

Mar 2026

Contents

Contents	i
1 Changes	1
1.1 .codespellignore	1
1.2 .github/workflows/labels_from_comment.yml	1
1.3 .gitignore	3
1.4 PhysLean.lean	3
1.5 PhysLean/ClassicalMechanics/Basic.lean	12
1.6 PhysLean/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean	12
1.7 PhysLean/ClassicalMechanics/EulerLagrange.lean	13
1.8 PhysLean/ClassicalMechanics/HamiltonsEquations.lean	14
1.9 PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean	14
1.10 PhysLean/ClassicalMechanics/HarmonicOscillator/ConfigurationSpace.lean	16
1.11 PhysLean/ClassicalMechanics/HarmonicOscillator/Solution.lean	17
1.12 PhysLean/ClassicalMechanics/Lagrangian/TotalDerivativeEquivalence.lean	18
1.13 PhysLean/ClassicalMechanics/Mass/MassUnit.lean	18
1.14 PhysLean/ClassicalMechanics/Pendulum/CoplanarDoublePendulum.lean	18
1.15 PhysLean/ClassicalMechanics/Pendulum/MiscellaneousPendulumPivotMotions.lean	18
1.16 PhysLean/ClassicalMechanics/Pendulum/SlidingPendulum.lean	18
1.17 PhysLean/ClassicalMechanics/RigidBody/Basic.lean	19
1.18 PhysLean/ClassicalMechanics/RigidBody/SolidSphere.lean	19
1.19 PhysLean/ClassicalMechanics/Scattering/RigidSphere.lean	19
1.20 PhysLean/ClassicalMechanics/Vibrations/LinearTriatomic.lean	19
1.21 PhysLean/ClassicalMechanics/WaveEquation/Basic.lean	19
1.22 PhysLean/ClassicalMechanics/WaveEquation/HarmonicWave.lean	20
1.23 PhysLean/CondensedMatter/Basic.lean	20
1.24 PhysLean/CondensedMatter/TightBindingChain/Basic.lean	20
1.25 PhysLean/Cosmology/Basic.lean	20
1.26 PhysLean/Cosmology/FLRW/Basic.lean	21
1.27 PhysLean/Electromagnetism/Basic.lean	21
1.28 PhysLean/Electromagnetism/Charge/ChargeUnit.lean	21
1.29 PhysLean/Electromagnetism/Current/CircularCoil.lean	21
1.30 PhysLean/Electromagnetism/Current/InfiniteWire.lean	22
1.31 PhysLean/Electromagnetism/Dynamics/Basic.lean	22

1.32	PhysLean/Electromagnetism/Dynamics/CurrentDensity.lean	22
1.33	PhysLean/Electromagnetism/Dynamics/Hamiltonian.lean	22
1.34	PhysLean/Electromagnetism/Dynamics/IsExtrema.lean	22
1.35	PhysLean/Electromagnetism/Dynamics/KineticTerm.lean	23
1.36	PhysLean/Electromagnetism/Dynamics/Lagrangian.lean	23
1.37	PhysLean/Electromagnetism/Kinematics/Boosts.lean	23
1.38	PhysLean/Electromagnetism/Kinematics/EMPotential.lean	23
1.39	PhysLean/Electromagnetism/Kinematics/ElectricField.lean	24
1.40	PhysLean/Electromagnetism/Kinematics/FieldStrength.lean	24
1.41	PhysLean/Electromagnetism/Kinematics/MagneticField.lean	24
1.42	PhysLean/Electromagnetism/Kinematics/ScalarPotential.lean	24
1.43	PhysLean/Electromagnetism/Kinematics/VectorPotential.lean	24
1.44	PhysLean/Electromagnetism/PointParticle/OneDimension.lean	25
1.45	PhysLean/Electromagnetism/PointParticle/ThreeDimension.lean	25
1.46	PhysLean/Electromagnetism/ThreeDimension/Basic.lean	25
1.47	PhysLean/Electromagnetism/Vacuum/Constant.lean	27
1.48	PhysLean/Electromagnetism/Vacuum/HarmonicWave.lean	27
1.49	PhysLean/Electromagnetism/Vacuum/IsPlaneWave.lean	27
1.50	PhysLean/Mathematics/Calculus/AdjFDeriv.lean	27
1.51	PhysLean/Mathematics/Calculus/Divergence.lean	28
1.52	PhysLean/Mathematics/DataStructures/FourTree/Basic.lean	28
1.53	PhysLean/Mathematics/DataStructures/FourTree/UniqueMap.lean	28
1.54	PhysLean/Mathematics/DataStructures/Matrix/LieTrace.lean	28
1.55	PhysLean/Mathematics/Distribution/Basic.lean	29
1.56	PhysLean/Mathematics/Distribution/PowMul.lean	29
1.57	PhysLean/Mathematics/FDerivCurry.lean	29
1.58	PhysLean/Mathematics/Fin.lean	29
1.59	PhysLean/Mathematics/Fin/Involutions.lean	30
1.60	PhysLean/Mathematics/Geometry/Metric/PseudoRiemannian/Defs.lean	30
1.61	PhysLean/Mathematics/Geometry/Metric/Riemannian/Defs.lean	30
1.62	PhysLean/Mathematics/InnerProductSpace/Adjoint.lean	31
1.63	PhysLean/Mathematics/InnerProductSpace/Basic.lean	31
1.64	PhysLean/Mathematics/InnerProductSpace/Calculus.lean	31
1.65	PhysLean/Mathematics/InnerProductSpace/Submodule.lean	31
1.66	PhysLean/Mathematics/KroneckerDelta.lean	32
1.67	PhysLean/Mathematics/LinearMaps.lean	35
1.68	PhysLean/Mathematics/List.lean	35
1.69	PhysLean/Mathematics/List/InsertIdx.lean	35
1.70	PhysLean/Mathematics/List/InsertionSort.lean	35
1.71	PhysLean/Mathematics/PiTensorProduct.lean	36
1.72	PhysLean/Mathematics/RatComplexNum.lean	36
1.73	PhysLean/Mathematics/SO3/Basic.lean	36
1.74	PhysLean/Mathematics/SchurTriangulation.lean	36
1.75	PhysLean/Mathematics/SpecialFunctions/PhysHermite.lean	37
1.76	PhysLean/Mathematics/Trigonometry/Tanh.lean	37
1.77	PhysLean/Mathematics/VariationalCalculus/Basic.lean	37

1.78	PhysLean/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean	39
1.79	PhysLean/Mathematics/VariationalCalculus/HasVarAdjoint.lean	39
1.80	PhysLean/Mathematics/VariationalCalculus/HasVarGradient.lean	39
1.81	PhysLean/Mathematics/VariationalCalculus/IsLocalizedfunctionTransform.lean	39
1.82	PhysLean/Mathematics/VariationalCalculus/IsTestFunction.lean	39
1.83	PhysLean/Meta/AllFilePaths.lean	40
1.84	PhysLean/Meta/Basic.lean	40
1.85	PhysLean/Meta/Informal/Basic.lean	40
1.86	PhysLean/Meta/Informal/Post.lean	41
1.87	PhysLean/Meta/Informal/SemiFormal.lean	41
1.88	PhysLean/Meta/Linters/Sorry.lean	41
1.89	PhysLean/Meta/Notes/Basic.lean	41
1.90	PhysLean/Meta/Notes/HTMLNote.lean	42
1.91	PhysLean/Meta/Notes/NoteFile.lean	42
1.92	PhysLean/Meta/Notes/ToHTML.lean	42
1.93	PhysLean/Meta/Remark/Basic.lean	43
1.94	PhysLean/Meta/Remark/Properties.lean	43
1.95	PhysLean/Meta/Sorry.lean	43
1.96	PhysLean/Meta/TODO/Basic.lean	43
1.97	PhysLean/Meta/TransverseTactics.lean	44
1.98	PhysLean/Optics/Basic.lean	44
1.99	PhysLean/Optics/Polarization/Basic.lean	44
1.100	PhysLean/Particles/BeyondTheStandardModel/GeorgiGlashow/Basic.lean	44
1.101	PhysLean/Particles/BeyondTheStandardModel/PatiSalam/Basic.lean	44
1.102	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Basic.lean	44
1.103	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/FamilyMaps.lean	44
1.104	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/NoGrav/Basic.lean	44
1.105	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Ordinary/Basic.lean	44
1.106	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Ordinary/Dim.lean	44
1.107	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Ordinary/Fam.lean	44
1.108	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Permutations.lean	44
1.109	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/BMinu.lean	44
1.110	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/Basic.lean	44
1.111	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/Bound.lean	44
1.112	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/Famil.lean	44
1.113	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/Hyper.lean	44
1.114	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/Plane.lean	44
1.115	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadS.lean	44
1.116	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadS.lean	44
1.117	PhysLean/Particles/BeyondTheStandardModel/Spin10/Basic.lean	48
1.118	PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Basic.lean	48
1.119	PhysLean/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean	48
1.120	PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean	48
1.121	PhysLean/Particles/FlavorPhysics/CKMMatrix/Basic.lean	49
1.122	PhysLean/Particles/FlavorPhysics/CKMMatrix/Invariants.lean	49
1.123	PhysLean/Particles/FlavorPhysics/CKMMatrix/PhaseFreedom.lean	49

1.124	PhysLean/Particles/FlavorPhysics/CKMMatrix/Relations.lean	49
1.125	PhysLean/Particles/FlavorPhysics/CKMMatrix/Rows.lean	50
1.126	PhysLean/Particles/FlavorPhysics/CKMMatrix/StandardParameterization/B	
1.127	PhysLean/Particles/FlavorPhysics/CKMMatrix/StandardParameterization/S	
1.128	PhysLean/Particles/NeutrinoPhysics/Basic.lean	50
1.129	PhysLean/Particles/StandardModel/AnomalyCancellation/Basic.lean	51
1.130	PhysLean/Particles/StandardModel/AnomalyCancellation/FamilyMaps.lean	
1.131	PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav/Basic.lean	
1.132	PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Lemma	
1.133	PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Linea	
1.134	PhysLean/Particles/StandardModel/AnomalyCancellation/Permutations.lean	
1.135	PhysLean/Particles/StandardModel/Basic.lean	52
1.136	PhysLean/Particles/StandardModel/HiggsBoson/Basic.lean	52
1.137	PhysLean/Particles/StandardModel/HiggsBoson/Potential.lean	52
1.138	PhysLean/Particles/StandardModel/Representations.lean	53
1.139	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/B3.lean	5
1.140	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Basic.lean	
1.141	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/HyperChar	
1.142	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/LineY3B3.	
1.143	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/OrthogY3B	
1.144	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/OrthogY3B	
1.145	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/OrthogY3B	
1.146	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Permutati	
1.147	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Y3.lean	5
1.148	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/AllowsTerm.lean	5
1.149	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Basic.lean	55
1.150	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Completions.lean	
1.151	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Map.lean	55
1.152	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimalSuperSet.1	
1.153	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllowsTe	
1.154	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllowsTe	
1.155	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllowsTe	
1.156	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/OfFieldLabel.lean	
1.157	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/OfPotentialTerm.1	
1.158	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoClosed.lean	
1.159	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoConstrained.	
1.160	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Yukawa.lean	57
1.161	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/ZMod.lean	58
1.162	PhysLean/Particles/SuperSymmetry/SU5/FieldLabels.lean	58
1.163	PhysLean/Particles/SuperSymmetry/SU5/Potential.lean	58
1.164	PhysLean/QFT/AnomalyCancellation/Basic.lean	58
1.165	PhysLean/QFT/AnomalyCancellation/GroupActions.lean	59
1.166	PhysLean/QFT/PerturbationTheory/CreateAnnihilate.lean	59
1.167	PhysLean/QFT/PerturbationTheory/FeynmanDiagrams/Basic.lean	59
1.168	PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/Basic.lean	59
1.169	PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/Grading.lean	59

1.170	PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormTimeOrder.lean	60
1.171	PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormalOrder.lean	60
1.172	PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/SuperCommute.lean	60
1.173	PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/TimeOrder.lean	60
1.174	PhysLean/QFT/PerturbationTheory/FieldSpecification/Basic.lean	61
1.175	PhysLean/QFT/PerturbationTheory/FieldSpecification/CrAnFieldOp.lean	61
1.176	PhysLean/QFT/PerturbationTheory/FieldSpecification/CrAnSection.lean	61
1.177	PhysLean/QFT/PerturbationTheory/FieldSpecification/Filters.lean	61
1.178	PhysLean/QFT/PerturbationTheory/FieldSpecification/NormalOrder.lean	62
1.179	PhysLean/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean	62
1.180	PhysLean/QFT/PerturbationTheory/FieldStatistics/Basic.lean	62
1.181	PhysLean/QFT/PerturbationTheory/FieldStatistics/ExchangeSign.lean	62
1.182	PhysLean/QFT/PerturbationTheory/FieldStatistics/OffInset.lean	63
1.183	PhysLean/QFT/PerturbationTheory/Koszul/KoszulSign.lean	63
1.184	PhysLean/QFT/PerturbationTheory/Koszul/KoszulSignInsert.lean	63
1.185	PhysLean/QFT/PerturbationTheory/WickAlgebra/Basic.lean	63
1.186	PhysLean/QFT/PerturbationTheory/WickAlgebra/Grading.lean	63
1.187	PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Basic.lean	64
1.188	PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Lemmas.lean	64
1.189	PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/WickContractions.lean	
1.190	PhysLean/QFT/PerturbationTheory/WickAlgebra/StaticWickTerm.lean	64
1.191	PhysLean/QFT/PerturbationTheory/WickAlgebra/StaticWickTheorem.lean	65
1.192	PhysLean/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean	65
1.193	PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeContraction.lean	65
1.194	PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeOrder.lean	65
1.195	PhysLean/QFT/PerturbationTheory/WickAlgebra/Universality.lean	66
1.196	PhysLean/QFT/PerturbationTheory/WickAlgebra/WickTerm.lean	66
1.197	PhysLean/QFT/PerturbationTheory/WickAlgebra/WicksTheorem.lean	66
1.198	PhysLean/QFT/PerturbationTheory/WickAlgebra/WicksTheoremNormal.lean	66
1.199	PhysLean/QFT/PerturbationTheory/WickContraction/Basic.lean	66
1.200	PhysLean/QFT/PerturbationTheory/WickContraction/Card.lean	67
1.201	PhysLean/QFT/PerturbationTheory/WickContraction/Erase.lean	67
1.202	PhysLean/QFT/PerturbationTheory/WickContraction/ExtractEquiv.lean	67
1.203	PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndContract.lean	67
1.204	PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndContractNat.lean	68
1.205	PhysLean/QFT/PerturbationTheory/WickContraction/Involutions.lean	68
1.206	PhysLean/QFT/PerturbationTheory/WickContraction/IsFull.lean	68
1.207	PhysLean/QFT/PerturbationTheory/WickContraction/Join.lean	68
1.208	PhysLean/QFT/PerturbationTheory/WickContraction/Perm.lean	68
1.209	PhysLean/QFT/PerturbationTheory/WickContraction/Sign/Basic.lean	69
1.210	PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertNone.lean	69
1.211	PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertSome.lean	69
1.212	PhysLean/QFT/PerturbationTheory/WickContraction/Sign/Join.lean	70
1.213	PhysLean/QFT/PerturbationTheory/WickContraction/Singleton.lean	70
1.214	PhysLean/QFT/PerturbationTheory/WickContraction/StaticContract.lean	70
1.215	PhysLean/QFT/PerturbationTheory/WickContraction/SubContraction.lean	70

1.216	PhysLean/QFT/PerturbationTheory/WickContraction/TimeCond.lean	71
1.217	PhysLean/QFT/PerturbationTheory/WickContraction/TimeContract.lean	71
1.218	PhysLean/QFT/PerturbationTheory/WickContraction/Uncontracted.lean	71
1.219	PhysLean/QFT/PerturbationTheory/WickContraction/UncontractedList.lean	71
1.220	PhysLean/QFT/QED/AnomalyCancellation/Basic.lean	71
1.221	PhysLean/QFT/QED/AnomalyCancellation/BasisLinear.lean	72
1.222	PhysLean/QFT/QED/AnomalyCancellation/ConstAbs.lean	72
1.223	PhysLean/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean	72
1.224	PhysLean/QFT/QED/AnomalyCancellation/Even/LineInCubic.lean	72
1.225	PhysLean/QFT/QED/AnomalyCancellation/Even/Parameterization.lean	73
1.226	PhysLean/QFT/QED/AnomalyCancellation/LineInPlaneCond.lean	73
1.227	PhysLean/QFT/QED/AnomalyCancellation/LowDim/One.lean	73
1.228	PhysLean/QFT/QED/AnomalyCancellation/LowDim/Three.lean	73
1.229	PhysLean/QFT/QED/AnomalyCancellation/LowDim/Two.lean	73
1.230	PhysLean/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean	74
1.231	PhysLean/QFT/QED/AnomalyCancellation/Odd/LineInCubic.lean	74
1.232	PhysLean/QFT/QED/AnomalyCancellation/Odd/Parameterization.lean	74
1.233	PhysLean/QFT/QED/AnomalyCancellation/Permutations.lean	74
1.234	PhysLean/QFT/QED/AnomalyCancellation/Sorts.lean	74
1.235	PhysLean/QFT/QED/AnomalyCancellation/VectorLike.lean	75
1.236	PhysLean/QuantumMechanics/DDimensions/Hydrogen/Basic.lean	75
1.237	PhysLean/QuantumMechanics/DDimensions/Hydrogen/LaplaceRungeLenzVector	
1.238	PhysLean/QuantumMechanics/DDimensions/Operators/AngularMomentum.lean	
1.239	PhysLean/QuantumMechanics/DDimensions/Operators/Commutation.lean	81
1.240	PhysLean/QuantumMechanics/DDimensions/Operators/Momentum.lean	87
1.241	PhysLean/QuantumMechanics/DDimensions/Operators/Position.lean	89
1.242	PhysLean/QuantumMechanics/DDimensions/Operators/Unbounded.lean	93
1.243	PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.lean	100
1.244	PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubm	
1.245	PhysLean/QuantumMechanics/FiniteTarget/Basic.lean	104
1.246	PhysLean/QuantumMechanics/FiniteTarget/HilbertSpace.lean	105
1.247	PhysLean/QuantumMechanics/OneDimension/GeneralPotential/Basic.lean	105
1.248	PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Basic.lean	105
1.249	PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Completeness	
1.250	PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Eigenfunction	
1.251	PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Examples.lean	
1.252	PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/TISE.lean	106
1.253	PhysLean/QuantumMechanics/OneDimension/HilbertSpace/Basic.lean	106
1.254	PhysLean/QuantumMechanics/OneDimension/HilbertSpace/Gaussians.lean	106
1.255	PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PlaneWaves.lean	106
1.256	PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PositionStates.lean	
1.257	PhysLean/QuantumMechanics/OneDimension/HilbertSpace/SchwartzSubmodule	
1.258	PhysLean/QuantumMechanics/OneDimension/Operators/Commutation.lean	107
1.259	PhysLean/QuantumMechanics/OneDimension/Operators/Momentum.lean	107
1.260	PhysLean/QuantumMechanics/OneDimension/Operators/Parity.lean	108
1.261	PhysLean/QuantumMechanics/OneDimension/Operators/Position.lean	108

1.262	PhysLean/QuantumMechanics/OneDimension/Operators/Unbounded.lean	108
1.263	PhysLean/QuantumMechanics/OneDimension/ReflectionlessPotential/Basic.lean	108
1.264	PhysLean/QuantumMechanics/PlanckConstant.lean	109
1.265	PhysLean/Relativity/Bispinors/Basic.lean	109
1.266	PhysLean/Relativity/CliffordAlgebra.lean	109
1.267	PhysLean/Relativity/LorentzAlgebra/Basic.lean	109
1.268	PhysLean/Relativity/LorentzAlgebra/Basis.lean	110
1.269	PhysLean/Relativity/LorentzAlgebra/ExponentialMap.lean	110
1.270	PhysLean/Relativity/LorentzGroup/Basic.lean	110
1.271	PhysLean/Relativity/LorentzGroup/Boosts/Apply.lean	110
1.272	PhysLean/Relativity/LorentzGroup/Boosts/Basic.lean	111
1.273	PhysLean/Relativity/LorentzGroup/Boosts/Generalized.lean	111
1.274	PhysLean/Relativity/LorentzGroup/Orthochronous/Basic.lean	111
1.275	PhysLean/Relativity/LorentzGroup/Proper.lean	111
1.276	PhysLean/Relativity/LorentzGroup/Restricted/Basic.lean	111
1.277	PhysLean/Relativity/LorentzGroup/Restricted/FromBoostRotation.lean	112
1.278	PhysLean/Relativity/LorentzGroup/Rotations.lean	112
1.279	PhysLean/Relativity/LorentzGroup/ToVector.lean	112
1.280	PhysLean/Relativity/MinkowskiMatrix.lean	112
1.281	PhysLean/Relativity/PauliMatrices/AsTensor.lean	113
1.282	PhysLean/Relativity/PauliMatrices/Basic.lean	113
1.283	PhysLean/Relativity/PauliMatrices/CliffordAlgebra.lean	113
1.284	PhysLean/Relativity/PauliMatrices/Relations.lean	113
1.285	PhysLean/Relativity/PauliMatrices/SelfAdjoint.lean	113
1.286	PhysLean/Relativity/PauliMatrices/ToTensor.lean	114
1.287	PhysLean/Relativity/SL2C/Basic.lean	114
1.288	PhysLean/Relativity/SL2C/SelfAdjoint.lean	114
1.289	PhysLean/Relativity/Special/ProperTime.lean	114
1.290	PhysLean/Relativity/Special/TwinParadox/Basic.lean	115
1.291	PhysLean/Relativity/SpeedOfLight.lean	115
1.292	PhysLean/Relativity/Tensors/Basic.lean	115
1.293	PhysLean/Relativity/Tensors/Color/Basic.lean	115
1.294	PhysLean/Relativity/Tensors/Color/Discrete.lean	116
1.295	PhysLean/Relativity/Tensors/Color/Lift.lean	116
1.296	PhysLean/Relativity/Tensors/ComplexTensor/Basic.lean	116
1.297	PhysLean/Relativity/Tensors/ComplexTensor/Lemmas.lean	116
1.298	PhysLean/Relativity/Tensors/ComplexTensor/Matrix/Pre.lean	117
1.299	PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean	117
1.300	PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean	117
1.301	PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Pre.lean	117
1.302	PhysLean/Relativity/Tensors/ComplexTensor/OfRat.lean	117
1.303	PhysLean/Relativity/Tensors/ComplexTensor/Units/Basic.lean	118
1.304	PhysLean/Relativity/Tensors/ComplexTensor/Units/Pre.lean	118
1.305	PhysLean/Relativity/Tensors/ComplexTensor/Units/Symm.lean	118
1.306	PhysLean/Relativity/Tensors/ComplexTensor/Vector/Pre/Basic.lean	118
1.307	PhysLean/Relativity/Tensors/ComplexTensor/Vector/Pre/Contraction.lean	118

1.308	PhysLean/Relativity/Tensors/ComplexTensor/Vector/Pre/Modules.lean	119
1.309	PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Basic.lean	119
1.310	PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Contraction.lean	119
1.311	PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Metric.lean	119
1.312	PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Modules.lean	120
1.313	PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Two.lean	120
1.314	PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Unit.lean	120
1.315	PhysLean/Relativity/Tensors/Constructors.lean	120
1.316	PhysLean/Relativity/Tensors/Contraction/Basic.lean	120
1.317	PhysLean/Relativity/Tensors/Contraction/Basis.lean	121
1.318	PhysLean/Relativity/Tensors/Contraction/Products.lean	121
1.319	PhysLean/Relativity/Tensors/Contraction/Pure.lean	121
1.320	PhysLean/Relativity/Tensors/Dual.lean	121
1.321	PhysLean/Relativity/Tensors/Elab.lean	122
1.322	PhysLean/Relativity/Tensors/Evaluation.lean	123
1.323	PhysLean/Relativity/Tensors/MetricTensor.lean	123
1.324	PhysLean/Relativity/Tensors/OfInt.lean	123
1.325	PhysLean/Relativity/Tensors/Product.lean	123
1.326	PhysLean/Relativity/Tensors/RealTensor/Basic.lean	124
1.327	PhysLean/Relativity/Tensors/RealTensor/CoVector/Basic.lean	124
1.328	PhysLean/Relativity/Tensors/RealTensor/Derivative.lean	124
1.329	PhysLean/Relativity/Tensors/RealTensor/Matrix/Pre.lean	124
1.330	PhysLean/Relativity/Tensors/RealTensor/Metrics/Basic.lean	125
1.331	PhysLean/Relativity/Tensors/RealTensor/Metrics/Pre.lean	125
1.332	PhysLean/Relativity/Tensors/RealTensor/ToComplex.lean	125
1.333	PhysLean/Relativity/Tensors/RealTensor/Units/Pre.lean	125
1.334	PhysLean/Relativity/Tensors/RealTensor/Vector/Basic.lean	125
1.335	PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/Basic.lean	126
1.336	PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/LightLike.lean	126
1.337	PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/TimeLike.lean	126
1.338	PhysLean/Relativity/Tensors/RealTensor/Vector/MinkowskiProduct.lean	126
1.339	PhysLean/Relativity/Tensors/RealTensor/Vector/Pre/Basic.lean	126
1.340	PhysLean/Relativity/Tensors/RealTensor/Vector/Pre/Contraction.lean	127
1.341	PhysLean/Relativity/Tensors/RealTensor/Vector/Pre/Modules.lean	127
1.342	PhysLean/Relativity/Tensors/RealTensor/Velocity/Basic.lean	127
1.343	PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean	127
1.344	PhysLean/Relativity/Tensors/Tensorial.lean	128
1.345	PhysLean/Relativity/Tensors/UnitTensor.lean	128
1.346	PhysLean/SpaceAndTime/Space/Basic.lean	128
1.347	PhysLean/SpaceAndTime/Space/ConstantSliceDist.lean	132
1.348	PhysLean/SpaceAndTime/Space/CrossProduct.lean	132
1.349	PhysLean/SpaceAndTime/Space/Derivatives/Basic.lean	132
1.350	PhysLean/SpaceAndTime/Space/Derivatives/Curl.lean	134
1.351	PhysLean/SpaceAndTime/Space/Derivatives/Div.lean	135
1.352	PhysLean/SpaceAndTime/Space/Derivatives/Grad.lean	135
1.353	PhysLean/SpaceAndTime/Space/Derivatives/Laplacian.lean	136

1.354	PhysLean/SpaceAndTime/Space/DistConst.lean	136
1.355	PhysLean/SpaceAndTime/Space/DistOfFunction.lean	137
1.356	PhysLean/SpaceAndTime/Space/Integrals/Basic.lean	137
1.357	PhysLean/SpaceAndTime/Space/Integrals/NormPow.lean	142
1.358	PhysLean/SpaceAndTime/Space/Integrals/RadialAngularMeasure.lean	146
1.359	PhysLean/SpaceAndTime/Space/IsDistBounded.lean	153
1.360	PhysLean/SpaceAndTime/Space/LengthUnit.lean	153
1.361	PhysLean/SpaceAndTime/Space/Module.lean	153
1.362	PhysLean/SpaceAndTime/Space/Norm.lean	170
1.363	PhysLean/SpaceAndTime/Space/Slice.lean	170
1.364	PhysLean/SpaceAndTime/Space/Translations.lean	170
1.365	PhysLean/SpaceAndTime/SpaceTime/Basic.lean	171
1.366	PhysLean/SpaceAndTime/SpaceTime/Boosts.lean	171
1.367	PhysLean/SpaceAndTime/SpaceTime/Derivatives.lean	171
1.368	PhysLean/SpaceAndTime/SpaceTime/LorentzAction.lean	171
1.369	PhysLean/SpaceAndTime/SpaceTime/TimeSlice.lean	172
1.370	PhysLean/SpaceAndTime/Time/Basic.lean	172
1.371	PhysLean/SpaceAndTime/Time/Derivatives.lean	172
1.372	PhysLean/SpaceAndTime/Time/TimeMan.lean	172
1.373	PhysLean/SpaceAndTime/Time/TimeTransMan.lean	173
1.374	PhysLean/SpaceAndTime/Time/TimeUnit.lean	173
1.375	PhysLean/SpaceAndTime/TimeAndSpace/Basic.lean	173
1.376	PhysLean/SpaceAndTime/TimeAndSpace/ConstantTimeDist.lean	173
1.377	PhysLean/StatisticalMechanics/BoltzmannConstant.lean	174
1.378	PhysLean/StatisticalMechanics/CanonicalEnsemble/Basic.lean	174
1.379	PhysLean/StatisticalMechanics/CanonicalEnsemble/Finite.lean	174
1.380	PhysLean/StatisticalMechanics/CanonicalEnsemble/Lemmas.lean	174
1.381	PhysLean/StatisticalMechanics/CanonicalEnsemble/TwoState.lean	175
1.382	PhysLean/StringTheory/Basic.lean	175
1.383	PhysLean/StringTheory/FTheory/SU5/Basic.lean	175
1.384	PhysLean/StringTheory/FTheory/SU5/Charges/AnomalyFree.lean	175
1.385	PhysLean/StringTheory/FTheory/SU5/Charges/OfRationalSection.lean	175
1.386	PhysLean/StringTheory/FTheory/SU5/Charges/Viable.lean	176
1.387	PhysLean/StringTheory/FTheory/SU5/Fluxes/Basic.lean	176
1.388	PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/ChiralIndices.lean	176
1.389	PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/Completeness.lean	176
1.390	PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/Elms.lean	176
1.391	PhysLean/StringTheory/FTheory/SU5/Quanta/Basic.lean	177
1.392	PhysLean/StringTheory/FTheory/SU5/Quanta/FiveQuanta.lean	177
1.393	PhysLean/StringTheory/FTheory/SU5/Quanta/IsViable.lean	177
1.394	PhysLean/StringTheory/FTheory/SU5/Quanta/TenQuanta.lean	177
1.395	PhysLean/Thermodynamics/Basic.lean	178
1.396	PhysLean/Thermodynamics/IdealGas/Basic.lean	178
1.397	PhysLean/Thermodynamics/Temperature/Basic.lean	178
1.398	PhysLean/Thermodynamics/Temperature/TemperatureUnits.lean	178
1.399	PhysLean/Units/Basic.lean	178

1.400PhysLean/Units/Dimension.lean	179
1.401PhysLean/Units/Examples.lean	179
1.402PhysLean/Units/FDeriv.lean	179
1.403PhysLean/Units/Integral.lean	179
1.404PhysLean/Units/UnitDependent.lean	180
1.405PhysLean/Units/WithDim/Area.lean	180
1.406PhysLean/Units/WithDim/Basic.lean	180
1.407PhysLean/Units/WithDim/Energy.lean	180
1.408PhysLean/Units/WithDim/Mass.lean	180
1.409PhysLean/Units/WithDim/Momentum.lean	181
1.410PhysLean/Units/WithDim/Pressure.lean	181
1.411PhysLean/Units/WithDim/Speed.lean	181
1.412PhysLean/Units/WithDim/Velocity.lean	181
1.413QuantumInfo.lean	181
1.414QuantumInfo/ClassicalInfo/Capacity.lean	182
1.415QuantumInfo/ClassicalInfo/Channel.lean	184
1.416QuantumInfo/ClassicalInfo/Distribution.lean	185
1.417QuantumInfo/ClassicalInfo/Entropy.lean	192
1.418QuantumInfo/ClassicalInfo/ForMathlib/Analysis/SpecialFunctions/Log/Ne	
1.419QuantumInfo/ClassicalInfo/Prob.lean	197
1.420QuantumInfo/Finite/AxiomatizedEntropy/Defs.lean	208
1.421QuantumInfo/Finite/AxiomatizedEntropy/Renyi.lean	214
1.422QuantumInfo/Finite/Braket.lean	215
1.423QuantumInfo/Finite/CPTPMap.lean	224
1.424QuantumInfo/Finite/CPTPMap/Bundled.lean	224
1.425QuantumInfo/Finite/CPTPMap/CPTP.lean	234
1.426QuantumInfo/Finite/CPTPMap/Dual.lean	252
1.427QuantumInfo/Finite/CPTPMap/MatrixMap.lean	262
1.428QuantumInfo/Finite/CPTPMap/Unbundled.lean	272
1.429QuantumInfo/Finite/Capacity.lean	289
1.430QuantumInfo/Finite/Capacity_doc.lean	295
1.431QuantumInfo/Finite/Channel/DegradableOrder.lean	296
1.432QuantumInfo/Finite/Distance.lean	297
1.433QuantumInfo/Finite/Distance/Fidelity.lean	297
1.434QuantumInfo/Finite/Distance/TraceDistance.lean	299
1.435QuantumInfo/Finite/Ensemble.lean	301
1.436QuantumInfo/Finite/Entanglement.lean	308
1.437QuantumInfo/Finite/Entropy.lean	315
1.438QuantumInfo/Finite/Entropy/DPI.lean	315
1.439QuantumInfo/Finite/Entropy/Relative.lean	325
1.440QuantumInfo/Finite/Entropy/SSA.lean	376
1.441QuantumInfo/Finite/Entropy/VonNeumann.lean	405
1.442QuantumInfo/Finite/MState.lean	412
1.443QuantumInfo/Finite/POVM.lean	443
1.444QuantumInfo/Finite/Pinching.lean	447
1.445QuantumInfo/Finite/Qubit/Basic.lean	456

1.446	QuantumInfo/Finite/ResourceTheory/FreeState.lean	460
1.447	QuantumInfo/Finite/ResourceTheory/HypothesisTesting.lean	472
1.448	QuantumInfo/Finite/ResourceTheory/ResourceTheory.lean	484
1.449	QuantumInfo/Finite/ResourceTheory/SteinsLemma.lean	488
1.450	QuantumInfo/Finite/Unitary.lean	535
1.451	QuantumInfo/ForMathlib.lean	537
1.452	QuantumInfo/ForMathlib/ContinuousLinearMap.lean	538
1.453	QuantumInfo/ForMathlib/ContinuousSup.lean	540
1.454	QuantumInfo/ForMathlib/Filter.lean	546
1.455	QuantumInfo/ForMathlib/HermitianMat.lean	547
1.456	QuantumInfo/ForMathlib/HermitianMat/Basic.lean	547
1.457	QuantumInfo/ForMathlib/HermitianMat/CFC.lean	564
1.458	QuantumInfo/ForMathlib/HermitianMat/Inner.lean	596
1.459	QuantumInfo/ForMathlib/HermitianMat/Jordan.lean	608
1.460	QuantumInfo/ForMathlib/HermitianMat/LogExp.lean	611
1.461	QuantumInfo/ForMathlib/HermitianMat/NonSingular.lean	627
1.462	QuantumInfo/ForMathlib/HermitianMat/Order.lean	631
1.463	QuantumInfo/ForMathlib/HermitianMat/Proj.lean	645
1.464	QuantumInfo/ForMathlib/HermitianMat/Reindex.lean	657
1.465	QuantumInfo/ForMathlib/HermitianMat/Rpow.lean	659
1.466	QuantumInfo/ForMathlib/HermitianMat/Schatten.lean	674
1.467	QuantumInfo/ForMathlib/HermitianMat/Sqrt.lean	681
1.468	QuantumInfo/ForMathlib/HermitianMat/Trace.lean	683
1.469	QuantumInfo/ForMathlib/HermitianMat/Unitary.lean	689
1.470	QuantumInfo/ForMathlib/IsMaximalSelfAdjoint.lean	690
1.471	QuantumInfo/ForMathlib/Isometry.lean	691
1.472	QuantumInfo/ForMathlib/Lieb.lean	706
1.473	QuantumInfo/ForMathlib/LimSupInf.lean	706
1.474	QuantumInfo/ForMathlib/LinearEquiv.lean	718
1.475	QuantumInfo/ForMathlib/Majorization.lean	720
1.476	QuantumInfo/ForMathlib/Matrix.lean	742
1.477	QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean	778
1.478	QuantumInfo/ForMathlib/Minimax.lean	782
1.479	QuantumInfo/ForMathlib/Misc.lean	786
1.480	QuantumInfo/ForMathlib/SionMinimax.lean	790
1.481	QuantumInfo/ForMathlib/Superadditive.lean	807
1.482	QuantumInfo/ForMathlib/Tactic/Commutes.lean	808
1.483	QuantumInfo/ForMathlib/Tactic/Commutes/Attribute.lean	812
1.484	QuantumInfo/ForMathlib/ULift.lean	813
1.485	QuantumInfo/ForMathlib/Unitary.lean	814
1.486	QuantumInfo/InfiniteDim/QState.lean	816
1.487	QuantumInfo/QI_DOC.md	818
1.488	QuantumInfo/Regularized.lean	854
1.489	QuantumInfo/StatMech/Hamiltonian.lean	858
1.490	QuantumInfo/StatMech/IdealGas.lean	859
1.491	QuantumInfo/StatMech/ThermoQuantities.lean	864

1.492QuantumInfo/WildeTODO.md	868
1.493README.md	873
1.494docs/ReviewGuidelines.md	875
1.495lake-manifest.json	877
1.496lakefile.lean	877
1.497scripts/lint-style.py	880

1 Changes

1.1 .codespellignore

Commun
braket
dOut

1.2 .github/workflows/labels_from_comment.yml

```
# This workflow allows any user to add or remove one of the labels in the array `labels`  
# by commenting on the PR or issue.  
# The script reacts to lines in the comment whose entire content is, up to whitespace, equal to  
# the label name, optionally preceded by `!`.  
# For each label, the bot follows the instruction of the *last* line that matches the label name
```

name: Label PR based on Comment

```
on:  
  issue_comment:  
    types: [created]  
  pull_request_review:  
    types: [submitted]  
  pull_request_review_comment:  
    types: [created]
```

```
# Limit permissions for GITHUB_TOKEN for the entire workflow  
permissions:  
  contents: read  
  pull-requests: write # Only allow PR comments/labels  
  # All other permissions are implicitly 'none'
```

```
jobs:  
  update-label:  
    env:
```

```

    COMMENT: ${github.event.comment.body}
runs-on: ubuntu-latest

steps:
- name: Add / remove label based on comment
  run: |
    labelArray=("awaiting-author" "blocked-by-mathlib-
PR" "blocked-by-PR" "documentation" "easy" "help-wanted" "merge-
conflict" "new-contributor" "please-adopt" "RFC" "WIP" "t-
classical-mechanics" "t-condensed-matter" "t-cosmology" "t-
electromagnetism" "t-mathematics" "t-meta" "t-optics" "t-
particles" "t-quantum-field-theory" "t-quantum-mechanics" "t-
relativity" "t-space-time" "t-statistical-mechanics" "t-string-
theory" "t-thermodynamics" "t-units")

    # we strip `r` since line endings from GitHub contain this character
    COMMENT="${COMMENT//`r`/}"

    # trim leading/trailing whitespace and collapse "internal" whitespace
    COMMENT="$(printf '%s' "${COMMENT}" | awk '{$1=$1};1')"

    # for debugging, we print some information
    printf '%s' "${COMMENT}" | hexdump -cC
    printf 'Comment: "%s"\n' "${COMMENT}"

    for i in "${!labelArray[@]}"; do
        inComment=""
        label="${labelArray[$i]}"
        printf `nProcessing label `'%s`'\n' "${label}"
        # extract the last line that, up to leading/trailing whitespace,
        inComment="$(printf '%s' "${COMMENT}" | grep "[-
]\?${label}$" | tail -1)"
        if [ -n "${inComment}" ]
        then
            printf $'Found `'%s`'\n' "${inComment}"
            if [ "${inComment:0:1}" == "-" ]
            then
                printf $'Removing label `'%s`'\n' "${label}"
                # we use curl rather than octokit/request-action so that the
                # (and send an annoying email) if the labels don't exist
                curl --request DELETE \
                    --url "https://api.github.com/repos/${github.repository} \
                    --header 'authorization: Bearer ${secrets.GITHUB_TOKEN}"
                printf $'%s DELETE done\n' "${inComment}"
            else

```



```

data="$(printf $'{"labels":["%s"]}' "${label}")"
printf $'Using data: %s\n' "${data}"
# we use curl rather than octokit/request-action so that the job won't
# (and send an annoying email) if the labels don't exist
curl --request POST \
  --header 'Accept: application/vnd.github+json' \
  --header 'authorization: Bearer ${ secrets.GITHUB_TOKEN }' \
  --header 'X-GitHub-API-Version: 2022-11-28' \
  --url "https://api.github.com/repos/${ github.repository }/issues/"
  --data "${data}"
printf $'%s POST done\n' "${inComment}"

fi

else
  printf $'Label \'%s\' not found.\n' "${label}"
fi

done

```

1.3 .gitignore

```
.lake/*
```

1.4 PhysLean.lean

```
module
```

```

public import PhysLean.ClassicalMechanics.Basic
public import PhysLean.ClassicalMechanics.DampedHarmonicOscillator.Basic
public import PhysLean.ClassicalMechanics.EulerLagrange
public import PhysLean.ClassicalMechanics.HamiltonsEquations
public import PhysLean.ClassicalMechanics.HarmonicOscillator.Basic
public import PhysLean.ClassicalMechanics.HarmonicOscillator.ConfigurationSpace
public import PhysLean.ClassicalMechanics.HarmonicOscillator.Solution
public import PhysLean.ClassicalMechanics.Lagrangian.TotalDerivativeEquivalence
public import PhysLean.ClassicalMechanics.Mass.MassUnit
public import PhysLean.ClassicalMechanics.Pendulum.CoplanarDoublePendulum
public import PhysLean.ClassicalMechanics.Pendulum.MiscellaneousPendulumPivotMotions
public import PhysLean.ClassicalMechanics.Pendulum.SlidingPendulum
public import PhysLean.ClassicalMechanics.RigidBody.Basic
public import PhysLean.ClassicalMechanics.RigidBody.SolidSphere
public import PhysLean.ClassicalMechanics.Scattering.RigidBody

```

```

public import PhysLean.ClassicalMechanics.Vibrations.LinearTriatomic
public import PhysLean.ClassicalMechanics.WaveEquation.Basic
public import PhysLean.ClassicalMechanics.WaveEquation.HarmonicWave
public import PhysLean.CondensedMatter.Basic
public import PhysLean.CondensedMatter.TightBindingChain.Basic
public import PhysLean.Cosmology.Basic
public import PhysLean.Cosmology.FLRW.Basic
public import PhysLean.Electromagnetism.Basic
public import PhysLean.Electromagnetism.Charge.ChargeUnit
public import PhysLean.Electromagnetism.Current.CircularCoil
public import PhysLean.Electromagnetism.Current.InfiniteWire
public import PhysLean.Electromagnetism.Dynamics.Basic
public import PhysLean.Electromagnetism.Dynamics.CurrentDensity
public import PhysLean.Electromagnetism.Dynamics.Hamiltonian
public import PhysLean.Electromagnetism.Dynamics.IsExtrema
public import PhysLean.Electromagnetism.Dynamics.KineticTerm
public import PhysLean.Electromagnetism.Dynamics.Lagrangian
public import PhysLean.Electromagnetism.Kinematics.Boosts
public import PhysLean.Electromagnetism.Kinematics.EMPotential
public import PhysLean.Electromagnetism.Kinematics.ElectricField
public import PhysLean.Electromagnetism.Kinematics.FieldStrength
public import PhysLean.Electromagnetism.Kinematics.MagneticField
public import PhysLean.Electromagnetism.Kinematics.ScalarPotential
public import PhysLean.Electromagnetism.Kinematics.VectorPotential
public import PhysLean.Electromagnetism.PointParticle.OneDimension
public import PhysLean.Electromagnetism.PointParticle.ThreeDimension
public import PhysLean.Electromagnetism.ThreeDimension.Basic
public import PhysLean.Electromagnetism.Vacuum.Constant
public import PhysLean.Electromagnetism.Vacuum.HarmonicWave
public import PhysLean.Electromagnetism.Vacuum.IsPlaneWave
public import PhysLean.Mathematics.Calculus.AdjFDeriv
public import PhysLean.Mathematics.Calculus.Divergence
public import PhysLean.Mathematics.DataStructures.FourTree.Basic
public import PhysLean.Mathematics.DataStructures.FourTree.UniqueMap
public import PhysLean.Mathematics.DataStructures.Matrix.LieTrace
public import PhysLean.Mathematics.Distribution.Basic
public import PhysLean.Mathematics.Distribution.PowMul
public import PhysLean.Mathematics.FDerivCurry
public import PhysLean.Mathematics.Fin
public import PhysLean.Mathematics.Fin.Involutions
public import PhysLean.Mathematics.Geometry.Metric.PseudoRiemannian.Defs
public import PhysLean.Mathematics.Geometry.Metric.Riemannian.Defs
public import PhysLean.Mathematics.InnerProductSpace.Adjoint
public import PhysLean.Mathematics.InnerProductSpace.Basic
public import PhysLean.Mathematics.InnerProductSpace.Calculus
public import PhysLean.Mathematics.InnerProductSpace.Submodule

```

```

public import PhysLean.Mathematics.KroneckerDelta
public import PhysLean.Mathematics.LinearMaps
public import PhysLean.Mathematics.List
public import PhysLean.Mathematics.List.InsertIdx
public import PhysLean.Mathematics.List.InsertionSort
public import PhysLean.Mathematics.PiTensorProduct
public import PhysLean.Mathematics.RatComplexNum
public import PhysLean.Mathematics.S03.Basic
public import PhysLean.Mathematics.SchurTriangulation
public import PhysLean.Mathematics.SpecialFunctions.PhysHermite
public import PhysLean.Mathematics.Trigonometry.Tanh
public import PhysLean.Mathematics.VariationalCalculus.Basic
public import PhysLean.Mathematics.VariationalCalculus.HasVarAdjDeriv
public import PhysLean.Mathematics.VariationalCalculus.HasVarAdjoint
public import PhysLean.Mathematics.VariationalCalculus.HasVarGradient
public import PhysLean.Mathematics.VariationalCalculus.IsLocalizedfunctionTransform
public import PhysLean.Mathematics.VariationalCalculus.IsTestFunction
public import PhysLean.Meta.AllFilePaths
public import PhysLean.Meta.Basic
public import PhysLean.Meta.Informal.Basic
public import PhysLean.Meta.Informal.Post
public import PhysLean.Meta.Informal.SemiFormal
public import PhysLean.Meta.Linters.Sorry
public import PhysLean.Meta.Notes.Basic
public import PhysLean.Meta.Notes.HTMLNote
public import PhysLean.Meta.Notes.NoteFile
public import PhysLean.Meta.Notes.ToHTML
public import PhysLean.Meta.Remark.Basic
public import PhysLean.Meta.Remark.Properties
public import PhysLean.Meta.Sorry
public import PhysLean.Meta.TODO.Basic
public import PhysLean.Meta.TransverseTactics
public import PhysLean.Optics.Basic
public import PhysLean.Optics.Polarization.Basic
public import PhysLean.Particles.BeyondTheStandardModel.GeorgiGlashow.Basic
public import PhysLean.Particles.BeyondTheStandardModel.PatiSalam.Basic
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Basi
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Fami
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.NoGr
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Ord
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Ord
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Ord
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Perm
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Plus
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Plus
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Plus

```

```

public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation
public import PhysLean.Particles.BeyondTheStandardModel.Spin10.Basic
public import PhysLean.Particles.BeyondTheStandardModel.TwoHDM.Basic
public import PhysLean.Particles.BeyondTheStandardModel.TwoHDM.GramMatrix
public import PhysLean.Particles.BeyondTheStandardModel.TwoHDM.Potential
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Basic
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Invariants
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.PhaseFreedom
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Relations
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Rows
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.StandardParameteri
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.StandardParameteri
public import PhysLean.Particles.NeutrinoPhysics.Basic
public import PhysLean.Particles.StandardModel.AnomalyCancellation.Basic
public import PhysLean.Particles.StandardModel.AnomalyCancellation.FamilyMa
public import PhysLean.Particles.StandardModel.AnomalyCancellation.NoGrav.B
public import PhysLean.Particles.StandardModel.AnomalyCancellation.NoGrav.0
public import PhysLean.Particles.StandardModel.AnomalyCancellation.NoGrav.0
public import PhysLean.Particles.StandardModel.AnomalyCancellation.Permutat
public import PhysLean.Particles.StandardModel.Basic
public import PhysLean.Particles.StandardModel.HiggsBoson.Basic
public import PhysLean.Particles.StandardModel.HiggsBoson.Potential
public import PhysLean.Particles.StandardModel.Representations
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.B
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.B
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.H
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.L
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.0
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.0
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.0
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.P
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Y
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.AllowsTer
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Basic
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completio
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Map
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimalSu
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Minimally
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Minimally
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Minimally
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.OfFieldLa
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.OfPotenti

```

```

public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoClosed
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoConstrained
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.ZMod
public import PhysLean.Particles.SuperSymmetry.SU5.FieldLabels
public import PhysLean.Particles.SuperSymmetry.SU5.Potential
public import PhysLean.QFT.AnomalyCancellation.Basic
public import PhysLean.QFT.AnomalyCancellation.GroupActions
public import PhysLean.QFT.PerturbationTheory.CreateAnnihilate
public import PhysLean.QFT.PerturbationTheory.FeynmanDiagrams.Basic
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Grading
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormTimeOrder
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormalOrder
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommute
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.TimeOrder
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.Basic
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.CrAnFieldOp
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.CrAnSection
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.Filters
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.NormalOrder
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder
public import PhysLean.QFT.PerturbationTheory.FieldStatistics.Basic
public import PhysLean.QFT.PerturbationTheory.FieldStatistics.ExchangeSign
public import PhysLean.QFT.PerturbationTheory.FieldStatistics.OffFinset
public import PhysLean.QFT.PerturbationTheory.Koszul.KoszulSign
public import PhysLean.QFT.PerturbationTheory.Koszul.KoszulSignInsert
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.Basic
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.Grading
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Basic
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemmas
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.WickContraction
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.StaticWickTerm
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.StaticWickTheorem
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.SuperCommute
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeContraction
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeOrder
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.Universality
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.WickTerm
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.WicksTheorem
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.WicksTheoremNormal
public import PhysLean.QFT.PerturbationTheory.WickContraction.Basic
public import PhysLean.QFT.PerturbationTheory.WickContraction.Card
public import PhysLean.QFT.PerturbationTheory.WickContraction.Erase
public import PhysLean.QFT.PerturbationTheory.WickContraction.ExtractEquiv
public import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContract

```

```

public import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndCont
public import PhysLean.QFT.PerturbationTheory.WickContraction.Involutions
public import PhysLean.QFT.PerturbationTheory.WickContraction.IsFull
public import PhysLean.QFT.PerturbationTheory.WickContraction.Join
public import PhysLean.QFT.PerturbationTheory.WickContraction.Perm
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Basic
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertNo
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertSo
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Join
public import PhysLean.QFT.PerturbationTheory.WickContraction.Singleton
public import PhysLean.QFT.PerturbationTheory.WickContraction.StaticContrac
public import PhysLean.QFT.PerturbationTheory.WickContraction.SubContractio
public import PhysLean.QFT.PerturbationTheory.WickContraction.TimeCond
public import PhysLean.QFT.PerturbationTheory.WickContraction.TimeContract
public import PhysLean.QFT.PerturbationTheory.WickContraction.Uncontracted
public import PhysLean.QFT.PerturbationTheory.WickContraction.UncontractedL
public import PhysLean.QFT.QED.AnomalyCancellation.Basic
public import PhysLean.QFT.QED.AnomalyCancellation.BasisLinear
public import PhysLean.QFT.QED.AnomalyCancellation.ConstAbs
public import PhysLean.QFT.QED.AnomalyCancellation.Even.BasisLinear
public import PhysLean.QFT.QED.AnomalyCancellation.Even.LineInCubic
public import PhysLean.QFT.QED.AnomalyCancellation.Even.Parameterization
public import PhysLean.QFT.QED.AnomalyCancellation.LineInPlaneCond
public import PhysLean.QFT.QED.AnomalyCancellation.LowDim.One
public import PhysLean.QFT.QED.AnomalyCancellation.LowDim.Three
public import PhysLean.QFT.QED.AnomalyCancellation.LowDim.Two
public import PhysLean.QFT.QED.AnomalyCancellation.Odd.BasisLinear
public import PhysLean.QFT.QED.AnomalyCancellation.Odd.LineInCubic
public import PhysLean.QFT.QED.AnomalyCancellation.Odd.Parameterization
public import PhysLean.QFT.QED.AnomalyCancellation.Permutations
public import PhysLean.QFT.QED.AnomalyCancellation.Sorts
public import PhysLean.QFT.QED.AnomalyCancellation.VectorLike
public import PhysLean.QuantumMechanics.DDimensions.Hydrogen.Basic
public import PhysLean.QuantumMechanics.DDimensions.Hydrogen.LaplaceRungeLe
public import PhysLean.QuantumMechanics.DDimensions.Operators.AngularMoment
public import PhysLean.QuantumMechanics.DDimensions.Operators.Commutation
public import PhysLean.QuantumMechanics.DDimensions.Operators.Momentum
public import PhysLean.QuantumMechanics.DDimensions.Operators.Position
public import PhysLean.QuantumMechanics.DDimensions.Operators.Unbounded
public import PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Basi
public import PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Schw
public import PhysLean.QuantumMechanics.FiniteTarget.Basic
public import PhysLean.QuantumMechanics.FiniteTarget.HilbertSpace
public import PhysLean.QuantumMechanics.OneDimension.GeneralPotential.Basic
public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Bas
public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Com

```

```

public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Eigenfunction
public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Examples
public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.TISE
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Basic
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Gaussians
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PlaneWaves
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PositionStates
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule
public import PhysLean.QuantumMechanics.OneDimension.Operators.Commutation
public import PhysLean.QuantumMechanics.OneDimension.Operators.Momentum
public import PhysLean.QuantumMechanics.OneDimension.Operators.Parity
public import PhysLean.QuantumMechanics.OneDimension.Operators.Position
public import PhysLean.QuantumMechanics.OneDimension.Operators.Unbounded
public import PhysLean.QuantumMechanics.OneDimension.ReflectionlessPotential.Basic
public import PhysLean.QuantumMechanics.PlanckConstant
public import PhysLean.Relativity.Bispinors.Basic
public import PhysLean.Relativity.CliffordAlgebra
public import PhysLean.Relativity.LorentzAlgebra.Basic
public import PhysLean.Relativity.LorentzAlgebra.Basis
public import PhysLean.Relativity.LorentzAlgebra.ExponentialMap
public import PhysLean.Relativity.LorentzGroup.Basic
public import PhysLean.Relativity.LorentzGroup.Boosts.Apply
public import PhysLean.Relativity.LorentzGroup.Boosts.Basic
public import PhysLean.Relativity.LorentzGroup.Boosts.Generalized
public import PhysLean.Relativity.LorentzGroup.Orthochronous.Basic
public import PhysLean.Relativity.LorentzGroup.Proper
public import PhysLean.Relativity.LorentzGroup.Restricted.Basic
public import PhysLean.Relativity.LorentzGroup.Restricted.FromBoostRotation
public import PhysLean.Relativity.LorentzGroup.Rotations
public import PhysLean.Relativity.LorentzGroup.ToVector
public import PhysLean.Relativity.MinkowskiMatrix
public import PhysLean.Relativity.PauliMatrices.AsTensor
public import PhysLean.Relativity.PauliMatrices.Basic
public import PhysLean.Relativity.PauliMatrices.CliffordAlgebra
public import PhysLean.Relativity.PauliMatrices.Relations
public import PhysLean.Relativity.PauliMatrices.SelfAdjoint
public import PhysLean.Relativity.PauliMatrices.ToTensor
public import PhysLean.Relativity.SL2C.Basic
public import PhysLean.Relativity.SL2C.SelfAdjoint
public import PhysLean.Relativity.Special.ProperTime
public import PhysLean.Relativity.Special.TwinParadox.Basic
public import PhysLean.Relativity.SpeedOfLight
public import PhysLean.Relativity.Tensors.Basic
public import PhysLean.Relativity.Tensors.Color.Basic
public import PhysLean.Relativity.Tensors.Color.Discrete
public import PhysLean.Relativity.Tensors.Color.Lift

```

```

public import PhysLean.Relativity.Tensors.ComplexTensor.Basic
public import PhysLean.Relativity.Tensors.ComplexTensor.Lemmas
public import PhysLean.Relativity.Tensors.ComplexTensor.Matrix.Pre
public import PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Basic
public import PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Lemmas
public import PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Pre
public import PhysLean.Relativity.Tensors.ComplexTensor.OfRat
public import PhysLean.Relativity.Tensors.ComplexTensor.Units.Basic
public import PhysLean.Relativity.Tensors.ComplexTensor.Units.Pre
public import PhysLean.Relativity.Tensors.ComplexTensor.Units.Symm
public import PhysLean.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic
public import PhysLean.Relativity.Tensors.ComplexTensor.Vector.Pre.Contract
public import PhysLean.Relativity.Tensors.ComplexTensor.Vector.Pre.Modules
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Basic
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Contraction
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Metric
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Modules
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Two
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Unit
public import PhysLean.Relativity.Tensors.Constructors
public import PhysLean.Relativity.Tensors.Contraction.Basic
public import PhysLean.Relativity.Tensors.Contraction.Basis
public import PhysLean.Relativity.Tensors.Contraction.Products
public import PhysLean.Relativity.Tensors.Contraction.Pure
public import PhysLean.Relativity.Tensors.Dual
public import PhysLean.Relativity.Tensors.Elab
public import PhysLean.Relativity.Tensors.Evaluation
public import PhysLean.Relativity.Tensors.MetricTensor
public import PhysLean.Relativity.Tensors.OfInt
public import PhysLean.Relativity.Tensors.Product
public import PhysLean.Relativity.Tensors.RealTensor.Basic
public import PhysLean.Relativity.Tensors.RealTensor.CoVector.Basic
public import PhysLean.Relativity.Tensors.RealTensor.Derivative
public import PhysLean.Relativity.Tensors.RealTensor.Matrix.Pre
public import PhysLean.Relativity.Tensors.RealTensor.Metrics.Basic
public import PhysLean.Relativity.Tensors.RealTensor.Metrics.Pre
public import PhysLean.Relativity.Tensors.RealTensor.ToComplex
public import PhysLean.Relativity.Tensors.RealTensor.Units.Pre
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Basic
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.Basic
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.Light
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.TimeL
public import PhysLean.Relativity.Tensors.RealTensor.Vector.MinkowskiProduc
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Pre.Basic
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Pre.Contraction
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Pre.Modules

```



```

public import PhysLean.Relativity.Tensors.RealTensor.Velocity.Basic
public import PhysLean.Relativity.Tensors.TensorSpecies.Basic
public import PhysLean.Relativity.Tensors.Tensorial
public import PhysLean.Relativity.Tensors.UnitTensor
public import PhysLean.SpaceAndTime.Space.Basic
public import PhysLean.SpaceAndTime.Space.ConstantSliceDist
public import PhysLean.SpaceAndTime.Space.CrossProduct
public import PhysLean.SpaceAndTime.Space.Derivatives.Basic
public import PhysLean.SpaceAndTime.Space.Derivatives.Curl
public import PhysLean.SpaceAndTime.Space.Derivatives.Div
public import PhysLean.SpaceAndTime.Space.Derivatives.Grad
public import PhysLean.SpaceAndTime.Space.Derivatives.Laplacian
public import PhysLean.SpaceAndTime.Space.DistConst
public import PhysLean.SpaceAndTime.Space.DistOfFunction
public import PhysLean.SpaceAndTime.Space.Integrals.Basic
public import PhysLean.SpaceAndTime.Space.Integrals.NormPow
public import PhysLean.SpaceAndTime.Space.Integrals.RadialAngularMeasure
public import PhysLean.SpaceAndTime.Space.IsDistBounded
public import PhysLean.SpaceAndTime.Space.LengthUnit
public import PhysLean.SpaceAndTime.Space.Module
public import PhysLean.SpaceAndTime.Space.Norm
public import PhysLean.SpaceAndTime.Space.Slice
public import PhysLean.SpaceAndTime.Space.Translations
public import PhysLean.SpaceAndTime.SpaceTime.Basic
public import PhysLean.SpaceAndTime.SpaceTime.Boosts
public import PhysLean.SpaceAndTime.SpaceTime.Derivatives
public import PhysLean.SpaceAndTime.SpaceTime.LorentzAction
public import PhysLean.SpaceAndTime.SpaceTime.TimeSlice
public import PhysLean.SpaceAndTime.Time.Basic
public import PhysLean.SpaceAndTime.Time.Derivatives
public import PhysLean.SpaceAndTime.Time.TimeMan
public import PhysLean.SpaceAndTime.Time.TimeTransMan
public import PhysLean.SpaceAndTime.Time.TimeUnit
public import PhysLean.SpaceAndTime.TimeAndSpace.Basic
public import PhysLean.SpaceAndTime.TimeAndSpace.ConstantTimeDist
public import PhysLean.StatisticalMechanics.BoltzmannConstant
public import PhysLean.StatisticalMechanics.CanonicalEnsemble.Basic
public import PhysLean.StatisticalMechanics.CanonicalEnsemble.Finite
public import PhysLean.StatisticalMechanics.CanonicalEnsemble.Lemmas
public import PhysLean.StatisticalMechanics.CanonicalEnsemble.TwoState
public import PhysLean.StringTheory.Basic
public import PhysLean.StringTheory.FTheory.SU5.Basic
public import PhysLean.StringTheory.FTheory.SU5.Charges.AnomalyFree
public import PhysLean.StringTheory.FTheory.SU5.Charges.OfRationalSection
public import PhysLean.StringTheory.FTheory.SU5.Charges.Viable
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.Basic

```

```

public import PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.ChiralIndi
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Completene
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems
public import PhysLean.StringTheory.FTheory.SU5.Quanta.Basic
public import PhysLean.StringTheory.FTheory.SU5.Quanta.FiveQuanta
public import PhysLean.StringTheory.FTheory.SU5.Quanta.IsViable
public import PhysLean.StringTheory.FTheory.SU5.Quanta.TenQuanta
public import PhysLean.Thermodynamics.Basic
public import PhysLean.Thermodynamics.IdealGas.Basic
public import PhysLean.Thermodynamics.Temperature.Basic
public import PhysLean.Thermodynamics.Temperature.TemperatureUnits
public import PhysLean.Units.Basic
public import PhysLean.Units.Dimension
public import PhysLean.Units.Examples
public import PhysLean.Units.FDeriv
public import PhysLean.Units.Integral
public import PhysLean.Units.UnitDependent
public import PhysLean.Units.WithDim.Area
public import PhysLean.Units.WithDim.Basic
public import PhysLean.Units.WithDim.Energy
public import PhysLean.Units.WithDim.Mass
public import PhysLean.Units.WithDim.Momentum
public import PhysLean.Units.WithDim.Pressure
public import PhysLean.Units.WithDim.Speed
public import PhysLean.Units.WithDim.Velocity

```

1.5 PhysLean/ClassicalMechanics/Basic.lean

```
module
```

```
-/@[expose] public section
```

1.6 PhysLean/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean

```
module
```

```

public import PhysLean.Meta.Informal.SemiFormal
public import PhysLean.ClassicalMechanics.EulerLagrange
public import PhysLean.ClassicalMechanics.HamiltonsEquations
@[expose] public section

```

```
open ContDiff
```

```
open Time
```

```

/-- Equalities for energy eqns -/
lemma kineticEnergy_eq (x: Time → ℝ) :
  kineticEnergy S x = fun t => (1 / 2 : ℝ) * S.m * (Time.deriv x t)^2 := by rfl

lemma potentialEnergy_eq (x: Time → ℝ) :
  potentialEnergy S x = fun t => (1 / 2 : ℝ) * S.k * (x t)^2 := by rfl

/-- Derives the energy dissipation rate from the equation of motion -
/
lemma energy_dissipation_rate (x: Time → ℝ) (h1 : S.EquationOfMotion x)
  (hx : ContDiff ℝ ∞ x) :
  Time.deriv (energy S x) t = - S.γ * (Time.deriv x t)^2 := by

  -- Rearrange Equation of Motion
  have heom' : S.m * Time.deriv (Time.deriv x) t + S.k * x t =
    - S.γ * Time.deriv x t := by linarith [h1 t]

  -- Break equation apart
  rw [energy, kineticEnergy_eq, potentialEnergy_eq]
  rw [Time.deriv_eq]

  -- Perform derivative
  have hdx : Differentiable ℝ x := hx.differentiable (by norm_num)
  have hddx : Differentiable ℝ (∂t x) := deriv_differentiable_of_contDiff x hx
  have hKE := ((hddx t).hasFDerivAt.pow 2).const_mul (1/2 * S.m)
  have hPE := ((hdx t).hasFDerivAt.pow 2).const_mul (1/2 * S.k)
  rw [(hKE.add hPE).fderiv]
  norm_num
  rw [← Time.deriv, ← Time.deriv]
  linear_combination (Time.deriv x t) * heom'

lemma energy_not_conserved (x: Time → ℝ) (h1 : S.EquationOfMotion x)
  (hx : ContDiff ℝ ∞ x)
  (hdx : Time.deriv x t ≠ 0)
  (hy : 0 < S.γ) :
  Time.deriv (energy S x) t < 0 := by
  rw [energy_dissipation_rate S x h1 hx]
  rw [neg_mul S.γ (∂t x t ^ 2)]
  refine neg_neg_of_pos (mul_pos hy (sq_pos_iff.mpr hdx))

```

1.7 PhysLean/ClassicalMechanics/EulerLagrange.lean

module

```
public import PhysLean.Mathematics.VariationalCalculus.HasVarGradient
@[expose] public section
```

1.8 PhysLean/ClassicalMechanics/HamiltonsEquations.lean

```
module
```

```
public import PhysLean.Mathematics.VariationalCalculus.HasVarGradient
@[expose] public section
```

1.9 PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean

```
module
```

```
public import PhysLean.ClassicalMechanics.EulerLagrange
public import PhysLean.ClassicalMechanics.HamiltonsEquations
```

In this file, a coordinate system is assumed where position and velocity both live in $\text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}$. This is a simpler model often used for pedagogy but only works because both the configuration space (position) and its tangent space are isomorphic to Euclidean Space. A proper formalisation should include the identification of the state space via manifolds and tangent bundles.

```
@[expose] public section
```

```
TODO "4DK2M" "Create a new file for the geometric model which properly models
configuration space and velocity as its tangent space, then show explicitly that the
coordinate model is a simplification of the geometric model.
```

```
A nice reference for such an analysis is:
```

```
https://web.williams.edu/Mathematics/it3/texts/var\_noether.pdf"
```

```
noncomputable def kineticEnergy (x_t : Time → EuclideanSpace ℝ (Fin 1)) : Time → ℝ :=
noncomputable def potentialEnergy (x : EuclideanSpace ℝ (Fin 1)) : ℝ :=
noncomputable def energy (x_t : Time → EuclideanSpace ℝ (Fin 1)) : Time → ℝ :=
lemma kineticEnergy_eq (x_t : Time → EuclideanSpace ℝ (Fin 1)) :
lemma potentialEnergy_eq (x : EuclideanSpace ℝ (Fin 1)) :
lemma energy_eq (x_t : Time → EuclideanSpace ℝ (Fin 1)) :
lemma kineticEnergy_differentiable (x_t : Time → EuclideanSpace ℝ (Fin 1)) :
lemma potentialEnergy_differentiable (x_t : Time → EuclideanSpace ℝ (Fin 1)) :
lemma energy_differentiable (x_t : Time → EuclideanSpace ℝ (Fin 1)) (hx : Continuous x_t) :
lemma kineticEnergy_deriv (x_t : Time → EuclideanSpace ℝ (Fin 1)) (hx : Continuous x_t) :
```

1.9

PHYSLEAN/CLASSICALMECHANICS/HARMONICOSCILLATOR/BASIC.LEAN

```

lemma potentialEnergy_deriv (x_t : Time → EuclideanSpace ℝ (Fin 1)) (hx : ContDiff ℝ
lemma energy_deriv (x_t : Time → EuclideanSpace ℝ (Fin 1)) (hx : ContDiff ℝ ∞ x_t) :
noncomputable def lagrangian (t : Time) (x : EuclideanSpace ℝ (Fin 1))
  (v : EuclideanSpace ℝ (Fin 1)) : ℝ :=
  (x_t : Time → EuclideanSpace ℝ (Fin 1)) :
lemma toDual_symm_innerSL (x : EuclideanSpace ℝ (Fin 1)) :
  (InnerProductSpace.toDual ℝ (EuclideanSpace ℝ (Fin 1))).symm (innerSL ℝ x) = x :
  apply (innerSL_inj (λ:=ℝ) (E:=EuclideanSpace ℝ (Fin 1))).1
lemma gradient_inner_self (x : EuclideanSpace ℝ (Fin 1)) :
  gradient (fun y : EuclideanSpace ℝ (Fin 1) => ⟨y, y⟩_ℝ) x = (2 : ℝ) • x := by
  have hid : DifferentiableAt ℝ (fun y : EuclideanSpace ℝ (Fin 1) => y) x := differ
  rw [show (fun y : EuclideanSpace ℝ (Fin 1) => ⟨y, y⟩_ℝ) =
    simp only [fderiv_id', ContinuousLinearMap.coe_id', id_eq, real_inner_comm, inner_
      ringHom_apply]
lemma gradient_const_mul_inner_self (c : ℝ) (x : EuclideanSpace ℝ (Fin 1)) :
  gradient (fun y : EuclideanSpace ℝ (Fin 1) => c * ⟨y, y⟩_ℝ) x = (2 * c) • x := b
  gradient (fun y : EuclideanSpace ℝ (Fin 1) => c * ⟨y, y⟩_ℝ) x
    = (InnerProductSpace.toDual ℝ (EuclideanSpace ℝ (Fin 1))).symm
      (fderiv ℝ (fun y : EuclideanSpace ℝ (Fin 1) => c * ⟨y, y⟩_ℝ) x) := rfl
  _ = (InnerProductSpace.toDual ℝ (EuclideanSpace ℝ (Fin 1))).symm
      (c • fderiv ℝ (fun y : EuclideanSpace ℝ (Fin 1) => ⟨y, y⟩_ℝ) x) := by
    rw [fderiv_const_mul]; fun_prop
  _ = c • gradient (fun y : EuclideanSpace ℝ (Fin 1) => ⟨y, y⟩_ℝ) x := by
private lemma gradient_add_const' {f : EuclideanSpace ℝ (Fin 1) → ℝ} {c : ℝ}
  (x : EuclideanSpace ℝ (Fin 1)) :
lemma gradient_lagrangian_position_eq (t : Time) (x : EuclideanSpace ℝ (Fin 1))
  (v : EuclideanSpace ℝ (Fin 1)) :
  have h_eq : (fun y : EuclideanSpace ℝ (Fin 1) => lagrangian S t y v) =
    ext; simp
lemma gradient_lagrangian_velocity_eq (t : Time) (x : EuclideanSpace ℝ (Fin 1))
  (v : EuclideanSpace ℝ (Fin 1)) :
  have h_eq : (fun y : EuclideanSpace ℝ (Fin 1) => lagrangian S t x y) =
    change gradient (fun y : EuclideanSpace ℝ (Fin 1) => lagrangian S t x y) v = S.m •
    ext; simp
noncomputable def gradLagrangian (x_t : Time → EuclideanSpace ℝ (Fin 1)) :
  Time → EuclideanSpace ℝ (Fin 1) :=
lemma gradLagrangian_eq_eulerLagrangeOp (x_t : Time → EuclideanSpace ℝ (Fin 1))
def EquationOfMotion (x_t : Time → EuclideanSpace ℝ (Fin 1)) : Prop :=
lemma equationOfMotion_iff_gradLagrangian_zero (x_t : Time → EuclideanSpace ℝ (Fin 1))
noncomputable def force (S : HarmonicOscillator) (x : EuclideanSpace ℝ (Fin 1)) :
  EuclideanSpace ℝ (Fin 1) :=
lemma force_eq_linear (x : EuclideanSpace ℝ (Fin 1)) : force S x = - S.k • x := by
  have hpot : (fun y : EuclideanSpace ℝ (Fin 1) => (1 / (2 : ℝ)) • S.k • ⟨y, y⟩_ℝ) =
  have hgrad : gradient (fun y : EuclideanSpace ℝ (Fin 1) => ((1 / (2 : ℝ)) * S.k) *
lemma gradLagrangian_eq_force (x_t : Time → EuclideanSpace ℝ (Fin 1)) (hx : ContDiff
lemma equationOfMotion_iff_newtons_2nd_law (x_t : Time → EuclideanSpace ℝ (Fin 1))

```

```

lemma energy_conservation_of_equationOfMotion (xt : Time → EuclideanSpace ℝ) :
lemma energy_conservation_of_equationOfMotion' (xt : Time → EuclideanSpace ℝ) :
noncomputable def toCanonicalMomentum (t : Time) (x : EuclideanSpace ℝ (Fin 1)) :
  EuclideanSpace ℝ (Fin 1) ≈1[ℝ] EuclideanSpace ℝ (Fin 1) where
lemma toCanonicalMomentum_eq (t : Time) (x : EuclideanSpace ℝ (Fin 1))
  (v : EuclideanSpace ℝ (Fin 1)) :
noncomputable def hamiltonian (t : Time) (p : EuclideanSpace ℝ (Fin 1))
  (x : EuclideanSpace ℝ (Fin 1)) : ℝ :=
lemma gradient_hamiltonian_position_eq (t : Time) (x : EuclideanSpace ℝ (Fin 1))
  (p : EuclideanSpace ℝ (Fin 1)) :
  have h_eq : (fun y : EuclideanSpace ℝ (Fin 1) => hamiltonian S t p y) =
    funext y; unfold hamiltonian; simp only [toCanonicalMomentum, lagrangian,
      inner_self_eq_norm_sq_to_K, ringHom_apply, potentialEnergy, smul_eq_mul,
      LinearEquiv.coe_symm_mk', inner_smul_right, norm_smul, norm_inv, norm_nsmul,
      ring_nf
    ]
    simp [mul_two]
  change gradient (fun y : EuclideanSpace ℝ (Fin 1) => hamiltonian S t p y)
  ext; simp
lemma gradient_hamiltonian_momentum_eq (t : Time) (x : EuclideanSpace ℝ (Fin 1))
  (p : EuclideanSpace ℝ (Fin 1)) :
  have h_eq : (fun y : EuclideanSpace ℝ (Fin 1) => hamiltonian S t y x) =
    funext y; unfold hamiltonian; simp only [toCanonicalMomentum, lagrangian,
      inner_self_eq_norm_sq_to_K, ringHom_apply, potentialEnergy, smul_eq_mul,
      LinearEquiv.coe_symm_mk', inner_smul_right, norm_smul, norm_inv, norm_nsmul,
      ring_nf
    ]
    simp [mul_two]
  change gradient (fun y : EuclideanSpace ℝ (Fin 1) => hamiltonian S t y x)
  ext; simp
lemma hamiltonian_eq_energy (xt : Time → EuclideanSpace ℝ (Fin 1)) :
  simp only [toCanonicalMomentum_eq, inner_smul_left, one_div, smul_eq_mul]
  simp only [ringHom_apply, inner_self_eq_norm_sq_to_K, one_div, add_left_i,
    field_simps
  ]
  ring
noncomputable def hamiltonEqOp (p : Time → EuclideanSpace ℝ (Fin 1))
  (q : Time → EuclideanSpace ℝ (Fin 1)) :=
lemma equationOfMotion_iff_hamiltonEqOp_eq_zero (xt : Time → EuclideanSpace ℝ) :
lemma equationOfMotion_tfae (xt : Time → EuclideanSpace ℝ (Fin 1)) (hx : Co

```

1.10 PhysLean/ClassicalMechanics/HarmonicOscillator/Configurations

module

```

public import Mathlib.Geometry.Manifold.Diffeomorphisms
public import PhysLean.SpaceAndTime.Time.Basic

```

1.11.

PHYSLEAN/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION1LEAN

@[expose] public section

TODO "4DLKC" "Configuration Space should be refactored to take $\mathbb{R}$ (Fin 1) as its value."

TODO "4DLL5" "The API around this should be improved to allow further development of geometric model of the Harmonic Oscillator (see TODO item 4DK2M)."

1.11 PhysLean/ClassicalMechanics/HarmonicOscillator/Solution.lean

module

public import Mathlib.Analysis.CStarAlgebra.Classes

public import PhysLean.ClassicalMechanics.HarmonicOscillator.Basic

@[expose] public section

x_0 : EuclideanSpace $\mathbb{R}$ (Fin 1)

v_0 : EuclideanSpace $\mathbb{R}$ (Fin 1)

x_{t_0} : EuclideanSpace $\mathbb{R}$ (Fin 1)

v_{t_0} : EuclideanSpace $\mathbb{R}$ (Fin 1)

noncomputable def trajectory (IC : InitialConditions) : Time $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin 1)

lemma trajectories_unique (IC : InitialConditions) (x : Time $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin 1))

set y : Time $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin 1) := fun t $\Rightarrow$ x t - IC.trajectory S t with h

ext i

simp only [smul_add, PiLp.add_apply, PiLp.smul_apply, PiLp.sub_apply, smul_eq_mul]

nth_rw 2 [$\leftarrow$ mul_one (S. ω * IC. x_{t_0} .ofLp i)]

rw [$\leftarrow$ h1]

ring

ext i

simp only [neg_smul, smul_add, PiLp.add_apply, PiLp.neg_apply, PiLp.smul_apply, PiLp.smul_eq_mul]

nth_rw 3 [$\leftarrow$ mul_one (IC. v_{t_0} .ofLp i)]

rw [$\leftarrow$ h1]

ring

ext i

fin_cases i

simp only [Fin.isValue, PiLp.zero_apply, add_zero]

· simp only [Fin.isValue, PiLp.add_apply, PiLp.neg_apply, PiLp.smul_apply, smul_eq_mul]

ext i

· fin_cases i

simp only [Fin.isValue, Fin.zero_eta]

ring

ext i

```
fin_cases i
```

1.12 PhysLean/ClassicalMechanics/Lagrangian/TotalDerivativeEqui

```
module
```

```
public import PhysLean.Mathematics.VariationalCalculus.HasVarGradient
@[expose] public section
```

1.13 PhysLean/ClassicalMechanics/Mass/MassUnit.lean

```
module
```

```
public import Mathlib.Geometry.Manifold.Diffeomorph
public import PhysLean.SpaceAndTime.Time.Basic
@[expose] public section
```

1.14 PhysLean/ClassicalMechanics/Pendulum/CoplanarDoublePendulum

```
module
```

```
public import PhysLean.Meta.Informal.Basic
public import PhysLean.Meta.Sorry
@[expose] public section
```

1.15 PhysLean/ClassicalMechanics/Pendulum/MiscellaneousPendulum

```
module
```

```
-/@[expose] public section
```

1.16 PhysLean/ClassicalMechanics/Pendulum/SlidingPendulum.lean

```
module
```

```
public import PhysLean.Meta.Sorry
```



```
@[expose] public section
```

1.17 PhysLean/ClassicalMechanics/RigidBody/Basic.lean

```
module
```

```
public import PhysLean.SpaceAndTime.Space.Derivatives.Curl  
public import Mathlib.Geometry.Manifold.Algebra.SmoothFunctions  
@[expose] public section
```

1.18 PhysLean/ClassicalMechanics/RigidBody/SolidSphere.lean

```
module
```

```
public import PhysLean.ClassicalMechanics.RigidBody.Basic  
@[expose] public section
```

1.19 PhysLean/ClassicalMechanics/Scattering/RigidSphere.lean

```
module
```

```
public import PhysLean.Meta.TODO.Basic  
@[expose] public section
```

1.20 PhysLean/ClassicalMechanics/Vibrations/LinearTriatomic.lean

```
module
```

```
public import PhysLean.Meta.TODO.Basic  
@[expose] public section
```

1.21 PhysLean/ClassicalMechanics/WaveEquation/Basic.lean

```
module
```

```
public import PhysLean.SpaceAndTime.Space.CrossProduct
public import PhysLean.SpaceAndTime.TimeAndSpace.Basic
@[expose] public section
```

1.22 PhysLean/ClassicalMechanics/WaveEquation/HarmonicWave.lean

```
module
```

```
public import PhysLean.ClassicalMechanics.WaveEquation.Basic
@[expose] public section
```

1.23 PhysLean/CondensedMatter/Basic.lean

```
module
```

```
-/@[expose] public section
```

1.24 PhysLean/CondensedMatter/TightBindingChain/Basic.lean

```
module
```

```
public import PhysLean.Meta.Informal.Basic
public import PhysLean.Meta.Informal.SemiFormal
public import PhysLean.Meta.Linters.Sorry
public import PhysLean.QuantumMechanics.FiniteTarget.HilbertSpace
@[expose] public section
```

1.25 PhysLean/Cosmology/Basic.lean

```
module
```

```
-/@[expose] public section
```

1.26 PhysLean/Cosmology/FLRW/Basic.lean

module

```
public import PhysLean.SpaceAndTime.Space.Basic
public import Mathlib.Analysis.SpecialFunctions.Trigonometric.DerivHyp
@[expose] public section
```

1.27 PhysLean/Electromagnetism/Basic.lean

module

```
public import PhysLean.SpaceAndTime.SpaceTime.Basic
@[expose] public section
```

1.28 PhysLean/Electromagnetism/Charge/ChargeUnit.lean

module

```
public import Mathlib.Algebra.EuclideanDomain.Basic
public import Mathlib.Algebra.EuclideanDomain.Field
public import Mathlib.Analysis.RCLike.Basic
public import PhysLean.Meta.TODO.Basic
@[expose] public section
```

1.29 PhysLean/Electromagnetism/Current/CircularCoil.lean

module

```
public import PhysLean.Electromagnetism.Dynamics.IsExtrema
public import PhysLean.SpaceAndTime.Space.Norm
public import PhysLean.SpaceAndTime.Space.Translations
public import PhysLean.SpaceAndTime.Space.ConstantSliceDist
public import PhysLean.SpaceAndTime.TimeAndSpace.ConstantTimeDist
@[expose] public section
```

1.30 PhysLean/Electromagnetism/Current/InfiniteWire.lean

module

```
public import PhysLean.Electromagnetism.Dynamics.IsExtrema
public import PhysLean.SpaceAndTime.Space.Norm
public import PhysLean.SpaceAndTime.Space.Translations
public import PhysLean.SpaceAndTime.Space.ConstantSliceDist
public import PhysLean.SpaceAndTime.TimeAndSpace.ConstantTimeDist
@[expose] public section
```

1.31 PhysLean/Electromagnetism/Dynamics/Basic.lean

module

```
public import PhysLean.Electromagnetism.Basic
@[expose] public section
```

1.32 PhysLean/Electromagnetism/Dynamics/CurrentDensity.lean

module

```
public import PhysLean.SpaceAndTime.SpaceTime.TimeSlice
@[expose] public section
```

1.33 PhysLean/Electromagnetism/Dynamics/Hamiltonian.lean

module

```
public import PhysLean.Electromagnetism.Dynamics.Lagrangian
@[expose] public section
```

1.34 PhysLean/Electromagnetism/Dynamics/IsExtrema.lean

module

```
public import PhysLean.Electromagnetism.Dynamics.Lagrangian
```

1.35. *PHYSLEAN/ELECTROMAGNETISM/DYNAMICS/KINETICTERM.LEAN*²³

@[expose] public section

1.35 PhysLean/Electromagnetism/Dynamics/KineticTerm.lean

module

```
public import PhysLean.Electromagnetism.Kinematics.MagneticField
public import PhysLean.Electromagnetism.Dynamics.Basic
@[expose] public section
```

1.36 PhysLean/Electromagnetism/Dynamics/Lagrangian.lean

module

```
public import PhysLean.Electromagnetism.Dynamics.CurrentDensity
public import PhysLean.Electromagnetism.Dynamics.KineticTerm
@[expose] public section
```

1.37 PhysLean/Electromagnetism/Kinematics/Boosts.lean

module

```
public import PhysLean.Electromagnetism.Kinematics.MagneticField
public import PhysLean.SpaceAndTime.SpaceTime.Boosts
@[expose] public section
```

1.38 PhysLean/Electromagnetism/Kinematics/EMPotential.lean

module

```
public import PhysLean.Electromagnetism.Basic
public import PhysLean.SpaceAndTime.SpaceTime.TimeSlice
public import PhysLean.Mathematics.VariationalCalculus.HasVarGradient
@[expose] public section
```

1.39 PhysLean/Electromagnetism/Kinematics/ElectricField.lean

```
module
```

```
public import PhysLean.Electromagnetism.Kinematics.VectorPotential
public import PhysLean.Electromagnetism.Kinematics.ScalarPotential
public import PhysLean.Electromagnetism.Kinematics.FieldStrength
```

```
@[expose] public section
```

1.40 PhysLean/Electromagnetism/Kinematics/FieldStrength.lean

```
module
```

```
public import PhysLean.Electromagnetism.Kinematics.EMPotential
```

```
@[expose] public section
```

1.41 PhysLean/Electromagnetism/Kinematics/MagneticField.lean

```
module
```

```
public import PhysLean.Electromagnetism.Kinematics.ElectricField
` \vec B` as `(∇ × (A.vectorPotential t)) x`.
```

```
@[expose] public section
```

```
fun t x => (∇ × (A.vectorPotential c t)) x
A.magneticField c = fun t x => (∇ × (A.vectorPotential c t)) x := rfl
(∇ × A.magneticField c t) x i = ∑ j, Space.deriv j (A.magneticFieldMatr
```

1.42 PhysLean/Electromagnetism/Kinematics/ScalarPotential.lean

```
module
```

```
public import PhysLean.Electromagnetism.Kinematics.EMPotential
```

```
@[expose] public section
```

1.43 PhysLean/Electromagnetism/Kinematics/VectorPotential.lean

```
module
```

1.44.

PHYSLEAN/ELECTROMAGNETISM/POINTPARTICLE/ONEDIMENSION.LEAN

```
public import PhysLean.Electromagnetism.Kinematics.EMPotential
@[expose] public section
```

1.44 PhysLean/Electromagnetism/PointParticle/OneDimension.lean

module

```
public import PhysLean.Electromagnetism.Dynamics.IsExtrema
public import PhysLean.SpaceAndTime.Space.Norm
public import PhysLean.SpaceAndTime.Space.Translations
public import PhysLean.SpaceAndTime.TimeAndSpace.ConstantTimeDist
@[expose] public section
```

1.45 PhysLean/Electromagnetism/PointParticle/ThreeDimension.lean

module

```
public import PhysLean.Electromagnetism.Dynamics.IsExtrema
public import PhysLean.SpaceAndTime.Space.Norm
public import PhysLean.SpaceAndTime.Space.Translations
public import PhysLean.SpaceAndTime.TimeAndSpace.ConstantTimeDist
@[expose] public section
```

```
  simp only [map_smul, smul_smul]
  ext η
  simp [constantTime_apply, diracDelta_apply, distTranslate_apply]
  left
  ring_nf
  field_simp
```

1.46 PhysLean/Electromagnetism/ThreeDimension/Basic.lean

/-

Copyright (c) 2026 Zhi Kai Pong. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Zhi Kai Pong

-/

module

```

public import PhysLean.Electromagnetism.Kinematics.MagneticField
/-!

# Three-Dimensional Electromagnetism

This directory provides a three-dimensional, vector-calculus-
facing layer for
electromagnetism.

The backend theory is formulated in a tensorial and dimension-
general way.
Here we re-express the relevant constructions in the familiar language of
scalar and vector potentials, electric and magnetic fields, and spatial
derivatives.

-/

@[expose] public section

namespace Electromagnetism
namespace ThreeDimension

open Time
open Space
open ElectromagneticPotential

variable (c : SpeedOfLight) (V : ElectromagneticPotential 3)

local notation "φ" => V.scalarPotential c
local notation "A" => V.vectorPotential c
local notation "E" => V.electricField c

/-!

# Fields from potentials

In this section we rewrite the electric and magnetic fields associated to a
electromagnetic potential in terms of the scalar and vector potentials, using
the standard vector-calculus expressions.

-/

/-- The electric field written in terms of the scalar and vector potentials
/
theorem electricField_eq_3D :
  E = fun t x => - ∇ (φ t) x - ∂t (fun t => A t x) t := by

```



```

    rw [electricField_eq]

local notation "B" => V.magneticField c

/-- The magnetic field written as the curl of the vector potential as  $\nabla \times A$ . -
/
theorem magneticField_eq_3D :
  B = fun t x => ( $\nabla \times (A\ t)$ ) x := by
  rw [magneticField_eq]

end ThreeDimension
end Electromagnetism

```

1.47 PhysLean/Electromagnetism/Vacuum/Constant.lean

```

module

public import PhysLean.Electromagnetism.Dynamics.IsExtrema
@[expose] public section

```

1.48 PhysLean/Electromagnetism/Vacuum/HarmonicWave.lean

```

module

public import PhysLean.Electromagnetism.Vacuum.IsPlaneWave

@[expose] public section

```

1.49 PhysLean/Electromagnetism/Vacuum/IsPlaneWave.lean

```

module

public import PhysLean.ClassicalMechanics.WaveEquation.Basic
public import PhysLean.Electromagnetism.Dynamics.IsExtrema
@[expose] public section

```

1.50 PhysLean/Mathematics/Calculus/AdjFDeriv.lean

```

module

```

```
public import Mathlib.Analysis.Calculus.Gradient.Basic
public import PhysLean.Mathematics.FDerivCurry
public import PhysLean.Mathematics.InnerProductSpace.Adjoint
public import PhysLean.Mathematics.InnerProductSpace.Calculus
@[expose] public section
```

1.51 PhysLean/Mathematics/Calculus/Divergence.lean

```
module
```

```
public import Mathlib.Analysis.InnerProductSpace.Trace
public import PhysLean.Mathematics.Calculus.AdjFDeriv
public import PhysLean.SpaceAndTime.TimeAndSpace.Basic
```

```
@[expose] public section
```

1.52 PhysLean/Mathematics/DataStructures/FourTree/Basic.lean

```
module
```

```
public import Mathlib.Data.Multiset.Bind
public import Mathlib.Data.Multiset.Sort
@[expose] public section
```

1.53 PhysLean/Mathematics/DataStructures/FourTree/UniqueMap.lean

```
module
```

```
public import PhysLean.Mathematics.DataStructures.FourTree.Basic
@[expose] public section
```

1.54 PhysLean/Mathematics/DataStructures/Matrix/LieTrace.lean

```
module
```

```
public import Mathlib.Analysis.Complex.Polynomial.Basic
```

```
public import Mathlib.Analysis.Normed.Algebra.MatrixExponential
public import PhysLean.Mathematics.SchurTriangulation
@[expose] public section
```

1.55 PhysLean/Mathematics/Distribution/Basic.lean

```
module
```

```
public import Mathlib.Analysis.Distribution.TemperedDistribution
public import PhysLean.Meta.TODO.Basic
@[expose] public section
```

1.56 PhysLean/Mathematics/Distribution/PowMul.lean

```
module
```

```
public import PhysLean.Mathematics.Distribution.Basic
public import Mathlib.MeasureTheory.Constructions.HaarToSphere
```

```
@[expose] public section
lemma norm_iteratedFDeriv_ofRealCLM {x} (i : ℕ) :
  · apply le_trans ?_ (ψ.le_seminorm □ _ _ x)
  · exact ψ.le_seminorm □ k n x
  · exact ψ.le_seminorm □ (k + 1) (n + 1) x
```

1.57 PhysLean/Mathematics/FDerivCurry.lean

```
module
```

```
public import Mathlib.Analysis.Calculus.FDeriv.Symmetric
```

```
@[expose] public section
```

1.58 PhysLean/Mathematics/Fin.lean

```
module
```

```
public import Mathlib.Tactic.Polyrith
```

```

public import Mathlib.Tactic.Linarith
public import Mathlib.Logic.Equiv.Fin.Basic

@[expose] public section

```

1.59 PhysLean/Mathematics/Fin/Involutions.lean

```

module

public import Mathlib.Tactic.Polyrith
public import Mathlib.Tactic.Linarith
public import Mathlib.Data.Nat.Factorial.DoubleFactorial
public import Mathlib.Data.Finset.Sort
public import Mathlib.Data.Fintype.Pi
public import Mathlib.Data.Fintype.Prod

@[expose] public section

```

1.60 PhysLean/Mathematics/Geometry/Metric/PseudoRiemannian/Defs

```

module
public import Mathlib.Analysis.InnerProductSpace.Basic
public import Mathlib.Analysis.RCLike.Lemmas
public import Mathlib.Geometry.Manifold.MFDeriv.Defs
public import Mathlib.LinearAlgebra.BilinearForm.Properties
public import Mathlib.LinearAlgebra.QuadraticForm.Real
public import Mathlib.Topology.LocallyConstant.Basic
@[expose] public section

def pseudoRiemannianMetricValToQuadraticForm

```

1.61 PhysLean/Mathematics/Geometry/Metric/Riemannian/Defs.lean

```

module

public import PhysLean.Mathematics.Geometry.Metric.PseudoRiemannian.Defs
public import Mathlib.Algebra.Lie.OfAssociative
public import Mathlib.MeasureTheory.Integral.IntervalIntegral.Basic
@[expose] public section

```

1.62. *PHYSLEAN/MATHEMATICS/INNERPRODUCTSPACE/ADJOINT.LEAN*31

1.62 PhysLean/Mathematics/InnerProductSpace/Adjoint.lean

module

```
public import PhysLean.Mathematics.InnerProductSpace.Basic
public import Mathlib.Analysis.InnerProductSpace.Adjoint
```

@[expose] public section

1.63 PhysLean/Mathematics/InnerProductSpace/Basic.lean

module

```
public import Mathlib.Analysis.InnerProductSpace.Calculus
public import Mathlib.Analysis.InnerProductSpace.Completion
public import Mathlib.Analysis.InnerProductSpace.ProdL2
public import Mathlib.Data.Real.CompleteField
public import Mathlib.Data.Real.StarOrdered
@[expose] public section
```

1.64 PhysLean/Mathematics/InnerProductSpace/Calculus.lean

module

```
public import PhysLean.Mathematics.InnerProductSpace.Basic
public import Mathlib.Analysis.InnerProductSpace.Adjoint
```

@[expose] public section

1.65 PhysLean/Mathematics/InnerProductSpace/Submodule.lean

module

```
public import Mathlib.Analysis.InnerProductSpace.LinearPMap
# Submodules of  $E \times F$ 
In this module we define submoduleToLp which reinterprets a submodule of  $E \times F$ ,
where  $E$  and  $F$  are inner product spaces, as a submodule of  $\text{WithLp } 2 (E \times F)$ .
by default  $E \times F$  is given the sup norm.
@[expose] public section
```

```
{F : Type*} [NormedAddCommGroup F] [InnerProductSpace  $\mathbb{C}$  F]
```

```

    (M : Submodule ℂ (E × F))
    (f : E × F) (g : F × E)
/-- The submodule of `WithLp 2 (E × F)` defined by `M`. -
/
def submoduleToLp : Submodule ℂ (WithLp 2 (E × F)) where
lemma mem_submodule_iff_mem_submoduleToLp : f ∈ M ↔ (WithLp.toLp 2 f) ∈ sub
Eq.to_iff rfl
lemma mem_submodule_closure_iff_mem_submoduleToLp_closure :
lemma mem_submodule_adjoint_iff_mem_submoduleToLp_orthogonal :
    g ∈ M.adjoint ↔ WithLp.toLp 2 (g.2, -g.1) ∈ (submoduleToLp M)[] := by
    have h' : inner ℂ u.snd g.1 = inner ℂ u.fst g.2 := by
    have h' : inner ℂ a g.2 = inner ℂ b g.1 := by
lemma mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthog
lemma mem_submodule_closure_adjoint_iff_mem_submoduleToLp_closure_orthogona
    g ∈ M.topologicalClosure.adjoint ↔
    WithLp.toLp 2 (g.2, -g.1) ∈ (submoduleToLp M).topologicalClosure[] := by

```

1.66 PhysLean/Mathematics/KroneckerDelta.lean

module

```

public import Mathlib.Algebra.BigOperators.Group.Finset.Piecewise
public import Mathlib.Algebra.CharZero.Defs
public import Mathlib.Algebra.Field.Defs
public import Mathlib.Algebra.Module.Defs
This module defines the Kronecker delta.

```

```

@[expose] public section
variable {α M : Type*} [DecidableEq α]

```

```

def kroneckerDelta (i j : α) : ℕ := if i = j then 1 else 0

```

```

@[inherit_doc]
notation "δ[" i ", " j "]" => kroneckerDelta i j

```

```

@[simp]
lemma eq_one_of_same (i : α) : δ[i,i] = 1 := if_pos rfl

```

```

lemma eq_zero_of_ne {i j : α} (h : i ≠ j) : δ[i,j] = 0 := if_neg h

```

```

@[simp]
lemma eq_of_coe {p : α → Prop} (i j : Subtype p) : δ[(i : α),j] = δ[i,j] :=
    rcases eq_or_ne i j with (rfl | hne)
    · repeat rw [eq_one_of_same]

```

```

    · rw [eq_zero_of_ne hne, eq_zero_of_ne <| Subtype.coe_ne_coe.mpr hne]

lemma eq_zero_of_not {p :  $\alpha \rightarrow \text{Prop}$ } {i j :  $\alpha$ } (hi :  $\neg p\ i$ ) (hj :  $p\ j$ ) :  $\delta[i,j] = 0 :=$ 
  eq_zero_of_ne (fun h  $\mapsto$  hi (h  $\triangleright$  hj))

/-!
### Conditions for smul to vanish
-/

lemma smul_of_eq_zero [AddMonoid M] (i j :  $\alpha$ ) {f :  $\alpha \rightarrow \alpha \rightarrow M$ } (hf :  $f\ i\ i = 0$ ) :
   $\delta[i,j] \cdot f\ i\ j = 0 :=$  by
  rcases eq_or_ne i j with (rfl | hne)
  · exact smul_eq_zero_of_right _ hf
  · exact smul_eq_zero_of_left (eq_zero_of_ne hne) _

lemma smul_eq_zero_iff [AddMonoid M] (i j :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :
   $\delta[i,j] \cdot f\ i\ j = 0 \Leftrightarrow i \neq j \vee f\ i\ i = 0 :=$  by
  rcases eq_or_ne i j with (rfl | hne)
  · simp [one_nsmul]
  · simp [eq_zero_of_ne, hne]

lemma smul_eq_zero_iff' [AddMonoid M] (i :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :
  ( $\forall j : \alpha, \delta[i,j] \cdot f\ i\ j = 0$ )  $\Leftrightarrow f\ i\ i = 0 :=$  by
  refine (fun h  $\mapsto$  ?_, fun hf j  $\mapsto$  smul_of_eq_zero i j hf)
  simpa [one_nsmul] using h i

lemma smul_eq_zero_iff'' [AddMonoid M] (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :
  ( $\forall i j : \alpha, \delta[i,j] \cdot f\ i\ j = 0$ )  $\Leftrightarrow \forall i : \alpha, f\ i\ i = 0 :=$ 
  forall_congr' fun j  $\mapsto$  smul_eq_zero_iff' j f

/-!
### Symmetrization
-/

lemma symm (i j :  $\alpha$ ) :  $\delta[i,j] = \delta[j,i] :=$  ite_cond_congr <| Eq.propIntro Eq.symm Eq.

lemma smul_symm [AddMonoid M] (i j :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :  $\delta[i,j] \cdot f\ j\ i = \delta[i,j] \cdot$ 
  rcases eq_or_ne i j with (rfl | hne)
  · rfl
  · simp only [eq_zero_of_ne hne, zero_smul]

lemma symmetrize [AddMonoid M] (i j :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :
   $\delta[i,j] \cdot (f\ i\ j + f\ j\ i) = (2 * \delta[i,j]) \cdot f\ i\ j :=$  by
  rcases eq_or_ne i j with (rfl | hne)
  · simp [one_nsmul, two_nsmul]
  · simp [eq_zero_of_ne hne]

```

```

lemma symmetrize' [AddCommMonoid M] {K : Type*} [Semifield K] [CharZero K]
  (i j :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :  $\delta[i,j] \cdot (2 : K)^{-1} \cdot (f\ i\ j + f\ j\ i) = \delta[i,j]$ 
  rcases eq_or_ne i j with (rfl | hne)
  · simp only [eq_one_of_same, one_nsmul, ← two_smul K, smul_smul]
    rw [inv_mul_cancel₀ (OfNat.zero_ne_ofNat 2).symm, one_smul]
  · simp [eq_zero_of_ne hne]

@[simp]
lemma smul_sub_eq_zero [AddGroup M] (i j :  $\alpha$ ) (f :  $\alpha \rightarrow \alpha \rightarrow M$ ) :  $\delta[i,j] \cdot (f\ i\ j - f\ j\ i) = 0$ 
  rcases eq_or_ne i j with (rfl | hne)
  · exact smul_eq_zero_of_right _ (sub_self <| f i i)
  · exact smul_eq_zero_of_left (eq_zero_of_ne hne) _

/-!
### Sums
-/

section Sums
open Finset

variable [AddCommMonoid M]

@[simp]
lemma sum_mul [Fintype  $\alpha$ ] (i j :  $\alpha$ ) :  $\sum k : \alpha, \delta[i,k] * \delta[k,j] = \delta[i,j]$  :=
  simp [kroneckerDelta]

@[simp]
lemma sum_smul [Fintype  $\alpha$ ] (i :  $\alpha$ ) (f :  $\alpha \rightarrow M$ ) :  $\sum j : \alpha, \delta[i,j] \cdot f\ j = f\ i$  :=
  simp [kroneckerDelta, one_nsmul]

lemma sum_sum_smul_eq_zero [Fintype  $\alpha$ ] {f :  $\alpha \rightarrow \alpha \rightarrow M$ } (hf :  $\forall i : \alpha, f\ i\ i = 0$ ) :
   $\sum i : \alpha, \sum j : \alpha, \delta[i,j] \cdot f\ i\ j = 0$  := by
  simp [sum_smul, hf, sum_const_zero]
lemma finset_sum_smul (s : Finset  $\alpha$ ) (i :  $\alpha$ ) (f :  $\alpha \rightarrow M$ ) :
   $\sum j \in s, \delta[i,j] \cdot f\ j = \text{if } i \in s \text{ then } f\ i \text{ else } 0$  := by
  simp [kroneckerDelta, one_nsmul]
lemma finset_sum_sum_smul_eq_zero {s s' : Finset  $\alpha$ } {f :  $\alpha \rightarrow \alpha \rightarrow M$ }
  (hf :  $\forall i \in s \cap s', f\ i\ i = 0$ ) :  $\sum i \in s, \sum j \in s', \delta[i,j] \cdot f\ i\ j = 0$ 
  simp only [finset_sum_smul, Finset.sum_ite_mem]
  rw [← sum_coe_sort]
  simp [hf]
end Sums

```


1.67 PhysLean/Mathematics/LinearMaps.lean

```

module

public import Mathlib.Tactic.Polyrith
public import Mathlib.Algebra.Module.LinearMap.Defs
public import Mathlib.Data.Fintype.BigOperators
public import PhysLean.Meta.TODO.Basic
public import Mathlib.Algebra.Ring.Rat

@[expose] public section

```

1.68 PhysLean/Mathematics/List.lean

```

module

public import PhysLean.Mathematics.Fin
public import Mathlib.Data.Nat.Lattice
public import Mathlib.Data.List.TakeWhile
import all Mathlib.Data.List.Sort
public import Mathlib.Logic.Equiv.Defs
public section

@[expose]
@[expose]
@[expose] def optionErase {I : Type} (l : List I) (i : Option (Fin l.length)) : List I
@[expose]
@[expose] def insertionSortMin {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) : α
@[expose]
@[expose] def optionEraseZ {I : Type} (l : List I) (a : I) (i : Option (Fin l.length)) : List I

```

1.69 PhysLean/Mathematics/List/InsertIdx.lean

```

module

public import Mathlib.Data.List.Sort
public import Mathlib.Algebra.Order.Ring.Nat

@[expose] public section

```

1.70 PhysLean/Mathematics/List/InsertionSort.lean

```

module

```

```
public import PhysLean.Mathematics.List
import all PhysLean.Mathematics.List
import all Mathlib.Data.List.Sort
```

```
@[expose] public section
```

1.71 PhysLean/Mathematics/PiTensorProduct.lean

```
module
```

```
public import Mathlib.LinearAlgebra.PiTensorProduct
```

```
@[expose] public section
```

1.72 PhysLean/Mathematics/RatComplexNum.lean

```
module
```

```
public import Mathlib.Analysis.Complex.Basic
```

```
@[expose] public section
```

1.73 PhysLean/Mathematics/S03/Basic.lean

```
module
```

```
public import Mathlib.LinearAlgebra.Eigenspace.Basic
```

```
public import Mathlib.Analysis.InnerProductSpace.PiL2
```

```
@[expose] public section
```

1.74 PhysLean/Mathematics/SchurTriangulation.lean

```
module
```

```
public import Mathlib.LinearAlgebra.Eigenspace.Triangularizable
```

```
public import Mathlib.Analysis.InnerProductSpace.Adjoint
```

```
@[expose] public section
```

1.75.

PHYSLEAN/MATHEMATICS/SPECIALFUNCTIONS/PHYSHERMITE.LEAN 37

```
noncomputable def schurTriangulationAux :
```

1.75 PhysLean/Mathematics/SpecialFunctions/PhysHermite.lean

```
module
```

```
public import Mathlib.Analysis.Calculus.Deriv.Polynomial
public import Mathlib.Analysis.SpecialFunctions.Gaussian.GaussianIntegral
public import Mathlib.Analysis.SpecialFunctions.Trigonometric.Series
public import Mathlib.Tactic.Cases
@[expose] public section
```

1.76 PhysLean/Mathematics/Trigonometry/Tanh.lean

```
module
```

```
public import Mathlib.Topology.Algebra.Polynomial
public import Mathlib.Analysis.SpecialFunctions.Trigonometric.DerivHyp
public import Mathlib.Analysis.Distribution.SchwartzSpace.Deriv
@[expose] public section
```

1.77 PhysLean/Mathematics/VariationalCalculus/Basic.lean

```
module
```

```
public import PhysLean.Mathematics.VariationalCalculus.IsTestFunction
## Overview of variational calculus
```

The variational calculus API in PhysLib is designed to match and formalize the physics of variational calculus. It is not designed to be a general API for variational calculus.

Within variational calculus we are interested in function transformations, $F : (X \rightarrow \mathbb{R}) \rightarrow \mathbb{R}$. In physics this functional is often of the form $L : (\text{Time} \rightarrow U) \rightarrow \text{Time} \rightarrow \mathbb{R}$, which represents the Lagrangian of a system. We will use this to explain the formalism within this API.

The action is nominally given by
$$S[u] = \int L(u, t) dt,$$
however it is convenient to introduce another function ϕ and define the action as

$$S[u] = \int \varphi(t) L(u, t) dt.$$
In the end we will set $\varphi := \text{fun } _ \Rightarrow 1$.

We now consider $\frac{\partial}{\partial s} S[u + s * \delta u]$ at $s = 0$ which is the variational derivative of S at u in the direction δu . This is equal to

$$\int \varphi(t) * \left. \frac{\partial}{\partial s} L(u + s * \delta u, t) \right|_{s=0} dt$$

Let us denote the function

$$\delta u, \varphi, t \mapsto \left. \frac{\partial}{\partial s} L(u + s * \delta u, t) \right|_{s=0}$$
as $L_u : (\text{Time} \rightarrow U) \rightarrow (\text{Time} \rightarrow \mathbb{R})$.

Then the variational derivative is

$$\int \varphi(t) L_u(\delta u, t) dt.$$

It may then be possible to find a function $G_u : (\text{Time} \rightarrow \mathbb{R}) \rightarrow \text{Time} \rightarrow U$ such that

$$\int \varphi(t) L_u(\delta u, t) dt = \int \langle G_u(\varphi, t), \delta u(t) \rangle dt$$

This is usually done by integration by parts.

We now set $\varphi := \text{fun } _ \Rightarrow 1$ and get $\text{grad } u := G_u(\text{fun } _ \Rightarrow 1)$, which is the variational gradient at u . The Euler–Lagrange equations, for example, are

In our API, the relationship between

- L_u and G_u is captured by the `HasVarAdjoint`.
- L and G_u by `HasVarAdjDeriv`.
- L and $\text{grad } u$ by `HasVarGradientAt`.

In practice we assume that L has a certain locality property `IsLocalizedFunctionTransform`, which allows us to work with functions φ and δu which have compact support.

This API assumes that U is an inner-product space. This can be considered as a configuration space, or a local chart thereof.

References

- <https://leanprover.zulipchat.com/#narrow/channel/479953-PhysLib/topic/Variational.20Calculus/with/529022834>

@[expose] public section

1.78.

PHYSLEAN/MATHEMATICS/VARIATIONALCALCULUS/HASVARADJDERIV31LEAN

1.78 PhysLean/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean

module

```
public import PhysLean.Mathematics.VariationalCalculus.HasVarAdjoint
@[expose] public section
```

1.79 PhysLean/Mathematics/VariationalCalculus/HasVarAdjoint.lean

module

```
public import PhysLean.Mathematics.VariationalCalculus.IsLocalizedfunctionTransform
@[expose] public section
```

1.80 PhysLean/Mathematics/VariationalCalculus/HasVarGradient.lean

module

```
public import PhysLean.Mathematics.VariationalCalculus.HasVarAdjDeriv
@[expose] public section
```

1.81 PhysLean/Mathematics/VariationalCalculus/IsLocalizedfunctionTrans

module

```
public import PhysLean.Mathematics.VariationalCalculus.Basic
@[expose] public section
```

1.82 PhysLean/Mathematics/VariationalCalculus/IsTestFunction.lean

module

```
public import PhysLean.Mathematics.Calculus.Divergence
@[expose] public section
```

1.83 PhysLean/Meta/AllFilePaths.lean

```

module

public import PhysLean.Meta.TODO.Basic
@[expose] public section

```

1.84 PhysLean/Meta/Basic.lean

```

module

public import Mathlib.Lean.Expr.Basic
public import ImportGraph.Imports.RequiredModules
public import Lean.Elab.DocString
@[expose] public section

  match env.getModuleIdxFor? c with
  | some idx =>
    pure <| env.header.moduleNames[idx]!
  match env.getModuleIdxFor? c with
  | some idx =>
    let modName := env.header.moduleNames[idx]!
    pure s! "{modName.toRelativeFilePath}:{← c.lineNumber} {c}"
  return (← findInternalDocString? env c).isSome
  match ← findInternalDocString? env c with
  | some (.inl s) => return s
  | some (.inr _) => return ""    -- Verso docstring, ignore for now
  | none => return ""
  match env.getModuleIdxFor? name with
  | some idx =>
    let modName := env.header.moduleNames[idx]!
    let fileContent ← IO.FS.readFile modName.toRelativeFilePath

```

1.85 PhysLean/Meta/Informal/Basic.lean

```

module

public meta import Lean.Parser.Term
@[expose] public section

```

1.86 PhysLean/Meta/Informal/Post.lean

```

module

public import PhysLean.Meta.Basic
public import PhysLean.Meta.Informal.Basic
public import PhysLean.Meta.TODO.Basic

@[expose] public section

```

1.87 PhysLean/Meta/Informal/SemiFormal.lean

```

module

public meta import Lean.Elab.Command

@[expose] public section

meta initialize wantedExtension : SimplePersistentEnvExtension WantedInfo (Array WantedInfo) := leading_parser
meta def «semiinformal_result» := leading_parser
meta def elabLemmaWanted : CommandElab := fun stx =>

```

1.88 PhysLean/Meta/Linters/Sorry.lean

```

module

public meta import PhysLean.Meta.Basic
@[expose] public meta section

```

1.89 PhysLean/Meta/Notes/Basic.lean

```

module

public meta import Lean.Elab.Command
@[expose] public section

meta initialize noteExtension : SimplePersistentEnvExtension NoteInfo (Array NoteInfo) := leading_parser
meta def elabNote : Lean.Elab.Command.CommandElab := fun stx =>
meta initialize noteDeclExtension : SimplePersistentEnvExtension Name (Array Name) := leading_parser
meta initialize noteAttribute : Unit := leading_parser
meta initialize noteInformalDeclExtension : SimplePersistentEnvExtension Name (Array Name) := leading_parser

```

```
meta initialize noteInformalAttribute : Unit ←
```

1.90 PhysLean/Meta/Notes/HTMLNote.lean

```
module
```

```
public import PhysLean.Meta.Basic
public import PhysLean.Meta.Notes.Basic
public import PhysLean.Meta.Informal.Basic
@[expose] public section
```

1.91 PhysLean/Meta/Notes/NoteFile.lean

```
module
```

```
public import Lean.Environment
@[expose] public section
```

1.92 PhysLean/Meta/Notes/ToHTML.lean

```
module
```

```
public meta import PhysLean.Meta.Notes.HTMLNote
public import PhysLean.Meta.Notes.NoteFile
@[expose] public section
```

```
meta def sortLE (ni1 ni2 : HTMLNote) : Bool :=
meta unsafe def getNodeInfo : CoreM (List HTMLNote) := do
meta def codeBlockHTML : String := "
meta def informalDefStyle : String :=
meta def codeButton : String := "
meta def mathJaxScript : String := "
meta def headerHTML : String :=
meta def titleHTML : String :=
meta def leanNote : String := "
meta def footerHTML : String :=
meta unsafe def toHTMLString : MetaM String := do
```


1.93 PhysLean/Meta/Remark/Basic.lean

```
module
```

```
public meta import Lean.Elab.Command  
@[expose] public section
```

```
meta initialize remarkExtension : SimplePersistentEnvExtension RemarkInfo (Array RemarkInfo)  
meta def elabRemark : Elab.Command.CommandElab := fun stx =>
```

1.94 PhysLean/Meta/Remark/Properties.lean

```
module
```

```
public import PhysLean.Meta.Remark.Basic  
@[expose] public section
```

```
meta def allRemarkInfo : m (Array RemarkInfo) := do  
meta def RemarkInfo.toFullName (r : RemarkInfo) : Name :=  
meta def RemarkInfo.IsRemark (n : Name) : m Bool := do  
meta def RemarkInfo.getRemarkInfo (n : Name) : m RemarkInfo := do
```

1.95 PhysLean/Meta/Sorry.lean

```
module
```

```
public import PhysLean.Meta.Linters.Sorry  
@[expose] public meta section
```

1.96 PhysLean/Meta/TODO/Basic.lean

```
module
```

```
public meta import Lean.Elab.Command  
@[expose] public section
```

```
meta initialize todoExtension : SimplePersistentEnvExtension todoInfo (Array todoInfo)  
meta def elabTODO : Elab.Command.CommandElab := fun stx =>
```

1.97 PhysLean/Meta/TransverseTactics.lean

```
module

public import Lean
public import PhysLean.Meta.TODO.Basic
@[expose] public section
```

1.98 PhysLean/Optics/Basic.lean

```
module

-/@[expose] public section
```

1.99 PhysLean/Optics/Polarization/Basic.lean

```
module

public import PhysLean.Electromagnetism.Vacuum.HarmonicWave
@[expose] public section
```

1.100 PhysLean/Particles/BeyondTheStandardModel/GeorgiGlashow/B

```
module

public import PhysLean.Particles.StandardModel.Basic
@[expose] public section
```

1.101 PhysLean/Particles/BeyondTheStandardModel/PatiSalam/Basic

```
module

public import PhysLean.Particles.StandardModel.Basic
@[expose] public section
```

1.102.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/RHN/ANOMALYANCELLATION/BASIC.LEAN

1.102 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.QFT.AnomalyCancellation.Basic
@[expose] public section
```

1.103 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Basic
@[expose] public section
```

1.104 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Perm
public import PhysLean.QFT.AnomalyCancellation.GroupActions
@[expose] public section
```

1.105 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Perm
public import PhysLean.QFT.AnomalyCancellation.GroupActions
@[expose] public section
```

1.106 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Order
@[expose] public section
```

1.107 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCanc

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
@[expose] public section
```

1.108 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCanc

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import Mathlib.RepresentationTheory.Basic
@[expose] public section
```

1.109 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCanc

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
@[expose] public section
```

1.110 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCanc

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellat
public import PhysLean.QFT.AnomalyCancellation.GroupActions
@[expose] public section
```

1.111 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCanc

module

1.112.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/RHN/ANOMALYANCELLATION/PLUSU1/FAMILYMAPS.

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Plus
@[expose] public section
```

1.112 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Plus
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Fami
@[expose] public section
```

1.113 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Plus
@[expose] public section
```

1.114 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Plus
public import Mathlib.Tactic.LinearCombination
@[expose] public section
```

1.115 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Plus
@[expose] public section
```

1.116 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancel

module

```
public import PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation
@[expose] public section
```

1.117 PhysLean/Particles/BeyondTheStandardModel/Spin10/Basic.le

module

```
public import PhysLean.Particles.BeyondTheStandardModel.PatiSalam.Basic
public import PhysLean.Particles.BeyondTheStandardModel.GeorgiGlashow.Basic
@[expose] public section
```

1.118 PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Basic.le

module

```
public import PhysLean.Particles.StandardModel.HiggsBoson.Potential
public import Mathlib.Analysis.Matrix.Normed
public import Mathlib.Analysis.Matrix.Order
@[expose] public section
```

1.119 PhysLean/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix

module

```
public import PhysLean.Particles.BeyondTheStandardModel.TwoHDM.Basic
@[expose] public section
```

1.120 PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Potential

module

```
public import PhysLean.Particles.BeyondTheStandardModel.TwoHDM.GramMatrix
```

1.121.

PHYSLEAN/PARTICLES/FLAVORPHYSICS/CKMMATRIX/BASIC.LEAN 49

```
@[expose] public section
```

1.121 PhysLean/Particles/FlavorPhysics/CKMMatrix/Basic.lean

```
module
```

```
public import Mathlib.LinearAlgebra.UnitaryGroup
public import Mathlib.Analysis.Complex.Trigonometric
public import Mathlib.Tactic.Polyrith
@[expose] public section
```

1.122 PhysLean/Particles/FlavorPhysics/CKMMatrix/Invariants.lean

```
module
```

```
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Basic
public import Mathlib.Analysis.Complex.Basic

@[expose] public section
```

1.123 PhysLean/Particles/FlavorPhysics/CKMMatrix/PhaseFreedom.lean

```
module
```

```
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Relations

@[expose] public section
```

1.124 PhysLean/Particles/FlavorPhysics/CKMMatrix/Relations.lean

```
module
```

```
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Rows
@[expose] public section
```

1.125 PhysLean/Particles/FlavorPhysics/CKMMatrix/Rows.lean

```
module
```

```
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Basic
public import Mathlib.Analysis.SpecialFunctions.Complex.Arg
public import Mathlib.LinearAlgebra.CrossProduct
public import Mathlib.LinearAlgebra.FiniteDimensional.Lemmas
public import Mathlib.Tactic.Cases
@[expose] public section
```

1.126 PhysLean/Particles/FlavorPhysics/CKMMatrix/StandardParameters.lean

```
module
```

```
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Rows
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.Invariants
@[expose] public section
```

1.127 PhysLean/Particles/FlavorPhysics/CKMMatrix/StandardParameters.lean

```
module
```

```
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.PhaseFreedom
public import PhysLean.Particles.FlavorPhysics.CKMMatrix.StandardParameters
@[expose] public section
```

1.128 PhysLean/Particles/NeutrinoPhysics/Basic.lean

```
module
```

```
public import Mathlib.LinearAlgebra.UnitaryGroup
public import Mathlib.Analysis.Complex.Trigonometric
public import Mathlib.Tactic.Polyrith
@[expose] public section
```


1.129.

PHYSLEAN/PARTICLES/STANDARDMODEL/ANOMALYANCELLATION/BASIC.LEAN

1.129 PhysLean/Particles/StandardModel/AnomalyCancellation/Basic.lean

module

```
public import PhysLean.QFT.AnomalyCancellation.Basic
public import Mathlib.Tactic.LinearCombination
@[expose] public section
```

1.130 PhysLean/Particles/StandardModel/AnomalyCancellation/FamilyMaps.lean

module

```
public import PhysLean.Particles.StandardModel.AnomalyCancellation.Basic
@[expose] public section
```

1.131 PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav/Basic.lean

module

```
public import PhysLean.Particles.StandardModel.AnomalyCancellation.Basic
@[expose] public section
```

1.132 PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav/One/LinearCombination.lean

module

```
public import PhysLean.Particles.StandardModel.AnomalyCancellation.NoGrav.One.LinearCombination
@[expose] public section
```

1.133 PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav/One/LinearCombination.lean

module

```
public import PhysLean.Particles.StandardModel.AnomalyCancellation.NoGrav.Basic
public import Mathlib.NumberTheory.FLT.Three
@[expose] public section
```

1.134 PhysLean/Particles/StandardModel/AnomalyCancellation/Perm

module

```
public import PhysLean.Particles.StandardModel.AnomalyCancellation.Basic
public import Mathlib.RepresentationTheory.Basic
@[expose] public section
```

1.135 PhysLean/Particles/StandardModel/Basic.lean

module

```
public import Mathlib.Geometry.Manifold.Instances.Real
public import PhysLean.Meta.Informal.SemiFormal
public import PhysLean.SpaceAndTime.SpaceTime.Basic
```

```
@[expose] public section
```

1.136 PhysLean/Particles/StandardModel/HiggsBoson/Basic.lean

module

```
public import PhysLean.Particles.StandardModel.Basic
public import Mathlib.Analysis.InnerProductSpace.Adjoint
public import Mathlib.Geometry.Manifold.VectorBundle.SmoothSection
@[expose] public section
```

1.137 PhysLean/Particles/StandardModel/HiggsBoson/Potential.lean

module

```
public import PhysLean.Particles.StandardModel.HiggsBoson.Basic
public import Mathlib.Tactic.Cases
@[expose] public section
```

1.138.

PHYSLEAN/PARTICLES/STANDARDMODEL/REPRESENTATIONS.LEAN 53

1.138 PhysLean/Particles/StandardModel/Representations.lean

module

```
public import Mathlib.Geometry.Manifold.Instances.Real
@[expose] public section
```

1.139 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/B3.lean

module

```
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Basic
public import Mathlib.Tactic.LinearCombination
@[expose] public section
```

1.140 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Basic.lean

module

```
public import PhysLean.QFT.AnomalyCancellation.Basic
@[expose] public section
```

1.141 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Hypercharge.lean

module

```
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Basic
@[expose] public section
```

1.142 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Linearity.lean

module

```
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Y3
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.B3
@[expose] public section
```

1.143 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation.L

module

```
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.L
@[expose] public section
```

1.144 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation.O

module

```
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.O
@[expose] public section
```

1.145 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation.B

module

```
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.O
@[expose] public section
```

1.146 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation.B

module

```
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.B
public import Mathlib.RepresentationTheory.Basic
@[expose] public section
```

1.147 PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation.B

module

```
public import PhysLean.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.B
```

1.148.

PHYSLEAN/PARTICLES/SUPERSYMMETRY/SU5/CHARGESPECTRUM/ALLOWSTERM.LEAN

```
public import Mathlib.Tactic.LinearCombination
@[expose] public section
```

1.148 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/AllowsTerm.lean

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.OfPotentialTerm
public import Mathlib.Tactic.FinCases
@[expose] public section
```

1.149 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Basic.lean

module

```
public import Mathlib.Data.Finset.Powerset
public import Mathlib.Data.Finset.Prod
public import Mathlib.Data.Finset.Sort
public import Mathlib.Data.Finset.Option
public import PhysLean.Particles.SuperSymmetry.SU5.Potential
@[expose] public section
```

1.150 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Completions.lean

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTerm
@[expose] public section
```

1.151 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Map.lean

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completions
@[expose] public section
```

1.152 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Minimal

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completio
@[expose] public section
```

1.153 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Minimal

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.AllowsTer
@[expose] public section
```

1.154 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Minimal

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoCons
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Minimally
@[expose] public section
```

1.155 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Minimal

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Minimally
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoCons

@[expose] public section
```

1.156 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/OfFie

module

1.157.

PHYSLEAN/PARTICLES/SUPERSYMMETRY/SU5/CHARGESPECTRUM/OFPOTENTIALTERM.LEAN

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Basic
@[expose] public section
```

1.157 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/OfPotentialT

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.OfFieldLabel
public import Mathlib.Tactic.Abel
@[expose] public section
```

1.158 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoClosed.

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimalSuperSet
public import PhysLean.Meta.TODO.Basic

@[expose] public section
```

1.159 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoConstra

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.AllowsTerm
public import Mathlib.Tactic.Polyrith

@[expose] public section
```

1.160 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Yukawa.lean

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoConstrained
@[expose] public section
```

1.161 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/ZMod.lean

module

```
meta import Mathlib.Data.ZMod.Defs
meta import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
meta import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completions
public import Mathlib.Data.Fintype.Prod
public import Mathlib.Data.ZMod.Defs
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Yukawa
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Completio
public import PhysLean.Meta.Linters.Sorry
@[expose] public section
```

1.162 PhysLean/Particles/SuperSymmetry/SU5/FieldLabels.lean

module

```
public import Mathlib.Tactic.DeriveFintype
@[expose] public section
```

1.163 PhysLean/Particles/SuperSymmetry/SU5/Potential.lean

module

```
public import PhysLean.Particles.SuperSymmetry.SU5.FieldLabels
@[expose] public section
```

1.164 PhysLean/QFT/AnomalyCancellation/Basic.lean

module

```
public import PhysLean.Mathematics.LinearMaps
public import Mathlib.LinearAlgebra.FiniteDimensional.Defs
public import PhysLean.Meta.Linters.Sorry
public import Mathlib.Tactic.Cases
@[expose] public section
```


1.165. *PHYSLEAN/QFT/ANOMALYANCELLATION/GROUPACTIONS.LEAN* 59

1.165 PhysLean/QFT/AnomalyCancellation/GroupActions.lean

module

```
public import PhysLean.QFT.AnomalyCancellation.Basic
public import Mathlib.RepresentationTheory.Basic
@[expose] public section
```

1.166 PhysLean/QFT/PerturbationTheory/CreateAnnihilate.lean

module

```
public import Mathlib.Algebra.BigOperators.Group.Finset.Basic
public import Mathlib.Data.Fintype.Card
@[expose] public section
```

1.167 PhysLean/QFT/PerturbationTheory/FeynmanDiagrams/Basic.lean

module

```
public import Mathlib.Tactic.FinCases
public import Mathlib.CategoryTheory.Comma.Over.Basic
public import Mathlib.CategoryTheory.IsomorphismClasses
public import Mathlib.Data.Fintype.Perm
public import Mathlib.Combinatorics.SimpleGraph.Connectivity.WalkCounting

@[expose] public section
```

1.168 PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/Basic.lean

module

```
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.CrAnSection

@[expose] public section
```

1.169 PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/Grading.lean

module

```

public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic
public import Mathlib.RingTheory.GradedAlgebra.Basic
@[expose] public section

```

1.170 PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormTi

```

module

```

```

public import PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic
@[expose] public section

```

1.171 PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/Normal

```

module

```

```

public import PhysLean.QFT.PerturbationTheory.FieldSpecification.NormalOrder
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommu
@[expose] public section

```

1.172 PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/SuperC

```

module

```

```

public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Grading
public import PhysLean.QFT.PerturbationTheory.FieldStatistics.ExchangeSign
@[expose] public section

```

1.173 PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/TimeOr

```

module

```

```

public import PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommu
@[expose] public section

```

1.174.

PHYSLEAN/QFT/PERTURBATIONTHEORY/FIELDSPECIFICATION/BASIC1 LEAN

1.174 PhysLean/QFT/PerturbationTheory/FieldSpecification/Basic.lean

module

```
public import PhysLean.SpaceAndTime.SpaceTime.Basic
public import PhysLean.Units.WithDim.Momentum
public import PhysLean.QFT.PerturbationTheory.FieldStatistics.OfFinset
@[expose] public section
```

1.175 PhysLean/QFT/PerturbationTheory/FieldSpecification/CrAnFieldOp.lean

module

```
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.Basic
public import PhysLean.QFT.PerturbationTheory.CreateAnnihilate
@[expose] public section
```

1.176 PhysLean/QFT/PerturbationTheory/FieldSpecification/CrAnSection.lean

module

```
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.CrAnFieldOp
public import PhysLean.Mathematics.List
@[expose] public section

simp only [eraseIdxEquiv_symm_eq_take_cons_drop, append, take, cons, drop, congr_f
List.length_take, length_eq, inf_le_left, List.getElem_append_right]
```

1.177 PhysLean/QFT/PerturbationTheory/FieldSpecification/Filters.lean

module

```
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.CrAnFieldOp
@[expose] public section
```

1.178 PhysLean/QFT/PerturbationTheory/FieldSpecification/NormalOrder

module

```
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.Filters
public import PhysLean.QFT.PerturbationTheory.Koszul.KoszulSign
import all Mathlib.Data.List.Sort
@[expose] public section
```

1.179 PhysLean/QFT/PerturbationTheory/FieldSpecification/TimeOrder

module

```
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.CrAnSection
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.NormalOrder
import all Mathlib.Data.List.Sort
@[expose] public section
```

1.180 PhysLean/QFT/PerturbationTheory/FieldStatistics/Basic.lean

module

```
public import PhysLean.Mathematics.List.InsertIdx
public import Mathlib.Tactic.FinCases
public import Mathlib.Algebra.BigOperators.Group.Finset.Piecewise
public import Mathlib.Data.Fintype.Card
public import Mathlib.Algebra.FreeMonoid.Basic
@[expose] public section
```

1.181 PhysLean/QFT/PerturbationTheory/FieldStatistics/ExchangeSymmetry

module

```
public import PhysLean.QFT.PerturbationTheory.FieldStatistics.Basic
public import Mathlib.Analysis.Complex.Basic
@[expose] public section
```

1.182.

PHYSLEAN/QFT/PERTURBATIONTHEORY/FIELDSTATISTICS/OFFINSET.lean

1.182 PhysLean/QFT/PerturbationTheory/FieldStatistics/OffFinset.lean

module

```
public import PhysLean.QFT.PerturbationTheory.FieldStatistics.Basic
public import Mathlib.Data.Finset.Sort
@[expose] public section
```

1.183 PhysLean/QFT/PerturbationTheory/Koszul/KoszulSign.lean

module

```
public import PhysLean.QFT.PerturbationTheory.Koszul.KoszulSignInsert
public import PhysLean.Mathematics.List.InsertionSort
import all PhysLean.Mathematics.List
import all Mathlib.Data.List.Sort
@[expose] public section
```

1.184 PhysLean/QFT/PerturbationTheory/Koszul/KoszulSignInsert.lean

module

```
public import PhysLean.QFT.PerturbationTheory.FieldStatistics.ExchangeSign
public import PhysLean.Mathematics.List
import all Mathlib.Data.List.Sort
@[expose] public section
```

1.185 PhysLean/QFT/PerturbationTheory/WickAlgebra/Basic.lean

module

```
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommute
public import Mathlib.Algebra.RingQuot
public import Mathlib.RingTheory.TwoSidedIdeal.Operations
@[expose] public section
```

1.186 PhysLean/QFT/PerturbationTheory/WickAlgebra/Grading.lean

module

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.Basic
@[expose] public section
```

1.187 PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/B

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormalOrder
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.Basic
@[expose] public section
```

1.188 PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/L

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Basic
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.SuperCommute
@[expose] public section
```

1.189 PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/W

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemma
public import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndCont
@[expose] public section
```

1.190 PhysLean/QFT/PerturbationTheory/WickAlgebra/StaticWickTer

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.WickC
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertNo
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertSo
public import PhysLean.QFT.PerturbationTheory.WickContraction.StaticContrac
```

1.191.

PHYSLEAN/QFT/PERTURBATIONTHEORY/WICKALGEBRA/STATICWICKTHEOREM.LEAN

@[expose] public section

1.191 PhysLean/QFT/PerturbationTheory/WickAlgebra/StaticWickTheorem.lean

module

public import PhysLean.QFT.PerturbationTheory.WickAlgebra.StaticWickTerm
@[expose] public section

1.192 PhysLean/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean

module

public import PhysLean.QFT.PerturbationTheory.WickAlgebra.Basic
@[expose] public section

1.193 PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeContraction.lean

module

public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemmas
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeOrder
@[expose] public section

1.194 PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeOrder.lean

module

public import PhysLean.QFT.PerturbationTheory.WickAlgebra.SuperCommute
public import PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.TimeOrder
@[expose] public section

1.195 PhysLean/QFT/PerturbationTheory/WickAlgebra/Universality.

module

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.Basic
@[expose] public section
```

1.196 PhysLean/QFT/PerturbationTheory/WickAlgebra/WickTerm.lean

module

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertNo
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertSo
public import PhysLean.QFT.PerturbationTheory.WickContraction.TimeContract
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.WickC
@[expose] public section
```

1.197 PhysLean/QFT/PerturbationTheory/WickAlgebra/WicksTheorem.

module

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.WickTerm
@[expose] public section
```

1.198 PhysLean/QFT/PerturbationTheory/WickAlgebra/WicksTheoremM

module

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.StaticWickTheorem
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.WicksTheorem
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Join
public import PhysLean.QFT.PerturbationTheory.WickContraction.TimeCond
@[expose] public section
```

1.199 PhysLean/QFT/PerturbationTheory/WickContraction/Basic.lean

module

1.200.

PHYSLEAN/QFT/PERTURBATIONTHEORY/WICKCONTRACTION/CARD. LEAN

```
public import PhysLean.QFT.PerturbationTheory.FieldSpecification.Basic
```

```
@[expose] public section
```

1.200 PhysLean/QFT/PerturbationTheory/WickContraction/Card.lean

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.ExtractEquiv
```

```
@[expose] public section
```

1.201 PhysLean/QFT/PerturbationTheory/WickContraction/Erase.lean

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Uncontracted
```

```
public import PhysLean.Mathematics.Fin
```

```
@[expose] public section
```

1.202 PhysLean/QFT/PerturbationTheory/WickContraction/ExtractEquiv.lean

```
module
```

```
meta import Mathlib.Data.Fintype.Sigma
```

```
meta import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContractNat
```

```
import all Init.Data.Fin.Fold
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContractNat
```

```
@[expose] public section
```

1.203 PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndContract

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.UncontractedList
```

```
@[expose] public section
```

1.204 PhysLean/QFT/PerturbationTheory/WickContraction/InsertAnd

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Erase
@[expose] public section
```

1.205 PhysLean/QFT/PerturbationTheory/WickContraction/Involution

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Basic
@[expose] public section
```

1.206 PhysLean/QFT/PerturbationTheory/WickContraction/IsFull.lean

```
module
```

```
public import PhysLean.Mathematics.Fin.Involutions
public import PhysLean.QFT.PerturbationTheory.WickContraction.ExtractEquiv
public import PhysLean.QFT.PerturbationTheory.WickContraction.Involutions
@[expose] public section
```

1.207 PhysLean/QFT/PerturbationTheory/WickContraction/Join.lean

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Singleton
@[expose] public section
```

1.208 PhysLean/QFT/PerturbationTheory/WickContraction/Perm.lean

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.WickTerm
public import PhysLean.QFT.PerturbationTheory.WickContraction.IsFull
public import PhysLean.Meta.Informal.SemiFormal
```

1.209

PHYSLEAN/QFT/PERTURBATIONTHEORY/WICKCONTRACTION/SIGN/BASIC.LEAN

@[expose] public section

```
/-- If `Perm  $\phi s \Lambda_1$   $\phi s \Lambda_2$ ` and both contractions are grading-
compliant,
  then if ` $\phi s \Lambda_1$ ` is a full Wick contraction, so is ` $\phi s \Lambda_2$ `. -
/
lemma isFull_of_isFull (h : Perm  $\phi s \Lambda_1$   $\phi s \Lambda_2$ )
  (h1 : GradingCompliant  $\phi s$   $\phi s \Lambda_1$ ) (h2 : GradingCompliant  $\phi s$   $\phi s \Lambda_2$ )
  (hf : IsFull  $\phi s \Lambda_1$ ) : IsFull  $\phi s \Lambda_2$  := by
/-- If `Perm  $\phi s \Lambda_1$   $\phi s \Lambda_2$ ` and both contractions are grading-
compliant,
  then their uncontracted lists are permutations of each other. -
/
lemma perm_uncontractedList (h : Perm  $\phi s \Lambda_1$   $\phi s \Lambda_2$ )
  (h1 : GradingCompliant  $\phi s$   $\phi s \Lambda_1$ ) (h2 : GradingCompliant  $\phi s$   $\phi s \Lambda_2$ ) :
  [ $\phi s \Lambda_1$ ]u c.Perm [ $\phi s \Lambda_2$ ]u c := by
```

1.209 PhysLean/QFT/PerturbationTheory/WickContraction/Sign/Basic.lean

module

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Basic
public import PhysLean.QFT.PerturbationTheory.Koszul.KoszulSign
@[expose] public section
```

1.210 PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertNone.lean

module

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Basic
public import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContract
@[expose] public section
```

1.211 PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertSome.lean

module

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Basic
public import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContract
@[expose] public section
```

1.212 PhysLean/QFT/PerturbationTheory/WickContraction/Sign/Join

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Join
@[expose] public section
```

1.213 PhysLean/QFT/PerturbationTheory/WickContraction/Singleton

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.SubContraction
public import PhysLean.QFT.PerturbationTheory.WickContraction.StaticContraction
public import PhysLean.QFT.PerturbationTheory.WickContraction.TimeContraction
public import PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Basic
@[expose] public section
```

1.214 PhysLean/QFT/PerturbationTheory/WickContraction/StaticContraction

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContract
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemma
@[expose] public section
```

1.215 PhysLean/QFT/PerturbationTheory/WickContraction/SubContraction

```
module
```

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.UncontractedL
@[expose] public section
```

1.216.

PHYSLEAN/QFT/PERTURBATIONTHEORY/WICKCONTRACTION/TIMECOND1 LEAN

1.216 PhysLean/QFT/PerturbationTheory/WickContraction/TimeCond.lean

module

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Join
@[expose] public section
```

1.217 PhysLean/QFT/PerturbationTheory/WickContraction/TimeContract.lean

module

```
public import PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeContraction
public import PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContract
@[expose] public section
```

1.218 PhysLean/QFT/PerturbationTheory/WickContraction/Uncontracted.lean

module

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.Basic
@[expose] public section
```

1.219 PhysLean/QFT/PerturbationTheory/WickContraction/UncontractedList

module

```
public import PhysLean.QFT.PerturbationTheory.WickContraction.ExtractEquiv
@[expose] public section
```

- simp only [List.orderedInsert_cons, ha, ↓reduceIte, Fin.val_fin_lt, decide_eq_not_lt, List.filter_cons_of_neg, Fin.val_fin_le, decide_true, List.filter_cons_of_pos, Fin.val_fin_lt, decide_false, Bool.false_eq_true, not_false_eq_true, List.filter_cons_of_neg]
- simp only [List.orderedInsert_cons, ha, ↓reduceIte, Fin.val_fin_lt, Fin.val_fin_le, decide_false, Bool.false_eq_true, not_false_eq_true, List.filter_cons_of_pos]

1.220 PhysLean/QFT/QED/AnomalyCancellation/Basic.lean

module

```
public import PhysLean.QFT.AnomalyCancellation.Basic
public import Mathlib.Tactic.LinearCombination
@[expose] public section
```

1.221 PhysLean/QFT/QED/AnomalyCancellation/BasisLinear.lean

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.Basic
public import Mathlib.LinearAlgebra.FreeModule.StrongRankCondition
@[expose] public section
```

1.222 PhysLean/QFT/QED/AnomalyCancellation/ConstAbs.lean

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.VectorLike
@[expose] public section
```

1.223 PhysLean/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.BasisLinear
public import PhysLean.QFT.QED.AnomalyCancellation.VectorLike
@[expose] public section
```

1.224 PhysLean/QFT/QED/AnomalyCancellation/Even/LineInCubic.lean

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.Even.BasisLinear
public import PhysLean.QFT.QED.AnomalyCancellation.LineInPlaneCond
@[expose] public section
```

1.225.

PHYSLEAN/QFT/QED/ANOMALYANCELLATION/EVEN/PARAMETERIZATION.lean

1.225 PhysLean/QFT/QED/AnomalyCancellation/Even/Parameterization.lean

module

```
public import PhysLean.QFT.QED.AnomalyCancellation.Even.LineInCubic
@[expose] public section
```

1.226 PhysLean/QFT/QED/AnomalyCancellation/LineInPlaneCond.lean

module

```
public import PhysLean.QFT.QED.AnomalyCancellation.ConstAbs
@[expose] public section
```

1.227 PhysLean/QFT/QED/AnomalyCancellation/LowDim/One.lean

module

```
public import PhysLean.QFT.QED.AnomalyCancellation.Basic
@[expose] public section
```

1.228 PhysLean/QFT/QED/AnomalyCancellation/LowDim/Three.lean

module

```
public import PhysLean.QFT.QED.AnomalyCancellation.Basic
@[expose] public section
```

1.229 PhysLean/QFT/QED/AnomalyCancellation/LowDim/Two.lean

module

```
public import PhysLean.QFT.QED.AnomalyCancellation.Basic
@[expose] public section
```

1.230 PhysLean/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.BasisLinear
public import PhysLean.QFT.QED.AnomalyCancellation.VectorLike
@[expose] public section
```

1.231 PhysLean/QFT/QED/AnomalyCancellation/Odd/LineInCubic.lean

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.LineInPlaneCond
public import PhysLean.QFT.QED.AnomalyCancellation.Odd.BasisLinear

@[expose] public section
```

1.232 PhysLean/QFT/QED/AnomalyCancellation/Odd/Parameterization

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.Odd.LineInCubic

@[expose] public section
```

1.233 PhysLean/QFT/QED/AnomalyCancellation/Permutations.lean

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.Basic
public import PhysLean.QFT.AnomalyCancellation.GroupActions
@[expose] public section
```

1.234 PhysLean/QFT/QED/AnomalyCancellation/Sorts.lean

```
module
```

```
public import PhysLean.QFT.QED.AnomalyCancellation.Permutations
public import Mathlib.Data.Fin.Tuple.Sort
```


1.235.

PHYSLEAN/QFT/QED/ANOMALYANCELLATION/VECTORLIKE.LEAN 75

@[expose] public section

1.235 PhysLean/QFT/QED/AnomalyCancellation/VectorLike.lean

module

public import PhysLean.QFT.QED.AnomalyCancellation.Sorts
@[expose] public section

1.236 PhysLean/QuantumMechanics/DDimensions/Hydrogen/Basic.lean

module

public import PhysLean.QuantumMechanics/DDimensions.Operators.Momentum
public import PhysLean.QuantumMechanics/DDimensions.Operators.Position
@[expose] public section

/-- The hydrogen atom Hamiltonian regularized by $\epsilon \neq 0$ is defined to be
def hamiltonianReg ($\epsilon : \mathbb{R}^\times$) : $\square(\text{Space } H.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } H.d, \mathbb{C}) :=$
 ($2 * H.m$)⁻¹ • $\square^2$ - $H.k$ • ($\square[\epsilon, -1]$ + ($2^{-1} * \epsilon.1$ ^ 2) • $\square[\epsilon, -$
 3])

1.237 PhysLean/QuantumMechanics/DDimensions/Hydrogen/LaplaceRungeLenzV

module

public import PhysLean.QuantumMechanics/DDimensions.Hydrogen.Basic
public import PhysLean.QuantumMechanics/DDimensions.Operators.Commutation
@[expose] public section

open Complex Constants

def lrlOperator ($\epsilon : \mathbb{R}^\times$) (i : Fin H.d) : $\square(\text{Space } H.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } H.d, \mathbb{C}) :=$
def lrlOperatorSqr ($\epsilon : \mathbb{R}^\times$) : $\square(\text{Space } H.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } H.d, \mathbb{C}) :=$

lemma lrlOperator_eq ($\epsilon : \mathbb{R}^\times$) (i : Fin H.d) :

+ ($2^{-1} * I * \hbar * (H.d - 1)$) • $\square[i]$ - ($H.m * H.k$) • $\square[\epsilon, -$

1] • L $\square[i]$:= by

dsimp [lrlOperator]

rw [sub_left_inj] -- $m k \cdot r^{-1} x$ terms match exactly

calc

_ = ($2 : \mathbb{C}$)⁻¹ • $\sum j, ((\square[j] \circ L \square[i]) \circ L \square[j] + \square[i] \circ L \square[j] \circ L \square[j])$

```

- (([j] •L [j]) •L [i] + [j] •L [j] •L [i])) := by
simp [angularMomentumOperator, add_sub, comp_assoc, sub_add_eq_add_sub,
momentum_comp_commute, ← Complex.coe_smul]
_ = (2 : ℂ)-1 • ∑ j, ((2 : ℂ) • [i] •L [j] •L [j] - (I * ħ) • δ[i,j])
- ((2 : ℂ) • ([j] •L [j]) •L [i] - (I * ħ) • [i])) := by
simp only [momentum_comp_position_eq, sub_comp, smul_comp, id_comp, s
comp_assoc, two_smul, eq_one_of_same, one_smul]
_ = (2 : ℂ)-1 • ∑ j, ((2 : ℂ) • [i] •L [j] •L [j] - (2 : ℂ) • ([j]
+ (I * ħ) • [i] - (I * ħ) • δ[i,j] • [j])) := by
conv_lhs =>
enter [2, 2, j]
rw [sub_sub_sub_comm, sub_sub_eq_add_sub]
simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib, ← Finset.smul
← finset_sum_comp, sum_smul, Finset.sum_const, Finset.card_univ, Fintyp
smul_add, ← add_sub]
simp only [momentumOperatorSqr, ← Nat.cast_smul_eq_nsmul ℂ, smul_smul, ←
grind [one_smul]
lemma lrlOperator_eq' (ε : ℝ×) (i : Fin H.d) : H.lrlOperator ε i = ∑ j, [i
+ (2-1 * I * ħ * (H.d - 1)) • [i] - (H.m * H.k) • [ε,-
1] •L [i] := by
rw [lrlOperator_eq, sub_left_inj, add_left_inj]
symm
trans ∑ j, [i] •L [j] •L [j] - ∑ j, ([j] •L [j]) •L [i]
· simp [angularMomentumOperator, comp_assoc, momentum_comp_commute]
simp [← comp_finset_sum, ← finset_sum_comp, momentumOperatorSqr]
lemma lrlOperator_eq'' (ε : ℝ×) (i : Fin H.d) : H.lrlOperator ε i = ∑ j, [i
- (2-1 * I * ħ * (H.d - 1)) • [i] - (H.m * H.k) • [ε,-
1] •L [i] := by
trans (2 : ℝ) • H.lrlOperator ε i - H.lrlOperator ε i
· simp [two_smul]
nth_rw 2 [lrlOperator_eq']
simp only [lrlOperator, Finset.sum_add_distrib, smul_add, smul_sub, smul
ring_nf
ext
simp only [one_smul, coe_sub', coe_smul', Pi.sub_apply, ContinuousLinearM
Pi.smul_apply, SchwartzMap.sub_apply, SchwartzMap.add_apply, SchwartzMa
ofReal_mul, ofReal_ofNat]
private def constRadiusRegInvCompPosition (ε : ℝ×) (i : Fin H.d) :=
(H.m * H.k) • [ε,-1] •L [i]
@[sorryful]
lemma angularMomentum_commutation_lrl (ε : ℝ×) (i j k : Fin H.d) :
sorry
@[sorryful]
lemma angularMomentum_commutation_lrlSqr (ε : ℝ×) (i j : Fin H.d) :
simp only [lie_sum, lie_leibniz, H.angularMomentum_commutation_lrl]
@[sorryful]

```

1.237.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACEROTUNGELENZVECTOR.LEAN

```

lemma angularMomentumSqr_commutation_lrlSqr (ε : ℝ×) :
  simp [sum_lie, leibniz_lie, H.angularMomentum_commutation_lrlSqr]
private lemma positionDotMomentum_commutation_radiusRegPow {d} (ε : ℝ×) :
  (-s * Complex.I * ħ) • (□[d,ε,s] - ε.1 ^ 2 • □[d,ε,s-
2]) := by
  rw [leibniz_lie, position_commutation_radiusRegPow, zero_comp, add_zero]
  rw [← lie_skew, radiusRegPow_commutation_momentum, comp_neg, comp_smul]
  rw [comp_eq_comp_sub_commute □[d,ε,_] □[_], position_commutation_radiusRegPow, s
  rw [positionOperatorSqr_eq]
  rw [sub_comp, smul_comp, radiusRegPowOperator_comp_eq]
  simp only [add_sub_cancel, neg_mul, neg_smul, neg_inj]
  congr
  rw [symm j i]
  position_commutation_momentum, smul_comp, smul_comp, id_comp]
  rw [symm j i, add_neg_eq_zero]
  (ε : ℝ×) (i j : Fin H.d) :
  repeat rw [position_commutation_radiusRegPow]
  rw [radiusRegPow_commutation_momentumSqr]
  rw [← comp_assoc, position_comp_radiusRegPow_commute, comp_assoc]
  (ε : ℝ×) (i j : Fin H.d) :
  (H.m * H.k * Complex.I * ħ * ε.1 ^ 2) • □[ε,-3] °L □[i,j] := by
  repeat rw [← lie_skew □[_] _, radiusRegPow_commutation_momentum]
  repeat rw [positionDotMomentum_commutation_radiusRegPow]
  rw [position_comp_commute j i, symm j i]
  simp only [comp_neg, comp_smuls1, RingHom.id_apply, Complex.ofReal_neg,
private lemma constMomentum_comm_constRadiusRegInvCompPosition_add (ε : ℝ×) (i j : F
  repeat rw [position_commutation_momentum, radiusRegPow_commutation_momentum]
  rw [position_comp_commute j i, symm j i]
  simp only [comp_neg, comp_smuls1, RingHom.id_apply, Complex.ofReal_neg,
  (ε : ℝ×) (i j : Fin H.d) :
  rw [radiusRegPow_commutation_radiusRegPow]
  repeat rw [position_commutation_radiusRegPow]
lemma lrl_commutation_lrl (ε : ℝ×) (i j : Fin H.d) : [H.lrlOperator ε i, H.lrlOperat
  rw [positionCompMomentumSqr_comm_constRadiusRegInvCompPosition_add H]
  rw [positionDotMomentumCompMomentum_comm_constRadiusRegInvCompPosition_add H]
  rw [constMomentum_comm_constRadiusRegInvCompPosition_add H]
  rw [constRadiusRegInvCompPosition_comm_constRadiusRegInvCompPosition H]
private lemma rr_comm_rx {d : ℕ} (ε : ℝ×) (i : Fin d) :
  [□[d,ε,-1] + (2-1 * ε.1 ^ 2) • □[ε,-3], □[ε,-1] °L □[i]] = 0 := by
  repeat rw [radiusRegPow_commutation_radiusRegPow]
  repeat rw [← lie_skew, position_commutation_radiusRegPow]
private lemma pSqr_comm_rx (ε : ℝ×) (i : Fin H.d) :
  + (3 * ħ ^ 2 * ε.1 ^ 2) • □[ε,-5] °L □[i]
  rw [← lie_skew, radiusRegPow_commutation_momentumSqr]
  mul_neg, mul_one, neg_mul, neg_smul, one_mul,
  ContinuousLinearMap.add_apply, ContinuousLinearMap.neg_apply, coe_smul', coe_com

```

```

private lemma rr_comm_pL_Lp {d : ℕ} (ε : ℝ×) (i : Fin d) :
  {□[d,ε,-1] + (2-1 * ε.1 ^ 2) • □[ε,-3], ∑ j, (□[j] •L □[i,j] + □[i,j] •
  (- Complex.I * ħ) • (□[ε,-3] +
  (3 * 2-1 * ε.1 ^ 2) • □[ε,-5]) •L ∑ j, (□[j] •L □[i,j] + □[i,j] •L □[
  repeat rw [← lie_skew _ □[_,_], angularMomentum_commutation_radiusRegPow]
  repeat rw [radiusRegPow_commutation_momentum]
  angularMomentum_commutation_radiusRegPow]
private lemma xL_Lx_eq {d : ℕ} (ε : ℝ×) (i : Fin d) : ∑ j, (□[j] •L □[i,j]
  (2 : ℝ) • (∑ j, □[j] •L □[j]) •L □[i]
  - (2 : ℝ) • □[ε,2] •L □[i] + (2 * ε.1 ^ 2) • □[i] - (Complex.I * ħ * (d
  + (2 * Complex.I * ħ) • δ[i,j] • □[j] - (Complex.I * ħ) • □[i] :=
  rw [symm j i, eq_one_of_same]
  ext
  simp only [nsmul_eq_mul, comp_add, comp_smuls1, RingHom.id_apply, c
  coe_comp', coe_smul', coe_mul, coe_id', CompTriple.comp_eq, Pi.su
  ContinuousLinearMap.add_apply, Function.comp_apply, Pi.smul_apply
  map_smul_of_tower, SchwartzMap.sub_apply, SchwartzMap.add_apply,
  momentumOperator_apply, neg_mul, mul_neg, SchwartzMap.smul_apply,
  sub_neg_eq_add, one_smul, comp_id, smul_neg, real_smul, ofReal_of
  + (2 * Complex.I * ħ) • δ[i,j] • □[j] - (Complex.I * ħ) • □[i] :=
  + (3 * Complex.I * ħ) • δ[i,j] • □[j] - (Complex.I * ħ) • □[i] :=
  ext
  simp only [nsmul_eq_mul, comp_add, comp_smuls1, RingHom.id_apply, c
  Pi.sub_apply, ContinuousLinearMap.add_apply, coe_comp', Function.
  coe_id', CompTriple.comp_eq, Pi.smul_apply, natCast_apply, map_sm
  SchwartzMap.sub_apply, SchwartzMap.add_apply, positionOperator_ap
  momentumOperator_apply, neg_mul, mul_neg, SchwartzMap.smul_apply,
  real_smul, ofReal_ofNat, sub_neg_eq_add, sub_left_inj]
  rw [positionOperatorSqr_eq ε, sub_comp, smul_comp, sum_smul, id_comp]
  simp only [smul_sub, ← sub_add, smul_smul, Finset.sum_const, Finset.card
  ← Nat.cast_smul_eq_nsmul ℂ]
  simp only [sub_eq_add_neg, add_assoc, ← neg_smul, ← add_smul]
  congr
lemma hamiltonianReg_commutation_lrl (ε : ℝ×) (i : Fin H.d) :
  {H.hamiltonianReg ε, H.lrlOperator ε i} = (Complex.I * ħ * H.k * ε.1 ^
  rw [rr_comm_rx]
  rw [pSqr_comm_rx]
  rw [rr_comm_pL_Lp]
  rw [xL_Lx_eq ε]
  simp only [← comp_assoc, radiusRegPowOperator_comp_eq]
private lemma sum_Lprx (ε : ℝ×) :
  simp only [comp_eq_comp_sub_commute □[ε,-1] □[_], position_commutation_ra
  rw [symm j i, sub_sub_sub_cancel_right]
  exact angularMomentumSqr_commutation_radiusRegPow ε _
private lemma sum_rxpL (ε : ℝ×) :
private lemma sum_prx (d : ℕ) (ε : ℝ×) : ∑ i : Fin d, □[i] •L □[ε,-

```

1.238.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/ANGULARMOMENTUM.LEAN

```

1] •L [i] =
  [ε,-1] •L ∑ i : Fin d, [i] •L [i] - (Complex.I * ħ * (d - 1)) • [ε,-
1]
- (Complex.I * ħ * ε.1 ^ 2) • [ε,-3] := by
  rw [← comp_assoc, comp_eq_comp_sub_commute [i] [ε,-
1], radiusRegPow_commutation_momentum]
  rw [eq_one_of_same]
  rw [comp_sub, comp_smul, one_smul, comp_id]
  rw [positionOperatorSqr_eq, comp_sub, radiusRegPowOperator_comp_eq, comp_smul]
  SchwartzMap.sum_apply, Finset.sum_const, Finset.card_univ, Fintype.card_fin,
  congr
private lemma sum_rxp (d : ℕ) (ε : ℝ×) : ∑ i : Fin d, [ε,-
1] •L [i] •L [i] =
  [ε,-1] •L ∑ i : Fin d, [i] •L [i] := by rw [comp_finset_sum]
private lemma sum_rxr (d : ℕ) (ε : ℝ×) : ∑ i, [d,ε,-1] •L [i] •L [ε,-
1] •L [i] =
  1 - (ε.1 ^ 2) • [ε,-2] := by
  _ = [d,ε,-1] •L [d,ε,-1] •L [i] •L [i] := by
  rw [comp_eq_comp_add_commute [i] [d,ε,-1], position_commutation_radiusRegP
  _ = [d,ε,-2] •L [i] •L [i] := by
  rw [← comp_assoc, radiusRegPowOperator_comp_eq]
  rw [← comp_finset_sum, positionOperatorSqr_eq, comp_sub, comp_smul,
  radiusRegPowOperator_comp_eq, neg_add_cancel, radiusRegPowOperator_zero]
  congr
lemma lrlOperatorSqr_eq (ε : ℝ×) : H.lrlOperatorSqr ε =
  rw [sum_Lprx] -- ∑∑ Lp-1x = r-1L2
  rw [sum_prx] -- ∑ p r-1x = r-1 ∑ xp - iħ(d-1)r-1 - iħε2r-3
  rw [sum_rxr] -- ∑ r-1x r-1x = 1 - ε2r-2
  Complex.ofReal_sub, radiusRegPowOperator_apply, one_apply, ne_eq,
  not_false_eq_true, Complex.ofReal_eq_zero, m_ne_zero, mul_inv_cancel_left

```

1.238 PhysLean/QuantumMechanics/DDimensions/Operators/AngularMomentum.

module

```

public import PhysLean.QuantumMechanics.DDimensions.Operators.Position
public import PhysLean.QuantumMechanics.DDimensions.Operators.Momentum
## i. Overview

```

In this module we introduce several angular momentum operators for quantum mechanics

ii. Key results

Definitions:

- ``angularMomentumOperator`` : (components of) the angular momentum operator $\varphi(\text{Space } d, \mathbb{C})$ as $\varphi_i \circ \varphi_j - \varphi_j \circ \varphi_i$.
- ``angularMomentumOperatorSqr`` : the operator acting on Schwartz maps $\varphi(\text{Space } d, \mathbb{C})$ as $\frac{1}{2} \sum_{i,j} \varphi_{ij} \circ \varphi_{ij}$.
- ``angularMomentumOperator2D`` : the (pseudo)scalar angular momentum operator
- ``angularMomentumOperator3D`` : the (pseudo)vector angular momentum operator

Notation:

- ``[[i,j]]`` for ``angularMomentumOperator i j``
- ``[]^2`` for ``angularMomentumOperatorSqr``

iii. Table of contents

- A. Angular momentum operator
 - A.1 Antisymmetry
- B. Angular momentum squared operator
- C. Special cases in low dimensions

iv. References

@[expose] public section

/-!

A. Angular momentum operator

notation "[[i ", " j "]" => angularMomentumOperator i j

/-!

A.1 Antisymmetry

-/

/-- The angular momentum operator is antisymmetric, $\varphi_{ij} = -$

φ_{ji} -/

lemma angularMomentumOperator_antisymm {d : ℕ} (i j : Fin d) : [[i,j]] = - [[j,i]]
Eq.symm (neg_sub _ _)

/-- Angular momentum operator components with repeated index vanish, $\varphi_{ii} = 0$ -/

lemma angularMomentumOperator_eq_zero {d : ℕ} (i : Fin d) : [[i,i]] = 0 := s

/-!

B. Angular momentum squared operator

-/

/-!

1.239.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COMMUTATION.LEAN

C. Special cases in low dimensions

momentum operator with components $\hat{p}_0 = \hat{p}_{12}$, $\hat{p}_1 = \hat{p}_{20}$ and $\hat{p}_2 = \hat{p}_{01}$.

lemma angularMomentumOperator1D_trivial (i j : Fin 1) : $\hat{p}[i,j] = 0$:= by

1.239 PhysLean/QuantumMechanics/DDimensions/Operators/Commutation.lean

module

public import PhysLean.Mathematics.KroneckerDelta

public import PhysLean.QuantumMechanics.DDimensions.Operators.AngularMomentum

i. Overview

In this module we compute the commutators for common operators acting on Schwartz ma

Commutator lemmas come in three flavors:

- 1. ``a_commutation_b`` lemmas are of the form ``{a, b} = (...)``.
- 2. ``a_comp_b_commute`` and ``a_comp_commute`` lemmas are of the form ``a ∘ b = b ∘ a`
- 3. ``a_comp_b_eq`` lemmas are of the form ``a ∘ b = b ∘ a + (...)``.

ii. Key results

- ``position_commutation_momentum`` : The canonical commutation relations.
- ``angularMomentum_commutation_position`` : The position operator transforms as a vec
infinitesimal rotations.
- ``angularMomentum_commutation_radiusRegPow`` : Functions of ``||x||2`` commute with the
- ``angularMomentum_commutation_momentum`` : The momentum operator transforms as a vec
infinitesimal rotations.
- ``angularMomentum_commutation_angularMomentum`` : Angular momenta generate an ``so(d)``
- ``angularMomentumSqr_commutation_angularMomentum`` : ``L2`` is a quadratic Casimir of

iii. Table of contents

- A. General
- B. Commutators
 - B.1. Position / position
 - B.2. Momentum / momentum
 - B.3. Position / momentum
 - B.4. Angular momentum / position
 - B.5. Angular momentum / momentum
 - B.6. Angular momentum / angular momentum

iv. References

@[expose] public section

```

open Complex Constants
open Bracket
variable {d : ℕ} (i j k l : Fin d) (ε : ℝ×) (s t : ℝ)
/-!

## A. General

-/

lemma leibniz_lie (A B C :  $\mathcal{L}(\text{Space } d, \mathbb{C}) \rightarrow \mathcal{L}[\mathbb{C}] \mathcal{L}(\text{Space } d, \mathbb{C})$ ) :
lemma lie_leibniz (A B C :  $\mathcal{L}(\text{Space } d, \mathbb{C}) \rightarrow \mathcal{L}[\mathbb{C}] \mathcal{L}(\text{Space } d, \mathbb{C})$ ) :
/-!

## B. Commutators

-/

/-!

### B.1. Position / position

@[simp]
lemma position_commutation_position : { $\mathcal{L}[i]$ ,  $\mathcal{L}[j]$ } = 0 := by
  ext
  simp [bracket, ← mul_assoc, mul_comm]
lemma position_comp_commute :  $\mathcal{L}[i] \circ_L \mathcal{L}[j] = \mathcal{L}[j] \circ_L \mathcal{L}[i]$  := by
  rw [comp_eq_comp_add_commute, position_commutation_position, add_zero]
@[simp]
lemma position_commutation_radiusRegPow : { $\mathcal{L}[i]$ ,  $\mathcal{L}[d, \epsilon, s]$ } = 0 := by
  ext
  simp [bracket, ← mul_assoc, mul_comm]
lemma position_comp_radiusRegPow_commute :  $\mathcal{L}[i] \circ_L \mathcal{L}[\epsilon, s] = \mathcal{L}[\epsilon, s] \circ_L \mathcal{L}[i]$ 
  rw [comp_eq_comp_add_commute, position_commutation_radiusRegPow, add_zero]
@[simp]
lemma radiusRegPow_commutation_radiusRegPow : { $\mathcal{L}[d, \epsilon, s]$ ,  $\mathcal{L}[d, \epsilon, t]$ } = 0 := by
  simp [bracket, mul_def, radiusRegPowOperator_comp_eq, add_comm]

/-!

### B.2. Momentum / momentum

@[simp]
lemma momentum_commutation_momentum : { $\mathcal{L}[i]$ ,  $\mathcal{L}[j]$ } = 0 := by
  have hdiff (k : Fin d) : Differentiable ℝ (∂[k] ψ) := Space.deriv_differe
  show  $\mathcal{L}[i] (\mathcal{L}[j] \psi) \times - \mathcal{L}[j] (\mathcal{L}[i] \psi) \times = 0$ 
  simp only [momentumOperator_apply_fun, Space.deriv_const_smul _ (hdiff _)]

```


1.239.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COMMUTATION.LEAN

```

    Space.deriv_commute _ (ψ.smooth 2), sub_self]
lemma momentum_comp_commute : [i] ⋅L [j] = [j] ⋅L [i] := by
  rw [comp_eq_comp_add_commute, momentum_commutation_momentum, add_zero]
@[simp]
lemma momentumSqr_commutation_momentum : {momentumOperatorSqr (d := d), [i]} = 0 :=
  simp [momentumOperatorSqr, sum_lie, leibniz_lie]
lemma momentumSqr_comp_momentum_commute : [i]² ⋅L [i] = [i] ⋅L [i]² := by
  rw [comp_eq_comp_add_commute, momentumSqr_commutation_momentum, add_zero]
/-!
### B.3. Position / momentum

-/

/-- The canonical commutation relations:  $[x_i, p_j] = i\hbar \delta_{ij}$ . -
/
lemma position_commutation_momentum : {[i], [j]} =
  (I * ħ) • δ[i,j] • ContinuousLinearMap.id ℂ (Space d, ℂ) := by
  show [i] ([j] ψ) x - [j] ([i] ψ) x =
  trans (I * ħ) * (-x i * ∂[j] ψ x + ∂[j] ((fun x : Space d ↦ x i) • ↑ψ) x)
  · simp only [positionOperator_apply, momentumOperator_apply, positionOperator_apply,
    ring
  rw [Space.deriv_smul (by fun_prop) (by fun_prop)]
  rw [Space.deriv_component]
  rcases eq_or_ne i j with (rfl | hne)
  · simp
  · simp [eq_zero_of_ne hne, hne.symm]

lemma momentum_comp_position_eq : [j] ⋅L [i] =
  [i] ⋅L [j] - (I * ħ) • δ[i,j] • ContinuousLinearMap.id ℂ (Space d, ℂ) := by
  rw [comp_eq_comp_sub_commute, position_commutation_momentum]

lemma position_position_commutation_momentum : {[i] ⋅L [j], [k]} =
  (I * ħ) • (δ[i,k] • [j] + δ[j,k] • [i]) := by
  simp only [leibniz_lie, position_commutation_momentum, comp_smul, smul_comp, comp_
    smul_add, add_comm]

lemma position_commutation_momentum_momentum : {[i], [j] ⋅L [k]} =
  (I * ħ) • (δ[i,k] • [j] + δ[i,j] • [k]) := by
  simp only [lie_leibniz, position_commutation_momentum, comp_smul, smul_comp, comp_
    smul_add]

lemma position_commutation_momentumSqr : {[i], [i]²} = (2 * I * ħ) • [i] := by
  simp only [momentumOperatorSqr, lie_sum, lie_leibniz, position_commutation_momentum,
    smul_comp, comp_id, id_comp, ← two_smul ℂ, smul_smul, mul_assoc, ← Finset.smul_s

lemma radiusRegPow_commutation_momentum :
```

```

    { $\square[d, \varepsilon, s]$ ,  $\square[i]$ } = ( $s * I * \hbar$ ) •  $\square[\varepsilon, s-2]$  •  $\square[i]$  := by
  ext  $\psi$  x
  have hne := Ne.symm (ne_of_lt <| norm_sq_add_unit_sq_pos  $\varepsilon$  x)
  have hdiff1 : DifferentiableAt  $\mathbb{R}$  (fun x => ( $\|x\|^2 + \varepsilon^2$ ) ^ ( $s / 2$ ))
    refine DifferentiableAt.rpow_const ?_ (Or.intro_left _ hne)
  have hdiff2 := Real.differentiableAt_rpow_const_of_ne ( $s / 2$ ) hne
  have hdiff3 : DifferentiableAt  $\mathbb{R}$  (fun x =>  $\|x\|^2 + \varepsilon^2$ ) x :=
    Differentiable.differentiableAt (by fun_prop)
  show  $\square[\varepsilon, s]$  ( $\square[i] \psi$ ) x -  $\square[i]$  ( $\square[\varepsilon, s]$   $\psi$ ) x = ( $s * I * \hbar$ ) *  $\square[\varepsilon, s-$ 
2] ( $\square[i] \psi$ ) x
  simp only [momentumOperator_apply, positionOperator_apply, radiusRegPowOp
  rw [ $\leftarrow$  Pi.smul_def', Space.deriv_smul hdiff1 (by fun_prop)]
  suffices  $\partial[i]$  (fun x => ( $\|x\|^2 + \varepsilon^2$ ) ^ ( $s / 2$ )) x =
    s * ( $\|x\|^2 + \varepsilon^2$ ) ^ ( $s / 2 - 1$ ) * x i by
    simp only [this, real_smul, ofReal_mul]
    ring_nf
  change  $\partial[i]$  ((fun r => r ^ ( $s / 2$ )) • (fun x =>  $\|x\|^2 + \varepsilon^2$ )) x = _
  rw [Space.deriv_eq, fderiv_comp x hdiff2 hdiff3, fderiv_add_const, fderiv
  simp [Real.deriv_rpow_const, mul_comm,  $\leftarrow$  mul_assoc, mul_div_cancel, s (Ne

lemma momentum_comp_radiusRegPow_eq :
   $\square[i]$  •  $\square[\varepsilon, s]$  =  $\square[\varepsilon, s]$  •  $\square[i]$  - ( $s * I * \hbar$ ) •  $\square[\varepsilon, s-$ 
2] •  $\square[i]$  := by
  rw [comp_eq_comp_sub_commute, radiusRegPow_commutation_momentum]

lemma radiusRegPow_commutation_momentumSqr :
  { $\square[d, \varepsilon, s]$ , momentumOperatorSqr (d := d)} = ( $2 * s * I * \hbar$ ) •  $\square[\varepsilon, s-$ 
2] •  $\sum i$ ,  $\square[i]$  •  $\square[i]$ 
  + ( $s * (d + s - 2) * \hbar^2$ ) •  $\square[\varepsilon, s-2]$  - ( $\varepsilon^2 * s * (s - 2) * \hbar^2$ )
4] := by
  calc
  _ = ( $s * I * \hbar$ ) •  $\sum i$ , (( $\square[i]$  •  $\square[\varepsilon, s-2]$ ) •  $\square[i]$  +  $\square[\varepsilon, s-$ 
2] •  $\square[i]$  •  $\square[i]$ ) := by
    simp [momentumOperatorSqr, lie_sum, lie_leibniz, radiusRegPow_commuta
     $\leftarrow$  smul_add,  $\leftarrow$  Finset.smul_sum, comp_assoc]
  _ = ( $s * I * \hbar$ ) •  $\sum i$ , ( $\square[\varepsilon, s-2]$  •  $\square[i]$  •  $\square[i]$  +  $\square[\varepsilon, s-$ 
2] •  $\square[i]$  •  $\square[i]$ 
    - ( $(s - 2) * I * \hbar$ ) •  $\square[\varepsilon, s-4]$  •  $\square[i]$  •  $\square[i]$ ) := by
    simp only [momentum_comp_radiusRegPow_eq, sub_comp, smul_comp, sub_ad
    ring_nf
  _ = ( $s * I * \hbar$ ) •  $\sum i$ , (( $2 : \mathbb{C}$ ) •  $\square[\varepsilon, s-2]$  •  $\square[i]$  •  $\square[i]$  - ( $I * \hbar$ ) •
2]
    - ( $(s - 2) * I * \hbar$ ) •  $\square[\varepsilon, s-4]$  •  $\square[i]$  •  $\square[i]$ ) := by
    simp [momentum_comp_position_eq, sub_add_eq_add_sub,  $\leftarrow$  two_smul  $\mathbb{C}$ ]
  simp only [Finset.sum_sub_distrib,  $\leftarrow$  Finset.smul_sum, smul_sub, smul_smul
  have hsumr :  $\sum i : \text{Fin } d$ ,  $\square[d, \varepsilon, s-2]$  = ( $d : \mathbb{C}$ ) •  $\square[\varepsilon, s-$ 

```

1.239.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COMMUTATION.LEAN

```

2] := by
  simp only [Finset.sum_const, Finset.card_univ, Fintype.card_fin, Nat.cast_smul_eq]
  have hrxx :  $\sum i, \langle i | \langle i | = \langle \epsilon, s-2 \rangle - (\epsilon^2 : \mathbb{C}) \cdot \langle \epsilon, s-4 \rangle$  := by
4] := by
  rw [positionOperatorSqr_eq  $\epsilon$ , comp_sub, comp_smul, comp_id, radiusRegPowOperator]
  ← Complex.coe_smul ( $\epsilon \cdot 1^2$ ), ofReal_pow]
  ring_nf
  rw [hsumr, hrxx, smul_smul, smul_sub, ← sub_add, sub_sub, ← add_smul, smul_smul]
  simp only [sub_eq_add_neg, ← neg_smul]
  congr 3 -- match coefficients of  $\langle s-4 \rangle$ ,  $\langle s-2 \rangle$  and  $\langle s-4 \rangle$ 
4]`
  · ring
  · ring_nf
  simp only [I_sq, ofReal_add, ofReal_neg, RingHom.toMonoidHom_eq_coe, OneHom.toFun,
    MonoidHom.toOneHom_coe, MonoidHom.coe_coe, coe_algebraMap, ZeroHom.coe_mk, ofReal_mul, ofReal_natCast, ofReal_pow]
  ring
  · ring_nf
  simp only [I_sq, ofReal_add, ofReal_neg, RingHom.toMonoidHom_eq_coe, OneHom.toFun,
    MonoidHom.toOneHom_coe, MonoidHom.coe_coe, coe_algebraMap, ZeroHom.coe_mk, ofReal_mul, ofReal_pow]
  ring
/-!
### B.4. Angular momentum / position
lemma angularMomentum_commutation_position :
   $\langle [i, j], [k] \rangle = (I * \hbar) \cdot (\delta[i, k] \cdot \langle [j] \rangle - \delta[j, k] \cdot \langle [i] \rangle)$  := by
  trans  $\langle [i] \rangle \circ_L \langle [j] \rangle, [k] \rangle - \langle [j] \rangle \circ_L \langle [i] \rangle, [k] \rangle$ 
  · simp [angularMomentumOperator, leibniz_lie]
  simp only [← lie_skew  $\langle [ ] \rangle \langle [ ] \rangle$ , comp_neg, sub_neg_eq_add, add_comm, ← sub_eq_add,
    position_commutation_momentum, comp_smul, comp_id, smul_sub, symm k_]

@[simp]
lemma angularMomentum_commutation_radiusRegPow :  $\langle [i, j], [d, \epsilon, s] \rangle = 0$  := by
  trans  $\langle [i] \rangle \circ_L \langle [j] \rangle, [\epsilon, s] \rangle - \langle [j] \rangle \circ_L \langle [i] \rangle, [\epsilon, s] \rangle$ 
  · simp [angularMomentumOperator, leibniz_lie]
  simp [← lie_skew  $\langle [ ] \rangle \langle [ ] \rangle$ , radiusRegPow_commutation_momentum, comp_neg,
    ← position_comp_radiusRegPow_commute, ← comp_assoc, position_comp_commute]

@[simp]
lemma angularMomentumSqr_commutation_radiusRegPow :
   $\langle \text{angularMomentumOperatorSqr } (d := d), [d, \epsilon, s] \rangle = 0$  := by
  simp [angularMomentumOperatorSqr, sum_lie, leibniz_lie]

/-!
### B.5. Angular momentum / momentum

```

```

lemma angularMomentum_commutation_momentum : {[i,j], [k]} =
  (I * ħ) • (δ[i,k] • [j] - δ[j,k] • [i]) := by
  trans {[i], [k]} •L [j] - {[j], [k]} •L [i]
  · simp [angularMomentumOperator, leibniz_lie]
  simp only [position_commutation_momentum, smul_comp, id_comp, smul_sub]

lemma momentum_comp_angularMomentum_eq : [k] •L [i,j] =
  [i,j] •L [k] - (I * ħ) • (δ[i,k] • [j] - δ[j,k] • [i]) := by
  rw [comp_eq_comp_sub_commute, angularMomentum_commutation_momentum]

@[simp]
lemma angularMomentum_commutation_momentumSqr : {[i,j], momentumOperatorSqr}
  simp only [momentumOperatorSqr, lie_sum, lie_leibniz, angularMomentum_commutation_momentum,
    comp_smul, comp_sub, smul_comp, sub_comp, ← smul_add, ← Finset.smul_sum,
    Finset.sum_sub_distrib, sum_smul, sub_add_sub_cancel, sub_self, smul_zero]
lemma momentumSqr_comp_angularMomentum_commute : [i,j] •L [i,j] = [i,j] •L [i,j]
  rw [comp_eq_comp_sub_commute, angularMomentum_commutation_momentumSqr, smul_zero]
@[simp]
lemma angularMomentumSqr_commutation_momentumSqr :
  simp [angularMomentumOperatorSqr, sum_lie, leibniz_lie]

/-!

### B.6. Angular momentum / angular momentum
lemma angularMomentum_commutation_angularMomentum : {[i,j], [k,l]} =
  (I * ħ) • (δ[i,k] • [j,l] - δ[i,l] • [j,k] - δ[j,k] • [i,l] + δ[j,l] • [i,k]) := by
  simp only [angularMomentum_commutation_position, angularMomentum_commutation_momentum,
    lie_sub, lie_leibniz, comp_smul, smul_comp, comp_sub, sub_comp, ← smul_add,
    dsimp [angularMomentumOperator]
  ext
  simp only [nsmul_eq_mul, coe_smul', coe_sub', Pi.smul_apply, Pi.sub_apply,
    ContinuousLinearMap.add_apply, coe_mul, SchwartzMap.smul_apply, SchwartzMap.add_apply,
    smul_eq_mul, map_comp_sub]
private lemma angularMomentum_comp_antisymm_sum {d : ℕ} (a b : Fin d) :
  ∑ x : Fin d, [x,a] •L [b,x] = ∑ x : Fin d, [a,x] •L [x,b] := by
  congr 1; ext x
  rw [angularMomentumOperator_antisymm x a, angularMomentumOperator_antisymm x b]
  simp only [neg_comp, comp_neg]

simp only [smul_lie, sum_lie, leibniz_lie,
  angularMomentum_commutation_angularMomentum,
  comp_sub, comp_add, sub_comp, add_comp, comp_smul, smul_comp,
  smul_add, smul_sub, Finset.sum_add_distrib, Finset.sum_sub_distrib,
  ← Finset.smul_sum]
simp_rw [Finset.sum_ite_eq' Finset.univ, Finset.mem_univ, if_true]
rw [angularMomentum_comp_antisymm_sum i j, angularMomentum_comp_antisymm_

```

1.240.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/MOMENTUM.lean

abel

1.240 PhysLean/QuantumMechanics/DDimensions/Operators/Momentum.lean

module

```
public import PhysLean.QuantumMechanics.DDimensions.Operators.Unbounded
public import PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmo
public import PhysLean.QuantumMechanics.PlanckConstant
public import PhysLean.SpaceAndTime.Space.Derivatives.Basic
import Mathlib.Analysis.Calculus.FDeriv.Star
# Momentum operators
```

i. Overview

In this module we introduce several momentum operators for quantum mechanics on `Space

ii. Key results

Definitions:

- `momentumOperator` : (components of) the momentum vector operator acting on Schwartz maps $\varphi(\text{Space } d, \mathbb{C})$ as $-i\hbar\partial_i$.
- `momentumOperatorSqr` : operator acting on Schwartz maps $\varphi(\text{Space } d, \mathbb{C})$ as $-\sum_i \partial_i^2$.
- `momentumUnboundedOperator` : a symmetric unbounded operator acting on the Schwartz space of the Hilbert space $\text{SpaceDHilbertSpace } d$.

Notation:

- $\partial[i]$ for `momentumOperator i`
- ∂^2 for `momentumOperatorSqr`

iii. Table of contents

- A. Momentum vector operator
- B. Momentum-squared operator
- C. Unbounded momentum vector operator

iv. References

`@[expose] public section`

`variable {d : ℕ} (i : Fin d)`

`/-!`

A. Momentum vector operator

-/

```
def momentumOperator :  $\Pi(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \Pi(\text{Space } d, \mathbb{C}) :=$ 
notation " $\Pi[" i "]" \Rightarrow$  momentumOperator i
lemma momentumOperator_apply_fun ( $\psi : \Pi(\text{Space } d, \mathbb{C})$ ) :
@[simp]
lemma momentumOperator_apply ( $\psi : \Pi(\text{Space } d, \mathbb{C})$ ) ( $x : \text{Space } d$ ) :
/-!
```

B. Momentum-squared operator

-/

```
def momentumOperatorSqr :  $\Pi(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \Pi(\text{Space } d, \mathbb{C}) := \sum i, \Pi[i] \circ L$ 
lemma momentumOperatorSqr_apply ( $\psi : \Pi(\text{Space } d, \mathbb{C})$ ) ( $x : \text{Space } d$ ) :
/-!
```

C. Unbounded momentum vector operator

-/

open SpaceDHilbertSpace MeasureTheory

```
/-- The momentum operators defined on the Schwartz submodule. -
/
```

```
def momentumOperatorSchwartz : schwartzSubmodule d  $\rightarrow_1$   $L[\mathbb{C}]$  schwartzSubmodule
schwartzEquiv.toLinearMap  $\circ_1 \Pi[i].toLinearMap \circ_1$  schwartzEquiv.symm.toLin
```

```
lemma momentumOperatorSchwartz_isSymmetric :
(momentumOperatorSchwartz i).IsSymmetric := by
intro  $\psi \psi'$ 
obtain {f, rfl} := schwartzEquiv.surjective  $\psi$ 
obtain {f', rfl} := schwartzEquiv.surjective  $\psi'$ 
simp only [momentumOperatorSchwartz, LinearMap.coe_comp, LinearEquiv.coe_
ContinuousLinearMap.coe_coe, Function.comp_apply, LinearEquiv.symm_appl
schwartzEquiv_inner, momentumOperator_apply, neg_mul, map_neg, map_mul,
Complex.conj_ofReal, neg_neg, mul_neg, integral_neg]
field_simp
simp only [Space.deriv_eq, mul_assoc, integral_const_mul, neg_mul_eq_mul
congr 2
rw [integral_mul_fderiv_eq_neg_fderiv_mul_of_integrable _ _ _ (by fun_pro
· simp only [neg_neg, fderiv_star]
rfl
· simp only [fderiv_star]
exact .mul_of_top_left (((starL'  $\mathbb{R} : \mathbb{C} \approx L[\mathbb{R}] \mathbb{C})$ ).integrable_comp_iff).mp
((f.fderivCLM  $\mathbb{C} \_ \mathbb{C}$ ).evalCLM  $\mathbb{C} \_ \mathbb{C}$  (basis i)).integrable)
```

1.241.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/POSITION.lean

```

    (SchwartzMap.memLp_top f' volume)
  · exact .mul_of_top_left (((starL' ℝ : ℂ ≈ L[ℝ] ℂ).integrable_comp_iff).mpr f.integ
    (((f'.fderivCLM ℂ _ ℂ).evalCLM ℂ _ ℂ (basis i)).memLp_top volume)
  · exact .mul_of_top_left (((starL' ℝ : ℂ ≈ L[ℝ] ℂ).integrable_comp_iff).mpr f.integ
    (f'.memLp_top volume)

/-- The symmetric momentum unbounded operators with domain the Schwartz
submodule of the Hilbert space. -/
def momentumUnboundedOperator :
  UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d) :=
  UnboundedOperator.ofSymmetric (schwartzSubmodule_dense d)
  (momentumOperatorSchwartz_isSymmetric i)

```

1.241 PhysLean/QuantumMechanics/DDimensions/Operators/Position.lean

module

```

public import PhysLean.QuantumMechanics.DDimensions.Operators.Unbounded
public import PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmo
public import PhysLean.SpaceAndTime.Space.Derivatives.Basic
## i. Overview

```

In this module we introduce several position operators for quantum mechanics on `Spa

ii. Key results

Definitions:

- `positionOperator` : (components of) the position vector operator acting on Schwar
 ` $(\text{Space } d, \mathbb{C})$ ` by multiplication by ` $x_i$ `.
- `radiusRegPowOperator` : operator acting on Schwartz maps by multiplication by
 ` $(\|x\|^2 + \varepsilon^2)^{(s/2)}$ `, a smooth regularization of ` $\|x\|^s$ `.
- `positionUnboundedOperator` : a symmetric unbounded operator acting on the Schwart
 of the Hilbert space `SpaceDHilbertSpace d`.
- `radiusRegPowUnboundedOperator` : a symmetric unbounded operator acting on the Sc
 submodule of the Hilbert space `SpaceDHilbertSpace d`. For ` $s \leq 0$ ` this operator
 bounded (by ` $|\varepsilon|^s$ `) and has natural domain the entire Hilbert space, but for uni
 use the same domain for all `s`.

Notation:

- ` $\square[i]$ ` for `positionOperator i`
- ` $\square[\varepsilon, s]$ ` for `radiusRegPowOperator ε s`

iii. Table of contents

- A. Position vector operator
- B. Regularized radius operator
- C. Unbounded position vector operator
- D. Unbounded regularized radius operator

iv. References

@[expose] public section

variable {d : ℕ} (i : Fin d)

/-!

A. Position vector operator

from $\square(\text{Space } d, \mathbb{C})$ to itself which maps ψ to $x_i \psi$. -
/

def positionOperator : $\square(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } d, \mathbb{C}) :=$
SchwartzMap.smulLeftCLM $\mathbb{C}$ (Complex.ofRealCLM $\circ$ L coordCLM i)

notation " $\square[" i "]"$ " => positionOperator i

lemma positionOperator_apply_fun ($\psi : \square(\text{Space } d, \mathbb{C})$) : $\square[i] \psi = (\text{fun } x : \text{Space } d \rightarrow \mathbb{C} \text{ : } x[i] \psi)$
ext

simp [positionOperator, coordCLM_apply, coord_apply,
smulLeftCLM_apply_apply (g := Complex.ofRealCLM $\circ$ (coordCLM i)) (by fun _ => simp)]

lemma positionOperator_apply ($\psi : \square(\text{Space } d, \mathbb{C})$) (x : Space d) : $\square[i] \psi x = x[i] \psi$
simp [positionOperator_apply_fun]

/-!

B. Regularized radius operator

/-- Power of regularized norm, $(\|x\|^2 + \varepsilon^2)^{(s/2)}$. -/

def normRegularizedPow (d : ℕ) (ε s : ℝ) : Space d $\rightarrow$ ℝ :=

lemma norm_sq_add_unit_sq_pos {d : ℕ} ($\varepsilon : \mathbb{R}^+$) (x : Space d) : $0 < \|x\|^2 + \varepsilon^2$
Left.add_pos_of_nonneg_of_pos (sq_nonneg $\|x\|^2$) (sq_pos_iff.mpr <| Units.one)

lemma normRegularizedPow_pos (d : ℕ) ($\varepsilon : \mathbb{R}^+$) (s : ℝ) (x : Space d) :

$0 < \text{normRegularizedPow } d \ \varepsilon \ s \ x :=$

Real.rpow_pos_of_pos (norm_sq_add_unit_sq_pos $\varepsilon \ x$) (s / 2)

lemma normRegularizedPow_hasTemperateGrowth (d : ℕ) ($\varepsilon : \mathbb{R}^+$) (s : ℝ) :

HasTemperateGrowth (normRegularizedPow d $\varepsilon \ s$) := by

-- to take advantage of `hasTemperateGrowth\_one\_add\_norm\_sq\_rpow`.

let f3 := fun (x : Space d) $\mapsto \varepsilon \cdot 1^{-1} \cdot x$

have h123 : normRegularizedPow d $\varepsilon \ s = f1 \circ f2 \circ f3 := by$

ext

simp only [normRegularizedPow, f1, f2, f3, comp_apply, norm_smul, norm_mul]

rw [← Real.mul_rpow (sq_nonneg ε) (add_nonneg (zero_le_one' _) (sq_nonneg $\|x\|^2$))]

1.241.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/POSITION1 LEAN

```

    simp [mul_add, mul_pow, add_comm]
def radiusRegPowOperator {d : ℕ} (ε : ℝ+) (s : ℝ) :  $\mathcal{L}(\mathbb{C})$   $\mathcal{L}(\text{Space } d, \mathbb{C})$ 
  SchwartzMap.smulLeftCLM  $\mathbb{C}$  (Complex.ofReal  $\circ$  normRegularizedPow d ε s)
notation " $\mathcal{L}$ " ε "," s "]" => radiusRegPowOperator ε s
@[inherit_doc radiusRegPowOperator]
notation " $\mathcal{L}$ " d "," ε "," s "]" => radiusRegPowOperator (d := d) ε s

lemma radiusRegPowOperator_apply_fun {d : ℕ} (ε : ℝ+) (s : ℝ) (ψ :  $\mathcal{L}(\text{Space } d, \mathbb{C})$ ) :
   $\mathcal{L}[d, \varepsilon, s]$  ψ = fun x ↦ ( $\|x\|^2 + \varepsilon^2$ )s / 2 • ψ x := by
  dsimp [radiusRegPowOperator]
  refine smulLeftCLM_apply_apply ?_ ψ x
  exact HasTemperateGrowth.comp (by fun_prop) (normRegularizedPow_hasTemperateGrowth)
@[simp]
lemma radiusRegPowOperator_apply {d : ℕ} (ε : ℝ+) (s : ℝ) (ψ :  $\mathcal{L}(\text{Space } d, \mathbb{C})$ ) (x : S) :
  rw [radiusRegPowOperator_apply_fun]
@[simp]
lemma radiusRegPowOperator_comp_eq {d : ℕ} (ε : ℝ+) (s t : ℝ) :
   $\mathcal{L}[d, \varepsilon, s] \circ_L \mathcal{L}[d, \varepsilon, t] = \mathcal{L}[d, \varepsilon, s+t]$  := by
  simp [add_div, Real.rpow_add (norm_sq_add_unit_sq_pos ε x), mul_assoc]
@[simp]
lemma radiusRegPowOperator_zero {d : ℕ} (ε : ℝ+) :
   $\mathcal{L}[d, \varepsilon, 0] = \text{ContinuousLinearMap.id } \mathbb{C} \mathcal{L}(\text{Space } d, \mathbb{C})$  := by
  ext
  simp

lemma positionOperatorSqr_eq {d : ℕ} (ε : ℝ+) :
   $\sum i, \mathcal{L}[i] \circ_L \mathcal{L}[i] = \mathcal{L}[d, \varepsilon, 2] - \varepsilon \cdot \text{ContinuousLinearMap.id } \mathbb{C} \mathcal{L}(\text{Space } d, \mathbb{C})$ 
  ext
  simp [Space.norm_sq_eq, add_mul, ← mul_assoc, ← pow_two, Finset.sum_mul]

/-!

## C. Unbounded position vector operator

-/

open SpaceDHilbertSpace

/-- The position operators defined on the Schwartz submodule. -
/
def positionOperatorSchwartz : schwartzSubmodule d →1  $\mathbb{C}$  schwartzSubmodule d :=
  schwartzEquiv.toLinearMap  $\circ_1$   $\mathcal{L}[i]$ .toLinearMap  $\circ_1$  schwartzEquiv.symm.toLinearMap

lemma positionOperatorSchwartz_isSymmetric : (positionOperatorSchwartz i).IsSymmetric
  intro ψ ψ'
  obtain ⟨_, rfl⟩ := schwartzEquiv.surjective ψ

```

```

obtain ⟨_, rfl⟩ := schwartzEquiv.surjective ψ'
unfold positionOperatorSchwartz
simp only [LinearMap.coe_comp, LinearEquiv.coe_coe, comp_apply, schwartzEquiv.coe_coe]
congr with x
simp only [LinearEquiv.symm_apply_apply, ContinuousLinearMap.coe_coe, positionOperator_apply, map_mul, Complex.conj_ofReal]
ring

/-- The symmetric position unbounded operators with domain the Schwartz submodule
of the Hilbert space. -/
def positionUnboundedOperator : UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d) :=
  UnboundedOperator.ofSymmetric (schwartzSubmodule_dense d) (positionOperator_apply)

/-!

## D. Unbounded regularized radius operator

-/

/-- The (regularized) radius operators defined on the Schwartz submodule. -/
def radiusRegPowOperatorSchwartz {d : ℕ} (ε : ℝ×) (s : ℝ) :
  schwartzSubmodule d →1 [ℂ] schwartzSubmodule d :=
  schwartzEquiv.toLinearMap ∘1 [ε, s].toLinearMap ∘1 schwartzEquiv.symm.toLinearMap

lemma radiusRegPowOperatorSchwartz_isSymmetric {d : ℕ} (ε : ℝ×) (s : ℝ) :
  (radiusRegPowOperatorSchwartz (d := d) ε s).IsSymmetric := by
  intro ψ ψ'
  obtain ⟨_, rfl⟩ := schwartzEquiv.surjective ψ
  obtain ⟨_, rfl⟩ := schwartzEquiv.surjective ψ'
  simp only [radiusRegPowOperatorSchwartz, LinearMap.coe_comp, LinearEquiv.coe_coe, schwartzEquiv_inner]
  congr with x -- match integrands
  simp only [LinearEquiv.symm_apply_apply, ContinuousLinearMap.coe_coe, radiusRegPowOperatorSchwartz, Complex.real_smul, map_mul, Complex.conj_ofReal]
  ring

/-- The symmetric (regularized) radius unbounded operators with domain the
of the Hilbert space. -/
def radiusRegPowUnboundedOperator {d : ℕ} (ε : ℝ×) (s : ℝ) :
  UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d) :=
  UnboundedOperator.ofSymmetric (schwartzSubmodule_dense d)
  (radiusRegPowOperatorSchwartz_isSymmetric ε s)

```

1.242.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED.lean

1.242 PhysLean/QuantumMechanics/DDimensions/Operators/Unbounded.lean

module

public import PhysLean.Mathematics.InnerProductSpace.Submodule
i. Overview

In this module we define unbounded operators between Hilbert spaces.

ii. Key results

Definitions:

- ``UnboundedOperator``: Densely defined, closable unbounded operator between Hilbert
 - ``partialOrder``: Poset structure for unbounded operators.
 - ``ofSymmetric``: Construction of an unbounded operator from a symmetric ``LinearPMap``
 - ``IsGeneralizedEigenvector``: The notion of eigenvectors/values for linear functionals
- (In)equalities:
- ``le_closure``: $U \leq U.\text{closure}$
 - ``adjoint_adjoint_eq_closure``: $U^\dagger = U.\text{closure}$
 - ``adjoint_ge_adjoint_of_le``: $U_1 \leq U_2 \rightarrow U_2^\dagger \leq U_1^\dagger$
 - ``closure_mono``: $U_1 \leq U_2 \rightarrow U_1.\text{closure} \leq U_2.\text{closure}$
 - ``isSymmetric_iff_le_adjoint``: $\text{IsSymmetric } T \leftrightarrow T \leq T^\dagger$

iii. Table of contents

- A. Definition
- B. Basic identities
- C. Instances
 - C.1. Partial order
 - C.2. Zero
 - C.3. AddZeroClass
 - C.4. DistribSMul
- D. Closure
- E. Adjoint
- F. Symmetric operators
- G. Self-adjoint operators
- H. Generalized eigenvectors

iv. References

- M. Reed & B. Simon, (1972). "Methods of modern mathematical physics: Vol. 1. Functional Analysis". Academic Press. <https://doi.org/10.1016/B978-0-12-585001-8.X5001-6>

@[expose] public section

open InnerProductSpace InnerProductSpaceSubmodule
/-!

A. Definition

```

-/

/-- An `UnboundedOperator` is a linear map from a submodule of `H` to `H'`,
    assumed to be both densely defined and closable. -/
(H : Type*) [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H]
(H' : Type*) [NormedAddCommGroup H'] [InnerProductSpace ℂ H'] [CompleteSpace H']
extends LinearPMap ℂ H H' where
/-- The domain of an unbounded operator is dense in `H`. -
/
dense_domain : Dense (domain : Set H)
{H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H]
{H' : Type*} [NormedAddCommGroup H'] [InnerProductSpace ℂ H'] [CompleteSpace H']
@[ext]
lemma ext {U₁ U₂ : UnboundedOperator H H'} (h : U₁.toLinearPMap = U₂.toLinearPMap) :
  U₁ = U₂ := by
  cases U₁
  simp_all
instance : CoeFun (UnboundedOperator H H') (fun U ↦ U.domain → H') :=
  {fun U ↦ U.toLinearPMap.toFun'}
## B. Basic identities
section
open Complex
lemma inner_map_polarization {T : UnboundedOperator H H} (x y : T.domain) :
  ⟨T x, ↑y⟩_ℂ = (⟨T (x + y), ↑(x + y)⟩_ℂ - ⟨T (x - y), ↑(x - y)⟩_ℂ
    - I * ⟨T (x + I • y), ↑(x + I • y)⟩_ℂ + I * ⟨T (x - I • y), ↑(x - I • y)⟩_ℂ)
  simp only [LinearPMap.map_add, coe_add, inner_add_right, inner_add_left,
    AddSubgroupClass.coe_sub, inner_sub_right, inner_sub_left, LinearPMap.map_sub,
    SetLike.val_smul, inner_smul_left, conj_I, inner_smul_right]
  grind [I_sq]

lemma inner_map_polarization' {T : UnboundedOperator H H} (x y : T.domain) :
  ⟨↑x, T y⟩_ℂ = (⟨↑(x + y), T (x + y)⟩_ℂ - ⟨↑(x - y), T (x - y)⟩_ℂ
    - I * ⟨↑(x + I • y), T (x + I • y)⟩_ℂ + I * ⟨↑(x - I • y), T (x - I • y)⟩_ℂ)
  simp only [coe_add, LinearPMap.map_add, inner_add_right, inner_add_left,
    LinearPMap.map_sub, inner_sub_right, inner_sub_left, SetLike.val_smul,
    inner_smul_left, conj_I, inner_smul_right]
  grind [I_sq]

end

/-!
## C. Instances
-/

section

```

1.242.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED. LEAN

open Classical

/-!

C.1. Partial order

Unbounded operators inherit the structure of a poset from ``LinearPMap``,
but *not* that of a ``SemilatticeInf`` because ``U1.domain ∩ U2.domain`` may not be dense.
-/

instance : PartialOrder (UnboundedOperator H H') where

le U₁ U₂ := U₁.toLinearPMap ≤ U₂.toLinearPMap

le_refl _ := le_refl _

le_trans _ _ h₁₂ h₂₃ := le_trans h₁₂ h₂₃

le_antisymm _ _ h h' := ext <| le_antisymm h h'

/-!

C.2. Zero

-/

-- A zero LinearPMap (any domain) is closable

lemma isClosable_of_zero {E F : Type*} [NormedAddCommGroup E] [InnerProductSpace ℂ E]
[NormedAddCommGroup F] [InnerProductSpace ℂ F] {f : LinearPMap ℂ E F} (hf : f.to
f.IsClosable := by
use f.graph.topologicalClosure.toLinearPMap
refine Eq.symm <| toLinearPMap_graph_eq f.graph.topologicalClosure fun x hx _ => ?
obtain ⟨b, hb, hb'⟩ := mem_closure_iff_seq_limit.mp hx
have (n : ℕ) : (b n).2 = 0 := by specialize hb n; simp_all
rw [nhds_prod_eq, Filter.tendsto_prod_iff'] at hb'
simp_all

instance : Zero (UnboundedOperator H H') := ⟨0, by simp, isClosable_of_zero rfl⟩

@[simp]

lemma zero_toLinearPMap : (0 : UnboundedOperator H H').toLinearPMap = 0 := rfl

instance : Inhabited (UnboundedOperator H H') := ⟨instZero.zero⟩

/-!

C.3. AddZeroClass

In defining addition for unbounded operators we use two junk values.

- If ``U1.domain ∩ U2.domain`` is not dense, then ``U1 + U2 = 0`` (domain ``T``)
- If ``U1.domain ∩ U2.domain`` is dense but ``U1.toLinearPMap + U2.toLinearPMap`` is not closable, then ``U1 + U2 = 0`` with domain ``U1.domain ∩ U2.domain``.

This ensures that distributivity, ``c • (U1 + U2) = c • U1 + c • U2``, holds for ``(c : ℂ)``.
-/

```

noncomputable instance : Add (UnboundedOperator H H') where
  add U1 U2 :=
    if hD : Dense (U1.domain  $\sqcap$  U2.domain : Set H) then
      if hC : (U1.toLinearPMap + U2.toLinearPMap).IsClosable then
        (U1.toLinearPMap + U2.toLinearPMap, hD, hC)
      else ((U1.domain  $\sqcap$  U2.domain,  $\emptyset$ ), hD, isClosable_of_zero rfl)
    else 0

/-- The domain of `U1 + U2` is `D(U1)  $\cap$  D(U2)` when it is dense. -
/
lemma add_domain_of_dense {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain  $\sqcap$  U2.domain : Set H)) : (U1 + U2).domain = U1.domain  $\cap$  U2.domain :=
  rw [HAdd.hAdd, instHAdd, Add.add, instAdd]
  by_cases hC : (U1.toLinearPMap + U2.toLinearPMap).IsClosable
  · simp only [hD, hC,  $\downarrow$ reduceDite, LinearPMap.add_domain]
  · simp only [hD, hC,  $\downarrow$ reduceDite]

/-- The junk value for `U1 + U2` has domain all of `H` when `D(U1)  $\cap$  D(U2)` is not dense. -
/
lemma add_domain_of_not_dense {U1 U2 : UnboundedOperator H H'}
  (hD :  $\neg$ Dense (U1.domain  $\sqcap$  U2.domain : Set H)) : (U1 + U2).domain =  $\top$  :=
  rw [HAdd.hAdd, instHAdd, Add.add, instAdd]
  simp only [hD,  $\downarrow$ reduceDite, zero_toLinearPMap, zero_domain]

lemma mem_domain_of_dense {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain  $\sqcap$  U2.domain : Set H)) ( $\psi$  : (U1 + U2).domain) :
   $\uparrow\psi \in U_1$ .domain  $\wedge$   $\uparrow\psi \in U_2$ .domain :=
  mem_inf.mp <| (add_domain_of_dense hD)  $\triangleright$   $\psi$ .2

/-- `U1 + U2` is the unbounded operator corresponding to `U1.toLinearPMap + U2.toLinearPMap` provided it is both densely defined and closable. -/
lemma add_toLinearPMap_of_dense_closable {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain  $\sqcap$  U2.domain : Set H))
  (hC : (U1.toLinearPMap + U2.toLinearPMap).IsClosable) :
  (U1 + U2).toLinearPMap = U1.toLinearPMap + U2.toLinearPMap := by
  rw [HAdd.hAdd, instHAdd, Add.add, instAdd]
  simp only [hD, hC,  $\downarrow$ reduceDite]

/-- The junk value for `U1 + U2` when `D(U1)  $\cap$  D(U2)` is dense and `U1 + U2` is not closable. -
/
lemma add_toLinearPMap_of_dense_not_closable {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain  $\sqcap$  U2.domain : Set H))
  (hC :  $\neg$ (U1.toLinearPMap + U2.toLinearPMap).IsClosable) :
  (U1 + U2).toLinearPMap = (U1.domain  $\sqcap$  U2.domain,  $\emptyset$ ) := by
  rw [HAdd.hAdd, instHAdd, Add.add, instAdd]

```

1.242.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED. LEAN

```
simp only [hD, hC, ↓reduceDite]

/-- The junk value for `U1 + U2` when `D(U1) ∩ D(U2)` is not dense. -
/
lemma add_toLinearPMap_of_not_dense {U1 U2 : UnboundedOperator H H'}
  (hD : ¬Dense (U1.domain ∩ U2.domain : Set H)) : (U1 + U2).toLinearPMap = 0 := by
  rw [HAdd.hAdd, instHAdd, Add.add, instAdd]
  simp only [hD, ↓reduceDite, zero_toLinearPMap]

/-- `(U1 + U2)ψ = U1ψ + U2ψ` provided `U1 + U2` is not given by a junk value. -
/
lemma add_apply_of_dense_closable {U1 U2 : UnboundedOperator H H'}
  (hD : Dense (U1.domain ∩ U2.domain : Set H))
  (hC : (U1.toLinearPMap + U2.toLinearPMap).IsClosable) (ψ : (U1 + U2).domain) :
  (U1 + U2) ψ = U1 ⟨ψ, (mem_domain_of_dense hD ψ).1⟩ + U2 ⟨ψ, (mem_domain_of_dense
obtain ⟨_, hb⟩ := LinearPMap.dExt_iff.mp <| add_toLinearPMap_of_dense_closable hD
simp only [Subtype.forall] at hb
specialize hb ψ ψ.2 ψ (mem_domain_of_dense hD ψ) rfl
simp_all [LinearPMap.add_apply]

noncomputable instance : AddZeroClass (UnboundedOperator H H') where
  zero_add U := by
    apply UnboundedOperator.ext
    rw [← zero_add U.toLinearPMap]
    exact add_toLinearPMap_of_dense_closable (by simp [U.dense_domain]) (by simp [U.
  add_zero U := by
    apply UnboundedOperator.ext
    rw [← add_zero U.toLinearPMap]
    exact add_toLinearPMap_of_dense_closable (by simp [U.dense_domain]) (by simp [U.

/-- Addition of unbounded operators is associative when neither of the intermediate
`U1 + U2` and `U2 + U3`, is given by a junk value. -/
lemma add_assoc {U1 U2 U3 : UnboundedOperator H H'}
  (hD12 : Dense (U1.domain ∩ U2.domain : Set H)) (hC12 : (U1.1 + U2.1).IsClosable)
  (hD23 : Dense (U2.domain ∩ U3.domain : Set H)) (hC23 : (U2.1 + U3.1).IsClosable)
  U1 + U2 + U3 = U1 + (U2 + U3) := by
  apply UnboundedOperator.ext
  by_cases hD : Dense (U1.domain ∩ U2.domain ∩ U3.domain : Set H)
  · have hD' : Dense (U1.domain ∩ (U2.domain ∩ U3.domain) : Set H) := by grind
    have hD12' : (U1 + U2).domain = U1.domain ∩ U2.domain := add_domain_of_dense hD12
    have hD23' : (U2 + U3).domain = U2.domain ∩ U3.domain := add_domain_of_dense hD23
    have h12 : (U1 + U2).1 = U1.1 + U2.1 := add_toLinearPMap_of_dense_closable hD12
    have h23 : (U2 + U3).1 = U2.1 + U3.1 := add_toLinearPMap_of_dense_closable hD23
    by_cases hC : (U1.1 + U2.1 + U3.1).IsClosable
    · -- No junk values anywhere: addition is associative
      have hC' : (U1.1 + (U2.1 + U3.1)).IsClosable := by grind
```

```

    rw [add_toLinearPMap_of_dense_closable (by simp_all) (h12 ▸ hC), h12,
        add_toLinearPMap_of_dense_closable (by simp_all) (h23 ▸ hC'), h23]
    grind
  · -- D(U1) ∩ D(U2) ∩ D(U3) is dense but neither side is closable:
    -- both sides are the junk zero operator with domain D(U1) ∩ D(U2) ∩
    have hC' : ¬(U1.1 + (U2.1 + U3.1)).IsClosable := by grind
    rw [add_toLinearPMap_of_dense_not_closable (by simp_all) (h12 ▸ hC),
        add_toLinearPMap_of_dense_not_closable (by simp_all) (h23 ▸ hC'), h
    grind
  · -- D(U1) ∩ D(U2) ∩ D(U3) is not dense: both sides are the junk zero op
    have hD' : ¬Dense (U1.domain ∩ (U2.domain ∩ U3.domain) : Set H) := by g
    have hD12' : ¬Dense ((U1 + U2).domain ∩ U3.domain : Set H) := add_domai
    have hD23' : ¬Dense (U1.domain ∩ (U2 + U3).domain : Set H) := add_domai
    rw [add_toLinearPMap_of_not_dense hD12', add_toLinearPMap_of_not_dense

/-!
### C.4. DistribSMul

Scalar multiplication by complex numbers is inherited from `LinearPMap`. No
the same domain as `U` for all constants; in particular, `(0 : ℂ) • U ≤ 0`
if `U.domain = τ`.
-/

lemma smul_mem_graph_of_mem_smul_graph {E F : Type*} [NormedAddCommGroup E] [I
  [NormedAddCommGroup F] [InnerProductSpace ℂ F] {f : LinearPMap ℂ E F} {
    {x : E × F} (h : x ∈ (c • f).graph.topologicalClosure) :
    (x.fst, c-1 • x.snd) ∈ f.graph.topologicalClosure := by
  obtain ⟨b, hb, hb'⟩ := mem_closure_iff_seq_limit.mp h
  apply mem_closure_iff_seq_limit.mpr
  use fun n ↦ ((b n).fst, c-1 • (b n).snd)
  rw [nhds_prod_eq, Filter.tendsto_prod_iff'] at *
  refine {?_, ⟨hb'.1, ?_⟩}
  · have {y : f.domain} {z : F} : c • f y = z ↔ f y = c-1 • z := by aesop
    simp_all
  · rw [nhds_smul₀ (inv_ne_zero hc), ← Pi.smul_def, Filter.smul_tendsto_smu
    exact hb'.2

lemma smul_isClosable_of_isClosable {E F : Type*} [NormedAddCommGroup E] [I
  [NormedAddCommGroup F] [InnerProductSpace ℂ F] {f : LinearPMap ℂ E F} (
    (c : ℂ) : (c • f).IsClosable := by
  rcases eq_zero_or_neZero c with (rfl | hc)
  · exact isClosable_of_zero (by simp)
  · use (c • f).graph.topologicalClosure.toLinearPMap
    refine Eq.symm <| toLinearPMap_graph_eq _ (fun x hx hx1 ↦ ?_)
    suffices c-1 • x.snd = 0 by aesop
    have hx := smul_mem_graph_of_mem_smul_graph hc.ne hx

```


1.242.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED. LEAN

```
rw [IsClosable.graph_closure_eq_closure_graph hf, hx1] at hx
exact graph_fst_eq_zero_snd f.closure hx rfl
```

```
noncomputable instance : SMul ℂ (UnboundedOperator H H') where
  smul c U := (c • U.toLinearPMap, U.dense_domain, smul_isClosable_of_isClosable U.i
```

```
variable (U : UnboundedOperator H H')
```

```
@[simp]
```

```
lemma smul_domain (c : ℂ) : (c • U).domain = U.domain := rfl
```

```
lemma smul_toLinearPMap (c : ℂ) : (c • U).toLinearPMap = c • U.toLinearPMap := rfl
```

```
lemma zero_smul_le_zero : (0 : ℂ) • U ≤ 0 := (by simp, by simp)
```

```
noncomputable instance : DistribSMul ℂ (UnboundedOperator H H') where
```

```
  smul_zero _ := ext <| by ext <;> simp
```

```
  smul_add c U1 U2 := by
```

```
    apply UnboundedOperator.ext
```

```
    by_cases hD : Dense (U1.domain ∪ U2.domain : Set H)
```

```
    · have hD' : Dense ((c • U1).domain ∪ (c • U2).domain : Set H) := by grind
```

```
      by_cases hC : (U1.l + U2.l).IsClosable
```

```
      · -- No junk values: smul distributes as it does for `LinearPMap`
```

```
        have h : c • (U1.l + U2.l) = c • U1.l + c • U2.l := by
```

```
          ext
```

```
          · simp [LinearPMap.add_domain]
```

```
          · simp [LinearPMap.add_apply]
```

```
        have hC' : ((c • U1).l + (c • U2).l).IsClosable := by
```

```
          simp [← h, smul_isClosable_of_isClosable hC]
```

```
        simp only [h, smul_toLinearPMap, add_toLinearPMap_of_dense_closable hD hC,
          add_toLinearPMap_of_dense_closable hD' hC']
```

```
      · -- `D(U1) ∩ D(U2)` is dense and `U1 + U2` is not closable:
```

```
        -- both sides are the zero operator on `D(U1) ∩ D(U2)`.
```

```
        -- `c = 0` must be treated separately because in this case the RHS side `is`
```

```
        rw [smul_toLinearPMap, add_toLinearPMap_of_dense_not_closable hD hC]
```

```
        rcases eq_zero_or_neZero c with (rfl | hc)
```

```
        · have hC' : (((0 : ℂ) • U1).l + ((0 : ℂ) • U2).l).IsClosable :=
```

```
          isClosable_of_zero (by ext; simp [LinearPMap.add_apply])
```

```
          rw [add_toLinearPMap_of_dense_closable hD' hC']
```

```
          ext
```

```
          · simp [LinearPMap.add_domain]
```

```
          · simp [LinearPMap.add_apply]
```

```
        · have h : U1.l + U2.l = c-1 • (c • U1.l + c • U2.l) := by
```

```
          ext
```

```
          · simp [LinearPMap.add_domain]
```

```
          · simp [LinearPMap.add_apply, smul_smul, inv_mul_cancel₀ hc.ne]
```

```
          have hC' := fun h' ↦ hC (h ▸ smul_isClosable_of_isClosable h' c-1)
```

```

      rw [add_toLinearPMap_of_dense_not_closable hD' hC']
      ext <;> simp
    · -- `D(U1) ∩ D(U2)` is not dense: both sides are the junk zero operator
      have hD' : ¬Dense ((c • U1).domain ∩ (c • U2).domain : Set H) := by g
      rw [smul_toLinearPMap, add_toLinearPMap_of_not_dense hD, add_toLinearPMap]
      ext <;> simp

end
## D. Closure
section

variable (U : UnboundedOperator H H')
noncomputable def closure : UnboundedOperator H H' where
lemma le_closure : U ≤ U.closure := LinearPMap.le_closure U.toLinearPMap

lemma isClosed_def : IsClosed U ↔ U.closure = U := by
  refine {fun h => ?_, fun h => h ▸ closure_isClosed U}
  rw [UnboundedOperator.ext_iff, closure_toLinearPMap]
  apply eq_of_eq_graph
  rw [← IsClosable.graph_closure_eq_closure_graph U.is_closable]
  exact IsClosed.submodule_topologicalClosure_eq h

end
## E. Adjoint
section

variable (U : UnboundedOperator H H')
lemma adjoint_dense_of_isClosable {f : LinearPMap ℂ H H'} (h_dense : Dense
  (h_closable : f.IsClosable) : Dense (f†.domain : Set H')) := by
noncomputable def adjoint : UnboundedOperator H' H where
  dense_domain := adjoint_dense_of_isClosable U.dense_domain U.is_closable
lemma adjoint_isClosed : U†.IsClosed := LinearPMap.adjoint_isClosed U.dense
lemma adjoint_closure_eq_adjoint : U†.closure = U† := (isClosed_def U†).mp
  apply UnboundedOperator.ext
  ext
  apply UnboundedOperator.ext
  ext
lemma le_adjoint_adjoint : U ≤ U†† := adjoint_adjoint_eq_closure U ▸ le_closure
lemma isClosed_iff : IsClosed U ↔ U†† = U := adjoint_adjoint_eq_closure U ▸
lemma adjoint_ge_adjoint_of_le {U1 U2 : UnboundedOperator H H'} (h : U1 ≤ U2)
  obtain {h_domain, h_agree} := h
  simp only [Subtype.forall] at h_agree
  have heq (x : U1.domain) (v : U2†.domain) : ∃ U2† v, x ∩ ℂ = ∃ (v : H'), U1
    have hx2 : ↑x ∈ U2.domain := h_domain <| coe_mem x
    have h : U1 x = U2 ⟨x, hx2⟩ := h_agree x x.2 x hx2 rfl

```

1.242.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED.LEAN

```
    exact h ▸ adjoint_isFormalAdjoint U2.dense_domain v ⟨x, hx2⟩
  constructor
  · intro v hv
    apply mem_adjoint_domain_of_exists v
    use U2† ⟨v, hv⟩
    exact fun x ↦ heq x ⟨v, hv⟩
  · exact fun u v huv ↦ (adjoint_apply_eq U1.dense_domain v <| fun x ↦ huv ▸ heq x u)

lemma closure_mono {U1 U2 : UnboundedOperator H H'} (h : U1 ≤ U2) : U1.closure ≤ U2.closure
  repeat rw [← adjoint_adjoint_eq_closure]
  exact adjoint_ge_adjoint_of_le <| adjoint_ge_adjoint_of_le h

end

## F. Symmetric operators
section

variable
  {E : Submodule ℂ H} (hE : Dense (E : Set H))
  {f : E →ℂ E} (hf : f.IsSymmetric)
  (T : UnboundedOperator H H)

/-- An `UnboundedOperator` constructed from a symmetric linear map on a dense submodule -/
def ofSymmetric : UnboundedOperator H H where
  toLinearPMap := LinearPMap.mk E (E.subtype ∘ℂ f)
  dense_domain := hE
  is_closable := by
    refine isClosable_iff_exists_closed_extension.mpr ?_
    use (LinearPMap.mk E (E.subtype ∘ℂ f))†
    exact ⟨LinearPMap.adjoint_isClosed hE, IsFormalAdjoint.le_adjoint hE hf⟩
@[simp]
lemma ofSymmetric_apply (ψ : E) : (ofSymmetric hE hf) ψ = E.subtype (f ψ) := rfl

-- Note that cannot simply co-opt `LinearMap.IsSymmetric` because
-- the domain and codomain of `T` need not be the same.
/-- `T` is symmetric if `⟨T x, y⟩ = ⟨x, T y⟩` for all `x, y ∈ T.domain`. -/
def IsSymmetric : Prop := ∀ x y : T.domain, ⟨T x, y⟩ℂ = ⟨x, T y⟩ℂ

lemma isSymmetric_iff_inner_map_self_real :
  IsSymmetric T ↔ ∀ x : T.domain, (starRingEnd ℂ) ⟨T x, x⟩ℂ = ⟨T x, x⟩ℂ := by
  simp only [inner_conj_symm]
  refine ⟨fun hT x ↦ (hT x x).symm, fun h x y ↦ ?_⟩
  rw [inner_map_polarization, inner_map_polarization']
  rw [h (x + y), h (x - y), h (x + Complex.I • y), h (x - Complex.I • y)]
```

```

lemma isSymmetric_iff_le_adjoint : IsSymmetric  $T \leftrightarrow T \leq T^\dagger$  := by
  refine (fun hT  $\mapsto$  IsFormalAdjoint.le_adjoint T.dense_domain <| IsFormalAdj
  intro h x y
  obtain (h_domain, h_agree) := h
  simp only [Subtype.forall] at h_agree
  have hy :  $\uparrow y \in T^\dagger$ .domain := h_domain <| coe_mem y
  have heq := (IsFormalAdjoint.symm <| adjoint_isFormalAdjoint T.dense_doma
  exact (h_agree y y.2 y hy rfl)  $\triangleright$  heq

end

/-!
## G. Self-adjoint operators
-/

section

variable (T : UnboundedOperator H H)

noncomputable instance instStar : Star (UnboundedOperator H H) := {adjoint}

lemma isSelfAdjoint_def : IsSelfAdjoint  $T \leftrightarrow T^\dagger = T$  := Iff.rfl

lemma isSelfAdjoint_iff : IsSelfAdjoint  $T \leftrightarrow$  IsSelfAdjoint T.toLinearPMap :=
  rw [isSelfAdjoint_def, LinearPMap.isSelfAdjoint_def,  $\leftarrow$  adjoint_toLinearPM
  UnboundedOperator.ext_iff]

lemma isClosed_of_isSelfAdjoint {T : UnboundedOperator H H} (hT : IsSelfAdj
  hT  $\triangleright$  adjoint_isClosed T

lemma isSymmetric_of_isSelfAdjoint {T : UnboundedOperator H H} (hT : IsSelf
  IsSymmetric T := by
  rw [isSymmetric_iff_le_adjoint]
  exact ge_of_eq hT

/-- `T` is essentially self-adjoint if its closure is self-
adjoint. -/
def IsEssentiallySelfAdjoint : Prop := IsSelfAdjoint T.closure

lemma isEssentiallySelfAdjoint_def : IsEssentiallySelfAdjoint  $T \leftrightarrow T^\dagger = T.cl$ 
  rw [IsEssentiallySelfAdjoint, isSelfAdjoint_def, closure_adjoint_eq_adjoin

lemma isSelfAdjoint_isEssentiallySelfAdjoint {T : UnboundedOperator H H} (h
  IsEssentiallySelfAdjoint T := by
  rw [isEssentiallySelfAdjoint_def]
  nth_rw 2 [ $\leftarrow$  hT]

```

1.243.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/SPACEDHILBERTSPACE/BASIC.LEAN

```

    exact Eq.symm <| adjoint_closure_eq_adjoint T

end

/-!
## H. Generalized eigenvectors
-/

section

variable
  {E : Submodule ℂ H} (hE : Dense (E : Set H))
  {f : E →ℂ E} (hf : f.IsSymmetric)
  (T : UnboundedOperator H H)

/-- A map `F : D(T) → L[ℂ] ℂ` is a generalized eigenvector of an unbounded operator `T`
    if there is an eigenvalue `c` such that for all `ψ ∈ D(T)`, `F (T ψ) = c • F ψ`. -/
/
def IsGeneralizedEigenvector (F : T.domain → L[ℂ] ℂ) (c : ℂ) : Prop :=
  ∀ ψ : T.domain, ∃ ψ' : T.domain, ψ' = T ψ ∧ F ψ' = c • F ψ

lemma isGeneralizedEigenvector_ofSymmetric_iff (F : E → L[ℂ] ℂ) (c : ℂ) :
  IsGeneralizedEigenvector (ofSymmetric hE hf) F c ↔ ∀ ψ : E, F (f ψ) = c • F ψ :=
  exact (SetLike.coe_eq_coe.mp hψ') ▸ hψ''
end

```

1.243 PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.l

module

```

public import Mathlib.Analysis.InnerProductSpace.Dual
public import Mathlib.MeasureTheory.Function.L2Space
public import PhysLean.SpaceAndTime.Space.Module
@[expose] public section

```

1.244 PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Schwartz

module

```

public import Mathlib.Analysis.Distribution.SchwartzSpace.Basic
public import PhysLean.QuantumMechanics/DDimensions.SpaceDHilbertSpace.Basic

```

```

@[expose] public section

variable {d : ℕ}
/-- The continuous linear map including Schwartz functions into `SpaceDHilb
/
def schwartzIncl :  $\square$ (Space d,  $\mathbb{C}$ )  $\rightarrow$  L[ $\mathbb{C}$ ] SpaceDHilbertSpace d := toLpCLM  $\mathbb{C}$  (E
instance : CoeFun (schwartzSubmodule d) fun  $\_ \mapsto$  Space d  $\rightarrow$   $\mathbb{C}$  :=  $\langle$ fun  $\psi \mapsto \psi.v$ 

@[simp]
lemma val_eq_coe ( $\psi$  : schwartzSubmodule d) (x : Space d) :  $\psi.val$  x =  $\psi$  x :=

lemma schwartzSubmodule_dense (d : ℕ) :
/-- The linear equivalence between the Schwartz functions ` $\square$ (Space d,  $\mathbb{C}$ )`
and the Schwartz submodule of `SpaceDHilbertSpace d`. -
/
def schwartzEquiv :  $\square$ (Space d,  $\mathbb{C}$ )  $\approx_{\iota[\mathbb{C}]}$  schwartzSubmodule d :=
  LinearEquiv.ofInjective schwartzIncl.toLinearMap (injective_toLp (E := Sp

variable (f g :  $\square$ (Space d,  $\mathbb{C}$ ))

lemma schwartzEquiv_coe_ae : (schwartzEquiv f) ==[volume] f := coeFn_toLp f

lemma schwartzEquiv_inner :
   $\square$ schwartzEquiv f, schwartzEquiv g  $\square \mathbb{C}$  =  $\int$  x : Space d, starRingEnd  $\mathbb{C}$  (f
  apply integral_congr_ae
  filter_upwards [schwartzEquiv_coe_ae f, schwartzEquiv_coe_ae g] with  $\_ hf$ 
  simp [hf, hg, mul_comm]

lemma schwartzEquiv_ae_eq (h : schwartzEquiv f ==[volume] schwartzEquiv g)
  (EmbeddingLike.apply_eq_iff_eq  $\_$ ).mp (SetLike.coe_eq_coe.mp (ext_iff.mpr

```

1.245 PhysLean/QuantumMechanics/FiniteTarget/Basic.lean

```

module

public import Mathlib.Analysis.InnerProductSpace.Adjoint
public import Mathlib.Analysis.Normed.Algebra.Exponential
public import PhysLean.Meta.TODO.Basic
public import PhysLean.QuantumMechanics.PlanckConstant

@[expose] public section

```

1.246.

PHYSLEAN/QUANTUMMECHANICS/FINITETARGET/HILBERTSPACE.lean

1.246 PhysLean/QuantumMechanics/FiniteTarget/HilbertSpace.lean

module

```
public import Mathlib.Analysis.CStarAlgebra.Classes
public import PhysLean.Meta.TODO.Basic
public import Mathlib.Analysis.InnerProductSpace.PiL2
@[expose] public section
```

1.247 PhysLean/QuantumMechanics/OneDimension/GeneralPotential/Basic.lean

module

```
public import PhysLean.QuantumMechanics.OneDimension.Operators.Momentum
public import PhysLean.QuantumMechanics.OneDimension.Operators.Position
@[expose] public section
```

1.248 PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Basic.lean

module

```
public import PhysLean.QuantumMechanics.OneDimension.Operators.Parity
public import PhysLean.QuantumMechanics.OneDimension.Operators.Momentum
public import PhysLean.QuantumMechanics.OneDimension.Operators.Position
@[expose] public section
```

1.249 PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Complex.lean

module

```
public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Eigenfunctions
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Gaussians
@[expose] public section
```

1.250 PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Eigenfunctions.lean

module

```

public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Bas
public import PhysLean.Mathematics.SpecialFunctions.PhysHermite
@[expose] public section

```

1.251 PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator

module

```

public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.TIS
@[expose] public section

```

1.252 PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator

module

```

public import PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Eig
@[expose] public section

```

1.253 PhysLean/QuantumMechanics/OneDimension/HilbertSpace/Basic

module

```

public import Mathlib.MeasureTheory.Function.L2Space
public import Mathlib.Analysis.SpecialFunctions.Gaussian.GaussianIntegral
public import PhysLean.Meta.Linters.Sorry
public import Mathlib.Analysis.InnerProductSpace.Dual
@[expose] public section

```

1.254 PhysLean/QuantumMechanics/OneDimension/HilbertSpace/Gauss

module

```

public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Basic
@[expose] public section

```


1.255.

PHYSLEAN/QUANTUMMECHANICS/ONEDIMENSION/HILBERTSPACE/PLANEWAVES.LEAN

1.255 PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PlaneWaves.lean

module

```
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule
public import Mathlib.Analysis.Distribution.TemperedDistribution
@[expose] public section
```

1.256 PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PositionState.lean

module

```
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule
public import Mathlib.Analysis.Distribution.TemperedDistribution

@[expose] public section
```

1.257 PhysLean/QuantumMechanics/OneDimension/HilbertSpace/SchwartzSubmodule.lean

module

```
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Basic
public import Mathlib.Analysis.Distribution.SchwartzSpace.Fourier
@[expose] public section
```

1.258 PhysLean/QuantumMechanics/OneDimension/Operators/Commutation.lean

module

```
public import PhysLean.QuantumMechanics.OneDimension.Operators.Position
public import PhysLean.QuantumMechanics.OneDimension.Operators.Momentum
@[expose] public section
```

1.259 PhysLean/QuantumMechanics/OneDimension/Operators/Momentum.lean

module

```
public import PhysLean.QuantumMechanics.OneDimension.Operators.Unbounded
```

```
@[expose] public section
```

1.260 PhysLean/QuantumMechanics/OneDimension/Operators/Parity.1

```
module
```

```
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PositionS
public import PhysLean.QuantumMechanics.OneDimension.Operators.Unbounded
@[expose] public section
```

1.261 PhysLean/QuantumMechanics/OneDimension/Operators/Position

```
module
```

```
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PositionS
public import PhysLean.QuantumMechanics.OneDimension.Operators.Unbounded
public import PhysLean.Mathematics.Distribution.PowMul
@[expose] public section
```

1.262 PhysLean/QuantumMechanics/OneDimension/Operators/Unbounded

```
module
```

```
public import PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PlaneWave
public import PhysLean.QuantumMechanics.PlanckConstant
public import Mathlib.Analysis.Calculus.FDeriv.Star
@[expose] public section
```

1.263 PhysLean/QuantumMechanics/OneDimension/ReflectionlessPote

```
module
```

```
public import PhysLean.QuantumMechanics.OneDimension.Operators.Parity
public import PhysLean.QuantumMechanics.OneDimension.Operators.Momentum
public import PhysLean.QuantumMechanics.OneDimension.Operators.Position
public import PhysLean.SpaceAndTime.Time.Basic
```

```
public import PhysLean.Mathematics.Trigonometry.Tanh
@[expose] public section
```

1.264 PhysLean/QuantumMechanics/PlanckConstant.lean

```
module
```

```
public import Mathlib.Data.NNReal.Defs
```

```
@[expose] public section
```

1.265 PhysLean/Relativity/Bispinors/Basic.lean

```
module
```

```
public import PhysLean.Relativity.PauliMatrices.ToTensor
```

```
@[expose] public section
```

1.266 PhysLean/Relativity/CliffordAlgebra.lean

```
module
```

```
public import Mathlib.Analysis.Complex.Basic
```

```
public import Mathlib.Data.Matrix.Reflection
```

```
public import Mathlib.LinearAlgebra.CliffordAlgebra.Basic
```

```
public import PhysLean.Meta.TODO.Basic
```

```
@[expose] public section
```

1.267 PhysLean/Relativity/LorentzAlgebra/Basic.lean

```
module
```

```
public import PhysLean.Relativity.MinkowskiMatrix
```

```
@[expose] public section
```

1.268 PhysLean/Relativity/LorentzAlgebra/Basis.lean

module

```
public import PhysLean.Relativity.LorentzAlgebra.Basic
public import PhysLean.Meta.TODO.Basic
@[expose] public section
```

1.269 PhysLean/Relativity/LorentzAlgebra/ExponentialMap.lean

module

```
public import Mathlib.Analysis.Normed.Field.Instances
public import PhysLean.Mathematics.DataStructures.Matrix.LieTrace
public import PhysLean.Relativity.LorentzAlgebra.Basic
public import PhysLean.Relativity.LorentzGroup.Restricted.Basic
@[expose] public section
```

1.270 PhysLean/Relativity/LorentzGroup/Basic.lean

module

```
public import PhysLean.Relativity.MinkowskiMatrix
public import PhysLean.Meta.TODO.Basic
public import Mathlib.Analysis.Complex.Basic
public import Mathlib.Topology.Instances.Matrix

@[expose] public section
```

1.271 PhysLean/Relativity/LorentzGroup/Boosts/Apply.lean

module

```
public import PhysLean.Relativity.LorentzGroup.Boosts.Basic
@[expose] public section
```

1.272. *PHYSLEAN/RELATIVITY/LORENTZGROUP/BOOSTS/BASIC.LEAN*11

1.272 PhysLean/Relativity/LorentzGroup/Boosts/Basic.lean

module

```
public import PhysLean.SpaceAndTime.SpaceTime.Basic
@[expose] public section
```

1.273 PhysLean/Relativity/LorentzGroup/Boosts/Generalized.lean

module

```
public import PhysLean.Relativity.LorentzGroup.Restricted.Basic
public import PhysLean.Meta.Linters.Sorry
```

```
@[expose] public section
```

1.274 PhysLean/Relativity/LorentzGroup/Orthochronous/Basic.lean

module

```
public import PhysLean.Relativity.LorentzGroup.Proper
public import PhysLean.Relativity.LorentzGroup.ToVector
public import PhysLean.Relativity.Tensors.RealTensor.Velocity.Basic
```

```
@[expose] public section
```

1.275 PhysLean/Relativity/LorentzGroup/Proper.lean

module

```
public import PhysLean.Relativity.LorentzGroup.Basic
public import Mathlib.Topology.Connected.PathConnected
public import Mathlib.Tactic.Cases
@[expose] public section
```

1.276 PhysLean/Relativity/LorentzGroup/Restricted/Basic.lean

module

```

public import PhysLean.Meta.Informal.SemiFormal
public import PhysLean.Relativity.LorentzGroup.Orthochronous.Basic
@[expose] public section

```

1.277 PhysLean/Relativity/LorentzGroup/Restricted/FromBoostRotations.lean

```

module

```

```

public import PhysLean.Relativity.LorentzGroup.Rotations
@[expose] public section

```

1.278 PhysLean/Relativity/LorentzGroup/Rotations.lean

```

module

```

```

public import PhysLean.Relativity.LorentzGroup.Boosts.Generalized
@[expose] public section

```

1.279 PhysLean/Relativity/LorentzGroup/ToVector.lean

```

module

```

```

public import PhysLean.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
@[expose] public section

```

1.280 PhysLean/Relativity/MinkowskiMatrix.lean

```

module

```

```

public import Mathlib.Algebra.Lie.Classical
public import Mathlib.Analysis.Normed.Ring.Lemmas
@[expose] public section

```

1.281 PhysLean/Relativity/PauliMatrices/AsTensor.lean

module

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Two
public import PhysLean.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic
@[expose] public section
```

1.282 PhysLean/Relativity/PauliMatrices/Basic.lean

module

```
public import Mathlib.Algebra.Lie.OfAssociative
public import Mathlib.Analysis.Complex.Basic
public import Mathlib.LinearAlgebra.Matrix.Trace
```

```
@[expose] public section
```

1.283 PhysLean/Relativity/PauliMatrices/CliffordAlgebra.lean

module

```
public import PhysLean.Relativity.PauliMatrices.Basic
public import Mathlib.LinearAlgebra.CliffordAlgebra.Basic
```

```
@[expose] public section
```

1.284 PhysLean/Relativity/PauliMatrices/Relations.lean

module

```
public import PhysLean.Relativity.PauliMatrices.ToTensor
public import PhysLean.Relativity.Tensors.ComplexTensor.Units.Basic
```

```
@[expose] public section
```

1.285 PhysLean/Relativity/PauliMatrices/SelfAdjoint.lean

module

```
public import PhysLean.Relativity.PauliMatrices.Basic
public import PhysLean.Relativity.MinkowskiMatrix
```

```
@[expose] public section
```

1.286 PhysLean/Relativity/PauliMatrices/ToTensor.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Basic
public import PhysLean.Relativity.PauliMatrices.AsTensor
public import PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Basic
```

```
@[expose] public section
```

1.287 PhysLean/Relativity/SL2C/Basic.lean

```
module
```

```
public import PhysLean.Relativity.SL2C.SelfAdjoint
public import Mathlib.Analysis.Complex.Polynomial.Basic
public import PhysLean.Relativity.LorentzGroup.Restricted.Basic
```

```
@[expose] public section
```

1.288 PhysLean/Relativity/SL2C/SelfAdjoint.lean

```
module
```

```
public import PhysLean.Mathematics.SchurTriangulation
public import Mathlib.LinearAlgebra.Matrix.Hermitian
@[expose] public section
```

1.289 PhysLean/Relativity/Special/ProperTime.lean

```
module
```

```
public import PhysLean.SpaceAndTime.SpaceTime.Basic
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.Light
```


1.290. *PHYSLEAN/RELATIVITY/SPECIAL/TWINPARADOX/BASIC.LEAN*15

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.TimeLike
@[expose] public section
```

1.290 PhysLean/Relativity/Special/TwinParadox/Basic.lean

module

```
public import PhysLean.Relativity.Special.ProperTime
@[expose] public section
```

1.291 PhysLean/Relativity/SpeedOfLight.lean

module

```
public import Mathlib.Algebra.Lie.Classical
public import Mathlib.Analysis.Normed.Ring.Lemmas

@[expose] public section
```

1.292 PhysLean/Relativity/Tensors/Basic.lean

module

```
public import PhysLean.Relativity.Tensors.TensorSpecies.Basic
public import Mathlib.Topology.Algebra.Module.ModuleTopology
public import Mathlib.Analysis.RCLike.Basic
@[expose] public section
```

1.293 PhysLean/Relativity/Tensors/Color/Basic.lean

module

```
public import Mathlib.CategoryTheory.Core
public import Mathlib.CategoryTheory.Comma.Over.Basic
public import Mathlib.CategoryTheory.Monoidal.Braided.Basic
public import PhysLean.Meta.TODO.Basic
@[expose] public section
```

1.294 PhysLean/Relativity/Tensors/Color/Discrete.lean

module

```
public import Mathlib.RepresentationTheory.Rep
@[expose] public section
```

1.295 PhysLean/Relativity/Tensors/Color/Lift.lean

module

```
public import PhysLean.Relativity.Tensors.Color.Basic
public import Mathlib.RepresentationTheory.Rep
public import PhysLean.Mathematics.PiTensorProduct
public import Mathlib.Algebra.Lie.OfAssociative
public import PhysLean.Meta.Informal.Basic
@[expose] public section
```

1.296 PhysLean/Relativity/Tensors/ComplexTensor/Basic.lean

module

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Pre
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Metric

@[expose] public section
```

1.297 PhysLean/Relativity/Tensors/ComplexTensor/Lemmas.lean

module

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Basic

@[expose] public section
```

1.298.

PHYSLEAN/RELATIVITY/TENSORS/COMPLEXTENSOR/MATRIX/PRE.LEAN

1.298 PhysLean/Relativity/Tensors/ComplexTensor/Matrix/Pre.lean

module

public import PhysLean.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic

@[expose] public section

1.299 PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean

module

public import PhysLean.Relativity.Tensors.ComplexTensor.OfRat

@[expose] public section

1.300 PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean

module

public import PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Basic

public import PhysLean.Relativity.Tensors.ComplexTensor.Units.Basic

@[expose] public section

1.301 PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Pre.lean

module

public import PhysLean.Relativity.Tensors.ComplexTensor.Units.Pre

@[expose] public section

1.302 PhysLean/Relativity/Tensors/ComplexTensor/OfRat.lean

module

public import PhysLean.Relativity.Tensors.ComplexTensor.Basic

public import PhysLean.Mathematics.RatComplexNum

public import PhysLean.Relativity.Tensors.Dual

@[expose] public section

1.303 PhysLean/Relativity/Tensors/ComplexTensor/Units/Basic.lean

module

public import PhysLean.Relativity.Tensors.ComplexTensor.OfRat

@[expose] public section

1.304 PhysLean/Relativity/Tensors/ComplexTensor/Units/Pre.lean

module

public import PhysLean.Relativity.Tensors.ComplexTensor.Matrix.Pre

public import PhysLean.Relativity.Tensors.ComplexTensor.Vector.Pre.Contract

@[expose] public section

1.305 PhysLean/Relativity/Tensors/ComplexTensor/Units/Symm.lean

module

public import PhysLean.Relativity.Tensors.ComplexTensor.Units.Basic

@[expose] public section

1.306 PhysLean/Relativity/Tensors/ComplexTensor/Vector/Pre/Basi

module

public import PhysLean.Relativity.Tensors.ComplexTensor.Vector.Pre.Modules

@[expose] public section

1.307 PhysLean/Relativity/Tensors/ComplexTensor/Vector/Pre/Cont

module

1.308.

PHYSLEAN/RELATIVITY/TENSORS/COMPLEXTENSOR/VECTOR/PRE/MODULES.LEAN

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic
@[expose] public section
```

1.308 PhysLean/Relativity/Tensors/ComplexTensor/Vector/Pre/Modules.lean

module

```
public import PhysLean.Relativity.SL2C.Basic
@[expose] public section
```

1.309 PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Basic.lean

module

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Modules
public import PhysLean.Relativity.SL2C.Basic
@[expose] public section
```

1.310 PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Contraction.lean

module

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Basic
@[expose] public section
```

1.311 PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Metric.lean

module

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Unit
@[expose] public section
```

1.312 PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Modules.lean

```
module
```

```
public import Mathlib.Algebra.Lie.OfAssociative
public import Mathlib.Analysis.Complex.Basic
@[expose] public section
```

1.313 PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Two.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Basic
@[expose] public section
```

1.314 PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Unit.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Two
public import PhysLean.Relativity.Tensors.ComplexTensor.Weyl.Contraction
@[expose] public section
```

1.315 PhysLean/Relativity/Tensors/Constructors.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.Contraction.Products
@[expose] public section
```

```
    fromPairT (TensorProduct.map LinearMap.id
      (S.FD.map (eqToHom (congrArg Discrete.mk h))).hom.hom' x) =
```

1.316 PhysLean/Relativity/Tensors/Contraction/Basic.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.Contraction.Pure
```

1.317. *PHYSLEAN/RELATIVITY/TENSORS/CONTRACTION/BASIS.LEAN*121

```
@[expose] public section
```

1.317 PhysLean/Relativity/Tensors/Contraction/Basis.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.Contraction.Basic  
@[expose] public section
```

```
    Fin.cast (by simp)
```

1.318 PhysLean/Relativity/Tensors/Contraction/Products.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.Contraction.Basic  
public import PhysLean.Relativity.Tensors.Product  
@[expose] public section
```

1.319 PhysLean/Relativity/Tensors/Contraction/Pure.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.Basic  
@[expose] public section
```

1.320 PhysLean/Relativity/Tensors/Dual.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.MetricTensor  
@[expose] public section
```

1.321 PhysLean/Relativity/Tensors/Elab.lean

module

```
public import PhysLean.Relativity.Tensors.Contraction.Basic
public import PhysLean.Relativity.Tensors.Evaluation
public import PhysLean.Relativity.Tensors.Tensorial
- `{T | [ $\mu$ ] v}T is `eval 0  $\mu$  (tensorNode T)`.

```

```
@[expose] public meta section

```

```
/-- Notation to describe the evaluation of a tensor index. -
/

```

```
syntax "[" ident "]" : indexExpr

```

```
/-- Bool which is true if an index is evaluated bracket `[ $\mu$ ]`. -
/

```

```
def indexExprIsBracketEval(stx : Syntax) : Bool :=
  match stx with
  | `(indexExpr|[$_]) => true
  | _ => false

```

```
  | `(indexExpr| [$a:ident]) => return a

```

```
/-- For list of `indexExpr` e.g. `[ $\alpha$ , 3,  $\beta$ , 2, [ $\gamma$ ]]`, `getEvalPos`
returns a list of pairs ` $\mathbb{N} \times \text{Term}$ ` related to indices which are evaluated
e.g. `[ $\mu$ ]`.

```

The second element of each pair is the value corresponding to that index.

The first element is the position of that number in the list of indices w
all other numbered indices before it are removed. Thus for the example gi

`getEvalBracketPos` outputs `[4,  $\gamma$ ]`. -/

```
def getEvalBracketPos (ind : List (TSyntax `indexExpr)) : TermElabM (List (
  let indEnum := ind.zipIdx
  let evals := indEnum.filter (fun x => indexExprIsBracketEval x.1)
  let evals2 ← (evals.mapM (fun x => indexToIdent x.1))
  let pos := evalAdjustPos (evals.map (fun x => x.2))
  return List.zip pos evals2

```

```
  let indFilt : List (TSyntax `indexExpr) := ind.filter (fun x => ¬ indexExprIsBracketEval x)
  let indFilt := indFilt.filter (fun x => ¬ indexExprIsBracketEval x)

```

```
/-- Given a list `l` of pairs ` $\mathbb{N} \times \text{Term}$ ` and a term `T` corresponding to a
for each `(a, b)` in `l`, `evalSyntax` applies `TensorTree.eval a b` to `T`.
Here `a` is the position of the index to be evaluated and
`b` is the value it is evaluated to from the `[ $\mu$ ]` syntax.

```

For example, if `l` is `[(1,  $\mu$ ), (1,  $v$ )]` and `T` is a tensor tree then
is `TensorTree.eval 1 v (TensorTree.eval 1  $\mu$  T)`.

The list `l` is expected to be the output of `getEvalBracketPos`.
 -/

```
def evalTermBracketMap (l : List (ℕ × Term)) (T : Term) : Term :=
  l.foldl (fun T' (x1, x2) => Syntax.mkApp (mkIdent ``Tensor.evalT)
    #[Syntax.mkNumLit (toString x1), x2, T']) T
```

```
let evalBracketSyntax := evalTermBracketMap (← getEvalBracketPos indices) eval
let contrSyntax := contrTermMap indices.length (← getContrPos indices) evalBra
```

1.322 PhysLean/Relativity/Tensors/Evaluation.lean

module

```
public import PhysLean.Relativity.Tensors.Basic
@[expose] public section
```

1.323 PhysLean/Relativity/Tensors/MetricTensor.lean

module

```
public import PhysLean.Relativity.Tensors.UnitTensor
@[expose] public section
```

```
(unitTensor (S := S) c) := by
(unitTensor (S := S) (S.τ c)) := by
permT id (by simp; exact ⟨rfl,rfl⟩) (unitTensor (S := S) c) := by
```

1.324 PhysLean/Relativity/Tensors/OfInt.lean

module

```
public import PhysLean.Relativity.Tensors.Basic
@[expose] public section
```

1.325 PhysLean/Relativity/Tensors/Product.lean

module

```
public import PhysLean.Relativity.Tensors.Basic
@[expose] public section
```

1.326 PhysLean/Relativity/Tensors/RealTensor/Basic.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.RealTensor.Metrics.Pre
public import PhysLean.Relativity.Tensors.Contraction.Basis
```

```
@[expose] public section
```

1.327 PhysLean/Relativity/Tensors/RealTensor/CoVector/Basic.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.RealTensor.Metrics.Basic
public import Mathlib.Geometry.Manifold.IsManifold.Basic
public import PhysLean.Relativity.Tensors.Elab
```

```
@[expose] public section
```

1.328 PhysLean/Relativity/Tensors/RealTensor/Derivative.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.ComplexTensor.Basic
```

```
@[expose] public section
```

1.329 PhysLean/Relativity/Tensors/RealTensor/Matrix/Pre.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Pre.Basic
public import Mathlib.LinearAlgebra.TensorProduct.Matrix
```

```
@[expose] public section
```

1.330.

~~PHYSLEAN/RELATIVITY/TENSORS/REALTENSOR/METRICS/BASIC.lean~~

1.330 PhysLean/Relativity/Tensors/RealTensor/Metrics/Basic.lean

module

```
public import PhysLean.Relativity.Tensors.RealTensor.Basic
public import PhysLean.Relativity.Tensors.MetricTensor
```

```
@[expose] public section
```

1.331 PhysLean/Relativity/Tensors/RealTensor/Metrics/Pre.lean

module

```
public import PhysLean.Relativity.Tensors.RealTensor.Units.Pre
```

```
@[expose] public section
```

1.332 PhysLean/Relativity/Tensors/RealTensor/ToComplex.lean

module

```
public import PhysLean.Meta.Linters.Sorry
public import PhysLean.Relativity.Tensors.ComplexTensor.Basic
@[expose] public section
```

1.333 PhysLean/Relativity/Tensors/RealTensor/Units/Pre.lean

module

```
public import PhysLean.Relativity.Tensors.RealTensor.Matrix.Pre
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Pre.Contraction
```

```
@[expose] public section
```

1.334 PhysLean/Relativity/Tensors/RealTensor/Vector/Basic.lean

module

```
public import PhysLean.Relativity.Tensors.RealTensor.Metrics.Basic
public import PhysLean.Relativity.Tensors.Elab
```

```
public import Mathlib.Geometry.Manifold.IsManifold.Basic
@[expose] public section
```

1.335 PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/B

```
module
```

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
@[expose] public section
```

1.336 PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/L

```
module
```

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.Basic
@[expose] public section
```

1.337 PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/T

```
module
```

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.Basic
@[expose] public section
```

1.338 PhysLean/Relativity/Tensors/RealTensor/Vector/MinkowskiPr

```
module
```

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Basic
@[expose] public section
```

1.339 PhysLean/Relativity/Tensors/RealTensor/Vector/Pre/Basic.1

```
module
```

1.340.

PHYSLEAN/RELATIVITY/TENSORS/REALTENSOR/VECTOR/PRE/CONTRACTION. LEAN

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Pre.Modules
public import Mathlib.RepresentationTheory.Rep
@[expose] public section
```

1.340 PhysLean/Relativity/Tensors/RealTensor/Vector/Pre/Contraction.lean

module

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.Pre.Basic
@[expose] public section
```

1.341 PhysLean/Relativity/Tensors/RealTensor/Vector/Pre/Modules.lean

module

```
public import PhysLean.Relativity.PauliMatrices.SelfAdjoint
public import Mathlib.RepresentationTheory.Basic
public import PhysLean.Relativity.LorentzGroup.Basic
public import Mathlib.Analysis.InnerProductSpace.PiL2
@[expose] public section
```

1.342 PhysLean/Relativity/Tensors/RealTensor/Velocity/Basic.lean

module

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
@[expose] public section
```

1.343 PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean

module

```
public import PhysLean.Relativity.Tensors.Color.Discrete
public import PhysLean.Relativity.Tensors.Color.Lift
@[expose] public section
```

1.344 PhysLean/Relativity/Tensors/Tensorial.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.Product
public import Mathlib.Topology.Algebra.Module.FiniteDimension
@[expose] public section
```

1.345 PhysLean/Relativity/Tensors/UnitTensor.lean

```
module
```

```
public import PhysLean.Relativity.Tensors.Constructors
@[expose] public section
```

1.346 PhysLean/SpaceAndTime/Space/Basic.lean

```
module
```

```
public import PhysLean.Meta.Informal.Basic
public import PhysLean.Meta.TODO.Basic
public import PhysLean.Meta.Linters.Sorry
public import Mathlib.Topology.ContinuousMap.CompactlySupported
public import Mathlib.Geometry.Manifold.IsManifold.Basic
public import Mathlib.MeasureTheory.Measure.Lebesgue.VolumeOfBalls
public import Mathlib.Analysis.InnerProductSpace.Calculus
public import Mathlib.Geometry.Manifold.Diffeomorph
to `d`-dimensional flat Euclidean space and prove some properties about it.
```

The scope of this module is to define on `Space d` basic instances related to the metric. We do not here define the structure of a `Module` on `Space d`, `PhysLean.SpaceAndTime.Space.Module`.

Implementation details

The exact implementation of `Space d` into PhysLean is discussed in numerous posts on the Lean Zulip, including:

- <https://leanprover.zulipchat.com/#narrow/channel/479953-PhysLean/topic/Space.20vs.20EuclideanSpace/with/575332888>

There is a choice between defining ``Space d`` as an ``abbrev`` of ``EuclideanSpace  $\mathbb{R}$  (Fin d)`` or as a structure with a field ``val : Fin d  $\rightarrow$   $\mathbb{R}$ `` :

	abbrev	def	structure
allows casting from <code>`EuclideanSpace`</code>	yes	yes	no
carries instances from <code>`EuclideanSpace`</code>	yes	no	no
requires reproving of lemmas from <code>`EuclideanSpace`</code>	no	yes	yes

The ``structure`` is the most conservative choice, as everything needs redefining. However, there are two benefits of using it:

1. It allows us to be precise about the instances we define on ``Space d``, and makes future refactoring of those instances easier.
2. It allows us to give the necessary physics context to results about ``Space d``, which would not otherwise be possible if we reuse results from Mathlib.

In this module we give ``Space d`` the instances of a ``NormedAddTorsor`` and a ``MetricSpace``. These physically correspond to the statement that you can add a vector to a point in space, and that there is a notion of distance between points in space. This notion of distance corresponds to a choice of length unit.

In ``PhysLean.SpaceAndTime.Space.Module`` we give ``Space d`` the structure of a ``Module`` (aka vector space), a ``Norm`` and an ``InnerProductSpace``. These require certain choices, such as the choice of a zero in ``Space d``.

This module structure is necessary in numerous places. For example, the normal derivatives used by physicists `` $\partial_x$ ,  $\partial_y$ ,  $\partial_z$ `` require a module structure.

Because of this, one should be careful to avoid using the explicit zero in ``Space d`` or adding two ``Space d`` values together. Where possible one should use the ``VAdd (EuclideanSpace  $\mathbb{R}$  (Fin d)) (Space d)`` instance instead.

@[expose] public section

A. The ``Space`` type

`/-- The type `Space d` is the world-volume which corresponds to
`d` dimensional (flat) Euclidean space with a given (but arbitrary)
choice of length unit, and a given (but arbitrary) choice of zero.`

The default value of `d` is `3`. Thus ``Space = Space 3`` -/

B.2. The Non-emptiness

```
instance {d} : Nonempty (Space d) := Nonempty.intro
  (fun _ => Classical.choice instNonemptyOfInhabited)
instance : Subsingleton (Space 0) := Subsingleton.intro <| fun _ _ =>
  eq_of_apply <| fun i => Fin.elim0 i
```

B.3.1. The additive action

B.3.2. Subtraction

```
noncomputable instance {d} : VSub (EuclideanSpace ℝ (Fin d)) (Space d) where
  vsub s1 s2 := WithLp.toLp 2 <| fun i => s1 i - s2 i
lemma vsub_apply {d} (s1 s2 : Space d) (i : Fin d) :
  (s1 -v s2) i = s1 i - s2 i := rfl
```

/-!

B.3.3. AddTorsor instance

-/

```
noncomputable instance {d} : AddTorsor (EuclideanSpace ℝ (Fin d)) (Space d)
  vsub_vadd' s1 s2 := by
  vadd_vsub' s1 s2 := by
```

B.4. PseudoMetricSpace

```
noncomputable instance {d} : Dist (Space d) where
  dist p q := √ (∑ i, (p i - q i) ^ 2)
lemma dist_eq {d} (p q : Space d) : dist p q = √ (∑ i, (p i - q i) ^ 2) :=
noncomputable instance {d} : PseudoMetricSpace (Space d) where
  dist_self x := by simp [dist_eq]
  dist_comm x y := by grind [dist_eq]
  dist_triangle x y z := by
    convert dist_triangle (WithLp.toLp 2 fun i => x i) (WithLp.toLp 2 fun i =>
      (WithLp.toLp 2 fun i => z i))
  all_goals
    rw [EuclideanSpace.dist_eq]
    simp only [dist, sq_abs]
```

B.5. NormAddTorsor instance

```
noncomputable instance {d} : NormedAddTorsor (EuclideanSpace ℝ (Fin d)) (Space d)
  dist_eq_norm' p q := by simp [dist, EuclideanSpace.norm_eq]
```

B.6. Metric space instance

```
noncomputable instance {d} : MetricSpace (Space d) where
  eq_of_dist_eq_zero {p q} := by simp [NormedAddTorsor.dist_eq_norm']
```

B.7. Non-triviality

```
instance {d : ℕ} : Nontrivial (Space d.succ) where
  exists_pair_ne := by
    obtain k := Classical.choice Space.instNonempty
    obtain ⟨v1, hv⟩ := exists_ne (0 : EuclideanSpace ℝ (Fin d.succ))
```



```

    use k, v1 +v k
    simp only [ne_eq, eq_vadd_iff_vsub_eq, vsub_self] using hv.symm
## C. Model structure (i.e. structure of a manifold)
/-- The manifold structure on `Space d`. -/
noncomputable def manifoldStructure (d : ℕ) :
  ModelWithCorners ℝ (EuclideanSpace ℝ (Fin d)) (Space d) where
  toFun := (Equiv.vaddConst (Classical.choice Space.instNonempty)).symm
  invFun := Equiv.vaddConst (Classical.choice Space.instNonempty)
  source := Set.univ
  target := Set.univ
  map_source' := by simp
  map_target' := by simp
  left_inv' := by simp
  right_inv' := by simp
  source_eq := by simp
  convex_range' := by
    rw [dif_pos (instIsRCLikeNormedField ℝ), Equiv.range_eq_univ]
    exact fun _ _ => by simp
  nonempty_interior' := by
    rw [Equiv.range_eq_univ]
    simp only [Equiv.coe_vaddConst_symm]
    simp only [Equiv.coe_vaddConst]
lemma manifoldStructure_comp_manifoldStructure_symm {d : ℕ} :
  (↑(manifoldStructure d) ∘ ↑(manifoldStructure d).symm) = id := by
  ext1 x
  simp using (manifoldStructure d).right_inv' (x := x) (by simp [manifoldStructure])

lemma manifoldStructure_comp_manifoldStructure_symm_apply {d : ℕ}
  (x : EuclideanSpace ℝ (Fin d)) :
  (manifoldStructure d) ((manifoldStructure d).symm x) = x := by
  simp using (manifoldStructure d).right_inv' (x := x) (by simp [manifoldStructure])

@[simp]
lemma range_manifoldStructure {d : ℕ} :
  (Set.range ↑(manifoldStructure d)) = Set.univ := by
  ext x
  simp using ((manifoldStructure d).symm x, manifoldStructure_comp_manifoldStructure)

open Manifold in
lemma contMDiff_vaddConst (d : ℕ) : ContMDiff
  (manifoldStructure d) (□(ℝ, EuclideanSpace ℝ (Fin d))) ⊤ (manifoldStructure d).
  rw [contMDiff_iff]
  refine ((manifoldStructure d).continuous_toFun, fun x y => ?_)
  simp only [extChartAt, OpenPartialHomeomorph.extend, OpenPartialHomeomorph.refl_pa
    PartialEquiv.refl_source, OpenPartialHomeomorph.singletonChartedSpace_chartAt_eq
    modelWithCornersSelf_partialEquiv, PartialEquiv.trans_refl, PartialEquiv.refl_co

```

```

    ModelWithCorners.toPartialEquiv_coe, PartialEquiv.refl_trans,
    ModelWithCorners.toPartialEquiv_coe_symm, manifoldStructure_comp_manifo
    CompTriple.comp_eq, ModelWithCorners.target_eq, Set.preimage_univ, Set.
    exact contDiffOn_id

```

1.347 PhysLean/SpaceAndTime/Space/ConstantSliceDist.lean

```

module

public import PhysLean.SpaceAndTime.Space.Slice

@[expose] public section

```

1.348 PhysLean/SpaceAndTime/Space/CrossProduct.lean

```

module

public import Mathlib.LinearAlgebra.CrossProduct
public import PhysLean.SpaceAndTime.Time.Derivatives
@[expose] public section

```

1.349 PhysLean/SpaceAndTime/Space/Derivatives/Basic.lean

```

module

public import Mathlib.Analysis.Calculus.FDeriv.Symmetric
public import Mathlib.Analysis.Calculus.Gradient.Basic
public import PhysLean.SpaceAndTime.Space.DistanceOfFunction
public import Mathlib.Geometry.Manifold.MFDeriv.FDeriv
@[expose] public section

lemma deriv_eq_fderiv_fun [AddCommGroup M] [Module ℝ M] [TopologicalSpace
  (μ : Fin d) (f : Space d → M) :
  deriv μ f = fun x => fderiv ℝ (fun x => f x) x (basis μ) := by rfl

open Manifold in
/-- The spatial-derivative in terms of the derivative of functions between
  manifolds. -/
lemma deriv_eq_mfderiv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (μ : Fin d) (f : Space d → M) (x : Space d) :
  deriv μ f x = mfderiv (ℝ, Space d) (ℝ, M) f x (basis μ) := by

```

```

    rw [deriv_eq_fderiv_basis, ← mfderiv_eq_fderiv]
    rfl

open Manifold in
lemma mdifferentiable_manifoldStructure_iff_differentiable {M d} [NormedAddCommGroup M] [NormedSpace ℝ M] {f : Space d → M} {x : Space d} :
  MDifferentiableAt (Space.manifoldStructure d) (x : ℝ, M) f x ↔ DifferentiableAt ℝ f x
constructor
· intro h
  rw [← mdifferentiableAt_iff_differentiableAt]
  apply h.comp (I' := Space.manifoldStructure d)
  exact (modelDiffeo.symm.mdifferentiable (WithTop.top_ne_zero)).mdifferentiableAt
· intro h
  apply (mdifferentiableAt_iff_differentiableAt.mpr h).comp (I' := (x : ℝ, Space d))
  exact (modelDiffeo.mdifferentiable (WithTop.top_ne_zero)).mdifferentiableAt

TODO "3XMN6" "Make the version of the derivative described through
`deriv_eq_mfderiv_manifoldStructure` the definition of `deriv` and prove the
equivalence with the current definition, under suitable conditions."

open Manifold in
/-- The spatial-derivative in terms of the derivative of functions between
manifolds with the manifold structure `Space.manifoldStructure d`. -
/
lemma deriv_eq_mfderiv_manifoldStructure {M d} [NormedAddCommGroup M] [NormedSpace ℝ M] (μ : Fin d) (f : Space d → M) (x : Space d) :
  deriv μ f x = mfderiv (Space.manifoldStructure d) (x : ℝ, M) f x (EuclideanSpace.single μ 1)
by_cases hf : DifferentiableAt ℝ f x
· rw [deriv_eq_mfderiv]
  change _ = mfderiv (Space.manifoldStructure d) (x : ℝ, M)
    (f ∘ modelDiffeo) x (EuclideanSpace.single μ 1)
  rw [mfderiv_comp (I' := (x : ℝ, Space d)) hf.mdifferentiableAt
    (modelDiffeo.mdifferentiable WithTop.top_ne_zero).mdifferentiableAt]
  simp only [Function.comp_apply, modelDiffeo_apply, mfderiv_eq_fderiv,
    ContinuousLinearMap.coe_comp']
  rw [basis_eq_mfderiv_modelDiffeo_single]
  rfl
· rw [deriv_eq, fderiv_zero_of_not_differentiableAt hf,
  mfderiv_zero_of_not_mdifferentiableAt <|
  mdifferentiable_manifoldStructure_iff_differentiable.mp.mt hf]
  simp

rw [deriv_eq_fderiv_fun]
rw [deriv_eq_fderiv_fun, deriv_eq_fderiv_fun, deriv_eq_fderiv_fun]
rw [deriv_eq_fderiv_basis, deriv_eq_fderiv_basis, deriv_eq_fderiv_basis, fderiv_sm]

```

```

rw [deriv_eq_fderiv_fun, deriv_eq_fderiv_fun]
rw [fderiv_const_smul, ContinuousLinearMap.coe_smul', Pi.smul_apply, Pi.s
rw [deriv_eq_fderiv_basis, deriv_eq_fderiv_basis, fderiv_const_mul]
rw [deriv_eq_fderiv_fun, deriv_eq_fderiv_fun, deriv_eq_fderiv_fun, deriv_
open ContDiff

lemma deriv_contDiff {d} {f : Space d → ℝ} (hf : ContDiff ℝ (n + 1) f) :
  ContDiff ℝ n fun x i => deriv i f x := by
  unfold deriv
  fun_prop

```

1.350 PhysLean/SpaceAndTime/Space/Derivatives/Curl.lean

```
module
```

```

public import PhysLean.SpaceAndTime.Space.Derivatives.Laplacian
- A.3. Basic operations on curl
- A.4. The curl of a linear map is a linear map
- A.5. Preliminary lemmas about second derivatives
- A.6. The div of a curl is zero
- A.7. The curl of a grad is zero
@[expose] public section

macro (name := curlNotation) "∇" "×" f:term:100 : term => `(curl $f)
lemma curl_zero : ∇ × (0 : Space → EuclideanSpace ℝ (Fin 3)) = 0 := by
lemma curl_const : ∇ × (fun _ : Space => v₃) = 0 := by
### A.3. Basic operations on curl
  ∇ × (f1 + f2) = ∇ × f1 + ∇ × f2 := by
  ∇ × (k • f) = k • ∇ × f := by
lemma curl_neg (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : Differentiable
  ∇ × (-f) = -∇ × f := by
  rw [← neg_one_smul ℝ, curl_smul, neg_one_smul]
  · exact hf

lemma curl_sub (f1 f2 : Space → EuclideanSpace ℝ (Fin 3))
  (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
  ∇ × (f1 - f2) = ∇ × f1 - ∇ × f2 := by
  rw [sub_eq_add_neg, curl_add, curl_neg, sub_eq_add_neg]
  repeat fun_prop

### A.4. The curl of a linear map is a linear map
  IsLinearMap ℝ (fun w => ∇ × (f w)) := by
### A.5. Preliminary lemmas about second derivatives

```

```

    repeat rw [deriv_eq_fderiv_fun]
    repeat rw [deriv_eq_fderiv_fun]
### A.6. The div of a curl is zero
 $\nabla \cdot (\nabla \times f) = 0$  := by
### A.7. The curl of a grad is zero

-/

lemma curl_of_grad_eq_zero (f : Space → ℝ) (hf : ContDiff ℝ 2 f) :
 $\nabla \times (\nabla f) = 0$  := by
  unfold curl grad
  ext x i
  fin_cases i <=>
  simp only [Fin.isValue, Pi.ofNat_apply, Fin.zero_eta, PiLp.zero_apply] <=>
  · rw [deriv_commute]
    simp only [Fin.isValue, sub_self]
    · exact hf

/-!

 $\nabla \times (\nabla \times f) = \nabla (\nabla \cdot f) - \Delta f$  := by

```

1.351 PhysLean/SpaceAndTime/Space/Derivatives/Div.lean

```
module
```

```
public import PhysLean.SpaceAndTime.Space.Derivatives.Grad
@[expose] public section
```

1.352 PhysLean/SpaceAndTime/Space/Derivatives/Grad.lean

```
module
```

```
public import PhysLean.SpaceAndTime.Space.Derivatives.Basic
- A.14. Differentiability of gradient
- B.6. The gradient of a Schwartz map
@[expose] public section
```

```
### A.14. Differentiability of gradient
```

```
-/
```

```

open ContDiff

@[fun_prop]
lemma contDiff_grad {n} {f : Space → ℝ} (hf : ContDiff ℝ (n + 1) f) :
  ContDiff ℝ n (fun x => ∇ f x) := by
  unfold grad
  apply ContDiff.fun_comp
  · fun_prop
  · exact deriv_contDiff hf

/-!
/-!

### B.6. The gradient of a Schwartz map

-/

/-- The gradient on Schwartz maps. -/
noncomputable def gradSchwartz {d} : (Space d, ℝ) → L[ℝ] (Space d, EuclideanSpace.single i (1 : ℝ))
  ∑ i, SchwartzMap.bilinLeftCLM (ContinuousLinearMap.lsmul ℝ ℝ)
    (.const (EuclideanSpace.single i (1 : ℝ)))
    ◦ L SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i)
    ◦ L SchwartzMap.fderivCLM ℝ (Space d) ℝ

lemma gradSchwartz_apply_eq_grad {d} (η : (Space d, ℝ)) (x : Space d):
  gradSchwartz η x = grad η x := by
  simp [gradSchwartz, grad_eq_sum]
  rfl

```

1.353 PhysLean/SpaceAndTime/Space/Derivatives/Laplacian.lean

```

module

public import PhysLean.SpaceAndTime.Space.Derivatives.Div
@[expose] public section

```

1.354 PhysLean/SpaceAndTime/Space/DistConst.lean

```

module

```

```
public import PhysLean.SpaceAndTime.Space.Derivatives.Curl
@[expose] public section
```

1.355 PhysLean/SpaceAndTime/Space/DistOfFunction.lean

```
module
```

```
public import PhysLean.SpaceAndTime.Space.IsDistBounded
public import Mathlib.MeasureTheory.SpecificCodomains.WithLp
```

```
@[expose] public section
```

1.356 PhysLean/SpaceAndTime/Space/Integrals/Basic.lean

```
/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module
```

```
public import PhysLean.SpaceAndTime.Space.Module
/-!
```

```
# Integrals in Space
```

```
## i. Overview
```

In this module we give general properties of integrals over ``Space d``. We focus here on the volume measure, which is the usual measure on ``Space d``, i.e. ``dx dy dz``.

```
## ii. Key results
```

- ``volume_eq_addHaar`` : The volume measure on ``Space d`` is the same as the Haar measure associated with the basis of ``Space d``.
- ``integral_volume_eq_spherical`` : The integral of a function over ``Space d.succ`` with respect to the volume measure can be expressed as an integral over the unit sphere and the positive reals.
- ``lintegral_volume_eq_spherical`` : The lower Lebesgue integral of a function over ``Space d.succ`` with respect to the volume measure can be expressed as a lower Lebesgue integral over the unit sphere and the positive reals.

```

-/

@[expose] public section

namespace Space

open InnerProductSpace MeasureTheory

/-!

## A. Properties of the volume measure

-/

lemma volume_eq_addHaar {d} : (volume (α := Space d)) = Space.basis.toBasis
  exact (OrthonormalBasis.addHaar_eq_volume _).symm

lemma volume_closedBall_ne_zero {d : ℕ} (x : Space d.succ) {r : ℝ} (hr : 0
  volume (Metric.closedBall x r) ≠ 0 := by
  obtain ⟨k,hk⟩ := Nat.even_or_odd' d.succ
  rcases hk with hk | hk
  · rw [InnerProductSpace.volume_closedBall_of_dim_even (k := k)]
    simp only [Nat.succ_eq_add_one, finrank_eq_dim, ne_eq, mul_eq_zero, Nat
      one_ne_zero, and_false, not_false_eq_true, pow_eq_zero_iff, ENNReal.o
      not_le]
    apply And.intro
    · simp_all
    · positivity
    · simpa using hk
  · rw [InnerProductSpace.volume_closedBall_of_dim_odd (k := k)]
    simp only [Nat.succ_eq_add_one, finrank_eq_dim, ne_eq, mul_eq_zero, Nat
      one_ne_zero, and_false, not_false_eq_true, pow_eq_zero_iff, ENNReal.o
      not_le]
    apply And.intro
    · simp_all
    · positivity
    · simpa using hk

lemma volume_closedBall_ne_top {d : ℕ} (x : Space d.succ) (r : ℝ) :
  volume (Metric.closedBall x r) ≠ ⊤ := by
  obtain ⟨k,hk⟩ := Nat.even_or_odd' d.succ
  rcases hk with hk | hk
  · rw [InnerProductSpace.volume_closedBall_of_dim_even (by simpa using hk)]
    simp only [Nat.succ_eq_add_one, finrank_eq_dim, ne_eq]
    apply not_eq_of_beq_eq_false

```



```

    rfl
    · rw [InnerProductSpace.volume_closedBall_of_dim_odd (by simp using hk)]
      simp only [Nat.succ_eq_add_one, finrank_eq_dim, ne_eq]
      apply not_eq_of_beq_eq_false
      rfl

@[simp]
lemma volume_metricBall_three :
  volume (Metric.ball (0 : Space 3) 1) = ENNReal.ofReal (4 / 3 * Real.pi) := by
  rw [InnerProductSpace.volume_ball_of_dim_odd (k := 1)]
  simp only [ENNReal.ofReal_one, finrank_eq_dim, one_pow, pow_one, Nat.reduceAdd,
    Nat.doubleFactorial_eq_3, Nat.doubleFactorial, mul_one, Nat.cast_ofNat, one_mul]
  ring_nf
  simp

@[simp]
lemma volume_metricBall_two :
  volume (Metric.ball (0 : Space 2) 1) = ENNReal.ofReal Real.pi := by
  rw [InnerProductSpace.volume_ball_of_dim_even (k := 1)]
  simp [finrank_eq_dim]
  simp [finrank_eq_dim]

@[simp]
lemma volume_metricBall_two_real :
  (volume.real (Metric.ball (0 : Space 2) 1)) = Real.pi := by
  trans (volume (Metric.ball (0 : Space 2) 1)).toReal
  · rfl
  rw [volume_metricBall_two]
  simp only [ENNReal.toReal_ofReal_eq_iff]
  exact Real.pi_nonneg

@[simp]
lemma volume_metricBall_three_real :
  (volume.real (Metric.ball (0 : Space 3) 1)) = 4 / 3 * Real.pi := by
  trans (volume (Metric.ball (0 : Space 3) 1)).toReal
  · rfl
  rw [volume_metricBall_three]
  simp only [ENNReal.toReal_ofReal_eq_iff]
  positivity

/-!

## B. Integrals over one-dimensional space

-/

```

```

lemma integral_one_dim_eq_integral_real {f : Space 1 → ℝ} :
  ∫ x, f x ∂volume = ∫ x, f (oneEquiv.symm x) ∂volume := by rw [integral_

/-!

## C. Integrals over volume to spherical

-/

lemma integral_volume_eq_spherical (d : ℕ) (f : Space d.succ → F)
  [NormedAddCommGroup F] [NormedSpace ℝ F] :
  ∫ x, f x ∂volume = ∫ x, f (x.2.1 • x.1.1) ∂(volume (α := Space d.succ).
    (Measure.volumeIoiPow (Module.finrank ℝ (Space d.succ) - 1))) := by
  rw [← MeasureTheory.MeasurePreserving.integral_comp (f := homeomorphUnitS
    (MeasureTheory.Measure.measurePreserving_homeomorphUnitSphereProd
      (volume (α := Space d.succ)))
    (Homeomorph.measurableEmbedding (homeomorphUnitSphereProd (Space d.succ
  simp only [Nat.succ_eq_add_one, homeomorphUnitSphereProd_apply_snd_coe,
    homeomorphUnitSphereProd_apply_fst_coe]
  let f' : (x : (Space d.succ)) → F := fun x => f (‖↑x‖ • ‖↑x‖-1 • ↑x)
  conv_rhs =>
    enter [2, x]
    change f' x.1
  rw [MeasureTheory.integral_subtype_comap (by simp), ← setIntegral_univ]
  simp [f']
  refine integral_congr_ae ?_
  have h1 : ∀m x ∂(volume (α := Space d.succ)), x ≠ 0 := by
    exact Measure.ae_ne volume 0
  filter_upwards [Measure.ae_ne volume 0] with x hx
  congr
  simp [smul_smul]
  have hx : ‖x‖ ≠ 0 := by
    simpa using hx
  field_simp
  simp

/- An instance of `sfinite` on the spherical integral measure on `Space d`.
   This is needed in many of the calculations related to spherical integrals
   but cannot be inferred by Lean without this. -/
instance : SFinite (@Measure.comap ↑(Set.Ioi 0) ℝ Subtype.instMeasurableSpace
  Real.measureSpace.toMeasurableSpace Subtype.val volume) := by
  refine { out' := ?_ }
  have h1 := SFinite.out' (μ := volume (α := ℝ))
  obtain ⟨m, h⟩ := h1
  use fun n => Measure.comap Subtype.val (m n)
  apply And.intro

```

```

    · intro n
      refine (isFiniteMeasure_iff (Measure.comap Subtype.val (m n))).mpr ?_
      rw [MeasurableEmbedding.comap_apply (MeasurableEmbedding.subtype_coe measura
      simp only [Set.image_univ, Subtype.range_coe_subtype, Set.mem_Ioi]
      have hm := h.1 n
      exact measure_lt_top (m n) {x | 0 < x}
    · ext s hs
      rw [MeasurableEmbedding.comap_apply, Measure.sum_apply]
      conv_rhs =>
        enter [1, i]
          rw [MeasurableEmbedding.comap_apply (MeasurableEmbedding.subtype_coe measu
          have h2 := h.2
          rw [Measure.ext_iff'] at h2
          rw [← Measure.sum_apply]
          exact h2 (Subtype.val '' s)
          refine MeasurableSet.subtype_image measurableSet_Ioi hs
          exact hs
          apply MeasurableEmbedding.subtype_coe
          simp

/-!

## D. Lower Lebesgue integral over volume to spherical

-/

lemma lintegral_volume_eq_spherical (d : ℕ) (f : Space d.succ → ENNReal) (hf : Measu
  ∫- x, f x ∂volume = ∫- x, f (x.2.1 • x.1.1) ∂(volume (α := Space d.succ).toSpher
    (Measure.volumeIoiPow (Module.finiteRank ℝ (Space d.succ) - 1))) := by
  have h0 := MeasureTheory.MeasurePreserving.lintegral_comp
    (f := fun x => f (x.2.1 • x.1.1)) (g := homeomorphUnitSphereProd _)
    (MeasureTheory.Measure.measurePreserving_homeomorphUnitSphereProd
    (volume (α := Space d.succ)))
    (by fun_prop)
  rw [← h0]
  simp only [Nat.succ_eq_add_one, homeomorphUnitSphereProd_apply_snd_coe,
    homeomorphUnitSphereProd_apply_fst_coe]
  let f' : (x : (Space d.succ)) → ENNReal := fun x => f (||↑x|| • ||↑x||-1 • ↑x)
  conv_rhs =>
    enter [2, x]
      change f' x.1
  rw [MeasureTheory.lintegral_subtype_comap (by simp)]
  simp [f']
  refine lintegral_congr_ae ?_
  filter_upwards [Measure.ae_ne volume 0] with x hx
  congr

```

```

simp [smul_smul]
have hx :  $\|x\| \neq 0$  := by
  simpa using hx
field_simp
rw [one_smul]

lemma lintegral_volume_eq_spherical_mul (d :  $\mathbb{N}$ ) (f : Space d.succ  $\rightarrow$  ENNReal)
   $\int^- x, f x \, dv_{\text{volume}} = \int^- x, f (x.2.1 \cdot x.1.1) * \text{.ofReal } (x.2.1 ^ d)$ 
   $\partial(\text{volume } (\alpha := \text{Space } d.\text{succ}).\text{toSphere}.\text{prod } (\text{Measure.volumeIoiPow } 0))$ 
  rw [lintegral_volume_eq_spherical, Measure.volumeIoiPow,
    MeasureTheory.prod_withDensity_right,
    MeasureTheory.lintegral_withDensity_eq_lintegral_mul,
    Measure.volumeIoiPow, MeasureTheory.prod_withDensity_right,
    MeasureTheory.lintegral_withDensity_eq_lintegral_mul]
  · refine lintegral_congr_ae ?_
    simp only [Nat.succ_eq_add_one, finrank_eq_dim, add_tsub_cancel_right,
      ENNReal.ofReal_one]
    filter_upwards with x
    simp only [Pi.mul_apply, one_mul]
    ring
  all_goals fun_prop

end Space

```

1.357 PhysLean/SpaceAndTime/Space/Integrals/NormPow.lean

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
module

public import PhysLean.SpaceAndTime.Space.Norm
/-!

# Integrability of norm powers on subsets of Space

## i. Overview

This module contains results on the integrability of  $x \mapsto \|x\|^p$  for real ex
`Space d`.

The integrability of  $x \mapsto \|x\|^p$  on `ball 0 b` and `(ball 0 b)^c` follows from

```

`integrable0n\_Ioo\_rpow\_iff` and `integrable0n\_Ioi\_rpow\_iff` after changing to spheri

ii. Key results

- `integrable0n\_norm\_rpow\_iff\_of\_isBounded\_nhds`: Necessary and sufficient condition for integrability on a set `s` which is a bounded neighborhood of the origin.
- `integrable0n\_norm\_rpow\_of\_isBounded\_compl\_nhds`: Integrability for all bounded sets with ` $0 \notin \text{closure } s$ `.
- `integrable0n\_norm\_rpow\_of\_compl\_nhds`: A sufficient condition for integrability on sets with ` $0 \notin \text{closure } s$ `.

iii. Table of contents

iv. References

-/

@[expose] public section

namespace Space

open MeasureTheory

```
private lemma npow_indicator_rpow_eq {n : ℕ} {s : Set ℝ} (hs : 0 ∉ s) (p : ℝ) :
  (fun r ↦ r ^ n • s.indicator (fun r ↦ r ^ p) r) = s.indicator (fun r ↦ r ^ (n + p)) r
by
  ext r
  by_cases hr : r ∈ s
  · grind [Set.indicator_of_mem, smul_eq_mul, add_comm, Real.rpow_add_natCast]
  · simp [hr]
```

```
private lemma _root_.MeasureTheory.integrable0n_indicator_iff {s : Set ℝ} (hs : MeasureTheory.isBounded s)
  (t : Set ℝ) (f : ℝ → ℝ) :
  Integrable0n (s.indicator f) t volume ↔ Integrable0n f (s ∩ t) volume := by
  refine and_congr ?_ ?_
  · rw [← Measure.restrict_restrict hs]
    constructor <=> intro ⟨g, hg, hg'⟩
    · use g
      refine ⟨hg, (ae_eq_restrict_iff_indicator_ae_eq hs).mpr ?_⟩
      grind [hg'.indicator (s := s), Set.indicator_indicator, Set.inter_self]
    · use s.indicator g
      exact ⟨StronglyMeasurable.indicator hg hs, (ae_eq_restrict_iff_indicator_ae_eq hs).mpr ?_⟩
  · have hInt : (fun r ↦ ||s.indicator f r||_e) = s.indicator (fun r ↦ ||f r||_e) := by
      ext r; by_cases hr : r ∈ s <=> simp [hr]
    simp only [HasFiniteIntegral, hInt, lintegral_indicator hs, Measure.restrict_restrict]
```

/-- The function ` $x \mapsto \|x\|^p$ ` is integrable on ` $\{x : \text{Space } d \mid 0 \leq \|x\| < b\}$ ` iff ` $0 < p$ `

```

/
lemma integrableOn_norm_rpow_ball_iff {d : ℕ} (hd : 0 < d) {b : ℝ} (hb : 0
  IntegrableOn (fun x : Space d ↦ ||x|| ^ p) (Metric.ball 0 b) ↔ 0 < d + p
have : Nontrivial (Space d) := (Nat.succ_pred_eq_of_pos hd) ▸ Space.instN
let f : Space d → ENNReal := (Metric.ball 0 b).indicator (fun x ↦ ||x|| ^
let g : ℝ → ℝ := (Set.Ioo 0 b).indicator (fun r ↦ r ^ p)
have hfg : f =ᵐ[volume] fun x ↦ ||g ||x||ᵉ := by
  apply ae_iff.mpr
  suffices {x | f x ≠ ||g ||x||ᵉ} = if p = 0 then {0} else ∅ by by_cases p
ext x
  by_cases ||x|| ∈ Set.Ioo 0 b
  · simp_all [f, g]
  · by_cases x = 0 <=> simp_all [f, g, Real.zero_rpow_eq_iff]
trans Integrable (fun x : Space d ↦ g ||x||)
· refine and_congr ?_ ?_
  · refine iff_of_true ?_ ?_
    repeat exact StronglyMeasurable.aestronglyMeasurable (by measurabilit
  · rw [HasFiniteIntegral, ← lintegral_indicator measurableSet_ball, lint
    rfl
trans IntegrableOn (fun r => r ^ (d - 1 + p)) (Set.Ioo 0 b)
· have hInter : Set.Ioo 0 b ∩ Set.Ioi 0 = Set.Ioo 0 b := by ext; grind
  simp_rw [integrable_fun_norm_addHaar, g, Space.finrank_eq_dim,
    npow_indicator_rpow_eq (Set.left_notMem_Ioo 0 b),
    _root_.MeasureTheory.integrableOn_indicator_iff measurableSet_Ioo, hI
  rw [intervalIntegral.integrableOn_Ioo_rpow_iff hb, neg_lt_iff_pos_add']
  ring_nf

/-- The function `x ↦ ||x||^p` is integrable on `{x : Space d | 0 < a ≤ ||x||}`
/
lemma integrableOn_norm_rpow_ball_compl_iff {d : ℕ} (hd : 0 < d) {a : ℝ} (h
  IntegrableOn (fun x : Space d ↦ ||x|| ^ p) (Metric.ball 0 a)ᶜ ↔ d + p < 0
have : Nontrivial (Space d) := (Nat.succ_pred_eq_of_pos hd) ▸ Space.instN
let f : Space d → ENNReal := (Metric.ball 0 a)ᶜ.indicator (fun x ↦ ||x|| ^
let g : ℝ → ℝ := (Set.Ici a).indicator (fun r ↦ r ^ p)
have hfg : f = fun x ↦ ||g ||x||ᵉ := by ext x; by_cases ||x|| ∈ Set.Ici a <=>
trans Integrable (fun x : Space d ↦ g ||x||)
· refine and_congr ?_ ?_
  · refine iff_of_true ?_ ?_
    repeat exact StronglyMeasurable.aestronglyMeasurable (by measurabilit
  · dsimp [HasFiniteIntegral]
    rw [← lintegral_indicator (by measurability), lintegral_congr_ae hfg.
trans IntegrableOn (fun r => r ^ (d - 1 + p)) (Set.Ici a)
· have hInter : Set.Ici a ∩ Set.Ioi 0 = Set.Ici a := by ext; grind
  simp_rw [integrable_fun_norm_addHaar, g, Space.finrank_eq_dim,
    npow_indicator_rpow_eq (Set.notMem_Ici.mpr ha),
    _root_.MeasureTheory.integrableOn_indicator_iff measurableSet_Ici, hI

```

```

rw [integrable0n_Ici_iff_integrable0n_Ioi, integrable0n_Ioi_rpow_iff ha, lt_neg_iff]
ring_nf

/-- The function  $x \mapsto \|x\|^p$  is integrable on the shell  $\{x : \text{Space } d \mid 0 < a \leq \|x\| \wedge \|x\| \leq b\}$ 
-/
lemma integrable0n_norm_rpow_shell {d : ℕ} {a : ℝ} (ha : 0 < a) (b p : ℝ) :
  Integrable0n (fun x : Space d ↦ \|x\| ^ p) ((Metric.ball 0 b) \ (Metric.ball 0 a)ᶜ)
  refine (StronglyMeasurable.aestronglyMeasurable (by measurability), ?_)
by_cases hab : a < b
· refine setLIntegral_lt_top_of_le_nreal ?_ ?_
  · exact measure_inter_ne_top_of_left_ne_top measure_ball_ne_top
  · use \|max (a ^ p) (b ^ p)\|+
    intro x hx
    simp only [Set.mem_inter_iff, Metric.mem_ball, dist_zero_right, Set.mem_compl_iff]
    rw [enorm_le_coe]
    refine abs_le_abs_of_nonneg (Real.rpow_nonneg (norm_nonneg x) p) ?_
    rw [le_sup_iff]
    by_cases hp : 0 ≤ p
    · exact 0r.inr <| Real.rpow_le_rpow (norm_nonneg x) (by linarith) hp
    · exact 0r.inl <| Real.rpow_le_rpow_of_nonpos ha (by linarith) (by linarith)
· suffices (Metric.ball 0 b) \ (Metric.ball 0 a)ᶜ = (∅ : Set (Space d)) by simp [t]
  ext x
  have : \|x\| < b → \|x\| < a := fun _ ↦ by linarith
  simp

/-- The function  $x \mapsto \|x\|^p$  is integrable on a bounded neighborhood of the origin
iff  $0 < d + p$ . -/
lemma integrable0n_norm_rpow_iff_of_isBounded_nhds {d : ℕ} (hd : 0 < d) {s : Set (Space d)}
  (hs : Bornology.IsBounded s) (hs' : s ∈ nhds 0) (p : ℝ) :
  Integrable0n (fun x : Space d ↦ \|x\| ^ p) s ↔ 0 < d + p := by
  obtain ⟨a, ha, ha'⟩ := Metric.eventually_nhds_iff_ball.mp hs'
  obtain ⟨b, hb, hb'⟩ := Bornology.IsBounded.subset_ball_lt hs 0 0
  constructor <=> intro h
  · exact (integrable0n_norm_rpow_ball_iff hd ha p).mp (Integrable0n.mono_set h ha')
  · exact Integrable0n.mono_set ((integrable0n_norm_rpow_ball_iff hd hb p).mpr h) hb

/-- The function  $x \mapsto \|x\|^p$  is integrable on a bounded subset with the origin in its interior
-/
lemma integrable0n_norm_rpow_of_isBounded_compl_nhds {d : ℕ} {s : Set (Space d)}
  (hs : Bornology.IsBounded s) (hs' : sᶜ ∈ nhds 0) (p : ℝ) :
  Integrable0n (fun x : Space d ↦ \|x\| ^ p) s := by
  obtain ⟨a, ha, ha'⟩ := Metric.eventually_nhds_iff_ball.mp hs'
  obtain ⟨b, hb, hb'⟩ := Bornology.IsBounded.subset_ball_lt hs 0 0
  have hsc : s ⊆ (Metric.ball 0 a)ᶜ := Set.subset_compl_comm.mp ha'
  exact Integrable0n.mono_set (integrable0n_norm_rpow_shell ha b p) (Set.subset_inter s (Metric.ball 0 a)ᶜ)

```

```

/-- The function  $x \mapsto \|x\|^p$  is integrable on a subset with the origin in it
    the decay at infinity is fast enough. -/
lemma integrableOn_norm_rpow_of_compl_nhds {d : ℕ} (hd : 0 < d) {s : Set (Space d)}
  (hs : sᶜ ∈ nhds 0) {p : ℝ} (h : d + p < 0) : IntegrableOn (fun x : Space d => ‖x‖^p) s
  obtain ⟨a, ha, ha'⟩ := Metric.eventually_nhds_iff_ball.mp hs
  have hs' : s ⊆ (Metric.ball 0 a)ᶜ := Set.subset_compl_comm.mp ha'
  exact IntegrableOn.mono_set ((integrableOn_norm_rpow_ball_compl_iff hd ha) s)
end Space

```

1.358 PhysLean/SpaceAndTime/Space/Integrals/RadialAngularMeasure

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import PhysLean.Mathematics.Distribution.Basic
public import PhysLean.SpaceAndTime.Space.Integrals.Basic
/-!

# The radial angular measure on Space

## i. Overview

The normal measure on `Space d` is  $r^{(d-1)} dr d\Omega$  in spherical coordinates
where  $d\Omega$  is the angular measure on the unit sphere. The radial angular measure
is the measure  $dr d\Omega$ , cancelling the radius contribution from the measure
coordinates.

This file is equivalent to `invPowMeasure`, which will slowly be deprecated

## ii. Key results

- `radialAngularMeasure`: The radial angular measure on `Space d`.

## iii. Table of contents

- A. The definition of the radial angular measure
  - A.1. Basic equalities
- B. Integrals with respect to radialAngularMeasure
- C. The radialAngularMeasure on balls

```


1.358.

PHYSLEAN/SPACEANDTIME/SPACE/INTEGRALS/RADIALANGULARMEASURE. LEAN

- D. Integrability conditions
- E. HasTemperateGrowth of measures
 - E.1. Integrability of powers
 - E.2. radialAngularMeasure has temperate growth

iv. References

-/

@[expose] public section
open SchwartzMap NNReal Real
noncomputable section

variable (α : Type) {E F F' : Type} [RCLike α] [NormedAddCommGroup E] [NormedAddCommGroup F']
[NormedAddCommGroup F']

variable [NormedSpace $\mathbb{R}$ E] [NormedSpace $\mathbb{R}$ F]

namespace Space

open MeasureTheory

/-!

A. The definition of the radial angular measure

-/

/-- The measure on `Space d` weighted by `1 / ||x|| ^ (d - 1)`. -

/

def radialAngularMeasure {d : $\mathbb{N}$ } : Measure (Space d) :=
volume.withDensity (fun x : Space d => ENNReal.ofReal (1 / ||x|| ^ (d - 1)))

/-!

A.1. Basic equalities

-/

lemma radialAngularMeasure_eq_volume_withDensity {d : $\mathbb{N}$ } : radialAngularMeasure =
volume.withDensity (fun x : Space d => ENNReal.ofReal (1 / ||x|| ^ (d - 1))) := by
rfl

@[simp]

lemma radialAngularMeasure_zero_eq_volume :
radialAngularMeasure (d := 0) = volume := by

```

    simp [radialAngularMeasure]

/-!

### A.2. SFinite property

-/

instance (d : ℕ) : SFinite (radialAngularMeasure (d := d)) := by
  dsimp [radialAngularMeasure]
  infer_instance

/-!

## B. Integrals with respect to radialAngularMeasure

-/

lemma integral_radialAngularMeasure {d : ℕ} (f : Space d → F) :
  ∫ x, f x ∂radialAngularMeasure = ∫ x, (1 / ||x|| ^ (d - 1)) • f x := by
  dsimp [radialAngularMeasure]
  erw [integral_withDensity_eq_integral_smul (by fun_prop)]
  congr
  funext x
  simp only [one_div]
  refine Eq.symm (Mathlib.Tactic.LinearCombination.smul_eq_const ?_ (f x))
  simp

lemma lintegral_radialMeasure {d : ℕ} (f : Space d → ENNReal) (hf : Measurable f) :
  ∫- x, f x ∂radialAngularMeasure = ∫- x, ENNReal.ofReal (1 / ||x|| ^ (d - 1)) • f x := by
  dsimp [radialAngularMeasure]
  rw [lintegral_withDensity_eq_lintegral_mul]
  simp only [one_div, Pi.mul_apply]
  · fun_prop
  · fun_prop

lemma lintegral_radialMeasure_eq_spherical_mul (d : ℕ) (f : Space d.succ → ENNReal)
  (hf : Measurable f) :
  ∫- x, f x ∂radialAngularMeasure = ∫- x, f (x.2.1 • x.1.1)
    ∂(volume (α := Space d.succ).toSphere.prod (Measure.volumeIoiPow 0))
  rw [lintegral_radialMeasure, lintegral_volume_eq_spherical_mul]
  apply lintegral_congr_ae
  filter_upwards with x
  have hx := x.2.2
  simp [Nat.succ_eq_add_one, -Subtype.coe_prop] at hx
  simp [norm_smul]

```

1.358.

PHYSLEAN/SPACEANDTIME/SPACE/INTEGRALS/RADIALANGULARMEASURE. LEAN

```

rw [abs_of_nonneg (le_of_lt x.2.2)]
trans ENNReal.ofReal (↑x.2 ^ d * (x.2.1 ^ d)-1) * f (x.2.1 • ↑x.1.1)
· rw [ENNReal.ofReal_mul]
  ring
  have h2 := x.2.2
  simp at h2
  positivity
trans ENNReal.ofReal 1 * f (x.2.1 • ↑x.1.1)
· congr
  field_simp
simp only [ENNReal.ofReal_one, Nat.succ_eq_add_one, one_mul]
fun_prop
fun_prop

```

/-!

C. The radialAngularMeasure on balls

-/

@[simp]

```

lemma radialAngularMeasure_closedBall (r : ℝ) :
  radialAngularMeasure (Metric.closedBall (0 : Space 3) r) = ENNReal.ofReal (4 * π)
  rw [← setLIntegral_one, ← MeasureTheory.lintegral_indicator measurableSet_closedBall
    lintegral_radialMeasure_eq_spherical_mul _ _
    ((measurable_indicator_const_iff 1).mpr measurableSet_closedBall)]
  have h1 (x : (Metric.sphere (0 : Space) 1) × ↑(Set.Ioi (0 : ℝ))) :
    (Metric.closedBall (0 : Space) r).indicator (fun x => (1 : ENNReal)) (x.2.1 •
      (Set.univ ×s {a | a.1 ≤ r})).indicator (fun x => 1) x :=
      Set.indicator_const_eq_indicator_const <| by
        simp [norm_smul]
        rw [abs_of_nonneg (le_of_lt x.2.2)]
  simp [h1]
  rw [MeasureTheory.lintegral_indicator <|
    MeasurableSet.prod MeasurableSet.univ (measurableSet_setOf.mpr (by fun_prop))]]
  simp [MeasureTheory.Measure.prod_prod, Measure.volumeIoiPow]
  rw [MeasureTheory.Measure.comap_apply _ Subtype.val_injective
    (fun s hs => MeasurableSet.subtype_image measurableSet_Ioi hs)
    _ (measurableSet_setOf.mpr (by fun_prop))]]
  trans 3 * ENNReal.ofReal (4 / 3 * π) * volume (α := ℝ) (Set.Ioc 0 r)
  · congr
    ext x
    simp only [Set.mem_image, Set.mem_setOf_eq, Subtype.exists, Set.mem_Ioi, exists_
      exists_prop, exists_eq_right_right, Set.mem_Ioc]
    grind
  simp only [volume_Ioc, sub_zero]

```

```

trans ENNReal.ofReal (3 * ((4 / 3 * π))) * ENNReal.ofReal r
· simp [ENNReal.ofReal_mul]
field_simp
rw [← ENNReal.ofReal_mul (by positivity)]

lemma radialAngularMeasure_real_closedBall (r : ℝ) (hr : 0 < r) :
  radialAngularMeasure.real (Metric.closedBall (0 : Space 3) r) = 4 * π *
  change (radialAngularMeasure (Metric.closedBall (0 : Space 3) r)).toReal
  simp only [radialAngularMeasure_closedBall, ENNReal.toReal_ofReal_eq_iff]
  positivity

/-!

## D. Integrability conditions

-/

lemma integrable_radialAngularMeasure_iff {d : ℕ} {f : Space d → F} :
  Integrable f (radialAngularMeasure (d := d)) ↔
  Integrable (fun x => (1 / ||x|| ^ (d - 1)) • f x) volume := by
  dsimp [radialAngularMeasure]
  erw [integrable_withDensity_iff_integrable_smul₀ (by fun_prop)]
  simp only [one_div]
  refine integrable_congr ?_
  filter_upwards with x
  rw [Real.toNNReal_of_nonneg (by positivity), NNReal.smul_def, coe_mk]

omit [NormedSpace ℝ F] in
lemma integrable_radialAngularMeasure_of_spherical {d : ℕ} (f : Space d.succ
  (hae : StronglyMeasurable f)
  (hf : Integrable (fun x => f (x.2.1 • x.1.1))
  (volume (α := Space d.succ).toSphere.prod (Measure.volumeIoiPow 0))) :
  Integrable f radialAngularMeasure := by
  refine ⟨StronglyMeasurable.astronglyMeasurable hae, ?_⟩
  rw [hasFiniteIntegral_iff_norm, lintegral_radialMeasure_eq_spherical_mul
  (by simp using StronglyMeasurable.enorm hae), ← hasFiniteIntegral_iff_
  exact hf.2

/-!

## E. HasTemperateGrowth of measures

-/

/-!

```

1.358.

PHYSLEAN/SPACEANDTIME/SPACE/INTEGRALS/RADIALANGULARMEASURE.LEAN

E.1. Integrability of powers

-/

```
private lemma integrable_neg_pow_on_ioi (n : ℕ) :
  IntegrableOn (fun x : ℝ => (|((1 : ℝ) + x) ^ (- (n + 2) : ℝ)|)) (Set.Ioi 0) := b
  rw [MeasureTheory.integrableOn_iff_comap_subtypeVal]
  rw [← MeasureTheory.integrable_smul_measure (c := n + 1)]
  apply MeasureTheory.integrable_of_integral_eq_one
  trans (n + 1) * ∫ (x : ℝ) in Set.Ioi 0, ((1 + x) ^ (- (n + 2) : ℝ)) ∂volume
  · rw [← MeasureTheory.integral_subtype_comap measurableSet_Ioi]
    simp only [neg_add_rev, Function.comp_apply, integral_smul_measure, smul_eq_mul]
    congr
    funext x
    simp only [abs_eq_self]
    apply Real.rpow_nonneg
    grind
  have h0 (x : ℝ) (hx : x ∈ Set.Ioi 0) : ((1 : ℝ) + ↑x) ^ (- (n + 2) : ℝ) =
    ((1 + x) ^ ((n + 2)))-1 := by
    rw [← Real.rpow_natCast, ← Real.inv_rpow, ← Real.rpow_neg_one, ← Real.rpow_mul]
    · simp [neg_add_rev, Nat.cast_add, Nat.cast_ofNat]
    · rw [Set.mem_Ioi] at hx
      positivity
    · grind
  trans (n + 1) * ∫ (x : ℝ) in Set.Ioi 0, ((1 + x) ^ (n + 2))-1 ∂volume
  · congr 1
    refine setIntegral_congr_ae ?_ ?_
    · simp
    filter_upwards with x hx
    rw [h0 x hx]
  trans (n + 1) * ∫ (x : ℝ) in Set.Ioi 1, (x ^ (n + 2))-1 ∂volume
  · congr 1
    let f := fun x : ℝ => (x ^ (n + 2))-1
    change ∫ (x : ℝ) in Set.Ioi 0, f (1 + x) ∂volume = ∫ (x : ℝ) in Set.Ioi 1, f x ∂
    let fa := fun x : ℝ => 1 + x
    change ∫ (x : ℝ) in Set.Ioi 0, f (fa x) ∂volume = _
    rw [← MeasureTheory.MeasurePreserving.setIntegral_image_emb (v := volume)]
    · simp [fa]
    · simpa [fa] using measurePreserving_add_left volume 1
    · simpa [fa] using measurableEmbedding_addLeft 1
  · trans (n + 1) * ∫ (x : ℝ) in Set.Ioi 1, (x ^ (- (n + 2) : ℝ)) ∂volume
    · congr 1
      refine setIntegral_congr_ae ?_ ?_
      · simp
      filter_upwards with x hx
      have hx : 1 < x := hx
      rw [← Real.rpow_natCast, ← Real.inv_rpow, ← Real.rpow_neg_one, ← Real.rpow_mul]
```

```

      · simp
      · positivity
      · positivity
    rw [integral_Ioi_rpow_of_lt (by linarith) (by linarith)]
    field_simp
    have h0 : (-2 + -(n : ℝ) + 1) ≠ 0 := by
      by_contra h
      have h1 : (1 : ℝ) - 0 = 2 + n := by grind
      simp at h1
      linarith
    simp only [neg_add_rev, Real.one_rpow, mul_one]
    grind
    · simp
    · simp
    · simp

lemma radialAngularMeasure_integrable_pow_neg_two {d : ℕ} :
  Integrable (fun x : Space d => (1 + ||x||) ^ (- (d + 1) : ℝ))
    radialAngularMeasure := by
  match d with
  | 0 => simp
  | dm1 + 1 =>
    apply integrable_radialAngularMeasure_of_spherical _ (by fun_prop)
    simp [norm_smul]
    rw [MeasureTheory.integrable_prod_iff]
    rotate_left
    · apply AEMeasurable.aestronglyMeasurable
      fun_prop
    refine {?_, by simp}
    filter_upwards with x
    simp [Measure.volumeIoiPow]
    let f := fun x : ℝ => |(1 + x) ^ (- (dm1 + 2) : ℝ)|
    have h0 : (fun (y : ↑(Set.Ioi 0)) => (1 + |y.1|) ^ (-1 + (-
1 + -↑dm1) : ℝ)) =
      f ∘ Subtype.val := by
      funext x
      rcases x with ⟨x, hx⟩
      simp at hx
      simp [f]
      ring_nf
      rw [abs_of_nonneg (by positivity), abs_of_nonneg (by positivity)]
    rw [h0]
    change Integrable (f ∘ Subtype.val)
    rw [← MeasureTheory.integrableOn_iff_comap_subtypeVal measurableSet_Ioi]
    exact integrable_neg_pow_on_ioi dm1

```

```

/-!
### E.2. radialAngularMeasure has temperate growth
-/

instance (d : ℕ) : Measure.HasTemperateGrowth (radialAngularMeasure (d := d)) where
  exists_integrable := by
    use d + 1
    simpa using radialAngularMeasure_integrable_pow_neg_two (d := d)

end Space

```

1.359 PhysLean/SpaceAndTime/Space/IsDistBounded.lean

```

module

public import PhysLean.SpaceAndTime.Space.Integrals.RadialAngularMeasure
public import PhysLean.SpaceAndTime.Time.Derivatives
public import Mathlib.Tactic.Cases

@[expose] public section

```

1.360 PhysLean/SpaceAndTime/Space/LengthUnit.lean

```

module

public import Mathlib.Geometry.Manifold.Diffeomorph
public import PhysLean.SpaceAndTime.Time.Basic
@[expose] public section

```

1.361 PhysLean/SpaceAndTime/Space/Module.lean

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import PhysLean.SpaceAndTime.Space.Basic

```

```
public import Mathlib.Geometry.Manifold.Diffeomorph
public import Mathlib.Analysis.Distribution.TemperateGrowth
/-!
```

```
# The structure of a module on Space
```

The scope of this module is to define on ``Space d`` the structure of a ``Modu`` (aka vector space), a ``Norm`` and an ``InnerProductSpace``, and give properties

These instances require certain non-canonical choices to be made, for example of a zero and for a basis, a choice of orientation.

```
-/
```

```
@[expose] public section
```

```
namespace Space
```

```
/-!
```

```
## A.1. Instance of an additive commutative monoid
```

```
-/
```

```
instance {d} : Add (Space d) where
  add p q := ⟨fun i => p.val i + q.val i⟩
```

```
@[simp]
lemma add_val {d : ℕ} (x y : Space d) :
  (x + y).val = x.val + y.val := rfl
```

```
@[simp]
lemma add_apply {d : ℕ} (x y : Space d) (i : Fin d) :
  (x + y) i = x i + y i := by
  simp [add_val]
```

```
instance {d} : Zero (Space d) where
  zero := ⟨fun _ => 0⟩
```

```
@[simp]
lemma zero_val {d : ℕ} : (0 : Space d).val = fun _ => 0 := rfl
```

```
@[simp]
lemma zero_apply {d : ℕ} (i : Fin d) :
  (0 : Space d) i = 0 := by
  simp [zero_val]
```



```

instance {d} : AddCommMonoid (Space d) where
  add_assoc a b c := by
    apply eq_of_val
    simp only [add_val]
    ring
  zero_add a := by
    apply eq_of_val
    simp only [zero_val, add_val, add_eq_right]
    rfl
  add_zero a := by
    apply eq_of_val
    simp only [zero_val, add_val, add_eq_left]
    rfl
  add_comm a b := by
    apply eq_of_val
    simp only [add_val]
    ring
  nsmul n a := ⟨fun i => n • a.val i⟩

@[simp]
lemma nsmul_val {d : ℕ} (n : ℕ) (a : Space d) :
  (n • a).val = fun i => n • a.val i := rfl

@[simp]
lemma nsmul_apply {d : ℕ} (n : ℕ) (a : Space d) (i : Fin d) :
  (n • a) i = n • (a i) := by rfl

lemma eq_vadd_zero {d} (s : Space d) :
  ∃ v : EuclideanSpace ℝ (Fin d), s = v +v (0 : Space d) := by
  obtain ⟨v, h⟩ := vadd_transitive 0 s
  use v
  rw [h]

@[simp]
lemma add_vadd_zero {d} (v1 v2 : EuclideanSpace ℝ (Fin d)) :
  (v1 +v (0 : Space d)) + (v2 +v (0 : Space d)) = (v1 + v2) +v (0 : Space d) := by
  ext i
  simp

/-!

## A.2. Instance of a module over `ℝ`

-/

```

```

instance {d} : SMul ℝ (Space d) where
  smul c p := ⟨fun i => c * p.val i⟩

@[simp]
lemma smul_val {d : ℕ} (c : ℝ) (p : Space d) :
  (c • p).val = fun i => c * p.val i := rfl

@[simp]
lemma smul_apply {d : ℕ} (c : ℝ) (p : Space d) (i : Fin d) :
  (c • p) i = c * (p i) := by rfl

@[simp]
lemma smul_vadd_zero {d} (k : ℝ) (v : EuclideanSpace ℝ (Fin d)) :
  k • (v +v (0 : Space d)) = (k • v) +v (0 : Space d) := by
  ext i
  simp

instance {d} : Module ℝ (Space d) where
  one_smul x := by
    ext i
    simp
  mul_smul a b x := by
    ext i
    simp only [smul_apply]
    ring
  smul_add a x y := by
    ext i
    simp only [smul_apply, add_apply]
    ring
  smul_zero a := by
    ext i
    simp
  add_smul a b x := by
    ext i
    simp only [smul_apply, add_apply]
    ring
  zero_smul x := by
    ext i
    simp

/-!

## A.3. Instance of an inner product space

```

```

-/

noncomputable instance {d} : Norm (Space d) where
  norm p :=  $\sqrt{\sum i, (p\ i)^2}$ 

lemma norm_eq {d} (p : Space d) :  $\|p\| = \sqrt{\sum i, (p\ i)^2}$  := by
  rfl

@[simp]
lemma abs_eval_le_norm {d} (p : Space d) (i : Fin d) :
  |p i| ≤  $\|p\|$  := by
  simp [norm_eq]
  refine Real.abs_le_sqrt ?_
  trans  $\sum j \in \{i\}, (p\ j)^2$ 
  · simp
  refine Finset.sum_le_univ_sum_of_nonneg (fun i => by positivity)

lemma norm_sq_eq {d} (p : Space d) :
   $\|p\|^2 = \sum i, (p\ i)^2$  := by
  rw [norm_eq]
  refine Real.sq_sqrt ?_
  positivity

lemma point_dim_zero_eq (p : Space 0) : p = 0 := by
  ext i
  fin_cases i

@[simp]
lemma norm_vadd_zero {d} (v : EuclideanSpace ℝ (Fin d)) :
   $\|v +_v (0 : Space\ d)\| = \|v\|$  := by
  simp [norm_eq, PiLp.norm_eq_of_L2]

instance : Neg (Space d) where
  neg p := ⟨fun i => - (p.val i)⟩

@[simp]
lemma neg_val {d : ℕ} (p : Space d) :
  (-p).val = fun i => - (p.val i) := rfl

@[simp]
lemma neg_apply {d : ℕ} (p : Space d) (i : Fin d) :
  (-p) i = - (p i) := by rfl

noncomputable instance {d} : AddCommGroup (Space d) where
  zsmul z p := ⟨fun i => z * p.val i⟩
  neg_add_cancel p := by

```

```

    ext i
    simp
zsmul_zero' p := by
  ext i
  simp
zsmul_succ' n p := by
  ext i
  simp only [Nat.succ_eq_add_one, Nat.cast_add, Nat.cast_one, Int.cast_add,
    Int.cast_one, add_apply]
  ring
zsmul_neg' n p := by
  ext i
  simp only [Int.cast_negSucc, Nat.cast_add, Nat.cast_one, neg_add_rev, N
    Int.cast_add, Int.cast_natCast, Int.cast_one, neg_apply]
  ring

@[simp]
lemma sub_apply {d} (p q : Space d) (i : Fin d) :
  (p - q) i = p i - q i := by
  simp [sub_eq_add_neg, neg_apply, add_apply]

@[simp]
lemma sub_val {d} (p q : Space d) :
  (p - q).val = fun i => p.val i - q.val i := by rfl

@[simp]
lemma vadd_zero_sub_vadd_zero {d} (v1 v2 : EuclideanSpace ℝ (Fin d)) :
  (v1 +v (0 : Space d)) - (v2 +v (0 : Space d)) = (v1 - v2) +v (0 : Space d)
  ext i
  simp [sub_apply, vadd_apply]

noncomputable instance {d} : SeminormedAddCommGroup (Space d) where

@[simp]
lemma dist_eq_norm {d} (p q : Space d) :
  dist p q = ||p - q|| := rfl

noncomputable instance : NormedAddCommGroup (Space d) where

instance {d} : Inner ℝ (Space d) where
  inner p q :=  $\sum i, p\ i * q\ i$ 

@[simp]
lemma inner_vadd_zero {d} (v1 v2 : EuclideanSpace ℝ (Fin d)) :
  inner ℝ (v1 +v (0 : Space d)) (v2 +v (0 : Space d)) = Inner.inner ℝ v1

```

```

    simp [inner, vadd_apply]
    apply Finset.sum_congr rfl
    intro i hi
    ring

lemma inner_apply {d} (p q : Space d) :
  inner ℝ p q =  $\sum i, p\ i * q\ i$  := by rfl

instance {d} : InnerProductSpace ℝ (Space d) where
  norm_smul_le a x := by
    obtain ⟨v, rfl⟩ := eq_vadd_zero x
    simp only [smul_vadd_zero, norm_vadd_zero, Real.norm_eq_abs]
    exact norm_smul_le a v
  norm_sq_eq_re_inner x := by
    obtain ⟨v, rfl⟩ := eq_vadd_zero x
    simp
  conj_inner_symm x y := by
    simp [inner_apply]
    congr
    funext i
    ring
  add_left x y z := by
    obtain ⟨v1, rfl⟩ := eq_vadd_zero x
    obtain ⟨v2, rfl⟩ := eq_vadd_zero y
    obtain ⟨v3, rfl⟩ := eq_vadd_zero z
    simp only [add_vadd_zero, inner_vadd_zero]
    exact InnerProductSpace.add_left v1 v2 v3
  smul_left x y a := by
    obtain ⟨v1, rfl⟩ := eq_vadd_zero x
    obtain ⟨v2, rfl⟩ := eq_vadd_zero y
    simp only [smul_vadd_zero, inner_vadd_zero, conj_trivial]
    exact InnerProductSpace.smul_left v1 v2 a

/-!

## A.4. Instance of a measurable space

-/

instance {d : ℕ} : MeasurableSpace (Space d) := borel (Space d)

instance {d : ℕ} : BorelSpace (Space d) where
  measurable_eq := by rfl

TODO "HB6YZ" "In the above documentation describe what an instance is, and why
it is useful to have instances for `Space d`."

```

```

TODO "HB6WN" "After TODO 'HB6VC', give `Space d` the necessary instances
      using `inferInstanceAs`."
/-!

## The norm on `Space`

-/

/-!

## Inner product

-/

lemma inner_eq_sum {d} (p q : Space d) :
  inner ℝ p q = ∑ i, p i * q i := by
  simp [inner]

@[simp]
lemma sum_apply {ι : Type} [Fintype ι] (f : ι → Space d) (i : Fin d) :
  (∑ x, f x) i = ∑ x, f x i := by
  let P (ι : Type) [Fintype ι] : Prop := ∀ (f : ι → Space d) (i : Fin d), (
  have h1 : P ι := by
    apply Fintype.induction_empty_option
    · intro α β h e h f i
      rw [← @e.sum_comp _, h, ← @e.sum_comp _]
    · simp [P]
    · intro α _ h f i
      simp only [Fintype.sum_option, add_apply, add_right_inj]
      rw [h]
  exact h1 f i

/-!

## Basis

-/

TODO "HB6Z4" "In the above documentation describe the notion of a basis
in Lean."

/-- The standard basis of Space based on `Fin d`. -/
noncomputable def basis {d} : OrthonormalBasis (Fin d) ℝ (Space d) where
  repr := {
    toFun p := WithLp.toLp 2 fun i => p i

```

```

    invFun := fun v => ⟨v⟩
    left_inv := by
      intro p
      rfl
    right_inv := by
      intro v
      rfl
    map_add' := by
      intro v1 v2
      rfl
    map_smul' := by
      intro c v
      rfl
    norm_map' := by
      intro x
      simp [norm_eq, PiLp.norm_eq_of_L2]}

lemma apply_eq_basis_repr_apply {d} (p : Space d) (i : Fin d) :
  p i = basis.repr p i := by
  simp [basis]

@[simp]
lemma basis_repr_apply {d} (p : Space d) (i : Fin d) :
  basis.repr p i = p i := by
  simp [apply_eq_basis_repr_apply]

@[simp]
lemma basis_repr_symm_apply {d} (v : EuclideanSpace ℝ (Fin d)) (i : Fin d) :
  basis.repr.symm v i = v i := by rfl

lemma basis_apply {d} (i j : Fin d) :
  basis i j = if i = j then 1 else 0 := by
  simp [apply_eq_basis_repr_apply]
  congr 1
  exact Lean.Grind.eq_congr' rfl rfl

@[simp]
lemma basis_self {d} (i : Fin d) : basis i i = 1 := by
  simp [basis_apply]

@[simp high]
lemma inner_basis {d} (p : Space d) (i : Fin d) :
  inner ℝ p (basis i) = p i := by
  simp [inner_eq_sum, basis_apply]

@[simp high]

```

```

lemma basis_inner {d} (i : Fin d) (p : Space d) :
  inner ℝ (basis i) p = p i := by
  simp [inner_eq_sum, basis_apply]

open InnerProductSpace

lemma basis_repr_inner_eq {d} (p : Space d) (v : EuclideanSpace ℝ (Fin d)) :
  (basis_repr p, v) = (p, basis_repr.symm v) := by
  exact LinearIsometryEquiv.inner_map_eq_flip basis_repr p v

instance {d : ℕ} : FiniteDimensional ℝ (Space d) :=
  Module.Basis.finiteDimensional_of_finite (h := basis.toBasis)

@[simp]
lemma finrank_eq_dim {d : ℕ} : Module.finrank ℝ (Space d) = d := by
  simp [Module.finrank_eq_nat_card_basis (basis.toBasis)]

@[simp]
lemma rank_eq_dim {d : ℕ} : Module.rank ℝ (Space d) = d := by
  simp [rank_eq_card_basis (basis.toBasis)]

@[simp]
lemma fderiv_basis_repr {d} (p : Space d) :
  fderiv ℝ basis_repr p = basis_repr.toContinuousLinearMap := by
  change fderiv ℝ basis_repr.toContinuousLinearMap p = _
  rw [ContinuousLinearMap.fderiv]

@[simp]
lemma fderiv_basis_repr_symm {d} (v : EuclideanSpace ℝ (Fin d)) :
  fderiv ℝ basis_repr.symm v = basis_repr.symm.toContinuousLinearMap := by
  change fderiv ℝ basis_repr.symm.toContinuousLinearMap v = _
  rw [ContinuousLinearMap.fderiv]

/-!

## Coordinates

-/

/-- The standard coordinate functions of Space based on `Fin d`.

The notation ` $\mu$  p` can be used for this. -/
noncomputable def coord ( $\mu$  : Fin d) (p : Space d) : ℝ :=
  inner ℝ p (basis  $\mu$ )

lemma coord_apply ( $\mu$  : Fin d) (p : Space d) :
```



```

    coord  $\mu$  p = p  $\mu$  := by
    simp [coord]

/-- The standard coordinate functions of Space based on `Fin d`, as a continuous lin
/
noncomputable def coordCLM {d} ( $\mu$  : Fin d) : Space d  $\rightarrow$  L[ $\mathbb{R}$ ]  $\mathbb{R}$  where
  toFun := coord  $\mu$ 
  map_add' := fun p q => by
    simp [coord, basis, inner_add_left]
  map_smul' := fun c p => by
    simp [coord, basis, inner_smul_left]
  cont := by
    unfold coord
    fun_prop

open ContDiff

@[fun_prop]
lemma coord_contDiff {i} : ContDiff  $\mathbb{R} \infty$  (fun x : Space d => x.coord i) := by
  change ContDiff  $\mathbb{R} \infty$  (coordCLM i)
  fun_prop

lemma coordCLM_apply ( $\mu$  : Fin d) (p : Space d) :
  coordCLM  $\mu$  p = coord  $\mu$  p := by
  rfl

@[inherit_doc coord]
scoped notation "[]" => coord

@[fun_prop]
lemma eval_continuous {d} (i : Fin d) :
  Continuous (fun p : Space d => p i) := by
  convert (coordCLM i).continuous
  simp [coordCLM_apply, coord]

@[fun_prop]
lemma eval_differentiable {d} (i : Fin d) :
  Differentiable  $\mathbb{R}$  (fun p : Space d => p i) := by
  convert (coordCLM i).differentiable
  simp [coordCLM_apply, coord]

@[fun_prop]
lemma eval_contDiff {d n} (i : Fin d) :
  ContDiff  $\mathbb{R} n$  (fun p : Space d => p i) := by
  convert (coordCLM i).contDiff
  simp [coordCLM_apply, coord]

```

```

/-- The continuous linear equivalence between `Space d` and the correspondi
/
def equivPi (d : ℕ) :
  Space d ≈L[ℝ]  $\prod$  (_ : Fin d), ℝ := LinearEquiv.toContinuousLinearEquiv <
  {
    toFun := fun p i => p i
    map_add' p1 p2 := by funext i; simp
    map_smul' p r := by funext i; simp
    invFun := fun f => ⟨f⟩
  }

/-!

## Basic differentiability conditions

-/

@[fun_prop]
lemma mk_continuous {d : ℕ} :
  Continuous (fun (f : Fin d → ℝ) => ⟨f⟩ : Space d) := (equivPi d).symm

@[fun_prop]
lemma mk_differentiable {d : ℕ} :
  Differentiable ℝ (fun (f : Fin d → ℝ) => ⟨f⟩ : Space d) := (equivPi d)

@[fun_prop]
lemma mk_contDiff {d : ℕ} {n : WithTop ℕ∞} :
  ContDiff ℝ n (fun (f : Fin d → ℝ) => ⟨f⟩ : Space d) := (equivPi d).sy

@[simp]
lemma fderiv_mk {d : ℕ} (f : Fin d → ℝ) :
  fderiv ℝ Space.mk f = (equivPi d).symm := by
  change fderiv ℝ (equivPi d).symm f = _
  rw [ContinuousLinearEquiv.fderiv]

@[simp]
lemma fderiv_val {d : ℕ} (p : Space d) :
  fderiv ℝ Space.val p = (equivPi d) := by
  change fderiv ℝ (equivPi d) p = _
  rw [ContinuousLinearEquiv.fderiv]

@[fun_prop]
lemma contDiff0n_vadd (s : Space d) :
  ContDiff0n ℝ  $\omega$  (fun (v : EuclideanSpace ℝ (Fin d)) => v +v s) Set.univ
  rw [contDiff0n_univ]

```

```

    refine fun_comp ?_ ?_
    · exact mk_contDiff (n := ω)
    · fun_prop

@[fun_prop]
lemma vadd_differentiable {d} (s : Space d) :
  Differentiable ℝ (fun (v : EuclideanSpace ℝ (Fin d)) => v +v s) :=
  mk_differentiable.comp <| by fun_prop

@[fun_prop]
lemma contDiffOn_vsub (s1 : Space d) :
  ContDiffOn ℝ ω (fun (s : Space d) => s -v s1) Set.univ :=
  contDiffOn_univ.mpr <| fun_comp (PiLp.contDiff_toLp) (by fun_prop)

@[fun_prop]
lemma vsub_differentiable {d} (s1 : Space d) :
  Differentiable ℝ (fun (s : Space d) => s -v s1) :=
  (PiLp.contDiff_toLp.differentiable (NeZero.ne' 2).symm).comp (by fun_prop)

lemma fderiv_space_components {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (μ : Fin d) (f : M → Space d) (hf : Differentiable ℝ f) (m dm : M):
  fderiv ℝ f m dm μ = fderiv ℝ (fun m' => f m' μ) m dm := by
  trans fderiv ℝ (Space.coordCLM μ ∘ fun m' => f m') m dm
  · rw [fderiv_comp _ (by fun_prop) (by fun_prop), ContinuousLinearMap.fderiv,
    ContinuousLinearMap.coe_comp', Function.comp_apply]
  · simp [coordCLM, coord_apply]
  · congr
  · ext i
  · simp [coordCLM, coord_apply]

/-!

## Directions

-/

/-- Notion of direction where `unit` returns a unit vector in the direction specifie
/
structure Direction (d : ℕ := 3) where
  /-- Unit vector specifying the direction. -/
  unit : Space d
  norm : ||unit|| = 1

/-- Direction of a `Space` value with respect to the origin. -
/
noncomputable def toDirection {d : ℕ} (x : Space d) (h : x ≠ 0) : Direction d where

```

```

    unit := (||x||-1) • x
    norm := norm_smul_inv_norm h

@[simp]
lemma direction_unit_sq_sum {d} (s : Direction d) :
  ∑ i : Fin d, (s.unit i) ^ 2 = 1 := by
  trans (||s.unit||) ^ 2
  · rw [norm_sq_eq]
  · rw [s.norm]
  simp

/-!

## One equiv

-/

/-- The linear isometric equivalence between `Space 1` and `ℝ`. -
/
noncomputable def oneEquiv : Space 1 ≈1[ℝ] ℝ where
  toFun x := x 0
  invFun x := ⟨fun _ => x⟩
  left_inv x := by
    ext i; fin_cases i; simp
  right_inv x := by simp
  map_add' x y := by rfl
  map_smul' c x := by rfl
  norm_map' x := by
    simp only [Fin.isValue, LinearEquiv.coe_mk, LinearMap.coe_mk, AddHom.coe_mk]
    rw [norm_eq]
    simp only [Fin.isValue, Finset.univ_unique, Fin.default_eq_zero, Finset.prod_eq_zero_iff]
    exact Eq.symm (Real.sqrt_sq_eq_abs (x 0))

lemma oneEquiv_coe :
  (oneEquiv : Space 1 → ℝ) = fun x => x 0 := by
  rfl

lemma oneEquiv_symm_coe :
  (oneEquiv.symm : ℝ → Space 1) = (fun x => ⟨fun _ => x⟩) := by
  rfl

lemma oneEquiv_symm_apply (x : ℝ) (i : Fin 1) :
  oneEquiv.symm x i = x := by
  fin_cases i
  rfl

```

```

lemma oneEquiv_continuous :
  Continuous (oneEquiv : Space 1 → ℝ) := by
  simp [oneEquiv_coe]
  fun_prop

lemma oneEquiv_symm_continuous :
  Continuous (oneEquiv.symm : ℝ → Space 1) := by
  simp [oneEquiv_symm_coe]
  fun_prop

/-- The continuous linear equivalence between `Space 1` and `ℝ`. -
/
noncomputable def oneEquivCLE : Space 1 ≈L[ℝ] ℝ where
  toLinearEquiv := oneEquiv
  continuous_toFun := by
    simp only [AddHom.toFun_eq_coe, LinearMap.coe_toAddHom, LinearEquiv.coe_coe]
    erw [oneEquiv_coe]
    fun_prop
  continuous_invFun := by
    simp only [LinearEquiv.invFun_eq_symm]
    erw [oneEquiv_symm_coe]
    fun_prop

open MeasureTheory
lemma oneEquiv_measurableEmbedding : MeasurableEmbedding oneEquiv where
  injective := oneEquiv.injective
  measurable := by fun_prop
  measurableSet_image' := by
    intro s hs
    change MeasurableSet (↑oneEquivCLE '' s)
    rw [ContinuousLinearEquiv.image_eq_preimage_symm]
    exact oneEquiv.symm.continuous.measurable hs

lemma oneEquiv_symm_measurableEmbedding : MeasurableEmbedding oneEquiv.symm where
  injective := oneEquiv.symm.injective
  measurable := by fun_prop
  measurableSet_image' := by
    intro s hs
    change MeasurableSet (↑oneEquivCLE.symm '' s)
    rw [ContinuousLinearEquiv.image_eq_preimage_symm]
    exact oneEquiv.continuous.measurable hs

lemma oneEquiv_measurePreserving : MeasurePreserving oneEquiv volume volume :=
  LinearIsometryEquiv.measurePreserving oneEquiv

lemma oneEquiv_symm_measurePreserving : MeasurePreserving oneEquiv.symm volume volum

```

```

    exact LinearIsometryEquiv.measurePreserving oneEquiv.symm

/-!

## Relation to tangent space

-/

open Manifold in
/-- A diffeomorphism between the two different manifold structures on `Space d`
    that equivalent to `manifoldStructure d` and that equivalent to ` $\mathbb{R}$ , Space d`
    /
noncomputable def modelDiffeo {d} :
  Diffeomorph (manifoldStructure d)  $\mathbb{R}$  (Space d) (Space d) (Space d)  $\top$  w
  toFun p := p
  invFun p := p
  left_inv _ := rfl
  right_inv _ := rfl
  contMDiff_toFun := by
    refine contMDiff_iff.mpr <continuous_id', fun x y => ?_>
    simp [manifoldStructure]
    fun_prop
  contMDiff_invFun := by
    refine contMDiff_iff.mpr <continuous_id', fun x y => ?_>
    simp [manifoldStructure]
    fun_prop

@[simp]
lemma modelDiffeo_apply (p : Space d) :
  modelDiffeo p = p := rfl

open Manifold in
/-- The derivative of `modelDiffeo` provides an equivalence between
    `Space d` and `EuclideanSpace  $\mathbb{R}$  (Fin d)`. This equivalences takes the basis
    of `EuclideanSpace  $\mathbb{R}$  (Fin d)` to the basis of `Space d`, and vice versa.
    /
lemma basis_eq_mfderiv_modelDiffeo_single (d :  $\mathbb{N}$ ) ( $\mu$  : Fin d) (x : Space d) :
  basis  $\mu$  = mfderiv (manifoldStructure d)  $\mathbb{R}$  (Space d) (modelDiffeo (d :  $\mathbb{N}$ )
    (EuclideanSpace.single  $\mu$  1)) := by
  simp [mfderiv]
  rw [if_pos (modelDiffeo.mdifferentiable (WithTop.top_ne_zero)).mdifferentiable]
  change _ = fderiv  $\mathbb{R}$  (manifoldStructure d).symm (manifoldStructure d x) (E
  simp [manifoldStructure]
  ext i
  rw [fderiv_space_components _ _ (by fun_prop)]
  simp only [vadd_apply, fderiv_add_const]

```

```

change _ = fderiv ℝ (EuclideanSpace.proj i) (x -v Classical.choice _) (EuclideanSp
simp [basis_apply]
congr 1
exact Eq.propIntro (fun a => Eq.symm a) fun a => (Eq.symm a)

/-!

## Properties of vadd with module structure

-/

lemma norm_vadd_le_add {d} (v : EuclideanSpace ℝ (Fin d)) (s : Space d) :
  ||v +v s|| ≤ ||v|| + ||s|| := by
  trans ||s - (-v +v (0 : Space d))||
  · apply le_of_eq
    congr
    ext i
    simp only [vadd_apply, sub_apply, PiLp.neg_apply, zero_apply, add_zero, sub_neg_
    ring
  · apply (norm_sub_le _ _).trans <| le_of_eq _
    simp only [norm_vadd_zero, norm_neg]
    ring

@[fun_prop]
lemma differentiable_vadd {d} (v : EuclideanSpace ℝ (Fin d)) :
  Differentiable ℝ (fun (s : Space d) => v +v s) :=
  mk_differentiable.comp <| by fun_prop

@[simp]
lemma fderiv_vadd {d} (v : EuclideanSpace ℝ (Fin d)) :
  fderiv ℝ (fun s => v +v s) = fun (_ : Space d) => ContinuousLinearMap.id ℝ _ :=
  ext s ds i
  change fderiv ℝ (fun s => v +v s) s ds i = _
  rw [fderiv_space_components]
  simp only [vadd_apply, fderiv_const_add, ContinuousLinearMap.coe_id', id_eq]
  trans fderiv ℝ (coordCLM i) s ds
  · congr
    ext j
    simp [coordCLM, coord_apply]
  · rw [ContinuousLinearMap.fderiv]
    simp [coordCLM, coord_apply]
  · fun_prop

@[fun_prop]
lemma vadd_hasTemperateGrowth {d} (v : EuclideanSpace ℝ (Fin d)) :
```

```

    Function.HasTemperateGrowth (fun s : Space d => v +v s) := by
    apply Function.HasTemperateGrowth.of_fderiv (k := 1) (C := 1 + ||v||)
    · rw [fderiv_vadd]
      simp
    · fun_prop
    · intro x
      simp only [pow_one]
      apply (norm_vadd_le_add _ _).trans
      have : 0 ≤ ||v|| := by positivity
      have : 0 ≤ ||x|| := by positivity
      nlinarith
end Space

```

1.362 PhysLean/SpaceAndTime/Space/Norm.lean

```

module

public import PhysLean.SpaceAndTime.Space.Derivatives.Div
public import Mathlib.Analysis.InnerProductSpace.NormPow
public import Mathlib.Analysis.Calculus.FDeriv.Norm

@[expose] public section

```

1.363 PhysLean/SpaceAndTime/Space/Slice.lean

```

module

public import Mathlib.Analysis.Calculus.ContDiff.FiniteDimension
public import PhysLean.SpaceAndTime.Space.Derivatives.Basic

@[expose] public section

```

1.364 PhysLean/SpaceAndTime/Space/Translations.lean

```

module

public import PhysLean.SpaceAndTime.Space.Derivatives.Curl
@[expose] public section

```


1.365 PhysLean/SpaceAndTime/SpaceTime/Basic.lean

module

```
public import PhysLean.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct
public import PhysLean.Relativity.SpeedOfLight
public import PhysLean.SpaceAndTime.Time.Basic
public import PhysLean.SpaceAndTime.Space.Integrals.Basic
@[expose] public section
```

1.366 PhysLean/SpaceAndTime/SpaceTime/Boosts.lean

module

```
public import PhysLean.Relativity.LorentzGroup.Boosts.Basic
public import PhysLean.Meta.Informal.SemiFormal
public import PhysLean.Mathematics.FDerivCurry
@[expose] public section
```

1.367 PhysLean/SpaceAndTime/SpaceTime/Derivatives.lean

module

```
public import PhysLean.SpaceAndTime.SpaceTime.LorentzAction
public import PhysLean.Relativity.Tensors.RealTensor.CoVector.Basic
public import Mathlib.Analysis.InnerProductSpace.TensorProduct

@[expose] public section
```

1.368 PhysLean/SpaceAndTime/SpaceTime/LorentzAction.lean

module

```
public import PhysLean.SpaceAndTime.Space.Derivatives.Basic

@[expose] public section
```

1.369 PhysLean/SpaceAndTime/SpaceTime/TimeSlice.lean

```

module

public import PhysLean.SpaceAndTime.SpaceTime.Derivatives
public import PhysLean.SpaceAndTime.TimeAndSpace.Basic
public import PhysLean.Meta.Informal.SemiFormal
public import PhysLean.Mathematics.FDerivCurry
@[expose] public section

```

1.370 PhysLean/SpaceAndTime/Time/Basic.lean

```

module

public import PhysLean.SpaceAndTime.Space.Basic
@[expose] public section

```

1.371 PhysLean/SpaceAndTime/Time/Derivatives.lean

```

module

public import PhysLean.SpaceAndTime.SpaceTime.Basic
@[expose] public section

@[fun_prop]
lemma deriv_contDiff_of_space {n} {M : Type} [NormedAddCommGroup M] [Normed
  (f : Time → Space d → M) (hf : ContDiff ℝ (n + 1) 1f) :
  ContDiff ℝ n fun (x : Space d) => (∂t fun t => f t x) t := by
  unfold deriv
  fun_prop

```

1.372 PhysLean/SpaceAndTime/Time/TimeMan.lean

```

module

public import Mathlib.Analysis.InnerProductSpace.Basic
public import Mathlib.Geometry.Manifold.Diffeomorph
@[expose] public section

```

1.373 PhysLean/SpaceAndTime/Time/TimeTransMan.lean

```
module
```

```
public import PhysLean.SpaceAndTime.Time.Derivatives
public import PhysLean.SpaceAndTime.Time.TimeUnit
@[expose] public section
```

1.374 PhysLean/SpaceAndTime/Time/TimeUnit.lean

```
module
```

```
public import Mathlib.Geometry.Manifold.Diffeomorph
public import PhysLean.SpaceAndTime.Time.Basic
@[expose] public section
```

1.375 PhysLean/SpaceAndTime/TimeAndSpace/Basic.lean

```
module
```

```
public import PhysLean.SpaceAndTime.Space.Derivatives.Curl
```

```
@[expose] public section
```

```
  Differentiable  $\mathbb{R}$  (fun t => ( $\nabla \times f_t$  t) x) := by
     $\partial_t$  (fun t => ( $\nabla \times f_t$  t) x) t = ( $\nabla \times$  fun x => ( $\partial_t$  (fun t =>  $f_t$  t x) t)) x := by
```

1.376 PhysLean/SpaceAndTime/TimeAndSpace/ConstantTimeDist.lean

```
module
```

```
public import PhysLean.SpaceAndTime.TimeAndSpace.Basic
public import Mathlib.Analysis.Calculus.ContDiff.FiniteDimension
```

```
@[expose] public section
```

1.377 PhysLean/StatisticalMechanics/BoltzmannConstant.lean

```

module

public import Mathlib.Data.NNReal.Defs

@[expose] public section

```

1.378 PhysLean/StatisticalMechanics/CanonicalEnsemble/Basic.lean

```

module

public import PhysLean.Thermodynamics.Temperature.Basic
public import Mathlib.MeasureTheory.Measure.ProbabilityMeasure
public import Mathlib.Analysis.Calculus.ParametricIntegral
public import PhysLean.Meta.Informal.SemiFormal
public import PhysLean.Meta.Linters.Sorry
public import Mathlib.Analysis.SpecialFunctions.Log.Summable
public import Mathlib.MeasureTheory.Integral.Prod
@[expose] public section

```

1.379 PhysLean/StatisticalMechanics/CanonicalEnsemble/Finite.lean

```

module

public import Mathlib.Algebra.Lie.OfAssociative
public import PhysLean.StatisticalMechanics.CanonicalEnsemble.Lemmas
@[expose] public section

```

1.380 PhysLean/StatisticalMechanics/CanonicalEnsemble/Lemmas.lean

```

module

public import PhysLean.StatisticalMechanics.CanonicalEnsemble.Basic

@[expose] public section

```

1.381.

PHYSLEAN/STATISTICALMECHANICS/CANONICALENSEMBLE/TWOSTATE.lean

1.381 PhysLean/StatisticalMechanics/CanonicalEnsemble/TwoState.lean

module

```
public import PhysLean.StatisticalMechanics.CanonicalEnsemble.Finite
public import PhysLean.Meta.Informal.Basic
@[expose] public section
```

1.382 PhysLean/StringTheory/Basic.lean

module

```
@[expose] public section
```

1.383 PhysLean/StringTheory/FTheory/SU5/Basic.lean

module

```
-/@[expose] public section
```

1.384 PhysLean/StringTheory/FTheory/SU5/Charges/AnomalyFree.lean

module

```
public import PhysLean.StringTheory.FTheory.SU5.Quanta.Basic
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.Map
@[expose] public section
```

1.385 PhysLean/StringTheory/FTheory/SU5/Charges/OfRationalSection.lean

module

```
public import PhysLean.Meta.TODO.Basic
public import Mathlib.Data.Fintype.Sets
public import Mathlib.Algebra.BigOperators.Group.List.Basic
@[expose] public section
```

1.386 PhysLean/StringTheory/FTheory/SU5/Charges/Viable.lean

```
module
```

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoClos
public import PhysLean.StringTheory.FTheory.SU5.Charges.OfRationalSection
```

```
@[expose] public section
def viableCompletions (I : CodimensionOneConfig) : Multiset (ChargeSpectrum
```

1.387 PhysLean/StringTheory/FTheory/SU5/Fluxes/Basic.lean

```
module
```

```
public import Mathlib.Analysis.Normed.Ring.Lemmas
```

```
@[expose] public section
```

1.388 PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/Chiral

```
module
```

```
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.Basic
public import Mathlib.Tactic.FinCases
```

```
@[expose] public section
```

1.389 PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/Comple

```
module
```

```
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.ChiralIndi
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems
```

```
@[expose] public section
```

1.390 PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/Elems.

```
module
```

```
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.Basic
```

1.391. *PHYSLEAN/STRINGTHEORY/FTHEORY/SU5/QUANTA/BASIC.LEAN* ^{W7}

```
public import Mathlib.Tactic.FinCases
```

```
@[expose] public section
```

1.391 PhysLean/StringTheory/FTheory/SU5/Quanta/Basic.lean

```
module
```

```
public import PhysLean.StringTheory.FTheory.SU5.Quanta.FiveQuanta
```

```
public import PhysLean.StringTheory.FTheory.SU5.Quanta.TenQuanta
```

```
public import PhysLean.StringTheory.FTheory.SU5.Charges.Viable
```

```
@[expose] public section
```

1.392 PhysLean/StringTheory/FTheory/SU5/Quanta/FiveQuanta.lean

```
module
```

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTer
```

```
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Completeness
```

```
@[expose] public section
```

1.393 PhysLean/StringTheory/FTheory/SU5/Quanta/IsViable.lean

```
module
```

```
public import PhysLean.StringTheory.FTheory.SU5.Charges.AnomalyFree
```

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.ZMod
```

```
@[expose] public section
```

1.394 PhysLean/StringTheory/FTheory/SU5/Quanta/TenQuanta.lean

```
module
```

```
public import PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimallyAllowsTer
```

```
public import PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Completeness
```

```
@[expose] public section
```

1.395 PhysLean/Thermodynamics/Basic.lean

module

@[expose] public section

1.396 PhysLean/Thermodynamics/IdealGas/Basic.lean

module

public import Mathlib.Analysis.SpecialFunctions.Pow.Real -- for Real.rpow_d
@[expose] public section

1.397 PhysLean/Thermodynamics/Temperature/Basic.lean

module

public import Mathlib.Analysis.Calculus.Deriv.Inv
public import Mathlib.Analysis.InnerProductSpace.Basic
public import PhysLean.StatisticalMechanics.BoltzmannConstant
public import PhysLean.Meta.TODO.Basic

@[expose] public section

1.398 PhysLean/Thermodynamics/Temperature/TemperatureUnits.lean

module

public import Mathlib.Geometry.Manifold.Diffeomorph
public import PhysLean.SpaceAndTime.Time.Basic
@[expose] public section

1.399 PhysLean/Units/Basic.lean

module

public import PhysLean.SpaceAndTime.Time.TimeUnit
public import PhysLean.SpaceAndTime.Space.LengthUnit


```
public import PhysLean.ClassicalMechanics.Mass.MassUnit
public import PhysLean.Electromagnetism.Charge.ChargeUnit
public import PhysLean.Thermodynamics.Temperature.TemperatureUnits
public import PhysLean.Units.Dimension
@[expose] public section
```

1.400 PhysLean/Units/Dimension.lean

module

```
public import Mathlib.Analysis.Normed.Field.Lemmas
@[expose] public section
```

1.401 PhysLean/Units/Examples.lean

module

```
public import PhysLean.Units.WithDim.Speed
@[expose] public section
```

1.402 PhysLean/Units/FDeriv.lean

module

```
public import PhysLean.Units.WithDim.Basic
@[expose] public section
```

1.403 PhysLean/Units/Integral.lean

module

```
public import PhysLean.Units.UnitDependent
@[expose] public section
```

1.404 PhysLean/Units/UnitDependent.lean

module

```
public import PhysLean.Units.Basic
@[expose] public section
```

1.405 PhysLean/Units/WithDim/Area.lean

module

```
public import PhysLean.Units.WithDim.Basic
@[expose] public section
```

1.406 PhysLean/Units/WithDim/Basic.lean

module

```
public import PhysLean.Units.UnitDependent
@[expose] public section
```

1.407 PhysLean/Units/WithDim/Energy.lean

module

```
public import PhysLean.Units.WithDim.Basic
@[expose] public section
```

1.408 PhysLean/Units/WithDim/Mass.lean

module

```
public import PhysLean.Units.Basic
@[expose] public section
```

1.409 PhysLean/Units/WithDim/Momentum.lean

module

```
public import PhysLean.Units.WithDim.Velocity
public import PhysLean.Units.WithDim.Mass
public import PhysLean.Units.WithDim.Basic
```

```
@[expose] public section
```

1.410 PhysLean/Units/WithDim/Pressure.lean

module

```
public import PhysLean.Units.WithDim.Basic
```

```
@[expose] public section
```

1.411 PhysLean/Units/WithDim/Speed.lean

module

```
public import PhysLean.Units.WithDim.Basic
```

```
@[expose] public section
```

1.412 PhysLean/Units/WithDim/Velocity.lean

module

```
public import PhysLean.Units.Basic
```

```
@[expose] public section
```

1.413 QuantumInfo.lean

/-

Copyright (c) 2025 Alex Meiburg. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Alex Meiburg

-/

```

--Mathlib imports
import QuantumInfo.ForMathlib

--Code
import QuantumInfo.Finite.Channel.DegradableOrder
import QuantumInfo.Finite.CPTPMap
import QuantumInfo.Finite.Distance
import QuantumInfo.Finite.Qubit.Basic
import QuantumInfo.Finite.ResourceTheory.FreeState
-- import QuantumInfo.Finite.ResourceTheory.ResourceTheory --
Commenting out for now -- pretty broken
import QuantumInfo.Finite.ResourceTheory.SteinsLemma
import QuantumInfo.Finite.Braket
import QuantumInfo.Finite.Capacity
import QuantumInfo.Finite.Ensemble
import QuantumInfo.Finite.Entanglement
import QuantumInfo.Finite.Entropy
-- import QuantumInfo.Finite.AxiomatizedEntropy.Defs --Experimental
-- import QuantumInfo.Finite.AxiomatizedEntropy.Renyi --Experimental
import QuantumInfo.Finite.MState
import QuantumInfo.Finite.Pinching
import QuantumInfo.Finite.POVm
import QuantumInfo.Finite.Unitary

--Documentation without code
import QuantumInfo.Finite.Capacity_doc

--Classical information theory
-- import QuantumInfo.ClassicalInfo.Capacity
-- import QuantumInfo.ClassicalInfo.Channel
import QuantumInfo.ClassicalInfo.Distribution
import QuantumInfo.ClassicalInfo.Entropy
import QuantumInfo.ClassicalInfo.Prob

--Statistical mechanics
import QuantumInfo.StatMech.Hamiltonian
import QuantumInfo.StatMech.IdealGas
import QuantumInfo.StatMech.ThermoQuantities

```

1.414 QuantumInfo/ClassicalInfo/Capacity.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.

```

Authors: Alex Meiburg

-/

import QuantumInfo.ClassicalInfo.Entropy

import Mathlib.Data.Finset.Fin

import Mathlib.Data.Fintype.Fin

--Classical capacity

-- * Define "code"

-- * Define (Shannon) capacity of a (iid, memoryless) channel

-- * Prove Shannon's capacity theorems

variable (A I O : Type*)

/-- Here we define a *Code* by an encoder and a decoder. The encoder is a function strings (*List*'s) of any length over an alphabet *A*, and returns strings over *I*; the decoder takes *O* and gives back strings over *A*. The idea is that a channel would map from strings of *I* to *O*. Important special cases are where *I=O* (the change the symbol set), and *FixedLengthCode* where the output lengths only depend /

structure Code where

encoder : List A → List I

decoder : List O → List A

/-- A *FixedLengthCode* is a *Code* whose output lengths don't depend on the input or the input length. -/

structure FixedLengthCode extends Code A I O where

enc_length : ℕ → ℕ

enc_maps_length : ∀ as, (encoder as).length = enc_length (as.length)

dec_length : ℕ → ℕ

dec_maps_length : ∀ is, (decoder is).length = dec_length (is.length)

/-- A *BlockCode* is a *FixedLengthCode* that (1) maps symbols in discrete blocks of (2) the encoded alphabet *I* has a canonical injection into *O*, and (3) has encoders that are inverses of each other. This is well-suited to describing noise and erasures: if the channel merely "corrupts" the data then this *O = I*; the erasure channel might *O = I ⊕ Unit* or *Option I*.

We define the behavior of a block code to "fail" if the input is not a multiple of block_in by having it return an empty list. -/

structure BlockCode (io : I → O) extends FixedLengthCode A I O where

block_in : ℕ

block_out : ℕ

block_enc : (Fin block_in → A) → (Fin block_out → I)

block_dec : (Fin block_out → O) → (Fin block_in → A)

block_enc_dec_inv : ∀ as, block_dec (io ∘ (block_enc as)) = as

enc_length na := if na % block_in != 0 then 0 else (na / block_in) * block_out

```

dec_length no := if no % block_out != 0 then 0 else (no / block_out) * bl
encoder as := if as.length % block_in != 0 then [] else
  sorry
decoder os := if os.length % block_out != 0 then [] else
  sorry
enc_maps_length := sorry
dec_maps_length := sorry

```

1.415 QuantumInfo/ClassicalInfo/Channel.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ClassicalInfo.Distribution
import Mathlib.MeasureTheory.Measure.MeasureSpaceDef

open BigOperators

/- Channels, as in information theory.

A `Channel` is as a function from `List`s over an input alphabet A to a dis
of lists over an output alphabet B. The most important case of interest her
is the memoryless channel, which preserves lengths and acts on each symbol
identically and independently.
-/

variable (I O : Type*) [Fintype O]

/- Actually defining Channels this way gets messy fast because of the measu

/-- Although we could simply `def Channel := List A → List B`, defining it
a structure lets us extend this structure by adding additional properties w
staying a `Channel`. -/
structure Channel [MeasurableSpace O] where
  map : List I → MeasureTheory.Measure (List O)
  map_prob : ∀is, IsProbabilityMeasure (map is)

namespace Channel

variable (A B C D : Type*)
variable (Cab : Channel A B) (Cbc : Channel B C) (Ccd : Channel C D)

```

```

/-- Identity channel -/
def id : Channel A A :=
  { _root_.id }

/-- Composition of two channels -/
def comp : Channel A C :=
  { Cbc.map ∘ Cab.map }

/-- Product of two channels, acting on two product types in parallel. -
/
def product : Channel (A × C) (B × D) :=
  { Cab.map × Ccd.map }
-/

/-- Discrete Memoryless Channel. Each input symbol `I` has a corresponding
    output distribution `Distribution O`, and this process is applied
    independently on each symbol in the list. -/
structure DMChannel where
  symb_dist : I → Distribution O

namespace DMChannel

/-- Apply a discrete memoryless channel to an n-character string. -
/
def on_fin (C : DMChannel I O) {n : ℕ} (is : Fin n → I) : ProbDistribution (Fin n → O) :=
  { fun os => ∏ k, C.symb_dist (is k) (os k), by
    -- change ∑ os in Fintype.piFinset fun x => (Finset.univ : Finset O), ∏ k : Fin n,
    -- have : ∀ i, Finset.sum Finset.univ (C.symb_dist i) = 1 :=
    --   fun i => Distribution.prop <| C.symb_dist i
    -- rw [Finset.sum_prod_piFinset]
    -- simp [this]
    sorry }

/-- Apply a discrete memoryless channel to a list. -/
def on_list (C : DMChannel I O) (is : List I) : ProbDistribution (Fin (is.length) → O) :=
  C.on_fin is.get

end DMChannel

```

1.416 QuantumInfo/ClassicalInfo/Distribution.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.

```

Authors: Alex Meiburg

-/

import QuantumInfo.ClassicalInfo.Prob

import Mathlib.Analysis.Convex.Combination

/-! # Distributions on finite sets

We define the type ``Distribution  $\alpha`$`  on a ``Fintype $\alpha`$` . By restricting ourselves to a lot of notation and casts are greatly simplified. This suffices for (most) dimensional quantum theory.

-/

noncomputable section

open NNReal

open Classical

open BigOperators

/--

We define our own (discrete) probability distribution notion here, instead of using ``PMF`` from Mathlib, because that uses ENNReals everywhere to maintain with ``MeasureTheory.Measure``.

The probabilities internal to a `Distribution` are NNReals. This lets us more easily write the statement that they sum to 1, since NNReals can be added. (Probabilities on their own, cannot.) But the `FunLike` instance gives ``Prob`` out, which carries information that they are all in the range $[0,1]$.

-/

```
def ProbDistribution ( $\alpha$  : Type u) [Fintype  $\alpha$ ] : Type u :=
  { f :  $\alpha \rightarrow \text{Prob}$  // Finset.sum Finset.univ (fun i  $\mapsto$  (f i :  $\mathbb{R}$ )) = 1 }
```

namespace ProbDistribution

variable { α β : Type*} [Fintype α] [Fintype β]

/-- Make a distribution, proving only that the values are nonnegative and the sum is 1. The fact that the values are at most 1 is derived as a consequence. /

```
def mk' (f :  $\alpha \rightarrow \mathbb{R}$ ) (h1 :  $\forall i, 0 \leq f\ i$ ) (hN :  $\sum i, f\ i = 1$ ) : ProbDistribution
  have h3 :  $\forall x, f\ x \leq 1$  := by
    intro x
    simp [← hN, Fintype.sum_eq_sum_compl_add x]
    exact Finset.sum_nonneg' h1
  (fun i  $\mapsto$  (f i, (h1 i, h3 i)), hN)
```

instance instFunLikeProb : FunLike (ProbDistribution α) α Prob where


```

    coe p a := p.1 a
    coe_injective' _ _ h :=
      Subtype.ext <| funext fun v ↦ by
        simpa only [Subtype.mk.injEq, coe_inj] using congrFun h v

@[simp]
theorem normalized (d : ProbDistribution  $\alpha$ ) : Finset.sum Finset.univ (fun i ↦ (d i :  $\mathbb{R}$ )) = d.2

abbrev prob (d : ProbDistribution  $\alpha$ ) := (d :  $\alpha \rightarrow \text{Prob}$ )

@[simp]
theorem fun_eq_val (d : ProbDistribution  $\alpha$ ) : d.val = d :=
  rfl

@[simp]
theorem funlike_apply (d :  $\alpha \rightarrow \text{Prob}$ ) (h : _) (x :  $\alpha$ ) :
  DFunLike.coe (self := instFunLikeProb) ⟨d, h⟩ x = d x :=
  rfl

@[ext]
theorem ext {p q : ProbDistribution  $\alpha$ } (h :  $\forall x, p x = q x$ ) : p = q :=
  DFunLike.ext p q h

/-- A distribution provides a witness that d is nonempty. -
/
theorem nonempty (d : ProbDistribution  $\alpha$ ) : Nonempty  $\alpha$  := by
  by_contra h
  simpa [not_nonempty_iff.mp h] using d.2

/-- Make an constant distribution: supported on a single element. This is also called
"One-point distribution", a "Degenerate distribution", a "Deterministic distribution",
"Delta function", or a "Point mass distribution". -/
def constant (x :  $\alpha$ ) : ProbDistribution  $\alpha$  :=
  ⟨fun y ↦ if x = y then 1 else 0,
    by simp [apply_ite]⟩

theorem constant_def (x :  $\alpha$ ) : (constant x :  $\alpha \rightarrow \text{Prob}$ ) = fun y ↦ if x = y then 1 else 0 := by
  rfl

@[simp]
theorem constant_eq (x :  $\alpha$ ) : constant x y = if x = y then 1 else 0 := by
  rfl

@[simp]
theorem constant_def' (x y :  $\alpha$ ) : (constant x :  $\alpha \rightarrow \text{Prob}$ ) y = if x = y then 1 else 0 := by
  rfl

```

```

    rw [← Prob.eq_iff_nnreal]
    change (Prob.toNNReal ∘ (constant x)) y = (if x = y then 1 else 0 : Prob)
    rw [constant_def x]
    split_ifs with h
    <;> simp [h]

/-- If a distribution has an element with probability 1, the distribution h
/
theorem constant_of_exists_one {D : ProbDistribution  $\alpha$ } {x :  $\alpha$ } (h : D x =
  ext y
  by_cases h2 : x = y
  · simp [h, ← h2]
  · simp only [constant_eq, h2, ↓reduceIte, Prob.coe_zero]
  by_contra h3
  replace h3 : 0 < (D y :  $\mathbb{R}$ ) := by
    linarith (config := {splitNe := true}) only [h3, @Prob.zero_le_coe (D
  have := D.normalized
  rw [← Finset.add_sum_erase _ _ (Finset.mem_univ x), h, Prob.coe_one] at
  rw [← Finset.add_sum_erase _ _ (a := y) (by simp using (Ne.symm h2))]
  have : 0 ≤  $\sum x \in \text{Finset.erase (Finset.erase Finset.univ x) y, (D x : } \mathbb{R})$ 
    Finset.sum_nonneg' (fun _ ↦ Prob.zero_le_coe)
  linarith

/-- Make an uniform distribution. -/
def uniform [n : Nonempty  $\alpha$ ] : ProbDistribution  $\alpha$  :=
  {fun _ ↦ {1 / (Finset.univ.card ( $\alpha$  :=  $\alpha$ )), by
    have : 0 < Finset.univ.card ( $\alpha$  :=  $\alpha$ ) :=
      Finset.Nonempty.card_pos (Finset.univ_nonempty_iff.mpr n)
    bound}, by simp}

@[simp]
theorem uniform_def [Nonempty  $\alpha$ ] (y :  $\alpha$ ) : ((uniform y) :  $\mathbb{R}$ ) = 1 / (Finset.
  rfl

/-- Make a distribution on a product of two Fintypes. -/
def prod (d1 : ProbDistribution  $\alpha$ ) (d2 : ProbDistribution  $\beta$ ) : ProbDistribu
  {fun x ↦ (d1 x.1) * (d2 x.2), by
    simp [← Finset.mul_sum, Fintype.sum_prod_type]}

@[simp]
theorem prod_def (x :  $\alpha$ ) (y :  $\beta$ ) : prod d1 d2 ⟨x, y⟩ = (d1 x) * (d2 y) :=
  rfl

/-- Given a distribution on  $\alpha$ , extend it to a distribution on ` $\text{Sum } \alpha \beta$ ` by
/
def extend_right (d : ProbDistribution  $\alpha$ ) : ProbDistribution ( $\alpha \oplus \beta$ ) :=

```

```

    {fun x ↦ Sum.casesOn x d.val (Function.const _ 0), by simp}

/-- Given a distribution on  $\alpha$ , extend it to a distribution on ` $\text{Sum } \beta \ \alpha$ ` by giving it
/
def extend_left (d : ProbDistribution  $\alpha$ ) : ProbDistribution ( $\beta \oplus \alpha$ ) :=
  {fun x ↦ Sum.casesOn x (Function.const _ 0) d.val, by simp}

/-- Make a convex mixture of two distributions on the same set. -
/
instance instMixable : Mixable ( $\alpha \rightarrow \mathbb{R}$ ) (ProbDistribution  $\alpha$ ) :=
  Mixable.instSubtype (inferInstance) (fun _ _ hab hx hy ↦ by
    simp [Mixable.mix_ab, Finset.sum_add_distrib, ← Finset.mul_sum, hab, hx, hy]
  )

/-- Given a distribution on type  $\alpha$  and an equivalence to type  $\beta$ , get the corresponding
distribution on type  $\beta$ . -/
def relabel (d : ProbDistribution  $\alpha$ ) ( $\sigma : \beta \approx \alpha$ ) : ProbDistribution  $\beta$  :=
  {fun b ↦ d ( $\sigma$  b),
   by rw [Equiv.sum_comp  $\sigma$  (fun a ↦ (d a :  $\mathbb{R}$ ))]; exact d.prop}

-- The two properties below (and congrRandVar) follow from the fact that Distribution
-- However, mathlib does not seem to support that outside of the CategoryTheory namespace
/-- ProbDistribution on  $\alpha$  and  $\beta$  are equivalent for equivalent types  $\alpha \approx \beta$ . -
/
def congr ( $\sigma : \alpha \approx \beta$ ) : ProbDistribution  $\alpha \approx$  ProbDistribution  $\beta$  := by
  constructor
  case toFun => exact fun d ↦ relabel d  $\sigma$ .symm
  case invFun => exact fun d ↦ relabel d  $\sigma$ 
  case left_inv =>
    intro d
    ext i
    unfold relabel
    simp only [← fun_eq_val, Equiv.symm_apply_apply, Subtype.coe_eta]
  case right_inv =>
    intro d
    ext i
    unfold relabel
    simp only [← fun_eq_val, Equiv.apply_symm_apply, Subtype.coe_eta]

@[simp]
theorem congr_apply ( $\sigma : \alpha \approx \beta$ ) (d : ProbDistribution  $\alpha$ ) (j :  $\beta$ ): (congr  $\sigma$  d) j = d
  rfl

/-- The inverse and congruence operations for distributions commute -
/
@[simp]

```

```

theorem congr_symm_apply ( $\sigma : \alpha \approx \beta$ ) : (ProbDistribution.congr  $\sigma$ ).symm = Pr
  rfl

/-- The distribution on Fin 2 corresponding to a coin with probability p. C
p of 0. -/
def coin (p : Prob) : ProbDistribution (Fin 2) :=
  ((if · = 0 then p else 1 - p), by simp)

@[simp]
theorem coin_val_zero (p : Prob) : coin p 0 = p := by
  simp [coin]

@[simp]
theorem coin_val_one (p : Prob) : coin p 1 = 1 - p := by
  simp [coin]

/-- Every distribution on two variable is some coin. -/
theorem fin_two_eq_coin (d : ProbDistribution (Fin 2)) : d = coin (d 0) :=
  ext i
  fin_cases i
  · simp [coin]
  · simp only [coin, Fin.mk_one, funlike_apply, one_ne_zero, ↓reduceIte,
    Prob.coe_one_minus, Subtype.ext_iff, Prob.coe_one_minus, eq_sub_iff_add,
    fun_eq_val, Fin.sum_univ_two] using d.property

theorem coin_eq_iff (p : Prob) (f : ProbDistribution (Fin 2)) :
  ProbDistribution.coin p = f  $\leftrightarrow$  p = f 0 := by
  constructor
  · rintro rfl
    rfl
  · rintro rfl
    rw [← ProbDistribution.fin_two_eq_coin f]

section randvar

/-- A `T`-valued random variable over ` $\alpha$ ` is a map `var :  $\alpha \rightarrow T$ ` along
with a probability distribution `distr : Distribution  $\alpha$ `. -
/
structure RandVar ( $\alpha : \text{Type}^*$ ) [Fintype  $\alpha$ ] (T : Type*) where
  var :  $\alpha \rightarrow T$ 
  distr : ProbDistribution  $\alpha$ 

instance instFunctor : Functor (RandVar  $\alpha$ ) where map f e := (f  $\circ$  e.1, e.2)

instance instLawfulFunctor : LawfulFunctor (RandVar  $\alpha$ ) where
  map_const { $\alpha$ } { $\beta$ } := by rfl

```

```

    id_map _ := by rfl
    comp_map _ _ _ := by rfl

-- `U` is required to be a group just because mix below uses Convex.sum_mem,
-- but it should be provable with just `AddCommMonoid U`
variable {T U : Type*} [AddCommGroup U] [Module ℝ U] [inst : Mixable U T]

/-- `Distribution.exp_val` is the expectation value of a random variable `X`. Under
it is the "convex combination over a finite family" on the type `T`, afforded by the
with the probability distribution of `X` as weights. -/
def expect_val (X : RandVar α T) : T := by
  let u : U := ∑ i ∈ Finset.univ, (X.distr i : ℝ) • (inst.to_U (X.var i))
  have ht : ∃ t : T, inst.to_U t = u := by
    have h₀ : ∀ i ∈ Finset.univ, 0 ≤ ↑(X.distr i) := by simp
    have h₁ : ∑ i ∈ Finset.univ, (X.distr i : ℝ) = 1 := by simp
    have h₂ : ∀ i ∈ Finset.univ, inst.to_U (X.var i) ∈ Set.range inst.to_U := by simp
    exact Set.mem_range.mp (inst.convex.sum_mem h₀ h₁ h₂)
  exact (inst.mkT ht).1

/-- The expectation value of a random variable over `α = Fin 2` is the same as `Mixa
with probabiliy weight `X.distr 0` -/
theorem expect_val_eq_mixable_mix (d : ProbDistribution (Fin 2)) (x₁ x₂ : T) : expect
  apply Mixable.to_U_inj
  simp only [Mixable.mix, expect_val, DFunLike.coe, Mixable.to_U_of_mkT]
  calc
    ∑ i : Fin (Nat.succ 0).succ, (d i : ℝ) • Mixable.to_U (![x₁, x₂] i) = ∑ i, (d i
      simp
    _ = (d 0 : ℝ) • Mixable.to_U (![x₁, x₂] 0) + (d 1 : ℝ) • Mixable.to_U (![x₁, x₂]
      simp
    _ = (d 0 : ℝ) • Mixable.to_U x₁ + (1 - d 0).val • Mixable.to_U x₂ := by
      congr
      simpa only [Subtype.ext_iff, Prob.coe_one_minus, eq_sub_iff_add_eq, add_comm,
        fun_eq_val, Fin.sum_univ_two] using d.property

/-- The expectation value of a random variable with constant probability distributio
/
theorem expect_val_constant (x : α) (f : α → T) : expect_val ⟨f, (constant x)⟩ = f x
  apply Mixable.to_U_inj
  simp only [expect_val, constant, DFunLike.coe, Mixable.to_U_of_mkT, apply_ite, Pro
    Prob.coe_zero, ite_smul, one_smul, zero_smul, Finset.sum_ite_eq, Finset.mem_univ

/-- The expectation value of a nonnegative real random variable is also nonnegative
/
theorem zero_le_expect_val (d : ProbDistribution α) (f : α → ℝ) (hpos : 0 ≤ f) : 0 ≤
  simp only [expect_val, Mixable.mkT, Mixable.to_U, id]
  apply Fintype.sum_nonneg

```

```

    intro x
    specialize hpos x
    exact mul_nonneg Prob.zero_le_coe hpos

/-- `T`-valued random variables on ` $\alpha$ ` and ` $\beta$ ` are equivalent if ` $\alpha \approx \beta$ ` -
/
def congrRandVar ( $\sigma : \alpha \approx \beta$ ) : RandVar  $\alpha$  T  $\approx$  RandVar  $\beta$  T := by
  constructor
  case toFun => exact fun X  $\mapsto$  { var := X.var  $\circ$   $\sigma$ .symm, distr := ProbDistrib
  case invFun => exact fun X  $\mapsto$  { var := X.var  $\circ$   $\sigma$ , distr := ProbDistributio
  case left_inv =>
    intro e
    dsimp
    congr
    · simp [Function.comp_assoc]
    · rw [← ProbDistribution.congr_symm_apply, Equiv.symm_apply_apply]
  case right_inv =>
    intro e
    dsimp
    congr
    · simp [Function.comp_assoc]
    · rw [← ProbDistribution.congr_symm_apply, Equiv.apply_symm_apply]

/-- Given a `T`-valued random variable `X` over ` $\alpha$ `, mapping over `T` commu
/
def map_congr_eq_congr_map {S : Type _} [Mixable U S] (f : T  $\rightarrow$  S) ( $\sigma : \alpha \approx \beta$ ) :
  f <$> congrRandVar  $\sigma$  X = congrRandVar  $\sigma$  (f <$> X) := by rfl

/-- The expectation value is invariant under equivalence of random variable
/
@[simp]
theorem expect_val_congr_eq_expect_val ( $\sigma : \alpha \approx \beta$ ) (X : RandVar  $\alpha$  T) : expect_val
  apply Mixable.to_U_inj
  simp only [expect_val, congrRandVar, Equiv.coe_fn_mk, Function.comp_apply,
    congr_apply]
  rw [Equiv.sum_comp  $\sigma$ .symm (fun i :  $\alpha \mapsto$  (X.distr i :  $\mathbb{R}$ ) • Mixable.to_U (X.

end randvar

end ProbDistribution

```

1.417 QuantumInfo/ClassicalInfo/Entropy.lean

```

/-

```

Copyright (c) 2025 Alex Meiburg. All rights reserved.
 Released under Apache 2.0 license as described in the file LICENSE.
 Authors: Alex Meiburg

```
-/
import QuantumInfo.ClassicalInfo.Distribution
import Mathlib.Analysis.SpecialFunctions.Log.NegMulLog
import Mathlib.Analysis.SpecialFunctions.BinaryEntropy
import QuantumInfo.ClassicalInfo.ForMathlib.Analysis.SpecialFunctions.Log.NegMulLog
```

```
/-! # Shannon entropy
```

Definitions and facts about the Shannon entropy function -
 $x \ln(x)$, both on a single
 variable and on a distribution.

There is significant overlap with `Real.negMulLog` and `Real.binEntropy` in Mathlib,
 and probably these files could be combined in some form. -
 /

```
noncomputable section
open NNReal
```

```
variable {α β : Type*} [Fintype α] [Fintype β]
```

```
/-- The one-event entropy function,  $H_1(p) = -p \ln(p)$ . Uses nits. -  

/
```

```
def H1 : Prob → ℝ :=
  fun x ↦ Real.negMulLog x
```

```
/--  $H_1$  of 0 is zero.-/
```

```
@[simp]
def H1_zero_eq_zero : H1 0 = 0 := by
  simp [H1]
```

```
/--  $H_1$  of 1 is zero.-/
```

```
@[simp]
def H1_one_eq_zero : H1 1 = 0 := by
  simp [H1]
```

```
/-- Entropy is nonnegative. -/
```

```
theorem H1_nonneg (p : Prob) : 0 ≤ H1 p := by
  rw [H1, Real.negMulLog, neg_mul, Left.nonneg_neg_iff]
  exact Real.mul_log_nonpos p.zero_le_coe p.coe_le_one
```

```
/-- Entropy is less than 1. -/
```

```
theorem H1_le_1 (p : Prob) : H1 p < 1 := by
```

```

rw [H1]
by_cases h : p = 0
· norm_num [h]
· have hp0 : 0 < p := lt_of_le_of_ne' p.zero_le h
  have h2 := Real.abs_log_mul_self_lt p hp0 p.coe_le_one
  rw [mul_comm, ← abs_neg, ← neg_mul] at h2
  exact lt_of_abs_lt h2

/-- Entropy is at most 1/e. -/
theorem H1_le_exp_m1 (p : Prob) : H1 p ≤ Real.exp (-1) :=
  Real.negMulLog_le_rexp_neg_one p.zero_le_coe

theorem H1_concave : ∀ (x y : Prob), ∀ (p : Prob), p[H1 x ↔ H1 y] ≤ H1 (p[x
  intros x y p
  simp only [H1, smul_eq_mul, Prob.coe_one_minus, Mixable.mix, Mixable.mix_
    Prob.mkT_mixable, Prob.to_U_mixable, Mixable.to_U_instUniv, Prob.to_U_m
  by_cases hxy : x = y
  · subst hxy
    ring_nf
    exact le_refl _
  by_cases hp : (p:ℝ) = 0
  · norm_num [hp]
  by_cases hp1 : (p:ℝ) = 1
  · norm_num [hp1]
  rw [← ne_eq] at hxy hp hp1
  have := Real.strictConcave0n_negMulLog.2
  replace := @this x ?_ y ?_ ?_ p (1 - p) ?_ ?_ (by linarith)
  · simp only [smul_eq_mul] at this
    apply le_of_lt
    convert this
  · simp only [Set.mem_Ici, Prob.zero_le_coe]
  · simp only [Set.mem_Ici, Prob.zero_le_coe]
  · simp only [Prob.ne_iff]
  · exact lt_of_le_of_ne p.zero_le_coe hp.symm
  · linarith (config := {splitNe := true}) [p.coe_le_one]

/-- The Shannon entropy of a discrete distribution,  $H(X) = \sum H_1(p_x)$ . -
/
def Hs (d : ProbDistribution α) : ℝ :=
  Finset.sum Finset.univ (fun x ↦ H1 (d.prob x))

/-- Shannon entropy of a distribution is nonnegative. -/
theorem Hs_nonneg (d : ProbDistribution α) : 0 ≤ Hs d :=
  Finset.sum_nonneg' fun _ ↦ H1_nonneg _

/-- Shannon entropy of a distribution is at most  $\ln d$ . -/

```



```

theorem Hs_le_log_d (d : ProbDistribution α) : Hs d ≤ Real.log (Fintype.card α) := by
  --Thanks Aristotle
  by_cases h : Fintype.card α = 0
  · simp_all [Hs, Fintype.card_eq_zero_iff.mp h]
  -- Since the sum of the probabilities is 1, we can apply Jensen's inequality for t
  x log x.
  have h_jensen {p : α → ℝ} (hsum : ∑ i, p i = 1) (hp : ∀ i, 0 ≤ p i ∧ p i ≤ 1) :
    -∑ i, p i * (p i).log ≤ Real.log (Fintype.card α) := by
    have h_jensen : (∑ i, (Fintype.card α : ℝ)-1 * p i) * (∑ i, (Fintype.card α : ℝ)
      (∑ i, (Fintype.card α : ℝ)-1 * (p i * (p i).log))) := by
    have h_convex : ConvexOn ℝ (Set.Icc 0 1) (fun x ↦ x * Real.log x) :=
      Real.convexOn_mul_log.subset Set.Icc_subset_Ici_self (convex_Icc 0 1)
    convert h_convex.map_sum_le _ _ _ <=> aesop
    simp_rw [← Finset.mul_sum, hsum, mul_one, Real.log_inv] at h_jensen
    have : 0 < (Fintype.card α : ℝ)-1 := by positivity
    have := mul_inv_cancel 0 <| show (Fintype.card α : ℝ) ≠ 0 by positivity
    nlinarith
    simp [Hs, H1, Real.negMulLog] using h_jensen d.2 (by grind)

/-- The shannon entropy of a constant variable is zero. -
/
@[simp]
theorem Hs_constant_eq_zero {i : α} : Hs (ProbDistribution.constant i) = 0 := by
  simp [Hs, apply_ite]

/-- Shannon entropy of a uniform distribution is ln d. -/
theorem Hs_uniform [Nonempty α] :
  Hs (ProbDistribution.uniform (α := α)) = Real.log (Finset.univ.card (α := α)) :=
  simp [Hs, ProbDistribution.prob, H1, Real.negMulLog]

/-- Shannon entropy of two-event distribution. -/
theorem Hs_coin (p : Prob) : Hs (ProbDistribution.coin p) = Real.binEntropy p := by
  simp [Hs, H1, ProbDistribution.coin, Real.binEntropy_eq_negMulLog_add_negMulLog_on]

lemma Hs_eq_of_multiset_map_eq (d1 : ProbDistribution α) (d2 : ProbDistribution β)
  (h : Multiset.map d1.prob Finset.univ.val = Multiset.map d2.prob Finset.univ.val) :
  Hs d1 = Hs d2 := by
  convert congr_arg (fun m ↦ m.map (fun x ↦ -Real.log x.1 * x.1 ) |> Multiset.sum )
  <=> simp [Hs, H1, mul_comm, Real.negMulLog]

--TODO:
-- * Shannon entropy is concave under mixing distributions.
-- * Shannon entropy as an expectation value

```

1.418 QuantumInfo/ClassicalInfo/ForMathlib/Analysis/SpecialFunc

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.SpecialFunctions.Log.NegMulLog

noncomputable section
open NNReal

namespace Real

theorem negMulLog_strictMonoOn : StrictMonoOn Real.negMulLog (Set.Icc 0 (exp
1))) := by
  apply strictMonoOn_of_deriv_pos
  · exact convex_Icc 0 (exp (-1))
  · exact continuous_negMulLog.continuousOn
  · intro x hx
    rw [interior_Icc, Set.mem_Ioo] at hx
    linarith only [log_exp (-1), log_lt_log hx.left hx.right, deriv_negMulLog h

theorem negMulLog_strictAntiOn : StrictAntiOn Real.negMulLog (Set.Ici (exp
1))) := by
  apply strictAntiOn_of_deriv_neg
  · exact convex_Ici (exp (-1))
  · exact continuous_negMulLog.continuousOn
  · intro x hx
    rw [interior_Ici' Set.nonempty_Ioo, Set.mem_Ioi] at hx
    have hx' : x ≠ 0 := by grind [exp_nonneg]
    linarith [log_exp (-1), log_lt_log (exp_pos (-1)) hx, deriv_negMulLog h

theorem negMulLog_le_rexp_neg_one {x : ℝ} (hx : 0 ≤ x) : negMulLog x ≤ exp
1) := by
  by_cases hp : x < exp (-1)
  · grw [negMulLog_strictMonoOn.monotoneOn (by grind) (by grind) hp.le]
    simp [negMulLog]
  by_cases hp' : Real.exp (-1) < x
  · grw [negMulLog_strictAntiOn.antitoneOn (by grind) (by grind) hp'.le]
    simp [negMulLog]
  · simp [show x = exp (-1) by order, negMulLog]

end Real

```

1.419 QuantumInfo/ClassicalInfo/Prob.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.Convex.Mul
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Analysis.SpecialFunctions.Log.ENNRealLog
import Mathlib.Data.NNReal.Basic
import Mathlib.Data.EReal.Basic
import Mathlib.Tactic.Finiteness
import Mathlib.Topology.UnitInterval

/-! # Probabilities

This defines a type `Prob`, which is simply any real number in the interval 0 to 1.
additional statements such as its application to convex sets, and it makes useful ty
functions that only make sense on probabilities.

A significant application is in the `Mixable` typeclass, also in this file, which is
of convex combination that applies to types as opposed to sets; elements are `Mixabl
-/

noncomputable section
open NNReal
open Classical

/-- `Prob` is a real number in the interval [0,1]. Similar to NNReal in many definit
allows other nice things more 'safely' such as convex combination. -
/
@[reducible]
def Prob := { p : ℝ // 0 ≤ p ∧ p ≤ 1 }

namespace Prob

instance : Coe Prob ℝ := ⟨Subtype.val⟩

instance canLift : CanLift ℝ Prob Subtype.val fun r => 0 ≤ r ∧ r ≤ 1 :=
  Subtype.canLift _

instance instZero : Zero Prob :=
  ⟨0, by simp⟩

instance instOne : One Prob :=

```

```

    (1, by simp)

instance instMul : Mul Prob :=
  (fun x y ↦ ⟨x.1 * y.1,
    ⟨mul_nonneg x.2.1 y.2.1, mul_le_one x.2.2 y.2.1 y.2.2⟩⟩)

@[simp, norm_cast]
theorem coe_zero : (0 : Prob).val = 0 :=
  rfl

@[simp, norm_cast]
theorem coe_one : (1 : Prob).val = 1 :=
  rfl

@[simp, norm_cast]
theorem coe_mul (x y : Prob) : (x * y).val = x.val * y.val :=
  rfl

@[simp, norm_cast]
theorem coe_inf (x y : Prob) : (x ⊓ y).val = x.val ⊓ y.val :=
  rfl

@[simp, norm_cast]
theorem coe_sup (x y : Prob) : (x ⊔ y).val = x.val ⊔ y.val :=
  rfl

instance instCommMonoidWithZero : CommMonoidWithZero Prob where
  mul_assoc := by intros; ext; simp [mul_assoc]
  one_mul := by intros; ext; simp
  mul_one := by intros; ext; simp
  mul_comm := by intros; ext; simp [mul_comm]
  mul_zero := by intros; ext; simp
  zero_mul := by intros; ext; simp

instance instDenselyOrdered : DenselyOrdered Prob :=
  show DenselyOrdered (Set.Icc 0 1) from Set.instDenselyOrdered

instance instCompleteLinearOrder : CompleteLinearOrder Prob :=
  instCompleteLinearOrderElemIccOfFactLe

instance instInhabited : Inhabited Prob where
  default := 0

instance : LinearOrderedCommMonoidWithZero Prob where
  mul_lt_mul_of_pos_left := by
    intros a ha b h hb

```

```

    rw [← Subtype.coe_lt_coe]
    exact mul_lt_mul_of_pos_left hb ha
    zero_le a := a.2.1

@[simp]
theorem zero_le_coe {p : Prob} :  $0 \leq (p : \mathbb{R})$  :=
  p.2.1

@[simp]
theorem coe_le_one {p : Prob} :  $(p : \mathbb{R}) \leq 1$  :=
  p.2.2

@[simp]
theorem zero_le {p : Prob} :  $0 \leq p$  :=
  zero_le_coe

@[simp]
theorem le_one {p : Prob} :  $p \leq 1$  :=
  coe_le_one

@[ext] protected theorem ext {n m : Prob} :  $(n : \mathbb{R}) = (m : \mathbb{R}) \rightarrow n = m$  :=
  Subtype.ext

theorem ne_iff {x y : Prob} :  $(x : \mathbb{R}) \neq (y : \mathbb{R}) \leftrightarrow x \neq y$  :=
  not_congr <| Prob.ext_iff.symm

@[simp, norm_cast]
theorem toReal_mul (x y : Prob) :  $(x * y : Prob) = (x : \mathbb{R}) * (y : \mathbb{R})$  := by
  simp only [coe_mul]

/-- Coercion `Prob → ℝ≥0`. -/
@[coe] def toNNReal : Prob →  $\mathbb{R}_{\geq 0}$  :=
  fun p ↦ ⟨p.val, zero_le_coe⟩

@[simp]
theorem toNNReal_mk : toNNReal { val := x, property := hx } = { val := x, property :=
  rfl

instance : Coe Prob  $\mathbb{R}_{\geq 0}$  := ⟨toNNReal⟩

instance canLiftNN : CanLift  $\mathbb{R}_{\geq 0}$  Prob toNNReal fun r =>  $r \leq 1$  :=
  ⟨fun x hx ↦ ⟨x, ⟨x.2, hx⟩⟩, rfl⟩

protected theorem eq_iff_nnreal (n m : Prob) :  $(n : \mathbb{R}_{\geq 0}) = (m : \mathbb{R}_{\geq 0}) \leftrightarrow n = m$  := by
  obtain ⟨n, hn⟩ := n
  obtain ⟨m, hn⟩ := m

```

```

    simp only [toNNReal_mk, Subtype.mk.injEq, NNReal]

@[simp, norm_cast]
theorem toNNReal_zero : (0 : Prob) = (0 :  $\mathbb{R}_{\geq 0}$ ) :=
  rfl

@[simp, norm_cast]
theorem toNNReal_one : (1 : Prob) = (1 :  $\mathbb{R}_{\geq 0}$ ) :=
  rfl

theorem ofNNReal_toNNReal : ENNReal.ofNNReal (toNNReal p) = ENNReal.ofReal
  simp [toNNReal, ENNReal.ofReal_eq_coe_nnreal]

def NNReal.asProb (p :  $\mathbb{R}_{\geq 0}$ ) (hp : p ≤ 1) : Prob :=
  ⟨p, ⟨p.2, hp⟩⟩

def NNReal.asProb' (p :  $\mathbb{R}_{\geq 0}$ ) (hp : p.1 ≤ 1) : Prob :=
  ⟨p, ⟨p.2, hp⟩⟩

theorem zero_lt_coe {p : Prob} (hp : p ≠ 0) : (0 :  $\mathbb{R}$ ) < p :=
  lt_of_le_of_ne' p.zero_le (unitInterval.coe_ne_zero.mpr hp)

/-- Subtract a probability from another. Truncates to zero, so this is often
to work with, for the same reason that Nat subtraction is a pain. But, it l
`1 - p`, which is sufficiently useful on its own that this seems worth havi
/
instance instSub : Sub Prob where
  sub p q := ⟨(p - q)  $\sqcap$  (0 :  $\mathbb{R}$ ), by
    simp using le_add_of_le_of_nonneg p.2.2 q.2.1
  ⟩

theorem coe_sub (p q : Prob) : (p - q : Prob) = (p.val - q.val)  $\sqcap$  (0 :  $\mathbb{R}$ )
  rfl

@[simp, norm_cast]
theorem coe_one_minus (p : Prob) : (1 - p : Prob) = 1 - (p :  $\mathbb{R}$ ) := by
  simp [coe_sub]

theorem add_one_minus (p : Prob) : p.val + (1 - p).val = 1 := by
  simp

@[simp]
theorem one_minus_inv (p : Prob) : 1 - (1 - p) = p := by
  ext
  simp

```

```

instance : OrderTopology Prob :=
  orderTopology_of_ordConnected (ht := Set.ordConnected_Icc)

@[simp, norm_cast]
theorem coe_iInf {ι : Type*} [Nonempty ι] (f : ι → Prob) : ↑(⋂ t, f t) = (⋂ t, f t :
  apply Monotone.map_ciInf_of_continuousAt
    · fun_prop
    · exact fun _ => id
    · exact OrderBot.bddBelow _)

instance : Nontrivial Prob where
  exists_pair_ne := ⟨0, 1, by simp [← Prob.ne_iff]⟩

@[simp]
theorem top_eq_one : (⊤ : Prob) = 1 := by
  rfl

@[simp]
theorem sub_zero (p : Prob) : p - 0 = p := by
  ext1; simp [coe_sub]

@[fun_prop]
theorem toNNReal_Continuous : Continuous Prob.toNNReal := by
  unfold Prob.toNNReal
  fun_prop

@[simp]
theorem mul_eq_one_iff (p q : Prob) : p * q = 1 ↔ p = 1 ∧ q = 1 := by
  cases p
  cases q
  refine ⟨fun h => ?_, fun h => by simp [h]⟩
  simp [Prob.ext_iff] at h ⊢
  constructor <=> nlinarith

end Prob

/-- A `Mixable T` typeclass instance gives a compact way of talking about the action
for forming linear combinations in convex spaces. The notation `p [ x₁ ↔ x₂ ]` mea
combination, equal to `x₁` if `p=1` and to `x₂` if `p=0`.

Mixable is defined by an "underlying" data type `U` with addition and scalar multi
bijection between the `T` and a convex set of `U`. For instance, in `Mixable (Dist
`U` is `n`-element vectors (which form the probability simplex, degenerate in one
`QuantumInfo.Finite.MState` density matrices in quantum mechanics, which are PSD m
`U` is the underlying matrix.

```

Why not just stick with existing notions of ``Convex``? ``Convex`` requires `t`AddCommMonoid`` and ``Module ℝ``. But many types, such as ``Distribution``, a "multiplying a probability distribution by 0.3" to get another distribution into, say, a vector or a function, but then we are not doing arithmetic with the expression ``0.3 * distA + 0.7 * distB`` cannot represent a distribution.

```

/
class Mixable (U : outParam (Type u)) (T : Type v) [AddCommMonoid U] [Module ℝ U]
  /-- Getter for the underlying data -/
  to_U : T → U
  /-- Proof that this getter is injective -/
  to_U_inj : ∀ {T₁ T₂}, to_U T₁ = to_U T₂ → T₁ = T₂
  /-- Proof that this image is convex -/
  convex : Convex ℝ (Set.range to_U)
  /-- Function to get a T from a proof that U is in the set. -
  /
  mkT : {u : U} → (∃ t, to_U t = u) → { t : T // to_U t = u }

namespace Mixable

variable {T U : Type*} [AddCommMonoid U] [Module ℝ U]

@[reducible]
def mix_ab [inst : Mixable U T] {a b : ℝ} (ha : 0 ≤ a) (hb : 0 ≤ b) (hab :
  inst.mkT <| inst.convex
    (x := to_U x₁) (exists_apply_eq_apply _ _)
    (y := to_U x₂) (exists_apply_eq_apply _ _)
    ha hb hab

/-- `Mixable.mix` represents the notion of "convex combination" on the type
instance. It takes a `Prob`, that is, a `Real` between 0 and 1. For working
/
def mix [inst : Mixable U T] (p : Prob) (x₁ x₂ : T) : T :=
  inst.mix_ab p.zero_le_coe (1 - p).zero_le_coe p.add_one_minus x₁ x₂

@[simp]
theorem to_U_of_mkT [inst : Mixable U T] (u : U) {h} : inst.to_U (mkT (u :=
  (mkT (u := u) h)).2

notation p "[" x₁:80 "↔" x₂ "]" => mix p x₁ x₂

notation p "[" x₁:80 "↔" x₂ ":" M "]" => mix (inst := M) p x₁ x₂

@[simp]
theorem mix_zero [inst : Mixable U T] (x₁ x₂ : T) : (0 : Prob) [ x₁ ↔ x₂ :
  apply inst.to_U_inj
  simp [mix, mix_ab]

```



```

/--When T is the whole space, and T is a suitable vector space over  $\mathbb{R}$ , we get a Mixable
/
instance instUniv [AddCommMonoid T] [Module  $\mathbb{R}$  T] : Mixable T T where
  to_U := id
  to_U_inj := id
  convex := by
    convert convex_univ
    simp only [Set.range_id]
  mkT := fun _ => {_, rfl}

@[simp]
theorem mkT_instUniv [AddCommMonoid T] [Module  $\mathbb{R}$  T] {t : T} (h :  $\exists t', \text{to\_U } t' = t$ ) :
  rfl

@[simp]
theorem to_U_instUniv [AddCommMonoid T] [Module  $\mathbb{R}$  T] {t : T} : instUniv.to_U t = t :
  rfl

section pi

theorem instPi.lem_1 {D : Type*} {T U : D → Type*} [Vi, AddCommMonoid (U i)] [V i, Module  $\mathbb{R}$  (U i)]
  {u : (i : D) → U i} (h :  $\exists (t : (i : D) \rightarrow T i), (\text{fun } d \Rightarrow \text{to\_U } (t d)) = u$ ) (d : D) :
  obtain (t, h) := h
  use t d
  exact congrFun h d

variable {D : Type*} {T U : D → Type*} [Vi, AddCommMonoid (U i)] [V i, Module  $\mathbb{R}$  (U i)]
/-- Mixable instance on Pi types. -/
instance instPi : Mixable ((i:D) → U i) ((i:D) → T i) where
  to_U x := fun d => (inst d).to_U (x d)
  to_U_inj h := funext fun d => (inst d).to_U_inj (congrFun h d)
  mkT := fun {u} h => {fun d => (inst d).mkT (u := u d) (instPi.lem_1 h d),
    by funext d; simp}
  convex := by
    simp [Convex, StarConvex]
    intro f1 f2 a b ha hb hab
    use fun d => (inst d).mix_ab ha hb hab (f1 d) (f2 d)
    funext d
    simp only [to_U_of_mkT, Pi.add_apply, Pi.mul_apply]

@[simp]
theorem val_mkT_instPi (D : Type*) [inst : Mixable U T] {u : D → U} (h :  $\exists t, \text{to\_U } t = u$ ) :
  fun d => (inst.mkT (instPi.lem_1 h d)).val :=
  rfl

```

```

@[simp]
theorem to_U_instPi (D : Type*) [inst : Mixable U T] {t : D → T} : (instPi)
  rfl

end pi

/-- Mixable instances on subtypes (of other mixable types), assuming that t
  have the correct closure properties. -/
def instSubtype {T : Type*} {P : T → Prop} (inst : Mixable U T)
  (h : ∀{x y:T},
    ∀a b : ℝ, (ha : 0 ≤ a) → (hb : 0 ≤ b) → (hab : a + b = 1) →
    P x → P y → P (inst.mix_ab ha hb hab x y))
  : Mixable U { t // P t } where
to_U x := inst.to_U (x.val)
to_U_inj h := Subtype.ext (inst.to_U_inj h)
mkT := fun {u} h ↦ {by
  have ⟨t, hu⟩ := inst.mkT (u := u) $ h.casesOn fun t h ↦ ⟨t, h⟩
  use t
  have ⟨t₁, ht₁⟩ := h
  exact (inst.to_U_inj $ hu.trans ht₁.symm) ▸ t₁.prop,
  by simp only [to_U_of_mkT]}
convex := by
  have hi := inst.convex
  simp [Convex, StarConvex] at hi
  intro x hx y hy a b ha hb hab
  let ⟨z, hz⟩ := hi x y ha hb hab
  refine ⟨z, {?, hz}⟩
  convert h ha hb hab hx hy
  apply inst.to_U_inj
  convert hz
  simp only [to_U_of_mkT]

end Mixable

namespace Prob

/-- Probabilities `Prob` themselves are convex. -/
instance instMixable : Mixable ℝ Prob where
to_U := Subtype.val
to_U_inj := Prob.ext
mkT := fun h ↦ {⟨_, Exists.casesOn h fun t ht => ht ▸ t.prop⟩, rfl}
convex := by
  simp [Convex, StarConvex]
  intro x hx0 hx1 y hy0 hy1 a b ha hb hab
  constructor
  · positivity

```

```

    · nlinarith

@[simp]
theorem to_U_mixable [AddCommMonoid T] [SMul ℝ T] (t : Prob) : instMixable.to_U t =
  rfl

@[simp]
theorem mkT_mixable (u : ℝ) (h : ∃ t : Prob, Mixable.to_U t = u) : Mixable.mkT h =
  ⟨(u, Exists.casesOn h fun t ht => ht ▸ t.2), rfl⟩ :=
  rfl

/-- `Prob.mix` is an alias of `Mixable.mix` so it can be accessed from a probability
dot notation, e.g. `p.mix x y`. -/
abbrev mix [AddCommMonoid U] [Module ℝ U] [inst : Mixable U T] (p : Prob) (x₁ x₂ : T)
  := p.mix x₁ x₂

section negLog
open ENNReal

/-- Map a probability [0,1] to [0,+∞] with -log p. Special case that 0 maps to +∞ (n
does). This makes it `Antitone`.
-/
noncomputable def negLog : Prob → ENNReal :=
  fun p => if p = 0 then ∞ else .ofNNReal (-Real.log p,
    Left.nonneg_neg_iff.mpr (Real.log_nonpos p.2.1 p.2.2))

--Note that this is an em-dash `−` and not a minus `−`, to make the notation work.
scoped notation "−log " => negLog

--TODO: Upgrade to `StrictAnti`. Even better: bundle negLog as `Prob ≈o ENNReal^od`.
theorem negLog_Antitone : Antitone negLog := by
  intro x y h
  dsimp [negLog]
  split_ifs with h₁ h₂ h₂
  · rfl
  · subst y
    exfalso
    change x.1 ≤ 0 at h
    have : ¬(x.1 = 0) := unitInterval.coe_ne_zero.mpr (by assumption)
    have : 0 ≤ x.1 := zero_le
    linarith +splitNe
  · exact OrderTop.le_top _
  · rw [ENNReal.coe_le_coe, ← NNReal.coe_le_coe, coe_mk, coe_mk, neg_le_neg_iff]
    apply (Real.log_le_log_iff _ _).mpr h
    <=> exact lt_of_le_of_ne zero_le (unitInterval.coe_ne_zero.mpr (by assumption)).

@[simp]

```

```

theorem negLog_zero :  $-\log (0 : \text{Prob}) = \top$  := by
  simp [negLog]

@[simp]
theorem negLog_one :  $-\log 1 = 0$  := by
  simp [negLog]

@[simp]
theorem negLog_eq_top_iff {p : Prob} :  $-\log p = \top \leftrightarrow p = 0$  := by
  simp [negLog]

theorem negLog_pos_ENNReal {p : Prob} (hp :  $p \neq 0$ ) :  $-\log p = .\text{ofNNReal } (-\text{Real.log } p,$ 
  Left.nonneg_neg_iff.mpr (Real.log_nonpos p.2.1 p.2.2)) := by
  simp [negLog, hp]

@[simp]
theorem negLog_pos_Real {p : Prob} :  $(-\log p).\text{toReal} = -\text{Real.log } p$  := by
  rw [negLog]
  split_ifs with hp
  · simp [hp]
  · simp

theorem le_negLog_of_le_exp {p : Prob} {x :  $\mathbb{R}$ } (h :  $p \leq \text{Real.exp } (-x)$ ) :  $\text{ENNReal.ofReal } x \leq -\log p$  := by
  by_cases hx :  $0 \leq x$ 
  · rw [negLog]
    split_ifs with hp
    · exact le_top
    · replace hp :  $0 < p := \text{lt\_of\_le\_of\_ne' } p.\text{zero\_le } hp$ 
      rw [le_iff_lt_or_eq] at h
      rcases h with h|h
      · apply le_of_lt
        replace h := Real.strictMonoOn_log hp (Real.exp_pos _) h
        rw [Real.log_exp] at h
        rw [← ENNReal.toReal_lt_toReal ofReal_ne_top coe_ne_top, toReal_ofR]
        simp using lt_neg_of_lt_neg h
      · apply le_of_eq
        rw [← ENNReal.toReal_eq_toReal_iff' ofReal_ne_top coe_ne_top,
          coe_toReal, coe_mk, h, Real.log_exp, neg_neg, toReal_ofReal hx]
  · trans 0
    · simp only [nonpos_iff_eq_zero, ofReal_eq_zero, le_of_not_ge hx]
    · exact _root_.zero_le _

@[aesop (rule_sets := [finiteness]) safe apply]
theorem negLog_ne_top {p : Prob} (hp :  $0 < p.\text{val}$ ) :  $-\log p \neq \infty$  := by

```

```

    simp [negLog] using ne_of_gt hp

theorem negLog_eq_neg_ENNReal_log (p : Prob) : -log p = -
ENNReal.log p := by
  rw [negLog]
  split_ifs with hp
  · simp [hp]
  · rw [log, if_neg, if_neg]
    · norm_cast
    · finiteness
    · rw [Subtype.ext_iff] at hp
      rw [toNNReal, ENNReal.coe_eq_zero]
      exact NNReal.coe_ne_zero.mp hp

theorem negLog_eq_ofReal_neg_log {p : Prob} (hp : 0 < p) :
  ENNReal.ofReal (-Real.log p) = -log p := by
  rcases p with ⟨p, p0, p1⟩
  rw [negLog]
  split_ifs with h
  · simp_all
  · exact ENNReal.ofReal_eq_coe_nnreal (neg_nonneg_of_nonpos (Real.log_nonpos p0 p1))

@[simp]
theorem zero_lt_negLog {p : Prob} : 0 < -log p ↔ p ≠ 1 := by
  --This is messy enough it's probably a sign we're missing other simp lemmas
  rw [negLog]
  split_ifs with h
  · simp [h]
  constructor <=> intro h₂ <=> contrapose! h₂
  · simp [h₂]
  simp only [nonpos_iff_eq_zero, ENNReal.coe_eq_zero] at h₂
  rw [Subtype.ext_iff] at h₂
  simp only [NNReal.val_eq_coe, NNReal.coe_zero, neg_eq_zero, Real.log_eq_zero,
    Set.Icc.coe_eq_zero, Set.Icc.coe_eq_one] at h₂
  rcases h₂ with h₂|h₂|h₂
  · contradiction
  · assumption
  · linarith [p.zero_le_coe]

@[fun_prop]
theorem Continuous_negLog : Continuous negLog := by
  --Thanks Aristotle
  have h_cont_at_zero : ContinuousAt -log 0 := by
    unfold Prob.negLog
    rw [ContinuousAt, if_pos rfl, ENNReal.tendsto_nhds_top_iff_nnreal]
    intro x

```

```

    rw [Metric.eventually_nhds_iff]
    use Real.exp (-x), by positivity
    rintro (a, ha0, ha1) ha'
    rw [Subtype.dist_eq, Set.Icc.coe_zero, dist_zero_right, Real.norm_eq_ab]
    split_ifs with h; · simp
    rw [Subtype.mk_eq_mk, Set.Icc.coe_zero] at h
    simp only [ENNReal.coe_lt_coe, ← NNReal.coe_lt_coe, NNReal.coe_mk]
    replace ha' := Real.log_lt_log (by positivity) ((le_abs_self _).trans_l)
    simp only [Real.log_exp] at ha'
    linarith
  have h_cont_on_pos : ContinuousOn -log (Set.Ioi 0) := by
    intro p hp
    apply Filter.Tendsto.congr'
    · filter_upwards [self_mem_nhdsWithin] with x hx using negLog_eq_ofReal
    · rw [← negLog_eq_ofReal_neg_log hp]
      apply ENNReal.continuous_ofReal.continuousAt.tendsto.comp
      exact (continuous_subtype_val.continuousWithinAt.tendsto.log hp.ne').
  rw [continuous_iff_continuousAt]
  rintro (p, (·, ·))
  rcases lt_trichotomy p 0 with h | rfl | h
  · order
  · exact h_cont_at_zero
  · exact h_cont_on_pos.continuousAt (Ioi_mem_nhds h)

end negLog

end Prob

```

1.420 QuantumInfo/Finite/AxiomatizedEntropy/Defs.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ClassicalInfo.Entropy
import QuantumInfo.Finite.MState
import QuantumInfo.Finite.CPTPMap

```

```

/-! # Generalized quantum entropy and relative entropy

```

Here we define a broad notion of entropy axiomatically, ``Entropy``, and the ``Entropy f`` means that the function ``f : MState → ℝ`` acts like a generalized entropy. For instance, min-, max-, α -Renyi, and von Neumann entropies all f

into this category. We prove various properties about the entropy for anything supporting this type class. Any entropy automatically gets corresponding notions of conditional entropy, mutual information, and so on.

Similarly, ``RelEntropy f`` means that ``f : MState → HermitianMat → ENNReal`` is a kind of relative entropy. Every ``RelEntropy`` leads to a notion of entropy, as well, by fixing one argument to the fully mixed state.

Of course relative entropies are "usually" used with a pair of (normalized) quantum states, but it's still very common in literature to specifically let the second argument be an arbitrary (PSD, Hermitian) matrix, so we do allow this. The behavior when not a density matrix is left unspecified by the axioms.

In terms of the file structure, we start with ``RelEntropy`` as the more "general" function, and then derive much of ``Entropy`` from it.

References:

- [Khinchin's Fourth Axiom of Entropy Revisited](<https://www.mdpi.com/2571-905X/6/3/49>)
- [α -z Relative Entropies](<https://warwick.ac.uk/fac/sci/math/research/events/2013/2014/statmech/su/Nilanjana-slides.pdf>)
- Watrous's notes, [Max-relative entropy and conditional min-entropy](<https://cs.uwaterloo.ca/~watrous/QIT-notes/QIT-notes.02.pdf>)
- [Quantum Relative Entropy - An Axiomatic Approach](<https://www.marcotom.info/files/masterclass2022.pdf>)
by Marco Tomamichel
- [StackExchange](<https://quantumcomputing.stackexchange.com/a/12953/10115>)

-/

noncomputable section
universe u

open scoped NNReal
open scoped ENNReal

variable (f : ∀ {d : Type u} [Fintype d] [DecidableEq d], MState d → HermitianMat d

/-- The axioms to be a well-behaved quantum relative entropy, as given by [Tomamichel](<https://www.marcotom.info/files/entropy-masterclass2022.pdf>). -

This simpler class allows for `_trivial_` relative entropies, such as ``-log tr(ρ0σ)``.

Use mixing ``RelEntropy.Nontrivial`` to only allow nontrivial relative entropies. -
/

```

class RelEntropy : Prop where
  /-- The data processing inequality -/
  DPI {d1 d2 : Type u} [Fintype d1] [DecidableEq d1] [Fintype d2] [DecidableEq d2]
    (ρ σ : MState d1) (Λ : CPTPMap d1 d2) : f (Λ ρ) (Λ σ) ≤ f ρ σ
  /-- Entropy is additive under tensor products -/
  of_kron {d1 d2 : Type u} [Fintype d1] [Fintype d2] [DecidableEq d1] [DecidableEq d2]
    ∀ (ρ1 σ1 : MState d1) (ρ2 σ2 : MState d2), f (ρ1 ⊗ ρ2) (σ1 ⊗ σ2) = f ρ1 σ1 + f ρ2 σ2
  /-- Normalization of entropy to be `ln N` for a pure state vs. uniform on d -/
  /
  normalized {d : Type u} [fin : Fintype d] [DecidableEq d] [Nonempty d] (i : i == 1) :
    f (.ofClassical (.constant i)) MState.uniform.M =
      some (Real.log fin.card, Real.log_nonneg (mod_cast Fintype.card_pos))

  /-- The axioms to be a well-behaved quantum relative entropy, as given by
  [Tomamichel](https://www.marcotom.info/files/entropy-masterclass2022.pdf). -/
  /
  class RelEntropy.Nontrivial [RelEntropy f] where
    /-- Nontriviality condition for a relative entropy. -/
    nontrivial (d) [Fintype d] [DecidableEq d] : ∃ (ρ σ : MState d),
      ρ.M.support = τ ∧ σ.M.support = τ ∧ 0 < f ρ σ

namespace RelEntropy

variable {d : Type u} [Fintype d] [DecidableEq d]
variable {d2 : Type u} [Fintype d2] [DecidableEq d2]

variable [RelEntropy f]

section possibly_trivial

/-
At some point we might want to offer a different constructor so that `normalized`
it for domains of size 2, which is sufficient (see Tomamichel's proof). In
fact that it's still zero when `Unique d` has to be proven, and this (now u
can be used inpart for that:

-- have h_uniq (ρ') := (Subsingleton.allEq ρ ρ').symm
-- have h_kron := of_kron (f := f) ρ ρ ρ ρ
-- let e : d ≈ (d × d) := (Equiv.prodUnique d d).symm
-- rw [← relabel_eq f e] at h_kron
-- rw [h_uniq ((ρ⊗ρ).relabel e)] at h_kron
-- rw [h_uniq σ]

At that point we need the fact that it's not `τ`, and then it must be zero.
-/

```



```

/-- Relabelling a state with `CPTPMap.of_equiv` leaves relative entropies unchanged. -/
@[simp]
theorem of_equiv_eq (e : d  $\approx$  d2) (ρ σ : MState d) :
  f (CPTPMap.of_equiv e ρ) (CPTPMap.of_equiv e σ) = f ρ σ := by
  apply le_antisymm
  · apply DPI
  · convert DPI (f := f) ((CPTPMap.of_equiv e) ρ) ((CPTPMap.of_equiv e) σ) (CPTPMap.
    · symm
    exact congrFun (CPTPMap.equiv_inverse e.symm) ρ
    · symm
    exact congrFun (CPTPMap.equiv_inverse e.symm) σ

/-- Relabelling a state with `MState.relabel` leaves relative entropies unchanged. -/
@[simp]
theorem relabel_eq (e : d2  $\approx$  d) (ρ σ : MState d) :
  f (ρ.relabel e) (σ.relabel e) = f ρ σ := by
  apply of_equiv_eq

--Tomamichel's "4. Positivity" theorem is implicit true in our description because w
--only allow ENNReals. The only part to prove is that "D(ρ||σ) = 0 if ρ = σ".

/-- The relative entropy is zero between any two states on a 1-
D Hilbert space. -/
private lemma wrt_self_eq_zero' [Unique d] (ρ σ : MState d) : f ρ σ = 0 := by
  convert normalized (f := f) (d := d) default
  · apply Subsingleton.allEq
  · apply Subsingleton.allEq
  · simp

/-- The relative entropy `D(ρ||ρ) = 0`. -/
@[simp]
theorem wrt_self_eq_zero (ρ : MState d) : f ρ ρ.M = 0 := by
  rw [← nonpos_iff_eq_zero, ← wrt_self_eq_zero' f (d := PUnit) default default]
  convert DPI (f := f) _ _ (CPTPMap.const_state ρ)
  · rw [CPTPMap.const_state_apply]
  · rw [CPTPMap.const_state_apply]

end possibly_trivial

section nontrivial
variable [Nontrivial f]

/-- A nontrivial relative entropy is faithful, it can distinguish when two state

```

```

/
theorem faithful (ρ σ : MState d) : f ρ σ = 0 ↔ ρ = σ := by
  sorry

end nontrivial

section bounds

open Prob in
/-- Quantum relative min-entropy. -/
def min (ρ : MState d) (σ : HermitianMat d ℂ) : ENNReal :=
  -log ⟨_, ρ.exp_val_prob ⟨proj_le_nonneg 0 σ, proj_le_le_one _ _⟩⟩

@[aesop (rule_sets := [finiteness]) simp]
theorem min_eq_top_iff (ρ : MState d) (σ : HermitianMat d ℂ) :
  (min ρ σ) = 1 ↔ ρ.M.support ≤ σ.ker := by
  open scoped HermitianMat in
  have h₂ : {0 ≤ₚ σ}.ker = σ.ker := by
    sorry --missing simp lemma
  simp [min, Prob.negLog_eq_top_iff, MState.exp_val_eq_zero_iff, Subtype.ex

open scoped HermitianMat in
protected theorem toReal_min (ρ : MState d) (σ : HermitianMat d ℂ) :
  (min ρ σ).toReal = -Real.log (ρ.exp_val {0 ≤ₚ σ}) :=
  Prob.negLog_pos_Real

/-- Min-relative entropy is a valid entropy function, albeit trivial (and n
/
instance : RelEntropy min where
  DPI := sorry
  of_kron := sorry
  normalized := sorry

theorem not_Nontrivial_min : ¬Nontrivial min := by
  rintro ⟨h⟩
  obtain ⟨ρ, σ, h₁, h₂, h₃⟩ := h (ULift (Fin 2))
  replace h₂ : proj_le 0 σ = (1 : HermitianMat (ULift (Fin 2)) ℂ) := by
    sorry--TODO
  simp [min, Subtype.ext_iff, MState.exp_val_eq_one_iff, proj_le_le_one, h₁

/-- The relative min-entropy is a lower bound on all relative entropies. -
/
theorem min_le (ρ σ : MState d) : min ρ σ ≤ f ρ σ := by
  sorry --Tomamichel, https://www.marcotom.info/files/entropy-masterclass2022.pdf, (1.28)

```

```

open Classical in
/-- Quantum relative max-entropy. -/
def max (p : MState d) (σ : HermitianMat d ℂ) : ENNReal :=
  if ∃ (x : ℝ), p.M ≤ Real.exp x • σ then
    some (sInf { x : NNReal | p.M ≤ Real.exp x • σ })
  else
    T

@[aesop (rule_sets := [finiteness]) simp]
protected theorem max_not_top (p : MState d) (σ : HermitianMat d ℂ) :
  (max p σ) ≠ T ↔ σ.ker ≤ p.M.ker := by
  open ComplexOrder in
  constructor
  · intro h
    contrapose! h
    simp only [max, ENNReal.some_eq_coe, ite_eq_right_iff, ENNReal.coe_ne_top, imp_f
      not_exists]
    intro x
    contrapose! h
    intro v hv
    rw [HermitianMat.ker, LinearMap.mem_ker] at hv ⊢
    replace hv : σ.toMat.mulVec v = 0 := sorry --why is this not defeq??
    replace h := h.right v
    rw [Matrix.sub_mulVec] at h
    simp [hv, Matrix.smul_mulVec_assoc] at h
    have := p.pos.right v
    -- have := le_antisymm (p.pos.right v) (by )
    sorry
  · intro
    rw [max, if_pos]
    · nofun
      sorry --log ("min nonzero eigenvalue of σ" / "max eigenvalue of p") should work

protected theorem toReal_max (p : MState d) (σ : HermitianMat d ℂ) :
  (max p σ).toReal = sInf { x : ℝ | p.M ≤ Real.exp x • σ } := by
  rw [max]
  split_ifs with h
  · have : { x : ℝ | p.M ≤ Real.exp x • σ }.Nonempty := by
      convert h
      simp
      sorry
  · push_neg at h
    simp [h]

/-- The relative max-entropy is a lower bound on all relative entropies. -
/

```

```
theorem le_max (p σ : MState d) : f p σ ≤ max p σ := by
  sorry --Tomamichel, https://www.marcotom.info/files/entropy-
  masterclass2022.pdf, (1.28)
```

```
end bounds
```

```
end RelEntropy
```

```
class Entropy (f : ∀ {d : Type u} [Fintype d] [DecidableEq d], MState d → ℝ) : Type u :=
  /-- The entropy of a pure state is zero -/
  of_const {d : Type u} [Fintype d] [DecidableEq d] (ψ : Ket d) : f (.pure ψ) = 0
  /-- Entropy is additive under tensor products -/
  of_kron {d₁ d₂ : Type u} [Fintype d₁] [Fintype d₂] [DecidableEq d₁] [DecidableEq d₂]
    (f₁ : Entropy f₁) (f₂ : Entropy f₂) : f (p ⊗ q) = f₁ p + f₂ q
  /-- Entropy is convex. TODO def? Or do we even need this? -/
  /
  -- convex : True := by trivial
```

1.421 QuantumInfo/Finite/AxiomatizedEntropy/Renyi.lean

```
/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.Entropy.Defs

/-! # Quantum Relative Entropy and α-Renyi Entropy -/

variable {d : Type*} [Fintype d] [DecidableEq d]

/-- The quantum relative entropy  $S(\rho||\sigma) = \text{Tr}[\rho (\log \rho - \log \sigma)]$ . -/
/
@[irreducible]
noncomputable def qRelativeEnt (p : MState d) (σ : HermitianMat d ℂ) : ENNR d :=
  open Classical in (if σ.ker ≤ p.M.ker then
    some (p.exp_val (HermitianMat.log p - HermitianMat.log σ),
      /- Quantum relative entropy is nonnegative. This can be proved by an application of
      Klein's inequality. -/
      sorry)
  else
    T)

notation "⊔(" p "||" σ ")" => qRelativeEnt p σ
```

```

instance : RelEntropy qRelativeEnt where
  DPI := sorry
  of_kron := sorry
  normalized := sorry

instance : RelEntropy.Nontrivial qRelativeEnt where
  nontrivial := sorry

/-- Quantum relative entropy as `Tr[p (log p - log σ)]` when supports are correct. -
/
theorem qRelativeEnt_ker {p σ : MState d} (h : σ.M.ker ≤ p.M.ker) :
  (⊔(p||σ) : EReal) = p.M.inner (HermitianMat.log p - HermitianMat.log σ) := by
  simp only [qRelativeEnt, h]
  congr

```

1.422 QuantumInfo/Finite/Bracket.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg, Rodolfo Soldati
-/
import QuantumInfo.ForMathlib
import QuantumInfo.ClassicalInfo.Distribution

/-!
Finite dimensional quantum pure states, bra and kets. Mixed states are `MState` in t

These could be done with a Hilbert space of Fintype, which would look like
```lean4
(H : Type*) [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H] [Finite
```
or by choosing a particular `Basis` and asserting it is `Fintype`. But frankly it se
mostly focus on the basis-dependent notion of `Matrix`, which has the added benefit
"classical" interpretation (as the basis elements, or diagonal elements of a mixed s
sense, this quantum theory comes with the a particular classical theory always prefe
-/

noncomputable section

open Classical
open BigOperators
open ComplexConjugate

```

```

open Kronecker
open scoped Matrix ComplexOrder

section
variable (d : Type*) [Fintype d]

/-- A ket as a vector of unit norm. We follow the convention in `Matrix` of
    from a Fintype. Kets are distinctly not a vector space in our notion, as t
    states and so cannot (in general) be added or scaled. -/
structure Ket where
  vec : d → ℂ
  normalized' : ∑ x, ||vec x|| ^ 2 = 1
  --TODO: change to `vec : EuclideanSpace ℂ d` / `normalized' : ||vec|| = 1`

/-- A bra is identical in definition to a `Ket`, but are separate to avoid
    They can be interconverted with the adjoint: `Ket.to_bra` and `Bra.to_ket`
    /
structure Bra where
  vec : d → ℂ
  normalized' : ∑ x, ||vec x|| ^ 2 = 1

end section

namespace Bracket

scoped notation:max "⊠" ψ:90 "⊠" => (ψ : Bra _)
scoped notation:max "⊠" ψ:90 "⊠" => (ψ : Ket _)

variable {d : Type*} [Fintype d]

instance instFunLikeKet : FunLike (Ket d) d ℂ where
  coe ψ := ψ.vec
  coe_injective' _ _ h := by rwa [Ket.mk.injEq]

instance instFunLikeBra : FunLike (Bra d) d ℂ where
  coe ψ := ψ.vec
  coe_injective' _ _ h := by rwa [Bra.mk.injEq]

def dot (ξ : Bra d) (ψ : Ket d) : ℂ := ∑ x, (ξ x) * (ψ x)

scoped notation "⊠" ξ:90 "⊡" ψ:90 "⊠" => dot (ξ : Bra _) (ψ : Ket _)

theorem dot_eq_dotProduct (ψ : Bra d) (φ : Ket d) : ⊠ψ⊡φ⊠= dotProduct (m :=
  rfl

```

```

end Braket

section braket
open Braket

variable {d : Type*} [Fintype d]

theorem Ket.apply (ψ : Ket d) (i : d) : ψ i = ψ.vec i :=
  rfl

theorem Bra.apply (ψ : Bra d) (i : d) : ψ i = ψ.vec i :=
  rfl

@[ext]
theorem Ket.ext {ξ ψ : Ket d} (h : ∀ x, ξ x = ψ x) : ξ = ψ :=
  DFunLike.ext ξ ψ h

@[ext]
theorem Bra.ext {ξ ψ : Bra d} (h : ∀ x, ξ x = ψ x) : ξ = ψ :=
  DFunLike.ext ξ ψ h

theorem Ket.normalized (ψ : Ket d) : ∑ x, Complex.normSq (ψ x) = 1 := by
  convert ψ.normalized'
  rw [Complex.normSq_eq_norm_sq]
  rfl

theorem Bra.normalized (ψ : Bra d) : ∑ x, Complex.normSq (ψ x) = 1 := by
  convert ψ.normalized'
  rw [Complex.normSq_eq_norm_sq]
  rfl

/-- Any Bra can be turned into a Ket by conjugating the elements. -
/
@[coe]
def Ket.to_bra (ψ : Ket d) : Bra d :=
  ⟨conj ψ, by simp using ψ.2⟩

/-- Any Ket can be turned into a Bra by conjugating the elements. -
/
@[coe]
def Bra.to_ket (ψ : Bra d) : Ket d :=
  ⟨conj ψ, by simp using ψ.2⟩

instance instBraOfKet : Coe (Ket d) (Bra d) := ⟨Ket.to_bra⟩
instance instKetOfBra : Coe (Bra d) (Ket d) := ⟨Bra.to_ket⟩

```

```

@[simp]
theorem Bra.eq_conj (ψ : Ket d) (x : d) :  $\overline{\psi x} = \text{conj } (\overline{\psi} x)$  :=
  rfl

theorem Bra.apply' (ψ : Ket d) (i : d) :  $\overline{\psi} i = \text{conj } (\psi.\text{vec } i)$  :=
  rfl

theorem Ket.exists_ne_zero (ψ : Ket d) :  $\exists x, \psi x \neq 0$  := by
  have hzerolt :  $\sum x : d, \text{Complex.normSq } (\psi x) > \sum x : d, 0$  := by rw [ψ.norm]
  have hpos :  $\exists x \in \text{Finset.univ}, 0 < \text{Complex.normSq } (\psi x)$  := Finset.exists_pos
  obtain ⟨x, _, hpos⟩ := hpos
  rw [Complex.normSq_pos] at hpos
  use x

theorem Bra.exists_ne_zero (ψ : Bra d) :  $\exists x, \psi x \neq 0$  := by
  have hzerolt :  $\sum x : d, \text{Complex.normSq } (\psi x) > \sum x : d, 0$  := by rw [ψ.norm]
  have hpos :  $\exists x \in \text{Finset.univ}, 0 < \text{Complex.normSq } (\psi x)$  := Finset.exists_pos
  obtain ⟨x, _, hpos⟩ := hpos
  rw [Complex.normSq_pos] at hpos
  use x

/-- Create a ket out of a vector given it has a nonzero component -
/
def Ket.normalize (v : d → ℂ) (h :  $\exists x, v x \neq 0$ ) : Ket d :=
  { vec := fun x ↦ v x /  $\sqrt{(\sum x : d, \|v x\|^2)}$ ,
    normalized' := by
      simp only [← Complex.normSq_eq_norm_sq, Complex.normSq_div,
        Complex.normSq_ofReal, ←sq]
      have hnonneg :  $\sum x : d, \text{Complex.normSq } (v x) \geq 0$  := Fintype.sum_nonneg
      simp only [Real.sq_sqrt hnonneg, div_eq_inv_mul, ←Finset.mul_sum]
      apply inv_mul_cancel₀
      by_contra hzero
      rw [Fintype.sum_eq_zero_iff_of_nonneg (fun x => Complex.normSq_nonneg)]
      obtain ⟨a, ha⟩ := h
      have h₁ : (fun x => Complex.normSq (v x)) a ≠ 0 := by simp only [ne_eq]
      exact h₁ (congrFun hzero a)
  }

/-- A ket is already normalized -/
theorem Ket.normalize_ket_eq_self (ψ : Ket d) : Ket.normalize (ψ.vec) (Ket.ext x)
  unfold normalize
  simp only [apply, ψ.normalized', Real.sqrt_one, Complex.ofReal_one, div_one]

/-- Create a bra out of a vector given it has a nonzero component -

```



```

/
def Bra.normalize (v : d → ℂ) (h : ∃ x, v x ≠ 0) : Bra d :=
{ vec := fun x ↦ v x / √(∑ x : d, ||v x|| ^ 2),
  normalized' := by
    simp only [← Complex.normSq_eq_norm_sq, Complex.normSq_div,
      Complex.normSq_ofReal, ←sq]
    have hnonneg : ∑ x : d, Complex.normSq (v x) ≥ 0 := Fintype.sum_nonneg (fun x
    simp only [Real.sq_sqrt hnonneg, div_eq_inv_mul, ←Finset.mul_sum]
    apply inv_mul_cancel₀
    by_contra hzero
    rw [Fintype.sum_eq_zero_iff_of_nonneg (fun x => Complex.normSq_nonneg (v x))]
    obtain ⟨a, ha⟩ := h
    have h₁ : (fun x => Complex.normSq (v x)) a ≠ 0 := by simp only [ne_eq, map_eq]
    exact h₁ (congrFun hzero a)
}

/-- A bra is already normalized -/
def Bra.normalize_ket_eq_self (ψ : Bra d) : Bra.normalize (ψ.vec) (Bra.exists_ne_zero
ext x
  unfold normalize
  simp only [apply, ψ.normalized', Real.sq_sqrt_one, Complex.ofReal_one, div_one]

/-- Ket form by the superposition of all elements in `d`.
Commonly denoted by |+⟩, especially for qubits -/
def uniform_superposition [hdne : Nonempty d] : Ket d := by
  let f : d → ℂ := fun _ ↦ 1
  have hfnezero : ∃ x, f x ≠ 0 := by
    obtain ⟨i⟩ := hdne
    use i
    simp only [f, ne_eq, one_ne_zero, not_false_eq_true]
  exact Ket.normalize f hfnezero

/-- There exists a ket for every nonempty `d`.
Here, we use the uniform superposition -/
instance instInhabited [Nonempty d] : Inhabited (Ket d) where
  default := uniform_superposition

/-- Construct the Ket corresponding to a basis vector, with a +1 phase. -
/
def Ket.basis (i : d) : Ket d :=
  ⟨fun j ↦ if i = j then 1 else 0, by simp [apply_ite]⟩

/-- Construct the Bra corresponding to a basis vector, with a +1 phase. -
/
def Bra.basis (i : d) : Bra d :=
  ⟨fun j ↦ if i = j then 1 else 0, by simp [apply_ite]⟩

```

```

/-- A Bra can be viewed as a function from Ket's to  $\mathbb{C}$ . -/
instance instFunLikeBraket : FunLike (Bra d) (Ket d)  $\mathbb{C}$  where
  coe  $\xi$  := dot  $\xi$ 
  coe_injective' x y h := by
    ext i
    simp [Ket.basis, dot, Ket.apply] using congrFun h (Ket.basis i)

/-- The inner product of any state with itself is 1. -/
@[simp]
theorem Braket.dot_self_eq_one ( $\psi$  : Ket d) :  $\langle \psi | \psi \rangle = 1$  := by
  have h1 x : conj ( $\psi$  x) *  $\psi$  x = Complex.normSq ( $\psi$  x) := by
    rw [Complex.normSq_eq_conj_mul_self]
  simp [dot, Bra.eq_conj, h1] using congr(Complex.ofReal  $\psi$ .normalized)

section prod
variable {d d1 d2 : Type*} [Fintype d] [Fintype d1] [Fintype d2]

/-- The outer product of two kets, creating an unentangled state. -
/
def Ket.prod ( $\psi_1$  : Ket d1) ( $\psi_2$  : Ket d2) : Ket (d1  $\times$  d2) where
  vec := fun (i,j)  $\mapsto$   $\psi_1$  i *  $\psi_2$  j
  normalized' := by
    simp [Fintype.sum_prod_type,  $\leftarrow$  Complex.normSq_eq_norm_sq, mul_pow,
       $\leftarrow$  Finset.mul_sum,  $\psi_1$ .normalized,  $\psi_2$ .normalized]

infixl:100 "  $\otimes$  " => Ket.prod

/-- A Ket is a product if it's `Ket.prod` of two kets. -/
def Ket.IsProd ( $\psi$  : Ket (d1  $\times$  d2)) : Prop :=  $\exists \xi \phi, \psi = \xi \otimes \phi$ 

/-- A Ket is entangled if it's not `Ket.prod` of two kets. -
/
def Ket.IsEntangled ( $\psi$  : Ket (d1  $\times$  d2)) : Prop :=  $\neg \psi$ .IsProd

/-- `Ket.prod` states are product states. -/
@[simp]
theorem Ket.IsProd_prod ( $\psi_1$  : Ket d1) ( $\psi_2$  : Ket d2) : ( $\psi_1$ .prod  $\psi_2$ ).IsProd :
  ( $\psi_1, \psi_2$ , rfl)

/-- `Ket.prod` states are not entangled states. -/
@[simp]
theorem Ket.not_IsEntangled_prod ( $\psi_1$  : Ket d1) ( $\psi_2$  : Ket d2) :  $\neg(\psi_1$ .prod  $\psi_2$ 
  ( $\cdot$  ( $\psi_1$ .IsProd_prod  $\psi_2$ )))

/-- A ket is a product state iff its components are cross-

```

multiplicative. -/

```

theorem Ket.IsProd_iff_mul_eq_mul (ψ : Ket (d₁ × d₂)) : ψ.IsProd ↔
  ∀ i₁ i₂ j₁ j₂, ψ (i₁, j₁) * ψ (i₂, j₂) = ψ (i₁, j₂) * ψ (i₂, j₁) := by
  constructor
  · rintro (ξ, ϕ, rfl) i₁ i₂ j₁ j₂
    simp only [prod, apply]
    ring_nf
  · intro hcrossm
    obtain ⟨(a, b), hψnonZero⟩ := Ket.exists_ne_zero ψ
    -- May be able to simplify proof below by using Ket.normalize
    let v₁ : d₁ → ℂ := fun x => ||ψ (a, b)|| / (ψ (a, b)) * ((ψ (x, b)) / √(∑ i : d₁,
    let v₂ : d₂ → ℂ := fun y => ψ (a, y) / √(∑ j : d₂, ||ψ (a, j)||^2)
    have hv1Norm : ∑ x, ||v₁ x||^2 = 1 := by
      simp only [← Complex.normSq_eq_norm_sq, v₁, Complex.normSq_mul, Complex.normSq
      Complex.normSq_ofReal, ← sq]
      rw [div_self _]
      have hnonneg : ∑ i : d₁, Complex.normSq (ψ (i, b)) ≥ 0 := Fintype.sum_nonneg (
      · simp_rw [Real.sq_sqrt hnonneg, one_mul, div_eq_inv_mul, ← Finset.mul_sum]
        apply inv_mul_cancel₀
        by_contra hzero
        rw [Fintype.sum_eq_zero_iff_of_nonneg (fun i => Complex.normSq_nonneg (ψ (i,
        have h₁ : (fun i => Complex.normSq (ψ (i, b))) a ≠ 0 := by simp only [ne_eq, m
        rw [hzero, Pi.zero_apply, ne_eq, eq_self, not_true_eq_false] at h₁
        exact h₁
      · simp_all only [ne_eq, map_eq_zero, not_false_eq_true]
    have hv2Norm : ∑ x, ||v₂ x||^2 = 1 := by
      simp only [← Complex.normSq_eq_norm_sq, v₂, Complex.normSq_div,
      Complex.normSq_ofReal, ← sq]
      have hnonneg : ∑ j : d₂, Complex.normSq (ψ (a, j)) ≥ 0 := Fintype.sum_nonneg (
      simp_rw [Real.sq_sqrt hnonneg, div_eq_inv_mul, ← Finset.mul_sum]
        apply inv_mul_cancel₀
        by_contra hzero
        rw [Fintype.sum_eq_zero_iff_of_nonneg (fun j => Complex.normSq_nonneg (ψ (a, j
        have h₁ : (fun j => Complex.normSq (ψ (a, j))) b ≠ 0 := by simp only [ne_eq, m
        rw [hzero, Pi.zero_apply, ne_eq, eq_self, not_true_eq_false] at h₁
        exact h₁
    let ψ₁ : Ket d₁ := ⟨v₁, hv1Norm⟩
    let ψ₂ : Ket d₂ := ⟨v₂, hv2Norm⟩
    use ψ₁, ψ₂
    ext ⟨x, y⟩
    have hψfun : ψ (x, y) = (ψ (x, b) * ψ (a, y)) / ψ (a, b) := eq_div_of_mul_eq hψnon
    have hψnorm : (∑ z : d₁ × d₂, Complex.normSq (ψ.vec (z.1, b) * ψ.vec (a, z.2)))
    calc
      ∑ z : d₁ × d₂, Complex.normSq (ψ.vec (z.1, b) * ψ.vec (a, z.2)) =
      ∑ z : d₁ × d₂, Complex.normSq (ψ.vec (a, b) * ψ.vec (z.1, z.2)) := by simp o
      _ = ∑ z : d₁ × d₂, Complex.normSq (ψ.vec (a, b)) * Complex.normSq (ψ.vec (z.1,

```

```

    _ = Complex.normSq (ψ.vec (a, b)) * ∑ z : d1 × d2, Complex.normSq (ψ
    _ = Complex.normSq (ψ.vec (a, b)) := by simp only [← apply, ψ.normali
simp [prod, apply, ψ1, ψ2, v1, v2]
rw [mul_assoc, ←mul_div_mul_comm, ←Complex.ofReal_mul, ←Real.sqrt_mul (
.
  simp_rw [Fintype.sum_mul_sum, ←Fintype.sum_prod_type']
  simp only [← Complex.normSq_eq_norm_sq]
  simp_rw [Fintype.sum_congr _ _ (fun z : d1 × d2 => (Complex.normSq_mu
  simp_rw [hψnorm, Complex.normSq_eq_norm_sq, Real.sqrt_sq_eq_abs, abs_
  ring_nf
  rw [mul_comm, ←mul_assoc, ←mul_assoc, ←mul_assoc]
  nth_rw 2 [←inv_inv (Complex.ofReal (||ψ.vec (a, b)||))]
  rw [Complex.mul_inv_cancel _]
  · rw [one_mul]
    ring_nf at hψfun
    simp_rw [Ket.apply, mul_comm, mul_comm (ψ.vec (a, y)) _, ←mul_assoc
    exact hψfun
  · simp_all [Ket.apply]
  · simp
end prod

section mes
/-- The Maximally Entangled State, or MES, on a d×d system. In principle th
is specifically the MES with an all-positive phase. For instance on `d := F
Bell state. -/
def Ket.MES (d) [Fintype d] [Nonempty d] : Ket (d × d) where
  vec := fun (i,j) ↦ if i = j then 1 / Real.sqrt (Fintype.card (α := d)) el
  normalized' := by
    simp [apply_ite, Fintype.sum_prod_type]

/-- On any space of dimension at least two, the maximally entangled state `
/
theorem Ket.MES_isEntangled [Nontrivial d] : (Ket.MES d).IsEntangled := by
  obtain ⟨x, y, h⟩ := @Nontrivial.exists_pair_ne d _
  rw [IsEntangled, MES, IsProd_iff_mul_eq_mul]
  push_neg
  use x, y, x, y
  simp [apply, h]

/-- The transpose trick -/
theorem transposeTrick {d} [Fintype d] [Nonempty d] [DecidableEq d] {M : Ma
  (M ⊗k 1) *v (Ket.MES d).vec = (1 ⊗k M.transpose) *v (Ket.MES d).vec :=
  ext i
  simp only [Ket.MES, Matrix.mulVec, dotProduct, Matrix.kroneckerMap, Matri
  simp
  conv =>

```

```

    enter [1, 2, a]
    equals if (i.2, i.2) = a then (M i.1 i.2) / √(Fintype.card d) else 0 =>
      grind
conv =>
  enter [2, 2, a]
  equals if (i.1, i.1) = a then (M i.1 i.2) / √(Fintype.card d) else 0 =>
    grind
simp
end mes

section equiv

/-- The equivalence relation on `Ket` where two kets equivalent if they are equal up
global phase, i.e.  $\exists z, \|z\| = 1 \wedge a.\text{vec} = z \cdot b.\text{vec}$  -/
def Ket.PhaseEquiv : Setoid (Ket d) where
  r a b :=  $\exists z : \mathbb{C}, \|z\| = 1 \wedge a.\text{vec} = z \cdot b.\text{vec}$ 
  iseqv := {
    refl := fun x ↦ ⟨1, by simp⟩,
    symm := fun ⟨z, h₁, h₂⟩ ↦ ⟨conj z,
      by simp [h₁],
      by simp [h₁, h₂, smul_smul, ← Complex.normSq_eq_conj_mul_self, Complex.normSq_
    trans := fun ⟨z₁, h₁₁, h₁₂⟩ ⟨z₂, h₂₁, h₂₂⟩ ↦ ⟨z₁ * z₂,
      by simp [h₁₁, h₂₁],
      by simp [h₁₂, h₂₂, smul_smul]⟩
  }

variable (d) in
/-- The type of `Ket`s up to a global phase equivalence, as given by `Ket.PhaseEquiv`
In particular, `MState`s really only care about a KetUpToPhase, and not Kets themselves
-/
def KetUpToPhase :=
  @Quotient (Ket d) Ket.PhaseEquiv

/-- Construct a `KetUpToPhase` from a `Ket`. -/
def KetUpToPhase.mk (ψ : Ket d) : KetUpToPhase d :=
  @Quotient.mk _ Ket.PhaseEquiv ψ

/-- Lift a function on `Ket d` to `KetUpToPhase d`, given that it respects phase equivalence
-/
def KetUpToPhase.lift {α : Sort*} (f : Ket d → α)
  (hf : ∀ ψ φ, Ket.PhaseEquiv.r ψ φ → f ψ = f φ) : KetUpToPhase d → α :=
  @Quotient.lift _ _ Ket.PhaseEquiv f hf

@[simp]
theorem KetUpToPhase.lift_mk {α : Sort*} (f : Ket d → α)

```

```

(hf :  $\forall \psi \phi$ , Ket.PhaseEquiv.r  $\psi \phi \rightarrow f \psi = f \phi$ ) ( $\psi$  : Ket d) :
KetUpToPhase.lift f hf (KetUpToPhase.mk  $\psi$ ) = f  $\psi$  := rfl

theorem KetUpToPhase.ind {p : KetUpToPhase d  $\rightarrow$  Prop}
(h :  $\forall \psi$  : Ket d, p (KetUpToPhase.mk  $\psi$ )) :  $\forall q$ , p q :=
@Quotient.ind _ Ket.PhaseEquiv p h

theorem KetUpToPhase.surjective_mk : Function.Surjective (KetUpToPhase.mk (
fun q => @Quotient.ind _ Ket.PhaseEquiv (fun q =>  $\exists a$ , KetUpToPhase.mk a

end equiv
end bracket

```

1.423 QuantumInfo/Finite/CPTPMap.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.CPTPMap.Bundled
import QuantumInfo.Finite.CPTPMap.CTP
import QuantumInfo.Finite.CPTPMap.Dual
import QuantumInfo.Finite.CPTPMap.MatrixMap
import QuantumInfo.Finite.CPTPMap.Unbundled

```

1.424 QuantumInfo/Finite/CPTPMap/Bundled.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.CPTPMap.Unbundled
import QuantumInfo.Finite.MState

import Mathlib.Topology.Order.Hom.Basic

/-! # Classes of Matrix Maps

The bundled `MatrixMap`s: `HMap`, `UnitalMap`, `TMap`, `PMap`, and `CMap`
These are defined over the bare minimum rings (`Semiring` or `RCLike`, resp

```

The combinations ``PTPMap`` (positive trace-preserving), ``CPTPMap``, and ``CPUMap`` (CP unital maps) take $\mathbb{C}$ as the default class.

The majority of quantum theory revolves around ``CPTPMap``'s, so those are explored more thoroughly in their file `CPTP.lean`.

-/

```
variable (dIn dOut R : Type*) (α : Type := ℂ)
variable [Semiring R] [RCLike α]
```

```
/-- Hermitian-preserving linear maps. -/
structure HPMMap extends MatrixMap dIn dOut α where
  HP : MatrixMap.IsHermitianPreserving toLinearMap
```

```
/-- Unital linear maps. -/
structure UnitalMap [DecidableEq dIn] [DecidableEq dOut]
  extends MatrixMap dIn dOut R where
  unital : MatrixMap.Unital toLinearMap
```

```
/-- Trace-preserving linear maps. -/
structure TPMMap [Fintype dIn] [Fintype dOut] extends MatrixMap dIn dOut R where
  TP : MatrixMap.IsTracePreserving toLinearMap
```

```
--Mark this as [simp] so that simp lemmas requiring `IsTracePreserving` can pick it
--In theory this could be making "IsTracePreserving" a typeclass ... or more realistically
--defining a `TracePreservingClass` similar to `AddHomClass`
attribute [simp] TPMMap.TP
```

```
/-- Positive linear maps. -/
structure PMap [Fintype dIn] [Fintype dOut]
  extends HPMMap dIn dOut α where
  pos : MatrixMap.IsPositive toLinearMap
  HP := pos.IsHermitianPreserving
```

```
/-- Completely positive linear maps. -/
structure CPMMap [Fintype dIn] [Fintype dOut] [DecidableEq dIn]
  extends PMap dIn dOut α where
  cp : MatrixMap.IsCompletelyPositive toLinearMap
  pos := cp.IsPositive
```

```
/-- Positive trace-preserving linear maps. These includes all channels, but aren't
necessarily *completely* positive, see `CPTPMap`. -/
structure PTPMap [Fintype dIn] [Fintype dOut]
  extends PMap dIn dOut α, TPMMap dIn dOut α
```

```

/-- Positive unital maps. These are important because they are the
    dual to `PTPMap`: they are the most general way to map *observables*. -
/
structure PUMap [Fintype dIn] [Fintype dOut] [DecidableEq dIn] [DecidableEq dOut]
  extends PMap dIn dOut [], UnitalMap dIn dOut []

attribute [simp] PTPMap.TP

/-- Completely positive trace-preserving linear maps. This is the most common
    meaning of "channel", often described as "the most general physically realizable
    quantum operation". -/
structure CPTPMap [Fintype dIn] [Fintype dOut] [DecidableEq dIn] [DecidableEq dOut]
  extends PTPMap dIn dOut ([] := []), CPMMap dIn dOut [] where

/-- Completely positive unital maps. These are important because they are the
    dual to `CPTPMap`: they are the physically realizable ways to map *observables*. -
/
structure CPUMap [Fintype dIn] [Fintype dOut] [DecidableEq dIn] [DecidableEq dOut]
  extends CPMMap dIn dOut [], PUMap dIn dOut []

variable {dIn dOut R} {α : Type} [RCLike α]

--Hermitian-preserving maps: continuous linear maps on HermitianMats.
namespace HPMMap
variable {Λ₁ Λ₂ : HPMMap dIn dOut α}
variable {CΛ₁ CΛ₂ : HPMMap dIn dOut ℂ}

abbrev map (M : HPMMap dIn dOut α) : MatrixMap dIn dOut α := M.toLinearMap

@[ext]
theorem ext (h : Λ₁.map = Λ₂.map) : Λ₁ = Λ₂ := by
  rwa [HPMMap.mk.injEq]

/-- Two maps are equal if they agree on all Hermitian inputs. -
/
theorem funext_hermitian (h : ∀ M : HermitianMat dIn ℂ, CΛ₁.map M = CΛ₂.map M) :
  CΛ₁ = CΛ₂ := by
  ext M : 2
  have hH := h (realPart M)
  have hA := h (imaginaryPart M)
  convert congr($hH + Complex.I • $hA)
  <=> rw (occs := [1]) [← realPart_add_I_smul_imaginaryPart M, map_add, map_smul]
  <=> rfl

/-- Two maps are equal if they agree on all positive inputs. -
/

```



```

theorem funext_pos [Fintype dIn] (h :  $\forall M : \text{HermitianMat } dIn \mathbb{C}, 0 \leq M \rightarrow C\Lambda_1.\text{map } M = C\Lambda_2.\text{map } M$ ) :
  C $\Lambda_1 = C\Lambda_2 :=$  by
  classical
  open scoped HermitianMat in
  apply funext_hermitian
  intro M
  rw [← M.posPart_add_negPart]
  simp [HermitianMat.posPart_nonneg, HermitianMat.negPart_nonneg, h]

/-- Two maps are equal if they agree on all positive inputs with trace one -
/
theorem funext_pos_trace [Fintype dIn]
(h :  $\forall M : \text{HermitianMat } dIn \mathbb{C}, 0 \leq M \rightarrow M.\text{trace} = 1 \rightarrow C\Lambda_1.\text{map } M = C\Lambda_2.\text{map } M$ ) :
  C $\Lambda_1 = C\Lambda_2 :=$  by
  apply funext_pos
  intro M hM'
  rcases hM'.eq_or_lt with rfl | hM
  · simp
  have h_tr :  $0 < M.\text{trace} := M.\text{trace\_pos } hM$ 
  have := h (M.trace-1 • M) ?_ ?_
  · simp only [HermitianMat.mat_smul, LinearMap.map_smul_of_tower] at this
    convert congr(M.trace • $this)
    · rw [smul_smul]
      field_simp
      simp
    · rw [smul_smul]
      field_simp
      simp
  · apply smul_nonneg (by positivity) hM'
  · simp [field]

/-- Two maps are equal if they agree on all `MState`s. -/
theorem funext_mstate [Fintype dIn] [DecidableEq dIn] { $\Lambda_1 \Lambda_2 : \text{HPMap } dIn dOut \mathbb{C}$ }
(h :  $\forall \rho : \text{MState } dIn, \Lambda_1.\text{map } \rho.m = \Lambda_2.\text{map } \rho.m$ ) :
   $\Lambda_1 = \Lambda_2 :=$ 
  funext_pos_trace fun M hM_pos hM_tr  $\mapsto h \langle M, hM\_pos, hM\_tr \rangle$ 

/-- Hermitian-preserving maps are functions from `HermitianMat`s to `HermitianMat`s.
/
instance instFunLike : FunLike (HPMap dIn dOut  $\mathbb{C}$ ) (HermitianMat dIn  $\mathbb{C}$ ) (HermitianMat dOut  $\mathbb{C}$ )
  coe  $\Lambda \rho := \langle \Lambda.\text{map } \rho.1, \Lambda.HP \rho.2 \rangle$ 
  coe_injective' x y h := funext_hermitian fun M  $\mapsto$ 
    by simpa using congrFun h M

instance [Fintype dIn] : ContinuousLinearMapClass
  (HPMap dIn dOut  $\mathbb{C}$ )  $\mathbb{R}$  (HermitianMat dIn  $\mathbb{C}$ ) (HermitianMat dOut  $\mathbb{C}$ ) where

```

```

    map_add f x y := HermitianMat.ext <| LinearMap.map_add f.toLinearMap x y
    map_smulsl f c x := HermitianMat.ext <| by simp [instFunLike]
    map_continuous f := .subtype_mk (by fun_prop) _

end HPMMap

variable [Fintype dIn] [Fintype dOut]

--Positive-preserving maps: continuous linear order-preserving maps on Herm
namespace PMap

@[ext]
theorem ext {Λ1 Λ2 : PMap dIn dOut []} (h : Λ1.map = Λ2.map) : Λ1 = Λ2 := by
  rw [PMap.mk.injEq]
  exact HPMMap.ext h

theorem injective_toHPMap : (PMap.toHPMap (dIn := dIn) (dOut := dOut) (□ :=
  fun _ _ ↦ (mk.injEq _ _ _ _).mpr

/-- Positive maps are functions from `HermitianMat`s to `HermitianMat`s. -
/
instance instFunLike : FunLike (PMap dIn dOut ℂ) (HermitianMat dIn ℂ) (Herm
  coe := DFunLike.coe ∘ toHPMap
  coe_injective' := DFunLike.coe_injective'.comp injective_toHPMap

set_option synthInstance.maxHeartbeats 40000 in
instance instLinearMapClass : LinearMapClass (PMap dIn dOut ℂ) ℝ (Hermitian
  map_add f x y := HermitianMat.ext <| LinearMap.map_add f.toLinearMap x y
  map_smulsl f c x := HermitianMat.ext <| by simp [instFunLike]

instance instContinuousOrderHomClass : ContinuousOrderHomClass (PMap dIn dO
  (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
  map_continuous f := ContinuousMapClass.map_continuous f.toHPMap
  map_monotone f x y h := by
    simpa using f.pos h

/-- Positive-preserving maps also preserve positivity on, specifically, Herm
/
@[simp]
theorem pos_Hermitian (M : PMap dIn dOut ℂ) {x : HermitianMat dIn ℂ} (h : 0
  simpa only [map_zero] using ContinuousOrderHomClass.map_monotone M h

end PMap

namespace CPMMap

```

```

def of_kraus_CPMMap {κ : Type*} [Fintype κ] [DecidableEq dIn] (M : κ → Matrix dOut dIn) : CPMMap dIn dOut :=
  toLinearMap := MatrixMap.of_kraus M M
  cp := MatrixMap.of_kraus_isCompletelyPositive M

end CPMMap

--Positive trace-preserving maps:
-- * Continuous linear order-preserving maps on HermitianMats.
-- * Continuous maps on MStates.
namespace PTPMap

@[ext]
theorem ext {Λ₁ Λ₂ : PTPMap dIn dOut ℂ} (h : Λ₁.map = Λ₂.map) : Λ₁ = Λ₂ := by
  rw [PTPMap.mk.injEq]
  exact PMap.ext h

theorem injective_toPMap : (PTPMap.toPMap (dIn := dIn) (dOut := dOut) (ℂ := ℂ)).InjEq :=
  fun _ _ => (mk.injEq _ _ _).mpr

/-- Positive trace-preserving maps are functions from `HermitianMat`s to `HermitianMat`s. -
/
instance instFunLike : FunLike (PTPMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) :=
  { coe := DFunLike.coe ∘ toPMap
    coe_injective' := DFunLike.coe_injective'.comp injective_toPMap }

instance instLinearMapClass : LinearMapClass (PTPMap dIn dOut ℂ) ℝ (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) :=
  { map_add f x y := by simp [instFunLike]
    map_smulₛₗ f c x := by simp [instFunLike] }

instance instHContinuousOrderHomClass : ContinuousOrderHomClass (PTPMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
  map_continuous f := ContinuousMapClass.map_continuous f.toPMap
  map_monotone f x y h := by
    simp using f.pos h

/-- PTP maps also preserve positivity on Hermitian matrices. -
/
@[simp]
theorem pos_Hermitian (M : PTPMap dIn dOut ℂ) {x : HermitianMat dIn ℂ} (h : 0 ≤ x) : 0 ≤ M x :=
  simp only [map_zero] using ContinuousOrderHomClass.map_monotone M h

/-- `PTPMap`s are functions from `MState`s to `MState`s. -
/
instance instMFunLike [DecidableEq dIn] [DecidableEq dOut] :
  FunLike (PTPMap dIn dOut) (MState dIn) (MState dOut) where
  coe Λ ρ := MState.mk

```

```

      (Λ.toHPMap ρ.M) (HermitianMat.zero_le_iff.mpr (Λ.pos ρ.psd)) (by
        rw [HermitianMat.trace_eq_one_iff, ← ρ.tr']
        exact Λ.TP ρ)
    coe_injective' x y h := injective_toPMap <| PMap.injective_toHPMap <|
      HPMap.funext_mstate fun ρ ↦ by
        have := congr($h ρ);
        rwa [MState.ext_iff, HermitianMat.ext_iff] at this

instance instMContinuousMapClass [DecidableEq dIn] [DecidableEq dOut] :
  ContinuousMapClass (PTPMap dIn dOut) (MState dIn) (MState dOut) where
  map_continuous f := by
    rw [continuous_induced_rng]
    exact (map_continuous f.toHPMap).comp MState.Continuous_HermitianMat

-- @[norm_cast]
theorem val_apply_MState [DecidableEq dIn] (M : PTPMap dIn dOut) (ρ : MState dIn)
  (M ρ : HermitianMat dOut ℂ) = (instFunLike.coe M) ρ := by
  rfl

-- If we have a PTPMap, the input and output dimensions are always both none
-- we can't preserve trace) - or they're both empty. So `[Nonempty dIn]` will
-- This would be nice as an `instance` but that would leave `dIn` as a meta
theorem nonemptyOut (Λ : PTPMap dIn dOut) [hIn : Nonempty dIn] [DecidableEq dIn]
  by_contra h
  simp only [not_nonempty_iff] at h
  let M := (1 : Matrix dIn dIn ℂ)
  have := calc (Finset.univ.card (α := dIn) : ℂ)
    _ = M.trace := by simp [Matrix.trace, M]
    _ = (Λ.map M).trace := (Λ.TP M).symm
    _ = 0 := by simp only [Matrix.trace_eq_zero_of_isEmpty]
  norm_num [Finset.univ_eq_empty_iff] at this

end PTPMap

namespace CPTPMap
variable [DecidableEq dIn]

/-- Two `CPTPMap`s are equal if their `MatrixMap`s are equal. -
/
@[ext]
theorem ext {Λ₁ Λ₂ : CPTPMap dIn dOut []} (h : Λ₁.map = Λ₂.map) : Λ₁ = Λ₂ :=
  rw [CPTPMap.mk.injEq]
  exact PTPMap.ext h

theorem injective_toPTPMap : (CPTPMap.toPTPMap (dIn := dIn) (dOut := dOut))
  fun _ _ ↦ (mk.injEq _ _ _ _).mpr

```

```

-- /-- Positive trace-preserving maps are functions from `HermitianMat`s to `HermitianMat`s. -
-- /
-- instance instFunLike : FunLike (CPTPMap dIn dOut []) (HermitianMat dIn []) (HermitianMat dOut [])
--   coe := DFunLike.coe ∘ toPTPMap
--   coe_injective' := DFunLike.coe_injective'.comp injective_toPTPMap

-- set_option synthInstance.maxHeartbeats 40000 in
-- instance instLinearMapClass : LinearMapClass (CPTPMap dIn dOut []) ℝ (HermitianMat dIn [])
--   map_add f x y := by simp [instFunLike]
--   map_smul s x f c x := by simp [instFunLike]

-- instance instContinuousOrderHomClass : ContinuousOrderHomClass (CPTPMap dIn dOut [])
--   (HermitianMat dIn []) (HermitianMat dOut []) where
--   map_continuous f := ContinuousMapClass.map_continuous f.toPMap
--   map_monotone f x y h := by
--     simp using f.pos h

-- /-- PTP maps also preserve positivity on Hermitian matrices. -
-- /
-- @[simp]
-- theorem pos_Hermitian (M : CPTPMap dIn dOut []) {x : HermitianMat dIn []} (h : 0 ≤ x) :
--   0 ≤ M x := by
--     simp only [map_zero] using ContinuousOrderHomClass.map_monotone M h

/-- `CPTPMap`s are functions from `MState`s to `MState`s. -
-
instance instMFunLike [DecidableEq dOut] : FunLike (CPTPMap dIn dOut) (MState dIn) (MState dOut)
  coe := DFunLike.coe ∘ toPTPMap
  coe_injective' := DFunLike.coe_injective'.comp injective_toPTPMap

-- @[norm_cast]
-- theorem val_apply_MState [DecidableEq dOut] (M : CPTPMap dIn dOut) (ρ : MState dIn) :
--   (M ρ : HermitianMat dOut ℂ) = (instFunLike.coe M) ρ := by
--     rfl

@[simp]
theorem IsTracePreserving (Λ : CPTPMap dIn dOut []) : Λ.map.IsTracePreserving :=
  Λ.TP

def of_kraus_CPTPMap {κ : Type*} [Fintype κ] [DecidableEq dIn]
  (M : κ → Matrix dOut dIn ℂ)
  (hTP : (∑ k, (M k).conjTranspose * (M k)) = 1) : CPTPMap dIn dOut [] where
  toLinearMap := MatrixMap.of_kraus M M
  cp := MatrixMap.of_kraus_isCompletelyPositive M
  TP := MatrixMap.IsTracePreserving.of_kraus_isTracePreserving M M hTP

```

end CPTPMap

namespace PUMap

variable [DecidableEq dIn] [DecidableEq dOut]

@[ext]

theorem ext { $\Lambda_1 \Lambda_2$: PUMap dIn dOut $\mathbb{C}$ } (h : Λ_1 .map = Λ_2 .map) : $\Lambda_1 = \Lambda_2$:= by
 rw [PUMap.mk.injEq]
 exact PMap.ext h

theorem injective_toPMap : (PUMap.toPMap (dIn := dIn) (dOut := dOut)) ($\square$:=
 intro
 rwa [PUMap.mk.injEq]

/-- `PUMap`s are functions from `HermitianMat`s to `HermitianMat`s. -
 /

instance instFunLike : FunLike (PUMap dIn dOut $\mathbb{C}$) (HermitianMat dIn $\mathbb{C}$) (HermitianMat dOut $\mathbb{C}$) where
 coe Λ := Λ .toPMap
 coe_injective' := (DFunLike.coe_injective' (F := PMap dIn dOut $\mathbb{C}$)).comp id

instance instLinearMapClass : LinearMapClass (PUMap dIn dOut $\mathbb{C}$) $\mathbb{R}$ (HermitianMat dIn $\mathbb{C}$) (HermitianMat dOut $\mathbb{C}$) where
 map_add f x y := HermitianMat.ext <| LinearMap.map_add f.toLinearMap x y
 map_smul_{sc} f c x := HermitianMat.ext <| by simp [instFunLike]

instance instHContinuousOrderHomClass : ContinuousOrderHomClass (PUMap dIn dOut $\mathbb{C}$) (HermitianMat dIn $\mathbb{C}$) (HermitianMat dOut $\mathbb{C}$) where
 map_continuous f := ContinuousMapClass.map_continuous f.toPMap
 map_monotone f x y h := by
 simpa using f.pos h

instance instOneHomClass : OneHomClass (PUMap dIn dOut $\mathbb{C}$) (HermitianMat dIn $\mathbb{C}$) (HermitianMat dOut $\mathbb{C}$) where
 map_one f := HermitianMat.ext (f.unital)

/-- CPTP maps also preserve positivity on Hermitian matrices. -
 /

@[simp]

theorem pos_Hermitian (M : PUMap dIn dOut $\mathbb{C}$) {x : HermitianMat dIn $\mathbb{C}$ } (h :
 simpa only [map_zero] using ContinuousOrderHomClass.map_monotone M h

end PUMap

namespace CPUMap

variable [DecidableEq dIn] [DecidableEq dOut]

@[ext]

```

theorem ext {Λ₁ Λ₂ : CPUMap dIn dOut ℂ} (h : Λ₁.map = Λ₂.map) : Λ₁ = Λ₂ := by
  rw [CPUMap.mk.injEq, CPMAP.mk.injEq]
  exact PMap.ext h

theorem injective_toPMap : (CPMap.toPMap ∘ CPUMap.toCPMap (dIn := dIn) (dOut := dOut))
  intro
  rwa [CPUMAP.mk.injEq, CPMAP.mk.injEq]

/-- `CPUMap`s are functions from `HermitianMat`s to `HermitianMat`s. -
/
instance instFunLike : FunLike (CPUMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
  coe Λ := Λ.toPMap
  coe_injective' := (DFunLike.coe_injective' (F := PMap dIn dOut ℂ)).comp injective_toPMap

instance instLinearMapClass : LinearMapClass (CPUMap dIn dOut ℂ) ℝ (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
  map_add f x y := HermitianMat.ext <| LinearMap.map_add f.toLinearMap x y
  map_smul f c x := HermitianMat.ext <| by simp [instFunLike]

instance instHContinuousOrderHomClass : ContinuousOrderHomClass (CPUMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
  map_continuous f := ContinuousMapClass.map_continuous f.toPMap
  map_monotone f x y h := by
    simp using f.pos h

instance instOneHomClass : OneHomClass (CPUMap dIn dOut ℂ) (HermitianMat dIn ℂ) (HermitianMat dOut ℂ) where
  map_one f := HermitianMat.ext (f.unital)

/-- CPTP maps also preserve positivity on Hermitian matrices. -
/
@[simp]
theorem pos_Hermitian (M : CPUMap dIn dOut ℂ) {x : HermitianMat dIn ℂ} (h : 0 ≤ x) :
  simp only [map_zero] using ContinuousOrderHomClass.map_monotone M h

end CPUMap

--Tests to make sure that our `simp`s and classes are all working like we want them

section test
variable [DecidableEq dIn] [DecidableEq dOut]

#guard_msgs in
example (M : HPMAP dIn dOut ℂ) : (M (Real.pi • 1)) = Real.pi • M 1 := by simp

#guard_msgs in
example (M : PTPMap dIn dOut ℂ) : (M.toHPMap (Real.pi • 1)) = Real.pi • M.toHPMap 1

```

```
#guard_msgs in
example (M : CPTPMap dIn dOut  $\square$ ) (p : Matrix dIn dIn  $\square$ ) : (M.map p).trace =

#guard_msgs in
example (M : CPUMap dIn dOut  $\mathbb{C}$ ) (T : HermitianMat dIn  $\mathbb{C}$ ) : M (1 + 2 • T) =

end test
```

1.425 QuantumInfo/Finite/CPTPMap/CPTP.lean

```
/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.CPTPMap.Bundled
import QuantumInfo.Finite.Unitary

/-! # Completely Positive Trace Preserving maps

A `CPTPMap` is a  $\mathbb{C}$ -linear map between matrices (`MatrixMap` is an alias),
`IsCompletelyPositive` and `IsTracePreserving`. CPTP maps are typically reg
quantum operation", as they map density matrices (`MState`s) to density mat
for maps that are positive (but not necessarily completely positive) is als

A large portion of the theory is in terms of the Choi matrix (`MatrixMap.ch
positive-definiteness of this matrix corresponds to being a CP map. This is
[Choi's theorem on CP maps](https://en.wikipedia.org/wiki/Choi%27s_theorem_

This file also defines several important examples of, classes of, and opera
* `compose`: Composition of maps
* `id`: The identity map
* `replacement`: The replacement channel that always outputs the same stat
* `prod`: Tensor product of two CPTP maps, with notation  $M_1 \otimes M_2$ 
* `piProd`: Tensor product of finitely many CPTP maps (Pi-
type product)
* `of_unitary`: The CPTP map corresponding to a unitary operation `U`
* `IsUnitary`: Predicate whether the map corresponds to any unitary
* `purify`: Purifying a channel into a unitary on a larger Hilbert space
* `complementary`: The complementary channel to its purification
* `IsEntanglementBreaking`, `IsDegradable`, `IsAntidegradable`: Entangleme
and antidegradable channels.
* `SWAP`, `assoc`, `assoc'`, `traceLeft`, `traceRight`: The CPTP maps corr
```



```

    operations on states. These correspond directly to `MState.SWAP`, `MState.assoc`,
    `MState.traceLeft`, and `MState.traceRight`.
-/

variable {dIn dOut dOut₂ : Type*} [Fintype dIn] [Fintype dOut] [Fintype dOut₂]

namespace CPTPMap
noncomputable section
open scoped Matrix ComplexOrder

variable [DecidableEq dIn]

variable {dM : Type*} [Fintype dM] [DecidableEq dM]
variable {dM₂ : Type*} [Fintype dM₂] [DecidableEq dM₂]
variable (Λ : CPTPMap dIn dOut)

/-- The Choi matrix of a CPTPMap. -/
@[reducible]
def choi := Λ.map.choi_matrix

/-- Two CPTPMaps are equal if their Choi matrices are equal. -
/
theorem choi_ext {Λ₁ Λ₂ : CPTPMap dIn dOut} (h : Λ₁.choi = Λ₂.choi) : Λ₁ = Λ₂ := by
  ext1
  exact MatrixMap.choi_equiv.injective h

/-- The Choi matrix of a channel is PSD. -/
theorem choi_PSD_of_CPTP : Λ.map.choi_matrix.PosSemidef :=
  Λ.map.choi_PSD_iff_CP_map.1 Λ.cp

/-- The trace of a Choi matrix of a CPTP map is the cardinality of the input space.
/
@[simp]
theorem Tr_of_choi_of_CPTP : Λ.choi.trace =
  (Finset.univ (α := dIn)).card :=
  Λ.TP.trace_choi

/-- Construct a CPTP map from a PSD Choi matrix with correct partial trace. -
/
def CPTP_of_choi_PSD_Tr {M : Matrix (dOut × dIn) (dOut × dIn) ℂ} (h₁ : M.PosSemidef)
  (h₂ : M.traceLeft = 1) : CPTPMap dIn dOut where
  toLinearMap := MatrixMap.of_choi_matrix M
  cp := (MatrixMap.choi_PSD_iff_CP_map (MatrixMap.of_choi_matrix M)).2
    ((MatrixMap.map_choi_inv M).symm ▸ h₁)
  TP := (MatrixMap.of_choi_matrix M).IsTracePreserving_iff_trace_choi.2
    ((MatrixMap.map_choi_inv M).symm ▸ h₂)

```

```

@[simp]
theorem choi_of_CPTP_of_choi (M : Matrix (dOut × dIn) (dOut × dIn) ℂ) {h₁}
  (CPTP_of_choi_PSD_Tr (M := M) h₁ h₂).choi = M := by
  simp only [choi, CPTP_of_choi_PSD_Tr]
  rw [MatrixMap.map_choi_inv]

theorem mat_coe_eq_apply_mat [DecidableEq dOut] (ρ : MState dIn) : (Λ ρ).m
  rfl

@[ext]
theorem funext [DecidableEq dOut] {Λ₁ Λ₂ : CPTPMap dIn dOut} (h : ∀ ρ, Λ₁ ρ
  DFunLike.ext _ _ h

/-- The composition of CPTPMaps, as a CPTPMap. -/
def compose (Λ₂ : CPTPMap dM dOut) (Λ₁ : CPTPMap dIn dM) : CPTPMap dIn dOut
  toLinearMap := Λ₂.map ∘l Λ₁.map
  cp := Λ₁.cp.comp Λ₂.cp
  TP := Λ₁.TP.comp Λ₂.TP

infixl:75 "∘m" => CPTPMap.compose

/-- Composition of CPTPMaps by `CPTPMap.compose` is compatible with the `in
/
@[simp]
theorem compose_eq [DecidableEq dOut] {Λ₁ : CPTPMap dIn dM} {Λ₂ : CPTPMap dM dOut}
  fun _ ↦ rfl

/-- Composition of CPTPMaps is associative. -/
theorem compose_assoc [DecidableEq dOut] (Λ₃ : CPTPMap dM₂ dOut) (Λ₂ : CPTPMap dM₁ dM₂)
  (Λ₁ : CPTPMap dIn dM₁) : (Λ₃ ∘m Λ₂) ∘m Λ₁ = Λ₃ ∘m (Λ₂ ∘m Λ₁) := by
  ext1 ρ
  simp

/-- CPTPMaps have a convex structure from their Choi matrices. -
/
instance instMixable : Mixable (Matrix (dOut × dIn) (dOut × dIn) ℂ) (CPTPMap dIn dOut)
  to_U := CPTPMap.choi
  to_U_inj := choi_ext
  mkT {u} h := ⟨CPTP_of_choi_PSD_Tr (M := u)
    (Exists.recOn h fun t ht => ht ▶ t.choi_PSD_of_CPTP)
    (Exists.recOn h fun t ht => (by
      rw [← ht, ← MatrixMap.IsTracePreserving_iff_trace_choi]
      exact t.TP)),
    by apply choi_of_CPTP_of_choi⟩
  convex := by

```

```

have h_convex : ∀ (M₁ M₂ : Matrix (dOut × dIn) (dOut × dIn) ℂ), M₁.PosSemidef →
  intro M₁ M₂ h₁ h₂ t ht₁ ht₂;
  convert Matrix.PosSemidef.add ( h₁.smul ht₁ ) ( h₂.smul ( sub_nonneg.mpr ht₂ ) )
intro M hM N hN a b ha hb hab;
obtain ⟨Λ₁, hΛ₁⟩ := hM
obtain ⟨Λ₂, hΛ₂⟩ := hN;
obtain ⟨Λ, hΛ⟩ : ∃ Λ : MatrixMap dIn dOut ℂ, (a • M + b • N).traceLeft = 1 ∧ (a
  refine ⟨_, ?_, ?_, rfl⟩
  · have h_trace_M : M.traceLeft = 1 := by
    convert Λ₁.TP using 1;
    rw [← hΛ₁, MatrixMap.IsTracePreserving_iff_trace_choi ]
  have h_trace_N : N.traceLeft = 1 := by
    convert Λ₂.map.IsTracePreserving_iff_trace_choi.1 Λ₂.TP;
    exact hΛ₂.symm;
  convert congr_arg₂ ( fun x y : Matrix dIn dIn ℂ => a • x + b • y ) h_trace_M
  · ext i j
    simp [ Matrix.traceLeft ]
    simp only [Finset.sum_add_distrib, Finset.mul_sum _ _ _];
  · rw [← add_smul, hab, one_smul ];
  · convert h_convex M N ( by simp [← hΛ₁ ] using Λ₁.choi_PSD_of_CPTP ) ( by s
    ring_nf
  use CPTP_of_choi_PSD_Tr hΛ.2.1 hΛ.1;
  exact MatrixMap.map_choi_inv (a • M + b • N)

/-- The identity channel, which leaves the input unchanged. -
/
def id : CPTPMap dIn dIn where
  toLinearMap := .id
  cp := .id
  TP := .id

/-- The map `CPTPMap.id` leaves any matrix unchanged. -/
@[simp]
theorem id_map : (id (dIn := dIn)).map = LinearMap.id := by
  rfl

/-- The map `CPTPMap.id` leaves the input state unchanged. -
/
@[simp]
theorem id_MState (ρ : MState dIn) : CPTPMap.id (dIn := dIn) ρ = ρ := by
  apply MState.ext_m
  rw [mat_coe_eq_apply_mat]
  simp

/-- The map `CPTPMap.id` composed with any map is the same map. -
/

```

```

@[simp]
theorem id_compose [DecidableEq dOut] ( $\Lambda$  : CPTPMap dIn dOut) :  $\text{id} \circ_m \Lambda = \Lambda$ 
  apply funext
  simp

/-- Any map composed with `CPTPMap.id` is the same map. -
/
@[simp]
theorem compose_id ( $\Lambda$  : CPTPMap dIn dOut) :  $\Lambda \circ_m \text{id} = \Lambda$  := by
  classical ext1
  simp

section equiv
variable [DecidableEq dOut]

/-- Given a equivalence (a bijection) between the types  $d_1$  and  $d_2$ , that is,
the same dimension, then there's a CPTP channel for this. This is what we
defining e.g. the SWAP channel, which is 'unitary' but takes heterogeneous
and outputs types  $(d_1 \times d_2)$  and  $(d_2 \times d_1)$ . -/
def ofEquiv ( $\sigma$  :  $d_{\text{In}} \approx d_{\text{Out}}$ ) : CPTPMap dIn dOut where
  toLinearMap := MatrixMap.submatrix  $\mathbb{C}$   $\sigma.\text{symm}$ 
  cp := .submatrix  $\sigma.\text{symm}$ 
  TP x := by rw [MatrixMap.IsTracePreserving.submatrix]

@[simp]
theorem ofEquiv_apply ( $\sigma$  :  $d_{\text{In}} \approx d_{\text{Out}}$ ) ( $\rho$  : MState dIn) :
  ofEquiv  $\sigma$   $\rho$  =  $\rho.\text{relabel } \sigma.\text{symm}$  := by
  rfl

@[simp]
theorem equiv_inverse ( $\sigma$  :  $d_{\text{In}} \approx d_{\text{Out}}$ ) : (ofEquiv  $\sigma$ )  $\circ$  (ofEquiv  $\sigma.\text{symm}$ ) =
  ext1; simp

variable { $d_1$   $d_2$   $d_3$  : Type*} [Fintype  $d_1$ ] [Fintype  $d_2$ ] [Fintype  $d_3$ ]
variable [DecidableEq  $d_1$ ] [DecidableEq  $d_2$ ] [DecidableEq  $d_3$ ]

--TODO: of_equiv (id) = id
--(of_equiv  $\sigma$ ).compose (of_equiv  $\tau$ ) = of_equiv ( $\sigma \circ \tau$ )

/-- The SWAP operation, as a channel. -/
def SWAP : CPTPMap ( $d_1 \times d_2$ ) ( $d_2 \times d_1$ ) :=
  ofEquiv (Equiv.prodComm  $d_1$   $d_2$ )

/-- The associator, as a channel. -/
def assoc : CPTPMap (( $d_1 \times d_2$ )  $\times$   $d_3$ ) ( $d_1 \times d_2 \times d_3$ ) :=
  ofEquiv (Equiv.prodAssoc  $d_1$   $d_2$   $d_3$ )

```

```

/-- The inverse associator, as a channel. -/
def assoc' : CPTPMap (d1 × d2 × d3) ((d1 × d2) × d3) :=
  ofEquiv (Equiv.prodAssoc d1 d2 d3).symm

@[simp]
theorem SWAP_eq_MState_SWAP (ρ : MState (d1 × d2)) : SWAP (d1 := d1) (d2 := d2) ρ =
  rfl

@[simp]
theorem assoc_eq_MState_assoc (ρ : MState ((d1 × d2) × d3)) : assoc (d1 := d1) (d2 := d2) ρ =
  rfl

@[simp]
theorem assoc'_eq_MState_assoc' (ρ : MState (d1 × d2 × d3)) : assoc' (d1 := d1) (d2 := d2) ρ =
  rfl

@[simp]
theorem assoc_assoc' : (assoc (d1 := d1) (d2 := d2) (d3 := d3)) ∘m assoc' = id := by
  ext1 ρ
  simp

end equiv

section trace
variable {d1 d2 : Type*} [Fintype d1] [Fintype d2] [DecidableEq d1] [DecidableEq d2]

/-- Partial tracing out the left, as a CPTP map. -/
@[simps]
def traceLeft : CPTPMap (d1 × d2) d2 :=
  --TODO: make Matrix.traceLeft a linear map, a `MatrixMap`.
  letI f (d) [Fintype d] [DecidableEq d]: Matrix (d1 × d) (d1 × d) ℂ →1 [ℂ] Matrix d d
  toFun x := Matrix.traceLeft x
  map_add' := by
    intros; ext
    simp [Matrix.traceLeft, Finset.sum_add_distrib]
  map_smul' := by
    intros; ext
    simp [Matrix.traceLeft, Finset.mul_sum]
  }
  {
    toLinearMap := f d2
    TP := by intro; simp [f]
    cp := by
      --(traceLeft ⊗km I) = traceLeft ∘m (ofEquiv prod_assoc)
      --Both go (A × B) × C → B × C
  }

```

```

--So then it suffices to show both are positive, and we have PosSemid
intro n
classical
suffices MatrixMap.IsPositive
  (f (d2 × Fin n) ◦1 (MatrixMap.submatrix ℂ (Equiv.prodAssoc d1 d2
  convert this
  ext
  rw [MatrixMap.kron_def]
  simp [f, Matrix.submatrix, Matrix.single, ite_and, Matrix.traceLeft]
  apply MatrixMap.IsPositive.comp
  · exact (MatrixMap.IsCompletelyPositive.submatrix _).IsPositive
  · intro x h
    exact h.traceLeft
}

/-- Partial tracing out the right, as a CPTP map. -/
def traceRight : CPTPMap (d1 × d2) d1 :=
  traceLeft ◦m SWAP

@[simp]
theorem traceLeft_eq_MState_traceLeft (p : MState (d1 × d2)) :
  traceLeft (d1 := d1) (d2 := d2) p = p.traceLeft := by
  rfl

@[simp]
theorem traceRight_eq_MState_traceRight (p : MState (d1 × d2)) :
  traceRight (d1 := d1) (d2 := d2) p = p.traceRight := by
  rfl --It's actually pretty crazy that this is a definitional equality, co

end trace

/--The replacement channel that maps all inputs to a given state. -
/
def replacement [Nonempty dIn] [DecidableEq dOut] (p : MState dOut) : CPTPMap
  traceLeft ◦m {
    toFun := fun M => Matrix.kroneckerMap (fun x1 x2 => x1 * x2) M p.m
    map_add' := by simp [Matrix.add_kronecker]
    map_smul' := by simp [Matrix.smul_kronecker]
    cp := MatrixMap.kron_kronecker_const p.psd
    TP := by intro; simp [Matrix.trace_kronecker]
  }

/-- The output of `replacement p` is always that `p`. -/
@[simp]
theorem replacement_apply [Nonempty dIn] [DecidableEq dOut] (p : MState dOut)
  replacement p p0 = p := by

```

```

simp [replacement, instMFunLike, PTPMap.instMFunLike, HPMMap.instFunLike, HPMMap.map
  MState.traceLeft]
--This should be simp...
ext i j
simp
rw [HermitianMat.instFun]
simp [-HermitianMat.mat_apply, Matrix.traceLeft, ← Finset.sum_mul]
convert one_mul _
exact  $\rho_0.\text{tr}$ 

--In principle we can relax the `Nonempty dIn`: for the case where `IsEmpty dIn`, we
-- 0 map, and it's CPTP.
instance [Nonempty dIn] [Nonempty dOut] [DecidableEq dOut] : Inhabited (CPTPMap dIn
  (replacement default))

instance [Nonempty dIn] [Nonempty dOut] : Nonempty (CPTPMap dIn dOut) := by
  classical infer_instance

/-- There is a CPTP map that takes a system of any (nonzero) dimension and outputs t
trivial Hilbert space, 1-dimensional, indexed by any `Unique` type. We can think of
as "destroying" the whole system; tracing out everything. -
/
def destroy [Nonempty dIn] [Unique dOut] : CPTPMap dIn dOut :=
  replacement default

/-- Two CPTP maps into the same one-dimensional output space must be equal -
/
theorem eq_if_output_unique [Unique dOut] ( $\Lambda_1 \Lambda_2$  : CPTPMap dIn dOut) :  $\Lambda_1 = \Lambda_2$  :=
  funext fun _  $\mapsto$  (Unique.eq_default _).trans (Unique.eq_default _).symm

/-- There is exactly one CPTPMap to a 1-dimensional space. -
/
instance instUnique [Nonempty dIn] [Unique dOut] : Unique (CPTPMap dIn dOut) where
  default := destroy
  uniq := fun _  $\mapsto$  eq_if_output_unique _ _

@[simp]
theorem destroy_comp {dOut2 : Type*} [Unique dOut2] [DecidableEq dOut] [Nonempty dIn]
  ( $\Lambda$  : CPTPMap dIn dOut) :
  destroy (dOut := dOut2)  $\circ_m$   $\Lambda$  = destroy :=
  Unique.eq_default _

section prod
open Kronecker

variable {dI1 dI2 dO1 dO2 : Type*} [Fintype dI1] [Fintype dI2] [Fintype dO1] [Fintyp

```

```

variable [DecidableEq dI1] [DecidableEq dI2] [DecidableEq dO1] [DecidableEq dO2]

set_option maxRecDepth 1000 in -- ??? what the heck is recursing
/-- The tensor product of two CPTPMaps. -/
def prod (Λ1 : CPTPMap dI1 dO1) (Λ2 : CPTPMap dI2 dO2) : CPTPMap (dI1 × dI2) (dO1 × dO2) :=
  toLinearMap := Λ1.map.kron Λ2.map
  cp := Λ1.cp.kron Λ2.cp
  TP := Λ1.TP.kron Λ2.TP

infixl:70 "⊗CP" => CPTPMap.prod

end prod

section finprod

variable {ι : Type u} [DecidableEq ι] [fι : Fintype ι]
variable {dI : ι → Type v} [∀(i : ι), Fintype (dI i)] [∀(i : ι), DecidableEq (dI i)]
variable {dO : ι → Type w} [∀(i : ι), Fintype (dO i)] [∀(i : ι), DecidableEq (dO i)]

/-- Finitely-indexed tensor products of CPTPMaps. -/
def piProd (Λi : (i : ι) → CPTPMap (dI i) (dO i)) : CPTPMap ((i : ι) → dI i) ((i : ι) → dO i) :=
  toLinearMap := MatrixMap.piProd (fun i => (Λi i).map)
  cp := MatrixMap.IsCompletelyPositive.piProd (fun i => (Λi i).cp)
  TP := sorry

theorem fin_1_piProd
  {dI : Fin 1 → Type v} [Fintype (dI 0)] [DecidableEq (dI 0)]
  {dO : Fin 1 → Type w} [Fintype (dO 0)] [DecidableEq (dO 0)]
  (Λi : (i : Fin 1) → CPTPMap (dI i) (dO i)) :
    piProd Λi = sorry ∘m ((Λi 1) ∘m sorry) :=
  sorry --TODO: permutations

/--
The tensor product of composed maps is the composition of the tensor products.
-/
theorem piProd_comp
  {d1 d2 d3 : ι → Type*}
  [∀ i, Fintype (d1 i)] [∀ i, DecidableEq (d1 i)]
  [∀ i, Fintype (d2 i)] [∀ i, DecidableEq (d2 i)]
  [∀ i, Fintype (d3 i)] [∀ i, DecidableEq (d3 i)]
  (Λ1 : ∀ i, CPTPMap (d1 i) (d2 i)) (Λ2 : ∀ i, CPTPMap (d2 i) (d3 i)) :
    piProd (fun i => (Λ2 i) ∘m (Λ1 i)) = (piProd Λ2) ∘m (piProd Λ1) := by
  apply CPTPMap.ext
  convert MatrixMap.piProd_comp _ _;
  infer_instance

```


end finprod

section unitary

-- Conjugating density matrices by a unitary as a channel. This is standard unitary
/

```
def ofUnitary (U :  $\mathbb{C}^{dIn \times dIn}$ ) : CPTPMap dIn dIn where
  toLinearMap := MatrixMap.conj U
  cp := MatrixMap.conj_isCompletelyPositive U.val
  TP := by intro; simp [Matrix.trace_mul_cycle U.val,  $\leftarrow$  Matrix.star_eq_conjTranspose]
```

-- The unitary channel U conjugated by U. -/
theorem ofUnitary_eq_conj (U : $\mathbb{C}^{dIn \times dIn}$) (ρ : MState dIn) :
 (ofUnitary U) ρ = ρ.U_conj U :=
 rfl

-- A channel is unitary iff it is `ofUnitary U`. -/
def IsUnitary (Λ : CPTPMap dIn dIn) : Prop :=
 $\exists U, \Lambda = \text{ofUnitary } U$

-- A channel is unitary iff it can be written as conjugation by a unitary. -
/

```
theorem IsUnitary_iff_U_conj (Λ : CPTPMap dIn dIn) : IsUnitary Λ  $\leftrightarrow$   $\exists U, \forall \rho, \Lambda \rho = \rho.U_{\text{conj}}$ 
  simp_rw [IsUnitary,  $\leftarrow$  ofUnitary_eq_conj, CPTPMap.funext_iff]
```

```
theorem IsUnitary_equiv (σ : dIn  $\approx$  dIn) : IsUnitary (ofEquiv σ) := by
  have h_unitary :  $\exists U : \text{Matrix } dIn \ dIn \ \mathbb{C}, U * U.conjTranspose = 1 \wedge U.conjTranspose$ 
    simp only [Matrix.conjTranspose, RCLike.star_def];
    refine' { fun y x => if y = σ x then 1 else 0, ?_, ?_, by simp }
    · ext y x
      simp [Matrix.mul_apply, Matrix.transpose_apply];
      rw [Finset.sum_eq_single ( σ.symm x )] <|> aesop
    · ext y x
      simp [Matrix.mul_apply, Matrix.transpose_apply, Matrix.map_apply];
      simp [Matrix.one_apply, eq_comm]
  obtain ⟨U, hU_unitary, hU_eq⟩ := h_unitary;
  use ⟨U, Matrix.mem_unitaryGroup_iff.mpr hU_unitary⟩
  have h_mul :  $\forall \rho : \text{Matrix } dIn \ dIn \ \mathbb{C}, U * \rho * U^H = \text{Matrix.submatrix } \rho \ \sigma.symm \ \sigma.symm$ 
    intro ρ
    ext i j
    have hU_i_x :  $\forall x : dIn, U \ i \ x = \text{if } x = \sigma.symm \ i \text{ then } 1 \text{ else } 0$  := by grind
    have hU_j_x :  $\forall x : dIn, U \ j \ x = \text{if } x = \sigma.symm \ j \text{ then } 1 \text{ else } 0$  := by grind
    simp [Matrix.mul_apply, Matrix.submatrix, hU_i_x, hU_j_x]
  ext ρ : 3
  exact (h_mul ρ).symm
```

end unitary

```
-- /-- A channel is *entanglement breaking* iff its product with the identity
-- only outputs separable states. -/
-- def IsEntanglementBreaking ( $\Lambda$  : CPTPMap dIn dOut) : Prop :=
--    $\forall$  (dR : Type u_1) [Fintype dR] [DecidableEq dR],
--    $\forall$  ( $\rho$  : MState (dR  $\times$  dIn)), ((CPTPMap.id (dIn := dR)  $\otimes_k \Lambda$ )  $\rho$ ).IsSeparable
```

--TODO:

--Theorem: entanglement breaking iff it holds for all channels, not just id
 --Theorem: entanglement break iff it breaks a Bell pair (Wilde Exercise 4.6)
 --Theorem: entanglement break if c-q or q-c, e.g. measurements
 --Theorem: eb iff Kraus operators can be written as all unit rank (Wilde Th

section purify

variable [DecidableEq dOut] [Inhabited dOut]

--PULLOUT

omit [DecidableEq dOut] [Inhabited dOut] in

/-

PROBLEM

If a MatrixMap of_kraus K K is trace-preserving, then $\sum_k K_k^\dagger K_k = 1$.

PROVIDED SOLUTION

The TP condition says for all X, $\text{trace}((\text{of_kraus } K \ K) \ X) = \text{trace}(X)$.

Unfolding of_kraus: $\text{trace}(\sum_k K_k \ X \ K_k^\dagger) = \sum_k \text{trace}(K_k^\dagger K_k \ X)$ (by cycle

So $\text{trace}(A \ X) = \text{trace}(X)$ for all X where $A = \sum_k K_k^\dagger K_k$, which means $A = 1$.

Use `Matrix.eq\_of\_trace\_mul\_eq` or the fact that trace is a faithful pairing

to conclude $A = 1$. The TP condition ` $\Lambda$ .TP` gives us ` $\forall x, (\Lambda.\text{map } x).\text{trace} = \text{trace } x$ '

since ` $\Lambda.\text{map} = \text{of_kraus } K \ K$ ', we substitute and simplify.

-/

private lemma kraus_sum_eq_one_of_TP

{ κ : Type*} [Fintype κ]

{ $K_1 \ K_2$: $\kappa \rightarrow \text{Matrix } dOut \ dIn \ \mathbb{C}$ }

(hTP : (MatrixMap.of_kraus $K_1 \ K_2$).IsTracePreserving) :

$\sum k, (K_2 \ k).\text{conjTranspose} * (K_1 \ k) = 1 := \text{by}$

ext1 i j

have := hTP (Matrix.of fun x y $\mapsto$ if x = j then if y = i then 1 else 0 else 0)

simp only [Matrix.trace, Matrix.diag_apply, Matrix.of_apply, Finset.sum_i

Finset.mem_univ, $\downarrow$ reduceIte] at this

convert this using 1

· simp [MatrixMap.of_kraus, Matrix.sum_apply, Matrix.mul_apply]

rw [Finset.sum_comm]

congr! 2

ring

· simp [Matrix.one_apply, eq_comm]

/-

PROBLEM

Given an $m \times n$ matrix V over $\mathbb{C}$ with $V^\dagger V = 1$, and an injection $\text{emb} : n \hookrightarrow m$, there exists a unitary matrix $U \in \text{unitaryGroup } m \ \mathbb{C}$ such that for all i and j , $U \ i \ (\text{emb } j) = V \ i \ j$.

PROVIDED SOLUTION

The columns of V form an orthonormal set in $\mathbb{C}^m$ (this follows from $V^\dagger V = 1$). Using the embedding emb , assign each column V_j to position $\text{emb}(j)$ in the larger matrix. The remaining columns can be filled by extending to an orthonormal basis of $\mathbb{C}^m$. This extension exists by ``Orthonormal.exists_orthonormalBasis_extension`` in Mathlib. The resulting matrix has orthonormal columns spanning $\mathbb{C}^m$, hence it is unitary.

Concretely, define the column vectors of V as an orthonormal family in `EuclideanSpace` indexed by the range of emb . Then use ``Orthonormal.exists_orthonormalBasis_extension`` to extend this to a full `OrthonormalBasis`. The matrix of this basis is unitary.

Alternatively, use V to define a linear isometry on the subspace spanned by the image of emb , then use ``LinearIsometry.extend`` to extend to the full space. Since in finite dimensions a linear isometry from a space to itself is surjective, this gives a `LinearIsometryEquiv`, hence a unitary matrix.

-/

```
set_option maxHeartbeats 1600000 in
private lemma exists_unitary_extending_isometry
  {m n : Type*} [Fintype m] [Fintype n] [DecidableEq m] [DecidableEq n]
  (V : Matrix m n ℂ) (hV : V.conjTranspose * V = 1)
  (emb : n ↪ m) :
  ∃ U : Matrix m m ℂ, ∀ i j, U.val i (emb j) = V i j := by
-- Let $u_i$ be the $i$-th column of $V$.
set u : n → EuclideanSpace ℂ m := fun j => WithLp.toLp 2 (fun i => V i j)
-- Since $u$ is an orthonormal set, we can extend it to an orthonormal basis of $ℂ^m$.
obtain ⟨b, hb⟩ : ∃ b : OrthonormalBasis m ℂ (EuclideanSpace ℂ m), ∀ j, b (emb j) = u j
have h_orthonormal : Orthonormal ℂ (fun j => u j) := by
  rw [orthonormal_iff_ite];
  intro i j
  replace hV := congr_fun (congr_fun hV i) j
  simpa only [mul_comm, Matrix.mul_apply] using hV
have := Orthonormal.exists_orthonormalBasis_extension_of_card_eq (1 := ℂ) (E := ℂ)
simp only [finrank_euclideanSpace, forall_const] at this
contrapose! this
· refine (fun i => if hi : i ∈ Set.range emb then u (Classical.choose hi) else 0)
  · simp +contextual only [Orthonormal, h_orthonormal.1, implies_true, true_and,
    Set.mem_range, Set.restrict_apply, Subtype.forall, ↓reduceDite]
    intro i j hij
    split_ifs with h₁ h₂
```

```

      · apply h_orthonormal.2
      have := Classical.choose_spec <∃ y, emb y = ↑i>
      have := Classical.choose_spec <∃ y, emb y = ↑j>
      grind
      · simp
      · simp
      · simp
      · simp_all only [Orthonormal, ne_eq, Set.mem_range, exists_exists_eq,
        EmbeddingLike.apply_eq_iff_eq, exists_eq, ↓reduceDite, Classical.
refine ⟨⟨Matrix.of (fun i j ↦ b j i), ?_⟩, ?_⟩
      · simp only [Matrix.mem_unitaryGroup_iff]
      ext1 i j
      simp [inner] using b.sum_inner_mul_inner (EuclideanSpace.single i 1) (
      · simp [hb, u]

omit [DecidableEq dOut] [Inhabited dOut] in
/--
Given Kraus operators K indexed by (dOut × dIn), define the isometry matrix
V : Matrix (dIn × dOut × dOut) dIn ℂ by V_{(a, b, d), a'} = (K (b, a))_{d,
Then V†V = 1.
-/
private lemma purify_isometry_condition
  {K : (dOut × dIn) → Matrix dOut dIn ℂ}
  (hTP : ∑ k, (K k).conjTranspose * (K k) = 1) :
  let V : Matrix (dIn × dOut × dOut) dIn ℂ :=
    fun ⟨a, b, d⟩ a' => (K (b, a)) d a'
  V.conjTranspose * V = 1 := by
  rw [← hTP]
  ext1 i j
  simp only [Matrix.mul_apply, Fintype.sum_prod_type];
  rw [Finset.sum_comm]
  simp [Matrix.sum_apply, Matrix.mul_apply]

private lemma purify_MState_pure_basis_default_entry (i j : dOut × dOut) :
  (MState.pure (Ket.basis (default : dOut × dOut))).m i j =
  if i = default ∧ j = default then 1 else 0 := by
  change (MState.pure (Ket.basis (default : dOut × dOut))).M.val i j = _
  simp only [MState.pure, Matrix.vecMulVec_apply, Ket.basis, Ket.to_bra,
    Braket.instFunLikeKet, Braket.instFunLikeBra]
  split_ifs <=> simp_all [eq_comm]

omit [Inhabited dOut] in
private lemma purify_replacement_single_eq (ρ₀ : MState (dOut × dOut)) (b₁
  ((replacement ρ₀).map (Matrix.single () () 1)) b₁ b₂ = ρ₀.m b₁ b₂ := by
  open Kronecker in
  suffices h : (Matrix.single () () 1 ⊗ₖ ρ₀.m).traceLeft = ρ₀.m from

```

```

    congr_fun (congr_fun h b₁) b₂
  ext : 1
  simp [Matrix.traceLeft, Matrix.kroneckerMap]

private lemma purify_prep_append_entry (X : Matrix dIn dIn ℂ)
  (a₁ : dIn) (b₁c₁ : dOut × dOut) (a₂ : dIn) (b₂c₂ : dOut × dOut) :
  let ρ₀ := MState.pure (Ket.basis (default : dOut × dOut))
  let zero_prep : CPTPMap Unit (dOut × dOut) := replacement ρ₀
  let prep := (id ⊗ℂ zero_prep)
  let append : CPTPMap dIn (dIn × Unit) := CPTPMap.ofEquiv (Equiv.prodPUnit dIn).s
  (prep ∘m append).map X (a₁, b₁c₁) (a₂, b₂c₂) =
  X a₁ a₂ * ρ₀.m b₁c₁ b₂c₂ := by
  --TODO Cleanup. The proof tactics aren't too long, but the intermediate state is
  -- distgusting; probably this can be repackaged as several intermediate simp lemma
  -- and generally needs better API.
  unfold replacement CPTPMap.traceLeft CPTPMap.prod CPTPMap.ofEquiv
  simp only [id_map, Matrix.traceLeft, Finset.univ_unique, PUnit.default_eq_unit,
    Finset.sum_singleton, Matrix.kroneckerMap, MState.pure_apply, Equiv.symm_symm]
  erw [LinearMap.comp_apply, MatrixMap.kron_def]
  erw [Finset.sum_eq_single a₁]
  · simp only [Matrix.single]
    erw [Matrix.of_apply]
    simp only [Matrix.of_apply]
    erw [Matrix.of_apply]
    simp [Matrix.of_apply]
    ring!
  · simp +contextual
  · simp

private lemma purify_conj_entry (X : Matrix dIn dIn ℂ) (U : [[dIn × dOut × dOut]])
  (i j : dIn × dOut × dOut) :
  let ρ₀ := MState.pure (Ket.basis (default : dOut × dOut))
  let zero_prep : CPTPMap Unit (dOut × dOut) := replacement ρ₀
  let prep := (id ⊗ℂ zero_prep)
  let append : CPTPMap dIn (dIn × Unit) := CPTPMap.ofEquiv (Equiv.prodPUnit dIn).s
  ((ofUnitary U) ∘m prep ∘m append).map X i j =
  ∑ α₁ : dIn, ∑ α₂ : dIn,
  U.val i (α₁, default, default) * X α₁ α₂ *
  starRingEnd ℂ (U.val j (α₂, default, default)) := by
  open Kronecker in
  have h_conj :
    let ρ₀ := MState.pure (Ket.basis (default : dOut × dOut))
    let zero_prep : CPTPMap Unit (dOut × dOut) := replacement ρ₀
    let prep := (id ⊗ℂ zero_prep)
    let append : CPTPMap dIn (dIn × Unit) := CPTPMap.ofEquiv (Equiv.prodPUnit dIn).s
    (prep ∘m append).map X = X ⊗k (MState.pure (Ket.basis (default : dOut × dOut)))

```

```

    ext {a1, b1c1} {a2, b2c2}
    simp [purify_prep_append_entry]
  have h_conj :
    let ρ0 := MState.pure (Ket.basis (default : dOut × dOut))
    let zero_prep : CPTPMap Unit (dOut × dOut) := replacement ρ0
    let prep := (id ⊗C P zero_prep)
    let append := CPTPMap.ofEquiv (Equiv.prodPUnit dIn).symm
    (ofUnitary U).map ((prep ∘m append).map X) i j =
      ∑ k, ∑ l, U.val i k * (X ⊗k (MState.pure (Ket.basis (default : dOut × dOut))))
    simp [h_conj, Matrix.kroneckerMap]
    convert congr_arg (fun m : Matrix (dIn × dOut × dOut) (dIn × dOut × dOut) => m)
    simp [Matrix.mul_apply, Matrix.conjTranspose_apply]
    ring_nf!
    exact Finset.sum_comm.trans (Finset.sum_congr rfl fun _ => by rw [Finset.sum_congr])
  have h_restrict : ∀ x : dIn × dOut × dOut, (Ket.basis default) x.2 = if x.1 = 0 then 0 else 1
    intro x
    simp [Ket.basis, eq_comm]
    exact rfl
  simp_all [Finset.sum_ite]
  convert h_conj using 1
  · congr! 1
    convert congr_arg (fun f => (U.val * f * U.val.conjTranspose)) <_> using 1
  · rw [← Finset.sum_product', ← Finset.sum_product']
    apply Finset.sum_bij (fun x _ => ((x.1, default, default), (x.2, default, default)))
    · simp
      exact fun a b => rfl
    · simp
    · simp
      exact fun _ _ => rfl
    · simp

private lemma purify_rhs_entry (X : Matrix dIn dIn ℂ) (d1 d2 : dOut)
  (U : [][dIn × dOut × dOut]) :
  let ρ0 := MState.pure (Ket.basis (default : dOut × dOut))
  let zero_prep : CPTPMap Unit (dOut × dOut) := replacement ρ0
  let prep := (id ⊗C P zero_prep)
  let append : CPTPMap dIn (dIn × Unit) := CPTPMap.ofEquiv (Equiv.prodPUnit dIn)
  (traceLeft ∘m traceLeft ∘m (ofUnitary U) ∘m prep ∘m append).map X d1 d2
  ∑ a : dIn, ∑ b : dOut, ∑ α1 : dIn, ∑ α2 : dIn,
    U.val (a, b, d1) (α1, default, default) * X α1 α2 *
    starRingEnd ℂ (U.val (a, b, d2) (α2, default, default)) := by
  apply Eq.symm
  have h := purify_conj_entry X U
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun y _ => Finset.sum_congr rfl fun x _ =>
    (h (x, y, d1) (x, y, d2)).symm

```

```

omit [DecidableEq dIn] [DecidableEq dOut] [Inhabited dOut] in
private lemma purify_of_kraus_entry (K : (dOut × dIn) → Matrix dOut dIn ℂ) (X : Matrix dOut dIn ℂ) :
  (MatrixMap.of_kraus K K) X d₁ d₂ =
    ∑ k : dOut × dIn, ∑ α₁ : dIn, ∑ α₂ : dIn,
      (K k) d₁ α₁ * X α₁ α₂ * starRingEnd ℂ ((K k) d₂ α₂) := by
  have h_lhs : (MatrixMap.of_kraus K K) X = ∑ k, K k * X * (K k).conjTranspose := by
    simp [MatrixMap.of_kraus]
  simp only [h_lhs, Matrix.sum_apply, Matrix.mul_apply]
  simp only [Matrix.conjTranspose_apply, RCLike.star_def, Finset.sum_mul]
  refine Finset.sum_congr rfl fun _ _ => ?_
  rw [Finset.sum_comm]

theorem exists_purify (Λ : CPTPMap dIn dOut) :
  ∃ (Λ' : CPTPMap (dIn × dOut × dOut) (dIn × dOut × dOut)),
    Λ'.IsUnitary Λ
  Λ = (
    let zero_prep : CPTPMap Unit (dOut × dOut) := replacement (MState.pure (Ket.ba
    let prep := (id ⊗C P zero_prep)
    let append : CPTPMap dIn (dIn × Unit) := CPTPMap.ofEquiv (Equiv.prodPUnit dIn)
    CPTPMap.traceLeft ◦m CPTPMap.traceLeft ◦m Λ' ◦m prep ◦m append
  ) := by
  obtain ⟨K, hK⟩ := Λ.cp.exists_kraus _
  have hTP_kraus : ∑ k, (K k).conjTranspose * (K k) = 1 :=
    kraus_sum_eq_one_of_TP (hK ▸ Λ.TP)
  let V : Matrix (dIn × dOut × dOut) dIn ℂ :=
    fun ⟨a, b, d⟩ a' => (K (b, a)) d a'
  have hV : V.conjTranspose * V = 1 :=
    purify_isometry_condition hTP_kraus
  let emb : dIn ↪ (dIn × dOut × dOut) :=
    ⟨fun a => (a, default, default), fun a₁ a₂ h => by simp using h⟩
  obtain ⟨U, hU⟩ := exists_unitary_extending_isometry V hV emb
  use ofUnitary U, ⟨U, rfl⟩
  apply CPTPMap.ext
  ext X d₁ d₂ : 2
  -- LHS: rewrite using hK and purify_of_kraus_entry
  -- RHS: use purify_rhs_entry
  change Λ.map = _ at hK
  rw [hK, purify_of_kraus_entry, purify_rhs_entry]
  rw [Fintype.sum_prod_type, Finset.sum_comm]
  simp only [Function.Embedding.coeFn_mk, emb, V] at hU
  simp_rw [hU]

```

*/-- Every channel can be written as a unitary channel on a larger system. In general the original channel was $A \rightarrow B$, we may need to go as big as dilating the output system (environment) by a factor of $A*B$. One way of stating this would be that it forms an*

isometry from A to $(B \times A \times B)$. So that we can instead talk about the cleaner, say that this is a unitary on $(A \times B \times B)$. The defining properties that this purification comes are ``purify_IsUnitary`` and ``purify_trace``. This means that always has type ``dIn × dOut``.

Furthermore, since we need a canonical " $\emptyset$ " state on B in order to add with we require a typeclass instance `[Inhabited dOut]`. -/

```
def purify (Λ : CTPMap dIn dOut) : CTPMap (dIn × dOut × dOut) (dIn × dOut)
  exists_purify Λ |>.choose
```

```
theorem purify_IsUnitary (Λ : CTPMap dIn dOut) : Λ.purify.IsUnitary :=
  exists_purify Λ |>.choose_spec.1
```

```
/-- With a channel  $\Lambda : A \rightarrow B$ , a valid purification  $(A \times B \times B) \rightarrow (A \times B \times B)$  is such
* Preparing the default  $\emptyset$  state on two copies of  $B$ 
* Appending these to the input
* Applying the purified unitary channel
* Tracing out the two left parts of the output
is equivalent to the original channel. This theorem states that the channel
has this property. -/
```

```
theorem purify_trace (Λ : CTPMap dIn dOut) : Λ = (
  let zero_prep : CTPMap Unit (dOut × dOut) := replacement (MState.pure (K
  let prep := (id  $\otimes^{c,p}$  zero_prep)
  let append : CTPMap dIn (dIn × Unit) := CTPMap.ofEquiv (Equiv.prodPUnit
  CTPMap.traceLeft  $\circ_m$  CTPMap.traceLeft  $\circ_m$  Λ.purify  $\circ_m$  prep  $\circ_m$  append
) :=
  exists_purify Λ |>.choose_spec.2
```

```
--TODO Theorem: `purify` is unique up to unitary equivalence.
```

```
--TODO: Best to rewrite the "zero_prep / prep / append" as one CTPMap.append
-- define that.
```

```
/-- The complementary channel comes from tracing out the other half (the right)
/
```

```
def complementary (Λ : CTPMap dIn dOut) : CTPMap dIn (dIn × dOut) :=
  let zero_prep : CTPMap Unit (dOut × dOut) := replacement (MState.pure (K
  let prep := (id  $\otimes^{c,p}$  zero_prep)
  let append : CTPMap dIn (dIn × Unit) := CTPMap.ofEquiv (Equiv.prodPUnit
  CTPMap.traceRight  $\circ_m$  CTPMap.assoc'  $\circ_m$  Λ.purify  $\circ_m$  prep  $\circ_m$  append
```

```
end purify
```

```
section degradable
```

```
variable [DecidableEq dOut] [Inhabited dOut] [DecidableEq dOut₂] [Inhabited
```



```

/-- A channel is *degradable to* another, if the other can be written as a composition
a _degrading_ channel D with the original channel. -/
def IsDegradableTo ( $\Lambda$  : CPTPMap dIn dOut) ( $\Lambda_2$  : CPTPMap dIn dOut2) : Prop :=
   $\exists$  (D : CPTPMap dOut (dOut2)), D  $\circ_m$   $\Lambda$  =  $\Lambda_2$ 

/-- A channel is *antidegradable to* another, if the other `IsDegradableTo` this one
-/
@[reducible]
def IsAntidegradableTo ( $\Lambda$  : CPTPMap dIn dOut) ( $\Lambda_2$  : CPTPMap dIn dOut2) : Prop :=
  IsDegradableTo  $\Lambda_2$   $\Lambda$ 

/-- A channel is *degradable* if it `IsDegradableTo` its complementary channel. -
-/
def IsDegradable ( $\Lambda$  : CPTPMap dIn dOut) : Prop :=
  IsDegradableTo  $\Lambda$   $\Lambda$ .complementary

/-- A channel is *antidegradable* if it `IsAntidegradableTo` its complementary channel
-/
@[reducible]
def IsAntidegradable ( $\Lambda$  : CPTPMap dIn dOut) : Prop :=
  IsAntidegradableTo  $\Lambda$   $\Lambda$ .complementary

--Theorem (Wilde Exercise 13.5.7): Entanglement breaking channels are antidegradable
end degradable

/-- `CPTPMap`s inherit a topology from their choi matrices. -
-/
instance instTop : TopologicalSpace (CPTPMap dIn dOut) :=
  TopologicalSpace.induced (CPTPMap.choi) instTopologicalSpaceMatrix

/-- The projection from `CPTPMap` to the Choi matrix is an embedding -
-/
theorem choi_IsEmbedding : Topology.IsEmbedding (CPTPMap.choi (dIn := dIn) (dOut :=
  eq_induced := rfl
  injective _ _ := choi_ext

instance instT3Space : T3Space (CPTPMap dIn dOut) :=
  Topology.IsEmbedding.t3Space choi_IsEmbedding

end
end CPTPMap

```

1.426 QuantumInfo/Finite/CPTPMap/Dual.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.CPTPMap.Bundled
import Mathlib.LinearAlgebra.Matrix.FiniteDimensional

/-! # Duals of matrix map

Definitions and theorems about the dual of a matrix map. -
/

noncomputable section
open ComplexOrder

variable {dIn dOut : Type*} [Fintype dIn] [Fintype dOut]
variable {R : Type*} [CommRing R]
variable {α : Type*} [RCLike α]

namespace MatrixMap

variable [DecidableEq dIn] [DecidableEq dOut] {M : MatrixMap dIn dOut α}

--This should be definable with LinearMap.adjoint, but that requires InnerP
--that is currently causing issues and pains (tried `open scoped Frobenius`

/-- The dual of a map between matrices, defined by  $\text{Tr}[A M(B)] = \text{Tr}[(\text{dual } M) A]$ 
called the adjoint of the map instead. -/
@[irreducible]
def dual (M : MatrixMap dIn dOut R) : MatrixMap dOut dIn R :=
  let iso1 := (Module.Basis.toDualEquiv <| Matrix.stdBasis R dIn dIn).symm
  let iso2 := (Module.Basis.toDualEquiv <| Matrix.stdBasis R dOut dOut)
  iso1 ∘1 LinearMap.dualMap M ∘1 iso2

/-- The defining property of a dual map: inner products are preserved on the
/
theorem Dual.trace_eq (M : MatrixMap dIn dOut R) (A : Matrix dIn dIn R) (B : Matrix dOut dOut R) :
  (M A * B).trace = (A * M.dual B).trace := by
  unfold dual
  dsimp [Matrix.trace]
  rw [LinearMap.dualMap_apply']
  simp_rw [Matrix.mul_apply]
  sorry

```

```

--all properties below should provable just from `inner_eq`, since the definition of
-- is pretty hairy (and maybe could be improved...)

/-- The dual of a `IsHermitianPreserving` map also `IsHermitianPreserving`. -
/
theorem IsHermitianPreserving.dual (h : M.IsHermitianPreserving) : M.dual.IsHermitia
  sorry

--TODO Cleanup, find home, abstract out to HermitianMats...?
theorem _root_.Matrix.PosSemidef.trace_mul_nonneg {n : Type*} [Fintype n] [Decidable
  {A B : Matrix n n []} (hA : A.PosSemidef) (hB : B.PosSemidef) :
  0 ≤ (A * B).trace := by
  open scoped Matrix in
  obtain {sqrtB, rfl} : ∃ sqrtB : Matrix n n [], B = sqrtBH * sqrtB := by
    exact Matrix.posSemidef_iff_eq_conjTranspose_mul_self.mp hB;
  simp only [← Matrix.mul_assoc, ← Matrix.trace_mul_comm sqrtB]
  have h : (sqrtB * A * sqrtBH).PosSemidef := by
    convert hA.conjTranspose_mul_mul_same sqrtBH using 1
    simp [Matrix.mul_assoc]
  rw [Matrix.posSemidef_iff_dotProduct_mulVec] at h
  simp [Matrix.mulVec, dotProduct, Matrix.trace, Pi.single_apply] using
    Finset.sum_nonneg fun i _ => h.2 (Pi.single i 1)

/-- The dual of a `IsPositive` map also `IsPositive`. -/
theorem IsPositive.dual (h : M.IsPositive) : M.dual.IsPositive := by
  intro x hx
  rw [Matrix.posSemidef_iff_dotProduct_mulVec] at hx ⊢
  use IsHermitianPreserving.dual h.IsHermitianPreserving hx.1
  intro v
  have h_dual_pos : 0 ≤ (M (Matrix.vecMulVec v (star v)) * x).trace := by
    --TODO Cleanup. Should be all in terms of HermitianMat
    apply Matrix.PosSemidef.trace_mul_nonneg;
    · apply h;
      exact Matrix.posSemidef_vecMulVec_self_star v;
    · rw [← Matrix.posSemidef_iff_dotProduct_mulVec] at hx
      exact hx;
  convert h_dual_pos using 1;
  rw [MatrixMap.Dual.trace_eq];
  simp [Matrix.vecMulVec, Matrix.mul_apply, Matrix.trace];
  simp [Matrix.mulVec, dotProduct, Finset.mul_sum _ _ _, mul_assoc, mul_comm, mul_l
  exact Finset.sum_comm.trans (Finset.sum_congr rfl fun _ => Finset.sum_congr rfl

/-- The dual of TracePreserving map is *not* trace-preserving, it's *unital*, that i
/
theorem dual_Unital (h : M.IsTracePreserving) : M.dual.Unital := by

```

```

-- By definition of dual, we know that for any matrix A,  $\text{Tr}(M(A) * I) = \text{Tr}(A * M(\text{dual } I))$ 
have h_dual_trace :  $\forall A : \text{Matrix } dIn\ dIn\ [], (M\ A * 1).\text{trace} = (A * M.\text{dual } 1).\text{trace}$ 
  exact fun A => Dual.trace_eq M A 1;
ext i j
specialize h_dual_trace ( Matrix.of ( fun k l => if k = j then if l = i then 1 else 0 ) )
simp_all [ Matrix.trace, Matrix.mul_apply ] ;
specialize h ( Matrix.of ( fun k l => if k = j then if l = i then 1 else 0 ) )
simp_all [ Matrix.trace ]
simp [ Matrix.one_apply, eq_comm ]

alias IsTracePreserving.dual := dual_Unital

/--
If two matrix maps satisfy the trace duality property, they are equal.
-/
lemma dual_unique
  (M : MatrixMap dIn dOut []) (M' : MatrixMap dOut dIn [])
  (h :  $\forall A\ B, (M\ A * B).\text{trace} = (A * M'\ B).\text{trace}$ ) : M.dual = M' := by
-- By definition of dual, we know that for any A and B, the trace of (M A * B) equals the trace of (A * M.dual B)
have h_dual :  $\forall A : \text{Matrix } dIn\ dIn\ [], \forall B : \text{Matrix } dOut\ dOut\ [], (M\ A * B).\text{trace} = (A * M.\text{dual } B).\text{trace}$ 
  exact fun A B => Dual.trace_eq M A B;
-- Since these two linear maps agree on all bases, they must be equal.
have h_eq :  $\forall A : \text{Matrix } dIn\ dIn\ [], \forall B : \text{Matrix } dOut\ dOut\ [], (A * M.\text{dual } B).\text{trace} = (M\ A * B).\text{trace}$ 
  exact fun A B => h_dual A B ▸ h A B;
refine' LinearMap.ext fun B => _;
exact Matrix.ext_iff_trace_mul_left.mpr fun x => h_eq x B

/--
The Choi matrix of the dual map is the transpose of the reindexed Choi matrix of the map.
-/
lemma dual_choi_matrix (M : MatrixMap dIn dOut []) :
  M.dual.choi_matrix = (M.choi_matrix.transpose).reindex (Equiv.prodComm dOut dIn)
-- By definition of dual, we know that  $(M.\text{dual } (single\ j_1\ j_2\ 1))\ i_1\ i_2 = (single\ i_1\ i_2\ 1) * M\ (single\ j_1\ j_2\ 1)$ 
have h_dual_def :  $\forall (i_1 : dIn) (j_1 : dOut) (i_2 : dIn) (j_2 : dOut), (M.\text{dual } (single\ j_1\ j_2\ 1))\ i_1\ i_2 = (single\ i_1\ i_2\ 1) * M\ (single\ j_1\ j_2\ 1)$ 
  intro i_1 j_1 i_2 j_2
  have h_dual_def :  $(M.\text{dual } (Matrix.single\ j_1\ j_2\ 1))\ i_1\ i_2 = \text{Matrix.trace } (single\ i_1\ i_2\ 1 * M\ (single\ j_1\ j_2\ 1))$ 
    simp [ Matrix.trace, Matrix.mul_apply ];
    simp [ Matrix.single ];
    rw [ Finset.sum_eq_single i_2 ]
    · aesop
    · intro b a a_1
      simp [ a_1.symm ]
    · aesop
  rw [ h_dual_def, ← Dual.trace_eq ];
  rw [ Matrix.trace ];
  rw [ Finset.sum_eq_single j_2 ] <=> aesop;

```

```

aesop

/--
If the Choi matrix of a map is positive semidefinite, then the Choi matrix of its dual is also positive semidefinite.
-/
lemma dual_choi_matrix_posSemidef_of_posSemidef (M : MatrixMap dIn dOut  $\alpha$ ) (h : M.choi_matrix.PosSemidef) :
  M.dual.choi_matrix.PosSemidef := by
  rw [dual_choi_matrix];
  simp +zetaDelta at *;
  apply_rules [Matrix.PosSemidef.submatrix];
  convert h.transpose using 1

/--
The dual of the identity map is the identity map.
-/
lemma dual_id : (MatrixMap.id dIn  $\alpha$ ).dual = MatrixMap.id dIn  $\alpha$  := by
  exact dual_unique (id dIn  $\alpha$ ) (id dIn  $\alpha$ ) fun A_1 => congrFun rfl

set_option maxHeartbeats 600000 in
/--
The dual of a Kronecker product of maps is the Kronecker product of their duals.
-/
lemma dual_kron {A B C D : Type*} [Fintype A] [Fintype B] [Fintype C] [Fintype D]
  [DecidableEq A] [DecidableEq B] [DecidableEq C] [DecidableEq D]
  (M : MatrixMap A B  $\alpha$ ) (N : MatrixMap C D  $\alpha$ ) :
  (M  $\otimes_{k_m}$  N).dual = M.dual  $\otimes_{k_m}$  N.dual := by
  have h_trace :  $\forall$  (X : Matrix (A  $\times$  C) (A  $\times$  C)  $\alpha$ ) (Y : Matrix (B  $\times$  D) (B  $\times$  D)  $\alpha$ ), (
    -- By definition of dual, we know that  $(M \times x_1 * y_1).trace = (x_1 * M.dual \ y_1).trace$ 
    have h_dual :  $\forall$  (x1 : Matrix A A  $\alpha$ ) (y1 : Matrix B B  $\alpha$ ), (M x1 * y1).trace = (x1 * M.dual y1).trace
    intro x1 y1
    convert MatrixMap.Dual.trace_eq M x1 y1 using 1
    have h_dual_N :  $\forall$  (x2 : Matrix C C  $\alpha$ ) (y2 : Matrix D D  $\alpha$ ), (N x2 * y2).trace = (x2 * N.dual y2).trace
    exact fun x2 y2 => MatrixMap.Dual.trace_eq N x2 y2;
    intro X Y;
    -- By definition of Kronecker product, we can write X and Y as sums of Kronecker products
    obtain ⟨X_sum, hX_sum⟩ :  $\exists$  X_sum : Finset (Matrix A A  $\alpha$   $\times$  Matrix C C  $\alpha$ ), X =  $\sum$  p in X_sum, p
    refine' { Finset.univ.image fun p : A  $\times$  A  $\times$  C  $\times$  C => (Matrix.of fun i j => if i = a  $\wedge$  j = a' then X (a, c) (a', c')
    ext (a, c) (a', c');
    rw [Finset.sum_apply, Finset.sum_apply];
    rw [Finset.sum_eq_single ( (Matrix.of fun i j => if i = a  $\wedge$  j = a' then X (a, c) (a', c')
    · intro a_1 b x x_1 x_2 x_3 a_2 a_3 a_4
      subst a_3 a_2
      contrapose! a_4; aesop;
    · exact fun h => False.elim ( h a a' c c' ( by ext i j; aesop ) ( by ext i j;
    obtain ⟨Y_sum, hY_sum⟩ :  $\exists$  Y_sum : Finset (Matrix B B  $\alpha$   $\times$  Matrix D D  $\alpha$ ), Y =  $\sum$  p in Y_sum, p
    use Finset.image (fun p => (Matrix.of (fun i j => Y (i, p.1) (j, p.2))), Matrix

```

```

ext {i, j} {k, l}; simp [ Matrix.kroneckerMap ] ;
rw [ Finset.sum_image ] <=> simp [ Matrix.sum_apply ];
· rw [ Finset.sum_eq_single ( j, l ) ] <=> aesop;
· intro p q h
  subst hX_sum
  simp_all only [Prod.mk.injEq, EmbeddingLike.apply_eq_iff_eq]
  obtain {fst, snd} := p
  obtain {fst_1, snd_1} := q
  obtain {left, right} := h
  simp_all only [Prod.mk.injEq]
  apply And.intro
  · have := congr_fun ( congr_fun right fst ) snd; aesop;
  · replace right := congr_fun ( congr_fun right fst ) snd; aesop;
-- By linearity of the trace and the properties of the Kronecker product
have h_expand : ∀ (x1 y1 : Matrix A A []) (x2 y2 : Matrix C C []) (x3 y3
intro x1 y1 x2 y2 x3 y3 x4 y4
simp [MatrixMap.kron_map_of_kron_state]
convert congr_arg2 ( · * · ) ( h_dual x1 x3 ) ( h_dual_N x2 x4 ) using
· simp only [Finset.sum_sigma', Finset.sum_mul _ _ _, Finset.mul_sum]
  refine' Finset.sum_bij ( fun x _ => { { x.fst.1, x.snd.1 }, { x.fst
· bound;
· exact fun b => { _, _, _, _, rfl };
· simp only [mul_assoc, Finset.mul_sum _ _ _, Finset.sum_mul];
  simp only [← Finset.sum_product', mul_left_comm];
  refine' Finset.sum_bij ( fun x _ => { x.1.2, x.2.2, x.1.1, x.2.1 } )
simp_all [ Matrix.trace_sum, Finset.sum_mul _ _ _ ];
simp [Matrix.mul_sum, h_expand]
apply dual_unique; assumption;

--The dual of a CompletelyPositive map is always CP, more generally it's k-
positive
-- see Lemma 3.1 of https://www.math.uwaterloo.ca/~krdavids/Preprints/CDPRp
theorem IsCompletelyPositive.dual (h : M.IsCompletelyPositive) : M.dual.IsC
intro n
have h_dual_pos : (MatrixMap.dual (M ⊗k m MatrixMap.id (Fin n) [])).IsPositi
  exact IsPositive.dual (h n);
-- By definition of complete positivity, we know that $(M ⊗k m id) dually
have h_dual_kron : (MatrixMap.dual (M ⊗k m MatrixMap.id (Fin n) [])) = (Mat
  convert dual_kron M ( MatrixMap.id ( Fin n ) [] ) using 1;
convert h_dual_pos using 1;
rw [ h_dual_kron, dual_id ]

/--
The composition of the dual of the inverse of the dual basis isomorphism wi
-/
lemma Module.Basis.dualMap_toDualEquiv_symm_comp_toDualEquiv {ι R M : Type*

```

```

    b.toDualEquiv.symm.toLinearMap.dualMap ∘₁ b.toDualEquiv.toLinearMap = (Module.eval
ext x f;
-- Since $b.toDual$ and $b.toDualEquiv.symm$ are inverses, we have $b.toDual (b.to
have h_inv : b.toDual (b.toDualEquiv.symm f) = f := by
  convert LinearEquiv.apply_symm_apply b.toDualEquiv f;
convert congr_arg ( fun g => g x ) h_inv using 1;
-- By definition of the dual basis, we know that $(b.toDual x) (b.toDualEquiv.symm
simp [Module.Basis.toDual];
ac_refl

/--
The composition of the inverse of the dual basis isomorphism with the dual of the du
-/
lemma Module.Basis.toDualEquiv_symm_comp_dualMap_toDualEquiv {ι R M : Type*} [Fintyp
  b.toDualEquiv.symm.toLinearMap ∘₁ b.toDualEquiv.toLinearMap.dualMap = (Module.ev
simp [ LinearMap.ext_iff ];
intro x
obtain ⟨y, hy⟩ : ∃ y, x = (Module.evalEquiv R M).toLinearMap y := by
  exact ⟨_, Eq.symm <| LinearEquiv.apply_symm_apply ( Module.evalEquiv R M ) x ⟩;
rw [ hy ];
simp [ Module.evalEquiv, LinearEquiv.symm_apply_eq ];
ext; simp [ Module.Dual.eval ] ;
simp [ Module.Basis.toDual ];
ac_refl

@[simp]
theorem dual_dual : M.dual.dual = M := by
  rw [dual, dual]
  simp only [← LinearMap.dualMap_comp_dualMap]
  have h₁ : (Matrix.stdBasis [] dOut dOut).toDualEquiv.symm.toLinearMap ∘₁
    ((Matrix.stdBasis [] dOut dOut).toDualEquiv).toLinearMap.dualMap =
    (Module.evalEquiv [] (Matrix dOut dOut [])).symm.toLinearMap := by
    apply Module.Basis.toDualEquiv_symm_comp_dualMap_toDualEquiv
  have h₂ : (Matrix.stdBasis [] dIn dIn).toDualEquiv.symm.toLinearMap.dualMap ∘₁
    (Matrix.stdBasis [] dIn dIn).toDualEquiv.toLinearMap =
    (Module.evalEquiv [] (Matrix dIn dIn [])).toLinearMap := by
    ext x y
    simp
    generalize Matrix.stdBasis [] dIn dIn = L
    -- Since $L$ is a basis, we can write $y$ as a linear combination of the basis e
    obtain ⟨c, hc⟩ : ∃ c : dIn × dIn → ℝ, y = ∑ i, c i • L.toDual (L i) := by
      have h_dual_basis : ∀ y : Module.Dual [] (Matrix dIn dIn []), ∃ c : dIn × dIn →
        intro y
        have h_dual_basis : y ∈ Submodule.span [] (Set.range (fun i => L.toDual (L i)
          have h_dual_basis : Submodule.span [] (Set.range (fun i => L.toDual (L i)))
            refine' Submodule.eq_top_of_finrank_eq _;

```

```

      rw [ finrank_span_eq_card ];
      · simp [ Module.finrank_eq_card_basis L ];
      · convert L.dualBasis.linearIndependent;
      exact h_dual_basis.symm ▸ Submodule.mem_top
      rw [ Finsupp.mem_span_range_iff_exists_finsupp ] at h_dual_basis;
      exact ( h_dual_basis.choose, by simpa [ Finsupp.sum_fintype ] using
        exact h_dual_basis y;
      subst hc
      simp [ map_sum, map_smul ];
      congr! 2;
      simp [ Module.Basis.toDualEquiv ];
      simp [ Module.Basis.toDual ]
      rw [← Module.Dual.eval_comp_comp_evalEquiv_eq]
      rw [← Module.evalEquiv_toLinearMap]
      simp only [← LinearMap.comp_assoc, LinearEquiv.comp_coe, LinearEquiv.self_t
        LinearMap.id_comp, h1]
      simp only [LinearMap.comp_assoc, LinearEquiv.comp_coe, LinearEquiv.self_t
        LinearMap.comp_id, h2]

end MatrixMap

namespace CPTPMap

variable [DecidableEq dIn] [DecidableEq dOut]

def dual (M : CPTPMap dIn dOut) : CPUMap dOut dIn where
  toLinearMap := M.map.dual
  unital := M.TP.dual
  cp := .dual M.cp

theorem dual_pos (M : CPTPMap dIn dOut) {T : HermitianMat dOut ℂ} (hT : 0 ≤
  0 ≤ M.dual T := by
  exact M.dual.pos_Hermitian hT

/-- The dual of a CPTP map preserves POVMs. Stated here just for two-
element POVMs, that is, an
operator `T` between 0 and 1. -/
theorem dual.PTP_POVM (M : CPTPMap dIn dOut) {T : HermitianMat dOut ℂ} (hT
  (0 ≤ M.dual T ∧ M.dual T ≤ 1) := by
  rcases hT with ⟨hT1, hT2⟩
  have hT_psd := HermitianMat.zero_le_iff.mp hT1
  use M.dual.pos_Hermitian hT1
  simpa using ContinuousOrderHomClass.map_monotone M.dual hT2

/-- The defining property of a dual channel, as specialized to `MState.exp_
/

```



```

theorem exp_val_Dual (dIn dOut : CPTPMap dIn dOut) (ρ : MState dIn) (T : HermitianMat dOut)
  (ρ ρ).exp_val T = ρ.exp_val (dIn.dual T) := by
  simp only [MState.exp_val, HermitianMat.inner_eq_re_trace, RCLike.re_to_complex]
  congr 1
  apply MatrixMap.Dual.trace_eq

```

```

end CPTPMap

```

```

section hermDual

```

```

--PULLOUT to Bundled.lean. Also use this to improve the definitions in POVM.lean.
def HPMat.ofHermitianMat {dOut : Type*} (f : HermitianMat dIn ℂ → ℝ) HermitianMat dOut
  toFun x := f (realPart x) + Complex.I • f (imaginaryPart x)
  map_add' x y := by
    simp only [map_add, HermitianMat.mat_add, smul_add]
    abel
  map_smul' c m := by
    have h_expand : realPart (c • m) = c.re • realPart m - c.im • imaginaryPart m ∧
      imaginaryPart (c • m) = c.re • imaginaryPart m + c.im • realPart m := by
      simp only [Subtype.ext_iff, AddSubgroupClass.coe_sub, selfAdjoint.val_smul,
        AddSubgroup.coe_add, realPart, selfAdjointPart_apply_coe, invOf_eq_inv, star
          smul_add, imaginaryPart, LinearMap.coe_comp, Function.comp_apply,
          skewAdjoint.negISMul_apply_coe, skewAdjointPart_apply_coe,
          ← Matrix.ext_iff, Matrix.add_apply, Matrix.smul_apply, smul_eq_mul, Complex.
            Complex.ofReal_inv, Complex.ofReal_ofNat, Matrix.star_apply, RCLike.star_def,
            Matrix.sub_apply, Complex.ext_iff, Complex.add_re, Complex.mul_re, Complex.i
              Complex.normSq_ofNat, Complex.mul_im, Complex.conj_re, Complex.conj_im, Comp
                Complex.sub_re, Complex.sub_im, Complex.add_im, Complex.neg_re, Complex.neg
                  ring_nf
                    simp
                      ext
                        simp only [h_expand, map_sub, map_smul, map_add, Matrix.add_apply, Matrix.smul_a
                          smul_eq_mul, RingHom.id_apply, Complex.ext_iff, Complex.add_re, Complex.mul_re
                            Complex.I, Complex.mul_im, Complex.add_im]
                        simp only [HermitianMat.mat_sub, HermitianMat.mat_smul, Matrix.sub_apply, Matrix
                          Complex.real_smul, Complex.sub_re, Complex.mul_re, Complex.ofReal_re, Complex.
                            zero_mul, sub_zero, HermitianMat.mat_add, Matrix.add_apply, Complex.add_re, Co
                              Complex.mul_im, add_zero, one_mul, zero_sub, neg_add_rev, zero_add, Complex.su
                                ring_nf
                                  simp
                                    HP _ h := by
                                      apply Matrix.IsHermitian.add
                                        · apply HermitianMat.H
                                        · simp [IsSelfAdjoint.imaginaryPart h]

```

```

--PULLOUT

```

```

@[simp]
theorem HMap.linearMap_ofHermitianMat (f : HermitianMat dIn  $\mathbb{C}$   $\rightarrow$   $\mathbb{R}$ ] Hermitian
  LinearMapClass.linearMap (HMap.ofHermitianMat f) = f := by
  ext1 (x, hx)
  ext1
  simp only [ofHermitianMat, LinearMap.coe_coe]
  simp only [HMap.instFunLike, HMap.map, HermitianMat.mat_mk,
    LinearMap.coe_mk, AddHom.coe_mk]
  conv => enter [2, 1, 2, 1]; rw [← realPart_add_I_smul_imaginaryPart x]
  suffices imaginaryPart x = 0 by simp [this]
  simp [imaginaryPart, skewAdjoint.negISMul, show star x = x from hx]

--PULLOUT
omit [Fintype dOut] in
@[simp]
theorem HMap.ofHermitianMat_linearMap (f : HMap dIn dOut  $\mathbb{C}$ ) :
  ofHermitianMat (LinearMapClass.linearMap f) = f := by
  ext : 3
  simp only [map, ofHermitianMat, instFunLike, LinearMap.coe_coe, Hermitian
    HermitianMat.mat_mk, LinearMap.coe_mk, AddHom.coe_mk,
    ← map_smul, ← map_add]
  simp only [map_add, map_smul, realPart, imaginaryPart, LinearMap.coe_comp]
  simp only [selfAdjointPart, LinearMap.coe_mk, AddHom.coe_mk,
    HermitianMat.mat_mk, LinearMap.map_smul_of_tower, skewAdjoint.negISMul]
  simp only [Matrix.add_apply, Matrix.smul_apply, smul_eq_mul]
  ring_nf
  simp
  ring

variable (f : HMap dIn dOut) (A : HermitianMat dIn  $\mathbb{C}$ )

--Can define one for HMap's that has 'easier' definitional properties, use
--doesn't go through Module.Basis the same way. Requires the equivalence be
linear maps of HermitianMats
--and  $\mathbb{C}$ -linear maps of matrices.
def HMap.hermDual : HMap dOut dIn :=
  HMap.ofHermitianMat (LinearMapClass.linearMap f).adjoint

@[simp]
theorem HMap.hermDual_hermDual : f.hermDual.hermDual = f := by
  simp [hermDual]

open RealInnerProductSpace

/-- The defining property of a dual map: inner products are preserved on th

```

```

/
theorem HPMat.inner_hermDual (B : HermitianMat dOut ℂ) :
  ∀ f A, B = A, f.hermDual B := by
  change (LinearMapClass.linearMap f) A, B = A, (LinearMapClass.linearMap f.hermDual A)
  rw [hermDual, ← LinearMap.adjoint_inner_right, HPMat.linearMap_ofHermitianMat]

/-- Version of `HPMat.inner_hermDual` that uses HermitianMat.inner directly. TODO clean up
/
theorem HPMat.inner_hermDual' (B : HermitianMat dOut ℂ) :
  ∀ f A, B = A, f.hermDual B :=
  HPMat.inner_hermDual f A B

/-- The dual of a `IsPositive` map also `IsPositive`. -/
theorem MatrixMap.IsPositive.hermDual (h : MatrixMap.IsPositive f.map) : f.hermDual.IsPositive
  unfold IsPositive at h ⊢
  intro x hx
  set xH : HermitianMat dOut ℂ := ⟨x, hx.left⟩ with hxH
  have hx' : x = xH := rfl; clear_value xH; subst x; clear hxH
  change Matrix.PosSemidef (f.hermDual xH).mat
  rw [← HermitianMat.zero_le_iff] at hx ⊢
  classical
  rw [HermitianMat.nonneg_iff_inner_nonneg]
  intro y hy
  rw [HermitianMat.zero_le_iff] at hy
  specialize h hy
  change Matrix.PosSemidef (f y).mat at h
  rw [← HermitianMat.zero_le_iff] at h
  rw [HPMat.inner_hermDual, HPMat.hermDual_hermDual]
  apply HermitianMat.inner_ge_zero hx h

/-- The dual of TracePreserving map is *not* trace-preserving, it's *unital*, that is
/
theorem HPMat.hermDual_Unital [DecidableEq dIn] [DecidableEq dOut] (h : MatrixMap.IsTracePreserving f.map) :
  f.hermDual.map.Unital := by
  suffices f.hermDual 1 = 1 by --todo: make this is an accessible 'constructor' for
  rw [HermitianMat.ext_iff] at this
  exact this
  open RealInnerProductSpace in
  apply ext_inner_left ℝ
  intro v
  rw [← HPMat.inner_hermDual]
  rw [HermitianMat.inner_one, HermitianMat.inner_one] --TODO change to Inner.inner
  exact congr(Complex.re $(h v)) --TODO: HPMat with IsTracePreserving give the HermitianMat
alias MatrixMap.IsTracePreserving.hermDual := HPMat.hermDual_Unital

```

```

namespace PTPMap

variable [DecidableEq dIn] [DecidableEq dOut]

def hermDual (M : PTPMap dIn dOut) : PUMap dOut dIn where
  toHMap := M.toHMap.hermDual
  pos := M.pos.hermDual
  unital := M.TP.hermDual

theorem hermDual_pos (M : PTPMap dIn dOut) {T : HermitianMat dOut  $\mathbb{C}$ } (hT :
   $0 \leq M.hermDual\ T$  := by
  exact M.hermDual.pos_Hermitian hT

/-- The dual of a PTP map preserves POVMs. Stated here just for two-
element POVMs, that is, an
operator `T` between 0 and 1. -/
theorem hermDual.PTP_POVM (M : PTPMap dIn dOut) {T : HermitianMat dOut  $\mathbb{C}$ } (
  ( $0 \leq M.hermDual\ T \wedge M.hermDual\ T \leq 1$ ) := by
  rcases hT with (hT1, hT2)
  have hT_psd := HermitianMat.zero_le_iff.mp hT1
  use M.hermDual.pos_Hermitian hT1
  simpa using ContinuousOrderHomClass.map_monotone M.hermDual hT2

/-- The defining property of a dual channel, as specialized to `MState.exp_
/
theorem exp_val_hermDual ( $\square$  : PTPMap dIn dOut) ( $\rho$  : MState dIn) (T : Hermit
  ( $\square\ \rho$ ).exp_val T =  $\rho$ .exp_val ( $\square.hermDual\ T$ ) := by
  simp only [MState.exp_val]
  apply HMap.inner_hermDual'

end PTPMap

end hermDual

```

1.427 QuantumInfo/Finite/CPTPMap/MatrixMap.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.LinearAlgebra.TensorProduct.Matrix
import Mathlib.LinearAlgebra.PiTensorProduct
import Mathlib.Data.Set.Card

```

```
import Mathlib.Algebra.Module.LinearMap.Basic
import QuantumInfo.ForMathlib
import QuantumInfo.Finite.Braket
import QuantumInfo.Finite.MState
```

```
/-! # Linear maps of matrices
```

This file works with ``MatrixMap`s`, that is, linear maps from square matrices to square matrices. Although this is just a shorthand for ``Matrix A A R →1[R] Matrix B B R``, there are some concepts that specifically make sense in this context.

```
* `toMatrix` is the rectangular "transfer matrix", where matrix multiplication commutes
* `choi_matrix` is the square "Choi matrix", see `MatrixMap.choi_PSD_iff_CP_map` for details
* `kron` is the Kronecker product of matrix maps
* `IsTracePreserving` states the trace of the output is always equal to the trace of the input
```

We provide some lemmas for relating these facts, prove basic facts e.g. composition of linear maps about ``IsTracePreserving`` maps.

```
-/
```

```
/-- A `MatrixMap` is a linear map between square matrices of size A to size B, over semiring R
/
```

```
abbrev MatrixMap (A B R : Type*) [Semiring R] := Matrix A A R →1[R] Matrix B B R
```

```
variable {A B C D E F R : Type*} [Fintype A] [DecidableEq A]
```

```
namespace MatrixMap
section matrix
```

```
variable [Semiring R]
```

```
variable (A R) in
```

```
/-- Alias of LinearMap.id, but specifically as a MatrixMap. -
```

```
/
```

```
@[reducible]
```

```
def id : MatrixMap A A R := LinearMap.id
```

```
/-- Choi matrix of a given linear matrix map. Note that this is defined even for maps that aren't CPTP, it's just rarely talked about in those contexts. This is the inverse of `MatrixMap.of_choi_matrix`. Compare with `MatrixMap.toMatrix`, which gives the transfer matrix.
/
```

```
def choi_matrix (M : MatrixMap A B R) : Matrix (B × A) (B × A) R :=
  fun (j1, i1) (j2, i2) ↦ M (Matrix.single i1 i2 1) j1 j2
```

```
/-- Given the Choi matrix, generate the corresponding R-linear map between matrices MatrixMap. This is the inverse of `MatrixMap.choi_matrix`. -
```

```

/
def of_choi_matrix (M : Matrix (B × A) (B × A) R) : MatrixMap A B R where
  toFun X := fun b₁ b₂ ↦ ∑ (a₁ : A), ∑ (a₂ : A), X a₁ a₂ * M (b₁, a₁) (b₂,
  map_add' x y := by funext b₁ b₂; simp [add_mul, Finset.sum_add_distrib]
  map_smul' r x := by
    funext b₁ b₂
    simp only [Matrix.smul_apply, smul_eq_mul, RingHom.id_apply, Finset.mul]

/-- Proves that `MatrixMap.of_choi_matrix` and `MatrixMap.choi_matrix` inverse
/
@[simp]
theorem map_choi_inv (M : Matrix (B × A) (B × A) R) : choi_matrix (of_choi_
  ext {i₁,i₂} {j₁,j₂})
  simp [of_choi_matrix, choi_matrix, Matrix.single, ite_and]

/-- Proves that `MatrixMap.choi_matrix` and `MatrixMap.of_choi_matrix` inverse
/
@[simp]
theorem choi_map_inv (M : MatrixMap A B R) : of_choi_matrix (choi_matrix M)
  -- By definition of `MatrixMap.of_choi_matrix`, we know that applying it
  ext X b₁ b₂; simp [MatrixMap.of_choi_matrix, MatrixMap.choi_matrix];
  -- By linearity of  $M$ , we can distribute  $M$  over the sum.
  have h_linear : M X = ∑ x : A, ∑ x₁ : A, X x x₁ • M (Matrix.single x x₁)
  have h_linear : M X = M (∑ x : A, ∑ x₁ : A, X x x₁ • Matrix.single x x₁)
    congr with i j ; simp ( config := { decide := Bool.true } ) [ Matrix.
    simp ( config := { decide := Bool.true } ) [ Matrix.single ];
    rw [ Finset.sum_eq_single i ] <=> aesop;
    simp +decide only [h_linear, map_sum, LinearMap.map_smul_s1];
    simp +zetaDelta at *;
  -- By linearity of  $M$ , we can distribute  $M$  over the sum and then apply
  simp [h_linear, Matrix.sum_apply]

/-- The correspondence induced by `MatrixMap.of_choi_matrix` is injective.
/
theorem choi_matrix_inj : Function.Injective (@choi_matrix A B R _ _) := by
  intro _ _ h
  simpa only [choi_map_inv] using congrArg of_choi_matrix h

variable {R : Type*} [CommSemiring R]

/-- The linear equivalence between linear maps of matrices, and Choi matrices
/
@[simps]
def choi_equiv : MatrixMap A B R ≈1[R] Matrix (B × A) (B × A) R where
  toFun := choi_matrix

```

```

    invFun := of_choi_matrix
    left_inv _ := by simp
    right_inv _ := by simp
    map_add' _ _ := by ext; simp [choi_matrix]
    map_smul' _ _ := by ext; simp [choi_matrix]

/-- The linear equivalence between MatrixMap's and transfer matrices on a larger space.
Compare with `MatrixMap.choi_matrix`, which gives the Choi matrix instead of the transfer matrix.
/
noncomputable def toMatrix [Fintype B] : MatrixMap A B R  $\approx$ ₗ[R] Matrix (B × B) (A × A)
  LinearMap.toMatrix (Matrix.stdBasis R A A) (Matrix.stdBasis R B B)

/-- Multiplication of transfer matrices, `MatrixMap.toMatrix`, is equivalent to composition.
/
theorem toMatrix_comp [Fintype B] [Fintype C] [DecidableEq B] (M₁ : MatrixMap A B R) (M₂ : MatrixMap B C R)
  LinearMap.toMatrix_comp _ _ M₂ M₁

end matrix

section kraus

variable [Star R] [CommSemiring R]
variable {κ : Type*} [Fintype κ]

/-- Construct a matrix map out of families of matrices  $M N : \Sigma \rightarrow \text{Matrix } B A R$ 
indexed by  $\kappa$  via  $X \mapsto \sum k : \kappa, (M k) * X * (N k)^H$  -/
def of_kraus (M N : κ → Matrix B A R) : MatrixMap A B R :=
  ∑ k : κ, {
    toFun X := M k * X * (N k).conjTranspose
    map_add' x y := by rw [Matrix.mul_add, Matrix.add_mul]
    map_smul' r x := by rw [RingHom.id_apply, Matrix.mul_smul, Matrix.smul_mul]
  }

def exists_kraus (Φ : MatrixMap A B R) : ∃ r : ℕ, ∃ (M N : Fin r → Matrix B A R), Φ
  sorry

end kraus

section submatrix

variable {A B : Type*} (R : Type*) [Semiring R]

/-- The `MatrixMap` corresponding to applying a `submatrix` operation on each side.
/
@[simps]
def submatrix (f : B → A) : MatrixMap A B R where

```

```

toFun x := x.submatrix f f
map_add' := by simp [Matrix.submatrix_add]
map_smul' := by simp [Matrix.submatrix_smul]

@[simp]
theorem submatrix_id : submatrix R _root_.id = id A R := by
  ext1; simp

@[simp]
theorem submatrix_comp (f : C → B) (g : B → A) :
  submatrix R f ∘1 submatrix R g = submatrix R (g ∘ f) := by
  ext1; simp

end submatrix

section kron
open Kronecker

variable {A B C D R : Type*} [Fintype A] [Fintype B] [Fintype C] [Fintype D]
variable [DecidableEq A] [DecidableEq C]

/-- The Kronecker product of MatrixMaps. Defined here using `TensorProduct`
reindexing operations and `LinearMap.toMatrix`/`Matrix.toLin`. Notation `⊗k`
/
noncomputable def kron [CommSemiring R] (M1 : MatrixMap A B R) (M2 : MatrixMap C D R) : MatrixMap (A × C) (B × D) R :=
  let h1 := (LinearMap.toMatrix (Module.Basis.tensorProduct (Matrix.stdBasis A B) (Matrix.stdBasis C D))
    (Module.Basis.tensorProduct (Matrix.stdBasis R B) (Matrix.stdBasis R D))
    (TensorProduct.map M1 M2));
  let r1 := Equiv.prodProdProdComm B B D D;
  let r2 := Equiv.prodProdProdComm A A C C;
  let h2 := Matrix.reindex r1 r2 h1;
  Matrix.toLin (Matrix.stdBasis R (A × C) (A × C)) (Matrix.stdBasis R (B × D) (B × D)) h2

scoped[MatrixMap] infixl:100 " ⊗k " => MatrixMap.kron

set_option maxHeartbeats 800000 in
set_option synthInstance.maxHeartbeats 60000 in
/-- The extensional definition of the Kronecker product `MatrixMap.kron`, i.e.
/
theorem kron_def [CommSemiring R] (M1 : MatrixMap A B R) (M2 : MatrixMap C D R) :
  (M1 ⊗k M2) M (b1, d1) (b2, d2) = ∑ a1, ∑ a2, ∑ c1, ∑ c2,
  (M1 (Matrix.single a1 a2 1) b1 b2) * (M2 (Matrix.single c1 c2 1) d1 d2)
  rw [kron]
  have h_expand : (Matrix.toLin (Matrix.stdBasis R (A × C) (A × C)) (Matrix.toLin (Matrix.stdBasis R (B × D) (B × D)) h2) M) (a1, c1) =
  have h_expand : M = ∑ a1 : A, ∑ a2 : A, ∑ c1 : C, ∑ c2 : C, M (a1, c1)
  ext {a1, c1} {a2, c2}

```



```

    simp only [Matrix.single, Matrix.sum_apply]
    rw [Finset.sum_eq_single a1, Finset.sum_eq_single a2, Finset.sum_eq_single c1,
    <=> simp +contextual
    nth_rw 1 [h_expand]
    simp only [map_sum, LinearMap.map_smuls1]
    rfl
    rw [h_expand]
    clear h_expand
    simp only [Matrix.sum_apply]
    congr! 8 with a1 _ a2 _ c1 _ c2 _
    rw [Matrix.smul_apply, smul_eq_mul, mul_comm]
    congr
    classical
    simp only [Matrix.stdBasis,
    Matrix.reindex_apply, Equiv.prodProdProdComm_symm, Matrix.toLin_apply,
    Matrix.mulVec, dotProduct, Matrix.submatrix_apply, Equiv.prodProdProdComm_apply,
    Module.Basis.tensorProduct_apply, Module.Basis.map_apply, Module.Basis.coe_reind
    Equiv.sigmaEquivProd_symm_apply, Pi.basis_apply, Pi.basisFun_apply, Matrix.coe_o
    Module.Basis.tensorProduct_repr_tmul_apply, Module.Basis.map_repr, LinearEquiv.t
    Module.Basis.repr_reindex, Finsupp.mapDomain_equiv_apply, Pi.basis_repr, Pi.basi
    Matrix.of_symm_single, Pi.single_apply, Matrix.smul_of, Matrix.sum_apply, Matrix
    rw [ Finset.sum_eq_single ( ( b1, d1 ), ( b2, d2 ) ) ]
    · rw [ Finset.sum_eq_single ( ( a1, c1 ), ( a2, c2 ) ) ]
    · simp only [↓reduceIte, Pi.single_eq_same, mul_one]
      rw [ mul_comm ]
      congr! 2
      · ext i j
        by_cases hi : i = a1
        <=> by_cases hj : j = a2
        <=> simp only [hi, hj, Matrix.of_apply, ne_eq, not_false_eq_true, Pi.single_
          Pi.single_eq_same, Pi.zero_apply, Matrix.single]
        <=> grind only
      · ext i j
        by_cases hi : i = c1
        <=> by_cases hj : j = c2
        <=> simp only [hi, hj, Matrix.of_apply, ne_eq, not_false_eq_true, Pi.single_
          Pi.single_eq_same, Pi.zero_apply, Matrix.single]
        <=> grind only
    · intros
      split
      · grind [Prod.mk.injEq, Pi.single_eq_of_ne, mul_zero]
      · simp
    · simp
  · simp only [Finset.mem_univ, ne_eq, forall_const, Prod.forall, Prod.mk.injEq, not
    intro a b c d h
    split_ifs

```

```

      · simp_all
      · simp
    · simp

section kron_lemmas
variable [CommSemiring R]

theorem add_kron (ML1 ML2 : MatrixMap A B R) (MR : MatrixMap C D R) : (ML1
  simp [kron, TensorProduct.map_add_left, Matrix.submatrix_add]

theorem kron_add (ML : MatrixMap A B R) (MR1 MR2 : MatrixMap C D R) : ML  $\otimes_k$ 
  simp [kron, TensorProduct.map_add_right, Matrix.submatrix_add]

theorem smul_kron (r : R) (ML : MatrixMap A B R) (MR : MatrixMap C D R) : (
  simp [kron, TensorProduct.map_smul_left, Matrix.submatrix_smul]

theorem kron_smul (r : R) (ML : MatrixMap A B R) (MR : MatrixMap C D R) : M
  simp [kron, TensorProduct.map_smul_right, Matrix.submatrix_smul]

@[simp]
theorem zero_kron (MR : MatrixMap C D R) : (0 : MatrixMap A B R)  $\otimes_{k\ m}$  MR = 0
  simp [kron]

@[simp]
theorem kron_zero (ML : MatrixMap A B R) : ML  $\otimes_{k\ m}$  (0 : MatrixMap C D R) = 0
  simp [kron]

variable [DecidableEq B] in
theorem kron_id_id : (id A R  $\otimes_{k\ m}$  id B R) = id (A  $\times$  B) R := by
  simp [kron]

variable {Dl1 Dl2 Dl3 Dr1 Dr2 Dr3 : Type*}
  [Fintype Dl1] [Fintype Dl2] [Fintype Dl3] [Fintype Dr1] [Fintype Dr2] [Fi
  [DecidableEq Dl1] [DecidableEq Dl2] [DecidableEq Dr1] [DecidableEq Dr2] i
/-- For maps L1, L2, R1, and R2, the product (L2  $\circ_1$  L1)  $\otimes_{k\ m}$  (R2  $\circ_1$  R1) = (L
/
theorem kron_comp_distrib (L1 : MatrixMap Dl1 Dl2 R) (L2 : MatrixMap Dl2 Dl
  (R2 : MatrixMap Dr2 Dr3 R) : (L2  $\circ_1$  L1)  $\otimes_{k\ m}$  (R2  $\circ_1$  R1) = (L2  $\otimes_{k\ m}$  R2)  $\circ_1$ 
  simp [kron, TensorProduct.map_comp, ← Matrix.toLin_mul, Matrix.submatrix_

end kron_lemmas

-- /-- The canonical tensor product on linear maps between matrices, where
--   M[A,B] to M[C,D] is given by M[A $\times$ C,B $\times$ D]. This tensor product acts inde
--   Kronecker products and gives Kronecker products as outputs. -
/

```

```

-- def matrixMap_kron (M1 : Matrix (A1 × B1) (C1 × D1) R) (M2 : Matrix (A2 × B2) (C2 × D2) R)
--   Matrix.of fun ((a1, a2), (b1, b2)) ((c1, c2), (d1, d2)) ↦
--     (M1 (a1, b1) (c1, d1)) * (M2 (a2, b2) (c2, d2))

/-- The operational definition of the Kronecker product `MatrixMap.kron`, that it maps
inputs to the Kronecker product of outputs. It is the unique bilinear map doing so.
/
theorem kron_map_of_kron_state [CommRing R] (M1 : MatrixMap A B R) (M2 : MatrixMap C D R)
  ext bd1 bd2
  let (b1, d1) := bd1
  let (b2, d2) := bd2
  rw [kron_def]
  simp only [Matrix.kroneckerMap_apply]
  simp_rw [mul_assoc, ← Finset.mul_sum]
  simp_rw [mul_comm (M2 _ _), mul_assoc, ← Finset.mul_sum, ← mul_assoc]
  simp_rw [← Finset.sum_mul]
  congr
  --TODO: Cleanup, these two branches are nearly identical (separate lemma?)
  · have h_linear : M1 MA = ∑ i : A, ∑ i_1 : A, MA i i_1 • M1 (Matrix.single i i_1 1)
    have h_linear : M1 MA = M1 (∑ i : A, ∑ i_1 : A, Matrix.single i i_1 (MA i i_1))
      congr;
      exact Matrix.matrix_eq_sum_single MA
    simp [h_linear, Matrix.single]
    congr! 2 with i _ j _
    convert M1.map_smul (MA i j) (Matrix.of fun i' j' ↦ if i = i' ∧ j = j' then 1 else 0)
    ext
    simp
    simp [h_linear, mul_comm, Matrix.sum_apply]
  · have h_expand : M2 MC = ∑ i : C, ∑ j : C, MC i j • M2 (Matrix.single i j 1) := by
    have h_expand : MC = ∑ i : C, ∑ j : C, MC i j • Matrix.single i j 1 := by
      ext i j
      simp [Matrix.sum_apply, Matrix.single]
      rw [Finset.sum_eq_single i] <=> aesop
    conv_lhs => rw [h_expand];
    simp [map_sum]
    congr! 2 with i _ j _
    rw [← M2.map_smul (MC i j) (Matrix.single i j 1)]
    exact congr_arg _ (by ext; simp [Matrix.single])
    simp [h_expand, Matrix.sum_apply]

theorem choi_matrix_state_rep {B : Type*} [Fintype B] [Nonempty A] (M : MatrixMap A B)
  M.choi_matrix = (↑(Fintype.card (α := A)) : ℂ) • (M ⊗k m (LinearMap.id : MatrixMap A A))
  ext i j
  simp [choi_matrix, kron_def M, Ket.MES, Ket.apply, Finset.mul_sum]
  conv =>
  rhs

```

```

conv =>
  enter [2, x, 2, a_1]
  conv =>
    enter [2, a_2]
    simp [apply_ite]
  simp only [Finset.sum_ite_eq, Finset.mem_univ, ↓reduceIte]
  rw [← mul_inv, ← Complex.ofReal_mul, ← Real.sqrt_mul (Fintype.card A)
    Real.sqrt_mul_self (Fintype.card A).cast_nonneg', mul_comm, mul_ass]
  simp
  conv =>
    right
    rw [Matrix.single, Matrix.of_apply]
    enter [1]
    rw [and_comm]
  simp [apply_ite, ite_and]
conv =>
  enter [2, x]
  simp [Finset.sum_ite]
simp [Finset.sum_ite]

theorem submatrix_kron_submatrix [CommSemiring R] (f : B → A) (g : D → C) :
  submatrix R f  $\otimes_{k\ m}$  submatrix R g = submatrix R (Prod.map f g) := by
  ext m i j
  rw [kron_def]
  simp [Prod.map, Matrix.single, ite_and]

theorem submatrix_kron_id [CommSemiring R] (f : B → A) :
  submatrix R f  $\otimes_{k\ m}$  id C R = submatrix R (Prod.map f _root_.id) := by
  simp [← submatrix_kron_submatrix]

theorem id_kron_submatrix [CommSemiring R] (f : B → A) :
  id C R  $\otimes_{k\ m}$  submatrix R f = submatrix R (Prod.map _root_.id f) := by
  simp [← submatrix_kron_submatrix]

end kron

section pi
section basis

--Missing from Mathlib

variable {ι : Type*}
variable {R : Type*} [CommSemiring R]
variable {s : ι → Type*} [∀ i, AddCommMonoid (s i)] [∀ i, Module R (s i)]
variable {L : ι → Type* }

```

```

/-- Like `Basis.tensorProduct`, but for `PiTensorProduct` -
/
noncomputable opaque _root_.Module.Basis.piTensorProduct [Vi, Fintype (L i)]
  (b : (i:ι) → Module.Basis (L i) R (s i)) :
  Module.Basis ((i:ι) → L i) R (PiTensorProduct R s) :=
  --Marking as opaque so that ATPs don't run into unpacking the sorried data.
  --TODO: This is actually defined appropriately in a later Mathlib version.
  Finsupp.basisSingleOne.map sorry

end basis

variable {R : Type*} [CommSemiring R]
variable {ι : Type u} [DecidableEq ι] [fι : Fintype ι]
variable {dI : ι → Type v} [Vi, Fintype (dI i)] [Vi, DecidableEq (dI i)]
variable {d0 : ι → Type w} [Vi, Fintype (d0 i)] [Vi, DecidableEq (d0 i)]

/-- Finite Pi-type tensor product of MatrixMaps. Defined as `PiTensorProduct.tprod`
Linear maps. Notation  $\square_{tm}[R] i, f i$ , eventually. -/
noncomputable def piProd (Λi : ∀ i, MatrixMap (dI i) (d0 i) R) : MatrixMap (Vi, dI i) R :=
  let map₁ := PiTensorProduct.map Λi;
  let map₂ := LinearMap.toMatrix
    (Module.Basis.piTensorProduct (fun i ↦ Matrix.stdBasis R (dI i) (dI i)))
    (Module.Basis.piTensorProduct (fun i ↦ Matrix.stdBasis R (d0 i) (d0 i))) map₁
  let r₁ : ((i : ι) → d0 i × d0 i) ≈ ((i : ι) → d0 i) × ((i : ι) → d0 i) := Equiv.ar
  let r₂ : ((i : ι) → dI i × dI i) ≈ ((i : ι) → dI i) × ((i : ι) → dI i) := Equiv.ar
  let map₃ := Matrix.reindex r₁ r₂ map₂;
  Matrix.toLin
    (Matrix.stdBasis R ((i:ι) → dI i) ((i:ι) → dI i))
    (Matrix.stdBasis R ((i:ι) → d0 i) ((i:ι) → d0 i)) map₃

-- notation3:100 " $\square_{tm}$ " "...)", "r:(scoped f => tprod R f) => r
-- syntax (name := bigsum) " $\sum$ " bigOpBinders ("with " term)? ", " term:67 : term

/--
Composition of `MatrixMap.piProd` maps distributes over the tensor product.
-/
theorem piProd_comp
  {d₁ d₂ d₃ : ι → Type*}
  [V i, Fintype (d₁ i)] [V i, DecidableEq (d₁ i)]
  [V i, Fintype (d₂ i)] [V i, DecidableEq (d₂ i)]
  [V i, Fintype (d₃ i)] [V i, DecidableEq (d₃ i)]
  (Λ₁ : ∀ i, MatrixMap (d₁ i) (d₂ i) R) (Λ₂ : ∀ i, MatrixMap (d₂ i) (d₃ i) R) :
  piProd (fun i ↦ (Λ₂ i) ∘₁ (Λ₁ i)) = (piProd Λ₂) ∘₁ (piProd Λ₁) := by
  simp [piProd, PiTensorProduct.map_comp, ← Matrix.toLin_mul]
  rw [← LinearMap.toMatrix_comp]

```

end pi

1.428 QuantumInfo/Finite/CPTPMap/Unbundled.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.CPTPMap.MatrixMap

/-! # Properties of Matrix Maps

Building on `MatrixMap`s, this defines the properties: `IsTracePreserving`,
`IsHermitianPreserving`, `IsPositive` and `IsCompletelyPositive`. They have
such as closure under composition, addition, and scaling.

These are the unbundled versions, which just state the relevant properties.
The bundled versions are `HPMap`, `UnitalMap`, `TPMap`, `PMap`, and `CPMap`
in Bundled.lean.
-/

namespace MatrixMap

variable {A B C D : Type*} [Fintype A] [Fintype B] [Fintype C] [Fintype D]
variable {κ [] : Type*} [Fintype κ] [RCLike []]

section tp

variable {R : Type*} [Semiring R]
variable {M : MatrixMap A B R} {M₂ : MatrixMap B C R}

/-- A linear matrix map is trace preserving if trace of the output equals t
/
def IsTracePreserving (M : MatrixMap A B R) : Prop :=
  ∀ (x : Matrix A A R), (M x).trace = x.trace

/-- A map is trace preserving iff the partial trace of the Choi matrix is t
/
theorem IsTracePreserving_iff_trace_choi [DecidableEq A] (M : MatrixMap A B R)
  ↔ M.choi_matrix.traceLeft = 1 := by
  constructor
  · intro h
    ext a₁ a₂

```

```

    replace h := h (Matrix.single a₁ a₂ 1)
    simp_rw [Matrix.trace, Matrix.diag] at h
    simp only [Matrix.traceLeft, choi_matrix, Matrix.of_apply, h]
    simp only [Matrix.single, Matrix.of_apply, Finset.sum_boole, Matrix.one_apply]
    have : (fun x => a₁ = x ∧ a₂ = x) = (fun x => a₁ = a₂ ∧ a₂ = x) := by
      funext x
      rw [eq_iff_iff, and_congr_left_iff]
      rintro rfl
      trivial
    split_ifs with h
    <=> simp [this, h, Finset.filter_eq]
  · intro h X
    replace h := fun (a₁ a₂ : A) ↦ congrFun₂ h a₁ a₂
    simp [Matrix.traceLeft, Matrix.trace] at h
    rw [← M.choi_map_inv, of_choi_matrix]
    dsimp
    rw [Finset.sum_comm_cycle, Finset.sum_comm_cycle]
    simp_rw [← Finset.mul_sum, h, Matrix.one_apply]
    simp

namespace IsTracePreserving

/-- Simp lemma: the trace of the image of a IsTracePreserving map is the same as the
/
@[simp]
theorem apply_trace (h : M.IsTracePreserving) (ρ : Matrix A A R)
  : (M ρ).trace = ρ.trace :=
  h ρ

/-- The trace of a Choi matrix of a TP map is the cardinality of the input space. -
/
theorem trace_choi [DecidableEq A] (h : M.IsTracePreserving) :
  M.choi_matrix.trace = (Finset.univ (α := A)).card := by
  rw [← Matrix.traceLeft_trace, (IsTracePreserving_iff_trace_choi M).mp h,
    Matrix.trace_one, Finset.card_univ]

/-- The composition of IsTracePreserving maps is also trace preserving. -
/
theorem comp (h₁ : M.IsTracePreserving) (h₂ : M₂.IsTracePreserving) :
  IsTracePreserving (M₂ ∘₁ M) := by
  intro x
  simp [h₂ _, h₁ _]

/-- The identity MatrixMap IsTracePreserving. -/
@[simp]
theorem id : (id A R).IsTracePreserving := by

```

```

simp [IsTracePreserving, MatrixMap.id]

variable {R : Type*} [CommSemiring R] in
/-- Unit linear combinations of IsTracePreserving maps are IsTracePreserving
/
theorem unit_linear {M1 M2 : MatrixMap A B R} {x y : R}
  (h1 : M1.IsTracePreserving) (h2 : M2.IsTracePreserving) (hxy : x + y =
    (x • M1 + y • M2).IsTracePreserving := by
  rw [IsTracePreserving] at h1 h2 ⊢
  simp [h1, h2, ← add_mul, hxy]

variable {R : Type*} [CommSemiring R] [DecidableEq C] [DecidableEq A] in
/-- The kronecker product of IsTracePreserving maps is also trace preserving
/
theorem kron {M1 : MatrixMap A B R} {M2 : MatrixMap C D R} (h1 : M1.IsTracePreserving)
  (M1 ⊗k m M2).IsTracePreserving := by
  intro x
  simp_rw [Matrix.trace, Matrix.diag]
  rw [Fintype.sum_prod_type, Fintype.sum_prod_type]
  simp_rw [kron_def]
  have h_simp : ∑ x_1, ∑ x_2, ∑ a_1, ∑ a_2, ∑ c_1, ∑ c_2,
    M1 (Matrix.single a_1 a_2 1) x_1 x_1 * M2 (Matrix.single c_1 c_2 1) x_2 x_2
    ∑ a_1, ∑ a_2, ∑ c_1, ∑ c_2, (if a_1 = a_2 then 1 else 0) * (if c_1 = c_2 then 1 else 0)
  --Sort the sum into AACCBDD order
  simp only [@Finset.sum_comm A _ D, @Finset.sum_comm A _ B, @Finset.sum_comm C _ A]
  simp only [← Finset.mul_sum, ← Finset.sum_mul]
  congr! 8 with a_1 _ a_2 _ c_1 _ c_2 _
  · refine (h1 _).trans ?_
    split_ifs with h
    · subst h
      exact Matrix.trace_single_eq_same _ _
    · exact Matrix.trace_single_eq_of_ne _ _ _ h
  · refine (h2 _).trans ?_
    split_ifs with h
    · subst h
      exact Matrix.trace_single_eq_same _ _
    · exact Matrix.trace_single_eq_of_ne _ _ _ h
  simp [h_simp]

variable {S : Type*} [CommSemiring S] [Star S] [DecidableEq A] in
/-- The channel  $X \mapsto \sum k : \kappa, (M k) * X * (N k)^H$  formed by Kraus operators  $M$ 
is trace-preserving if  $\sum k : \kappa, (N k)^H * (M k) = 1$  -/
theorem of_kraus_isTracePreserving
  (M N :  $\kappa \rightarrow$  Matrix B A S)
  (hTP : ( $\sum k, (N k).conjTranspose * (M k) = 1$ ) :
  (MatrixMap.of_kraus M N).IsTracePreserving := by

```



```

intro x
simp only [of_kraus, LinearMap.coe_sum, LinearMap.coe_mk, AddHom.coe_mk, Finset.sum_mul,
  Matrix.trace_sum]
conv =>
  enter [1,2,i]
  rw [Matrix.trace_mul_cycle (M i) x (N i).conjTranspose]
  rw [← Matrix.trace_sum, ← Finset.sum_mul, hTP, one_mul]

/-- `MatrixMap.submatrix` is trace-preserving when the function is an equivalence. -
/
theorem submatrix (e : A ≈ B) : (MatrixMap.submatrix R e).IsTracePreserving := by
  intro; simp

end IsTracePreserving
end tp

section unital

variable [DecidableEq A] [DecidableEq B] [Semiring R]

/-- A linear matrix map is *unital* if it preserves the identity. -
/
def Unital (M : MatrixMap A B R) : Prop :=
  M 1 = 1

namespace Unital

variable {M : MatrixMap A B R}

omit [Fintype A] [Fintype B]

@[simp]
theorem map_1 (h : M.Unital) : M 1 = 1 :=
  h

/-- The identity `MatrixMap` is `Unital`. -/
@[simp]
theorem id : (id A R).Unital := by
  simp [Unital, MatrixMap.id]

--TODO: Closed under composition, kronecker products, it's iff M.choi_matrix.traceLe

end Unital
end unital

```

```

variable {A B C R : Type*}

open Kronecker
open TensorProduct

open ComplexOrder
variable [RCLike R]

/-- A linear matrix map is *Hermitian preserving* if it maps `IsHermitian`
/
def IsHermitianPreserving (M : MatrixMap A B R) : Prop :=
  ∀ x, x.IsHermitian → (M x).IsHermitian

/-- A linear matrix map is *positive* if it maps `PosSemidef` matrices to `
/
def IsPositive [Fintype A] [Fintype B] (M : MatrixMap A B R) : Prop :=
  ∀ x, x.PosSemidef → (M x).PosSemidef

/-- A linear matrix map is *completely positive* if, for any integer n, the
with `I(n)` is positive. -/
def IsCompletelyPositive [Fintype A] [Fintype B] [DecidableEq A] (M : MatrixMap A B R) : Prop :=
  ∀ (n : ℕ), (M ⊗k m (LinearMap.id : MatrixMap (Fin n) (Fin n) R)).IsPositive

namespace IsHermitianPreserving

variable {A : Type*} [Fintype A] in
/-- The identity MatrixMap IsHermitianPreserving. -/
theorem id : (id A R).IsPositive :=
  fun _ h ↦ h

/-- The composition of IsHermitianPreserving maps is also Hermitian preserving. -/
theorem comp {M1 : MatrixMap A B R} {M2 : MatrixMap B C R}
  (h1 : M1.IsHermitianPreserving) (h2 : M2.IsHermitianPreserving) : IsHermitianPreserving (M2 ∘ M1) :=
  fun _ h ↦ h2 (h1 h)

end IsHermitianPreserving

namespace IsPositive
variable [Fintype A] [Fintype B] [Fintype C]

/- Every `MatrixMap` that `IsPositive` is also `IsHermitianPreserving`. -/
theorem IsHermitianPreserving {M : MatrixMap A B R}
  (hM : IsPositive M) : IsHermitianPreserving M := by
  intro x hx

```

```

let xH : HermitianMat _ _ := ⟨x, hx⟩
classical --because PosPart requires DecidableEq
have hMPos := hM (HermitianMat.zero_le_iff.mp xH.posPart_nonneg)
have hMNeg := hM (HermitianMat.zero_le_iff.mp xH.negPart_nonneg)
have hSub := hMPos.isHermitian.sub hMNeg.isHermitian
rw [← map_sub] at hSub
convert ← hSub
exact HermitianMat.ext_iff.1 (HermitianMat.posPart_add_negPart xH)

/-- The composition of IsPositive maps is also positive. -
/
theorem comp {M₁ : MatrixMap A B R} {M₂ : MatrixMap B C R} (h₁ : M₁.IsPositive)
  (h₂ : M₂.IsPositive) : IsPositive (M₂ ∘₁ M₁) :=
  fun _ h ↦ h₂ (h₁ h)

variable {A : Type*} [Fintype A] in
/-- The identity MatrixMap IsPositive. -/
@[simp]
theorem id : (id A R).IsPositive :=
  fun _ h ↦ h

/-- Sums of IsPositive maps are IsPositive. -/
theorem add {M₁ M₂ : MatrixMap A B R} (h₁ : M₁.IsPositive) (h₂ : M₂.IsPositive) :
  (M₁ + M₂).IsPositive :=
  fun _ h ↦ Matrix.PosSemidef.add (h₁ h) (h₂ h)

/-- Nonnegative scalings of IsPositive maps are IsPositive. -
/
theorem smul {M : MatrixMap A B R} (hM : M.IsPositive) {x : R} (hx : 0 ≤ x) :
  (x • M).IsPositive :=
  fun _ h ↦ (hM h).smul hx

end IsPositive

namespace IsCompletelyPositive
variable [Fintype A] [Fintype B] [Fintype C] [DecidableEq A]

/-- Definition of a CP map, but with `Fintype T` in the definition instead of a `Fin`
/
theorem of_Fintype {M : MatrixMap A B R} (h : IsCompletelyPositive M)
  (T : Type*) [Fintype T] [DecidableEq T] :
  (M.kron (LinearMap.id : MatrixMap T T R)).IsPositive := by
  obtain ⟨n, ⟨e⟩⟩ : ∃ n : ℕ, Nonempty (T ≈ Fin n) :=
    Finite.exists_equiv_fin T
  convert h n using 1
  have h_submatrix : (M ⊗ₖₘ (LinearMap.id : MatrixMap T T R)) = (MatrixMap.submatrix

```

```

    ext
    simp only [submatrix, LinearMap.coe_comp, LinearMap.coe_mk, AddHom.coe_
      Matrix.submatrix_apply]
    rw [MatrixMap.kron_def, MatrixMap.kron_def]
    simp only [Matrix.single, LinearMap.id_coe, id_eq, Matrix.of_apply, mul
      ite_mul, zero_mul, Matrix.submatrix];
    congr! 4
    rw [← Equiv.sum_comp e]
    congr! 2
    rw [← Equiv.sum_comp e]
    simp only [EmbeddingLike.apply_eq_iff_eq, Equiv.symm_apply_apply]
  constructor
  · intro h₂
    simp [MatrixMap.IsPositive]
    exact h n
  · intro h x hx
    specialize h (hx.submatrix (fun p : A × Fin n => (p.1, e.symm p.2)))
    rw [h_submatrix]
    simp only [LinearMap.coe_comp, Function.comp_apply, submatrix_apply]
    exact h.submatrix _

/- Every `MatrixMap` that `IsCompletelyPositive` also `IsPositiveMap`. -
/
theorem IsPositive {M : MatrixMap A B R}
  (hM : IsCompletelyPositive M) : IsPositive M := by
  intro x hx
  let x' : Matrix (A × Fin 1) (A × Fin 1) R := x ⊗k 1
  let eqA : (A × Fin 1) ≈ A :=
    (Equiv.prodCongrRight (fun _ ↦ finOneEquiv)).trans (Equiv.prodPUnit A)
  let eqB : (B × Fin 1) ≈ B :=
    (Equiv.prodCongrRight (fun _ ↦ finOneEquiv)).trans (Equiv.prodPUnit B)
  specialize @hM 1 (x.submatrix eqA eqA) (Matrix.PosSemidef.submatrix hx _)
  convert Matrix.PosSemidef.submatrix hM eqB.symm; clear hM
  --TODO Cleanup
  ext i j
  simp only [Matrix.submatrix, Matrix.of_apply]
  rw [MatrixMap.kron_def]
  suffices h : M x = ∑ a₁, ∑ a₂, x a₁ a₂ • M (Matrix.single a₁ a₂ 1) by
    simp [h, Matrix.sum_apply, Matrix.single, eqA, eqB]
    ac rfl
  simp only [← M.map_smul, ← map_sum]
  congr
  ext k l
  simp [Matrix.sum_apply, Matrix.single]
  rw [Finset.sum_eq_single k]
  · simp

```

```

    · simp +contextual
    · simp +contextual

/-- The composition of IsCompletelyPositive maps is also completely positive. -
/
theorem comp [DecidableEq B] {M₁ : MatrixMap A B R} {M₂ : MatrixMap B C R} (h₁ : M₁.
  (h₂ : M₂.IsCompletelyPositive) : IsCompletelyPositive (M₂ ∘ₗ M₁) := by
  --sketch: (M₂ ∘ₗ M₁) ⊗ₖₘ id[n] = (M₂ ⊗ₖₘ id[n]) ∘ₗ (M₁ ⊗ₖₘ id[n]), which is a comp
  intro n x hx
  specialize h₁ n hx
  specialize h₂ n h₁
  conv in LinearMap.id =>
    change LinearMap.id ∘ₗ LinearMap.id
  rw [kron_comp_distrib]
  simpa using h₂

/-- The identity MatrixMap IsCompletelyPositive. -/
@[simp]
theorem id : (id A R).IsCompletelyPositive := by
  intro n ρ h
  rwa [show LinearMap.id = MatrixMap.id (Fin n) R from rfl, kron_id_id]

/-- Sums of IsCompletelyPositive maps are IsCompletelyPositive. -
/
theorem add {M₁ M₂ : MatrixMap A B R} (h₁ : M₁.IsCompletelyPositive) (h₂ : M₂.IsComp
  (M₁ + M₂).IsCompletelyPositive :=
  fun n _ h ↦ by
  simpa only [add_kron] using Matrix.PosSemidef.add (h₁ n h) (h₂ n h)

/-- Nonnegative scalings of `IsCompletelyPositive` maps are `IsCompletelyPositive`.
/
theorem smul {M : MatrixMap A B R} (hM : M.IsCompletelyPositive) {x : R} (hx : 0 ≤ x
  (x • M).IsCompletelyPositive :=
  fun n ρ h ↦ by
  rw [MatrixMap.smul_kron]
  exact (hM n h).smul hx

variable (A B) in
/-- The zero map `IsCompletelyPositive`. -/
theorem zero : (0 : MatrixMap A B R).IsCompletelyPositive :=
  fun _ _ _ ↦ by simpa using Matrix.PosSemidef.zero

/-- A finite sum of completely positive maps is completely positive. -
/
theorem finset_sum {ι : Type*} [Fintype ι] {m : ι → MatrixMap A B R} (hm : ∀ i, (m i
  (∑ i, m i).IsCompletelyPositive :=

```

```

    Finset.sum_induction m _ (fun _ _ => add) (.zero A B) (by simp)

end IsCompletelyPositive

variable [Fintype A] [Fintype B] [Fintype C] [DecidableEq A]
variable {d : Type*} [Fintype d]

/-- The map that takes M and returns  $M \otimes_k C$ , where C is positive semidefinite.
/
theorem kron_kronecker_const {C : Matrix d d R} (h : C.PosSemidef) {h1 h2 :
  (({fun M => M  $\otimes_k$  C, h1}, h2) : MatrixMap A (A  $\times$  d) R) := by
  intros n x hx
  have h_kronecker_pos : (x  $\otimes_k$  C).PosSemidef := by
    -- Since  $x$  and  $C$  are positive semidefinite, there exist matrices  $U$ 
    obtain (U, hU) :  $\exists U : \text{Matrix } (A \times \text{Fin } n) (A \times \text{Fin } n) R, x = \text{star } U * U$ 
    Matrix.posSemidef_iff_eq_conjTranspose_mul_self.mp hx
    obtain (V, hV) :  $\exists V : \text{Matrix } d d R, C = \text{star } V * V :=$ 
    Matrix.posSemidef_iff_eq_conjTranspose_mul_self.mp h
    --  $W = (U \otimes V)^* (U \otimes V)$  is positive semidefinite.
    have hW_pos : (U  $\otimes_k$  V).conjTranspose * (U  $\otimes_k$  V) = x  $\otimes_k$  C := by
      rw [Matrix.kroneckerMap_conjTranspose, ← Matrix.mul_kronecker_mul]
      rw [hU, hV, Matrix.star_eq_conjTranspose, Matrix.star_eq_conjTranspose]
      rw [← hW_pos]
    exact Matrix.posSemidef_conjTranspose_mul_self (U  $\otimes_k$  V)
  --TODO clean up this mess (but, thanks Aristotle)
  convert h_kronecker_pos.submatrix (fun (⟨a, d'⟩, n') : (A  $\times$  d)  $\times$  Fin
  ext (⟨a, d⟩, n) (⟨a', d'⟩, n')
  simp [Matrix.kroneckerMap_apply, Matrix.submatrix_apply]
  erw [MatrixMap.kron_def]
  simp [Matrix.single, Matrix.kroneckerMap_apply]
  simp [Finset.sum_ite, Finset.filter_eq', Finset.filter_and]
  rw [Finset.sum_eq_single a]
  · simp_all only [RingHom.id_apply, ↓reduceIte, Finset.mem_univ, Finset.in
    simp_all only
    rw [Finset.sum_eq_single n]
    · simp_all only [↓reduceIte, Finset.mem_univ, Finset.inter_singleton_of
      ring
    · intro b a_1 a_2
      simp_all only [Finset.mem_univ, ne_eq, ↓reduceIte, Finset.notMem_empty
        Finset.inter_singleton_of_notMem, Finset.sum_empty]
    · intro a_1
      simp_all only [Finset.mem_univ, not_true_eq_false]
  · intro b a_1 a_2
    simp_all only [RingHom.id_apply, Finset.mem_univ, ne_eq, ↓reduceIte, Fi
      Finset.inter_singleton_of_notMem, Finset.sum_empty]
  · intro a_1

```

```

simp_all only [RingHom.id_apply, Finset.mem_univ, not_true_eq_false]

omit [Fintype B] in
theorem choi_of_kraus (K :  $\kappa \rightarrow$  Matrix B A  $\square$ ) :
  (MatrixMap.of_kraus K K).choi_matrix =  $\sum$  k, Matrix.vecMulVec (fun (x : B  $\times$  A) =>
    -- By definition of Choi matrix, we can expand the left-
    hand side using the linearity of the map and the properties of the Choi matrix.
    ext (b1, a1) (b2, a2)
    simp [MatrixMap.choi_matrix, MatrixMap.of_kraus];
    -- By definition of the sum, the entry (b1, b2) of the sum of the Choi matrices of
    simp [Matrix.sum_apply, Matrix.mul_apply, Matrix.single];
    -- Since the inner sum over `x2` will only contribute when `x2 = a1` and `x1 =
    have h_inner :  $\forall$  x :  $\kappa$ ,  $\sum$  x1 : A, ( $\sum$  x2 : A, if a1 = x2  $\wedge$  a2 = x1 then K x b1)
    simp [Finset.sum_ite, Finset.filter_eq, Finset.filter_and];
    intro x; rw [Finset.sum_eq_single a2] <|> aesop;
    exact Finset.sum_congr rfl fun _ => h_inner _

/-- The linear map of conjugating a matrix by another, `x  $\rightarrow$  y * x * yH`. -
/
@[simps]
def _root_.MatrixMap.conj (y : Matrix B A R) : MatrixMap A B R where
  toFun x := y * x * y.conjTranspose
  map_add' x y := by rw [Matrix.mul_add, Matrix.add_mul]
  map_smul' r x := by rw [RingHom.id_apply, Matrix.mul_smul, Matrix.smul_mul]

omit [DecidableEq A] in
theorem conj_isPositive (M : Matrix B A  $\square$ ) : (conj M).IsPositive := by
  exact fun X hX => hX.mul_mul_conjTranspose_same M

omit [DecidableEq A] in
theorem IsPositive_sum { $\iota$  : Type*} [Fintype  $\iota$ ] (f :  $\iota \rightarrow$  MatrixMap A B  $\mathbb{C}$ ) (h :  $\forall$  i, (
  ( $\sum$  i, f i).IsPositive := by
  intro X hX;
  replace hX :  $\forall$  i, ((f i) X).PosSemidef := fun i => h i hX;
  simp [Matrix.posSemidef_iff_dotProduct_mulVec, Matrix.IsHermitian] at hX  $\vdash$ 
  simp [Matrix.conjTranspose_sum, Matrix.mulVec, dotProduct] at hX  $\vdash$ 
  simp [Matrix.sum_apply, Finset.mul_sum, Finset.sum_mul] at hX  $\vdash$ 
  constructor
  · simp [hX]
  field_simp
  simp_rw [mul_assoc, mul_comm ((f _) X _),  $\leftarrow$  mul_assoc]
  intro x
  rw [Finset.sum_comm_cycle]
  exact Finset.sum_nonneg fun i _ => by simpa [mul_assoc, mul_comm, mul_left_comm,
omit [DecidableEq A] in

```

```

theorem of_kraus_isPositive (K :  $\kappa \rightarrow$  Matrix B A  $\mathbb{C}$ ) :
  (of_kraus K K).IsPositive := by
  rw [of_kraus]
  apply IsPositive_sum
  intro k
  exact conj_isPositive (K k)

theorem conj_kron (M : Matrix B A  $\square$ ) (N : Matrix D C  $\square$ ) [DecidableEq C] :
  conj M  $\otimes_{k,m}$  conj N = conj (M  $\otimes_k$  N) := by
  apply LinearMap.ext
  intro x
  have h_eq :  $\forall$  (X : Matrix A A  $\square$ ) (Y : Matrix C C  $\square$ ), (conj M  $\otimes_{k,m}$  conj N)
    intro X Y;
    convert MatrixMap.kron_map_of_kron_state _ _ X Y using 1;
    ext { b1, d1 } { b2, d2 }
    simp only [Matrix.kroneckerMap]
    ring_nf
    simp only [conj, LinearMap.coe_mk, AddHom.coe_mk, Matrix.mul_apply, Mat
      Matrix.conjTranspose_apply, star_mul', RCLike.star_def]
    simp only [mul_left_comm, mul_comm, Finset.mul_sum, mul_assoc]
    simp only [ $\leftarrow$  Finset.univ_product_univ,  $\leftarrow$  Finset.sum_product'];
    apply Finset.sum_bij (fun x _  $\mapsto$  (x.1.2, x.2.2, x.1.1, x.2.1)) <=> simp
  -- By linearity, it suffices to show that the maps agree on a basis.
  have h_basis :  $\forall$  (x : Matrix (A  $\times$  C) (A  $\times$  C)  $\square$ ), x  $\in$  Submodule.span  $\square$  (Se
    intro x hx;
    induction hx using Submodule.span_induction;
    · rename_i h
      simp only [Set.mem_range, Prod.exists] at h
      obtain (w, (w', rfl)) := h
      apply h_eq
    · simp [conj];
    · simp_all only [map_add]
    · simp_all only [map_smul]
  convert h_basis x _;
  -- By definition of matrix multiplication and the properties of the Krone
  have h_decomp :  $\forall$  (x : Matrix (A  $\times$  C) (A  $\times$  C)  $\square$ ),  $\exists$  (coeffs : A  $\rightarrow$  C  $\rightarrow$  A  $\rightarrow$ 
    intro x
    use fun a1 c1 a2 c2 => x (a1, c1) (a2, c2);
    ext { a1, c1 } { a2, c2 }
    simp only [Matrix.single]
    simp only [Matrix.sum_apply, Matrix.kroneckerMap]
    rw [Finset.sum_eq_single a1] <=> simp [Finset.sum_ite]
    · rw [Finset.sum_eq_single c1] <=> simp +contextual
      rw [Finset.sum_eq_single c2] <=> simp +contextual;
    · simp +contextual [Finset.filter_eq', Finset.filter_and ];
  obtain (coeffs, rfl) := h_decomp x;

```



```

exact Submodule.sum_mem _ fun a₁ _ => Submodule.sum_mem _ fun c₁ _ => Submodule.sum_mem _

theorem congruence_one_eq_id : conj (1 : Matrix A A ℂ) = MatrixMap.id A ℂ := by
  ext x
  simp [conj]

theorem congruence_CP {A B : Type*} [Fintype A] [Fintype B] [DecidableEq A] [DecidableEq B]
  intro n;
  -- The tensor product of congruence maps is a congruence map.
  have h_tensor_congruence : conj M ⊗k LinearMap.id = conj (M ⊗k (1 : Matrix (Fin n) (Fin n) ℂ))
    convert conj_kron M (1 : Matrix (Fin n) (Fin n) ℂ);
  ext M
  simp
  convert conj_isPositive (M ⊗k (1 : Matrix (Fin n) (Fin n) ℂ)) using 1

theorem IsCompletelyPositive_sum {ι : Type*} [Fintype ι] (f : ι → MatrixMap A B ℂ) (h :
  (∑ i, f i).IsCompletelyPositive := by
    convert IsCompletelyPositive.finset_sum h using 1

omit [Fintype B] [DecidableEq A] in
theorem of_kraus_eq_sum_conj (K : κ → Matrix B A ℂ) :
  of_kraus K K = ∑ k, conj (K k) := by
  ext
  simp [MatrixMap.of_kraus, conj]

theorem of_kraus_CP (K : κ → Matrix B A ℂ) : (of_kraus K K).IsCompletelyPositive :=
  -- By definition of `MatrixMap.of_kraus`, we know that it is a sum of congruence maps
  have h_sum_congruence : MatrixMap.of_kraus K K = ∑ k, conj (K k) := by
    -- By definition of `MatrixMap.of_kraus`, we know that it is equal to the sum of congruence maps
    apply of_kraus_eq_sum_conj
  have h_congruence_CP : ∀ k, (conj (K k)).IsCompletelyPositive := by
    intro k; exact (by
      convert congruence_CP (K k) using 1;
      -- Since B is a finite type, we can use the fact that finite types have decidable equality
      apply Classical.decEq);
  exact h_sum_congruence.symm ▸ IsCompletelyPositive.finset_sum h_congruence_CP

theorem exists_kraus_of_choi_PSD
  (C : Matrix (B × A) (B × A) ℂ) (hC : C.PosSemidef) :
  ∃ (K : (B × A) → Matrix B A ℂ), C = (MatrixMap.of_kraus K K).choi_matrix := by
  classical
  use fun k i j => (hC.1.eigenvectorUnitary.val : Matrix _ _ ℂ) (i, j) k * (hC.1.eigenvalue k)
  convert Matrix.IsHermitian.spectral_theorem hC.1 using 1;
  ext i j
  simp [choi_of_kraus, Matrix.mul_apply, Matrix.vecMulVec]
  ring_nf

```

```

simp [ Matrix.sum_apply, Matrix.diagonal ];
refine Finset.sum_congr rfl fun _ => ?_
rw [ ← RCLike.ofReal_pow, Real.sq_sqrt ( hC.eigenvalues_nonneg _ ) ]

/-
The Choi matrix of M is the image of the unnormalized maximally entangled s
-/
theorem choi_matrix_eq_map_proj (M : MatrixMap A B R) :
  M.choi_matrix = (M  $\otimes_{k\ m}$  MatrixMap.id A R) (Matrix.vecMulVec (fun (x : A
  have h_choi :  $\forall$  (M : MatrixMap A B R), MatrixMap.kron M (MatrixMap.id A R
  intro M
  ext {b1, d1} {b2, d2}
  simp [MatrixMap.kron_def, MatrixMap.choi_matrix];
  simp +decide [ Matrix.single, Matrix.vecMulVec ];
  rw [ Finset.sum_eq_single d1 ] <=> aesop;
  convert h_choi M |> Eq.symm

/-- Choi's theorem on completely positive maps: A map `IsCompletelyPositive
/
theorem choi_PSD_iff_CP_map (M : MatrixMap A B R) :
  M.IsCompletelyPositive  $\leftrightarrow$  M.choi_matrix.PosSemidef := by
  constructor
  · intro hcp
    have := MatrixMap.IsCompletelyPositive.of_Fintype hcp A
    rw [ MatrixMap.choi_matrix_eq_map_proj ] at *;
    exact this ( Matrix.PosSemidef.outer_self_conj _ )
  · intro h_psd
    obtain {K, hK} := exists_kraus_of_choi_PSD M.choi_matrix h_psd
    rw [choi_matrix_inj hK]
    exact of_kraus_CP K

omit [Fintype B] [DecidableEq A] in
theorem conj_eq_mulRightLinearMap_comp_mulRightLinearMap (y : Matrix B A R)
  conj y = mulRightLinearMap B R y.conjTranspose  $\circ$  mulLeftLinearMap A R
  ext1; simp

/-- The act of conjugating (not necessarily by a unitary, just by any matrix
/
theorem conj_isCompletelyPositive (M : Matrix B A R) : (conj M).IsCompletely
--TODO: This is identical to congruence_CP
intro n m h
classical
open ComplexOrder in
open Kronecker in
suffices ((M  $\otimes_k$  1 : Matrix (B  $\times$  Fin n) (A  $\times$  Fin n) R) * m * (M.conjTransp
  convert this

```

```

--TODO cleanup. Thanks Aristotle
ext { b₁, c₁ } { b₂, c₂ }
rw [ MatrixMap.kron_def ];
simp [Matrix.mul_apply, Matrix.single];
have h_split :  $\sum x, \sum x_1, \sum x_2, \sum x_3, (\text{if } x_2 = c_1 \wedge x_3 = c_2 \text{ then } (\sum x_4, (\sum$ 
  refine Finset.sum_congr rfl fun _ => Finset.sum_congr rfl fun _ => ?_
  rw [ Finset.sum_eq_single c₁ ]
  · simp_all only [Finset.mem_univ, true_and, Finset.sum_ite_eq', ↓reduceIte]
  · intro b a a₁
    simp_all only [Finset.mem_univ, ne_eq, false_and, ↓reduceIte, Finset.sum_congr]
  · intro a
    simp_all only [Finset.mem_univ, not_true_eq_false]
convert h_split using 1;
rw [ Matrix.mul_assoc ];
simp only [Matrix.mul_apply, Matrix.kroneckerMap_apply, Matrix.conjTranspose_apply,
  RCLike.star_def, Finset.mul_sum _ _ _ , Finset.sum_mul, ite_mul, zero_mul];
simp [ Matrix.one_apply, Finset.sum_ite, Finset.filter_eq, Finset.filter_and ];
have h_reindex :  $\sum x \in \{x \mid c_1 = x.2\}, \sum x_1 \in \{x \mid x.2 = c_2\}, M b_1 x.1 * (m \times x$ 
  rw [ show ( Finset.univ.filter fun x : A × Fin n => c₁ = x.2 ) = Finset.image
    · simp [Finset.sum_image, Set.InjOn]
    · ext { x, y }
      simp only [Finset.mem_filter, Finset.mem_univ, true_and, Finset.mem_image, P
        exists_eq_left]
      exact eq_comm;
    · ext { x, y }
      simp [ eq_comm ];
  have h_inner :  $\forall x x_1, \sum x_2, \sum x_3 \in \{x\} \text{ n if } x_1 = x_2 \text{ then Finset.univ else}$ 
    intro x x₁
    rw [ Finset.sum_eq_single x₁ ] <=> simp +contextual;
    simp +contextual [ eq_comm ];
  simp only [ h_inner ];
  simp only [ mul_assoc, mul_comm, mul_left_comm ] using h_reindex
obtain ⟨m', rfl⟩ := Matrix.posSemidef_iff_eq_conjTranspose_mul_self.mp h
convert Matrix.posSemidef_conjTranspose_mul_self (m' * (M ⊗ₖ 1 : Matrix (B × Fin n
simp only [Matrix.conjTranspose_mul, Matrix.conjTranspose_conjTranspose, Matrix.mu
rw [Matrix.mul_assoc, Matrix.mul_assoc]
congr
ext
simp +contextual only [Matrix.kroneckerMap_apply, Matrix.conjTranspose_apply, RCLi
  Matrix.one_apply, apply_ite, mul_one, mul_zero, star_zero, ↓reduceIte, ite_eq_r
  map_eq_zero, if_true_left]
tauto

/-- `MatrixMap.submatrix` is completely positive -/
theorem IsCompletelyPositive.submatrix (f : B → A) : (MatrixMap.submatrix R f).IsCom
  convert conj_isCompletelyPositive (Matrix.submatrix (α := R) 1 f _root_.id : Matri

```

```

ext1 m
simp [m.submatrix_eq_mul_mul]

/-- The channel  $X \mapsto \sum k : \kappa, (M\ k) * X * (M\ k)^H$  formed by Kraus operators  $M$ 
is completely positive -/
theorem of_kraus_isCompletelyPositive (M :  $\kappa \rightarrow \text{Matrix } B\ A\ R$ ) :
  (of_kraus M M).IsCompletelyPositive := by
  rw [of_kraus]
  exact IsCompletelyPositive.finset_sum (fun i  $\mapsto$  conj_isCompletelyPositive

omit [Fintype B] [DecidableEq A] in
/-
The Choi matrix of a map in symmetric Kraus form is a sum of rank-
1 projectors.
-/
theorem choi_of_kraus_R [DecidableEq A] (K :  $\kappa \rightarrow \text{Matrix } B\ A\ \square$ ) :
  (of_kraus K K).choi_matrix =  $\sum k, \text{Matrix.vecMulVec } (\text{fun } (x : B \times A) \Rightarrow$ 
  unfold of_kraus choi_matrix
  ext i j : 2
  simp only [LinearMap.coe_sum, LinearMap.coe_mk, AddHom.coe_mk, Finset.sum
    Matrix.sum_apply, Matrix.mul_apply, Matrix.conjTranspose_apply, RCLike.
    Matrix.vecMulVec, Matrix.of_apply]
  simp only [Matrix.single, Matrix.of_apply, mul_ite, mul_one, mul_zero]
  simp +contextual only [Finset.sum_ite, Finset.filter_and, Finset.filter_c
    Finset.sum_const_zero, add_zero];
  congr! with k
  simp_all only [Finset.mem_univ, Finset.filter_eq,  $\downarrow$ reduceIte]
  rw [Finset.sum_eq_single j.2]
  · simp
  · aesop
  · simp

/-
The Choi matrix of  $M$  is the result of applying  $M \otimes I$  to the unnormali
-/
variable {A B R : Type*} [Fintype A] [Fintype B] [DecidableEq A] [RCLike R]

theorem choi_eq_kron_id_apply_choi_id (M :  $\text{MatrixMap } A\ B\ R$ ) :
  M.choi_matrix = (M  $\otimes_{\kappa m}$   $\text{MatrixMap.id } A\ R$ ) (( $\text{MatrixMap.id } A\ R$ ).choi_matri
  ext (j1, a1) (j2, a2) : 2
  rw [MatrixMap.kron_def]
  simp only [choi_matrix, Matrix.single, LinearMap.id_coe, id_eq, Matrix.of
    mul_zero]
  simp only [ $\leftarrow$  Finset.sum_filter, Finset.filter_and, Finset.filter_const, F
    Finset.mem_univ,  $\downarrow$ reduceIte, Finset.mem_inter, Finset.mem_singleton, Fi
    nsmul_eq_mul]

```

```

rw [ Finset.sum_eq_single a₁ ]
· simp +contextual only [↓reduceIte, Finset.inter_singleton_of_mem]
  rw [ Finset.sum_eq_single a₂, Finset.sum_eq_single a₁ ]
  · simp
  · simp +contextual
  · intro h
    simp at h
  · intro b _ hb
    rw [Finset.sum_eq_zero]
    aesop
  · simp
· simp +contextual only
  intro b a
  split_ifs with h <=> simp [h]
· simp +contextual

```

/-

The Choi matrix of the identity map is positive semidefinite.

-/

```

theorem choi_id_is_PSD {A R : Type*} [Fintype A] [DecidableEq A] [RCLike R] :
  (MatrixMap.id A R).choi_matrix.PosSemidef := by
  -- Let $v$ be the vector with $v_{(a,b)} = \delta_{ab}$.
  set v : A × A → R := fun p => if p.1 = p.2 then 1 else 0;
  -- By definition of $C$, we know that $C = v v^*$.
  have hC : (MatrixMap.id A R).choi_matrix = Matrix.of (fun (i j : A × A) => v i * v j)
    ext (i, j) (k, l) ; simp [ MatrixMap.choi_matrix, Matrix.single ] ; aesop;
  rw [Matrix.posSemidef_iff_dotProduct_mulVec]
  refine' (λ x, fun x => _);
  · ext i j; aesop;
  · -- By definition of $v$, we know that $star x \sqcup_v v v^* x = |star x \sqcup_v v|^2$.
    have h_inner : star x \sqcup_v (MatrixMap.id A R).choi_matrix.mulVec x = star (star x
      simp [ hC, Matrix.mulVec, dotProduct ];
      simp [ Finset.mul_sum, mul_comm, v]
      exact Finset.sum_congr rfl fun i hi => by split_ifs <=> simp [ * ] ;
    rw [ h_inner, mul_comm ];
    exact mul_star_self_nonneg (star x \sqcup_v v)

```

/-

If a map is completely positive, its Choi matrix is positive semidefinite.

-/

```

theorem is_CP_implies_choi_PSD {A B R : Type*} [Fintype A] [Fintype B] [DecidableEq A]
  M.choi_matrix.PosSemidef := by
  rw [choi_eq_kron_id_apply_choi_id]
  exact MatrixMap.IsCompletelyPositive.of_Fintype hCP A choi_id_is_PSD

```

```

theorem IsCompletelyPositive.exists_kraus (Φ : MatrixMap A B R) (hCP : Φ.IsCompletelyPositive) :
  ∃ (K : List (MatrixMap A B R)),
    (K.map (λ k, k.mulVec)).mulVec Φ.mulVec = Φ.mulVec

```

```

    ∃ (M : (B × A) → Matrix B A R), Φ = of_kraus M M := by
  rw [choi_PSD_iff_CP_map] at hCP
  convert exists_kraus_of_choi_PSD Φ.choi_matrix hCP using 1;
  funext
  rw [eq_iff_iff, iff_comm]
  exact MatrixMap.choi_matrix_inj.eq_iff

/--
The Kronecker product of two Kraus maps is the Kraus map of the Kronecker p
-/
theorem kron_of_kraus {A B C D R : Type*} [Fintype A] [Fintype B] [Fintype
  [DecidableEq A] [DecidableEq C] [CommSemiring R] [StarRing R] [SMulComm
  {κ ι : Type*} [Fintype κ] [Fintype ι]
  (M : κ → Matrix B A R) (N : ι → Matrix D C R) :
  of_kraus M M ⊗κ m of_kraus N N =
  of_kraus (fun (k : κ × ι) => M k.1 ⊗κ N k.2) (fun k => M k.1 ⊗κ N k.2)
open Classical
in convert MatrixMap.choi_map_inv _ using 1;
rotate_left
· infer_instance
· infer_instance
rw [choi_map_inv]
apply MatrixMap.choi_matrix_inj
ext { b, a } { d, c }
simp only [of_kraus] ;
simp only [choi_matrix, LinearMap.coe_sum, LinearMap.coe_mk, AddHom.coe_m
rw [ MatrixMap.kron_def ]
simp only [LinearMap.coe_sum, LinearMap.coe_mk, AddHom.coe_mk, Finset.sum
  Matrix.sum_apply, Matrix.mul_apply, Matrix.conjTranspose_apply, Matrix.
  star_mul'] ;
simp only [Matrix.single, Matrix.of_apply, mul_ite, mul_one, mul_zero, Fi
  Finset.sum_const_zero, add_zero, ne_eq];
simp only [Finset.sum_sigma', Finset.univ_sigma_univ];
simp only [mul_comm, Finset.mul_sum, Finset.sum_sigma', mul_left_comm, mu
refine Finset.sum_bij ( fun x _ => { { { x.snd.snd.fst.fst, x.snd.fst.fst
· simp only [Finset.mem_sigma, Finset.mem_univ, Finset.mem_filter, true_a
  grind
· simp only [Finset.mem_sigma, Finset.mem_univ, Finset.mem_filter, true_a
  Prod.mk.injEq, heq_eq_eq, and_imp]
  grind
· simp only [Finset.mem_sigma, Finset.mem_univ, Finset.mem_filter, true_a
  Sigma.exists, existsAndEq, and_true, exists_and_left, and_imp]
  grind
· simp

namespace IsCompletelyPositive

```

```

/-- The Kronecker product of IsCompletelyPositive maps is also completely positive.
/
theorem kron {D : Type*} [DecidableEq C] [Fintype D] {M1 : MatrixMap A B R} {M2 : MatrixMap A B R}
  (h1 : M1.IsCompletelyPositive) (h2 : M2.IsCompletelyPositive) : IsCompletelyPositive (M1.kron M2) :=
  obtain ⟨K1, rfl⟩ := exists_kraus M1 h1
  obtain ⟨K2, rfl⟩ := exists_kraus M2 h2
  rw [kron_of_kraus K1 K2]
  apply of_kraus_isCompletelyPositive

section piProd

variable {ι : Type u} [DecidableEq ι] [fι : Fintype ι]
variable {dI : ι → Type v} [∀i, Fintype (dI i)] [∀i, DecidableEq (dI i)]
variable {dO : ι → Type w} [∀i, Fintype (dO i)] [∀i, DecidableEq (dO i)]

/-- The `MatrixMap.piProd` product of IsCompletelyPositive maps is also completely positive.
/
theorem piProd {Λi : ∀ i, MatrixMap (dI i) (dO i) R} (h1 : ∀ i, (Λi i).IsCompletelyPositive) :
  IsCompletelyPositive (MatrixMap.piProd Λi) := by
  sorry

end piProd

end IsCompletelyPositive

end MatrixMap

```

1.429 QuantumInfo/Finite/Capacity.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.SpecialFunctions.Log.Base

import QuantumInfo.Finite.Entropy
import QuantumInfo.Finite.CPTPMap
import QuantumInfo.Finite.Distance

/--! # Quantum Capacity

```

This focuses on defining and proving theorems about the quantum capacity, t

1. A channel A ``Emulates`` another channel B if there are D and E such that
2. A channel A ``εApproximates`` channel B (of the same dimensions) if the f
- ε.
3. A channel A ``AchievesRate`` $R:\mathbb{R}$ if for every $\varepsilon>0$, n copies of A emulates
4. The ``quantumCapacity`` of the channel A is the supremum of the achievabl

The most basic facts:

- * ``emulates_self``: Every channel emulates itself.
- * ``emulates_trans``: If A emulates B and B emulates C, then A emulates C. (
- * ``εApproximates A B ε`` is equivalent to the existence of some δ (dependin
- B| has diamond norm at most δ , and $\delta\rightarrow 0$ as $\varepsilon\rightarrow 0$.
- * ``achievesRate_0``: Every channel achievesRate 0. So, the set of achievabl
- * If a channel achievesRate R_1 , it also every achievesRate R_2 every $R_2 \leq R_1$
- ∞. Achievable rates are ``-BddBelow``.
- * ``bddAbove_achievesRate``: A channel $C : \dim X \rightarrow \dim Y$ cannot achievesRate R

The nice lemmas we would want:

- * Capacity of a replacement channel is zero.
- * Capacity of an identity channel is ``log2(D)``.
- * Capacity is superadditive under tensor products. (That is, at least addi
- * Capacity of a kth tensor power is exactly k times the capacity of the or
- * Capacity does not decrease under tensor sums.
- * Capacity does not increase under composition.

Then, we should show that our definition is equivalent to some above. Most,

Then the LSD theorem establishes that the single-copy coherent information
copy coherent information converges to the capacity, is ``quantumCapacity_eq``

TODO

The only notion of "capacity" here currently is "quantum capacity" in the u
equal capacities relevant to quantum channels, see e.g. [Watrous's notes](h

- * Quantum capacity (``quantumCapacity``)
- * Quantum 1-shot capacity
- * Entanglement-assisted classical capacity
- * Qss, the quantum side-channel capacity
- * Holevo capacity, aka Holevo χ . The Holevo–Schumacher–Westmoreland theore
- * Entanglement-assisted Holevo capacity
- * Entanglement-assisted quantum capacity
- * One- and two-way distillable entanglement

And other important theorems like superdense coding, nonadditivity, superac
-/


```
namespace CTPMap
```

```
variable {d1 d2 d3 d4 d5 d6 : Type*}
variable [Fintype d1] [Fintype d2] [Fintype d3] [Fintype d4] [Fintype d5] [Fintype d6]
```

```
variable [DecidableEq d3] [DecidableEq d4] in
/--
A channel  $\Lambda_1$  `Emulates` another channel  $\Lambda_2$  if there are D and E such that  $D \circ \Lambda_1 \circ E = \Lambda_2$ 
-/  
def Emulates ( $\Lambda_1$  : CTPMap d1 d2) ( $\Lambda_2$  : CTPMap d3 d4) : Prop :=
   $\exists$  (E : CTPMap d3 d1) (D : CTPMap d2 d4), D.compose ( $\Lambda_1$ .compose E) =  $\Lambda_2$ 
```

```
/--
A channel A `εApproximates` channel B of the same dimensions if the for every state
ε.
-/  
def εApproximates (A B : CTPMap d1 d2) (ε : ℝ) : Prop :=
   $\forall$  (ρ : MState d1), (A ρ).fidelity (B ρ) ≥ 1-ε
```

```
/--
A channel A `AchievesRate` R:ℝ if for every ε>0, some n copies of A emulates a channel B
-/  
def AchievesRate (A : CTPMap d1 d2) (R : ℝ) : Prop :=
   $\forall$  ε : ℝ, ε > 0 →
     $\exists$  n > 0,  $\exists$  (dimB : ℕ) (B : CTPMap (Fin dimB) (Fin dimB)),
      (CTPMap.piProd (fun _ : Fin n => A)).Emulates B ∧
      Real.logb 2 dimB ≥ R*n ∧
      B.εApproximates CTPMap.id ε
```

```
noncomputable def quantumCapacity (A : CTPMap d1 d2) : ℝ :=
  sSup { R : ℝ | AchievesRate A R }
```

```
section emulates
```

```
variable [DecidableEq d3] [DecidableEq d4] [DecidableEq d5]
```

```
/-- Every quantum channel emulates itself. -/  
@[refl]
```

```
theorem emulates_self ( $\Lambda$  : CTPMap d1 d2) :  $\Lambda$ .Emulates  $\Lambda$  :=
  (CTPMap.id, CTPMap.id, by simp)
```

```
/-- If a quantum channel A emulates B, and B emulates C, then A emulates C. -  
/
```

```
@[trans]
```

```
theorem emulates_trans ( $\Lambda_1$  : CTPMap d1 d2) ( $\Lambda_2$  : CTPMap d3 d4) ( $\Lambda_3$  : CTPMap d5 d6)
  (h12 :  $\Lambda_1$ .Emulates  $\Lambda_2$ ) (h23 :  $\Lambda_2$ .Emulates  $\Lambda_3$ ) :  $\Lambda_1$ .Emulates  $\Lambda_3$  := by
```

```

    obtain ⟨E1, D1, hED1⟩ := h12
    obtain ⟨E2, D2, hED2⟩ := h23
    exact ⟨E1.compose E2, D2.compose D1, by classical simp [← hED1, ← hED2, c
end emulates

section εApproximates

/-- Every quantum channel perfectly approximates itself, that is, `εApproximates self`
/
theorem εApproximates_self (Λ : CPTPMap d1 d2) : Λ.εApproximates Λ 0 :=
  fun ρ ↦ ((Λ ρ).fidelity_self_eq_one.trans (sub_zero 1).symm).ge

/-- If a quantum channel A approximates B with ε0, it also approximates B with ε1
/
theorem εApproximates_monotone {A B : CPTPMap d1 d2} {ε0 : ℝ} (h : A.εApproximates B ε0)
  {ε1 : ℝ} (h2 : ε0 ≤ ε1) : A.εApproximates B ε1 :=
  fun ρ ↦ (tsub_le_tsub_left h2 1).trans (h ρ)

end εApproximates

section AchievesRate

/-- Every quantum channel achieves a rate of zero. -/
theorem achievesRate_0 (Λ : CPTPMap d1 d2) : Λ.AchievesRate 0 := by
  intro ε hε
  use 1, zero_lt_one, 1, default
  constructor
  · have : Nonempty d1 := by sorry--having a CPTPMap should be enough to construct
spaces are nonempty
  have : Nonempty d2 := by sorry
  use Classical.ofNonempty, Classical.ofNonempty
  sorry--exact Unique.eq_default _
  constructor
  · norm_num
  · rw [Unique.eq_default id]
  sorry--apply εApproximates_monotone (εApproximates_self default) hε.le

/-- The identity channel on D dimensional space achieves a rate of log2(D).
/
theorem id_achievesRate_log_dim : (id (dIn := d1)).AchievesRate (Real.logb 2 d1)
  intro ε hε
  use 1, zero_lt_one, Fintype.card d1, id
  constructor
  · --they are equivalent up to permutation
  · -- TODO: Instead this proof should be `@[simp] piProd (fun x => id) = id`

```

```

    refine' { _, _, _ };
    exact CTPMap.ofEquiv ( Fintype.equivFinOfCardEq ( by simp +decide ) ).symm;
    exact CTPMap.ofEquiv ( Fintype.equivFinOfCardEq ( by simp +decide ) );
    apply CTPMap.ext;
    ext; simp +decide [ CTPMap.piProd ];
    unfold MatrixMap.piProd
    simp_all only [gt_iff_lt, PiTensorProduct.map_id, LinearMap.toMatrix_id_eq_basis
      Module.Basis.toMatrix_self, Matrix.reindex_apply, Matrix.submatrix_one_equiv,
      erw [ MatrixMap.submatrix_apply ]
      simp_all only [Equiv.symm_symm, Equiv.apply_symm_apply, Matrix.submatrix_apply]
    constructor
    · norm_num
    · exact εApproximates_monotone (εApproximates_self id) hε.le

/-- A channel cannot achieve a rate greater than log2(D), where D is the input dimension
/
theorem not_achievesRate_gt_log_dim_in (Λ : CTPMap d₁ d₂) {R : ℝ} (hR : Real.logb 2
  ¬Λ.AchievesRate R := by
  sorry

noncomputable section AristotleLemmas

end AristotleLemmas

/-- A channel cannot achieve a rate greater than log2(D), where D is the output dimension
/
theorem not_achievesRate_gt_log_dim_out (Λ : CTPMap d₁ d₂) {R : ℝ} (hR : Real.logb
  intro h;
  -- We show that the identity channel on the output space `d₂` emulates `Λ`. Since
  have h_emulate : (CTPMap.id (dIn := d₂)).Emulates Λ := by
    exact ⟨Λ, CTPMap.id, by simp⟩
  -- If `Λ` achieves rate `R`, then `id_{d₂}` achieves rate `R`. This follows because
  have h_id_achieves : (CTPMap.id (dIn := d₂)).AchievesRate R := by
    intro ε hε_pos
    obtain ⟨n, hn, dimB, B, hB_emulate, hB_rate, hB_approx⟩ := h ε hε_pos
    have h_id_emulate : (CTPMap.piProd (fun (_ : Fin n) => CTPMap.id (dIn := d₂)))
      -- Since `id_{d₂}` emulates `Λ`, we can use the fact that the tensor product of
      have h_tensor_emulate : ∀ (n : ℕ), (CTPMap.piProd (fun (_ : Fin n) => CTPMap
        intro n
        obtain ⟨E, D, hD⟩ := h_emulate
        use CTPMap.piProd (fun (_ : Fin n) => E), CTPMap.piProd (fun (_ : Fin n) =>
          simp [ ← hD, ← CTPMap.piProd_comp];
        exact emulates_trans _ _ _ ( h_tensor_emulate n ) hB_emulate;
        exact ⟨ n, hn, dimB, B, h_id_emulate, hB_rate, hB_approx ⟩;
    refine not_le_of_gt hR <| not_lt.mp fun h => ?_
    exact not_lt_of_ge ( le_of_not_gt fun h' => not_achievesRate_gt_log_dim_in _ h' h_

```

```

/-- The achievable rates are a bounded set. -/
theorem bddAbove_achievesRate ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) : BddAbove {R |  $\Lambda$ .AchievesRate R}
  use Real.logb 2 (Fintype.card  $d_1$ )
  intro R h
  contrapose h
  exact not_achievesRate_gt_log_dim_in  $\Lambda$  (lt_of_not_ge h)

end AchievesRate

section capacity

/-- Quantum channel capacity is nonnegative. -/
theorem zero_le_quantumCapacity ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) :  $0 \leq \Lambda$ .quantumCapacity
  le_csSup (bddAbove_achievesRate  $\Lambda$ ) (achievesRate_0  $\Lambda$ )

/-- Quantum channel capacity is at most  $\log_2(D)$ , where  $D$  is the input dimension. -/
theorem quantumCapacity_ge_log_dim_in ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) :  $\Lambda$ .quantumCapacity
  Real.sSup_le (by
    intro R h
    contrapose h
    exact not_achievesRate_gt_log_dim_in  $\Lambda$  (lt_of_not_ge h))
  (by
    by_cases h : Nonempty  $d_1$ 
    · apply Real.logb_nonneg one_lt_two (Nat.one_le_cast.mpr Fintype.card_pos)
    · simp [not_nonempty_iff.mp h])

/-- The LSD (Lloyd-Shor-Devetak) theorem: the quantum capacity is at least the coherent
information. The "coherent information" is used in literature to refer to the coherent
information of a channel (coherentInfo), or a function of just a channel. In the latter case,
it is maximized over. Here we use the former definition and state that the lower bound is
achieved. -/
theorem coherentInfo_le_quantumCapacity ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) ( $\rho$  : MState  $d_1$ )
  sorry

/-- The quantum capacity is the limit of the coherent information of  $n$ -copy uses of the channel. -/
theorem quantumCapacity_eq_piProd_coherentInfo ( $\Lambda$  : CPTPMap  $d_1$   $d_2$ ) :  $\Lambda$ .quantumCapacity
  sSup { r :  $\mathbb{R}$  |  $\exists n \rho, r = \text{coherentInfo } \rho (\text{CPTPMap.piProd } (\text{fun } \_ : \text{Fin } n \Rightarrow \Lambda))$  }
  sorry

end capacity

```

1.430 QuantumInfo/Finite/Capacity_doc.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
/-!
# Defining Quantum Capacity

```

Quantum capacity is defined in different reference works in at least seven meaningful

(1) <https://www.uio.no/studier/emner/matnat/math/MAT4430/v23/timeplan/lectureblup.pdf>

Defines the notion of " (n, m, δ) -coding scheme", a code for m qubits in n copies of norm distance of δ from the identity channel. Then a rate R is "achievable" if there

(2) <https://cs.uwaterloo.ca/~watrous/TQI/TQI.8.pdf>, Defn 8.42.

Watrous doesn't use the word "coding scheme", but rather define "emulating" (8.1) and "approximation" (8.2) of the identity. This is equivalent to the coding scheme and decoding scheme. Now the set of achievable rates may be an open or closed interval, and

(3) Works like <https://arxiv.org/pdf/1007.2855> (equation 3) and https://arxiv.org/pdf/shot_channel_capacity), and so it is the average coherent information achieved across n limit. This definition makes the LSD theorem (which states that $Q \geq Q^{(1)}$) actually

(4) <https://arxiv.org/pdf/quant-ph/0304127> (Section 5) and <https://arxiv.org/pdf/quant-ph/9809010> specifically distinguish between "subspace transmission", "entanglement transmission" and "entanglement generation". Convergence in fidelity is surely equivalent to convergence in diamond norm on the d -dimensional space of qubits. arxiv:9809010 specifically requires a sequence of codes

(5) "Entanglement transmission" asks for a high-entropy state (thus, a large subspace)

(6) "Entanglement generation" changes the task from coding. Instead we need a bipartite state ϵ . The rates are $(1/n) \cdot (\log \kappa)$, achievable if for all δ and sufficiently large n (etc)

Note: Option (6) is what Devetak, in arxiv:0304127, actually proves the LSD theorem

(7) Wilde's book also of "coding scheme", but defines it in terms of entanglement generation

To capture the idea of "quantum capacity", and some other idea that turns out to be "coding scheme" is cleaner work with as a way to directly construct or extract codes,

This leads to the final definition in [Capacity.lean](./QuantumInfo/Finite/
-/

1.431 QuantumInfo/Finite/Channel/DegradableOrder.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.CPTPMap

/-! # The Degradable Order

We define the "degradable order" on channels, where  $M \leq N$  if  $N$  can be degra
This is a scoped instance, because there are other reasonable orders on cha

The degradable order is actually only a preorder, because channels that are
each other are not necessarily equal. (For instance, unitary channels are d
the identity and vice versa.) It can compare channels of different output d
they must have the same input dimension. For this reason the `DegradablePre
by the channel input type.

One might hope that if this was defined on the quotient type of "channels u
that it would become an order; but this is not the case. That is, there are
are degradable to each other, but cannot be degraded to each other by unita
let  $A$  be the replacement channel that goes to a mixed state, and  $B$  be a rep
to a pure state.

Technical notes: to model "channels of different output types", the preorder
type of channels parameterized by their output type. And since the output t
Fintype and DecidableEq, the argument is also a Sigma to bring this along.
-/

```

section

```

def DegradablePreorder (dIn : Type*) [Fintype dIn] [DecidableEq dIn] : Preo
  (Σ dOut : (Σ t, Fintype t × DecidableEq t), let _ := dOut.snd.1; CPTPMap
  le  $\Lambda_1$   $\Lambda_2$  :=
    let _ :=  $\Lambda_1$ .fst.snd.1;
    let _ :=  $\Lambda_2$ .fst.snd.1;
    let _ :=  $\Lambda_2$ .fst.snd.2;
     $\Lambda_2$ .snd.IsDegradableTo  $\Lambda_1$ .snd;
  le_refl  $\Lambda$  :=

```

```

    let _ :=  $\Lambda$ .fst.snd.1;
    let _ :=  $\Lambda$ .fst.snd.2;
    (CPTPMap.id, CPTPMap.compose_id  $\Lambda$ .snd)
le_trans  $\Lambda_1$   $\Lambda_2$   $\Lambda_3$   $h_{12}$   $h_{23}$  := by
  let _ :=  $\Lambda_1$ .fst.snd.1;
  let _ :=  $\Lambda_1$ .fst.snd.2;
  let _ :=  $\Lambda_2$ .fst.snd.1;
  let _ :=  $\Lambda_2$ .fst.snd.2;
  let _ :=  $\Lambda_3$ .fst.snd.1;
  let _ :=  $\Lambda_3$ .fst.snd.2;
  obtain (D12, hD12) :=  $h_{12}$ ;
  obtain (D23, hD23) :=  $h_{23}$ ;
  use D12.compose D23
  rwa [CPTPMap.compose_assoc, hD23]
end

```

1.432 QuantumInfo/Finite/Distance.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.Distance.Fidelity
import QuantumInfo.Finite.Distance.TraceDistance

/-!
Distances between quantum states.
* Fidelity
* Trace distance
* Bures metric
* Superfidelity
* Quantum Wasserstein Distance

All share some axioms: https://quantiki.org/wiki/distance-measures-between-states
-/

```

1.433 QuantumInfo/Finite/Distance/Fidelity.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.

```

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Alex Meiburg

-/

import QuantumInfo.Finite.CPTPMap

noncomputable section

open BigOperators

open ComplexConjugate

open Kronecker

open scoped Matrix ComplexOrder

variable {d d₂ : Type*} [Fintype d] [DecidableEq d] [Fintype d₂] (ρ σ : MSt

namespace MState

/-- The fidelity of two quantum states. This is the quantum version of the

/

def fidelity (ρ σ : MState d) : ℝ :=

(σ.M.conj ρ.M.sqrt.mat).sqrt.trace

theorem fidelity_ge_zero : 0 ≤ fidelity ρ σ := by

apply HermitianMat.trace_nonneg

apply HermitianMat.sqrt_nonneg

theorem fidelity_le_one : fidelity ρ σ ≤ 1 :=

sorry --submultiplicativity of trace and sqrt

/-- The fidelity, as a `Prob` probability with value between 0 and 1. -

/

def fidelity_prob : Prob :=

(fidelity ρ σ, (fidelity_ge_zero ρ σ, fidelity_le_one ρ σ))

/-- A state has perfect fidelity with itself. -/

theorem fidelity_self_eq_one : fidelity ρ ρ = 1 := by

simp only [fidelity, HermitianMat.sqrt_eq_cfc_rpow_half]

conv =>

enter [1, 1, 1, 2]

rw [← HermitianMat.cfc_id ρ.M]

rw [HermitianMat.cfc_conj, ← HermitianMat.cfc_comp_apply]

convert ρ.tr using 2

convert ρ.M.cfc_id using 1

apply HermitianMat.cfc_congr_of_nonneg ρ.nonneg

intro x hx

simp only [one_div, Pi.mul_apply, id_eq, Pi.pow_apply]

rw [← Real.rpow_two, Real.rpow_inv_rpow hx (by norm_num), ← sq, ← Real.rp


```

    exact Real.rpow_rpow_inv hx (by norm_num)

/-- The fidelity is 1 if and only if the two states are the same. -
/
theorem fidelity_eq_one_iff_self : fidelity  $\rho$   $\sigma$  = 1  $\leftrightarrow$   $\rho$  =  $\sigma$  :=
  {by sorry,
    fun h  $\mapsto$  h  $\triangleright$  fidelity_self_eq_one  $\rho$ }

/-- The fidelity is a symmetric quantity. -/
theorem fidelity_symm : fidelity  $\rho$   $\sigma$  = fidelity  $\sigma$   $\rho$  :=
  sorry --break into sqrts

/-- The fidelity cannot decrease under the application of a channel. -
/
theorem fidelity_channel_nondecreasing [DecidableEq  $d_2$ ] ( $\Lambda$  : CPTPMap  $d$   $d_2$ ) : fidelity
  sorry

--TODO: Real.arccos  $\circ$  fidelity forms a metric (triangle inequality), the Fubini–Stud
--Matches with classical (squared) Bhattacharyya coefficient
--Invariance under unitaries
--Uhlmann's theorem

end MState

```

1.434 QuantumInfo/Finite/Distance/TraceDistance.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.MState

import QuantumInfo.ForMathlib

noncomputable section

open Classical
open BigOperators
open ComplexConjugate
open Kronecker
open scoped Matrix ComplexOrder

variable { $d$  : Type*} [Fintype  $d$ ] [DecidableEq  $d$ ]

```

```

/--The trace distance between two quantum states: half the trace norm of th
/
def TrDistance (ρ σ : MState d) : ℝ :=
  (1/2:ℝ) * (ρ.m - σ.m).traceNorm

namespace TrDistance

variable {d d₂ : Type*} [Fintype d] [Fintype d₂] (ρ σ : MState d)

theorem ge_zero : 0 ≤ TrDistance ρ σ := by
  rw [TrDistance]
  simp [Matrix.traceNorm_nonneg]

theorem le_one : TrDistance ρ σ ≤ 1 :=
  calc TrDistance ρ σ
    _ = (1/2:ℝ) * (ρ.m - σ.m).traceNorm := by rfl
    _ ≤ (1/2:ℝ) * (ρ.m.traceNorm + σ.m.traceNorm) := by
      linarith [Matrix.traceNorm_triangleIneq' ρ.m σ.m]
    _ = (1/2:ℝ) * (1 + 1) := by
      rw [ρ.traceNorm_eq_1, σ.traceNorm_eq_1]
    _ = 1 := by norm_num

/-- The trace distance, as a `Prob` probability with value between 0 and 1.
/
def prob : Prob :=
  ⟨TrDistance ρ σ, ⟨ge_zero ρ σ, le_one ρ σ⟩⟩

/-- The trace distance is a symmetric quantity. -/
theorem symm : TrDistance ρ σ = TrDistance σ ρ := by
  dsimp [TrDistance]
  rw [← Matrix.traceNorm_eq_neg_self, neg_sub]

/-- The trace distance is equal to half the 1-norm of the eigenvalues of th
/
theorem eq_abs_eigenvalues : TrDistance ρ σ = (1/2:ℝ) *
  ∑ i, abs ((ρ.Hermitian.sub σ.Hermitian).eigenvalues i) := by
  rw [TrDistance, Matrix.traceNorm_Hermitian_eq_sum_abs_eigenvalues]

-- Fuchs-van de Graaf inequalities
-- Relation to classical TV distance

```

1.435 QuantumInfo/Finite/Ensemble.lean

```

/-
Copyright (c) 2025 Leonardo A Lessa. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Leonardo A Lessa
-/
import QuantumInfo.Finite.MState

open MState
open BigOperators

noncomputable section

/-- A mixed-state ensemble is a random variable valued in `MState d`. That is,
a collection of mixed states `var :  $\alpha \rightarrow \text{MState } d$ `, each with their own probability w
described by `distr : ProbDistribution  $\alpha$ `. -/
abbrev MEnsemble (d : Type*) ( $\alpha$  : Type*) [Fintype d] [DecidableEq d] [Fintype  $\alpha$ ] :=

/-- A pure-state ensemble is a random variable valued in `Ket d`. That is,
a collection of pure states `var :  $\alpha \rightarrow \text{Ket } d$ `, each with their own probability weigh
described by `distr : ProbDistribution  $\alpha$ `. -/
abbrev PEnsemble (d : Type*) ( $\alpha$  : Type*) [Fintype d] [Fintype  $\alpha$ ] := ProbDistribution

variable { $\alpha$   $\beta$  d : Type*} [Fintype  $\alpha$ ] [Fintype  $\beta$ ] [Fintype d] [DecidableEq d]

/-- Alias for `ProbDistribution.var` for mixed-state ensembles. -
/
abbrev MEnsemble.states [Fintype  $\alpha$ ] : MEnsemble d  $\alpha \rightarrow (\alpha \rightarrow \text{MState } d) := \text{ProbDistribution}$ 

/-- Alias for `ProbDistribution.var` for pure-state ensembles. -
/
abbrev PEnsemble.states [Fintype  $\alpha$ ] : PEnsemble d  $\alpha \rightarrow (\alpha \rightarrow \text{Ket } d) := \text{ProbDistribution}$ 

namespace Ensemble

/-- A pure-state ensemble is a mixed-state ensemble if all kets are interpreted as m
/
@[coe] def toMEnsemble : PEnsemble d  $\alpha \rightarrow \text{MEnsemble } d \alpha := \text{Functor.map pure}$ 

instance : Coe (PEnsemble d  $\alpha$ ) (MEnsemble d  $\alpha$ ) := ⟨toMEnsemble⟩

@[simp]
theorem toMEnsemble_mk : (toMEnsemble ⟨ps, distr⟩ : MEnsemble d  $\alpha$ ) = ⟨pure  $\circ$  ps, dis
rfl

```

```

/-- A mixed-state ensemble comes from a pure-state ensemble if and only if
/
theorem coe_PEnsemble_iff_pure_states (me : MEnsemble d  $\alpha$ ): ( $\exists$  pe : PEnsemble d  $\alpha$ ) :
  constructor
  · intro ⟨pe, hpe⟩
    use pe.states
    ext1 i
    subst hpe
    rfl
  · intro ⟨ $\psi$ , h $\psi$ ⟩
    use ⟨ $\psi$ , me.distr⟩
    simp only [toMEnsemble_mk]
    congr
    exact h $\psi$ .symm

/-- The resulting mixed state after mixing the states in an ensemble with their
    respective probability weights. Note that, generically, a single mixed state
    ensembles that mixes into it. -/
def mix (e : MEnsemble d  $\alpha$ ) : MState d := ProbDistribution.expect_val e

@[simp]
theorem mix_of (e : MEnsemble d  $\alpha$ ) : (mix e).m =  $\sum i$ , (e.distr i :  $\mathbb{R}$ ) • (e.distr i).pure
  apply AddSubgroup.val_finset_sum -- *laughs in defeq*

/-- Two mixed-state ensembles indexed by ` $\alpha$ ` and ` $\beta$ ` are equivalent
/
def congrMEnsemble ( $\sigma$  :  $\alpha \approx \beta$ ) : MEnsemble d  $\alpha \approx$  MEnsemble d  $\beta$  := ProbDistribution.map_congr_eq_congr_map  $\sigma$ 

/-- Two pure-state ensembles indexed by ` $\alpha$ ` and ` $\beta$ ` are equivalent
/
def congrPEnsemble ( $\sigma$  :  $\alpha \approx \beta$ ) : PEnsemble d  $\alpha \approx$  PEnsemble d  $\beta$  := ProbDistribution.map_congr_eq_congr_map  $\sigma$ 

/-- Equivalence of mixed-state ensembles leaves the resulting mixed state invariant
/
@[simp]
theorem mix_congrMEnsemble_eq_mix ( $\sigma$  :  $\alpha \approx \beta$ ) (e : MEnsemble d  $\alpha$ ) : mix (congrMEnsemble  $\sigma$  e) =
  ProbDistribution.expect_val_congr_eq_expect_val  $\sigma$  e

/-- Equivalence of pure-state ensembles leaves the resulting mixed state invariant
/
@[simp]
theorem mix_congrPEnsemble_eq_mix ( $\sigma$  :  $\alpha \approx \beta$ ) (e : PEnsemble d  $\alpha$ ) : mix (toMEnsemble (congrPEnsemble  $\sigma$  e)) =
  unfold toMEnsemble congrPEnsemble mix
  rw [ProbDistribution.map_congr_eq_congr_map MState.pure  $\sigma$  e]
  exact ProbDistribution.expect_val_congr_eq_expect_val  $\sigma$  (MState.pure <$> e)

```

```

/-- The average of a function `f : MState d → T`, where `T` is of `Mixable U T` instance
state ensemble `e`
is the expectation value of `f` acting on the states of `e`, with the corresponding
/
def average {T : Type _} {U : Type*} [AddCommGroup U] [Module ℝ U] [inst : Mixable U T]
  ProbDistribution.expect_val <| f <$> e

/-- A version of `average` conveniently specialized for functions `f : MState d → ℝ≥0`
Notably, the average is also a nonnegative real number. -
/
def average_NNReal {d : Type _} [Fintype d] [DecidableEq d] (f : MState d → NNReal)
  (average (NNReal.toReal ∘ f) e,
   ProbDistribution.zero_le_expect_val e.distr (NNReal.toReal ∘ f ∘ e.states) (fun
/- The average of a function `f : Ket d → T`, where `T` is of `Mixable U T` instance
state ensemble `e`
is the expectation value of `f` acting on the states of `e`, with the corresponding
/
def pure_average {T : Type _} {U : Type*} [AddCommGroup U] [Module ℝ U] [inst : Mixable U T]
  ProbDistribution.expect_val <| f <$> e

/-- A version of `average` conveniently specialized for functions `f : Ket d → ℝ≥0`
Notably, the average is also a nonnegative real number. -
/
def pure_average_NNReal {d : Type _} [Fintype d] (f : Ket d → NNReal) (e : PEnsemble d)
  (pure_average (NNReal.toReal ∘ f) e,
   ProbDistribution.zero_le_expect_val e.distr (NNReal.toReal ∘ f ∘ e.states) (fun
/- The average of `f : MState d → T` on a coerced pure-state ensemble `e : MEnsemble d`
is equal to averaging the restricted function over Kets `f ∘ pure : Ket d → T` on `e`
/
theorem average_of_pure_ensemble {T : Type _} {U : Type*} [AddCommGroup U] [Module ℝ U]
  (f : MState d → T) (e : PEnsemble d) (α) :
  average f (toMEnsemble e) = pure_average (f ∘ pure) e := by
  simp only [average, pure_average, toMEnsemble, comp_map]

variable {ψ : Ket d}

@[simp]
theorem distr_toMEnsemble (e : PEnsemble d) (α) : (toMEnsemble e).distr = e.distr := by
  rfl

/-
A pure-state ensemble mixes into a pure state if and only if
the only states in the ensemble with nonzero probability are equal
to the same Ket `ψ` up to a global phase.

```

```

-/
theorem mix_pEnsemble_pure_iff_pure {e : PEnsemble d α} :
  mix (toMEnsemble e) = MState.pure ψ ↔
  ∀ i : α, e.distr i ≠ 0 → MState.pure (e.states i) = MState.pure ψ := by
  refine {fun h i hi => ?_, fun h => ?_}
  · apply MState.eq_of_sum_eq_pure ?_ ?_ ?_ e.distr.normalized i (Finset.me
    · exact_mod_cast lt_of_le_of_ne (e.distr i).zero_le hi.symm
    · exact (MState.pure_iff_purity_one _).mp {ψ, rfl}
    · exact congr_arg MState.M h.symm
    · grind
  · have h_sum : mix (toMEnsemble e) = ∑ i, (e.distr i).val • (MState.pure
    refine Finset.sum_congr rfl fun i _ => ?_
    by_cases hi : e.distr i = 0
    · simp [hi]
    · rw [← h i hi]
      rfl
    simp [MState.ext_iff, h_sum, ← Finset.sum_smul]

/- The theorem below is also false for the same reason as the original `mix
   knowing `MState.pure (e.states i) = MState.pure ψ` does not imply `e.sta
   so `f (e.var i) ≠ f ψ` in general for a non-phase-invariant `f : Ket d →
/
/-- The average of `f : Ket d → T` on an ensemble that mixes to a pure stat
/
theorem mix_pEnsemble_pure_average {e : PEnsemble d α} {T : Type _} {U : Ty
  [inst : Mixable U T] (f : Ket d → T) (hf : ∀ ψ φ, Ket.PhaseEquiv.r ψ φ
  (hmix : mix (toMEnsemble e) = MState.pure ψ) :
  pure_average f e = f ψ := by
  have hpure := mix_pEnsemble_pure_iff_pure.mp hmix
  -- Each state with nonzero probability is phase-equivalent to ψ
  have hfeq : ∀ i, e.distr i ≠ 0 → f (e.var i) = f ψ := fun i hdi =>
    hf _ _ ((MState.PhaseEquiv_iff_pure_eq _ _).mpr (hpure i hdi))
  simp only [pure_average, Functor.map, ProbDistribution.expect_val]
  apply Mixable.to_U_inj
  simp only [Mixable.to_U_of_mkT, Function.comp_apply]
  have h1 : ∀ i ∈ Finset.univ, (e.distr i : ℝ) • (Mixable.to_U (f (e.var i)
  -- For nonzero summands, distr i ≠ 0 so f (e.var i) = f ψ
  have h2 : ∀ i ∈ Finset.univ, (e.distr i : ℝ) • Mixable.to_U (f (e.var i))
    intro i _
    by_cases hdi : e.distr i = 0
    · simp [hdi]
    · rw [hfeq i hdi]
  rw [Finset.sum_congr rfl h2, ← Finset.sum_smul, ProbDistribution.normaliz

theorem sum_prob_mul_eq_one_iff {ι : Type*} [Fintype ι] (p : ι → ℝ) (x : ι
  (hp : ∀ i, 0 ≤ p i) (hsum : ∑ i, p i = 1) (hx : ∀ i, x i ≤ 1) :

```

```

    (∑ i, p i * x i = 1) ↔ ∀ i, p i ≠ 0 → x i = 1 := by
  constructor
  · intro a i a_1
    contrapose! a
    have h : ∃ i, p i * x i < p i := by
      use i
      apply mul_lt_of_lt_one_right
      · exact lt_of_le_of_ne' (hp i) a_1
      · exact lt_of_le_of_ne (hx i) a
    replace h : ∑ i, p i * x i < ∑ i, p i :=
      Finset.sum_lt_sum (fun i _ => by nlinarith [hp i, hx i]) (by simp using h)
    exact (h.trans_le (by simp [hsum])).ne
  · intro a
    rw [← hsum]
    congr! with i
    by_cases hi : p i = 0
    · simp [hi]
    · simp [a i hi]

```

--CLEANUP. This proof used to work but it took forever to build. Also now it's broke
theorem MState.exp_val_pure_eq_one_iff {d : Type*} [Fintype d] [DecidableEq d] (ρ :

```

  ρ.exp_val (pure ψ) = 1 ↔ ρ = pure ψ := by
  stop
  constructor <=> intro h <=> simp_all +decide [ MState.exp_val ];
  · have h_eq : ρ.M = (MState.pure ψ).M := by
      have h_eq : (ρ.M - (MState.pure ψ).M).inner (ρ.M - (MState.pure ψ).M) = 0 := b
      have h_eq_inner : (ρ.M - (MState.pure ψ).M).inner (ρ.M - (MState.pure ψ).M)
        norm_num [ HermitianMat.inner, Matrix.mul_apply ];
        simp +decide [ Matrix.mul_sub, Matrix.sub_mul, Matrix.trace_sub, Matrix.tr
        ring;
      have h_eq_inner : ρ.M.inner ρ.M ≤ 1 ∧ (MState.pure ψ).M.inner (MState.pure ψ
        aesop;
        · have := ρ.M.inner_le_mul_trace ρ.zero_le ρ.zero_le;
          aesop;
        · simp +decide [ HermitianMat.inner ];
          have := MState.pure_mul_self ψ; aesop;
      have h_eq_inner : (ρ.M - (MState.pure ψ).M).inner (ρ.M - (MState.pure ψ).M)
        exact?;
      linarith;
      have h_eq : ∀ (A : HermitianMat d ℂ), A.inner A = 0 → A = 0 := by
        intro A hA;
        exact?;
        exact sub_eq_zero.mp ( h_eq _ <_> );
      cases ρ ; cases ψ ; aesop;
  · unfold HermitianMat.inner; aesop;
  rw [ MState.pure_mul_self ] ; aesop

```

```

theorem mix_mEnsemble_pure_iff_pure {e : MEnsemble d  $\alpha$ } :
  mix e = pure  $\psi \leftrightarrow \forall i : \alpha, e.\text{distr } i \neq 0 \rightarrow e.\text{states } i = \text{MState.pure } \psi :=
  \text{have } h : (\text{mix } e).\text{exp\_val } \uparrow(\text{MState.pure } \psi) = \sum i, \uparrow(e.\text{distr } i) * (e.\text{states } i) :=
  \text{simp } [\text{MState.exp\_val}, \text{HermitianMat.inner\_def}, \text{Finset.sum\_mul}]
  \text{rw } [\leftarrow \text{MState.exp\_val\_pure\_eq\_one\_iff}, h, \text{sum\_prob\_mul\_eq\_one\_iff}]
  · \text{simp only } [\text{MState.exp\_val\_pure\_eq\_one\_iff}, \text{ne\_eq}, \text{Set.Icc.coe\_eq\_zero}]
  · \text{exact fun } i \Rightarrow (e.\text{distr } i).2.1;
  · \text{simp}
  · \text{intro } i
    \text{apply } (e.\text{states } i).\text{exp\_val\_le\_one } (\text{MState.le\_one } _)

/-- The average of `f : MState d  $\rightarrow$  T` on an ensemble that mixes to a pure state is the average of `f` on the pure state. -/
theorem mix_mEnsemble_pure_average {e : MEnsemble d  $\alpha$ } {T : Type _} {U : Type _}
  average f e = f (pure  $\psi$ ) := by
  \text{have } hpure := \text{mix\_mEnsemble\_pure\_iff\_pure.mp } hmix
  \text{simp only } [\text{average}, \text{Functor.map}, \text{ProbDistribution.expect\_val}]
  \text{apply } \text{Mixable.to\_U\_inj}
  \text{rw } [\text{MEnsemble.states}] \text{ at } hpure
  \text{simp only } [\text{Mixable.to\_U\_of\_mkT}, \text{Function.comp\_apply}]
  \text{have } h1 : \forall i \in \text{Finset.univ}, (e.\text{distr } i : \mathbb{R}) \cdot (\text{Mixable.to\_U } (f (e.\text{var } i))) =
    \text{have } h2 : e.\text{distr } i = 0 \rightarrow (e.\text{distr } i : \mathbb{R}) \cdot (\text{Mixable.to\_U } (f (e.\text{var } i))) = 0 :=
    \text{simp only } [h0, \text{Prob.coe\_zero}, \text{zero\_smul}]
    \text{exact } (hpure i) \circ h2.mt
  \text{classical rw } [\leftarrow \text{Finset.sum\_filter\_of\_ne } h1, \text{Finset.sum\_filter}]
  \text{classical conv } \Rightarrow
    \text{enter } [1, 2, a]
    \text{rw } [\leftarrow \text{dite\_eq\_ite}]
    \text{enter } [2, hvar]
    \text{rw } [hvar]
  \text{classical conv } \Rightarrow
    \text{enter } [1, 2, a]
    \text{rw } [\text{dite\_eq\_ite}]
    \text{rw } [\leftarrow \text{ite\_zero\_smul}]
  \text{have } hpure' : \forall i \in \text{Finset.univ}, (\uparrow(e.\text{distr } i) : \mathbb{R}) \neq 0 \rightarrow e.\text{var } i = \text{pure } \psi :=
  \text{apply } hpure i
  \text{simpa using } hne0
  \text{classical rw } [\leftarrow \text{Finset.sum\_smul}, \leftarrow \text{Finset.sum\_filter}, \text{Finset.sum\_filter\_of\_ne}]

/-- The trivial mixed-state ensemble of ` $\rho$ ` consists of copies of ` $\rho$ `, with one having probability 1. -/
def trivial_mEnsemble ( $\rho$  : MState d) (i :  $\alpha$ ) : MEnsemble d  $\alpha$  :=  $\langle \text{fun } _ \mapsto \rho, 1 \rangle$ 

/-- The trivial mixed-state ensemble of ` $\rho$ ` mixes to ` $\rho$ ` -$ 
```



```

/
theorem trivial_mEnsemble_mix (ρ : MState d) : ∀ i : α, mix (trivial_mEnsemble ρ i)
  apply MState.ext_m
  classical simp only [trivial_mEnsemble, ProbDistribution.constant, mix_of, DFunLike,
    Prob.coe_one, Prob.coe_zero, ite_smul, one_smul, zero_smul, Finset.sum_ite_eq,
    Finset.mem_univ, ↓reduceIte]

/-- The average of `f : MState d → T` on a trivial ensemble of `ρ` is `f ρ` -
/
theorem trivial_mEnsemble_average {T : Type _} {U : Type*} [AddCommGroup U] [Module
  U] (f : MState d → T) (ρ : MState d) : average f (trivial_mEnsemble ρ) = f ρ := fun i => by
  simp only [average, Functor.map, ProbDistribution.expect_val, trivial_mEnsemble]
  apply Mixable.to_U_inj
  classical simp [apply_ite]

instance MEnsemble.instInhabited [Nonempty d] [Inhabited α] : Inhabited (MEnsemble d α)
  default := trivial_mEnsemble default default

/-- The trivial pure-state ensemble of `ψ` consists of copies of `ψ`, with the `i`-th
  one having
  probability 1. -/
def trivial_pEnsemble (ψ : Ket d) (i : α) : PEnsemble d α := ⟨fun _ => ψ, ProbDistribution.one⟩

variable (ψ : Ket d)

/-- The trivial pure-state ensemble of `ψ` mixes to `ψ` -
/
theorem trivial_pEnsemble_mix : ∀ i : α, mix (toMEnsemble (trivial_pEnsemble ψ i)) =
  apply MState.ext_m
  classical simp only [trivial_pEnsemble, ProbDistribution.constant, toMEnsemble_mk,
    apply_ite, Prob.coe_one, Prob.coe_zero, MEnsemble.states, Function.comp_apply,
    one_smul, zero_smul, Finset.sum_ite_eq, Finset.mem_univ, ↓reduceIte]

omit [DecidableEq d] in
/-- The average of `f : Ket d → T` on a trivial ensemble of `ψ` is `f ψ` -
/
theorem trivial_pEnsemble_average {T : Type _} {U : Type*} [AddCommGroup U] [Module
  U] (f : Ket d → T) (ψ : Ket d) : pure_average f (trivial_pEnsemble ψ) = f ψ := fun i => by
  simp only [pure_average, Functor.map, ProbDistribution.expect_val, trivial_pEnsemble]
  apply Mixable.to_U_inj
  classical simp [apply_ite]

instance PEnsemble.instInhabited [Nonempty d] [Inhabited α] : Inhabited (PEnsemble d α)
  default := trivial_pEnsemble default default

/-- The spectral pure-state ensemble of `ρ`. The states are its eigenvectors, and the

```

```

/
def spectral_ensemble (p : MState d) : PEnsemble d d where
  var i :=
    { vec := p.Hermitian.eigenvectorBasis i
      normalized' := by
        rw [←one_pow 2, ←p.Hermitian.eigenvectorBasis.orthonormal.1 i]
        have hnonneg : 0 ≤ ∑ x : d, Complex.normSq (p.Hermitian.eigenvectorBasis i x)
        simp_rw [Complex.normSq_eq_norm_sq]
        positivity
        simp only [← Complex.normSq_eq_norm_sq, EuclideanSpace.norm_eq, Real.norm_eq]
      }
  distr := p.spectrum

--PULLOUT
theorem spectral_decomposition_sum {d : Type*} [Fintype d] [DecidableEq d]
  {A : Matrix d d ℂ} (hA : A.IsHermitian) :
  A = ∑ i, (hA.eigenvalues i) • (Matrix.vecMulVec (hA.eigenvectorBasis i)
nth_rw 1 [hA.spectral_theorem])
ext
simp only [Matrix.sum_apply]
simp only [Unitary.conjStarAlgAut_apply, mul_assoc]
simp only [Matrix.mul_apply, Matrix.IsHermitian.eigenvectorUnitary_apply,
Function.comp_apply, Matrix.star_apply, RCLike.star_def, mul_comm, mul_
Finset.sum_ite_eq, Finset.mem_univ, λreduceIte, Matrix.vecMulVec, Pi.star_def,
Matrix.smul_apply, Matrix.of_apply, Algebra.smul_def]
simp_rw [mul_assoc]

/-- The spectral pure-state ensemble of `p` mixes to `p` -
/
theorem spectral_ensemble_mix {p : MState d} : mix (↑(spectral_ensemble p))
ext i j
convert rfl;
convert rfl;
apply MState.ext_m;
convert Ensemble.mix_of _
convert (spectral_decomposition_sum p.Hermitian) using 1

end Ensemble

```

1.436 QuantumInfo/Finite/Entanglement.lean

```

/-
Copyright (c) 2025 Leonardo A Lessa. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.

```

Authors: Leonardo A Lessa

```
-/
import QuantumInfo.Finite.Braket
import QuantumInfo.Finite.Ensemble
import QuantumInfo.Finite.Entropy
import QuantumInfo.ClassicalInfo.Entropy
```

```
/-!
```

Entanglement measures. (Mixed) convex roof extensions. Definition of product / separable states are in `Braket.lean` and/or `MState.lean`

Important definitions:

- * `convex\_roof`: Convex roof extension of $g : \text{KetUpToPhase } d \rightarrow \mathbb{R}_{\geq 0}$
- * `mixed\_convex\_roof`: Mixed convex roof extension of $f : \text{MState } d \rightarrow \mathbb{R}_{\geq 0}$
- * `EoF`: Entanglement of Formation

TODO:

- Other entanglement measures (not necessarily based on convex roof extensions). In particular, it is difficult to implement: (Logarithmic) Negativity, Entanglement of Purification, Entropy of Entanglement, Entanglement Cost, Distillable Entanglement. For a compendium on the zoo of entanglement measures, see [1] Christandl, Matthias. "The Structure of Bipartite Quantum States - Insights from Entanglement Theory." <https://doi.org/10.48550/arXiv.quant-ph/0604183>.
- Define classes of entanglement measures with good properties, including: monotonicity, monotonicity on average under LOCC, convexity (if the latter two are present, it implies the former by some), vanishing on separable states, normalized on the maximally entangled state. For other properties, see [1] above and: [2] Szalay, Szilárd. "Multipartite Entanglement Measures." (mainly Sec. IV) <https://doi.org/10.1103/PhysRevA.92.042329>. [3] Horodecki, Ryszard, Paweł Horodecki, Michał Horodecki, and Karol Horodecki. "Entanglement Measures and Properties." <https://doi.org/10.1103/RevModPhys.81.865>.
- Useful properties of convex roof extensions:
 1. If f is monotonically non-increasing under LOCC, so is its convex roof.
 2. If $f(\psi)$ is zero if and only if ψ is a product state, then its convex roof is faithful, i.e. the mixed state is separable.
 For other properties, see Sec. IV.F of [2] above.

```
-/
```

noncomputable section

```
open ENNReal
open NNReal
open MState
open Ensemble
```

```
/-- Convex roof extension of a function  $g : \text{KetUpToPhase } d \rightarrow \mathbb{R}_{\geq 0}$ , defined as the i
```

state
ensembles of a given ρ of the average of g in that ensemble.

This is valued in the extended nonnegative real numbers $\mathbb{R}_{\geq 0\infty}$ to have good come from the fact that $\mathbb{R}_{\geq 0\infty}$ is a complete lattice. For example, it is ne However, it is also proven in `convex_roof_ne_top` that the convex roof is be used in most applications. -/

```
def convex_roof_ENNReal {d : Type _} [Fintype d] [DecidableEq d] (g : KetUp
  [] (n : ℕ+) (e : PEnsemble d (Fin n)) ( _ : mix (toMEnsemble e) = ρ), ↑(pur
```

```
/-- Mixed convex roof extension of a function  $f : MState\ d \rightarrow \mathbb{R}_{\geq 0}$ , defined
state
ensembles of a given  $\rho$  of the average of  $f$  on that ensemble.
```

This is valued in the extended nonnegative real numbers $\mathbb{R}_{\geq 0\infty}$ to have good come from the fact that $\mathbb{R}_{\geq 0\infty}$ is a complete lattice (see `ENNReal.instComp` it is also proven in `mixed_convex_roof_ne_top` that the mixed convex roof be used in most applications. -/

```
def mixed_convex_roof_ENNReal {d : Type _} [Fintype d] [DecidableEq d] (f :
  [] (n : ℕ+) (e : MEnsemble d (Fin n)) ( _ : mix e = ρ), ↑(average_NNReal f
```

```
variable {d d1 d2 : Type _} [Fintype d] [Fintype d1] [Fintype d2] [Nonempty
variable [DecidableEq d] [DecidableEq d1] [DecidableEq d2]
variable (f : MState d → ℝ≥0)
variable (g : KetUpToPhase d → ℝ≥0)
```

```
/-- The convex roof extension convex_roof_ENNReal is never  $\infty$ . -
/
```

```
theorem convex_roof_ne_top : ∀ ρ, convex_roof_ENNReal g ρ ≠ ∞ := fun ρ => b
  simp only [convex_roof_ENNReal, ne_eq, iInf_eq_top, coe_ne_top, imp_false
  use {Fintype.card d, Fintype.card_pos}
  have ed : d ≈ Fin ↑({Fintype.card d, Fintype.card_pos} : ℕ+) := by
    simp only
    exact Fintype.equivFin d
  use (congrPEnsemble ed) <| spectral_ensemble ρ
  rw [mix_congrPEnsemble_eq_mix ed]
  push_neg
  convert spectral_ensemble_mix
```

```
omit [Nonempty d] in
```

```
/-- The convex roof extension mixed_convex_roof_ENNReal is never  $\infty$ . -
/
```

```
theorem mixed_convex_roof_ne_top : ∀ ρ, mixed_convex_roof_ENNReal f ρ ≠ ∞ :
  simp only [mixed_convex_roof_ENNReal, ne_eq, iInf_eq_top, coe_ne_top, imp
  use 1, trivial_mEnsemble ρ 0
  push_neg
```

```

    exact trivial_mEnsemble_mix p 0

/-- Convex roof extension of a function `g : KetUpToPhase d → ℝ≥0`, defined as the i
state
ensembles of a given `p` of the average of `g` in that ensemble.

This is valued in the nonnegative real numbers `ℝ≥0` by applying `ENNReal.toNNReal`
it should be used in proofs alongside `convex_roof_ne_top`. -
/
def convex_roof : MState d → ℝ≥0 := fun x ↦ (convex_roof_ENNReal g x).untop (convex_

/-- Mixed convex roof extension of a function `f : MState d → ℝ≥0`, defined as the i
state
ensembles of a given `p` of the average of `f` on that ensemble.

This is valued in the nonnegative real numbers `ℝ≥0` by applying `ENNReal.toNNReal`
it should be used in proofs alongside `mixed_convex_roof_ne_top`. -
/
def mixed_convex_roof : MState d → ℝ≥0 := fun x ↦ (mixed_convex_roof_ENNReal f x).un

/-- Auxiliary function. Convex roof of a function `f : MState d → ℝ≥0` defined over
restricting `f` to pure states, via `pureQ : KetUpToPhase d → MState d`. -
/
def convex_roof_of_MState_fun : MState d → ℝ≥0 := convex_roof (f ∘ pureQ)

-- TODO: make `le_convex_roof`, `convex_roof_le`, `le_mixed_convex_roof` and `mixed_
and-only-if` statements.

omit [Nonempty d] in
theorem le_mixed_convex_roof (p : MState d) :
  (∀ n > 0, ∀ e : MEnsemble d (Fin n), mix e = p → c ≤ average_NNReal f e) → (c ≤ mi
  unfold mixed_convex_roof
  rw [WithTop.le_untop_iff]
  apply le_iInf; intro ⟨n, hnpos⟩; apply le_iInf; intro e; apply le_iInf; intro hmix
  rw [some_eq_coe', ENNReal.coe_le_coe]
  exact h n hnpos e hmix

theorem le_convex_roof (p : MState d) :
  (∀ n > 0, ∀ e : PEnsemble d (Fin n), mix (toMEnsemble e) = p → c ≤ pure_average_NN
  unfold convex_roof
  rw [WithTop.le_untop_iff]
  apply le_iInf; intro ⟨n, hnpos⟩; apply le_iInf; intro e; apply le_iInf; intro hmix
  rw [some_eq_coe', ENNReal.coe_le_coe]
  exact h n hnpos e hmix

theorem convex_roof_le (p : MState d):

```

```

(∃ n > 0, ∃ e : PEnsemble d (Fin n), mix (toMEnsemble e) = ρ ∧ pure_average
  obtain ⟨n, hnpos, e, hmix, h⟩ := h
  unfold convex_roof
  rw [WithTop.untop_le_iff]
  apply iInf_le_of_le ⟨n, hnpos⟩; apply iInf_le_of_le e; apply iInf_le_of_l
  rw [some_eq_coe', ENNReal.coe_le_coe]
  exact h

omit [Nonempty d] in
theorem mixed_convex_roof_le (ρ : MState d):
(∃ n > 0, ∃ e : MEnsemble d (Fin n), mix e = ρ ∧ average_NNReal f e ≤ c) →
  obtain ⟨n, hnpos, e, hmix, h⟩ := h
  unfold mixed_convex_roof
  rw [WithTop.untop_le_iff]
  apply iInf_le_of_le ⟨n, hnpos⟩; apply iInf_le_of_le e; apply iInf_le_of_l
  rw [some_eq_coe', ENNReal.coe_le_coe]
  exact h

/-- The mixed convex roof extension of `f` is smaller than or equal to its
the former minimizes over a larger set of ensembles. -/
theorem mixed_convex_roof_le_convex_roof : mixed_convex_roof f ≤ convex_roof
  intro ρ
  apply le_convex_roof (f ∘ pureQ) ρ
  intro n hnpos e hmix
  apply mixed_convex_roof_le
  use n
  apply And.intro hnpos
  use f e
  apply And.intro hmix
  exact le_of_eq <| NNReal.coe_inj.mp <| average_of_pure_ensemble (toReal ∘

/-- The convex roof extension of `g : KetUpToPhase d → ℝ≥0` applied to a pu
/
theorem convex_roof_of_pure (ψ : Ket d) : convex_roof g (pure ψ) = g (KetUp
  rw [le_antisymm_iff]
  constructor
  · apply convex_roof_le
    use 1; simp only [gt_iff_lt, zero_lt_one, true_and]; use trivial_pEnsem
    constructor
    · exact trivial_pEnsemble_mix ψ 0
    · simp only [pure_average_NNReal, Fin.isValue, ← NNReal.coe_le_coe, coe
      rw [trivial_pEnsemble_average ψ _ 0]
      rfl
  · apply le_convex_roof
    intro n hnpos e hmix
    apply le_of_eq

```

```

simp only [pure_average_NNReal, ← NNReal.coe_inj, coe_mk]
have hphase_inv : ∀ a b, Ket.PhaseEquiv.r a b →
  (NNReal.toReal ∘ g ∘ KetUpToPhase.mk) a = (NNReal.toReal ∘ g ∘ KetUpToPhase.
  intro a b hab
  simp only [Function.comp_apply]
  congr 1
  exact congrArg g (Quotient.sound hab)
rw [mix_pEnsemble_pure_average (NNReal.toReal ∘ g ∘ KetUpToPhase.mk) hphase_inv
rfl

omit [Nonempty d] in
/-- The mixed convex roof extension of `f : MState d → ℝ≥0` applied to a pure state
/
theorem mixed_convex_roof_of_pure (ψ : Ket d) : mixed_convex_roof f (pure ψ) = f (pure
rw [le_antisymm_iff]
constructor
· apply mixed_convex_roof_le
  use 1; simp only [gt_iff_lt, zero_lt_one, true_and]; use trivial_mEnsemble (pure
  constructor
  · exact trivial_mEnsemble_mix (pure ψ) 0
  · simp only [average_NNReal, Fin.isValue, ← NNReal.coe_le_coe, coe_mk]
    rw [trivial_mEnsemble_average _ (pure ψ) 0]
    rfl
· apply le_mixed_convex_roof
  intro n hnpos e hmix
  replace hpure := mix_mEnsemble_pure_iff_pure.mp hmix
  apply le_of_eq
  simp only [average_NNReal, ← NNReal.coe_inj, coe_mk]
  rw [mix_mEnsemble_pure_average (toReal ∘ f) hmix, Function.comp_apply]

/-- Entanglement of Formation of bipartite systems. It is the convex roof extension
von Neumann entropy of one of the subsystems (here chosen to be the left one, but se

The function `Svn ∘ traceRight ∘ pure` is phase-invariant because `pure` maps phase-
equivalent kets to the
same mixed state, so it descends to `KetUpToPhase` via `pureQ`. -
/
def EoF : MState (d1 × d2) → ℝ≥0 :=
  convex_roof (KetUpToPhase.lift
    (fun ψ ↦ ⟨Svn (pure ψ).traceRight, Svn_nonneg (pure ψ).traceRight⟩)
    (fun ψ φ h ↦ by
      simp only
      congr 1
      congr 1
      exact congrArg MState.traceRight ((MState.PhaseEquiv_iff_pure_eq ψ φ).mp h)))

```

```

/-
The partial trace of the maximally entangled state is the maximally mixed s
-/
theorem traceRight_pure_MES (d : Type*) [Fintype d] [DecidableEq d] [Nonempty d]
  (MState.pure (Ket.MES d)).traceRight = MState.uniform := by
  -- By definition of partial trace, we sum over the second system.
  have h_partial_trace :  $\forall (i j : d), \sum k : d, (\text{Ket.MES } d).\text{vec } (i, k) * (\text{state } k)$ 
    unfold Ket.MES
    intro i j
    simp only [one_div, Pi.star_apply, RCLike.star_def, ite_mul, zero_mul,
      Finset.mem_univ,  $\downarrow$ reduceIte, Complex.ofReal_inv]
    split
    · subst i
      simp only [map_inv₀, Complex.conj_ofReal]
      ring_nf; norm_cast; norm_num;
    · grind
  unfold MState.pure MState.traceRight MState.uniform
  ext i j
  convert h_partial_trace i j
  simp_all only [Pi.star_apply, RCLike.star_def, one_div, Complex.ofReal_inv,
    Complex.ofReal_natCast, mul_ite, mul_one, mul_zero, HermitianMat.mat_apply,
    coe_ofClassical, ProbDistribution.uniform_def, Finset.card_univ]
  unfold HermitianMat.diagonal
  simp_all only [map_inv₀, map_natCast]
  rfl

/-
The von Neumann entropy of a state is equal to the trace of  $-\rho \log \rho$  (technically)
-/
theorem S_vn_eq_trace_cfc {d : Type*} [Fintype d] [DecidableEq d] (ρ : MState.d)
  S_vn ρ = (HermitianMat.cfc ρ.M Real.negMulLog).trace := by
  -- By definition of von Neumann entropy, we have  $S_{vn} \rho = \text{Finset.sum } \text{Finset.univ } (\text{fun } x \mapsto \text{Real.negMulLog } (\rho.M.\text{eigenvalues } x))$ 
  have h_def :  $S_{vn} \rho = \text{Finset.sum } \text{Finset.univ } (\text{fun } x \mapsto \text{Real.negMulLog } (\rho.M.\text{eigenvalues } x))$ 
    rfl
  -- By definition of trace, the trace of  $\text{cfc } \rho.M \text{ Real.negMulLog}$  is the sum of the eigenvalues of  $\text{cfc } \rho.M \text{ Real.negMulLog}$ 
  have h_trace :  $(\rho.M.\text{cfc } \text{Real.negMulLog}).\text{trace} = \sum x, (\rho.M.\text{cfc } \text{Real.negMulLog}).H.\text{eigenvalues } x := by
    exact (HermitianMat.sum_eigenvalues_eq_trace _).symm
  obtain (e, he) :  $\exists e : d \approx d, (\rho.M.\text{cfc } \text{Real.negMulLog}).H.\text{eigenvalues } e = \text{Real.negMulLog } \circ \rho.M.H.\text{eigenvalues } \circ e := by
    exact Matrix.IsHermitian.cfc_eigenvalues _ _
  rw [h_def, h_trace, he]
  simp only [Function.comp_apply]
  conv_lhs => rw [← Equiv.sum_comp e ]$$ 
```


The von Neumann entropy of a classical state (diagonal in the basis) is equal to the
 -/

```
theorem Svn_ofClassical {d : Type*} [Fintype d] [DecidableEq d] (dist : ProbDistribution d) :
  Svn (MState.ofClassical dist) = Hs dist := by
  -- Let's unfold the definition of `Svn` using `Svn_eq_trace_cfc`.
  have h_def : Svn (MState.ofClassical dist) = (HermitianMat.cfc (MState.ofClassical dist)
    exact Svn_eq_trace_cfc (ofClassical dist);
  convert h_def using 1;
  -- By definition of $MState.ofClassical$, we know that $(MState.ofClassical dist).M = HermitianMat.diagonal C (fun x => dist x)$.
  have h_diag : (MState.ofClassical dist).M = HermitianMat.diagonal C (fun x => dist x) :=
    exact rfl;
  rw [ h_diag, HermitianMat.cfc_diagonal, HermitianMat.trace_diagonal ] ; aesop
```

/-- The entanglement of formation of the maximally entangled state with on-site dimension d is $\log(d)$. -/

```
theorem EoF_of_MES : EoF (pure <| Ket.MES d) = Real.log (Finset.card Finset.univ (α d))
  simp only [EoF, convex_roof_of_pure, Finset.card_univ]
  simp only [KetUpToPhase.lift_mk]
  -- The von Neumann entropy of the maximally mixed state is  $\log(d)$ .
  have h_von_neumann : Svn (MState.uniform : MState d) = Real.log (Fintype.card d) :
    rw [MState.uniform, Svn_ofClassical ProbDistribution.uniform, Hs_uniform, Finset.trace]
  simp only [NNReal.coe_mk]
  rw [traceRight_pure_MES]
  exact h_von_neumann
```

1.437 QuantumInfo/Finite/Entropy.lean

```
/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
```

```
import QuantumInfo.Finite.Entropy.VonNeumann
import QuantumInfo.Finite.Entropy.SSA
import QuantumInfo.Finite.Entropy.Relative
import QuantumInfo.Finite.Entropy.DPI
```

1.438 QuantumInfo/Finite/Entropy/DPI.lean

```
/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
```

```

-/
import QuantumInfo.Finite.Entropy.Relative
import QuantumInfo.ForMathlib.HermitianMat.Sqrt

noncomputable section

variable {d d1 d2 d3 : Type*}
variable [Fintype d] [Fintype d1] [Fintype d2] [Fintype d3]
variable [DecidableEq d] [DecidableEq d1] [DecidableEq d2] [DecidableEq d3]
variable {dA dB dC dA1 dA2 : Type*}
variable [Fintype dA] [Fintype dB] [Fintype dC] [Fintype dA1] [Fintype dA2]
variable [DecidableEq dA] [DecidableEq dB] [DecidableEq dC] [DecidableEq dA1] [DecidableEq dA2]
variable { $\square$  : Type*} [RCLike  $\square$ ]
variable { $\alpha$  :  $\mathbb{R}$ } { $\rho$   $\sigma$  : MState d}

open scoped InnerProductSpace RealInnerProductSpace HermitianMat

/-!
# DPI (Data Processing Inequality)

The Data Processing Inequality (DPI) for the sandwiched Rényi relative entropy as a consequence, the quantum relative entropy.
-/

open scoped Matrix ComplexOrder
open BigOperators

/-- The weighted norm  $\|X\|_{p, \sigma}$  defined in the paper. -
/
noncomputable def weighted_norm (p :  $\mathbb{R}$ ) ( $\sigma$  : MState d) (X : Matrix d d  $\mathbb{C}$ ) :  $\mathbb{R}$  :=
  let  $\sigma_{\text{pow}}$  : HermitianMat d  $\mathbb{C}$  :=  $\sigma.M.cfc$  (fun x => x ^ (1 / (2 * p)))
  schattenNorm ( $\sigma_{\text{pow}}$ .mat * X *  $\sigma_{\text{pow}}$ .mat) p

/-- The spectral norm (operator norm) of a matrix. -/
noncomputable def spectralNorm_mat (A : Matrix d d  $\mathbb{C}$ ) :  $\mathbb{R}$  :=
  if h : Finset.univ.Nonempty then
    let A_dag_A : HermitianMat d  $\mathbb{C}$  := (A.conjTranspose * A, by
      have h := Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose
      rwa [Matrix.conjTranspose_conjTranspose] at h)
    Real.sqrt ((Finset.univ.image A_dag_A.H.eigenvalues).max' (Finset.Nonempty))
  else 0

/-- The weighted norm for  $p = \infty$ . -/
noncomputable def weighted_norm_infty ( $\sigma$  : MState d) (X : Matrix d d  $\mathbb{C}$ ) :  $\mathbb{R}$  :=
  spectralNorm_mat X

```

```

/-- The map  $\Gamma_\sigma(X) = \sigma^{\{1/2\}} X \sigma^{\{1/2\}}$ . -/
noncomputable def Gamma (σ : MState d) (X : Matrix d d ℂ) : Matrix d d ℂ :=
  let σ_half : HermitianMat d ℂ := σ.M.cfc (fun x => x ^ (1/2 : ℝ))
  σ_half.mat * X * σ_half.mat

/-- The inverse map  $\Gamma_{\sigma^{-1}}(X) = \sigma^{\{-1/2\}} X \sigma^{\{-1/2\}}$ . -/
noncomputable def Gamma_inv (σ : MState d) (X : Matrix d d ℂ) : Matrix d d ℂ :=
  let σ_inv_half : HermitianMat d ℂ := σ.M.cfc (fun x => x ^ (-
1/2 : ℝ))
  σ_inv_half.mat * X * σ_inv_half.mat

/-- The operator  $T = \Gamma_{\{\Phi(\sigma)\}^{-1}} \circ \Phi \circ \Gamma_\sigma$ . -/
noncomputable def T_op (Φ : CPTPMap d d₂) (σ : MState d) (X : Matrix d d ℂ) : Matrix
  Gamma_inv (Φ σ) (Φ.map (Gamma σ X))

/-- The induced norm of a map  $\Psi: M_d \rightarrow M_{d_2}$  with respect to weighted norms. -
/
noncomputable def induced_norm (p : ℝ) (σ : MState d) (Φ : CPTPMap d d₂) (Ψ : Matrix
  sSup { weighted_norm p (Φ σ) (Ψ X) / weighted_norm p σ X | (X : Matrix d d ℂ) }

/-
The induced norm of a map  $\Psi$  with respect to the weighted infinity norm.
-/
noncomputable def induced_norm_infty_map (σ : MState d) (Φ : CPTPMap d d₂) (Ψ : Matrix
  sSup { weighted_norm_infty (Φ σ) (Ψ X) / weighted_norm_infty σ X | (X : Matrix d d ℂ) }

/-
The operator  $T = \Gamma_{\{\Phi(\sigma)\}^{-1}} \circ \Phi \circ \Gamma_\sigma$  as a linear map.
-/
noncomputable def T_map (σ : MState d) (Φ : CPTPMap d d₂) : MatrixMap d d₂ ℂ :=
  { toFun := fun X => T_op Φ σ X,
    map_add' := fun X Y => by
      unfold T_op Gamma Gamma_inv
      simp [Matrix.mul_add, Matrix.add_mul]
    map_smul' := fun c X => by
      unfold T_op
      simp
      unfold Gamma Gamma_inv
      simp [mul_assoc]
  }

/-
Gamma_map is the conjugation by the square root of sigma.
-/
noncomputable def Gamma_map (σ : MState d) : MatrixMap d d ℂ :=
  MatrixMap.conj (σ.M.cfc (fun x => x ^ (1/2 : ℝ))).mat

```

```

lemma Gamma_map_eq (σ : MState d) (X : Matrix d d ℂ) :
  Gamma_map σ X = Gamma σ X := by
  ext; simp [ Gamma_map, Gamma ];
  apply_rules [ IsSelfAdjoint.cfc ]

/-
Gamma_map is completely positive.
-/
lemma Gamma_map_CP (σ : MState d) : (Gamma_map σ).IsCompletelyPositive := b
  have := @MatrixMap.conj_isCompletelyPositive;
  exact this _

/-
Gamma_inv_map is the conjugation by the inverse square root of sigma.
-/
noncomputable def Gamma_inv_map (σ : MState d) : MatrixMap d d ℂ :=
  MatrixMap.conj (σ.M.cfc (fun x => x ^ (-1/2 : ℝ))).mat

lemma Gamma_inv_map_eq (σ : MState d) (X : Matrix d d ℂ) :
  Gamma_inv_map σ X = Gamma_inv σ X := by
  simp [Gamma_inv_map, Gamma_inv];
  congr;
  apply IsSelfAdjoint.cfc

/-
The inverse square root of sigma.
-/
noncomputable def sigma_inv_sqrt (σ : MState d) : Matrix d d ℂ :=
  (σ.M.cfc (fun x => x ^ (-1/2 : ℝ))).mat

/-
Gamma_inv_map is conjugation by sigma_inv_sqrt.
-/
lemma Gamma_inv_map_eq_conj (σ : MState d) :
  Gamma_inv_map σ = MatrixMap.conj (sigma_inv_sqrt σ) := by
  exact rfl

/-
Gamma_inv_map is completely positive.
-/
lemma Gamma_inv_map_CP (σ : MState d) : (Gamma_inv_map σ).IsCompletelyPosit
  convert MatrixMap.conj_isCompletelyPositive _;
  · infer_instance;
  · infer_instance

```

```

/-
T_map is the composition of Gamma_inv_map, Phi, and Gamma_map.
-/
lemma T_map_eq_comp (σ : MState d) (Φ : CPTPMap d d₂) :
  T_map σ Φ = (Gamma_inv_map (Φ σ)).comp (Φ.map.comp (Gamma_map σ)) := by
  ext;
  unfold T_map;
  simp [T_op]
  congr! 1;
  · exact funext fun x => Gamma_inv_map_eq ( Φ σ ) x ▸ rfl;
  · rw [ Gamma_map_eq ]

/-
T_map is completely positive.
-/
lemma T_is_CP (σ : MState d) (Φ : CPTPMap d d₂) :
  (T_map σ Φ).IsCompletelyPositive := by
  rw [ T_map_eq_comp ];
  apply MatrixMap.IsCompletelyPositive.comp;
  · apply MatrixMap.IsCompletelyPositive.comp;
    · exact Gamma_map_CP σ;
    · exact Φ.cp;
  · exact Gamma_inv_map_CP (Φ σ)

/-
T_map is positive.
-/
lemma T_is_positive (σ : MState d) (Φ : CPTPMap d d₂) :
  (T_map σ Φ).IsPositive := by
  exact T_is_CP σ Φ |> fun h => h.IsPositive

/-
The weighted 1-norm of X is the trace norm of Gamma(X).
-/
lemma weighted_norm_one_eq_trace_norm_Gamma (σ : MState d) (X : Matrix d d ℂ) :
  weighted_norm 1 σ X = schattenNorm (Gamma σ X) 1 := by
  unfold weighted_norm Gamma;
  norm_num

/-
The induced norm of a super-operator between weighted Schatten spaces.
-/
noncomputable def general_induced_norm
  (p q : ℝ) (σ : MState d) (σ' : MState d₂)
  (Ψ : MatrixMap d d₂ ℂ) : ℝ :=
  sSup { weighted_norm q σ' (Ψ X) / weighted_norm p σ X | (X : Matrix d d ℂ) ( _ : we

```

```

/-
Multiplication property for HermitianMat functional calculus.
-/
lemma HermitianMat.cfc_mul {d : Type*} [Fintype d] [DecidableEq d]
  (A : HermitianMat d ℂ) (f g : ℝ → ℝ) :
  (A.cfc f).mat * (A.cfc g).mat = (A.cfc (fun x => f x * g x)).mat := by
  symm
  apply mat_cfc_mul

/-
Gamma of identity is sigma.
-/
lemma Gamma_one (σ : MState d) : Gamma σ 1 = σ.M.mat := by
  have h_gamma_one : (σ.M.cfc (fun x => x^(1/2 : ℝ))).mat * (σ.M.cfc (fun x =>
    symm
    exact HermitianMat.mat_cfc_mul σ.M ( fun x => x ^ ( 1 / 2 : ℝ ) ) ( fun
  convert h_gamma_one using 1;
  · unfold Gamma; aesop;
  · norm_num [ ← Real.sqrt_eq_rpow, Real.sqrt_mul_self ( show 0 ≤ _ from _
    have h_gamma_one : ∀ x ∈ spectrum ℝ σ.m, Real.sqrt x * Real.sqrt x = x
      intro x hx; rw [ Real.mul_self_sqrt ] ; exact (by
        rw [ spectrum.mem_iff ] at hx;
        exact Matrix.PosSemidef.pos_of_mem_spectrum σ.psd x hx);
    rw [ cfc ];
    split_ifs <;> simp_all
    · convert rfl;
      convert cfcHom_id _;
      ext x; aesop;
    · exact False.elim ( <IsSelfAdjoint σ.m → ¬ContinuousOn ( fun x => Real

/-
Gamma inverse of sigma is identity.
-/
lemma Gamma_inv_self (σ : MState d) (hσ : σ.m.PosDef) :
  Gamma_inv σ σ.M.mat = 1 := by
  -- We use `HermitianMat.cfc_mul` and the fact that  $x^{-1/2} * x * x^{-1/2} = 1$  for  $x > 0$ .
  have h_gamma_inv_sigma : (σ.M.cfc (fun x => x ^ (-1/2 : ℝ))).mat * (σ.M.cfc
  1/2 : ℝ))).mat = (σ.M.cfc (fun x => x ^ (-1/2 : ℝ) * x * x ^ (-
  1/2 : ℝ))).mat := by
  have h_gamma_inv_sigma : (σ.M.cfc (fun x => x ^ (-1/2 : ℝ))).mat * (σ.M.cfc
  1/2 : ℝ))).mat = (σ.M.cfc (fun x => x ^ (-1/2 : ℝ) * x * x ^ (-
  1/2 : ℝ))).mat := by
  have h_gamma_inv_sigma : ∀ (f g h : ℝ → ℝ), ContinuousOn f (spectrum
    intro f g h hf hg hh

```

```

    have h_gamma_inv_sigma : (σ.M.cfc f).mat * (σ.M.cfc g).mat = (σ.M.cfc (fun x => x
      symm
      convert HermitianMat.mat_cfc_mul σ.M f g using 1;
      rw [ h_gamma_inv_sigma, ← HermitianMat.mat_cfc_mul ];
      congr! 2
    have h : ∀ x ∈ spectrum ℝ σ.M.mat, x ≠ 0 := by
      norm_num
      intro x hx h_zero
      have h_eigenvalue : ∃ v : d → ℂ, v ≠ 0 ∧ σ.m.mulVec v = x • v := by
        simp_all [ spectrum.mem_iff ]
        contrapose! hx;
        exact Matrix.PosDef.isUnit hσ;
      obtain ( v, hv_ne_zero, hv_eigenvalue ) := h_eigenvalue
      rw [Matrix.posDef_iff_dotProduct_mulVec] at hσ
      have := hσ.2 hv_ne_zero
      simp [hv_eigenvalue, h_zero] at this
      apply h_gamma_inv_sigma
      · fun_prop
      · fun_prop
      · fun_prop
      convert h_gamma_inv_sigma using 1;
      ext i j ; simp [ Matrix.mul_apply ]
      -- Since  $x^{-1/2} * x * x^{-1/2} = 1$  for  $x > 0$ , we have  $(\sigma.M.cfc (fun x => x
      1/2 : ℝ))) .mat * (\sigma.M.mat) * (\sigma.M.cfc (fun x => x ^ (-1/2 : ℝ))) .mat = (\sigma.M.cfc (fun
      have h_gamma_inv_sigma_simplified : (\sigma.M.cfc (fun x => x ^ (-
      1/2 : ℝ))) .mat * (\sigma.M.mat) * (\sigma.M.cfc (fun x => x ^ (-1/2 : ℝ))) .mat = (\sigma.M.cfc (fun
      convert h_gamma_inv_sigma using 1;
      congr! 1;
      -- Since  $x^{-1/2} * x * x^{-1/2} = 1$  for all  $x > 0$ , the functions are equal.
      have h_eq : ∀ x : ℝ, 0 < x → x ^ (-1 / 2 : ℝ) * x * x ^ (-
      1 / 2 : ℝ) = 1 := by
        intro x hx
        ring_nf
        norm_num [ hx.ne' ];
        rw [ ← Real.rpow_natCast, ← Real.rpow_mul hx.le ] ; norm_num [ hx.ne' ];
        rw [ Real.rpow_neg_one, inv_mul_cancel₀ hx.ne' ];
        exact Eq.symm (HermitianMat.cfc_congr_of_posDef hσ h_eq);
      convert h_gamma_inv_sigma_simplified using 1;
      ext i j
      simp

/-
The matrix of the output state is the map applied to the input matrix.
-/
lemma CPTPMap_apply_MState_M (Φ : CPTPMap d d₂) (σ : MState d) :
  (Φ σ).M.mat = Φ.map σ.M.mat := by$ 
```

```

exact rfl

/-
The map T is unital.
-/
theorem T_map_unital (σ : MState d) (Φ : CPTPMap d d₂) (hΦσ : (Φ σ).m.PosDef) :
  (T_map σ Φ) 1 = 1 := by
  dsimp [T_map, T_op]
  rw [Gamma_one σ]
  rw [← CPTPMap_apply_MState_M]
  rw [Gamma_inv_self (Φ σ) hΦσ]

/-
The map T is completely positive.
-/
theorem T_map_is_CP_proof (σ : MState d) (Φ : CPTPMap d d₂) :
  (T_map σ Φ).IsCompletelyPositive := by
  apply T_is_CP

/-
Gamma composed with Gamma inverse is identity.
-/
lemma Gamma_Gamma_inv (σ : MState d) (hσ : σ.m.PosDef) (X : Matrix d d ℂ) :
  Gamma σ (Gamma_inv σ X) = X := by
  -- By definition of Gamma and Gamma_inv, we can simplify the expression.
  have h_simp : (σ.M.cfc (fun x => x ^ (1 / 2 : ℝ))).mat * (σ.M.cfc (fun x
1 / 2 : ℝ))).mat = 1 := by
  symm
  convert HermitianMat.mat_cfc_mul _ _ _ using 1;
  · have h_gamma_gamma_inv : ∀ x ∈ spectrum ℝ σ.M.mat, x ^ (1 / 2 : ℝ) *
1 / 2 : ℝ) = 1 := by
    intro x hx
    have hx_pos : 0 < x := by
      have := (Matrix.posDef_iff_dotProduct_mulVec.mp hσ).2;
    obtain ⟨v, hv⟩ : ∃ v : d → ℂ, v ≠ 0 ∧ σ.m.mulVec v = x • v := by
      rw [spectrum.mem_iff] at hx;
    simp_all [Matrix.isUnit_iff_isUnit_det];
    obtain ⟨v, hv⟩ := Matrix.exists_mulVec_eq_zero_iff.mpr hx;
    simp_all [sub_eq_iff_eq_add, Matrix.sub_mulVec];
    exact ⟨v, hv.1, hv.2.symm.trans (by ext i; erw [Matrix.mulVec
specialize this hv.1;
simp_all [dotProduct];
simp_all [mul_assoc, mul_comm];
simp_all [mul_left_comm (v _), Complex.mul_conj, Complex.normS
norm_cast at this;
exact lt_of_not_ge fun hx' => this.not_ge <| Finset.sum_nonpos fu

```



```

      rw [ ← Real.rpow_add hx_pos ] ; norm_num;
      rw [ HermitianMat.cfc_congr (g := fun x ↦ 1) ]
    · rw [ HermitianMat.cfc_const ]
      norm_num
    · exact fun x hx => h_gamma_gamma_inv x hx;
  unfold Gamma Gamma_inv; simp_all [ ← mul_assoc ] ;
  simp_all [ mul_assoc, mul_eq_one_comm.mp h_simp ]

/-
If a Hermitian matrix is bounded by M*I, then all its eigenvalues are at most M.
-/
theorem HermitianMat.le_smul_one_imp_eigenvalues_le (A : HermitianMat d ℂ) (M : ℝ)
  (h : A ≤ M • (1 : HermitianMat d ℂ)) (i : d) :
  A.H.eigenvalues i ≤ M := by
  -- By definition of eigenvalues, for any unit vector $v$, we have $\langle v, A v \rangle \le M$.
  have h_eigenvalue_le_M_step : ∀ (v : EuclideanSpace ℂ d), ‖v‖ = 1 → ⟨v, A.vmul v⟩ ≤ M := by
    intro v hv
    have h_inner : ⟨v, A.vmul v⟩ ≤ ⟨v, (M • 1).vmul v⟩ := by
      have h_inner : ⟨v, A.vmul v⟩ ≤ ⟨v, ((M • 1 : Matrix d d ℂ) - A.mat).vmul v⟩ ≥ 0
      have h_inner_le_M : ∀ (X : HermitianMat d ℂ), X ≥ 0 → ∀ (v : EuclideanSpace ℂ d), ‖v‖ = 1 → ⟨v, X.vmul v⟩ ≤ 0
      intro X hX v
      rw [ge_iff_le, HermitianMat.zero_le_iff, Matrix.posSemidef_iff_dotProduct]
      have := hX.2 v
      simp [ Matrix.mulVec, dotProduct ] at *
      convert this using 1;
      refine Finset.sum_congr rfl fun _ _ => ?_
      sorry
    convert h_inner_le_M { _, _ } v;
    all_goals norm_num [ HermitianMat.le_iff ] at *;
    · convert h.1;
    · exact h;
  simp_all [ Matrix.sub_mulVec ]
  simp_all [ EuclideanSpace.norm_eq ];
  convert h_inner using 1;
  simp [ Matrix.mulVec, dotProduct, inner ];
  simp [ Matrix.one_apply, mul_assoc ];
  simp [ ← Finset.mul_sum ];
  simp [ Complex.mul_conj, Complex.normSq_eq_norm_sq ];
  norm_cast ; aesop;
  have := A.H.eigenvectorBasis.orthonormal;
  have := this.1 i;
  have := h_eigenvalue_le_M_step ( A.H.eigenvectorBasis i ) this;
  rw [ show A.mat.mulVec _ = ( Matrix.IsHermitian.eigenvalues A.H i : ℂ ) • ( Matrix.mulVec _ ( A.H.eigenvectorBasis i ) ) ]
  · simp_all
  · convert A.H.mulVec_eigenvectorBasis i using 1

```

```

set_option maxHeartbeats 400000 in
open MatrixOrder in
/-
If all eigenvalues of a Hermitian matrix are at most M, then the matrix is
-/
theorem HermitianMat.eigenvalues_le_imp_le_smul_one (A : HermitianMat d ℂ)
  (h : ∀ i, A.H.eigenvalues i ≤ M) :
  A ≤ M • (1 : HermitianMat d ℂ) := by
  have := A.H.spectral_theorem.symm;
  -- Since $A$ is Hermitian, we can write it as $A = UDU^*$ where $U$ is un
  have h_decomp : ∃ U : Matrix d d ℂ, U * star U = 1 ∧ ∃ D : HermitianMat d ℂ
    use A.H.eigenvectorUnitary
    constructor; · simp
    use HermitianMat.diagonal ℂ A.H.eigenvalues
    constructor
    · exact this.symm
    · simp [HermitianMat.diagonal, ← HermitianMat.mat_apply] using h
  obtain (U, hU_unitary, D, hA_eq, hD_le) := h_decomp;
  have hA_le : U * D * star U ≤ U * (M • 1) * star U := by
    have hD_le : D ≤ M • (1 : HermitianMat d ℂ) := by
      sorry
    have := HermitianMat.conj_mono (M := U) hD_le
    simp only [conj, AddMonoidHom.coe_mk, ZeroHom.coe_mk] at this
    replace this := Subtype.coe_le_coe.mpr this
    simp only [mat_smul, mat_one] at this
    exact this
  rw [← hA_eq] at hA_le
  simp only [Algebra.mul_smul_comm, mul_one, Algebra.smul_mul_assoc, hU_uni
  exact hA_le

/-- The block matrix  $\begin{bmatrix} 1 & X \\ X^\dagger & X^\dagger X \end{bmatrix}$  is positive semidefinite. -
/
theorem block_matrix_posSemidef {m n k : Type*} [Fintype m] [Fintype n] [Fintype k]
  (X : Matrix k n ℂ) (Y : Matrix k m ℂ) :
  (Matrix.fromBlocks (YH * Y) (YH * X) (XH * Y) (XH * X)).PosSemidef := by
  set Z : Matrix (m ⊕ n) (m ⊕ n) ℂ := Matrix.fromBlocks (YH * Y) (YH * X) (XH * Y) (XH * X)
  have hZ : Z = Matrix.fromBlocks (m := k) YH 0 XH 0 * Matrix.fromBlocks Y 0 X 0
    simp +zetaDelta [Matrix.fromBlocks_multiply]
  have hZ : Z = (Matrix.fromBlocks (o := k) Y X 0 0)H * Matrix.fromBlocks Y 0 X 0
    rw [hZ]
    ext i j ; simp [Matrix.mul_apply];
    cases i <=> cases j <=> simp [Matrix.fromBlocks];
  rw [hZ]
  exact Matrix.posSemidef_conjTranspose_mul_self _

theorem block_matrix_one_posSemidef {m n : Type*} [Fintype m] [Fintype n] [Fintype n]

```

```

    (X : Matrix m n ℂ) :
      (Matrix.fromBlocks 1 X XH (XH * X)).PosSemidef := by
        simp using block_matrix_posSemidef X (1 : Matrix m m ℂ)

/-- The Data Processing Inequality for the Sandwiched Renyi relative entropy.
Proved in `https://arxiv.org/pdf/1306.5920`. Seems kind of involved. -
/
theorem sandwichedRenyiEntropy_DPI (hα : 1 ≤ α) (ρ σ : MState d) (Φ : CPTMap d d₂)
  D_α(Φ ρ || Φ σ) ≤ D_α(ρ || σ) := by
  --If we want, we can prove this just for 1 < α, and then use continuity (above) to
  -- α → 1.
  sorry

--Helps us track this one sorry for the GQSL
axiom sandwichedRenyiEntropy_DPI_ax : type_of% @sandwichedRenyiEntropy_DPI

/--
info: 'sandwichedRenyiEntropy_DPI_ax' depends on axioms: [propext,
sandwichedRenyiEntropy_DPI_ax,
Classical.choice,
Quot.sound]
-/
#guard_msgs in
#print axioms sandwichedRenyiEntropy_DPI_ax

```

1.439 QuantumInfo/Finite/Entropy/Relative.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.Entropy.VonNeumann

import QuantumInfo.ForMathlib.HermitianMat.Unitary

noncomputable section

variable {d d₁ d₂ d₃ : Type*}
variable [Fintype d] [Fintype d₁] [Fintype d₂] [Fintype d₃]
variable [DecidableEq d] [DecidableEq d₁] [DecidableEq d₂] [DecidableEq d₃]
variable {dA dB dC dA₁ dA₂ : Type*}
variable [Fintype dA] [Fintype dB] [Fintype dC] [Fintype dA₁] [Fintype dA₂]
variable [DecidableEq dA] [DecidableEq dB] [DecidableEq dC] [DecidableEq dA₁] [Decid

```

```

variable {α : Type*} [RCLike α]
variable {α : ℝ} {ρ σ : MState d}

open scoped InnerProductSpace RealInnerProductSpace HermitianMat

/-!
To do relative entropies, we start with the _sandwiched Renyi Relative Entropy.
Then instead of proving many theorems (like DPI, relabelling, additivity, etc),
it for this one quantity, then it follows for other quantities (like the relative entropy).
-/

--Note: without the assumption `h`, we could still get nonnegativity, just
private theorem sandwiched_trace_pos (h : σ.M.ker ≤ ρ.M.ker) :
  0 < ((ρ.M.conj (σ.M ^ ((1 - α)/(2 * α))) ).mat) ^ α).trace := by
  apply HermitianMat.trace_pos
  apply HermitianMat.rpow_pos
  apply HermitianMat.conj_pos ρ.pos
  grw [← h]
  exact HermitianMat.ker_rpow_le_of_nonneg σ.nonneg

--TODO: We don't actually use this, and it's not clear that it's useful (since
-- specialization); remove?
omit [DecidableEq d] in
/--
Weighted Jensen inequality: for weights  $w_j \geq 0$  with  $\sum w_j = 1$ , values  $b_j$ 
and  $q \geq 1$ :  $(\sum_j w_j * b_j)^q \leq \sum_j w_j * b_j^q$ .

This is the special case of `Real.rpow_arith_mean_le_arith_mean_rpow` applied to
`Finset.univ`
-/
lemma weighted_jensen_rpow (b w : d → ℝ) (q : ℝ)
  (hb : ∀ j, 0 ≤ b j) (hw : ∀ j, 0 ≤ w j) (hsum : ∑ j, w j = 1) (hq : 1 ≤ q) :
  (∑ j, w j * b j) ^ q ≤ ∑ j, w j * b j ^ q :=
  Real.rpow_arith_mean_le_arith_mean_rpow Finset.univ _ _ (fun i _ => hw i)

omit [DecidableEq d] in
/--
Doubly stochastic Hölder inequality: for nonneg a, b, doubly stochastic w,
and conjugate p, q > 1:
 $\sum_{ij} a_i * b_j * w_{ij} \leq (\sum a_i^p)^{1/p} * (\sum b_j^q)^{1/q}$ .
-/
lemma doubly_stochastic_holder (a b : d → ℝ) (w : d → d → ℝ)
  (ha : ∀ i, 0 ≤ a i) (hb : ∀ j, 0 ≤ b j)
  (hw : ∀ i j, 0 ≤ w i j)
  (hrow : ∀ i, ∑ j, w i j = 1) (hcol : ∀ j, ∑ i, w i j = 1)
  (p q : ℝ) (hp : 1 < p) (hpq : 1/p + 1/q = 1) :

```

```

    
$$\sum_i, \sum_j, a_i * b_j * w_{ij} \leq (\sum_i, a_i^p)^{1/p} * (\sum_j, b_j^q)^{1/q} :=$$

    by_contra h_contra
    contrapose! h_contra with h_contra
    simp_all [ ← Finset.mul_sum, mul_assoc ]
    have h0q : 0 < q :=
      lt_of_le_of_ne ( le_of_not_gt fun hq => by { rw [ inv_eq_one_div, div_eq_mul_inv ] } )
      -- Apply Hölder's inequality with the sequences $a_i$ and $g_i = \sum_j w_{ij} b_j$
    have h_holder : (  $\sum_i, a_i * (\sum_j, w_{ij} * b_j)$  )  $\leq (\sum_i, a_i^p)^{1/p} * (\sum_i,$ 
      have := @Real.inner_le_Lp_mul_Lq
      simp_all
      convert this Finset.univ a ( fun i =>  $\sum_j, w_{ij} * b_j$  ) ( show p.HolderConjugate
        · simp only [ha, abs_of_nonneg, mul_eq_mul_left_iff]
          left
          congr!
          symm
          exact abs_of_nonneg (Finset.sum_nonneg fun _ => mul_nonneg (hw _ _) (hb _))
        · constructor <=> linarith
      -- By Fubini's theorem, we can interchange the order of summation.
    have h_fubini :  $\sum_i, (\sum_j, w_{ij} * b_j)^q \leq \sum_j, b_j^q :=$  by
      -- Apply Jensen's inequality to the convex function $x^q$ with weights $w_{ij}$.
    have h_jensen :  $\forall i, (\sum_j, w_{ij} * b_j)^q \leq \sum_j, w_{ij} * b_j^q :=$  by
      intro i
      have h_jensen : ConvexOn ℝ (Set.Ici 0) (fun x : ℝ => x ^ q) := by
        apply convex0n_rpow
        nlinarith [ inv_pos.2 ( zero_lt_one.trans hp ), inv_pos.2 h0q, mul_inv_cancel
          convert h_jensen.map_sum_le _ _ _ <=> aesop
          refine' le_trans ( Finset.sum_le_sum fun i _ => h_jensen i ) _;
          rw [ Finset.sum_comm ]
          simp [ ← Finset.sum_mul, hcol ]
        simp_all only [mul_comm];
        simp using h_holder.trans ( mul_le_mul_of_nonneg_left ( Real.rpow_le_rpow ( Finset
--PULLOUT
/--
Hermitian trace Hölder inequality: for PSD A, B and conjugate exponents p, q > 1,
 $\|A\|_p \leq \text{Tr}[A^p]^{1/p} * \text{Tr}[B^q]^{1/q}$ .
-/
lemma HermitianMat.inner_le_trace_rpow_mul
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B)
  (p q : ℝ) (hp : 1 < p) (hpq : 1/p + 1/q = 1) :
   $\|A\|_p, \|B\|_q \leq (A^p).\text{trace}^{1/p} * (B^q).\text{trace}^{1/q} :=$  by
  by_cases hq : q > 1;
  · -- Apply the doubly_stochastic_holder lemma with the weights $w_{ij} = |C_{ij}|$
    rw [trace_rpow_eq_sum, trace_rpow_eq_sum, inner_eq_doubly_stochastic_sum]
    refine doubly_stochastic_holder
      A.H.eigenvalues B.H.eigenvalues

```

```

      (fun i j ↦ ||(A.H.eigenvectorUnitary.val.conjTranspose * B.H.eigenvect
      (fun i ↦ by simpa using hA.eigenvalues_nonneg i)
      (fun i ↦ by simpa using hB.eigenvalues_nonneg i)
      (by bound) ?_ ?_ p q hp hpq
    · apply Matrix.unitary_row_sum_norm_sq (A.H.eigenvectorUnitary.val.conj
      simp [mul_assoc]
      simp [← mul_assoc, Matrix.IsHermitian.eigenvectorUnitary]
    · apply Matrix.unitary_col_sum_norm_sq (A.H.eigenvectorUnitary.val.conj
      simp [mul_assoc]
      simp [← mul_assoc, Matrix.IsHermitian.eigenvectorUnitary]
    · rcases eq_or_ne q 0 with _ | _
      · grind only [cases 0r]
      · field_simp at hpq
      nlinarith

--PULLOUT
lemma MState.rpow_le_one' {r : ℝ} (hσ : 0 < r) : σ.M ^ r ≤ 1 := by
  rw [HermitianMat.le_iff]
  have h1 : 1 - σ.M ^ r = σ.M.cfc (fun x => 1 - x ^ r) := by
    rw [HermitianMat.rpow_eq_cfc]
    have : σ.M.cfc (fun _ => (1:ℝ)) = 1 := by ext1; simp
    rw [← this, ← HermitianMat.cfc_sub_apply]
  rw [h1, ← HermitianMat.zero_le_iff, HermitianMat.cfc_nonneg_iff]
  intro i
  have hge : 0 ≤ σ.M.H.eigenvalues i :=
    (HermitianMat.zero_le_iff.mp σ.nonneg).eigenvalues_nonneg i
  have hle : σ.M.H.eigenvalues i ≤ 1 := σ.eigenvalue_le_one i
  linarith [Real.rpow_le_one hge hle hσ.le]

--PULLOUT
/-- If  $A \geq 0$  and  $A \leq 1$ , then each eigenvalue of  $A$  is in  $[0, 1]$ . -
/
lemma HermitianMat.eigenvalues_le_one_of_le_one
  (A : HermitianMat d ℂ) (hA1 : A ≤ 1) (i : d) :
  A.H.eigenvalues i ≤ 1 := by
  by_contra! h
  obtain ⟨v, hv₁, hv₂⟩ : ∃ v : EuclideanSpace ℂ d, ||v|| = 1 ∧ A.mat.mulVec v
  use A.H.eigenvectorBasis i
  exact ⟨A.H.eigenvectorBasis.orthonormal.1 i, A.H.mulVec_eigenvectorBasi
  have h_eigenvalue : star v □v A.mat.mulVec v = (A.H.eigenvalues i) * star
  rw [hv₂, dotProduct_smul, Complex.real_smul]
  have h_unit : star v □v v = 1 := by
    simp only [EuclideanSpace.norm_eq, Real.sqrt_eq_one, dotProduct, Pi.sta
    RCLike.star_def] at hv₁ ⊢
    simp only [sq, Complex.ext_iff, Complex.re_sum, Complex.mul_re, Complex
    Complex.conj_im, Complex.mul_im, neg_mul, sub_neg_eq_add, Complex.im_

```

```

    Complex.one_re, Complex.one_im] at hv₁ ⊢
  simp only [Complex.norm_def, Complex.normSq, MonoidWithZeroHom.coe_mk, ZeroHom.coe_mk,
    mul_comm, add_neg_cancel, Finset.sum_const_zero, and_true] at hv₁ ⊢
  rw [← hv₁]
  refine Finset.sum_congr rfl fun _ => ?_
  rw [Real.mul_self_sqrt (add_nonneg (mul_self_nonneg _) (mul_self_nonneg _))]
open ComplexOrder in
have := (Matrix.posSemidef_iff_dotProduct_mulVec.mp hA1).2 v
simp only [val_eq_coe, mul_one, mat_one, Matrix.sub_mulVec,
  Matrix.one_mulVec, dotProduct_sub, h_eigenvalue, h_unit] at this
norm_cast at this
linarith

--PULLOUT
/-- For positive  $A \leq 1$  and  $p \geq 1$ ,  $\text{Tr}[A^p] \leq \text{Tr}[A]$ . -/
lemma HermitianMat.trace_rpow_le_trace_of_le_one
  (A : HermitianMat d ℂ) (hA : 0 ≤ A) (hA1 : A ≤ 1)
  (p : ℝ) (hp : 1 ≤ p) :
  (A ^ p).trace ≤ A.trace := by
  -- Rewrite both sides using trace_rpow_eq_sum:  $\text{Tr}[A^p] = \sum \lambda_i^p$  and  $\text{Tr}[A] = \sum \lambda_i$ 
  have h_trace_eq_sum : (A ^ p).trace =  $\sum i, (A.H.eigenvalues i) ^ p \wedge A.trace = \sum i, (A.H.eigenvalues i)$ 
  exact { by rw [ HermitianMat.trace_rpow_eq_sum ], by rw [ show A.trace =  $\sum i, (A.H.eigenvalues i)$  ] }
  rw [ h_trace_eq_sum.1, h_trace_eq_sum.2 ];
  apply_rules [ Finset.sum_le_sum ];
  intro i hi; by_cases hi0 : A.H.eigenvalues i = 0 <|> simp_all
  · rw [ Real.zero_rpow ( by positivity ) ];
  · conv_rhs => rw [← (A.H.eigenvalues i).rpow_one]
    apply Real.rpow_le_rpow_of_exponent_ge
    · exact lt_of_le_of_ne' (le_of_not_gt fun hi => hi0 <| by linarith [ show 0 ≤ A.H.eigenvalues i ])
    · exact A.eigenvalues_le_one_of_le_one hA1 i
    · exact hp

private lemma trace_conj_rpow_eq_inner (hα₀ : 0 < α) (hα : α < 1) :
  ((p.M ^ α).conj (σ.M ^ ((1 - α) / (2 * α) * α)).mat).trace =  $\langle p.M ^ α, σ.M ^ (1 - α) \rangle$ 
  convert congr_arg _ ( HermitianMat.inner_eq_trace_rc _ _ ) using 2;
  rotate_left;
  rotate_left;
  rotate_left;
  exact d;
  exact ℂ;
  all_goals try infer_instance;
  exact p ^ α;
  exact σ ^ ( 1 - α );
  rotate_right;
  exact fun x => x.re;

```

```

· unfold HermitianMat.conj;
simp [ Matrix.trace, Matrix.mul_apply, inner]
rw [ show ( 1 - α ) / ( 2 * α ) * α = ( 1 - α ) / 2 by rw [ div_mul_eq_
-- By the properties of the trace, we can rearrange the terms inside th
have h_trace : Matrix.trace ((σ.M ^ ((1 - α) / 2)).mat * (ρ.M ^ α).mat)
  have h_trace : (σ.M ^ ((1 - α) / 2)).mat * (σ.M ^ ((1 - α) / 2)).mat
    have := σ.nonneg;
    rw [ ← HermitianMat.mat_rpow_add this ]
    ring_nf
    linarith;
    rw [ ← h_trace, Matrix.mul_assoc ];
    rw [ ← Matrix.trace_mul_comm ]
    simp [ Matrix.mul_assoc ]
  convert congr_arg Complex.re h_trace using 1;
  simp [ Matrix.trace, Matrix.mul_apply ];
· exact HermitianMat.inner_eq_re_trace _ _

private lemma inner_rpow_le_one (hα₀ : 0 < α) (hα : α < 1) :
  [ρ.M ^ α, σ.M ^ (1 - α)]_ℝ ≤ 1 := by
  convert HermitianMat.inner_le_trace_rpow_mul
    (HermitianMat.rpow_nonneg ρ.nonneg) (HermitianMat.rpow_nonneg σ.nonneg)
    (1 / α) (1 / (1 - α)) _ _ using 1
· rw [← HermitianMat.rpow_mul ρ.nonneg, ← HermitianMat.rpow_mul σ.nonneg]
  simp [hα₀.ne', (sub_pos.mpr hα).ne']
· field_simp
  exact hα
· simp

private theorem sandwiched_trace_of_lt_1 (hα₀ : 0 < α) (hα : α < 1) :
  ((ρ.M.conj (σ.M ^ ((1 - α)/(2 * α))) .mat) ^ α).trace ≤ 1 := by
  have h1α : 0 < 1 - α := sub_pos.mpr hα
  -- Apply trace_rpow_conj_le with p = 2 and q = 2α/(1-
α)
  set t := (1 - α) / (2 * α) with ht_def
  have ht_pos : 0 < t := by positivity
  have hp : (0 : ℝ) < 2 := by positivity
  have hq : (0 : ℝ) < 2 * α / (1 - α) := by positivity
  have hpq : 1 / (2 * α) = 1 / 2 + 1 / (2 * α / (1 - α)) := by
    field_simp
    ring
  calc ((ρ.M.conj (σ.M ^ t).mat) ^ α).trace
    ≤ (((ρ.M ^ (2 / 2)).trace) ^ (1 / 2) *
      ((σ.M ^ t) ^ (2 * α / (1 - α))).trace) ^ (1 / (2 * α / (1 - α)))
      HermitianMat.trace_rpow_conj_le ρ.nonneg (HermitianMat.rpow_nonneg
_ = 1 := by
  -- Simplify: ρ.M ^ (2/2) = ρ.M ^ 1 = ρ.M, Tr[ρ.M] = 1

```



```

-- (σ.M ^ t) ^ (2α/(1-α)) = σ.M ^ (t * 2α/(1-α)) = σ.M ^ 1, Tr[σ.M] = 1
have h1 : (2 : ℝ) / 2 = 1 := by norm_num
have h2 : t * (2 * α / (1 - α)) = 1 := by
  rw [ht_def]; field_simp
rw [h1, HermitianMat.rpow_one,
  ← HermitianMat.rpow_mul σ.nonneg, h2, HermitianMat.rpow_one,
  p.tr, σ.tr]
simp

/-- For PSD A and p ≠ 0, `A^{ -p} * A^p = HermitianMat.supportProj A`. -
/
lemma HermitianMat.rpow_neg_mul_rpow_eq_supportProj
  {A : HermitianMat d ℂ} (hA : 0 ≤ A) {p : ℝ} (hp : p ≠ 0) :
  (A ^ (-p)).mat * (A ^ p).mat = A.supportProj.mat := by
  unfold HermitianMat.supportProj;
  have h_cfc : (A ^ (-p)).mat * (A ^ p).mat = (A.cfc (fun x => x ^ (-
p))) * (A.cfc (fun x => x ^ p)) := by
    congr! 1;
  rw [h_cfc, ← mat_cfc_mul ];
  have h_cfc : ∀ x : ℝ, 0 ≤ x → x ^ (-p) * x ^ p = if x = 0 then 0 else 1 := by
    intro x hx; by_cases hx' : x = 0 <=> simp [ hx', Real.rpow_neg, hx, hp ] ;
  have h_cfc_eq : (A.cfc (fun x => x ^ (-p) * x ^ p)) = (A.cfc (fun x => if x = 0 th
    apply cfc_congr_of_nonneg hA;
    exact fun x hx => h_cfc x hx;
  convert congr_arg ( fun x : HermitianMat d ℂ => x.val ) h_cfc_eq using 1;
  exact congr_arg ( fun x : HermitianMat d ℂ => x.val ) ( supportProj_eq_cfc A )

lemma HermitianMat.supportProj_mul_self (A : HermitianMat d ℂ) :
  A.supportProj.mat * A.mat = A.mat := by
  have h_supportProj_mul_A : ∀ (v : d → ℂ), (A.supportProj.val.mulVec (A.val.mulVec
    intro v
  have h_range : WithLp.toLp 2 (A.val.mulVec v) ∈ LinearMap.range A.val.toEuclidean
    exact { _, rfl }
  have h_supportProj_mul_A : ∀ (v : EuclideanSpace ℂ d), v ∈ LinearMap.range A.val
    intro v hv
  have h_supportProj_mul_A : (A.supportProj.val.toEuclideanLin v) = (Submodule.o
    simp only [val_eq_coe, Submodule.coe_orthogonalProjection_apply]
    simp [supportProj, projector]
    simp only [Submodule.starProjection]
    simp [-Submodule.coe_orthogonalProjection_apply]
  have key : ∀ (f : EuclideanSpace ℂ d →1 [ℂ] EuclideanSpace ℂ d),
    Matrix.toEuclideanLin
      ((LinearMap.toMatrix (EuclideanSpace.basisFun d ℂ).toBasis (EuclideanS
    intro f
  rw [Matrix.toEuclideanLin, Matrix.toLpLin_eq_toLin]
  exact Matrix.toLin_toMatrix _ _ f

```

```

    have hsup : A.support = (Matrix.toEuclideanLin (↑A : Matrix d d ℂ))
      simp [HermitianMat.support, HermitianMat.lin]
    rw [key, LinearMap.comp_apply, Submodule.subtype_apply, hsup]
    rfl
    rw [h_supportProj_mul_A]
    exact Submodule.eq_starProjection_of_mem_of_inner_eq_zero (by simp)
    exact congr(WithLp.ofLp $(h_supportProj_mul_A _ h_range))
    exact Matrix.toLin'.injective (LinearMap.ext fun v => by simp using h_s

lemma HermitianMat.inner_supportProj_self (A : HermitianMat d ℂ) :
  [A, A.supportProj] = A.trace := by
  simp only [trace, IsMaximalSelfAdjoint.RCLike_selfadjMap, Matrix.trace, M
    mat_apply, map_sum, RCLike.re_to_complex]
  simp only [inner, IsMaximalSelfAdjoint.RCLike_selfadjMap, RCLike.re_to_co
  convert congr_arg Complex.re (congr_arg Matrix.trace (HermitianMat.supp
  · simp only [Matrix.trace, Matrix.diag_apply, Matrix.mul_apply, mat_apply
    Complex.mul_re, Finset.sum_sub_distrib, mul_comm];
  exact congrArg₂ _ (Finset.sum_comm) (Finset.sum_comm);
  · simp only [Matrix.trace, Matrix.diag_apply, mat_apply, Complex.re_sum]

lemma HermitianMat.mul_supportProj_of_ker_le {A B : HermitianMat d ℂ}
  (h : LinearMap.ker B.lin.toLinearMap ≤ LinearMap.ker A.lin.toLinearMap) :
  A.mat * B.supportProj.mat = A.mat := by
  -- Since $B.supportProj$ is the projection onto $range B$, we have $B.supp
  have h_supportProj_mul_B : B.supportProj.mat * B.mat = B.mat := by
    exact supportProj_mul_self B
  have h_range_A_subset_range_B : LinearMap.range A.lin.toLinearMap ≤ Linea
  have h_orthogonal_complement : LinearMap.range (B.lin : EuclideanSpace
    have h_orthogonal_complement : ∀ (T : EuclideanSpace ℂ d →₁[ℂ] Euclid
      intro T hT;
      refine' Submodule.eq_of_le_of_finrank_eq _ _;
      · rintro x ⟨y, rfl⟩
        rw [Submodule.mem_orthogonal' (LinearMap.ker T) (T y)]
        intro z hz
        rw [LinearMap.mem_ker] at hz
        simp [← LinearMap.adjoint_inner_right, ← hT, hz]
      · have := LinearMap.finrank_range_add_finrank_ker T;
        have := Submodule.finrank_add_finrank_orthogonal (LinearMap.ker T
          linarith;
        apply h_orthogonal_complement
        simp
  have h_orthogonal_complement_A : LinearMap.range (A.lin : EuclideanSpac
    intro x hx y hy
    simp_all only [LinearMap.mem_range, ContinuousLinearMap.coe_coe, Line
    obtain ⟨z, rfl⟩ := hx;
    have h_orthogonal_complement_A : ∀ (y z : EuclideanSpace ℂ d), [y, A.

```

```

      simp
      rw [ h_orthogonal_complement_A, hy, inner_zero_left ];
      exact h_orthogonal_complement.symm ▸ le_trans h_orthogonal_complement_A ( Submod
-- Since $B.supportProj$ is the projection onto $range B$, and $range A \subseteq range B$,
have h_supportProj_mul_A : ∀ (v : EuclideanSpace ℂ d), B.supportProj.mat.mulVec (A
intro v
obtain ⟨w, hw⟩ : WithLp.toLp 2 (A.mat.mulVec v) ∈ LinearMap.range B.lin.toLinearMap
  exact h_range_A_subset_range_B ( Set.mem_range_self v );
  replace h_supportProj_mul_B := congr(Matrix.mulVec $h_supportProj_mul_B w)
  replace hw := congr(WithLp.ofLp $hw)
  simp only [ContinuousLinearMap.coe_coe] at hw
  simp only [← hw, ← Matrix.mulVec_mulVec] using h_supportProj_mul_B
-- By definition of matrix multiplication, if B.supportProj * A * v = A * v for all v,
have h_matrix_eq : ∀ (M N : Matrix d d ℂ), (∀ v : EuclideanSpace ℂ d, M.mulVec (N
intro M N hMN
ext i j
specialize hMN (WithLp.toLp 2 ( Pi.single j 1 ))
replace hMN := congr_fun hMN i
aesop;
rw [← Matrix.conjTranspose_inj]
simp_all only [Matrix.mulVec_mulVec, Matrix.conjTranspose_mul, conjTranspose_mat,

lemma HermitianMat.inner_supportProj_of_ker_le {A B : HermitianMat d ℂ}
(h : LinearMap.ker B.lin.toLinearMap ≤ LinearMap.ker A.lin.toLinearMap) :
  [A, B.supportProj] = A.trace := by
  rw [inner_def, mul_supportProj_of_ker_le h, trace]

lemma supportProj_inner_density (h : σ.M.ker ≤ ρ.M.ker) :
  [σ.M.supportProj, ρ.M]_ℝ = 1 := by
  rw [HermitianMat.inner_comm, HermitianMat.inner_supportProj_of_ker_le h]
  simp

/-
[ρ.M.conj (σ.M ^ t).mat, σ.M ^ (-2 * t)]_ℝ = 1 for density matrices ρ, σ with ker(σ) ⊆ ker(ρ)
-/
private lemma sandwiched_inner_eq_one (h : σ.M.ker ≤ ρ.M.ker) (t : ℝ) :
  [ρ.M.conj (σ.M ^ t).mat, σ.M ^ (-2 * t)]_ℝ = 1 := by
  rcases eq_or_ne t 0 with rfl | ht
  · simp
  · have h_combine : (σ.M ^ (-2 * t)).mat * (σ.M ^ t).mat = (σ.M ^ (-t)).mat := by
      rw [show (-t : ℝ) = -2 * t + t by ring, HermitianMat.mat_rpow_add σ.nonneg]
      contrapose! ht
      linarith
  have h_support : (σ.M ^ t).mat * (σ.M ^ (-t)).mat = σ.M.supportProj.mat := by
      rw [← neg_ne_zero] at ht

```

```

    simp only [neg_neg] using  $\sigma.M.\text{rpow\_neg\_mul\_rpow\_eq\_supportProj}$   $\sigma.\text{nonneg}$ 
    rw [HermitianMat.inner_def, HermitianMat.conj_apply_mat, HermitianMat.conj_apply_mat]
    rw [Matrix.trace_mul_comm, ← mul_assoc, ← mul_assoc, h_combine]
    rw [Matrix.trace_mul_cycle, h_support, ← HermitianMat.inner_def]
    exact supportProj_inner_density h

private theorem sandwiched_trace_of_gt_1 (h :  $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$ ) (h $\alpha$  :  $\alpha > 0$ 
   $1 \leq ((\rho.M.\text{conj } (\sigma.M ^ ((1 - \alpha)/(2 * \alpha))) .mat) ^ \alpha).\text{trace} :=$  by
  -- Let  $t = (1 - \alpha) / (2 * \alpha)$ ,  $A = \rho.M.\text{conj } (\sigma.M ^ t)$  and  $B = \sigma.M ^ (-2 * t)$ 
  set t :  $\mathbb{R} := (1 - \alpha) / (2 * \alpha)$ 
  set A :=  $\rho.M.\text{conj } (\sigma.M ^ t).\text{mat}$ 
  set B :=  $\sigma.M ^ (-2 * t)$ 
  have h_trace :  $\langle A, B \rangle = 1 :=$  sandwiched_inner_eq_one h t
  have h_inner :  $\langle A, B \rangle \leq (A ^ \alpha).\text{trace} ^ (1 / \alpha) * (B ^ (\alpha / (\alpha - 1))).\text{trace} :=$  by
    apply HermitianMat.inner_le_trace_rpow_mul
    · positivity
    · exact HermitianMat.rpow_nonneg  $\sigma.\text{nonneg}$  --TODO positivity
    · exact h $\alpha$ 
    · rw [div_div_eq_mul_div, div_add_div, div_eq_iff]
      · ring
      · positivity
      · positivity
      · positivity
  have h_trace_B :  $(B ^ (\alpha / (\alpha - 1))).\text{trace} = 1 :=$  by
    have hB :  $B ^ (\alpha / (\alpha - 1)) = \sigma.M ^ (-2 * t * (\alpha / (\alpha - 1))) :=$  by
      rw [HermitianMat.rpow_mul  $\sigma.\text{nonneg}$ ]
    have h :  $-2 * t * (\alpha / (\alpha - 1)) = 1 :=$  by
      rw [mul_div, div_eq_iff]
      · linarith [mul_div_cancel0 (1 -  $\alpha$ ) (by linarith :  $(2 * \alpha) \neq 0$ )]
      · linarith only [h $\alpha$ ]
    rw [hB, h]
    simp
  have h_final :  $1 \leq (A ^ \alpha).\text{trace} ^ (1 / \alpha) :=$  by
    simp only [h_trace, one_div, h_trace_B, Real.one_rpow, mul_one] using
  have :  $0 \leq (A ^ \alpha).\text{trace} :=$  by
    --TODO: Make this all discharge via just `positivity`.
    apply HermitianMat.trace_nonneg
    apply HermitianMat.rpow_nonneg
    positivity
  refine le_of_not_gt fun h => h_final.not_gt ?_
  simp using Real.rpow_lt_one this h (by positivity)

private theorem sandwichedRelEntropy_nonneg_alpha_lt_1 (h :  $\sigma.M.\text{ker} \leq \rho.M.\text{ker}$ )
   $0 \leq ((\rho.M.\text{conj } (\sigma.M ^ ((1 - \alpha)/(2 * \alpha))) .mat) ^ \alpha).\text{trace}.\log / (\alpha - 1)$ 
  apply div_nonneg_of_nonpos

```

```

· apply Real.log_nonpos
· exact (sandwiched_trace_pos h).le
· exact sandwiched_trace_of_lt_1 hα0 hα
· linarith

private theorem sandwichedRelEntropy_nonneg_α_gt_1 (h : σ.M.ker ≤ ρ.M.ker) (hα : α
  0 ≤ ((ρ.M.conj (σ.M ^ ((1 - α)/(2 * α)) ).mat) ^ α).trace.log / (α - 1) := by
  grw [← sandwiched_trace_of_gt_1 h hα]
  · simp
  · linarith

private lemma sandwichedRelEntropy.trace_at_one (ρ σ : MState d) :
  ((ρ.M.conj (σ.M ^ ((1 - (1:ℝ)) / (2 * (1:ℝ))))).mat) ^ (1:ℝ)).trace = 1 := by
  simp

/-
For fixed PSD B, the derivative of  $\alpha \mapsto \text{Tr}[B^\alpha]$  at  $\alpha = 1$  is  $\square[B, B.\log] = \text{Tr}[B \log B]$ .
-/
private lemma hasDerivAt_trace_rpow_at_one (B : HermitianMat d ℂ) (hB : 0 ≤ B) :
  HasDerivAt (fun α : ℝ => (B ^ α).trace) □B, B.log□ 1 := by
  have h_inner : □B, B.log□ = ∑ i, (B.H.eigenvalues i) * Real.log (B.H.eigenvalues i)
  -- By definition of the inner product, we have □B, B.log□ = tr(B * B.log).
  have h_inner_def : □B, B.log□ = Matrix.trace (B.mat * B.log.mat) := by
    simp [Matrix.trace, Inner.inner];
    refine' Finset.sum_congr rfl fun i _ => _;
    have h_herm : (B * B.log : Matrix d d ℂ).IsHermitian := by
      have h_comm : Commute (B : Matrix d d ℂ) (B.log : Matrix d d ℂ) := by
        commutes
      simp_all [Matrix.IsHermitian]
      rw [h_comm]
    have := h_herm.apply i i
    simp_all [Complex.ext_iff]
    linarith
  -- By definition of the trace, we have tr(B * B.log) = ∑ i, B.eigenvalues i * log
  have h_trace : (B.mat * B.log.mat).trace = ∑ i, B.H.eigenvalues i * Real.log (B.H.eigenvalues i)
  have h_trace : (B ^ 1 * B.log.mat).trace = (B.cfc (fun x => x * Real.log x)).trace
  have h_trace : B ^ 1 * B.log.mat = B.cfc (fun x => x * Real.log x) := by
    have h_log : B ^ 1 * B.log.mat = B.cfc id * B.cfc Real.log := by
      simp [pow_one, HermitianMat.log]
    rw [h_log]
    exact (B.mat_cfc_mul_apply id Real.log).symm
  rw [h_trace, ← HermitianMat.trace_eq_trace_rc]
  rfl
  simp_all [HermitianMat.trace_cfc_eq];
  exact_mod_cast h_inner_def.trans h_trace;

```

```

have h_deriv : ∀ i, HasDerivAt (fun α : ℝ => (B.H.eigenvalues i) ^ α) (B
  intro i
  by_cases h_pos : 0 < B.H.eigenvalues i;
  · convert HasDerivAt.rpow ( hasDerivAt_const _ _ ) ( hasDerivAt_id 1 )
    simp only [id_eq, mul_one, sub_self, Real.rpow_zero, Real.rpow_one, o
  · have h_zero : B.H.eigenvalues i = 0 := by
    exact le_antisymm ( le_of_not_gt h_pos ) ( by simp using hB.eigenv
    simp [h_zero]
    exact (hasDerivAt_const _ _).congr_of_eventuallyEq (Filter.eventually
    simp only [HermitianMat.trace_rpow_eq_sum, ← Finset.sum_apply]
    convert HasDerivAt.sum fun i _ => h_deriv i using 1

/-
PROBLEM
Trace cyclicity for conj: Tr[conj(σ^t, ρ)] = [ρ, σ^{2t}] = Tr[ρ σ^{2t}].
Since σ^t is Hermitian: Tr[σ^t ρ σ^t] = Tr[ρ (σ^t)^2] = Tr[ρ σ^{2t}].
PROVIDED SOLUTION
By definition, (ρ.M.conj (σ.M ^ t).mat).mat = (σ.M ^ t).mat * ρ.M.mat * (σ
By Matrix.trace_mul_comm applied to the product ((σ^t).mat * ρ.mat) and (σ^
Tr[(σ^t).mat * ρ.mat * (σ^t).mat] = Tr[(σ^t).mat * (σ^t).mat * ρ.mat].
Now use mat_rpow_add with σ.nonneg and t + t = 2t (≠ 0 since t ≠ 0): (σ.M ^
By inner_eq_trace_rc: [ρ.M, σ.M ^ (2*t)] = (ρ.M.mat * (σ.M ^ (2*t)).mat).tr
By Matrix.trace_mul_comm: Tr[ρ.mat * (σ^{2t}).mat] = Tr[(σ^{2t}).mat * ρ.ma
Combine: the conj trace = Tr[(σ^{2t}).mat * ρ.mat] = Tr[ρ.mat * (σ^{2t}).ma
Note: show t + t = 2 * t by ring, and 2 * t ≠ 0 from ht using two_mul_ne_ze
-/
private lemma trace_conj_eq_inner_rpow {ρ σ : MState d} {t : ℝ} (ht : t ≠ 0
  (ρ.M.conj (σ.M ^ t).mat).trace = [ρ.M, σ.M ^ (2 * t)] := by
  have h_cyclic : ((σ.M ^ t).mat * ρ.M.mat * (σ.M ^ t).mat).trace = ((σ.M ^
  -- Since σ.M ^ t is Hermitian, we can use the property that the trace o
  have h_cyclic : Matrix.trace ((σ.M ^ t).mat * ρ.M.mat * (σ.M ^ t).mat)
    rw [← Matrix.trace_mul_comm]
    simp [Matrix.mul_assoc]
    rw [h_cyclic, two_mul]
    ring_nf
  have h_exp : (σ.M ^ (t + t)).mat = (σ.M ^ t).mat * (σ.M ^ t).mat := by
    have h_nonneg : 0 ≤ σ.M := by
      exact σ.nonneg
    have h_ne_zero : t + t ≠ 0 := by
      exact fun h => ht ( by linarith )
    exact HermitianMat.mat_rpow_add h_nonneg h_ne_zero
    rw [mul_two, h_exp]
  have h_inner : [ρ.M, σ.M ^ (2 * t)] = ((ρ.M.mat * (σ.M ^ (2 * t)).mat).tr
    exact rfl
  simp_all
  convert congr_arg Complex.re h_cyclic using 1 ; simp [ HermitianMat.conj

```

```

rw [ Matrix.trace_mul_comm ]

-- The weight of eigenvalue i in the inner product decomposition
private def eigenWeight (p σ : MState d) (i : d) : ℝ :=
  RCLike.re ((Matrix.vecMul (star (σ.M.H.eigenvectorBasis i : d → ℂ)) p.M.mat) []_v (σ

private lemma inner_cfc_eq_sum_eigenWeight (p σ : MState d) (f : ℝ → ℝ) :
  []p.M, σ.M.cfc f[] = ∑ i, f (σ.M.H.eigenvalues i) * eigenWeight p σ i := by
  -- By definition of the inner product in the context of Hermitian matrices, we can
  have h_inner : []p.M, σ.M.cfc f[] = RCLike.re (Matrix.trace (p.M.mat * (σ.M.cfc f).m
    exact rfl;
  have h_trace : Matrix.trace (p.M.mat * (σ.M.cfc f).mat) = ∑ i, f (σ.M.H.eigenvalue
    rw [ Matrix.trace ];
  have h_cfc_def : (σ.M.cfc f).mat = ∑ i, (f (Matrix.IsHermitian.eigenvalues σ.M.H
    convert σ.M.cfc_toMat_eq_sum_smul_proj f using 1;
    ext i j; simp [ Matrix.single ] ; ring_nf
    simp [ Matrix.sum_apply, Matrix.mul_apply, Matrix.conjTranspose_apply, Matrix.
    refine' Finset.sum_congr rfl fun x _ => _ ; simp [ Finset.sum_ite, Finset.filt
    rw [ Finset.sum_eq_single x ] <=> aesop;
    simp [ h_cfc_def, Matrix.mulVec, dotProduct, Finset.mul_sum, mul_left_comm ];
    simp [ Matrix.sum_apply, Matrix.mul_apply ];
    rw [ Finset.sum_comm ] ; congr ; ext ; congr ; ext ; congr ; ext ; ring!;
    simp_all [ eigenWeight ];
    simp [ Matrix.dotProduct_mulVec ]

private lemma eigenWeight_nonneg (p σ : MState d) (i : d) : 0 ≤ eigenWeight p σ i :=
  -- By definition of `eigenWeight`, we have:
  set v := σ.M.H.eigenvectorBasis i
  set w := p.M.mat.mulVec v
  have h_eigenWeight : eigenWeight p σ i = RCLike.re (star v []_v w) := by
    unfold eigenWeight;
    simp +zetaDelta at *;
    simp [ Matrix.dotProduct_mulVec ]
  rw [h_eigenWeight];
  -- Since p is positive semi-definite, we have that the inner product of any vector
  negative. Hence, we can write:
  have := p.pos
  obtain ⟨ h₁, h₂ ⟩ := this;
  open ComplexOrder in
  have := (Matrix.posSemidef_iff_dotProduct_mulVec.mp h₁).2 v;
  exact this.1.trans (by simp [w])

private lemma eigenWeight_zero_of_eigenvalue_zero {i : d} (hσ : σ.M.ker ≤ p.M.ker)
  (hei : σ.M.H.eigenvalues i = 0) :
  eigenWeight p σ i = 0 := by
  unfold eigenWeight

```

```

have h_mulVec_zero :  $\sigma.M.mat.mulVec (\sigma.M.H.eigenvectorBasis i) = 0$  := by
  convert Matrix.IsHermitian.mulVec_eigenvectorBasis  $\sigma.M.H$  i using 1
  simp [hei]
have h_mulVec_zero' :  $\rho.M.mat.mulVec (\sigma.M.H.eigenvectorBasis i) = 0$  := by
  specialize h $\sigma$  congr(WithLp.toLp 2 $h_mulVec_zero)
  exact congr(WithLp.ofLp $h $\sigma$ )
convert congr_arg (fun x : d  $\rightarrow$   $\mathbb{C} \Rightarrow$  RCLike.re (star ( $\sigma.M.H.eigenvector$ 
· simp [Matrix.dotProduct_mulVec]
· simp [dotProduct]

/-
The derivative of  $u \mapsto [\rho, \sigma^u]$  at  $u = 0$  is  $[\rho, \sigma.log]$ .
Use inner_cfc_eq_sum_eigenWeight to write  $[\rho, \sigma^u] = \sum i, q_i^u * eigen$ 
differentiate term by term using HasDerivAt.sum.
-/
private lemma hasDerivAt_inner_rpow_at_zero (h :  $\sigma.M.ker \leq \rho.M.ker$ ) :
  HasDerivAt (fun u :  $\mathbb{R} \Rightarrow [\rho.M, \sigma.M^u]$ )  $[\rho.M, \sigma.M.log]$  0 := by
  convert HasDerivAt.congr_of_eventuallyEq ?_ ?_;
  exact fun u  $\Rightarrow \sum i, (\sigma.M.H.eigenvalues i)^u * eigenWeight \rho \sigma i$ ;
  · have h_deriv :  $\forall i, HasDerivAt (fun u : \mathbb{R} \Rightarrow (\sigma.M.H.eigenvalues i)^u$ 
    intro i
    rcases ( $\sigma.eigenvalue\_nonneg i$ ).lt_or_eq' with h_pos | h_zero
    · simp only [h_pos, Real.rpow_def_of_pos, mul_comm]
      convert ((hasDerivAt_id 0).mul (hasDerivAt_const _ _)).exp.const_mu
      simp
    · simp [Real.rpow_def_of_nonpos, mul_comm, h_zero]
      convert hasDerivAt_const (0 :  $\mathbb{R}$ ) (0 :  $\mathbb{R}$ ) using 1
      ext1 u
      split_ifs with hu
      · simp [eigenWeight_zero_of_eigenvalue_zero h h_zero]
      · rfl
  convert HasDerivAt.sum (u := Finset.univ) fun i _  $\Rightarrow$  h_deriv i using 1
  · ext x : 1
    simp only [Finset.sum_apply]
  · exact inner_cfc_eq_sum_eigenWeight  $\rho \sigma$  Real.log
  · filter_upwards [Metric.ball_mem_nhds 0 zero_lt_one] with u hu
    exact inner_cfc_eq_sum_eigenWeight  $\rho \sigma$  ( $\cdot^u$ )

/- The derivative of  $\alpha \mapsto Tr[\rho \sigma^{((1-\alpha)/\alpha)}]$  at  $\alpha = 1$  is -
 $[\rho, log \sigma]$ .
Uses trace cyclic:  $Tr[\sigma^{2t} \rho \sigma^{2t}] = Tr[\rho \sigma^{(2t)}]$ .
With  $2t(\alpha) = (1-\alpha)/\alpha$ ,  $d/d\alpha (2t) = -1/\alpha^2$ , and  $d/d\epsilon \sigma^\epsilon|_{\epsilon=0} = log \sigma$ . -
/
private lemma hasDerivAt_trace_conj_at_one { $\rho \sigma$  : MState d}
  (h :  $\sigma.M.ker \leq \rho.M.ker$ ) :
  HasDerivAt

```



```

      (fun α : ℝ => ((p.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat)).trace)
      (-□p.M, σ.M.log□)
      1 := by
    have h_chain : HasDerivAt (fun α : ℝ => □p.M, σ.M ^ ((1 - α) / α)□) (□p.M, σ.M.log
1)) 1 := by
      apply HasDerivAt.comp (h₂ := fun u => □p.M, σ.M ^ u□) (h := fun α => (1 - α) / α)
      · simpa using hasDerivAt_inner_rpow_at_zero h
      · simpa using HasDerivAt.div (hasDerivAt_id (1 : ℝ) |> HasDerivAt.const_sub 1
ring_nf at h_chain
    apply h_chain.congr_of_eventuallyEq _
    filter_upwards [lt_mem_nhds zero_lt_one] with α hα
    by_cases h : (1 - α) / (2 * α) = 0
      · simp [ne_of_gt, hα] at h
        obtain {} : α = 1 := by linarith
        simp [*]
      · simp only [trace_conj_eq_inner_rpow h]
        ring_nf

/-
For a differentiable function b with b(1) = c ≥ 0 and b(α) ≥ 0 near α = 1,
the function α ↦ b(α)^α - b(α) has derivative c * log c at α = 1.
-/
private lemma scalar_rpow_cross_term {b : ℝ → ℝ} {c : ℝ}
  (hb : HasDerivAt b (deriv b 1) 1) (hc : b 1 = c) (hc_pos : 0 < c) :
  HasDerivAt (fun α => b α ^ α - b α) (c * Real.log c) 1 := by
  subst c
  simpa using (hb.rpow (hasDerivAt_id 1) hc_pos).sub hb

/-- For a PSD matrix A, Tr[A^s] - Tr[A] has derivative □A, log A□ at s = 1.
This generalizes `hasDerivAt_trace_rpow_at_one` to give the derivative of the
difference Tr[A^s] - Tr[A], which equals □A, log A□ since d/ds Tr[A] = 0. -
/
private lemma hasDerivAt_trace_rpow_sub_trace (A : HermitianMat d ℂ) (hA : 0 ≤ A) :
  HasDerivAt (fun s : ℝ => (A ^ s).trace - A.trace) □A, A.log□ 1 := by
  simpa using hasDerivAt_trace_rpow_at_one A hA

-- Abbreviation for the "B(α)" matrix appearing in the sandwiched trace.
private abbrev B_of (p σ : MState d) (α : ℝ) : HermitianMat d ℂ :=
  p.M.conj (σ.M ^ ((1 - α) / (2 * α))).mat

-- At α = 1, B(α) = p.M since (1-1)/(2·1) = 0 and conj by 1 is identity.
private lemma B_of_one (p σ : MState d) : B_of p σ 1 = p.M := by
  simp [B_of, HermitianMat.rpow_zero, HermitianMat.conj_one]

-- B(α) is nonneg for α > 0, because it's a conj of a nonneg matrix.
private lemma B_of_nonneg (p σ : MState d) (α : ℝ) : 0 ≤ B_of p σ α := by

```

```

exact HermitianMat.conj_nonneg _ ρ.nonneg

-- The function  $g(M) = \text{Tr}[M^s] - \text{Tr}[M]$  satisfies  $g(M) = 0$  when  $s = 1$ .
private lemma trace_rpow_sub_trace_at_one (M : HermitianMat d ℂ) :
  (M ^ (1 : ℝ)).trace - M.trace = 0 := by
  simp [HermitianMat.rpow_one]

-- The cross term function value at  $\alpha = 1$  is zero.
private lemma cross_term_at_one (ρ σ : MState d) :
  ((B_of ρ σ 1) ^ (1 : ℝ)).trace - (B_of ρ σ 1).trace
  - (ρ.M ^ (1 : ℝ)).trace + 1 = 0 := by
  simp [B_of_one, HermitianMat.rpow_one, ρ.tr]

/-
Scalar rpow cross term with just continuity: for a continuous function  $b$  with
 $b(1) = c > 0$ ,  $b(\alpha) > 0$  near 1, the function  $\alpha \mapsto b(\alpha)^\alpha - b(\alpha)$  has derivative
 $c * \log c$  at  $\alpha = 1$ . The key insight is that  $\partial/\partial x(x^\alpha - x)|_{\alpha=1} = 0$ ,
so the derivative of  $b$  doesn't matter.
-/
private lemma scalar_rpow_cross_term_of_continuous {b : ℝ → ℝ} {c : ℝ}
  (hb_cont : ContinuousAt b 1) (hc : b 1 = c) (hc_pos : 0 < c)
  (hb_pos : ∀f α in nhds 1, 0 < b α) :
  HasDerivAt (fun α => b α ^ α - b α) (c * Real.log c) 1 := by
  rw [hasDerivAt_iff_tendsto_slope_zero];
  -- Use the fact that  $b(1+t)^{1+t} - b(1+t)$  can be rewritten as  $b(1+t)^{1+t} - b(1+t) = b(1+t)^{1+t} - b(1+t) + b(1+t) - b(1+t)$ 
  suffices h_rewrite : Filter.Tendsto (fun t => t-1 * (b (1 + t) * (b (1 + t) ^ t - b (1 + t)))) (nhds 0) (nhds (c * log c))
  refine' h_rewrite.congr' _;
  rw [Filter.EventuallyEq, eventually_nhdsWithin_iff];
  rw [Metric.eventually_nhds_iff] at *;
  obtain (ε, ε_pos, hε) := hb_pos; use ε, ε_pos; intros y hy hy'; rw [ring_nf]
  norm_num [hc]; ring;
  -- Use the fact that  $b(1+t) \rightarrow c$  as  $t \rightarrow 0$ .
  have h_b : Filter.Tendsto (fun t => b (1 + t)) (nhdsWithin 0 {0}c) (nhds c)
  exact hc ▸ hb_cont.tendsto.comp (tendsto_nhdsWithin_of_tendsto_nhds (
  -- Use the fact that  $b(1+t)^t - 1 \sim t \log(b(1+t))$  as  $t \rightarrow 0$ .
  have h_exp : Filter.Tendsto (fun t => t-1 * (b (1 + t) ^ t - 1)) (nhdsWithin 0 {0}c) (nhds (c * log c))
  have h_exp : Filter.Tendsto (fun t => (b (1 + t) ^ t - 1) / t) (nhdsWithin 0 {0}c) (nhds (c * log c))
  have h_log : Filter.Tendsto (fun t => (Real.log (b (1 + t)) * t) / t) (nhdsWithin 0 {0}c) (nhds (c * log c))
  exact Filter.Tendsto.congr' (by filter_upwards [self_mem_nhdsWithin])
  have h_exp : Filter.Tendsto (fun t => (Real.exp (Real.log (b (1 + t))) * t - t) / t) (nhdsWithin 0 {0}c) (nhds (c * log c))
  have h_exp : HasDerivAt (fun t => Real.exp (Real.log (b (1 + t))) * t) (Real.exp (Real.log (b (1 + t)))) 0
  have h_log : HasDerivAt (fun t => Real.log (b (1 + t)) * t) (Real.log (b (1 + t))) 0
  rw [hasDerivAt_iff_tendsto_slope_zero];
  simp [div_eq_inv_mul] using h_log
  convert h_log.exp using 1; norm_num [hc];

```

```

    simpa [ div_eq_inv_mul ] using h_exp.tendsto_slope_zero;
    refine' h_exp.congr'_;
    filter_upwards [ h_b.eventually ( lt_mem_nhds hc_pos ) ] with t ht using by rw
    simp only [ div_eq_inv_mul ] using h_exp;
    convert h_b.mul h_exp using 2 ; ring

/-
Scalar rpow cross term for the zero case: for continuous b with b(1) = 0,
0 ≤ b(α) near 1, the function α ↦ b(α)^α - b(α) has derivative 0 at α = 1.
Uses the convention 0 * log 0 = 0.
-/
private lemma scalar_rpow_cross_term_of_continuous_zero {b : ℝ → ℝ}
  (hb_cont : ContinuousAt b 1) (hc : b 1 = 0)
  (hb_nonneg : ∀f α in nhds 1, 0 ≤ b α) :
  HasDerivAt (fun α => b α ^ α - b α) 0 1 := by
  -- Let's choose any ε > 0$.
  have h_eps : ∀ ε > 0, ∃ δ > 0, ∀ α, abs (α - 1) < δ → abs (b α ^ α - b α) ≤ ε * abs α
  -- Use the fact that $|b(α)^α - b(α)| ≤ |h| · sqrt(b(α)) · |log b(α)|$ for $0 < b(α)$
  have h_bound : ∀f α in nhds 1, |b α ^ α - b α| ≤ |α - 1| * Real.sqrt (|b α|) * |log (b α)|
  have h_bound : ∀f α in nhds 1, 0 ≤ b α ∧ b α ≤ 1 ∧ |α - 1| ≤ 1 / 2 → |b α ^ α - b α| ≤ |α - 1|
  filter_upwards [ hb_nonneg ] with α hα1 hα2 ; rcases eq_or_lt_of_le hα2.1 with
  · norm_num [ ← hα3 ] at *;
  · linarith [ abs_le.mp hα2 ];
  · split_ifs <=> simp_all [ne_of_gt]
  -- Use the fact that $|e^x - 1| \leq |x| e^{|x|}$ for any $x$.
  have h_exp_bound : ∀ x : ℝ, |Real.exp x - 1| ≤ |x| * Real.exp |x| := by
    intro x; rw [ abs_le ] ; constructor <=> cases abs_cases x <=> simp [ *
x ), Real.exp_le_exp.2 ( by linarith : x ≤ |x| ), Real.exp_le_exp.2 ( by linarith :
x ≤ |x| ) ] ;
  -- Apply the exponential bound to $x = \log(b(\alpha)) \cdot (\alpha - 1)$
  have h_exp_bound_applied : |Real.exp (Real.log (b α) * (α - 1)) - 1| ≤ |Real.log (b α)| * |α - 1|
  simp only [ abs_mul, mul_assoc ] using h_exp_bound ( Real.log ( b α ) *
  -- Use the fact that $|b(\alpha)| \leq \sqrt{b(\alpha)}$ for $0 < b(\alpha)$
  have h_sqrt_bound : |b α| * Real.exp (|Real.log (b α)| * |α - 1|) ≤ Real.sqrt (|b α|)
  rw [ abs_of_nonneg hα2.1 ] ; rw [ Real.sqrt_eq_rpow ] ; rw [ ← Real.log
cases abs_cases ( Real.log ( b α ) ) <=> cases abs_cases ( -
1 + α ) <=> nlinarith [ Real.log_le_sub_one_of_pos hα3, abs_le.mp hα2.2.2 ] ;
  rw [ show Real.exp ( Real.log ( b α ) * α ) = Real.exp ( Real.log ( b α )
field_simp;
  rw [ abs_mul ] ; nlinarith [ abs_nonneg ( Real.log ( b α ) ), abs_nonneg (
have h_bound : ∀f α in nhds 1, 0 ≤ b α ∧ b α ≤ 1 ∧ |α - 1| ≤ 1 / 2 := by
  have h_bound : ∀f α in nhds 1, 0 ≤ b α ∧ b α ≤ 1 := by
    filter_upwards [ hb_nonneg, hb_cont.eventually ( Metric.ball_mem_nhds _ ze
    filter_upwards [ h_bound, Metric.ball_mem_nhds 1 ( show ( 0 : ℝ ) < 1 / 2 by
    filter_upwards [ h_bound, <∀f α in nhds 1, 0 ≤ b α ∧ b α ≤ 1 ∧ |α - 1| ≤ 1 / 2
  -- Use the fact that $\sqrt{|b(\alpha)|} \cdot |\log(|b(\alpha)|)| \to 0$ as $b(\alpha) \to 0$.
```

```

have h_sqrt_log : Filter.Tendsto (fun α => Real.sqrt (|b α|) * |Real.log x|)
have h_sqrt_log : Filter.Tendsto (fun x => Real.sqrt x * |Real.log x|)
have h_sqrt_log : Filter.Tendsto (fun x => Real.sqrt x * Real.log x)
-- Let $y = \sqrt{x}$, so we can rewrite the limit as $\lim_{y \rightarrow 0} y \log y$
suffices h_log_y : Filter.Tendsto (fun y => 2 * y * Real.log y) (
  have h_subst : Filter.Tendsto (fun x => 2 * Real.sqrt x * Real.log x)
    exact h_log_y.comp <| Filter.Tendsto.inf (Real.continuous_sqrt)
    generalize_proofs at *; (
      exact h_subst.congr' (Filter.eventuallyEq_of_mem self_mem_nhds)
      exact tendsto_nhdsWithin_of_tendsto_nhds (by simpa [mul_assoc])
      exact tendsto_zero_iff_norm_tendsto_zero.mpr (by simpa using h_sqrt_log)
have h_sqrt_log : Filter.Tendsto (fun α => Real.sqrt (|b α|) * |Real.log x|)
  refine' h_sqrt_log.comp _;
  rw [tendsto_nhdsWithin_iff];
  exact (tendsto_nhdsWithin_of_tendsto_nhds (by simpa [hc]) using
    rw [Metric.tendsto_nhdsWithin_nhds] at h_sqrt_log;
    exact Metric.tendsto_nhds_nhds.mpr fun ε hε => by rcases h_sqrt_log ε
  intro ε hε_pos
  obtain (δ1, hδ1_pos, hδ1) : ∃ δ1 > 0, ∀ α, abs (α - 1) < δ1 → Real.sqrt
    simpa using Metric.tendsto_nhds_nhds.mp h_sqrt_log ε hε_pos |> fun (
  obtain (δ2, hδ2_pos, hδ2) : ∃ δ2 > 0, ∀ α, abs (α - 1) < δ2 → |b α ^ α|
    exact Metric.mem_nhds_iff.mp h_bound |> fun (δ2, hδ2_pos, hδ2) => (
    exact (Min.min δ1 δ2, lt_min hδ1_pos hδ2_pos, fun α hα => le_trans (h
  rw [hasDerivAt_iff_isLittle0_nhds_zero];
  rw [Asymptotics.isLittle0_iff];
  intro ε hε; rcases h_eps ε hε with (δ, hδ, H); filter_upwards [Metric

/-- If  $\ker A \leq \ker \rho M$ , then conjugating  $\rho M$  by the support projection of  $A$  gives  $\rho M$ .
This is because  $\rho M$  is supported entirely on the support (= range) of  $A$ .
/
private lemma conj_supportProj_eq_of_ker_le (A ρM : HermitianMat d ℂ) (hker :
  ρM.conj (A.supportProj).mat = ρM := by
  have h_conj_support : ρM.mat * A.supportProj.mat = ρM.mat := by
    apply HermitianMat.mul_supportProj_of_ker_le; assumption;
  -- Using the conjugate transpose property and the fact that A.supportProj
  have h_conj_support : (A.supportProj.mat).conjTranspose * ρM.mat * A.supportProj.mat = ρM.mat
    rw [← Matrix.conjTranspose_inj] at *; aesop;
  simp_all [HermitianMat.conj, Matrix.mul_assoc]

/-- For a PSD matrix  $A$ , the function  $r \mapsto A^r$  converges to  $A$ .supportProj as  $r \rightarrow 0$  through nonzero values. On positive eigenvalues  $\lambda$ ,  $\lambda^r \rightarrow 1$ . On zero eigenvalues,  $0^r = 0$  for  $r \neq 0$ . So the limit is the support projection (indicator of nonzero eigenvalues). -/
private lemma rpow_tendsto_supportProj (A : HermitianMat d ℂ) :
  Filter.Tendsto (fun r : ℝ => A ^ r) (nhdsWithin 0 {(0 : ℝ)}c) (nhds A.supportProj)

```



```

    have h_exp : ContinuousAt (fun  $\alpha$  :  $\mathbb{R}$  => (1 -  $\alpha$ ) / (2 *  $\alpha$ )) 1 := by
      exact ContinuousAt.div ( continuousAt_const.sub continuousAt_id ) (c
      exact ContinuousAt.comp ( by simp using h_inner ) h_exp |> ContinuousA
    exact h_cont

private lemma trace_cfc_sub_le (A : HermitianMat d  $\mathbb{C}$ ) (f g :  $\mathbb{R}$  →  $\mathbb{R}$ ) :
  |(A.cfc f).trace - (A.cfc g).trace| ≤
    (Fintype.card d :  $\mathbb{R}$ ) * (∑ i, |f (A.H.eigenvalues i) - g (A.H.eigenval
  rw [HermitianMat.trace_cfc_eq, HermitianMat.trace_cfc_eq]
  convert Finset.abs_sum_le_sum_abs _ Finset.univ |> le_trans <| Finset.sum
  · simp [← Finset.sum_sub_distrib]
  · simp

/-- Eigenvalues of M( $\alpha$ ) are uniformly bounded near  $\alpha = 1$ . -
/
private lemma eigenvalues_bounded_near {M :  $\mathbb{R}$  → HermitianMat d  $\mathbb{C}$ }
  (hM_nonneg : ∀f  $\alpha$  in nhds 1, 0 ≤ M  $\alpha$ )
  (hM_cont : ContinuousAt M 1) :
  ∃ K > 0, ∀f  $\alpha$  in nhds 1, ∀ i, 0 ≤ (M  $\alpha$ ).H.eigenvalues i ∧ (M  $\alpha$ ).H.eigen
  have h_op_norm_bound : ∃ K > 0, ∀f  $\alpha$  in nhds 1, ||M  $\alpha$ || ≤ K := by
    exact ( ||M 1|| + 1, by positivity, hM_cont.norm.eventually ( ge_mem_nhds
  have h_eigenvalue_bound : ∀f  $\alpha$  in nhds 1, ∀ i, 0 ≤ (M  $\alpha$ ).H.eigenvalues i
  have h_eigenvalue_bound : ∀f  $\alpha$  in nhds 1, ∀ i, |(M  $\alpha$ ).H.eigenvalues i|
    filter_upwards [ hM_nonneg, h_op_norm_bound.choose_spec.2 ] with  $\alpha$  h $\alpha$ 
    exact HermitianMat.eigenvalue_norm_le (M  $\alpha$ ) i
    filter_upwards [ h_eigenvalue_bound, hM_nonneg ] with  $\alpha$  h $\alpha$  h $\alpha'$  i
    exact ( h $\alpha'$  |> fun h => by simp using h.eigenvalues_nonneg i, le_of_ab
    exact ( h_op_norm_bound.choose, h_op_norm_bound.choose_spec.1, h_eigenval

/-
Uniform convergence of (x^{1+h} - x)/h to x * log x on [0, K] as h → 0.
This is the uniform version of the derivative of s ↦ x^s at s = 1.
-/
set_option maxHeartbeats 800000 in
private lemma rpow_slope_tendsto_uniformly (K :  $\mathbb{R}$ ) :
  ∀  $\varepsilon$  > 0, ∃  $\delta$  > 0, ∀ h :  $\mathbb{R}$ , 0 < |h| → |h| <  $\delta$  →
  ∀ x ∈ Set.Icc 0 K, |(x ^ (1 + h) - x) / h - x * Real.log x| <  $\varepsilon$  := by
  intro  $\varepsilon$   $\varepsilon_{\text{pos}}$ 
  obtain (δ1, δ1_pos, hδ1) : ∃ δ1 > 0, ∀ x ∈ Set.Icc 0 K, 0 < x → x < δ1 →
  obtain (δ1, δ1_pos, hδ1) : ∃ δ1 > 0, ∀ x ∈ Set.Icc 0 K, 0 < x → x < δ1
  have h_cont : Filter.Tendsto (fun x => x * Real.log x) (nhdsWithin 0
    exact tendsto_nhdsWithin_of_tendsto_nhds ( by simp using Real.cont
  have := Metric.tendsto_nhdsWithin_nhds.mp h_cont (  $\varepsilon$  / 4 ) ( by linar
  exact ( this.choose, this.choose_spec.1, fun x hx1 hx2 hx3 => by simp
  have h_bound : ∃ δ1 > 0, ∀ x ∈ Set.Icc 0 K, 0 < x → x < δ1 → ∀ h, 0 < |
  have h_bound : ∀ x ∈ Set.Icc 0 K, 0 < x → ∀ h :  $\mathbb{R}$ , 0 < |h| → |h| < 1

```

```

intros x hx hx_pos h hh_pos hh_lt_half
have h_exp_bound : |Real.exp (h * Real.log x) - 1| ≤ |h| * |Real.log x| * Re
  have h_exp_bound : ∀ y : ℝ, |Real.exp y - 1| ≤ |y| * Real.exp (|y|) := by
    intro y; rw [abs_le] ; constructor <;> cases abs_cases y <;> simp [ *
      · nlinarith [ Real.add_one_le_exp y ];
      · nlinarith [ Real.exp_pos y, Real.exp_neg y, mul_inv_cancel₀ ( ne_of_gt
y ) ];
      · nlinarith [ Real.exp_pos y, Real.exp_neg y, mul_inv_cancel₀ ( ne_of_gt
y ) ];
      · nlinarith [ Real.exp_pos y, Real.exp_neg y, mul_inv_cancel₀ ( ne_of_gt
y ) ];
    simp only [ abs_mul ] using h_exp_bound ( h * Real.log x );
    rw [ Real.rpow_add hx_pos, Real.rpow_one ];
    rw [ Real.rpow_def_of_pos hx_pos ];
    rw [ show x * Real.exp ( Real.log x * h ) - x = x * ( Real.exp ( h * Real.lo
    rw [ abs_mul, abs_of_nonneg hx_pos.le ]
    refine' le_trans ( mul_le_mul_of_nonneg_left h_exp_bound hx_pos.le ) _
    ring_nf
    exact le_add_of_le_of_nonneg ( mul_le_mul_of_nonneg_left ( Real.exp_le_exp.m
    exact ( δ₁, δ₁_pos, fun x hx hx' hx'' h hh hh' => h_bound x hx hx' h hh hh' );
    obtain (δ₂, δ₂_pos, hδ₂) : ∃ δ₂ > 0, ∀ x ∈ Set.Icc 0 K, 0 < x → x < δ₂ → x * (|R
    have h_bound : Filter.Tendsto (fun x => x * (|Real.log x| + 1) * Real.exp (1 /
      -- Let $y = -\log x$, so we can rewrite the limit as $y \to \infty$.
      suffices h_log : Filter.Tendsto (fun y => Real.exp (-
y) * (y + 1) * Real.exp ((y + 1) / 2)) Filter.atTop (nhds 0) by
        have h_subst : Filter.Tendsto (fun x => Real.exp (-
(-Real.log x)) * ((-Real.log x) + 1) * Real.exp ((-Real.log x + 1) / 2)) (nhdsWithin
          exact h_log.comp ( Filter.tendsto_neg_atBot_atTop.comp ( Real.tendsto_lo
          refine' h_subst.congr' _;
          filter_upwards [ Ioo_mem_nhdsGT_of_mem ( le_rfl, zero_lt_one ) ] with x hx
          rw [ abs_of_nonpos ( Real.log_nonpos hx.1.le hx.2.le ) ]
          rw [ neg_neg, Real.exp_log hx.1 ]
          ring_nf
          -- We can factor out $e^{-y/2}$ and use the fact that $e^{\{-
y/2\}} \to 0$ as $y \to \infty$.
          suffices h_factor : Filter.Tendsto (fun y => Real.exp (-
y / 2) * (y + 1)) Filter.atTop (nhds 0) by
            convert h_factor.const_mul ( Real.exp ( 1 / 2 ) ) using 2 <;> ring_nf
            norm_num [ mul_assoc, ← Real.exp_add ] ; ring_nf
            -- Let $z = \frac{y}{2}$, so we can rewrite the limit as $z \to \infty$.
            suffices h_z : Filter.Tendsto (fun z => Real.exp (-
z) * (2 * z + 1)) Filter.atTop (nhds 0) by
              convert h_z.comp ( Filter.tendsto_id.atTop_mul_const ( by norm_num : 0 < (
have := Real.tendsto_pow_mul_exp_neg_atTop_nhds_zero 1;
convert this.const_mul 2 |> Filter.Tendsto.add <| Real.tendsto_exp_atBot.com
ring;

```

```

have := Metric.tendo_stdo_hndsWithin_hnds.mp h_bound ( ε / 4 ) ( by linarith
obtain ( δ2, δ2_pos, H ) := this; exact ( δ2, δ2_pos, fun x hx1 hx2 h
obtain (δ3, δ3_pos, hδ3) : ∃ δ3 > 0, ∀ x ∈ Set.Icc 0 K, 0 < x → x < δ3
obtain ( δ3, δ3_pos, hδ3 ) := h_bound;
exact ( δ3, δ3_pos, fun x hx hx' hx'' h hh hh' => by rw [ abs_div, di
refine' ( Min.min δ1 ( Min.min δ2 δ3 ), lt_min δ1_pos ( lt_min δ2_pos δ
exact lt_of_le_of_lt ( mul_le_mul_of_nonneg_left ( Real.exp_le_exp.mpr
obtain (δ2, δ2_pos, hδ2) : ∃ δ2 > 0, ∀ x ∈ Set.Icc δ1 K, ∀ h, 0 < |h| → |
have h_mean_value : ∀ x ∈ Set.Icc δ1 K, ∀ h, 0 < |h| → |h| < 1 / 2 → |(
intros x hx h h_pos h_lt
have h_mean_value : |(x ^ h - 1) / h - Real.log x| ≤ |h| * (Real.log
-- Applying the inequality |e^y - 1 - y| ≤ |y|^2 e^|y| with y = h *
have h_exp_ineq : ∀ y : ℝ, |Real.exp y - 1 - y| ≤ |y|^2 * Real.exp
intro y; rw [ abs_le ] ; constructor <;> cases abs_cases y <;> si
· nlinarith [ Real.add_one_le_exp y, Real.exp_pos y ];
· nlinarith [ Real.add_one_le_exp y, Real.add_one_le_exp ( -
y ), Real.exp_pos y, Real.exp_pos ( -y ) ];
· -- Using the Taylor series expansion of e^y, we have e^y \le
have h_taylor : ∀ y : ℝ, 0 ≤ y → Real.exp y ≤ 1 + y + y^2 * Rea
intro y hy; nlinarith [ Real.exp_pos y, Real.exp_neg y, mul_i
y ), mul_nonneg hy ( Real.exp_nonneg y ), mul_nonneg hy ( Real.exp_nonneg (
y ) ) ] ;
linarith [ h_taylor y ( by linarith ) ];
· nlinarith [ Real.exp_pos y, Real.exp_neg y, mul_inv_cancel0 ( n
y ) ];
convert mul_le_mul_of_nonneg_left ( h_exp_ineq ( h * Real.log x ) )
· rw [ ← abs_inv, ← abs_mul ] ; ring_nf;
by_cases hh : h = 0 <;> aesop;
· simp [ sq, mul_assoc, mul_comm, mul_left_comm, h_pos.ne' ];
convert mul_le_mul_of_nonneg_left h_mean_value ( show 0 ≤ x by linari
rw [ show x ^ ( 1 + h ) * h-1 + ( - ( x * h-1 ) - x * Real.log x ) =
h-1 + ( h-1 * x ^ h - Real.log x ) ) by rw [ Real.rpow_add ( by linarith [
-- Choose δ2 such that |h| * x * (Real.log x) ^ 2 * Real.exp (|h| * |Re
obtain (δ2, δ2_pos, hδ2) : ∃ δ2 > 0, ∀ x ∈ Set.Icc δ1 K, ∀ h, 0 < |h| →
-- Since $x * (\log x)^2 * \exp(|h| * |\log x|)$ is continuous on the
obtain (M, hM) : ∃ M > 0, ∀ x ∈ Set.Icc δ1 K, ∀ h, 0 < |h| → |h| < 1
have h_cont : ContinuousOn (fun x => x * (Real.log x) ^ 2 * Real.ex
exact ContinuousOn.mul ( ContinuousOn.mul continuousOn_id ( Conti
obtain ( M, hM ) := IsCompact.exists_bound_of_continuousOn ( Compac
norm_num +zetaDelta at *;
exact ( Max.max M 1, by positivity, fun x hx1 hx2 h hh1 hh2 => le_t
exact ( Min.min ( 1 / 2 ) ( ε / 4 / M ), lt_min ( by norm_num ) ( div
exact ( Min.min δ2 ( 1 / 2 ), lt_min δ2_pos ( by norm_num ), fun x hx h
refine' ( Min.min ( 1 / 2 ) δ2, lt_min ( by positivity ) δ2_pos, fun h hh
· cases lt_or_eq_of_le hx.1 <;> simp_all [ abs_of_nonneg ];
· constructor <;> cases abs_cases ( Real.log x ) <;> nlinarith [ hδ1 x

```



```

    · by_cases h : 1 + h = 0 <;> simp_all [ division_def ] ; linarith [ Real.log_le_
      norm_num [ ← <0 = x> ] at * ; aesop;
    · constructor <;> linarith [ hδ₂ x <_> hx.2 h hh₁ hh₂.2.1 hh₂.2.2 ]

/-
CFC trace continuity: if  $M \rightarrow \rho.M$  and  $f$  is continuous on  $[0, \infty)$ ,
then  $\text{Tr}[(M \alpha).cfc(f)] \rightarrow \text{Tr}[\rho.M.cfc(f)]$ .
-/
private lemma trace_cfc_tendsto_of_tendsto (f :  $\mathbb{R} \rightarrow \mathbb{R}$ )
  (hf : ContinuousOn f (Set.Ici 0))
  {M :  $\mathbb{R} \rightarrow \text{HermitianMat } d \mathbb{C}$ }
  (hM_cont : ContinuousAt M 1) (hM_nonneg :  $\forall^\rho \alpha \text{ in nhds } 1, 0 \leq M \alpha$ )
  (hM_one :  $M \ 1 = \rho.M$ ) :
  Filter.Tendsto (fun  $\alpha \Rightarrow ((M \alpha).cfc f).trace$ ) (nhds 1) (nhds (( $\rho.M.cfc f$ ).trace))
have h_cfc_cont : ContinuousWithinAt (fun A :  $\text{HermitianMat } d \mathbb{C} \Rightarrow A.cfc f$ ) {A :  $\text{HermitianMat } d \mathbb{C}$ }
have h_cont : ContinuousOn (fun A :  $\text{HermitianMat } d \mathbb{C} \Rightarrow A.cfc f$ ) {A :  $\text{HermitianMat } d \mathbb{C}$ }
have h_cont_trace : ContinuousOn (fun A :  $\text{HermitianMat } d \mathbb{C} \Rightarrow (A.cfc f).trace$ ) {A :  $\text{HermitianMat } d \mathbb{C}$ }
have h_cont_cfc : ContinuousOn (fun A :  $\text{HermitianMat } d \mathbb{C} \Rightarrow A.cfc f$ ) {A :  $\text{HermitianMat } d \mathbb{C}$ }
intro A hA
exact HermitianMat.continuousWithinAt_cfc_of_continuousOn hf hA
have h_spectrum_subset :  $\forall A : \text{HermitianMat } d \mathbb{C}, 0 \leq A \rightarrow \text{spectrum } \mathbb{R} A.mat \subseteq$ 
intro A hA
exact (HermitianMat.posSemidef_iff_spectrum_Ici A).mp hA
exact h_cont_cfc.mono fun A hA  $\Rightarrow$  h_spectrum_subset A hA
exact h_cont_trace
exact h_cont _ ( by simp [  $\rho.2$  ] ) |> ContinuousWithinAt.mono <| Set.Subset.refl
have h_trace_cont : Continuous (fun A :  $\text{HermitianMat } d \mathbb{C} \Rightarrow A.trace$ ) := by
exact HermitianMat.trace_Continuous;
have h_comp_cont : Filter.Tendsto (fun  $\alpha \Rightarrow (M \alpha).cfc f$ ) (nhds 1) (nhds (( $\rho : \text{HermitianMat } d \mathbb{C} \Rightarrow \text{HermitianMat } d \mathbb{C}$ )).cfc f))
convert h_cfc_cont.tendsto.comp _ using 2;
exact tendsto_nhdsWithin_iff.mpr { hM_cont.tendsto.trans ( by simp [ hM_one ] ) },
exact h_trace_cont.continuousAt.tendsto.comp h_comp_cont

/-
The remainder term  $r(1+h)/h \rightarrow 0$  where
` $r(\alpha) = \text{Tr}[M(\alpha)^\alpha] - \text{Tr}[M(\alpha)] - \text{Tr}[\rho.M^\alpha] + \text{Tr}[\rho.M]$ `
-/
set_option maxHeartbeats 800000 in
private lemma cross_term_slope_tendsto_zero
  {M :  $\mathbb{R} \rightarrow \text{HermitianMat } d \mathbb{C}$ }
  (hM_nonneg :  $\forall^\rho \alpha \text{ in nhds } 1, 0 \leq M \alpha$ )
  (hM_cont : ContinuousAt M 1)
  (hM_one :  $M \ 1 = \rho.M$ ) :
  Filter.Tendsto
    (fun h :  $\mathbb{R} \Rightarrow ((M (1 + h)) ^ (1 + h)).trace - (M (1 + h)).trace$ 
      - ( $\rho.M ^ (1 + h)).trace + \rho.M.trace$ ) / h)

```

```

(nhdsWithin 0 {0}^c)
(nhds 0) := by
obtain ⟨ K, hK_pos, hK ⟩ := eigenvalues_bounded_near hM_nonneg hM_cont;
clear hK_pos
-- Let  $G_h(x) = \frac{x^{1+h} - x}{h}$  for  $h \neq 0$  and  $G_0(x) = x \log x$ 
set Gh :  $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} := \text{fun } h \ x \Rightarrow \text{if } h = 0 \text{ then } x * \text{Real.log } x \text{ else } (x ^ (1 + h) - x) / h$ 
-- Using the triangle inequality decomposition with  $G_0(x) = x \log x$ ,
have h_triangle : Filter.Tendsto (fun h =>  $\sum i, Gh \ h \ ((M \ (1 + h)).H.eigenvalues)$ ) 0 0
-- By the properties of the trace and the continuity of  $G_h$ , we can bound the difference
have h_bound :  $\forall \varepsilon > 0, \exists \delta > 0, \forall h : \mathbb{R}, 0 < |h| \rightarrow |h| < \delta \rightarrow \forall x \in \text{Set.univ}, |G_h(x) - G_0(x)| < \varepsilon$ 
  intro  $\varepsilon$   $\varepsilon_{\text{pos}}$ 
  obtain ⟨  $\delta$ ,  $\delta_{\text{pos}}$ , h $\delta$  ⟩ :  $\exists \delta > 0, \forall h : \mathbb{R}, 0 < |h| \rightarrow |h| < \delta \rightarrow \forall x \in \text{Set.univ}, |G_h(x) - G_0(x)| < \delta$ 
  have := rpow_slope_tendsto_uniformly K  $\varepsilon_{\text{pos}}$ ; aesop;
  use  $\delta$ ,  $\delta_{\text{pos}}$ 
  intro h hh_pos hh_lt
  aesop;
-- Using the bound, we can show that the difference tends to zero.
have h_diff_zero : Filter.Tendsto (fun h =>  $\sum i, (Gh \ h \ ((M \ (1 + h)).H.eigenvalues))$ ) 0 0
  constructor;
  · have h_diff_zero :  $\forall \varepsilon > 0, \exists \delta > 0, \forall h : \mathbb{R}, 0 < |h| \rightarrow |h| < \delta \rightarrow \forall i, 0 \leq |G_h(h_i) - G_0(h_i)| < \varepsilon$ 
    intro  $\varepsilon$   $\varepsilon_{\text{pos}}$ 
    obtain ⟨  $\delta$ , h $\delta_{\text{pos}}$ , h $\delta$  ⟩ := h_bound  $\varepsilon$   $\varepsilon_{\text{pos}}$ 
    obtain ⟨  $\delta'$ , h $\delta'_{\text{pos}}$ , h $\delta'$  ⟩ :  $\exists \delta' > 0, \forall h : \mathbb{R}, |h| < \delta' \rightarrow \forall i, 0 \leq |G_h(h_i) - G_0(h_i)| < \delta'$ 
    rcases Metric.mem_nhds_iff.mp hK with ⟨  $\delta'$ , h $\delta'_{\text{pos}}$ , h $\delta'$  ⟩;
    exact ⟨  $\delta'$ , h $\delta'_{\text{pos}}$ , fun h hh i => h $\delta'$  ( mem_ball_iff_norm.mpr (Metric.mem_nhds_iff.mp hK) ) ⟩
    exact ⟨ Min.min  $\delta \ \delta'$ , lt_min h $\delta_{\text{pos}}$  h $\delta'_{\text{pos}}$ , fun h hh1 hh2 i => h $\delta$  (Metric.mem_nhds_iff.mp hK) ⟩
    rw [ Metric.tendsto_nhdsWithin_nhds ];
    intro  $\varepsilon$   $\varepsilon_{\text{pos}}$ 
    obtain ⟨  $\delta$ , h $\delta_{\text{pos}}$ , h $\delta$  ⟩ := h_diff_zero ( $\varepsilon / (\text{Fintype.card } d + 1)$ ) (by positivity);
    refine' ⟨  $\delta$ , h $\delta_{\text{pos}}$ , fun x hx hx' => _ ⟩;
    simp +zetaDelta at *;
    rw [ if_neg hx ];
    rw [ ← Finset.sum_sub_distrib ];
    exact lt_of_le_of_lt ( Finset.abs_sum_le_sum_abs _ _ ) ( lt_of_le_of_lt (Metric.tendsto_nhdsWithin_nhds) (Metric.tendsto_nhdsWithin_nhds) );
    intro  $\varepsilon$   $\varepsilon_{\text{pos}}$ ; rcases h_bound ( $\varepsilon / (\text{Fintype.card } d + 1)$ ) (div_pos (Metric.tendsto_nhdsWithin_nhds) (Metric.tendsto_nhdsWithin_nhds));
    rw [ ← Finset.sum_sub_distrib ];
    refine' lt_of_le_of_lt ( Finset.abs_sum_le_sum_abs _ _ ) _;
    refine' lt_of_le_of_lt ( Finset.sum_le_sum fun i _ => le_of_lt (Metric.tendsto_nhdsWithin_nhds) ) _;
    · have := hK i; have := this.1.self_of_nhds; aesop;
    · exact hK i |>.2.self_of_nhds |> fun h => by simp [ hM_one ] using Metric.tendsto_nhdsWithin_nhds;
    · simp [ div_eq_mul_inv ];
      nlinarith [ mul_inv_cancel_left ( by positivity : ( Fintype.card d + 1 ) > 0 ) ];
have h_diff_zero : Filter.Tendsto (fun h =>  $\sum i, ((M \ (1 + h)).H.eigenvalues) \cdot h_i$ ) 0 0
  have h_diff_zero : Filter.Tendsto (fun h =>  $((M \ (1 + h)).cfc \ (fun x => x \log x))$ ) 0 0

```

```

have h_diff_zero : Filter.Tendsto (fun h => ((M (1 + h)).cfc (fun x => x * R
  have h_diff_zero : Filter.Tendsto (fun α => ((M α).cfc (fun x => x * Real.
    convert trace_cfc_tendsto_of_tendsto _ _ _ _ using 1;
    · exact Continuous.continuousOn ( Real.continuous_mul_log );
    · exact hM_cont;
    · exact hM_nonneg;
    · exact hM_one;
    exact h_diff_zero.comp ( tendsto_nhdsWithin_of_tendsto_nhds ( by norm_num
      simp using h_diff_zero.sub_const ( ( ρ.M.cfc fun x => x * Real.log x ).trac
    convert h_diff_zero using 2;
    rw [ HermitianMat.trace_cfc_eq, HermitianMat.trace_cfc_eq ];
    convert h_diff_zero.add ( <Filter.Tendsto ( fun h => ∑ i, ( Gh h ( ( M ( 1 + h )
  refine' h_triangle.congr' _;
  rw [ Filter.EventuallyEq, eventually_nhdsWithin_iff ];
  rw [ Metric.eventually_nhds_iff ] at *;
  obtain ( ε, ε_pos, hε ) := hK; use ε, ε_pos; intro y hy hy'; simp_all [ div_eq_inv
  have h_trace_rpow : ∀ (A : HermitianMat d ℂ) (p : ℝ), (A ^ p).trace = ∑ i, (A.H.ei
    exact fun A p => HermitianMat.trace_rpow_eq_sum A p;
  have := h_trace_rpow ( M ( 1 + y ) ) 1; have := h_trace_rpow ( ρ : HermitianMat d
  simp +zetaDelta at *;
  simp [ ← this, div_eq_inv_mul, mul_sub, hy' ];
  simp [ ← Finset.mul_sum _ _ _, this.symm ]
  ring

/-- For a differentiable family of PSD matrices M(α) with M(1) having eigenvalues p,
  the function α ↦ Tr[M(α)^α] - Tr[M(α)] has derivative [M(1), M(1).log] at α = 1.
  This is because at α = 1, the function x^s - x has zero x-
  derivative (since d/dx(x^1) = 1),
  so only the s-derivative contributes, giving the same answer as for fixed eigenv
  /
private lemma hasDerivAt_trace_rpow_sub_trace_variable_base
  {M : ℝ → HermitianMat d ℂ}
  (hM_nonneg : ∀ α in nhds 1, 0 ≤ M α)
  (hM_cont : ContinuousAt M 1)
  (hM_one : M 1 = ρ.M) :
  HasDerivAt (fun α : ℝ => (M α ^ α).trace - (M α).trace) [ρ.M, ρ.M.log] 1 := by
  have h_deriv : HasDerivAt (fun α : ℝ => ((M α) ^ α).trace - (M α).trace - ((ρ.M) ^
    convert hasDerivAt_iff_tendsto_slope_zero.mpr _ using 1
    convert cross_term_slope_tendsto_zero hM_nonneg hM_cont hM_one using 2 ; norm_num
  convert h_deriv.add ( hasDerivAt_trace_rpow_sub_trace ρ.M ρ.nonneg ) using 1 <=> n
  ring_nf
  ext; norm_num; ring

/-- The cross term in the derivative decomposition vanishes: the function
  α ↦ Tr[B(α)^α] - Tr[B(α)] - Tr[p^α] + 1 has derivative 0 at α = 1.
  This is because at α=1, B^1 = B, so ∂/∂B Tr[B^α] = Tr[·] (the trace is linear),

```

```

    making the cross term (variation in B times variation in  $\alpha$ ) vanish. -
/
private lemma rpow_trace_cross_term_vanishes { $\rho$   $\sigma$  : MState d}
  (h :  $\sigma.M.ker \leq \rho.M.ker$ ) :
  HasDerivAt
    (fun  $\alpha$  :  $\mathbb{R}$  => (( $\rho.M.conj$  ( $\sigma.M ^ ((1 - \alpha) / (2 * \alpha)))$ ).mat) ^  $\alpha$ ).trace
      - ( $\rho.M.conj$  ( $\sigma.M ^ ((1 - \alpha) / (2 * \alpha)))$ ).mat).trace
      - ( $\rho.M ^ \alpha$ ).trace + 1)
    0
  1 := by
  have h_cross_term : HasDerivAt (fun  $\alpha$  :  $\mathbb{R}$  => (( $\rho.M.conj$  ( $\sigma.M ^ ((1 - \alpha) / (2 * \alpha)))$ ).mat) ^  $\alpha$ ).trace /
  apply And.intro;
  · convert hasDerivAt_trace_rpow_sub_trace_variable_base _ _ _ using 1;
  · exact Filter.Eventually.of_forall fun  $\alpha$  => B_of_nonneg  $\rho$   $\sigma$   $\alpha$ ;
  · convert B_of_continuousAt  $\rho$   $\sigma$  h using 1;
  · simp [ HermitianMat.conj ];
  · convert hasDerivAt_trace_rpow_at_one  $\rho.M$  ( by exact  $\rho$ .nonneg ) using
  convert HasDerivAt.add ( HasDerivAt.sub h_cross_term.1 h_cross_term.2 ) (
  ring

private theorem sandwichedRelEntropy.hasDerivAt_trace_at_one { $\rho$   $\sigma$  : MState d}
  (h :  $\sigma.M.ker \leq \rho.M.ker$ ) :
  HasDerivAt
    (fun  $\alpha$  :  $\mathbb{R}$  => (( $\rho.M.conj$  ( $\sigma.M ^ ((1 - \alpha) / (2 * \alpha)))$ ).mat) ^  $\alpha$ ).trace)
    [ $\rho.M$ ,  $\rho.M.log - \sigma.M.log$ ]
  1 := by
  have h_deriv :=
    have h_cross_term := rpow_trace_cross_term_vanishes h
    have h_conj := hasDerivAt_trace_conj_at_one h
    have h_rpow := hasDerivAt_trace_rpow_at_one  $\rho.M$   $\rho$ .nonneg
    (h_cross_term.add (h_conj.add h_rpow)).sub (hasDerivAt_const 1 1)
  convert h_deriv using 2
  · simp only [Pi.sub_apply, Pi.add_apply]
    ring
  · simp only [inner_sub_right]
    ring

/--
The key limit: as  $\alpha \rightarrow 1$ ,  $\log(\text{Tr}[(\rho.conj \sigma^t)^\alpha]) / (\alpha-1) \rightarrow [\rho, \log \rho - \log \sigma]$ 
where  $t = (1-\alpha)/(2\alpha)$ . Derived from hasDerivAt_trace_at_one via L'Hôpitala
(or equivalently,  $\log(1+x)/x \rightarrow 1$  and  $(f(\alpha)-1)/(\alpha-1) \rightarrow f'(1)$ ).
-/
private theorem sandwichedRelEntropy.limit_at_one ( $\rho$   $\sigma$  : MState d)
  (h :  $\sigma.M.ker \leq \rho.M.ker$ ) :
  Filter.Tendsto
    (fun  $\alpha$  :  $\mathbb{R} \rightarrow ((\rho.M.conj (\sigma.M ^ ((1 - \alpha) / (2 * \alpha)))$ ).mat) ^  $\alpha$ ).trace.1

```

```

      (nhdsWithin 1 (Set.Ioi 0 \ {1}))
      (nhds [p.M, p.M.log - σ.M.log] := by
have h_log_approx : HasDerivAt (fun α : ℝ ↦ Real.log ((p.M.conj (σ.M ^ ((1 - α) /
      have h_deriv := sandwichedRelEntropy.hasDerivAt_trace_at_one h
      convert h_deriv.log (by simp) using 1
      simp
      rw [hasDerivAt_iff_tendsto_slope] at h_log_approx
      convert h_log_approx.mono_left (nhdsWithin_mono _ _) using 2
      · norm_num [div_eq_inv_mul, slope_def_field]
      · simp

theorem inner_log_sub_log_nonneg (h : σ.M.ker ≤ p.M.ker) :
  0 ≤ [p.M, p.M.log - σ.M.log] := by
  -- Take the limit  $\alpha \rightarrow 1+$  of the sandwiched Renyi relative entropy,
  -- which converges to  $[p.M, p.M.log - \sigma.M.log]$  and is nonneg for all  $\alpha > 1$ .
  -- Use the limit from the right: for  $\alpha > 1$ ,  $\log(\text{Tr}[\dots]^\alpha) / (\alpha - 1) \geq 0$ 
  have h_limit := sandwichedRelEntropy.limit_at_one p σ h
  -- Restrict the filter to  $(1, \infty) \subset (0, \infty) \setminus \{1\}$ 
  have h_mono : nhdsWithin (1 : ℝ) (Set.Ioi 1) ≤ nhdsWithin 1 (Set.Ioi 0 \ {1}) := b
    apply nhdsWithin_mono
    intro x hx
    exact (Set.mem_Ioi.mpr (lt_trans zero_lt_one hx), ne_of_gt hx)
  haveI : (nhdsWithin (1 : ℝ) (Set.Ioi 1)).NeBot := inferInstance
  apply ge_of_tendsto (h_limit.mono_left h_mono)
  filter_upwards [self_mem_nhdsWithin] with α hα
  exact sandwichedRelEntropy_nonneg_α_gt_1 h hα

theorem sandwichedRelEntropy_nonneg {α : ℝ} (hα : 0 < α) (h : σ.M.ker ≤ p.M.ker) :
  0 ≤ if α = 1 then [p.M, p.M.log - σ.M.log]
    else ((p.M.conj (σ.M ^ ((1 - α)/(2 * α))) ).mat) ^ α).trace.log / (α - 1) := by
  split_ifs with h1
  · exact inner_log_sub_log_nonneg h
  by_cases hα₂ : α > 1
  · exact sandwichedRelEntropy_nonneg_α_gt_1 h hα₂
  · have : α < 1 := by push_neg at hα₂; exact lt_of_le_of_ne hα₂ h1
    exact sandwichedRelEntropy_nonneg_α_lt_1 h hα this

section additivity

--TODO Cleanup. Ugh.

/--
If the kernels of the components are contained, then the kernel of the Kronecker pro
-/
lemma ker_kron_le_of_le {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂] [DecidableEq d₁] [
  (A C : Matrix d₁ d₁ ℂ) (B D : Matrix d₂ d₂ ℂ)

```

```

(hA : LinearMap.ker A.toEuclideanLin ≤ LinearMap.ker C.toEuclideanLin)
(hB : LinearMap.ker B.toEuclideanLin ≤ LinearMap.ker D.toEuclideanLin)
LinearMap.ker (A.kronecker B).toEuclideanLin ≤ LinearMap.ker (C.kronecker D).toEuclideanLin
intro x hx
simp only [Matrix.kronecker, LinearMap.mem_ker, Matrix.toLpLin_apply,
  WithLp.toLp_eq_zero] at hx ⊢
-- By definition of Kronecker product, we know that  $(A \otimes B)x = 0$ 
have h_kronecker : ∀ i j, ∑ k, A i k • ∑ l, B j l • x (k, l) = 0 := by
  intro i j
  replace hx := congr_fun hx (i, j)
  simp only [Matrix.mulVec, dotProduct, Matrix.kroneckerMap_apply,
    Pi.zero_apply, smul_eq_mul, Finset.mul_sum] at hx ⊢
  rw [← Finset.sum_product]
  simp only [mul_assoc, Finset.univ_product_univ] using hx
-- Apply the hypothesis `hA` to each term in the sum.
have h_apply_hA : ∀ i j, ∑ k, C i k • ∑ l, B j l • x (k, l) = 0 := by
  intro i j
  specialize hA (show (WithLp.toLp 2 (fun k => ∑ l, B j l • x (k, l)))
    · simp_all only [smul_eq_mul, LinearMap.mem_ker]
    ext i_1 : 1
    simp_all only [PiLp.zero_apply]
    apply h_kronecker
    · exact congr(WithLp.ofLp $hA i)
  ext (i, j)
  simp only [smul_eq_mul, Matrix.mulVec, dotProduct, Matrix.kroneckerMap_apply,
    Pi.zero_apply] at h_kronecker h_apply_hA ⊢
  have h_apply_hB : ∑ l, D j l • ∑ k, C i k • x (k, l) = 0 := by
    specialize hB
    simp_all only [funext_iff, Pi.zero_apply, Prod.forall, smul_eq_mul]
    have := hB (show (WithLp.toLp 2 (fun l => ∑ k, C i k • x (k, l)))
      · simp_all only [LinearMap.mem_ker] ;
      exact congr(WithLp.ofLp $this j)
    · ext j
      specialize h_apply_hA i j
      simp [Matrix.mulVec, dotProduct, Finset.mul_sum] at h_apply_hA ⊢
      simp_rw [mul_left_comm]
      rw [Finset.sum_comm]
      exact h_apply_hA
  rw [← h_apply_hB]
  simp only [smul_eq_mul, Finset.mul_sum]
  rw [Finset.sum_sigma];
  refine' Finset.sum_bij (fun x _ => (x.2, x.1)) _ _ _ _
  · simp
  · simp
  · simp
  · simp only [Finset.mem_univ, mul_assoc, Prod.mk.eta, mul_left_comm, imp_

```

```

--TODO: Generalize to arbitrary PSD matrices.
/--
If the kernel of a product state is contained in another, the left component kernel
-/
lemma ker_le_of_ker_kron_le_left (ρ₁ σ₁ : MState d₁) (ρ₂ σ₂ : MState d₂)
  (h : (σ₁ ⊗M σ₂).M.ker ≤ (ρ₁ ⊗M ρ₂).M.ker) :
  σ₁.M.ker ≤ ρ₁.M.ker := by
  intro u hu
  obtain ⟨v, hv⟩ : ∃ v : EuclideanSpace ℂ d₂, v ∉ (σ₂ : HermitianMat d₂ ℂ).ker ∧ v ∉
    have h_union : (σ₂ : HermitianMat d₂ ℂ).ker ≠ ⊤ ∧ (ρ₂ : HermitianMat d₂ ℂ).ker ≠ ⊤
    constructor <=> intro h_top;
    · have h_contra : σ₂.M = 0 := by
      ext1
      simp_all [ Submodule.eq_top_iff' ];
      ext i j
      specialize h_top ( EuclideanSpace.single j 1 )
      simp_all
      replace h_top := congr(WithLp.ofLp $h_top i)
      simp_all
      convert h_top using 1;
      erw [ Matrix.toLpLin_apply ]
      simp_all only [MState.mat_M, EuclideanSpace.ofLp_single, Matrix.mulVec_sin
        MulOpposite.op_one, Pi.smul_apply, Matrix.col_apply, one_smul]
      exact σ₂.pos.ne' h_contra;
    · have h_contra : ρ₂.M = 0 := by
      ext i j; simp_all [ Submodule.eq_top_iff' ] ;
      convert congr(WithLp.ofLp $(h_top (WithLp.toLp 2 ( Pi.single j 1 )) ) i) u
      simp
      simp [ HermitianMat.lin ];
      rfl
      exact ρ₂.pos.ne' h_contra;
  have h_union : ∀ (U V : Submodule ℂ (EuclideanSpace ℂ d₂)), U ≠ ⊤ → V ≠ ⊤ → ∃ v
    intros U V hU hV;
    by_contra h_contra;
    have h_union : U ⊔ V = ⊤ := by
      ext v
      simp only [Submodule.mem_top, iff_true]
      by_cases hvU : v ∈ U <=> by_cases hvV : v ∈ V <=> simp_all [ Submodule.mem_s
      · exact ⟨ v, hvU, 0, by simp, by simp ⟩;
      · exact ⟨ v, hvU, 0, by simp, by simp ⟩;
      · exact ⟨ 0, by simp, v, h_contra v hvU, by simp ⟩;
  have h_union : ∃ v : EuclideanSpace ℂ d₂, v ∉ U ∧ v ∈ V := by
    have h_union : ∃ v : EuclideanSpace ℂ d₂, v ∈ V ∧ v ∉ U := by
      have h_not_subset : ¬V ≤ U := by
        exact fun h => hU <| by rw [ eq_top_iff ] ; exact h_union ▸ sup_le ( by

```

```

    exact Set.not_subset.mp h_not_subset;
    exact { h_union.choose, h_union.choose_spec.2, h_union.choose_spec.1 };
    obtain { v, hv1, hv2 } := h_union;
    obtain { w, hw1, hw2 } :  $\exists w : \text{EuclideanSpace } \mathbb{C} \ d_2, w \notin V \wedge w \in U :=$ 
      obtain { w, hw } := ( show  $\exists w : \text{EuclideanSpace } \mathbb{C} \ d_2, w \notin V$  from by
        exact Classical.not_not.1 fun hw' => hw <| h_contra _ hw' );
    have h_union :  $v + w \notin U \wedge v + w \notin V :=$  by
      exact { fun h => hv1 <| by simp using U.sub_mem h hw2, fun h => hw2 <| h_contra { v + w, h_union.1, h_union.2 } };
    exact h_union _ _ ( by tauto ) ( by tauto );
-- Consider  $z = u \otimes v$ .
set z : EuclideanSpace  $\mathbb{C} \ (d_1 \times d_2) :=$  WithLp.toLp 2 ( fun p => u p.1 * v p.2 );
have hz :  $z \in (\sigma_1 \otimes^M \sigma_2 : \text{HermitianMat } (d_1 \times d_2) \ \mathbb{C}).\text{ker} :=$  by
  ext { i, j }
  simp [z]
  have h_kronecker :  $\forall (A : \text{Matrix } d_1 \ d_1 \ \mathbb{C}) (B : \text{Matrix } d_2 \ d_2 \ \mathbb{C}) (u : d_1 \rightarrow \mathbb{C}) (v : d_2 \rightarrow \mathbb{C})$ 
    intro A B u v; ext { i, j }; simp [ Matrix.mulVec, dotProduct, Finset.sum, Finset.sum_prod_type_right ];
  exact Fintype.sum_prod_type_right fun x => A i x.1 * (B j x.2 * (u x.1 * v x.2 ));
  convert congr_fun ( h_kronecker  $\sigma_1.1.\text{mat } \sigma_2.1.\text{mat } u \ v$  ) ( i, j ) using
  exact Or.inl ( by simp [ Matrix.mulVec ] using congr(WithLp.ofLp $hu i) );
have hz' :  $z \in (\rho_1 \otimes^M \rho_2 : \text{HermitianMat } (d_1 \times d_2) \ \mathbb{C}).\text{ker} :=$  by
  exact h hz;
have hz'' :  $\forall a \ b, (\rho_1.M.\text{val}.\text{mulVec } u) \ a * (\rho_2.M.\text{val}.\text{mulVec } v) \ b = 0 :=$  by
  intro a b
  have hz'' :  $(\rho_1.M.\text{val}.\text{mulVec } u) \ a * (\rho_2.M.\text{val}.\text{mulVec } v) \ b = ((\rho_1 \otimes^M \rho_2).M.\text{val}.\text{mulVec } z) \ (a \ b)$ 
    simp [ Matrix.mulVec, dotProduct ];
  simp [ Finset.sum_mul, mul_assoc, mul_comm ];
  simp [ z, Finset.mul_sum, mul_assoc, mul_left_comm ];
  erw [ Finset.sum_product ]; simp
  exact rfl;
  exact hz''.trans ( by simp using congr(WithLp.ofLp $hz' ( a, b )) );
ext a; specialize hz'' a; simp_all [ Matrix.mulVec, dotProduct ];
contrapose! hv;
intro hv'; ext b; specialize hz'' b; simp_all
exact hz''.resolve_left ( by simp [ Matrix.mulVec, dotProduct ] using hv' )

--TODO: Generalize to arbitrary PSD matrices.
--TODO: Rewrite the proof using the `ker_le_of_ker_kron_le_left` lemma, and
-- there's a unitary whose conjugation swaps the kronecker product.
/--
If the kernel of a product state is contained in another, the right component
is also contained in the other.
-/
lemma ker_le_of_ker_kron_le_right ( $\rho_1 \ \sigma_1 : \text{MState } d_1$ ) ( $\rho_2 \ \sigma_2 : \text{MState } d_2$ )
  (h :  $(\sigma_1 \otimes^M \sigma_2).M.\text{ker} \leq (\rho_1 \otimes^M \rho_2).M.\text{ker}$ ) :
   $\sigma_2.M.\text{ker} \leq \rho_2.M.\text{ker} :=$  by

```



```

intro v hv;
have h_z :  $\exists u : \text{EuclideanSpace } \mathbb{C} \ d_1, u \neq 0 \wedge u \notin \sigma_1.M.\text{ker} \wedge u \notin \rho_1.M.\text{ker} := by
  have h_z :  $\sigma_1.M.\text{ker} \neq \top \wedge \rho_1.M.\text{ker} \neq \top := by
    have h_ker_ne_top :  $\forall (\rho : \text{MState } d_1), \rho.M.\text{ker} \neq \top := by
      intro \rho hp_top
      have h_contra :  $\rho.M = 0 := by
        ext i j
        simp_all [ Submodule.eq_top_iff' ] ;
        convert congr(WithLp.ofLp $(hp_top ( EuclideanSpace.single j 1 ) ) i) using
        simp
        erw [ Matrix.toLpLin_apply ]
        aesop
      exact \rho.pos.ne' h_contra;
    exact ( h_ker_ne_top \sigma_1, h_ker_ne_top \rho_1 );
  have h_z :  $\exists u : \text{EuclideanSpace } \mathbb{C} \ d_1, u \notin \sigma_1.M.\text{ker} \wedge u \notin \rho_1.M.\text{ker} := by
    have h_z :  $\forall (U V : \text{Submodule } \mathbb{C} \ (\text{EuclideanSpace } \mathbb{C} \ d_1)), U \neq \top \rightarrow V \neq \top \rightarrow \exists u :
      intro U V hU hV
      by_contra h_contra
      push_neg at h_contra;
      have h_union :  $\exists u : \text{EuclideanSpace } \mathbb{C} \ d_1, u \notin U \wedge u \in V := by
        exact Exists.elim ( show  $\exists u : \text{EuclideanSpace } \mathbb{C} \ d_1, u \notin U$  from by simpa [
          obtain ( u, hu_1, hu_2 ) := h_union;
          have h_union :  $\forall v : \text{EuclideanSpace } \mathbb{C} \ d_1, v \in U \rightarrow v + u \in V := by
            intro v hv; specialize h_contra ( v + u ) ; simp_all [ Submodule.add_mem_i
          have h_union :  $\forall v : \text{EuclideanSpace } \mathbb{C} \ d_1, v \in U \rightarrow v \in V := by
            exact fun v hv => by simpa using V.sub_mem ( h_union v hv ) hu_2;
            exact hV ( eq_top_iff.mpr fun x hx => by by_cases hxU : x \in U <=> aesop );
            exact h_z _ _ ( by tauto ) ( by tauto );
            exact ( h_z.choose, by intro h; simpa [ h ] using h_z.choose_spec.1, h_z.choose_
          obtain ( u, hu_1, hu_2, hu_3 ) := h_z;
          -- Consider the vector $z = u \otimes v$.
          set z :  $\text{EuclideanSpace } \mathbb{C} \ (d_1 \times d_2) := .toLp \ 2 \ ( fun p => u \ p.1 * v \ p.2 );
          have hz :  $z \in (\sigma_1 \otimes^M \sigma_2).M.\text{ker} := by
            -- By definition of $z$, we have  $(\sigma_1 \otimes \sigma_2).mat.mulVec \ z = \sigma_1.mat.mulVec \ u \otimes \sigma_2.$ 
            have hz_mul :  $(\sigma_1 \otimes^M \sigma_2).M.mat.mulVec \ z = fun p => (\sigma_1.M.mat.mulVec \ u) \ p.1 * (\sigma_2.$ 
              ext p; simp [z, Matrix.mulVec]
              simp [ dotProduct, Finset.mul_sum, Finset.sum_mul, mul_assoc, mul_comm, mul_le
              rw [ ← Finset.sum_product' ];
              refine' Finset.sum_bij ( fun x _ => ( x.2, x.1 ) ) _ _ _ _ <=> simp;
              exact fun a b => Or.inl <| Or.inl <| rfl;
            simp_all [ funext_iff, Matrix.mulVec ];
            ext ( a, b )
            specialize hz_mul a b
            simp_all [ dotProduct]
            convert hz_mul using 1;
            simp_all only [zero_eq_mul]$$$$$$$$$$$ 
```

```

    exact Or.inr ( by simp [ Matrix.mulVec, dotProduct ] using congr(WithL
have hz' : z ∈ (ρ1 ⊗M ρ2).M.ker := by
  exact h hz;
have hz'' : ∀ i j, (ρ1.M.val.mulVec u) i * (ρ2.M.val.mulVec v) j = 0 := b
  intro i j;
  have hz''' : (ρ1.M.val.kronecker ρ2.M.val).mulVec (fun p => u p.1 * v p.
    simp [ Matrix.mulVec, dotProduct, Finset.mul_sum, mul_assoc, mul_comm
    simp [ mul_assoc, Finset.mul_sum, Finset.sum_mul ];
    rw [ ← Finset.sum_product' ];
    refine' Finset.sum_bij ( fun x _ => ( x.2, x.1 ) ) _ _ _ _
      · simp
      · simp
      · simp
      · intro _ _
        simp only [MState.m, HermitianMat.mat_apply]
        ring_nf
    exact hz'''.symm.trans ( by simp using congr(WithLp.ofLp $hz' ( i, j ))
contrapose! hz''';
obtain ⟨ i, hi ⟩ := Function.ne_iff.mp ( show ρ1.M.val.mulVec u ≠ 0 from
obtain ⟨ j, hj ⟩ := Function.ne_iff.mp ( show ρ2.M.val.mulVec v ≠ 0 from
use i, j
aesop;

/--
The kernel of a product state is contained in another product state's kerne
kernels are contained.
-/
lemma ker_prod_le_iff (ρ1 σ1 : MState d1) (ρ2 σ2 : MState d2) :
  (σ1 ⊗M σ2).M.ker ≤ (ρ1 ⊗M ρ2).M.ker ↔ σ1.M.ker ≤ ρ1.M.ker ∧ σ2.M.ker ≤
  constructor <;> intro h;
  · exact ⟨ ker_le_of_ker_kron_le_left ρ1 σ1 ρ2 σ2 h, ker_le_of_ker_kron_le
  · convert ker_kron_le_of_le _ _ _ _ h.1 h.2 using 1

--TODO: Generalize to RCLike.
omit [DecidableEq d1] [DecidableEq d2] in
lemma HermitianMat.inner_kron
  (A : HermitianMat d1 ℂ) (B : HermitianMat d2 ℂ) (C : HermitianMat d1 ℂ)
  [A ⊗k B, C ⊗k D] = [A, C] * [B, D] := by
  -- Apply the property of the trace of Kronecker products.
  have h_trace_kron : ∀ (A1 B1 : Matrix d1 d1 ℂ) (A2 B2 : Matrix d2 d2 ℂ),
    intro A1 B1 A2 B2
    rw [← Matrix.mul_kronecker_mul, Matrix.trace_kronecker]
  simp_all only [inner, IsMaximalSelfAdjoint.RCLike_selfadjMap, kronecker_m
    RCLike.re_to_complex, RCLike.im_to_complex, sub_eq_self, mul_eq_zero];
  simp only [Matrix.trace, Matrix.diag_apply, Matrix.mul_apply, mat_apply,
    Complex.mul_im];

```

```

left;
have h_symm : ∀ x x_1, (A x x_1).re * (C x_1 x).im + (A x x_1).im * (C x_1 x).re =
((A x_1 x).re * (C x x_1).im + (A x_1 x).im * (C x x_1).re) := by
  intro x y; have := congr_fun (congr_fun A.2 y) x; have := congr_fun (congr_fun
  grind;
have h_sum_zero : ∑ x, ∑ x_1, ((A x x_1).re * (C x_1 x).im + (A x x_1).im * (C x_1
((A x x_1).re * (C x_1 x).im + (A x x_1).im * (C x_1 x).re) := by
  rw [Finset.sum_comm];
  exact Finset.sum_congr rfl fun _ => Finset.sum_congr rfl fun _ => h_symm _
  norm_num [Finset.sum_add_distrib] at *; linarith

attribute [fun_prop] ContinuousAt.rpow

lemma continuousOn_rpow_uniform {K : Set ℝ} (hK : IsCompact K) :
  ContinuousOn (fun r : ℝ ↦ UniformOnFun.ofFun {K} (fun t : ℝ ↦ t ^ r)) (Set.Ioi 0)
  refine continuousOn_of_forall_continuousAt fun r hr => ?_
  rw [Set.mem_Ioi] at hr
  apply UniformOnFun.tendsto_iff_tendstoUniformlyOn.mpr
  simp only [Set.mem_singleton_iff, UniformOnFun.toFun_ofFun, Metric.tendstoUniforml
  Function.comp_apply, forall_eq]
  intro ε hεpos;
  have h_unif_cont : UniformContinuousOn (fun (p : ℝ × ℝ) => p.1 ^ p.2) (K ×s Set.Ic
  apply IsCompact.uniformContinuousOn_of_continuous
  · exact hK.prod CompactIccSpace.isCompact_Icc
  · refine continuousOn_of_forall_continuousAt fun p {hp1, {hp21, hp22}} ↦ ?_
    have _ : p.1 ≠ 0 ∨ 0 < p.2 := by right; linarith
    fun_prop (disch := assumption)
  rw [Metric.uniformContinuousOn_iff] at h_unif_cont
  obtain {δ, hδpos, H} := h_unif_cont ε hεpos
  filter_upwards [Ioo_mem_nhds (show r / 2 < r by linarith) (show r < r * 2 by linarith)]
  refine H (x, r) {hx, ?_} (x, n) {hx, ?_} ?_
  · constructor <=> linarith
  · constructor <=> linarith
  · have : |r - n| < δ := abs_lt.mpr {by linarith, by linarith}
    simp

theorem sandwichedRelEntropy_additive_alpha_one_aux (ρ1 σ1 : MState d1) (ρ2 σ2 : MS
(h1 : σ1.M.ker ≤ ρ1.M.ker) (h2 : σ2.M.ker ≤ ρ2.M.ker) :
  [(ρ1 ⊗M ρ2).M, (ρ1 ⊗M ρ2).M.log - (σ1 ⊗M σ2).M.log] =
  [ρ1.M, ρ1.M.log - σ1.M.log]_ℝ + [ρ2.M, ρ2.M.log - σ2.M.log] := by
  have h_log_kron : (ρ1 ⊗M ρ2).M.log = ρ1.M.log ⊗k ρ2.M.supportProj + ρ1.M.supportPr
  constructor <=> apply HermitianMat.log_kron_with_proj;
  have h_inner_supportProj : ∀ (A : HermitianMat d1 ℂ) (B : HermitianMat d2 ℂ), [A ⊗
  exact fun A B => HermitianMat.inner_kron A B ρ1 ρ2;
  simp only [HermitianMat.ker] at h1 h2
  simp_all only [inner_sub_right, inner_add_right, real_inner_comm,

```

```

    HermitianMat.inner_supportProj_self, MState.tr, mul_one, one_mul,
    HermitianMat.inner_supportProj_of_ker_le]
  abel

/-- The Sandwiched Renyi Relative Entropy, defined with ln (nits). Note the
switch to the standard Relative Entropy, for continuity. For  $\alpha \leq 0$ , this
is no conventional value for  $\alpha < 0$ ; there is a continuous limit at  $\alpha = 0$ ,
unnneeded at the moment.)-/
def SandwichedRelRentropy ( $\alpha : \mathbb{R}$ ) ( $\rho \ \sigma : \text{MState } d$ ) : ENNReal :=
  open Classical in
  if h $\alpha$  :  $0 < \alpha$  then
    if h :  $\sigma.M.ker \leq \rho.M.ker$ 
    then (.ofNNReal (if  $\alpha = 1$  then
       $\lfloor \rho.M, \rho.M.log - \sigma.M.log \rfloor$ 
    else
       $((\rho.M.conj (\sigma.M ^ ((1 - \alpha)/(2 * \alpha))) .mat) ^ \alpha).trace.log / (\alpha - 1)$ 
      sandwichedRelRentropy_nonneg h $\alpha$  h))
    else  $\top$ 
  else 0

notation "D~_"  $\alpha$  "("  $\rho$  "||"  $\sigma$  ")" => SandwichedRelRentropy  $\alpha$   $\rho$   $\sigma$ 

/-- The quantum relative entropy  $\lfloor (\rho || \sigma) := \text{Tr}[\rho (\log \rho - \log \sigma)] \rfloor$ . Also call
the Umegaki quantum relative entropy, when it's necessary to distinguish from
relative entropies. -/
def qRelativeEnt ( $\rho \ \sigma : \text{MState } d$ ) : ENNReal :=
  D~_1( $\rho || \sigma$ )

notation " $\lfloor$  ("  $\rho$  "||"  $\sigma$  ")" => qRelativeEnt  $\rho$   $\sigma$ 

/--
The Sandwiched Renyi Relative entropy is additive for  $\alpha=1$  (standard relative
-/
private theorem sandwichedRelRentropy_additive_alpha_one ( $\rho_1 \ \sigma_1 : \text{MState } d_1$ 
  D~_1( $\rho_1 \otimes^M \rho_2 || \sigma_1 \otimes^M \sigma_2$ ) = D~_1( $\rho_1 || \sigma_1$ ) + D~_1( $\rho_2 || \sigma_2$ ) := by
  by_cases h1 :  $\sigma_1.M.ker \leq \rho_1.M.ker$ 
  <=> by_cases h2 :  $\sigma_2.M.ker \leq \rho_2.M.ker$ 
  · simp only [SandwichedRelRentropy,  $\downarrow$ reduceIte,  $\downarrow$ reduceDIte, h1, h2]
    split_ifs <=> simp_all [ker_prod_le_iff ];
    simp only [sandwichedRelRentropy_additive_alpha_one_aux  $\rho_1 \ \sigma_1 \ \rho_2 \ \sigma_2$  h1
      rfl
    · simp only [SandwichedRelRentropy, zero_lt_one,  $\downarrow$ reduceDIte,  $\downarrow$ reduceIte,
      add_top, dite_eq_right_iff, ENNReal.coe_ne_top, imp_false]
      have := ker_prod_le_iff  $\rho_1 \ \sigma_1 \ \rho_2 \ \sigma_2$ 
      tauto
    · simp only [SandwichedRelRentropy, zero_lt_one,  $\downarrow$ reduceDIte,  $\downarrow$ reduceIte,
```

```

    top_add, dite_eq_right_iff, ENNReal.coe_ne_top, imp_false]
  contrapose! h1
  exact (ker_le_of_ker_kron_le_left ρ₁ σ₁ ρ₂ σ₂) h1
  · simp only [SandwichedRelEntropy, zero_lt_one, ↓reduceDite, ↓reduceIte, h1, h2,
    add_top, dite_eq_right_iff, ENNReal.coe_ne_top, imp_false]
  contrapose! h1
  exact (ker_le_of_ker_kron_le_left ρ₁ σ₁ ρ₂ σ₂) h1

lemma sandwiched_term_product (ρ₁ σ₁ : MState d₁) (ρ₂ σ₂ : MState d₂) (α β : ℝ) :
  (((ρ₁ ⊗M ρ₂).M.conj ((σ₁ ⊗M σ₂).M ^ β).mat) ^ α).trace =
  ((ρ₁.M.conj (σ₁.M ^ β).mat) ^ α).trace * ((ρ₂.M.conj (σ₂.M ^ β).mat) ^ α).trace
  simp only [MState.prod]
  rw [← HermitianMat.trace_kronecker]
  rw [← HermitianMat.rpow_kron α ?_ ?_, ← HermitianMat.conj_kron,
    HermitianMat.rpow_kron β σ₁.nonneg σ₂.nonneg, HermitianMat.kronecker_mat]
  · exact HermitianMat.conj_nonneg _ ρ₁.nonneg
  · exact HermitianMat.conj_nonneg _ ρ₂.nonneg

/-
The Sandwiched Renyi Relative entropy is additive for alpha != 1.
-/
theorem sandwichedRelEntropy_additive_alpha_ne_one {α : ℝ} (hα : α ≠ 1) (ρ₁ σ₁ : MS
   $\tilde{D}_\alpha(\rho_1 \otimes^M \rho_2 \| \sigma_1 \otimes^M \sigma_2) = \tilde{D}_\alpha(\rho_1 \| \sigma_1) + \tilde{D}_\alpha(\rho_2 \| \sigma_2)$  := by
  by_cases hα0 : 0 < α; swap
  · simp [SandwichedRelEntropy, hα0]
  by_cases h_ker : σ₁.M.ker ≤ ρ₁.M.ker ∧ σ₂.M.ker ≤ ρ₂.M.ker
  · simp_all [SandwichedRelEntropy]
    -- Apply the additivity of the trace term to split the logarithm into the sum of
    have h_trace_add : Real.log ((ρ₁ ⊗M ρ₂).M.conj ((σ₁ ⊗M σ₂).M ^ ((1 - α) / (2 * α)))
      rw [sandwiched_term_product, Real.log_mul ];
      · exact (sandwiched_trace_pos h_ker.1).ne'
      · exact (sandwiched_trace_pos h_ker.2).ne'
    split_ifs <=> simp_all
    · norm_num [ add_div ];
      exact rfl;
    · exact False.elim ( <¬ ( σ₁ ⊗M σ₂ |> MState.M |> HermitianMat.ker ) ≤ ( ρ₁ ⊗M ρ₂ |> MState.M |> HermitianMat.ker ) > )
  · have h_ker_prod : ¬((σ₁ ⊗M σ₂).M.ker ≤ (ρ₁ ⊗M ρ₂).M.ker) := by
      simp_all [ ker_prod_le_iff ]
    rw [not_and_or] at h_ker
    rcases h_ker with h_ker | h_ker
    · simp [SandwichedRelEntropy, h_ker_prod, h_ker, hα0]
    · simp [SandwichedRelEntropy, h_ker_prod, h_ker, hα0]

end additivity

/-- The Sandwiched Renyi Relative entropy is additive when the inputs are product st

```

```

/
@[simp]
theorem sandwichedRelEntropy_additive (α) (ρ1 σ1 : MState d1) (ρ2 σ2 : MState d2) :
   $\tilde{D}_\alpha(\rho_1 \otimes^M \rho_2 \| \sigma_1 \otimes^M \sigma_2) = \tilde{D}_\alpha(\rho_1 \| \sigma_1) + \tilde{D}_\alpha(\rho_2 \| \sigma_2) :=$  by
  rcases eq_or_ne α 1 with rfl | hα
  · exact sandwichedRelEntropy_additive_alpha_one ρ1 σ1 ρ2 σ2
  · apply sandwichedRelEntropy_additive_alpha_ne_one hα

/-- The quantum relative entropy is additive when the inputs are product states -/
/
@[simp]
theorem qRelativeEnt_additive (ρ1 σ1 : MState d1) (ρ2 σ2 : MState d2) :
   $\square(\rho_1 \otimes^M \rho_2 \| \sigma_1 \otimes^M \sigma_2) = \square(\rho_1 \| \sigma_1) + \square(\rho_2 \| \sigma_2) :=$  by
  --or `simp [SandwichedRelEntropy]`.
  exact sandwichedRelEntropy_additive_alpha_one ρ1 σ1 ρ2 σ2

@[simp]
theorem sandwichedRelEntropy_relabel (ρ σ : MState d) (e : d2 ≈ d) :
   $\tilde{D}_\alpha(\rho.\text{relabel } e \| \sigma.\text{relabel } e) = \tilde{D}_\alpha(\rho \| \sigma) :=$  by
  simp only [SandwichedRelEntropy, MState.relabel_M]
  rw! [HermitianMat.ker_reindex_le_iff] --Why doesn't this `simp`? Because
  simp [HermitianMat.conj_submatrix]

@[simp]
theorem sandwichedRelEntropy_self (hα : 0 < α) (ρ : MState d) :
  --Technically this holds for all α except for '-1' and '0'. But those are
  --TODO: Maybe SandwichedRelEntropy should actually be defined differently
   $\tilde{D}_\alpha(\rho \| \rho) = 0 :=$  by
  simp? [SandwichedRelEntropy, NNReal.eq_iff, hα] says
  simp only [SandwichedRelEntropy, hα, ↑reduceDite, le_refl, sub_self, i
    ENNReal.coe_eq_zero, NNReal.eq_iff, NNReal.coe_mk, NNReal.coe_zero, i
    div_eq_zero_iff, Real.log_eq_zero]
  intro hα
  left; right; left
  rw [HermitianMat.rpow_eq_cfc, HermitianMat.rpow_eq_cfc]
  nth_rw 2 [← HermitianMat.cfc_id ρ.M]
  rw [HermitianMat.cfc_conj, ← HermitianMat.cfc_comp]
  conv =>
  enter [1, 1]
  equals ρ.M.cfc id =>
  apply HermitianMat.cfc_congr_of_nonneg ρ.nonneg
  intro i (hi : 0 ≤ i)
  simp
  rw [← Real.rpow_mul_natCast hi, ← Real.rpow_one_add' hi]
  · rw [← Real.rpow_mul hi]
  field_simp

```

```

      ring_nf
      exact Real.rpow_one i
    · field_simp; ring_nf; positivity
  simp

@[aesop (rule_sets := [finiteness]) unsafe apply]
theorem sandwichedRelEntropy_ne_top {p σ : MState d} [σ.M.NonSingular] :  $\tilde{D}_\alpha(\rho||\sigma) < \infty$ 
  by_cases 0 < α
  · simp [SandwichedRelEntropy, HermitianMat.nonSingular_ker_bot, *]
  · simp [SandwichedRelEntropy, *]

@[fun_prop]
lemma continuousOn_exponent : ContinuousOn (fun α : ℝ => (1 - α) / (2 * α)) (Set.Ioi 1) := by
  fun_prop (disch := intros; linarith [Set.mem_Ioi.mp <_>])

@[fun_prop]
lemma Complex.continuousOn_cpow_const_Ioi (z : ℂ) :
  ContinuousOn (fun r : ℝ => z ^ (r : ℂ)) (Set.Ioi 0) := by
  apply ContinuousOn.const_cpow (f := Complex.ofReal)
  · fun_prop
  · grind [ofReal_ne_zero]

/--
The function  $\alpha \mapsto (1 - \alpha) / (2 * \alpha)$  maps the interval  $(1, \infty)$  to  $(-\infty, 0)$ .
-/
lemma maps_to_Iio_of_Ioi_1 : Set.Mapsto (fun α : ℝ => (1 - α) / (2 * α)) (Set.Ioi 1) (Set.Iio 0) := by
  intro x hx
  rw [Set.mem_Ioi] at hx
  rw [Set.mem_Iio]
  have h1 : 1 - x < 0 := by linarith
  have h2 : 0 < 2 * x := by linarith
  exact div_neg_of_neg_of_pos h1 h2

--PR'ed: #35494
@[simp]
theorem frontier_singleton {X : Type*} [TopologicalSpace X] [T1Space X] [PerfectSpace X]
  (p : X) : frontier {p} = {p} := by
  simp [frontier]

private theorem sandwichedRelEntropy.continuousOn_Ioi_1_aux (p σ : MState d) :
  ContinuousOn (fun (α : ℝ) => ((HermitianMat.conj (σ.M ^ ((1 - α) / (2 * α)))).mat) (p)) (Set.Ioi 1) := by
  have h_cont : ContinuousOn (fun α : ℝ => (HermitianMat.conj (σ.M ^ ((1 - α) / (2 * α)))).mat) (Set.Ioi 1) := by
    have h_cont : ContinuousOn (fun α : ℝ => (σ.M ^ ((1 - α) / (2 * α)))).mat (Set.Ioi 1) := by
      have h_cont : ContinuousOn (fun α : ℝ => σ.M ^ ((1 - α) / (2 * α))) (Set.Ioi 1) := by
        have h_cont : ContinuousOn (fun α : ℝ => (1 - α) / (2 * α)) (Set.Ioi 1) := by

```

```

    exact continuousOn_of_forall_continuousAt fun x hx => ContinuousA
    have h_cont : ContinuousOn (fun  $\alpha : \mathbb{R} \Rightarrow (\sigma.M ^ \alpha)$ ) (Set.Ioi 0) :=
      apply_rules [ HermitianMat.continuousOn_rpow_neg ];
    exact h_cont.comp <_> fun x hx => by rw [ Set.mem_Ioi ] ; rw [ div_
    exact Continuous.comp_continuousOn ( by continuity ) h_cont;
  fun_prop;
-- Apply the lemma HermitianMat.continuousOn_rpow_joint_nonneg_pos with t
apply HermitianMat.continuousOn_rpow_joint_nonneg_pos;
· exact h_cont;
· exact continuousOn_id;
· exact fun x hx => zero_lt_one.trans hx;

private theorem sandwichedRelEntropy.continuousOn_Ioi_1 ( $\rho \ \sigma : \text{MState } d$ ) :
  ContinuousOn (fun  $\alpha \Rightarrow \check{D}_\alpha(\rho||\sigma)$ ) (Set.Ioi 1) := by
  dsimp [SandwichedRelEntropy]
  split_ifs with hp
  · simp [← ENNReal.ofReal_eq_coe_nnreal]
    rw [continuousOn_congr (f := fun  $\alpha \mapsto \text{ENNReal.ofReal}$ 
      (Real.log ((HermitianMat.conj ( $\sigma.M ^ ((1 - \alpha) / (2 * \alpha)))$ )).mat)  $\rho.M ^$ 
    · apply (ENNReal.continuous_ofReal).comp_continuousOn
      apply ContinuousOn.div_
    · apply ContinuousOn.log
      · exact HermitianMat.trace_Continuous.comp_continuousOn
        (continuousOn_Ioi_1_aux  $\rho \ \sigma$ )
      · intro x hx
        apply LT.lt.ne'
        grw [← sandwiched_trace_of_gt_1 hp hx]
        exact zero_lt_one
    · fun_prop
    · clear hp; grind
  · intro  $\alpha$  (h $\alpha : 1 < \alpha$ )
    dsimp only
    rw [if_pos (zero_lt_one.trans h $\alpha$ ), if_neg h $\alpha$ .ne']
  · rw [continuousOn_congr (f := fun  $\alpha \mapsto \tau$ )]
  · fun_prop
  · clear  $\rho \ \sigma$  hp;
    grind only [→ Set.EqOn.eq_of_mem, = Set.mem_Ioi, Set.EqOn, cases Or]

private theorem sandwichedRelEntropy.continuousOn_Ioi_0_1_aux ( $\rho \ \sigma : \text{MStat}$ 
  ContinuousOn (fun ( $\alpha : \mathbb{R}$ )  $\mapsto ((\text{HermitianMat.conj } (\sigma.M ^ ((1 - \alpha) / (2 *
  have h_cont : ContinuousOn (fun  $\alpha : \mathbb{R} \Rightarrow (\text{HermitianMat.conj } (\sigma.M ^ ((1 -
  have h_cont : ContinuousOn (fun  $\alpha : \mathbb{R} \Rightarrow (\sigma.M ^ ((1 - \alpha) / (2 * \alpha)))$ ).ma
  have h_cont : ContinuousOn (fun  $\alpha : \mathbb{R} \Rightarrow \sigma.M ^ ((1 - \alpha) / (2 * \alpha)))$  (
    have h_exp_cont : ContinuousOn (fun  $\alpha : \mathbb{R} \Rightarrow (1 - \alpha) / (2 * \alpha)$ ) (Se
    exact continuousOn_of_forall_continuousAt fun x hx => ContinuousA
  have h_rpow_cont : ContinuousOn (fun  $\alpha : \mathbb{R} \Rightarrow (\sigma.M ^ \alpha)$ ) (Set.Ioi 0$$ 
```



```

        apply_rules [ HermitianMat.continuousOn_rpow_pos ]
        exact h_rpow_cont.comp h_exp_cont fun x hx => by rw [ Set.mem_Ioi ] ; apply
        exact Continuous.comp_continuousOn ( by continuity ) h_cont
    fun_prop
  apply HermitianMat.continuousOn_rpow_joint_nonneg_pos
  · exact h_cont
  · exact continuousOn_id
  · exact fun x hx => hx.1

/-- Continuity on (0,1): the sandwich relative Rényi entropy is continuous in  $\alpha$  on (
/
private theorem sandwichedRelRentropy.continuousOn_Ioo_0_1 (p  $\sigma$  : MState d) :
  ContinuousOn (fun  $\alpha$  =>  $\tilde{D}_\alpha(p||\sigma)$ ) (Set.Ioo 0 1) := by
  dsimp [SandwichedRelRentropy]
  split_ifs with hp
  · simp [← ENNReal.ofReal_eq_coe_nnreal]
    rw [continuousOn_congr (f := fun  $\alpha$   $\mapsto$  ENNReal.ofReal
      (Real.log ((HermitianMat.conj ( $\sigma.M$  ^ ((1 -  $\alpha$ ) / (2 *  $\alpha$ ))).mat) p.M ^  $\alpha$ ).trace
    · apply (ENNReal.continuous_ofReal).comp_continuousOn
      apply ContinuousOn.div
    · apply ContinuousOn.log
      · exact HermitianMat.trace_Continuous.comp_continuousOn
        (continuousOn_Ioo_0_1_aux p  $\sigma$ )
      · intro x hx
        exact (sandwiched_trace_pos hp).ne'
    · fun_prop
    · intro x hx; exact sub_ne_zero.mpr (ne_of_lt hx.2)
  · intro  $\alpha$  h $\alpha$ 
    dsimp only
    rw [if_pos h $\alpha$ .1, if_neg (ne_of_lt h $\alpha$ .2)]
  · rw [continuousOn_congr (f := fun  $\alpha$   $\mapsto$   $\tau$ )]
    · fun_prop
    · intro x hx
      dsimp only
      simp [hx.1]

/-- Continuity at 1: the sandwich relative Rényi entropy is continuous at  $\alpha = 1$ . -
/
private theorem sandwichedRelRentropy.continuousAt_1 (p  $\sigma$  : MState d) :
  ContinuousWithinAt (fun  $\alpha$  =>  $\tilde{D}_\alpha(p||\sigma)$ ) (Set.Ioi 0) 1 := by
  by_cases h :  $\sigma.M.ker \leq p.M.ker$ 
  · simp only [ContinuousWithinAt, SandwichedRelRentropy, dif_pos h, zero_lt_one, if
    -- Use the fact that the limit of the real-valued function is the inner product.
    have h_real_limit : Filter.Tendsto (fun  $\alpha$  :  $\mathbb{R}$  => if  $\alpha = 1$  then  $\langle p.M, p.M \log - \sigma$ 
      have h_real_limit : Filter.Tendsto (fun  $\alpha$  :  $\mathbb{R}$  => Real.log ((HermitianMat.conj
        exact sandwichedRelRentropy.limit_at_one p  $\sigma$  h

```

```

    rw [ Metric.tendsto_nhdsWithin_nhds ] at *
    intro ε hε
    rcases h_real_limit ε hε with ⟨δ, hδ, H⟩
    use δ, hδ
    intro x hx₁ hx₂
    by_cases hx₃ : x = 1 <=> simp [*]
-- Since the real-valued function tends to the inner product, the ENNRe
have h_enreal_limit : Filter.Tendsto (fun α : ℝ => ENNReal.ofReal (if
  exact (ENNReal.tendsto_ofReal h_real_limit).comp Filter.tendsto_id
convert h_enreal_limit.congr' _ using 2
· symm
  apply ENNReal.ofReal_eq_coe_nnreal
· filter_upwards [self_mem_nhdsWithin] with α (hα : 0 < α)
  simp only [ENNReal.ofReal, ENNReal.coe_inj, hα, ↓reduceDite]
  exact Real.toNNReal_of_nonneg _
· apply tendsto_const_nhds.congr'
  filter_upwards [self_mem_nhdsWithin] with α hα
  simp only [SandwichedRelRentropy, Set.mem_Ioi.mp hα, zero_lt_one, dif_n

@[fun_prop]
theorem sandwichedRelRentropy.continuousOn (p σ : MState d) :
  ContinuousOn (fun α => D_ α(p||σ)) (Set.Ioi 0) := by
--If this turns out too hard, we just need `ContinuousAt f 1`.
--If that's still too hard, we really _just_ need that `(λ[>] 1).Tendsto
intro α hα
rcases lt_trichotomy α 1 with hα1 | rfl | hα1
· have h := sandwichedRelRentropy.continuousOn_Ioo_0_1 p σ
  exact (h.continuousAt (Ioo_mem_nhds hα hα1)).continuousWithinAt
· exact sandwichedRelRentropy.continuousAt_1 p σ
· have h := sandwichedRelRentropy.continuousOn_Ioi_1 p σ
  exact (h.continuousAt (Ioi_mem_nhds hα1)).continuousWithinAt

/-- Quantum relative entropy as `Tr[p (log p - log σ)]` when supports are c
/
theorem qRelativeEnt_ker {p σ : MState d} (h : σ.M.ker ≤ p.M.ker) :
  (p||σ).toEReal = (p.M, p.M.log - σ.M.log) := by
  simp [qRelativeEnt, SandwichedRelRentropy, h, EReal.coe_nnreal_eq_coe_rea

open Classical in
theorem qRelativeEnt_eq_neg_Sv_n_add (p σ : MState d) :
  (qRelativeEnt p σ).toEReal = -(Sv_n p : EReal) +
  if σ.M.ker ≤ p.M.ker then (-(p.M, σ.M.log) : EReal) else (τ : EReal)
by_cases h : σ.M.ker ≤ p.M.ker
· simp [h, Sv_n_eq_neg_trace_log, qRelativeEnt_ker, inner_sub_right]
  rw [real_inner_comm, sub_eq_add_neg]
· simp [h, qRelativeEnt, SandwichedRelRentropy]

```

```

/-- The quantum relative entropy is unchanged by `MState.relabel` -
/

```

```

@[simp]

```

```

theorem qRelativeEnt_relabel (ρ σ : MState d) (e : d₂ ≈ d) :
  □(ρ.relabel e || σ.relabel e) = □(ρ || σ) := by
  simp [qRelativeEnt]

```

```

@[simp]

```

```

theorem sandwichedRelRentropy_of_unique [Unique d] (ρ σ : MState d) :
   $\tilde{D}_\alpha(\rho || \sigma) = 0$  := by
  rcases Subsingleton.allEq ρ default
  rcases Subsingleton.allEq σ default
  simp [SandwichedRelRentropy]

```

```

@[simp]

```

```

theorem qRelEntropy_of_unique [Unique d] (ρ σ : MState d) :
  □(ρ || σ) = 0 := by
  exact sandwichedRelRentropy_of_unique ρ σ

```

```

theorem sandwichedRelRentropy_heq_congr

```

```

  {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂]
  {ρ₁ σ₁ : MState d₁} {ρ₂ σ₂ : MState d₂} (hd : d₁ = d₂) (hp : ρ₁ ≈ ρ₂) (hσ : σ₁
   $\tilde{D}_\alpha(\rho_1 || \sigma_1) = \tilde{D}_\alpha(\rho_2 || \sigma_2)$  := by
  --Why does this thm need to exist? Why not just `subst d₁` and `simp [heq_eq_eq]`?
  --and d₂ are equal, we then end up with two distinct instances of `Fintype d₁` and
  --and ρ₁ and ρ₂ refer to them each and so have different types. And then we'd need
  --too. This is kind of tedious, so it's better to just have this theorem around.
  rw [heq_iff_exists_eq_cast] at hp hσ
  obtain ⟨_, rfl⟩ := hp
  obtain ⟨_, rfl⟩ := hσ
  simp [← MState.relabel_cast _ hd]

```

```

@[gcongr]

```

```

theorem sandwichedRelRentropy_congr {α : ℝ}
  {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [DecidableEq d₂]
  {ρ₁ σ₁ : MState d₁} {ρ₂ σ₂ : MState d₂} (hd : d₁ = d₂)
  (hp : ρ₁ = ρ₂.relabel (Equiv.cast hd)) (hσ : σ₁ = σ₂.relabel (Equiv.cast hd))
   $\tilde{D}_\alpha(\rho_1 || \sigma_1) = \tilde{D}_\alpha(\rho_2 || \sigma_2)$  := by
  subst ρ₁ σ₁
  simp

```

```

@[gcongr]

```

```

theorem qRelEntropy_heq_congr {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁] [Fintype
  {ρ₁ σ₁ : MState d₁} {ρ₂ σ₂ : MState d₂} (hd : d₁ = d₂) (hp : ρ₁ ≈ ρ₂) (hσ : σ₁
  □(ρ₁ || σ₁) = □(ρ₂ || σ₂) := by

```

```

exact sandwichedRelEntropy_heq_congr hd hp hσ

/-- Quantum relative entropy when σ has full rank -/
theorem qRelativeEnt_rank {ρ σ : MState d} [σ.M.NonSingular] :
  (⟦(ρ||σ)⟧ : EReal) = ⟦ρ.M, ρ.M.log - σ.M.log⟧ := by
  apply qRelativeEnt_ker
  simp [HermitianMat.nonSingular_ker_bot]

section lowerSemicontinuous_1

variable {d : Type*} [Fintype d] [DecidableEq d]

open scoped InnerProductSpace RealInnerProductSpace HermitianMat

private def approxLog (N : ℕ) : ℝ → ℝ := fun t => Real.log (max t (Real.exp
(N : ℝ)))

private lemma approxLog_continuous (N : ℕ) : Continuous (approxLog N) := by
  have h_cont : Continuous (fun t : ℝ => max t (Real.exp (-
N))) :=
    Continuous.max continuous_id continuous_const
  exact Continuous.log h_cont (fun x => ne_of_gt (lt_max_of_lt_right (Real.

private lemma approxLog_ge_log_pos {t : ℝ} (ht : 0 < t) (N : ℕ) :
  Real.log t ≤ approxLog N t := by
  unfold approxLog
  exact Real.log_le_log ht (le_max_left _ _)

private lemma continuous_inner_cfc_approxLog (ρ : MState d) (N : ℕ) :
  Continuous (fun σ : MState d => ⟦ρ.M, σ.M.cfc (approxLog N)⟧) := by
  refine Continuous.comp ?_ ?_
  · fun_prop (disch := solve_by_elim)
  · exact (HermitianMat.cfc_continuous (approxLog_continuous N)).comp conti

private lemma approxLog_tendsto_at_pos {t : ℝ} (ht : 0 < t) :
  Filter.Tendsto (fun N : ℕ => approxLog N t) Filter.atTop (nhds (Real.log t)) := by
  refine' Filter.Tendsto.congr' _ tendsto_const_nhds
  filter_upwards [Filter.eventually_gt_atTop [-Real.log t]_] with N hN
  unfold approxLog
  rw [max_eq_left (by rw [← Real.log_le_log_iff (by positivity) (by positivity)
Real.log t], show (N : ℝ) ≥ [-Real.log t]_ + 1 by exact_mod_cast hN, Real.log
N)]]

open ComplexOrder in
private lemma inner_cfc_approxLog_ge (ρ σ : MState d) (N : ℕ) (hσ : σ.M.ker
⟦ρ.M, σ.M.log⟧ ≤ ⟦ρ.M, σ.M.cfc (approxLog N)⟧ := by

```

```

rw [inner_cfc_eq_sum_eigenWeight, show  $\sigma.M.\log = \sigma.M.cfc \text{ Real.log}$  from rfl, inner_]
apply Finset.sum_le_sum
intro i _
have hpsd :  $\sigma.M.mat.PosSemidef$  := by
  have h :=  $\sigma.pos.le$ 
  rwa [HermitianMat.le_iff, sub_zero] at h
have hei_nn :  $0 \leq \sigma.M.H.eigenvalues\ i := hpsd.eigenvalues\_nonneg\ i$ 
by_cases hei :  $\sigma.M.H.eigenvalues\ i = 0$ 
· rw [eigenWeight_zero_of_eigenvalue_zero h $\sigma$  hei, mul_zero, mul_zero]
· exact mul_le_mul_of_nonneg_right (approxLog_ge_log_pos (lt_of_le_of_ne hei_nn (N
  (eigenWeight_nonneg  $\rho\ \sigma\ i$ )))

open ComplexOrder in
private lemma tendsto_inner_cfc_approxLog ( $\rho\ x : MState\ d$ ) ( $hx : x.M.ker \leq \rho.M.ker$ )
  Filter.Tendsto (fun  $N : \mathbb{N} \Rightarrow [\rho.M, x.M.cfc\ (approxLog\ N)]$ )
  Filter.atTop (nhds  $[\rho.M, x.M.log]$ ) := by
  rw [show  $x.M.log = x.M.cfc \text{ Real.log}$  from rfl, inner_cfc_eq_sum_eigenWeight]
  simp_rw [inner_cfc_eq_sum_eigenWeight]
  apply tendsto_finset_sum
  intro i _
  have hpsd :  $x.M.mat.PosSemidef$  := by
    have h :=  $x.pos.le$ 
    rwa [HermitianMat.le_iff, sub_zero] at h
  have hei_nn :  $0 \leq x.M.H.eigenvalues\ i := hpsd.eigenvalues\_nonneg\ i$ 
  by_cases hei :  $x.M.H.eigenvalues\ i = 0$ 
  · simp [eigenWeight_zero_of_eigenvalue_zero hx hei]
  · exact (approxLog_tendsto_at_pos (lt_of_le_of_ne hei_nn (Ne.symm hei))).mul_const

lemma inner_log_bounded_near ( $hx : \sigma.M.ker \leq \rho.M.ker$ ) { $y : \mathbb{R}$ } ( $hy : [\rho.M, \sigma.M.log] < y$ )
   $\forall^f x$  in nhds  $\sigma$ ,  $x.M.ker \leq \rho.M.ker \rightarrow [\rho.M, x.M.log] < y := by
  have h_tendsto := tendsto_inner_cfc_approxLog  $\rho\ \sigma\ hx$ 
  obtain (N, hN) :  $\exists N : \mathbb{N}, [\rho.M, \sigma.M.cfc\ (approxLog\ N)] < y := by
    by_contra h
    push_neg at h
    exact absurd (lt_of_lt_of_le hy (ge_of_tendsto h_tendsto (Filter.Eventually.of_f
      (lt_irrefl _)))
  have h_cont := continuous_inner_cfc_approxLog  $\rho\ N$ 
  have h_lt :  $\forall^f x$  in nhds  $\sigma$ ,  $[\rho.M, x.M.cfc\ (approxLog\ N)] < y :=
    h_cont.continuousAt.eventually (gt_mem_nhds hN)
  filter_upwards [h_lt] with  $\sigma\ h\sigma\_lt\ h\sigma\_ker$ 
  exact lt_of_le_of_lt (inner_cfc_approxLog_ge  $\rho\ \sigma\ N\ h\sigma\_ker$ )  $h\sigma\_lt$ 

end lowerSemicontinuous_1

section lowerSemicontinuous_2$$$ 
```

```

variable {d : Type*} [Fintype d] [DecidableEq d]

open scoped InnerProductSpace RealInnerProductSpace HermitianMat

private lemma eigenWeight_eq_zero_iff (p x : MState d) (i : d) :
  eigenWeight p x i = 0 ↔ (x.M.H.eigenvectorBasis i : EuclideanSpace ℂ d)
  have h_forward : eigenWeight p x i = 0 → (x.M.H.eigenvectorBasis i) ∈ p.M
  unfold eigenWeight
  intro h_zero
  have h_inner : star (x.M.H.eigenvectorBasis i : d → ℂ) []v (p.M.mat.mulVec v) = star (star v []v (p.M.mat.mulVec v))
  convert h_zero using 1
  have h_real : ∀ (v : d → ℂ), star v []v (p.M.mat.mulVec v) = star (star v []v (p.M.mat.mulVec v))
  intro v
  have h_real : star v []v (p.M.mat.mulVec v) = star (star v []v (p.M.mat.mulVec v))
  have h_inner : ∀ (v w : d → ℂ), star v []v (p.M.mat.mulVec w) = star (star v []v (p.M.mat.mulVec w))
  intro v w
  have h_inner : star v []v (p.M.mat.mulVec w) = star (star w []v (p.M.mat.mulVec v))
  have h_inner : star v []v (p.M.mat.mulVec w) = star (star w []v (p.M.mat.mulVec v))
  have h_inner : p.M.mat = star p.M.mat := by
    exact p.M.2.symm ▸ rfl
  conv_rhs => rw [h_inner]
  simp [Matrix.mulVec, dotProduct]
  ring_nf
  simp [Finset.mul_sum, mul_comm, mul_left_comm];
  rw [Finset.sum_comm]; congr; ext; congr; ext; ring!;
  exact h_inner
  exact h_inner
  exact h_inner v v ▸ by simp [Matrix.mulVec, dotProduct];
  exact h_real.trans (by simp [])
  have h_real : ∀ (v : d → ℂ), star v []v (p.M.mat.mulVec v) = RCLike.re
  intro v; specialize h_real v; rw [eq_comm] at h_real; simp_all [
    linarith! [h_real];
  ]
  rw [h_real]; norm_cast; simp [Matrix.dotProduct_mulVec]
  exact HermitianMat.mem_ker_of_inner_mulVec_zero p.2 _ h_inner
  refine (h_forward, fun h => ?_)
  -- Since p e_i = 0, we have e_i^* p e_i = 0.
  have h_zero : (Matrix.vecMul (star (x.M.H.eigenvectorBasis i : d → ℂ)) p.M.mat.mulVec (x.M.H.eigenvectorBasis i : d → ℂ)) = 0 :=
    exact congr(WithLp.ofLp $h)
  convert congr_arg (fun v => star (x.M.H.eigenvectorBasis i : d → ℂ))
  simp [Matrix.dotProduct_mulVec]
  ring_nf
  simp [dotProduct]
  exact congr_arg Complex.re h_zero

private lemma ker_le_iff_eigenWeight_zero (p x : MState d) :

```

```

    x.M.ker ≤ p.M.ker ↔ ∀ i, x.M.H.eigenvalues i = 0 → eigenWeight p x i = 0 := by
  constructor
  · exact fun h i ↦ eigenWeight_zero_of_eigenvalue_zero h
  · intro h v hv
    obtain ⟨w, hw⟩ : ∃ w : d → ℂ, v = ∑ i, w i • x.M.H.eigenvectorBasis i := by
      exact ⟨_, Eq.symm (x.M.H.eigenvectorBasis.sum_repr v)⟩;
    -- Since $v \in \ker(x.M)$, we have $x.M(v) = 0$. Using the eigenvector basis, t
    zero, then $w i = 0$.
    have h_w_zero : ∀ i, x.M.H.eigenvalues i ≠ 0 → w i = 0 := by
      intro i hi
      have h_eigenvalue : x.M.val.mulVec v = ∑ i, (x.M.H.eigenvalues i) • w i • x.M.H.e
      have h_eigenvalue : ∀ i, x.M.val.mulVec (x.M.H.eigenvectorBasis i) = x.M.H.e
      exact fun i => x.M.H.mulVec_eigenvectorBasis i |> fun h => by simpa [mul_
      rw [hw]
      simp only [WithLp.ofLp_sum, WithLp.ofLp_smul]
      rw [Matrix.mulVec_sum];
      exact Finset.sum_congr rfl fun i _ => by rw [Matrix.mulVec_smul, h_eigenval
      have h_eigenvalue_zero : ∑ i, (x.M.H.eigenvalues i) • w i • x.M.H.eigenvectorB
      replace h_eigenvalue := congr(WithLp.toLp 2 $h_eigenvalue)
      simp only [HermitianMat.val_eq_coe, MState.mat_M, WithLp.ofLp_sum, WithLp.of
      WithLp.toLp_sum, WithLp.toLp_smul, WithLp.toLp_ofLp] at h_eigenvalue
      rw [← h_eigenvalue, ← hv]
      rfl
      have h_eigenvalue_zero : ∀ i, (x.M.H.eigenvalues i) • w i = 0 := by
        intro i
        have h_eigenvalue_zero : (x.M.H.eigenvalues i) • w i = inner ℂ (x.M.H.eigenv
        simp [orthonormal_iff_ite.mp (show Orthonormal ℂ (fun i => x.M.H.eigenv
        aesop
        simpa [hi] using h_eigenvalue_zero i |> fun h => by simpa [hi] using h;
      simp_all [eigenWeight_eq_zero_iff];
      exact Submodule.sum_mem _ fun i _ => if hi : x.M.H.eigenvalues i = 0 then by sim

private lemma neg_ker_exists_eigenWeight_pos (p x : MState d) (hx : ¬(x.M.ker ≤ p.M.ker)
  ∃ i, x.M.H.eigenvalues i = 0 ∧ 0 < eigenWeight p x i := by
  -- By `ker_le_iff_eigenWeight_zero`, ¬(x.M.ker ≤ p.M.ker) iff ∃ i, eigenvalue_i =
  have h_eigenWeight_ne_zero : ∃ i, x.M.H.eigenvalues i = 0 ∧ eigenWeight p x i ≠ 0
  exact Classical.not_forall_not.1 fun h => hx <| by simpa using ker_le_iff_eigenW
  exact h_eigenWeight_ne_zero.imp fun i hi => ⟨hi.1, lt_of_le_of_ne (eigenWeight_n

private lemma approxLog_at_zero (N : ℕ) : approxLog N 0 = -
(N : ℝ) := by
  simp [approxLog, max_eq_right (Real.exp_pos (-N)).le]

private lemma inner_cfc_approxLog_tendsto_bot (p x : MState d) (hx : ¬(x.M.ker ≤ p.M.ker)
  Filter.Tendsto (fun N : ℕ => p.M, x.M.cfc (approxLog N)) Filter.atTop Filter.atBot
  have h_split_sum : Filter.Tendsto (fun N : ℕ => ∑ i ∈ Finset.univ.filter (fun i =>

```

```

    have h_split_sum : Filter.Tendsto (fun N : ℕ => ∑ i ∈ Finset.univ.filter
↑N) * eigenWeight ρ x i) Filter.atTop Filter.atBot := by
    have h_split_sum : ∑ i ∈ Finset.univ.filter (fun i => x.M.H.eigenvalu
        obtain ( i, hi, hi' ) := neg_ker_exists_eigenWeight_pos ρ x hx; exa
    simp only [neg_mul];
    simp only [ Finset.sum_neg_distrib, Finset.mul_sum _ _ _ ] using Fil
    apply h_split_sum.congr'
    filter_upwards [ Filter.eventually_gt_atTop 0 ] with N hN
    refine Finset.sum_congr rfl fun i hi => ?_
    rw [ show approxLog N ( x.M.H.eigenvalues i ) = -↑N from by rw [ show x
convert h_split_sum.atBot_add ( show Filter.Tendsto ( fun N : ℕ => ∑ i ∈
· rw [ inner_cfc_eq_sum_eigenWeight, Finset.sum_filter_add_sum_filter_not
· apply tendsto_finset_sum
  intro i hi
  exact Filter.Tendsto.mul ((approxLog_tendsto_at_pos ( show 0 < x.M.H.ei

end lowerSemicontinuous_2

open Classical in
theorem qRelativeEnt_lowerSemicontinuous_2 (ρ x : MState d) (hx : ¬(x.M.ker
  ∀f (x' : MState d) in nhds x,
    y < (if x'.M.ker ≤ ρ.M.ker then ⌊ρ.M, ρ.M.log - x'.M.log⌋ else τ : ER
-- Since $y < \top$, we can choose a neighborhood around $x$ where the in
have h_inner_lt_y : ∀f x' in nhds x, x'.M.ker ≤ ρ.M.ker → ⌊ρ.M, ρ.M.log -
  have h_inner_lt_y : Filter.Tendsto (fun N : ℕ => ⌊ρ.M, ρ.M.log - x.M.cfc
    have h_inner_lt_y : Filter.Tendsto (fun N : ℕ => ⌊ρ.M, ρ.M.log⌋ - ⌊ρ.
      exact Filter.Tendsto.add_atTop tendsto_const_nhds ( Filter.tendsto_
    convert h_inner_lt_y using 1
    ext1 N
    simp [inner_sub_right]
  obtain (N, hN) : ∃ N : ℕ, ⌊ρ.M, ρ.M.log - x.M.cfc (approxLog N)⌋ > y.to
    exact (h_inner_lt_y.eventually_gt_atTop _).exists
  have h_cont : Continuous (fun σ : MState d => ⌊ρ.M, ρ.M.log - σ.M.cfc (
    simp only [inner_sub_right]
    exact continuous_const.sub (continuous_inner_cfc_approxLog ρ N)
  have h_cont : ∀f x' in nhds x, ⌊ρ.M, ρ.M.log - x'.M.cfc (approxLog N)⌋
    exact h_cont.continuousAt.eventually ( lt_mem_nhds hN ) |> fun h => h
  filter_upwards [h_cont] with x' hx' hx''
  apply lt_of_lt_of_le hx'
  have h_inner_le : ⌊ρ.M, x'.M.log⌋ ≤ ⌊ρ.M, x'.M.cfc (approxLog N)⌋ := by
    exact inner_cfc_approxLog_ge ρ x' N hx''
  rw [inner_sub_right, inner_sub_right]
  exact sub_le_sub_left h_inner_le _
  filter_upwards [ h_inner_lt_y ] with x' hx';
  split_ifs <;> simp_all [ ENNReal.toReal ];
  · convert ENNReal.ofReal_lt_ofReal_iff (show 0 < ⌊ρ.M, ρ.M.log - x'.M.log

```



```

cases y
  · simp at hy
  simp only [ENNReal.ofReal, ENNReal.toNNReal_coe, Real.toNNReal_coe, ENNReal.coe_
  rw [← NNReal.coe_lt_coe, Real.toNNReal_of_nonneg (le_trans (by positivity) hx'.1
  simp [← ENNReal.ofReal_coe_nnreal]
  · rw [lt_top_iff_ne_top, ne_eq] at hy ⊢
  rwa [EReal.coe_ennreal_eq_top_iff]

/-
Relative entropy is lower semicontinuous (in each argument, actually, but we only ne
latter here). Will need the fact that all the cfc / eigenvalue stuff is continuous,
carefully handling what happens with the kernel subspace, which will make this a pai
-/
@[fun_prop]
theorem qRelativeEnt.lowerSemicontinuous (p : MState d) : LowerSemicontinuous fun σ
  simp_rw [qRelativeEnt, SandwichedRelEntropy, if_true, lowerSemicontinuous_iff]
  simp only [zero_lt_one, ↓reduceDite]
  intro x
  by_cases hx : x.M.ker ≤ p.M.ker
  · intro y hy
    obtain {y', hy'} : ∃ y' : ℝ, y < ENNReal.ofReal y' ∧ y' < [p.M, p.M.log - x.M.lo
      rcases ENNReal.lt_iff_exists_real_btwn.mp hy with { y', hy₁, hy₂ };
      rw [ ENNReal.ofReal_lt_iff_lt_toReal ] at hy₂ <=> aesop;
      simp only [↓reduceDite, inner_sub_right, hx] at hy hy' ⊢
      have := inner_log_bounded_near hx (y := [p.M, p.M.log] - y') (by linarith)
      filter_upwards [this] with σ hσ
      split
      · simp_all only [ENNReal.ofReal, forall_const]
        apply lt_of_lt_of_le hy'.1
        refine mod_cast max_le (a := y') (b := 0) (c := [p.M, p.M.log] - [p.M, σ.M.log
          · linarith
          · linarith [ show 0 ≤ y' from le_of_not_gt fun h => by norm_num [ Real.toNNRea
      · exact hy'.1.trans_le (by simp)
  · intro y hy
    simp only [hx, ↓reduceDite] at hy ⊢
    have h₂ := qRelativeEnt_lowerSemicontinuous_2 p x hx y hy
    filter_upwards [h₂] with x' hx'
    split_ifs with h₁ junk
    · simpa [← EReal.coe_ennreal_lt_coe_ennreal_iff, h₁] using hx'
    · simp at junk
    · exact hy

```

/-- Joint convexity of Quantum relative entropy. We can't state this with `ConvexOn`
 an `AddCommMonoid`, which `MState`s are not. Instead we state it with `Mixable`.

TODO:

```

* Add the `Mixable` instance that infers from the `Coe` so that the right
`p [⊔(ρ₁||σ₁) ↔ ⊔(ρ₂||σ₂)]`
* Define (joint) convexity as its own thing - a `ConvexOn` for `Mixable` t
* Maybe, more broadly, find a way to make `ConvexOn` work with the subset
-/
theorem qRelativeEnt_joint_convexity :
  ∀ (ρ₁ ρ₂ σ₁ σ₂ : MState d), ∀ (p : Prob),
    ⊔(p [ρ₁ ↔ ρ₂] || p [σ₁ ↔ σ₂]) ≤ p * ⊔(ρ₁||σ₁) + (1 - p) * ⊔(ρ₂||σ₂) := by
      sorry

@[simp]
theorem qRelEntropy_self (ρ : MState d) : ⊔(ρ||ρ) = 0 := by
  simp [qRelativeEnt]

@[aesop (rule_sets := [finiteness]) unsafe apply]
theorem qRelativeEnt_ne_top {ρ σ : MState d} [σ.M.NonSingular] : ⊔(ρ||σ) ≠ T
  rw [qRelativeEnt]
  finiteness

/-- `I(A:B) = ⊔(ρ^A||ρ^A ⊗ ρ^B)` -/
theorem qMutualInfo_as_qRelativeEnt (ρ : MState (dA × dB)) :
  qMutualInfo ρ = (⊔(ρ||ρ.traceRight ⊗M ρ.traceLeft) : EReal) := by
  sorry

/-
Helper: If σ₂ ≤ α • σ₁ for density matrices, then α > 0.
Proof: σ₂ has trace 1, so it's nonzero. If α ≤ 0, then α • σ₁ ≤ 0 (since
but σ₂ ≤ α • σ₁ ≤ 0 with σ₂ ≥ 0 forces σ₂ = 0, contradicting trace = 1.
-/
private lemma pos_of_MState_le_smul {σ₁ σ₂ : MState d} (hσ : σ₂.M ≤ α • σ₁.
  by_contra! h_nonpos
  apply σ₂.pos.ne'
  apply le_antisymm
  · convert ← hσ using 1
    apply le_antisymm
    · exact smul_nonpos_of_nonpos_of_nonneg h_nonpos ( by positivity)
    · exact hσ.trans' ( by positivity );
  · positivity

open ComplexOrder in
private lemma HermitianMat.inner_log_mono_of_posDef_of_le {A B C : Hermitia
  (hC : 0 ≤ C) (hA : A.mat.PosDef) (hAB : A ≤ B) :
  ⊔C, A.log ⊔ ≤ ⊔C, B.log ⊔ := by
  exact inner_mono hC (log_mono hA hAB)

open ComplexOrder in

```

```

private lemma posDef_add_eps {A : HermitianMat d ℂ} (hA : 0 ≤ A) {ε : ℝ} (hε : 0 < ε) :
  (A + ε • 1).mat.PosDef := by
  rw [HermitianMat.zero_le_iff] at hA
  rw [Matrix.posDef_iff_dotProduct_mulVec]
  constructor
  · exact HermitianMat.H _
  · intro x hx_ne_zero
    have h_inner : star x []_v (A.val.mulVec x) ≥ 0 := by
      simp [hA]
    have h_eps : star x []_v (ε • 1 : Matrix d d ℂ).mulVec x = ε * star x []_v x := by
      simp [Matrix.mulVec, dotProduct, Finset.mul_sum, mul_left_comm, Matrix.one_apply]
    have h_pos : 0 < ε * star x []_v x := by
      simp [*]
    simp [Matrix.add_mulVec, h_eps] using add_pos_of_nonneg_of_pos h_inner h_pos

private lemma log_add_eps_eq_cfc (A : HermitianMat d ℝ) (ε : ℝ) :
  (A + ε • 1).log = A.cfc (Real.log <| · + ε) := by
  have h_cfc : A + ε • 1 = A.cfc (· + ε) := by
    have h_add : A.cfc (fun u => u + ε) = A.cfc (fun u => u) + A.cfc (fun _ => ε) :=
      exact A.cfc_add_apply _
    simp only [HermitianMat.cfc_id', HermitianMat.cfc_const, h_add]
  rw [h_cfc, HermitianMat.log, A.cfc_comp_apply]

private lemma inner_cfc_eq_sum (A C : HermitianMat d ℝ) (f : ℝ → ℝ) :
  []C, A.cfc f[] = ∑ i, f (A.H.eigenvalues i) *
  RCLike.re ((C.mat * (A.H.eigenvectorUnitary.val * Matrix.single i i 1 * A.H.ei
  apply congr(RCLike.re ((C.val * $(A.cfc_toMat_eq_sum_smul_proj f)).trace)).trans
  simp only [HermitianMat.val_eq_coe, Matrix.mul_sum, Algebra.mul_smul_comm, Matrix.
  Matrix.trace_smul, RCLike.real_smul_eq_coe_mul, map_sum, RCLike.mul_re,
  RCLike.ofReal_re, RCLike.ofReal_im, zero_mul, sub_zero]

private lemma eigenproj_coeff_zero_of_ker_le {A C : HermitianMat d ℝ}
  (hker : A.ker ≤ C.ker) {i : d} (hi : A.H.eigenvalues i = 0) :
  RCLike.re ((C.mat * (A.H.eigenvectorUnitary.val * Matrix.single i i 1 * A.H.ei
  have h_eigenvector_in_ker_C : C.mat.mulVec (A.H.eigenvectorBasis i) = 0 := by
  have h_eigenvector_in_ker : (A.H.eigenvectorBasis i) ∈ LinearMap.ker A.lin.toLin
    have := congr(WithLp.toLp 2 $(A.H.mulVec_eigenvectorBasis i))
    simp [hi] at this
    exact congr(WithLp.toLp 2 $this)
  specialize hker h_eigenvector_in_ker
  exact congr(WithLp.ofLp $hker)
  have h_trace_zero : (C.mat * (A.H.eigenvectorUnitary.val * Matrix.single i i 1 * A
    simp [Matrix.trace, Matrix.mulVec, dotProduct]
    simp [Matrix.mul_apply, Matrix.single, Matrix.conjTranspose_apply]
    simp [Finset.sum_ite, Finset.filter_eq, Finset.filter_and, mul_comm, mul_left_co
    refine Finset.sum_congr rfl fun j hj => Finset.sum_congr rfl fun k hk => ?_

```

```

rw [Finset.sum_eq_single i]
· simp_all only [Finset.mem_univ, ↓reduceIte, Finset.inter_univ, Finset
  rw [mul_assoc]
· intro b a a_1
  simp_all only [Finset.mem_univ, ne_eq]
  split <;> simp_all only [not_true_eq_false, Finset.inter_empty, Finse
· intro a
  simp_all only [Finset.mem_univ, not_true_eq_false]
simp [h_eigenvector_in_ker_C, h_trace_zero]

open scoped Topology in
private lemma inner_log_shift_tendsto {A C : HermitianMat d []} (hker : A.ker
  ([]> 0).Tendsto (fun (ε : ℝ) ↦ []C, (A + ε • 1).log[]) ([] []C, A.log[]) :=
  simp only [log_add_eps_eq_cfc A]
  refine only [inner_cfc_eq_sum, HermitianMat.log]
  refine tendsto_finset_sum _ fun i _ ↦ ?_
  by_cases hi : A.H.eigenvalues i = 0
  · simp [eigenproj_coeff_zero_of_ker_le hker hi]
  · have h := log_add_eps_eq_cfc A
    simp_rw [HermitianMat.log] at h
    refine Filter.Tendsto.mul ?_ tendsto_const_nhds
    conv in [] (Real.log _) => rw [← add_zero (A.H.eigenvalues i)]
    apply (tendsto_const_nhds.add (Filter.tendsto_id.mono_left inf_le_left)
    rwa [add_zero]

private lemma HermitianMat.inner_log_mono_of_psd_of_le {A B C : HermitianMa
  (hA : 0 ≤ A) (hAB : A ≤ B) (hC : 0 ≤ C) (hker : A.ker ≤ C.ker) :
  []C, A.log[] ≤ []C, B.log[] := by
  open scoped Topology in
  have h_eventually : ∀f ε in []> (0 : ℝ),
    []C, (A + ε • 1).log[] ≤ []C, (B + ε • 1).log[] := by
    refine eventually_nhdsWithin_of_forall fun ε hε ↦ ?_
    exact inner_log_mono_of_posDef_of_le hC (posDef_add_eps hA hε) (add_le_
  refine le_of_tendsto_of_tendsto ?_ ?_ h_eventually
  · exact inner_log_shift_tendsto hker
  · apply inner_log_shift_tendsto
    exact (ker_antitone hA hAB).trans hker

private lemma HermitianMat.inner_log_sub_le_log_alpha (p : MState d) {σ1 σ2
  (hσ : σ2.M ≤ α • σ1.M)
  (hker1 : σ1.M.ker ≤ p.M.ker) (hker2 : σ2.M.ker ≤ p.M.ker) :
  []p.M, σ2.M.log - σ1.M.log[] ≤ Real.log α := by
  have h_log_mono : []p.M, σ2.M.log - (α • σ1.M).log[] ≤ 0 := by
    have h_log_mono : []p.M, σ2.M.log[] ≤ []p.M, (α • σ1.M).log[] := by
      exact inner_log_mono_of_psd_of_le σ2.nonneg hσ p.nonneg hker2
    simp [inner_sub_right] using sub_nonpos_of_le h_log_mono

```

```

have h_log_smul : ( $\alpha \cdot \sigma_1.M$ ).log = (Real.log  $\alpha$ ) *  $\sigma_1.M$ .supportProj +  $\sigma_1.M$ .log := by
  apply HermitianMat.log_smul_of_pos
  rintro rfl
  simp using pos_of_MState_le_smul h $\sigma$ 
rw [h_log_smul] at h_log_mono
simp only [add_comm, sub_eq_add_neg, neg_add_rev] at h_log_mono h_log_smul
have h_inner_support :  $\llbracket \rho.M, \sigma_1.M$ .supportProj  $\rrbracket$  = 1 := by
  rw [HermitianMat.inner_supportProj_of_ker_le hker $_1$ ,  $\rho$ .tr]
simp_all [← add_assoc, inner_add_right, inner_smul_right]

theorem qRelEntropy_le_add_of_le_smul ( $\rho$  : MState d) { $\sigma_1 \sigma_2$  : MState d} (h $\sigma$  :  $\sigma_2.M \leq \sigma_1.M$ ) :
   $\llbracket \rho \parallel \sigma_1 \rrbracket \leq \llbracket \rho \parallel \sigma_2 \rrbracket + \text{ENNReal.ofReal (Real.log } \alpha \rrbracket$ 
  := by
  -- Consider two cases: when the kernel of  $\sigma_2$  is contained in the kernel of  $\rho$  and w
  by_cases hker :  $\sigma_2.M$ .ker  $\leq \rho.M$ .ker;
  · by_cases hker $_1$  :  $\sigma_1.M$ .ker  $\leq \rho.M$ .ker;
    · -- Using `qRelativeEnt_ker` to get  $D(\rho \parallel \sigma_1).$ toEReal =  $\llbracket \rho.M, \rho.M$ .log -  $\sigma_1.M$ .log  $\rrbracket$ 
      have h_log :  $\llbracket \rho.M, \sigma_2.M$ .log -  $\sigma_1.M$ .log  $\rrbracket \leq \text{Real.log } \alpha$  := by
        apply HermitianMat.inner_log_sub_le_log_alpha
        · exact h $\sigma$ 
        · exact hker $_1$ 
        · exact hker
      have h_final : (qRelativeEnt  $\rho \sigma_1$ ).toEReal  $\leq$  (qRelativeEnt  $\rho \sigma_2$ ).toEReal + ENNReal.ofReal (Real.log  $\alpha$ )
      simp_all only [qRelativeEnt_ker, inner_sub_right, tsub_le_iff_right, EReal.cases max_cases (Real.log  $\alpha$ ) 0
        <=> simp only [sup_of_le_left, *]
        <=> norm_cast at *
        <=> linarith [Real.log_nonneg one_le_two]
      have h_final' : qRelativeEnt  $\rho \sigma_1 \leq$  qRelativeEnt  $\rho \sigma_2$  + ENNReal.ofReal (Real.log  $\alpha$ )
      exact_mod_cast h_final
      exact h_final'
    · by_contra h_contra;
      have hker_le :  $\sigma_1.M$ .ker  $\leq \sigma_2.M$ .ker := by
        apply_rules [ HermitianMat.ker_le_of_le_smul, h $\sigma$  ];
        · rintro rfl
          apply  $\sigma_2$ .pos.not_ge
          simp using h $\sigma$ 
        · positivity
      exact hker $_1$  (hker_le.trans hker)
  · simp [hker, SandwichedRelEntropy, qRelativeEnt]

theorem qRelativeEnt_op_le { $\rho \sigma$  : MState d} (h :  $\rho.M \leq \alpha \cdot \sigma.M$ ) :
   $\llbracket \rho \parallel \sigma \rrbracket \leq \text{.ofReal (Real.log } \alpha \rrbracket$  := by
  simp using qRelEntropy_le_add_of_le_smul  $\rho$  h

```

1.440 QuantumInfo/Finite/Entropy/SSA.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.Entropy.VonNeumann
import QuantumInfo.ForMathlib.HermitianMat.Sqrt

/-!
Quantities of quantum _relative entropy_, namely the (standard) quantum rel
entropy, and generalizations such as sandwiched Rényi relative entropy.
-/

noncomputable section

variable {d d₁ d₂ d₃ m n : Type*}
variable [Fintype d] [Fintype d₁] [Fintype d₂] [Fintype d₃] [Fintype m] [Fi
variable [DecidableEq d] [DecidableEq d₁] [DecidableEq d₂] [DecidableEq d₃]
variable {dA dB dC dA₁ dA₂ : Type*}
variable [Fintype dA] [Fintype dB] [Fintype dC] [Fintype dA₁] [Fintype dA₂]
variable [DecidableEq dA] [DecidableEq dB] [DecidableEq dC] [DecidableEq dA
variable {α : Type*} [RCLike α]
variable {α : ℝ}

open scoped InnerProductSpace RealInnerProductSpace Kronecker Matrix

/-
The operator norm of a matrix is the operator norm of the linear map it rep
-/
/-- The operator norm of a matrix, with respect to the Euclidean norm (l2 n
/
noncomputable def Matrix.opNorm [RCLike α] (A : Matrix m n α) : ℝ :=
  ||LinearMap.toContinuousLinearMap (Matrix.toEuclideanLin A)||

/-
An isometry preserves the Euclidean norm.
-/
theorem Matrix.isometry_preserves_norm (A : Matrix n m α) (hA : A.Isometry)
  ||Matrix.toEuclideanLin A x|| = ||x|| := by
  rw [← sq_eq_sq₀ (by positivity) (by positivity), Matrix.Isometry] a
  simp [EuclideanSpace.norm_eq]
  have h_inner : ∀ x y : EuclideanSpace α m, inner α (toEuclideanLin A x) (
    intro x y

```

```

    have h_inner_eq : inner [] (toEuclideanLin A x) (toEuclideanLin A y) = inner [] x y
    simp [ Matrix.toEuclideanLin, inner ];
    simp [ Matrix.mulVec, dotProduct, Finset.mul_sum, mul_comm, ];
    simp [ Matrix.mul_apply, mul_assoc, mul_comm, mul_left_comm, Finset.mul_sum ]
    rw [ Finset.sum_comm, Finset.sum_comm_cycle ]
    simp_all [ Matrix.toEuclideanLin, toLpLin ];
    convert congr_arg Real.sqrt ( congr_arg ( fun z => ||z|| ) ( h_inner x x ) ) using 1
    · simp [ EuclideanSpace.norm_eq, inner_self_eq_norm_sq_to_K ];
    · simp [ EuclideanSpace.norm_eq, inner_self_eq_norm_sq_to_K ]

/-
The operator norm of an isometry is 1 (assuming the domain is non-
empty).
-/
theorem Matrix.opNorm_isometry [Nonempty m] (A : Matrix n m ℝ) (hA : A.Isometry) : M
  have h_opNorm : ∀ x : EuclideanSpace ℝ m, ||Matrix.toEuclideanLin A x|| = ||x|| := by
    convert Matrix.isometry_preserves_norm A hA;
    refine' le_antisymm ( csInf_le _ _ ) ( le_csInf _ _ );
    · exact ( 0, fun c hc => hc.1 );
    · aesop;
    · exact ( 1, ⟨zero_le_one, fun x => le_of_eq (by simp [h_opNorm])⟩ );
    · norm_num +zetaDelta at *;
    intro b hb h; specialize h ( EuclideanSpace.single ( Classical.arbitrary m ) 1 )

variable (d1 d2) in
/-- The matrix representation of the map $v \mapsto v \otimes \sum_k |k\rangle\langle k|$ is
The output index is `(d1 \times d2) \times d2`. -/
def map_to_tensor_MES : Matrix ((d1 × d2) × d2) d1 ℂ :=
  Matrix.of fun ((i, j), k) l => if i = l ∧ j = k then 1 else 0

theorem map_to_tensor_MES_prop (A : Matrix (d1 × d2) (d1 × d2) ℂ) :
  (map_to_tensor_MES d1 d2).conjTranspose * (Matrix.kronecker A (1 : Matrix d2 d2
  A.traceRight := by
  ext i j; simp [map_to_tensor_MES, Matrix.mul_apply]
  simp [ Finset.sum_ite, Matrix.one_apply ]
  rw [ Finset.sum_sigma ];
  rw [ Finset.sum_congr rfl (g := fun x => A ( i, x.1.2 ) ( j, x.1.2 )) ]
  · apply Finset.sum_bij (fun x _ => x.1.2)
    · simp
    · aesop
    · simp
    · simp
    · aesop

/- The isometry V_rho from the paper. -/
noncomputable def V_rho (pAB : HermitianMat (dA × dB) ℂ) : Matrix ((dA × dB) × dB) d

```

```

let pA_inv_sqrt := pAB.traceRight-1.sqrt
let pAB_sqrt := pAB.sqrt
let I_B := (1 : Matrix dB dB ℂ)
let term1 := pAB_sqrt.mat ⊗k I_B
let term2 := map_to_tensor_MES dA dB
term1 * term2 * pA_inv_sqrt.mat

open scoped MatrixOrder ComplexOrder

/-- The isometry V_sigma from the paper. -/
noncomputable def V_sigma (σBC : HermitianMat (dB × dC) ℂ) : Matrix (dB × (
  (V_rho (σBC.reindex (Equiv.prodComm dB dC))).reindex
    ((Equiv.prodComm _ _).trans (Equiv.prodCongr (Equiv.refl _)) (Equiv.prod
      (Equiv.refl _))

/--
V_rho^H * V_rho simplifies to sandwiching the traceRight by the inverse squ
-/
lemma V_rho_conj_mul_self_eq (pAB : HermitianMat (dA × dB) ℂ) (hp : pAB.mat
  let pA := pAB.traceRight
  let pA_inv_sqrt := (pA-1.sqrt : Matrix dA dA ℂ)
  (V_rho pAB)H * (V_rho pAB) = pA_inv_sqrt * pAB.traceRight.mat * pA_inv_sqrt
  -- By definition of $V_rho$, we can write out the product $V_rho^H * V_rho$
  simp [V_rho];
  simp [← Matrix.mul_assoc];
  have h_simp : (Matrix.kroneckerMap (fun x1 x2 => x1 * x2) (pAB_sqrt : Mat
    have h_simp : (pAB_sqrt : Matrix (dA × dB) (dA × dB) ℂ)H * (pAB_sqrt :
      convert pAB_sqrt_sq ( show 0 ≤ pAB from ?_ ) using 1;
      · simp [ HermitianMat.sqrt ];
      · positivity
  ext { i, j } { k, l }
  simp [← h_simp, Matrix.mul_apply ]
  ring_nf
  by_cases hij : j = l
  · simp [ hij, Matrix.one_apply ]
    simp [← Finset.sum_filter]
    refine' Finset.sum_bij ( fun x _ => x.1 ) _ _ _ _ <;> simp
    intro a b
    exact Or.inl ( by simp using congr_fun ( congr_fun ( pAB_sqrt.2 ) i
  · simp [ hij, Matrix.one_apply ]
    exact Finset.sum_eq_zero (by aesop)
  simp_all [ mul_assoc, Matrix.mul_assoc ];
  simp [← Matrix.mul_assoc, ← map_to_tensor_MES_prop ]

/--
The partial trace (left) of a positive definite matrix is positive definite

```



```

-/
lemma PosDef_traceRight [Nonempty dB] (A : HermitianMat (dA × dB) ℂ) (hA : A.mat.PosDef)
  A.traceRight.mat.PosDef := by
  have h_trace_right_pos_def : ∀ (x : EuclideanSpace ℂ dA), x ≠ 0 → 0 < ∑ k : dB, (s
    intro x hx_ne_zero
  have h_inner_pos : ∀ k : dB, 0 ≤ (star x) ⌊v (Matrix.mulVec (A.val.submatrix (fun i : dA =>
    have := (Matrix.posDef_iff_dotProduct_mulVec.mp hA).2;
    intro k
    specialize @this ( fun i => if h : i.2 = k then x i.1 else 0 )
    simp_all only [ne_eq, dite_eq_ite, dotProduct, Pi.star_apply, RCLike.star_def,
      HermitianMat.mat_apply, mul_ite, mul_zero, HermitianMat.val_eq_coe, Matrix.s
    convert this ( show ( fun i : dA × dB => if i.2 = k then x i.1 else 0 ) ≠ 0 fr
  rw [← Finset.sum_subset ( Finset.subset_univ ( Finset.image ( fun i : dA => (
    · simp only [Finset.sum_ite, Finset.sum_const_zero, add_zero, Set.InjOn, Finse
      Set.mem_univ, Prod.mk.injEq, and_true, imp_self, implies_true, Finset.sum_
      refine' Finset.sum_congr rfl fun i hi => _;
      refine' congr_arg _ ( Finset.sum_bij ( fun j _ => ( j, k ) ) _ _ _ ) <=> s
    · simp only [Finset.mem_univ, Finset.mem_image, true_and, not_exists, ne_eq, F
      Finset.sum_const_zero, add_zero, mul_eq_zero, map_eq_zero, ite_eq_right_if
      Prod.forall, Prod.mk.injEq, not_and, forall_eq];
      exact fun a b hb => Or.inl fun h => False.elim <| hb <| h.symm;
    obtain ⟨k, hk⟩ : ∃ k : dB, (star x) ⌊v (Matrix.mulVec (A.val.submatrix (fun i =>
      have := @(Matrix.posDef_iff_dotProduct_mulVec.mp hA).2 ( fun i => x i.1 * ( if
      simp_all only [ne_eq, dotProduct, Pi.star_apply, RCLike.star_def, Matrix.mulVe
      HermitianMat.val_eq_coe, Matrix.submatrix_apply, HermitianMat.mat_apply, mul
      contrapose! this
      simp_all only [ne_eq, funext_iff, Pi.zero_apply, ite_eq_right_iff, Prod.forall
        not_forall, Finset.sum_ite, Finset.sum_const_zero, add_zero] ;
      constructor
      · rw [← Function.ne_iff]
        change _ ≠ 0
        simpa using hx_ne_zero
      convert this ( Classical.arbitrary dB ) using 1;
      rw [← Finset.sum_subset ( Finset.subset_univ ( Finset.univ.image fun i : dA =>
      · simp only [Finset.coe_univ, Prod.mk.injEq, and_true, implies_true, Set.injOn
        Finset.sum_image, ↓reduceIte, gt_iff_lt]
      congr! 3;
      refine' Finset.sum_bij ( fun y hy => y.1 ) _ _ _ <=> simp
      · simp only [Finset.mem_univ, Finset.mem_image, true_and, not_exists, ne_eq, m
        map_eq_zero, ite_eq_right_iff, forall_const, Prod.forall, Prod.mk.injEq, n
        exact fun a b hb => Or.inl fun h => False.elim <| hb <| h.symm ▸ rfl
      exact lt_of_lt_of_le hk ( Finset.single_le_sum ( fun k _ => h_inner_pos k ) ( Fi
    rw [Matrix.posDef_iff_dotProduct_mulVec]
    refine' ⟨A.traceRight.2, fun x hx => _ ⟩;
    · convert h_trace_right_pos_def (WithLp.toLp 2 x) (by simpa using hx) using 1;
      unfold HermitianMat.traceRight

```

```

simp only [dotProduct, Pi.star_apply, RCLike.star_def, HermitianMat.mat
simp only [Matrix.mulVec, dotProduct, mul_comm, Matrix.submatrix_apply,
simp only [Matrix.traceRight, HermitianMat.mat_apply, Matrix.of_apply,
rw [Finset.sum_comm_cycle]

lemma PosDef_traceLeft [Nonempty dA] (A : HermitianMat (dA × dB) ℂ) (hA : A
  A.traceLeft.mat.PosDef := by
  exact PosDef_traceRight (A.reindex (Equiv.prodComm _ _)) (hA.reindex _)

/--
V_rho is an isometry.
-/
theorem V_rho_isometry [Nonempty dB] (pAB : HermitianMat (dA × dB) ℂ) (hp :
  (V_rho pAB)H * (V_rho pAB) = 1 := by
  -- Since pA is positive definite, we can use the fact that pA_inv_sqrt *
  have h_pos_def : (pAB.traceRight-1.sqrt : Matrix dA dA ℂ) * (pAB.traceRig
    convert HermitianMat.sqrt_inv_mul_self_mul_sqrt_inv_eq_one _;
    exact PosDef_traceRight _ hp
  rw [← h_pos_def]
  exact V_rho_conj_mul_self_eq pAB hp

/--
V_sigma is an isometry.
-/
theorem V_sigma_isometry [Nonempty dB] (σBC : HermitianMat (dB × dC) ℂ) (hσ
  (V_sigma σBC).conjTranspose * (V_sigma σBC) = 1 := by
  simp [V_sigma]
  exact V_rho_isometry _ (hσ.reindex _)

/-
Definition of W_mat with correct reindexing.
-/
open HermitianMat
open scoped MatrixOrder

variable {dA dB dC : Type*} [Fintype dA] [Fintype dB] [Fintype dC]
variable [DecidableEq dA] [DecidableEq dB] [DecidableEq dC]

/-- The operator W from the paper, defined as a matrix product. -
/
noncomputable def W_mat (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMa
  let pA := pAB.traceRight
  let σC := σBC.traceLeft
  let pAB_sqrt := (pAB.sqrt : Matrix (dA × dB) (dA × dB) ℂ)
  let σC_inv_sqrt := (σC-1.sqrt : Matrix dC dC ℂ)
  let pA_inv_sqrt := (pA-1.sqrt : Matrix dA dA ℂ)

```

```

let oBC_sqrt := (oBC.sqrt : Matrix (dB × dC) (dB × dC) ℂ)

let term1 := pAB_sqrt ⊗k oC_inv_sqrt
let term2 := pA_inv_sqrt ⊗k oBC_sqrt
let term2_reindexed := term2.reindex (Equiv.prodAssoc dA dB dC).symm (Equiv.prodAs

term1 * term2_reindexed

/--
The operator norm of a matrix product is at most the product of the operator norms.
-/
theorem Matrix.opNorm_mul_le {l m n α : Type*} [Fintype l] [Fintype m] [Fintype n]
  [DecidableEq l] [DecidableEq m] [DecidableEq n] [RCLike α]
  (A : Matrix l m α) (B : Matrix m n α) :
  Matrix.opNorm (A * B) ≤ Matrix.opNorm A * Matrix.opNorm B := by
  have h_opNorm_mul_le : ∀ (A : Matrix l m α) (B : Matrix m n α), Matrix.opNorm (A *
    intro A B
    have h_comp : Matrix.toEuclideanLin (A * B) = Matrix.toEuclideanLin A ∘1 Matrix.
      ext; simp [toEuclideanLin]
    convert ContinuousLinearMap.opNorm_comp_le ( Matrix.toEuclideanLin A |> LinearMa
    unfold Matrix.opNorm;
    exact congr_arg _ ( by aesop );
    exact h_opNorm_mul_le A B

theorem Matrix.opNorm_reindex_proven {l m n p : Type*} [Fintype l] [Fintype m] [Fint
  [DecidableEq l] [DecidableEq m] [DecidableEq n] [DecidableEq p]
  (e : m ≈ l) (f : n ≈ p) (A : Matrix m n α) :
  Matrix.opNorm (A.reindex e f) = Matrix.opNorm A := by
  refine' le_antisymm _ _;
  · refine' csInf_le _ _;
    · exact { 0, fun c hc => hc.l };
    · refine' { norm_nonneg _, fun x => _ };
      convert ContinuousLinearMap.le_opNorm ( LinearMap.toContinuousLinearMap ( Matr
        · simp [ Matrix.toEuclideanLin, EuclideanSpace.norm_eq ];
          rw [ ← Equiv.sum_comp e.symm ] ; aesop;
        · simp [ EuclideanSpace.norm_eq, Matrix.opNorm ];
          exact 0.rinl ( by rw [ ← Equiv.sum_comp f ] );
    · refine' ContinuousLinearMap.opNorm_le_bound _ _ fun a => _;
      · exact ContinuousLinearMap.opNorm_nonneg _;
      · convert ContinuousLinearMap.le_opNorm ( LinearMap.toContinuousLinearMap ( toEu
        · simp [ EuclideanSpace.norm_eq, Matrix.toEuclideanLin ];
          rw [ ← Equiv.sum_comp e.symm ]
          simp [ Matrix.mulVec, dotProduct ] ;
        · congr! 2;
          simp [ EuclideanSpace.norm_eq]
          conv_lhs => rw [ ← Equiv.sum_comp f.symm ] ;

```

```

/--
Define U_rho as the Kronecker product of V_rho and the identity.
-/
noncomputable def U_rho (pAB : HermitianMat (dA × dB) ℂ) : Matrix ((dA × dB) × (dA × dB)) ℂ :=
  Matrix.kronecker (V_rho pAB) (1 : Matrix dC dC ℂ)

/--
Define U_sigma as the Kronecker product of the identity and V_sigma.
-/
noncomputable def U_sigma (oBC : HermitianMat (dB × dC) ℂ) : Matrix (dA × (dB × dC)) ℂ :=
  Matrix.kronecker (1 : Matrix dA dA ℂ) (V_sigma oBC)

/--
The operator norm of the conjugate transpose is equal to the operator norm.
-/
theorem Matrix.opNorm_conjTranspose_eq_opNorm {m n : Type*} [Fintype m] [Fintype n] [DecidableEq m] [DecidableEq n] (A : Matrix m n ℂ) :
  Matrix.opNorm AH = Matrix.opNorm A := by
  unfold Matrix.opNorm
  rw [← ContinuousLinearMap.adjoint.norm_map (toEuclideanLin A).toContinuousLinearMap,
    toEuclideanLin_conjTranspose_eq_adjoint]
  rfl

theorem isometry_mul_conjTranspose_le_one {m n : Type*} [Fintype m] [Fintype n] [DecidableEq m] [DecidableEq n]
  (V : Matrix m n ℂ) (hV : V.conjTranspose * V = 1) :
  V * V.conjTranspose ≤ 1 := by
  have h_pos : (1 - V * VH) * (1 - V * VH) = 1 - V * VH := by
    simp [sub_mul, mul_sub, ← Matrix.mul_assoc];
    simp [Matrix.mul_assoc, hV];
  have h_pos : (1 - V * VH) = (1 - V * VH)H * (1 - V * VH) := by
    simp_all [Matrix.conjTranspose_sub, Matrix.conjTranspose_one, Matrix.conjTranspose_mul];
  have h_pos : Matrix.PosSemidef (1 - V * VH) := by
    rw [h_pos] at *; apply Matrix.posSemidef_conjTranspose_mul_self;
  grind

/-
If `A†A = I` and `B†B = I` (both isometries into the same space), then `|(A†B)†(A†B)| ≤ I`.
-/
theorem conjTranspose_isometry_mul_isometry_le_one {m n k : Type*} [Fintype m] [Fintype n] [Fintype k] [DecidableEq m] [DecidableEq n] [DecidableEq k]
  (A : Matrix k m ℂ) (B : Matrix k n ℂ) (hA : A.conjTranspose * A = 1) (hB : B.conjTranspose * B = 1) :
  (A.conjTranspose * B).conjTranspose * (A.conjTranspose * B) ≤ 1 := by

```

```

have h_le : (BH * A) * (BH * A)H ≤ 1 := by
  have h_le : (BH * A) * (BH * A)H ≤ (BH * B) := by
    have h_le : (A * AH) ≤ 1 := by
      apply isometry_mul_conjTranspose_le_one A hA;
    -- Apply the fact that if  $X \leq Y$ , then  $CXC^* \leq CYC^*$  for any matrix  $C$ 
    have h_conj : ∀ (C : Matrix n k ℂ) (X Y : Matrix k k ℂ), X ≤ Y → C * X * CH ≤ C * Y * CH := by
      fun C X Y a => Matrix.PosSemidef.mul_mul_conjTranspose_mono C a
    simp [Matrix.mul_assoc] using h_conj BH (A * AH) 1 h_le;
  aesop;
simp [Matrix.mul_assoc] using h_le

/-! ### Helper lemmas for operator_ineq_SSA -/

/- Reindexing preserves the HermitianMat ordering. -/
theorem HermitianMat.reindex_le_reindex_iff {d d₂ : Type*} [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂] (e : d ≈ d₂) (A B : HermitianMat d ℂ) :
  A.reindex e ≤ B.reindex e ↔ A ≤ B := by
  constructor <=> intro h <=> rw [HermitianMat.le_iff] at * <=> aesop;

/- Inverse of a Kronecker product of HermitianMat. -/
theorem HermitianMat.inv_kronecker {m n : Type*} [Fintype m] [DecidableEq m] [Fintype n] [DecidableEq n] [Nonempty m] [Nonempty n]
  (A : HermitianMat m ℂ) (B : HermitianMat n ℂ)
  [A.NonSingular] [B.NonSingular] :
  (A ⊗k B)-1 = A-1 ⊗k B-1 := by
  have h_inv : (A ⊗k B).mat * (A-1 ⊗k B-1).mat = 1 := by
    have h_inv : (A ⊗k B).mat * (A-1 ⊗k B-1).mat = (A.mat * A-1.mat) ⊗k (B.mat * B-1.mat) := by
      ext i j; simp [Matrix.mul_apply, Matrix.kroneckerMap];
    simp only [mul_assoc, Finset.sum_mul]
    simp only [Finset.mul_sum]
    rw [← Finset.sum_product]; congr; ext; ring!;
  aesop;
  refine' Subtype.ext (Matrix.inv_eq_right_inv h_inv)

/- Inverse of a reindexed HermitianMat. -/
theorem HermitianMat.inv_reindex {d d₂ : Type*} [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂] (A : HermitianMat d ℂ) (e : d ≈ d₂) :
  (A.reindex e)-1 = A-1.reindex e := by
  ext1
  simp

/- Kronecker of PosDef matrices is PosDef. -/
theorem HermitianMat.PosDef_kronecker {m n : Type*} [Fintype m] [DecidableEq m] [Fintype n] [DecidableEq n]
  (A : HermitianMat m ℂ) (B : HermitianMat n ℂ)
  (hA : A.mat.PosDef) (hB : B.mat.PosDef) :

```

```

    (A ⊗k B).mat.PosDef := by
    exact Matrix.PosDef.kron hA hB

/- Reindex of PosDef is PosDef. -/
theorem HermitianMat.PosDef_reindex {d d₂ : Type*} [Fintype d] [DecidableEq d]
[Fintype d₂] [DecidableEq d₂] (A : HermitianMat d ℂ) (e : d ≈ d₂)
(hA : A.mat.PosDef) :
(A.reindex e).mat.PosDef := by
convert hA.reindex e

/-- The sandwich matrix S used in the proof of intermediate_ineq.
This is derived from W_mat_sq_le_one by algebraic manipulation (conjugation)
/
private noncomputable def S_mat (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ) :
Matrix ((dA × dB) × dC) ((dA × dB) × dC) ℂ :=
(pAB.traceRight-1.sqrt.mat ⊗k σBC.sqrt.mat).reindex
(Equiv.prodAssoc dA dB dC).symm (Equiv.prodAssoc dA dB dC).symm

/- W†W = S * (p_AB ⊗ σ_C-1) * S, i.e. W†W equals the conj of LHS by S.
This follows from: W = (pAB1/2 ⊗ σC{-1/2}) * S, so W†W = S† * (pAB1/2 ⊗ σC{-1/2})† *
(pAB1/2 ⊗ σC{-1/2}) * S
= S * (pAB ⊗ σC-1) * S (using sqrt_sq and Hermiticity of S).
-/
private lemma W_mat_sq_eq_conj [Nonempty dA] [Nonempty dB] [Nonempty dC]
(pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
(hp : pAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
(W_mat pAB σBC)H * (W_mat pAB σBC) =
S_mat pAB σBC * (pAB ⊗k (σBC.traceLeft-1)).mat * S_mat pAB σBC := by
unfold W_mat S_mat;
simp [Matrix.mul_assoc, Matrix.conjTranspose_mul, Matrix.conjTranspose_k];
have h_simp : (pAB.sqrt : Matrix (dA × dB) (dA × dB) ℂ) * (pAB.sqrt : Matrix (dA × dB) (dA × dB) ℂ) = pAB;
exact h_simp;
· exact sqrt_sq (by positivity)
· convert sqrt_sq (show 0 ≤ (σBC.traceLeft-1 : HermitianMat dC ℂ) from
have h_inv_pos : (σBC.traceLeft-1 : HermitianMat dC ℂ).mat.PosDef :=
have h_inv_pos : (σBC.traceLeft : Matrix dC dC ℂ).PosDef := by
exact PosDef_traceLeft σBC hσ;
convert h_inv_pos.inv using 1;
convert h_inv_pos.posSemidef using 1;
exact zero_le_iff;
have h_simp : (pAB.sqrt : Matrix (dA × dB) (dA × dB) ℂ) ⊗k (σBC.traceLeft : Matrix dC dC ℂ) = (pAB.sqrt : Matrix (dA × dB) (dA × dB) ℂ) * (σBC.traceLeft : Matrix dC dC ℂ);
have h_simp : ∀ (A B C D : Matrix (dA × dB) (dA × dB) ℂ) (E F : Matrix dC dC ℂ)
exact fun A B C D E F => Eq.symm (Matrix.mul_kronecker_mul A B E F);
aesop;
simp_all [← Matrix.mul_assoc]

```

```

/- **Step 2**:  $S * (\rho_A \otimes \sigma_{BC}^{-1}).\text{reindex} * S = I$ .
This follows from  $(\rho_A^{\{-\frac{1}{2}\}} * \rho_A * \rho_A^{\{-\frac{1}{2}\}}) \otimes (\sigma_{BC}^{\frac{1}{2}} * \sigma_{BC}^{-1} * \sigma_{BC}^{\frac{1}{2}}) = I \otimes I =$ 
-/
private lemma S_mat_conj_rhs_eq_one [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : pAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  S_mat pAB σBC * ((pAB.traceRight ⊗k σBC-1).reindex (Equiv.prodAssoc dA dB dC).symm) = 1 := by
  S_mat pAB σBC = 1 := by
  have h_comm : Commute (σBC.sqrt.mat) (σBC-1.mat) := by
  have h_comm : Commute (σBC.sqrt.mat) (σBC.mat) := by
    exact commute_sqrt_left rfl;
  have h_comm_inv : Commute (σBC.sqrt.mat) (σBC.mat) → Commute (σBC.sqrt.mat) (σBC-1.mat) := by
    intro h_comm
  have h_comm_inv : Commute (σBC.sqrt.mat) (σBC.mat) → Commute (σBC.sqrt.mat) (σBC-1.mat) := by
    intro h_comm
  have h_inv : σBC.mat-1 * σBC.sqrt.mat = σBC.sqrt.mat * σBC.mat-1 := by
  have h_inv : σBC.mat-1 * σBC.sqrt.mat * σBC.mat = σBC.sqrt.mat := by
    simp [mul_assoc, h_comm.eq];
    rw [← mul_assoc, Matrix.nonsing_inv_mul _];
    · rw [one_mul];
    · exact isUnit_iff_ne_zero.mpr hσ.det_pos.ne';
  have h_inv : σBC.mat-1 * σBC.sqrt.mat * σBC.mat * σBC.mat-1 = σBC.sqrt.mat := by
    rw [h_inv];
  convert h_inv using 1; simp [mul_assoc, hσ.det_pos.ne'];
  exact h_inv.symm;
  convert h_comm_inv h_comm using 1;
  exact h_comm_inv h_comm;
  have h_comm : σBC.sqrt.mat * σBC-1.mat * σBC.sqrt.mat = σBC.mat * σBC-1.mat := by
    rw [mul_assoc, ← h_comm.eq];
    rw [← Matrix.mul_assoc, HermitianMat.sqrt_sq];
  convert hσ.posSemidef using 1;
  exact zero_le_iff;
  have h_comm : pAB.traceRight-1.sqrt.mat * pAB.traceRight.mat * pAB.traceRight-1.sqrt.mat = pAB.traceRight.mat := by
  have h_comm : pAB.traceRight.mat.PosDef := by
    apply_rules [PosDef_traceRight];
  convert sqrt_inv_mul_self_mul_sqrt_inv_eq_one h_comm using 1;
  convert congr_arg2 (fun x y => Matrix.kronecker x y |> Matrix.reindex (Equiv.prodAssoc dA dB dC).symm) using 1;
  · unfold S_mat; simp [*, Matrix.mul_assoc];
  ext (i, j) (k, l); simp [Matrix.mul_apply, Matrix.kroneckerMap_apply];
  simp only [mul_assoc, Finset.mul_sum _ _ _, Finset.sum_mul];
  simp only [← Finset.sum_product'];
  refine' Finset.sum_bij (fun x _ => (x.1.2, x.2.2, x.1.1, x.2.1)) _ _ _ _ <=>
  exact fun _ _ _ _ => Or.inl <| by ring;
  · ext (i, j) (k, l)
  simp [Matrix.one_apply]
aesop

```

```

· have := hσ.det_pos.ne'
  have := Matrix.nonsing_inv_mul _ (isUnit_iff_ne_zero.mpr hσ.det_pos.ne')
  aesop;

/- Key factorization:  $W_{\text{mat}} = (F \otimes I_C) * (I_A \otimes G)$ .reindex, where
   $F = \rho_{AB}.\text{sqrt} * (\rho_A^{-1}.\text{sqrt} \otimes I_B)$  and  $G = (I_B \otimes \sigma_C^{-1}.\text{sqrt}) * \sigma_{BC}.\text{sqrt}$ .
  This expresses  $W$  as a "contraction over the shared  $B$  index". -
/
private lemma W_mat_eq_factored
  (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ) :
  W_mat ρAB σBC =
    let F := ρAB.sqrt.mat * (ρAB.traceRight-1.sqrt.mat ⊗k (1 : Matrix dB dB ℂ))
    let G := ((1 : Matrix dB dB ℂ) ⊗k σBC.traceLeft-1.sqrt.mat) * σBC.sqrt.mat
    (F ⊗k (1 : Matrix dC dC ℂ)) *
    ((1 : Matrix dA dA ℂ) ⊗k G).reindex
    (Equiv.prodAssoc dA dB dC).symm (Equiv.prodAssoc dA dB dC).symm :
  -- By definition of matrix multiplication and the properties of the Kronecker product
  ext ⟨a, b⟩, c ⟨a', b'⟩, c'; simp [Matrix.mul_apply, Matrix.kroneckerMap_apply, map_to_tensor_MES]
  unfold W_mat; simp [Matrix.mul_apply, Matrix.kroneckerMap_apply, Matrix.reindex_apply]
  simp [Finset.sum_ite, Finset.mul_sum, Finset.sum_mul, mul_assoc, mul_comm]
  simp [Finset.sum_filter, Finset.sum_sigma'];
  rw [← Finset.sum_filter];
  refine' Finset.sum_bij (fun x _ => ⟨⟨a', x.1.2⟩, c⟩, ⟨⟨x.1.2, x.2.1⟩, c⟩);
  aesop

/- Connection between  $F$  and  $V_{\text{rho}}$  via the MES:
   $(F \otimes I_B) * \text{MES} = V_{\text{rho}}$ , where  $F = \rho_{AB}.\text{sqrt} * (\rho_A^{-1}.\text{sqrt} \otimes I_B)$ . -
/
private lemma F_tensor_MES_eq_V_rho
  (ρAB : HermitianMat (dA × dB) ℂ) :
  let F := ρAB.sqrt.mat * (ρAB.traceRight-1.sqrt.mat ⊗k (1 : Matrix dB dB ℂ))
  (F ⊗k (1 : Matrix dB dB ℂ)) * map_to_tensor_MES dA dB = V_rho ρAB := by
  ext ⟨i, j⟩ k; simp [Matrix.mul_apply, Matrix.kroneckerMap_apply, map_to_tensor_MES]
  unfold V_rho; simp [Matrix.mul_apply, Matrix.kroneckerMap_apply, map_to_tensor_MES]
  simp [Finset.sum_ite, Matrix.one_apply, mul_comm, Finset.mul_sum]
  rw [Finset.sum_sigma']; simp [eq_comm, Finset.filter_filter];
  refine' Finset.sum_bij (fun x _ => ⟨⟨k, j⟩, j⟩, x, j) _ _ _ _ <
  · aesop (simp_config := { singlePass := true });
  · intro a; rw [Finset.sum_eq_single ⟨(a, j), j⟩] <;> aesop;

section Wmat_calculation

variable {dA dB dC : Type*} [Fintype dA] [Fintype dB] [Fintype dC]
variable [DecidableEq dA] [DecidableEq dB] [DecidableEq dC]

abbrev BigIdx (dA dB dC : Type*) := ((dA × dB) × dB) × (dB × dC)

```



```

abbrev SmallIdx (dA dB dC : Type*) := (dA × dB) × dC
abbrev MidIdx (dA dB dC : Type*) := (dA × dB) × (dB × (dB × dC))

/-- The associator equivalence (no swap needed).
    Maps (((dA×dB)×dB)×(dB×dC)) to ((dA×dB)×(dB×(dB×dC))). -
/
private def assoc_equiv (dA dB dC : Type*) :
  BigIdx dA dB dC ≈ MidIdx dA dB dC :=
  Equiv.prodAssoc (dA × dB) dB (dB × dC)

variable (dA dB dC) in
private def T1_mat (pAB : HermitianMat (dA × dB) ℂ) :
  Matrix (BigIdx dA dB dC) (SmallIdx dA dB dC) ℂ :=
  (V_rho pAB ⊗_k (1 : Matrix (dB × dC) (dB × dC) ℂ)).reindex
    (Equiv.refl _) (Equiv.prodAssoc dA dB dC).symm

variable (dA dB dC) in
private def T2_mat (σBC : HermitianMat (dB × dC) ℂ) :
  Matrix (SmallIdx dA dB dC) (MidIdx dA dB dC) ℂ :=
  (1 : Matrix (dA × dB) (dA × dB) ℂ) ⊗_k (V_sigma σBC)H

private def PERM_mat (dA dB dC : Type*) [Fintype dA] [Fintype dB] [Fintype dC]
  [DecidableEq dA] [DecidableEq dB] [DecidableEq dC] :
  Matrix (MidIdx dA dB dC) (BigIdx dA dB dC) ℂ :=
  (1 : Matrix (BigIdx dA dB dC) (BigIdx dA dB dC) ℂ).reindex
    (assoc_equiv dA dB dC) (Equiv.refl _)

private lemma T1_isometry [Nonempty dB]
  (pAB : HermitianMat (dA × dB) ℂ) (hp : pAB.mat.PosDef) :
  (T1_mat dA dB dC pAB)H * (T1_mat dA dB dC pAB) = 1 := by
  have h_kron : (V_rho pAB ⊗_k (1 : Matrix (dB × dC) (dB × dC) ℂ))H *
    (V_rho pAB ⊗_k (1 : Matrix (dB × dC) (dB × dC) ℂ)) = 1 := by
    have hV := V_rho_isometry pAB hp
    convert congr_arg (fun m => Matrix.kroneckerMap (· * ·) m (1 : Matrix (dB × dC)
      · ext i j
        simp [Matrix.mul_apply, Matrix.kroneckerMap, Matrix.one_apply]
        by_cases hij : i.2 = j.2 <=> simp [hij, Finset.sum_ite]
        · exact Finset.sum_bij (fun x _ => x.1) (by aesop) (by aesop) (by aesop) (by aesop)
        · exact Finset.sum_eq_zero (by aesop)
      · ext i j
        simp [Matrix.one_apply]
        aesop
    convert congr_arg (Matrix.reindex (Equiv.prodAssoc dA dB dC).symm (Equiv.prodAssoc
      ext i j
      simp [Matrix.one_apply]
      aesop

```

```

set_option maxHeartbeats 400000 in
private lemma T2_sq_le_one [Nonempty dB]
  (σBC : HermitianMat (dB × dC) ℂ) (hσ : σBC.mat.PosDef) :
  (T2_mat dA dB dC σBC)H * (T2_mat dA dB dC σBC) ≤ 1 := by
  have hT2_isometry : (V_sigma σBC).conjTranspose * (V_sigma σBC) = 1 :=
    V_sigma_isometry σBC hσ
  convert isometry_mul_conjTranspose_le_one (Matrix.kronecker (1 : Matrix (
    · ext {i, j} {k, l}
      simp [Matrix.mul_apply]
      ring_nf
      unfold T2_mat
      simp [Matrix.one_apply]
      ring_nf
      congr! 2
      · exact eq_comm
      · aesop
    · convert congr_arg (Matrix.kronecker (1 : Matrix (dA × dB) (dA × dB) ℂ))
      · ext {i, j} {k, l}
        simp [Matrix.mul_apply]
        ring_nf
        simp [Matrix.one_apply, mul_comm, mul_left_comm, Finset.mul_sum _ _
          split_ifs <=> simp_all [Finset.sum_ite]
          subst i
          refine' Finset.sum_bij (fun x _ => x.2) _ _ _
          · aesop
          · aesop
          · aesop
          · aesop
          · aesop
      · aesop

private lemma PERM_isometry : (PERM_mat dA dB dC)H * PERM_mat dA dB dC = 1
  simp [PERM_mat]

/-- Element-wise identity: W_mat = ∑_{b*} V_rho * V_sigma†.
    This is the key computation from Eq. (6) of Lin-Kim-Hsieh. -
/
private lemma W_mat_entry (pAB : HermitianMat (dA × dB) ℂ) (σBC : Hermitian
  (i j : SmallIdx dA dB dC) :
  W_mat pAB σBC i j =
    ∑ b_star : dB,
      V_rho pAB ((i.1, b_star)) j.1.1 *
      (V_sigma σBC)H i.2 (b_star, (j.1.2, j.2)) := by
  obtain ⟨(a, b), c⟩ := i
  obtain ⟨(a', b'), c'⟩ := j
  simp only [W_mat, V_sigma, V_rho, map_to_tensor_MES]

```

```

simp only [Matrix.mul_apply, Matrix.kroneckerMap_apply, Matrix.reindex_apply,
  Matrix.submatrix_apply, Matrix.conjTranspose_apply, Matrix.one_apply,
  Matrix.of_apply, Equiv.prodAssoc_apply,
  Equiv.prodComm_apply, Equiv.prodComm_symm, Equiv.refl_apply, Equiv.refl_symm,
  Equiv.prodCongr_apply, Equiv.prodCongr_symm,
  Equiv.symm_trans_apply,
  Prod.swap, Equiv.symm_symm, Prod.map_apply]
simp only [apply_ite_star, star_zero, mul_ite, mul_one, mul_zero, star_mul', star_
simp only [Finset.mul_sum, Finset.sum_mul]
rw [show Finset.univ (α := (dA × dB) × dC) =
  (Finset.univ ×s Finset.univ : Finset ((dA × dB) × dC))
  from Finset.univ_product_univ_symm]
rw [Finset.sum_product]
rw [show Finset.univ (α := dA × dB) =
  (Finset.univ ×s Finset.univ : Finset (dA × dB))
  from Finset.univ_product_univ_symm]
simp_rw [Finset.sum_product]
rw [Finset.sum_comm (s := Finset.univ (α := dA))]
simp_rw [← Finset.sum_mul, ← Finset.mul_sum]
-- Goal: LHS (triple sum, clean) = RHS (sums with ite conditions)
-- Strategy: simplify ite sums on RHS, relate reindex terms, match.
-- Step 1: collapse ite sums using Finset.sum_eq_single
-- Convert ↑A i j to A i j throughout
simp only [HermitianMat.mat_apply]
-- Collapse ite sums: ρ part
have h_ite_rho : ∀ (i : dA) (x : dB) (f : dA × dB → ℂ),
  (∑ i_1 : (dA × dB) × dB, if i_1.1.1 = i ∧ i_1.1.2 = i_1.2 then if x = i_1.2 th
    f (i, x) := by
    intro i x f; rw [Finset.sum_eq_single ((i, x), x)]
    · simp
    · intro ((a_0, b_0), b_1) _ hne; split_ifs with h h2 <=> simp_all [Prod.mk.injEq]
    · intro h; exact absurd (Finset.mem_univ _) h
simp_rw [h_ite_rho]
-- σ part: pull out constant factor, then collapse
simp_rw [← Finset.sum_mul]
-- Now the σ inner sum has the same shape as h_ite_rho but for dC×dB
have h_ite_sigma : ∀ (i : dC) (x : dB) (f : dC × dB → ℂ),
  (∑ i_1 : (dC × dB) × dB, if i_1.1.1 = i ∧ i_1.1.2 = i_1.2 then if x = i_1.2 th
    f (i, x) := by
    intro i x f; rw [Finset.sum_eq_single ((i, x), x)]
    · simp
    · intro ((a_0, b_0), b_1) _ hne; split_ifs with h h2 <=> simp_all [Prod.mk.injEq]
    · intro h; exact absurd (Finset.mem_univ _) h
-- Collapse σ ite sum using conv to navigate into the RHS
conv_rhs =>
  arg 2; ext x_outer

```

```

    arg 2; arg 2; ext x_1
    arg 1
    rw [h_ite_sigma x_1 x_outer (fun p => star ((σBC.reindex (Equiv.prodComm
-- Now relate reindexed σBC terms to original terms
-- cfc_reindex: (A.reindex e).sqrt = A.sqrt.reindex e
simp only [show (σBC.reindex (Equiv.prodComm dB dC)).sqrt =
    σBC.sqrt.reindex (Equiv.prodComm dB dC) from
    σBC.cfc_reindex Real.sqrt (Equiv.prodComm dB dC)]
-- traceRight of reindex(prodComm) = traceLeft
simp only [show (σBC.reindex (Equiv.prodComm dB dC)).traceRight = σBC.traceRight
    ext i j; simp only [HermitianMat.traceRight, HermitianMat.traceLeft,
        Matrix.traceRight, Matrix.traceLeft, HermitianMat.reindex,
        Matrix.reindex_apply, HermitianMat.mat_mk, Matrix.of_apply,
        Matrix.submatrix_apply, Equiv.prodComm_symm, Equiv.prodComm_apply, Pr
-- Hermiticity
have h1 : σBC.sqrt.matH = σBC.sqrt.mat := σBC.sqrt.2
have h2 : σBC.traceLeft-1.sqrt.matH = σBC.traceLeft-1.sqrt.mat := σBC.traceLeft-1.sqrt.mat.2
-- Expand product of sums and match
simp_rw [Finset.sum_mul, Finset.mul_sum]
-- Both sides are triple sums with the same structure. Match element-
wise.
refine Finset.sum_congr rfl fun x_bd _ => Finset.sum_congr rfl fun x_1 _ =>
    Finset.sum_congr rfl fun x_2 _ => ?_
-- Now a single-term equality
-- star of reindexed entry: def-eq gives star(σBC.sqrt (b',c') (x_bd,x_2))
-- then Hermiticity gives σBC.sqrt (x_bd,x_2) (b',c')
have hs1 : star ((σBC.sqrt.reindex (Equiv.prodComm dB dC)) (c', b') (x_2,
    σBC.sqrt (x_bd, x_2) (b', c')) :=
    CStarMatrix.star_apply_of_isSelfAdjoint h1
have hs2 : star (σBC.traceLeft-1.sqrt x_2 c) = σBC.traceLeft-1.sqrt c x_2
    CStarMatrix.star_apply_of_isSelfAdjoint h2
rw [hs1, hs2]; ring

/-- Element-wise identity: RHS = ∑_{b*} V_rho * V_sigma†. -
/
private lemma RHS_entry (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dB) ℂ) (i j : SmallIdx dA dB dC) :
    (T2_mat dA dB dC σBC * PERM_mat dA dB dC * T1_mat dA dB dC pAB) i j =
        ∑ b_star : dB,
            V_rho pAB ((i.1, b_star)) j.1.1 *
            (V_sigma σBC)H i.2 (b_star, (j.1.2, j.2)) := by
    obtain ⟨(a, b), c⟩ := i
    obtain ⟨(a', b'), c'⟩ := j
    simp only [Matrix.mul_apply, T2_mat, T1_mat, PERM_mat, assoc_equiv,
        Matrix.kroneckerMap_apply, Matrix.one_apply, Matrix.reindex_apply,
        Equiv.prodAssoc_symm_apply, Equiv.refl_symm, Equiv.refl_apply,

```

```

    Equiv.prodAssoc_apply, Matrix.conjTranspose_apply,
    Matrix.submatrix_apply, Equiv.symm_symm]
simp only [ite_mul, one_mul, zero_mul]
simp only [mul_ite, mul_one, mul_zero]
have h_inner : ∀ (x : BigIdx dA dB dC),
  (∑ x_1 : MidIdx dA dB dC,
    if (a, b) = x_1.1
      then if ((x_1.1, x_1.2.1), x_1.2.2) = x then star (V_sigma σBC x_1.2 c) else 0
      else 0) =
    if x.1.1 = (a, b) then star (V_sigma σBC (x.1.2, x.2) c) else 0 := by
  intro ⟨⟨p, q⟩, r⟩
  rw [Finset.sum_eq_single ⟨(a, b), (q, r)⟩]
  · simp [eq_comm]
  · intro ⟨s, ⟨t, u⟩⟩ _ hne; split_ifs with h1 h2 <=> try rfl
    exfalso; apply hne; ext <=> grind only
  · intro h; exact absurd (Finset.mem_univ _) h
simp_rw [h_inner, ite_mul, zero_mul]
rw [show Finset.univ (α := BigIdx dA dB dC) = (Finset.univ ×ˢ Finset.univ : Finset
  from Finset.univ_product_univ.symm)]
rw [Finset.sum_product]
simp only [Finset.sum_ite_eq', Finset.mem_univ, ↓reduceIte]
rw [show Finset.univ (α := (dA × dB) × dB) = (Finset.univ ×ˢ Finset.univ : Finset
  from Finset.univ_product_univ.symm)]
rw [Finset.sum_product]
simp only [RCLike.star_def, Finset.sum_ite_irrel, Finset.sum_const_zero, Finset.su
  Finset.mem_univ, ↓reduceIte]
simp_rw [mul_comm]

private lemma W_mat_eq_three_factors [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ) :
  W_mat ρAB σBC =
    T2_mat dA dB dC σBC * PERM_mat dA dB dC * T1_mat dA dB dC ρAB := by
  ext i j
  rw [W_mat_entry, RHS_entry]

/-- Core inequality:  $W^\dagger W \leq I$ .
This is the key step, following from the isometry argument:
 $V_\rho \otimes I_C$  and  $I_A \otimes V_\sigma$  are isometries, their cross product has norm  $\leq 1$ ,
and the result can be related to  $W_\text{mat}$  through the MES computation (Eq. 6 in Lin-
Kim-Hsieh). -/
theorem W_mat_sq_le_one [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : ρAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  (W_mat ρAB σBC)ᵂ * (W_mat ρAB σBC) ≤ 1 := by
  rw [W_mat_eq_three_factors]
  have h_T2 := T2_sq_le_one (dA := dA) σBC hσ

```

```

have h_step1 : (PERM_mat dA dB dC)H * ((T2_mat dA dB dC σBC)H * (T2_mat dA dB dC σBC))H * (T2_mat dA dB dC σBC) ≤ 1 := by
  calc _ ≤ (PERM_mat dA dB dC)H * 1 * PERM_mat dA dB dC :=
    Matrix.PosSemidef.conjTranspose_mul_mul_mono _ h_T2
  _ = 1 := by rw [Matrix.mul_one, PERM_isometry]
calc _ = (T1_mat dA dB dC ρAB)H * ((PERM_mat dA dB dC)H * ((T2_mat dA dB dC σBC)H * (T2_mat dA dB dC σBC)) * PERM_mat dA dB dC) * (T1_mat dA dB dC ρAB) := by
  simp [Matrix.conjTranspose_mul, Matrix.mul_assoc]
_ ≤ (T1_mat dA dB dC ρAB)H * 1 * (T1_mat dA dB dC ρAB) :=
  Matrix.PosSemidef.conjTranspose_mul_mul_mono _ h_step1
_ = 1 := by rw [Matrix.mul_one, T1_isometry _ hp]

end Wmat_calculation

/- S_mat is invertible (since ρ_A and σ_BC are positive definite). -
/
private lemma S_mat_isUnit [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : ρAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  IsUnit (S_mat ρAB σBC) := by
  rw [Matrix.isUnit_iff_isUnit_det];
  -- Since ρAB and σBC are positive definite, their square roots are also invertible
  have h_inv_sqrt : IsUnit (ρAB.traceRight-1.sqrt.mat.det) ∧ IsUnit (σBC.traceLeft-1.sqrt.mat.det) := by
    constructor;
    · have h_det_ne_zero : ∀ (A : HermitianMat (dA) ℂ), A.mat.PosDef → (A.traceRight-1.sqrt.mat.det) ≠ 0
      intro A hA
      have h_det_ne_zero : (A.traceRight-1.sqrt.mat.PosDef) := by
        exact sqrt_posDef hA;
        exact h_det_ne_zero.det_pos.ne';
      exact isUnit_iff_ne_zero.mpr (h_det_ne_zero _ <| by simp using PosDef_pos)
    · have h_inv_sqrt : σBC.traceLeft-1.sqrt.mat.PosDef := by
        exact sqrt_posDef hσ;
        exact isUnit_iff_ne_zero.mpr h_inv_sqrt.det_pos.ne';
  unfold S_mat;
  simp_all [Matrix.det_kronecker]

/-- The intermediate operator inequality: ρ_AB ⊗ σ_C-1 ≤ (ρ_A ⊗ σ_BC-1).reindex (Equiv.prodAssoc dA dB dC).symm :=
This is derived from W_mat_sq_le_one by algebraic manipulation (conjugation)
/
theorem intermediate_ineq [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (ρAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : ρAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  ρAB ⊗k (σBC.traceLeft-1)-1 ≤
  (ρAB.traceRight ⊗k σBC-1).reindex (Equiv.prodAssoc dA dB dC).symm :=
  have h_sorted : (ρAB.traceRight-1.sqrt.mat ⊗k σBC.traceLeft-1.sqrt.mat).reindex (Equiv.prodAssoc dA dB dC).symm ≤

```

```

    convert W_mat_sq_le_one pAB σBC hp hσ using 1;
    convert W_mat_sq_eq_conj pAB σBC hp hσ |> Eq.symm using 1;
    convert h_sorted using 1;
    rw [HermitianMat.le_iff];
    rw [← S_mat_conj_rhs_eq_one pAB σBC hp hσ ];
    simp only [ Matrix.posSemidef_iff_dotProduct_mulVec ]
    constructor <=> intro h <=> simp_all
  · convert h_sorted using 1;
    convert S_mat_conj_rhs_eq_one pAB σBC hp hσ using 1;
  · have := S_mat_isUnit pAB σBC hp hσ;
    cases' this.nonempty_invertible with u hu;
    have h_pos_semidef : Matrix.PosSemidef ((S_mat pAB σBC)-1 * (S_mat pAB σBC * ((p
      exact Matrix.PosSemidef.mul_mul_conjTranspose_same h (S_mat pAB σBC)-1;
    simp_all [ Matrix.mul_assoc, Matrix.sub_mul, Matrix.mul_sub ];
    simp_all [ Matrix.posSemidef_iff_dotProduct_mulVec, Matrix.IsHermitian ];
    have h_conj : (S_mat pAB σBC)H = S_mat pAB σBC := by
      unfold S_mat; simp [ Matrix.conjTranspose_kronecker, Matrix.conjTranspose_subm
    simp_all [ Matrix.conjTranspose_nonsing_inv ]

open HermitianMat in
/-- **Operator extension of SSA** (Main result of Lin-Kim-
Hsieh).
  For positive definite ρ_AB and σ_BC:
  `ρ_A-1 ⊗ σ_BC ≤ ρ_AB-1 ⊗ σ_C`
  where ρ_A = Tr_B(ρ_AB) and σ_C = Tr_B(σ_BC), and the RHS is reindexed
  via the associator `(dA × dB) × dC ≈ dA × (dB × dC)`. -
/
theorem operator_ineq_SSA [Nonempty dA] [Nonempty dB] [Nonempty dC]
  (pAB : HermitianMat (dA × dB) ℂ) (σBC : HermitianMat (dB × dC) ℂ)
  (hp : pAB.mat.PosDef) (hσ : σBC.mat.PosDef) :
  pAB.traceRight-1 ⊗k σBC ≤
  (pAB-1 ⊗k σBC.traceLeft).reindex (Equiv.prodAssoc dA dB dC) := by
  have h_inv_symm : ((pAB.traceRight ⊗k σBC-1).reindex (Equiv.prodAssoc dA dB dC).sy
    apply HermitianMat.inv_antitone;
  · apply HermitianMat.PosDef_kronecker pAB (σBC.traceLeft)-1 hp (PosDef_traceLeft
  · exact intermediate_ineq pAB σBC hp hσ;
  have h_inv_symm : ((pAB.traceRight ⊗k σBC-1).reindex (Equiv.prodAssoc dA dB dC).sy
    have h_inv_symm : (pAB.traceRight ⊗k σBC-1)-1 = pAB.traceRight-1 ⊗k (σBC-1)-1 :=
      convert HermitianMat.inv_kronecker _ _ using 1;
  · infer_instance;
  · exact { ( Classical.arbitrary dB, Classical.arbitrary dC ) };
  · have h_trace_right_pos_def : (pAB.traceRight).mat.PosDef := by
      exact PosDef_traceRight pAB hp
    exact (by exact PosDef_traceRight pAB hp |>.isUnit)
  · have h_inv_symm : σBC-1.NonSingular := by
      have h_inv_symm : σBC.NonSingular := by

```

```

        exact nonSingular_of_posDef hσ
        exact nonSingular_iff_inv.mpr h_inv_symm;
    exact h_inv_symm;
    convert congr_arg ( fun x : HermitianMat _ => x.reindex ( Equiv.prodA
    · apply HermitianMat.inv_reindex
    · convert rfl;
    apply HermitianMat.ext;
    convert Matrix.nonsing_inv_nonsing_inv _;
    exact isUnit_iff_ne_zero.mpr ( hσ.det_pos.ne' );
    have h_inv_symm : (ρAB ⊗k σBC.traceLeft-1)-1 = ρAB-1 ⊗k σBC.traceLeft :=
    have h_inv_symm : (ρAB ⊗k σBC.traceLeft-1)-1 = ρAB-1 ⊗k (σBC.traceLeft-1
    convert HermitianMat.inv_kronecker ρAB ( σBC.traceLeft-1 ) using 1;
    · exact nonSingular_of_posDef hp;
    · have h_inv_symm : σBC.traceLeft.mat.PosDef := by
        exact PosDef_traceLeft σBC hσ;
    -- Since σBC.traceLeft is positive definite, its inverse is also po
singular.
    have h_inv_pos_def : (σBC.traceLeft-1).mat.PosDef := by
        convert h_inv_symm.inv using 1;
    exact nonSingular_of_posDef h_inv_pos_def;
    convert h_inv_symm using 1;
    have h_inv_symm : (σBC.traceLeft-1)-1 = σBC.traceLeft := by
    have h_inv_symm : (σBC.traceLeft-1).mat * σBC.traceLeft.mat = 1 := by
    have h_inv_symm : (σBC.traceLeft-1).mat * σBC.traceLeft.mat = 1 :=
        have h_inv_symm : σBC.traceLeft.mat.PosDef := by
            exact PosDef_traceLeft σBC hσ
        convert Matrix.nonsing_inv_mul _;
        exact isUnit_iff_ne_zero.mpr h_inv_symm.det_pos.ne';
    exact h_inv_symm
    have h_inv_symm : (σBC.traceLeft-1 : HermitianMat dC ℂ).mat-1 = σBC.t
    rw [ Matrix.inv_eq_right_inv h_inv_symm ];
    exact Eq.symm (HermitianMat.ext (id (Eq.symm h_inv_symm)));
    rw [h_inv_symm];
    have h_inv_symm : (ρAB.traceRight-1 ⊗k σBC).reindex (Equiv.prodAssoc dA d
    aesop;
    convert HermitianMat.reindex_le_reindex_iff ( Equiv.prodAssoc dA dB dC )

open scoped InnerProductSpace RealInnerProductSpace

/-! ### Weak monotonicity and SSA proof infrastructure -/
section SSA_proof

omit [DecidableEq d₁] in
open HermitianMat in
private lemma inner_kron_one_eq_inner_traceRight
    (A : HermitianMat d₁ ℂ) (M : HermitianMat (d₁ × d₂) ℂ) :

```



```

    (A ⊗k (1 : HermitianMat d2 ℂ), M) = (A, M.traceRight) := by
  rw [inner_comm];
  -- By definition of partial trace, we have that the trace of M multiplied by (A ⊗
  have h_partial_trace : Matrix.trace (M.mat * (A.mat ⊗k 1 : Matrix (d1 × d2) (d1 ×
    simp [ Matrix.trace, Matrix.mul_apply ];
    simp [ Matrix.traceRight, Matrix.one_apply, mul_comm ];
    simp only [Finset.sum_sigma', Finset.mul_sum _ _ _];
    rw [ ← Finset.sum_filter ];
    refine' Finset.sum_bij ( fun x _ => ( x.snd.1, x.fst.1, x.fst.2 ) ) _ _ _ _ <=>
  exact congr_arg Complex.re h_partial_trace

omit [DecidableEq d2] in
open HermitianMat in
private lemma inner_one_kron_eq_inner_traceLeft
  (B : HermitianMat d2 ℂ) (M : HermitianMat (d1 × d2) ℂ) :
  ((1 : HermitianMat d1 ℂ) ⊗k B, M) = (B, M.traceLeft) := by
  convert inner_kron_one_eq_inner_traceRight B ( M.reindex (Equiv.prodComm d1 d2) )
  refine' congr_arg ( fun x : ℂ => x.re ) _;
  refine' Finset.sum_bij ( fun x y => ( x.2, x.1 ) ) _ _ _ _ <=> simp [ Matrix.mul_a
  intro a b; rw [ ← Equiv.sum_comp (Equiv.prodComm d1 d2) ]
  simp [ Matrix.one_apply, mul_comm ]

open HermitianMat in
private lemma hermitianMat_log_inv_eq_neg
  (A : HermitianMat d1 ℂ) [A.NonSingular] : A-1.log = -
  A.log := by
  -- By the property of continuous functional calculus, the logarithm of the inverse
  have h_log_inv : A-1.log = A.cfc (Real.log ∘ (·-1)) := by
    have h_log_inv : A-1 = A.cfc (·-1) := by
      exact Eq.symm HermitianMat.cfc_inv;
    rw [ h_log_inv, HermitianMat.log ];
    exact Eq.symm (HermitianMat.cfc_comp A (fun x => x-1) Real.log);
  simp [ HermitianMat.log ];
  convert congr_arg ( fun f => A.cfc f ) ( show Real.log ∘ ( fun x => x-1 ) = -
  Real.log from funext fun x => ?_ ) using 1
  · exact Eq.symm (HermitianMat.cfc_neg A Real.log);
  · by_cases hx : x = 0 <=> simp [ hx, Real.log_inv ]

private lemma PosDef_assoc'_traceRight
  (p : MState (d1 × d2 × d3)) (hp : p.M.mat.PosDef) :
  p.assoc'.traceRight.M.mat.PosDef := by
  have _ := p.nonempty |> nonempty_prod.mp |>.right |> nonempty_prod.mp |>.right
  apply PosDef_traceRight
  convert hp.reindex (Equiv.prodAssoc d1 d2 d3).symm

private lemma wm_inner_lhs [Nonempty d1] [Nonempty d2] [Nonempty d3]

```

```

    (p : MState (d1 × d2 × d3)) :
    [(-p.assoc'.traceRight.M.traceRight.log) ⊗k (1 : HermitianMat (d2 × d3))
    (1 : HermitianMat d1 ℂ) ⊗k p.traceLeft.M.log, p.M] =
    Svn p.traceRight - Svn p.traceLeft := by
    convert congr_arg2 ( · + · ) _ _ using 1;
    convert inner_add_left _ _ _ using 1;
    · rw [ Svn_eq_neg_trace_log ];
    convert inner_kron_one_eq_inner_traceRight _ _ using 1;
    simp [ HermitianMat.traceRight ];
    congr! 2;
    congr! 1;
    ext i j; simp [ Matrix.traceRight ];
    exact Fintype.sum_prod_type fun x => p.m (i, x) (j, x)
    · rw [ Svn_eq_neg_trace_log ];
    simp [ inner_one_kron_eq_inner_traceLeft ]

private lemma wm_inner_rhs [Nonempty d1] [Nonempty d2] [Nonempty d3]
    (p : MState (d1 × d2 × d3)) :
    [((-p.assoc'.traceRight.M.log) ⊗k (1 : HermitianMat d3 ℂ) +
    (1 : HermitianMat (d1 × d2) ℂ) ⊗k p.traceLeft.M.traceLeft.log).reindex
    (Equiv.prodAssoc d1 d2 d3), p.M] =
    Svn p.assoc'.traceRight - Svn p.traceLeft.traceLeft := by
    simp [ HermitianMat.traceLeft, HermitianMat.traceRight, Svn_eq_neg_trace_log ]
    simp [ inner_add_left, inner_one_kron_eq_inner_traceLeft, inner_kron_one_eq_inner_traceLeft ]
    congr! 2;
    convert MState.traceLeft_assoc' p using 1;
    unfold MState.assoc' MState.traceLeft; aesop;

/-- Weak monotonicity (form 2) for positive definite states:
    S(ρ12) + S(ρ23) ≥ S(ρ1) + S(ρ3).
    Proved by applying operator_ineq_SSA, taking log, then inner product with
    /
private lemma Svn_wm_pd [Nonempty d1] [Nonempty d2] [Nonempty d3]
    (p : MState (d1 × d2 × d3)) (hp : p.M.mat.PosDef) :
    Svn p.traceRight + Svn p.traceLeft.traceLeft ≤
    Svn p.assoc'.traceRight + Svn p.traceLeft := by
    -- Set up marginals and their PD properties
    have h12 := PosDef_assoc'_traceRight p hp
    have h23 := PosDef_traceLeft p.M hp
    haveI : p.assoc'.traceRight.M.NonSingular := nonSingular_of_posDef h12
    haveI : p.traceLeft.M.NonSingular := nonSingular_of_posDef h23
    haveI : p.assoc'.traceRight.M.traceRight.NonSingular :=
        nonSingular_of_posDef (PosDef_traceRight _ h12)
    haveI : p.traceLeft.M.traceLeft.NonSingular :=
        nonSingular_of_posDef (PosDef_traceLeft _ h23)
    -- Step 1: Operator inequality

```

```

have h_op := operator_ineq_SSA ρ.assoc'.traceRight.M ρ.traceLeft.M h12 h23
-- Step 2: Take log
have h_lhs_pd : (ρ.assoc'.traceRight.M.traceRight-1 ⊗k ρ.traceLeft.M).mat.PosDef :
  HermitianMat.PosDef_kronecker __ (PosDef_traceRight _ h12).inv h23
have h_log := HermitianMat.log_mono h_lhs_pd h_op
-- Step 3: Simplify logs
rw [HermitianMat.log_kron, hermitianMat_log_inv_eq_neg] at h_log
rw [HermitianMat.reindex_log, HermitianMat.log_kron, hermitianMat_log_inv_eq_neg]
-- Step 4: Take inner product with ρ.M (≥ 0)
have h_inner := HermitianMat.inner_mono ρ.nonneg h_log
-- Step 5: Use commutativity to match wm_inner_lhs/rhs forms
rw [HermitianMat.inner_comm, HermitianMat.inner_comm ρ.M] at h_inner
rw [wm_inner_lhs ρ, wm_inner_rhs ρ] at h_inner
linarith

```

```
private lemma MState.approx_by_pd
```

```

(ρ : MState d1) :
  ∃ (pn : ℕ → MState d1), (∀ n, (pn n).M.mat.PosDef) ∧
    Filter.Tendsto pn Filter.atTop (nhds ρ) := by
  have h_ne1 := ρ.nonneg
  -- Define the sequence of PD states pn as the mixture of ρ with the uniform state
  set εn : ℕ → ℝ := fun n => 1 / (n + 2)
  set pn : ℕ → MState d1 := fun n => Mixable.mix ⟨εn n, by
    exact ⟨ by positivity, by rw [div_le_iff₀] <=> linarith ⟩⟩ (MState.uniform) ρ
  refine' ⟨ pn, _, _ ⟩;
  · intro n
    have h_pos_def : (pn n).M = (1 - εn n) • ρ.M + εn n • (MState.uniform (d := d1))
      refine' add_comm _ _ |> Eq.trans <| _;
      congr! 1
      aesop;
    have h_pos_def : ∀ (A : Matrix d1 d1 ℂ), A.PosSemidef → ∀ (B : Matrix d1 d1 ℂ),
      intro A hA B hB ε hε
      simp only [Matrix.posSemidef_iff_dotProduct_mulVec, Matrix.posDef_iff_dotProd
        constructor <=> simp_all
      · simp_all [Matrix.IsHermitian, Matrix.conjTranspose_add, Matrix.conjTranspos
      · intro x hx_ne_zero
        have h_pos : 0 < (1 - ε) * (star x []v A *v x) + ε * (star x []v B *v x) := by
          exact add_pos_of_nonneg_of_pos (mul_nonneg (sub_nonneg.2 <| mod_cast hε.
            convert h_pos using 1 ; simp [Matrix.add_mulVec] ; ring_nf!
          simp [Matrix.mulVec, dotProduct, Finset.mul_sum ___, mul_assoc, mul_left
            convert h_pos_def _ _ _ _ _ ⟨_, _⟩ <=> norm_num [ * ] ;
            congr! 1
          exact psd ρ
          · exact uniform_posDef;
          · exact one_div_pos.mpr ( by linarith );
          · exact div_lt_one ( by positivity ) |>.2 ( by linarith )

```

```

· -- Show that the sequence  $p_n$  converges to  $p$ .
  have h_conv : Filter.Tendsto (fun n =>  $\varepsilon n \bullet$  (MState.uniform : MState
    exact le_trans ( Filter.Tendsto.add ( Filter.Tendsto.smul ( tendsto_c
  rw [ tendsto_iff_dist_tendsto_zero ] at *;
  convert h_conv using 1;
  ext n; simp [pn, Mixable.mix];
  congr! 1

@[fun_prop]
private lemma MState.traceLeft_continuous :
  Continuous (MState.traceLeft : MState ( $d_1 \times d_2$ )  $\rightarrow$  MState  $d_2$ ) := by
  -- Since the matrix traceLeft is continuous, the function that maps a sta
  have h_traceLeft_cont : Continuous (fun  $p$  : HermitianMat ( $d_1 \times d_2$ )  $\mathbb{C} \Rightarrow p$ 
  have h_cont : Continuous (fun  $p$  : Matrix ( $d_1 \times d_2$ ) ( $d_1 \times d_2$ )  $\mathbb{C} \Rightarrow p$ .tra
    exact continuous_pi fun _ => continuous_pi fun _ => continuous_finset
  convert h_cont.comp ( show Continuous fun  $p$  : HermitianMat (  $d_1 \times d_2$  )
    · constructor <=> intro h <=> rw [ continuous_induced_rng ] at * <=> ae
    · fun_prop;
  exact continuous_induced_rng.mpr ( by continuity )

@[fun_prop]
private lemma MState.traceRight_continuous :
  Continuous (MState.traceRight : MState ( $d_1 \times d_2$ )  $\rightarrow$  MState  $d_1$ ) := by
  rw [ continuous_iff_continuousAt ];
  intro  $p$ 
  simp [ContinuousAt] at *;
  rw [ tendsto_nhds ] at *;
  intro  $s$   $h_s$   $h_{p_s}$ ;
  rcases  $h_s$  with (  $t$ ,  $ht$ ,  $rfl$  );
  -- The traceRight map is continuous, so the preimage of an open set under
  have h_traceRight_cont : Continuous (HermitianMat.traceRight : HermitianM
  -- The traceRight map is a linear map, hence continuous.
  have h_traceRight_linear :  $\exists$  (  $f$  : HermitianMat ( $d_1 \times d_2$ )  $\mathbb{C} \rightarrow_1 [\mathbb{R}]$  Hermit
    refine' { _, _ };
    refine' { .. };
    exact fun  $A \Rightarrow A$ .traceRight;
    all_goals simp [ HermitianMat.traceRight ];
    · exact fun  $x$   $y \Rightarrow rfl$ ;
    · aesop
  obtain (  $f$ ,  $hf$  ) := h_traceRight_linear;
  convert f.continuous_of_finiteDimensional;
  exact funext fun  $A \Rightarrow hf$   $A \triangleright rfl$ ;
  have := h_traceRight_cont.isOpen_preimage  $t$   $ht$ ;
  exact Filter.mem_of_superset ( this.preimage ( continuous_induced_dom ) |

@[fun_prop]

```

```

private lemma MState.assoc'_continuous :
  Continuous (MState.assoc' : MState (d1 × d2 × d3) → MState ((d1 × d2) × d3)) :=
  apply continuous_induced_rng.mpr;
  -- The reindex function is continuous because it is a composition of continuous fu
  have h_reindex_cont : Continuous (fun ρ : HermitianMat (d1 × d2 × d3) ℂ => ρ.reind
    apply continuous_induced_rng.mpr;
    fun_prop (disch := norm_num);
  convert h_reindex_cont.comp _ using 2;
  exact Continuous_HermitianMat

/-- Weak monotonicity, version with partial traces. -/
lemma Svn_wm (ρ : MState (d1 × d2 × d3)) :
  Svn ρ.traceRight + Svn ρ.traceLeft.traceLeft ≤
  Svn ρ.assoc'.traceRight + Svn ρ.traceLeft := by
  have h_ne123 := ρ.nonempty
  obtain { pn, hpn_pos, hpn } := MState.approx_by_pd ρ;
  -- Apply the inequality for each pn and then take the limit.
  have h_lim : Filter.Tendsto (fun n => Svn (pn n).traceRight + Svn (pn n).traceLeft
    constructor <=> refine' Filter.Tendsto.add _ _;
    · exact Svn_continuous.continuousAt.tendsto.comp (MState.traceRight_continuous.
    · exact Svn_continuous.comp (MState.traceLeft_continuous.comp (MState.traceLef
    · convert Svn_continuous.continuousAt.tendsto.comp (MState.traceRight_continuou
    · exact Svn_continuous.continuousAt.tendsto.comp (MState.traceLeft_continuous.c
  have {_, hn23} := nonempty_prod.mp h_ne123
  have {_, _} := nonempty_prod.mp hn23
  exact le_of_tendsto_of_tendsto' h_lim.1 h_lim.2 fun n => Svn_wm_pd _ (hpn_pos n)

/-- Permutation to relabel (A×B×C)×R as A×(B×C×R). -/
private def perm_A_BCR' (dA dB dC : Type*) :
  (dA × dB × dC) × (dA × dB × dC) ≈ dA × (dB × dC × (dA × dB × dC)) where
  toFun x := let ((a,b,c), r) := x; (a, (b,c,r))
  invFun x := let (a, (b,c,r)) := x; ((a,b,c), r)
  left_inv := by intro x; simp
  right_inv := by intro x; simp

/-- The BCR state: trace out A from the purification of ρABC. -
/
private def pBCR (ρ : MState (dA × dB × dC)) : MState (dB × dC × (dA × dB × dC)) :=
  ((MState.pure ρ.purify).relabel (perm_A_BCR' dA dB dC).symm).traceLeft

private lemma S_BC_of_BCR_eq (ρ : MState (dA × dB × dC)) :
  Svn (pBCR ρ).assoc'.traceRight = Svn ρ.traceLeft := by
  -- By definition of pBCR, we know that its BC-marginal is equal to the BC-
  marginal of ρ.
  have h_marginal : (pBCR ρ).assoc'.traceRight = ρ.traceLeft := by
    unfold pBCR MState.traceLeft MState.traceRight MState.assoc';

```

```

simp [ MState.assoc, MState.SWAP, MState.relabel, MState.pure, perm_AB_B
unfold reindex HermitianMat.traceLeft HermitianMat.traceRight; ext
simp
simp [ Matrix.traceLeft, Matrix.traceRight, Matrix.submatrix ];
have :=  $\rho$ .purify_spec;
replace this := congr_arg ( fun x => x.M.val ) this ; simp_all [ MState
simp [  $\leftarrow$  this, Matrix.traceRight, Matrix.vecMulVec ];
exact Finset.sum_comm.trans ( Finset.sum_congr rfl fun _ => Finset.su
rw [h_marginal]

/-- Equivalence to relabel the purification as  $(dA \times dB) \times (dC \times R)$ . -
/
private def perm_AB_CR' (dA dB dC : Type*) :
   $(dA \times dB \times dC) \times (dA \times dB \times dC) \approx (dA \times dB) \times (dC \times (dA \times dB \times dC))$  whe
toFun x := let ((a,b,c), r) := x; ((a,b), (c,r))
invFun x := let ((a,b), (c,r)) := x; ((a,b,c), r)
left_inv := by intro x; simp
right_inv := by intro x; simp

/- The CR-marginal of  $\rho_{BCR}$  equals the traceLeft of the AB|CR-
relabelled purification. -/
private lemma BCR_traceLeft_eq_purify_traceLeft ( $\rho$  : MState  $(dA \times dB \times dC)$ )
  ( $\rho_{BCR} \rho$ ).traceLeft =
  ((MState.pure  $\rho$ .purify).relabel (perm_AB_CR' dA dB dC).symm).traceLeft
convert MState.ext ?_
convert MState.ext ?_
any_goals exact  $\rho$ .traceLeft.traceLeft;
· simp [ MState.traceLeft, MState.relabel, perm_AB_CR' ];
simp [ MState.traceLeft, MState.relabel,  $\rho_{BCR}$  ];
unfold HermitianMat.traceLeft
simp only [mat_reindex, MState.mat_M, Matrix.reindex_apply, mat_mk, Equ
unfold Matrix.traceLeft
simp
congr! 2
ext i1 j1
rw [  $\leftarrow$  Finset.sum_product' ]
simp [ perm_AB_CR' ]
exact Finset.sum_bij ( fun x _ => ( x.2, x.1 ) ) ( by simp ) ( by simp
· rfl

/- The traceRight of the AB|CR-relabelled purification has same entropy as  $\rho$ 
/
private lemma purify_AB_traceRight_eq ( $\rho$  : MState  $(dA \times dB \times dC)$ ) :
   $S_{v_n} ((MState.pure \rho.purify).relabel (perm_AB_CR' dA dB dC).symm).traceR$ 
 $S_{v_n} \rho.assoc'.traceRight :=$  by
have h_traceRight : ((MState.pure  $\rho$ .purify).relabel (perm_AB_CR' dA dB dC

```

```

    have h_traceRight : (MState.pure ρ.purify).traceRight = ρ := by
      exact MState.purify_spec ρ;
    convert congr_arg ( fun m => m.assoc'.traceRight ) h_traceRight using 1;
    ext i j; simp [ MState.traceRight, MState.assoc' ] ;
    simp [ HermitianMat.traceRight, MState.SWAP, MState.assoc ];
    simp [ Matrix.traceRight, Matrix.submatrix ];
    congr! 2;
    ext i j; simp [ perm_AB_CR' ] ;
    exact Fintype.sum_prod_type _
  rw [h_traceRight]

/-- The CR-marginal of pBCR has the same entropy as the AB-
marginal of ρ. -/
private lemma S_CR_of_BCR_eq (ρ : MState (dA × dB × dC)) :
  Svn (pBCR ρ).traceLeft = Svn ρ.assoc'.traceRight := by
  rw [BCR_traceLeft_eq_purify_traceLeft]
  rw [Svn_pure_complement ρ.purify (perm_AB_CR' dA dB dC).symm]
  exact purify_AB_traceRight_eq ρ

private lemma S_B_of_BCR_eq (ρ : MState (dA × dB × dC)) :
  Svn (pBCR ρ).traceRight = Svn ρ.traceLeft.traceRight := by
  unfold pBCR;
  unfold MState.traceLeft MState.traceRight MState.relabel MState.pure;
  simp [ HermitianMat.traceLeft, HermitianMat.traceRight, perm_A_BCR' ];
  unfold Matrix.traceLeft Matrix.traceRight; simp [Matrix.vecMulVec ];
  -- By definition of purification, we know that ρ.purify is a purification of ρ.m.
  have h_purify : ∀ (i j : dA × dB × dC), ρ.m i j = ∑ r : dA × dB × dC, ρ.purify (i,
    intro i j
    have := ρ.purify_spec;
    replace this := congr_arg ( fun m => m.M i j ) this
    simp_all [MState.traceRight]
    exact this.symm
  simp only [Finset.sum_sigma', h_purify];
  congr! 3;
  ext i2 j2
  simp
  ring_nf
  refine' Finset.sum_bij ( fun x _ => ( x.fst.1, x.snd, x.fst.2 ) ) _ _ _ _ <;> simp
  · grind
  · grind

private lemma S_R_of_BCR_eq (ρ : MState (dA × dB × dC)) :
  Svn (pBCR ρ).traceLeft.traceLeft = Svn ρ := by
  have h_trace : (pBCR ρ).traceLeft.traceLeft = (MState.pure ρ.purify).traceLeft :=
    unfold pBCR MState.traceLeft;
  ext i j;

```

```

simp [ HermitianMat.traceLeft];
simp [ perm_A_BCR', Matrix.traceLeft ];
simp [ Finset.sum_sigma' ];
refine' Finset.sum_bij ( fun x _ => ( x.snd.snd, x.snd.fst, x.fst ) ) _
grind;
convert Svn_of_partial_eq ρ.purify using 1;
· rw [h_trace];
· rw [ ρ.purify_spec ]

/-- Strong subadditivity on a tripartite system -/
theorem Svn_strong_subadditivity (ρ123 : MState (d1 × d2 × d3)) :
  let ρ12 := ρ123.assoc'.traceRight;
  let ρ23 := ρ123.traceLeft;
  let ρ2 := ρ123.traceLeft.traceRight;
  Svn ρ123 + Svn ρ2 ≤ Svn ρ12 + Svn ρ23 := by
  have _ := ρ123.nonempty |> nonempty_prod.mp |>.right |> nonempty_prod.mp
  -- Apply weak monotonicity to ρBCR, then substitute purification identities
  have hwm := Svn_wm (ρBCR ρ123)
  rw [S_BC_of_BCR_eq, S_CR_of_BCR_eq, S_B_of_BCR_eq, S_R_of_BCR_eq] at hwm
  linarith

/-- "Ordinary" subadditivity of von Neumann entropy -/
theorem Svn_subadditivity (ρ : MState (d1 × d2)) :
  Svn ρ ≤ Svn ρ.traceRight + Svn ρ.traceLeft := by
  have := Svn_strong_subadditivity (ρ.relabel (d2 := d1 × Unit × d2))
  {fun x ↦ (x.1, x.2.2), fun x ↦ (x.1, ((), x.2)), fun x ↦ by simp, fun x ↦ by simp}
  simp [Svn_relabel] at this
  convert this using 1
  congr 1
  · convert Svn_relabel _ (Equiv.prodPUnit _).symm
    exact rfl
  · convert Svn_relabel _ (Equiv.punitProd _).symm
    exact rfl

/-- Triangle inequality for pure tripartite states: S(A) ≤ S(B) + S(C). -
/
theorem Svn_pure_tripartite_triangle (ψ : Ket ((d1 × d2) × d3)) :
  Svn (MState.pure ψ).traceRight.traceRight ≤
  Svn (MState.pure ψ).traceRight.traceLeft + Svn (MState.pure ψ).traceLeft
  have hsubadd := Svn_subadditivity (MState.pure ψ).assoc.traceLeft
  obtain ⟨ψ', hψ'⟩ : ∃ ψ', (MState.pure ψ).assoc = _ :=
  MState.relabel_pure_exists ψ _
  grind [Svn_of_partial_eq, MState.traceLeft_left_assoc,
  MState.traceLeft_right_assoc, MState.traceRight_assoc]

/-- One direction of the Araki-Lieb triangle inequality: S(A) ≤ S(B) + S(AB)

```



```

/
theorem Svn_triangle_ineq_one_way (ρ : MState (d1 × d2)) : Svn ρ.traceRight ≤ Svn ρ.
  have := Svn_pure_tripartite_triangle ρ.purify
  have := Svn_of_partial_eq ρ.purify
  aesop

/-- Araki-Lieb triangle inequality on von Neumann entropy -
/
theorem Svn_triangle_subadditivity (ρ : MState (d1 × d2)) :
  abs (Svn ρ.traceRight - Svn ρ.traceLeft) ≤ Svn ρ := by
  rw [abs_sub_le_iff]
  constructor
  · have := Svn_triangle_ineq_one_way ρ
    grind only
  · have := Svn_triangle_ineq_one_way ρ.SWAP
    grind only [Svn_triangle_ineq_one_way, Svn_of_SWAP_eq, MState.traceRight_SWAP,
      MState.traceLeft_SWAP]

/-- Weak monotonicity of quantum conditional entropy: S(A|B) + S(A|C) ≥ 0. -
/
theorem Svn_weak_monotonicity (ρ : MState (dA × dB × dC)) :
  let pAB := ρ.assoc'.traceRight
  let pAC := ρ.SWAP.assoc.traceLeft.SWAP
  0 ≤ qConditionalEnt pAB + qConditionalEnt pAC := by
  simp only [qConditionalEnt, MState.traceRight_left_assoc', Svn_of_SWAP_eq]
  rw [add_sub, sub_add_eq_add_sub, le_sub_iff_add_le, le_sub_iff_add_le, zero_add]
  nth_rw 2 [add_comm]
  simpa using Svn_wm ρ.SWAP.assoc.SWAP.assoc

/-- Strong subadditivity, stated in terms of conditional entropies.
  Also called the data processing inequality. H(A|BC) ≤ H(A|B). -
/
theorem qConditionalEnt_strong_subadditivity (ρ123 : MState (d1 × d2 × d3)) :
  qConditionalEnt ρ123 ≤ qConditionalEnt ρ123.assoc'.traceRight := by
  have := Svn_strong_subadditivity ρ123
  dsimp at this
  simp only [qConditionalEnt, MState.traceRight_left_assoc']
  linarith

/-- Strong subadditivity, stated in terms of quantum mutual information.
  I(A;BC) ≥ I(A;B). -/
theorem qMutualInfo_strong_subadditivity (ρ123 : MState (d1 × d2 × d3)) :
  qMutualInfo ρ123 ≥ qMutualInfo ρ123.assoc'.traceRight := by
  have := Svn_strong_subadditivity ρ123
  dsimp at this
  simp only [qMutualInfo, MState.traceRight_left_assoc', MState.traceRight_right_ass

```

```

linarith

/-- The quantum conditional mutual information `QCMi` is nonnegative. -
/
theorem qcmi_nonneg (ρ : MState (dA × dB × dC)) :
  0 ≤ qcmi ρ := by
  simp [qcmi, qConditionalEnt]
  have := Svn_strong_subadditivity ρ
  linarith

/-- The quantum conditional mutual information `QCMi ρABC` is at most 2 log
/
theorem qcmi_le_2_log_dim (ρ : MState (dA × dB × dC)) :
  qcmi ρ ≤ 2 * Real.log (Fintype.card dA) := by
  have := Svn_subadditivity ρ.assoc'.traceRight
  have := abs_le.mp (Svn_triangle_subadditivity ρ)
  grind [qcmi, qConditionalEnt, Svn_nonneg, Svn_le_log_d]

/-- The quantum conditional mutual information `QCMi ρABC` is at most 2 log
/
theorem qcmi_le_2_log_dim' (ρ : MState (dA × dB × dC)) :
  qcmi ρ ≤ 2 * Real.log (Fintype.card dC) := by
  have h_araki_lieb_assoc' : Svn ρ.assoc'.traceRight - Svn ρ.traceLeft.traceRight
  apply le_of_abs_le
  rw [← ρ.traceLeft_assoc', ← Svn_of_assoc'_eq ρ]
  exact Svn_triangle_subadditivity ρ.assoc'
  have := Svn_subadditivity ρ.traceLeft
  grind [qcmi, qConditionalEnt, Svn_le_log_d, MState.traceRight_left_assoc']

/- The chain rule for quantum conditional mutual information:
`I(A1A2 : C | B) = I(A1:C|B) + I(A2:C|BA1)`.

It should be something like this, but it's hard to track the indices correctly
theorem qcmi_chain_rule (ρ : MState ((dA1 × dA2) × dB × dC)) :
  let ρA1BC := ρ.assoc.SWAP.assoc.traceLeft.SWAP;
  let ρA2BA1C : MState (dA2 × (dA1 × dB) × dC) :=
    ((CPTPMap.id ⊗k CPTPMap.assoc').compose (CPTPMap.assoc.compose (CPTPMap.assoc'
  qcmi ρ = qcmi ρA1BC + qcmi ρA2BA1C
  := by
  admit
-/

```

1.441 QuantumInfo/Finite/Entropy/VonNeumann.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.Braket
import QuantumInfo.Finite.CPTPMap
import QuantumInfo.ClassicalInfo.Entropy

/-!
Quantum notions of information and entropy.

We start with quantities of _entropy_, namely the von Neumann entropy and its derive
* Quantum conditional entropy, `qConditionalEnt`
* Quantum mutual information, `qMutualInfo`
* Coherent information, `coherentInfo`
* Quantum conditional mutual information, `qcmi`.
and then prove facts about them.
-/

/- # TODO / Goals:

--QConditionalEnt chain rule

--Quantum discord

--Entanglement:
-- * Entanglement entropy
-- * Entanglement of formation
-- * Relative entropy of entanglement
-- * Squashed entanglement
-- * Negativity (+ facts here: https://www.quantiki.org/wiki/strong-sub-additivity)
-- * Distillable entanglement (One way, Two way, --> Coherent Information)
-- * Entanglement cost ( $\neq$  EoF, prove; asymptotically  $=$  EoF.)
-- Bound entanglement (Prop)

-- https://arxiv.org/pdf/quant-ph/0406162

--https://en.wikipedia.org/wiki/Von\_Neumann\_entropy#Properties
-- in particular https://www.quantiki.org/wiki/strong-sub-additivity

--https://en.wikipedia.org/wiki/Quantum\_relative\_entropy#Relation\_to\_other\_quantum\_i

```

```

--QMutualInfo is symmetric

--TODO:
-- * Classical conditional entropy is nonnegative
-- * Not true of qConditionalEnt
-- * These measures track their classical values on `MState.ofClassical` st
-/

noncomputable section

variable {d d1 d2 d3 : Type*}
variable [Fintype d] [Fintype d1] [Fintype d2] [Fintype d3]
variable [DecidableEq d] [DecidableEq d1] [DecidableEq d2] [DecidableEq d3]
variable {dA dB dC dA1 dA2 : Type*}
variable [Fintype dA] [Fintype dB] [Fintype dC] [Fintype dA1] [Fintype dA2]
variable [DecidableEq dA] [DecidableEq dB] [DecidableEq dC] [DecidableEq dA1] [DecidableEq dA2]
variable {α : Type*} [RCLike α]
variable {α : ℝ}

open scoped InnerProductSpace RealInnerProductSpace

section entropy

/-- Von Neumann entropy of a mixed state. -/
def Svn (ρ : MState d) : ℝ :=
  Hs ρ.spectrum

/-- The Quantum Conditional Entropy S(ρA|ρB) is given by S(ρAB) - S(ρB). -
/
def qConditionalEnt (ρ : MState (dA × dB)) : ℝ :=
  Svn ρ - Svn ρ.traceLeft

/-- The Quantum Mutual Information I(A:B) is given by S(ρA) + S(ρB) - S(ρAB)
/
def qMutualInfo (ρ : MState (dA × dB)) : ℝ :=
  Svn ρ.traceLeft + Svn ρ.traceRight - Svn ρ

/-- The Coherent Information of a state ρ passing through a channel Λ is the
entropy of the image under Λ of the purification of ρ. -
/
def coherentInfo (ρ : MState d1) (Λ : CPTMap d1 d2) : ℝ :=
  let pPure : MState (d1 × d1) := MState.pure ρ.purify
  let pImg : MState (d2 × d1) := Λ.prod (CPTMap.id (dIn := d1)) pPure
  (- qConditionalEnt pImg)

```

```

/-- The Quantum Conditional Mutual Information,  $I(A;C|B) = S(A|B) - S(A|BC)$ . -
/
def qcmi (p : MState (dA × dB × dC)) : ℝ :=
  qConditionalEnt p.assoc'.traceRight - qConditionalEnt p

/-- von Neuman entropy is nonnegative. -/
theorem S_vn_nonneg (p : MState d) : 0 ≤ S_vn p :=
  Hs_nonneg _

/-- von Neuman entropy is at most log d. -/
theorem S_vn_le_log_d (p : MState d) : S_vn p ≤ Real.log (Finset.card Finset.univ (α :
  Hs_le_log_d _

/-- von Neuman entropy of pure states is zero. -/
@[simp]
theorem S_vn_of_pure_zero (ψ : Ket d) : S_vn (MState.pure ψ) = 0 := by
  obtain ⟨i, hi⟩ := MState.spectrum_pure_eq_constant ψ
  rw [S_vn, hi, Hs_constant_eq_zero]

theorem S_vn_eq_neg_trace_log (p : MState d) : S_vn p = -[p.M.log, p.M] := by
  open HermitianMat in
  rw [log, inner_eq_re_trace]
  nth_rw 2 [← cfc_id p.M]
  rw [← mat_cfc_mul]
  simp only [S_vn, Hs, H1, Real.negMulLog, neg_mul, Finset.sum_neg_distrib, neg_inj]
  rw [← trace_eq_re_trace, ← sum_eigenvalues_eq_trace]
  obtain ⟨e, he⟩ := p.M.cfc_eigenvalues (Real.log * id)
  apply Finset.sum_equiv e.symm (by simp)
  simp [MState.spectrum, ProbDistribution.mk', he, mul_comm]

/-- Von Neumann entropy is the trace of the matrix function  $x \mapsto -x \log x$ . -/
theorem S_vn_eq_trace_cfc_negMulLog (p : MState d) :
  S_vn p = (p.M.cfc Real.negMulLog).trace := by
  open HermitianMat in
  unfold Real.negMulLog
  rw [S_vn_eq_neg_trace_log, trace, log, inner_eq_re_trace, IsMaximalSelfAdjoint.RCLi
  nth_rw 2 [← cfc_id p.M]
  rw [← mat_cfc_mul, RCLike.re_to_complex, ← Complex.neg_re, ← Matrix.trace_neg]
  rw [← mat_neg, ← p.M.cfc_neg]
  congr! 5
  simp [mul_comm]

@[simp]
theorem S_vn_unit_zero [Unique d] (p : MState d) : S_vn p = 0 := by
  refine le_antisymm ?_ (S_vn_nonneg p)

```

```

    simp using SVN_le_log_d ρ

/-- Von Neumann entropy is invariant under relabeling of the basis. -
/
@[simp]
theorem SVN_relabel (ρ : MState d1) (e : d2 ≈ d1) :
  SVN (ρ.relabel e) = SVN ρ := by
  simp [SVN_eq_neg_trace_log]

/-- Von Neumann entropy is unchanged under SWAP. TODO: All unitaries-
/
@[simp]
theorem SVN_of_SWAP_eq (ρ : MState (d1 × d2)) : SVN ρ.SWAP = SVN ρ := by
  apply HS_eq_of_multiset_map_eq
  exact ρ.multiset_spectrum_relabel_eq (Equiv.prodComm d1 d2).symm

/-- Von Neumann entropy is unchanged under assoc. -/
@[simp]
theorem SVN_of_assoc_eq (ρ : MState ((d1 × d2) × d3)) : SVN ρ.assoc = SVN ρ
  apply HS_eq_of_multiset_map_eq
  apply ρ.multiset_spectrum_relabel_eq

/-- Von Neumann entropy is unchanged under assoc'. -/
@[simp]
theorem SVN_of_assoc'_eq (ρ : MState (d1 × (d2 × d3))) : SVN ρ.assoc' = SVN ρ
  rw [← SVN_of_assoc_eq, ρ.assoc_assoc']

@[fun_prop]
theorem selfAdjointMap_Continuous {α : Type*} [RCLike α] :
  Continuous (IsMaximalSelfAdjoint.selfadjMap : α →+ ℝ) := by
  rw [IsMaximalSelfAdjoint.RCLike_selfadjMap]
  fun_prop

@[fun_prop]
theorem HermitianMat.trace_Continuous {d α : Type*} [Fintype d] [RCLike α] :
  Continuous (HermitianMat.trace : HermitianMat d α → ℝ) := by
  rw [funext HermitianMat.trace_eq_re_trace]
  fun_prop

@[fun_prop]
theorem SVN_continuous : Continuous (SVN (d := d)) := by
  rw [funext SVN_eq_trace_cfc_negMulLog]
  fun_prop

section partial_trace_pure
--TODO Cleanup

```

```

/--
Convert a vector on a product space to a matrix.
-/
private def vecToMat {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂] (v : d₁ × d₂ → ℂ) : Matrix
  Matrix.of (fun i j => v (i, j))

/--
The right partial trace of a pure state is the product of the matrix representation
and its conjugate transpose.
-/
private lemma traceRight_eq_mul_conjTranspose (ψ : Ket (d₁ × d₂)) :
  (MState.pure ψ).traceRight.M.val = (vecToMat ψ.vec) * (vecToMat ψ.vec).conjTranspose
  unfold vecToMat; aesop;

/-
The left partial trace of a pure state is the product of the transpose of the
matrix representation and its conjugate.
-/
lemma traceLeft_eq_transpose_mul_conj (ψ : Ket (d₁ × d₂)) :
  (MState.pure ψ).traceLeft.M.val = (vecToMat ψ.vec).transpose * (vecToMat ψ.vec).conjTranspose
  unfold vecToMat; aesop;

/--
The left partial trace of `vecToMat` is the transpose of  $M^H * M$ .
-/
private lemma traceLeft_eq_transpose_conjTranspose_mul (ψ : Ket (d₁ × d₂)) :
  (MState.pure ψ).traceLeft.M.val =
    ((vecToMat ψ.vec).conjTranspose * (vecToMat ψ.vec)).transpose := by
    norm_num +zetaDelta at *;
    convert traceLeft_eq_transpose_mul_conj ψ using 1

/--
Shannon entropy is determined by the multiset of non-zero probabilities.
-/
private lemma Hs_eq_of_nonzero_multiset_eq {α β : Type*} [Fintype α] [Fintype β]
  (d₁ : ProbDistribution α) (d₂ : ProbDistribution β)
  (h : (Finset.univ.val.map d₁.prob).filter (· ≠ 0) = (Finset.univ.val.map d₂.prob).filter (· ≠ 0)) :
  Hs d₁ = Hs d₂ := by
  -- By reindexing the sums, we can show that the Shannon entropies are equal.
  simp [Hs]
  -- Since the multisets of non-zero probabilities are equal, their sums are equal.
  have h_sum_eq : ∑ x ∈ Finset.univ.filter (fun x => d₁.prob x ≠ 0), H₁ (d₁.prob x) =
    ∑ x ∈ Finset.univ.filter (fun x => d₂.prob x ≠ 0), H₁ (d₂.prob x) := by
    convert congr_arg ( fun m => m.map ( fun x => H₁ x ) ) h using 1
    · simp [ Finset.sum, Multiset.filter_map ];
    · simp [ Finset.sum, Multiset.filter_map ];

```

```

    rwa [Finset.sum_filter_of_ne, Finset.sum_filter_of_ne] at h_sum_eq
    · aesop
    · aesop

/--
If two polynomials differ by a factor of  $X^k$ , their non-zero roots are the
-/
private lemma nonzero_roots_eq_of_charpoly_eq_X_pow {R : Type*} [CommRing R]
  [DecidableEq R] (p q : Polynomial R) (k : ℕ) (h : p = Polynomial.X ^ k *
    p.roots.filter (· ≠ 0) = q.roots.filter (· ≠ 0) := by
  by_cases hq : q = 0 <|> simp_all [Polynomial.roots_mul, Polynomial.X_ne_0]

/--
The non-zero eigenvalues of AB and BA are the same.
-/
private lemma nonzero_eigenvalues_eq_of_mul_comm {n m : Type*} [Fintype n]
  [DecidableEq n] [DecidableEq m] (A : Matrix n m ℂ) (B : Matrix m n ℂ) :
  (A * B).charpoly.roots.filter (· ≠ 0) = (B * A).charpoly.roots.filter (· ≠ 0) :=
  have h_roots : (Polynomial.X ^ Fintype.card m * (Matrix.charpoly (A * B)))
    rw [A.charpoly_mul_comm' B]
    convert h_roots using 1
    · simp [Polynomial.roots_mul, Matrix.charpoly_monoid, Polynomial.Monic.ne_0]
    · simp [Polynomial.roots_mul, Matrix.charpoly_monoid, Polynomial.Monic.ne_0]

/--
If two states have the same non-zero eigenvalues (spectrum probabilities),
the same von Neumann entropy.
-/
private lemma S_vn_eq_of_nonzero_eigenvalues_eq (ρ₁ : MState d₁) (ρ₂ : MState d₂)
  (h : (Finset.univ.val.map ρ₁.spectrum.prob).filter (· ≠ 0) =
    (Finset.univ.val.map ρ₂.spectrum.prob).filter (· ≠ 0)) :
  S_vn ρ₁ = S_vn ρ₂ := by
  exact Hs_eq_of_nonzero_multiset_eq _ _ h

/--
Filtering non-zero elements commutes with mapping `RCLike.ofReal` for multisets.
-/
private lemma multiset_filter_map_ofReal_eq {R : Type*} [RCLike R] (M : Multiset R)
  (M.map (RCLike.ofReal : ℝ → R)).filter (· ≠ 0) = (M.filter (· ≠ 0)).map (RCLike.ofReal)
  simp [Multiset.filter_map]

/-
The non-zero roots of the characteristic polynomial are the non-zero
eigenvalues mapped
to complex numbers.
-/

```



```

private lemma charpoly_roots_filter_ne_zero_eq_eigenvalues_filter_ne_zero {d : Type*}
  A.charpoly.roots.filter (· ≠ 0) = ((Finset.univ.val.map hA.eigenvalues).filter (
    convert congr_arg _ ?_);
  rw [ multiset_filter_map_ofReal_eq ];
  -- Since A is Hermitian, its characteristic polynomial is equal to the product of
  have h_charpoly : A.charpoly = Multiset.prod (Multiset.map (fun i => Polynomial.X
    convert Matrix.IsHermitian.charpoly_eq hA using 1;
  rw [ h_charpoly, Polynomial.roots_multiset_prod ];
  · simp [ Multiset.bind_map ];
  · simp [ Polynomial.X_sub_C_ne_zero ]

/--
Mapping `RCLike.ofReal` over a multiset is injective.
-/
private lemma multiset_map_ofReal_injective {R : Type*} [RCLike R] {M N : Multiset R}
  M.map (RCLike.ofReal : R → R) = N.map RCLike.ofReal ↔ M = N := by
  exact (fun h => by simp using congr(($h).map RCLike.re), fun h => by rw [h])

/--
If the non-zero roots of the characteristic polynomials of two states are equal,
their von Neumann entropies are equal.
-/
private lemma S_vn_eq_of_charpoly_roots_eq (p₁ : MState d₁) (p₂ : MState d₂)
  (h : (p₁.M.1.charpoly.roots.filter (· ≠ 0)) = (p₂.M.1.charpoly.roots.filter (· ≠ 0))) :
  S_vn p₁ = S_vn p₂ := by
  convert S_vn_eq_of_nonzero_eigenvalues_eq p₁ p₂ _;
  have h_spectrum_eq_aux : (Matrix.charpoly (p₁.M.1)).roots.filter (· ≠ 0) = (Multiset
    constructor;
  · convert charpoly_roots_filter_ne_zero_eq_eigenvalues_filter_ne_zero _ _;
    any_goals exact p₁.M.isSelfAdjoint;
    simp [ ProbDistribution.prob, MState.spectrum ];
    simp [ Multiset.filter_map, ProbDistribution.mk' ];
    simp [ Subtype.ext_iff ];
  · convert charpoly_roots_filter_ne_zero_eq_eigenvalues_filter_ne_zero p₂.M.1 (He
    simp [ ProbDistribution.prob, MState.spectrum ];
    simp [ Multiset.filter_map, ProbDistribution.mk' ];
    simp [ Subtype.ext_iff ];
  norm_num +zetaDelta at *;
  rw [ h_spectrum_eq_aux.1, h_spectrum_eq_aux.2 ] at h;
  exact Multiset.map_injective ( by intros a b; aesop ) h

end partial_trace_pure

/-- von Neuman entropies of the left- and right- partial trace of pure states are equal
-/
theorem S_vn_of_partial_eq (ψ : Ket (d₁ × d₂)) :

```

```

      Svn (MState.pure  $\psi$ ).traceLeft = Svn (MState.pure  $\psi$ ).traceRight := by
    apply Svn_eq_of_charpoly_roots_eq
    rw [traceLeft_eq_transpose_conjTranspose_mul, traceRight_eq_mul_conjTrans
      ← Matrix.charpoly_transpose]
    exact nonzero_eigenvalues_eq_of_mul_comm _ _

/-- Quantum conditional entropy is symmetric for pure states. -
/
@[simp]
theorem qConditionalEnt_of_pure_symm ( $\psi$  : Ket ( $d_1 \times d_2$ )) :
  qConditionalEnt (MState.pure  $\psi$ ).SWAP = qConditionalEnt (MState.pure  $\psi$ )
  simp [qConditionalEnt, Svn_of_partial_eq]

/-- Quantum mutual information is symmetric. -/
@[simp]
theorem qMutualInfo_symm ( $\rho$  : MState ( $d_1 \times d_2$ )) :
  qMutualInfo  $\rho$ .SWAP = qMutualInfo  $\rho$  := by
  simp [qMutualInfo, add_comm]

/-- For a pure state, the entropy of one subsystem equals the entropy of it
even after relabeling. -/
@[simp]
theorem Svn_pure_complement ( $\psi$  : Ket  $d_1$ ) ( $e$  :  $d_2 \times d_3 \approx d_1$ ) :
  Svn ((MState.pure  $\psi$ ).relabel  $e$ ).traceLeft = Svn ((MState.pure  $\psi$ ).relabel
  obtain ( $\psi'$ ,  $h\psi'$ ) := MState.relabel_pure_exists  $\psi$   $e$ 
  simp only [ $h\psi'$ , Svn_of_partial_eq]

end entropy

```

1.442 QuantumInfo/Finite/MState.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg, Leonardo A. Lessa
-/
import QuantumInfo.ForMathlib
import QuantumInfo.ClassicalInfo.Distribution
import QuantumInfo.Finite.Braket

import Mathlib.Logic.Equiv.Basic

/-!
Finite dimensional quantum mixed states,  $\rho$ .

```

The same comments apply as in `Braket`:

These could be done with a Hilbert space of Fintype, which would look like

```
```lean4
```

```
(H : Type*) [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H] [Finite
```

```
```
```

or by choosing a particular `Basis` and asserting it is `Fintype`. But frankly it seems mostly focus on the basis-dependent notion of `Matrix`, which has the added benefit of a "classical" interpretation (as the basis elements, or diagonal elements of a mixed state). In this sense, this quantum theory comes with the a particular classical theory always preferred.

Important definitions:

```
* `instMixable`: the `Mixable` instance allowing convex combinations of `MState`s
* `ofClassical`: Mixed states representing classical distributions
* `purity`: The purity  $\text{Tr}[\rho^2]$  of a state
* `spectrum`: The spectrum of the matrix
* `uniform`: The maximally mixed state
* `mix`: The total state corresponding to an ensemble
* `average`: Averages a function over an ensemble, with appropriate weights
-/
```

noncomputable section

```
open BigOperators
open ComplexConjugate
open HermitianMat
open scoped Matrix ComplexOrder
```

```
/-- A mixed quantum state is a PSD matrix with trace 1.
```

```
We don't `extend (M : HermitianMat d ℂ)` because that gives an annoying thing where
`M` is actually a `Subtype`, which means  $\rho.M.foo$  notation doesn't work. -
/
```

```
@[ext]
```

```
structure MState (d : Type*) [Fintype d] [DecidableEq d] where
  M : HermitianMat d ℂ
  nonneg : 0 ≤ M
  tr : M.trace = 1
```

```
variable {d d₁ d₂ d₃ : Type*}
```

```
variable [Fintype d] [Fintype d₁] [Fintype d₂] [Fintype d₃]
```

```
variable [DecidableEq d] [DecidableEq d₁] [DecidableEq d₂] [DecidableEq d₃]
```

```
variable (ψ φ : Ket d)
```

```
variable (ρ σ : MState d)
```

```

namespace MState

attribute [coe] MState.M
instance instCoe : Coe (MState d) (HermitianMat d  $\mathbb{C}$ ) := ⟨MState.M⟩

attribute [simp] MState.tr

/-- The underlying `Matrix` in an MState. Prefer `MState.M` for the `Hermitian`
/
def m (p : MState d) : Matrix d d  $\mathbb{C}$  := p.M.mat

@[simp]
theorem mat_M : p.M.mat = p.m := by
  rfl

theorem pos (p : MState d) :  $0 < p.M$  := by
  apply p.nonneg.lt_of_ne'
  intro h
  have := p.tr
  simp [h] at this

open Lean Meta Mathlib.Meta.Positivity in
/-- Positivity extension for `MState.M`: it is always positive ( $0 < p.M$ ).
Note: we must not call `whnfR` on `e` because `MState.M` is a structure
projection (reducible), so `whnfR` would reduce it and destroy the pattern.
/
@[positivity MState.M _]
def evalMStateM : PositivityExt where eval {u _ $\alpha$ } _z $\alpha$  _p $\alpha$  e := do
  let p := e.appArg!
  pure (.positive (← mkAppM `MState.pos #[p]))

--TODO: There should be a bunch of places where we can use `positivity` to
-- that are currently proved manually.
example (p : MState d) :  $0 < p.M$  := by positivity

--XXX These are methods that directly reference the matrix, "m" or ".val".
-- We'd like to remove these (where possible) so that mostly go through Her
-- where possible.
theorem psd : p.m.PosSemidef :=
  HermitianMat.zero_le_iff.mp p.nonneg

/-- Every mixed state is Hermitian. -/
theorem Hermitian : p.m.IsHermitian :=
  p.M.H

```

```

@[simp]
theorem tr' :  $\rho.m.trace = 1$  := by
  rw [MState.m.eq_def, ← HermitianMat.trace_eq_trace_rc,  $\rho.tr$ ]
  simp

theorem ext_m { $\rho_1 \rho_2$  : MState d} (h :  $\rho_1.m = \rho_2.m$ ) :  $\rho_1 = \rho_2$  := by
  rw [MState.mk.injEq]
  ext1
  exact h

/-- The map from mixed states to their matrices is injective -
/
theorem m_inj : (MState.m (d := d)).Injective :=
  fun _ _ h => by ext1; ext1; exact h

theorem M_Injective : Function.Injective (MState.M (d := d)) := by
  intro _ _
  exact MState.ext

variable (d) in
/-- The matrices corresponding to MStates are `Convex  $\mathbb{R}$ ` -
/
theorem convex : Convex  $\mathbb{R}$  (Set.range (MState.M (d := d))) := by
  simp only [Convex, Set.mem_range, StarConvex,
    forall_exists_index, forall_apply_eq_imp_iff]
  intro x y a b ha hb hab
  replace hab :  $a + b = (1 : \mathbb{C})$  := by norm_cast
  have := HermitianMat.convex_cone x.nonneg y.nonneg ha hb
  exact ⟨_, this, by simp using mod_cast hab, rfl⟩

instance instMixable : Mixable (HermitianMat d  $\mathbb{C}$ ) (MState d) where
  to_U := MState.M
  to_U_inj := MState.ext
  mkT {u} := fun h =>
    ⟨(u, h.casesOn fun t ht => ht ▸ t.nonneg,
      h.casesOn fun t ht => ht ▸ t.tr), rfl⟩
  convex := convex d

--An MState is a witness that d is nonempty.
instance nonempty : Nonempty d := by
  by_contra h
  simp [HermitianMat.trace_eq_re_trace, not_nonempty_iff.mp h] using  $\rho.tr$ 

-- Could have used properties of  $\rho.spectrum$ 
theorem eigenvalue_nonneg :  $\forall i, 0 \leq \rho.Hermitian.eigenvalues i$  := by

```

```

rw [← Matrix.PosSemidef.nonneg_iff_eigenvalue_nonneg p.Hermitian]
exact p.nonneg

-- Could have used properties of p.spectrum
theorem eigenvalue_le_one : ∀ i, p.Hermitian.eigenvalues i ≤ 1 := by
  intro i
  convert Finset.single_le_sum (fun y _ => p.psd.eigenvalues_nonneg y) (Fins
  rw [p.M.sum_eigenvalues_eq_trace, p.tr]

theorem le_one : p.M ≤ 1 := by
  open MatrixOrder in
  suffices h : p.m ≤ (1 : ℝ) • 1 by
    rw [one_smul] at h
    exact h
  rw [← Matrix.PosSemidef.le_smul_one_of_eigenvalues_iff p.Hermitian]
  exact eigenvalue_le_one p

open scoped RealInnerProductSpace InnerProductSpace

/-- The inner product of two MState's, as a real number between 0 and 1. -
/
scoped instance : Inner Prob (MState d) where
  inner := fun p σ => ⟨p.M, σ.M⟩,
  inner_ge_zero p.nonneg σ.nonneg,
  (inner_le_mul_trace p.nonneg σ.nonneg).trans (by simp))

theorem inner_def : ⟨p, σ⟩_Prob = ⟨p.M, σ.M⟩,
  inner_ge_zero p.nonneg σ.nonneg,
  (inner_le_mul_trace p.nonneg σ.nonneg).trans (by simp)) := by
  rfl

theorem val_inner : (⟨p, σ⟩_Prob : ℝ) = ⟨p.M, σ.M⟩ := by
  rfl

section exp_val

def exp_val_ℂ (T : Matrix d d ℂ) : ℂ :=
  (T * p.m).trace

--TODO: Bundle as a ContinuousLinearMap.
/-- The expectation value of an operator on a quantum state. -
/
def exp_val (T : HermitianMat d ℂ) : ℝ :=
  ⟨p.M, T⟩

theorem exp_val_nonneg {T : HermitianMat d ℂ} (h : 0 ≤ T) : 0 ≤ p.exp_val T

```

```

    inner_ge_zero ρ.nonneg h

--TODO: Positivity extension for `MState.exp_val`. (Use the `inner` extension that w
-- to write first.)

@[simp]
theorem exp_val_zero : ρ.exp_val 0 = 0 := by
  simp [MState.exp_val]

@[simp]
theorem exp_val_one : ρ.exp_val 1 = 1 := by
  simp [MState.exp_val]

theorem exp_val_le_one {T : HermitianMat d ℂ} (h : T ≤ 1) : ρ.exp_val T ≤ 1 := by
  have hmono := inner_mono ρ.nonneg h
  rwa [inner_one ρ.M, ρ.tr] at hmono

theorem exp_val_prob {T : HermitianMat d ℂ} (h : 0 ≤ T ∧ T ≤ 1) :
  0 ≤ ρ.exp_val T ∧ ρ.exp_val T ≤ 1 :=
  ⟨ρ.exp_val_nonneg h.1, ρ.exp_val_le_one h.2⟩

theorem exp_val_sub (A B : HermitianMat d ℂ) :
  ρ.exp_val (A - B) = ρ.exp_val A - ρ.exp_val B := by
  simp [exp_val, inner_sub_right]

/-- If a PSD observable `A` has expectation value of 0 on a state `ρ`, it must entire
support of `ρ` in its kernel. -/
theorem exp_val_eq_zero_iff {A : HermitianMat d ℂ} (hA₁ : 0 ≤ A) :
  ρ.exp_val A = 0 ↔ ρ.M.support ≤ A.ker := by
  exact inner_zero_iff ρ.nonneg hA₁

/-- If an observable `A` has expectation value of 1 on a state `ρ`, it must entirely
support of `ρ` in its 1-eigenspace. -/
theorem exp_val_eq_one_iff {A : HermitianMat d ℂ} (hA₂ : A ≤ 1) :
  ρ.exp_val A = 1 ↔ ρ.M.support ≤ (1 - A).ker := by
  rw [← exp_val_eq_zero_iff ρ (A := 1 - A) (HermitianMat.zero_le_iff.mpr hA₂)]
  rw [exp_val_sub, exp_val_one]
  rw [sub_eq_zero, eq_comm]

theorem exp_val_add (A B : HermitianMat d ℂ) :
  ρ.exp_val (A + B) = ρ.exp_val A + ρ.exp_val B := by
  simp [exp_val, inner_add_right]

@[simp]
theorem exp_val_smul (r : ℝ) (A : HermitianMat d ℂ) :
  ρ.exp_val (r • A) = r * ρ.exp_val A := by

```

```

    simp [MState.exp_val]

@[gcongr]
theorem exp_val_le_exp_val (ρ : MState d) {A B : HermitianMat d ℂ} (h : A ≤ B) :
  ρ.exp_val A ≤ ρ.exp_val B := by
  simp only [MState.exp_val]
  refine inner_mono ρ.nonneg h

end exp_val

section pure

/-- A mixed state can be constructed as a pure state arising from a ket. -
/
def pure (ψ : Ket d) : MState d where
  M := {
    val := Matrix.vecMulVec ψ (ψ : Bra d)
    property := (Matrix.PosSemidef.outer_self_conj ψ).1
  }
  nonneg := HermitianMat.zero_le_iff.mpr (.outer_self_conj ψ)
  tr := by
    have h₁ (x) : ψ x * conj (ψ x) = Complex.normSq (ψ x) := by
      rw [mul_comm, Complex.normSq_eq_conj_mul_self]
    simp [HermitianMat.trace_eq_re_trace, Matrix.trace, Matrix.vecMulVec_apply]
    exact ψ.normalized

proof_wanted pure_inner : ∀ pure ψ, pure φ → Prob = ||Braket.dot ψ φ||^2

@[simp]
theorem pure_apply {i j : d} : (pure ψ).m i j = (ψ i) * conj (ψ j) := by
  rfl

theorem pure_mul_self : (pure ψ).m * (pure ψ).m = (pure ψ : Matrix d d ℂ) :
  dsimp [pure, MState.m]
  simp [Matrix.vecMulVec_mul_vecMulVec, ← Braket.dot_eq_dotProduct]

/-- The purity of a state is  $\text{Tr}[\rho^2]$ . This is a `Prob`, because it is always
between zero and one. -/
def purity (ρ : MState d) : Prob := ∀ ρ, ρ → Prob

/-- The eigenvalue spectrum of a mixed quantum state, as a `Distribution`.
/
def spectrum (ρ : MState d) : ProbDistribution d :=
  ProbDistribution.mk'
    (ρ.M.H.eigenvalues .)
    (ρ.psd.eigenvalues_nonneg .)

```



```

    (by rw [sum_eigenvalues_eq_trace, p.tr])

/-- The spectrum of a pure state is (1,0,0,...), i.e. a constant distribution. -
/
theorem spectrum_pure_eq_constant :
  ∃ i, (pure ψ).spectrum = ProbDistribution.constant i := by
  let ρ := pure ψ
  -- Prove 1 is in the spectrum of pure ψ by exhibiting an eigenvector with value 1.
  have h1 : ∃ i, (pure ψ).spectrum i = 1 := by
    simp [spectrum, ProbDistribution.mk']
  have hEig : ∃ i, (pure ψ).M.H.eigenvalues i = 1 := by
    -- Prove ψ is an eigenvector of ρ = pure ψ
    have hv : ρ.M *v ψ = ψ := by
      ext
      simp_rw [ρ, pure, Matrix.mulVec, mat, Matrix.vecMulVec_apply, dotProduct,
        Bra.apply', Ket.apply, mul_assoc, ← Finset.mul_sum, ← Complex.normSq_eq_conj,
        ← Complex.ofReal_sum, ← Ket.apply, ψ.normalized, Complex.ofReal_one, mul_one]
    let U : Matrix.unitaryGroup d ℂ := star ρ.M.H.eigenvectorUnitary -- Diagonaliz
    let w : d → ℂ := U *v ψ
    -- Prove w = U ψ is an eigenvector of the diagonalized matrix of ρ = pure ψ
    have hDiag : Matrix.diagonal (RCLike.ofReal ∘ ρ.M.H.eigenvalues) *v w = w := by
      simp_rw [← Matrix.IsHermitian.conjStarAlgAut_star_eigenvectorUnitary,
        eq_comm, Unitary.conjStarAlgAut_apply,
        ← Matrix.mulVec_mulVec, w, U, Matrix.mulVec_mulVec] -- Uses spectral theorem
      simp_all
      rw [←Matrix.mulVec_mulVec, hv]
    -- Prove w = U ψ is nonzero by contradiction
    have hwNonZero : ∃ j, w j ≠ 0 := by
      by_contra hwZero
      simp at hwZero
      rw [←funext_iff] at hwZero
      -- If w is zero, then ψ is zero, since U is invertible
      have hψZero : ∀ x, ψ x = 0 := by
        apply congr_fun
        -- Prove U is invertible
        have hUdetNonZero : (U : Matrix d d ℂ).det ≠ 0 := by
          by_contra hDetZero
          obtain ⟨u, huUni⟩ := U
          have h0uni : 0 ∈ unitary ℂ := by
            rw [←hDetZero]
            simp
            exact Matrix.det_of_mem_unitary huUni
          rw [Unitary.mem_iff] at h0uni
          simp_all
          exact Matrix.eq_zero_of_mulVec_eq_zero hUdetNonZero hwZero
        -- Reach an contradiction that ψ has norm 0

```

```

have hψn := Ket.normalized ψ
have hnormZero : ∀ x : d, Complex.normSq (ψ x) = 0 := fun x => by
  rw [hψZero x, Complex.normSq_zero]
have hsumZero : ∑ x : d, Complex.normSq (ψ x) = 0 := by
  apply Finset.sum_eq_zero
  intros x _
  exact hnormZero x
simp_all
obtain ⟨j, hwNonZero'⟩ := hwNonZero
have hDiagj := congr_fun hDiag j
rw [Matrix.mulVec_diagonal, mul_eq_right, hwNonZero'] at hDiagj
use j
simp_all
obtain ⟨i, hEig'⟩ := hEig
use i
ext
exact hEig'
-- If 1 is in a distribution, the distribution is a constant.
obtain ⟨i, hi⟩ := this
use i
exact ProbDistribution.constant_of_exists_one hi

/-- If the spectrum of a mixed state is (1,0,0...) i.e. a constant distribution
a pure state. -/
theorem pure_of_constant_spectrum (h : ∃ i, ρ.spectrum = ProbDistribution.c
  ∃ ψ, ρ = pure ψ := by
  obtain ⟨i, h'⟩ := h
  -- Translate assumption to eigenvalues being (1,0,0,...)
  have hEig : ρ.M.H.eigenvalues = fun x => if x = i then 1 else 0 := by
    ext x
    simp [spectrum, ProbDistribution.constant, ProbDistribution.mk'] at h'
    rw [Subtype.mk.injEq] at h'
    have h'x := congr_fun h' x
    rw [if_congr (Eq.comm) (Eq.refl 1) (Eq.refl 0)]
    rw [Prob.ext_iff] at h'x
    dsimp at h'x
    rw [h'x]
    split_ifs
    case pos => rfl
    case neg => rfl
  -- Choose the eigenvector v of ρ with eigenvalue 1 to make ψ
  let ⟨u, huUni⟩ := ρ.M.H.eigenvectorUnitary -- Diagonalizing unitary of ρ
  let D : Matrix d d ℂ := Matrix.diagonal (RCLike.ofReal ∘ ρ.M.H.eigenvalue
  let v : EuclideanSpace ℂ d := ρ.M.H.eigenvectorBasis i
  -- Prove v is normalized
  have hUvNorm : ∑ x, ||v x||^2 = 1 := by

```

```

have hinnerv : Inner.inner  $\mathbb{C}$  v v = 1 := by
  have :=  $\rho$ .M.H.eigenvectorBasis.orthonormal
  rw [orthonormal_iff_ite] at this
  convert this i i
  simp
  simp only [PiLp.inner_apply, RCLike.inner_apply, Complex.mul_conj'] at hinnerv
  rw [← Fintype.sum_equiv (Equiv.refl d) _ (fun x => (Complex.ofReal ||v x||) ^ 2) (
  rw [← Complex.ofReal_sum Finset.univ (fun x => ||v x|| ^ 2), Complex.ofReal_eq_one
  exact hinnerv
let  $\psi$  : Ket d := (v, hUvNorm) -- Construct  $\psi$ 
use  $\psi$ 
ext j k
-- Use spectral theorem to prove that  $\rho$  = pure  $\psi$ 
rw [Matrix.IsHermitian.spectral_theorem  $\rho$ .M.H, Unitary.conjStarAlgAut_apply, Matrix
simp [ $\psi$ , v, hEig]
have hsum :  $\forall x \in \text{Finset.univ}, x \notin (\{i\} : \text{Finset } d) \rightarrow (\rho$ .M.H.eigenvectorBasis x j)
  intros x hx hxnoti
  rw [Finset.mem_singleton] at hxnoti
  rw [if_neg hxnoti, Complex.ofReal_zero]
  ring
simp_rw [←Finset.sum_subset (Finset.subset_univ {i}) hsum, Finset.sum_singleton, r
rfl

/-- A state  $\rho$  is pure iff its spectrum is (1,0,0,...) i.e. a constant distribution.
/
theorem pure_iff_constant_spectrum : ( $\exists \psi, \rho$  = pure  $\psi$ )  $\leftrightarrow$ 
   $\exists i, \rho$ .spectrum = ProbDistribution.constant i :=
  (fun h  $\mapsto$  h.rec fun  $\psi$  h2  $\mapsto$  h2  $\triangleright$  spectrum_pure_eq_constant  $\psi$ ,
  pure_of_constant_spectrum  $\rho$ )

theorem pure_iff_purity_one : ( $\exists \psi, \rho$  = pure  $\psi$ )  $\leftrightarrow$   $\rho$ .purity = 1 := by
  --purity = exp(-Collision entropy)
  --purity eq 1 iff collision entropy is zero
  --entropy is zero iff distribution is constant
  --distribution is constant iff pure
  constructor <=> intro h;
  · obtain (w, rfl) := h
    dsimp [purity, inner]
    have := pure_mul_self w
    aesop;
  · --TODO Cleanup
    -- Apply the theorem that states a mixed state is pure if and only if its spectr
    apply (pure_iff_constant_spectrum  $\rho$ ).mpr;
    have h_eigenvalues :  $\sum i, (\rho$ .spectrum i).val ^ 2 = 1 := by
      -- By definition of purity, we know that the sum of the squares of the eigenva
      have h_trace_sq :  $\sum i, (\rho$ .spectrum i).val ^ 2 =  $\rho$ .purity := by

```

```

have h_eigenvalues :  $\sum i, (\rho.M.H.eigenvalues\ i) ^ 2 = (\rho.M.mat * \rho$ 
  have := Matrix.IsHermitian.spectral_theorem  $\rho.M.H$ ;
  conv_rhs => rw [ this ];
  simp [ Matrix.trace_mul_comm, Matrix.mul_assoc ];
  exact Finset.sum_congr rfl fun _ => by ring;
  convert congr_arg Complex.re h_eigenvalues using 1;
  simp_all only [Set.Icc.coe_one]
have h_eigenvalues :  $\sum i, (\rho.spectrum\ i).val * ((\rho.spectrum\ i).val - 1)$ 
  simp_all [ sq, mul_sub ];
-- Since each term in the sum is non-positive and their sum is zero, ea
have h_each_zero :  $\forall i, (\rho.spectrum\ i).val * ((\rho.spectrum\ i).val - 1) =$ 
  have h_each_zero :  $\forall i, (\rho.spectrum\ i).val * ((\rho.spectrum\ i).val - 1)$ 
    exact fun i => by nlinarith only [ show (  $\rho.spectrum\ i : \mathbb{R}$  )  $\geq 0$  by
    exact fun i => le_antisymm ( h_each_zero i ) ( by simp [ h_eigenvalu
-- Since each term in the sum is non-positive and their sum is zero, ea
have h_each_zero :  $\forall i, (\rho.spectrum\ i).val = 0 \vee (\rho.spectrum\ i).val = 1$ 
  exact fun i => mul_eq_zero.mp ( h_each_zero i ) |> Or.imp id fun h =>
have h_sum_one :  $\sum i, (\rho.spectrum\ i).val = 1 :=$  by
  grind;
obtain (i, hi) :  $\exists i, (\rho.spectrum\ i).val = 1 :=$  by
  contrapose! h_sum_one; aesop;
-- Since the sum of the eigenvalues is 1 and one of them is 1, the rema
have h_sum_zero :  $\sum j \in Finset.univ.erase\ i, (\rho.spectrum\ j).val = 0 :=$ 
  rw [  $\leftarrow$  Finset.sum_erase_add _ ( Finset.mem_univ i ), hi ] at h_sum_
  rw [ Finset.sum_eq_zero_iff_of_nonneg ] at h_sum_zero
· simp_all only [Finset.sum_const_zero, mul_eq_zero, Set.Icc.coe_eq_zer
  ProbDistribution.normalized, Finset.mem_erase, ne_eq, Finset.mem_un
  apply Exists.intro
· ext x : 2
  simp_all only [ProbDistribution.constant_eq]
  split
  next h_1 =>
    subst h_1
    simp_all only [Set.Icc.coe_one, Set.Icc.coe_eq_one]
    exact hi
  next h_1 =>
    simp_all only [Set.Icc.coe_zero, Set.Icc.coe_eq_zero]
    apply h_sum_zero
    apply Aesop.BuiltinRules.not_intro
    intro a
    subst a
    simp_all only [not_true_eq_false]
· intro i_1 a
  simp_all only [Finset.sum_const_zero, mul_eq_zero, Set.Icc.coe_eq_zer
  ProbDistribution.normalized, Finset.mem_univ, Finset.sum_erase_eq_s
  ne_eq, and_true, Prob.zero_le_coe]

```

```

--TODO: Would be better if there was an `MState.eigenstate` or similar (maybe extend
-- a similar thing for `HermitianMat`) and then this could be an equality with that,
-- an explicit formula, instead of this `Exists`.
theorem spectralDecomposition (ρ : MState d) :
  ∃ (ψs : d → Ket d), ρ.M = ∑ i, (ρ.spectrum i : ℝ) • (MState.pure (ψs i)).M := by
  use (fun i ↦ ((ρ.M.H.eigenvectorUnitary · i), Matrix.unitaryGroup_row_norm _ i))
  ext i j
  nth_rw 1 [ρ.M.H.spectral_theorem]
  --TODO Cleanup
  simp only [Complex.coe_algebraMap, spectrum, ProbDistribution.mk',
    ProbDistribution.funlike_apply, pure, Matrix.IsHermitian.eigenvectorUnitary_appl
  rw [HermitianMat.mat_finset_sum]
  simp only [Unitary.conjStarAlgAut_apply]
  rw [Finset.sum_apply, Finset.sum_apply, Matrix.mul_apply]
  congr!
  simp only [Matrix.mul_diagonal, Matrix.IsHermitian.eigenvectorUnitary_apply,
    mul_comm, Matrix.star_apply, RCLike.star_def]
  simp only [Function.comp_apply, mat_M, mat_apply, smul_apply, Complex.real_smul]
  rw [mul_assoc]
  rfl

end pure

section prod

def prod (ρ₁ : MState d₁) (ρ₂ : MState d₂) : MState (d₁ × d₂) where
  M := ρ₁.M ⊗ₖ ρ₂.M
  nonneg := HermitianMat.zero_le_iff.mpr (ρ₁.psd.PosSemidef_kronecker ρ₂.psd)
  tr := by simp

infixl:100 " ⊗ᵐ " => MState.prod

theorem prod_inner_prod (ξ₁ ψ₁ : MState d₁) (ξ₂ ψ₂ : MState d₂) :
  ∑ ξ₁ ⊗ᵐ ξ₂, ψ₁ ⊗ᵐ ψ₂ ⊔_Prob = ∑ ξ₁, ψ₁ ⊔_Prob * ∑ ξ₂, ψ₂ ⊔_Prob := by
  ext1
  simp only [inner_def, Prob.coe_mul, ← Complex.ofReal_inj]
  --Lots of this should actually be facts about HermitianMat first
  simp only [prod, Complex.ofReal_mul]
  simp only [← RCLike.ofReal_eq_complex_ofReal, inner_eq_trace_rc]
  simp only [kronecker, ← Matrix.trace_kronecker]
  simp only [mat_M, mat_mk, Matrix.mul_kronecker_mul]

/-- The product of pure states is a pure product state , `Ket.prod`. -
/
theorem pure_prod_pure (ψ₁ : Ket d₁) (ψ₂ : Ket d₂) : pure (ψ₁ ⊗ ψ₂) = (pure ψ₁) ⊗ᵐ

```

```

    ext : 3
    simp [Ket.prod, Ket.apply, prod, -mat_apply]
    ac_rfl

end prod

/-- A representation of a classical distribution as a quantum state, diagonal
/
def ofClassical (dist : ProbDistribution d) : MState d where
  M := diagonal ℂ (fun x ↦ dist x)
  nonneg := by simp [zero_le_iff, diagonal, Matrix.posSemidef_diagonal_iff]
  tr := by simp [trace_diagonal]

@[simp]
theorem coe_ofClassical (dist : ProbDistribution d) :
  (ofClassical dist).M = diagonal ℂ (dist ·) := by
  rfl

theorem ofClassical_pow (dist : ProbDistribution d) (p : ℝ) :
  (ofClassical dist).M ^ p = diagonal ℂ (fun i ↦ (dist i) ^ p) := by
  rw [coe_ofClassical, diagonal_pow]

/-- The maximally mixed state. -/
def uniform [Nonempty d] : MState d :=
  ofClassical ProbDistribution.uniform

/-- There is exactly one state on a dimension-1 system. -
/
--note that this still takes (and uses) the `Fintype d` and `DecidableEq d`
--Even though instances for those can be derived from `Unique d`, we want to
--apply on `@MState d ?x ?y` for _any_ x and y.
instance instUnique [Unique d] : Unique (MState d) where
  default := @uniform _ _ _ _
  uniq := by
    intro p
    ext
    have h₁ := p.tr
    have h₂ := (@uniform _ _ _ _ : MState d).tr
    simp [Matrix.trace, Unique.eq_default, -MState.tr, HermitianMat.trace_e
    apply Complex.ext
    · exact h₁.trans h₂.symm
    · rw [complex_im_eq_zero, complex_im_eq_zero]

/-- There exists a mixed state for every nonempty `d`.
Here, the maximally mixed one is chosen. -/
instance instInhabited [Nonempty d] : Inhabited (MState d) where

```

```

    default := uniform

@[simp]
theorem M_default [Unique d] : (default : MState d).M = 1 := by
  simp [instInhabited, uniform]
  rfl

section ptrace

/-- Partial tracing out the left half of a system. -/
@[simps]
def traceLeft (ρ : MState (d1 × d2)) : MState d2 where
  M := ρ.M.traceLeft
  nonneg := zero_le_iff.mpr ρ.psd.traceLeft
  tr := by simp [trace]

/-- Partial tracing out the right half of a system. -/
@[simps]
def traceRight (ρ : MState (d1 × d2)) : MState d1 where
  M := ρ.M.traceRight
  nonneg := zero_le_iff.mpr ρ.psd.traceRight
  tr := by simp [trace]

/-- Taking the direct product on the left and tracing it back out gives the same state. -/
@[simp]
theorem traceLeft_prod_eq (ρ1 : MState d1) (ρ2 : MState d2) : (ρ1 ⊗M ρ2).traceLeft =
  ext1
  simp [prod]

/-- Taking the direct product on the right and tracing it back out gives the same state. -/
@[simp]
theorem traceRight_prod_eq (ρ1 : MState d1) (ρ2 : MState d2) : (ρ1 ⊗M ρ2).traceRight =
  ext1
  simp [prod]

end ptrace

-- TODO: direct sum (by zero-padding)

--TODO: Spectra of left- and right- partial traces of a pure state are equal.

/-- Spectrum of direct product. There is a permutation σ so that the spectrum of the
    ρ1 and ρ2, as permuted under σ, is the pairwise products of the spectra of ρ1 and
    /

```

```

theorem spectrum_prod (p1 : MState d1) (p2 : MState d2) : ∃ (σ : d1 × d2 ≈ d1 × d2)
  ∀ i, ∀ j, (p1 ⊗M p2).spectrum (σ (i, j)) = (p1.spectrum i) * (p2.spectrum j)
--TODO Cleanup
by_contra! h;
-- Apply `Matrix.IsHermitian.eigenvalues_eq_of_unitary_similarity_diagonal`
obtain ⟨σ, hσ⟩ : ∃ σ : d1 × d2 ≈ d1 × d2, (p1.prod p2).M.H.eigenvalues σ = 0
have h_unitary : ∃ U : Matrix (d1 × d2) (d1 × d2) ℂ, U ∈ Matrix.unitaryGroup
-- Let $U_A$ and $U_B$ be the eigenvector unitaries of $\rho_1$ and $\rho_2$
obtain ⟨U_A, hU_A⟩ : ∃ U_A : Matrix d1 d1 ℂ, U_A ∈ Matrix.unitaryGroup
have := p1.M.H.spectral_theorem;
refine' { _, _, this };
simp
obtain ⟨U_B, hU_B⟩ : ∃ U_B : Matrix d2 d2 ℂ, U_B ∈ Matrix.unitaryGroup
have := p2.M.H.spectral_theorem;
refine' { _, _, this };
simp
refine' { Matrix.kroneckerMap (fun x y => x * y) U_A U_B, _, _ };
· simp_all only [ne_eq, Matrix.mem_unitaryGroup_iff, mat_M, Matrix.mul_assoc]
have h_unitary : Matrix.kroneckerMap (fun x y => x * y) U_A U_B * M
have h_unitary : Matrix.kroneckerMap (fun x y => x * y) U_A U_B *
  ext { i, j } { k, l } ; simp [ Matrix.mul_apply, Matrix.kroneckerMap_apply,
    ring_nf
    erw [ Finset.sum_product ]
    simp [ mul_assoc, mul_comm, mul_left_comm, Finset.mul_sum ]
    exact Finset.sum_comm.trans ( Finset.sum_congr rfl fun _ => F
    simp_all only [zero_mul, implies_true, mul_zero, mul_one, Matrix.mul_assoc]
  exact h_unitary
· simp_all [ MState.prod, Matrix.mul_assoc, Matrix.mul_kronecker_mul ]
  congr 2;
· ext { i, j } { i', j' } ; by_cases hi : i = i' <=> by_cases hj : j = j'
· ext i j; simp [ Matrix.conjTranspose_apply, Matrix.kroneckerMap_apply ]
obtain ⟨ U, hU1, hU2 ⟩ := h_unitary;
apply Matrix.IsHermitian.eigenvalues_eq_of_unitary_similarity_diagonal;
exact hU1;
convert hU2 using 1;
norm_num +zetaDelta at *;
obtain ⟨ i, j, h ⟩ := h σ; have := congr_fun hσ ( i, j ) ; simp_all [ MState.prod ]
exact h ( by exact Subtype.ext this )

theorem sInf_spectrum_prod (p : MState d) (σ : MState d2) :
  sInf ( _root_.spectrum ℝ (p ⊗M σ).m ) = sInf ( _root_.spectrum ℝ p.m ) * sInf ( _root_.spectrum ℝ σ.m )
rcases isEmpty_or_nonempty d with _ | _; · simp
rcases isEmpty_or_nonempty d2 with _ | _; · simp
rw [MState.m, MState.prod, HermitianMat.spectrum_prod, ← MState.m, ← MState.prod]
apply csInf_mul_nonneg
· exact IsSelfAdjoint.spectrum_nonempty p.M.H

```



```

    · rw [MState.m, p.M.H.spectrum_real_eq_range_eigenvalues]
      rintro _ {i, rfl}
      apply p.eigenvalue_nonneg
    · exact IsSelfAdjoint.spectrum_nonempty σ.M.H
    · rw [MState.m, σ.M.H.spectrum_real_eq_range_eigenvalues]
      rintro _ {i, rfl}
      apply σ.eigenvalue_nonneg

--TODO: Spectrum of direct sum. Spectrum of partial trace?

/-- A mixed state is separable iff it can be written as a convex combination of prod
/
def IsSeparable (p : MState (d₁ × d₂)) : Prop :=
  ∃ pLRs : Finset (MState d₁ × MState d₂), --Finite set of (pL, pR) pairs
  ∃ ps : ProbDistribution pLRs, --ProbDistribution over those pairs, an ensemble
  p.M = ∑ pLR : pLRs, (ps pLR : ℝ) • (Prod.fst pLR.val).M ⊗k (Prod.snd pLR.val).

/-- A product state `MState.prod` is separable. -/
theorem IsSeparable_prod (p₁ : MState d₁) (p₂ : MState d₂) : IsSeparable (p₁ ⊗M p₂)
  let only := (p₁, p₂)
  use { only }, ProbDistribution.constant {only, Finset.mem_singleton_self only}
  simp [prod, Unique.eq_default, only]

theorem eq_of_sum_eq_pure {d : Type*} [Fintype d] [DecidableEq d]
  {ι : Type*} {s : Finset ι} {p : ι → ℝ} {ps : ι → MState d}
  {ρ : MState d} (h_pure : ρ.purity = 1) (h_sum : ρ.M = ∑ i ∈ s, p i • (ps i).M)
  (hp_nonneg : ∀ i ∈ s, 0 ≤ p i) (hp_sum : ∑ i ∈ s, p i = 1) (i : ι) (hi : i ∈ s)
  ps i = ρ := by
  have h_trace : ∀ j ∈ s, 0 < p j → (⊖ ρ.M, (ps j).M) = 1 := by
  have h_tr_pure : ∑ j ∈ s, p j • ⊖ ρ.M, (ps j).M = 1 := by
    have h_tr_pure : ⊖ ρ.M, ∑ j ∈ s, p j • (ps j).M = ∑ j ∈ s, p j • ⊖ ρ.M, (ps j).
      simp [HermitianMat.inner_def, ← val_eq_coe];
      rw [AddSubgroup.val_finset_sum]
      simp [Finset.mul_sum]
      rw [← h_tr_pure, ← h_sum];
      convert h_pure using 1;
      exact beq_eq_beq.mp rfl;
  have h_tr_le_one : ∀ j ∈ s, ⊖ ρ.M, (ps j).M ≤ 1 := by
    intro j hj
    have h_tr_le_one_j : ⊖ ρ.M, (ps j).M ≤ ρ.M.trace * (ps j).M.trace := by
      apply HermitianMat.inner_le_mul_trace
      · exact p.nonneg;
      · exact (ps j).nonneg;
    simp_all only [smul_eq_mul, tr, mul_one, ge_iff_le]
    exact h_tr_le_one_j.trans (h_sum ▸ p.tr.le);
  intro j hj hj_pos

```

```

by_contra h_contra;
have h_tr_lt_one :  $\sum j \in s, p j \cdot \lceil \rho.M, (\rho s j).M \rceil < \sum j \in s, p j :=$  by
  apply Finset.sum_lt_sum;
  · exact fun i hi => mul_le_of_le_one_right (hp_nonneg i hi) (h_tr
  · exact (j, hj, mul_lt_of_lt_one_right hj_pos (lt_of_le_of_ne (h_t
linarith;
have h_eq :  $\rho.M = (\rho s i).M :=$  by
  have h_eq :  $\lceil \rho.M - (\rho s i).M, \rho.M - (\rho s i).M \rceil = 0 :=$  by
  have h_eq :  $\lceil \rho.M - (\rho s i).M, \rho.M - (\rho s i).M \rceil = \lceil \rho.M, \rho.M \rceil - 2 * \lceil \rho.M,$ 
    simp only [HermitianMat.inner_def, IsMaximalSelfAdjoint.RCLike_self
    RCLike.re_to_complex];
  simp [Matrix.mul_sub, Matrix.sub_mul, Matrix.trace_sub, Matrix.trace
  ring
  have h_eq :  $\lceil \rho.M, \rho.M \rceil = 1 \wedge \lceil (\rho s i).M, (\rho s i).M \rceil \leq 1 :=$  by
  have h_eq :  $\lceil \rho.M, \rho.M \rceil = 1 :=$  by
    convert h_pure using 1;
    exact beq_eq_beq.mp rfl;
  have h_eq :  $\forall (A : \text{HermitianMat } d \mathbb{C}), 0 \leq A \rightarrow A.\text{trace} = 1 \rightarrow \lceil A, A \rceil$ 
    intros A hA_nonneg hA_trace
  have h_eq :  $\lceil A, A \rceil \leq A.\text{trace} * A.\text{trace} :=$  by
    apply HermitianMat.inner_le_mul_trace hA_nonneg hA_nonneg;
    aesop;
  exact (by assumption, h_eq _ (ps i |>.2) (ps i |>.3));
  linarith [h_trace i hi hpi, (rho.M - (rho s i).M).inner_self_nonneg];
-- Since the inner product of a matrix with itself is zero if and only
have h_zero :  $\rho.M - (\rho s i).M = 0 :=$  by
  apply inner_self_eq_zero.mp h_eq;
  exact eq_of_sub_eq_zero h_zero;
exact MState.ext h_eq.symm

theorem purity_prod {d1 d2 : Type*} [Fintype d1] [Fintype d2] [DecidableEq
  (rho1 : MState d1) (rho2 : MState d2) : (rho1  $\otimes^M$  rho2).purity = rho1.purity * rho2.
  exact prod_inner_prod rho1 rho1 rho2 rho2

theorem pure_eq_pure_iff {d : Type*} [Fintype d] [DecidableEq d] (psi phi : Ket
  pure psi = pure phi  $\leftrightarrow \exists z : \mathbb{C}, \|z\| = 1 \wedge \psi.\text{vec} = z \cdot \phi.\text{vec} :=$  by
  refine' (fun h => _, fun h => _);
  · -- By definition of pure state, we have that  $\psi.\text{vec} * \text{conj } \psi.\text{vec} = \phi.\text{vec}$ 
  have h_eq :  $\forall i j, \psi.\text{vec } i * \text{starRingEnd } \mathbb{C} (\psi.\text{vec } j) = \phi.\text{vec } i * \text{starRi}$ 
    intro i j;
  replace h := congr_arg (fun rho => rho.M.mat i j) h; aesop;
  -- Let  $\psi_k$  be such that  $\psi_k \neq 0$ .
  obtain (k, hk) :  $\exists k, \phi.\text{vec } k \neq 0 :=$  by
    exact phi.exists_ne_zero;
  -- Let  $z = \frac{\psi_k}{\phi_k}$ . Then  $|z| = 1$ .
  obtain (z, hz) :  $\exists z : \mathbb{C}, \psi.\text{vec } k = z * \phi.\text{vec } k \wedge \|z\| = 1 :=$  by

```

```

    specialize h_eq k k
    simp_all only [ne_eq]
    refine' (  $\psi.\text{vec } k / \phi.\text{vec } k, \_, \_$  ) <=> simp_all [ Complex.mul_conj, Complex.n
    rw [ div_eq_iff ] <=> norm_cast at * <=> aesop;
    refine' (  $\bar{z}, \text{hz.2}, \text{funext fun } i \Rightarrow \_$  );
    specialize h_eq i k
    simp_all
    -- Since  $\overline{z} \cdot \overline{\varphi_k} \neq 0$ , we can divide both si
    have h_div :  $\psi.\text{vec } i * \text{starRingEnd } \mathbb{C} \, z = \phi.\text{vec } i$  := by
      exact mul_left_cancel₀ ( show starRingEnd  $\mathbb{C} \, (\phi.\text{vec } k) \neq 0$  from by simp [ Co
    rw [ ← h_div, mul_left_comm, Complex.mul_conj, Complex.normSq_eq_norm_sq ] ; aes
    · cases h
      rename_i h
      obtain (left, right) := h
      -- Since  $|w| = 1$ , we have  $w \overline{w} = 1$ , which simplifies the matrix to
      have h_simp :  $\forall i j, \psi.\text{vec } i * \text{star } (\psi.\text{vec } j) = \phi.\text{vec } i * \text{star } (\phi.\text{vec } j)$  := by
        simp [ *, Complex.ext_iff ];
        intro i j; rw [ Complex.norm_def ] at left; simp_all [ Complex.normSq ];
        grind +ring;
      exact MState.ext_m ( by ext i j; simp [ Matrix.vecMulVec ] using h_simp i j )

/-- Two kets are phase-equivalent if and only if their pure states are equal. -
/
theorem PhaseEquiv_iff_pure_eq {d : Type*} [Fintype d] [DecidableEq d] ( $\psi \, \phi$  : Ket d)
  Ket.PhaseEquiv.r  $\psi \, \phi \leftrightarrow \text{MState.pure } \psi = \text{MState.pure } \phi$  := by
  exact (pure_eq_pure_iff  $\psi \, \phi$ ).symm

/-- `MState.pure` descends to the quotient `KetUpToPhase`. -
/
def pureQ {d : Type*} [Fintype d] [DecidableEq d] : KetUpToPhase d → MState d :=
  @Quotient.lift _ _ Ket.PhaseEquiv MState.pure (fun a b h => (PhaseEquiv_iff_pure_e

@[simp]
theorem pureQ_mk {d : Type*} [Fintype d] [DecidableEq d] ( $\psi$  : Ket d) :
  pureQ (Quotient.mk Ket.PhaseEquiv  $\psi$ ) = MState.pure  $\psi$  := rfl

theorem pureQ_injective {d : Type*} [Fintype d] [DecidableEq d] : Function.Injective
  intro a b h
  induction a using Quotient.ind
  induction b using Quotient.ind
  simp [pureQ] at h
  exact Quotient.sound ((PhaseEquiv_iff_pure_eq _ _).mpr h)

theorem pure_separable_imp_IsProd {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂] [Decidab
  ( $\psi$  : Ket (d₁ × d₂)) (h : IsSeparable (pure  $\psi$ )) :  $\psi$ .IsProd := by
  obtain ( pLRs, ps, hps ) := h;

```

```

-- Since `pure  $\psi$ ` is pure (`purity = 1`), by `MState.eq_of_sum_eq_pure`,
obtain (k, hk) :  $\exists k : \{ x : \text{MState } d_1 \times \text{MState } d_2 \mid x \in \text{pLRs} \}, 0 < (\text{ps}$ 
  have h_pure : (MState.pure  $\psi$ ).purity = 1 := by
    exact ( pure_iff_purity_one _ ).mp (  $\psi$ , rfl );
obtain (k, hk) :  $\exists k : \{ x : \text{MState } d_1 \times \text{MState } d_2 \mid x \in \text{pLRs} \}, 0 < ($ 
  exact ( Classical.choose ( show  $\exists k : \text{pLRs}, 0 < (\text{ps } k : \mathbb{R})$  ) from by
  refine' ( k, hk, _ );
convert MState.eq_of_sum_eq_pure h_pure _ _ _ k _ _;
rotate_left;
exact Finset.univ;
use fun x => ( ps x :  $\mathbb{R}$  );
use fun x => MState.prod x.val.1 x.val.2;
all_goals norm_cast;
· exact fun i a => unitInterval.nonneg (ps i);
· exact ps.2;
· exact Finset.mem_univ k;
· obtain (val, property) := k
  obtain (fst, snd) := val
  have fwd :  $0 \leq (\text{ps } ((\text{fst}, \text{snd}), \text{property})).1 := \text{le\_of\_lt } \text{hk}$ 
  apply Iff.intro
  · intro a
    exact MState.ext_iff.mpr a.symm
  · intro a
    rw [← a]
    rfl
-- Since `pure  $\psi$ ` is pure (`purity = 1`), by `MState.pure_iff_purity_one`
obtain ( $\xi$ , h $\xi$ ) :  $\exists \xi : \text{MState } d_1, k.\text{val}.1 = \xi \wedge \xi.\text{purity} = 1 := \text{by}$ 
  have h_purity : (pure  $\psi$ ).purity = (k.val.1).purity * (k.val.2).purity :
    convert MState.purity_prod _ _;
    exact MState.ext hk.2;
  have h_purity_one : (pure  $\psi$ ).purity = 1 := by
    exact ( pure_iff_purity_one _ ).mp (  $\psi$ , rfl );
  rw [ h_purity, Prob.mul_eq_one_iff ] at h_purity_one ; aesop
obtain ( $\phi$ , h $\phi$ ) :  $\exists \phi : \text{MState } d_2, k.\text{val}.2 = \phi \wedge \phi.\text{purity} = 1 := \text{by}$ 
  have h_purity_prod : (pure  $\psi$ ).purity = (k.val.1).purity * (k.val.2).purity :
    convert MState.purity_prod _ _;
    exact MState.ext hk.2;
  have h_purity_one : (MState.pure  $\psi$ ).purity = 1 := by
    exact pure_iff_purity_one _ |>.1 (  $\psi$ , rfl );
  aesop;
-- Since ` $\xi$ ` and ` $\phi$ ` are pure states, we have ` $\xi = \text{pure } \xi'$ ` and ` $\phi = \text{pure } \phi'$ `
obtain ( $\xi'$ , h $\xi'$ ) :  $\exists \xi' : \text{Ket } d_1, \xi = \text{MState.pure } \xi' := \text{by}$ 
  have := MState.pure_iff_purity_one  $\xi$ ;
  exact this.mpr h $\xi$ .2
obtain ( $\phi'$ , h $\phi'$ ) :  $\exists \phi' : \text{Ket } d_2, \phi = \text{MState.pure } \phi' := \text{by}$ 
  have := MState.pure_iff_purity_one  $\phi$ ; aesop;

```

```

-- Since `pure  $\psi$  = pure  $\xi \otimes \text{pure } \phi$ `, we have ` $\psi = \xi \otimes \text{pure } \phi$ ` up to a global phase `z`
have h_eq : (pure  $\psi$ ).M = (pure ( $\xi' \otimes \text{pure } \phi'$ )).M := by
  rw [hk.2, h $\xi$ .1, h $\xi'$ , h $\phi$ .1, h $\phi'$ , MState.pure_prod_pure ];
  exact rfl;
-- Since `pure  $\psi$  = pure ( $\xi' \otimes \text{pure } \phi'$ )`, we have ` $\psi = \xi' \otimes \text{pure } \phi'$ ` up to a global phase `z`
have h_eq_ket :  $\exists z : \mathbb{C}, \|z\| = 1 \wedge \psi.\text{vec} = z \cdot (\xi' \otimes \text{pure } \phi').\text{vec}$  := by
  have := MState.pure_eq_pure_iff  $\psi$  ( $\xi' \otimes \text{pure } \phi'$ );
  exact this.mp (MState.ext h_eq);
obtain { z, hz1, hz2 } := h_eq_ket;
use { fun i => z *  $\xi'$  i, ?_ },  $\phi'$ ;
ext { i, j }; simp [Ket.prod];
convert congr_fun hz2 (i, j) using 1;
exact mul_assoc _ _;
simp [hz1]
simp [Complex.normSq_eq_norm_sq] using  $\xi'.\text{normalized'}$ 

/-- A pure state is separable iff the ket is a product state. -
/
theorem pure_separable_iff_IsProd ( $\psi : \text{Ket } (d_1 \times d_2)$ ) :
  IsSeparable (pure  $\psi$ )  $\leftrightarrow \psi.\text{IsProd}$  := by
  apply Iff.intro
  · exact pure_separable_imp_IsProd  $\psi$ 
  · rintro {  $\xi, \phi, \text{rfl}$  }
    rw [pure_prod_pure  $\xi \phi$ ]
    exact IsSeparable_prod _ _;

/--
A mixed state is pure if and only if its rank is 1.
-/
theorem pure_iff_rank_eq_one {d : Type*} [Fintype d] [DecidableEq d] ( $\rho : \text{MState } d$ )
  ( $\exists \psi, \rho = \text{pure } \psi$ )  $\leftrightarrow \rho.\text{m.rank} = 1$  := by
  constructor <=> intro h;
  · obtain {w, rfl} := h
    -- The rank of the outer product of a vector with itself is 1.
    have h_rank :  $\forall (v : d \rightarrow \mathbb{C}), v \neq 0 \rightarrow \text{Matrix.rank } (\text{Matrix.vecMulVec } v (\text{conj } v)) = 1$ 
      intro v hv_ne_zero
      have h_outer_product :  $\forall (u : d \rightarrow \mathbb{C}), \exists (c : \mathbb{C}), \text{Matrix.mulVec } (\text{Matrix.vecMulVec } u (\text{conj } u)) = c \cdot u$ 
        intro u
        use  $\sum i, (\text{starRingEnd } \mathbb{C} (v \ i)) * (u \ i)$ ;
        ext i; simp [Matrix.vecMulVec, Matrix.mulVec, dotProduct, mul_comm, mul_left_distrib];
        apply le_antisymm
      · have h_outer_product : LinearMap.range ( $\text{Matrix.mulVecLin } (\text{Matrix.vecMulVec } u (\text{conj } u))$ )
          rintro x { u, rfl }; obtain { c, hc } := h_outer_product u; aesop;
        exact le_trans (Submodule.finrank_mono h_outer_product) (finrank_span_le (Submodule.span  $\{u\}$ )));
      · contrapose! hv_ne_zero; simp_all [Matrix.rank, Submodule.eq_bot_iff];
        ext i; specialize hv_ne_zero (Pi.single i 1); simp_all [Matrix.vecMulVec]

```

```

    simp using congr_fun hv_ne_zero i;
    exact h_rank _ ( fun h => by simp [ h ] using w.exists_ne_zero );
    · -- Since  $\rho$  is Hermitian and has rank 1, it must be of the form  $|\psi\rangle\langle\psi|$  for
      obtain  $\langle\psi, h\psi\rangle : \exists \psi : d \rightarrow \mathbb{C}, \rho.m = \text{Matrix.of } (\text{fun } i \ j \Rightarrow \psi \ i * \text{star } (\psi \ j))$ 
      -- Since  $\rho$  is Hermitian and has rank 1, it must be of the form  $|\psi\rangle\langle\psi|$ 
      have h_pure :  $\exists \psi : d \rightarrow \mathbb{C}, \rho.m = \text{Matrix.of } (\text{fun } i \ j \Rightarrow \psi \ i * \text{star } (\psi \ j))$ 
      have h_rank :  $\rho.m.\text{rank} = 1 := h$ 
      have h_herm :  $\rho.m.\text{IsHermitian} :=$  by
        exact  $\rho.M.\text{property}$ 
      have := h_herm.spectral_theorem;
      -- Since the rank of  $\rho.m$  is 1, the diagonal matrix in the spectral
      zero entry.
      obtain  $\langle i, h_i \rangle : \exists i : d, h_herm.\text{eigenvalues } i \neq 0 \wedge \forall j : d, j \neq i$ 
      have h_diag :  $\sum i : d, (\text{if } h_herm.\text{eigenvalues } i = 0 \text{ then } 0 \text{ else } 1)$ 
      have h_diag :  $\text{Matrix.rank } (\text{Matrix.diagonal } (h_herm.\text{eigenvalues}))$ 
      have h_diag :  $\text{Matrix.rank } (\text{Matrix.diagonal } (h_herm.\text{eigenvalues}))$ 
      exact Eq.symm (Matrix.IsHermitian.rank_eq_rank_diagonal h_herm)
      exact h_diag.trans h_rank;
      rw [ Matrix.rank_diagonal ] at h_diag;
      simp [ Finset.sum_ite ];
      rw [ Fintype.card_subtype ] at h_diag ; exact h_diag;
      obtain  $\langle i, h_i \rangle : \exists i : d, h_herm.\text{eigenvalues } i \neq 0 :=$  by
        exact not_forall.mp fun h => by simp [ h ] at h_diag;
      rw [ Finset.sum_eq_add_sum_diff_singleton ( Finset.mem_univ i ) ]
      exact  $\langle i, h_i, \text{fun } j \ hj \Rightarrow \text{Classical.not\_not.1 fun } hj' \Rightarrow \text{absurd } hj' \rangle$ 
      -- Since the diagonal matrix in the spectral theorem has exactly one
      zero entry, we can write  $\rho.m$  as  $|\psi\rangle\langle\psi|$  for some ket  $\psi$ .
      use fun j => (h_herm.eigenvectorUnitary : Matrix d d  $\mathbb{C}$ ) j i * Real.sqrt h_i
      convert this using 1
      ext j k; simp [ Matrix.mul_apply, Matrix.diagonal ]
      ring_nf
      rw [ Finset.sum_eq_single i ] <|> simp +contextual [ h_i ];
      exact Or.inl <| Or.inl <| mod_cast Real.sqrt <| by
        have :=  $\rho.psd.\text{eigenvalues\_nonneg } i$ ;
        exact this
      exact h_pure;
      have h_norm :  $\sum x, \text{Complex.normSq } (\psi \ x) = 1 :=$  by
        have :=  $\rho.\text{tr}'$ ;
        simp_all [ Complex.ext_iff, Matrix.trace ];
        simp only [ Complex.normSq_apply, mul_comm ] using this.1;
      use  $\langle\psi, \text{by}$ 
        simp [ Complex.normSq_eq_norm_sq ] using h_norm)
      generalize_proofs at *;
      refine' MState.ext_m _ ; aesop
  /--

```

A ket on a product space is a product state if and only if its coefficient matrix has rank 1.

```

theorem Ket.IsProd_iff_rank_eq_one {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂] [DecidableEq d₁] [DecidableEq d₂] :
  (ψ : Ket (d₁ × d₂)) :
  ψ.IsProd ↔ (Matrix.of (fun i j => ψ (i, j))).rank = 1 := by
  rw [Ket.IsProd_iff_mul_eq_mul];
  constructor;
  · intro h;
    obtain ⟨ξ, ψ', hξψ'⟩ : ∃ ξ : d₁ → ℂ, ∃ ψ' : d₂ → ℂ, ∀ i j, ψ (i, j) = ξ i * ψ' j := h
    -- Let's choose any j₀ such that ψ(i, j₀) ≠ 0 for some i.
    obtain ⟨j₀, h_j₀⟩ : ∃ j₀ : d₂, ∃ i₀ : d₁, ψ (i₀, j₀) ≠ 0 := by
      have := ψ.exists_ne_zero;
      exact ⟨ this.choose.2, this.choose.1, this.choose_spec ⟩;
    choose i₀ h_i₀ using h_j₀;
    exact ⟨ fun i => ψ (i, j₀) / ψ (i₀, j₀), fun j => ψ (i₀, j), fun i j =>
      -- Since the matrix is a product of two vectors, its rank is 1.
      have h_rank : Matrix.rank (Matrix.of (fun i j => ξ i * ψ' j)) ≤ 1 := by
        -- The range of the matrix is spanned by the single vector ξ.
        have h_range : LinearMap.range (Matrix.mulVecLin (Matrix.of (fun i j => ξ i *
          rintro x ⟨ y, rfl ⟩);
          rw [Submodule.mem_span_singleton];
          exact ⟨ ∑ j, ψ' j * y j, by ext i; simp [Matrix.mulVec, dotProduct, mul_comm] ⟩;
          exact le_trans (Submodule.finite_rank_mono h_range) (finite_rank_span_le_card _) |
        cases h_rank.eq_or_lt <=> simp_all [Matrix.rank, Submodule.eq_bot_iff];
      · convert ⟨Module.finite_rank ℂ (LinearMap.range (Matrix.mulVecLin (Matrix.of fun
        · aesop;
        · ext; simp [hξψ']⟩;
      · have := ψ.exists_ne_zero
        simp_all only [ne_eq, mul_eq_zero, not_or, Prod.exists, exists_and_left, exist
        obtain ⟨left, right⟩ := this
        obtain ⟨w, h₂⟩ := left
        obtain ⟨w₁, h₃⟩ := right
        rename_i h₁
        specialize h₁ (Pi.single w₁ 1)
        simp_all [funext_iff]
      · rw [Matrix.rank];
      rw [finite_rank_eq_one_iff];
      intro a i₁ i₂ j₁ j₂
      simp_all only [ne_eq, Subtype.forall, LinearMap.mem_range, Matrix.mulVecBilin_ap
        Subtype.exists, Submodule.mk_eq_zero, SetLike.mk_smul_mk, Subtype.mk.injEq, fo
        exists_and_left, exists_prop]
      obtain ⟨w, h⟩ := a
      obtain ⟨left, right⟩ := h
      obtain ⟨left₁, right⟩ := right
      obtain ⟨w₁, h⟩ := left₁
      subst h

```

```

obtain { c, hc } := right ( Pi.single j1 1 ) ;
obtain { d, hd } := right ( Pi.single j2 1 ) ;
simp_all only [funext_iff, Matrix.mulVec, Matrix.of_apply, Pi.zero_apply,
  Pi.smul_apply, smul_eq_mul, Matrix.mulVec_single, MulOpposite.op_one,
  Matrix.col_apply]
rw [ ← hc i1, ← hd i1, ← hc i2, ← hd i2 ] ; ring

/-- A pure state is separable iff the partial trace on the left is pure. -
/
theorem pure_separable_iff_traceLeft_pure (ψ : Ket (d1 × d2)) : IsSeparable
  ∃ ψ1, pure ψ1 = (pure ψ).traceLeft := by
  have h1 := MState.pure_separable_iff_IsProd ψ;
  have h2 := Ket.IsProd_iff_rank_eq_one ψ;
  have h3 := MState.pure_iff_rank_eq_one ( ( MState.pure ψ ).traceLeft )
  simp_all
  have h4 : Matrix.rank ((MState.pure ψ).traceLeft.m) = Matrix.rank (Matrix
    have h4 : (MState.pure ψ).traceLeft.m = Matrix.transpose (Matrix.conjTr
      ext i j
      simp [ MState.traceLeft, Matrix.mul_apply ] ;
      exact Finset.sum_congr rfl fun _ => mul_comm _ _;
      rw [ h4, Matrix.rank_transpose, Matrix.rank_conjTranspose_mul_self ];
      grind

--TODO: Separable states are convex

section purification

/-- The purification of a mixed state. Always uses the full dimension of the
  purify, so e.g. an existing pure state with d=4 still becomes d=16 in the
  property is `MState.traceRight_of_purify`; see also `MState.purify` for t
/
def purify (ρ : MState d) : Ket (d × d) where
  vec := fun (i,j) ↦
    let p2 := ρ.Hermitian.eigenvectorUnitary i j
    p2 * (ρ.Hermitian.eigenvalues j).sqrt
  normalized' := by
    have h1 := fun i ↦ ρ.psd.eigenvalues_nonneg i
    simp only [Complex.norm_mul,
      Complex.norm_real, Real.norm_eq_abs, mul_pow, sq_abs, h1, Real.sq_sqrt,
      Fintype.sum_prod_type_right]
    simp_rw [← Finset.sum_mul]
    have : ∀ x, ∑ i : d, ||ρ.Hermitian.eigenvectorUnitary i x|| ^ 2 = 1 :=
      Matrix.unitaryGroup_row_norm ρ.Hermitian.eigenvectorUnitary
    apply @RCLike.ofReal_injective ℂ
    simp_rw [this, one_mul, Matrix.IsHermitian.sum_eigenvalues_eq_trace]
    exact ρ.tr'

```



```

/-- The defining property of purification, that tracing out the purifying system gives
    original mixed state. -/
@[simp]
theorem purify_spec (ρ : MState d) : (pure ρ.purify).traceRight = ρ := by
  ext i j
  simp_rw [purify, traceRight, HermitianMat.traceRight, Matrix.traceRight]
  simp only [Matrix.IsHermitian.eigenvectorUnitary_apply, mat_M, pure_apply,
    mat_mk, Matrix.of_apply]
  simp only [Ket.apply]
  simp only [map_mul]
  simp_rw [mul_assoc, mul_comm, ← mul_assoc (Complex.ofReal _), Complex.mul_conj]
  -- By definition of eigenvectorUnitary and the properties of the unitary matrix and
  have h_eigenvectorUnitary : ∀ i j, ∑ x, ρ.Hermitian.eigenvectorUnitary i x * ((ρ.H
    intro i j
    have h_eigenvectorUnitary : ρ.M = Matrix.of (fun i j => ∑ x, ρ.Hermitian.eigenv
      have := ρ.Hermitian.spectral_theorem;
      convert this using 1;
      ext i j; simp [Matrix.mul_apply, Matrix.diagonal] ;
      replace h_eigenvectorUnitary := congr_fun (congr_fun h_eigenvectorUnitary i) j
      simp_all only [mat_apply, Matrix.IsHermitian.eigenvectorUnitary_apply, Matrix.of
        congr! 2;
        norm_num [Complex.ext_iff, sq];
        exact 0r.inl (Real.mul_self_sqrt (ρ.psd.eigenvalues_nonneg _))
      simp_all [Complex.normSq, sq];
      simp only [mul_assoc] using h_eigenvectorUnitary i j

/-- `MState.purify` bundled with its defining property `MState.traceRight_of_purify`
/
def purifyX (ρ : MState d) : { ψ : Ket (d × d) // (pure ψ).traceRight = ρ } :=
  {ρ.purify, ρ.purify_spec}

end purification

@[simps]
def relabel (ρ : MState d₁) (e : d₂ ≈ d₁) : MState d₂ where
  M := ρ.M.reindex e.symm
  nonneg := by simp [zero_le_iff, ρ.psd]
  tr := by simp [trace]

@[simp]
theorem relabel_m (ρ : MState d₁) (e : d₂ ≈ d₁) :
  (ρ.relabel e).m = ρ.m.submatrix e e := by
  rfl

@[simp]

```

```

theorem relabel_refl {d : Type*} [Fintype d] [DecidableEq d] (ρ : MState d)
  ρ.relabel (Equiv.refl d) = ρ := by
  ext
  simp

/-- Relabeling a pure state by a bijection yields another pure state. -
/
theorem relabel_pure_exists (ψ : Ket d₁) (e : d₂ ≈ d₁) :
  ∃ ψ' : Ket d₂, (pure ψ).relabel e = pure ψ' := by
  refine ⟨(fun i => ψ (e i), ?_), rfl⟩
  rw [← ψ.normalized', Fintype.sum_equiv e]
  congr!

@[simp]
theorem relabel_relabel {d d₂ d₃ : Type*}
  [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂] [Fintype d₃]
  (ρ : MState d) (e : d₂ ≈ d) (e₂ : d₃ ≈ d₂) : (ρ.relabel e).relabel e₂ =
  rfl

theorem eq_relabel_iff {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁] [Fint
  (ρ : MState d₁) (σ : MState d₂) (h : d₁ ≈ d₂) :
  ρ = σ.relabel h ↔ ρ.relabel h.symm = σ := by
  simp only [MState.ext_iff, HermitianMat.ext_iff, mat_M, relabel_m]
  exact ⟨(by simp[·]), (by simp[← ·])⟩

theorem relabel_comp {d₁ d₂ d₃ : Type*} [Fintype d₁] [DecidableEq d₁] [Fint
  [Fintype d₃] [DecidableEq d₃] (ρ : MState d₁) (e : d₂ ≈ d₁) (f : d₃ ≈
  (ρ.relabel e).relabel f = ρ.relabel (f.trans e) := by
  ext
  simp

theorem relabel_cast {d₁ d₂ : Type u} [Fintype d₁] [DecidableEq d₁]
  [Fintype d₂] [DecidableEq d₂]
  (ρ : MState d₁) (e : d₂ ≈ d₁) :
  ρ.relabel (Equiv.cast e) = cast (by have := e.symm; congr <=> (apply Su
  ext i j
  simp only [relabel_M, mat_reindex, mat_M, Matrix.reindex_apply, Matrix.su
  subst e
  congr!
  symm
  apply cast_heq

@[simp]
theorem spectrum_relabel {ρ : MState d} (e : d₂ ≈ d) :
  _root_.spectrum ℝ (ρ.relabel e).m = _root_.spectrum ℝ ρ.m := by
  ext1 v

```

```

rw [spectrum.mem_iff] --TODO make a plain `Matrix` version of this
rw [Algebra.algebraMap_eq_smul_one v]
rw [MState.relabel_m, ← Matrix.submatrix_one_equiv e]
rw [← Matrix.smul_apply, ← Matrix.submatrix_smul]
rw [← Matrix.sub_apply, ← Matrix.submatrix_sub]
rw [Matrix.isUnit_submatrix_equiv]
rw [← Algebra.algebraMap_eq_smul_one v, ← spectrum.mem_iff]

/-- The purity of a state is invariant under relabeling of the basis. -
/
@[simp]
theorem purity_relabel (ρ : MState d₁) (e : d₂ ≈ d₁) : (ρ.relabel e).purity = ρ.purity :=
  simp [purity, inner_def, -inner_self_eq_norm_sq_to_K]
--TODO: Swap and assoc for kets.
--TODO: Connect these to unitaries (when they can be)

/-- The heterogeneous SWAP gate that exchanges the left and right halves of a quantum state.
This can apply even when the two "halves" are of different types, as opposed to (swap)
gate on quantum circuits that leaves the qubit dimensions unchanged. Notably, it is
/
def SWAP (ρ : MState (d₁ × d₂)) : MState (d₂ × d₁) :=
  ρ.relabel (Equiv.prodComm d₁ d₂).symm

/--
The multiset of values in the spectrum of a relabeled state is the same as the multiset of
-/
lemma multiset_spectrum_relabel_eq {d₁ d₂ : Type*} [Fintype d₁] [DecidableEq d₁] [Fintype d₂]
  (ρ : MState d₁) (e : d₂ ≈ d₁) :
  Multiset.map (ρ.relabel e).spectrum Finset.univ.val = Multiset.map ρ.spectrum Finset.univ.val :=
  have h_charpoly : Matrix.charpoly (ρ.relabel e).m = Matrix.charpoly ρ.m := by
    exact Matrix.charpoly_reindex e.symm ρ.m
  have h_eigenvalues : Multiset.map (ρ.relabel e).M.H.eigenvalues Finset.univ.val =
    Multiset.map ρ.M.H.eigenvalues Finset.univ.val :=
    have h_eigenvalues : Polynomial.roots (Matrix.charpoly (ρ.relabel e).m) = Polynomial.roots (Matrix.charpoly ρ.m) :=
      rw [h_charpoly];
    have := ρ.M.H.roots_charpoly_eq_eigenvalues
    have := (ρ.relabel e).M.H.roots_charpoly_eq_eigenvalues
  simp_all only [relabel_m, mat_M, Complex.coe_algebraMap, Function.comp_apply, reindex,
    mat_reindex, Matrix.reindex_apply, Equiv.symm_symm]
  replace this := congr_arg ( fun m => m.map ( fun x => x.re ) ) this
  aesop;
  unfold MState.spectrum;
  ext
  rename_i a
  simp_all only [relabel_m, relabel_M, mat_reindex, mat_M, Matrix.reindex_apply, Equiv.symm_symm]
  obtain ⟨val, property⟩ := a
  obtain ⟨left, right⟩ := property

```

```

convert congr_arg ( fun m => Multiset.count val m ) h_eigenvalues using 1
· rw [ Multiset.count_map, Multiset.count_map ];
  simp [ Subtype.ext_iff ];
  congr! 2;
· erw [ Multiset.count_map, Multiset.count_map ];
  congr! 2;
  exact beq_eq_beq.mp rfl

def spectrum_SWAP (p : MState (d1 × d2)) : ∃ e, p.SWAP.spectrum.relabel e =
-- Apply the lemma exists_equiv_of_multiset_map_eq with the appropriate p
obtain ⟨w, h⟩ := exists_equiv_of_multiset_map_eq (fun p => p.spectrum p)
  (p.multiset_spectrum_relabel_eq (Equiv.prodComm _ _).symm ▸ rfl)
use w
ext x
simp_rw [h]
rfl

@[simp]
theorem SWAP_SWAP (p : MState (d1 × d2)) : p.SWAP.SWAP = p :=
  rfl

@[simp]
theorem traceLeft_SWAP (p : MState (d1 × d2)) : p.SWAP.traceLeft = p.traceR
  rfl

@[simp]
theorem traceRight_SWAP (p : MState (d1 × d2)) : p.SWAP.traceRight = p.traceL
  rfl

/-- The associator that re-clusters the parts of a quantum system. -
/
def assoc (p : MState ((d1 × d2) × d3)) : MState (d1 × d2 × d3) :=
  p.relabel (Equiv.prodAssoc d1 d2 d3).symm

/-- The associator that re-clusters the parts of a quantum system. -
/
def assoc' (p : MState (d1 × d2 × d3)) : MState ((d1 × d2) × d3) :=
  p.SWAP.assoc.SWAP.assoc.SWAP

@[simp]
theorem assoc_assoc' (p : MState (d1 × d2 × d3)) : p.assoc'.assoc = p := by
  rfl

@[simp]
theorem assoc'_assoc (p : MState ((d1 × d2) × d3)) : p.assoc.assoc' = p :=
  rfl

```

```

@[simp]
theorem traceLeft_right_assoc (p : MState ((d1 × d2) × d3)) :
  p.assoc.traceLeft.traceRight = p.traceRight.traceLeft := by
  ext
  simp [assoc, relabel, Matrix.traceLeft, traceLeft, Matrix.traceRight, traceRight]
  using Finset.sum_comm

@[simp]
theorem traceRight_left_assoc' (p : MState (d1 × d2 × d3)) :
  p.assoc'.traceRight.traceLeft = p.traceLeft.traceRight := by
  rw [← p.assoc'.traceLeft_right_assoc, assoc_assoc']

@[simp]
theorem traceRight_assoc (p : MState ((d1 × d2) × d3)) :
  p.assoc.traceRight = p.traceRight.traceRight := by
  ext : 3
  apply Finset.sum_product

@[simp]
theorem traceLeft_assoc' (p : MState (d1 × d2 × d3)) :
  p.assoc'.traceLeft = p.traceLeft.traceLeft := by
  convert p.SWAP.assoc.SWAP.traceRight_assoc
  simp

@[simp]
theorem traceLeft_left_assoc (p : MState ((d1 × d2) × d3)) :
  p.assoc.traceLeft.traceLeft = p.traceLeft := by
  simp [← traceLeft_assoc']

@[simp]
theorem traceRight_right_assoc' (p : MState (d1 × d2 × d3)) :
  p.assoc'.traceRight.traceRight = p.traceRight := by
  simp [assoc']

@[simp]
theorem traceNorm_eq_1 (p : MState d) : p.m.traceNorm = 1 :=
  have := calc (p.m.traceNorm : ℂ)
    _ = p.m.trace := p.psd.traceNorm_PSD_eq_trace
    _ = 1 := p.tr'
  Complex.ofReal_eq_one.mp this

--TODO: This naming is very inconsistent. Should be better about "prod" vs "kron"

theorem relabel_kron (p : MState d1) (σ : MState d2) (e : d3 ≈ d1) :
  ((p.relabel e) ⊗M σ) = (p ⊗M σ).relabel (e.prodCongr (Equiv.refl d2)) := by

```

```

    rfl --is this defeq abuse? I don't know

theorem kron_relabel (ρ : MState d1) (σ : MState d2) (e : d3 ≈ d2) :
  (ρ ⊗M σ).relabel e = (ρ ⊗M σ).relabel ((Equiv.refl d1).prodCongr e) :=
  rfl

theorem prod_assoc (ρ : MState d1) (σ : MState d2) (τ : MState d3) :
  (ρ ⊗M (σ ⊗M τ)) = (ρ ⊗M σ ⊗M τ).relabel (Equiv.prodAssoc d1 d2 d3).symm
  ext : 2
  simp [-Matrix.kronecker_assoc']
  exact (Matrix.kronecker_assoc' ρ.m σ.m τ.m).symm

section topology

/-- Mixed states inherit the subspace topology from matrices -
/
instance : TopologicalSpace (MState d) :=
  TopologicalSpace.induced MState.M inferInstance

/-- The projection from mixed states to their Hermitian matrices is an embed
/
theorem toMat_IsEmbedding : Topology.IsEmbedding (MState.M (d := d)) where
  eq_induced := rfl
  injective := @MState.ext _ _ _

instance : T3Space (MState d) :=
  Topology.IsEmbedding.t3Space toMat_IsEmbedding

instance : CompactSpace (MState d) := by
  constructor
  rw [(Topology.IsInducing.induced MState.M).isCompact_iff]
  suffices IsCompact (Set.Icc 0 1 ∩ { m | m.trace = 1 } : Set (HermitianMat
    convert this
    ext1 m
    constructor
    · rintro ⟨ρ, _, rfl⟩
      simp [ρ.nonneg, ρ.le_one]
    · simpa using fun m_pos _ m_tr => ⟨⟨m, m_pos, m_tr⟩, rfl⟩
  apply isCompact_Icc.inter_right
  refine isClosed_eq ?_ continuous_const
  rw [funext trace_eq_re_trace]
  fun_prop

noncomputable instance : MetricSpace (MState d) :=
  MetricSpace.induced MState.M MState.M_Injective inferInstance

```

```
theorem dist_eq (x y : MState d) : dist x y = dist x.M y.M := by
  rfl
```

```
instance : BoundedSpace (MState d) where
  bounded_univ :=
    CompactSpace.isCompact_univ.isBounded
```

```
@[fun_prop]
theorem Continuous_HermitianMat : Continuous (MState.M (d := d)) :=
  continuous_iff_le_induced.mpr fun _ => id
```

```
@[fun_prop]
theorem Continuous_Matrix : Continuous (MState.m (d := d)) := by
  unfold MState.m
  fun_prop
```

```
theorem image_M_isBounded (S : Set (MState d)) : Bornology.IsBounded (MState.M '' S)
  rw [← Bornology.isBounded_induced]
  exact Bornology.IsBounded.all S
```

```
end topology
```

```
section finprod
```

```
variable {ι : Type u} [DecidableEq ι] [fι : Fintype ι]
variable {dI : ι → Type v} [∀(i : ι), Fintype (dI i)] [∀(i : ι), DecidableEq (dI i)]
```

```
def piProd (pi : (i : ι) → MState (dI i)) : MState ((i : ι) → dI i) where
  M := {
    val := Matrix.piProd (fun i ↦ (pi i).m)
    property := Matrix.IsHermitian.piProd (fun i ↦ (pi i).Hermitian)
  }
  nonneg := by
    rw [zero_le_iff]
    exact Matrix.PosSemidef.piProd (fun i => psd (pi i))
  tr := by simp [trace, Matrix.trace_piProd]
```

```
/-- The n-copy "power" of a mixed state, with the standard basis indexed by pi types
/
```

```
def npow (p : MState d) (n : ℕ) : MState (Fin n → d) :=
  piProd (fun _ ↦ p)
```

```
@[inherit_doc]
infixl:110 " ⊗^ " => MState.npow
```

```
end finprod
```

section posdef

```

theorem PosDef.kron {d1 d2 : Type*} [Fintype d1] [DecidableEq d1] [Fintype
  {σ1 : MState d1} {σ2 : MState d2} (hσ1 : σ1.m.PosDef) (hσ2 : σ2.m.PosDe
  hσ1.kron hσ2

theorem PosDef.relabel {d1 d2 : Type*} [Fintype d1] [DecidableEq d1] [Finty
  {ρ : MState d1} (hp : ρ.m.PosDef) (e : d2 ≈ d1) : (ρ.relabel e).m.PosDe
  Matrix.PosDef.reindex hp e.symm

/-- If both states positive definite, so is their mixture. -
/
theorem PosDef_mix {d : Type*} [Fintype d] [DecidableEq d] {σ1 σ2 : MState
  (hσ1 : σ1.m.PosDef) (hσ2 : σ2.m.PosDef) (p : Prob) : (p [σ1 ↔ σ2]).m.Po
  Matrix.PosDef.Convex hσ1 hσ2 p.zero_le (1 - p).zero_le (by simp)

/-- If one state is positive definite and the mixture is nondegenerate, the
/
theorem PosDef_mix_of_ne_zero {d : Type*} [Fintype d] [DecidableEq d] {σ1 σ
  (hσ1 : σ1.m.PosDef) (p : Prob) (hp : p ≠ 0) : (p [σ1 ↔ σ2]).m.PosDef :=
  rw [← zero_lt_iff] at hp
  exact (hσ1.smul hp).add_posSemidef (σ2.psd.rsmul (1 - p).zero_le)

/-- If the second state is positive definite and the mixture is nondegenera
/
theorem PosDef_mix_of_ne_one {d : Type*} [Fintype d] [DecidableEq d] {σ1 σ2
  (hσ2 : σ2.m.PosDef) (p : Prob) (hp : p ≠ 1) : (p [σ1 ↔ σ2]).m.PosDef :=
  have : 0 < 1 - p := by
    --TODO this is ridiculous, move to Prob
    contrapose! hp
  have : (1 : ℝ) - (p : ℝ) = (0 : ℝ) := by
    have := le_antisymm hp (1 - p).zero_le
    rw [Subtype.ext_iff] at this
    simpa using this
  ext
  change (p : ℝ) = 1
  linarith
  exact (hσ2.smul this).posSemidef_add (σ1.psd.rsmul p.zero_le)

theorem uniform_posDef {d : Type*} [Nonempty d] [Fintype d] [DecidableEq d]
  (uniform (d := d)).m.PosDef := by
  simp [uniform, ofClassical, m, HermitianMat.diagonal]
  exact Fintype.card_pos

theorem posDef_of_unique {d : Type*} [Fintype d] [DecidableEq d] (ρ : MStat

```



```

    rw [Subsingleton.allEq p uniform]
    exact uniform_posDef

end posdef

end MState

```

1.443 QuantumInfo/Finite/POVM.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.CPTPMap

/-! # Positive Operator-Valued Measures

A Positive Operator-Valued Measures, or POVM, is the most general notion of a quantum
measurement: it is a collection of positive semidefinite (PSD) operators that sum to the identity. These
operators, called `POVM.measure`, of measurement outcomes; and they induce a CPTP map, `POVM.measuremap`,
which adds learned information.

Developing this theory is important if one wants to discuss classical information and
quantum information. The route to get back to classical information (a `ProbDistribution` of outcomes)
is through the CPTP map.

TODO: They can also evolve under CPTP maps themselves (the Heisenberg picture of quantum
information). If they do, with each other or not, they might be projective or not.
-/
noncomputable section
open BigOperators
open ComplexOrder
open Matrix
open scoped RealInnerProductSpace

/-- A POVM is a (finite) collection of PSD matrices on the same Hilbert space
that sum to the identity. Here `X` indexes the matrices, and `d` is the space
dimension.

Applied to an `MState` on that on that space with
`POVM.measure`, this produces a distribution of outcomes indexed by the same
type as the collection.

This measurement action can be composed with `MState.of_classical`, in which

```

```

case it is equal to a CPTP map `measurement_map`. -/
structure POVM (X : Type*) (d : Type*) [Fintype X] [Fintype d] [DecidableEq d]
  mats : X → HermitianMat d ℂ
  nonneg : ∀ x, 0 ≤ (mats x)
  normalized : ∑ x, mats x = 1

namespace POVM

variable {X : Type*} {d : Type*} [Fintype X] [Fintype d] [DecidableEq d] [D

/-- The act of measuring is a quantum channel, that maps a `d`-
dimensional quantum
state to an `d × X`-dimensional quantum-classical state. -
/
def measurementMap (Λ : POVM X d) : CPTPMap d (d × X) where
  toLinearMap :=
    ∑ (x : X), open Kronecker in {
      toFun := fun ρ ↦ (((Λ.mats x) ^ (1/2:ℝ)).mat * ρ * ((Λ.mats x)^(1/2:
      map_add' := by simp [mul_add, add_mul, Matrix.kroneckerMap_add_left]
      map_smul' := by simp [Matrix.smul_kronecker]
    }
  cp := by
    apply Finset.sum_induction
    · exact fun _ _ ha ↦ ha.add
    · exact MatrixMap.IsCompletelyPositive.zero _ _
    · intro x _
      --Note: this map M1 would do as well as an object on its own, it's "m
      let M1 : MatrixMap d d ℂ := {
        fun ρ ↦ ((Λ.mats x) ^ (1/2:ℝ)).mat * ρ * ((Λ.mats x)^(1/2:ℝ)).mat,
        by simp [mul_add, add_mul]},
        by simp
      let M2 : MatrixMap d (d × X) ℂ := {
        fun ρ ↦ (ρ.kronecker (Matrix.single x x 1)),
        by simp [add_mul, Matrix.kroneckerMap_add_left]},
        by simp [Matrix.smul_kronecker]
      set M3 := LinearMap.comp M2 M1 with hM3
      simp only [M1, M2, LinearMap.comp, kronecker, LinearMap.coe_mk, AddHo
      unfold Function.comp at hM3
      rw [← hM3]
      apply MatrixMap.IsCompletelyPositive.comp
      · dsimp [M1]
        conv =>
          enter [1, 1, 1, ρ, 2]
          rw [← HermitianMat.conjTranspose_mat]
          exact MatrixMap.conj_isCompletelyPositive (Λ.mats x ^ (1 / 2)).mat
      · apply MatrixMap.kron_kronecker_const

```

```

    exact (Matrix.PosSemidef.stdBasisMatrix_iff_eq x x (zero_lt_one' ℂ)).2 rfl
TP := by
  intro x
  rw [LinearMap.sum_apply, trace_sum]
  dsimp
  simp only [Matrix.trace_kronecker, Matrix.trace_mul_cycle (B := x),
    Matrix.trace_single_eq_same, mul_one]
  rw [← trace_sum, ← Finset.sum_mul]
  congr
  convert one_mul x
  rw [show (1 : Matrix d d ℂ) = (1 : HermitianMat d ℂ).mat by rfl, ← Λ.normalized]
  rw [HermitianMat.mat_finset_sum]
  congr! with i _
  exact HermitianMat.pow_half_mul (Λ.nonneg i)

open Kronecker in
theorem measurementMap_apply_matrix (Λ : POVM X d) (m : Matrix d d ℂ) :
  Λ.measurementMap.map m = ∑ x : X,
    (((Λ.mats x) ^ (1/2:ℝ)).mat * m * ((Λ.mats x)^(1/2:ℝ)).mat) ⊗k Matrix.single x
  dsimp [measurementMap, HPMat.map]
  rw [LinearMap.sum_apply]
  rfl

open HermitianMat in
theorem measurementMap_apply_hermitianMat (Λ : POVM X d) (m : HermitianMat d ℂ) :
  Λ.measurementMap.toHPMap m = ∑ x : X,
    --TODO: Something like `HermitianMat.single` to make this better
    ((m.conj ((Λ.mats x)^(1/2:ℝ)).mat : HermitianMat d ℂ) ⊗k HermitianMat.diagonal ℂ)
  ext1
  convert Λ.measurementMap_apply_matrix m.mat
  simp only [conj_apply, conjTranspose_mat, HermitianMat.mat_finset_sum,
    kronecker_mat, mat_mk]
  congr!
  ext i j
  simp only [HermitianMat.diagonal, mat_mk, diagonal_apply, single, of_apply]
  split_ifs <=> grind only [= map_zero, = map_one]

/-- A POVM leads to a distribution of outcomes on any given mixed state p. -
/
def measure (Λ : POVM X d) (p : MState d) : ProbDistribution X := .mk'
  (f := fun x => ∑ Λ.mats x, p.M)
  (h₁ := fun x => HermitianMat.inner_ge_zero (Λ.nonneg x) p.nonneg)
  (hN := by
    simp [HermitianMat.inner_eq_re_trace, ← Complex.re_sum, ← trace_sum, ← Finset.
      ← HermitianMat.mat_finset_sum, Λ.normalized])

```

```

/-- The quantum-classical `POVM.measurement_map`, gives a marginal on the r
/
theorem traceLeft_measurementMap_eq_measure (Λ : POVM X d) (ρ : MState d) :
  (Λ.measurementMap ρ).traceLeft = MState.ofClassical (Λ.measure ρ) := by
  open Kronecker in
  ext i j
  rcases ρ with ⟨⟨ρ, ρH⟩, hp0, hp1⟩
  change (Matrix.traceLeft (Λ.measurementMap.map ρ)) i j = _
  rw [measurementMap_apply_matrix]
  --TODO: a lemma for Matrix.traceLeft (Σ x, _) = Σ x, (Matrix.traceLeft _)
  simp_rw [Matrix.traceLeft, Matrix.of_apply, Matrix.sum_apply]
  rw [Finset.sum_comm]
  simp only [kroneckerMap_apply, MState.coe_ofClassical]
  simp only [single, of_apply, mul_ite, mul_one, mul_zero, Finset.sum_ite_i
    Finset.sum_const_zero]
  simp only [HermitianMat.diagonal, HermitianMat.mat_mk, diagonal_apply]
  symm; split
  · subst j
    simp only [measure, ProbDistribution.mk', ProbDistribution.funlike_appl
      Finset.mem_univ, ↓reduceIte]
    change _ = Matrix.trace
    rw [Matrix.trace_mul_cycle, HermitianMat.pow_half_mul (Λ.nonneg i)]
    exact HermitianMat.inner_eq_trace_rc _
  · conv => enter [2, 2, x]; rw [if_neg (by grind)]
    simp

/-- The action of measuring a state with the POVM `Λ`, discarding the resul
the mixed state recording the outcome. This resulting state is purely diagon
`POVM.measureDiscard_apply`. -/
noncomputable def measureDiscard (Λ : POVM X d) : CPTPMap d X :=
  CPTPMap.traceLeft ∘m Λ.measurementMap

theorem measureDiscard_apply (Λ : POVM X d) (ρ : MState d) :
  Λ.measureDiscard ρ = MState.ofClassical (Λ.measure ρ) := by
  simp [measureDiscard, traceLeft_measurementMap_eq_measure]

/-- The action of measuring a state with the POVM `Λ`, forgetting the measu
keeping the disturbed state. -/
noncomputable def measureForget (Λ : POVM X d) : CPTPMap d d :=
  CPTPMap.traceRight ∘m Λ.measurementMap

proof_wanted measureForget_eq_kraus (Λ : POVM X d) :
  Λ.measureForget = CPTPMap.of_kraus_CPTPMap (fun i ↦ (Λ.mats i) ^ (1/2 :
    simp [-one_div, fun x ↦ HermitianMat.pow_half_mul (Λ.nonneg x), Herm
    using Λ.normalized
  )

```

end POVM

1.444 QuantumInfo/Finite/Pinching.lean

```

/-
Copyright (c) 2025 Leonardo A Lessa. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Leonardo A Lessa, Alex Meiburg
-/
import QuantumInfo.Finite.CPTMap
import QuantumInfo.Finite.MState
import QuantumInfo.Finite.Entropy
import QuantumInfo.ForMathlib.HermitianMat.CFC

/-! # Pinching channels
A pinching channel decoheres in the eigenspaces of a given state.
More precisely, given a state  $\rho$ , the pinching channel with respect to  $\rho$  is defined as

$$E(\sigma) = \sum_i P_i \sigma P_i$$

where the  $P_i$  are the projectors onto the  $i$ -th eigenspaces of  $\rho = \sum_i p_i P_i$ , with  $i \neq$ 
TODO: Generalize to pinching with respect to arbitrary P(0)VM.
-/

noncomputable section
open scoped Matrix RealInnerProductSpace

variable {d : Type*} [Fintype d] [DecidableEq d]

def pinching_kraus ( $\rho$  : MState d) : spectrum  $\mathbb{R}$   $\rho.m$   $\rightarrow$  HermitianMat d  $\mathbb{C}$  :=
  fun x  $\mapsto$   $\rho.M.cfc$  (fun y  $\mapsto$  if y = x then 1 else 0)

theorem pinching_kraus_commutates ( $\rho$  : MState d) (i : spectrum  $\mathbb{R}$   $\rho.m$ ) :
  Commute (pinching_kraus  $\rho$  i).mat  $\rho.m$  := by
  rw [MState.m,  $\leftarrow$   $\rho.M.cfc\_id$ , commute_iff_eq, pinching_kraus]
  rw [ $\leftarrow$   $\rho.M.mat\_cfc\_mul$ ,  $\leftarrow$   $\rho.M.mat\_cfc\_mul$ ]
  congr 2; ext; simp

theorem pinching_kraus_mul_self ( $\rho$  : MState d) (i : spectrum  $\mathbb{R}$   $\rho.m$ ) :
  (pinching_kraus  $\rho$  i).mat *  $\rho.m$  = i.val • pinching_kraus  $\rho$  i := by
  dsimp [MState.m]
  nth_rw 1 [ $\leftarrow$   $\rho.M.cfc\_id$ ]
  rw [pinching_kraus]
  rw [ $\leftarrow$   $\rho.M.mat\_cfc\_mul$ ,  $\leftarrow$  HermitianMat.mat_smul]

```

```

    rw [← p.M.cfc_const_mul]
    congr! 3
    simp +contextual

instance finite_spectrum_inst (p : MState d) : Fintype (spectrum ℝ p.m) :=
  Fintype.ofFinite (spectrum ℝ p.m)

theorem pinching_kraus_orthogonal (p : MState d) {i j : spectrum ℝ p.m} (h
  (pinching_kraus p i).mat * (pinching_kraus p j).mat = 0 := by
  convert (HermitianMat.mat_cfc_mul p.M _ _).symm
  congr ($((p.M.cfc_const 0).symm).mat)
  · simp
  · grind [Pi.mul_apply]

/-- The Kraus operators of the pinching channel are projectors: they square
/
@[simp]
theorem pinching_sq_eq_self (p : MState d) (k) : (pinching_kraus p k) ^ 2
  ext1
  push_cast
  rw [pow_two, pinching_kraus, ← p.M.mat_cfc_mul]
  congr! 3
  simp

/-- The Kraus operators of the pinching channel are orthogonal projectors.
/
theorem pinching_kraus_ortho (p : MState d) (i j : spectrum ℝ p.m) :
  (pinching_kraus p i).mat * (pinching_kraus p j).mat = if i = j then (p
  split_ifs with hij
  · grind [sq, HermitianMat.mat_pow, pinching_sq_eq_self]
  · exact pinching_kraus_orthogonal p hij

theorem pinching_sum (p : MState d) :  $\sum k$ , pinching_kraus p k = 1 := by
  ext i j
  simp only [pinching_kraus, HermitianMat.cfc]
  have heq : Set.EqOn (fun x =>  $\sum i$  : spectrum ℝ p.m, if x = ↑i then (1 : ℝ)
  intro x hx
  dsimp
  rw [Finset.sum_set_coe (f := fun i => if x = i then 1 else 0), Finset.s
  rwa [Set.mem_toFinset]
  simp [-HermitianMat.mat_apply]
  rw [← cfc_sum, Finset.sum_fn, cfc_congr heq, cfc_one (R := ℝ) (ha := _)]
  rw [IsSelfAdjoint, Matrix.star_eq_conjTranspose, p.Hermitian]

def pinching_map (p : MState d) : CPTPMap d d ℂ :=
  CPTPMap.of_kraus_CPTPMap (HermitianMat.mat ∘ pinching_kraus p) (by

```

```

conv =>
  enter [1, 2, k]
  rw [Function.comp_apply, (pinching_kraus ρ k).H, ←pow_two]
  norm_cast
  rw [pinching_sq_eq_self ρ k]
  norm_cast
  simp [pinching_sum]
)

theorem pinchingMap_apply_M (σ ρ : MState d) : (pinching_map σ ρ).M =
  (_, (MatrixMap.of_kraus_isCompletelyPositive
    (HermitianMat.mat ◦ pinching_kraus σ)).IsPositive.IsHermitianPreserving ρ.M.H) :
  rfl

theorem pinching_eq_sum_conj (σ ρ : MState d) : (pinching_map σ ρ).M =
  ∑ k, (pinching_kraus σ k).mat * ρ.M * (pinching_kraus σ k).mat := by
  rw [pinchingMap_apply_M]
  simp [MatrixMap.of_kraus, Matrix.mul_assoc]

theorem pinching_commutes_kraus (σ ρ : MState d) (i : spectrum ℝ σ.m) :
  Commute (pinching_map σ ρ).m (pinching_kraus σ i).mat := by
  have h_expand := pinching_eq_sum_conj σ ρ
  simp only [MState.mat_M] at h_expand
  simp only [Commute, h_expand];
  simp only [SemiconjBy, Finset.sum_mul]
  simp only [mul_assoc, Finset.mul_sum]
  congr! 1 with x
  by_cases h : x = i <=> simp [ h, ← mul_assoc, pinching_kraus_ortho ];
  grind

theorem pinching_commutes (ρ σ : MState d) :
  Commute (pinching_map σ ρ).m σ.m := by
  dsimp [MState.m, Commute, SemiconjBy]
  rw [pinchingMap_apply_M]
  simp only [MatrixMap.of_kraus, Function.comp_apply]
  simp only [HermitianMat.conjTranspose_mat, MState.mat_M, LinearMap.coe_sum,
    LinearMap.coe_mk, AddHom.coe_mk, Finset.sum_apply]
  simp only [HermitianMat.mat_mk, Finset.sum_mul, Finset.mul_sum]
  congr! 1 with i
  have hl : (pinching_kraus σ i) * σ.m = i.val • (pinching_kraus σ i) :=
    pinching_kraus_mul_self σ i
  have hr : σ.m * (pinching_kraus σ i) = i.val • (pinching_kraus σ i) := by
    rwa [pinching_kraus_commutes] at hl
  simp only [mul_assoc, hl]
  simp only [← mul_assoc, hr]
  simp

```

```

@[simp]
theorem pinching_self (ρ : MState d) : pinching_map ρ ρ = ρ := by
  ext1
  rw [pinchingMap_apply_M]
  simp only [MatrixMap.of_kraus, Function.comp_apply]
  simp only [HermitianMat.conjTranspose_mat, MState.mat_M, LinearMap.coe_sum,
    LinearMap.coe_mk, AddHom.coe_mk, Finset.sum_apply]
  simp_rw [(pinching_kraus_commutes ρ _).eq, mul_assoc, ← sq]
  conv_lhs =>
    enter [1, 2, x, 2]
    change (pinching_kraus ρ x ^ 2).mat
    rw [pinching_sq_eq_self]
  simp_rw [← Finset.mul_sum, ← HermitianMat.mat_finset_sum]
  simp only [pinching_sum, HermitianMat.mat_one, mul_one]
  rfl

/-- Lemma 3.10 of Hayashi's book "Quantum Information Theory - Mathematical
Also, Lemma 5 in https://arxiv.org/pdf/quant-ph/0107004.
-- Used in (S60) -/
theorem pinching_bound (ρ σ : MState d) : ρ.M ≤ (↑(Fintype.card (spectrum ℝ
  rw [pinchingMap_apply_M]
  suffices ρ.M ≤ (Fintype.card (spectrum ℝ σ.m) : ℝ) • ∑ c, ρ.M.conj (pinch
    convert this
    ext1
    simp [MatrixMap.of_kraus, HermitianMat.conj]
  --Rewrite ρ as its spectral decomposition
  obtain ⟨ψs, hpm⟩ := ρ.spectralDecomposition
  simp only [hpm, map_sum, Finset.smul_sum]
  rw [Finset.sum_comm (α := d)]
  gcogr with i _
  --
  open ComplexOrder in
  rw [HermitianMat.le_iff_mulVec_le_mulVec]
  intro v
  simp [← Finset.smul_sum, smul_comm _ (ρ.spectrum i : ℝ), Matrix.smul_mulV
  gcogr
  · --The spectrum of a MState is always positive.
    --Should be a helper lemma, it's only like two lines though.
    --*Could* be a positivity extension.
    dsimp [MState.spectrum, ProbDistribution.mk']
    rw [Complex.zero_le_real]
    exact (HermitianMat.zero_le_iff.mp ρ.nonneg).eigenvalues_nonneg _
  have h1 : (1 : Matrix d d ℂ) = (1 : HermitianMat d ℂ) := by exact selfAdj
  conv_lhs =>
    enter [2, 1]

```



```

    rw [← one_mul (MState.m _), h1, ← congr(HermitianMat.mat $(pinching_sum σ))]
    enter [2]
    rw [← mul_one (MState.m _), h1, ← congr(HermitianMat.mat $(pinching_sum σ))]
    simp only [HermitianMat.mat_finset_sum]
    simp only [Matrix.mul_sum, Matrix.sum_mul, Matrix.sum_mulVec, dotProduct_sum]
    simp only [MState.pure]
    dsimp [MState.m]
    --This out to be Cauchy-Schwarz.
    have hschwarz := inner_mul_inner_self_le (⊆ := ℂ) (E := EuclideanSpace ℂ (↑(spectrum
      (x := .toLp 2 fun i ↦ 1) (y := .toLp 2 fun k ↦ (
        star v ⊆_v ((pinching_kraus σ k).mat *v (ψs i))
      ))
    ))
    rw [← Complex.real_le_real] at hschwarz
    simp only [PiLp.inner_apply] at hschwarz
    simp only [RCLike.inner_apply, map_one, mul_one, one_mul, Complex.ofReal_mul, Finset
      Finset.card_univ, nsmul_eq_mul, RCLike.natCast_re, map_sum, RCLike.re_to_complex
      Complex.ofReal_natCast, Complex.ofReal_sum] at hschwarz
    simp only [HermitianMat.mat_mk] at ⊢
    have h_mul (x y : spectrum ℝ σ.m) :
      star v ⊆_v ((pinching_kraus σ y).mat *
        (Matrix.vecMulVec ↑(ψs i) ((ψs i).to_bra) : Matrix d d ℂ)
        * (pinching_kraus σ x).mat) *v v =
      star v ⊆_v (pinching_kraus σ y).mat *v (ψs i)
        * (starRingEnd ℂ) (star v ⊆_v (pinching_kraus σ x).mat *v (ψs i)) := by
    rw [← Matrix.mulVec_mulVec, ← Matrix.mulVec_mulVec]
    rw [Matrix.vecMulVec_mulVec, op_smul_eq_smul]
    rw [Matrix.mulVec_smul, dotProduct_smul, smul_eq_mul, mul_comm]
    congr
    rw [starRingEnd_apply, ← Matrix.star_dotProduct, Matrix.star_mulVec]
    rw [← Matrix.dotProduct_mulVec, HermitianMat.conjTranspose_mat]
    --Uses the defeq of `star (_ : Bra)` and `(_ : Ket)`. Would be good to have a lemma
    -- so that we don't 'abuse' this defeq, TODO. (Maybe even make the coercions between
    -- kets and bras irreducible?)
    rfl
    convert hschwarz withx <=> clear hschwarz
    · rw [← map_sum, ← Complex.ofReal_mul, ← norm_mul]
      rw [Complex.mul_conj, Complex.norm_real, Real.norm_of_nonneg (Complex.normSq_nonneg)]
      simp_rw [← Complex.mul_conj, map_sum, Finset.mul_sum, Finset.sum_mul]
      congr! with x _ y _
      rw [← Matrix.mul_assoc]
      exact h_mul x y
    · have hc (c d : ℂ) : d = starRingEnd ℂ d → c = d → c = d.re := by
        rintro h rfl; simp [Complex.ext_iff] at h ⊢; linarith
      apply hc <=> clear hc
      · simpa using mul_comm _ _
      · exact h_mul x x

```

```

open ComplexOrder in
theorem ker_le_ker_pinching_of_PosDef (ρ σ : MState d) (hpos : σ.m.PosDef)
  have h_ker : σ.M.ker = ⊥ := by
    have := hpos.toLin_ker_eq_bot
    simp [LinearMap.ker_eq_bot', HermitianMat.ker] at this ⊢
    intro m hm
    simp only [WithLp.ofLp_eq_zero] using this m congr($hm)
  rw [h_ker]
  exact bot_le

theorem pinching_idempotent (ρ σ : MState d) :
  (pinching_map σ) (pinching_map σ ρ) = (pinching_map σ ρ) := by
  rw [MState.ext_iff]
  have h_idempotent : (∑ k, (pinching_kraus σ k).mat * (∑ l, (pinching_kraus
    simp only [Matrix.mul_sum, Matrix.sum_mul, ← mul_assoc, pinching_kraus_
    simp [mul_assoc, pinching_kraus_ortho]
  ext1
  grind [pinching_eq_sum_conj]

theorem inner_cfc_pinching (ρ σ : MState d) (f : ℝ → ℝ) :
  []ρ.M, (pinching_map σ ρ).M.cfc f[] = [](pinching_map σ ρ).M, (pinching_ma
nth_rw 2 [pinchingMap_apply_M]
rw [HermitianMat.inner_eq_re_trace, HermitianMat.inner_eq_re_trace]
congr 1
simp only [MState.mat_M, HermitianMat.mat_mk]
conv_rhs =>
  rw [MatrixMap.of_kraus, LinearMap.sum_apply, Finset.sum_mul]
  rw [Matrix.trace_sum]
  simp only [Function.comp_apply, HermitianMat.conjTranspose_mat, LinearM
    AddHom.coe_mk]
  enter [2, x]
  rw [mul_assoc, ← Matrix.trace_mul_cycle, mul_assoc]
conv_rhs =>
  rw [← Matrix.trace_sum, ← Finset.mul_sum]
  enter [1, 2, 2, x]
  rw [(pinching_commutes_kraus σ ρ x).symm.cfc_right]
  rw [mul_assoc, ← sq]
  change _ * (pinching_kraus σ x ^ 2).mat
  rw [pinching_sq_eq_self σ x]
congr 2
rw [← Finset.mul_sum]
convert (mul_one _).symm
convert congr($(pinching_sum σ).mat)
simp

```



```

    exact Submodule.sum_mem _ fun k _ ↦ by simpa [HermitianMat.ker_conj p.non
noncomputable section AristotleLemmas

/-
If  $v$  is in the kernel of  $\sigma$ , then for any non-zero eigenvalue  $k$ , the project
eigenspace is  $\emptyset$ .
-/
theorem pinching_kraus_ker_of_ne_zero {d : Type*} [Fintype d] [DecidableEq
  (σ : MState d) (v : d → ℂ) (hv : σ.m.mulVec v = 0)
  (k : spectrum ℝ σ.m) (hk : k.val ≠ 0) :
  (pinching_kraus σ k).mat *v v = 0 := by
  -- Applying the equation  $(\text{pinching\_kraus } \sigma k).toMat * \sigma.m = k.$ 
  have h_eq_zero : ((pinching_kraus σ k).mat * σ.m) *v v = k.val • ((pinchi
  have h_eq_zero : ((pinching_kraus σ k).mat * σ.m) = k.val • (pinching_k
    convert pinching_kraus_mul_self σ k using 1;
  simp [ h_eq_zero ];
  ext i
  simp [ Matrix.mulVec, dotProduct ] ;
  simp only [ mul_assoc, Finset.mul_sum ];
  simp_all [ ← Matrix.mulVec_mulVec ];
  rw [ eq_comm ] at h_eq_zero
  aesop

end AristotleLemmas

set_option maxHeartbeats 2000000 in
theorem ker_le_ker_pinching_map_ker (ρ σ : MState d) (h : σ.M.ker ≤ ρ.M.ker
  σ.M.ker ≤ (pinching_map σ ρ).M.ker := by
  --TODO Cleanup
  intro v hv;
  -- Since  $v \in \ker \sigma$ , we have  $P_k v = 0$  for all  $k$  where the e
zero.
  have h_proj_zero : ∀ k : spectrum ℝ σ.m, k.val ≠ 0 → (pinching_kraus σ k)
    intro k hk
    exact pinching_kraus_ker_of_ne_zero σ v congr ($hv) k hk
  -- Since  $v \in \ker \sigma$ , we have  $P_k v = v$  for all  $k$  where the e
  have h_proj_one : ∀ k : spectrum ℝ σ.m, k.val = 0 → (pinching_kraus σ k).
    intro k hk
    have := pinching_kraus_mul_self σ k
    simp_all only [ ne_eq, Subtype.forall, zero_smul ]
    -- Since  $v$  is in the kernel of  $\sigma$ , we have  $\sum_i P_i v = v$ 
  have h_sum_proj : ∑ i : spectrum ℝ σ.m, (pinching_kraus σ i).mat *v v =
    have h_sum_proj : ∑ i : spectrum ℝ σ.m, (pinching_kraus σ i).mat = 1
    convert pinching_sum σ;
    simp [ ← Matrix.ext_iff, HermitianMat.ext_iff ];

```

```

    convert congr_arg ( fun m => m *v v ) h_sum_proj using 1;
    · induction' ( Finset.univ : Finset ( spectrum ℝ σ.m ) ) using Finset.induction
    · simp_all only [Finset.sum_empty, Matrix.zero_mulVec];
    · simp_all only [not_false_eq_true, Finset.sum_insert];
      simp [ Matrix.add_mulVec ];
    · norm_num;
    rw [ Finset.sum_eq_single k ] at h_sum_proj <=> aesop;
-- Since  $v \in \ker \sigma$ , we have  $\sum_k P_k \rho v = \sum_k P_k \rho v$ 
have h_sum : (pinching_map σ ρ).M.mat *v v =  $\sum k : \text{spectrum } \mathbb{R} \sigma.m, (\text{pinching\_kraus } k) \cdot v$ 
    convert congr_arg ( fun x : Matrix d d ℂ => x.mulVec v ) ( pinching_eq_sum_conj
      simp [ Matrix.mul_assoc, Matrix.sum_mulVec ];
    refine' Eq.trans congr(WithLp.toLp 2 $(h_sum)) _;
    simp only [MState.mat_M, Matrix.mulVec_mulVec, WithLp.toLp_sum]
    refine' Finset.sum_eq_zero fun k hk => _;
    by_cases hk_zero : k.val = 0
    · simp_all only [ne_eq, Subtype.forall, Matrix.mulVec_mulVec, Finset.mem_univ]
      simp only [WithLp.toLp_eq_zero]
      convert congr_arg ( fun x => (pinching_kraus σ k).mat *v x ) congr($(h hv)) using 1
    · specialize h_proj_one k.val k.2 hk_zero
      rw! [← hk_zero] at h_proj_one
      simp only [Subtype.coe_eta] at h_proj_one
      simp only [← Matrix.mul_assoc, ← Matrix.mulVec_mulVec, ContinuousLinearMap.coe]
      rw [h_proj_one]
      rfl
    · simp
    · simp [← Matrix.mulVec_mulVec, h_proj_zero _ hk_zero]

/-- Exercise 2.8 of Hayashi's book "A group theoretic approach to Quantum Information Theory"
/
theorem pinching_pythagoras (ρ σ : MState d) :
   $\| \rho \|_\sigma = \| \rho \|_{\text{pinching\_map } \sigma \rho} + \| \text{pinching\_map } \sigma \rho \|_\sigma :=$  by
  by_cases h_ker : σ.M.ker ≤ ρ.M.ker
  · have h_ker1 : (pinching_map σ ρ).M.ker ≤ ρ.M.ker := pinching_map_ker_le ρ σ
    have h_ker2 : σ.M.ker ≤ (pinching_map σ ρ).M.ker := ker_le_ker_pinching_map_ker
    rw [← EReal.coe_enreal_eq_coe_enreal_iff, EReal.coe_enreal_add]
    rw [qRelativeEnt_ker h_ker, qRelativeEnt_ker h_ker1, qRelativeEnt_ker h_ker2]
    have h_eq1 := inner_cfc_pinching_right ρ σ Real.log
    have h_eq2 := inner_cfc_pinching ρ σ Real.log
    rw [← HermitianMat.log] at h_eq1 h_eq2
    simp only [inner_sub_right]
    rw [h_eq2, h_eq1]
    simp only [EReal.coe_sub]
    rw [← add_sub_assoc, EReal.sub_add_cancel]
  · simp only [qRelativeEnt, SandwichedRelEntropy, dif_pos zero_lt_one]
    trans T
    · exact dif_neg h_ker

```

```

· convert (add_top _).symm
  apply dif_neg ?_
  contrapose! h_ker
  exact h_ker.trans (pinching_map_ker_le ρ σ)

```

1.445 QuantumInfo/Finite/Qubit/Basic.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.Finite.CPTPMap

/-!
Quantum theory and operations specific to qubits.
- Standard named (single-qubit) gates: Z, X, Y, H, S, T
- Controlled versions of gates
- Completeness of the PPT test: a state is separable iff it is PPT.
- Fidelity for qubits:  $F(\rho, \sigma) = 2\sqrt{(\rho.\det * \sigma.\det)}$ .
- The singlet/triplet split.
-/

abbrev Qubit := Fin 2

open Lean.Parser.Tactic in
open Lean in
/--
Proves goals equating small matrices by expanding out products and simplifying.
-/
syntax (name := matrix_expand) "matrix_expand"
  (" [" ((simpStar <|> simpErase <|> simpLemma),*,?) "]" )?
  (" with " rcasesPat+)? : tactic

macro_rules
| `(tactic| matrix_expand $[[$rules,*]]? $[with $withArg*]?) => do
  let id1 := (withArg.getD {[]}).getD 0 (← `(rcasesPat| _))
  let id2 := (withArg.getD {[]}).getD 1 (← `(rcasesPat| _))
  let rules' := rules.getD {#[]}
  `(tactic| (
    ext i j
    repeat rcases (i : Prod _ _) with ⟨i, $id1⟩
    repeat rcases (j : Prod _ _) with ⟨j, $id2⟩
    fin_cases i

```

```

    <=> fin_cases j
    <=> simp [Complex.ext_iff,
      Matrix.mul_apply, Fintype.sum_prod_type, Matrix.one_apply, field,
      $rules',* ]
    <=> norm_num
    <=> try field_simp
    <=> try ring_nf
  ))

namespace Qubit
open Real
open Complex

variable {k : Type*} [Fintype k] [DecidableEq k]

/-- The Pauli Z gate on a qubit. -/
def Z : Matrix k k :=
  (!! [1, 0; 0, -1], by constructor <=> matrix_expand)

/-- The Pauli X gate on a qubit. -/
def X : Matrix k k :=
  (!! [0, 1; 1, 0], by constructor <=> matrix_expand)

/-- The Pauli Y gate on a qubit. -/
def Y : Matrix k k :=
  (!! [0, -I; I, 0], by constructor <=> matrix_expand)

/-- The H gate, a Hadamard gate, on a qubit. -/
noncomputable def H : Matrix k k :=
  (√(1/2) • (!! [1, 1; 1, -1]), by constructor <=> matrix_expand)

/-- The S gate, or Rz(π/2) rotation on a qubit. -/
def S : Matrix k k :=
  (!! [1, 0; 0, I], by constructor <=> matrix_expand)

/-- The T gate, or Rz(π/4) rotation on a qubit. -/
noncomputable def T : Matrix k k :=
  (!! [1, 0; 0, (1 + I)/√2], by constructor <=> matrix_expand)

@[simp]
theorem Z_sq : Z * Z = 1 := by
  matrix_expand [Z]

@[simp]
theorem X_sq : X * X = 1 := by
  matrix_expand [X]

```

```

@[simp]
theorem Y_sq : Y * Y = 1 := by
  matrix_expand [Y]

@[simp]
theorem H_sq : H * H = 1 := by
  matrix_expand [H]

@[simp]
theorem S_sq : S * S = Z := by
  matrix_expand [S, Z]

@[simp]
theorem T_sq : T * T = S := by
  matrix_expand [T, S]

/-- The anticommutator `{X,Y}` is zero. Marked simp as to put Pauli product
X-Z order. -/
@[simp]
theorem X_Y_anticomm : X * Y = -Y * X := by
  matrix_expand [X, Y]

/-- The anticommutator `{Y,Z}` is zero. -/
theorem Y_Z_anticomm : Z * Y = -Y * Z := by
  matrix_expand [Z, Y]

/-- The anticommutator `{Z,X}` is zero. -/
theorem Z_X_anticomm : Z * X = -X * Z := by
  matrix_expand [Z, X]

@[simp]
theorem H_mul_X_eq_Z_mul_H : H * X = Z * H := by
  matrix_expand [H, X, Z]

@[simp]
theorem H_mul_Z_eq_X_mul_H : H * Z = X * H := by
  matrix_expand [H, X, Z]

@[simp]
theorem S_Z_comm : Z * S = S * Z := by
  simp [← S_sq, mul_assoc]

@[simp]
theorem T_Z_comm : Z * T = T * Z := by
  simp [← S_sq, ← T_sq, mul_assoc]

```



```

@[simp]
theorem S_T_comm : S * T = T * S := by
  simp [← T_sq, mul_assoc]

/-- Given a unitary `U` on some Hilbert space `k`, we have the controllized version
where `U` is conditionally applied if the first qubit is `1`. -
/
def controllize (g :  $\mathbb{C}[k]$ ) :  $\mathbb{C}[\text{Qubit} \times k]$  :=
  (Matrix.of fun (q1,t1) (q2,t2) ↦
    if (q1,q2) = (0,0) then
      (if t1 = t2 then 1 else 0)
    else if (q1,q2) = (1,1) then
      g t1 t2
    else 0
  , by
    rw [Matrix.mem_unitaryGroup_iff]
    matrix_expand [-Complex.ext_iff] with ti tj;
    · congr 1
      exact propext eq_comm
    · exact congrFun2 g.2.2 ti tj
  )

scoped notation "C[" g "]" => controllize g

/-- Controlled-NOT gate on two qubits.
The first qubit is the control and the second qubit is the target. -
/
def CNOT :  $\mathbb{C}[\text{Fin } 2 \times \text{Fin } 2]$  := C[X]

/-- The matrix representation of CNOT is the standard 4x4 permutation matrix. -
/
lemma CNOT_matrix :
  Matrix.reindex finProdFinEquiv finProdFinEquiv CNOT.val =
    ![
      [(1:ℂ), 0, 0, 0],
      [0, 1, 0, 0],
      [0, 0, 0, 1],
      [0, 0, 1, 0]
    ] := by
  ext i j
  simp only [CNOT, Qubit.X, Matrix.reindex_apply]
  fin_cases i <=> fin_cases j <=> rfl

variable (g :  $\mathbb{C}[k]$ ) (j1 j2 : k)

@[simp]
theorem controllize_apply_zero_zero : C[g] (0, j1) (0, j2) = (1 :  $\mathbb{C}[k]$ ) j1 j2 := by

```

```

    rfl

@[simp]
theorem controllize_apply_zero_one :  $C[g] \ (0, j_1) \ (1, j_2) = 0 := by$ 
  rfl

@[simp]
theorem controllize_apply_one_zero :  $C[g] \ (1, j_1) \ (0, j_2) = 0 := by$ 
  rfl

@[simp]
theorem controllize_apply_one_one :  $C[g] \ (1, j_1) \ (1, j_2) = g \ j_1 \ j_2 := by$ 
  rfl

@[simp]
theorem controllize_mul (g1 g2 :  $\square[k]$ ) :  $C[g_1] * C[g_2] = C[g_1 * g_2] := by$ 
  matrix_expand

@[simp]
theorem controllize_one :  $C[(1 : \square[k])] = 1 := by$ 
  matrix_expand

@[simp]
theorem controllize_mul_inv :  $C[g] * C[g^{-1}] = 1 := by$ 
  simp

open scoped Matrix in
@[simp]
theorem X_controllize_X :  $(X \otimes_u 1) * C[g] * (X \otimes_u 1) = (1 \otimes_u g) * C[g^{-1}] :=$ 
  matrix_expand [X, -Complex.ext_iff] with ki kj;
  suffices (1 : Matrix k k  $\mathbb{C}$ ) ki kj = (g * g-1) ki kj by
    convert this
  simp

end Qubit

```

1.446 QuantumInfo/Finite/ResourceTheory/FreeState.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg, Leonardo A. Lessa, Rodolfo R. Soldati
-/
import Mathlib.Algebra.Module.Submodule.Lattice

```

```

import Mathlib.Analysis.Subadditive
import Mathlib.CategoryTheory.Functor.FullyFaithful
import Mathlib.CategoryTheory.Monoidal.Braided.Basic
import Mathlib.Data.EReal.Basic
import Mathlib.Tactic

```

```

import QuantumInfo.Finite.CPTPMap
import QuantumInfo.Finite.Entropy

```

```

open scoped Topology

```

```

/-- A `ResourcePretheory` is a family of Hilbert spaces closed under tensor products
`Fintype` and `DecidableEq` for each. It forms a pre-structure then on which to disc
theories. For instance, to talk about "two-party scenarios", we could write `Resourc
with `H (a,b) := (Fin a) × (Fin b)`.
```

```

The `Semigroup ι` structure means we have a way to take products of our labels of Hi
in a way that is associative (with actual equality). The `prodEquiv` lets us reinter
a product-labelled Hilbert spaces, and an actual pair of Hilbert spaces.
-/

```

```

class ResourcePretheory (ι : Type*) extends Semigroup ι where

```

```

  /- The indexing of each Hilbert space -/

```

```

  H : ι → Type*

```

```

  /- Each space is finite -/

```

```

  [FinH : ∀ i, Fintype (H i)]

```

```

  /- Each object has decidable equality -/

```

```

  [DecEqH : ∀ i, DecidableEq (H i)]

```

```

  /- Each space is nonempty (dimension at least 1) -/

```

```

  [NonemptyH : ∀ i, Nonempty (H i)]

```

```

  /- The product structure induces an isomorphism of Hilbert spaces -
/

```

```

  prodEquiv i j : H (i * j) ≈ (H i) × (H j)

```

```

  --Possible we want some fact like the associativity of `prod` or the existence of
  -- which would then imply MonoidalCategory structure later (instead of just Catego
  -- take the (logically equivalent, in the appropriate model) assumption that the a
  -- actually an equality. This is captured in the `Semigroup ι` assumption. If we w
  -- this into something more flexible, we would replace that with `Mul ι` (dropping
  -- and get an appropriate associator `Equiv` here.

```

```

  hAssoc i j k :

```

```

    ((prodEquiv (i * j) k).trans <|
      ((prodEquiv i j).prodCongr (Equiv.refl (H k))).trans <|
      (Equiv.prodAssoc _ _ _).trans <|
      ((Equiv.refl (H i)).prodCongr ((prodEquiv j k).symm))).trans
      (prodEquiv i (j * k)).symm
    )

```

```

  = Equiv.cast (congrArg H <| mul_assoc i j k)

```

```

attribute [instance] ResourcePretheory.FinH
attribute [instance] ResourcePretheory.DecEqH
attribute [instance] ResourcePretheory.NonemptyH

namespace ResourcePretheory

variable {ι : Type*} [ResourcePretheory ι] {i j k l : ι}

/-- The `prod` operation of `ResourcePretheory` gives the natural product of
that puts us in a new Hilbert space of the category. Accessible by the notation
/
noncomputable def prodRelabel (ρ₁ : MState (H i)) (ρ₂ : MState (H j)) : MState (H (i * j)) :=
  (ρ₁ ⊗M ρ₂).relabel (prodEquiv i j)

@[inherit_doc]
scoped infixl:65 " ⊗r " => prodRelabel

theorem prodRelabel_assoc (ρ₁ : MState (H i)) (ρ₂ : MState (H j)) (ρ₃ : MState (H k)) :
  ρ₁ ⊗r ρ₂ ⊗r ρ₃ ≈ ρ₁ ⊗r (ρ₂ ⊗r ρ₃) := by
  simp [prodRelabel, MState.relabel_kron]
  have h_equiv := hAssoc i j k
  rw [← Equiv.trans_assoc, Equiv.trans_cancel_right] at h_equiv
  have h_cong := congrArg (MState.relabel ((ρ₁ ⊗M ρ₂) ⊗M ρ₃)) h_equiv
  rw [← eq_cast_iff_heq]; swap
  · rw [mul_assoc]
  convert h_cong; clear h_equiv h_cong
  rw [← MState.relabel_cast]; swap
  · rw [mul_assoc]
  rw [MState.kron_relabel, MState.prod_assoc]
  rw [MState.relabel_comp, MState.relabel_comp, MState.relabel_comp]
  rfl

/-- A `MState.relabel` can be distributed across a `prodRelabel`, if you have the
correspond correctly. -/
theorem prodRelabel_relabel_cast_prod
  (ρ₁ : MState (H i)) (ρ₂ : MState (H j))
  (h : H (k * l) = H (i * j)) (hik : k = i) (hlj : l = j) :
  (ρ₁ ⊗r ρ₂).relabel (Equiv.cast h) =
  (ρ₁.relabel (Equiv.cast (congrArg H hik))) ⊗r (ρ₂.relabel (Equiv.cast (congrArg H hlj)))
  subst hik
  subst hlj
  rfl

/-- The `prod` operation of `ResourcePretheory` gives the natural product of
by the notation `M₁ ⊗c pr M₂`. -/

```

```

noncomputable def prodCPTPMap (M1 : CPTPMap (H i) (H j)) (M2 : CPTPMap (H k) (H l))
  (CPTPMap (H (i * k)) (H (j * l))) :=
  (CPTPMap.ofEquiv (prodEquiv j l).symm).compose ((M1  $\otimes^{c p}$  M2).compose (CPTPMap.ofEquiv

@[inherit_doc]
scoped notation M1 "  $\otimes^{c p}_r$  " M2 => prodCPTPMap M1 M2

open ComplexOrder in
theorem PosDef.prod {ρ : MState (H i)} {σ : MState (H j)} (hp : ρ.m.PosDef) (hσ : σ.
  : (ρ  $\otimes_r$  σ).m.PosDef := by
  have : (ρ  $\otimes^M$  σ).m.PosDef := MState.PosDef.kron hp hσ
  rw [prodRelabel]
  exact MState.PosDef.relabel this (prodEquiv i j)

--BAD old attempt at PNat powers
-- /-- Powers of spaces. Defined for `PNat` so that we don't have zeroth powers. -
/
-- noncomputable def spacePow (i : ι) (n : ℕ+) : ι :=
--   n.natPred.rec i (fun _ j => prod j i)

-- scoped notation i "  $\otimes^H[" n "]"$  => spacePow i n

-- @[simp]
-- theorem spacePow_one (i : ι) : i  $\otimes^H[1]$  = i :=
--   rfl

-- theorem spacePow_succ (i : ι) (n : ℕ+) : i  $\otimes^H[n + 1]$  = prod (i  $\otimes^H[n]$ ) i := by
--   rcases n with ⟨_, hn⟩
--   · contradiction
--   · rfl

-- /-- Powers of states. Defined for `PNat`, so that we don't have zeroth powers -
/
-- noncomputable def statePow {i : ι} (ρ : MState (H i)) (n : ℕ+) : MState (H (i  $\otimes^H[n]$ 
--   (n.natPred.rec ρ (fun _ σ => σ  $\otimes_r$  ρ) : MState (H (i  $\otimes^H[n.natPred.succPNat]$ )))

-- scoped notation ρ "  $\otimes_r^H[" n "]"$  => statePow ρ n

-- @[simp]
-- theorem statePow_one {i : ι} (ρ : MState (H i)) : ρ  $\otimes_r^H[1]$  = ρ :=
--   rfl

-- theorem statePow_succ {i : ι} (ρ : MState (H i)) (n : ℕ+) : ρ  $\otimes_r^H[n + 1]$  = ρ  $\otimes_r^H[n]$ 
--   rcases n with ⟨_, hn⟩
--   · contradiction
--   · rfl

```

```

@[simp]
theorem qRelEntropy_prodRelabel (ρ1 ρ2 : MState (H i)) (σ1 σ2 : MState (H j)) :
   $\square(\rho_1 \otimes_r \sigma_1 \| \rho_2 \otimes_r \sigma_2) = \square(\rho_1 \| \rho_2) + \square(\sigma_1 \| \sigma_2)$  := by
  simp [prodRelabel]

@[simp]
theorem sandwichedRelEntropy_prodRelabel {α : ℝ} (ρ1 ρ2 : MState (H i)) (σ1 σ2 : MState (H j)) :
   $\check{D}_\alpha(\rho_1 \otimes_r \sigma_1 \| \rho_2 \otimes_r \sigma_2) = \check{D}_\alpha(\rho_1 \| \rho_2) + \check{D}_\alpha(\sigma_1 \| \sigma_2)$  := by
  simp [prodRelabel]

end ResourcePretheory

open ResourcePretheory

/-- A ResourcePretheory is `Unital` if it has a Hilbert space of size 1, i.e.
/
class UnitalPretheory (ι : Type*) extends ResourcePretheory ι, MulOneClass
  prod_default {i} (ρ : MState (H i)) :
    (toResourcePretheory.prodRelabel ρ (Inhabited.default : MState (H 1)))
  default_prod {i} (ρ : MState (H i)) :
    (toResourcePretheory.prodRelabel (Inhabited.default : MState (H 1)) ρ)

namespace UnitalPretheory

variable {ι : Type*} [UnitalPretheory ι] {i j : ι}

instance : Monoid ι where

/-- Powers of spaces.

We define it for `Nat` in a `UnitalPretheory`. In principal this could be defined
in `ResourcePretheory` and be defined for `PNat` so that we don't have zeroth
anticipation that we might some day want that, and that we might do everything
non-equality associator, we keep this as its own definition and keep our own
rewriting theorems where possible.-/
noncomputable def spacePow (i : ι) (n : ℕ) : ι :=
  i ^ n

--This notation is less necessary now since we can just write `i ^ n` as long as
--a monoid.
scoped notation i "⊗^H[" n "]" => spacePow i n

@[simp]
theorem spacePow_zero (i : ι) : i ^ 0 = 1 := by
  rfl

```

```

@[simp]
theorem spacePow_one (i :  $\iota$ ) :  $i^1 = i$  := by
  simp

theorem spacePow_succ (i :  $\iota$ ) (n :  $\mathbb{N}$ ) :  $i^{(n+1)} = (i^n) * i$  := by
  rfl

theorem spacePow_add (m n :  $\mathbb{N}$ ) :
   $i^{(m+n)} = (i^m) * (i^n)$  := by
  induction n
  · simp
  · rename_i n ih
    rw [spacePow_succ,  $\leftarrow$  mul_assoc,  $\leftarrow$  add_assoc,  $\leftarrow$  ih, spacePow_succ]

theorem spacePow_mul (m n :  $\mathbb{N}$ ) :
   $i^{(m*n)} = (i^m)^n$  :=
  pow_mul i m n

/-- Powers of states, using the resource theory's notion of product. Accessible via
`p  $\otimes_r^n$ `.-/
noncomputable def statePow (p : MState (H i)) (n :  $\mathbb{N}$ ) : MState (H ( $i^n$ )) :=
  n.rec default (fun _  $\sigma \mapsto \sigma \otimes_r p$ )

@[inherit_doc]
scoped notation p "  $\otimes_r^n$  [" n "]" => statePow p n

@[simp]
theorem statePow_zero (p : MState (H i)) :  $p \otimes_r^{[0]} = \text{default}$  :=
  rfl

@[simp]
theorem statePow_one (p : MState (H i)) :  $p \otimes_r^{[1]} \asymp p$  := by
  rw [ $\leftarrow$  eq_cast_iff_heq]; swap
  · rw [spacePow_one]
  · rw [eq_cast_iff_heq, statePow]
    exact default_prod p

theorem statePow_succ (p : MState (H i)) (n :  $\mathbb{N}$ ) :  $p \otimes_r^{[n+1]} = p \otimes_r^{[n]} \otimes_r p$  := by
  rfl

theorem statePow_add (p : MState (H i)) (m n :  $\mathbb{N}$ ) :  $p \otimes_r^{[m+n]} \asymp p \otimes_r^{[m]} \otimes_r p \otimes_r^{[n]}$ 
  rw [ $\leftarrow$  eq_cast_iff_heq]; swap
  · rw [spacePow_add]
  rw [eq_cast_iff_heq]
  induction n

```

```

· rw [add_zero, statePow_zero]
  exact (prod_default _).symm
· rename_i n ih
  rw [statePow_succ, ← add_assoc, statePow_succ]
  refine HEq.trans ?_ (prodRelabel_assoc _ _ _)
  congr
  apply spacePow_add

theorem statePow_add_relabel (p : MState (H i)) (m n : ℕ) :
  p ⊗r^[m + n] = (p ⊗r^[m] ⊗r p ⊗r^[n]).relabel (Equiv.cast (by congr; ex
have h := statePow_add p m n
rw [heq_iff_exists_eq_cast] at h
obtain ⟨h, h₂⟩ := h
rw [h₂, MState.relabel_cast]

theorem statePow_mul (p : MState (H i)) (m n : ℕ) : p ⊗r^[m * n] ≈ (p ⊗r^[m
rw [← eq_cast_iff_heq]; swap
· rw [spacePow_mul]
rw [eq_cast_iff_heq]
induction n
· simp
· rename_i n ih
  rw [statePow_succ, mul_add]
  --This is TERRIBLE. There has to be a better way. TODO Cleanup
  trans p ⊗r^[m * n] ⊗r p ⊗r^[m * 1]
  · apply statePow_add
  · rw [← eq_cast_iff_heq] at ih; swap
    · congr 2 <;> simp [pow_mul]
    rw [← eq_cast_iff_heq]; swap
    · congr 2 <;> simp [pow_mul]
    rw [← MState.relabel_cast _ (by simp [pow_mul])]
    rw [prodRelabel_relabel_cast_prod]
    · congr
      · rw [ih, MState.relabel_cast]
      · rw [MState.relabel_cast]
        rw [eq_cast_iff_heq]
        · rw [mul_one]
    · rw [pow_mul]
    · rw [mul_one]

theorem statePow_mul_relabel {i : ι} (p : MState (H i)) (m n : ℕ) :
  p ⊗r^[m * n] = (p ⊗r^[m]) ⊗r^[n].relabel (Equiv.cast (congrArg H (pow_mu
have h := statePow_mul p m n
rw [heq_iff_exists_eq_cast] at h
obtain ⟨h, h₂⟩ := h
rw [h₂, MState.relabel_cast]

```



```

open ComplexOrder in
theorem PosDef.npow {ρ : MState (H i)} (hp : ρ.m.PosDef) (n : ℕ)
  : (ρ ⊗r ^[n]).m.PosDef := by
  induction n
  · rw [statePow_zero, spacePow_zero]
    exact MState.posDef_of_unique default
  · apply ResourcePretheory.PosDef.prod <_> hp

theorem statePow_rw {n m : ℕ} (h : n = m) (ρ : MState (H i)) :
  ρ ⊗r ^[n] = (ρ ⊗r ^[m]).relabel (Equiv.cast (by congr)) := by
  subst n
  simp

@[simp]
theorem qRelEntropy_statePow (ρ σ : MState (H i)) (n : ℕ) :
  □(ρ ⊗r ^[n] || σ ⊗r ^[n]) = n * □(ρ || σ) := by
  induction n
  · simp
  · rename_i n ih
    rw [statePow_succ, statePow_succ, qRelEntropy_prodRelabel]
    simp [ih, add_mul]

theorem sInf_spectrum_rprod {j : ι} (ρ : MState (H i)) (σ : MState (H j)) :
  sInf (spectrum ℝ (ρ ⊗r σ).m) = sInf (spectrum ℝ ρ.m) * sInf (spectrum ℝ σ.m) :=
  rw [← MState.sInf_spectrum_prod, prodRelabel, MState.spectrum_relabel]

lemma sInf_spectrum_spacePow (σ : MState (H i)) (n : ℕ) :
  sInf (spectrum ℝ (σ ⊗r ^[n]).m) = sInf (spectrum ℝ σ.m) ^ n := by
  induction n
  · simp only [statePow_zero, pow_zero]
    conv =>
      enter [1, 1, 2]
      equals 1 =>
        ext1
        simp [default, MState.uniform, MState.ofClassical, MState.m, HermitianMat.diagonal]
        rw [spectrum.one_eq, csInf_singleton]
  · rename_i n ih
    rw [statePow_succ, sInf_spectrum_rprod, ih, pow_succ]

@[simp]
theorem sandwichedRelEntropy_statePow {α : ℝ} (ρ σ : MState (H i)) (n : ℕ) :
   $\tilde{D}_\alpha(\rho \otimes_r^n || \sigma \otimes_r^n) = n * \tilde{D}_\alpha(\rho || \sigma)$  := by
  induction n
  · rw [statePow_zero, statePow_zero, spacePow_zero]
    simp

```

```

    · rename_i n ih
      rw [statePow_succ, statePow_succ, sandwichedRelEntropy_prodRelabel]
      simp [ih, add_mul]

end UnitalPretheory

/- FreeStateTheories: theories defining some sets of "free states" within a
/

/-- A `FreeStateTheory` is a collection of mixed states (`MState`s) in a `R`
some necessary axioms:
* For each Hilbert space `H i`, the free states are a closed, convex set
* For each Hilbert space `H i`, there is a free state of full rank
* If ` $\rho_i \in H i$ ` and ` $\rho_j \in H j$ ` are free, then ` $\rho_i \otimes \rho_j$ ` is free in `H` (prod)
-/
class FreeStateTheory (ι : Type*) extends ResourcePretheory ι where
  /- The set of states we're calling "free" -/
  IsFree : Set (MState (toResourcePretheory.H i))
  /- The set F(H) of free states is closed -/
  free_closed : IsClosed (@IsFree i)
  /- The set F(H) of free states is convex (more precisely, their matrices)
  /
  free_convex : Convex ℝ (MState.M '' (@IsFree i))
  /- The set of free states is closed under tensor product -
  /
  free_prod {ρ₁ : MState (H i)} {ρ₂ : MState (H j)} (h₁ : IsFree ρ₁) (h₂ :
  /- The set F(H) of free states contains a full-rank state `ρfull`, equiv
  /
  free_fullRank (i : ι) : open ComplexOrder in ∃ (ρ : MState (H i)), ρ.m.Po

namespace FreeStateTheory

variable {ι : Type*} [FreeStateTheory ι] {i : ι}

noncomputable instance Inhabited_IsFree : Inhabited (IsFree (i := i)) :=
  ⟨((free_fullRank i).choose, (free_fullRank i).choose_spec.right)⟩

theorem IsFree.of_unique [Unique (H i)] (ρ : MState (H i)) : ρ ∈ IsFree :=
  obtain ⟨σ, h₁, h₂⟩ := free_fullRank i
  convert h₂
  apply Subsingleton.allEq

/--The set of free states is compact because it's a closed subset of a comp
/
theorem IsCompact_IsFree : IsCompact (IsFree (i := i)) :=
  .of_isClosed_subset isCompact_univ free_closed (Set.subset_univ _)

```

```

--Also this needs to be generalized to other convex sets. I think it should work for
--(well-behaved?) Mixable instance, it certainly works for any `Convex` set (of which
-- is one, the only relevant property here is `free_convex`.
theorem IsFree.mix {ι : Type*} [FreeStateTheory ι] {i : ι} {σ₁ σ₂ : MState (H i)}
  (hσ₁ : IsFree σ₁) (hσ₂ : IsFree σ₂) (p : Prob) : IsFree (p [σ₁ ↔ σ₂]) := by
  obtain ⟨m, hm₁, hm₂⟩ := free_convex (i := i) ⟨σ₁, hσ₁, rfl⟩ ⟨σ₂, hσ₂, rfl⟩ p.zero_
  simp [Mixable.mix, Mixable.mix_ab, MState.instMixable]
  simp at hm₂
  convert ← hm₁

end FreeStateTheory

open UnitalPretheory
open FreeStateTheory

class UnitalFreeStateTheory (ι : Type*) extends FreeStateTheory ι, UnitalPretheory ι

namespace UnitalFreeStateTheory

--Things like asymptotically free operations, measures of non-
--freeness, etc. that can be stated
--entirely in terms of the free states (without referring to operations) go here.

variable {ι : Type*} [UnitalFreeStateTheory ι] {i : ι}

theorem _root_.FreeStateTheory.IsFree.npow {i : ι} {p : MState (H i)}
  (hp : IsFree p) (n : ℕ) : IsFree (p ⊗r^[n]) := by
  induction n
  · rw [statePow_zero, spacePow_zero]
    apply IsFree.of_unique
  · rw [statePow_succ]
    exact FreeStateTheory.free_prod <_> hp

@[simp]
theorem relabel_cast_isFree {i j : ι} (p : MState (H i)) (h : j = i) {h' : H j = H i}
  p.relabel (Equiv.cast h') ∈ IsFree ↔ p ∈ IsFree := by
  subst h
  simp

open NNReal

/-- In a `FreeStateTheory`, we have free states of full rank, therefore the minimum
of any state `p` to a free state is finite. -/
@[aesop (rule_sets := [finiteness]) safe]
lemma relativeEntResource_ne_top (p : MState (H i)) : ∃ σ ∈ IsFree, ∃ (p||σ) ≠ ⊤ := by

```

```

let {w,h} := free_fullRank i
apply ne_top_of_le_ne_top _ (iInf_le _ w)
simp only [ne_eq, iInf_eq_top, Classical.not_imp]
constructor
· exact h.2
· refine ne_of_apply_ne ENNReal.toEReal (qRelativeEnt_ker (p := p) (?_) ▶
  convert @bot_le _ _ (Submodule.instOrderBot) _
  have := h.1.toLin_ker_eq_bot
  simp [LinearMap.ker_eq_bot', HermitianMat.ker] at this ⊢
  intro m hm
  simp only [WithLp.ofLp_eq_zero] using this m congr($hm)

noncomputable def RelativeEntResource : MState (H i) → ℝ≥0 :=
  fun p ↦ (⊓ σ ∈ IsFree, ⊓ (p||σ)).untop (relativeEntResource_ne_top p)

scoped notation "⊓r" => RelativeEntResource

theorem exists_isFree_relativeEntResource (p : MState (H i)) :
  ∃ σ ∈ IsFree, ⊓ (p||σ) = ⊓r p := by
  obtain {σ, hσ1, hσ2} := IsCompact_IsFree.exists_isMinOn_lowerSemicontinuous
    (s := IsFree (i := i)) (f := fun σ ↦ ⊓ (p||σ))
  Set.Nonempty.of_subtype (by fun_prop)
  use σ, hσ1
  rw [RelativeEntResource, ← hσ2.iInf_eq hσ1, ENNReal.ofNNReal, WithTop.coe]

theorem RelativeEntResource.Subadditive (p : MState (H i)) : Subadditive fun
  intro m n
  obtain {σ2, hσ2f, hσ2d} := exists_isFree_relativeEntResource (p ⊗r^[m])
  obtain {σ3, hσ3f, hσ3d} := exists_isFree_relativeEntResource (p ⊗r^[n])
  simp only [RelativeEntResource, ← NNReal.coe_add, coe_le_coe]
  rw [← ENNReal.coe_le_coe]
  simp [RelativeEntResource, ENNReal.ofNNReal] at hσ2d hσ3d ⊢
  rw [← hσ2d, ← hσ3d]
  clear hσ2d hσ3d
  have ht1 : i ^ (m + n) = i ^ m * i ^ n :=
    spacePow_add m n
  have ht := congrArg H ht1
  refine le_trans (biInf_le (i := (σ2 ⊗r σ3).relabel (Equiv.cast ht)) _ ?_)
  · simp [ht1] using free_prod hσ2f hσ3f
  · apply le_of_eq
    rw [← qRelEntropy_prodRelabel]
    gcongr
    · apply statePow_add
    · rw [← eq_cast_iff_heq]
      apply MState.relabel_cast
      rw [spacePow_add]

```

```

noncomputable def RegularizedRelativeEntResource (p : MState (H i)) :  $\mathbb{R}_{\geq 0}$  :=
  ((RelativeEntResource.Subadditive p).lim, by
    rw [Subadditive.lim]
    apply Real.sInf_nonneg
    rintro x ⟨x, hx, rfl⟩
    positivity)

```

```

scoped notation "□r∞" => RegularizedRelativeEntResource

```

```

/-- Lemma 5 -/

```

```

theorem RelativeEntResource.tendsto (p : MState (H i)) :
  Filter.atTop.Tendsto (fun n ↦ □r (p ⊗r^[n]) / n) (□ (□r∞ p)) := by
  rw [← NNReal.tendsto_coe]
  apply (RelativeEntResource.Subadditive p).tendsto_lim
  use 0
  rintro _ ⟨y, rfl⟩
  positivity

```

```

/-- Alternate version of Lemma 5 which states the convergence with the `ENNReal`
expression for `RelativeEntResource`, as opposed its `untop`-
ped `NNReal` value. -/

```

```

theorem RelativeEntResource.tendsto_ennreal (p : MState (H i)) :
  Filter.atTop.Tendsto (fun n ↦ (□ σ ∈ IsFree, □ (p ⊗r^[n] || σ)) / ↑n) (□ (□r∞ p)) :=
  refine Filter.Tendsto.congr' ?_ (ENNReal.tendsto_coe.mpr <| RelativeEntResource.te
  rw [Filter.EventuallyEq, Filter.eventually_atTop]
  use 1; intros
  rw [RelativeEntResource, ENNReal.coe_div (by positivity), ENNReal.coe_natCast]
  congr
  apply WithTop.coe_untop

```

```

noncomputable def GlobalRobustness {i : ι} : MState (H i) →  $\mathbb{R}_{\geq 0}$  :=
  fun p ↦ sInf {s | ∃ σ, ((1 / (1+s), by bound) [p ↦ σ]) ∈ IsFree}

```

```

/-- A sequence of operations `f_n` is asymptotically nongenerating if `lim_{n→∞} RG(
RG is `GlobalRobustness` and `ρ_n` is any sequence of free states. Equivalently, we
over operations and states) on the left-hand side inside the limit.
-/

```

```

def IsAsymptoticallyNongenerating (dI d0 : ι) (f : (n : ℕ) → CPTPMap (H (dI ⊗H H[n])))
  ∀ (ps : (n : ℕ) → MState (H (dI ⊗H H[n]))), (∀ n, IsFree (ps n)) →
  Filter.atTop.Tendsto (fun n ↦ GlobalRobustness ((f n) (ps n))) (□ 0)

```

```

end UnitalFreeStateTheory

```

1.447 QuantumInfo/Finite/ResourceTheory/HypothesisTesting.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg, Leonardo A. Lessa, Rodolfo R. Soldati
-/
import Mathlib.Algebra.Module.Submodule.Lattice
import Mathlib.Analysis.Subadditive
import Mathlib.CategoryTheory.Functor.FullyFaithful
import Mathlib.CategoryTheory.Monoidal.Braided.Basic
import Mathlib.Data.EReal.Basic

import QuantumInfo.Finite.CPTPMap
import QuantumInfo.Finite.Entropy
import QuantumInfo.Finite.POVm

/-!
Defines `OptimalHypothesisRate`, the optimal rate of distinguishing an `MST`
mixed states `S`, with at most Type I error  $\epsilon$ .

That is to say: take a projective measurement (a `POVM`) with elements `{T,
T}`, where measuring `T`
will mean we conclude our unknown state was  $\rho$ , and measuring `1-T`
will mean we think the state was
something in `S`. We only accept T's such that  $\text{Tr}[(1-T)\rho] \leq \epsilon$ , that is, w
probability of incorrectly concluding it was  $\rho$ . The Type II error associate
 $\max_{\sigma \in S} \text{Tr}[T \sigma]$ , that is, the (worst possible, over possible states)
concluding our state was in `S`. Optimize over `T` to get the lowest possib
and the resulting error rate is `OptimalHypothesisRate  $\rho \in S$ `.

We make this accessible through the notation  $\beta_{\epsilon}(\rho||S)$ .

See [The tangled state of quantum hypothesis testing](https://doi.org/10.10
023-02289-9) by
Mario Berta et al. for a broader overview.
-/

open NNReal
open ComplexOrder
open Topology
open Prob
open scoped RealInnerProductSpace InnerProductSpace

variable {d : Type*} [Fintype d] [DecidableEq d]

```

1.447.

QUANTUMINFO/FINITE/RESOURCE THEORY/HYPOTHESISTESTING.LEAN3

-- The optimal hypothesis testing rate, for a tolerance ε : given a state ρ and a set S , the optimum distinguishing rate that allows a probability ε of errors. -

```

/
noncomputable def OptimalHypothesisRate (ρ : MState d) (ε : Prob) (S : Set (MState d)) : ℝ :=
  ⌊ T : { m : HermitianMat d ℂ // ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 },
    ⌊ σ ∈ S, ⟨_, σ.exp_val_prob T.prop.right⟩

```

scoped[OptimalHypothesisRate] notation "β_" ε " (" ρ "||" S ")" => OptimalHypothesisRate

namespace OptimalHypothesisRate

-- The space of strategies `T` in `OptimalHypothesisRate` is inhabited, we always have a strategy.

```

/
instance iInf_Inhabited (ρ : MState d) (ε : Prob) :
  Inhabited { m // ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 } :=
  ⟨1, by simp⟩

```

```

instance (ρ : MState d) (ε : Prob) : Inhabited {m | ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1} :=
  iInf_Inhabited ρ ε

```

-- The space of strategies `T` in `OptimalHypothesisRate` is compact. -

```

/
theorem iInf_IsCompact (ρ : MState d) (ε : Prob) : IsCompact { m | ρ.exp_val (1 - m) ≤ ε } :=
  have hC1 : IsCompact {m : HermitianMat d ℂ | 0 ≤ m ∧ m ≤ 1} :=
    HermitianMat.unitInterval_IsCompact
  have hC2 : IsClosed {m | ρ.exp_val (1 - m) ≤ ε} := by
    --This is a linear constraint and so has a closed image
    change IsClosed ((fun m ↦ ρ.M.inner_1 (1 - m)) ⁻¹' (Set.Iic ε))
    refine IsClosed.preimage ?_ isClosed_Iic
    fun_prop
  exact hC1.inter_left hC2

```

-- The space of strategies `T` in `OptimalHypothesisRate` is convex. -

```

/
theorem iInf_IsConvex (ρ : MState d) (ε : Prob) : Convex ℝ { m | ρ.exp_val (1 - m) ≤ ε } :=
  --We *could* get this from a more general fact that any linear subspace is convex,
  --and the intersection of convex spaces is convex, and this is an intersection of
  --three convex spaces. That would be more broken-down and lemmaified.
  rintro x ⟨hx1, hx2, hx3⟩ y ⟨hy1, hy2, hy3⟩ a b ha hb hab
  rw [← eq_sub_iff_add_eq'] at hab
  subst b
  refine And.intro ?_ (And.intro ?_ ?_)
  · simp only [MState.exp_val, inner_sub_right, HermitianMat.inner_one, MState.tr,
    tsub_le_iff_right, inner_add_right, inner_smul_right] at hx1 hy1
    linear_combination a * hx1 + (1 - a) * hy1
  · apply HermitianMat.convex_cone <=> assumption

```

```

· rw [← sub_nonneg] at hx₃ hy₃ ⊢
  convert HermitianMat.convex_cone hx₃ hy₃ ha hb using 1
  simp only [sub_smul, one_smul, smul_sub]
  abel

/-- When `S` is empty, the optimal hypothesis testing rate is zero. -
/
@[simp]
theorem of_empty {ρ : MState d} (ε : Prob) : β_ ε(ρ||∅) = 0 := by
  simp [OptimalHypothesisRate]
  rfl

theorem le_sup_exp_val {ρ : MState d} (ε : Prob) {S : Set (MState d)}
  (m : HermitianMat d ℂ) (hExp : ρ.exp_val (1 - m) ≤ ε) (hm : 0 ≤ m ∧ m ≤ 1) :
  β_ ε(ρ||S) ≤ ⌊ σ ∈ S, ⟨_, σ.exp_val_prob hm⟩ := by
  unfold OptimalHypothesisRate
  apply iInf_le_of_le ⟨m, ⟨hExp, hm⟩⟩ _
  simp only [le_refl]

theorem le_of_subset (ρ : MState d) (ε : Prob) {S1 S2 : Set (MState d)} (h
  β_ ε(ρ||S1) ≤ β_ ε(ρ||S2) :=
  iInf_mono (fun _ ↦ iSup_le_iSup_of_subset h)

theorem of_singleton {ρ σ : MState d} {ε : Prob} :
  β_ ε(ρ||{σ}) =
  ⌊ T : { m : HermitianMat d ℂ // ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1
  ⟨_, σ.exp_val_prob T.2.right) := by
  simp only [OptimalHypothesisRate, iSup_singleton]

open scoped Prob in
theorem negLog_le_singleton (ρ : MState d) (ε : Prob) (S : Set (MState d))
  (σ : MState d) (h : σ ∈ S) : -log β_ ε(ρ||S) ≤ -log β_ ε(ρ||{σ}) := by
  apply Prob.negLog_Antitone
  apply le_of_subset
  exact Set.singleton_subset_iff.mpr h

theorem singleton_le_exp_val {ρ σ : MState d} {ε : Prob} (m : HermitianMat
  (hExp : ρ.exp_val (1 - m) ≤ ε) (hm : 0 ≤ m ∧ m ≤ 1) :
  β_ ε(ρ||{σ}) ≤ ⟨_, σ.exp_val_prob hm⟩ := by
  rw [of_singleton]
  apply iInf_le_of_le ⟨m, ⟨hExp, hm⟩⟩ _
  simp only [le_refl]

/-- There exists an optimal T for the hypothesis testing, that is, it's a m
and not just an infimum. This states we have `1 - ε ≤ ρ.exp_val T`, but we
"worsen" T to make that bound tight, which is `exists_min`. -

```


1.447.

QUANTUMINFO/FINITE/RESOURCE THEORY/HYPOTHESIS TESTING. LEAN 5

```
/
theorem exists_min' (ρ : MState d) (ε : Prob) (S : Set (MState d)) :
  ∃ (T : { m : HermitianMat d // ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 }),
    (∃ σ ∈ S, ⟨_, σ.exp_val_prob T.prop.right⟩ = β_ ε(ρ||S))
    ∧ 1 - ε ≤ ρ.exp_val T := by
  have _ : Nonempty d := ρ.nonempty
  rcases S.eq_empty_or_nonempty with rfl | hS
  · simp [-Subtype.exists] using {rfl, ⟨1, by simp⟩, by simp}
  rw [← Set.nonempty_coe_sort] at hS
  obtain (T, hT1, hT2) := IsCompact.exists_isMinOn (α := Prob)
    (isCompact_iff_isCompact_univ.mp (iInf_IsCompact ρ ε)) Set.univ_nonempty
    (f := fun T ↦ ∃ σ ∈ S, ⟨_, σ.exp_val_prob T.prop.right⟩)
    (by
      have h := HermitianMat.inner1.continuous_iSup_fst
        (Bornology.isBounded_induced.mp (Bornology.IsBounded.all S))
      apply Continuous.continuousOn
      simp_rw [← iSup_subtype'', subtype_val_iSup' (ι := S)]
      refine Continuous.subtype_mk ?_ _
      refine Continuous.comp (g := fun T ↦ ∃ (i : S), i.val.exp_val T) ?_ continuous
      convert h with T
      rw [← sSup_image' (s := S) (f := fun i ↦ i.exp_val T)]
      rw [← sSup_image' (s := (MState.M '' S)) (f := fun i ↦ i.inner1 T)]
      simp [Set.image, MState.exp_val, HermitianMat.inner1]
    )
  clear hT1

  use T
  constructor
  · simp only [isMinOn_univ_iff] at hT2
    rw [OptimalHypothesisRate]
    --Why is the following three bundled together not a theorem? Is it, and I can't
    apply le_antisymm
    · exact le_iInf hT2
    · exact iInf_le_iff.mpr fun _ a ↦ a T
  · simp [MState.exp_val_sub, add_comm (ε : ℝ) _] using T.2.1

--TODO: Maybe we should define these two instances.
-- Then we can get rid of `Prob.sub_zero` and use the generic `tsub_zero`.
-- #synth AddCommMonoid Prob
-- #synth OrderedSub Prob

/-- There exists an optimal T for the hypothesis testing, that is, it's a minimum and
not just an infimum. This tightens the `T` from `exists_min'` to a `∃ ρ, T = 1 - ε` b
/
theorem exists_min (ρ : MState d) (ε : Prob) (S : Set (MState d)) :
  ∃ (T : { m : HermitianMat d // ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1 }),
```

```

    (∏ σ ∈ S, ⟨_, σ.exp_val_prob T.prop.right⟩ = β_ ε(ρ||S))
    ∧ ρ.exp_val T = 1 - ε := by
  obtain ⟨T, hT1, hT2⟩ := exists_min' ρ ∈ S

```

```

--Instead of just `use T`, we (may) have to reduce it so that it saturate
--We do this by multiplying it by a scalar less than 1 to get a `T'`. Sin
--than T, it's still optimal in terms of achieving `β_ ε(ρ||S)`, but it ca
set δ := ρ.exp_val ↑T - (1 - ε)-- with δ_def

```

```

by_cases hδ : δ = 0
· use T, hT1
  linarith
replace hδ : 0 < δ := by linarith +splitNe
have hδ_le : δ ≤ 1 := by
  linarith [ρ.exp_val_le_one T.2.2.2, ε.2]

```

```

have hTr : 0 < ρ.exp_val T := by
  linarith [ε.coe_le_one]

```

```

set T' : HermitianMat d ℂ :=
  (1 - δ / ρ.exp_val T) • T with hT'_def
have hT'_le : T' ≤ T := by
  rw [← one_smul ℝ T.val, hT'_def]
  gcongr
  · exact T.2.2.1
  · simp; positivity
have hpT' : ρ.exp_val (1 - T') = ε := by
  simp [T', MState.exp_val_sub, δ, field]

```

```

have hT' : ρ.exp_val (1 - T') ≤ ε ∧ 0 ≤ T' ∧ T' ≤ 1 := by
  use hpT'.le
  constructor
  · simp [T']
    refine smul_nonneg ?_ T.2.2.1
    bound
  · exact hT'_le.trans T.2.2.2
use ⟨T', hT'⟩

```

```

constructor
· rw [OptimalHypothesisRate] at hT1
  apply le_antisymm
  · apply le_iInf
    intro i
    refine le_trans ?_ (le_of_eq_of_le hT1 ?_)
    · gcongr
    · exact iInf_le_iff.mpr fun _ a ↦ a i
  · exact iInf_le_iff.mpr fun _ a ↦ a ⟨T', hT'⟩

```

1.447.

QUANTUMINFO/FINITE/RESOURCE THEORY/HYPOTHESIS TESTING. LEAN

```

    · simp [MState.exp_val_sub, ← hpT']

/-- When the allowed Type I error  $\epsilon$  is less than 1 (so, we have some limit on our
and the kernel of the state  $\rho$  contains the kernel of some element in  $S$ , then the
hypothesis rate is positive - there is some lower bound on the type II errors we'll
other words, under these conditions, we cannot completely avoid type II errors. -
/
theorem pos_of_lt_one {ρ : MState d} (S : Set (MState d))
  (hp : ∃ σ ∈ S, σ.M.ker ≤ ρ.M.ker)
  {ε : Prob} (hε : ε < 1) : 0 < β_ε(ρ||S) := by
  obtain {σ, hσ₁, hσ₂} := hp
  --Assume the converse: that the infimum is zero. The set of such T's is inhabited
  --and closed, so there is some T that attains the value zero. This T has zero
  --inner product with σ ( $\sigma.exp\_val\ T = 0$ ), and yet (by definition of T's type) we
  --also have that  $\rho.exp\_val\ (1 - T) \leq \epsilon < 1$ . So  $T$  lives entirely in  $\sigma$ 's kernel,
  --which (by  $h\_supp$ ) is contained in  $\rho$ 's kernel. So
  -- $\rho.exp\_val\ (1 - T) = \rho.exp\_val\ 1 - \rho.exp\_val\ T = \rho.trace - 0 = 1$ , a contradiction
  by_contra h
  obtain {(T, hT₁, hT₂, hT₃), hT₄, hT₅} := exists_min ρ ε S
  rw [← bot_eq_zero'', not_bot_lt_iff] at h
  rw [h, iSup_eq_bot, bot_eq_zero''] at hT₄
  specialize hT₄ σ
  simp only [iSup_pos hσ₁, Subtype.ext_iff, Set.Icc.coe_zero, MState.exp_val] at hT₄
  rw [HermitianMat.inner_zero_iff σ.nonneg hT₂] at hT₄
  replace hT₁ : ρ.exp_val (1 - T) ≠ 1 := (lt_of_le_of_lt hT₁ hε).ne
  absurd hT₁
  rw [ρ.exp_val_eq_one_iff ?_, sub_sub_cancel]
  · grw [← hT₄]
    rwa [HermitianMat.ker, HermitianMat.ker, ContinuousLinearMap.ker_le_ker_iff_range]
    · simp
    · simp
  · exact sub_le_self 1 hT₂

--Lemma 3 from Hayashi
theorem Lemma3 {ρ : MState d} (ε : Prob) {S : Set (MState d)} (hS₁ : IsCompact S)
  (hS₂ : Convex ℝ (MState.M '' S)) : ∃ σ ∈ S, β_ε(ρ||{σ}) = β_ε(ρ||S) := by

  --Work out the case where S is empty, so we can now assume it's nonempty
  rcases S.eq_empty_or_nonempty with rfl|hnS
  · simp using bot_eq_zero''
  --Upgrade this fact to an instance
  have _ : Nonempty S := hnS.to_subtype

  simp only [OptimalHypothesisRate, Set.mem_singleton_iff, iSup_iSup_eq_left]

  --This parts needs the minimax theorem. Set up the relevant sets and hypotheses.

```

```

--The function `f` will be the `MState.exp_val` function, but bundled as
let f : LinearMap.BilinForm ℝ (HermitianMat d ℂ) := HermitianMat.inner
let S' : Set (HermitianMat d ℂ) := MState.M '' S
let T' : Set (HermitianMat d ℂ) := { m | ρ.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧

have hS'_1 : IsCompact S' := hS_1.image MState.Continuous_HermitianMat

have hT'_1 : IsCompact T' := iInf_IsCompact ρ ε

have hT'_2 : Convex ℝ T' := iInf_IsConvex ρ ε

have hS'_3 : S'.Nonempty := by simpa only [Set.image_nonempty, S']

have hT'_3 : T'.Nonempty := Set.Nonempty.of_subtype

ext1 --turn it from Prob equality into ℝ equality
convert LinearMap.BilinForm.minimax' (M := HermitianMat d ℂ) f S' T' hS'_1
--The remaining twiddling is about moving the casts inside the iInf's and
--In a better world, this would be mostly handled by some clever simps or
· have hi := iSup_range' (ι := S) (fun x ↦ ⌊ (y : T'), (f x) ↑y) (·)
  rw [← Set.image_eq_range] at hi
  rw [← iSup_subtype'', Set.Icc.coe_iSup (zero_le_one), hi]
  congr!
  exact Set.Icc.coe_iInf zero_le_one
· rw [Set.Icc.coe_iInf (ι := T') zero_le_one]
  congr! 2 with y
  have hi := iSup_range' (ι := S) (fun x ↦ (f x) ↑y) (·)
  rw [← Set.image_eq_range] at hi
  rw [← iSup_subtype'', Set.Icc.coe_iSup zero_le_one, hi]
  rfl

--Maybe should be phrased in terms of `0 < ...` instead? Maybe belongs in a
theorem ker_diagonal_prob_eq_bot {q : Prob} (hq_1 : 0 < q) (hq_2 : q < 1) :
  HermitianMat.ker (.diagonal ℂ (ProbDistribution.coin q ·)) = ⊥ := by
  have hA : (Matrix.toLin' (HermitianMat.diagonal ℂ (ProbDistribution.coin
    apply Matrix.PosDef.toLin_ker_eq_bot
    apply Matrix.PosDef.diagonal
    intro i; fin_cases i
  · simpa
  · simpa [← Complex.ofReal_one, Complex.real_lt_real]
  simp [LinearMap.ker_eq_bot', HermitianMat.ker] at hA
  intro m hm
  simpa only [WithLp.ofLp_eq_zero] using hA m congr($hm)

variable {d_2 : Type*} [Fintype d_2] [DecidableEq d_2] in
/-- Lemma S1 -/

```

1.447.

QUANTUMINFO/FINITE/RESOURCE THEORY/HYPOTHESIS TESTING. LEMMA 5

```

theorem optimalHypothesisRate_antitone (p σ : MState d) (ϕ : CPTMap d d₂) (ε : Prob)
  β_ε(p||{σ}) ≤ β_ε(ϕ p||{ϕ σ}) := by
  simp only [of_singleton]
  obtain (ϕDualSubtype, h) :
    ∃ e : ({ m : HermitianMat d₂ ℂ // (ϕ p).exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1} →
      { m : HermitianMat d ℂ // p.exp_val (1 - m) ≤ ε ∧ 0 ≤ m ∧ m ≤ 1}),
    ∀ x, e x = ϕ.hermDual x
    := by
    constructor; swap
    · rintro (m, hm₁, hm₂)
      refine (ϕ.toPTPMap.hermDual m, ?_, PTPMap.hermDual.PTP_POVM ϕ.toPTPMap hm₂)
      have hϕd : (ϕ p).exp_val (1 - m) = p.exp_val (ϕ.hermDual (1 - m)) :=
        ϕ.exp_val_hermDual p (1 - m)
      simpa [hϕd] using hm₁
    · rintro (m, hm₁, hm₂)
      rfl
  convert le_iInf_comp _ ϕDualSubtype
  rename_i T'
  specialize h T'
  rw [h]
  exact ϕ.exp_val_hermDual σ T'

```

open scoped HermitianMat in

open scoped Prob in

-- This is from [Strong converse exponents for a quantum channel discrimination problem and quantum-feedback-assisted communication](<https://doi.org/10.1007/s00220-016-2645-4>), Lemma 5.

It seems like this is actually true for all $0 < \alpha$ (with appropriate modifications at the end) and it for the case of $1 < \alpha$.

--

```

theorem Ref81Lem5 (p σ : MState d) (ε : Prob) (hε : ε < 1) (α : ℝ) (hα : 1 < α) :
  -log β_ε(p||{σ}) ≤ D̃_α(p||σ) + -log (1 - ε) *
    (ofNNReal (α, zero_le_one.trans hα.le)) / (ofNNReal (α - 1, sub_nonneg_of_le α 1))
  := by
  generalize_proofs pf1 pf2
  --If p isn't in the support of σ, the right hand side is just ∞. (The left hand side is 0)
  by_cases h_supp : σ.M.ker ≤ p.M.ker
  swap
  · simp [SandwichedRelEntropy, h_supp, zero_lt_one.trans hα]

```

--Now we know that p.support ≤ σ.support. This is the main case we actually care about.

--Proof from <https://link.springer.com/article/10.1007/s00220-016-2645-4> reproduced below.

--

Lemma 5. Let $\rho, \sigma \in \mathcal{S}(H)$ be such that $\text{supp } \rho \subseteq \text{supp } \sigma$. For any $Q \in \mathcal{B}(H)$ such

that $0 \leq Q \leq I$, and any $\alpha > 1$,

$$-\log \text{Tr}[Q\sigma] \leq \tilde{D}^\alpha(\rho||\sigma) - \alpha / (\alpha-1) * \log \text{Tr}[Q\rho]. \quad (3.7)$$

In particular, for any $\alpha > 1$ and any $\varepsilon \in (0, 1)$,

$$D^{\varepsilon}_H(\rho||\sigma) \leq \tilde{D}^\alpha(\rho||\sigma) + \alpha / (\alpha-1) * \log(1 / (1-\varepsilon)). \quad (3.8)$$

Proof. Let $p \equiv \text{Tr} \{Q\rho\}$ and $q \equiv \text{Tr} \{Q\sigma\}$. By the monotonicity of the sand Rényi relative entropy for $\alpha > 1$, we find that

$$\tilde{D}^\alpha(\rho||\sigma) \geq \tilde{D}^\alpha((p, 1-p) || (q, 1-q)) \quad (3.9)$$

$$= 1 / (\alpha-1) * \log[p^\alpha * q^{(1-\alpha)} + (1-p)^\alpha * (1-q)^{(1-\alpha)}] \quad (3.10)$$

$$\geq 1 / (\alpha-1) * \log[p^\alpha * q^{(1-\alpha)}] \quad (3.11)$$

$$= \alpha / (\alpha-1) * \log p - \log q, \quad (3.12)$$

from which (3.7) follows. The statement in (3.8) follows by optimizing that $\text{Tr} \{Q\rho\} \geq 1 - \varepsilon$.

-/

-- The "monotonicity of the ..." part here refers to the data processing
-- the $(p, 1-p)$ and $(q, 1-q)$ refer to states which are qubits ("coins") of
-- q , respectively. The states ρ and σ can be "processed" into these coins

have $h_2 : 0 < 1 - \varepsilon$.val := by
 change ε .val < 1 at $h\varepsilon$
 linarith

let $p : \text{Prob} := 1 - \varepsilon$
 set $q : \text{Prob} := \beta_\varepsilon(\rho||\{\sigma\})$
 let $p2 : \text{MState} (\text{Fin } 2) := .\text{ofClassical} <| .\text{coin } p$
 let $q2 : \text{MState} (\text{Fin } 2) := .\text{ofClassical} <| .\text{coin } q$

--Show there's a lower bound on β_ε , that you can't do perfect discrimination
--It's possible that we actually don't want this here, that it should "follow"
--from the main proof.

have $h_q : 0 < q := \text{pos_of_lt_one } \{\sigma\} \langle \sigma, \text{rfl}, h_{\text{supp}} \rangle h\varepsilon$

suffices $-\log q \leq \tilde{D}_\alpha(p2||q2) + -\log(1 - \varepsilon) * (.\text{ofNNReal } (\alpha, pf1)) / (. . .)$
 refine this.trans (add_le_add_left ?_ _)

--Show that this is an instance of the Data Processing Inequality

obtain $\langle \Phi, h\Phi_1, h\Phi_2 \rangle : \exists (\Phi : \text{CPTPMap } d (\text{Fin } 2)), p2 = \Phi \rho \wedge q2 = \Phi \sigma :$
 --The relevant map here is to take the T that optimizes inside β_ε (Φ)
 --and use the projective measurement onto $\{T, 1 - T\}$. This achieves the
 --discrimination rate on σ , and so gives the outcome distribution $q2$.
 --is tight on the bound that $\text{Tr}[p T] = \varepsilon$.

--Get the measurement operator T .
 obtain $\langle T, hT_1, hT_2 \rangle := \text{exists_min } p \varepsilon \{\sigma\}$
 simp only [Set.mem_singleton_iff, iSup_iSup_eq_left] at hT_1
 --Turn it into a POVM (probably want to have lemmas around this ideal)
 let $\Lambda : \text{POVM} (\text{Fin } 2) d := \{$
 mats $i := \text{if } i = 0 \text{ then } T \text{ else } 1 - T$

1.447.

QUANTUMINFO/FINITE/RESOURCE THEORY/HYPOTHESISTESTING.LEAN

```

    nonneg i := by
      split
      · exact T.prop.2.1
      · exact HermitianMat.zero_le_iff.mpr T.prop.2.2
    normalized := by simp
  }
  use  $\Lambda$ .measureDiscard
  simp only [POVM.measureDiscard_apply, p2, q2]
  constructor
  · congr
    rw [ProbDistribution.coin_eq_iff]
    ext
    dsimp [MState.exp_val] at hT2
    simp [POVM.measure,  $\Lambda$ , p, ProbDistribution.mk', coe_one_minus,  $\leftarrow$  hT2, HermitianMat.zero_le_iff]
  · congr
    rw [ProbDistribution.coin_eq_iff]
    ext
    dsimp [POVM.measure,  $\Lambda$ , q]
    rw [ $\leftarrow$  hT1]
    exact HermitianMat.inner_comm _ _
  rw [h $\Phi_1$ , h $\Phi_2$ ]
  exact sandwichedRenyiEntropy_DPI_ax h $\alpha$ .le  $\rho$   $\sigma$   $\Phi$ 

--If q = 1, this inequality is trivial
by_cases hq2 : q = 1
· rw [hq2]
  simp

replace hq2 : q < 1 := show q.val < 1 by
  linarith +splitNe [q.coe_le_one, unitInterval.coe_ne_one.mpr hq2]

--The Renyi entropy is finite
rw [SandwichedRelRentropy, dif_pos (zero_lt_one.trans h $\alpha$ ), dif_pos ?_]; swap
· suffices q2.M.ker = 1 by
  simp only [this, bot_le]
  --q2 has eigenvalues  $\beta_{- \epsilon}(\rho||\{\sigma\})$  and  $1-\beta_{- \epsilon}(\rho||\{\sigma\})$ , so as long as  $\beta_{- \epsilon}(\rho||\{\sigma\})$  is
  --this is true.
  exact ker_diagonal_prob_eq_bot hq hq2

conv => enter [2, 1, 1, 1]; rw [if_neg h $\alpha$ .ne']

--The logs are finite
rw [Prob.negLog, Prob.negLog, if_neg hq.ne']
rw [if_neg (show 1 -  $\epsilon \neq 0$  by simp [Subtype.ext_iff, Prob.coe_sub] using h2.ne')]

```

```

--Turn the ENNReal problem into a Real problem
have hα₂ : Subtype.mk _ pf2 ≠ 0 := by
  change ¬( _ = Subtype.mk 0 _ )
  simp only [mk_zero, Nonneg.mk_eq_zero]
  linarith
rw [← ENNReal.coe_mul, ← ENNReal.coe_div hα₂, ← ENNReal.coe_add, ENNReal.coe_sub]
clear hα₂
simp only [← coe_le_coe, coe_mk, NNReal.coe_add, NNReal.coe_div, NNReal.coe_sub]
clear pf1 pf2

rw [← add_div, ← sub_eq_add_neg]
conv =>
  enter [2,1,1,1]
  equals (p^α * q^(1-α) + (1-p)^α * (1-q)^(1-α) : ℝ) =>
    unfold q2
    rw [MState.ofClassical_pow]
    unfold p2
    rw [MState.coe_ofClassical]
    rw [HermitianMat.diagonal_conj_diagonal, HermitianMat.diagonal_pow]
    rw [HermitianMat.trace_diagonal]
    simp only [Fin.sum_univ_two, Fin.isValue, ProbDistribution.coin_val_zero,
      ProbDistribution.coin_val_one, Prob.coe_one_minus]
    rw [Real.mul_rpow p.zero_le (by positivity)]
    rw [← Real.rpow_natCast_mul (by bound)]
    rw [← Real.rpow_mul q.zero_le]
    rw [Real.mul_rpow (by bound) (by positivity)]
    rw [← Real.rpow_natCast_mul (by bound)]
    rw [← Real.rpow_mul (by bound)]
    congr <=> simp [field]

by_cases h : ε = 0
· simp [h, p, @Real.zero_rpow α (by positivity)]
  apply Eq.le
  rw [Real.log_rpow hq]
  have : α - 1 ≠ 0 := by linarith
  field_simp
  ring_nf

have hp : 0 < p := show (0 : ℝ) < p by simp [p, hε]

replace h : 0 < ε := zero_lt_iff.mpr h
trans (Real.log (p ^ α * q ^ (1 - α)) - Real.log (1 - ε.val) * α) / (α - 1)
· rw [Real.log_mul]
  rotate_left
  · exact (Real.rpow_pos_of_pos hp _).ne'
  · exact (Real.rpow_pos_of_pos hq _).ne'

```


1.447.

QUANTUMINFO/FINITE/RESOURCE THEORY/HYPOTHESIS TESTING. LEARN3

```

    simp only [p, Prob.coe_one_minus]
    rw [Real.log_rpow (by linarith), mul_comm α, add_sub_cancel_left]
    rw [Real.log_rpow (x := q.val) hq]
    rw [mul_comm, ← mul_div, mul_comm, show (1 - α) = -(α - 1) by abel]
    simp [-neg_sub, neg_div, div_self (a := α - 1) (by linarith)]
    · rw [div_le_div_iff_of_pos_right (by linarith), tsub_le_iff_right]
    nth_rewrite 4 [Prob.coe_sub]
    simp only [Set.Icc.coe_one, sub_nonneg, Prob.coe_le_one, sup_of_le_left, sub_add]
    apply Real.log_le_log
    · refine mul_pos (Real.rpow_pos_of_pos hp _) (Real.rpow_pos_of_pos hq _)
    rw [le_add_iff_nonneg_right]
    refine mul_nonneg (Real.rpow_nonneg ?_ _) (Real.rpow_nonneg ?_ _)
    · exact sub_nonneg_of_le p.2.2
    · exact sub_nonneg_of_le q.2.2

theorem rate_pos_of_smul_pos {ε : Prob} {d : Type*} [Fintype d] [DecidableEq d] {ρ σ
  (hσ₂ : 0 < β_ε(ρ||{σ₂})) {c : ℝ} (hc : 0 < c) (hσ : c • σ₂ ≤ σ₁.M) : 0 < β_ε(ρ||
  simp only [of_singleton, lt_iInf_iff] at hσ₂ ⊢
  rcases hσ₂ with ⟨b, _, hb_le⟩, hb_pos, hb)
  change 0 < b at hb_pos --TODO simp thm / lemma
  use ⟨(min c 1) * b, by positivity, by bound⟩
  constructor
  · change 0 < (min c 1) * b --TODO simp thm / lemma
    positivity
  intro i
  specialize hb i
  rw [Subtype.mk_le_mk, MState.exp_val] at hb ⊢
  replace hb : c * b ≤ □c • σ₂.M, i□ := by
    rwa [← mul_le_mul_iff_of_pos_left hc, ← HermitianMat.inner_smul_left] at hb
  grw [min_le_left]
  refine hb.trans (HermitianMat.inner_mono' i.2.2.1 hσ)

@[fun_prop]
theorem rate_Continuous_singleton {ε : Prob} {d : Type*} [Fintype d] [DecidableEq d]
  Continuous fun σ ↦ β_ε(ρ||{σ}) := by
  have h := HermitianMat.inner_.flip.continuous_iInf_fst
    (S := { m | ρ.exp_val (1 - m) ≤ ↑ε ∧ 0 ≤ m ∧ m ≤ 1 })
    ((Metric.isBounded_Icc 0 1).subset (Set.setOf_subset_setOf_of_imp fun _ ↦ And.r
  simp only [of_singleton]
  conv => enter [1, σ]; rw [subtype_val_iInf']
  exact Continuous.subtype_mk (h.comp MState.Continuous_HermitianMat) _

/-- On the 1D Hilbert space, the optimal hypothesis testing rate is simply 1 - ε,
since there's nothing to learn. (More generally this would hold whenever ρ=σ.) --
/
theorem optimalHypothesisRate_unique {d : Type*} [Fintype d] [DecidableEq d]

```

```

    (ε : Prob) (ρ σ : MState d) [Unique d] : β_ ε(ρ||{σ}) = 1 - ε := by
  obtain rfl := Unique.eq_default ρ
  obtain rfl := Unique.eq_default σ
  rw [OptimalHypothesisRate.of_singleton]
  apply le_antisymm
  · refine iInf_le_of_le (((1 - ε : Prob) : ℝ) • 1, {?, ?, ?}) ?_
    · simp [MState.exp_val_sub]
    · apply smul_nonneg ?_ zero_le_one
      simp
    · apply smul_le_of_le_one_left zero_le_one
      simp
    · simp [-Prob.coe_one_minus]
  · simp
    intro a he1 ha0 ha1
    rw [MState.exp_val_sub, MState.exp_val_one, tsub_le_iff_right] at he1
    rw [← tsub_le_iff_left, ← Prob.coe_one_minus] at he1
    exact he1

```

1.448 QuantumInfo/Finite/ResourceTheory/ResourceTheory.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg, Leonardo A. Lessa, Rodolfo R. Soldati
-/
import QuantumInfo.Finite.ResourceTheory.FreeState

/-- A quantum resource theory extends a `FreeStateTheory` with a collection
required that any state we can always prepare for free must be free, and the
theory is "minimal", but we can have a more restricted set of operations. -/
/
class ResourceTheory (ι : Type*) extends FreeStateTheory ι where
  freeOps i j : Set (CPTPMap (H i) (H j))
  /-- Free operations in a ResourceTheory are nongenerating: they only crea
  /
  nongenerating {i j : ι} {f} (h : f ∈ freeOps i j) : ∀ ρ, IsFree ρ → IsFree
  --We might need to require some more closure properties on `freeOps`, lik
  --For now we just require that they include the identity and composition,
  /-- The identity operation is free -/
  free_id i : CPTPMap.id ∈ freeOps i i
  /-- Free operations are closed under composition -/
  free_comp {i j k} (Y : freeOps j k) (X : freeOps i j) : Y.1.compose X.1 ∈

namespace ResourceTheory

```

1.448.

QUANTUMINFO/FINITE/RESOURCE THEORY/RESOURCE THEORY. LEAN 485

```
open ResourcePretheory
open FreeStateTheory
```

```
variable {ι : Type*}
```

```
/-- Given a `FreeStateTheory`, there is a maximal set of free operations compatible
That is the set of all operations that don't generate non-
free states from free states. We
call this the maximal resource theory. -/
```

```
def maximal [FreeStateTheory ι] : ResourceTheory ι where
  freeOps i j := { f | ∀ ρ, IsFree ρ → IsFree (f ρ) }
  nongenerating := id
  free_id _ _ := by rwa [CPTPMap.id_MState]
  free_comp f g ρ h := f.prop _ (g.prop ρ h)
```

```
/-- A resource theory `IsMaximal` if it includes all non-
generating operations. -/
```

```
def IsMaximal (r : ResourceTheory ι) : Prop :=
  ∀ (i j), r.freeOps i j = { f | ∀ ρ, IsFree ρ → IsFree (f ρ) }
```

```
/-- A resource theory `IsTensorial` if it includes tensor products of operations, cr
free states, and discarding. This implies that includes a unit object. -
```

```
/
```

```
structure IsTensorial [UnitalPretheory ι] : Prop where
  prod : ∀ {i j k l : ι} {f g}, f ∈ freeOps i k → g ∈ freeOps j l → (f ⊗CPTP g) ∈ fr
  create : ∀ {i : ι} (ρ), IsFree ρ → CPTPMap.replacement ρ ∈ freeOps Unital.unit i
  destroy : ∀ (i : ι), CPTPMap.destroy ∈ freeOps i Unital.unit
```

```
/-- The theory `ResourceTheory.maximal` always `IsMaximal`. -
```

```
/
```

```
theorem maximal_IsMaximal [FreeStateTheory ι] : IsMaximal (maximal (ι := ι)) :=
  fun _ _ => rfl
```

```
-- --Helper theorem for ResourceTheory.mk_of_ops
```

```
-- private lemma convex_states_of_convex_ops [ResourcePretheory ι] (O : ∀ (i j : ι),
-- (h_convex : ∀ {i j}, Convex ℝ (CPTPMap.choi ' ' 0 i j)) (i : ι) :
--   Convex ℝ (MState.M ' ' fun ρ => ∀ {j} σ, ∃ f, 0 ≤ f ∧ f σ = ρ)) := by
--   intro _ hx _ hy a b ha hb hab
--   rw [Set.mem_image] at hx hy
--   obtain ⟨x, hx1, hx2⟩ := hx
--   obtain ⟨y, hy1, hy2⟩ := hy
--   use Mixable.mix_ab ha hb hab x y
--   constructor
--   · change forall _, _
--     intro j σ
--     obtain ⟨fx, ⟨hfx1, hfx2⟩⟩ := hx1 σ
```

```

--      obtain {fy,{hfy1,hfy2}} := hyl σ
--      use Mixable.mix_ab ha hb hab fx fy
--      constructor
--      · specialize h_convex {fx, hfx1, rfl} {fy, hfy1, rfl} ha hb hab
--      rw [Set.mem_image] at h_convex
--      obtain {w,hw1,hw2} := h_convex
--      have : w = Mixable.mix_ab ha hb hab fx fy := by
--        apply CPTPMap.choi_ext
--        convert hw2
--        --Should be a theorem
--        simp only [Mixable.mix_ab, Mixable.mkT]
--        exact CPTPMap.choi_of_CPTP_of_choi (a • fx.choi + b • fy.choi)
--      exact this ▸ hw1
--      · --Should be a theorem about CPTPMap.instMixable, really. Also, thi
--      subst x y
--      simp only [Mixable.mix_ab, Mixable.mkT, MState.instMixable, CPTPMap
--      CPTPMap.CPTP_of_choi_PSD_Tr, CPTPMap.mk, MatrixMap.of_choi_matri
--      ext
--      change (Finset.sum _) = ((_ : ℂ) + _)
--      simp only [Matrix.add_apply, Matrix.smul_apply, Complex.real_smul]
--      simp_rw [mul_add, Finset.sum_add_distrib]
--      congr
--      · sorry
--      · sorry
--      · rw [Mixable.mix_ab, Mixable.mkT, MState.instMixable, ← hx2, ← hy2]
--      rfl

-- /-- A `ResourceTheory` can be constructed from a set of operations (sati
-- and then the free states are taken to be the set of states that can be p
-- -/
-- def mk_of_ops [ResourcePretheory ι] (O : ∀ (i j : ι), Set (CPTPMap (H i)
-- (h_id : ∀ i, CPTPMap.id ∈ O i i) --Operations include identity
-- (h_comp : ∀ {i j k} (Y : O j k) (X : O i j), Y.1.compose X.1 ∈ O i k)
Operations include compositions
-- (h_closed : ∀ {i j}, IsClosed (O i j)) -- Operations are topological
-- (h_convex : ∀ {i j}, Convex ℝ (CPTPMap.choi '' O i j)) -- (The choi
-- (h_prod : ∀ {i j k l f g} (hf : f ∈ O i k) (hg : g ∈ O j l), (f ⊗ℂ pr
Closed under products
-- (h_fullRank : ∀ {i : ι}, sorry) --Some statement about having full r
-- (h_appendFree : ∀ {i j k : ι}, sorry) --Some statement that appendin
-- : ResourceTheory ι where
-- freeOps := O
-- free_id := h_id
-- free_comp := h_comp

-- IsFree {i} ρ := ∀ {j} σ, ∃ f, O j i f ∧ f σ = ρ

```

1.448.

QUANTUMINFO/FINITE/RESOURCE THEORY/RESOURCE THEORY. LEAN 487

```
-- free_closed := sorry
-- free_convex {i} := convex_states_of_convex_ops 0 @h_convex i
-- free_prod {i j ρ σ} hp hσ k ρ₀ := by
--   obtain ⟨f,hf1⟩ := hp ρ
--   obtain ⟨g,hg⟩ := hσ σ
--   sorry
-- free_fullRank := sorry
-- nongenerating := by
--   intro i j f hf ρ hFree k σ
--   obtain ⟨g,hg1,hg2⟩ := hFree σ
--   use f.compose g
--   constructor
--   · exact h_comp ⟨f,hf⟩ ⟨g,hg1⟩
--   · simp only [CPTPMap.compose_eq, hg2]

-- /-- A `ResourceTheory` provides a category structure -
-- /
-- instance instQRTCATEGORY (ι : Type*) [ResourceTheory ι] : CategoryTheory.Category
--   Hom x y := freeOps x y
--   id := fun _ => ⟨CPTPMap.id, free_id _⟩
--   comp f g := ⟨CPTPMap.compose g.1 f.1, free_comp g f⟩
--   id_comp X := by simp
--   comp_id := by simp
--   assoc := fun f g h => by simp using CPTPMap.compose_assoc h.1 g.1 f.1

-- open ComplexOrder in
-- /-- The 'fully free' quantum resource theory: the category is all finite Hilbert
-- free and all states are free. Marked noncomputable because we use `Fintype.ofFinite`
-- /
-- noncomputable def fullyFreeQRT : ResourceTheory { ι : Type // Finite ι ∧ Nonempty
--   H := Subtype.val
--   FinH := fun i => have := i.prop.left; Fintype.ofFinite i
--   DecEqH := fun i a b => Classical.propDecidable (a = b)
--   NonemptyH := fun i => i.prop.right

--   prod := fun ⟨i,⟨hi,hi2⟩⟩ ⟨j,⟨hj,hj2⟩⟩ => ⟨i × j, ⟨Finite.instProd, instNonempt
--   prodEquiv := fun ⟨_,⟨_,_⟩⟩ ⟨_,⟨_,_⟩⟩ => Equiv.refl _

--   IsFree := Set.univ
--   free_closed := isClosed_univ
--   free_convex {i} := by
--     -- convert MState.convex (H i) --For MState.m, not MState.M
--     sorry
--   free_prod _ _ := trivial
--   free_fullRank := by
```

```

--      intro i
--      have := i.prop.left
--      have := i.prop.right
--      let _ := Fintype.ofFinite i
--      let _ : DecidableEq i := fun _ _ => Classical.propDecidable _
--      use MState.uniform --use the fully mixed state
--      --The fact that the fully mixed state is PosDef should be stated s
--      suffices Matrix.PosDef (@MState.uniform (d := i.val) _ _ this).M.t
--      change _ ^ True
--      rw [and_true]
--      exact this
--      simp only [MState.uniform, MState.ofClassical, Distribution.unifor
--      classical apply Matrix.PosDef.diagonal
--      intro
--      rw [Finset.card_univ, one_div, Complex.ofReal_inv]
--      exact RCLike.inv_pos_of_pos (Nat.cast_pos'.mpr Fintype.card_pos)

--      freeOps _ _ := Set.univ
--      nongenerating _ _ _ := trivial
--      free_id := Set.mem_univ
--      free_comp _ _ := Set.mem_univ _

-- end ResourceTheory

```

1.449 QuantumInfo/Finite/ResourceTheory/SteinsLemma.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg, Leonardo A. Lessa, Rodolfo R. Soldati
-/
import QuantumInfo.Finite.ResourceTheory.FreeState
import QuantumInfo.Finite.ResourceTheory.HypothesisTesting
import QuantumInfo.Finite.Pinching
import QuantumInfo.ForMathlib.Matrix
import QuantumInfo.ForMathlib.LimSupInf
import QuantumInfo.ForMathlib.HermitianMat
import QuantumInfo.ForMathlib.HermitianMat.Jordan

import Mathlib.Tactic.Bound

open NNReal
open scoped ENNReal
open ComplexOrder

```

```

open Topology
open scoped Prob
open scoped OptimalHypothesisRate
open ResourcePretheory
open FreeStateTheory
open UnitalPretheory
open UnitalFreeStateTheory
open scoped RealInnerProductSpace InnerProductSpace

namespace SteinsLemma

variable {ι : Type*} [UnitalFreeStateTheory ι]
variable {i : ι}

/-- The  $\tilde{\sigma}_n$  defined in Lemma 6, also in equation (S40) in Lemma 7. -
/
noncomputable def Lemma6_on (m : ℕ) (σf : MState (H i)) (σm : MState (H (i ^ m))) :
  fun n ↦ (σm ⊗r^[n / m] ⊗r σf ⊗r^[n % m]).relabel <| .cast <| congrArg H (by
    rw [← pow_mul, ← spacePow_add, Nat.div_add_mod n m]
  )

theorem Lemma6_on_IsFree {σ1 : MState (H i)} {σm : (m : ℕ) → MState (H (i ^ m))} (hσm :
  (hσm : ∀ (m : ℕ), σm m ∈ IsFree) (m n : ℕ) : Lemma6_on m σ1 (σm m) n ∈ IsFree :=
  rw [Lemma6_on, relabel_cast_isFree]
  · apply free_prod --pick a better name / alias for this
    · exact (hσm m).npow (n / m)
    · exact hσ1_free.npow (n % m)
  · rw [← pow_mul, ← spacePow_add, Nat.div_add_mod n m]

/-- Lemma 6 from the paper.
We _did_ end up doing the version that "works also in the case of  $\varepsilon = 0$ ", which is n
-/
private theorem Lemma6 {m : ℕ} (hm : 0 < m) (ρ σf : MState (H i)) (σm : MState (H (i
  [σf.M.NonSingular] {ε : Prob} (hε : ε < 1) :
  Filter.atTop.limsup (fun n ↦ -log β_ ε(ρ ⊗r^[n]||{Lemma6_on m σf σm n}) / n) ≤ [](ρ
:= by

set on := Lemma6_on m σf σm with hσn

have h_add : ∀ α n,  $\tilde{D}_\alpha(\rho \otimes_r^n || \sigma_n) = (n/m : \mathbb{N}) * \tilde{D}_\alpha(\rho \otimes_r^m || \sigma_m) + (n \% m :$ 
  --"Break apart" on, and apply additivity of `SandwichedRelEntropy`.
  intro α n
  rw [hσn, Lemma6_on]
  have hnm_add := Nat.div_add_mod n m
  rw [statePow_rw hnm_add.symm, statePow_add_relabel]
  have hnm_eq :  $(i^{(m * (n / m))} * i^{(n \% m)}) = (i^m)^{(n / m)} * i^{(n \% m)}$ 

```

```

    rw [pow_mul]
    have h_Hn_eq : H (i ^ n) = H ((i ^ m) ^ (n / m) * i ^ (n % m)) := by
      rw [← pow_mul, ← pow_add, hnm_add]
    simp only [MState.relabel_relabel, ← Equiv.cast_trans]
    rw [← sandwichedRelEntropy_statePow]
    rw [← sandwichedRelEntropy_statePow]
    rw [← sandwichedRelEntropy_prodReLabel]

gcongr
· rw [MState.eq_relabel_iff]
  simp only [MState.relabel_relabel, ← Equiv.cast_symm, ← Equiv.cast_tr
  rw [prodReLabel_relabel_cast_prod _ _ _ ((pow_mul ..).symm) rfl]
  congr
  rw [statePow_mul_relabel]
  simp [← Equiv.cast_trans]
· simp

--This will probably need  $1 < \alpha$  actually
have h_α : ∀ α, (1 < α) → Filter.atTop.limsup (fun n ↦ -log β_ε(p ⊗_r^n[n]||
  ð_α(p ⊗_r^n[m]||σ_m)) / m := by
  intro α hα
  apply le_of_le_of_eq (b := Filter.atTop.limsup (fun n ↦ ð_α(p ⊗_r^n[n]||
· --Apply the "[81] Lemma 5" to p⊗^n and σn
  have h_lem5 (n) := OptimalHypothesisRate.Ref81Lem5 (p ⊗_r^n[n]) (σn n)

--Upper-bound β on the LHS with this lemma
--Distribute the limsup over subtraction
--The term on the right is a constant, divided by n, which converges
--Dropping that leaves the identity
generalize_proofs pf1 pf2 at h_lem5
let x n := -log β_ε(p ⊗_r^n[n]||{σn n})
let y n := ð_α(p ⊗_r^n[n]||σn n)
set z := -log (1 - ε) * (ENNReal.ofNNReal ⟨α, pf1⟩) / (ENNReal.ofNNRe

have hz : z ≠ τ := by
  unfold z
  have hz1 : -log (1 - ε) ≠ τ := by
    --TODO: should be `bound`, ideally
    simp [Subtype.ext_iff]
    have : (ε : ℝ) < 1 := hε
    linarith
  have hz2 : (ENNReal.ofNNReal ⟨α - 1, pf2⟩) ≠ 0 := by
    --TODO: should be `bound`, ideally
    simp [NNReal.eq_iff]
    linarith
  finiteness

```



```

change ∀ n, x n ≤ y n + z at h_lem5
change Filter.atTop.limsup (fun n ↦ x n / n) ≤ Filter.atTop.limsup (fun n ↦ y n / n) at h_lem5
exact extracted_limsup_inequality z hz y x h_lem5

· suffices Filter.atTop.Tendsto (fun n ↦  $\tilde{D}_\alpha(\rho \otimes_r^n \| \sigma_n) / n$ ) (□) ( $\tilde{D}_\alpha(\rho \otimes_r \| \sigma)$ )
  exact this.limsup_eq
conv =>
  enter [1,n]
  equals ((↑(n / m) *  $\tilde{D}_\alpha(\rho \otimes_r^m \| \sigma_m)$ ) / n + (↑(n % m) *  $\tilde{D}_\alpha(\rho \| \sigma)$ ) / n) =
    simp_rw [h_add, ENNReal.add_div]
conv => enter [3,1]; apply (add_zero _).symm
apply Filter.Tendsto.add
· simp_rw [div_eq_mul_inv, mul_comm, ← mul_assoc]
  conv =>
    enter [3,1]
    apply (one_mul _).symm
  rw [← mul_assoc]
  cases  $\tilde{D}_\alpha(\rho \otimes_r^m \| \sigma_m)$ 
  · simp
    -- This is true for all x past m.
    apply tendsto_nhds_of_eventually_eq
    refine Filter.eventually_atTop.mpr ?_
    use m
    intros
    rw [ENNReal.mul_top]
    apply (ENNReal.mul_pos ?_ ?_).ne'
    · simp only [ne_eq, ENNReal.inv_eq_zero, ENNReal.natCast_ne_top, not_false_eq_true]
    · simp
      omega
  · rename_i v
    suffices Filter.atTop.Tendsto (fun x ↦  $(x : \mathbb{R})^{-1} * \uparrow(x / m) * (v : \mathbb{R}) : \mathbb{N} \rightarrow \mathbb{R}$ )
      -- Similar to the "convert ENNReal.tendsto_ofReal this" below. Just push
      convert ENNReal.tendsto_ofReal this
    · rename_i x
      cases x
      · simp
        rw [ENNReal.ofReal_mul (by positivity), ENNReal.ofReal_mul (by positivity)]
        simp
        norm_cast
      · rw [ENNReal.ofReal_mul (by positivity), one_div, ENNReal.ofReal_inv_ofReal]
        simp
        exact (Filter.Tendsto_inv_nat_mul_div_real m).mul tendsto_const_nhds
  · suffices Filter.atTop.Tendsto (fun x ↦  $(x \% m : \mathbb{N}) * (\tilde{D}_\alpha(\rho \| \sigma))$ ).toReal / x
    -- Convert a Tendsto over ENNReal to one over Real
    convert ENNReal.tendsto_ofReal this

```

```

      · rename_i x
      cases x
      · simp
      rw [ENNReal.ofReal_div_of_pos (by positivity), ENNReal.ofReal_m
congr
      · simp
      · rw [ENNReal.ofReal_toReal (by finiteness)]
      · rw [ENNReal.ofReal_natCast]
      · simp
    apply bdd_le_mul_tendsto_zero (b := 0) (B := m *  $\tilde{D}_\alpha(p||\sigma)$ .toReal)
      · exact Filter.Eventually.of_forall (fun _ => by positivity)
      · apply Filter.Eventually.of_forall (fun _ => ?_)
      exact mul_le_mul_of_nonneg_right (mod_cast (Nat.mod_lt _ hm).le)
      · exact tendsto_inv_atTop_nhds_zero_nat

--Take the limit as  $\alpha \rightarrow 1$ .
replace h_α : Filter.atTop.limsup (fun n =>  $-\log \beta_\epsilon(p \otimes_r^n || \sigma_n)$ ) / n
  refine ge_of_tendsto (x := ( $\square[>]$  1)) ?_ (eventually_nhdsWithin_of_forall)
  apply tendsto_nhdsWithin_of_tendsto_nhds
  convert ContinuousAt.tendsto ?_ using 3
  have _ := ENNReal.continuous_div_const m (by positivity)
  have _ := (sandwichedRelEntropy.continuousOn ( $p \otimes_r^m$ )  $\sigma_m$ ).continuous
  fun_prop

exact h_α

section Lemma7

open MatrixMap
open Matrix
open PosSemidef

-- TODO: Commutation and order relations about `proj_le` specified in the t
-- between Eqs. (S77) and (S78)

open scoped HermitianMat in
theorem LemmaS2liminf {ε3 : Prob} {ε4 :  $\mathbb{R}_{\geq 0}$ } (hε4 :  $0 < \epsilon_4$ )
  {d :  $\mathbb{N} \rightarrow \text{Type}^*$ } [∀ n, Fintype (d n)] [∀ n, DecidableEq (d n)] (ρ : (n :  $\mathbb{N}$ )
  {Rinf :  $\mathbb{R}_{\geq 0}$ } (hRinf : Rinf  $\geq$  Filter.atTop.liminf (fun (n :  $\mathbb{N}$ ) =>  $-\log \beta_\epsilon$ 
  :
  (Filter.atTop.liminf (fun (n :  $\mathbb{N}$ ) =>  $\square\{(p \ n).M \geq_p (\text{Real.exp } (n * (Rinf + \epsilon$ 
  := by
  by_contra h
  push_neg at h
  replace h := Filter.eventually_lt_of_lt_liminf h ?_
  · replace h := Filter.eventually_atTop.mp h

```

```

obtain ⟨n₀, h⟩ := h
--Can assume that n₀ is positive. Then we don't have to worry about nonzero values
wlog hn₀ : 0 < n₀
· exact this hε4 ρ σ hRinf 1 (fun b hb ↦ h _ <| by omega) zero_lt_one
let T (n : ℕ) := {(ρ n).M ≥ₚ (Real.exp (n * (Rinf + ε4))) • (σ n).M}
have hT : ∀ n ≥ n₀, (ρ n).exp_val (1 - (T n)) ≤ ε3 := fun n hn ↦ by -- Eq (S23)
  unfold MState.exp_val T
  rw [inner_sub_right, HermitianMat.inner_one, MState.tr,
    HermitianMat.inner_comm, tsub_le_iff_right, add_comm, ← tsub_le_iff_right]
  apply le_of_lt
  exact h n hn
have hβ : ∀ n ≥ n₀, β_ε3(ρ n||{σ n}) ≤ Real.exp (-n * (Rinf + ε4)) := fun n hn ↦
  open HermitianMat in
  calc
    β_ε3(ρ n||{σ n}) ≤ (σ n).exp_val (T n) := by
      have hβ' := OptimalHypothesisRate.singleton_le_exp_val (σ := σ n) (T n) (h
        simp only [Subtype.coe_le_coe.mpr hβ'])
      _ ≤ [T n, Real.exp (-n * (Rinf + ε4))] • (ρ n).M := by
        rw [← mul_le_mul_iff_right₀ (Real.exp_pos ((n * (Rinf + ε4))))], HermitianM
        simp only [isUnit_iff_ne_zero, ne_eq, Real.exp_ne_zero, not_false_eq_true,
          IsUnit.mul_inv_cancel_left]
        rw [MState.exp_val, HermitianMat.inner_comm, ← HermitianMat.inner_smul_right]
        unfold T
        exact projLE_inner_le (Real.exp (n * (Rinf + ε4))) • (σ n).M (ρ n).M
      _ ≤ Real.exp (-n * (Rinf + ε4)) := by
        simp [HermitianMat.inner_smul_right]
        rw [mul_comm]
        apply (mul_le_iff_le_one_left (Real.exp_pos (-(n * (Rinf + ε4))))).mpr
        rw [HermitianMat.inner_comm, ← MState.exp_val]
        exact (ρ n).exp_val_le_one (projLE_le_one _ _)
have h' : ∀ n ≥ n₀, Rinf + ε4 ≤ -log β_ε3(ρ n||{σ n}) / n := fun n hn ↦ by -- Eq
  have : 0 < n := by order
  have hn1 : (n : ℝ≥0∞) ≠ 0 := by positivity
  have hn2 : (n : ℝ≥0∞) ≠ ∞ := by finiteness
  have hh : n * (Rinf + ε4) = ENNReal.ofReal (n * (Rinf + ε4)) := by
    simp only [Nat.cast_nonneg, ENNReal.ofReal_mul, ENNReal.ofReal_natCast, zero
      ENNReal.ofReal_add, ENNReal.ofReal_coe_nnreal]
  apply (ENNReal.mul_le_mul_iff_right (a := n) (b := Rinf + ε4) (c := -log β_ε3
    rw [ENNReal.mul_div_cancel hn1 hn2, hh]
  apply Prob.le_negLog_of_le_exp
  rw [← neg_mul]
  exact hβ n hn
have hf : ∀ᶜ (n : ℕ) in Filter.atTop, Rinf + ε4 ≤ -log β_ε3(ρ n||{σ n}) / n := b
  rw [Filter.eventually_atTop]
  use n₀
replace hf := Filter.le_liminf_of_le ?_ hf

```

```

· replace hf := le_trans hf hRinf
  replace hf := tsub_eq_zero_iff_le.mpr hf
  simp_all
  apply Filter.IsCobounded.of_frequently_le (u :=  $\tau$ )
  simp [Filter.frequently_atTop]
  intro n; use n
  apply Filter.isBoundedUnder_of
  use 0; intro n
  rw [HermitianMat.inner_comm, ← MState.exp_val]
  exact ( $\rho$  n).exp_val_nonneg ((Real.exp (n * (Rinf +  $\epsilon_4$ ))) • ( $\sigma$  n).M).projLE

open scoped HermitianMat in
theorem LemmaS2limsup { $\epsilon_3$  : Prob} { $\epsilon_4$  :  $\mathbb{R}_{\geq 0}$ } (h $\epsilon_4$  :  $0 < \epsilon_4$ )
  {d :  $\mathbb{N} \rightarrow \text{Type}^*$ } [V n, Fintype (d n)] [V n, DecidableEq (d n)] ( $\rho$  : ( $n$  :  $\mathbb{N}$ ) →  $\mathbb{R}_{\geq 0}$ )
  {Rsup :  $\mathbb{R}_{\geq 0}$ } (hRsup : Rsup  $\geq$  Filter.atTop.limsup (fun (n :  $\mathbb{N}$ ) ↦  $-\log \beta_n$ )) :
  :
  (Filter.atTop.limsup (fun (n :  $\mathbb{N}$ ) ↦  $\square \{ \rho n \geq_p (\text{Real.exp } (n * (\text{Rsup} + \epsilon_4))) \}$ ))
  := by
  by_contra h
  push_neg at h
  replace h := Filter.frequently_lt_of_lt_limsup ?_ h
  · replace h := Filter.frequently_atTop.mp h
    let T (n :  $\mathbb{N}$ ) := {( $\rho$  n).M  $\geq_p (\text{Real.exp } (n * (\text{Rsup} + \epsilon_4)))$  • ( $\sigma$  n).M}
    have hT (n0) :  $\exists n \geq n_0, (\rho n).exp\_val (1 - (T n)) \leq \epsilon_3$  := by -- Eq (S3)
      obtain ⟨n, hn, h⟩ := h n0
      use n, hn
      unfold MState.exp_val T
      rw [inner_sub_right, HermitianMat.inner_one, MState.tr,
        HermitianMat.inner_comm, tsub_le_iff_right, add_comm, ← tsub_le_iff]
      apply le_of_lt
      exact h
    have h $\beta$  (n0) :  $\exists n \geq n_0, \beta_n \epsilon_3 (\rho n || \{\sigma n\}) \leq \text{Real.exp } (-$ 
n * (Rsup +  $\epsilon_4$ )) := by -- Eq (S32)
      obtain ⟨n, hn, hT⟩ := hT n0
      use n, hn
      open HermitianMat in
      calc
         $\beta_n \epsilon_3 (\rho n || \{\sigma n\}) \leq (\sigma n).exp\_val (T n)$  := by
          have h $\beta'$  := OptimalHypothesisRate.singleton_le_exp_val ( $\sigma := \sigma n$ )
          simp only [Subtype.coe_le_coe.mpr h $\beta'$ ]
          <=  $\square T n, \text{Real.exp } (-n * (\text{Rsup} + \epsilon_4)) \cdot \rho n \square$  := by
            rw [← mul_le_mul_iff_right0 (Real.exp_pos ((n * (Rsup +  $\epsilon_4$ )))), H
            simp only [isUnit_iff_ne_zero, ne_eq, Real.exp_ne_zero, not_false
              IsUnit.mul_inv_cancel_left]
            rw [MState.exp_val, HermitianMat.inner_comm, ← HermitianMat.inner
            unfold T

```

```

    exact projLE_inner_le (Real.exp (n * (Rsup + ε4)) • (σ n).M) (ρ n).M
  _ ≤ Real.exp (-n * (Rsup + ε4)) := by
    simp [HermitianMat.inner_smul_right]
    rw [mul_comm]
    apply (mul_le_iff_le_one_left (Real.exp_pos (-(n * (Rsup + ε4)))).mpr
    rw [HermitianMat.inner_comm, ← MState.exp_val]
    exact (ρ n).exp_val_le_one (projLE_le_one _)
have h' (n₀) : ∃ n ≥ n₀, Rsup + ε4 ≤ -log β_ ε3(ρ n||{σ n}) / n := by -- Eq (S33)
  obtain {n, hn, hβ} := hβ (n₀ + 1)
  use n, by linarith
  have hn0 : 0 < n := by omega
  have hn1 : (n : ℝ≥0∞) ≠ 0 := by positivity
  have hn2 : (n : ℝ≥0∞) ≠ ⊤ := by finiteness
  have hh : n * (Rsup + ε4) = ENNReal.ofReal (n * (Rsup + ε4)) := by
    simp [ENNReal.ofReal_add]
  apply (ENNReal.mul_le_mul_iff_right (a := n) (b := Rsup + ε4) (c := -log β_ ε3
  rw [ENNReal.mul_div_cancel hn1 hn2, hh]
  apply Prob.le_negLog_of_le_exp
  rwa [← neg_mul]
have hf : ∃f (n : ℕ) in Filter.atTop, Rsup + ε4 ≤ -log β_ ε3(ρ n||{σ n}) / n := b
  rwa [Filter.frequently_atTop]
replace hf := Filter.le_limsup_of_frequently_le hf (by isBoundedDefault)
· replace hf := le_trans hf hRsup
  replace hf := tsub_eq_zero_iff_le.mpr hf
  simp_all
apply Filter.atTop.isCoboundedUnder_le_of_le (x := 0)
intro n
rw [HermitianMat.inner_comm, ← MState.exp_val]
exact (ρ n).exp_val_nonneg ((Real.exp (n * (Rsup + ε4)) • (σ n).M).projLE_nonneg (

private theorem LemmaS3_helper {ε : Prob} {d : ℕ → Type*} [∀ n, Fintype (d n)] [∀ n,
  (ρ σ₁ σ₂ : (n : ℕ) → MState (d n))
  (f : ℕ → ℝ≥0) (hσ : ∀ (i : ℕ), Real.exp (-f i) • (σ₂ i).M ≤ (σ₁ i)) (n : ℕ) :
    -log β_ ε(ρ n||{σ₁ n}) ≤ -log β_ ε(ρ n||{σ₂ n}) + f n := by
have h₁ (T : HermitianMat (d n) ℂ) (hT : 0 ≤ T) :
  Real.exp (-f n) * ⌊T, σ₂ n⌋ ≤ ⌊T, σ₁ n⌋ := by
  simpα using HermitianMat.inner_mono hT (hσ n)
by_cases hσ₂ : β_ ε(ρ n||{σ₂ n}) = 0
· simp [hσ₂]
replace hσ₂ := Prob.zero_lt_coe hσ₂
have hσ₁ : (0 : ℝ) < β_ ε(ρ n||{σ₁ n}) := by
  refine OptimalHypothesisRate.rate_pos_of_smul_pos hσ₂ (Real.exp_pos (-
f n)) ?_
  exact hσ n --For some reason turning these two lines into one `exact` causes tim
  rw [← ENNReal.toReal_le_toReal (by finiteness) (by finiteness)]
  rw [ENNReal.toReal_add (by finiteness) (by finiteness)]

```

```

simp only [Prob.negLog_pos_Real, ENNReal.coe_toReal, OptimalHypothesisRat
  Set.mem_singleton_iff, iSup_iSup_eq_left] at hσ1 hσ2 ⊢
rw [← neg_le_neg_iff]
simp only [neg_add_rev, neg_neg]
rw [← Real.log_exp (-f n), ← Real.log_mul (by positivity) (by positivity)]
apply Real.log_le_log (by positivity)
simp only [Prob.coe_iInf]
rw [Real.mul_iInf_of_nonneg (by positivity)]
apply ciInf_mono
· use 0
  rintro a ⟨y, rfl⟩
  have := (σ2 n).exp_val_nonneg y.2.2.1
  positivity
intro ⟨x, hx1, hx2, hx3⟩
simp only [MState.exp_val, ← HermitianMat.inner_smul_left]
exact HermitianMat.inner_mono' hx2 (hσ n)

/-- Lemma S3 from the paper. What they denote as σn and σn', we denote as σn and σn'
o(n))`
we express as a function `f : ℕ → ℝ`, together with the fact that `f` is li
o of `n` (i.e. that
`f =o[atTop] id`), and then writing `exp(-f)`. We also split LemmaS3 into
and the `lim sup` part. The theorem as written is true for any `f`, but we
`f` (so, `ℕ → ℝ≥0`) which is easier to work with and more natural in the su
/
private theorem LemmaS3_inf {ε : Prob}
  {d : ℕ → Type*} [∀ n, Fintype (d n)] [∀ n, DecidableEq (d n)]
  (ρ σ1 σ2 : (n : ℕ) → MState (d n))
  (f : ℕ → ℝ≥0) (hf : (f · : ℕ → ℝ) =o[atTop] (· : ℕ → ℝ))
  (hσ : ∀ i, Real.exp (-f i) • (σ2 i).M ≤ σ1 i)
  :
  Filter.atTop.liminf (fun (n : ℕ) ↦ -log β_ε(ρ n||{σ1 n}) / n) ≤
    Filter.atTop.liminf (fun (n : ℕ) ↦ -log β_ε(ρ n||{σ2 n}) / n)
  := by
  --Starting with `helper`, divide by n and take the limits. Since f is o(n)
  --the (f n) / n term will go to zero.
  trans Filter.atTop.liminf fun n ↦ (-log β_ε(ρ n||{σ2 n}) + f n) / n
  · refine Filter.liminf_le_liminf (.of_forall ?_)
  intro
  grw [LemmaS3_helper _ _ _ hσ]
  · apply le_of_eq
  simp_rw [ENNReal.add_div]
  apply ENNReal.liminf_add_of_right_tendsto_zero
  convert Asymptotics.IsLittle0.tendsto_div_nhds_zero hf
  rw [← ENNReal.tendsto_toReal_iff_of_eventually_ne_top ?_ ENNReal.zero_n
  · simp

```

```

    · rw [Filter.eventually_atTop]
      use 1
      finiteness

private theorem LemmaS3_sup {ε : Prob}
  {d : ℕ → Type*} [∀ n, Fintype (d n)] [∀ n, DecidableEq (d n)]
  (ρ σ₁ σ₂ : (n : ℕ) → MState (d n))
  (f : ℕ → ℝ≥0) (hf : (f · : ℕ → ℝ) =o[atTop] (· : ℕ → ℝ))
  (hσ : ∀ i, Real.exp (-f i) • (σ₂ i).M ≤ σ₁ i)
  :
  Filter.atTop.limsup (fun (n : ℕ) ↦ -log β_ε(ρ n||{σ₁ n}) / n) ≤
    Filter.atTop.limsup (fun (n : ℕ) ↦ -log β_ε(ρ n||{σ₂ n}) / n)
  := by
  -- Starting with `helper`, divide by n and take the limits. Since f is o(n),
  -- the (f n) / n term will go to zero.
  trans Filter.atTop.limsup fun n ↦ (-log β_ε(ρ n||{σ₂ n}) + f n) / n
  · refine Filter.limsup_le_limsup (.of_forall ?_)
    dsimp
    intro x
    grw [LemmaS3_helper _ _ _ hσ]
  · apply le_of_eq
    simp_rw [ENNReal.add_div]
    apply ENNReal.limsup_add_of_right_tendsto_zero
    convert Asymptotics.IsLittle0.tendsto_div_nhds_zero hf
    rw [← ENNReal.tendsto_toReal_iff_of_eventually_ne_top ?_ ENNReal.zero_ne_top]
  · simp
  · rw [Filter.eventually_atTop]
    use 1
    finiteness

-- This is not exactly how  $R_{\{1, \varepsilon\}}$  is defined in Eq. (17), but it should be equal d
-- the monotonicity of log and Lemma 3.
private noncomputable def R1 (ρ : MState (H i)) (ε : Prob) : ℝ≥0∞ :=
  Filter.atTop.liminf fun n ↦ -log β_ε(ρ ⊗r^[n]||IsFree) / n

private noncomputable def R2 (ρ : MState (H i)) : ((n : ℕ) → IsFree (i := i ^ n)) →
  fun σ ↦ Filter.atTop.liminf fun n ↦ □(ρ ⊗r^[n]||σ n) / n

open MatrixOrder

open scoped HermitianMat

section projLE

variable {d : Type*} [Fintype d] [DecidableEq d] (A B : HermitianMat d ℂ)

```

```

private lemma commute_aux (n : ℕ) {x : ℝ}
  {E σ : HermitianMat d ℂ} (hσ : Commute σ.mat σ.mat)
  (hE : E = 1 - {Real.exp (n * x) • σ ≤p []})
  : Commute ((1 / n : ℝ) • E).mat (σ.log - σ.log).mat := by
  rw [HermitianMat.one_sub_projLT] at hE
  obtain ⟨C, ⟨f, rfl⟩, ⟨g, rfl⟩⟩ := hσ.exists_HermitianMat_cfc
  rw [HermitianMat.log, HermitianMat.log]
  rw [← HermitianMat.cfc_comp, ← HermitianMat.cfc_comp, ← HermitianMat.cfc_comp]
  rw [HermitianMat.projLT_def] at hE
  commutes

open HermMul in
private lemma rexp_mul_smul_proj_lt_mul_sub_le_mul_sub {n : ℕ} {x : ℝ}
  {E σ : HermitianMat d ℂ} (hσ : Commute σ.mat σ.mat) (hx : 0 < x)
  (hσ : σ.mat.PosSemidef) (hσ : σ.mat.PosDef)
  (hE : E = 1 - {Real.exp (n * x) • σ ≤p []})
  : [], ((1 / n : ℝ) • E) * (σ.log - σ.log) ≤ [], x • E := by
  rw [HermitianMat.inner_eq_re_trace, HermitianMat.inner_eq_re_trace]
  rcases n.eq_zero_or_pos with rfl | hn
  · have hE' : 0 ≤ E.mat := by
    rw [hE, HermitianMat.one_sub_projLT]
    apply HermitianMat.projLT_nonneg
  have hσ : 0 ≤ σ := by rwa [HermitianMat.zero_le_iff]
  replace hσ : 0 ≤ [], E := HermitianMat.inner_ge_zero hσ hE'
  rw [HermMul.mul_eq_symmMul, HermitianMat.symmMul_of_commute]
  · simp only [CharP.cast_eq_zero, div_zero, zero_smul, HermitianMat.mat_sub,
    HermitianMat.mat_sub, zero_mul, mul_zero, Matrix.trace_zero, RCLike
    Complex.zero_re, HermitianMat.mat_smul, Algebra.mul_smul_comm, trac
    RCLike.mul_re, Complex.ofReal_re, RCLike.im_to_complex, Complex.ofR
    positivity
  · simp
  rw [HermMul.mul_eq_symmMul, HermitianMat.symmMul_of_commute (commute_aux
  dsimp
  gcongr
  apply Matrix.PosSemidef.trace_mono
  rw [one_div, HermitianMat.mat_smul, smul_mul_assoc, mul_smul_comm]
  rw [inv_smul_le_iff_of_pos (mod_cast hn), HermitianMat.mat_smul]
  rw [mul_smul_comm]
  obtain ⟨C, ⟨f, hf⟩, ⟨g, hg⟩⟩ := hσ.exists_HermitianMat_cfc
  rw [hf, hg] at hE
  rw [HermitianMat.one_sub_projLT, HermitianMat.projLT_def] at hE
  rw [← HermitianMat.cfc_const_mul, ← HermitianMat.cfc_sub] at hE
  rw [← HermitianMat.cfc_comp] at hE
  unfold Function.comp at hE
  dsimp at hE
  rw [HermitianMat.log, HermitianMat.log]

```



```

rw [← HermitianMat.cfc_comp, ← HermitianMat.cfc_comp, smul_smul]
change _ * (E.mat * HermitianMat.mat (_ - _)) ≤ _
rw [← HermitianMat.cfc_sub]
subst E
rw [← HermitianMat.mat_cfc_mul, ← HermitianMat.mat_cfc_mul]
rw [← HermitianMat.mat_cfc_mul]
change _ ≤ HermitianMat.mat ((n * x) • C.cfc _)
rw [← HermitianMat.cfc_const_mul]
change (C.cfc _ : HermitianMat _) ≤ C.cfc _
rw [← sub_nonneg, ← HermitianMat.cfc_sub, HermitianMat.cfc_nonneg_iff]
intro i
simp only [mul_ite, Pi.sub_apply, Pi.mul_apply, ite_mul]
rw [ite_sub_ite]
simp only [sub_pos, mul_one, Function.comp_apply, one_mul, mul_zero, zero_mul, sub]
split_ifs with h; swap
· rfl
set fi := f (C.H.eigenvalues i) with hfi
set gi := g (C.H.eigenvalues i) with hgi
have hfi₀ : 0 ≤ fi := by
  rw [hf, ← HermitianMat.zero_le_iff, HermitianMat.cfc_nonneg_iff] at h₀
  exact h₀ i
have hgi₀ : 0 < gi := by
  rw [hg, HermitianMat.cfc_posDef] at h₀
  exact h₀ i
rcases hfi₀.eq_or_lt with hfi₂ | hfi₂
· simp [← hfi₂]
· simp [mul_comm fi]
  suffices Real.log fi < n * x + Real.log gi by
    grw [this]
    simp only [add_sub_cancel_right, le_refl]
  rw [← Real.log_exp (n * x), ← Real.log_mul (by positivity) (by positivity)]
  apply Real.strictMonoOn_log hfi₂ ?_ h
  change 0 < Real.exp (n * x) * gi
  positivity

private lemma rexp_mul_smul_proj_lt_mul_sub_le_mul_sub' {n : ℕ} {x : ℝ} {y : ℝ}
{E ⊆ σ : HermitianMat d ℂ} (hσ : Commute σ.mat σ.mat)
(hσ : σ.mat.PosSemidef) (hσ : σ.mat.PosDef)
(hE : E = {Real.exp (n * y) • σ ≤ₚ σ} - {Real.exp (n * x) • σ ≤ₚ σ})
: (1 / n : ℝ) • E.mat * (σ.log.mat - σ.log.mat) ≤ x • E := by
--Another version of the above. Once we've proved one it's probably very easy to a
--code for the other. This doesn't suffer from the zero eigenvalue issue as much.
rcases n.eq_zero_or_pos with rfl | hn
· simp_all
obtain ⟨C, ⟨f, hf⟩, ⟨g, hg⟩⟩ := hσ.exists_HermitianMat_cfc
rw [hf, hg] at hE ⊢

```

```

rw [HermitianMat.projLE_def, HermitianMat.projLE_def] at hE
rw [← HermitianMat.cfc_const_mul, ← HermitianMat.cfc_sub] at hE
rw [← HermitianMat.cfc_comp, ← HermitianMat.cfc_const_mul] at hE
rw [← HermitianMat.cfc_sub, ← HermitianMat.cfc_comp, ← HermitianMat.cfc_s
subst E
rw [HermitianMat.log, HermitianMat.log]
rw [← HermitianMat.cfc_comp, ← HermitianMat.cfc_comp]
conv =>
  enter [1]
  congr
  · change HermitianMat.mat ((1 / (n : ℝ)) • _)
  · change HermitianMat.mat (_ - _)
change _ ≤ HermitianMat.mat (x • C.cfc _)
rw [← HermitianMat.cfc_sub, ← HermitianMat.cfc_const_mul, ← HermitianMat.
rw [← HermitianMat.cfc_const_mul, ← sub_nonneg]
change 0 ≤ HermitianMat.mat (_ - _)
rw [← HermitianMat.cfc_sub]
change (0 : HermitianMat d ℂ) ≤ _
rw [HermitianMat.cfc_nonneg_iff]
intro i
simp only [Pi.sub_apply, Function.comp_apply, one_div, Pi.mul_apply]
set fi := f (C.H.eigenvalues i) with hfi
set gi := g (C.H.eigenvalues i) with hgi
have hfi₀ : 0 ≤ fi := by
  rw [hf, ← HermitianMat.zero_le_iff, HermitianMat.cfc_nonneg_iff] at hfi
  exact hfi i
have hgi₀ : 0 < gi := by
  rw [hg, HermitianMat.cfc_posDef] at hgi
  exact hgi i
split_ifs with ha hb hb <=> simp only [← hfi, ← hgi] at ha hb ⊢
· simp
· simp only [sub_nonneg, not_le, sub_zero, mul_one] at ha hb ⊢
  rw [inv_mul_le_iff₀ (by positivity)]
  rw [← Real.exp_le_exp, Real.exp_sub]
  rw [Real.exp_log (lt_of_lt_of_le (by positivity) ha), Real.exp_log hgi₀]
  rw [div_le_iff₀ hgi₀]
  exact hb.le
· simp only [sub_nonneg, not_le, zero_sub, mul_neg, mul_one, neg_mul, sub
  le_neg_add_iff_add_le, add_zero] at ha hb ⊢
  rw [le_inv_mul_iff₀ (by positivity)]
  rw [← Real.exp_le_exp, Real.exp_sub]
  rw [Real.exp_log (lt_of_lt_of_le (by positivity) hb), Real.exp_log hgi₀]
  rw [le_div_iff₀ hgi₀]
  exact hb
· simp

```

end projLE

```

private lemma f_image_bound (mineig : ℝ) (n : ℕ) (h : 0 < mineig) (hn : 0 < n) :
  let c : ℕ → ℝ := fun n => Real.log (1 / mineig) + Real.log 3 / (max n 1);
  let f : ℕ → ℝ → ℝ := fun n lam => ⌈Real.log lam / c n⌉ * c n;
  let S : Set ℝ := (fun x => Real.exp (f n x)) '' Set.Icc ((mineig ^ n) / 3) 1;
  (h_le_f : ∀ (n : ℕ) (lam : ℝ), Real.log lam ≤ f n lam) →
  (h_f_le : ∀ (n : ℕ) (lam : ℝ), f n lam < Real.log lam + c n) →
  S.ncard ≤ n + 1 ∧ S.Finite := by
  --Thanks Aristotle. TODO Cleanup
  -- To show that $$$ is finite, we need to show that the function $$$ maps the inte
  have h_finite : Set.Finite (Set.image (fun x => Real.exp (⌈Real.log x / (Real.log
    -- Since the interval [mineig^n / 3, 1] is bounded and the function Real.log is
    have h_bounded : ∃ m M : ℤ, ∀ x ∈ Set.Icc (mineig^n / 3) 1, m ≤ ⌈Real.log x / (R
      have h_bounded : ∃ m M : ℝ, ∀ x ∈ Set.Icc (mineig^n / 3) 1, m ≤ Real.log x / (
        -- Since the interval [mineig^n / 3, 1] is closed and bounded, and the funct
        have h_cont : ContinuousOn (fun x => Real.log x / (Real.log (1 / mineig) + R
          -- Since Real.log x is continuous on the interval (0, ∞) and the denominat
          zero constant, the function Real.log x / (Real.log (1 / mineig) + Real.log 3 / (max
            have h_cont : ContinuousOn Real.log (Set.Icc (mineig^n / 3) 1) := by
              exact continuousOn_of_forall_continuousAt fun x hx => Real.continuousAt_
              exact h_cont.div_const _;
            exact ( ( InfSet.sInf <| ( fun x => Real.log x / ( Real.log ( 1 / mineig ) ) +
              obtain ( m, M, hM ) := h_bounded
            exact ( [m], [M], fun x hx => ( Int.ceil_mono <| hM x hx |>.1, Int.ceil_mono <
          cases' h_bounded with m h_bounded
          cases' h_bounded with M h_bounded
          refine Set.Finite.subset ( Set.toFinite ( Finset.image ( fun i : ℤ => Real.exp (
            intro _ a_1
            simp_all only [Set.mem_Icc, one_div, Real.log_inv, Nat.cast_max, Nat.cast_one, a
              Finset.coe_image, Finset.coe_Icc]
            obtain (w, ⟨left, rfl⟩) := a_1
            simp_all only [Real.exp_eq_exp, mul_eq_mul_right_iff, Int.cast_inj]
            apply Exists.intro
            · apply And.intro
              on_goal 2 => {
                apply Or.inl
                rfl
              }
            · simp_all only [and_self]
          intro c f S h_le_f h_f_le
          simp_all only [one_div, Real.log_inv, Nat.cast_max, Nat.cast_one, and_true, f, c,
            -- Since the interval [(n * log(mineig) - log(3)) / c(n), 0 / c(n)] has length (lo
            have h_card : Set.ncard (Set.image (fun x => Real.exp (⌈Real.log x / (Real.log (1
              refine Set.ncard_le_ncard ?_;
              intro _ a_1

```

```

simp_all only [one_div, Real.log_inv, Nat.cast_max, Nat.cast_one, Set.m
obtain ⟨w, ⟨⟨left, right_1⟩, rfl⟩⟩ := a_1
simp_all only [Real.exp_eq_exp, mul_eq_mul_right_iff, Int.cast_inj]
refine' ⟨_, ⟨_, _⟩, Or.inl rfl⟩;
· gcongr;
· have := h_f_le n 1 ; norm_num at this ; linarith [ Real.log_le_sub
· -- Taking the logarithm of both sides of the inequality $mineig^n /
  have h_log : Real.log (mineig^n / 3) ≤ Real.log w := by
    exact Real.log_le_log ( by positivity ) left;
    rwa [ Real.log_div ( by positivity ) ( by positivity ), Real.log_po
· refine' Int.ceil_le.mpr _;
rw [ div_le_iff₀ ]
· simp_all only [Int.cast_zero, zero_mul]
  exact Real.log_nonpos ( by linarith [ pow_pos h n ] ) right_1;
· have := h_f_le n 1
  simp_all only [Real.log_one, zero_div, Int.ceil_zero, Int.cast_zero
have h_card_image : Set.ncard (Set.image (fun k : ℤ => Real.exp (k * (Rea
  apply Set.ncard_image_le;
  exact Set.finite_Icc _ _;
simp_all +decide [ Set.ncard_eq_toFinset_card' ];
refine le_trans h_card <| le_trans h_card_image ?_;
rcases n with _ | n
· simp at hn
simp_all only [lt_add_iff_pos_left, add_pos_iff, zero_lt_one, or_true, Na
  le_add_iff_nonneg_left, Nat.cast_nonneg, sup_of_le_left, Int.toNat_le,
specialize h_f_le ( n + 1 ) 1;
simp_all only [Real.log_one, Nat.cast_add, Nat.cast_one, le_add_iff_nonne
  zero_div, Int.ceil_zero, Int.cast_zero, zero_mul, zero_add, lt_neg_add_
apply Int.le_of_lt_add_one
rw [ ← @Int.cast_lt ℝ ]
push_cast
nlinarith [ Int.le_ceil ( ( ( n + 1 ) * Real.log mineig - Real.log 3 ) /
Real.log mineig + Real.log 3 / ( n + 1 ) ) ),
mul_div_cancel₀ ( ( n + 1 ) * Real.log mineig - Real.log 3 ) ( show ( -
Real.log mineig + Real.log 3 / ( n + 1 ) ) ≠ 0 by
nlinarith [ Real.log_pos ( show ( 3 : ℝ ) > 1 by norm_num ), mul_div_
Real.log_pos ( show ( 3 : ℝ ) > 1 by norm_num ), mul_div_cancel₀ (

private lemma c'_bounded {mineig : ℝ} {ε2 : ℕ → ℝ≥0}
(hε2 : ∀ (n : ℕ), ε2 n < 1) (o : ℝ) :
let c : ℕ → ℝ := fun n => Real.log (1 / mineig) + Real.log 3 / ↑(max n 1)
let c' : ℝ → ℕ → ℝ := fun ε2 n => (c n + (c n) / n) □ (o + ε2);
(∀ (n : ℕ), 0 < c n) →
∃ (C : NNReal), ∀f (n : ℕ) in Filter.atTop, c' (↑(ε2 n)) n ≤ ↑C := by
have h_bound : ∃ C : ℝ, ∀f n in Filter.atTop, Real.log (1 / mineig) + Rea
have h_bound : Filter.Tendsto (fun n => Real.log (1 / mineig) + Real.lo

```

```

    exact le_trans ( Filter.Tendsto.add ( tendsto_const_nhds.add <| Filter.Tendsto
    exact ( _, h_bound.eventually ( ge_mem_nhds <| lt_add_one _ ) );
  intro c c' a
  simp_all only [one_div, Real.log_inv, Filter.eventually_atTop, ge_iff_le, Nat.cast
    lt_neg_add_iff_add_lt, add_zero, sup_le_iff, c, c']
  obtain ⟨w, ⟨w_1, h⟩⟩ := h_bound
  use [w]_+ + [0]_+ + 1, [w_1]_+
  intro n hn
  constructor
  · norm_num
    linarith [ Nat.le_ceil w, h n ( Nat.le_of_ceil_le hn ) ]
  · norm_num
    linarith [ Nat.le_ceil o, show (  $\epsilon_2$  n :  $\mathbb{R}$  )  $\leq$  1 by exact_mod_cast le_of_lt ( h $\epsilon_2$ 
noncomputable section sigmas

/- Define the different (sequences of) states needed in the construction and prove t
properties. -/

section defs
section  $\sigma_1$ _c_and_f --Stuff that has variable (i) explicit because it only depends on
variable (i)

private def  $\sigma_1$  : MState (H i) :=
  Classical.choose (FreeStateTheory.free_fullRank i)

private def  $\sigma_1$ _mineig :=  $\square$  k, ( $\sigma_1$  i).M.H.eigenvalues k

/-- The sequence  $c_n$  given in (S44). In order to handle when  $c = 0$ , we've replaced t
(Real.log 3) / n term with (Real.log 3) / (max n 1). -/
private def  $\sigma_1$ _c (n :  $\mathbb{N}$ ) :  $\mathbb{R}$  :=
  Real.log (1 /  $\sigma_1$ _mineig i) + (Real.log 3) / (max n 1)

/-- The function  $f_n(\lambda)$  in (S45). -/
private def f_map (n :  $\mathbb{N}$ ) (lam :  $\mathbb{R}$ ) :  $\mathbb{R}$  :=
  [Real.log lam /  $\sigma_1$ _c i n] *  $\sigma_1$ _c i n

end  $\sigma_1$ _c_and_f
--From here on, sequences generally depend on other parameters + states.

variable (p : MState (H i)) ( $\epsilon$  : Prob) (m :  $\mathbb{N}$ ) ( $\sigma$  : (n :  $\mathbb{N}$ )  $\rightarrow$  IsFree (i := i ^ n))

private def « $\sigma$ » :
  (n :  $\mathbb{N}$ )  $\rightarrow$  MState (H (i ^ n)) :=
  fun n  $\mapsto$  Lemma6_on m ( $\sigma_1$  i) ( $\sigma$  m) n

```

```

private def «σ*» : (n : ℕ) → MState (H (i ^ n)) :=
  haveI σ_max_exists (n : ℕ) := IsCompact_IsFree.exists_isMaxOn Set.Nonempty
    (f := fun σ ↦ β_ε(ρ ⊗r ^[n] || {σ})) (hf := Continuous.continuousOn (by
      fun n ↦ Classical.choose (σ_max_exists n)

--TODO: would be nice to write a `Mixable` thing for mixing `k` things acco
-- in this case `ProbDistribution.uniform (Fin 3)` .
private def σ' : (n : ℕ) → MState (H (i ^ n)) :=
  fun n ↦ ⟨2/3, by norm_num⟩ [(1/2, by norm_num) [«σ» m σ n ↔ «σ*» ρ ε n] ↔

private def σ''_unnormalized : (n : ℕ) → HermitianMat (H (i ^ n)) ℂ :=
  fun n ↦ (σ' ρ ε m σ n).M.cfc fun e ↦ Real.exp (f_map i n e)

end defs

section proofs
/- Prove relevant properties about their positivity, trace, etc. -
/
section σ1_c_and_f

variable (i)

private theorem σ1_pos : (σ1 i).m.PosDef :=
  (FreeStateTheory.free_fullRank i).choose_spec.left

private theorem σ1_isFree : IsFree (σ1 i) :=
  (FreeStateTheory.free_fullRank i).choose_spec.right

private theorem mineig_pos : 0 < σ1_mineig i := by
  --because σ1 is PosDef, all eigenvalues are positive, so their minimum is
  obtain ⟨i_min, hi_min⟩ := exists_eq_ciInf_of_finite (f := (HermitianMat.H
  unfold σ1_mineig
  rw [← hi_min]
  exact (σ1_pos i).eigenvalues_pos i_min

private theorem mineig_le_one : σ1_mineig i ≤ 1 := by
  --all eigenvalues of a state are at most 1. (We might not actually need
  obtain ⟨i_min, hi_min⟩ := exists_eq_ciInf_of_finite (f := (HermitianMat
  unfold σ1_mineig
  rw [← hi_min]
  exact (σ1 i).eigenvalue_le_one i_min

private theorem σ1_c_pos (n) : 0 < σ1_c i n := by
  rw [σ1_c]
  have h_min_pos := mineig_pos i

```

```

have h_min_le_one := mineig_le_one i
have h1 : 0 ≤ Real.log (1 / σ1_mineig i) := by bound
positivity

private theorem σ1_c_div_lim : Filter.atTop.Tendsto (fun n ↦ (σ1_c i n) / n) (0) :
  unfold σ1_c
  have h_const : Filter.atTop.Tendsto (fun n : ℕ ↦ Real.log (1 / σ1_mineig i) / n) (
    tendsto_const_nhds.div_atTop tendsto_natCast_atTop_atTop;
  -- We can split the limit into two parts: the constant term and the term involving
  have h_div : Filter.atTop.Tendsto (fun n : ℕ ↦ Real.log 3 / (max n 1 * n)) (0) :
    tendsto_const_nhds.div_atTop <| Filter.tendsto_atTop_mono (fun n ↦ by
      norm_cast; nlinearith [le_max_left n 1, le_max_right n 1]) tendsto_natCast_atTop
  convert h_const.add h_div using 2 <=> ring

private lemma σ1_c_identity {n : ℕ} (hn : 0 < n) :
  Real.exp (-σ1_c i n) * (1 / 3) * σ1_mineig i ^ n =
  Real.exp (-n * (σ1_c i n + σ1_c i n / n)) := by
  have h (x : ℝ) : n * (x / n) = x := by field_simp
  simp only [σ1_c]
  simp only [neg_mul, show ((max n 1 : ℕ) : ℝ) = n from mod_cast (max_eq_left hn)]
  simp only [Real.exp_add, mul_add, neg_add_rev, mul_assoc, h]
  simp [Real.exp_neg, Real.exp_log, Real.exp_log (mineig_pos i), Real.exp_nat_mul]

theorem σ1_c_little0 : (fun n : ℕ ↦ σ1_c i n + Real.log 3) =o[Filter.atTop] (fun x ↦
  apply Asymptotics.IsLittle0.add
  · rw [Asymptotics.isLittle0_iff_tendsto']
  · exact σ1_c_div_lim i
  simp only [Nat.cast_eq_zero, Filter.eventually_atTop]
  use 1
  grind
  · --This `(fun x => Real.log 3) =o[Filter.atTop] fun x => x` really should be its
  refine Asymptotics.isLittle0_const_left.2 <| 0r.inr ?_
  convert tendsto_natCast_atTop_atTop (R := ℝ)
  ext
  simp

/-- (S46), part 1 -/
private theorem log_le_f (n : ℕ) (lam : ℝ) : Real.log lam ≤ f_map i n lam :=
  calc
  - ≤ ([Real.log lam / (σ1_c i n)]) * σ1_c i n := by
    rw [← mul_inv_le_iff0 (σ1_c_pos i n)]
    apply Int.le_ceil _
  - = _ := by
    rfl

/-- (S46), part 2 -/

```

```

private theorem f_le_log (n : ℕ) (lam : ℝ) : f_map i n lam < Real.log lam +
  calc
  _ = [Real.log lam / σ1_c i n] * σ1_c i n := by
    rfl
  _ < (Real.log lam / σ1_c i n + 1) * σ1_c i n := by
    gcongr
    · exact σ1_c_pos i n
    · exact Int.ceil_lt_add_one _
  _ ≤ _ := by
    have := σ1_c_pos i n
    field_simp
    rfl

private theorem le_exp_f (n : ℕ) (x : ℝ) (hx : 0 < x) : x ≤ Real.exp (f_map
  convert Real.exp_monotone (log_le_f i n x)
  rw [Real.exp_log hx]

private theorem exp_f_le (n : ℕ) (x : ℝ) (hx : 0 < x) : Real.exp (f_map i n
  convert Real.exp_strictMono (f_le_log i n x) using 1
  rw [Real.exp_add (Real.log x), Real.exp_log hx, mul_comm]

end σ1_c_and_f

variable (ρ : MState (H i)) (ε : Prob)
variable (m : ℕ) (σ : (n : ℕ) → IsFree (i := i ^ n)) (n : ℕ)

private theorem «σ~isFree» : IsFree («σ~ m σ n) :=
  Lemma6_σn_IsFree (σ1_isFree i) (fun n ↦ (σ n).2) m n

private theorem «σ*_free» : IsFree («σ*_ ρ ε n) :=
  haveI σ_max_exists (n : ℕ) := IsCompact_IsFree.exists_isMaxOn Set.Nonempty
    (f := fun σ ↦ β_ε(ρ ⊗r^[n]||{σ})) (hf := Continuous.continuousOn (by
    (σ_max_exists n).choose_spec.left

private theorem «σ*_max» :
  IsMaxOn (fun σ ↦ β_ε(ρ ⊗r^[n]||{σ})) IsFree («σ*_ ρ ε n) :=
  haveI σ_max_exists (n : ℕ) := IsCompact_IsFree.exists_isMaxOn Set.Nonempty
    (f := fun σ ↦ β_ε(ρ ⊗r^[n]||{σ})) (hf := Continuous.continuousOn (by
    (σ_max_exists n).choose_spec.right

private theorem σ'_free : IsFree (σ' ρ ε m σ n) := by
  -- by convexity of `IsFree` and that the three constituents are free
  unfold σ'
  apply IsFree.mix
  · exact («σ~isFree» m σ n).mix («σ*_free» ρ ε n) _
  · exact (σ1_isFree i).npow n

```



```

private theorem σ'_posdef : (σ' ρ ∈ m σ n).m.PosDef := by
  --because σ1 is PosDef, so is σ1⊗n, and so is any convex mixture.
  unfold σ'
  apply MState.PosDef_mix_of_ne_one
  · apply UnitalPretheory.PosDef.npow (σ1_pos i)
  · norm_num [← Prob.ne_iff]

```

```

private theorem hσ'_n_eq_sum_third : (σ' ρ ∈ m σ n).M =
  (1 / 3 : ℝ) • (⟨σ⟩ m σ n) + (1 / 3 : ℝ) • (⟨σ*⟩ ρ ∈ n) + (1 / 3 : ℝ) • ((σ1 i) ⊗
  unfold σ'
  change _ • _ + _ = _
  conv =>
    enter [1, 1, 2]
    change _ + _
  dsimp [Mixable.to_U]
  norm_num only [one_div, Prob.coe_one_minus, smul_add, smul_smul]

```

```

private theorem hσ1_le_σ' : (1 / 3 : ℝ) • ((σ1 i) ⊗n).M ≤ (σ' ρ ∈ m σ n).M := by
  rw [hσ'_n_eq_sum_third]
  apply le_add_of_nonneg_left
  have := (⟨σ*⟩ ρ ∈ n).nonneg
  have := (⟨σ⟩ m σ n).nonneg
  positivity

```

```

private theorem σ''_unnormalized_PosDef : Matrix.PosDef (σ''_unnormalized ρ ∈ m σ n)
  dsimp [σ''_unnormalized]
  rw [HermitianMat.cfc_posDef]
  intro
  positivity

```

```

private theorem σ''_tr_bounds : 1 ≤ (σ''_unnormalized ρ ∈ m σ n).trace ∧
  (σ''_unnormalized ρ ∈ m σ n).trace < Real.exp (σ1_c i n) := by
  have hσ' := (σ' ρ ∈ m σ n).tr
  constructor
  · rw [← HermitianMat.sum_eigenvalues_eq_trace] at hσ' ⊢
    rw [← hσ']
    obtain (e, he) := (σ' ρ ∈ m σ n).M.cfc_eigenvalues fun e ↦ Real.exp (f_map i n e)
    rw [he]
    simp only [Function.comp_apply]
    rw [Equiv.sum_comp e (fun k ↦ Real.exp (f_map i n (Matrix.IsHermitian.eigenvalue
    gcongr
    apply le_exp_f i n _
    exact (σ'_posdef ρ ∈ m σ n).eigenvalues_pos _
  · rw [← HermitianMat.sum_eigenvalues_eq_trace] at hσ' ⊢
    rw [← mul_one (Real.exp (σ1_c i n)), ← hσ', Finset.mul_sum]

```

```

obtain ⟨e, he⟩ := (σ' ρ ε m σ n).M.cfc_eigenvalues fun e ↦ Real.exp (f_
rw [he]; clear he
dsimp
rw [Equiv.sum_comp e (fun k ↦ Real.exp (f_map i n (Matrix.IsHermitian.e
gcongr
· exact Finset.univ_nonempty
· apply exp_f_le i n _
exact (σ'_posdef ρ ε m σ n).eigenvalues_pos _

end proofs

variable (ρ : MState (H i)) (ε : Prob)
variable (m : ℕ) (σ : (n : ℕ) → IsFree (i := i ^ n)) (n : ℕ)

--We're now finally ready to define the main sequence with the properties w
--This is the normalized version of σ''_unnormalized, which gives a state b
-- already PosDef
private def σ'' : (n : ℕ) → MState (H (i ^ n)) := fun n ↦ {
  --TODO make this its own definition: Normalizing a matrix to give a tr-
  1 op.
  M := (σ''_unnormalized ρ ε m σ n).trace⁻¹ • (σ''_unnormalized ρ ε m σ n)
  nonneg := by
    have h1 : 0 < (σ''_unnormalized ρ ε m σ n).trace :=
      zero_lt_one.trans_le (σ''_tr_bounds ρ ε m σ n).left
    have h2 : 0 < σ''_unnormalized ρ ε m σ n :=
      (σ''_unnormalized_PosDef ρ ε m σ n).zero_lt
    positivity
  tr := by
    rw [HermitianMat.trace_smul]
    apply inv_mul_cancel₀
    exact (zero_lt_one.trans_le (σ''_tr_bounds ρ ε m σ n).left).ne'
}

private lemma σ''_posdef n : (σ'' ρ ε m σ n).M.mat.PosDef := by
  apply (σ''_unnormalized_PosDef ρ ε m σ n).smul
  have := (σ''_tr_bounds ρ ε m σ n).left
  positivity

private lemma σ'_le_σ'' (n) : Real.exp (-σ₁_c i n) • (σ' ρ ε m σ n).M ≤ σ''
dsimp [σ'']
set x := (σ''_unnormalized ρ ε m σ n).trace
dsimp [σ''_unnormalized]
rw [← HermitianMat.cfc_const_mul_id, ← HermitianMat.cfc_const_mul_id,
  ← HermitianMat.cfc_comp]
rw [← sub_nonneg, ← HermitianMat.cfc_sub, HermitianMat.cfc_nonneg_iff]
intro k

```

```

set y := (σ' ρ ε m σ n).M.H.eigenvalues k
have hy : 0 < y := (σ'_posdef ρ ε m σ n).eigenvalues_pos k
dsimp only [Pi.sub_apply, Function.comp_apply]
have : 0 < x := zero_lt_one.trans_le (σ''_tr_bounds ρ ε m σ n).left
grw [← le_exp_f i n y hy]
rw [← sub_mul]
suffices 0 ≤ x-1 - Real.exp (-σ1_c i n) by positivity
rw [sub_nonneg, ← inv_le_inv₀]
· simpa [← Real.exp_neg] using (σ''_tr_bounds ρ ε m σ n).right.le
· positivity
· positivity

private lemma σ''_le_σ' (n) : σ'' ρ ε m σ n ≤ Real.exp (σ1_c i n) • (σ' ρ ε m σ n).M
  dsimp [σ'']
  set x := (σ''_unnormalized ρ ε m σ n).trace
  dsimp [σ''_unnormalized]
  rw [← HermitianMat.cfc_const_mul_id, ← HermitianMat.cfc_const_mul_id,
    ← HermitianMat.cfc_comp]
  rw [← sub_nonneg, ← HermitianMat.cfc_sub, HermitianMat.cfc_nonneg_iff]
  intro k
  set y := (σ' ρ ε m σ n).M.H.eigenvalues k
  have hy : 0 < y := (σ'_posdef ρ ε m σ n).eigenvalues_pos k
  dsimp only [Pi.sub_apply, Function.comp_apply]
  grw [← exp_f_le i n y hy]
  rw [← one_sub_mul]
  suffices 0 ≤ 1 - x-1 by positivity
  simpa using inv_le_one_of_one_le₀ (σ''_tr_bounds ρ ε m σ n).left

private theorem «σ''_ge_σ*» n : σ'' ρ ε m σ n ≥ (Real.exp (-
σ1_c i n) / 3) • («σ*» ρ ε n).M := by
  grw [ge_iff_le, ← σ'_le_σ'', div_eq_mul_inv, ← smul_smul, ← one_div]
  rw [smul_le_smul_iff_of_pos_left (by positivity), hσ'n_eq_sum_third]
  apply le_add_of_le_of_nonneg
  · apply le_add_of_nonneg_left
    have := («σ*» m σ n).nonneg
    positivity
  · have := ((σ1 i) ⊗r^[n]).nonneg
    positivity

private theorem «σ''_ge_σ*~» n : σ'' ρ ε m σ n ≥ (Real.exp (-
σ1_c i n) / 3) • («σ*~» m σ n).M := by
  grw [ge_iff_le, ← σ'_le_σ'', div_eq_mul_inv, ← smul_smul, ← one_div]
  rw [smul_le_smul_iff_of_pos_left (by positivity), hσ'n_eq_sum_third]
  apply le_add_of_le_of_nonneg
  · apply le_add_of_nonneg_right
    have := («σ*» ρ ε n).nonneg

```

```

    positivity
    · have := ((σ1 i) ⊗r[n]).nonneg
    positivity

private theorem σ''_ge_σ1 n : σ'' ρ ε m σ n ≥ (Real.exp (-
σ1_c i n) / 3) • ((σ1 i) ⊗r[n]).M := by
  grw [ge_iff_le, ← σ'_le_σ'', div_eq_mul_inv, ← smul_smul, ← one_div]
  rw [smul_le_smul_iff_of_pos_left (by positivity)]
  exact hσ1_le_σ' ρ ε m σ n

private abbrev ε0_func (ε' : Prob) : ℝ := (R2 ρ σ - R1 ρ ε).toReal * (ε - ε')

end sigmas

private theorem EquationS88 (ρ : MState (H i)) (σ : (n : ℕ) → ↑IsFree) {ε ε' : Prob}
(hR1R2 : R1 ρ ε < R2 ρ σ) (hR1 : R1 ρ ε ≠ ⊤) (hR2 : R2 ρ σ ≠ ⊤) (hε0_1 : ε' < ε) :
:
let □ := fun n => pinching_map (σ'' ρ ε m σ n);
let ε0 := ε0_func ρ ε σ ε';
(0 < ε0) →
(R1 ρ ε).toReal ≤ (R2 ρ σ).toReal + ε0 →
  let P1 := fun ε2 (n : ℕ) ↦ {Real.exp (↑n * ((R1 ρ ε).toReal + ε2)) •
  let P2 := fun ε2 (n : ℕ) ↦ {Real.exp (↑n * ((R2 ρ σ).toReal + ε0 + ε2)) •
  let E1 := 1 - P1;
  let E2 := P1 - P2;
  let E3 := P2;
  E1 + E2 + E3 = 1 →
  (∀ (ε2 : ℝ) (n : ℕ), E1 ε2 n = {↑((□ n) (ρ ⊗r[n])) <p Real.exp (↑n * ((R1 ρ ε).toReal + ε2)) •
  (∀ (ε2 : ℝ) (n : ℕ), E2 ε2 n ≤ {↑((□ n) (ρ ⊗r[n])) <p Real.exp (↑n * ((R2 ρ σ).toReal + ε0 + ε2)) •
  (∀ (ε2 : ℝ) (n : ℕ), 0 ≤ E2 ε2 n) →
  let c' : ℝ → ℕ → ℝ := fun ε2 n => max (σ1_c i n + σ1_c i n / ↑n) ((R2 ρ σ).toReal -
  ∀ (ε2 : ℝ) (n : ℕ), 0 < ε2 → 0 < n →
  (1 / n : ℝ) • (E3 ε2 n).mat * (((□ n) (ρ ⊗r[n])).M).log - ((σ'' ρ ε m σ n).toReal -
  ENNReal.ofReal
  ((R1 ρ ε).toReal - (R1 ρ ε).toReal * □(□ n (ρ ⊗r[n])).M, P1 ε2 n) +
  (ε2 - ε2 * □(□ n (ρ ⊗r[n])).M, P2 ε2 n) +
  □(□ n (ρ ⊗r[n])).M, P1 ε2 n * (R2 ρ σ).toReal +
  □(□ n (ρ ⊗r[n])).M, P1 ε2 n * ε0 +
  (-((R2 ρ σ).toReal * □(□ n (ρ ⊗r[n])).M, P2 ε2 n) -
  ε0 * □(□ n (ρ ⊗r[n])).M, P2 ε2 n) +
  □(□ n (ρ ⊗r[n])).M, P2 ε2 n * c' ε2 n) =
  R1 ρ ε + ENNReal.ofReal ε2 +
  ENNReal.ofReal □P1 ε2 n, □ n (ρ ⊗r[n]) *
  (ENNReal.ofReal ε2 + R2 ρ σ + ENNReal.ofReal ε0 -
  (R1 ρ ε + ENNReal.ofReal ε2)) +

```

```

      ENNReal.ofReal (P2 ε2 n, (n (ρ ⊗r ^[n])) *
        (ENNReal.ofReal (c' ε2 n) -
          (ENNReal.ofReal ε2 + R2 ρ σ + ENNReal.ofReal ε0))) := by
intros (ε0 hε0 h1 P1 P2 E1 E2 E3 hE hexp1 hexp2 hE2 c' ε2 n hε2 hn hlog
repeat rw [HermitianMat.inner_comm (P1 ε2 n)]
repeat rw [HermitianMat.inner_comm (P2 ε2 n)]
ring_nf
rw [← ENNReal.toReal_eq_toReal_iff' (by finiteness) (by finiteness)]
rw [ENNReal.toReal_ofReal]; swap
· apply add_nonneg
  · rw [← mul_one_sub, ← mul_one_sub, add_assoc, add_assoc, add_assoc]
    apply add_nonneg
    · apply mul_nonneg
      · positivity
      · rw [sub_nonneg]
        apply MState.exp_val_le_one
        apply HermitianMat.projLE_le_one
    · nth_rw 2 [sub_eq_add_neg]
      rw [← add_assoc, add_comm, add_assoc]
      apply add_nonneg
      · apply mul_nonneg
        · positivity
        · rw [sub_nonneg]
          apply MState.exp_val_le_one
          apply HermitianMat.projLE_le_one
      · rw [← mul_add, ← neg_add, ← mul_add, add_comm, ← sub_eq_add_neg]
        rw [← sub_mul]
        apply mul_nonneg
        · rw [sub_nonneg]
          apply HermitianMat.inner_mono
          · apply MState.nonneg
          · unfold P1 P2
            rw [← sub_nonneg]
            change 0 ≤ E2 ε2 n
            exact hE2 ε2 n
          · positivity
      · apply mul_nonneg
        · apply HermitianMat.inner_ge_zero
          · apply MState.nonneg
          · apply HermitianMat.projLE_nonneg
        · positivity
repeat rw [ENNReal.toReal_add (by finiteness) (by finiteness)]
rw [ENNReal.toReal_mul, ENNReal.toReal_mul]
rw [ENNReal.toReal_sub_of_le ?_ (by finiteness)]; swap
· grw [hR1R2, ← hε0, ENNReal.ofReal_zero, add_zero, add_comm]
repeat rw [ENNReal.toReal_ofReal]

```

```

rotate_left
· apply HermitianMat.inner_ge_zero --TODO: Positivity extension for Hermi
· apply MState.nonneg --TODO: Positivity extension for MState
· apply HermitianMat.projLE_nonneg --TODO: Positivity extension for pro
· apply HermitianMat.inner_ge_zero
· apply MState.nonneg
· apply HermitianMat.projLE_nonneg
· exact hε2.le
rw [ENNReal.toReal_sub_of_le ?_ (by finiteness)]; swap
· dsimp [c']
  rw [ENNReal.ofReal_max]
  convert le_max_right
  rw [ENNReal.ofReal_add (by positivity) (by positivity),
      ENNReal.ofReal_add (by positivity) (by positivity)]
  rw [ENNReal.ofReal_toReal (by finiteness)]
  abel
repeat rw [ENNReal.toReal_add (by finiteness) (by finiteness)]
repeat rw [ENNReal.toReal_ofReal (by positivity)]
ring

set_option maxHeartbeats 800000 in
private theorem EquationS62
  (ρ : MState (H i)) (σ : (n : ℕ) → IsFree (i := i ^ n))
  {ε ε' : Prob} (hε'_1 : 0 < ε') (hε'_2 : ε' < ε) (hε : ε < 1)
  (hR1R2 : R1 ρ ε < R2 ρ σ) (hR1 : R1 ρ ε ≠ ⊤) (hR2 : R2 ρ σ ≠ ⊤)
  (hε₀ : 0 < ε₀_func ρ ε σ ε') (hε₀' : (R1 ρ ε).toReal ≤ (R2 ρ σ).toReal
  (m : ℕ) (hm : m ≥ 1 ∧  $\prod (\rho \otimes_r^m) \uparrow (\sigma m)$ ) /  $\uparrow m < R2 \rho \sigma + (.ofNNReal (\epsilon_0$ 
  :
  let  $\prod n := pinching\_map (\sigma'' \rho \epsilon m \sigma n)$ ;
  Filter.atTop.liminf (fun n  $\mapsto \prod (\prod n (\rho \otimes_r^n) \uparrow (\sigma'' \rho \epsilon m \sigma n) / n) - R1$ 
   $\uparrow (1 - \epsilon') * (R2 \rho \sigma - R1 \rho \epsilon) := by$ 
intro  $\prod$ 
set ε₀ := ε₀_func ρ ε σ ε'

have hliminf_le : Filter.atTop.liminf (fun n  $\mapsto$ 
   $-\log \beta_{\epsilon}(\prod n (\rho \otimes_r^n) \uparrow \{(\sigma'' \rho \epsilon m \sigma n)\} / n) \leq (R1 \rho \epsilon).toNNReal :=$ 
(S66)
  rw [ENNReal.coe_toNNReal hR1R2.ne_top]
  unfold R1
  calc _ = Filter.atTop.liminf (fun n  $\Rightarrow -\log$ 
     $\beta_{\epsilon}(\prod n (\rho \otimes_r^n) \uparrow \{(pinching\_map (\sigma'' \rho \epsilon m \sigma n)) (\sigma'' \rho \epsilon m \sigma n)\})$ 
    enter [1, 1, n]
    rw [ $\leftarrow pinching\_self (\sigma'' \rho \epsilon m \sigma n)$ ]
   $\leq Filter.atTop.liminf (fun n \Rightarrow -\log \beta_{\epsilon}(\rho \otimes_r^n \uparrow \{(\sigma'' \rho \epsilon m \sigma n)\})$ 
(S67)
  refine Filter.liminf_le_liminf ?_

```

```

    apply Filter.Eventually.of_forall
    intro
    apply ENNReal.div_le_div_right
    apply Prob.negLog_Antitone
    apply OptimalHypothesisRate.optimalHypothesisRate_antitone
    ≤ Filter.atTop.liminf (fun n => -log β_ ε (ρ ⊗r ^[n]) || { («σ*» ρ ∈ n) }) / ↑n) := by
(S68)
    apply LemmaS3_inf _ _ _
    (f := fun n ↦ ⟨σ1_c i n + Real.log 3, add_nonneg (σ1_c_pos i n).le (Real.log
      (σ1_c_little0 i)
    intro n
    rw [coe_mk, neg_add', Real.exp_sub, Real.exp_log (by positivity)]
    exact «σ''_ge_σ*» ρ ∈ m σ n
    = _ := by --(S69)
    congr! 4 with n
    rw [← OptimalHypothesisRate.Lemma3 ε IsCompact_IsFree free_convex]
    rw [← iSup_subtype'']
    exact ((«σ*_max» ρ ∈ n).iSup_eq («σ*_free» ρ ∈ n)).symm

  have hlimsup_le (ε1 : Prob) (hε1 : 0 < (ε1 : ℝ) ∧ (ε1 : ℝ) < 1) :
    Filter.atTop.limsup (fun n ↦ -log β_ (1-ε1)(⊔ n (ρ ⊗r ^[n]) || {σ'' ρ ∈ m σ n}) /
      ((R2 ρ σ).toNNReal + ⟨ε0, hε0.le⟩ : NNReal)) := by --
(S70)
    rw [ENNReal.coe_add, ENNReal.coe_toNNReal hR2]
    unfold R2
    calc _ = Filter.atTop.limsup (fun n => -log
      β_ (1-ε1)(⊔ n (ρ ⊗r ^[n]) || {(pinching_map (σ'' ρ ∈ m σ n)) (σ'' ρ ∈ m σ n)}) / ↑
        enter [1, 1, n]
        rw [← pinching_self (σ'' ρ ∈ m σ n)]
    ≤ Filter.atTop.limsup (fun n => -log β_ (1-ε1)(ρ ⊗r ^[n]) || {(σ'' ρ ∈ m σ n)}) / ↑
    -- refine Filter.limsup_le_limsup ?
    apply Filter.Eventually.of_forall
    intro
    apply ENNReal.div_le_div_right
    apply Prob.negLog_Antitone
    apply OptimalHypothesisRate.optimalHypothesisRate_antitone
    ≤ Filter.atTop.limsup (fun n => -log β_ (1-ε1)(ρ ⊗r ^[n]) || { («σ~» m σ n) }) / ↑n) :
(S71)
    apply LemmaS3_sup _ _ _
    (f := fun n ↦ ⟨σ1_c i n + Real.log 3, add_nonneg (σ1_c_pos i n).le (Real.log
      (σ1_c_little0 i)
    intro n
    rw [coe_mk, neg_add', Real.exp_sub, Real.exp_log (by positivity)]
    exact «σ''_ge_σ~» ρ ∈ m σ n
    ≤ _ := by --(S72)
    -- have _ := HermitianMat.nonSingular_of_posDef (σ1_pos i)

```

```

    apply Lemma6
    · exact hm.left
    · change (_ : ℝ) < _
      simp using hε1.left
    _ ≤ _ := hm.right.le -- (S73)

let P1 ε2 n := {(∑ n (ρ ⊗r ^[n])) . M ≥p (Real.exp (↑n * ((R1 ρ ε).toReal + ε2))
let P2 ε2 n := {(∑ n (ρ ⊗r ^[n])) . M ≥p (Real.exp (↑n * ((R2 ρ σ).toReal + ε0))

have hEComm ε2 n : Commute ((∑ n) (ρ ⊗r ^[n])) . M - Real.exp ((n : ℝ) * ((
  ((∑ n) (ρ ⊗r ^[n])) . M - Real.exp ((n : ℝ) * ((R1 ρ ε).toReal + ε2)) •
  simp only [HermitianMat.mat_sub, MState.mat_M, HermitianMat.mat_smul]
  suffices h : Commute (∑ n (ρ ⊗r ^[n])) . m (σ' ρ ε m σ n) . m by
    apply Commute.sub_left
    · commutes
    · apply Commute.smul_left
      apply Commute.sub_right
      · exact Commute.symm <_>
      · commutes
    exact pinching_commutes (ρ ⊗r ^[n]) (σ' ρ ε m σ n)

have hPcomm ε2 n : Commute (P1 ε2 n).mat (P2 ε2 n).mat := by
  simp only [HermitianMat.projLE, HermitianMat.mat_cfc, P1, P2]
  apply IsSelfAdjoint.commute_cfc
  · apply HermitianMat.H
  symm
  apply IsSelfAdjoint.commute_cfc
  · apply HermitianMat.H
  exact hEComm ε2 n

let E1 := 1 - P1 -- (S78)
let E2 := P1 - P2 -- (S79)
let E3 := P2 -- (S80)

have hE_pos ε2 n : 0 ≤ E2 ε2 n := by
  dsimp [E2, P2, P1]
  rw [HermitianMat.projLE_def, HermitianMat.projLE_def, sub_nonneg]
  apply HermitianMat.cfc_le_cfc_of_commute
  · intro _ _ hxy; grind
  · exact hEComm ε2 n
  · grw [hR1R2, ← hε0, add_zero]
  exact hR2
clear hEComm

have Esum : E1 + E2 + E3 = 1 := by
  unfold E1 E2 E3

```



```

abel

have hE1proj ε2 n : E1 ε2 n = {(∑ n (ρ ⊗r ^[n]))}.M <p (Real.exp (↑n*((R1 ρ ε)).toReal))
  dsimp [E1, P1]
  rw [sub_eq_iff_eq_add]
  simp only [HermitianMat.proj_le_add_lt]

have hE2leProj ε2 n : E2 ε2 n ≤ {(∑ n (ρ ⊗r ^[n]))}.M <p (Real.exp (↑n*((R2 ρ σ)).toReal))
  dsimp [E2, P1, P2]
  rw [sub_le_iff_le_add]
  simp only [HermitianMat.proj_le_add_lt]
  exact HermitianMat.projLE_le_one _ _

-- (S83)
let c' ε2 n := (σ1_c i n + (σ1_c i n) / n) ∙ ((R2 ρ σ).toReal + ε0 + ε2)

have hc' (ε2 : ℕ → ℝ≥0) (hε2 : ∀ n, ε2 n < 1) :
  ∃ (C : ℝ≥0), ∀f (n : ℕ) in Filter.atTop, c' (ε2 n) n ≤ C := by
  apply c'_bounded hε2 _ (σ1_c pos i)

-- (S84)
have hσ'' ε2 n : Real.exp (-n * c' ε2 n) ∙ 1 ≤ (σ'' ρ ε m σ n).M := by
  rcases n.eq_zero_or_pos with rfl | hn
  · have _ : Unique (H (i ^ 0)) := by
      rw [spacePow_zero]
      infer_instance
      apply le_of_eq
      simp [Unique.eq_default (σ'' ρ ε m σ 0)]
  calc
    (σ'' ρ ε m σ n).M ≥ Real.exp (- σ1_c i n) ∙ (σ' ρ ε m σ n).M := σ'_le_σ'' ρ ε
  _ ≥ (Real.exp (- σ1_c i n) * (1 / 3)) ∙ ((σ1 i) ⊗r ^[n]).M := by
    grw [← hσ1_le_σ' ρ ε m σ n, smul_smul]
  _ ≥ (Real.exp (- σ1_c i n) * (1 / 3)) ∙ ((iInf ((σ1 i) ⊗r ^[n])).M.H.eigenvalues
    apply smul_le_smul_of_nonneg_left
    · exact ((σ1 i) ⊗r ^[n]).M.H.iInf_eigenvalues_smul_one_le
    · positivity
  _ = (Real.exp (- σ1_c i n) * (1 / 3) * (σ1_mineig i)^n) ∙ 1 := by
    dsimp [σ1_mineig, iInf]
    rw [← Matrix.IsHermitian.spectrum_real_eq_range_eigenvalues]
    rw [← Matrix.IsHermitian.spectrum_real_eq_range_eigenvalues]
    rw [MState.mat_M, sInf_spectrum_spacePow (σ1 i) n, MState.mat_M, smul_smul]
  _ = Real.exp (- n * (σ1_c i n + (σ1_c i n) / n)) ∙ 1 := by
    rw [σ1_c identity i hn]
  _ ≥ Real.exp (-n * c' ε2 n) ∙ 1 := by
    apply smul_le_smul_of_nonneg_right
    · apply Real.exp_le_exp_of_le

```

```

      simp only [neg_mul, neg_le_neg_iff]
      apply mul_le_mul_of_nonneg_left _ (by positivity)
      dsimp [c']
      exact le_sup_left
    · exact zero_le_one

have hE3comm :  $\epsilon_2 n$  : Commute (E3  $\epsilon_2 n$ ).mat (( $\sum n$  ( $\rho \otimes_r^n$ ))).M.log.mat) :
  unfold E3 P2
  rw [HermitianMat.projLE_def]
  apply HermitianMat.cfc_commute
  apply Commute.sub_left
  · rfl
  · apply Commute.smul_left
    apply Commute.symm
    exact pinching_commutes ( $\rho \otimes_r^n$ ) ( $\sigma'' \rho \in m \sigma n$ )

-- Leo: I think there's a typo in the third eq. of this step:  $\rho$  should be
-- The next set of equations also have  $\rho_n$  instead of  $\rho^n$ .
-- (S88)
have hDleq  $\epsilon_2 n$  (h $\epsilon_2$  :  $0 < \epsilon_2$ ) (hn :  $0 < n$ ) : ( $\sum n$  ( $\rho \otimes_r^n$ ))|| $\sigma'' \rho \in m$ 
  .ofReal  $\square P1 \epsilon_2 n$ ,  $\sum n$  ( $\rho \otimes_r^n$ )) * ((R2  $\rho \sigma$  + .ofReal  $\epsilon_0$  + .ofReal
  .ofReal  $\square P2 \epsilon_2 n$ ,  $\sum n$  ( $\rho \otimes_r^n$ )) * (.ofReal (c'  $\epsilon_2 n$ ) - (R2  $\rho \sigma$  +
  -- see (S81) for comments on why that statement had to be changed
  -- (S85)
  -- (first inequality should have  $\sigma_n''$  instead of
  --  $\tilde{\sigma}_n''$ ; corrected in v4, where  $\tilde{\sigma}_n''$  took
  -- the place of  $\sigma_n''$ ) in this and other steps
have hE3leq : ( $1/n : \mathbb{R}$ ) * (E3  $\epsilon_2 n$ ).mat * (( $\sum n$  ( $\rho \otimes_r^n$ ))).M.log.mat -
  calc
  ( $1/n : \mathbb{R}$ ) * (E3  $\epsilon_2 n$ ).mat * (( $\sum n$  ( $\rho \otimes_r^n$ ))).M.log.mat - ( $\sigma'' \rho \in m$ 
  -- first inequality of (S85)
  have h $\square$ plognonpos :  $0 \leq -(\sum n$  ( $\rho \otimes_r^n$ ))).M.log.mat := by
    -- log of state is non-positive
    simp only [HermitianMat.log]
    rw [← HermitianMat.mat_neg, ← HermitianMat.cfc_neg ( $\sum n$  ( $\rho \otimes_r^n$ ))]
    rw [← HermitianMat.mat_zero]
    rw [← HermitianMat.val_eq_coe, ← HermitianMat.val_eq_coe]
    rw [Subtype.coe_le_coe, HermitianMat.cfc_nonneg_iff ( $\sum n$  ( $\rho \otimes_r^n$ ))]
Real.log)]
  intro i
  simp
  apply Real.log_nonpos
  · apply MState.eigenvalue_nonneg _
  · apply MState.eigenvalue_le_one
  rw [← sub_nonneg, ← mul_sub_left_distrib]
  conv =>

```

```

    rhs
    congr; rfl
    rw [← sub_add (-(σ'' ρ ε m σ n).M.log.mat) ((∑ n (ρ ⊗r ^[n])).M.log.mat) (←
    rw [add_comm_sub]
    rw [← add_sub_assoc]
    simp [sub_self (σ'' ρ ε m σ n).M.log.mat]
    simp only [Algebra.smul_mul_assoc, one_div]
    have hE3nonneg : 0 ≤ (E3 ε2 n).mat * -(∑ n (ρ ⊗r ^[n])).M.log.mat := by
      apply Commute.mul_nonneg _ hlognonpos (Commute.neg_right (hE3comm ε2 n))
      apply HermitianMat.projLE_nonneg
    apply smul_nonneg (inv_nonneg_of_nonneg (Nat.cast_nonneg' n)) hE3nonneg
    _ ≤ (1/n : ℝ) • (E3 ε2 n).mat * (-(Real.exp (-n * c' ε2 n) • (1 : HermitianMa
    -- intermediate step in 2nd ineq of S85
    simp only [mul_neg, neg_mul, neg_le_neg_iff]
    have hllogleq : (Real.exp (-(n * c' ε2 n)) • 1 : HermitianMat (H (i ^ n)) ℂ)
      -- apply hσ'' ε2 n to log by monotonicity
      rw [← HermitianMat.val_eq_coe, ← HermitianMat.val_eq_coe]
      rw [Subtype.coe_le_coe]
      exact HermitianMat.log_mono
        (Matrix.PosDef.smul Matrix.PosDef.one (Real.exp_pos (-
    (n * c' ε2 n))))
      (by simp using hσ'' ε2 n)
      rw [← sub_nonneg] at hllogleq
      rw [← sub_nonneg, ← mul_sub_left_distrib]
      simp only [one_div, Algebra.smul_mul_assoc]
      have hE3commlog : Commute (E3 ε2 n).mat ((σ'' ρ ε m σ n).M.log.mat - (Real.e
    (n * c' ε2 n)) • 1 : HermitianMat _ ℂ).log.mat) := by
        -- projector commutes with logs
        apply Commute.sub_right
        · -- prove Commute (E3 _) (σ'' _).log
          unfold E3 P2
          rw [HermitianMat.projLE_def]
          apply HermitianMat.cfc_commute
          apply Commute.sub_left
          · exact pinching_commutes (ρ ⊗r ^[n]) (σ'' ρ ε m σ n)
          · apply Commute.smul_left
            rfl
        · -- prove `Commute (E3 _) (_ 1).log`
          conv_rhs =>
            rw [HermitianMat.log_smul (by positivity)]
            simp only [
              Real.log_exp, neg_smul, HermitianMat.log_one,
              add_zero, NegMemClass.coe_neg,
              HermitianMat.val_eq_coe, selfAdjoint.val_smul,
              selfAdjoint.val_one
            ]

```



```

have hE2comm : Commute (E2 ε2 n).mat (((⊠ n (ρ ⊗r ^[n]))).M.log - (σ'' ρ ε m σ n))
-- TODO this needs to be extracted from here, it's badly redundant
  apply Commute.sub_right
  · unfold E2 P1 P2
    simp only [Pi.sub_apply, HermitianMat.mat_sub, HermitianMat.val_eq_coe]
    rw [HermitianMat.projLE_def, HermitianMat.projLE_def]
    apply Commute.sub_left
    · apply HermitianMat.cfc_commute
      apply Commute.sub_left
      · rfl
      · simp only [HermitianMat.val_eq_coe, HermitianMat.mat_smul, MState.marginalize
        Real.exp_ne_zero, not_false_eq_true, Commute.smul_left_iff]
        apply Commute.symm
        exact pinching_commutes (ρ ⊗r ^[n]) (σ'' ρ ε m σ n)
    · apply HermitianMat.cfc_commute
      apply Commute.sub_left
      · rfl
      · simp only [HermitianMat.val_eq_coe, HermitianMat.mat_smul, MState.marginalize
        Real.exp_ne_zero, not_false_eq_true, Commute.smul_left_iff]
        apply Commute.symm
        exact pinching_commutes (ρ ⊗r ^[n]) (σ'' ρ ε m σ n)
  · unfold E2 P1 P2
    simp only [Pi.sub_apply, HermitianMat.mat_sub, HermitianMat.val_eq_coe]
    rw [HermitianMat.projLE_def, HermitianMat.projLE_def]
    apply Commute.sub_left
    · apply HermitianMat.cfc_commute
      apply Commute.sub_left
      · exact pinching_commutes (ρ ⊗r ^[n]) (σ'' ρ ε m σ n)
      · simp
    · apply HermitianMat.cfc_commute
      apply Commute.sub_left
      · exact pinching_commutes (ρ ⊗r ^[n]) (σ'' ρ ε m σ n)
      · simp
have hE2leqInner : (n : ℝ)-1 * ((⊠ n (ρ ⊗r ^[n]))).M.mat * (E2 ε2 n).mat * ((⊠ n
  ((R2 ρ σ).toReal + ε0 + ε2) * ⊠ (⊠ n (ρ ⊗r ^[n]))).M, E2 ε2 n) := by
  open HermMul in
  -- (S82) -- see (S81) for comments
  have hE2leq ε2 (n : ℕ) (hε2 : 0 < ε2) : (1/n : ℝ) • (E2 ε2 n).mat * ((⊠ n
    refine rexp_mul_smul_proj_lt_mul_sub_le_mul_sub'
      (pinching_commutes (ρ ⊗r ^[n]) (σ'' ρ ε m σ n)) ?_ (σ''_posdef ρ ε m σ n)
    rw [← HermitianMat.zero_le_iff]
    apply MState.nonneg
  simp at hE2leq
  rw [← Complex.re_ofReal_mul (↑n)-1, ← smul_eq_mul, ← Matrix.trace_smul]
  rw [← RCLike.re_to_complex]
  conv =>

```

```

    enter [1, 2]
    rw [mul_assoc]
    enter [1, 2, 2]
    norm_cast
    rw [← HermitianMat.symmMul_of_commute hE2comm]
conv at hE2leq =>
    enter [ε2, n, hε2, 1, 2, 2]
    norm_cast
specialize hE2leq ε2 n hε2
rw [← HermitianMat.symmMul_of_commute hE2comm] at hE2leq
rw [← Matrix.mul_smul (∑ n (ρ ⊗r ^[n])) .M.mat (Complex.ofReal (n : ℝ))]
conv =>
    enter [1, 2, 1, 2]
    equals HermitianMat.mat ((n : ℝ)-1 • (HermitianMat.symmMul (E2
      (((∑ n) (ρ ⊗r ^[n])) .M.log - (σ'' ρ ε m σ n).M.log)))) =>
      rfl --TODO: Lemma to work with this better
    rw [← HermitianMat.inner_eq_re_trace (∑ n (ρ ⊗r ^[n])) .M _]
    rw [← HermitianMat.inner_smul_right]
    exact ((HermitianMat.inner_mono ((∑ n (ρ ⊗r ^[n])))).nonneg) hE2leq
simp at hE3leq
/-this and the `have` above are duplicates-/
have hE3comm : Commute (E3 ε2 n).mat (((∑ n (ρ ⊗r ^[n])) .M.log - (σ'' ρ ε m σ n).M.log)) =>
  apply Commute.sub_right
  · simp [(hE3comm ∑ ε2 n)]
  · unfold E3 P2
    simp
    rw [HermitianMat.projLE_def]
    apply HermitianMat.cfc_commute
    apply Commute.sub_left
    · exact pinching_commutes (ρ ⊗r ^[n]) (σ'' ρ ε m σ n)
    · simp
/- hE3comm -/
have hE3leqInner : (n : ℝ)-1 * ((∑ n (ρ ⊗r ^[n])) .M.mat * (E3 ε2 n).mat) ≤ 0
  open HermMul in
  rw [← Complex.re_ofReal_mul (↑n)-1 _, ← smul_eq_mul, ← Matrix.trace]
  rw [← RCLike.re_to_complex]
conv =>
    enter [1, 2]
    rw [mul_assoc]
    enter [1, 2, 2, 2]
    norm_cast
conv =>
    enter [1, 2, 1, 2, 2]
    rw [← HermitianMat.symmMul_of_commute hE3comm]
conv at hE3leq =>
    enter [1, 2, 2]

```

```

norm_cast
rw [← HermitianMat.symmMul_of_commute hE3comm] at hE3leq
conv at hE3leq =>
  lhs
  rw [← HermitianMat.mat_smul (n : ℝ)⁻¹ (HermitianMat.symmMul _ _)]
conv at hE3leq =>
  rhs
  rw [← HermitianMat.mat_smul _ (E3 ε2 n)]
rw [← Matrix.mul_smul (∑ n (ρ ⊗r ^[n])) .M .mat (Complex.ofReal (n : ℝ)⁻¹) (H
conv =>
  enter [1, 2, 1, 2]
  equals HermitianMat.mat ((n : ℝ)⁻¹ • (HermitianMat.symmMul (E3 ε2 n)
    (((∑ n (ρ ⊗r ^[n])) .M .log - (σ'' ρ ε m σ n).M .log)))) =>
    rfl --TODO: Lemma to work with this better (see above TODO)
  rw [← HermitianMat.inner_eq_re_trace (∑ n (ρ ⊗r ^[n])) .M ((n : ℝ)⁻¹ • (Herm
  rw [← HermitianMat.inner_smul_right]
  exact ((HermitianMat.inner_mono (∑ n (ρ ⊗r ^[n]))).nonneg) hE3leq)
simp only [IsMaximalSelfAdjoint.RCLike_selfadjMap, MState.mat_M, HermitianMa
  RCLike.re_to_complex, HermitianMat.inner_smul_right, ge_iff_le]
conv =>
  enter [1, 1, 1, 1]
  rw [mul_add, mul_add, add_mul, add_mul, trace_add, trace_add]
simp [← ENNReal.ofReal_coe_nnreal, NNReal.toReal]
rw [← ENNReal.ofReal_natCast, ← ENNReal.ofReal_div_of_pos (by simp [hn]), di
rw [ENNReal.ofReal_le_ofReal_iff']
· left
  rw [mul_add, mul_add]
  apply add_le_add_three
  · rw [HermitianMat.inner_def]
    simp

-- (S81)
/- A literal translation of the paper would read:
  (1/n : ℝ) • (E1 ε2 n).mat * ((∑ n (ρ ⊗r ^[n])) .M .log .mat - (σ'' n).M.
But this is simply not true! Because what happens when `∑ n (ρ ⊗r ^[n])`
it can? Then (S81) is an inequality of operators where the LHS has an op
infinity" eigenvalue, intuitively. This isn't something very well define
in our definitions. This only becomes mathematically meaningful when we
(S88): both sides are traced against `∑ n (ρ ⊗r ^[n])`, so that the 0 eig
is the version we state and prove, then.

Luckily, (S82) and (S85) are correct as written (in a particular interpr
there the problematic subspaces are indeed projected out by the E2 and E
-/
have hE1leq ε2 (n : ℕ) (hε2 : 0 < ε2) :
  ∑ (∑ n (ρ ⊗r ^[n])) .M, (((1 / (n : ℝ)) • (E1 ε2 n)).symmMul

```

```

      (((pinching_map (σ'' ρ ε m σ n)) (ρ ⊗r^[n])).M).log - ((
        [] (n (ρ ⊗r^[n])).M, ((R1 ρ ε).toReal + ε2) • (E1 ε2 n) []
      refine rexp_mul_smul_proj_lt_mul_sub_le_mul_sub
        (pinching_commutates (ρ ⊗r^[n]) (σ'' ρ ε m σ n)) (by positivity)
      rw [← HermitianMat.zero_le_iff]
      apply MState.nonneg

    simp only [HermitianMat.inner_def] at hE1leq
    conv at hE1leq =>
      enter [ε2, n, hε2, 1]
      rw [HermitianMat.symmMul_of_commute (commute_aux n (E := E1 ε2 n))]
    simp at hE1leq
    conv at hE1leq =>
      intro ε2 n hε2
      rw [← mul_assoc]
      pattern (pinching_map (σ'' ρ ε m σ n))
      change [] n
      apply (hE1leq ε2 n hε2)
      · apply hE2leqInner
      · apply hE3leqInner
    -- (S88)
    - = _ := by
      unfold E1 E2 E3
      simp [inner_sub_right]
      ring_nf
      apply EquationS88 <=> assumption

  clear hE_pos

  -- (S91)
  have hliminfDleq : Filter.atTop.liminf (fun n => [] (n (ρ ⊗r^[n]) || σ'' ρ ε
    (R1 ρ ε) + .ofReal (1 - ε.val) * (R2 ρ σ + .ofReal ε0 - R1 ρ ε)) :=
    -- Pick a sequence `ε2 n` that converges slowly enough that we ensure bo
    -- which otherwise depend on a 'constant' ε1 and ε2, both converge to z
    -- by looking at the max of the P1 and P2 terms.
    have this :=
      exists_liminf_zero_of_forall_liminf_limsup_le_with_UB (1 - ε) 0
      (fun x n => ENNReal.ofNNReal (P1 x n, [] n (ρ ⊗r^[n]) []),
        HermitianMat.inner_ge_zero (HermitianMat.projLE_nonneg _ _) ([] n (ρ
      (fun x n => ENNReal.ofNNReal (P2 x n, [] n (ρ ⊗r^[n]) []),
        HermitianMat.inner_ge_zero (HermitianMat.projLE_nonneg _ _) ([] n (ρ
      zero_lt_one ?_ ?_; rotate_left
    · have hliminfP1 ε2 (hε2 : 0 < ε2) := --(S76)
      LemmaS2liminf hε2 (fun n => [] n (ρ ⊗r^[n])) (σ'' ρ ε m σ) hliminf_le

    conv at hliminfP1 =>

```



```

    enter [ε2, hε2, 1, 1, n, 2]
    change P1 _ _
--The goal is now hliminfP1, up to stupid casting
intro x hx
specialize hliminfP1 (x, hx.le) hx
apply ENNReal.ofReal_mono at hliminfP1
convert ← hliminfP1 using 1
dsimp
conv =>
  enter [2, 1, n]
  rw [← ENNReal.ofReal_eq_coe_nnreal]
  refine ENNReal.ofReal_mono.map_liminf_of_continuousAt _ ?_ ?_ ?_
  · apply ENNReal.continuous_ofReal.continuousAt
  · use 1
    simp only [Filter.eventually_map, Filter.eventually_atTop,
      forall_exists_index]
    intro a x hx
    apply (hx x le_rfl).trans
    rw [← (λ x (p ⊗r ^[x])).tr, ← HermitianMat.one_inner]
    apply HermitianMat.inner_mono' (λ x (p ⊗r ^[x])).nonneg
    apply HermitianMat.projLE_le_one
  · use 0
    simp only [Filter.eventually_map, Filter.eventually_atTop]
    use 0
    intro _ _
    apply HermitianMat.inner_ge_zero
    · apply HermitianMat.projLE_nonneg
    · apply MState.nonneg
  · have hlimsupP2' ε2 (hε2 : 0 < ε2) :
      Filter.atTop.limsup (fun n => λ P2 ε2 n, λ n (p ⊗r ^[n])) = 0 := by
    apply le_antisymm
    · apply le_of_forall_pos_le_add
      intro ε1' hε1'
      let ε1 := min ε1' (1/2)
      have hε1 : 0 < (ε1 : ℝ) ∧ (ε1 : ℝ) < 1 := by
        constructor
        · rw [lt_min_iff]
          exact {hε1', by norm_num}
        · rw [min_lt_iff]
          exact 0r.inr (by norm_num)
      have hlimsupP2 ε2 (hε2 : 0 < ε2) (ε1 : Prob) (hε1 : 0 < (ε1 : ℝ) ∧ (ε1 :
(S77)
      LemmaS2limsup hε2 (fun n => λ n (p ⊗r ^[n])) (σ'' p ε m σ) (hlimsup_le ε1
      specialize hlimsupP2 (ε2, hε2.le) hε2 (ε1, {hε1.1.le, hε1.2.le}) hε1
      trans ε1
      · convert hlimsupP2

```

```

      simp only [Prob.coe_one_minus, sub_sub_cancel]
    · simp only [one_div, zero_add, inf_le_left, ε1]
  · apply Filter.le_limsup_of_frequently_le ?_ ?_
  · rw [Filter.frequently_atTop]
    intro n
    refine {n, le_refl, ?_}
    exact HermitianMat.inner_ge_zero (HermitianMat.projLE_nonneg _
      (λ n (p ⊗r ^[n])).nonneg)
  · apply Filter.isBoundedUnder_of
    use 1
    intro n
    rw [← (λ n (p ⊗r ^[n])).tr, ← HermitianMat.one_inner]
    exact HermitianMat.inner_mono' (λ n (p ⊗r ^[n])).nonneg
      (HermitianMat.projLE_le_one _)
--The goal is now hlimsupP2', up to stupid casting
intro x hx
specialize hlimsupP2' x hx
apply le_of_eq at hlimsupP2'
apply ENNReal.ofReal_mono at hlimsupP2'
convert ← hlimsupP2' using 1
swap
· simp
dsimp
conv =>
  enter [2, 1, n]
  exact (ENNReal.ofReal_eq_coe_nnreal _).symm
refine ENNReal.ofReal_mono.map_limsup_of_continuousAt _ ?_ ?_ ?_
· apply ENNReal.continuous_ofReal.continuousAt
· use 1
  simp only [Filter.eventually_map, Filter.eventually_atTop]
  use 0
  intro x hx
  rw [← (λ x (p ⊗r ^[x])).tr, ← HermitianMat.one_inner]
  apply HermitianMat.inner_mono' (λ x (p ⊗r ^[x])).nonneg
  apply HermitianMat.projLE_le_one
· use 0
  simp only [Filter.eventually_map, Filter.eventually_atTop, forall_e]
  intro a x hx
  grw [← hx x le_refl]
  apply HermitianMat.inner_ge_zero
  · apply HermitianMat.projLE_nonneg
  · apply MState.nonneg
rcases this with (ε2, hg1, hg2, hg3, hliminf_g1, hliminf_g2)

replace hDleq := Filter.liminf_le_liminf (Filter.eventually_atTop.mpr (
apply le_trans hDleq -- (S89)

```

```

have hP2zero : Filter.atTop.Tendsto (fun n ↦ .ofReal (P2 (ε2 n) n, λ n (p ⊗r ^[n]
  (.ofReal (c' (ε2 n) n) - (R2 p σ + .ofReal ε0 + .ofReal (ε2 n)))) (λ 0) := b
have hf : Filter.atTop.Tendsto (fun n ↦ .ofReal (P2 (ε2 n) n, λ n (p ⊗r ^[n]))
  refine tendsto_of_le_liminf_of_limsup_le bot_le ?_
  convert hliminf_g2
  apply ENNReal.ofReal_eq_coe_nnreal
obtain ⟨C, hC⟩ := hc' ε2 hg2
refine ENNReal.bdd_le_mul_tendsto_zero (b := C) (by finiteness) hf ?_
filter_upwards [hC] with a ha
grw [ha]
simp
conv =>
  enter [1, 1]
  rw [← Pi.add_def]
rw [ENNReal.liminf_add_of_right_tendsto_zero hP2zero _]
conv =>
  enter [1, 1, n]
  rw [add_assoc]
rw [liminf_const_add (Filter.atTop) _ (R1 p ε) (by isBoundedDefault) (by isBound
conv =>
  enter [1, 2, 1]
  rw [← Pi.add_def]
  enter [2, n]
  rw [mul_comm]
rw [ENNReal.liminf_add_of_left_tendsto_zero ?_ _]
· rw [ENNReal.add_le_add_iff_left hR1]
  apply le_trans (ENNReal.liminf_mul_le ?_ ?_)
  · rw [mul_comm]
    gcongr
    · -- Alex: This is hard to prove with hliminfP1, because in hliminfP1 the ε2
      -- the liminf, but here it is allowed to vary with n. We need to 'upgrade'
      -- the following fact, which should (in some form) be its own theorem:
      /- (∀ x, x > 0 → liminf (n ↦ f x n) ≤ y) →
        ∃ g : ℕ → ℝ, (∀ x, g x > 0) ∧ (liminf g = 0) ∧ (liminf (n ↦ f (g n) n) ≤
      -/
      --this is stated above as exists_liminf_zero_of_forall_liminf_le.
      -- ... but then this needs to match up with the ε2 ...
      --Ahh, no, so actually this `g` is how we want to pick our `ε2` above!
      convert hliminf_g1 using 3 with n
      apply ENNReal.ofReal_eq_coe_nnreal
    · conv =>
        enter [1, 1, n]
        rw [ENNReal.add_sub_add_eq_sub_right (by finiteness)]
        rw [Filter.limsup_const]
  · left
    apply ne_bot_of_le_ne_bot (b := ENNReal.ofReal ε0)

```

```

      · rwa [← bot_lt_iff_ne_bot, ENNReal.bot_eq_zero, ENNReal.ofReal_pos
        apply Filter.le_limsup_of_frequently_le'
        apply Filter.Eventually.frequently
        apply Filter.Eventually.of_forall
        intro x
        rw [add_right_comm, ← ENNReal.sub_add_eq_add_sub (add_le_add_left h
        exact le_add_self
      · left
        apply ne_top_of_le_ne_top (b := R2 ρ σ + ENNReal.ofReal ε₀ + 1) (by
        apply Filter.limsup_le_of_le (by isBoundedDefault) ?_
        apply Filter.Eventually.of_forall
        intro x
        suffices h : ε₂ x ≤ 1 by
          nth_grw 1 [h]
          simp
          exact (hg₂ x).le
      · rw [← ENNReal.ofReal_zero ]
        apply ENNReal.tendsto_ofReal
        rw [← NNReal.tendsto_coe] at hg₃
        exact hg₃

simp only [tsub_le_iff_right]
convert hliminfDleq using 1
rw [add_comm, ENNReal.add_right_inj hR1]
--Rewriting through the ENNReal madness to get (S92)
conv =>
  rhs
  rw [← ENNReal.sub_add_eq_add_sub hR1R2.le hR1, ← ENNReal.ofReal_toReal
    ← ENNReal.ofReal_toReal hR2, ← ENNReal.ofReal_sub _ (ENNReal.toReal
    ← ENNReal.ofReal_add (sub_nonneg.mpr (ENNReal.toReal_mono hR2 hR1R2
    ← ENNReal.ofReal_mul (by simp))]
  rhs
  rw [← ENNReal.toReal_sub_of_le hR1R2.le hR2]

equals (1 - ε'.val) * (R2 ρ σ - R1 ρ ε).toReal =>
  unfold ε₀
  field_simp [show 1 - ε.val ≠ 0 from ne_of_gt (sub_pos.mpr hε)]
  ring_nf
  rw [ENNReal.ofReal_mul (by simp), Prob.ofNNReal_toNNReal,
    ENNReal.ofReal_toReal (by simp [hR1, hR2]), Prob.coe_one_minus]

/-- Lemma 7 from the paper. We write `ε` for their `̵`. -
/
private theorem Lemma7 (ρ : MState (H i)) {ε : Prob} (hε : 0 < ε ∧ ε < 1) (
  (R2 ρ σ ≥ R1 ρ ε) →
  ∀ ε' : Prob, (hε' : 0 < ε' ∧ ε' < ε) → -- ε' is written as ̵ in

```

```

    ∃ σ' : (n : ℕ) → IsFree (i := i ^ n),
    R2 ρ σ' - R1 ρ ε ≤ .ofNNReal (1 - ε' : Prob) * (R2 ρ σ - R1 ρ ε)
    := by
--This proof naturally splits out into LemmaS62:
-- `lim inf n→∞ 1/n D(E_n(ρ^⊗n)||σ'^n) - R1,ε ≤ (1 - ~ε)(R2 - R1,ε).`
--which is proved above. Here, set up the preliminaries (appropriate finiteness
-- conditions, nonzeroness, etc.) get S62, prove S61, and the conclusion is just `

--First deal with the easy case of R1 = R2.
intro hR1R2 ε' {hε'₁, hε'₂}
rw [ge_iff_le, le_iff_lt_or_eq, or_comm] at hR1R2
rcases hR1R2 with hR1R2|hR1R2
· use σ
  simp [hR1R2]
--This leaves us with the stronger statement that R1 < R2 strictly.
--Before proceeding, let's reduce to the case that they're finite.
have hR1 : R1 ρ ε ≠ τ := hR1R2.ne_top
rcases eq_or_ne (R2 ρ σ) τ with hR2|hR2
· rw [hR2, ENNReal.top_sub hR1, ENNReal.mul_top', if_neg]
  · simp
  · have : ε'.val < 1 := hε'₂.trans hε.2
    rcases ε' with {ε', hε'₁, hε'₂}
    simp only [Prob.toNNReal, Prob.coe_one_minus, ENNReal.coe_eq_zero]
    rw [Subtype.ext_iff, val_eq_coe, val_eq_coe, coe_zero, coe_mk]
    grind

let ε₀ : ℝ := (R2 ρ σ - R1 ρ ε).toReal * (ε - ε') / (1 - ε)
have hε₀ : 0 < ε₀ :=
  have := sub_pos.mpr (show ε.val < 1 from hε.2)
  have := sub_pos.mpr (show ε'.val < ε from hε'₂)
  have : 0 < (SteinsLemma.R2 ρ σ - SteinsLemma.R1 ρ ε).toReal :=
    ENNReal.toReal_pos (tsub_pos_of_lt hR1R2).ne' (ENNReal.sub_ne_top hR2)
  by positivity
have hε₀' : (R1 ρ ε).toReal ≤ (R2 ρ σ).toReal + ε₀ := by
  dsimp [ε₀]
  rw [← sub_nonneg]
  have _ := sub_pos.mpr (show ε.val < 1 from hε.2)
  have _ := sub_pos.mpr (show ε'.val < ε from hε'₂)
  rw [ENNReal.toReal_sub_of_le hR1R2.le (by finiteness)]
  field_simp
  suffices h : 0 ≤ ((R2 ρ σ).toReal - (R1 ρ ε).toReal) * ((↑ε - ↑ε') + (1 - ↑ε)) b
    convert h using 1
    · exact zero_mul _
    · ring_nf
  rw [← ENNReal.toReal_sub_of_le hR1R2.le (by finiteness)]
  positivity

```

```

-- m exists because  $R2 + \varepsilon_0$  is strictly above  $R2$ , which is the  $\liminf$ .
obtain (m, hm) :=
  have h :  $R2 \rho \sigma < R2 \rho \sigma + .ofNNReal (\varepsilon_0, h\varepsilon_0.le) :=$ 
    ENNReal.lt_add_right hR2 (by simp [← NNReal.coe_eq_zero, h\varepsilon_0.ne'])
  (Filter.frequently_lt_of_liminf_lt (h := h)).forall_exists_of_atTop 1

have qRel_σ''_le_σ' n :  $\square(\rho \otimes_r^{[n]} \|\sigma'' \rho \varepsilon m \sigma n\| \leq \square(\rho \otimes_r^{[n]} \|\sigma' \rho \varepsilon m \sigma n\|)$ 
  rw [← Real.log_exp (σ1_c i n)]
  apply qRelEntropy_le_add_of_le_smul
  rw [← inv_smul_le_iff_of_pos (by positivity), ← Real.exp_neg]
  apply σ'_le_σ''

have qRel_σ'_le_σ'' n :  $\square(\rho \otimes_r^{[n]} \|\sigma' \rho \varepsilon m \sigma n\| \leq \square(\rho \otimes_r^{[n]} \|\sigma'' \rho \varepsilon m \sigma n\|)$ 
  rw [← Real.log_exp (σ1_c i n)]
  apply qRelEntropy_le_add_of_le_smul
  apply σ''_le_σ'

-- Definition of the pinching map w.r.t.  $\sigma''$  in Eq. (S55)
let □ n := pinching_map (σ'' ρ ε m σ n)

-- Number of distinct eigenvalues of  $\sigma''$  in Eq. (S56) is
-- Fintype.card (spectrum  $\mathbb{R}$  (σ'' n).m), which is  $d_n$  in the paper.
have hdl n : Fintype.card (spectrum  $\mathbb{R}$  (σ'' ρ ε m σ n).m) ≤ n + 1 := by
  rw [(σ'' ρ ε m σ n).M.H.card_spectrum_eq_image (A := (σ'' ρ ε m σ n).m)]
  rcases n.eq_zero_or_pos with rfl | hn
  · have _ : Unique (H (i ^ 0)) := by
      rw [spacePow_zero]
      infer_instance
    simp
  rw [← Set.ncard_coe_finset]
  simp only [Finset.coe_image, Finset.coe_univ, Set.image_univ]
  have eq :  $\exists (e : \text{Equiv.Perm } \_), (\sigma'' \rho \varepsilon m \sigma n).M.H.eigenvalues =$ 
    (fun x ↦ (σ''_unnormalized ρ ε m σ n).trace-1 * Real.exp (f_map i n
      convert (σ' ρ ε m σ n).M.cfc_eigenvalues (fun x ↦ (σ''_unnormalized ρ
        rw [HermitianMat.cfc_const_mul, ← σ''_unnormalized, σ''])
      rcases eq with (eq, heq)
    rw [heq]
  simp only [Set.range_comp, MState.mat_M, EquivLike.range_eq_univ, Set.i
  let S : Set  $\mathbb{R}$  := (fun x => Real.exp (f_map i n x)) '' Set.Icc ((σ1_mine
  have h_card_subs : Set.ncard S ≤ n + 1 ∧ S.Finite := by
    exact f_image_bound (σ1_mineig i) n (mineig_pos i) hn (log_le_f i) (f
  let S2 : Set  $\mathbb{R}$  := (fun x => (σ''_unnormalized ρ ε m σ n).trace-1 * Real
  obtain (h_card_subs2, h_s2_finite) : Set.ncard S2 ≤ n + 1 ∧ S2.Finite :
    have hS2 : S2 = ((σ''_unnormalized ρ ε m σ n).trace-1 * .) '' S := by
      simp [S, S2, Set.image_image]

```

```

    constructor
    · rw [hS₂]
      exact (Set.ncard_image_le h_card_subs.right).trans h_card_subs.left
    · rw [hS₂]
      exact h_card_subs.right.image _
  refine (Set.ncard_le_ncard (t := (fun x => (σ''_unnormalized ρ ε m σ n).trace⁻¹
    Real.exp (f_map i n x)) '' Set.Icc (((σ₁_mineig i) ^ n) / 3) 1) ?_ h_s₂_finite
  apply Set.image_mono
  rintro _ {k, rfl}
  refine {?_, MState.eigenvalue_le_one _ _}
  refine le_trans ?_ (ciInf_le (Finite.bddBelow_range _) k)
  refine le_trans ?_ ((HermitianMat.H _).iInf_eigenvalues_le (hσ₁_le_σ' ρ ε m σ n))
  dsimp [σ₁_mineig, iInf]
  rw [← Matrix.IsHermitian.spectrum_real_eq_range_eigenvalues]
  rw [← Matrix.IsHermitian.spectrum_real_eq_range_eigenvalues]
  rw [HermitianMat.val_eq_coe, HermitianMat.mat_smul]
  rw [spectrum.smul_eq_smul _ _ (CFC.spectrum_nonempty ℝ _ ((σ₁ i) ⊗_r^[n]).M.H)]
  rw [Real.sInf_smul_of_nonneg (by norm_num)]
  simp [MState.mat_M, div_eq_inv_mul, sInf_spectrum_spacePow]

  have hdpos n : 0 < Fintype.card (spectrum ℝ (σ'' ρ ε m σ n).M) := by
    rw [Fintype.card_pos_iff, Set.nonempty_coe_sort]
    apply IsSelfAdjoint.spectrum_nonempty
    exact (σ'' ρ ε m σ n).M.H

  -- Eq. (S60)
  have qRel_ent_bound n : [] (ρ ⊗_r^[n] || σ' ρ ε m σ n) ≤ ENNReal.ofReal (Real.log (n +
    [] (ρ ⊗_r^[n] || σ' ρ ε m σ n) ≤ ENNReal.ofReal (Real.log (Fintype.card (spectrum ℝ
    qRelativeEnt_op_le (pinching_bound ..)
    _ ≤ ENNReal.ofReal (Real.log (n + 1)) := by
      grw [hdle n]
      · exact_mod_cast le_rfl
      · simp [hdpos n]

  -- Eq. (S61)
  have hliminf : Filter.atTop.liminf (fun n => [] (ρ ⊗_r^[n] || σ' ρ ε m σ n) / n) =
    Filter.atTop.liminf (fun n => [] ([] n (ρ ⊗_r^[n] || σ' ρ ε m σ n) / n) :
  trans Filter.atTop.liminf fun n => [] (ρ ⊗_r^[n] || σ' ρ ε m σ n) / n
  · have hg : Filter.atTop.Tendsto (fun n => ENNReal.ofReal (σ₁_c i n) / n) (0) :
    rw [← ENNReal.tendsto_toReal_iff_of_eventually_ne_top ?_ ENNReal.zero_ne_top]
    · simp [ENNReal.toReal_ofReal (σ₁_c_pos i _).le]
      exact σ₁_c_div_lim i
    · rw [Filter.eventually_atTop]
      use 1
      intros
      finiteness

```

```

apply le_antisymm
· nth_rw 2 [← ENNReal.liminf_add_of_right_tendsto_zero hg]
conv =>
  enter [2, 1, n]
  rw [Pi.add_apply, ← ENNReal.add_div]
  apply Filter.liminf_le_liminf (β := ℝ≥0∞)
  rw [Filter.eventually_atTop]
  use 1
  intro n_
  exact ENNReal.div_le_div (qRel_σ' _le_σ' n) (by rfl)
-- A copy of the · above with σ' and σ'' swapped
· nth_rw 2 [← ENNReal.liminf_add_of_right_tendsto_zero hg]
conv =>
  enter [2, 1, n]
  rw [Pi.add_apply, ← ENNReal.add_div]
  apply Filter.liminf_le_liminf (β := ℝ≥0∞)
  rw [Filter.eventually_atTop]
  use 1
  intro n_
  exact ENNReal.div_le_div (qRel_σ'' _le_σ' n) (by rfl)
· -- Eq (S59) has a minus sign, which gets complicated when one of the
-- However, I don't think we need this version with the minus sign.
have h_pinching := fun n ↦ pinching_pythagoras (ρ ⊗r ^[n]) (σ'' ρ ∈ m
simp only [h_pinching, ENNReal.add_div, ← Pi.add_apply]
conv_lhs =>
  apply ENNReal.liminf_add_of_left_tendsto_zero
  tactic =>
    apply tendsto_of_tendsto_of_tendsto_of_le_of_le
      (g := (0 : ℕ → ℝ≥0∞)) (h := fun n ↦ ENNReal.ofReal (Real.log n))
    · exact tendsto_const_nhds
    · -- Basically that  $\lim_{n \rightarrow \infty} \log n / n = 0$ 
      rw [← Filter.tendsto_sub_atTop_iff_nat 1]
      apply Filter.Tendsto.congr' (f₁ := fun (n : ℕ) ↦ ENNReal.ofReal
    · simp only [Filter.EventuallyEq, ← ENNReal.ofReal_natCast, Fil
      use 2; intros b hb
      convert ENNReal.ofReal_div_of_pos (y := b - 1) (by rify at b
    · norm_cast
      omega
    · norm_cast; symm; apply Int.subNatNat_of_le
      omega
      refine Filter.Tendsto.comp (g := fun r ↦ ENNReal.ofReal (Real.l
        ?_ tendsto_natCast_atTop_atTop
      rw [← ENNReal.ofReal_zero]
      apply ENNReal.tendsto_ofReal
      convert Real.tendsto_pow_log_div_mul_add_atTop 1 (-
1) 1 (zero_ne_one.symm) using 3

```



```

      · simp
      · ring
    · positivity
    · intro n
      exact ENNReal.div_le_div (qRel_ent_bound n) le_rfl

use fun n ↦ {σ' ρ ∈ m σ n, σ'_free ρ ∈ m σ n}
rw [R2, hliminf]
exact EquationS62 ρ σ hε'₁ hε'₂ hε.2 hR1R2 hR1 hR2 hε₀ hε₀' m hm

/-- Lemma 7 gives us a way to repeatedly "improve" a sequence σ to one with a smaller gap.
The paper paints this as pretty much immediate from Lemma7, but we need to handle the base case R1. -/
private noncomputable def Lemma7_improver (ρ : MState (H i)) {ε : Prob} (hε : 0 < ε)
  --The parameters above are the "fixed" parameters that we'll improve
  --It takes one sequence of free states, `(n : ℕ) → IsFree (i := i ^ n)`, and gives another
  ((n : ℕ) → IsFree (i := i ^ n)) → ((n : ℕ) → IsFree (i := i ^ n)) :=
fun σ ↦
  if h : R2 ρ σ ≥ R1 ρ ε then
    (Lemma7 ρ hε σ h ε' hε').choose
  else
    σ --The gap was already 0 (or even, negative!) so leave it unchanged.

/-- The Lemma7_improver does its job at shrinking the gap. -
/
theorem Lemma7_gap (ρ : MState (H i)) {ε : Prob} (hε : 0 < ε ∧ ε < 1) {ε' : Prob} (hε' : 0 < ε')
  ∀ σ,
    letI σ' := Lemma7_improver ρ hε hε' σ;
    R2 ρ σ' - R1 ρ ε ≤ .ofNNReal (1 - ε' : Prob) * (R2 ρ σ - R1 ρ ε) := by
  intro σ
  dsimp [SteinsLemma.Lemma7_improver]
  split_ifs with h
  · exact (SteinsLemma.Lemma7 ρ hε σ h ε' hε').choose_spec
  · push_neg at h
    rw [tsub_eq_zero_of_le h.le]
    exact zero_le _

end Lemma7

/-- Theorem 1 in https://arxiv.org/pdf/2408.02722v3 -/
theorem GeneralizedQSteinsLemma {i : ι} (ρ : MState (H i)) {ε : Prob} (hε : 0 < ε ∧ ε < 1)
  Filter.atTop.Tendsto (fun n ↦ -log β_ε(ρ ⊗_r^n || IsFree) / n) (□) (□_r^∞ ρ) := by
  --It suffices to show limsup LHS ≤ RHS and liminf LHS ≥ RHS.
  refine tendsto_of_le_liminf_of_limsup_le ?_ ?_
  · -- the "key part" of the "opposite inequality".

```

```

--We need to pick an  $\varepsilon'$  (a  $\tilde{\varepsilon}$  in the paper). The only constraint
--less than  $\varepsilon$ . We take  $\varepsilon' := \varepsilon/2$ .
--TODO: Should we have an HDiv Prob Nat instance?
let  $\varepsilon' : \text{Prob} := (\varepsilon/2, \text{by constructor } <;> \text{linarith } [\varepsilon.\text{zero\_le\_coe}, \varepsilon.\text{co}])$ 
have  $h\varepsilon' : 0 < \varepsilon' \wedge \varepsilon' < \varepsilon := \text{by constructor } <;> \text{change } (\_ : \mathbb{R}) < (\_ :$ 

--Take some initial sequence  $\sigma_1$ . We need to pick it so that  $R_2 \rho \sigma_1 \neq \tau$ 
--it by applying Lemma 7. Taking the full-rank state of dimension  $H i$ 
have  $(\sigma_{\text{free}}, h\sigma_{\text{free}}) := \text{free\_fullRank } i$ 
set  $\sigma_1 : (n : \mathbb{N}) \rightarrow \text{IsFree } (i := i \wedge n) := \text{fun } n \mapsto$ 
   $(\sigma_{\text{free}} \otimes_r^n, \text{IsFree.npow } h\sigma_{\text{free}}.2 \ n)$  with  $h\sigma_1$ 
have  $h\sigma_1_{\text{top}} : R_2 \rho \sigma_1 \neq \tau := \text{by}$ 
  rw  $[R_2, \leftarrow \text{Filter.liminf\_nat\_add } \_ \ 1]$ 
  simp  $[\sigma_1, \text{mul\_comm } \_ \ (q\text{RelativeEnt } \_ \ \_)]$ 
  conv =>
    enter  $[1, 1, 1, n]$ 
    rw  $[\text{ENNReal.mul\_div\_cancel\_right } (\text{by positivity}) (\text{by finiteness})]$ 
    have  $\_ := \text{HermitianMat.nonSingular\_of\_posDef } h\sigma_{\text{free}}.1$ 
    simp  $[q\text{RelativeEnt\_ne\_top}]$ 
clear  $h\sigma_1$ 
--Repeat the Lemma7 improvement process to drive the gap down
let  $\sigma_k : \mathbb{N} \rightarrow (n : \mathbb{N}) \rightarrow \text{IsFree } (i := i \wedge n) := \text{fun } k \mapsto$ 
   $(\text{Lemma7\_improver } \rho \ h\varepsilon \ h\varepsilon')^k \ \sigma_1$ 

--The gap between  $R_{\{1, \varepsilon\}}$  and  $R_2$  for  $\sigma_k$  goes to 0 as  $k \rightarrow \infty$ .
have  $h\sigma_k_{\text{gap}} : \text{Filter.atTop.Tendsto } (\text{fun } k \mapsto R_2 \rho (\sigma_k \ k) - R_1 \rho \varepsilon) \ (\square \ 0)$ 
  suffices  $h : \forall (k : \mathbb{N}), R_2 \rho (\sigma_k \ k) - R_1 \rho \varepsilon \leq (1 - \varepsilon')^k * (R_2 \rho \sigma_1$ 
    refine  $\text{tendsto\_nhds\_bot\_mono' } ?\_ \ h$ 
    conv =>
      enter  $[3, 1]$ 
      equals  $0 * (R_2 \rho \sigma_1 - R_1 \rho \varepsilon) => \text{simp}$ 
      apply  $\text{ENNReal.Tendsto.mul\_const}$ 
      · simp only  $[\text{ENNReal.tendsto\_pow\_atTop\_nhds\_zero\_iff}]$ 
      --This should just be `simp` or `bound` at this point. TODO.
      simp  $[\text{Prob.toNNReal}, \leftarrow \text{NNReal.coe\_lt\_coe}, h\varepsilon'.1]$ 
      · right; exact  $\text{ENNReal.sub\_ne\_top } h\sigma_1_{\text{top}}$ 
    suffices  $h : \forall (m \ k : \mathbb{N}), R_2 \rho (\sigma_k \ (m + k)) - R_1 \rho \varepsilon \leq (1 - \varepsilon')^k * ($ 
      convert  $h \ 0$ ; simp
    intro  $m \ k$ ; induction  $k$  generalizing  $m$ 
    · simp  $[\sigma_k]$ 
    rename  $i \ k \ ih$ 
    have  $\sigma_k_{\text{succ}} (n) : \sigma_k (n + 1) = \text{Lemma7\_improver } \rho \ h\varepsilon \ h\varepsilon' (\sigma_k \ n) :=$ 
       $\text{Function.iterate\_succ\_apply' } \_ \ .$ 
    rw  $[\leftarrow \text{add\_assoc}, \sigma_k_{\text{succ}}, \text{pow\_succ}]$ 
    grw  $[\text{Lemma7\_gap } \rho \ h\varepsilon \ h\varepsilon' (\sigma_k \ (m + k)), ih \ m]$ 
    ring_nf

```

```

rfl

replace hσk_gap : Filter.atTop.liminf (fun k ↦ R2 ρ (σk k)) ≤ R1 ρ ε := by
  rw [ENNReal.tendsto_sub_const_nhds_zero_iff] at hσk_gap
  grw [Filter.liminf_le_limsup, hσk_gap]

rw [R1] at hσk_gap
grw [← hσk_gap]; clear hσk_gap

have hReg := RelativeEntResource.tendsto_enreal ρ
replace hReg := hReg.liminf_eq
rw [← hReg]; clear hReg

unfold R2
/- The idea is now that: the LHS is the liminf over all n, of the minimum free σ
   the RHS is the liminf over a particular subsequence, given by σk, which is free
   the math is complicated a bit by the fact that the RHS is a _double_ liminf. To
   deal with by talking about the sequences `σ_{n_k, *} = σ_{n_k, k}` (below Eq (1))
   actually construct such a subsequence here, we just unfold the bounds repeated
-/
refine Filter.le_liminf_of_le (by isBoundedDefault) ?_
apply Filter.Eventually.of_forall fun _ ↦ ?_
refine Filter.liminf_le_liminf ?_
apply Filter.Eventually.of_forall fun _ ↦ ?_
gcongr
rw [iInf_subtype']
exact iInf_le _ _

· --the "strong converse" part
conv =>
  enter [1, 1, n, 1, 1]
  rw [← OptimalHypothesisRate.Lemma3 ε IsCompact_IsFree free_convex]

--Let σm be the state minimizing  $\|(\rho \otimes^m \sigma_m)\|$  over free states. This is guaranteed
-- (1) the divergence is continuous and (2) the set of free states is compact.
have σm_exists (m : ℕ) := IsCompact_IsFree.exists_isMinOn_lowerSemicontinuousOn
  Set.Nonempty.of_subtype (f := fun σ ↦  $\|(\rho \otimes^m \sigma)\|$ ) (by fun_prop)

have hσm1 (m) := (σm_exists m).choose_spec.left
have hσm2 (m) := (σm_exists m).choose_spec.right
generalize σm_def : (fun m ↦ (σm_exists m).choose) = σm
simp_rw [congrFun σm_def] at hσm1 hσm2
clear σm_def σm_exists

--Let σ1 be the full-rank free state
have ⟨σ1, hσ1_pos, hσ1_free⟩ := FreeStateTheory.free_fullRank i

```

```

--`h` is Eq (14)
have _ := HermitianMat.nonSingular_of_posDef hσ1_pos
have h (m : ℕ) (hm : m ≥ 1) := Lemma6 hm ρ σ1 (σm m) hε.2

--Update `h` to Eq (15)
have h2 (m : ℕ) : (fun n ↦ -log β- ε(ρ ⊗r ^[n]||IsFree) / n) ≤f[Filter.atTop]
  (fun n ↦ -log β- ε(ρ ⊗r ^[n]||{(Lemma6_on m σ1 (σm m)) n}) / n) := by
  rw [Filter.EventuallyLE]
  apply Filter.Eventually.of_forall
  intro n
  gcongr
  apply OptimalHypothesisRate.negLog_le_singleton
  apply Lemma6_on_IsFree hσ1_free hσm1
replace h (m) (hm) := (Filter.limsup_le_limsup (h2 m)).trans (h m hm)
clear h2

--Update `h` to Eq (16)
conv at h =>
  enter [m, hm, 2, 1]
  exact (IsMinOn.iInf_eq (hσm1 m) (hσm2 m)).symm

apply tendsto_le_of_eventuallyLE tendsto_const_nhds (RelativeEntResource)
rw [Filter.EventuallyLE, Filter.eventually_atTop]
use 1
convert h using 7
· exact OptimalHypothesisRate.Lemma3 ε IsCompact_IsFree free_convex
· symm
  apply iInf_subtype''

/-- Theorem 4, which is also called the Generalized quantum Stein's lemma i
What they state as an equality of limits, which don't exist per se in Mathlib
of a number (which happens to be `RegularizedRelativeEntResource`) to which
-/
theorem limit_hypotesting_eq_limit_rel_entropy (ρ : MState (H i)) (ε : Prob
  ∃ d : ℝ≥0,
    Filter.atTop.Tendsto (fun n ↦ -log β- ε(ρ ⊗r ^[n]||IsFree) / n) (□ d)
    ∧
    Filter.atTop.Tendsto (fun n ↦ (□ σ ∈ IsFree, □(ρ ⊗r ^[n]||σ)) / n) (□ d)
    := by
  use □r∞ ρ -- Regularized relative entropy of resource (RegularizedRelativeEntResource)
  constructor
  · exact GeneralizedQSteinsLemma ρ hε -- Theorem 1 in Hayashi & Yamasaki
  · exact RelativeEntResource.tendsto_enreal ρ -- The regularized relative

```

```

/--

```

```

info: 'SteinsLemma.limit_hypotesting_eq_limit_rel_entropy' depends on axioms: [prop
  sandwichedRenyiEntropy_DPI_ax,
  Classical.choice,
  Quot.sound]
-/
#guard_msgs in
#print axioms limit_hypotesting_eq_limit_rel_entropy

```

1.450 QuantumInfo/Finite/Unitary.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg, Rodolfo Soldati
-/
import QuantumInfo.Finite.MState

/-! # Unitary operators on quantum state

This file is intended for lemmas about unitary matrices (`Matrix.unitaryGroup`) and
`Bra`s, `Ket`s, and `MState` mixed states.

This is imported by `CPTPMap` to define things like unitary channels, Kraus operator
complementary channels, so this file itself does not discuss channels yet. -
/

noncomputable section

open RealInnerProductSpace
open InnerProductSpace

namespace HermitianMat

variable {α : Type*} [RCLike α] {n : Type*} [Fintype n] [DecidableEq n]
variable (A B : HermitianMat n α) (U : Matrix.unitaryGroup n α)

@[simp]
theorem trace_conj_unitary : (conj U.val A).trace = A.trace := by
  simp [Matrix.trace_mul_cycle, conj, ← Matrix.star_eq_conjTranspose, trace]

@[simp]
theorem le_conj_unitary : A.conj U.val ≤ B.conj U ↔ A ≤ B := by
  rw [← sub_nonneg, ← sub_nonneg (b := A), ← map_sub]
  constructor

```

```

    · intro h
      simp [HermitianMat.conj_conj] using conj_nonneg (star U).val h
    · exact fun h ↦ conj_nonneg U.val h

@[simp]
theorem inner_conj_unitary : [A.conj U.val, B.conj U.val] = [A, B] := by
  dsimp [conj]
  simp only [inner_eq_re_trace, mat_mk]
  rw [← mul_assoc, ← mul_assoc, mul_assoc _ _ U.val]
  rw [Matrix.trace_mul_cycle, ← mul_assoc, ← mul_assoc _ _ A.mat]
  simp [← Matrix.star_eq_conjTranspose]

/--
The eigenvalues of a Hermitian matrix conjugated by a unitary matrix are the
same as the eigenvalues of the original matrix.
-/
@[simp]
theorem eigenvalues_conj : (A.conj U.val).H.eigenvalues = A.H.eigenvalues :=
  rw [Matrix.IsHermitian.eigenvalues_eq_eigenvalues_iff]
  change (U.val * A.mat * star U.val).charpoly = _
  rw [Matrix.charpoly_mul_comm, ← mul_assoc, U.2.1, one_mul]

end HermitianMat

namespace MState

variable {d d₁ d₂ d₃ : Type*}
variable [Fintype d] [Fintype d₁] [Fintype d₂] [Fintype d₃]
variable [DecidableEq d]
variable {ψ φ f : Ket d}

/-- Conjugate a state by a unitary matrix (applying the unitary as an evolution)
-/
def U_conj (ρ : MState d) (U : [d]) : MState d where
  M := ρ.M.conj U.val
  nonneg := HermitianMat.conj_nonneg U.val ρ.nonneg
  tr := by simp

/-- `MState.U_conj`, the action of a unitary on a mixed state by conjugation.
The  $\triangleleft$  notation comes from the theory of racks and quandles, where this is a
conjugation-like operation. -/
scoped[MState] notation:80 U:80 "  $\triangleleft$  " ρ:81 => MState.U_conj ρ U

/-- You might think this should only be true up to permutation, so that it
is  $\exists \sigma : \text{Equiv.Perm } d, (\rho.U\_conj U).spectrum = \rho.spectrum.relabel \sigma$ . But since
the eigenvalues of a matrix are always canonically sorted, this is actually an equality.
-/

```

```

-/
@[simp]
theorem U_conj_spectrum_eq (ρ : MState d) (U :  $\mathbb{C}[d]$ ) :
  (ρ.U_conj U).spectrum = ρ.spectrum := by
  simp [spectrum, U_conj]

@[simp]
theorem inner_U_conj (ρ σ : MState d) (U :  $\mathbb{C}[d]$ ) :  $\langle U \triangleleft \rho, U \triangleleft \sigma \rangle_{\text{Prob}} = \langle \rho, \sigma \rangle_{\text{Prob}}$ 
  simp [U_conj, inner_def]

/-- The No-cloning theorem, saying that if states  $\psi$  and  $\phi$  can both be perfectly
unitary  $U$  and a fiducial state  $f$ , and they aren't identical (their inner product
then the two states must be orthogonal to begin with. In short: only orthogonal states
cloned. -/
theorem no_cloning {U :  $\mathbb{C}[d \times d]$ }
  (hψ : U  $\triangleleft$  pure (ψ  $\otimes$  f) = pure (ψ  $\otimes$  ψ))
  (hφ : U  $\triangleleft$  pure (φ  $\otimes$  f) = pure (φ  $\otimes$  φ))
  (H :  $\langle \text{pure } \psi, \text{pure } \phi \rangle_{\text{Prob}} < (1 : \mathbb{R})$ ) :
     $\langle \text{pure } \psi, \text{pure } \phi \rangle_{\text{Prob}} = (0 : \mathbb{R})$  := by
  set ρψ := pure ψ
  set ρφ := pure φ
  have h1 :  $\langle \rho\psi, \rho\phi \rangle_{\text{Prob}} * \langle \rho\psi, \rho\phi \rangle_{\text{Prob}} = \langle \text{pure } (\psi \otimes \psi), \text{pure } (\phi \otimes \phi) \rangle_{\text{Prob}}$  := by
    grind only [pure_prod_pure, prod_inner_prod]
  have h2 : ( $\langle \text{pure } (\psi \otimes \psi), \text{pure } (\phi \otimes \phi) \rangle_{\text{Prob}} : \mathbb{R}$ ) =  $\langle U \triangleleft \text{pure } (\psi \otimes f), U \triangleleft \text{pure } (\phi \otimes f) \rangle_{\text{Prob}}$ 
    grind only [pure_prod_pure]
  replace h2 : ((pure (ψ  $\otimes$  ψ)).m * (pure (φ  $\otimes$  φ)).m).trace.re = (ρψ.m * ρφ.m).trace.re
  convert ← h2
  simp +zetaDelta only [inner_U_conj, pure_prod_pure, prod]
  simp [inner, ← Matrix.mul_kronecker_mul, pure_mul_self,
    Matrix.trace_kronecker]
  have h3 : (ρψ.m * ρφ.m).trace.re * ((ρψ.m * ρφ.m).trace.re - 1) = 0 := by
    rw [mul_sub, sub_eq_zero, mul_one]
    exact congr(Subtype.val $h1).trans h2
  rw [mul_eq_zero] at h3
  apply h3.resolve_right
  exact sub_ne_zero_of_ne H.ne

end MState

```

1.451 QuantumInfo/ForMathlib.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.

```

Authors: Alex Meiburg

```

-/
import QuantumInfo.ForMathlib.ContinuousLinearMap
import QuantumInfo.ForMathlib.ContinuousSup
import QuantumInfo.ForMathlib.Filter
import QuantumInfo.ForMathlib.HermitianMat
import QuantumInfo.ForMathlib.Isometry
import QuantumInfo.ForMathlib.LinearEquiv
import QuantumInfo.ForMathlib.MatrixNorm.TraceNorm
import QuantumInfo.ForMathlib.Matrix
import QuantumInfo.ForMathlib.Minimax
import QuantumInfo.ForMathlib.Misc
import QuantumInfo.ForMathlib.Unitary

/-! Collects the various "missing" Mathlib facts we have in this folder. -
/

```

1.452 QuantumInfo/ForMathlib/ContinuousLinearMap.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
--For the first three lemmas
import Mathlib.Topology.Algebra.Module.LinearMap

--For the third lemma
import Mathlib.Analysis.CStarAlgebra.Classes
import Mathlib.Analysis.InnerProductSpace.Spectrum
import Mathlib.Order.CompletePartialOrder

namespace ContinuousLinearMap

variable {R S : Type*} [Semiring R] [Semiring S] (σ : R →+* S) (M M₂ : Type
variable [TopologicalSpace M] [AddCommMonoid M] [TopologicalSpace M₂] [AddC
variable [Module R M] [Module S M₂]

--These two theorems might look a bit silly as aliases of `LinearMap.____`,
--TODO: I think we can remove these now that we've bumped to Lean 4.28.0
@[simp]
theorem range_zero [RingHomSurjective σ] : (0 : M →SL[σ] M₂).range = 1 := b
simp

```



```

@[simp]
theorem ker_zero : (0 : M →SL[σ] M2).ker = 0 := by
  simp

theorem ker_mk (f : M →SL[σ] M2) (hf : Continuous f.toFun) :
  (ContinuousLinearMap.mk f hf).ker = LinearMap.ker f := by
  rfl

end ContinuousLinearMap

namespace ContinuousLinearMap

variable {n : Type*} [Fintype n] [RCLike R]

/-- The support of a Hermitian matrix is the sum of its nonzero eigenspaces. -
/
theorem support_eq_sup_eigenspace_nonzero (A : EuclideanSpace R n →L EuclideanSpace R n)
  (hA : A.IsSymmetric) : A.range = { μ ≠ 0, Module.End.eigenspace A μ } := by
  apply le_antisymm
  · rintro x ⟨y, hy⟩
    have h_decomp : y ∈ { μ : R, Module.End.eigenspace A.toLinearMap μ } := by
      have h_orth := hA.orthogonalComplement_iSup_eigenspaces_eq_bot
      rw [Submodule.orthogonal_eq_bot_iff] at h_orth
      rw [h_orth]
      exact Submodule.mem_top;
    rw [Submodule.mem_iSup_iff_exists_finsupp] at h_decomp
    rcases h_decomp with ⟨f, hf1, hf2⟩
    have h_apply_A : A y = ∑ i ∈ f.support, A (f i) := by
      rw [← hf2, map_finsuppSum]
      exact rfl
    have h_eigen (i) : A (f i) = (i : R) • f i :=
      Module.End.mem_eigenspace_iff.mp (hf1 i)
    rw [← hy, coe_coe, h_apply_A, Finset.sum_congr rfl (fun i _ => h_eigen i)]
    refine Submodule.sum_mem _ fun i _ => ?_
    by_cases hi0 : i = 0
    · simp [hi0]
    · apply Submodule.smul_mem
      apply Submodule.mem_iSup_of_mem i
      exact Submodule.mem_iSup_of_mem hi0 (hf1 i)
  · simp only [iSup_le_iff]
    intro μ hμ x hx
    use μ-1 • x
    simp_all

end ContinuousLinearMap

```

1.453 QuantumInfo/ForMathlib/ContinuousSup.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Analysis.Normed.Module.FiniteDimension
import Mathlib.Data.Real.StarOrdered
import Mathlib.Analysis.Normed.Operator.BoundedLinearMaps

variable {α β γ : Type*} {S : Set β} {f : γ → β → α}
variable [ConditionallyCompleteLinearOrder α] [TopologicalSpace α] [OrderTopologicalSpace α]
variable [TopologicalSpace γ]

namespace IsCompact
variable [TopologicalSpace β]

theorem sSup_image_eq_sSup_image_closure {f : β → α}
  (hS : IsCompact (closure S)) (hf : Continuous f) :
  sSup (f '' S) = sSup (f '' closure S) := by
  rcases S.eq_empty_or_nonempty with rfl | h; · simp
  refine csSup_eq_of_forall_le_of_forall_lt_exists_gt (by simpa) ?_ ?_
  · rintro a ⟨w, hw, rfl⟩
    exact le_csSup (hS.image hf).bddAbove (Set.mem_image_of_mem f <| subset_closure)
  · intro w hw
    simp only [Set.mem_image, exists_exists_and_eq_and]
    contrapose! hw
    have h_image_closure : f '' closure S ⊆ closure (f '' S) :=
      image_closure_subset_closure_image hf
    have h_closure_image : closure (f '' S) ⊆ Set.Iic w :=
      closure_minimal (Set.image_subset_iff.mpr hw) isClosed_Iic
    exact csSup_le ((h.mono subset_closure).image f) fun y hy =>
      (h_image_closure.trans h_closure_image) hy

theorem sInf_image_eq_sInf_image_closure {f : β → α} (hS : IsCompact (closure S))
  sInf (f '' S) = sInf (f '' closure S) :=
  sSup_image_eq_sSup_image_closure (α := αop) hS hf

/-- A version of `IsCompact.continuous_sSup` with a slightly weaker hypothesis
that its closure is compact (but the set itself need not be). -
/
theorem closure_continuous_sSup (hS : IsCompact (closure S)) (hf : Continuous f)
  Continuous fun x => sSup (f x '' S) := by
  simp_rw [fun x => sSup_image_eq_sSup_image_closure hS (f := f x)] (by fun_p

```

```

    exact hS.continuous_sSup hf

/-- A version of `IsCompact.continuous_sInf` with a slightly weaker hypothesis on th
that its closure is compact (but the set itself need not be). -
/
theorem closure_continuous_sInf (hS : IsCompact (closure S)) (hf : Continuous 1f) :
  Continuous fun x ↦ sInf (f x '' S) :=
  closure_continuous_sSup (α := αo d) hS hf

theorem continuous_iSup (hS : IsCompact (closure S)) (hf : Continuous 1f) :
  Continuous fun x ↦  $\bigvee$  y : S, f x y := by
  simp_rw [iSup, ← Set.image_eq_range]
  exact hS.closure_continuous_sSup hf

theorem continuous_iInf (hS : IsCompact (closure S)) (hf : Continuous 1f) :
  Continuous fun x ↦  $\bigwedge$  y : S, f x y :=
  continuous_iSup (α := αo d) hS hf

end IsCompact

namespace Bornology.IsBounded
variable [PseudoMetricSpace β] [ProperSpace β]

/-- Similar to `IsCompact.continuous_sSup`, but taking a bounded set in the bornolog
of a compact set. -/ --TODO: Can `ProperSpace` be relaxed to `CompleteSpace` here?
theorem continuous_sSup (hS : Bornology.IsBounded S) (hf : Continuous 1f) :
  Continuous fun x ↦ sSup (f x '' S) :=
  hS.isCompact_closure.closure_continuous_sSup hf

/-- Similar to `IsCompact.continuous_sInf`, but taking a bounded set in the bornolog
of a compact set. -/
theorem continuous_sInf (hS : Bornology.IsBounded S) (hf : Continuous 1f) :
  Continuous fun x ↦ sInf (f x '' S) :=
  hS.isCompact_closure.closure_continuous_sInf hf

theorem continuous_iSup (hS : Bornology.IsBounded S) (hf : Continuous 1f) :
  Continuous fun x ↦  $\bigvee$  y : S, f x y := by
  simp_rw [iSup, ← Set.image_eq_range]
  exact hS.isCompact_closure.closure_continuous_sSup <| by fun_prop

theorem continuous_iInf (hS : Bornology.IsBounded S) (hf : Continuous 1f) :
  Continuous fun x ↦  $\bigwedge$  y : S, f x y :=
  continuous_iSup (α := αo d) hS hf

end Bornology.IsBounded

```

```
namespace LinearMap
```

```
/-- For bilinear maps in suitably well-behaved spaces with `IsModuleTopology`  
argument is still `Continuous`, by `Bornology.IsBounded.continuous_iSup`. -/  
/
```

```
theorem continuous_iSup {E F  $\alpha$  : Type*}  
  [CommRing  $\alpha$ ] [TopologicalSpace  $\alpha$ ] [IsTopologicalRing  $\alpha$ ] [ConditionallyCompleteLattice  $\alpha$ ]  
  [AddCommGroup E] [TopologicalSpace E] [Module  $\alpha$  E] [IsModuleTopology  $\alpha$  E]  
  [AddCommGroup F] [Module  $\alpha$  F] [PseudoMetricSpace F] [ProperSpace F] [Module  $\alpha$  F]  
  (f : E  $\rightarrow$   $\alpha$  F  $\rightarrow$   $\alpha$ ) {S : Set F} (hS : Bornology.IsBounded S) :  
    Continuous fun x  $\mapsto$   $\bigsqcup$  y : S, f x y :=  
    hS.continuous_iSup <| by fun_prop
```

```
/-- For bilinear maps in suitably well-behaved spaces with `IsModuleTopology`  
argument is still `Continuous`, by `Bornology.IsBounded.continuous_iInf`. -/  
/
```

```
theorem continuous_iInf {E F  $\alpha$  : Type*}  
  [CommRing  $\alpha$ ] [TopologicalSpace  $\alpha$ ] [IsTopologicalRing  $\alpha$ ] [ConditionallyCompleteLattice  $\alpha$ ]  
  [AddCommGroup E] [TopologicalSpace E] [Module  $\alpha$  E] [IsModuleTopology  $\alpha$  E]  
  [AddCommGroup F] [Module  $\alpha$  F] [PseudoMetricSpace F] [ProperSpace F] [Module  $\alpha$  F]  
  (f : E  $\rightarrow$   $\alpha$  F  $\rightarrow$   $\alpha$ ) {S : Set F} (hS : Bornology.IsBounded S) :  
    Continuous fun x  $\mapsto$   $\bigsqcap$  y : S, f x y :=  
    hS.continuous_iInf <| by fun_prop
```

```
/-- A specialization of `LinearMap.continuous_iSup` to finite dimensional spaces  
of requiring a (non-instance) `IsModuleTopology`. -/
```

```
theorem continuous_iSup' {E F  $\alpha$  : Type*}  
  [NontriviallyNormedField  $\alpha$ ] [ConditionallyCompleteLinearOrder  $\alpha$ ] [OrderTopology  $\alpha$ ]  
  [AddCommGroup E] [TopologicalSpace E] [IsTopologicalAddGroup E] [T2Space E]  
  [Module  $\alpha$  E] [ContinuousSMul  $\alpha$  E] [FiniteDimensional  $\alpha$  E]  
  [PseudoMetricSpace F] [ProperSpace F] [AddCommGroup F] [IsTopologicalAddGroup F]  
  [Module  $\alpha$  F] [ContinuousSMul  $\alpha$  F] [FiniteDimensional  $\alpha$  F]  
  (f : E  $\rightarrow$   $\alpha$  F  $\rightarrow$   $\alpha$ ) {S : Set F} (hS : Bornology.IsBounded S) :  
    Continuous fun x  $\mapsto$   $\bigsqcup$  y : S, f x y :=  
    let _ : IsModuleTopology  $\alpha$  E := isModuleTopologyOfFiniteDimensional ( $\alpha$  :=  $\alpha$ )  
    let _ : IsModuleTopology  $\alpha$  F := isModuleTopologyOfFiniteDimensional ( $\alpha$  :=  $\alpha$ )  
    f.continuous_iSup hS
```

```
/-- A specialization of `LinearMap.continuous_iInf` to finite dimensional spaces  
of requiring a (non-instance) `IsModuleTopology`. -/
```

```
theorem continuous_iInf' {E F  $\alpha$  : Type*}  
  [NontriviallyNormedField  $\alpha$ ] [ConditionallyCompleteLinearOrder  $\alpha$ ] [OrderTopology  $\alpha$ ]  
  [AddCommGroup E] [TopologicalSpace E] [IsTopologicalAddGroup E] [T2Space E]  
  [Module  $\alpha$  E] [ContinuousSMul  $\alpha$  E] [FiniteDimensional  $\alpha$  E]  
  [PseudoMetricSpace F] [ProperSpace F] [AddCommGroup F] [IsTopologicalAddGroup F]  
  [Module  $\alpha$  F] [ContinuousSMul  $\alpha$  F] [FiniteDimensional  $\alpha$  F]
```

```

(f : E →1[ $\alpha$ ] F →1[ $\alpha$ ]  $\alpha$ ) {S : Set F} (hS : Bornology.IsBounded S) :
  Continuous fun x ↦  $\alpha$  y : S, f x y :=
let _ : IsModuleTopology  $\alpha$  E := isModuleTopologyOfFiniteDimensional ( $\alpha$  :=  $\alpha$ ) (E :=  $\alpha$ )
let _ : IsModuleTopology  $\alpha$  F := isModuleTopologyOfFiniteDimensional ( $\alpha$  :=  $\alpha$ ) (E :=  $\alpha$ )
f.continuous_iInf hS

/-- Alias of `LinearMap.continuous_iSup` ` that takes `LinearMap.BilinForm`. -
/
theorem BilinForm.continuous_iSup { $\alpha$  E : Type*}
  [RCLike  $\alpha$ ] [ConditionallyCompleteLinearOrder  $\alpha$ ] [OrderTopology  $\alpha$ ]
  [NormedAddCommGroup E] [InnerProductSpace  $\alpha$  E] [FiniteDimensional  $\alpha$  E] [ProperSpace E]
  (f : LinearMap.BilinForm  $\alpha$  E) {S : Set E} (hS : Bornology.IsBounded S) :
    Continuous fun x ↦  $\alpha$  y : S, f x y :=
  f.continuous_iSup' hS

/-- Alias of `LinearMap.continuous_iInf` ` that takes `LinearMap.BilinForm`. -
/
theorem BilinForm.continuous_iInf { $\alpha$  E : Type*}
  [RCLike  $\alpha$ ] [ConditionallyCompleteLinearOrder  $\alpha$ ] [OrderTopology  $\alpha$ ]
  [NormedAddCommGroup E] [InnerProductSpace  $\alpha$  E] [FiniteDimensional  $\alpha$  E] [ProperSpace E]
  (f : LinearMap.BilinForm  $\alpha$  E) {S : Set E} (hS : Bornology.IsBounded S) :
    Continuous fun x ↦  $\alpha$  y : S, f x y :=
  f.continuous_iInf' hS

end LinearMap

namespace ContinuousLinearMap

variable { $\alpha$   $\alpha_2$  : Type*} {E F G : Type*} [NontriviallyNormedField  $\alpha$ ] [Semiring  $\alpha_2$ ] {
  [SeminormedAddCommGroup E] [NormedSpace  $\alpha$  E] [ProperSpace E]
  [AddCommMonoid F] [TopologicalSpace F] [Module  $\alpha_2$  F]
  [SeminormedAddCommGroup G] [NormedSpace  $\alpha$  G] [ConditionallyCompleteLinearOrder G] [0

/-- A specialization of `Bornology.IsBounded.continuous_iSup_bilinear` to `Continuous
/
theorem continuous_iSup
  (f : F →SL[ $\sigma_{12}$ ]  $\bar{E}$  →L[ $\alpha$ ] G) {S : Set E} (hS : Bornology.IsBounded S) :
    Continuous fun x ↦  $\alpha$  y : S, f x y :=
  hS.continuous_iSup <| by fun_prop

theorem continuous_iInf
  (f : F →SL[ $\sigma_{12}$ ]  $\bar{E}$  →L[ $\alpha$ ] G) {S : Set E} (hS : Bornology.IsBounded S) :
    Continuous fun x ↦  $\alpha$  y : S, f x y :=
  hS.continuous_iInf ( $\alpha$  := G) <| by fun_prop

end ContinuousLinearMap

```

```

--This is the theorem we actually needed downstream...
theorem LinearMap.BilinForm.continuous_iSup_fst
  {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional E]
  (f : LinearMap.BilinForm ℝ E) {S : Set E} (hS : Bornology.IsBounded S) :
    Continuous fun x ↦ λ y : S, f y x := by
  exact LinearMap.BilinForm.continuous_iSup f.flip hS
--Old "direct" proof:
-- -- Since  $f$  is continuous, there exists  $C > 0$  such that for all  $y$ 
-- obtain (C, hC1, hC2) : ∃ C > 0, ∀ y ∈ S, ∀ x : E, |f y x| ≤ C * ||y|| * ||x||
-- -- Since  $f$  is continuous, there exists  $C > 0$  such that for all  $y$ 
dimensional spaces.
-- have h_cont : ∃ C > 0, ∀ y x : E, |f y x| ≤ C * ||y|| * ||x|| := by
--   have h_bounded : Continuous (fun p : E × E => f p.1 p.2) := by
--     have _ := isModuleTopologyOfFiniteDimensional (λ _ := ℝ) (E := E)
--     fun_prop
--   obtain (C, hC0, hC) : ∃ C > 0, ∀ y x : E, ||y|| ≤ 1 → ||x|| ≤ 1 → |f y x| ≤ C
--   have h_compact : IsCompact {p : E × E | ||p.1|| ≤ 1 ∧ ||p.2|| ≤ 1} := by
--     have h_closed_unit_ball : IsCompact {p : E | ||p|| ≤ 1} := by
--       convert ProperSpace.isCompact_closedBall (0 : E) 1
--     simp [Metric.closedBall, dist_eq_norm]
--     exact h_closed_unit_ball.prod h_closed_unit_ball;
--   obtain (C, hC) := h_compact.exists_bound_of_continuousOn h_bound
--   exact (C ≤ 1, zero_lt_one.trans_le (le_max_right _), fun y x h)
-- refine (C, hC0, fun y x => ?_);
-- rcases eq_or_ne y 0 with rfl | hy; · simp
-- rcases eq_or_ne x 0 with rfl | hx; · simp
-- have := hC (||y||-1 * y) (||x||-1 * x) (by simp [hy, norm_smul]) (by s
-- simp only [map_smul, LinearMap.smul_apply, smul_eq_mul] at this
-- rw [abs_le] at this
-- rw [← norm_ne_zero_iff] at hx hy
-- have : 0 < ||y|| * ||x|| := by positivity
-- have := inv_mul_cancel_left0 hy ((f y) x)
-- have := inv_mul_cancel_left0 hx ((f y) x)
-- have := mul_inv_cancel0 hy
-- constructor <=> nlinarith
-- exact ( h_cont.choose, h_cont.choose_spec.1, fun y hy x => h_cont.cho
-- -- Since  $S$  is bounded, there exists  $M > 0$  such that for all  $y \in S$ 
-- obtain (M, hM1, hM2) : ∃ M > 0, ∀ y ∈ S, ||y|| ≤ M :=
--   hS.exists_pos_norm_le
-- rw [Metric.continuous_iff]
-- intro b ∈ hε
-- refine (ε / (C * M + 1), div_pos hε (by positivity), fun a ha => ?_)
-- -- Using the triangle inequality and the continuity of  $f$ , we get:
-- have h_triangle (y) (hy : y ∈ S) : |f y a - f y b| ≤ C * M * ||a - b|| :=
--   rw [← map_sub]

```

```

--   apply (hC2 y hy ( a - b )).trans
--   refine mul_le_mul_of_nonneg_right ?_ (by positivity)
--   exact mul_le_mul_of_nonneg_left (hM2 y hy) hC1.le
--   rcases S.eq_empty_or_nonempty with rfl | ⟨y, hy⟩; · simp [*]
--   simp [dist_eq_norm] at *
--   -- Applying the triangle inequality to the suprema, we get:
--   have h_sup_triangle : |(∑ y : S, f y a) - (∑ y : S, f y b)| ≤ C * M * ||a - b|| :
--   rw [abs_sub_le_iff]
--   constructor
--   · -- Applying the inequality  $f y a \leq f y b + C * M * ||a - b||$  to each term
--     have h_le (y : S) : f y a ≤ f y b + C * M * ||a - b|| := by
--       linarith [abs_le.mp (h_triangle y y.2)]
--     rw [sub_le_iff_le_add, add_comm]
--     convert ciSup_le fun y => le_trans ( h_le y ) _;
--     · exact ⟨(y, hy)⟩
--     · refine add_le_add ?_ le_rfl
--       refine le_ciSup ?_ (Set.mem_range_self _)
--       exact ⟨C * M * ||b||, Set.forall_mem_range.2 fun y => le_of_abs_le ((hC2 _
--   · rw [sub_le_iff_le_add']
--     -- Applying the triangle inequality to each term in the supremum, we get:
--     have h_sup_triangle (y) (hy : y ∈ S) : f y b ≤ f y a + C * M * ||a - b|| := b
--       linarith [abs_le.mp (h_triangle y hy)]
--     convert ciSup_le _
--     · exact ⟨(y, hy)⟩
--     · intro x
--       refine (h_sup_triangle x x.2).trans (add_le_add_right ?_ _)
--       exact le_ciSup (show BddAbove (Set.range fun y : S => f y a) by
--         refine ⟨C * M * ||a||, Set.forall_mem_range.2 fun y => ?_⟩
--         refine le_of_abs_le ((hC2 _ y.2 _).trans ?_)
--         refine mul_le_mul_of_nonneg_right ?_ (by positivity)
--         exact mul_le_mul_of_nonneg_left (hM2 _ y.2) hC1.le
--       ) x;
--   apply h_sup_triangle.trans_lt
--   rw [lt_div_iff₀ (by positivity)] at ha
--   nlinarith [mul_pos hC1 hM1]

```

```

theorem LinearMap.BilinForm.continuous_iInf_fst
  {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E] [FiniteDimensional ℝ E]
  (f : LinearMap.BilinForm ℝ E) {S : Set E} (hS : Bornology.IsBounded S) :
  Continuous fun x => ∑ y : S, f y x :=
  LinearMap.BilinForm.continuous_iInf f.flip hS

```

1.454 QuantumInfo/ForMathlib/Filter.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib

import Mathlib.Tactic.Bound

open Topology

--This is a stupid name for a stupid lemma
theorem Filter.Tendsto_inv_nat_mul_div_real (m : ℕ)
  : Filter.Tendsto (fun (x : ℕ) => ((↑x)-1 * ↑(x / m) : ℝ)) Filter.atTop (
  --Thanks aristotle!
  -- This simplifies to  $\lim_{x \rightarrow \infty} \frac{\lfloor x / m \rfloor}{x}$ 
  have h_floor : Filter.Tendsto (fun x : ℕ => (Nat.floor (x / m : ℝ) : ℝ) /
  -- We'll use the fact that the floor function is bounded and apply the
  have h_floor_bound : ∀ x : ℕ, x > 0 → (Nat.floor (x / m : ℝ) : ℝ) / x ≥
  cases eq_or_ne m 0
  · rename_i h
    intro x a
    subst h
    simp_all only [gt_iff_lt, CharP.cast_eq_zero, div_zero, Nat.floor_z
      Left.neg_nonpos_iff, inv_nonneg, Nat.cast_nonneg, le_refl, and_se
  · intro x a
    simp_all only [ne_eq, gt_iff_lt, one_div, ge_iff_le, tsub_le_iff_r
    apply And.intro
    · rw [ inv_eq_one_div, div_add', div_le_div_iff₀ ] <=> first | posi
    · rw [ div_le_iff₀ ( by positivity ) ];
      simpa [ div_eq_inv_mul ] using Nat.floor_le ( by positivity : 0 ≤
  -- Apply the squeeze theorem to conclude the proof.
  have h_squeeze : Filter.Tendsto (fun x : ℕ => (1 / m : ℝ) - 1 / x) Filt
    simpa using tendsto_const_nhds.sub (tendsto_inv_atTop_nhds_zero_nat (
    exact tendsto_of_tendsto_of_tendsto_of_le_of_le' h_squeeze tendsto_cons
  -- Apply the hypothesis `h_floor` to conclude the proof.
  convert h_floor using 1;
  -- By definition of floor function, we know that  $\lfloor (x : ℝ) / m \rfloor$  is the gr
  funext x; simp [Nat.floor_div_natCast];
  ring

--Similar to `ENNReal.tendsto_toReal_iff` in `Mathlib/Topology/Instances/ENN
-- instead of requiring finiteness for all values, just eventually is needed
open Filter Topology ENNReal in

```



```

theorem ENNReal.tendsto_toReal_iff_of_eventually_ne_top
  {ι} {fi : Filter ι} {f : ι → ℝ≥0∞} (hf : ∀ i in fi, f i ≠ ∞) {x : ℝ≥0∞}
  (hx : x ≠ ∞) : Tendsto (fun n => (f n).toReal) fi (λ x.toReal) ↔ Tendsto f fi (λ
  have he₁ : f =ᶠ[fi] (fun n ↦ (f n).toNNReal) := by
    rw [EventuallyEq]
    peel hf with h
    simp [h]
  have he₂ : (fun n ↦ (f n).toReal) = (fun n ↦ ((f n).toNNReal : ℝ≥0∞).toReal) :=
    rfl
  rw [Filter.tendsto_congr' he₁, he₂]
  exact tendsto_toReal_iff (by finiteness) hx

```

1.455 QuantumInfo/ForMathlib/HermitianMat.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Basic
import QuantumInfo.ForMathlib.HermitianMat.CFC
import QuantumInfo.ForMathlib.HermitianMat.Inner
import QuantumInfo.ForMathlib.HermitianMat.LogExp
import QuantumInfo.ForMathlib.HermitianMat.Order
import QuantumInfo.ForMathlib.HermitianMat.Proj
import QuantumInfo.ForMathlib.HermitianMat.Rpow
import QuantumInfo.ForMathlib.HermitianMat.Reindex
import QuantumInfo.ForMathlib.HermitianMat.Schatten
import QuantumInfo.ForMathlib.HermitianMat.Sqrt
import QuantumInfo.ForMathlib.HermitianMat.Trace
import QuantumInfo.ForMathlib.HermitianMat.Unitary

```

1.456 QuantumInfo/ForMathlib/HermitianMat/Basic.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.Matrix
import QuantumInfo.ForMathlib.IsMaximalSelfAdjoint
import QuantumInfo.ForMathlib.ContinuousLinearMap
import QuantumInfo.ForMathlib.Tactic.Commutates

```

```

import Mathlib.Analysis.Matrix.Normed
import Mathlib.Analysis.SpecialFunctions.ContinuousFunctionalCalculus.ExpLo
import Mathlib.Analysis.SpecialFunctions.ContinuousFunctionalCalculus.Rpow.
import Mathlib.Analysis.SpecialFunctions.Bernstein
import Mathlib.Analysis.SpecialFunctions.Pow.NNReal
import Mathlib.Tactic.NormNum.GCD

/-- The type of Hermitian matrices, as a `Subtype`. Equivalent to a `Matrix`
with the fact that `Matrix.IsHermitian`. -/
def HermitianMat (n : Type*) (α : Type*) [AddGroup α] [StarAddMonoid α] :=
  (selfAdjoint (Matrix n n α) : Type (max u_1 u_2))

namespace HermitianMat

variable {α R [] : Type*} {m n : Type*}
variable [Star R] [TrivialStar R]
variable [RCLike []]

section addgroup

variable [AddGroup α] [StarAddMonoid α]

theorem eq_IsHermitian : HermitianMat n α = { m : Matrix n n α // m.IsHerm
  rfl

@[coe] def mat : HermitianMat n α → Matrix n n α :=
  Subtype.val

instance : Coe (HermitianMat n α) (Matrix n n α) := ⟨mat⟩

@[simp]
theorem val_eq_coe (A : HermitianMat n α) : A.val = A := by
  rfl

@[simp]
theorem mat_mk (x : Matrix n n α) (h) : mat ⟨x, h⟩ = x := by
  rfl

@[simp]
theorem mk_mat {A : HermitianMat n α} (h : A.mat.IsHermitian) : ⟨A.mat, h⟩
  rfl

/-- Alias for HermitianMat.property or HermitianMat.2, this gets the fact t
  is actually `IsHermitian`. -/
theorem H (A : HermitianMat n α) : A.mat.IsHermitian :=

```

A.2

```
@[ext] protected theorem ext {A B : HermitianMat n  $\alpha$ } : A.mat = B.mat  $\rightarrow$  A = B :=
  Subtype.ext
```

```
instance instFun : FunLike (HermitianMat n  $\alpha$ ) n (n  $\rightarrow$   $\alpha$ ) where
  coe M := (M : Matrix n n  $\alpha$ )
  coe_injective' _ _ h := HermitianMat.ext h
```

```
@[simp]
theorem mat_apply {A : HermitianMat n  $\alpha$ } {i j : n} : A.mat i j = A i j := by
  rfl
```

```
@[simp]
theorem conjTranspose_mat (A : HermitianMat n  $\alpha$ ) :
  A.mat.conjTranspose = A.mat :=
  A.H
```

```
instance : AddGroup (HermitianMat n  $\alpha$ ) :=
  AddSubgroup.toAddGroup _
```

```
instance [IsEmpty n] : Unique (HermitianMat n  $\alpha$ ) where
  default := 0
  uniq a := by ext; exact (IsEmpty.false <_>).elim
```

```
@[simp, norm_cast]
theorem mat_zero : (0 : HermitianMat n  $\alpha$ ).mat = 0 := by
  rfl
```

```
@[simp]
theorem mk_zero (h : (0 : Matrix n n  $\alpha$ ).IsHermitian) : (0, h) = (0 : HermitianMat n
  rfl
```

```
@[simp]
theorem zero_apply (i j : n) : (0 : HermitianMat n  $\alpha$ ) i j = 0 := by
  rfl
```

```
@[simp, norm_cast]
theorem mat_add (A B : HermitianMat n  $\alpha$ ) :
  (A + B).mat = A.mat + B.mat := by
  rfl
```

```
@[simp, norm_cast]
theorem mat_sub (A B : HermitianMat n  $\alpha$ ) :
  (A - B).mat = A.mat - B.mat := by
  rfl
```

```

@[simp, norm_cast]
theorem mat_neg (A : HermitianMat n  $\alpha$ ) :
  (-A).mat = -A.mat := by
  rfl

section smul
variable [SMul R  $\alpha$ ] [StarModule R  $\alpha$ ]

instance : SMul R (HermitianMat n  $\alpha$ ) :=
  {fun c A  $\mapsto$  {c • A.mat, (IsSelfAdjoint.all _).smul A.H}}

@[simp, norm_cast]
theorem mat_smul (c : R) (A : HermitianMat n  $\alpha$ ) :
  (c • A).mat = c • A.mat := by
  rfl

@[simp]
theorem smul_apply (c : R) (A : HermitianMat n  $\alpha$ ) (i j : n) :
  (c • A) i j = c • A i j := by
  rfl
end smul
section topology
variable [TopologicalSpace  $\alpha$ ]

instance : TopologicalSpace (HermitianMat n  $\alpha$ ) :=
  inferInstanceAs (TopologicalSpace (selfAdjoint _))

/- Amusingly, if we don't tag this fun_prop, then fun_prop fails to prove o
it will look through and see that `HermitianMat.mat` is `Subtype.val` *here
applications of the tactic. -/
@[fun_prop]
theorem continuous_mat : Continuous (HermitianMat.mat : HermitianMat n  $\alpha$  →
  fun_prop

lemma continuousOn_iff_coe {X : Type*} [TopologicalSpace X] {s : Set X}
  (f : X → HermitianMat n  $\alpha$ ) :
  ContinuousOn f s  $\leftrightarrow$  ContinuousOn (fun x  $\Rightarrow$  (f x).mat) s := by
  constructor
  · intro; fun_prop
  · intro h
    rw [continuousOn_iff_continuous_restrict] at *
    apply Continuous.subtype_mk h

variable [IsTopologicalAddGroup  $\alpha$ ]

```

```
--In principle, ContinuousAdd and ContinuousNeg just need corresponding instances for
-- not all of IsTopologicalAddGroup.
```

```
instance : ContinuousAdd (HermitianMat n  $\alpha$ ) :=
  inferInstanceAs (ContinuousAdd (selfAdjoint _))
```

```
instance : ContinuousNeg (HermitianMat n  $\alpha$ ) :=
  inferInstanceAs (ContinuousNeg (selfAdjoint _))
```

```
instance : IsTopologicalAddGroup (HermitianMat n  $\alpha$ ) where
```

```
variable [TopologicalSpace R] [SMul R  $\alpha$ ] [ContinuousSMul R  $\alpha$ ] [StarModule R  $\alpha$ ]
```

```
instance : ContinuousSMul R (HermitianMat n  $\alpha$ ) where
  continuous_smul := by
    rw [continuous_induced_rng]
    fun_prop
```

```
--Shortcut instances:
```

```
instance : IsTopologicalAddGroup (HermitianMat n  $\mathbb{C}$ ) := inferInstance
```

```
instance : ContinuousSMul  $\mathbb{R}$  (HermitianMat n  $\mathbb{C}$ ) := inferInstance
```

```
--TODO: Would be good to figure out the general (not just RLike) version of this.
```

```
instance : T3Space (HermitianMat n  $\mathbb{C}$ ) :=
  inferInstanceAs (T3Space (selfAdjoint _))
```

```
end topology
```

```
section mulAction
```

```
variable [Monoid R] [MulAction R  $\alpha$ ] [StarModule R  $\alpha$ ]
```

```
instance : MulAction R (HermitianMat n  $\alpha$ ) :=
  Function.Injective.mulAction Subtype.val Subtype.coe_injective mat_smul
```

```
end mulAction
```

```
end addgroup
```

```
section addcommgroup
```

```
variable [AddCommGroup  $\alpha$ ] [StarAddMonoid  $\alpha$ ]
```

```
instance : AddCommGroup (HermitianMat n  $\alpha$ ) :=
  AddSubgroup.toAddCommGroup _
```

```
@[simp, norm_cast]
```

```
theorem mat_finset_sum (f :  $\iota \rightarrow$  HermitianMat n  $\alpha$ ) (s : Finset  $\iota$ ) :
```

```

    ( $\sum i \in s, f i$ ).mat =  $\sum i \in s, (f i)$ .mat := by
      apply AddSubgroup.val_finset_sum

section module

variable [Semiring R] [Module R  $\alpha$ ] [StarModule R  $\alpha$ ]
instance : Module R (HermitianMat n  $\alpha$ ) :=
  inferInstanceAs (Module R (selfAdjoint (Matrix n n  $\alpha$ )))

variable [TopologicalSpace  $\alpha$ ]

/-- The projection from HermitianMat to Matrix, as a continuous linear map.
/
@[simps]
def mat : HermitianMat n  $\alpha$  → L[R] Matrix n n  $\alpha$  where
  toFun := mat
  cont := by fun_prop
  map_add' := by simp
  map_smul' := by simp

end module
end addcommgroup
section ring

variable [NonAssocRing  $\alpha$ ] [StarRing  $\alpha$ ] [DecidableEq n]

instance : One (HermitianMat n  $\alpha$ ) :=
  ⟨1, by
    simp [selfAdjoint.mem_iff, ← Matrix.ext_iff,
      Matrix.one_apply, apply_ite ( $\beta := \alpha$ ), eq_comm]⟩

@[simp, norm_cast]
theorem mat_one : (1 : HermitianMat n  $\alpha$ ).mat = 1 := by
  rfl

@[simp]
theorem mk_one (h : (1 : Matrix n n  $\alpha$ ).IsHermitian) : ⟨1, h⟩ = (1 : HermitianMat n  $\alpha$ ) := by
  rfl

@[simp]
theorem one_apply (i j : n) : (1 : HermitianMat n  $\alpha$ ) i j = (1 : Matrix n n  $\alpha$ ) i j := by
  rfl

noncomputable instance : AddCommMonoidWithOne (HermitianMat n  $\alpha$ ) where
  instance [i : Nonempty n] : NeZero (1 : HermitianMat n  $\alpha$ ) := by

```

```

    constructor
    intro h
    simp using congr($h i.some i.some)

end ring
section commring

variable [CommRing  $\alpha$ ] [StarRing  $\alpha$ ] [DecidableEq m] [Fintype m]
variable (A : HermitianMat m  $\alpha$ ) (n :  $\mathbb{N}$ ) (z :  $\mathbb{Z}$ )

noncomputable instance instInv : Inv (HermitianMat m  $\alpha$ ) :=
  {fun x  $\mapsto$  {x-1, x.H.inv}}

@[simp, norm_cast]
theorem mat_inv : (A-1).mat = A.mat-1 := by
  rfl

@[simp]
theorem zero_inv : ((0 : HermitianMat m  $\alpha$ )-1) = 0 := by
  ext1; simp

@[simp]
theorem one_inv : ((1 : HermitianMat m  $\alpha$ )-1) = 1 := by
  ext1; simp

noncomputable instance instPow : Pow (HermitianMat m  $\alpha$ )  $\mathbb{N}$  :=
  {fun x n  $\mapsto$  {x ^ n, x.H.pow n}}

@[simp, norm_cast]
theorem mat_pow (n :  $\mathbb{N}$ ) : (A ^ n).mat = A.mat ^ n := by
  rfl

@[simp]
theorem pow_zero : A ^ 0 = 1 := by
  ext1; simp

@[simp]
theorem zero_pow (hn : n  $\neq$  0): (0 : HermitianMat m  $\alpha$ ) ^ n = 0 := by
  ext1; simp [hn]

@[simp]
theorem one_pow : ((1 : HermitianMat m  $\alpha$ ) ^ n) = 1 := by
  ext1; simp

noncomputable instance instZPow : Pow (HermitianMat m  $\alpha$ )  $\mathbb{Z}$  :=
  {fun x z  $\mapsto$  {x ^ z, x.H.zpow z}}

```

```

@[simp]
theorem mat_zpow (z : ℤ) : (A ^ z).mat = A.mat ^ z := by
  rfl

@[simp, norm_cast]
theorem zpow_natCast : A ^ (n : ℤ) = A ^ n := by
  rfl

@[simp]
theorem zpow_zero : A ^ (0 : ℤ) = 1 := by
  ext1; simp

@[simp]
theorem zpow_one : A ^ (1 : ℤ) = A := by
  ext1; simp

@[simp]
theorem one_zpow : ((1 : HermitianMat m α) ^ z) = 1 := by
  ext1; simp

@[simp]
theorem zpow_neg_one : A ^ (-1) = A-1 := by
  ext1; exact A.mat.zpow_neg_one

@[simp]
theorem inv_zpow : A-1 ^ z = (A ^ z)-1 := by
  ext1; exact A.mat.inv_zpow z

add_aesop_rules safe norm (rule_sets := [Commutes])
  [mat_zero, mat_one, mat_smul, mat_add, mat_sub, mat_neg, mat_pow, mat_zpo

@[aesop safe apply (rule_sets := [Commutes])]
theorem _root_.Matrix.inv_commute {α : Type*} {A : Matrix m m α} [CommRing
  rcases A.nonsing_inv_cancel_or_zero with h | h
  · simp [Commute, SemiconjBy, h]
  · simp [h]

@[aesop safe apply (rule_sets := [Commutes])]
theorem commute_inv_self : Commute A-1.mat A.mat := by
  commutes

@[aesop safe apply (rule_sets := [Commutes])]
theorem commute_self_inv : Commute A.mat A-1.mat := by
  commutes

```



```
end commring
section rlike
```

```
variable [Finite n] in
instance FiniteDimensional : FiniteDimensional  $\mathbb{R}$  (HermitianMat n  $\square$ ) :=
  FiniteDimensional.finiteDimensional_submodule (selfAdjoint.submodule  $\mathbb{R}$  (Matrix n n  $\square$ ))
```

```
@[simp]
theorem im_diag_eq_zero (A : HermitianMat n  $\square$ ) (x : n) :
  RLike.im (A x x) = 0 := by
  simp [CharZero.eq_neg_self_iff] using congrArg (RLike.im <|  $\cdot$  x x) A.H.symm
```

```
--Repeat it explicitly for Complex.im so that simp can find it
```

```
@[simp]
theorem complex_im_eq_zero (A : HermitianMat n  $\mathbb{C}$ ) (x : n) :
  (A x x).im = 0 :=
  A.im_diag_eq_zero x
```

```
end rlike
```

```
section conj
```

```
variable [CommRing  $\alpha$ ] [StarRing  $\alpha$ ] [Fintype n]
variable (A : HermitianMat n  $\alpha$ )
```

```
/-- The Hermitian matrix given by conjugating by a (possibly rectangular) Matrix. If
square, this would apply to any `Semigroup`+`StarMul` (as proved by `IsSelfAdjoint.c`
us conjugate to other sizes too, as is done in e.g. Kraus operators. That is, it's a
-/
```

```
def conj {m} (B : Matrix m n  $\alpha$ ) : HermitianMat n  $\alpha$   $\rightarrow$  HermitianMat m  $\alpha$  where
  toFun A :=
    (B * A.mat * B.conjTranspose, by
      ext
        simp only [Matrix.star_apply, Matrix.mul_apply, Matrix.conjTranspose_apply, Fintype.star_sum, star_mul', star_star, show  $\forall$  (a b : n), star (A.mat b a) = A.mat a b]
        rw [Finset.sum_comm]
        congr! 2
        ring)
  map_add' _ _ := by ext1; simp [Matrix.mul_add, Matrix.add_mul]
  map_zero' := by simp
```

```
theorem conj_apply (B : Matrix m n  $\alpha$ ) (A : HermitianMat n  $\alpha$ ) :
  conj B A = (B * A.mat * B.conjTranspose, (conj B A).2) := by
  rfl
```

```
@[simp]
```

```

theorem conj_apply_mat (B : Matrix m n  $\alpha$ ) (A : HermitianMat n  $\alpha$ ) :
  (A.conj B).mat = B * A.mat * B.conjTranspose := by
  rfl

theorem conj_conj {m l} [Fintype m] (B : Matrix m n  $\alpha$ ) (C : Matrix l m  $\alpha$ ) :
  (A.conj B).conj C = A.conj (C * B) := by
  ext1
  simp [Matrix.conjTranspose_mul, Matrix.mul_assoc]

variable (B : HermitianMat n  $\alpha$ )

@[simp]
theorem conj_zero [DecidableEq n] : A.conj (0 : Matrix m n  $\alpha$ ) = 0 := by
  simp [conj_apply]

@[simp]
theorem conj_one [DecidableEq n] : A.conj 1 = A := by
  simp [conj_apply]

@[simp]
lemma conj_one_unitary [DecidableEq n] (U : Matrix.unitaryGroup n  $\alpha$ ) :
  conj U.val 1 = 1 := by
  ext1
  have h : U * U.val.conjTranspose = 1 := U.prop.2
  simp [h]

variable (R : Type*) [Star R] [TrivialStar R] [CommSemiring R] [Algebra R  $\alpha$ ]

/-- `HermitianMat.conj` as an ` $R$ -linear map, where ` $R$ ` is the ring of rele
/
def conjLinear {m} (B : Matrix m n  $\alpha$ ) : HermitianMat n  $\alpha$   $\rightarrow$  [R] HermitianMat n  $\alpha$ 
  toAddHom := conj B
  map_smul' _ _ := by
    ext1
    simp

@[simp]
theorem conjLinear_apply (B : Matrix m n  $\alpha$ ) : conjLinear R B A = conj B A
  rfl

@[fun_prop]
lemma continuous_conj ( $\rho$  : HermitianMat n  $\square$ ) : Continuous ( $\rho$ .conj (m := m))
  simp only [HermitianMat.conj, AddMonoidHom.coe_mk, ZeroHom.coe_mk]
  fun_prop

end conj

```

```
section eigenspace
```

```
variable [Fintype n] [DecidableEq n] (A : HermitianMat n  $\mathbb{R}$ )
```

```
instance [i : Nonempty n] : FaithfulSMul  $\mathbb{R}$  (HermitianMat n  $\mathbb{R}$ ) where
  eq_of_smul_eq_smul h := by
    simp [RCLike.smul_re, -mat_apply] using congr(RCLike.re ($(h 1).val i.some i.so
```

```
/-- The continuous linear map associated with a Hermitian matrix. -
/
```

```
def lin : EuclideanSpace  $\mathbb{R}$  n  $\rightarrow$  L[ $\mathbb{R}$ ] EuclideanSpace  $\mathbb{R}$  n where
  toLinearMap := A.mat.toEuclideanLin
  cont := LinearMap.continuous_of_finiteDimensional _
```

```
@[simp]
theorem isSymmetric : A.lin.IsSymmetric :=
  Matrix.isHermitian_iff_isSymmetric.mp A.H
```

```
@[simp]
theorem lin_zero : (0 : HermitianMat n  $\mathbb{R}$ ).lin = 0 := by
  simp [lin]; rfl
```

```
@[simp]
theorem lin_one : (1 : HermitianMat n  $\mathbb{R}$ ).lin = 1 := by
  simp [lin]; rfl
```

```
noncomputable def eigenspace ( $\mu$  :  $\mathbb{R}$ ) : Submodule  $\mathbb{R}$  (EuclideanSpace  $\mathbb{R}$  n) :=
  Module.End.eigenspace A.lin  $\mu$ 
```

```
/-- The kernel of a Hermitian matrix `A` as a submodule of Euclidean space, defined
`LinearMap.ker A.toMat.toEuclideanLin`. Equivalently, the zero-
eigenspace. -/
```

```
def ker : Submodule  $\mathbb{R}$  (EuclideanSpace  $\mathbb{R}$  n) :=
  LinearMap.ker A.lin.toLinearMap
```

```
theorem mem_ker_iff_mulVec_zero (x : EuclideanSpace  $\mathbb{R}$  n) : x  $\in$  A.ker  $\leftrightarrow$  A.mat.mulVec
  simp [ker, LinearMap.mem_ker, lin, Matrix.toLpLin_apply]
```

```
/-- The kernel of a Hermitian matrix is its zero eigenspace. -
/
```

```
theorem ker_eq_eigenspace_zero : A.ker = A.eigenspace 0 := by
  ext
  simp [ker, eigenspace]
```

```
@[simp]
```

```

theorem ker_zero : (0 : HermitianMat n  $\mathbb{C}$ ).ker =  $\top$  := by
  simp [ker]

@[simp]
theorem ker_one : (1 : HermitianMat n  $\mathbb{C}$ ).ker =  $\perp$  := by
  simp [ker]; rfl

theorem ker_pos_smul {c :  $\mathbb{R}$ } (hc : c  $\neq$  0) : (c • A).ker = A.ker := by
  ext x
  simp [mem_ker_iff_mulVec_zero, Matrix.smul_mulVec, hc]

/-- The support of a Hermitian matrix `A` as a submodule of Euclidean space
`LinearMap.range A.toMat.toEuclideanLin`. Equivalently, the sum of all nonz
/
def support : Submodule  $\mathbb{C}$  (EuclideanSpace  $\mathbb{C}$  n) :=
  LinearMap.range A.lin.toLinearMap

/-- The support of a Hermitian matrix is the sum of its nonzero eigenspaces
/
theorem support_eq_sup_eigenspace_nonzero : A.support =  $\sum \mu \neq 0, A.\text{eigenspace}$ 
  exact A.lin.support_eq_sup_eigenspace_nonzero A.isSymmetric

@[simp]
theorem support_zero : (0 : HermitianMat n  $\mathbb{C}$ ).support =  $\perp$  := by
  simp [support]

@[simp]
theorem support_one : (1 : HermitianMat n  $\mathbb{C}$ ).support =  $\top$  := by
  simp [support] using LinearMap.ker_eq_bot_iff_range_eq_top.mp rfl

@[simp]
theorem ker_orthogonal_eq_support : A.ker $^\perp$  = A.support := by
  rw [ker, support]
  convert ContinuousLinearMap.orthogonal_ker A.lin
  simp

@[simp]
theorem support_orthogonal_eq_range : A.support $^\perp$  = A.ker := by
  rw [ker, support]
  convert ContinuousLinearMap.orthogonal_range A.lin
  simp

end eigenspace

section diagonal

```

```

variable {α : Type*} [RCLike α] [DecidableEq n]

variable (α) in
def diagonal (f : n → ℝ) : HermitianMat n α :=
  (Matrix.diagonal (f ·),
   by simp [selfAdjoint.mem_iff, Matrix.star_eq_conjTranspose, Matrix.diagonal_conj])

variable (f g : n → ℝ)

@[simp]
theorem diagonal_mat : (diagonal α f).mat = Matrix.diagonal (f · : n → α) := by
  rfl

@[simp]
theorem diagonal_zero : (diagonal α 0) = (0 : HermitianMat n α) := by
  ext1; simp

@[simp]
theorem diagonal_one : (diagonal α 1) = (1 : HermitianMat n α) := by
  ext; rw [diagonal_mat]; simp

lemma diagonal_add : diagonal α (f + g) = diagonal α f + diagonal α g := by
  ext1; simp

lemma diagonal_add_apply : diagonal α (fun x ↦ f x + g x) = diagonal α f + diagonal α g := by
  ext1; simp

lemma diagonal_sub : diagonal α (f - g) = diagonal α f - diagonal α g := by
  ext1; simp

theorem diagonal_mul (c : ℝ) : diagonal α (fun x ↦ c * f x) = c • diagonal α f := by
  ext1; simp [← Matrix.diagonal_smul]

theorem diagonal_conj_diagonal [Fintype n] :
  (diagonal α f).conj (diagonal α g) = diagonal α (fun i ↦ f i * (g i)^2) := by
  ext1
  simp [diagonal, conj]
  intro
  ring

/--
A Hermitian matrix is equal to its diagonalization conjugated by its eigenvector unitary
-/
lemma eq_conj_diagonal [Fintype n] (A : HermitianMat n α) :
  A = (diagonal α A.H.eigenvalues).conj A.H.eigenvectorUnitary := by
  ext1

```

```

    exact Matrix.IsHermitian.spectral_theorem A.2

end diagonal

section kronecker
open Kronecker

variable {p q : Type*}
variable [CommRing  $\alpha$ ] [StarRing  $\alpha$ ]

/-- The kronecker product of two HermitianMats, see `Matrix.kroneckerMap`.
/
def kronecker (A : HermitianMat m  $\alpha$ ) (B : HermitianMat n  $\alpha$ ) : HermitianMat
  val := A.mat  $\otimes_k$  B.mat
  property := Matrix.kroneckerMap_IsHermitian A.H B.H

@[inherit_doc HermitianMat.kronecker]
scoped[HermitianMat] infixl:100 "  $\otimes_k$  " => HermitianMat.kronecker

@[simp, norm_cast]
theorem kronecker_mat (A : HermitianMat m  $\alpha$ ) (B : HermitianMat n  $\alpha$ ) :
  (A  $\otimes_k$  B).mat = A.mat  $\otimes_k$  B.mat := by
  rfl

@[simp]
theorem zero_kronecker (A : HermitianMat m  $\alpha$ ) : (0 : HermitianMat n  $\alpha$ )  $\otimes_k$  A
  ext1; simp

@[simp]
theorem kronecker_zero (A : HermitianMat m  $\alpha$ ) : A  $\otimes_k$  (0 : HermitianMat n  $\alpha$ )
  ext1; simp

variable [DecidableEq m] [DecidableEq n] in
@[simp]
theorem kronecker_one_one : (1 : HermitianMat m  $\alpha$ )  $\otimes_k$  (1 : HermitianMat n  $\alpha$ )
  ext1; simp

variable (A B : HermitianMat m  $\alpha$ ) (C : HermitianMat n  $\alpha$ ) in
theorem add_kronecker : (A + B)  $\otimes_k$  C = A  $\otimes_k$  C + B  $\otimes_k$  C := by
  ext1; simp [Matrix.add_kronecker]

variable (A : HermitianMat m  $\alpha$ ) (B C : HermitianMat n  $\alpha$ ) in
theorem kronecker_add : A  $\otimes_k$  (B + C) = A  $\otimes_k$  B + A  $\otimes_k$  C := by
  ext1; simp [Matrix.kronecker_add]

lemma kronecker_diagonal [DecidableEq m] [DecidableEq n] (d1 : m →  $\mathbb{R}$ ) (d2 :

```

```

    (diagonal [] d1 ⊗k diagonal [] d2) = diagonal [] (fun (i : m × n) => d1 i.1 * d2 i.2)
  ext1
  simp [Matrix.diagonal_kronecker_diagonal]

/--
A ⊗k B always commutes with C ⊗k D if the pairs commute.
-/
--Apply safely. It will almost always work, but there are cases where it's not sound
-- such as `A = 0`. But these can all get easily simp'ed away anyway.
@[aesop safe apply (rule_sets := [Commutes])]
theorem kron_commute [Fintype m] [Fintype n] [DecidableEq m] [DecidableEq n]
  {A C : HermitianMat m α} {B D : HermitianMat n α}
  (hAC : Commute A.mat C.mat) (hBD : Commute B.mat D.mat):
  Commute (A ⊗k B).mat (C ⊗k D).mat := by
  rw [commute_iff_eq] at hAC hBD ⊢
  simp only [kronecker_mat, ← Matrix.mul_kronecker_mul, hAC, hBD]

/--
A ⊗k 1 always commutes with 1 ⊗k B
-/
@[aesop safe apply (rule_sets := [Commutes])] --redundant but important shortcut
theorem kron_id_commute_id_kro [Fintype m] [Fintype n] [DecidableEq m] [DecidableEq n]
  (A : HermitianMat m α) (B : HermitianMat n α) :
  Commute (A ⊗k (1 : HermitianMat n α)).mat ((1 : HermitianMat m α) ⊗k B).mat := by
  commutes

/-
The conjugate of a Kronecker product by a Kronecker product is the Kronecker product
-/
lemma kronecker_conj [Fintype m] [Fintype n]
  (A : HermitianMat m α) (B : HermitianMat n α) (C : Matrix p m α) (D : Matrix q n α)
  (A ⊗k B).conj (C ⊗k D) = (A.conj C) ⊗k (B.conj D) := by
  ext1
  exact Matrix.kronecker_conj_eq A.mat B.mat C D

end kronecker

section more_range_stuff

variable {d d2 : Type*} [Fintype d] [DecidableEq d] [Fintype d2] [DecidableEq d2]

/-
If the range of a Hermitian matrix is contained in its kernel, the matrix is zero.
-/
theorem range_le_ker_imp_zero {A : HermitianMat d []}
  (h : LinearMap.range A.mat.toEuclideanLin ≤ LinearMap.ker A.mat.toEuclideanLin)

```

```

rw [HermitianMat.ext_iff, mat_zero]
ext i j
have hA_sq : (A.mat * A.mat) = 0 := by
  simp_all only [SetLike.le_def, LinearMap.mem_range, LinearMap.mem_ker,
    forall_apply_eq_imp_iff]
  simp_all only [← Matrix.ext_iff, Matrix.mul_apply, mat_apply, Matrix.zero_apply]
  intro i j
  specialize h (EuclideanSpace.single j 1)
  simp [Matrix.mulVec, dotProduct] using congr(WithLp.ofLp $(h) i)
simp_all only [mat_apply, Matrix.zero_apply]
replace hA_sq := congr_fun (congr_fun hA_sq i) i
simp_all only [Matrix.mul_apply, mat_apply, Matrix.zero_apply] ;
-- Since $A$ is Hermitian, we have $A i x * A x i = |A i x|^2$.
have h_abs : ∀ x, (A i x) * (A x i) = ||A i x|| ^ 2 := by
  intro x; have := A.2
  simp_all only [val_eq_coe, sq] ;
  have := congr_fun (congr_fun this i) x
  simp_all only [Matrix.star_apply, mat_apply, RCLike.star_def] ;
  simp only [← this, mul_comm, RCLike.norm_conj];
  simp [← sq, RCLike.mul_conj] ;
simp_rw [h_abs] at hA_sq
norm_cast at hA_sq
simp_all [Finset.sum_eq_zero_iff_of_nonneg]

/--
If  $\ker M \subseteq \ker A$ , then  $\text{range}(A M^H) = \text{range } A$ .
-/
theorem _root_.Matrix.range_mul_conjTranspose_of_ker_le_ker {A : Matrix d d}
  (h : LinearMap.ker M.toEuclideanLin ≤ LinearMap.ker A.toEuclideanLin) :
  LinearMap.range (A * M.conjTranspose).toEuclideanLin = LinearMap.range
  apply le_antisymm
  · rintro x ⟨y, rfl⟩
    use (M.conjTranspose.toEuclideanLin) y;
    simp [Matrix.toEuclideanLin]
  · intro x hx;
    -- Since $x \in \text{range}(A)$, there exists $y \in \text{range}(M^H)$
    obtain ⟨y, hy⟩ : ∃ y ∈ LinearMap.range (Matrix.toEuclideanLin (M.conjTr
      have h_range_MH : LinearMap.range (Matrix.toEuclideanLin (M.conjTrans
        have h_orthogonal : (LinearMap.range (Matrix.toEuclideanLin (M.conj
          ext x
          rw [Matrix.toEuclideanLin_conjTranspose_eq_adjoint]
          simp only [Submodule.mem_orthogonal, LinearMap.mem_ker, LinearMap
            constructor
            · intro h
              rw [← inner_self_eq_zero (□ := □)]
              have : ∀ y, @inner □ _ _ y (Matrix.toEuclideanLin M x) = 0 := b

```



```

      intro y
      rw [← LinearMap.adjoint_inner_left]
      exact h _ (y, rfl)
    exact this
  · intro h y (z, hz)
    rw [← hz, LinearMap.adjoint_inner_left, h, inner_zero_right]
  rw [← h_orthogonal, Submodule.orthogonal_orthogonal]
obtain (y, rfl) := hx;
-- Since $y$ is in the range of $M^H$, we can write $y$ as $y = y_1 + y_2$ where
obtain (y1, y2, hy1, hy2, hy) : ∃ y1 y2 : EuclideanSpace d, y1 ∈ LinearMap.range M, y2 ∈ LinearMap.range (M.conjTranspose) := h_decomp
have h_decomp : ∀ y : EuclideanSpace d, ∃ y1 ∈ LinearMap.range (Matrix.toEuclideanLin (M.conjTranspose M)) y1 ∈ LinearMap.range M, y = y1 + y2 := by
  intro y
  have h_decomp : y ∈ (LinearMap.range (Matrix.toEuclideanLin (M.conjTranspose M)) + LinearMap.range M) := by
    rw [h_range_MH];
    rw [sup_comm, Submodule.sup_orthogonal_of_hasOrthogonalProjection];
    exact Submodule.mem_top;
  rw [Submodule.mem_sup] at h_decomp; tauto;
  exact (y, y, h_decomp y |> Classical.choose_spec |> And.left, h_decomp y |> Classical.choose_spec |> And.right)
  exact (y1, hy1, by rw [hy, map_add, LinearMap.mem_ker.mp (h hy2)]) ; simp
obtain (z, rfl) := hy.1;
exact (z, by simp [Matrix.toEuclideanLin] using hy.2)

theorem conj_ne_zero {A : HermitianMat d} {M : Matrix d₂ d} (hA : A ≠ 0)
(h : LinearMap.ker M.toEuclideanLin ≤ A.ker) : A.conj M ≠ 0 := by
  by_contra h_contra
  have h_range : LinearMap.range A.mat.toEuclideanLin ≤ LinearMap.ker A.mat.toEuclideanLin
  have h_range : LinearMap.range (A.mat * M.conjTranspose).toEuclideanLin ≤ LinearMap.ker A.mat.toEuclideanLin
  rintro x (y, rfl)
  replace h_contra := congr ($(h_contra).mat)
  simp_all [Matrix.toLpLin_apply, Matrix.mul_assoc]
  rw [← Matrix.range_mul_conjTranspose_of_ker_le_ker h]
  exact h_range.trans h
  exact hA (range_le_ker_imp_zero h_range)

theorem conj_ne_zero_iff {A : HermitianMat d} {M : Matrix d₂ d}
(h : LinearMap.ker M.toEuclideanLin ≤ A.ker) : A.conj M ≠ 0 ↔ A ≠ 0 := by
  refine (⟦_, (conj_ne_zero · h)⟧)
  intro h rfl; simp at h--should be grind[= map_zero] but I don't know why. TODO.

section spectrum

variable [Fintype n] [DecidableEq n] [Fintype m] [DecidableEq m]

theorem _root_.Matrix.IsHermitian.spectrum_rcLike {A : Matrix n n} (hA : A.IsHermitian) :
  RCLike.ofReal '' spectrum ℝ A = spectrum ℂ A := by
  rw [hA.spectrum_eq_image_range, hA.spectrum_real_eq_range_eigenvalues]

```

```

/-- We fix a simp-normal form that, for HermitianMat, we always work in terms
of the real spectrum. -/
@[simp]
theorem spectrum_rcLike (A : HermitianMat n  $\mathbb{R}$ ) :
  spectrum  $\mathbb{R}$  A.mat = RCLike.ofReal '' spectrum  $\mathbb{R}$  A.mat := by
  exact A.H.spectrum_rcLike.symm

theorem ne_zero_iff_ne_zero_spectrum (A : HermitianMat n  $\mathbb{R}$ ) :
  A  $\neq$  0  $\leftrightarrow$   $\exists x \in$  spectrum  $\mathbb{R}$  A.mat, x  $\neq$  0 := by
  constructor;
  · intro h_nonzero
    contrapose! h_nonzero
    simp only [HermitianMat.ext_iff, mat_zero]
    rw [A.H.spectral_theorem]
    ext i j
    simp [Matrix.mul_apply, Matrix.diagonal]
    refine Finset.sum_eq_zero fun x _  $\mapsto$  ?_
    simp [h_nonzero _ <| A.H.spectrum_real_eq_range_eigenvalues.symm  $\triangleright$  Set.]
  · rintro (x, hx, hx') h
    simp [h, spectrum, resolventSet, Algebra.algebraMap_eq_smul_one,
      hx', Matrix.isUnit_iff_isUnit_det] at hx

open scoped Pointwise in
theorem spectrum_prod
  {A : HermitianMat m  $\mathbb{R}$ } {B : HermitianMat n  $\mathbb{R}$ } :
  spectrum  $\mathbb{R}$  (A  $\otimes_k$  B).mat = spectrum  $\mathbb{R}$  A.mat * spectrum  $\mathbb{R}$  B.mat :=
  Matrix.spectrum_prod A.H B.H

end spectrum

--Shortcut instance
noncomputable instance : AddCommMonoid (HermitianMat d  $\mathbb{C}$ ) :=
  inferInstance

```

1.457 QuantumInfo/ForMathlib/HermitianMat/CFC.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Inner
import QuantumInfo.ForMathlib.HermitianMat.NonSingular

```

```

import QuantumInfo.ForMathlib.Isometry
import QuantumInfo.ForMathlib.Unitary

import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Continuity
import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Basic
import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Instances
import Mathlib.Analysis.CStarAlgebra.ContinuousFunctionalCalculus.Commute
import Mathlib.Analysis.CStarAlgebra.CStarMatrix
import Mathlib.Algebra.Order.Group.Pointwise.CompleteLattice
import Mathlib.Topology.TietzeExtension

/-! Matrix operations on HermitianMats with the CFC -/
namespace HermitianMat

noncomputable section CFC

variable {d d₂ : Type*} [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂]
variable {X : Type*} [TopologicalSpace X]
variable (A : HermitianMat d) (f : ℝ → ℝ) (g : ℝ → ℝ) (q r : ℝ)

/- Adding this to the `CStarAlgebra` aesop set allows `cfc_tac` to use it. -
/
omit [Fintype d] [DecidableEq d] in
@[aesop safe apply (rule_sets := [CStarAlgebra])]
theorem isSelfAdjoint : IsSelfAdjoint A.mat := by
  exact A.H

/- Adding this to `fun_prop` allows `cfc_cont_tac` to use it. -
/
@[fun_prop]
theorem continuousOn_finite {α β : Type*} (f : α → β) (S : Set α)
  [TopologicalSpace α] [TopologicalSpace β] [T1Space α] [Finite S] : ContinuousOn
  rw [continuousOn_iff_continuous_restrict]
  exact continuous_of_discreteTopology

@[simp]
theorem conjTranspose_cfc : (cfc f A.mat).conjTranspose = cfc f A.mat := by
  exact cfc_predicate f A.mat

protected def cfc : HermitianMat d → :=
  ⟨cfc f A.mat, cfc_predicate _ _⟩

theorem cfc_eq : A.cfc f = ⟨cfc f A.mat, cfc_predicate f A.mat⟩ := by
  rfl

@[simp]
```

```
theorem mat_cfc : (A.cfc f).mat = _root_.cfc f A.mat := by
  rfl
```

```
section congr
```

```
variable {f g A}
```

```
theorem cfc_eq_cfc_iff_eq0n (f g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
  A.cfc f = A.cfc g  $\leftrightarrow$  Set.Eq0n f g (spectrum  $\mathbb{R}$  A.mat) := by
  rw [HermitianMat.ext_iff, mat_cfc, mat_cfc]
  exact _root_.cfc_eq_cfc_iff_eq0n A.H
```

```
@[gcongr]
```

```
nonrec theorem cfc_congr (hfg : Set.Eq0n f g (spectrum  $\mathbb{R}$  A.mat)) :
  A.cfc f = A.cfc g := by
  ext1
  exact cfc_congr hfg
```

```
/-- Version of `cfc_congr` specialized to PSD matrices. -
/
```

```
nonrec theorem cfc_congr_of_nonneg (hA :  $0 \leq A$ ) (hfg : Set.Eq0n f g (Set.Icc 0 (spectrum  $\mathbb{R}$  A.mat))) :
  A.cfc f = A.cfc g := by
  refine cfc_congr (hfg.mono ?_)
  open MatrixOrder in
  exact spectrum_nonneg_of_nonneg (a := A.mat) hA
```

```
open ComplexOrder in
```

```
/-- Version of `cfc_congr` specialized to positive definite matrices. -
/
```

```
nonrec theorem cfc_congr_of_posDef (hA : A.mat.PosDef) (hfg : Set.Eq0n f g (Set.Icc 0 (spectrum  $\mathbb{R}$  A.mat))) :
  A.cfc f = A.cfc g := by
  refine cfc_congr (hfg.mono ?_)
  rw [A.H.spectrum_real_eq_range_eigenvalues]
  rintro _ {i, rfl}
  exact hA.eigenvalues_pos i
```

```
end congr
```

```
section commute
```

```
variable {A B : HermitianMat d  $\square$ }
```

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
```

```
theorem _root_.Commute.cfc_left (hAB : Commute A.mat B.mat) :
  Commute (A.cfc f).mat B.mat := by
  exact hAB.cfc_real f
```

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
```

```

theorem _root_.Commute.cfc_right (hAB : Commute A.mat B.mat) :
  Commute A.mat (B.cfc f).mat :=
  (hAB.symm.cfc_left f).symm

theorem cfc_commute (f g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hAB : Commute A.mat B.mat) :
  Commute (A.cfc f).mat (B.cfc g).mat := by
  exact (hAB.cfc_right g).cfc_left f

@[aesop safe apply (rule_sets := [Commutes])]
theorem cfc_self_commute (A : HermitianMat d  $\square$ ) (f g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
  Commute (A.cfc f).mat (A.cfc g).mat := by
  commutes

end commute

/-- Reindexing a matrix commutes with applying the CFC. -
/
@[simp]
theorem cfc_reindex (e : d  $\approx$  d2) : (A.reindex e).cfc f = (A.cfc f).reindex e := by
  rw [HermitianMat.ext_iff]
  simp only [mat_cfc, mat_reindex]
  exact Matrix.cfc_reindex f e

theorem spectrum_cfc_eq_image (A : HermitianMat d  $\square$ ) (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
  spectrum  $\mathbb{R}$  (A.cfc f).mat = f '' (spectrum  $\mathbb{R}$  A.mat) := by
  exact cfc_map_spectrum f A.mat

/--
Spectral decomposition of `A.cfc f` as a sum of scaled projections (matrix version).
-/
theorem cfc_toMat_eq_sum_smul_proj : (A.cfc f).mat =
   $\sum i, f (A.H.eigenvalues i) \cdot (A.H.eigenvectorUnitary.val * (Matrix.single i i 1))$ 
  rw [A.mat_cfc, A.H.cfc_eq, Matrix.IsHermitian.cfc]
  have h : ( Matrix.diagonal ( RCLike.ofReal  $\circ$  f  $\circ$  Matrix.IsHermitian.eigenvalues A.
    ext i j ; by_cases hij : i = j <=> simp [ hij ];
    · simp [ Matrix.sum_apply, Matrix.single ];
      simp [ Algebra.smul_def ];
    · rw [Finset.sum_apply, Finset.sum_apply]
      simp_all
  rw [h]
  simp [Matrix.single, Matrix.mul_assoc]
  congr! 1
  ext j k
  simp [Matrix.mul_apply, Finset.mul_sum, Finset.smul_sum, smul_ite, smul_zero]

--Ensure we get this instance:

```

```

/-- info: locallyCompact_of_proper -/
#guard_msgs in
#synth LocallyCompactSpace (HermitianMat d [])

theorem cfc_eigenvalues (A : HermitianMat d []) :
   $\exists (e : d \approx d), (A.cfc f).H.eigenvalues = f \circ A.H.eigenvalues \circ e :=$ 
  A.H.cfc_eigenvalues f

/-! Here we give HermitianMat versions of many cfc theorems, like `cfc_id`,
etc. We need these because (as above) `HermitianMat.cfc` is different from
/

@[simp]
nonrec theorem cfc_id : A.cfc id = A := by
  simpa [HermitianMat.ext_iff] using cfc_id  $\mathbb{R}$  A.mat

@[simp]
nonrec theorem cfc_id' : A.cfc (·) = A :=
  cfc_id A

nonrec theorem cfc_add : A.cfc (f + g) = A.cfc f + A.cfc g := by
  ext1; exact cfc_add ..

theorem cfc_add_apply : A.cfc (fun x  $\mapsto$  f x + g x) = A.cfc f + A.cfc g :=
  cfc_add A f g

nonrec theorem cfc_sub : A.cfc (f - g) = A.cfc f - A.cfc g := by
  ext1; exact cfc_sub ..

theorem cfc_sub_apply : A.cfc (fun x  $\mapsto$  f x - g x) = A.cfc f - A.cfc g :=
  cfc_sub A f g

nonrec theorem cfc_neg : A.cfc (-f) = -A.cfc f := by
  ext1; exact cfc_neg ..

theorem cfc_neg_apply : A.cfc (fun x  $\mapsto$  -f x) = -A.cfc f :=
  cfc_neg A f

/-- We don't have a direct analog of `cfc_mul`, since we can't generally mu
to HermitianMat's to get another one, so the theorem statement wouldn't be
typed.
But, we can say that the matrices are always equal. See `cfc_conj` for the
free
analog to multiplication. -/
theorem mat_cfc_mul : (A.cfc (f * g)).mat = A.cfc f * A.cfc g := by
  simp only [mat_cfc]

```

```

exact cfc_mul ..

theorem mat_cfc_mul_apply : (A.cfc (fun x ↦ f x * g x)).mat = A.cfc f * A.cfc g := by
  exact mat_cfc_mul ..

nonrec theorem cfc_comp : A.cfc (g ∘ f) = (A.cfc f).cfc g := by
  ext1; exact cfc_comp ..

theorem cfc_comp_apply : A.cfc (fun x ↦ g (f x)) = (A.cfc f).cfc g :=
  cfc_comp A f g

nonrec theorem cfc_conj : (A.cfc f).conj (A.cfc g) = A.cfc (f * g^2) := by
  ext1
  simp only [conj_apply, mat_cfc, mat_mk, conjTranspose_cfc]
  rw [← cfc_mul, ← cfc_mul, Pi.mul_def, Pi.pow_def]
  grind only

@[simp]
theorem cfc_diagonal (g : d → ℝ) : (diagonal [] g).cfc f = diagonal [] (f ∘ g) := by
  ext1
  exact Matrix.cfc_diagonal g f

theorem cfc_conj_unitary (U : Matrix.unitaryGroup d []) :
  (A.conj U.val).cfc f = (A.cfc f).conj U := by
  ext1
  exact Matrix.cfc_conj_unitary f U

@[simp]
nonrec theorem cfc_const : (A.cfc (fun _ ↦ r)) = r • 1 := by
  ext1
  simp only [mat_cfc, mat_smul, mat_one]
  rw [cfc_const r A.mat]
  exact Algebra.algebraMap_eq_smul_one r

@[simp]
nonrec theorem cfc_const_mul_id : A.cfc (fun x ↦ r * x) = r • A := by
  ext1
  rw [mat_cfc, mat_smul, cfc_const_mul_id r A.mat]

@[simp]
nonrec theorem cfc_const_mul : A.cfc (fun x ↦ r * f x) = r • A.cfc f := by
  rw [← cfc_const_mul_id, ← cfc_comp]
  rfl

@[simp]
nonrec theorem cfc_apply_zero : (0 : HermitianMat d []).cfc f = f 0 • 1 := by

```

```

simp [HermitianMat.ext_iff, Algebra.algebraMap_eq_smul_one]

@[simp]
nonrec theorem cfc_apply_one : (1 : HermitianMat d []).cfc f = f 1 • 1 := by
  simp [HermitianMat.ext_iff, Algebra.algebraMap_eq_smul_one]

theorem cfc_pow {n : ℕ} : A.cfc (· ^ n) = A ^ n := by
  ext1
  induction n
  · simp
  · simp_rw [pow_succ, mat_pow, mat_cfc_mul_apply, pow_succ, cfc_id']
    congr

theorem cfc_nonneg_iff : 0 ≤ A.cfc f ↔ ∀ i, 0 ≤ f (A.H.eigenvalues i) := by
  open MatrixOrder in
  rw [cfc_eq, ← Subtype.coe_le_coe, ZeroMemClass.coe_zero]
  rw [_root_.cfc_nonneg_iff f A.mat, A.H.spectrum_real_eq_range_eigenvalues]
  grind

open ComplexOrder in
theorem cfc_posDef : (A.cfc f).mat.PosDef ↔ ∀ i, 0 < f (A.H.eigenvalues i)
  rw [(A.cfc f).H.posDef_iff_eigenvalues_pos]
  obtain ⟨e, he⟩ := A.cfc_eigenvalues f
  rw [he]
  refine ⟨fun h i ↦ ?_, fun h i ↦ h (e i)⟩
  simpa using h (e.symm i)

variable {A f} in
/-- If a rael function preserves nonnegativity, the CFC preserves PSDness. -/
theorem cfc_nonneg_of_nonneg (hA : 0 ≤ A) (hf : ∀ i ≥ 0, 0 ≤ f i) :
  0 ≤ A.cfc f := by
  rw [cfc_nonneg_iff]
  rw [zero_le_iff, A.H.posSemidef_iff_eigenvalues_nonneg] at hA
  exact fun i ↦ hf _ (hA i)

theorem cfc_nonSingular (hf : ∀ i, f (A.H.eigenvalues i) ≠ 0) : NonSingular
  rw [nonSingular_iff_eigenvalue_ne_zero]
  obtain ⟨e, he⟩ := cfc_eigenvalues f A
  simpa [he] using fun i ↦ hf (e i)

theorem trace_mul_cfc (A : HermitianMat d []) (f : ℝ → ℝ) :
  (A.mat * (A.cfc f).mat).trace = ∑ i, A.H.eigenvalues i * f (A.H.eigenvalues i)
  conv_lhs => rw [A.eq_conj_diagonal]
  rw [cfc_conj_unitary]

```



```

simp [conj, Matrix.mul_assoc, A.H.eigenvectorUnitary.val.trace_mul_comm]
simp [← Matrix.mul_assoc, Matrix.IsHermitian.eigenvectorUnitary ]

theorem norm_eq_sum_eigenvalues_sq (A : HermitianMat d []) :
  ||A|| ^ 2 = ∑ i, (A.H.eigenvalues i)^2 := by
  rw [← RCLike.ofReal_inj (K := []), RCLike.ofReal_pow, norm_eq_trace_sq]
  conv_lhs => change (A ^ 2).mat.trace; rw [(A ^ 2).H.trace_eq_sum_eigenvalues]
  simp only [map_sum, map_pow]
  rw [← cfc_pow]
  obtain (e, he) := cfc_eigenvalues (· ^ 2) A
  simp only [he, Function.comp_apply, map_pow]
  exact e.sum_comp (fun x => (algebraMap ℝ []) (A.H.eigenvalues x) ^ 2)

variable {A} in
theorem lt_smul_of_norm_lt {r : ℝ} (h : ||A|| ≤ r) : A ≤ r • 1 := by
  rcases lt_or_ge r 0 with _ | hr
  · have := norm_nonneg A
    order
  rcases isEmpty_or_nonempty d
  · exact le_of_subsingleton
  have h' := (sq_le_sq (by positivity) (by positivity)).mpr h
  rw [norm_eq_sum_eigenvalues_sq] at h'
  nth_rw 1 [← cfc_const A, ← cfc_id A]
  rw [le_iff, ← cfc_sub]
  rw [(HermitianMat.H _).posSemidef_iff_eigenvalues_nonneg]
  intro i; rw [Pi.zero_apply]
  obtain (e, he) := cfc_eigenvalues ((fun x => r) - id) A
  rw [he]; clear he
  dsimp only [Function.comp_apply, Pi.sub_apply, id_eq]
  rw [sub_nonneg]
  apply le_of_sq_le_sq _ hr
  refine le_trans ?_ h'
  exact Finset.single_le_sum (f := fun x => (A.H.eigenvalues x)^2) (by intros; posi

theorem ball_subset_Icc : Metric.ball A r ⊆ Set.Icc (A - r • 1) (A + r • 1) := by
  intro x
  simp only [Metric.mem_ball, dist_eq_norm, Set.mem_Icc, tsub_le_iff_right]
  intro h
  constructor
  · rw [← norm_neg] at h
    grw [← lt_smul_of_norm_lt h.le]
    simp
  · grw [← lt_smul_of_norm_lt h.le]
    simp

theorem spectrum_subset_of_mem_Icc (A B : HermitianMat d []) :

```

```

    ∃ a b, ∀ x, A ≤ x ∧ x ≤ B → spectrum ℝ x.mat ⊆ Set.Icc a b := by
    use ⌊ i, A.H.eigenvalues i, ⌊ i, B.H.eigenvalues i
    rintro x ⟨hl, hr⟩
    exact A.H.spectrum_subset_of_mem_Icc B.H hl hr

--TODO: Generalize this to real matrices (really, RCLike) too. The theorem
-- gives it for complex matrices only.
@[fun_prop]
-- protected theorem cfc_continuous {f : ℝ → ℝ} (hf : Continuous f) :
--   Continuous (cfc · f : HermitianMat d ℂ → HermitianMat d ℂ) := by
--   rcases isEmpty_or_nonempty d
--   · sorry
--   rw [Metric.continuous_iff] at hf ⊢
--   intro x ∈ hε
--   have _ : Nonempty (spectrum ℝ x.toMat) := by
--     sorry
--   replace hf b := hf b ∈ hε
--   choose fc hfc₀ hfc using hf
--   let δ : ℝ := ⌊ e : spectrum ℝ x.toMat, fc e
--   refine ⟨δ, ?_, ?_⟩
--   · --This whole block should just be `positivity`. TODO fix.
--     dsimp [δ]
--     --Why doesn't just `classical` make this happen automatically?
--     replace h_fin := Fintype.ofFinite (spectrum ℝ x.toMat)
--     rw [← Finset.sup'_univ_eq_ciSup, gt_iff_lt, Finset.lt_sup'_iff]
--     simp [hfc₀]
--   intro a ha
--   simp only [dist, AddSubgroupClass.subtype_apply, val_eq_coe, cfc_toMat]
--   sorry

@[fun_prop]
protected theorem cfc_continuous {f : ℝ → ℝ} (hf : Continuous f) :
  Continuous (HermitianMat.cfc · f : HermitianMat d ℂ → HermitianMat d ℂ)
  unfold HermitianMat.cfc
  suffices Continuous (fun A : HermitianMat d ℂ ↦ _root_.cfc f A.mat) by
  fun_prop
  have h_compact_cover := LocallyCompactSpace.local_compact_nhds (X := Herm
  apply continuous_of_continuousOn_iUnion_of_isOpen (ι := HermitianMat d ℂ
    (s := fun ab ↦ Metric.ball ab.1 ab.2))
  · rintro ⟨A, r, hr⟩
    apply ContinuousOn.mono ?_ (ball_subset_Icc A r)
    obtain ⟨a, b, hab⟩ := spectrum_subset_of_mem_Icc (A - r • 1) (A + r • 1)
    open ComplexOrder in
    refine ContinuousOn.cfc (s := fun _ ↦ Set.Icc a b) (t := Set.Icc (A - r
    · intro _
      exact isCompact_Icc

```

```

    · simp only [Set.mem_Icc]
      exact fun _ _ => eventually_nhdsWithin_of_forall hab
  · simp
  · ext x
    simp only [Set.mem_iUnion, Set.mem_univ, iff_true]
    use ⟨x, 1⟩
    simp

open ComplexOrder in
theorem Matrix.PosDef.spectrum_subset_Ioi {d : Type*} [Fintype d] [DecidableEq d]
  {A : Matrix d d ℝ} (hA : A.PosDef) : spectrum ℝ A ⊆ Set.Ioi 0 := by
  intro x hx;
  -- Since $A$ is positive definite, all its eigenvalues are positive.
  have h_eigenvalues_pos : ∀ i : d, 0 < hA.1.eigenvalues i := by
    exact hA.eigenvalues_pos;
  have h_spectrum_eq_range : spectrum ℝ A = Set.range (hA.1.eigenvalues) := by
    exact Matrix.IsHermitian.spectrum_real_eq_range_eigenvalues hA.left;
  aesop

/--
If f is a continuous family of functions parameterized by x, then (fun x => A.cfc (f
-/
@[fun_prop]
theorem continuous_cfc_fun {f : X → ℝ → ℝ} (hf : ∀ i, Continuous (f · i)) :
  Continuous (fun x => A.cfc (f x)) := by
  apply Continuous.subtype_mk
  conv => enter [1, x]; apply A.cfc_toMat_eq_sum_smul_proj (f x)
  fun_prop

variable {f : X → ℝ → ℝ} {S : Set ℝ}
/--
ContinuousOn variant for when all the matrices (A x) have a spectrum in a set T, and
-/
@[fun_prop]
theorem continuousOn_cfc_fun {T : Set ℝ}
  (hf : ∀ i ∈ T, ContinuousOn (f · i) S) (hA : spectrum ℝ A.mat ⊆ T) :
  ContinuousOn (fun x => A.cfc (f x)) S := by
  simp_rw [continuousOn_iff_continuous_restrict] at hf ⊢
  apply Continuous.subtype_mk
  conv => enter [1, x]; apply A.cfc_toMat_eq_sum_smul_proj (f x)
  unfold Set.restrict at hf
  apply continuous_finset_sum _
  rw [A.H.spectrum_real_eq_range_eigenvalues] at hA
  refine fun i _ => Continuous.smul (hf _ (by grind)) (by fun_prop)

section joint_continuity

```

--TODO Cleanup

/--

Bound the Frobenius norm of a functional calculus application.

-/

```
lemma norm_cfc_le_sqrt_card_mul_bound {A : HermitianMat d ℂ} {f : ℝ → ℝ} {C : ℝ}
  (hC : 0 ≤ C) (hf : ∀ x ∈ spectrum ℝ A.mat, ||f x|| ≤ C) :
  ||A.cfc f|| ≤ Real.sqrt (Fintype.card d) * C := by
  rw [← Real.sqrt_sq (norm_nonneg _)];
  -- Recall that the Frobenius norm of a Hermitian matrix is the square root of the sum of the squares of its eigenvalues.
  have h_frobenius_eigenvalues : ∀ (M : HermitianMat d ℂ), ||M|| ^ 2 = ∑ i ∈ spectrum ℝ M, i ^ 2
  exact fun M => norm_eq_sum_eigenvalues_sq M;
  -- Applying the bound on the eigenvalues to the Frobenius norm.
  have h_bound : ∑ i ∈ Finset.univ, ((A.cfc f).H.eigenvalues i) ^ 2 ≤ (Fintype.card d) * C ^ 2 := by
    have h_bound : ∀ i, ((A.cfc f).H.eigenvalues i) ^ 2 ≤ C ^ 2 := by
      intro i
      have h_eigenvalue_bound : |(A.cfc f).H.eigenvalues i| ≤ C := by
        obtain ⟨ x, hx, hx' ⟩ : (A.cfc f).H.eigenvalues i ∈ f '' spectrum ℝ A.mat
        have h_bound := (A.cfc f).H.eigenvalues_mem_spectrum_real i
        rwa [spectrum_cfc_eq_image A f] at h_bound
      specialize hf x hx
    aesop;
    nlinarith only [abs_le.mp h_eigenvalue_bound];
  exact le_trans (Finset.sum_le_sum fun _ => h_bound _) (by simp);
  rw [h_frobenius_eigenvalues, Real.sqrt_le_left] <=> nlinarith [Real.sqrt_sq]
```

/-

The norm of the difference of two functional calculus applications is bounded.

-/

```
lemma norm_cfc_sub_cfc_le_sqrt_card {A : HermitianMat d ℂ} {f g : ℝ → ℝ} :
  ||A.cfc f - A.cfc g|| ≤ Real.sqrt (Fintype.card d) * sup x ∈ spectrum ℝ A.mat, ||f x - g x||
  rw [← HermitianMat.cfc_sub];
  refine' le_trans (norm_cfc_le_sqrt_card_mul_bound _ _);
  exact sup x ∈ spectrum ℝ A.mat, ||f x - g x||;
  · exact Real.iSup_nonneg fun _ => Real.iSup_nonneg fun _ => norm_nonneg _;
  · intro x hx
    apply le_csSup;
    · -- The supremum of a finite set of real numbers is finite.
      have h_finite : Set.Finite (spectrum ℝ A.mat) := by
        exact Set.toFinite _;
      obtain ⟨ M, hM ⟩ := h_finite.exists_finset_coe;
      refine' { ∑ x ∈ M, ||f x - g x||, Set.forall_mem_range.2 fun x => _ };
      rw [← hM];
      rw [← @ciSup_eq_ite];
      split_ifs <=> [ exact Finset.single_le_sum (fun x _ => norm_nonneg (
```

```

    · exact { x, by aesop };
    · rfl

/-
If f and g are close on T, and the spectrum of A is in T, then A.cfc f and A.cfc g are close.
-/
lemma norm_cfc_sub_le_of_sup_le {A : HermitianMat d ℂ} {f g : ℝ → ℝ} {T : Set ℝ} {ε : ℝ}
  (hT : spectrum ℝ A.mat ⊆ T) (hε : 0 ≤ ε) (h_sup : ∀ x ∈ T, ||f x - g x|| ≤ ε) :
  ||A.cfc f - A.cfc g|| ≤ Real.sqrt (Fintype.card d) * ε := by
  refine' le_trans ( norm_cfc_sub_cfc_le_sqrt_card ) _;
  gcongr;
  refine' ciSup_le fun x => _;
  exact Real.iSup_le (fun i => h_sup x (hT i)) hε

/--
If $f$ is jointly continuous on $S \times T$ and $T$ is compact, then $x \mapsto f(x, t)$ is continuous.
-/
lemma dist_lt_of_continuous' {X : Type*} [TopologicalSpace X]
  {f : X → ℝ → ℝ} {S : Set X} {T : Set ℝ}
  (hT : IsCompact T)
  (hf : ContinuousOn (fun (p : X × ℝ) ↦ f p.1 p.2) (S ×s T))
  {x₀ : X} (hx₀ : x₀ ∈ S) {ε : ℝ} (hε : 0 < ε) :
  ∃ U ∈ nhds x₀, ∀ x ∈ U ∩ S, ∀ t ∈ T, ||f x t - f x₀ t|| < ε := by
  by_contra h_contra;
  -- For each $t \in T$, by continuity at $(x_0, t)$, there exist neighborhoods $U_t$ of $x_0$ and $V_t$ of $t$ such that
  have h_cont : ∀ t ∈ T, ∃ U_t ∈ nhds x₀, ∃ V_t ∈ nhds t, ∀ x ∈ U_t ∩ S, ∀ t' ∈ V_t,
    intro t ht
    have h_cont_t : ∀f (p : X × ℝ) in nhds (x₀, t), p ∈ S ×s T → ||f p.1 p.2 - f x₀ t|| < ε / 2 := by
      have := hf ( x₀, t ) { hx₀, ht };
      have := this.eventually ( Metric.ball_mem_nhds _ ( half_pos hε ) );
      rw [ eventually_nhdsWithin_iff ] at this; aesop;
      rcases mem_nhds_prod_iff.mp h_cont_t with ( U, hU, V, hV, hUV );
      exact { U, hU, V, hV, fun x hx t' ht' => hUV ( Set.mk_mem_prod hx.1 ht'.1 ) ( hx t' ht' ) };
  choose! U hU V hV hUV using h_cont;
  -- Since $T$ is compact, cover it by finitely many $V_{t_i}$. Let $U = \bigcap U_{t_i}$.
  obtain (t_fin, ht_fin) : ∃ t_fin : Finset ℝ, (∀ t ∈ t_fin, t ∈ T) ∧ T ⊆ ⋃ t ∈ t_fin, V t
  have := hT.elim_nhds_subcover V fun t ht => hV t ht;
  tauto;
  refine h_contra (⋂ t ∈ t_fin, U t, ?_, ?_)
  · exact Filter.biInter_mem ( Finset.finite_toSet t_fin ) |>.2 fun t ht => hU t ( hV t ht )
  · intro x hx t ht
    obtain (t', ht'_fin, ht'_t) : ∃ t' ∈ t_fin, t ∈ V t' := by
      simpa using ht_fin.2 ht;
    have := hUV t' ( ht'_fin.1 t' ht'_fin ) x ( Set.mem_iInter.1 hx.1 t' ht'_fin, hx t' ht' );
    have := hUV t' ( ht'_fin.1 t' ht'_fin ) x₀ ( mem_of_mem_nhds ( hU t' ( ht'_fin.1 t' ht'_fin ) ) );
    exact abs_lt.mpr ( by linarith [ abs_lt.mp <||f x t - f x₀ t'|| < ε / 2, abs_lt.mpr

```

```

/--
The functional calculus is continuous on matrices with spectrum in a compact set.
-/
lemma continuousOn_cfc_of_compact {K : Set ℝ} {g : ℝ → ℝ} (hK : IsCompact K) :
  ContinuousOn (fun (A : HermitianMat d ℂ) ↦ A.cfc g) {A | spectrum ℝ A.mat ⊆ K}
  by_contra! h_contra;
  -- By Stone-Weierstrass, there exists a sequence of polynomials `p_n` converging uniformly to `g` on `K`.
  obtain ⟨p_n, hp_n⟩ : ∃ p_n : ℕ → Polynomial ℝ, (∀ n, ∀ x ∈ K, |(p_n n).eval x - g x| < 1 / (n + 1))
  have h_stone_weierstrass : ∀ ε > 0, ∃ p : Polynomial ℝ, ∀ x ∈ K, |p.eval x - g x| < ε
  have := @exists_polynomial_near_of_continuousOn;
  obtain ⟨a, b, hab⟩ : ∃ a b : ℝ, K ⊆ Set.Icc a b := by
    exact ⟨ hK.bddBelow.some, hK.bddAbove.some, fun x hx => ⟨ hK.bddBelow.some, hK.bddAbove.some ⟩ ⟩
  -- Extend $g$ to a continuous function on $[a, b]$.
  obtain ⟨f, hf⟩ : ∃ f : ℝ → ℝ, ContinuousOn f (Set.Icc a b) ∧ ∀ x ∈ K, |f x - g x| < ε
  have := @ContinuousMap.exists_restrict_eq;
  specialize this ( show IsClosed K from hK.isClosed ) ( ContinuousOn f (Set.Icc a b) )
  exact ⟨ _, this.choose.continuous.continuousOn, fun x hx => by simp ⟩
  exact fun ε εpos => by rcases this a b f hf.1 ε εpos with ⟨ p, hp ⟩;
  exact ⟨ fun n => Classical.choose ( h_stone_weierstrass ( 1 / ( n + 1 ) ) ) ⟩
  -- The sequence `A ↦ A.cfc (p_n)` converges uniformly to `A ↦ A.cfc g` on `K`.
  have h_uniform : ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, ∀ A : HermitianMat d ℂ, spectrum ℝ A.mat ⊆ K,
    |A.cfc (p_n n).eval - A.cfc g| < ε
  -- By the properties of the functional calculus, we have `||A.cfc p_n - A.cfc g|| ≤ ...`
  have h_uniform_bound : ∀ n, ∀ A : HermitianMat d ℂ, spectrum ℝ A.mat ⊆ K,
    intro n A hA
    have h_uniform_bound : ||A.cfc (fun x => (p_n n).eval x) - A.cfc g|| ≤
      exact norm_cfc_sub_cfc_le_sqrt_card;
    refine' le_trans h_uniform_bound ( mul_le_mul_of_nonneg_left _ ( Real.sqrt ( Fintype.card d ) ) );
    refine' ciSup_le fun x => _;
    field_simp;
    by_cases hx : x ∈ spectrum ℝ A.mat <=> simp_all
    exact le_trans ( mul_le_mul_of_nonneg_right ( hp_n n x ( hA hx ) ) ( Real.sqrt ( Fintype.card d ) ) )
    exact fun ε εpos => ⟨ Nat.ceil ( ε⁻¹ * Real.sqrt ( Fintype.card d ) ),
  -- The uniform limit of continuous functions is continuous.
  have h_cont : ContinuousOn (fun A : HermitianMat d ℂ => A.cfc g) {A : HermitianMat d ℂ | spectrum ℝ A.mat ⊆ K}
  have h_seq_cont : ∀ n, ContinuousOn (fun A : HermitianMat d ℂ => A.cfc (p_n n).eval) {A : HermitianMat d ℂ | spectrum ℝ A.mat ⊆ K}
  fun_prop
  refine' Metric.continuousOn_iff.mpr _;
  intro A hA ε εpos
  obtain ⟨N, hN⟩ := h_uniform (ε / 3) (by linarith)
  obtain ⟨δ, δpos, hδ⟩ : ∃ δ > 0, ∀ a ∈ {A : HermitianMat d ℂ | spectrum ℝ A.mat ⊆ K},
    have := Metric.continuousOn_iff.mp (h_seq_cont N) A hA (ε / 3) (
      exact ⟨ this.choose, this.choose_spec.1, fun a ha ha' => by simp_all ⟩
    )
  refine' ⟨ δ, δpos, fun a ha ha' => _ ⟩;
  have := hN N le_rfl a ha;
  have := hN N le_rfl A hA;

```

```

    rw [ dist_eq_norm ];
    rw [ show a.cfc g - A.cfc g = ( a.cfc g - a.cfc ( fun x => Polynomial.eval x (p
    exact lt_of_le_of_lt ( norm_add3_le .. ) ( by linarith [ norm_sub_rev ( a.cfc g
    contradiction

```

```

end joint_continuity

```

```

theorem continuous_cfc_joint_compact {X d : Type*} [TopologicalSpace X] [Fintype d]
  {f : X → ℝ → ℝ} {A : X → HermitianMat d ℂ} {S : Set X} {T : Set ℝ}
  (hT : IsCompact T)
  (hf : ContinuousOn (fun (p : X × ℝ) ↦ f p.1 p.2) (S ×s T))
  (hA1 : ∀ x ∈ S, spectrum ℝ (A x).mat ≤ T)
  (hA2 : ContinuousOn (fun x ↦ A x) S) :
  ContinuousOn (fun x ↦ (A x).cfc (f x)) S := by
  intro x x_in_S
  have h_eps_delta : ContinuousWithinAt (fun y => (A y).cfc (f x)) S x := by
    refine ContinuousOn.continuousWithinAt ?_ x_in_S
    exact (continuousOn_cfc_of_compact hT (hf.uncurry_left x x_in_S)).comp hA2 hA1
  rw [ ContinuousWithinAt ] at *;
  rw [ Metric.tendsto_nhds ] at *;
  intro ε ε_pos
  obtain ⟨U, hU1, hU2⟩ : ∃ U ∈ nhds x, ∀ y ∈ U ∩ S, ||(A y).cfc (f y) - (A y).cfc (f
  have h_eps_delta1 : ∀ ε > 0, ∃ U ∈ nhds x, ∀ y ∈ U ∩ S, ||(A y).cfc (f y) - (A y)
  intro ε ε_pos
  obtain ⟨U, hU1, hU2⟩ := dist_lt_of_continuous' (f := f) hT hf x_in_S ε_pos
  use U, hU1
  intro y hy
  apply norm_cfc_sub_le_of_sup_le (hA1 y hy.right) ε_pos.le
  intro t ht
  exact le_of_lt (hU2 y hy t ht)
  exact h_eps_delta1 _ (by positivity)
  filter_upwards [ h_eps_delta ( ε / 2 ) ( half_pos ε_pos ), self_mem_nhdsWithin, me
  apply lt_of_le_of_lt (dist_triangle _ ((A y).cfc (f x)) _)
  nlinarith [hU2 y { hy3, hy2 },
    Real.sqrt_nonneg ( Fintype.card d : ℝ ),
    mul_div_cancel0 ε ( show ( 2 * Real.sqrt ( Fintype.card d : ℝ ) + 1 ) ≠ 0 by pos
    norm_nonneg ( ( A y ).cfc ( f y ) - ( A y ).cfc ( f x ) ),
    norm_nonneg ( ( A y ).cfc ( f x ) - ( A x ).cfc ( f x ) ),
    dist_eq_norm ( ( A y ).cfc ( f y ) ) ( ( A y ).cfc ( f x ) ),
    dist_eq_norm ( ( A y ).cfc ( f x ) ) ( ( A x ).cfc ( f x ) )]]

```

```

open scoped Matrix.Norms.Frobenius

```

```

/-

```

PROBLEM

Eigenvalues of a `HermitianMat` are bounded by its (Frobenius) norm.

PROVIDED SOLUTION

Use ``norm_eq_sum_eigenvalues_sq`` which gives $\|A\|^2 = \sum_i (A.H.eigenvalues\ i)$

```

-/
lemma eigenvalue_norm_le (A : HermitianMat d ℂ) (i : d) :
  |A.H.eigenvalues i| ≤ ||A|| := by
  have h_eigenvalue_bound : |A.H.eigenvalues i| ^ 2 ≤ ||A|| ^ 2 := by
    rw [norm_eq_sum_eigenvalues_sq A];
    simp [pow_two];
    exact Finset.single_le_sum (fun i _ => mul_self_nonneg (A.H.eigenvalues i))
      nlinarith [norm_nonneg A]

```

/--

The spectrum of a ``HermitianMat`` is contained in the closed ball of radius

-/

```

lemma spectrum_subset_closedBall (A : HermitianMat d ℂ) :
  spectrum ℝ A.mat ⊆ Metric.closedBall (0 : ℝ) ||A|| := by
  rw [A.H.spectrum_real_eq_range_eigenvalues]
  rintro _ {i, rfl}
  simp [Metric.mem_closedBall, dist_zero_right, eigenvalue_norm_le]

```

/-

PROBLEM

Upper semicontinuity of the spectrum of a Hermitian matrix: if the spectrum is contained in an open set U , then the spectrum of any Hermitian matrix sufficiently close to A_0 is also contained in U . This follows from the openness of the set of invertible matrices.

PROVIDED SOLUTION

The proof uses the resolvent approach and compactness.

1. Let $M = \|A_0\| + 1$. For B in a ball of radius 1 around A_0 , $\|B\| \leq M$, so $\text{spectrum } \mathbb{R} B.mat \subseteq \text{Metric.closedBall } (0 : \mathbb{R}) M$.
2. Let $K = \text{Metric.closedBall } (0 : \mathbb{R}) M \setminus U$. Then K is compact (closed and bounded).
3. For each $t \in K$: $t \notin \text{spectrum } \mathbb{R} A_0.mat$. By the definition of spectrum, $A_0 - t \cdot \text{Id}$ is invertible.
4. By compactness of K (it's compact since it's a closed subset of the compact ball $\text{Metric.closedBall } (0 : \mathbb{R}) M$), the open balls $B(t_j, \varepsilon_{t_j})$ cover K . Let $\delta = \min(1, \min_j \varepsilon_{t_j})$.
5. For B with $\|B - A_0\| < \delta$: $\text{spectrum } \mathbb{R} B.mat \subseteq \text{Metric.closedBall } (0 : \mathbb{R}) M$ (by step 1). Note: we need to connect $\text{spectrum } \mathbb{R} B.mat$ (the real spectrum) to IsUnit in $\text{Matrix } d d \mathbb{C}$. adjoint elements, $t \in \text{spectrum } \mathbb{R} A.mat$ iff $\text{algebraMap } \mathbb{R} (\text{Matrix } d d \mathbb{C}) t \in \text{IsUnit}$.

```

-/
lemma spectrum_subset_of_isOpen (A₀ : HermitianMat d ℂ) (U : Set ℝ)
  (hU : IsOpen U) (hAU : spectrum ℝ A₀.mat ⊆ U) :
  ∀ᶠ B in nhds A₀, spectrum ℝ B.mat ⊆ U := by
  -- Let $M = \|A_0\| + 1$. For $B$ in a ball of radius $1$ around $A_0$, $\|B\| \le M$.
  obtain ⟨M, hM⟩ : ∃ M : ℝ, ∀ B : HermitianMat d ℂ, \|B - A₀\| < 1 → spectrum ℝ B.mat ⊆ Metric.closedBall (0 : ℝ) M
  use \|A₀\| + 1
  intro B hB
  have h_norm : \|B\| ≤ \|A₀\| + 1 := by
    have := norm_sub_norm_le B A₀; linarith!;
  generalize_proofs at *; (exact spectrum_subset_closedBall B |> fun h =>
  generalize_proofs at *; (
  -- Let $K = \overline{B(0, M)} \setminus U$. Then $K$ is compact and $K \cap \text{spectrum } \mathbb{R} B.mat = \emptyset$ for all $B$ in the ball.

```



```

set K : Set ℝ := Metric.closedBall 0 M \ U
have hK_compact : IsCompact K := by
  exact IsCompact.diff ( ProperSpace.isCompact_closedBall _ _ ) hU
have hK_disjoint : K ∩ spectrum ℝ A₀.mat = ∅ := by
  exact Set.eq_empty_of_forall_notMem fun x hx => hx.1.2 <| hAU hx.2
generalize_proofs at *; (
  -- For each $t \in K$, there exist $\delta_t > 0$ and $\epsilon_t > 0$ such that f
  have h_unitary : ∀ t ∈ K, ∃ δ_t > 0, ∃ ε_t > 0, ∀ B : HermitianMat d ℂ, ||B - A₀|| <
    intro t ht
    have h_unitary : IsUnit (A₀.mat - algebraMap ℝ (Matrix d d ℂ) t) := by
      simp_all [ Set.ext_iff, spectrum.mem_iff ];
      simpa using hK_disjoint t ht |> IsUnit.neg |> IsUnit.mul <| isUnit_one
    generalize_proofs at *; (
      -- The set of units is open in the space of matrices.
      have h_unitary_open : IsOpen {B : Matrix d d ℂ | IsUnit B} := by
        exact Units.isOpen
      generalize_proofs at *; (
        have h_unitary_cont : Continuous (fun p : HermitianMat d ℂ × ℝ => p.1.mat - alge
          refine' Continuous.sub _ _ <=> fun_prop (disch := solve_by_elim)
        generalize_proofs at *; (
          have := Metric.isOpen_iff.mp ( h_unitary_open.preimage h_unitary_cont ) ( A₀, t
          generalize_proofs at *; (
            obtain ( ε, ε_pos, hε ) := this; exact ( ε, ε_pos, ε, ε_pos, fun B hB s hs => hε
          generalize_proofs at *; (
            -- By compactness of $K$, finitely many $\epsilon$-balls cover $K$. Take $\delta =
            obtain (δ, hδ_pos, hδ) : ∃ δ > 0, ∀ t ∈ K, ∃ ε_t > 0, ∀ B : HermitianMat d ℂ, ||B -
              choose! δ hδ ε hε h using h_unitary
            generalize_proofs at *; (
              have := hK_compact.elim_nhds_subcover ( fun t => Metric.ball t ( ε t ) ) fun t h
              obtain ( t, ht₁, ht₂ ) := this
              generalize_proofs at *; (
                -- Let $\delta = \min(1, \min_{i \in t} \delta_i)$.
                obtain (δ_min, hδ_min_pos, hδ_min) : ∃ δ_min > 0, ∀ i ∈ t, δ_min ≤ δ i := by
                  by_cases ht : t.Nonempty <=> simp_all [ Finset.Nonempty ];
                  · exact ( Finset.min' ( t.image δ ) ( _, Finset.mem_image_of_mem δ ht.choose_s
                  · exact ( 1, zero_lt_one )
                generalize_proofs at *; (
                  refine' ( Min.min δ_min 1, lt_min hδ_min_pos zero_lt_one, fun x hx => _ )
                generalize_proofs at *; (
                  obtain ( i, hi, hi' ) := ht₂ x hx
                  generalize_proofs at *; (
                    exact ( ε i - |x - i|, sub_pos.mpr ( by simpa [ abs_sub_comm ] using hi' ), fun
                  generalize_proofs at *; (
                    -- For any $B$ with $||B - A₀|| < \delta$, if $t \in \sigma(B)$, then $t \notin K$
                    have h_not_in_K : ∀ B : HermitianMat d ℂ, ||B - A₀|| < δ → ∀ t ∈ spectrum ℝ B.mat, t
                      intro B hB t ht htK

```

```

obtain {ε_t, hε_t_pos, hε_t} := hδ t htK
have h_unit : IsUnit (B.mat - algebraMap ℝ (Matrix d d ℂ) t) := by
  exact hε_t B hB t ( by simp using hε_t_pos )
generalize_proofs at *; (
  exact ht ( by simp [ sub_eq_iff_eq_add ] using h_unit.neg ))
generalize_proofs at *; (
  filter_upwards [ Metric.ball_mem_nhds A₀ ( show 0 < Min.min δ 1 by posi

```

/-
PROBLEM
 The CFC is continuous in the matrix argument when the function is continuous on the spectra, even when that set is not compact. This generalizes `continuousOn\_cfc\_of\_spectrum`. The proof uses Tietze extension from the finite spectrum to a compact interval, and bounds the error using upper semicontinuity of the spectral radius.
PROVIDED SOLUTION
 The proof uses Tietze extension and the compact version `continuousOn\_cfc\_of\_spectrum`.
 Step 1: Extend g from the finite set $\text{spectrum}(A_0)$ to all of $\mathbb{R}$ via Tietze. The spectrum $\Lambda := \text{spectrum } \mathbb{R} A_0.\text{mat}$ is finite (it equals $\text{Set.range } A_0.H$). The restriction of g to Λ is continuous (any function on a finite set is continuous). By `ContinuousMap.exists\_restrict\_eq`, there exists a continuous function $h : \mathbb{R} \rightarrow \mathbb{R}$ such that $h|_{\Lambda} = g|_{\Lambda}$.
 Step 2: $(A_0).cfc g = (A_0).cfc h$, since $g = h$ on $\text{spectrum}(A_0)$ (by `cfc\_congr`).
 Step 3: Show $B \mapsto B.cfc h$ is continuous at A_0 .
 Let $M = \|A_0\| + 1$ and $K = \text{Set.Icc } (-M) M$. Since h is continuous on $\mathbb{R}$ and hence on K , $[M, M] = K$ by $\text{spectrum_subset_closedBall}$, the map is $\text{ContinuousWithinAt}$ at A_0 .
 Step 4: Show $\|B.cfc g - B.cfc h\| \rightarrow 0$ as $B \rightarrow A_0$ within $\{B \mid \sigma(B) \subseteq T\}$. The function $|g - h|$ is 0 on $\Lambda = \text{spectrum}(A_0)$. For each eigenvalue $\lambda_i \in \Lambda$, g is continuous on T at λ_i and h is continuous everywhere. So $|g(t) - h(t)| = |g(t) - g(\lambda_i) + h(\lambda_i) - h(t)|$ is small for t near λ_i . Define $U_\varepsilon = \{t \in \mathbb{R} \mid \forall \lambda \in \Lambda, \text{ if } |t - \lambda| < \text{some } \delta \text{ then } |g(t) - h(t)| < \varepsilon\}$. Actually, more precisely: the set $V_\varepsilon = \{t : \mathbb{R} \mid t \in T \rightarrow |g(t) - h(t)| < \varepsilon\}$. By `spectrum\_subset\_of\_isOpen`, for B near A_0 , $\text{spectrum}(B) \subseteq V_\varepsilon$. Since $\text{spectrum}(B) \subseteq T$, for $t \in \text{spectrum}(B)$, $|g(t) - h(t)| < \varepsilon$. So $\|B.cfc g - B.cfc h\| \leq \sqrt{d} * \varepsilon$ by `norm\_cfc\_sub\_le\_of\_sup\_le`.
 Step 5: Combine. By the triangle inequality:
 $\|B.cfc g - A_0.cfc g\| \leq \|B.cfc g - B.cfc h\| + \|B.cfc h - A_0.cfc h\|$
 Both terms $\rightarrow 0$, so the map is $\text{ContinuousWithinAt}$.
 Use `Metric.continuousWithinAt\_iff` and an $\varepsilon/2$ argument.
 -/

```

set_option maxHeartbeats 800000 in
lemma continuousWithinAt_cfc_of_continuousOn {T : Set ℝ} {g : ℝ → ℝ}
  {A₀ : HermitianMat d ℂ}
  (hg : ContinuousOn g T) (hA₀ : spectrum ℝ A₀.mat ⊆ T) :
  ContinuousWithinAt (fun B ↦ B.cfc g) {B | spectrum ℝ B.mat ⊆ T} A₀ := by
  have h_ext : ∃ h : ℝ → ℝ, Continuous h ∧ ∀ x ∈ spectrum ℝ A₀.mat, h x = g x
  have h_finite : Set.Finite (spectrum ℝ A₀.mat) := by

```

```

    exact Set.toFinite _
  generalize_proofs at *; (
    have h_cont : ContinuousOn g (spectrum  $\mathbb{R}$  A0.val) := by
      exact hg.mono hA0
    generalize_proofs at *; (
      have := @ContinuousMap.exists_restrict_eq  $\mathbb{R}$ ;
      specialize this ( show IsClosed ( spectrum  $\mathbb{R}$  A0.val ) from h_finite.isClosed ) (
        obtain (h, hh_cont, hh_eq) := h_ext;
        have h_cfc_cont : ContinuousWithinAt (fun B => B.cfc h) {B : HermitianMat d  $\mathbb{C}$  | sp
          exact Continuous.continuousWithinAt (HermitianMat.cfc_continuous hh_cont)
        have h_diff_small :  $\forall \varepsilon > 0, \exists U \in \text{nhds } A_0, \forall B \in U \cap \{B : \text{HermitianMat } d \mathbb{C} \mid \text{spec}$ 
          intro  $\varepsilon$   $\varepsilon_{\text{pos}}$ 
          obtain (δ, δ_pos, hδ) :  $\exists \delta > 0, \forall x \in T, \forall y \in \text{spectrum } \mathbb{R} A_0.\text{mat}, |x - y| < \delta \rightarrow$ 
            have h_diff_small :  $\forall y \in \text{spectrum } \mathbb{R} A_0.\text{mat}, \exists \delta > 0, \forall x \in T, |x - y| < \delta \rightarrow$ 
              intro y hy
              have h_diff_small : Filter.Tendsto (fun x => |g x - h x|) (nhdsWithin y T) (
                have h_diff_small : Filter.Tendsto (fun x => g x - h x) (nhdsWithin y T) (
                  exact Filter.Tendsto.sub ( hg.continuousWithinAt ( hA0 hy ) ) ( hh_cont.
                    simp [ hh_eq y hy ] using h_diff_small.abs;
                  have := Metric.tendsto_nhdsWithin_nhds.mp h_diff_small (  $\varepsilon / (\text{Real.sqrt } F$ 
                choose! δ hδ_pos hδ using h_diff_small;
              have h_finite : Set.Finite (spectrum  $\mathbb{R}$  A0.mat) := by
                exact Set.toFinite _;
              obtain (δ_min, hδ_min_pos, hδ_min) :  $\exists \delta_{\text{min}} > 0, \forall y \in \text{spectrum } \mathbb{R} A_0.\text{mat}, \delta_{\text{m}}$ 
                by_cases h_empty : spectrum  $\mathbb{R} A_0.\text{mat} = \emptyset$ ;
                · exact ( 1, zero_lt_one, by simp [ h_empty ] );
                · have := h_finite.toFinset.exists_min_image δ;
                  exact Exists.elim ( this ( Finset.nonempty_of_ne_empty ( by simp [ Set.ex
                    exact ( δ_min, hδ_min_pos, fun x hx y hy hxy => hδ y hy x hx ( lt_of_lt_of_le
                    -- By the spectrum_subset_of_isOpen lemma, there exists a neighborhood U of A0 s
                    obtain (U, hU) :  $\exists U \in \text{nhds } A_0, \forall B \in U, \text{spectrum } \mathbb{R} B.\text{mat} \subseteq \{x \mid \exists y \in \text{spectrum}$ 
                      have h_spectrum_subset :  $\forall^f B \text{ in nhds } A_0, \text{spectrum } \mathbb{R} B.\text{mat} \subseteq \text{Metric.thickening}$ 
                      have h_open : IsOpen (Metric.thickening δ (spectrum  $\mathbb{R}$  A0.mat)) := by
                        exact Metric.isOpen_thickening
                      have := spectrum_subset_of_isOpen A0 ( Metric.thickening δ ( spectrum  $\mathbb{R}$  A0.m
                    generalize_proofs at *; (
                      exact ( _, h_spectrum_subset, fun B hB => fun x hx => by simp [ dist_eq_norm
                    generalize_proofs at *; (
                      refine' ( U, hU.1, fun B hB => _ )
                      have h_diff_small :  $\forall x \in \text{spectrum } \mathbb{R} B.\text{mat}, |g x - h x| \leq \varepsilon / (\text{Real.sqrt } (\text{Fintyp}$ 
                        exact fun x hx => le_of_lt ( hδ x ( hB.2 hx ) ) _ ( hU.2 B hB.1 hx |> Classical.
                      generalize_proofs at *; (
                      have h_diff_small :  $\|B.\text{cfc } g - B.\text{cfc } h\| \leq \text{Real.sqrt } (\text{Fintype.card } d) * (\varepsilon / (\text{Rea}$ 
                        apply_rules [ norm_cfc_sub_le_of_sup_le ];
                        · positivity;
                        · exact fun x hx => hx

```

```

    generalize_proofs at *; (
      exact h_diff_small.trans_lt ( by rw [ mul_div, div_lt_iff₀ ] <=> nlinarith
    rw [ Metric.continuousWithinAt_iff ] at *;
    intro ε hε
    obtain ⟨δ, hδ_pos, hδ⟩ := h_cfc_cont (ε / 2) (half_pos hε)
    obtain ⟨U, hU_nhds, hU⟩ := h_diff_small (ε / 2) (half_pos hε)
    use Min.min δ (Metric.mem_nhds_iff.mp hU_nhds).choose
    simp [hδ_pos];
    refine' { _, _ }
    all_goals generalize_proofs at *;
    · exact <∃ ε, 0 < ε ∧ Metric.ball A₀ ε ⊆ U>.choose_spec.1;
    · intro x hx hx' hx''
      have hd1 := hδ hx hx'
      have hd2 := hU x ((Metric.mem_nhds_iff.mp hU_nhds).choose_spec.2 hx''),
      have h_eq : A₀.cfc g = A₀.cfc h := by
        exact cfc_congr (show Set.EqOn g h (spectrum ℝ A₀.mat) from fun x hx
      rw [dist_eq_norm, h_eq]
      calc ||x.cfc g - A₀.cfc h||
        = ||(x.cfc g - x.cfc h) + (x.cfc h - A₀.cfc h)|| := by congr 1; abel
        _ ≤ ||x.cfc g - x.cfc h|| + ||x.cfc h - A₀.cfc h|| := norm_add_le _ _
        _ < ε / 2 + ε / 2 := by
          apply add_lt_add hd2
          rwa [dist_eq_norm] at hd1
        _ = ε := add_halves ε

```

/-

PROBLEM

For f jointly continuous on $S \times T$ and the spectrum of $A y$ contained in S , $f(y, t) - f(x_0, t)$ can be made uniformly small on $\text{spectrum}(\mathbb{R})(A y).mat$. This is the non-compact replacement for `dist_lt_of_continuous`: instead of on a fixed compact set, we get uniform convergence on the (moving, finite)

PROVIDED SOLUTION

Constructive proof. The key steps:

- Step 1: For each eigenvalue $\lambda_i := (A x_0).H.eigenvalues i$ (which is in T by $hf(x_0, \lambda_i)$ gives `ContinuousWithinAt` at (x_0, λ_i) within $S \times T$)
- Apply this to the ε -ball around $f(x_0, \lambda_i)$, use that $f(x_0, \lambda_i) - f(x_0, \lambda_i)$
 - Similarly, hf at (x_0, λ_i) restricted to $\{x_0\} \times T$ gives $\|f x_0 t - f x_0 \lambda_i\|$
 - Triangle inequality: $\|f y t - f x_0 t\| < \varepsilon$
 - Use `mem_nhds_prod_iff` to extract U_i and V_i from the product neighborhood
- Step 2: The open set $W := \bigcap_i V_i$ contains $\text{spectrum}(A x_0)$ (since each $\lambda_i \in V_i$)
- Step 3: By `ContinuousWithinAt` of A at x_0 (from hA_2), and the filter from step 2
- Step 4: Take $U = U' \cap \bigcap_i U_i$ (finite intersection since d is `Fintype`). For
- $t \in T$ (by hA_1)
 - $\text{spectrum}(A y) \subseteq W$ (by step 3), so $t \in V_i$ for some i
 - $y \in U_i \cap S$
 - So $\|f y t - f x_0 t\| < \varepsilon$ (by step 1)

Use `by\_contra` and arrive at contradiction, or construct the neighborhood directly
 IMPORTANT: To get the open sets V_i , use `ContinuousWithinAt` of f at (x_0, λ_i) which
 For the continuity of A composed with `spectrum_subset_of_isOpen`: use `ContinuousWith

```

-/
set_option maxHeartbeats 800000 in
lemma dist_lt_of_continuous_spectrum {X : Type*} [TopologicalSpace X]
  {f : X → ℝ → ℝ} {A : X → HermitianMat d ℂ} {S : Set X} {T : Set ℝ}
  (hf : ContinuousOn (fun (p : X × ℝ) ↦ f p.1 p.2) (S ×s T))
  (hA1 : ∀ x ∈ S, spectrum ℝ (A x).mat ⊆ T)
  (hA2 : ContinuousOn (fun x ↦ A x) S)
  {x0 : X} (hx0 : x0 ∈ S) {ε : ℝ} (hε : 0 < ε) :
  ∃ U ∈ nhds x0, ∀ y ∈ U ∩ S, ∀ t ∈ spectrum ℝ (A y).mat, ||f y t - f x0 t|| < ε :=
  by_contra h_contra;
  -- For each eigenvalue λi := (A x0).H.eigenvalues i (which is in T by hA1), th
  obtain ⟨Ui, Vi, hUi, hVi, h_cont⟩ : ∃ (Ui : d → Set X) (Vi : d → Set ℝ),
    have h_cont : ∀ i, ∃ (Ui : Set X) (Vi : Set ℝ), IsOpen Ui ∧ IsOpen Vi ∧
      intro i
      generalize_proofs at *; (
        have h_cont : ContinuousWithinAt (fun p : X × ℝ => f p.1 p.2 - f x0 p.2) (
          have := hf (x0, (A x0).H.eigenvalues i) (hx0, ?_)
          generalize_proofs at *;
          · convert this.sub ( ContinuousWithinAt.comp ( show ContinuousWithinAt (
            · have := hf ( x0, ( A x0 ).H.eigenvalues i ) ( hx0, hA1 x0 hx0 ((A x0
              generalize_proofs at *; (
                convert this.comp ( show ContinuousWithinAt ( fun p => ( x0, p ) ) T
                generalize_proofs at *; (
                  exact ContinuousWithinAt.prodMk ( continuousWithinAt_const ) continu
                  exact fun x hx => ( hx0, hx ));
                · exact fun x hx => hx.2;
                · exact hA1 x0 hx0 ((A x0).H.eigenvalues_mem_spectrum_real i) )
              generalize_proofs at *; (
                have := h_cont.eventually ( Metric.ball_mem_nhds _ hε )
                simp_all [ dist_eq_norm ]
                (
                  rw [ eventually_nhdsWithin_iff ] at this
                  generalize_proofs at *; (
                    rcases mem_nhds_prod_iff.mp this with ⟨ U, V, hU, hV, h ⟩
                    generalize_proofs at *; (
                      exact ⟨ interior U, isOpen_interior, interior hU, isOpen_interior, mem_int
                    generalize_proofs at *; (
                      choose Ui Vi hUi hVi hx0i hVii h_conti using h_cont; exact ⟨ Ui, Vi
                    -- The open set W := ⋃i Vi contains spectrum(A x0) (since each λi ∈ Vi and s
                    set W := ⋃ i, Vi i with hW_def
                    have hW_open : IsOpen W := by
                      exact isOpen_iUnion hVi
                    have hW_spectrum : spectrum ℝ (A x0).mat ⊆ W := by

```

```

intro t ht
obtain ⟨i, hi⟩ : ∃ i, t = (A x₀).H.eigenvalues i := by
  have h_eigenvalues : spectrum ℝ (A x₀).mat = Set.range (A x₀).H.e
    (A x₀).H.spectrum_real_eq_range_eigenvalues
  generalize_proofs at *; (
    exact h_eigenvalues.subset ht |> Exists.imp fun i hi => hi.symm)
aesop
have hW_subset : ∀f B in nhds (A x₀), spectrum ℝ B.mat ⊆ W := by
  exact spectrum_subset_of_isOpen (A x₀) W hW_open hW_spectrum
have hW_subset_S : ∀f y in nhdsWithin x₀ S, spectrum ℝ (A y).mat ⊆ W
  exact Filter.mem_of_superset ( hA₂.continuousWithinAt hx₀ |> fun h
obtain ⟨U', hU'⟩ : ∃ U' ∈ nhds x₀, ∀ y ∈ U' ∩ S, spectrum ℝ (A y).mat
  obtain ⟨ U', hU' ⟩ := mem_nhdsWithin_iff_exists_mem_nhds_inter.mp h
obtain ⟨U'', hU''⟩ : ∃ U'' ∈ nhds x₀, ∀ i, U'' ⊆ U_i i := by
  exact ⟨ ∅ i, U_i i, Filter.mem_of_superset ( Filter.iInter_mem.mpr
set U := U' ∩ U'' with hU_def
have hU_mem : U ∈ nhds x₀ := by
  exact Filter.inter_mem hU'.1 hU''.1
have hU_subset : ∀ y ∈ U ∩ S, spectrum ℝ (A y).mat ⊆ W := by
  exact fun y hy => hU'.2 y ⟨ hy.1.1, hy.2 ⟩ |> Set.Subset.trans <| b
have hU_cont : ∀ y ∈ U ∩ S, ∀ t ∈ spectrum ℝ (A y).mat, ||f y t - f x₀
  intro y hy t ht
  obtain ⟨i, hi⟩ : ∃ i, t ∈ V_i i := by
    exact Set.mem_iUnion.mp ( hU_subset y hy ht ) |> Exists.imp fun i
  have h_cont_i : ||f y t - f x₀ t|| < ε := by
    exact h_cont.2.2 i y ⟨ hU''.2 i ( by aesop ), hy.2 ⟩ t ⟨ hi, hA₁
  exact h_cont_i
exact h_contra ⟨U, hU_mem, hU_cont⟩

```

/-

PROBLEM

Joint continuity of the functional calculus, without requiring compactness
 This generalizes `continuous\_cfc\_joint\_compact` by removing the `IsCompact`
 The compactness is unnecessary because the spectrum of a `HermitianMat` (wh
 finite-dimensional) is always finite and hence compact. The proof works by
 the compact case locally: at each point `x₀ ∈ S`, the spectrum of `A x₀` is
 contained in a compact interval `K = [-M, M]`, and the compact version is a
 continuous extension of `f x₀` from the finite spectrum to `K`.

PROVIDED SOLUTION

The proof follows the same structure as `continuous\_cfc\_joint\_compact` but
 compact helpers `dist\_lt\_of\_continuous\_spectrum` and `continuousWithinAt\_cfc`
 Fix x ∈ S. Show ContinuousWithinAt.

Step 1: Show `ContinuousWithinAt (fun y => (A y).cfc (f x)) S x`.

The function f x (i.e., f(x, ·)) is continuous on T: this follows from `hf`.
 By `continuousWithinAt\_cfc\_of\_continuousOn` with g = f x and T = T:

```

`ContinuousWithinAt (fun B => B.cfc (f x)) {B | spectrum ℝ B.mat ⊆ T} (A x)

```

Compose with `hA₂.continuousWithinAt x\_in\_S` and `hA₁`:

`ContinuousWithinAt (fun y => (A y).cfc (f x)) S x`.

Note: We need that `fun y => A y` maps `S` into `{B | spectrum ℝ B.mat ≤ T}`, which

Step 2: Use the triangle inequality, exactly as in `continuous\_cfc\_joint\_compact`:

Decompose the goal using:

$\text{dist} ((A y).cfc (f y)) ((A x).cfc (f x)) \leq \text{dist} ((A y).cfc (f y)) ((A y).cfc (f x))$

For the first term (f varies, A = A(y)):

Use `dist\_lt\_of\_continuous\_spectrum` (our new helper) to get: for any $\varepsilon > 0$, there e

Then by `norm\_cfc\_sub\_le\_of\_sup\_le`, $\|(A y).cfc (f y) - (A y).cfc (f x)\| \leq \sqrt{d} *$

For the second term (A varies, f = f(x)):

Use step 1 directly (ContinuousWithinAt of $B \mapsto B.cfc (f x)$).

Combine with the same nlinarith/ε-δ argument as in `continuous\_cfc\_joint\_compact`.

In code, the proof structure should mirror continuous_cfc_joint_compact closely, jus

- `dist\_lt\_of\_continuous` hT hf x_in_S ε_pos` with `dist_lt_of_continuous_spectrum h

- `continuousOn\_cfc\_of\_compact hT (hf.uncurry\_left x x\_in\_S)` with `continuousWithin

-/

@[fun_prop]

theorem continuous_cfc_joint {X d : Type*} [TopologicalSpace X] [Fintype d] [Decidab

{f : X → ℝ → ℝ} {A : X → HermitianMat d ℂ} {S : Set X} {T : Set ℝ}

(hf : ContinuousOn (fun (p : X × ℝ) ↦ f p.1 p.2) (S ×ˢ T))

(hA₁ : ∀ x ∈ S, spectrum ℝ (A x).mat ≤ T)

(hA₂ : ContinuousOn (fun x ↦ A x) S) :

ContinuousOn (fun x ↦ (A x).cfc (f x)) S := by

by_contra h_not_cont_at_x₀

generalize_proofs at *;

have h_cont : ∀ x₀ ∈ S, ContinuousWithinAt (fun x => (A x).cfc (f x)) S x₀ :=
intro x₀ hx₀

have h_cont : ContinuousWithinAt (fun x => (A x).cfc (f x₀)) S x₀ := by

have h_cont : ContinuousWithinAt (fun B => B.cfc (f x₀)) {B | spectrum ℝ B

apply_rules [continuousWithinAt_cfc_of_continuousOn, hf.uncurry_left x₀

generalize_proofs at *; (

exact h_cont.comp (hA₂.continuousWithinAt hx₀) (by aesop) |> Continuou

generalize_proofs at *; (

-- By the triangle inequality, we can bound the distance between \$(A x).cfc

have h_triangle : ∀^f x in nhdsWithin x₀ S, $\|(A x).cfc (f x) - (A x).cfc (f x₀)\|$

refine' Filter.Eventually.of_forall fun x => _;

exact norm_cfc_sub_cfc_le_sqrt_card

generalize_proofs at *; (

-- By the properties of the supremum, we can bound the distance between \$(A

have h_sup : Filter.Tendsto (fun x => $\sup t \in \text{spectrum } \mathbb{R} (A x).mat, \|f x t - f$

have h_sup : ∀ ε > 0, ∃ U ∈ nhdsWithin x₀ S, ∀ x ∈ U, ∀ t ∈ spectrum ℝ (A

intro ε ε_pos

generalize_proofs at *; (

have := dist_lt_of_continuous_spectrum hf hA₁ hA₂ hx₀ ε_pos

generalize_proofs at *; (

obtain (U, hU₁, hU₂) := this; exact (U ∩ S, mem_nhdsWithin_iff_exists

```

      generalize_proofs at *; (
        refine' Metric.tendsto_nhds.mpr _;
        intro  $\varepsilon$   $\varepsilon_{\text{pos}}$ ; rcases h_sup (  $\varepsilon / 2$  ) ( half_pos  $\varepsilon_{\text{pos}}$  ) with ( U
        rw [ abs_of_nonneg ( Real.iSup_nonneg fun _ => Real.iSup_nonneg f
        · exact abs_le.mpr ( by linarith [ hU2 x hx t ht ], by linarith [
        · linarith [  $\varepsilon_{\text{pos}}$  ]))
      generalize_proofs at *; (
        have h_final : Filter.Tendsto (fun x =>  $\|(A x).cfc (f x) - (A x).cf$ 
          exact squeeze_zero_norm' ( by filter_upwards [ h_triangle ] with
        generalize_proofs at *; (
        convert h_cont.add ( show ContinuousWithinAt ( fun x => ( A x |> He
        generalize_proofs at *; (
        exact tendsto_zero_iff_norm_tendsto_zero.mpr h_final |> fun h => h.
      generalize_proofs at *; (
      exact h_not_cont_at_x0 <| fun x hx => h_cont x hx |> ContinuousWithin

/-- Specialization of `continuous0n_cfc_fun` for nonsingular matrices. -
/
@[fun_prop]
theorem continuous0n_cfc_fun_nonsingular {f : X → ℝ → ℝ} {S : Set X}
  (hf : ∀ i ≠ 0, Continuous0n (f · i) S) [NonSingular A] :
    Continuous0n (fun x ↦ A.cfc (f x)) S := by
  apply continuous0n_cfc_fun (T := {0}ᶜ)
  · exact hf
  · grind [nonSingular_zero_notMem_spectrum]

/-- Specialization of `continuous0n_cfc_fun` for positive semidefinite matr
/
@[fun_prop]
theorem continuous0n_cfc_fun_nonneg {f : X → ℝ → ℝ} {S : Set X}
  (hf : ∀ i ≥ 0, Continuous0n (f · i) S) (hA : 0 ≤ A) :
    Continuous0n (fun x ↦ A.cfc (f x)) S := by
  apply continuous0n_cfc_fun (T := Set.Ici 0)
  · exact hf
  · rw [zero_le_iff] at hA
    exact hA.pos_of_mem_spectrum

open ComplexOrder in
/-- Specialization of `continuous0n_cfc_fun` for positive definite matrices
/
@[fun_prop]
theorem continuous0n_cfc_fun_posDef {f : X → ℝ → ℝ} {S : Set X}
  (hf : ∀ i > 0, Continuous0n (f · i) S) (hA : A.mat.PosDef) :
    Continuous0n (fun x ↦ A.cfc (f x)) S := by
  apply continuous0n_cfc_fun (T := Set.Ioi 0)
  · exact hf

```



```

    • exact Matrix.PosDef.spectrum_subset_Ioi hA

variable {A B : HermitianMat d []} (f : ℝ → ℝ)

/--
The inverse of the CFC is the CFC of the inverse function.
-/
lemma inv_cfc_eq_cfc_inv (hf : ∀ i, f (A.H.eigenvalues i) ≠ 0) :
  (A.cfc f)-1 = A.cfc (fun u ↦ (f u)-1) := by
  suffices (A.cfc f).mat-1 = (A.cfc (fun u ↦ 1 / f u)).mat by
  ext1
  simpa using this
  have h_def : (A.cfc f).mat = ∑ i, f (A.H.eigenvalues i) • (A.H.eigenvectorUnitary.
    exact cfc_toMat_eq_sum_smul_proj A f;
  have h_subst : (A.cfc (fun u ↦ 1 / f u)).mat = ∑ i, (1 / f (A.H.eigenvalues i)) •
    exact cfc_toMat_eq_sum_smul_proj A fun u ↦ 1 / f u;
  have h_inv : (A.cfc f).mat * (A.cfc (fun u ↦ 1 / f u)).mat = 1 := by
    -- Since the eigenvectorUnitary is unitary, we have that the product of the proj
    have h_unitary : A.H.eigenvectorUnitary.val * A.H.eigenvectorUnitary.val.conjTra
      simp [ Matrix.IsHermitian.eigenvectorUnitary ];
  have h_inv : ∀ i j, (A.H.eigenvectorUnitary.val * (Matrix.single i i 1) * A.H.ei
    simp [ ← Matrix.mul_assoc ];
    intro i j; split_ifs <=> simp_all [ Matrix.mul_assoc, mul_eq_one_comm.mp h_uni
  simp_all [ Finset.sum_mul, Finset.mul_sum ];
  have h_sum : ∑ i, (A.H.eigenvectorUnitary.val * (Matrix.single i i 1) * A.H.eige
    simp [ Finset.mul_sum, Finset.sum_mul, Matrix.mul_assoc ];
  simp_all [ Matrix.single ];
  convert h_unitary using 2;
  ext i j; simp [ Matrix.mul_apply]
  simp [ Matrix.sum_apply, Finset.filter_eq', Finset.filter_and ];
  rw [ Finset.sum_eq_single j ] <=> aesop;
  rw [ Matrix.inv_eq_right_inv h_inv ];

theorem cfc_inv [NonSingular A] : A.cfc (fun u ↦ u-1) = A-1 := by
  simpa using (inv_cfc_eq_cfc_inv id nonSingular_eigenvalue_ne_zero).symm

section integral

open MeasureTheory
open scoped Matrix.Norms.Frobenius

omit [DecidableEq d] in
/--
The integral of a Hermitian matrix function commutes with `toMat`.
-/
lemma integral_toMat (A : ℝ → HermitianMat d []) (T₁ T₂ : ℝ) {μ : Measure ℝ}

```

```

(hA : IntervalIntegrable A μ T1 T2) :
  (∫ t in T1..T2, A t ∂μ).mat = ∫ t in T1..T2, (A t).mat ∂μ := by
  exact ((mat₁ (R := ℝ)).intervalIntegral_comp_comm hA).symm

/--
A sum of scaled constant matrices is integrable if the scalar functions are
-/
lemma intervalIntegrable_sum_smul_const (T1 T2 : ℝ) {μ : Measure ℝ} (g : ℝ
  (P : d → Matrix d d ℝ) (hg : ∀ i, IntervalIntegrable (fun t ↦ g t i) μ
    IntervalIntegrable (fun t ↦ ∑ i, g t i • P i) μ T1 T2 := by
  simp_all [intervalIntegrable_iff]
  exact integrable_finset_sum _ fun i _ ↦ Integrable.smul_const (hg i) _

/--
A function to Hermitian matrices is integrable iff its matrix values are in
-/
lemma intervalIntegrable_toMat_iff (A : ℝ → HermitianMat d ℝ) (T1 T2 : ℝ) {
  IntervalIntegrable (fun t ↦ (A t).mat) μ T1 T2 ↔ IntervalIntegrable A μ
  --TODO Cleanup
  simp [ intervalIntegrable_iff ];
  constructor <=> intro h;
  · -- Since `toMat` is a linear isometry, the integrability of `A.toMat` i
    have h_toMat_integrable : IntegrableOn (fun t ↦ (A t).mat) (Set.uIoc T1 T2)
      intro h_toMat_integrable
    have h_toMat_linear : ∃ (L : HermitianMat d ℝ →1 [ℝ] Matrix d d ℝ), ∀
      refine' { _, _ };
      refine' { .. };
      exacts [ fun x ↦ x.mat, fun x y ↦ rfl, fun m x ↦ rfl, fun x ↦ rfl ]
    obtain ⟨L, hL⟩ := h_toMat_linear;
    have h_toMat_linear : IntegrableOn (fun t ↦ L (A t)) (Set.uIoc T1 T2)
      intro h_toMat_integrable
    have h_toMat_linear : ∃ (L_inv : Matrix d d ℝ →1 [ℝ] HermitianMat d
      have h_toMat_linear : Function.Injective L := by
        intro x y hxy;
        simp_all only [HermitianMat.ext_iff]
      have h_toMat_linear : ∃ (L_inv : Matrix d d ℝ →1 [ℝ] HermitianMat
        exact IsSemisimpleModule.extension_property L h_toMat_linear Li
        exact ⟨ h_toMat_linear.choose, fun x ↦ by simpa using LinearMap.c
      obtain ⟨ L_inv, hL_inv ⟩ := h_toMat_linear;
      have h_toMat_linear : IntegrableOn (fun t ↦ L_inv (L (A t))) (Set.u
        exact ContinuousLinearMap.integrable_comp ( L_inv.toContinuousLin
      aesop;
      aesop;
      exact h_toMat_integrable h;
      · apply h.norm.mono'
      · have := h.aestronglyMeasurable;

```

```

    fun_prop
    · filter_upwards with t using le_rfl

/--
The CFC of an integrable function family is integrable.
-/
lemma integrable_cfc (T₁ T₂ : ℝ) (f : ℝ → ℝ → ℝ) {μ : Measure ℝ}
  (hf : ∀ i, IntervalIntegrable (fun t ↦ f t (A.H.eigenvalues i)) μ T₁ T₂) :
  IntervalIntegrable (fun t ↦ A.cfc (f t)) μ T₁ T₂ := by
  have h_expand : ∀ t, (A.cfc (f t)).mat = ∑ i, f t (A.H.eigenvalues i) • (A.H.eigen
    exact fun t ↦ cfc_toMat_eq_sum_smul_proj A (f t);
  rw [← intervalIntegrable_toMat_iff];
  rw [funext h_expand];
  apply intervalIntegrable_sum_smul_const
  exact hf

/--
The integral of the CFC is the CFC of the integral.
-/
lemma integral_cfc_eq_cfc_integral (T₁ T₂ : ℝ) {μ : Measure ℝ} (f : ℝ → ℝ → ℝ)
  (hf : ∀ i, IntervalIntegrable (fun t ↦ f t (A.H.eigenvalues i)) μ T₁ T₂) :
  ∫ t in T₁..T₂, A.cfc (f t) ∂μ = A.cfc (fun u ↦ ∫ t in T₁..T₂, f t u ∂μ) := by
  ext1
  rw [integral_toMat];
  · rw [intervalIntegral.integral_congr fun t ht ↦ HermitianMat.cfc_toMat_eq_sum_sm
    · rw [Finset.sum_congr rfl fun i _ ↦ intervalIntegral.integral_smul_const _ _]
      exact Eq.symm (cfc_toMat_eq_sum_smul_proj A fun u ↦ ∫ (t : ℝ) in T₁..T₂, f t u
    · simp_all [intervalIntegrable_iff];
      exact fun i ↦ (hf i).smul_const _
    · exact integrable_cfc T₁ T₂ f hf

end integral

theorem cfc_pos_of_pos {A : HermitianMat d []} {f : ℝ → ℝ} (hA : 0 < A)
  (hf : ∀ i > 0, 0 < f i) (hf₂ : 0 ≤ f 0) : 0 < A.cfc f := by
  have h_pos := (posSemidef_iff_spectrum_nonneg A).mp hA.le
  have h_f_pos : ∃ x ∈ spectrum ℝ (A.cfc f).mat, x ≠ 0 := by
    obtain ⟨ x, hx₁, hx₂ ⟩ := ne_zero_iff_ne_zero_spectrum A |>.1 hA.ne'
    exact ⟨ f x, by simp using HermitianMat.spectrum_cfc_eq_image A f ▸ Set.mem_ima
  have h_f_nonneg : 0 ≤ A.cfc f := by
    rw [HermitianMat.posSemidef_iff_spectrum_nonneg];
    rw [HermitianMat.spectrum_cfc_eq_image];
    rintro _ ⟨ x, hx, rfl ⟩; exact if hx0 : x = 0 then by simp [hx0] using hf₂ e
  have h_f_nonzero : A.cfc f ≠ 0 := by
    contrapose! h_f_pos;
    simp [h_f_pos, spectrum.mem_iff, Matrix.isUnit_iff_isUnit_det, Algebra.algebraMa

```

```

exact lt_of_le_of_ne h_f_nonneg h_f_nonzero.symm

/-- If two matrices A and B commute, then they is a common matrix with which
This is a variant of the common theorem that "commuting matrices can be sim
/
theorem _root_.Commute.exists_HermitianMat_cfc (hAB : Commute A.mat B.mat)
  ∃ C : HermitianMat d [], (∃ f : ℝ → ℝ, A = C.cfc f) ∧ (∃ g : ℝ → ℝ, B =
obtain ⟨C, ⟨g₁, hg₁⟩, ⟨g₂, hg₂⟩⟩ := hAB.exists_cfc A.H B.H
by_cases hC : C.IsHermitian
· use ⟨C, hC⟩
  constructor
  · exact ⟨g₁, by simp [HermitianMat.ext_iff, hg₁]⟩
  · exact ⟨g₂, by simp [HermitianMat.ext_iff, hg₂]⟩
· change ¬(IsSelfAdjoint C) at hC
  rw [cfc_apply_of_not_predicate C hC] at hg₁ hg₂
  use 0
  constructor
  · exact ⟨0, by simp [HermitianMat.ext_iff, hg₁]⟩
  · exact ⟨0, by simp [HermitianMat.ext_iff, hg₂]⟩

open ComplexOrder in
theorem cfc_le_cfc_of_PosDef (hfg : ∀ i, 0 < i → f i ≤ g i) (hA : A.mat.PosDef)
  A.cfc f ≤ A.cfc g := by
  rw [← sub_nonneg, ← HermitianMat.cfc_sub, cfc_nonneg_iff]
  intro i
  rw [Pi.sub_apply, sub_nonneg]
  rw [A.H.posDef_iff_eigenvalues_pos] at hA
  apply hfg
  apply hA

open ComplexOrder in
variable {f} in
/- TODO: Write a version of this that holds more broadly for some sets. Esp
which correspond nicely to closed intervals of matrices. Write the speciali
instead of MonotoneOn). Also a version that works for StrictMonoOn. -
/
theorem cfc_le_cfc_of_commute_monoOn (hf : MonotoneOn f (Set.Ioi 0))
  (hAB₁ : Commute A.mat B.mat) (hAB₂ : A ≤ B) (hA : A.mat.PosDef) (hB : B.mat.PosDef)
  A.cfc f ≤ B.cfc f := by
  obtain ⟨C, ⟨g₁, rfl⟩, ⟨g₂, rfl⟩⟩ := hAB₁.exists_HermitianMat_cfc
  -- Need to show that g₁ ≤ g₂ on spectrum ℝ C
  rw [← C.cfc_comp, ← C.cfc_comp]
  rw [← sub_nonneg, ← C.cfc_sub, cfc_nonneg_iff] at hAB₂ ⊢
  intro i
  simp only [Pi.sub_apply, Function.comp_apply, sub_nonneg]
  apply hf

```

```

    · rw [cfc_posDef] at hA
      exact hA i
    · rw [cfc_posDef] at hB
      exact hB i
    · simpa using hAB₂ i

/-- TODO: See above -/
theorem cfc_le_cfc_of_commute (hf : Monotone f) (hAB₁ : Commute A.mat B.mat) (hAB₂ :
  A.cfc f ≤ B.cfc f) := by
  obtain ⟨C, ⟨g₁, rfl⟩, ⟨g₂, rfl⟩⟩ := hAB₁.exists_HermitianMat_cfc
  -- Need to show that  $g_1 \leq g_2$  on spectrum  $\mathbb{R} C$ 
  rw [← C.cfc_comp, ← C.cfc_comp]
  rw [← sub_nonneg, ← C.cfc_sub, cfc_nonneg_iff] at hAB₂ ⊢
  intro i
  simp only [Pi.sub_apply, Function.comp_apply, sub_nonneg]
  apply hf
  simpa using hAB₂ i

--This is the more general version that requires operator concave functions but does
-- to commute. Requires the correct statement of operator convexity though, which we
open ComplexOrder in
proof_wanted cfc_mono0n_pos_of_mono0n_posDef {d : Type*} [Fintype d] [DecidableEq d]
  {f : ℝ → ℝ} (hf_is_operator_convex : False) :
  Monotone0n (HermitianMat.cfc · f) { A : HermitianMat d ℂ | A.mat.PosDef }

section uncategorized_cleanup

open ComplexOrder in
theorem inv_ge_one_of_le_one (hA : A.mat.PosDef) (h : A ≤ 1) : 1 ≤ A⁻¹ := by
  -- Since  $A$  is positive definite and  $A \leq 1$ , we have  $A.cfc (\text{fun } x \Rightarrow x^{-1} - 1)$ 
  have h_cfc_nonneg : 0 ≤ A.cfc (fun x => x⁻¹ - 1) := by
    have h_cfc_nonneg : ∀ i, 0 ≤ (A.H.eigenvalues i)⁻¹ - 1 := by
      have h_pos : ∀ i, 0 < A.H.eigenvalues i ∧ A.H.eigenvalues i ≤ 1 := by
        -- Since  $A$  is positive definite, all its eigenvalues are positive.
        have h_pos : ∀ i, 0 < A.H.eigenvalues i := by
          exact fun i => Matrix.PosDef.eigenvalues_pos hA i;
        -- Since  $A \leq 1$ , for any eigenvalue  $\lambda_i$  of  $A$ , we have  $\lambda_i \leq 1$ 
        have h_le_one : ∀ i, A.H.eigenvalues i ≤ 1 := by
          have h_le_one : ∀ i, A.H.eigenvalues i ≤ 1 := by
            intro i
            have h_eigenvalue : A.mat.PosSemidef := by
              exact hA.posSemidef
            have h_eigenvalue_le_one : ∀ x : d → ℝ, x ≠ 0 → (star x ⌊ₐ A.mat.mulVec
              intro x hx_ne_zero
              have h_eigenvalue_le_one : (star x ⌊ₐ (1 - A.mat).mulVec x) ≥ 0 := by
                exact Matrix.PosSemidef.dotProduct_mulVec_nonneg h x

```

```

      generalize_proofs at *; (
        rw [ div_le_iff₀ ] <=> simp_all [ Matrix.sub_mulVec, dotProdu
      generalize_proofs at *; (
        have := h_eigenvalue_le_one ( A.H.eigenvectorBasis i ) ?_ <=> s
        · have := Matrix.IsHermitian.mulVec_eigenvectorBasis ( show Mat
          by_cases h : ( A.H.eigenvectorBasis i |> WithLp.ofLp ) []_ sta
          exact absurd h ( by exact ne_of_apply_ne ( fun x => ||x|| ) ( b
        · exact fun h => by simp [ h ] using ( A.H.eigenvectorBasis.or
      generalize_proofs at *; (
        exact h_le_one)
        exact fun i => ⟨h_pos i, h_le_one i⟩;
        exact fun i => sub_nonneg_of_le ( one_le_inv₀ ( h_pos i |>.1 ) |>.2 (
        exact (cfc_nonneg_iff A fun x => x⁻¹ - 1).mpr h_cfc_nonneg;
      -- Since $A.cfc (fun x => x⁻¹ - 1) \geq 0$, we have $A.cfc (fun x => x⁻¹)
      have h_cfc_ge_one : A.cfc (fun x => x⁻¹) ≥ 1 := by
        have h_cfc_sub : A.cfc (fun x => x⁻¹ - 1) = A.cfc (fun x => x⁻¹) - A.cf
        exact cfc_sub_apply A Inv.inv fun x => 1;
        aesop;
        convert h_cfc_ge_one.le using 1;
        convert cfc_inv.symm;
        exact nonSingular_of_posDef hA

/-- The trace of cfc(f, A) equals the sum of f applied to eigenvalues. -
/
lemma trace_cfc_eq (A : HermitianMat d ℂ) (f : ℝ → ℝ) :
  (A.cfc f).trace = ∑ i, f (A.H.eigenvalues i) := by
  have h1 := HermitianMat.trace_eq_trace (A.cfc f)
  obtain ⟨e, he⟩ := HermitianMat.cfc_eigenvalues f A
  have h2 := (A.cfc f).H.trace_eq_sum_eigenvalues
  rw [he] at h2
  simp [Function.comp] at h2
  rw [HermitianMat.mat_cfc] at h1
  rw [h2] at h1
  have h3 : (Complex.ofReal) (A.cfc f).trace = Complex.ofReal (∑ i, f (A.H.
    convert h1 using 1
    simp
  have h4 := Complex.ofReal_injective h3
  rw [h4]
  exact Equiv.sum_comp e (fun x => f (A.H.eigenvalues x))

end uncategorized_cleanup

lemma mulVec_eq_zero_iff_inner_eigenvector_zero
  (A : HermitianMat d ℂ) (x : EuclideanSpace ℂ d) :
  A.mat.mulVec x = 0 ↔ ∀ i, A.H.eigenvalues i ≠ 0 → inner ℂ (A.H.eigenvec
  -- Since the eigenvectors form an orthonormal basis, we can express x as

```

```

obtain (c, hc) : ∃ c : d → ℂ, x = ∑ i, c i • A.H.eigenvectorBasis i := by
  have := A.H.eigenvectorBasis.sum_repr x;
  exact ( _, this.symm );
-- By definition of $A$, we know that $A.mulVec (x.ofLp) = \sum_{i} c_i \lambda_i$
have h_mulVec : A.val.mulVec (x.ofLp) = ∑ i, c i • (A.H.eigenvalues i) • (A.H.eigenvectorBasis i)
have h_mulVec : ∀ i, A.val.mulVec (A.H.eigenvectorBasis i) = (A.H.eigenvalues i) • (A.H.eigenvectorBasis i)
  intro i
  have := A.H.mulVec_eigenvectorBasis i
  aesop;
convert congr_arg ( fun y => ( ∑ i, c i • y i ) ) ( funext fun i => h_mulVec i )
· simp [ hc, ];
ext i; rw [ Matrix.mulVec, dotProduct ]
simp [ Finset.mul_sum _ _ _, mul_assoc, mul_comm ]
rw [ Finset.sum_comm ]
simp [ Matrix.mulVec, dotProduct, mul_comm, Finset.mul_sum _ _ _ ]
· ext i; simp [ Finset.sum_apply ] ;
constructor;
· intro h i hi
  have h_inner : inner ℂ (A.H.eigenvectorBasis i) (∑ j, c j • (A.H.eigenvalues j) • (A.H.eigenvectorBasis j)) = 0 := by
    convert congr_arg ( fun x => inner ℂ (A.H.eigenvectorBasis i) x ) ( show ( ∑ j, c j • (A.H.eigenvalues j) • (A.H.eigenvectorBasis j)) = 0 )
    · simp [ inner_zero_right ];
    · ext j; replace h := congr_fun h j; aesop;
  simp_all
  convert congr_arg ( fun x : ℂ => x / (A.H.eigenvalues i) ) h_inner using 1 <;>
  refine' Finset.sum_congr rfl fun j _ => _ ; by_cases hj : A.H.eigenvalues j = 0
  · by_cases hij : i = j <;> simp_all [ ];
  · by_cases hij : i = j <;> simp_all [ inner_self_eq_norm_sq_to_K ];
· intro h
  have h_zero_coeffs : ∀ i, A.H.eigenvalues i ≠ 0 → c i = 0 := by
    intro i hi; specialize h i hi; simp_all
    rw [ Finset.sum_eq_single i ] at h <;> simp_all [ orthonormal_iff_ite.mp ( A.H.eigenvectorBasis i ), inner_self_eq_norm_sq_to_K ]
    aesop;
  simp_all
  exact Finset.sum_eq_zero fun i _ => by by_cases hi : A.H.eigenvalues i = 0 <;> s

open InnerProductSpace in
lemma cfc_mulVec_expansion (A : HermitianMat d ℂ) (f : ℝ → ℝ) (x : EuclideanSpace ℂ) :
  (A.cfc f).mat.mulVec x = ∑ i, (f (A.H.eigenvalues i) : ℂ) • inner ℂ (A.H.eigenvectorBasis i) x := by
  ext i; simp [ Matrix.mulVec, dotProduct ] ; ring_nf
-- By definition of $cfc$, we know that $(A.cfc f).i j = \sum_k f(\lambda_k) \langle x, A.H.eigenvectorBasis k \rangle \cdot A.H.eigenvectorBasis k$
have h_cfc_def : (A.cfc f).mat i = ∑ k, f (A.H.eigenvalues k) • (A.H.eigenvectorBasis k) • inner ℂ (A.H.eigenvectorBasis i) (A.H.eigenvectorBasis k)
-- By definition of $cfc$, we know that $(A.cfc f).i j = \sum_k f(\lambda_k) \langle x, A.H.eigenvectorBasis k \rangle \cdot A.H.eigenvectorBasis k$
have h_cfc_def : (A.cfc f).mat = ∑ k, f (A.H.eigenvalues k) • (A.H.eigenvectorBasis k) • inner ℂ (A.H.eigenvectorBasis i) (A.H.eigenvectorBasis k)
  convert cfc_toMat_eq_sum_smul_proj A f using 1;
convert congr_fun h_cfc_def i using 1;
simp [ funext_iff, Matrix.single ];

```

```

    simp [ Matrix.mul_apply, Matrix.conjTranspose_apply, Matrix.sum_apply,
    intro x; congr; ext y; simp [ Finset.sum_ite, Finset.filter_eq, Finset.
    rw [ Finset.sum_eq_single y ] <=> aesop;
    simp_all [mul_comm, mul_left_comm ] ; ring_nf
    convert congr_arg ( fun y =>  $\sum j, x.\text{ofLp } j * y j$  ) h_cfc_def using 1
    simp [ Finset.mul_sum _ _ _ , mul_assoc, mul_left_comm ]
    ring_nf!
    rw [ Finset.sum_comm, Finset.sum_congr rfl ]
    intros
    simp [ mul_assoc, inner ]
    ring_nf!
    simp only [Finset.mul_sum _ _ _ , mul_assoc]

section ker_cfc

variable {A : HermitianMat d  $\mathbb{C}$ } {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } {s : Set  $\mathbb{R}$ }

lemma ker_cfc_le_ker_on_set
  (hs : spectrum  $\mathbb{R}$  A.mat  $\subseteq s$ )
  (h :  $\forall i \in s, f i = 0 \rightarrow i = 0$ ) :
  (A.cfc f).ker  $\leq$  A.ker := by
  intro x hx
  have h_inner :  $\forall i, A.H.\text{eigenvalues } i \neq 0 \rightarrow \text{inner } \mathbb{C} (A.H.\text{eigenvectorBasis } i) x = 0$  := by
    have h_inner_zero : (A.cfc f).mat.mulVec x = 0 := by
      exact (mem_ker_iff_mulVec_zero (A.cfc f) x).mp hx
    have h_inner_zero_expansion :  $\sum i, (f (A.H.\text{eigenvalues } i) : \mathbb{C}) \cdot \text{inner } \mathbb{C} (A.H.\text{eigenvectorBasis } i) x = 0$  := by
      convert h_inner_zero using 1;
      rw [ cfc_mulVec_expansion ];
      exact Iff.symm (WithLp.ofLp_eq_zero 2)
    have h_inner_zero_coeff :  $\forall i, f (A.H.\text{eigenvalues } i) \cdot \text{inner } \mathbb{C} (A.H.\text{eigenvectorBasis } i) x = 0$  := by
      intro i
      have h_inner_zero_coeff_i :  $f (A.H.\text{eigenvalues } i) \cdot \text{inner } \mathbb{C} (A.H.\text{eigenvectorBasis } i) x = 0$  := by
        simp [ orthonormal_iff_ite.mp ( A.H.eigenvectorBasis.orthonormal ) ]
      rw [ h_inner_zero_coeff_i, h_inner_zero_expansion ]
      simp [ inner_zero_right ] -- This line is just to prevent the proof from
    have h_inner_zero_final :  $\forall i, A.H.\text{eigenvalues } i \neq 0 \rightarrow \text{inner } \mathbb{C} (A.H.\text{eigenvectorBasis } i) x = 0$  := by
      -- Since  $A.H.\text{eigenvalues } i \neq 0$ , by hypothesis  $h$ , we have  $f(A.H.\text{eigenvalues } i) \neq 0$ .
      have h_f_nonzero :  $\forall i, A.H.\text{eigenvalues } i \neq 0 \rightarrow f (A.H.\text{eigenvalues } i) \neq 0$  := by
        intro i hi; specialize h ( A.H.eigenvalues i ) ( hs <| by
          exact Matrix.IsHermitian.eigenvalues_mem_spectrum_real (H A) i )
        generalize_proofs at *; (
          exact fun i hi => by simp [ h_f_nonzero i hi ] using h_inner_zero_coeff_i
        )
      exact h_inner_zero_final;
    convert mulVec_eq_zero_iff_inner_eigenvector_zero A x |>.2 h_inner using
    exact mem_ker_iff_mulVec_zero A x

```



```

lemma ker_cfc_le_ker (h :  $\forall i, f i = 0 \rightarrow i = 0$ ) :
  (A.cfc f).ker  $\leq$  A.ker := by
  exact ker_cfc_le_ker_on_set (Set.subset_univ _) (by simp using h)

lemma ker_cfc_le_ker_nonneg (hA :  $0 \leq A$ ) (h :  $\forall i \geq 0, f i = 0 \rightarrow i = 0$ ) :
  (A.cfc f).ker  $\leq$  A.ker := by
  rw [posSemidef_iff_spectrum_Ici] at hA
  exact ker_cfc_le_ker_on_set hA h

lemma ker_le_ker_cfc_on_set (hs : spectrum  $\mathbb{R}$  A.mat  $\subseteq s$ ) (h :  $\forall i \in s, i = 0 \rightarrow f i = 0$ ) :
  A.ker  $\leq$  (A.cfc f).ker := by
  intro x hx
  have h_inner_zero :  $\forall i, A.H.eigenvalues i \neq 0 \rightarrow \text{inner } \mathbb{C} (A.H.eigenvectorBasis i) x = 0$  := by
    intro i hi
    have h_inner_zero : A.mat.mulVec x = 0 := by
      exact (mem_ker_iff_mulVec_zero A x).mp hx;
    have := mulVec_eq_zero_iff_inner_eigenvector_zero A x; aesop;
  have h_mulVec_zero : (A.cfc f).mat.mulVec x =  $\sum i, (f (A.H.eigenvalues i) : \mathbb{C}) \cdot i$  := by
    convert cfc_mulVec_expansion A f x using 1;
  convert h_mulVec_zero using 1
  simp_all [funext_iff];
  ext i; specialize h_mulVec_zero i; simp_all [lin, Matrix.mulVec];
  refine' Finset.sum_eq_zero fun j _ => ?_
  by_cases hj : A.H.eigenvalues j = 0 <=> simp_all
  exact 0r.inl (h _ (hs (Matrix.IsHermitian.eigenvalues_mem_spectrum_real (H A) j)))

lemma ker_le_ker_cfc (h :  $\forall i, i = 0 \rightarrow f i = 0$ ) :
  A.ker  $\leq$  (A.cfc f).ker := by
  exact ker_le_ker_cfc_on_set (Set.subset_univ _) (by simp using h)

lemma ker_le_ker_cfc_nonneg (hA :  $0 \leq A$ ) (h :  $\forall i \geq 0, i = 0 \rightarrow f i = 0$ ) :
  A.ker  $\leq$  (A.cfc f).ker := by
  rw [posSemidef_iff_spectrum_Ici] at hA
  exact ker_le_ker_cfc_on_set hA h

theorem ker_cfc_eq_ker (h :  $\forall i, f i = 0 \leftrightarrow i = 0$ ) :
  (A.cfc f).ker = A.ker := by
  refine le_antisymm (ker_cfc_le_ker ?_) (ker_le_ker_cfc ?_)
  <=> grind only

theorem ker_cfc_eq_ker_nonneg (hA :  $0 \leq A$ ) (h :  $\forall i \geq 0, f i = 0 \leftrightarrow i = 0$ ) :
  (A.cfc f).ker = A.ker := by
  refine le_antisymm (ker_cfc_le_ker_nonneg hA ?_) (ker_le_ker_cfc_nonneg hA ?_)
  <=> grind only

end ker_cfc

```

end CFC

1.458 QuantumInfo/ForMathlib/HermitianMat/Inner.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Order
import Mathlib.Analysis.Convex.Contractible
import Mathlib.Topology.Instances.Real.Lemmas

/-! # Inner product of Hermitian Matrices

For general matrices there are multiple reasonable notions of "inner product" (the Frobenius inner product), and so Mathlib avoids giving a canonical `InnerProductSpace` instance for the particular case of Hermitian matrices, these all coincide, so we can put a canonical instance.

This _does_ however induce a `Norm` on `HermitianMat` as well, the Frobenius norm is a uniquely correct choice. It is something that one essentially has to live with. Currently structures the instances. (Thankfully, all norms induce the same topology on finite-dimensional matrices.)

Some care to be taken so that the topology induced by the `InnerProductSpace` instance on HermitianMat is the topology that HermitianMat inherits from the topology on Matrix. This can be done by using `InnerProductSpace.ofCoreOfTopology`.

-/

namespace HermitianMat

variable {R n α : Type*} [Star R] [TrivialStar R] [Fintype n]
open scoped InnerProductSpace RealInnerProductSpace
open IsMaximalSelfAdjoint

section defs

variable [Ring α] [StarAddMonoid α] [CommSemiring R] [Algebra R α] [IsMaximalSelfAdjoint α]

/-- The Hermitian inner product, `Tr[AB]`. This is equal to `Matrix.trace (A * B)` for real values when the matrices are complex, using `IsMaximalSelfAdjoint`. -
/
```

```

instance : Inner R (HermitianMat n  $\alpha$ ) where
  inner A B := selfadjMap (A.mat * B.mat).trace

theorem inner_def (A B : HermitianMat n  $\alpha$ ) :
   $\langle A, B \rangle_R$  = selfadjMap (A.mat * B.mat).trace := by
  rfl

end defs
section semiring

--We necessarily re-state and re-prove many of the theorems from InnerProductSpace/B
--because our inner product happens outside of just an `InnerProductSpace` instance.

variable [CommSemiring R] [Ring  $\alpha$ ] [StarAddMonoid  $\alpha$ ] [Algebra R  $\alpha$ ] [IsMaximalSelfAdj]
variable (A B C : HermitianMat n  $\alpha$ )

protected theorem inner_add_right :  $\langle A, B + C \rangle_R$  =  $\langle A, B \rangle_R$  +  $\langle A, C \rangle_R$  := by
  simp [inner_def, left_distrib]

protected theorem inner_add_left :  $\langle A + B, C \rangle_R$  =  $\langle A, C \rangle_R$  +  $\langle B, C \rangle_R$  := by
  simp [inner_def, right_distrib]

@[simp]
protected theorem inner_zero_right :  $\langle A, 0 \rangle_R$  = 0 := by
  simp [inner_def]

@[simp]
protected theorem inner_zero_left :  $\langle 0, A \rangle_R$  = 0 := by
  simp [inner_def]

end semiring
section ring

variable [CommRing R] [Ring  $\alpha$ ] [StarAddMonoid  $\alpha$ ] [Algebra R  $\alpha$ ] [IsMaximalSelfAdjoint]
variable (A B C : HermitianMat n  $\alpha$ )

@[simp]
protected theorem inner_neg_left :  $\langle -A, B \rangle_R$  = - $\langle A, B \rangle_R$  := by
  simp [inner_def]

@[simp]
protected theorem inner_neg_right :  $\langle A, -B \rangle_R$  = - $\langle A, B \rangle_R$  := by
  simp [inner_def]

protected theorem inner_sub_left :  $\langle A, B - C \rangle_R$  =  $\langle A, B \rangle_R$  -  $\langle A, C \rangle_R$  := by
  simp [inner_def, mul_sub]

```

```

protected theorem inner_sub_right :  $\square A - B$ ,  $C\square_R = \square A$ ,  $C\square_R - \square B$ ,  $C\square_R := b$ 
  simp [inner_def, sub_mul]

variable [StarModule R  $\alpha$ ]

@[simp]
protected theorem inner_smul_left (r : R) :  $\square r \cdot A$ ,  $B\square_R = r * \square A$ ,  $B\square_R := b$ 
  simp [inner_def, selfadj_smul]

@[simp]
protected theorem inner_smul_right (r : R) :  $\square A$ ,  $r \cdot B\square_R = r * \square A$ ,  $B\square_R := b$ 
  simp [inner_def, selfadj_smul]

/-- The Hermitian inner product as bilinear form. Compare with `inner1` (in
which requires an `InnerProductSpace` instance. -/
protected def inner1 : LinearMap.BilinForm R (HermitianMat n  $\alpha$ ) where
  toFun A := {
    toFun := ( $\square A$ ,  $\cdot\square_R$ )
    map_add' := A.inner_add_right
    map_smul' r B := by simp
  }
  map_add' A B := by ext1; apply A.inner_add_left B
  map_smul' A B := by ext1; simp

end ring
section starring

variable [CommSemiring R] [Ring  $\alpha$ ] [StarRing  $\alpha$ ] [Algebra R  $\alpha$ ] [IsMaximalSel
variable (A B : HermitianMat n  $\alpha$ )

@[simp]
theorem inner_one :  $\square A$ ,  $1\square_R = A.trace$  := by
  simp only [inner_def, mat_one, mul_one, trace]

@[simp]
theorem one_inner :  $\square 1$ ,  $A\square_R = A.trace$  := by
  simp only [inner_def, one_mul, mat_one, trace]

end starring
section commring

variable [CommSemiring R] [CommRing  $\alpha$ ] [StarRing  $\alpha$ ] [Algebra R  $\alpha$ ] [IsMaxima
variable (A B : HermitianMat n  $\alpha$ )

/-- The inner product for Hermtian matrices is equal to the trace of the pr

```

```

/
theorem inner_eq_trace_mul : algebraMap R  $\alpha$   $\square$ A, B  $\square$ _R = (A.mat * B.mat).trace := by
  apply IsMaximalSelfAdjoint.selfadj_algebra
  rw [IsSelfAdjoint, Matrix.trace]
  simp_rw [star_sum, Matrix.diag_apply, Matrix.mul_apply, star_sum, star_mul, mul_comm]
  rw [Finset.sum_comm]
  congr! <=> apply congrFun2 (H _)

theorem inner_comm :  $\square$ A, B  $\square$ _R =  $\square$ B, A  $\square$ _R := by
  rw [inner_def, inner_def, Matrix.trace_mul_comm]

end commring

section trivialstar
variable [CommRing  $\alpha$ ] [StarRing  $\alpha$ ] [TrivialStar  $\alpha$ ]
variable (A B : HermitianMat n  $\alpha$ )

/-- `HermitianMat.inner` reduces to `Matrix.trace (A * B)` when the elements are a `
/
theorem inner_eq_trace_trivial :  $\square$ A, B  $\square$ _ $\alpha$  = (A.mat * B.mat).trace := by
  rw [← inner_eq_trace_mul]
  rfl

end trivialstar

section RCLike
open ComplexOrder

variable {n  $\square$  : Type*} [Fintype n] [RCLike  $\square$ ] (A B C : HermitianMat n  $\square$ )

theorem inner_eq_re_trace :  $\square$ A, B  $\square$  = RCLike.re (A.mat * B.mat).trace := by
  rfl

theorem inner_eq_trace_rc :  $\square$ A, B  $\square$  = (A.mat * B.mat).trace := by
  rw [inner_eq_re_trace, ← RCLike.conj_eq_iff_re]
  convert (Matrix.trace_conjTranspose (A.mat * B.mat)).symm using 1
  rw [Matrix.conjTranspose_mul, A.H, B.H, Matrix.trace_mul_comm]

theorem inner_self_nonneg: 0  $\leq$   $\square$ A, A  $\square$  := by
  simp_rw [inner_eq_re_trace, Matrix.trace, Matrix.diag, Matrix.mul_apply, map_sum]
  refine Finset.sum_nonneg fun i _  $\mapsto$  Finset.sum_nonneg fun j _  $\mapsto$  ?_
  rw [← congrFun2 A.H, Matrix.conjTranspose_apply]
  refine And.left <| RCLike.nonneg_iff.mp ?_
  open ComplexOrder in
  exact star_mul_self_nonneg (A.mat j i)

```

```

variable {A B C}

open MatrixOrder in
theorem inner_mul_nonneg (h :  $0 \leq A.mat * B.mat$ ) :  $0 \leq \langle A, B \rangle$  := by
  rw [Matrix.nonneg_iff_posSemidef] at h
  exact (RCLike.nonneg_iff.mp h.trace_nonneg).left

/-- The inner product for PSD matrices is nonnegative. -/
theorem inner_ge_zero (hA :  $0 \leq A$ ) (hB :  $0 \leq B$ ) :  $0 \leq \langle A, B \rangle$  := by
  rw [zero_le_iff] at hB
  open MatrixOrder in
  open Classical in
  rw [inner_eq_re_trace, ← CFC.sqrt_mul_sqrt_self A.mat hA, Matrix.trace_mul]
  nth_rewrite 1 [← (Matrix.nonneg_iff_posSemidef.mp (CFC.sqrt_nonneg A.mat)
  exact (RCLike.nonneg_iff.mp (hB.conjTranspose_mul_mul_same _).trace_nonneg

theorem inner_mono (hA :  $0 \leq A$ ) :  $B \leq C \rightarrow \langle A, B \rangle \leq \langle A, C \rangle$  := by
  intro hBC
  classical have hTr :  $0 \leq \langle A, C - B \rangle$  := inner_ge_zero hA (zero_le_iff.mpr
  simp [inner_def, mul_sub] using hTr

theorem inner_mono' (hA :  $0 \leq A$ ) :  $B \leq C \rightarrow \langle B, A \rangle \leq \langle C, A \rangle$  := by
  intro hBC
  rw [inner_comm B A, inner_comm C A]
  exact inner_mono hA hBC

/-- The inner product for PSD matrices is at most the product of their traces. -/
theorem inner_le_mul_trace (hA :  $0 \leq A$ ) (hB :  $0 \leq B$ ) :  $\langle A, B \rangle \leq A.trace * B.trace$ 
  classical convert inner_mono hA (le_trace_smul_one hB)
  simp [mul_comm]

--TODO cleanup
private theorem inner_zero_iff_aux_lemma [DecidableEq n] (hA1 : A.mat.PosSemidef) :
  RCLike.re (A.val * B.val).trace = 0 ↔
    LinearMap.range (Matrix.toEuclideanLin A.val) ≤
      LinearMap.ker (Matrix.toEuclideanLin B.val) := by
  open MatrixOrder in
  --Thanks Aristotle
  have h_trace_zero : (RCLike.re ((A.val * B.val).trace)) = 0 ↔ (A.val * B.val).trace = 0 := by
    -- Since $A$ and $B$ are positive semidefinite, we can write them as $C^* C$ and $D^* D$.
    obtain ⟨C, hC⟩ : ∃ C : Matrix n n ℝ, A.val = C.conjTranspose * C := by
      rw [← Matrix.nonneg_iff_posSemidef] at hA1
      exact CStarAlgebra.nonneg_iff_eq_star_mul_self.mp hA1
    obtain ⟨D, hD⟩ : ∃ D : Matrix n n ℝ, B.val = D.conjTranspose * D := by

```

```

    erw [← Matrix.nonneg_iff_posSemidef] at hB₁
    exact CStarAlgebra.nonneg_iff_eq_star_mul_self.mp hB₁
  have h_trace_zero_iff : (RCLike.re ((A.val * B.val).trace)) = 0 ↔ (D * C.conjTranspose) = 0
  -- Since  $\text{Tr}((DC)^* DC) = \sum_{i,j} |(DC)_{ij}|^2$ , and this is non-negative, we have
  have h_trace_zero_iff : (RCLike.re ((D * C.conjTranspose).conjTranspose * (D * C.conjTranspose))) = 0 ↔ (D * C.conjTranspose) = 0
  have h_trace_zero_iff : ∀ (M : Matrix n n ℝ), (RCLike.re (M.conjTranspose * M)) = 0 ↔ M = 0
  simp [Matrix.trace, Matrix.mul_apply];
  intro M
  -- simp_all only
  obtain ⟨val, property⟩ := A
  obtain ⟨val₁, property₁⟩ := B
  subst hD hC
  apply Iff.intro
  · intro a
    rw [Finset.sum_eq_zero_iff_of_nonneg fun i _ => Finset.sum_nonneg fun j _ => (a i j).norm_num]
    ext i j
    specialize a j
    rw [Finset.sum_eq_zero_iff_of_nonneg fun _ _ => add_nonneg (mul_self_nonneg (a i j).norm_num) (0 : ℝ)]
    simp_all only [Finset.mem_univ, forall_const, Matrix.zero_apply]
    exact RCLike.ext (by norm_num; nlinarith only [a i]) (by norm_num; nlinarith only [a i])
  · intro a
    subst a
    simp_all only [Matrix.zero_apply, map_zero, mul_zero, add_zero, Finset.sum_eq_zero_iff]
    exact h_trace_zero_iff _;
  convert h_trace_zero_iff using 3
  simp [Matrix.mul_assoc];
  rw [← Matrix.trace_mul_comm]
  have h_trace_cyclic : Matrix.trace (D.conjTranspose * D * C.conjTranspose * C) = Matrix.trace (C * D.conjTranspose * D * C)
  rw [← Matrix.trace_mul_comm]
  simp [Matrix.mul_assoc];
  simp_all [Matrix.mul_assoc]
  simp_all only
  obtain ⟨val, property⟩ := A
  obtain ⟨val₁, property₁⟩ := B
  subst hD hC
  apply Iff.intro
  · intro a
    simp_all only [iff_true]
    simp [← Matrix.mul_assoc, ← Matrix.conjTranspose_inj, a];
  · intro a
    simp_all only [Matrix.trace_zero, map_zero, true_iff]
  have h_range_ker : (LinearMap.range (Matrix.toEuclideanLin A.val)) ≤ (LinearMap.range (Matrix.toEuclideanLin B.val))
  intro h_range_ker
  have hAB_zero : ∀ v, (Matrix.toEuclideanLin B.val) ((Matrix.toEuclideanLin A.val).val v) = 0
  exact fun v => h_range_ker (LinearMap.mem_range_self _ v)
  have h_herm : A.val * B.val = (B.val * A.val).conjTranspose := by

```

```

    simp [Matrix.conjTranspose_mul]
  have hBA_zero : (B.val * A.val) = 0 := by
    ext i j
    specialize hAB_zero (EuclideanSpace.single j 1)
    have h1 := hAB_zero
    simp only [Matrix.toEuclideanLin, Matrix.toLpLin_apply, Matrix.mulVec]
    have h2 := congr_fun (congrArg WithLp.ofLp h1) i
    simp only [WithLp.ofLp_toLp, WithLp.ofLp_zero, EuclideanSpace.single]
    simp [Matrix.mul_apply, Matrix.mulVec, dotProduct, Pi.single_apply]
    rw [h_herm, hBA_zero, Matrix.conjTranspose_zero]
  simp_all only
  obtain ⟨val, property⟩ := A
  obtain ⟨val_1, property_1⟩ := B
  simp_all only
  apply Iff.intro
  · rintro a _ ⟨y, rfl⟩
    have h_comm : val_1 * val = 0 := by
      rw [← Matrix.conjTranspose_inj]
      have h_conj_transpose : val.conjTranspose = val ∧ val_1.conjTranspose
        aesop
      simp [h_conj_transpose, Matrix.conjTranspose_mul, a]
    simp only [LinearMap.mem_ker]
    show (Matrix.toEuclideanLin val_1) ((Matrix.toEuclideanLin val) y) = 0
    simp [Matrix.toEuclideanLin, Matrix.toLpLin_apply, Matrix.mulVec_mulVec]
  · grind

/-- The inner product of two PSD matrices is zero iff they have disjoint su
in the other's kernel. -/
theorem inner_zero_iff [DecidableEq n] (hA1 : 0 ≤ A) (hB1 : 0 ≤ B)
  : [A, B] = 0 ↔ A.support ≤ B.ker := by
  rw [zero_le_iff] at hA1 hB1
  rw [inner_eq_re_trace]
  exact inner_zero_iff_aux_lemma hA1 hB1

variable {d d2 : Type*} (A B : HermitianMat d []) [Fintype d2] [Fintype d]

@[simp]
theorem reindex_inner (e : d ≈ d2) (B : HermitianMat d2 []) :
  [A.reindex e, B] = [A, B.reindex e.symm] := by
  simp only [inner_def, RCLike_selfadjMap, mat_reindex, Matrix.reindex_appl
  congr
  rw (occs := [3,4]) [← e.symm_symm]
  rw [← Matrix.submatrix_id_mul_right]
  rw (occs := [2]) [Matrix.trace_mul_comm]
  rw [Matrix.submatrix_id_mul_right, Matrix.trace_mul_comm, Equiv.symm_symm

```



```
end RCLike
```

```
section topology
```

```
/-!
```

```
Theorems about `HermitianMat`s that have to do with the topological structure. Pretty
assume these are matrices over  $\mathbb{C}$ , but changes to upgrade this to other types are wel
```

```
-/
```

```
open ComplexOrder
```

```
variable {d : Type*} [Fintype d] {α : Type*} [RCLike α]
```

```
--Check that it synthesizes ok
```

```
#guard_msgs(drop info) in
```

```
#synth ContractibleSpace (HermitianMat d  $\mathbb{C}$ )
```

```
@[fun_prop]
```

```
theorem inner_continuous : Continuous (Inner.inner  $\mathbb{R}$  (E := HermitianMat d α)) := by
  rw [funext₂ inner_eq_re_trace]
  fun_prop
```

```
@[fun_prop] --fun_prop can actually prove this, should I leave this on or not?
```

```
theorem inner_bilinForm_Continuous (A : HermitianMat d α) : Continuous ↑(HermitianMa
  LinearMap.continuous_of_finiteDimensional _
```

```
end topology
```

```
section innerproductspace
```

```
variable {d d₂ : Type*} [Fintype d] [Fintype d₂] {α : Type*} [RCLike α]
```

```
/-- We define the Hermitian inner product as our "canonical" inner product, which do
This disagrees slightly with Mathlib convention on the `Matrix` type, which avoids a
as there are several reasonable ones; for Hermitian matrices, though, this seem to b
/
```

```
noncomputable def InnerProductCore : InnerProductSpace.Core  $\mathbb{R}$  (HermitianMat d α) :=
{
  inner A B := αA, Bα
  conj_inner_symm := fun x y => by
    simpα using inner_comm y x
  re_inner_nonneg := inner_self_nonneg
  add_left := by simp [inner, add_mul]
  smul_left x y r := by simp
  definite x h := by
    replace h :  $\sum j, \sum i, (\text{RCLike.re } (x \ i \ j) ^ 2 + \text{RCLike.im } (x \ i \ j) ^ 2) = 0 := b$ 
    convert h
    simp only [inner_eq_re_trace, Matrix.trace, Matrix.diag_apply, Matrix.mul_ap
```

```

      RCLike.mul_re, sub_eq_add_neg]
    congr! 2 with i _ j
    simp only [Matrix.conjTranspose_apply, ← congrFun₂ x.H i j]
    simp [pow_two]
  ext i j
  rw [Fintype.sum_eq_zero_iff_of_nonneg (fun i ↦ by positivity)] at h
  replace h := congrFun h j
  rw [Pi.zero_apply, Fintype.sum_eq_zero_iff_of_nonneg (fun i ↦ by posi
  replace h := congrFun h i
  rw [Pi.zero_apply] at h
  rw [add_eq_zero_iff_of_nonneg (by positivity) (by positivity), sq_eq
  apply RCLike.ext (h.left.trans RCLike.zero_re.symm) (h.right.trans (m
}

open Matrix.Norms.Frobenius in
/-- The `HermitianMat` type inherits the Frobenius necessarily, since it's
Hermitian inner product, and in Mathlib an `InnerProductSpace` always carri
norm. -/
noncomputable instance instNormedGroup : NormedAddCommGroup (HermitianMat d
  AddSubgroupClass.normedAddCommGroup _

theorem norm_eq_frobenius (A : HermitianMat d []) :
  ||A|| = (∑ i : d, ∑ j : d, ||A i j|| ^ 2) ^ (1 / 2 : ℝ) := by
  convert ← Matrix.frobenius_norm_def A.mat
  exact Real.rpow_ofNat _ 2

theorem norm_eq_sqrt_inner_self (A : HermitianMat d []) : ||A|| = √(⟨A, A⟩) :=
  rw [norm_eq_frobenius, ← Real.sqrt_eq_rpow]
  congr
  simp_rw [inner_eq_re_trace, Matrix.trace, Matrix.diag, Matrix.mul_apply]
  simp only [map_sum]
  congr! with i _ j _
  simp only [RCLike.norm_sq_eq_def, RCLike.mul_re, sub_eq_add_neg,
    neg_mul_eq_mul_neg]
  congr 2 <=> (rw [← A.H]; simp)

noncomputable instance instNormedSpace : NormedSpace ℝ (HermitianMat d []) w
  norm_smul_le r x := by
  simp [norm_eq_sqrt_inner_self, ← mul_assoc, Real.sqrt_mul',
    inner_self_nonneg, Real.sqrt_mul_self_eq_abs]

noncomputable instance instInnerProductSpace : InnerProductSpace ℝ (Hermiti
  letI : Inner ℝ (HermitianMat d []) := InnerProductCore.toInner;
  letI : NormedSpace ℝ (HermitianMat d []) := instNormedSpace;
  { InnerProductCore with
    norm_sq_eq_re_inner := fun x => by

```

```

    rw [norm_eq_sqrt_inner_self, Real.sq_sqrt (inner_self_nonneg x), RCLike.re_to_
  }

instance : CompleteSpace (HermitianMat d  $\mathbb{R}$ ) :=
  inferInstance

--Shortcut instances
noncomputable instance : NormedAddCommGroup (HermitianMat d  $\mathbb{R}$ ) :=
  inferInstance

noncomputable instance : NormedAddCommGroup (HermitianMat d  $\mathbb{C}$ ) :=
  inferInstance

--PR'ed in #35056
open ComplexOrder in
def _root_.RCLike.instOrderClosed : OrderClosedTopology  $\mathbb{R}$  where
  isClosed_le' := by
    conv => enter [1, 1, p]; rw [RCLike.le_iff_re_im]
    simp_rw [Set.setOf_and]
    refine IsClosed.inter (isClosed_le ?_ ?_) (isClosed_eq ?_ ?_) <=> continuity

scoped[ComplexOrder] attribute [instance] RCLike.instOrderClosed

variable (A B : HermitianMat d  $\mathbb{R}$ )

variable {A B} in
theorem dist_le_of_mem_Icc (x : HermitianMat d  $\mathbb{R}$ ) (hA : A  $\leq$  x) (hB : x  $\leq$  B) :
   $\|x - A\| \leq \|B - A\|$  := by
  classical
  conv => enter [2, 1]; equals (B - x) + (x - A) => abel
  rw [← sq_le_sq (norm_nonneg _) (norm_nonneg _)]
  rw [norm_add_pow_two_real, le_add_iff_nonneg_left]
  suffices  $0 \leq \|B - x\|, x - A$  by positivity
  apply inner_ge_zero <=> rwa [sub_nonneg]

omit [Fintype n] in
theorem Matrix.IsHermitian_isClosed : IsClosed { A : Matrix n n  $\mathbb{R}$  | A.IsHermitian }
  conv =>
    enter [1, 1, A]
    rw [Matrix.IsHermitian, ← sub_eq_zero]
    convert isClosed_singleton.preimage (f := fun (x : Matrix n n  $\mathbb{R}$ ) ↦ (x.conjTranspos
      (by fun_prop) using 1

open ComplexOrder

theorem Matrix.PosSemiDef_isClosed : IsClosed { A : Matrix n n  $\mathbb{R}$  | A.PosSemidef } :=

```

```

rw [show { A : Matrix n n  $\mathbb{R}$  | A.PosSemidef } = { A | A.IsHermitian } n {
  ext A; simp [Matrix.posSemidef_iff_dotProduct_mulVec]]
refine IsHermitian_isClosed.inter ?_
suffices IsClosed ( $\lambda$  x : n  $\rightarrow$   $\mathbb{R}$ , { A : Matrix n n  $\mathbb{R}$  |  $0 \leq \text{star } x \cdot A \cdot \text{mulVec}$ 
  rwa [ $\leftarrow$  Set.setOf_forall] at this
exact isClosed_iInter fun  $\_ \mapsto$  (isClosed_Ici (a := 0)).preimage (by fun_pr

theorem isClosed_nonneg : IsClosed { A : HermitianMat n  $\mathbb{R}$  |  $0 \leq A$  } := by
  simp_rw [zero_le_iff]
  exact Matrix.PosSemiDef_isClosed.preimage_val

--TODO: The PosDef matrices are open *within* the HermitianMat space (not i

instance : OrderClosedTopology (HermitianMat d  $\mathbb{R}$ ) where
  isClosed_le' := by
    classical
    convert IsClosed.preimage (X := (HermitianMat d  $\mathbb{R}$   $\times$  HermitianMat d  $\mathbb{R}$ ))
      (f := fun xy  $\mapsto$  (xy.2 - xy.1)) (by fun_prop) isClosed_nonneg
    ext {x, y}
    simp only [Set.mem_setOf_eq, Set.mem_preimage,  $\leftarrow$  sub_nonneg (b := x)]

/-- Equivalently: the matrices `X` such that `X - A` is PSD and `B - X` is
/
instance : CompactIccSpace (HermitianMat d  $\mathbb{R}$ ) where
  isCompact_Icc := by
    intros A B
    apply Metric.isCompact_of_isClosed_isBounded isClosed_Icc
    rw [Metric.isBounded_iff]
    use 2 * ||B - A||
    rintro x {hxA, hxB} y {hyA, hyB}
    grw [dist_triangle_right (z := A), dist_eq_norm, dist_eq_norm]
    grw [dist_le_of_mem_Icc x hxA hxB, dist_le_of_mem_Icc y hyA hyB]
    rw [two_mul]

variable [DecidableEq d]

/-- The PSD matrices that are ` $\leq 1$ ` are a compact set. More generally, this
but stating that is a bit different because of how numerals are treated. Th
directly matrices, putting in an ` $(a : \mathbb{R}) \cdot 1 \leq m \wedge m \leq (b : \mathbb{R}) \cdot 1$ ` involv
easily from this. More generally ` $A \leq m \wedge m \leq B$ ` is compact.
-/
theorem unitInterval_IsCompact : IsCompact {m : HermitianMat d  $\mathbb{R}$  |  $0 \leq m \wedge$ 
  CompactIccSpace.isCompact_Icc

@[simp]
theorem norm_one : ||(1 : HermitianMat d  $\mathbb{R}$ )|| =  $\sqrt{(\text{Fintype.card } d : \mathbb{R})}$  := by

```

```

    rw [norm_eq_sqrt_real_inner (F := HermitianMat d ℂ)]
    congr 1
    simp [-inner_self_eq_norm_sq_to_K, inner_def]

theorem norm_eq_trace_sq : ||A|| ^ 2 = (A.mat ^ 2).trace := by
  rw [norm_eq_frobenius, ← RCLike.ofReal_pow, ← Real.rpow_two, ← Real.rpow_mul (by p
    simp only [one_div, ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, inv_mul_cancel
    simp only [sq A.mat, map_sum, map_pow, Matrix.trace, Matrix.diag_apply, Matrix.mul
    congr! with i _ j _
    rw [← star_star (A j i), ← A.mat_apply (i := j)]]
    rw [← A.mat.conjTranspose_apply j i, A.H, eq_comm]
    exact RCLike.mul_conj (A.mat i j)

end innerproductspace

--TODO: Cleanup, ew what?
/--
The inner product  $\langle A, B \rangle$  equals  $\sum_{ij} a_i b_j w_{ij}$  where  $a_i, b_j$  are eigenvalues
and  $w_{ij} = \|C_{ij}\|^2$  for  $C = U_A^* U_B$  unitary.
-/
lemma inner_eq_doubly_stochastic_sum {d : Type*} [Fintype d] [DecidableEq d]
  (A B : HermitianMat d ℂ) :
  let C := A.H.eigenvectorUnitary.val.conjTranspose * B.H.eigenvectorUnitary.val
   $\langle A, B \rangle_{\mathbb{R}} = \sum i, \sum j,$ 
  A.H.eigenvalues i * B.H.eigenvalues j * (||C i j||^2) := by
  -- By the properties of the trace and diagonalization, we can rewrite the trace of
  have h_trace_diag : Matrix.trace (A.mat * B.mat) = Matrix.trace ((A.H.eigenvectorU
  have h_trace_diag : Matrix.trace (A.mat * B.mat) = Matrix.trace ((A.H.eigenvecto
    simp [ Matrix.mul_assoc ];
    simp [ ← Matrix.mul_assoc, Matrix.IsHermitian.eigenvectorUnitary ];
    rw [ h_trace_diag, Matrix.trace_mul_comm ];
    simp [ ← mul_assoc, Matrix.IsHermitian.eigenvectorUnitary ];
  -- Since $A$ is Hermitian, its eigenvector matrix is unitary, and thus $(A.H.eigen
  have h_diag_A : (A.H.eigenvectorUnitary : Matrix d d ℂ).conjTranspose * A.mat * (A
    have := A.H.spectral_theorem;
    convert congr_arg ( fun x : Matrix d d ℂ => ( A.H.eigenvectorUnitary : Matrix d
    simp [ ← Matrix.mul_assoc, Matrix.IsHermitian.eigenvectorUnitary ];
  -- Since $B$ is Hermitian, its eigenvector matrix is unitary, and thus $(A.H.eigen
  have h_diag_B : (A.H.eigenvectorUnitary : Matrix d d ℂ).conjTranspose * B.mat * (A
    have h_diag_B : B.mat = (B.H.eigenvectorUnitary : Matrix d d ℂ) * Matrix.diagona
    convert B.H.spectral_theorem using 1;
    grind;
  -- Since $C = U_A^* U_B$ is unitary, we have $C_{ij} = \langle u_i, v_j \rangle$ w
  set C : Matrix d d ℂ := (A.H.eigenvectorUnitary : Matrix d d ℂ).conjTranspose * (B
  have hC_unitary : C * C.conjTranspose = 1 := by
    simp +zetaDelta at *;

```

```

    simp [ Matrix.mul_assoc ];
    simp [ ← Matrix.mul_assoc, Matrix.IsHermitian.eigenvectorUnitary ]
  have hC_norm : ∀ i j, ‖C i j‖ ^ 2 = (C i j) * (star (C i j)) := by
    simp [ Complex.mul_conj, Complex.normSq_eq_norm_sq ]
  have hC_trace : Matrix.trace (Matrix.diagonal (fun i => A.H.eigenvalues i)) =
    simp [ Matrix.trace, Matrix.mul_apply, hC_norm ];
    simp [ Matrix.diagonal, Finset.sum_ite_eq ];
    exact Finset.sum_congr rfl fun _ => Finset.sum_congr rfl fun _ => b
  convert congr_arg Complex.re hC_trace using 1;
  convert congr_arg Complex.re h_trace_diag using 1;
  rw [ h_diag_A, h_diag_B ] ; simp [ Matrix.mul_assoc ] ;
  simp +zetaDelta at *

```

1.459 QuantumInfo/ForMathlib/HermitianMat/Jordan.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Algebra.Jordan.Basic

import QuantumInfo.ForMathlib.HermitianMat.CFC
import QuantumInfo.ForMathlib.HermitianMat.Order

/-!
Hermitian matrices have a Jordan algebra structure given by
`A * B := 2⁻¹ • (A.toMat * B.toMat + B.toMat * A.toMat)`. We call this oper
`HermitianMat.symmMul`, but it's available as `*` multiplication scoped und
`HermMul`. When `A` and `B` commute, this reduces to standard matrix multiplicatio
-/

noncomputable section

section starRing

variable {d : Type*} [Fintype d] [Field α] [StarRing α]
variable (A B : HermitianMat d α)

namespace HermitianMat

def symmMul : HermitianMat d α :=
  ((2 : α)⁻¹ • (A.mat * B.mat + B.mat * A.mat),
    by simp [selfAdjoint, IsSelfAdjoint, add_comm, Matrix.star_eq_conjTrans

```

```

theorem symmMul_comm : A.symmMul B = B.symmMul A := by
  rw [symmMul, symmMul, Subtype.mk.injEq, add_comm]

@[simp]
theorem symmMul_zero : A.symmMul 0 = 0 := by
  simp [symmMul]

@[simp]
theorem zero_symmMul : symmMul 0 A = 0 := by
  simp [symmMul]

theorem symmMul_toMat : (A.symmMul B).mat =
  (2 :  $\square$ )-1 • (A.mat * B.mat + B.mat * A.mat) := by
  rfl

variable [Invertible (2 :  $\square$ )]

variable {A B} in
@[simp]
theorem symmMul_of_commute (hAB : Commute A.mat B.mat) :
  (A.symmMul B).mat = A.mat * B.mat := by
  rw [symmMul_toMat, hAB]
  rw [smul_add, ← add_smul, inv_eq_one_div, ← add_div]
  rw [add_self_div_two, one_smul]

theorem symmMul_self : (symmMul A A).mat = A.mat * A.mat := by
  simp

variable [DecidableEq d]

@[simp]
theorem symmMul_one : A.symmMul 1 = A := by
  ext1; simp

@[simp]
theorem one_symmMul : symmMul 1 A = A := by
  ext1; simp

@[simp]
theorem symmMul_neg_one : A.symmMul (-1) = -A := by
  ext1; simp

@[simp]
theorem neg_one_symmMul : symmMul (-1) A = -A := by
  ext1; simp

```

```

end HermitianMat
end starRing

namespace HermMul

section starRing

variable {d  $\alpha$  : Type*} [Fintype d] [Field  $\alpha$ ] [StarRing  $\alpha$ ]
variable (A B : HermitianMat d  $\alpha$ )

scoped instance : CommMagma (HermitianMat d  $\alpha$ ) where
  mul := HermitianMat.symmMul
  mul_comm := HermitianMat.symmMul_comm

-- --Stupid shortcut that might actually help a lot
-- scoped instance : Mul (HermitianMat d  $\alpha$ ) :=
--   CommMagma.toMul

theorem mul_eq_symmMul : A * B = A.symmMul B := by
  rfl

scoped instance : IsCommJordan (HermitianMat d  $\alpha$ ) where
  lmul_comm_rmul_rmul a b := by
    ext1
    simp only [mul_eq_symmMul, HermitianMat.symmMul_toMat, smul_add,
      mul_add, add_mul, Matrix.mul_smul, Matrix.smul_mul, Matrix.mul_assoc]
    abel

scoped instance : MulZeroClass (HermitianMat d  $\alpha$ ) where
  zero_mul := by simp [mul_eq_symmMul]
  mul_zero := by simp [mul_eq_symmMul]

variable [DecidableEq d] [Invertible (2 :  $\alpha$ )]

scoped instance : MulZeroOneClass (HermitianMat d  $\alpha$ ) where
  one_mul := by simp [mul_eq_symmMul]
  mul_one := by simp [mul_eq_symmMul]

end starRing

section field

variable {d  $\alpha$  : Type*} [Fintype d] [Field  $\alpha$ ] [StarRing  $\alpha$ ]

scoped instance : NonUnitalNonAssocRing (HermitianMat d  $\alpha$ ) where

```



```

left_distrib a b c := by
  ext1
  simp [mul_eq_symmMul, HermitianMat.symmMul_toMat, mul_add, add_mul]
  abel
right_distrib a b c := by
  ext1
  simp [mul_eq_symmMul, HermitianMat.symmMul_toMat, mul_add, add_mul]
  abel

variable [Invertible (2 :  $\alpha$ )] [DecidableEq  $\alpha$ ]

--TODO: Upgrade this to NonAssocCommRing, see #28604 in Mathlib
scoped instance : NonAssocRing (HermitianMat  $\alpha$ ) where

end field

section rlike

variable { $\alpha$  : Type*} [Fintype  $\alpha$ ] [RCLike  $\alpha$ ]

scoped instance : IsScalarTower  $\mathbb{R}$  (HermitianMat  $\alpha$ ) (HermitianMat  $\alpha$ ) where
  smul_assoc r x y := by
    ext : 2
    simp only [smul_eq_mul, mul_eq_symmMul, HermitianMat.symmMul_toMat,
      HermitianMat.mat_smul, smul_add]
    simp

end rlike

end HermMul

```

1.460 QuantumInfo/ForMathlib/HermitianMat/LogExp.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Proj

/-! # Properties of the matrix logarithm and exponential

In particular, operator monotonicity and concavity of the matrix logarithm.
These are proved using `inv_antitone`, so, first showing that the matrix inverse

```

is operator antitone for positive definite matrices.
-/

```
variable {d d₂ [] : Type*}
variable [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂]
variable [RCLike []]
variable {A B : HermitianMat d []} {x : ℝ}
```

noncomputable section

```
theorem Matrix.IsHermitian.log_smul_of_ne_zero {A : Matrix d d []} (hA : A.I
  cfc Real.log (x • A) = (Real.log x) • cfc (if · = 0 then (0 : ℝ) else 1
  have hCFC : cfc (Real.log • (x * ·)) A = cfc Real.log (x • A) := by
    exact cfc_comp_smul x Real.log _ (by fun_prop) hA
  rw [← hCFC, ← cfc_smul, ← cfc_add]
  apply cfc_congr
  intro t ht
  by_cases h : t = 0
  · simp [*]
  · simp [*, Real.log_mul]
```

namespace HermitianMat

section exp

/-- Matrix exponential of a Hermitian matrix, as given by the continuous functional calculus with `Real.exp` -/

```
def exp (A : HermitianMat d []) : HermitianMat d [] :=
  A.cfc Real.exp
```

/-- Primed because `Commute.exp\_left` refers to `NormedSpace.exp` instead of `Real.exp` -/

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem _root_.Commute.exp_left' (hAB : Commute A.mat B.mat) :
  Commute (A.exp).mat B.mat := by
  rw [exp]; commutes
```

/-- Primed because `Commute.exp\_right` refers to `NormedSpace.exp` instead of `Real.exp` -/

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem _root_.Commute.exp_right' (hAB : Commute A.mat B.mat) :
  Commute A.mat (B.exp).mat := by
  rw [exp]; commutes
```

@[simp]

```
theorem reindex_exp (e : d ≈ d₂) : (A.reindex e).exp = A.exp.reindex e :=
```

```

    cfc_reindex A Real.exp e

variable (A) in
instance nonSingular_exp : NonSingular A.exp := by
  exact cfc_nonSingular A Real.exp (fun i ↦ by positivity)

/-- The matrix exponential of a Hermitian matrix is nonnegative. -
/
theorem exp_nonneg (A : HermitianMat d []) : 0 ≤ A.exp := by
  rw [exp, HermitianMat.cfc_nonneg_iff]
  exact fun i ↦ le_of_lt (Real.exp_pos _)

/-- The matrix exponential of a Hermitian matrix is strictly positive (Loewner order)
Requires `Nonempty` since over an empty index type every matrix equals zero and `0 <`
is false. -/
theorem exp_pos [i : Nonempty d] (A : HermitianMat d []) : 0 < A.exp := by
  apply A.exp_nonneg.lt_of_ne'
  intro h
  simp [h] using A.nonSingular_exp.isUnit

open Lean Meta Mathlib.Meta.Positivity in
/-- Positivity extension for `HermitianMat.exp`: always strictly positive if `Nonempty d`
TODO: We could add a fallback to give `nonnegative` if `Nonempty d` is not available
possibly also print a warning. (Users might often not have `Nonempty d` in context,
they probably want to.) -/
@[positivity HermitianMat.exp _]
def evalHermitianMatExp : PositivityExt where eval {u α} _za _pa e := do
  let .app _exp (A : Expr) ← whnfR e | throwError "not HermitianMat.exp"
  pure (.positive (← mkAppM ``HermitianMat.exp_pos #[A]))

end exp

/-- Matrix logarithm (base e) of a Hermitian matrix, as given by the elementwise
real logarithm of the diagonal in a diagonalized form, using `Real.log`

Note that this means that the nullspace of the image includes all of the nullspace of the
original matrix. This contrasts to the standard definition, which is typically defined for
positive *definite* matrices, and the nullspace of the image is exactly the
(λ=1)-eigenspace of the original matrix. (We also get the (λ=-1)-eigenspace here!)

It coincides with a standard definition if A is positive definite. -
/
def log (A : HermitianMat d []) : HermitianMat d [] :=
  A.cfc Real.log

```

```

@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem _root_.Commute.log_left (hAB : Commute A.mat B.mat) :
  Commute (A.log).mat B.mat := by
  rw [log]; commutes

@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem _root_.Commute.log_right (hAB : Commute A.mat B.mat) :
  Commute A.mat (B.log).mat := by
  rw [log]; commutes

@[simp]
theorem reindex_log (e : d ≈ d₂) : (A.reindex e).log = A.log.reindex e :=
  cfc_reindex A Real.log e

@[simp]
theorem log_zero : (0 : HermitianMat d []).log = 0 := by
  simp [log]

@[simp]
theorem log_one : (1 : HermitianMat d []).log = 0 := by
  simp [log]

theorem log_smul_of_pos (A : HermitianMat d []) (hx : x ≠ 0) :
  (x • A).log = Real.log x • A.supportProj + A.log := by
  ext1
  convert A.H.log_smul_of_ne_zero hx
  simp [cfc, log, supportProj_eq_cfc]

theorem log_smul {A : HermitianMat d []} {x : ℝ} (hx : x ≠ 0) [NonSingular A] :
  (x • A).log = Real.log x • 1 + A.log := by
  simp [log_smul_of_pos A hx]

/-
The inverse function is operator antitone for positive definite matrices.
-/
open ComplexOrder in
theorem inv_antitone (hA : A.mat.PosDef) (h : A ≤ B) : B⁻¹ ≤ A⁻¹ := by
  -- Since $B - A$ is positive semidefinite, we can write it as $C^*C$ for
  -- obtain $(C, hC) : \exists C : \text{Matrix } d \ d \ \square, B.\text{mat} - A.\text{mat} = C.\text{conjTranspose} * C$
  -- Using the fact that $B = A + C^*C$, we can write $B^{-1}$ as
  -- $(A + C^*C)^{-1}$.
  have h_inv_posDef : (1 + C * A.mat⁻¹ * C.conjTranspose).PosDef := by
    exact Matrix.PosDef.one.add_posSemidef (hA.inv.posSemidef.mul_mul_conjT
  have hB_inv : B.mat⁻¹ = A.mat⁻¹ - A.mat⁻¹ * C.conjTranspose * (1 + C * A.

```

```

have hB_inv : (A.mat + C.conjTranspose * C)-1 = A.mat-1 - A.mat-1 * C.conjTransp
have hB_inv : (A.mat + C.conjTranspose * C) * (A.mat-1 - A.mat-1 * C.conjTransp
have h_inv : (1 + C * A.mat-1 * C.conjTranspose) * (1 + C * A.mat-1 * C.conj
  exact Matrix.mul_nonsing_inv _ ( show IsUnit _ from by simp [ Matrix.isUn
simp only [mul_assoc, Matrix.mul_sub] at *
simp only [← Matrix.mul_assoc, add_mul, one_mul] at *
simp only [isUnit_iff_ne_zero, ne_eq, hA.det_pos.ne', not_false_eq_true,
  Matrix.mul_nonsing_inv, one_mul, ← add_mul] at *
simp only [mul_assoc, add_mul] at *
simp_all only [← Matrix.mul_assoc, ← eq_sub_iff_add_eq']
grind only [cases eager Subtype]
rw [ Matrix.inv_eq_right_inv hB_inv ];
rw [ ← hB_inv, ← hC, add_sub_cancel ];
-- Since $(1 + C * A-1 * C.conjTranspose)$ is positive definite, its inverse is al
have h_inv_pos : (A.mat-1 * C.conjTranspose * (1 + C * A.mat-1 * C.conjTranspose)-1
have h_inv_pos : (C * A.mat-1).conjTranspose * (1 + C * A.mat-1 * C.conjTranspos
  simp [ Matrix.mul_assoc, Matrix.conjTranspose_mul ];
  rw [ Matrix.conjTranspose_nonsing_inv, A.H ];
  rw [ ← h_inv_pos ];
  exact Matrix.PosSemidef.conjTranspose_mul_mul_same h_inv_posDef.inv.posSemidef _
have h_inv_pos : (A.mat-1 - B.mat-1).PosSemidef := by
  simp_all [ Matrix.PosSemidef ];
exact h_inv_pos

/-
The integral of $1/(1+t) - 1/(x+t)$ from 0 to T is $\log x + \log((1+T)/(x+T))$.
-/
lemma Real.integral_inv_sub_inv_finite (x T : ℝ) (hx : 0 < x) (hT : 0 < T) :
  ∫ t in (0)..T, (1 / (1 + t) - 1 / (x + t)) = Real.log x + Real.log ((1 + T) / (x
  rw [ intervalIntegral.integral_sub, intervalIntegral.integral_comp_add_left, inter
  · rw [ ← Real.log_mul, intervalIntegral.integral_deriv_eq_sub' ];
    field_simp;
  rw [ intervalIntegral.integral_deriv_eq_sub' ];
  any_goals intro t ht; exact Real.differentiableAt_log ( by cases Set.mem_uIcc.mp
  any_goals positivity;
  · rw [ Real.log_div ( by positivity ) ( by positivity ), Real.log_mul ( by posit
    ring;
  · exact funext fun x => by simp [ div_eq_inv_mul ] ;
  · exact continuousOn_of_forall_continuousAt fun t ht => ContinuousAt.div continu
  · exact funext fun x => by simp [ div_eq_inv_mul ] ;
  · exact continuousOn_of_forall_continuousAt fun t ht => ContinuousAt.div continu
  · exact ContinuousOn.intervalIntegrable ( by exact continuousOn_of_forall_continuo
  · exact ContinuousOn.intervalIntegrable ( by exact continuousOn_of_forall_continuo

/--
The limit of $\log((1+T)/(x+T))$ as $T \to \infty$ is 0, for $x > 0$.

```

```

-/
lemma Real.tendsto_log_div_add_atTop (x : ℝ) :
  Filter.Tendsto (fun T => Real.log ((1 + T) / (x + T))) .atTop (nhds 0)
  -- We can divide the numerator and the denominator by $b$ and then take $t$
  suffices h_div : Filter.Tendsto (fun b => Real.log ((1 / b + 1) / (x / b + 1)) .atTop (nhds 0)
  refine h_div.congr' ( by filter_upwards [ Filter.eventually_gt_atTop 0 ]
  exact le_trans ( Filter.Tendsto.log ( Filter.Tendsto.div ( Filter.Tendsto
    .atTop (nhds 0) ) ) ) )

set_option maxHeartbeats 1000000 in
open ComplexOrder MeasureTheory intervalIntegral in
/--
Monotonicity of the finite integral approximation of the logarithm.
-/
theorem logApprox_mono {x y : HermitianMat d} (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  (hxy : x ≤ y) (T : ℝ) (hT : 0 < T) :
  ∫ t in (0)..T, ((1 + t)-1 • (1 : HermitianMat d)) - (x + t • 1)-1 ≤
  ∫ t in (0)..T, ((1 + t)-1 • (1 : HermitianMat d)) - (y + t • 1)-1 :=
  -- By the properties of the integral, we can bring the limit inside, so we
  have h_integrable : ContinuousOn (fun t : ℝ => (1 + t)-1 • (1 : HermitianMat d))
  refine' ( ContinuousOn.smul ( ContinuousOn.inv₀ ( continuousOn_const.adjoint )
  · refine' ContinuousOn.comp ( show ContinuousOn ( fun m : HermitianMat d => m.mat
    · intro m hm;
    refine' ContinuousAt.continuousWithinAt _;
    have h_inv_cont : ContinuousAt (fun m : Matrix d d => m-1) m.mat
    have h_inv_cont : ContinuousAt (fun m : Matrix d d => m-1) m.mat
    have h_det_cont : ContinuousAt (fun m : Matrix d d => m.det)
    exact Continuous.continuousAt ( continuous_id.matrix_det )
    have h_adj_cont : ContinuousAt (fun m : Matrix d d => m.adjugate
    exact Continuous.continuousAt ( continuous_id.matrix_adjugate )
    simp_all [ Matrix.inv_def ];
    exact ContinuousAt.smul ( h_det_cont.inv₀ ( by simp using hm.d ) )
    exact h_inv_cont;
    rw [ ContinuousAt ] at *;
    rw [ tendsto_subtype_rng ] at *;
    exact h_inv_cont.comp ( continuous_subtype_val.tendsto _ );
    · fun_prop;
    · intro t ht;
    simp only [Set.mem_setOf_eq, mat_add, mat_smul, mat_one]
    rw [Matrix.posDef_iff_dotProduct_mulVec] at hx
    refine' ( _, _ );
    · exact H ((fun t => x + t • 1) t);
    · intro v hv_ne_zero
    have h_pos : 0 < star v v • x.mat.mulVec v + t • star v v • (1 : Matrix d d)
    have := hx.2 hv_ne_zero;
    refine' add_pos_of_pos_of_nonneg this _;
    exact mul_nonneg ( mod_cast ht.1 ) ( Finset.sum_nonneg fun i _

```

```

simp_all [ Matrix.add_mulVec ];
simp_all [ Matrix.mulVec, dotProduct ];
simp_all [ Matrix.one_apply, Finset.mul_sum, mul_left_comm,];
convert h_pos using 3 ; simp [ mul_left_comm, Algebra.smul_def ];
· have h_cont : ContinuousOn (fun t : ℝ => (y + t • 1 : Matrix d d [])-1) (Set.Icc 0 T) := by
  have h_inv : ∀ t ∈ Set.Icc 0 T, (y + t • 1 : Matrix d d []).det ≠ 0 := by
    intro t ht;
  have h_det_pos : ∀ t ∈ Set.Icc (0 : ℝ) T, Matrix.PosDef (y.mat + t • 1) :=
    intro t ht;
  rw [Matrix.posDef_iff_dotProduct_mulVec] at hy ⊢
  refine' ( _ , _ );
  · simp [ Matrix.IsHermitian, Matrix.conjTranspose_add, Matrix.conjTranspose_mulVec ];
  · intro x hx_ne_zero
    have h_pos : 0 < star x []v y.mat.mulVec x + t * star x []v x := by
      have := hy.2 hx_ne_zero;
      exact add_pos_of_pos_of_nonneg this ( mul_nonneg ( mod_cast ht.1 ) ( dotProduct x x ) );
    simp_all [ Matrix.add_mulVec ];
    simp_all [ Matrix.mulVec, dotProduct ];
    simp_all [ Matrix.one_apply, Finset.mul_sum, mul_left_comm ];
    convert h_pos using 1;
    simp [ mul_assoc, mul_comm, mul_left_comm, Algebra.smul_def ];
    exact ne_of_gt ( h_det_pos t ht |> fun h => h.det_pos )
  have h_cont_inv : ContinuousOn (fun t : ℝ => (y + t • 1 : Matrix d d [])-1) (Set.Icc 0 T) := by
    have h_cont_det : ContinuousOn (fun t : ℝ => (y + t • 1 : Matrix d d []).det) (Set.Icc 0 T) := by
      fun_prop
    have h_cont_adj : ContinuousOn (fun t : ℝ => (y + t • 1 : Matrix d d []).adj) (Set.Icc 0 T) := by
      fun_prop;
    simp_all [ Matrix.inv_def ];
    exact ContinuousOn.smul ( h_cont_det.inv₀ fun t ht => h_inv t ht.1 ht.2 )
  convert h_cont_inv using 1;
  rw [ continuousOn_iff_continuous_restrict ] at *;
  exact continuous_induced_rng.mpr h_cont
rw [ intervalIntegral.integral_of_le hT.le, intervalIntegral.integral_of_le hT.le ]
apply_rules [ MeasureTheory.integral_mono_ae ];
· exact ContinuousOn.integrableOn_Icc (ContinuousOn.sub h_integrable.1 h_integrable.2)
· exact ContinuousOn.integrableOn_Icc (ContinuousOn.sub h_integrable.1 h_integrable.2)
have h_integral_limit : ∀ t ∈ Set.Icc (0 : ℝ) T, (y + t • 1)-1 ≤ (x + t • 1)-1 := by
  intro t ht;
  apply inv_antitone;
  · rw [Matrix.posDef_iff_dotProduct_mulVec] at hx ⊢
    constructor;
    · simp [ Matrix.IsHermitian];
    · intro v hv_ne_zero
      have h_pos : 0 < star v []v x.mat.mulVec v + t * star v []v v := by
        have := hx.2 hv_ne_zero
        exact add_pos_of_pos_of_nonneg this ( mul_nonneg ( mod_cast ht.1 ) ( dotProduct v v ) );

```

```

      simp_all [ Matrix.add_mulVec ];
      convert h_pos using 2 ; simp [ Matrix.mulVec, dotProduct ];
      simp [ Matrix.one_apply, Finset.mul_sum, mul_left_comm ];
      simp [ mul_left_comm, Algebra.smul_def ];
      · exact add_le_add_left hxy _;
    have h_integral_limit : ∀ t ∈ Set.Ioc 0 T, (1 + t)-1 • 1 - (x + t • 1)-1
      exact fun t ht => sub_le_sub_left ( h_integral_limit t <| Set.Ioc_subse
    filter_upwards [MeasureTheory.ae_restrict_mem measurableSet_Ioc] with t h
    exact h_integral_limit t ht

/-
Definition of the finite integral approximation of the logarithm.
-/
noncomputable def logApprox {n : Type*} [Fintype n] [DecidableEq n] [RCLi
  (x : HermitianMat n) (T : ℝ) : HermitianMat n :=
  ∫ t in (0)..T, ((1 + t)-1 • (1 : HermitianMat n) - (x + t • 1)-1)

/-
Definition of the scalar log approximation and its value.
-/
noncomputable def scalarLogApprox (T : ℝ) (u : ℝ) : ℝ :=
  ∫ t in (0)..T, ((1 + t)-1 - (u + t)-1)

theorem scalarLogApprox_eq (x T : ℝ) (hx : 0 < x) (hT : 0 < T) :
  scalarLogApprox T x = Real.log x + Real.log ((1 + T) / (x + T)) := by
  convert Real.integral_inv_sub_inv_finite x T hx hT using 1;
  unfold scalarLogApprox; norm_num

open ComplexOrder in
/--
The integrand in the log approximation is the CFC of the scalar integrand.
-/
private lemma integrand_eq
  (x : HermitianMat d) (hx : x.mat.PosDef) (t : ℝ) (ht : 0 ≤ t) :
  ((1 + t)-1 • (1 : HermitianMat d) - (x + t • 1)-1) = x.cfc (fun u =>
  have h_cfc_add : x.cfc (fun u => u + t) = x.cfc (fun u => u) + x.cfc (fun
  x.cfc_add id _
  have h_cfc_sub : (x + t • 1)-1 = x.cfc (fun u => (u + t)-1) := by
  convert inv_cfc_eq_cfc_inv (fun u => u + t) ?_ using 1;
  · simp [h_cfc_add]
  · exact fun i => ne_of_gt ( add_pos_of_pos_of_nonneg ( hx.eigenvalues_p
  rw [← cfc_const x (1 + t)-1, h_cfc_sub, ← cfc_sub ];
  rfl

open ComplexOrder MeasureTheory intervalIntegral in
/-

```


The matrix log approximation is the CFC of the scalar log approximation.

-/

```
theorem logApprox_eq_cfc_scalar
  (x : HermitianMat d  $\mathbb{C}$ ) (hx : x.mat.PosDef) (T :  $\mathbb{R}$ ) (hT : 0 < T) :
    logApprox x T = x.cfc (scalarLogApprox T) := by
  unfold scalarLogApprox logApprox;
  rw [intervalIntegral.integral_congr fun t ht => ?_];
  convert integral_cfc_eq_cfc_integral 0 T (fun t u => (1 + t)-1 - (u + t)-1)
  · intro i;
    apply_rules [ContinuousOn.intervalIntegrable];
    field_simp;
    apply_rules [ContinuousOn.sub, ContinuousOn.div, continuousOn_const, continuous]
  · fun_prop;
  · exact fun x hx => by cases Set.mem_uIcc.mp hx <=> linarith;
  · fun_prop;
  · have := hx.eigenvalues_pos i;
    exact fun t ht => ne_of_gt (add_pos_of_pos_of_nonneg this (by cases Set.mem_uIcc.mp ht <=> linarith))
  · apply integrand_eq x hx t
    cases Set.mem_uIcc.mp ht <=> linarith
```

open ComplexOrder in

/--

The log approximation is the log plus an error term.

-/

```
theorem logApprox_eq_log_add_error
  (x : HermitianMat d  $\mathbb{C}$ ) (hx : x.mat.PosDef) (T :  $\mathbb{R}$ ) (hT : 0 < T) :
    logApprox x T = x.log + x.cfc (fun u => Real.log ((1 + T) / (u + T))) := by
  have h_logApprox :  $\int t$  in (0)..T, ((1 + t)-1 • (1 : HermitianMat d  $\mathbb{C}$ ) - (x + t • 1))
  convert logApprox_eq_cfc_scalar x hx T hT using 1;
  apply cfc_congr_of_posDef hx;
  exact fun u hu => Eq.symm (scalarLogApprox_eq u T hu.out hT);
  have h_cfc_add : x.cfc (fun u => Real.log u + Real.log ((1 + T) / (u + T))) = x.cfc (fun u => Real.log ((1 + T) / (u + T)))
  apply cfc_add;
  exact h_logApprox.trans h_cfc_add
```

open ComplexOrder Filter Topology in

open scoped Matrix.Norms.Frobenius in

/--

The error term in the log approximation tends to 0 as T goes to infinity.

-/

```
lemma tendsto_cfc_log_div_add_atTop (x : HermitianMat d  $\mathbb{C}$ ) :
  Tendsto (fun T => x.cfc (fun u => Real.log ((1 + T) / (u + T)))) atTop (nhds 0)
  -- Expand `(cfc x ...).mat` using `cfc_toMat_eq_sum_smul_proj`.
  have h_expand :  $\forall T : \mathbb{R}, (x.cfc (fun u => Real.log ((1 + T) / (u + T))))$ .mat =  $\sum i$ 
    exact fun T => cfc_toMat_eq_sum_smul_proj x fun u => Real.log ((1 + T) / (u + T))
  -- The limit of a sum is the sum of the limits.
```

```

have h_sum : Filter.Tendsto (fun T : ℝ => ∑ i, Real.log ((1 + T) / (x.H.e
  refine' tendsto_finset_sum _ fun i _ => _;
  convert Filter.Tendsto.smul_const ( Real.tendsto_log_div_add_atTop ( x.
  all_goals try infer_instance;
  norm_num +zetaDelta at *
rw [ tendsto_iff_norm_sub_tendsto_zero ] at *;
convert h_sum using 2 ; simp [ ← h_expand]
rfl

open ComplexOrder Filter in
/--
The log approximation converges to the matrix logarithm.
-/
lemma tendsto_logApprox {x : HermitianMat d []} (hx : x.mat.PosDef) :
  Tendsto (fun T => logApprox x T) atTop (nhds x.log) := by
  have h_log_approx_eq : ∀f T in Filter.atTop, x.logApprox T = x.log + x.
    filter_upwards [ Filter.eventually_gt_atTop 0 ] with T hT using logAp
  rw [ Filter.tendsto_congr' h_log_approx_eq ];
  simpa using tendsto_const_nhds.add ( tendsto_cfc_log_div_add_atTop x )

--PULLOUT
open ComplexOrder in
omit [DecidableEq d] [Fintype d] in
theorem posDef_of_posDef_le (hA : A.mat.PosDef) (hAB : A ≤ B) : B.mat.PosDe
  rw [le_iff] at hAB
  convert hA.add_posSemidef hAB
  simp

open ComplexOrder in
/--
The matrix logarithm is operator monotone.
-/
theorem log_mono (hA : A.mat.PosDef) (hAB : A ≤ B) : A.log ≤ B.log := by
  have hB : B.mat.PosDef := posDef_of_posDef_le hA hAB
  apply le_of_tendsto_of_tendsto (tendsto_logApprox hA) (tendsto_logApprox
  rw [Filter.EventuallyLE, Filter.eventually_atTop]
  exact ⟨1, fun T hT => by simpa using logApprox_mono hA hB hAB T ( zero_lt

/-- Monotonicity of exp on commuting operators. -/
theorem le_of_exp_commute (hAB₂ : A.exp ≤ B.exp) :
  A ≤ B := by
  have hA : A = (A.exp).log := by simp [exp, log, ← HermitianMat.cfc_comp]
  have hB : B = (B.exp).log := by simp [exp, log, ← HermitianMat.cfc_comp]
  rw [hA, hB]
  apply HermitianMat.log_mono
  · rw [exp, HermitianMat.cfc_posDef]

```

```

    intro
    positivity
    · exact hAB₂

set_option maxHeartbeats 10000000 in
open ComplexOrder Matrix in
/--
The inverse function is operator convex on positive definite matrices.
-/
lemma inv_convex {x y : HermitianMat d} (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  (a b : ℝ) (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) :
  (a • x + b • y)⁻¹ ≤ a • x⁻¹ + b • y⁻¹ := by
  -- Using the fact that the set of positive semidefinite matrices is a convex cone,
  --  $\begin{pmatrix} a & \bullet & x + b & \bullet & y & I \\ I & a & \bullet & x^{-1} & \bullet & y^{-1} \end{pmatrix}$ 
  -- is positive semidefinite.
  have h_pos_semidef :
    (Matrix.fromBlocks (a • x.mat + b • y.mat) (1 : Matrix d d) (1 : Matrix d d))
    -- Since  $a + b = 1$ , we can use the fact that the block matrix  $\begin{pmatrix} a & \bullet & x + b & \bullet & y & I \\ I & a & \bullet & x^{-1} & \bullet & y^{-1} \end{pmatrix}$ 
    -- is positive semidefinite for any positive definite  $A$ .
    have h_block_pos : ∀ A : Matrix d d, A.PosDef → (Matrix.fromBlocks A 1 1 A⁻¹)
      intro A hA
      have h_block_pos : (Matrix.fromBlocks A (1 : Matrix d d) (1 : Matrix d d))
        have h_inv_pos : A⁻¹.PosSemidef := by
          exact hA.inv.posSemidef
        have h_block_pos : (Matrix.fromBlocks A (1 : Matrix d d) (1 : Matrix d d))
          simp [Matrix.fromBlocks_multiply];
          have := hA.det_pos;
          exact (by rw [Matrix.mul_nonsing_inv] (show IsUnit A.det from isUnit
            have h_block_pos : (Matrix.fromBlocks A 0 0 (A⁻¹ - A⁻¹ * A * A⁻¹)).PosSemi
            have h_block_pos : (Matrix.fromBlocks A 0 0 (A⁻¹ - A⁻¹ * A * A⁻¹)) = (Ma
              have h_inv : A⁻¹ * A = 1 := by
                rw [Matrix.nonsing_inv_mul];
                exact isUnit_iff_ne_zero.mpr hA.det_pos.ne';
              simp [h_inv];
              rw [h_block_pos];
              rw [Matrix.posDef_iff_dotProduct_mulVec] at hA
              rw [Matrix.posSemidef_iff_dotProduct_mulVec]
              constructor;
              · ext i j ; simp [Matrix.fromBlocks];
                cases i <=> cases j <=> simp
                exact hA.1.apply _ _;
              · intro x
                simp [Matrix.mulVec, dotProduct];
                have := @hA.2 (fun i => x (Sum.inl i));
                by_cases h : (fun i => x (Sum.inl i)) = 0

```

```

      · simp_all [ dotProduct, Matrix.mulVec ];
      simp_all [ funext_iff ];
      · exact le_of_lt (this h)
rw [ Matrix.posSemidef_iff_dotProduct_mulVec ] at *;
simp_all [ Matrix.IsHermitian, Matrix.mul_assoc ];
refine' { _, _ };
· simp [Matrix.fromBlocks_conjTranspose, h_inv_pos.1 ];
· intro x
  set y : d ⊕ d → ℝ := (Matrix.fromBlocks 1 A⁻¹ 0 1).mulVec x
  have h_y : star x ⊔_v (Matrix.fromBlocks 1 0 A⁻¹ 1 * (Matrix.fromBlocks
    simp +zetaDelta at *;
    simp [Matrix.dotProduct_mulVec ];
    simp [ Matrix.star_mulVec ];
    congr! 2;
    ext i j ; simp [ Matrix.mul_apply, Matrix.fromBlocks ];
    cases i <;> cases j <;> simp [ Matrix.one_apply];
    · rw [ ← Matrix.ext_iff ] at * ; aesop;
    · rw [ ← Matrix.ext_iff ] at * ; aesop;
    exact h_y.symm ▸ h_block_pos.2 y;
  exact h_block_pos;
  -- Since $a + b = 1$, we can use the fact that the block matrix $\begin{pmatrix} a & b \\ b & a \end{pmatrix}$ is positive semidefinite.
  have h_convex : Matrix.PosSemidef ((a • Matrix.fromBlocks (x.mat) (1
    apply_rules [ Matrix.PosSemidef.add, Matrix.PosSemidef.smul ];
  convert h_convex using 1;
  ext i j ; simp [ Matrix.fromBlocks ];
  rcases i with ( i | i ) <;> rcases j with ( j | j ) <;> simp [ Matrix
    · split_ifs <;> simp_all [ ← add_smul ];
    · split_ifs <;> simp_all [ ← add_smul ];
  have h_schur : (a • x.mat + b • y.mat).PosDef := by
    by_cases ha' : a = 0 <;> by_cases hb' : b = 0 <;> simp_all [ Matrix.pos
    rw [Matrix.posDef_iff_dotProduct_mulVec] at hx hy ⊢
    constructor;
    · simp_all [ Matrix.IsHermitian, Matrix.conjTranspose_add, Matrix.conjT
    · intro v hv_ne_zero
      have h_pos : 0 < a * (star v ⊔_v x.mat.mulVec v) + b * (star v ⊔_v y.mat
        have := hx.2 hv_ne_zero; have := hy.2 hv_ne_zero
        clear hx hy
        simp_all [ Matrix.mulVec, dotProduct ] ;
        exact add_pos_of_nonneg_of_pos ( mul_nonneg ( mod_cast ha ) ( le_of
        convert h_pos using 1 ; simp [ Matrix.add_mulVec]
        ring_nf
        simp [ Matrix.mulVec, dotProduct, Finset.mul_sum, mul_left_comm];
        simp [mul_left_comm, Algebra.smul_def ];
      change (a • (x.mat)⁻¹ + b • (y.mat)⁻¹ - (a • x.mat + b • y.mat)⁻¹).PosSem
      rw [Matrix.posSemidef_iff_dotProduct_mulVec] at h_pos_semidef ⊢

```

```

refine' { _, fun v => _ };
· simp_all [ Matrix.IsHermitian, Matrix.conjTranspose_nonsing_inv ];
· have := h_pos_semidef.2;
  specialize this (Sum.elim (- (a • x.mat + b • y.mat)-1.mulVec v) v);
  simp_all [ Matrix.fromBlocks_mulVec ];
  simp_all [ Matrix.mul_nonsing_inv _ ( show IsUnit (a • x.mat + b • y.mat).det fr
  simp_all [ dotProduct, Matrix.sub_mulVec ];
  refine this.trans_eq (Finset.sum_congr rfl fun _ _ => by ring );

open ComplexOrder in
/--
The shifted inverse function is operator convex on positive definite matrices.
-/
lemma inv_shift_convex {x y : HermitianMat d []} (hx : x.mat.PosDef) (hy : y.mat.PosD
  [] a b : ℝ (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) (t : ℝ) (ht : 0 ≤ t) :
  (a • x + b • y + t • 1)-1 ≤ a • (x + t • 1)-1 + b • (y + t • 1)-1 := by
  have hx' : (x + t • 1).mat.PosDef := hx.add_posSemidef (.smul .one ht)
  have hy' : (y + t • 1).mat.PosDef := hy.add_posSemidef (.smul .one ht)
  convert inv_convex hx' hy' ha hb hab using 1
  ext
  simp [add_assoc, add_left_comm, hab, ← add_smul]

open MeasureTheory intervalIntegral ComplexOrder Matrix in
open scoped Matrix.Norms.Frobenius in
/--
Definition of the approximation of the matrix logarithm.
-/
lemma integrable_inv_shift {A : HermitianMat d []} (hA : A.mat.PosDef) (b : ℝ) (hb :
  IntervalIntegrable (fun t => (A + t • 1)-1) volume 0 b := by
  -- Since A is positive definite, for any t ≥ 0, A + t • 1 is also positive definit
  have h_inv : ∀ t : ℝ, 0 ≤ t → IsUnit (A + t • 1).mat := by
    intro t ht
    have h_pos_def : (A + t • 1).mat.PosDef := by
      convert hA.add_posSemidef _ using 1
      constructor <=> simp [*]
      · simp [ Matrix.IsHermitian ];
      · simp [ Matrix.one_apply, Finsupp.sum ];
      intro x
      refine Finset.sum_nonneg fun i _ => ?_
      split_ifs <=> simp [ *, mul_comm, RCLike.mul_conj ]
      positivity
    exact Matrix.PosDef.isUnit h_pos_def
  have h_inv_cont : ContinuousOn (fun t : ℝ => (A + t • 1).mat-1) (Set.Icc 0 b) := b
    intro t ht
    have h_inv : IsUnit (A + t • 1).mat := h_inv t ht.1
    have h_inv_cont : ContinuousAt (fun t : ℝ => (A + t • 1).mat-1) t := by

```

```

    have h_det_cont : ContinuousAt (fun t : ℝ => (A + t • 1).mat.det) t :
      fun_prop (disch := solve_by_elim)
    have h_adj_cont : ContinuousAt (fun t : ℝ => (A + t • 1).mat.adjugate
      exact Continuous.continuousAt ( by exact Continuous.matrix_adjugate
      simp_all [ Matrix.inv_def ];
      exact ContinuousAt.smul ( h_det_cont.inv₀ <| by simp [ Matrix.isUnit
    exact h_inv_cont.continuousWithinAt
  have h_inv_cont : ContinuousOn (fun t : ℝ => (A + t • 1)⁻¹) (Set.Icc 0 b)
    exact (continuousOn_iff_coe fun t => (A + t • 1)⁻¹).mpr h_inv_cont
  exact h_inv_cont.intervalIntegrable_of_Icc hb

open ComplexOrder in
/--
The finite integral approximation of the matrix logarithm is operator concave
-/
theorem logApprox_concave {n : Type*} [Fintype n] [DecidableEq n] [RCLike
  {x y : HermitianMat n} (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  ∀ a b : ℝ (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) (T : ℝ) (hT : 0 ≤
  a • x.logApprox T + b • y.logApprox T ≤ (a • x + b • y).logApprox T :=
  have h_integrable {z : HermitianMat n} : z.mat.PosDef → IntervalIntegrable
    intro hz
    have h_integrable := integrable_inv_shift hz T hT
    rw [ intervalIntegrable_iff_integrableOn_Ioc_of_le hT ] at *
    refine MeasureTheory.Integrable.sub ?_ h_integrable
    exact ContinuousOn.integrableOn_Icc ( by exact continuousOn_of_forall_c
  have h_int2 : IntervalIntegrable (fun t => (1 + t)⁻¹ • (1 : HermitianMat
    exact h_integrable (Matrix.PosDef.Convex hx hy ha hb hab)
  have h_integral_mono : ∫ t in (0)..T, a • ((1 + t)⁻¹ • (1 : HermitianMat
    have h_integral_mono : ∀ t ∈ Set.Icc 0 T, a • ((1 + t)⁻¹ • (1 : Hermiti
      intros t ht
      have h_inv_shift_convex : ((a • x + b • y) + t • 1)⁻¹ ≤ a • (x + t •
        convert inv_shift_convex hx hy ha hb hab t ht.1 using 1;
      simp_all [smul_sub, ← smul_assoc ];
      rw [ show ( 1 + t ) ⁻¹ • ( 1 : HermitianMat n ) = ( a * ( 1 + t ) -
        convert sub_le_sub_left h_inv_shift_convex _ using 1 ; abel_nf;
      rw [ intervalIntegral.integral_of_le hT, intervalIntegral.integral_of_l
      apply MeasureTheory.integral_mono_ae
      · exact ( (h_integrable hx).1.smul a |> fun h => h.add ( (h_integrable
      · exact h_int2.1.mono_set (Set.Ioc_subset_Ioc le_rfl le_rfl)
      · filter_upwards [ MeasureTheory.ae_restrict_mem measurableSet_Ioc ] wi
    convert h_integral_mono using 1;
    rw [ intervalIntegral.integral_add ( by exact (h_integrable hx).smul a )
    rw [logApprox, logApprox]

open ComplexOrder in
/-
```

The matrix logarithm is operator concave.

-/

```
theorem log_concave {x y : HermitianMat d []} (hx : x.mat.PosDef) (hy : y.mat.PosDef)
  (a b : ℝ) (ha : 0 ≤ a) (hb : 0 ≤ b) (hab : a + b = 1) :
  a • x.log + b • y.log ≤ (a • x + b • y).log := by
  apply le_of_tendsto_of_tendsto (b := .atTop) (f := fun T => a • x.logApprox T + b • y.logApprox T)
  · exact ((tendsto_const_nhds.smul (tendsto_logApprox hx)).add (tendsto_const_nhds.smul (tendsto_logApprox hy)))
  · apply tendsto_logApprox
  exact Matrix.PosDef.Convex hx hy ha hb hab
  · rw [Filter.EventuallyLE, Filter.eventually_atTop]
  exact {0, logApprox_concave hx hy ha hb hab}
```

/-

The logarithm of the Kronecker product of two diagonal Hermitian matrices is the sum of the logarithms of the matrices.

-/

```
lemma log_kron_diagonal {m n [] : Type*} [Fintype m] [DecidableEq m] [Fintype n] [DecidableEq n]
  {d1 : m → ℝ} {d2 : n → ℝ} (h1 : ∀ i, 0 < d1 i) (h2 : ∀ j, 0 < d2 j) :
  (diagonal [] d1 ⊗_k diagonal [] d2).log =
  (diagonal [] d1).log ⊗_k 1 + 1 ⊗_k (diagonal [] d2).log := by
  have h_eq : (diagonal [] d1 ⊗_k diagonal [] d2) = (diagonal [] (fun (i : m × n) => d1 i.1 * d2 i.2))
  exact kronecker_diagonal d1 d2
  convert congr_arg _ h_eq using 1;
  -- By definition of logarithm, we can rewrite the right-hand side.
  have h_rhs : (diagonal [] (fun (i : m × n) => d1 i.1 * d2 i.2)).log =
    (diagonal [] (fun (i : m × n) => Real.log (d1 i.1) + Real.log (d2 i.2))) := by
    rw [log, cfc_diagonal];
    exact congr_arg _ (funext fun i => Real.log_mul (ne_of_gt (h1 i.1)) (ne_of_gt (h2 i.2)))
  rw [h_rhs];
  have h_rhs : (diagonal [] (fun (i : m × n) => Real.log (d1 i.1) + Real.log (d2 i.2))) =
    (diagonal [] (fun (i : m × n) => Real.log (d1 i.1))) + (diagonal [] (fun (i : m × n) => Real.log (d2 i.2)))
  ext1
  simp [diagonal]
  rw [h_rhs];
  congr! 1;
  · rw [show (diagonal [] d1 |> log) = diagonal [] (Real.log ∘ d1) from ?_];
  · rw [← diagonal_one, kronecker_diagonal]
  simp
  · exact cfc_diagonal Real.log d1
  · have h_rhs : (diagonal [] d2).log = (diagonal [] (fun (i : n) => Real.log (d2 i)))
  exact cfc_diagonal Real.log d2
  rw [h_rhs];
  convert kronecker_diagonal 1 (fun i => Real.log (d2 i)) using 1;
  all_goals try infer_instance;
  · exact congr_arg2 _ (diagonal_one.symm) rfl;
  · simp
```

```

/--
The logarithm of a Hermitian matrix conjugated by a unitary matrix is the c
-/
lemma log_conj_unitary (A : HermitianMat d []) (U : Matrix.unitaryGroup d []) :
  (A.conj U.val).log = A.log.conj U.val :=
  cfc_conj_unitary _ Real.log U

open RealInnerProductSpace in
theorem inner_log_smul_of [NonSingular A] {x : ℝ} (hx : x ≠ 0) :
  [(x • A).log, B] = Real.log x * B.trace + [A.log, B] := by
  simp [log_smul hx, inner_add_left]

section kron

lemma log_kron_diagonal_with_proj {f : d → ℝ} {g : d₂ → ℝ} :
  (diagonal [] f ⊗ₖ diagonal [] g).log =
  (diagonal [] f).log ⊗ₖ (diagonal [] g).supportProj +
  (diagonal [] f).supportProj ⊗ₖ (diagonal [] g).log := by
  have h_diag_kron : (diagonal [] f ⊗ₖ diagonal [] g).log = diagonal [] (fun i
    rw [kronecker_diagonal, log]
    exact cfc_diagonal _ _
  simp_all [HermitianMat.ext_iff, cfc_diagonal, log, supportProj_eq_cfc ];
  ext {i, j} {i', j'}
  by_cases hi' : i = i'; swap
  · simp [hi']
  by_cases hj' : j = j'; swap
  · simp [hj']
  simp [hi', hj']
  split_ifs <;> simp_all [Real.log_mul]

variable {B : HermitianMat d₂ []}

/--
Generalization of `HermitianMat.log_kron` for possibly singular matrices.
-/
lemma log_kron_with_proj : (A ⊗ₖ B).log = A.log ⊗ₖ B.supportProj + A.supportProj ⊗ₖ B.log
  obtain ⟨UA, DA, rfl⟩ : ∃ UA : Matrix.unitaryGroup d [], ∃ DA, A = (diagonal [] DA,
    ⟨_, _, eq_conj_diagonal A⟩)
  obtain ⟨UB, DB, rfl⟩ : ∃ UB : Matrix.unitaryGroup d₂ [], ∃ DB, B = (diagonal [] DB,
    ⟨_, _, eq_conj_diagonal B⟩)
  rw [← kronecker_conj, log_conj_unitary _ ⟨_, Matrix.kronecker_mem_unitary _ _⟩]
  rw [log_kron_diagonal_with_proj, map_add (conj _)]
  congr 1
  <;> rw [supportProj_eq_cfc, supportProj_eq_cfc, cfc_conj_unitary, log_conj_unitary]

```


1.461.

QUANTUMINFO/FORMATHLIB/HERMITIANMAT/NONSINGULAR.LEAN 627

/--

The matrix logarithm of the Kronecker product of two nonsingular Hermitian matrices
the sum of the Kronecker products of their logarithms with the identity matrix.

-/

theorem log_kron [NonSingular A] [NonSingular B] : (A $\otimes_k$ B).log = A.log $\otimes_k$ 1 + 1 $\otimes_k$
simp [log_kron_with_proj]

end kron

1.461 QuantumInfo/ForMathlib/HermitianMat/NonSingular.lean

/-

Copyright (c) 2025 Alex Meiburg. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Alex Meiburg

-/

import QuantumInfo.ForMathlib.HermitianMat.Order

import QuantumInfo.ForMathlib.Isometry

noncomputable section

attribute [instance] invertibleOne

namespace Matrix

variable {d R S F α : Type*} [Fintype d] [DecidableEq d]

variable [CommSemiring R] [Semiring S] [Algebra R S] [Field F] [RCLike α]

theorem isUnit_smul {c : R} (hA : IsUnit c) {M : Matrix d d S} (hM : IsUnit M) :
IsUnit (c • M : Matrix d d S) := by
obtain ⟨d, rfl⟩ := hA
obtain ⟨M', rfl⟩ := hM
use d • M'
rfl

theorem isUnit_natCast {n : $\mathbb{N}$ } (hn : n $\neq$ 0) [CharZero F] : IsUnit (n : Matrix d d F)
exact (IsUnit.mk0 (n : F) (mod_cast hn)).map (algebraMap F _)

theorem isUnit_real_smul {r : $\mathbb{R}$ } (hr : r $\neq$ 0) {M : Matrix d d α } (hM : IsUnit M) :
IsUnit (r • M : Matrix d d α) :=
isUnit_smul hr.isUnit hM

theorem isUnit_real_cast {r : $\mathbb{R}$ } (hr : r $\neq$ 0) : IsUnit (r • 1 : Matrix d d α) := by
exact isUnit_real_smul hr isUnit_one

```
end Matrix
```

```
namespace HermitianMat
```

```
variable {n m R  $\alpha$  : Type*} [Fintype n] [DecidableEq n] [Fintype m] [DecidableEq m]
```

```
variable [CommRing R] [StarRing R] [RCLike  $\alpha$ ]
```

```
variable (A : HermitianMat n R) (B : HermitianMat m R)
```

```
class NonSingular (A : HermitianMat n R) : Prop where
  isUnit : IsUnit A.mat
```

```
@[simp]
```

```
theorem isUnit_mat_of_nonSingular [NonSingular A] : IsUnit A.mat :=
  NonSingular.isUnit
```

```
theorem nonsingular_iff_isUnit : NonSingular A  $\leftrightarrow$  IsUnit A.mat := by
  exact Iff.intro (fun h  $\mapsto$  h.isUnit) NonSingular.mk
```

```
instance instHasInv_of_invertible [i : Invertible A.mat] : NonSingular A :=
  {isUnit_of_invertible _}
```

```
instance instInvertible_of_hasInv [h : NonSingular A] : Invertible A.mat :=
  h.isUnit.invertible
```

```
instance : NonSingular (1 : HermitianMat n R) :=
  instHasInv_of_invertible (i := invertibleOne)
```

```
variable {A : HermitianMat n  $\alpha$ } {B : HermitianMat m  $\alpha$ } {C : Matrix n n  $\alpha$ }
open ComplexOrder
```

```
theorem nonSingular_of_posDef (hA : A.mat.PosDef) : NonSingular A :=
  {hA.isUnit}
```

```
theorem nonSingular_iff_posDef_of_PSD (hA :  $0 \leq A$ ) :
  NonSingular A  $\leftrightarrow$  A.mat.PosDef := by
  refine {fun h  $\mapsto$  ?_, fun hA2  $\mapsto$  nonSingular_of_posDef hA2}
  have : IsUnit A.mat := h.isUnit
  grind [zero_le_iff]
```

```
theorem nonSingular_smul [i : NonSingular A] {c :  $\mathbb{R}$ } (hc : IsUnit c) : NonSingular (Matrix.isUnit_smul hc i.isUnit)
```

```
theorem nonSingular_iff_zero_notMem_spectrum : NonSingular A  $\leftrightarrow$   $0 \notin$  spectrum A
simp [nonsingular_iff_isUnit, spectrum, resolventSet]
```

1.461.

QUANTUMINFO/FORMATHLIB/HERMITIANMAT/NONSINGULAR.LEAN 629

```
theorem nonSingular_iff_eigenvalue_ne_zero : NonSingular A  $\leftrightarrow$   $\forall i$ , A.H.eigenvalues i  $\neq 0$  := by
  simp [nonSingular_iff_zero_notMem_spectrum, A.H.spectrum_real_eq_range_eigenvalues]
```

```
theorem nonSingular_iff_det_ne_zero : NonSingular A  $\leftrightarrow$  A.mat.det  $\neq 0$  := by
  simp [Matrix.isUnit_iff_isUnit_det, nonsingular_iff_isUnit]
```

```
theorem nonSingular_iff_ker_bot : NonSingular A  $\leftrightarrow$  A.ker =  $\perp$  := by
  rw [nonSingular_iff_det_ne_zero];
  have h_det_nonzero_to_ker_trivial : A.mat.det  $\neq 0 \rightarrow$  A.ker =  $\perp$  := by
    intro h_det_nonzero
    rw [Submodule.eq_bot_iff]
    have h_inv : Invertible A.mat := by
      convert Matrix.invertibleOfDetInvertible A.mat
      exact invertibleOfNonzero h_det_nonzero
    have h_ker_trivial (x : n  $\rightarrow$   $\square$ ) (hx : A.mat.mulVec x = 0) : x = 0 := by
      simpa using congr_arg (h_inv.1.mulVec) hx
    intro x hx
    rw [mem_ker_iff_mulVec_zero] at hx
    exact PiLp.ext (fun i  $\Rightarrow$  congr_fun (h_ker_trivial x.ofLp hx) i)
  refine (h_det_nonzero_to_ker_trivial, fun h h'  $\Rightarrow$  ?_)
  obtain (x, hx) := Matrix.exists_mulVec_eq_zero_iff.mpr h'
  have h_inj : Function.Injective (Matrix.mulVecLin A.mat) := by
    rw [ $\leftarrow$  LinearMap.ker_eq_bot, Submodule.eq_bot_iff]
    intro y hy
    rw [LinearMap.mem_ker, Matrix.mulVecLin_apply] at hy
    have hm : (WithLp.toLp 2 y)  $\in$  A.ker := (mem_ker_iff_mulVec_zero A _).mpr hy
    rw [h] at hm
    have := (Submodule.mem_bot (R :=  $\square$ )).mp hm
    simpa using this
  specialize @h_inj x 0
  simp_all
```

```
theorem nonSingular_iff_support_top : NonSingular A  $\leftrightarrow$  A.support =  $\top$  := by
  simp only [support, Submodule.eq_top_iff']
  refine (fun hA  $\mapsto$  ?_, fun hA  $\mapsto$  ?_)
  · intro x
    have hA_inv : IsUnit A.mat := hA.isUnit
    rcases hA_inv.exists_right_inv with (y, hy)
    exact (WithLp.toLp 2 (y.mulVec x.ofLp), by
      apply PiLp.ext; intro i
      simp [lin, Matrix.toLpLin_apply, Matrix.mulVec_mulVec, hy])
  · constructor
    have : Function.Surjective A.mat.mulVec := by
      intro y
      obtain (x, hx) := hA (WithLp.toLp 2 y)
      exact (x.ofLp, by
```

```

      have := congr_arg WithLp.ofLp hx
      simp [lin, Matrix.toLpLin_apply] at this
      exact this)
    exact Matrix.mulVec_surjective_iff_isUnit.mp this

@[simp]
theorem nonSingular_iff_neg : NonSingular (-A) ↔ NonSingular A := by
  simp [nonSingular_iff_det_ne_zero, Matrix.det_neg]

@[simp]
theorem nonSingular_iff_inv : NonSingular (A-1) ↔ NonSingular A := by
  rw [nonsingular_iff_isUnit, nonsingular_iff_isUnit]
  exact Matrix.isUnit_nonsing_inv_iff

@[simp]
theorem nonSingular_iff_kronecker [Nonempty n] [Nonempty m] :
  NonSingular (A ⊗k B) ↔ NonSingular A ∧ NonSingular B := by
  simp [nonSingular_iff_det_ne_zero, Matrix.det_kronecker]

theorem nonSingular_iff_conj (hC : IsUnit C) : NonSingular (A.conj C) ↔ Non
  simp_all [Matrix.isUnit_iff_isUnit_det, nonsingular_iff_isUnit]

@[simp]
theorem nonSingular_iff_reindex (e : n ≈ m) : NonSingular (A.reindex e) ↔ N
  rw [nonSingular_iff_det_ne_zero, nonSingular_iff_det_ne_zero]
  rw [reindex_eq_conj, ← Matrix.det_reindex_self e, conj_apply_mat]
  congr! 2
  ext : 2
  simp [Matrix.mul_apply, Matrix.one_apply]

section fwd

theorem nonSingular_empty [IsEmpty n] : NonSingular A := by
  rw [Subsingleton.eq_one A]
  infer_instance

variable [NonSingular A] [NonSingular B]

theorem nonSingular_det_ne_zero : A.mat.det ≠ 0 := by
  rwa [← nonSingular_iff_det_ne_zero]

@[simp]
theorem nonSingular_ker_bot : A.ker = ⊥ := by
  rwa [← nonSingular_iff_ker_bot]

@[simp]

```

```

theorem nonSingular_support_top : A.support = T := by
  rwa [← nonSingular_iff_support_top]

instance nonSingular_neg : NonSingular (-A) := by
  rwa [nonSingular_iff_neg]

instance nonSingular_inv : NonSingular (A-1) := by
  rwa [nonSingular_iff_inv]

instance nonSingular_kron [Nonempty n] [Nonempty m] : NonSingular (A ⊗k B) :=
  nonSingular_iff_kronecker.mpr (inferInstance, inferInstance)

theorem nonSingular_conj (hC : IsUnit C) : NonSingular (A.conj C) := by
  rwa [nonSingular_iff_conj hC]

instance nonSingular_conj_isometry {B : HermitianMat n []} [NonSingular B] :
  NonSingular (A.conj B.mat) := by
  simpa [nonSingular_iff_conj]

theorem nonSingular_zero_notMem_spectrum : 0 ∉ spectrum ℝ A.mat := by
  rwa [← nonSingular_iff_zero_notMem_spectrum]

theorem nonSingular_eigenvalue_ne_zero : ∀ i, A.H.eigenvalues i ≠ 0 := by
  rwa [← nonSingular_iff_eigenvalue_ne_zero]

end fwd

end HermitianMat

```

1.462 QuantumInfo/ForMathlib/HermitianMat/Order.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Trace

namespace HermitianMat

open ComplexOrder
open scoped Matrix

variable {α : Type*} [RCLike α]

```

```

variable {n m ι : Type*} [Fintype n] [Fintype m] [Fintype ι]
variable {A B C : HermitianMat n []}
variable {M : Matrix m n []} {N : Matrix n n []}

open MatrixOrder in
/-- The `MatrixOrder` instance for Matrix (the Loewner order) we keep open
HermitianMat, always. -/
instance : PartialOrder (HermitianMat n []) :=
  inferInstanceAs (PartialOrder (selfAdjoint _))

open MatrixOrder in
instance : IsOrderedAddMonoid (HermitianMat n []) :=
  inferInstanceAs (IsOrderedAddMonoid (selfAdjoint _))

omit [Fintype n] in
theorem le_iff :  $A \leq B \leftrightarrow (B - A).mat.PosSemidef$  := by
  rfl

omit [Fintype n] in
theorem zero_le_iff :  $0 \leq A \leftrightarrow A.mat.PosSemidef$  := by
  rw [le_iff, sub_zero]

theorem le_iff_mulVec_le :  $A \leq B \leftrightarrow$ 
   $\forall x, \text{star } x \sqsubseteq_v A.mat * _v x \leq \text{star } x \sqsubseteq_v B.mat * _v x$  := by
  simp [le_iff, Matrix.posSemidef_iff_dotProduct_mulVec, B.H.sub A.H, Matrix]

instance [DecidableEq n] : ZeroLEOneClass (HermitianMat n []) where
  zero_le_one := by
    rw [zero_le_iff]
    exact Matrix.PosSemidef.one

omit [Fintype n] in
theorem lt_iff_posdef :  $A < B \leftrightarrow (B - A).mat.PosSemidef \wedge A \neq B$  :=
  lt_iff_le_and_ne

instance : IsStrictOrderedModule  $\mathbb{R}$  (HermitianMat n []) where
  smul_lt_smul_of_pos_left a ha b b2 hb := by
    rw [HermitianMat.lt_iff_posdef] at hb ⊢
    simp only [← smul_sub, ne_eq, smul_right_inj ha.ne']
    exact ⟨hb.left.smul ha.le, hb.right⟩
  smul_lt_smul_of_pos_right a ha b b2 hb := by
    rw [HermitianMat.lt_iff_posdef] at ha ⊢
    rw [sub_zero] at ha
    rw [← sub_pos] at hb
    convert And.intro (ha.left.smul hb.le) ha.right using 1
    · simp [← sub_smul]

```

```

simp only [ne_eq, not_iff_not]
constructor
· intro h
  rw [eq_comm, ← sub_eq_zero, ← sub_smul] at h
  simp [eq_comm, hb.ne'] using h
· rintro rfl; simp

theorem posSemidef_iff_spectrum_Ici [DecidableEq n] (A : HermitianMat n) :
   $0 \leq A \leftrightarrow \text{spectrum } \mathbb{R} A.\text{mat} \subseteq \text{Set.Ici } 0$  := by
  rw [zero_le_iff, Matrix.posSemidef_iff_isHermitian_and_spectrum_nonneg]
  simp [A.H, Set.Ici.eq_1]

theorem posSemidef_iff_spectrum_nonneg [DecidableEq n] (A : HermitianMat n) :
   $0 \leq A \leftrightarrow \forall x \in \text{spectrum } \mathbb{R} A.\text{mat}, 0 \leq x$  := by
  exact A.posSemidef_iff_spectrum_Ici

theorem trace_nonneg (hA :  $0 \leq A$ ) :  $0 \leq A.\text{trace}$  := by
  exact (RCLike.nonneg_iff.mp (zero_le_iff.mp hA).trace_nonneg).1

theorem trace_pos (hA :  $0 < A$ ) :  $0 < A.\text{trace}$  := by
  open ComplexOrder in
  have hA' := hA.le
  rw [HermitianMat.zero_le_iff] at hA'
  have h_pos := Matrix.PosSemidef.trace_pos hA' (by simp [HermitianMat.ext_iff] usi
  rw [HermitianMat.trace_eq_re_trace]
  rw [RCLike.pos_iff] at h_pos
  exact h_pos.left

open Lean Meta Mathlib.Meta.Positivity in
/-- Positivity extension for `HermitianMat.trace`: nonneg when the matrix is nonneg,
positive when the matrix is positive. -/
@[positivity HermitianMat.trace _]
def evalHermitianMatTrace : PositivityExt where eval {u _α} _zα _pα e := do
  let .app_tr (A : Expr) ← whnfR e | throwError "not HermitianMat.trace"
  let (isStrict, pfA) ← bestResult A
  if isStrict then
    pure (.positive (← mkAppM ``HermitianMat.trace_pos #[pfA]))
  else
    pure (.nonnegative (← mkAppM ``HermitianMat.trace_nonneg #[pfA]))

--Without these shortcut instances, `gcongr` fails to close certain goals...? Why? T
instance : PosSMulMono  $\mathbb{R}$  (HermitianMat n) := inferInstance
instance : SMulPosMono  $\mathbb{R}$  (HermitianMat n) := inferInstance

--Without explicitly giving this instance, Lean times out trying to find it sometime
instance : PosSMulReflectLE  $\mathbb{R}$  (HermitianMat n) :=

```

```

PosSMulMono.toPosSMulReflectLE

theorem le_trace_smul_one [DecidableEq n] (hA : 0 ≤ A) : A ≤ A.trace • 1 :=
  have hA' : A.mat.PosSemidef := zero_le_iff.mp hA
  refine (Matrix.PosSemidef.le_smul_one_of_eigenvalues_iff hA'.1 A.trace).m
  rw [← A.sum_eigenvalues_eq_trace]
  intro i
  exact Finset.single_le_sum (fun j _ ↦ hA'.eigenvalues_nonneg j) (Finset.m

/-- The Kronecker product of two nonnegative Hermitian matrices is nonnegat
/
theorem kronecker_nonneg {A : HermitianMat m []} (hA : 0 ≤ A) (hB : 0 ≤ B) :
  rw [zero_le_iff, kronecker_mat]
  classical exact (zero_le_iff.mp hA).PosSemidef_kronecker (zero_le_iff.mp

/-- The Kronecker product of two positive Hermitian matrices is positive. -
/
theorem kronecker_pos {A : HermitianMat m []} (hA : 0 < A) (hB : 0 < B) : 0
  apply lt_of_le_of_ne (kronecker_nonneg hA.le hB.le)
  intro h
  replace h := congr(trace $h)
  simp only [trace_zero, trace_kronecker, zero_eq_mul] at h
  apply trace_pos at hA
  apply trace_pos at hB
  grind only [cases Or]

omit [Fintype n] in
open MatrixOrder in
theorem posSemidef_to_nonneg {A : Matrix n n []} (hA : A.PosSemidef) : 0 ≤ A
  exact hA.nonneg

open MatrixOrder in
theorem posDef_to_pos {A : Matrix n n []} (hA : A.PosDef) [Nonempty n] : 0 <
  apply lt_of_le_of_ne hA.posSemidef.nonneg
  rintro rfl
  classical simp [Matrix.det_zero] using hA.det_pos

open Lean Meta in
/-- Given an expression `e` (a matrix) and a proof expression `p` whose type
`Matrix.PosSemidef A`, `Matrix.PosDef A`, or `And P Q` (syntactically), att
find a proof of nonnegativity or positivity for `e`. Only syntactic matchin
head constant is used; `isDefEq` is used only to compare the matrix argumen
/
private partial def findMatrixPSDInExpr (e : Expr) (p : Expr) (ty : Expr) :
  MetaM (Option (Bool × Expr)) := do
  let head := ty.getAppFn

```



```

if head.isConst then
  let name := head.constName!
  if name == ``Matrix.PosSemidef then
    -- Last argument is the matrix
    let args := ty.getAppArgs
    let A := args.back!
    if ← isDefEq A e then
      let pf ← mkAppM ``HermitianMat.posSemidef_to_nonneg #[p]
      return some (false, pf)
  if name == ``Matrix.PosDef then
    let args := ty.getAppArgs
    let A := args.back!
    if ← isDefEq A e then
      -- Try strict (needs Nonempty n); extract the index type from PosDef args
      -- PosDef args: [n, R, Fintype n, Ring R, PartialOrder R, StarRing R, A]
      let nType := args[0]!
      let nonemptyType ← mkAppM ``Nonempty #[nType]
      match ← try? (synthInstance nonemptyType) with
      | some nonemptyInst =>
        -- posDef_to_pos : {[]} → [RLike []] → {n} → [Fintype n] → {A} → (hA : A.PosDef) → A.PosSemidef
        let pf ← mkAppOptM ``HermitianMat.posDef_to_pos #[none, none, none, none, none, none, nonemptyInst]
        return some (true, pf)
      | none =>
        let pSemidef ← mkAppM ``Matrix.PosDef.posSemidef #[p]
        let pf ← mkAppM ``HermitianMat.posSemidef_to_nonneg #[pSemidef]
        return some (false, pf)
  if name == ``And then
    let args := ty.getAppArgs
    if args.size == 2 then
      -- Recurse on left and right
      let pLeft ← mkAppM ``And.left #[p]
      if let some result ← findMatrixPSDInExpr e pLeft args[0]! then
        return some result
      let pRight ← mkAppM ``And.right #[p]
      if let some result ← findMatrixPSDInExpr e pRight args[1]! then
        return some result
    return none

open Lean Meta Mathlib.Meta.Positivity in
/-- Positivity extension for `Matrix`: looks for `A.PosSemidef` or `A.PosDef` in the
local context (including syntactic `And` conjunctions) to prove ` $\theta \leq A$ ` or ` $\theta < A$ `.
/
@[positivity (_ : HermitianMat _)]
def evalMatrixPSD : PositivityExt where eval {u _α} _zα _pα e := do
  let lctx ← getLCtx
  let mut best : Strictness _zα _pα e := .none

```

```

for ldecl in lctx do
  if ldecl.isImplementationDetail then continue
  let ty := ldecl.type
  let p : Expr := .fvar ldecl.fvarId
  if let some (isStrict, pf) ← findMatrixPSDInExpr e p ty then
    if isStrict then
      return .positive pf
    else
      best := .nonnegative pf
  match best with
  | .none => throwError "evalMatrixPSD: no PosSemidef or PosDef hypothesis"
  | other => return other

omit [Fintype n] in
theorem mat_posSemidef_to_nonneg (hA : A.mat.PosSemidef) : 0 ≤ A :=
  zero_le_iff.mpr hA

theorem mat_posDef_to_pos [Nonempty n] (hA : A.mat.PosDef) : 0 < A := by
  exact posDef_to_pos hA

open Lean Meta in
/-- Given an expression `e` (a `HermitianMat`) and a proof expression `p` with
`Matrix.PosSemidef A.mat`, `Matrix.PosDef A.mat`, or `And P Q` (syntactical)
find a proof of nonnegativity or positivity for `e`. Only syntactic matching
head constant is used; `isDefEq` is used only to compare the `HermitianMat`
/
private partial def findHermitianMatPSDInExpr (e : Expr) (p : Expr) (ty : Expr) :
  MetaM (Option (Bool × Expr)) := do
  let head := ty.getAppFn
  if head.isConst then
    let name := head.constName!
    if name == ``Matrix.PosSemidef || name == ``Matrix.PosDef then
      -- Last argument should be `A.mat` i.e. `HermitianMat.mat A`
      let args := ty.getAppArgs
      let matExpr := args.back!
      -- Check if matExpr is `HermitianMat.mat A` (or equivalently `Subtype
      let matHead := matExpr.getAppFn
      if matHead.isConst && (matHead.constName! == ``HermitianMat.mat ||
        matHead.constName! == ``Subtype.val) then
        let matArgs := matExpr.getAppArgs
        let A := matArgs.back!
        if ← isDefEq A e then
          if name == ``Matrix.PosSemidef then
            let pf ← mkAppM ``HermitianMat.mat_posSemidef_to_nonneg #[p]
            return some (false, pf)

```

```

else
  -- PosDef: try strict (needs Nonempty n)
  let psdArgs := ty.getAppArgs
  let nType := psdArgs[0]!
  let nonemptyType ← mkAppM ``Nonempty #[nType]
  match ← try? (synthInstance nonemptyType) with
  | some nonemptyInst =>
    -- mat_posDef_to_pos : {[]} → [RCLike []] → {n} → [Fintype n] → {A} → [N
    let pf ← mkAppOptM ``HermitianMat.mat_posDef_to_pos #[none, none, none
    return some (true, pf)
  | none =>
    let pSemidef ← mkAppM ``Matrix.PosDef.posSemidef #[p]
    let pf ← mkAppM ``HermitianMat.mat_posSemidef_to_nonneg #[pSemidef]
    return some (false, pf)
if name == ``And then
  let args := ty.getAppArgs
  if args.size == 2 then
    let pLeft ← mkAppM ``And.left #[p]
    if let some result ← findHermitianMatPSDInExpr e pLeft args[0]! then
      return some result
    let pRight ← mkAppM ``And.right #[p]
    if let some result ← findHermitianMatPSDInExpr e pRight args[1]! then
      return some result
return none

open Lean Meta Mathlib.Meta.Positivity in
/-- Positivity extension for `HermitianMat`: looks for `A.mat.PosSemidef` or `A.mat.
the local context (including syntactic `And` conjunctions) to prove ` $\theta \leq A$ ` or ` $\theta <
/
@[positivity (_ : HermitianMat _ _)]
def evalHermitianMatPSD : PositivityExt where eval {_u _α} _zα _pα e := do
  trace[Tactic.positivity] "evalHermitianMatPSD: {e}"
  let lctx ← getLCtx
  let mut best : Strictness _zα _pα e := .none
  for ldecl in lctx do
    if ldecl.isImplementationDetail then continue
    let ty := ldecl.type
    let p : Expr := .fvar ldecl.fvarId
    if let some (isStrict, pf) ← findHermitianMatPSDInExpr e p ty then
      if isStrict then
        return .positive pf
      else
        best := .nonnegative pf
  match best with
  | .none => throwError "evalHermitianMatPSD: no A.mat.PosSemidef or A.mat.PosDef hy
  | other => return other$ 
```

```

open Lean Meta Mathlib.Meta.Positivity in
/-- Positivity extension for `HermitianMat.kronecker`: nonneg when both fac
/
@[positivity HermitianMat.kronecker _ _]
def evalHermitianMatKronecker : PositivityExt where eval {u _α} _zα _pα e
  let .app (.app _kron A) B ← whnfR e | throwError "not HermitianMat.kronec
  let (isStrictA, pfA) ← bestResult A
  let (isStrictB, pfB) ← bestResult B
  if isStrictA && isStrictB then
    pure (.positive (← mkAppM ``HermitianMat.kronecker_pos #[pfA, pfB]))
  else
    let pfA' ← try mkAppM ``le_of_lt #[pfA] catch _ => pure pfA
    let pfB' ← try mkAppM ``le_of_lt #[pfB] catch _ => pure pfB
    let pfAB' ← mkAppM ``HermitianMat.kronecker_nonneg #[pfA', pfB']
    pure (.nonnegative pfAB')

variable (M) in
open Lean Meta Mathlib.Meta.Positivity in
/-- Positivity extension for `HermitianMat.conj`: nonneg when the inner mat
/
theorem conj_nonneg (hA : 0 ≤ A) : 0 ≤ A.conj M := by
  rw [zero_le_iff] at hA
  exact Matrix.PosSemidef.mul_mul_conjTranspose_same hA M

theorem conj_pos [DecidableEq n] {A : HermitianMat n ℂ} {M : Matrix m n ℂ}
  (h : LinearMap.ker M.toEuclideanLin ≤ A.ker) : 0 < A.conj M := by
  classical exact (A.conj_nonneg M hA.le).lt_of_ne' (A.conj_ne_zero hA.ne')

open Lean Meta Mathlib.Meta.Positivity in
/-- Positivity extension for `HermitianMat.conj`: nonneg when the inner mat
/
@[positivity HermitianMat.conj _ _]
def evalHermitianMatConj : PositivityExt where eval {u _α} _zα _pα e := do
  let .app (.app _coe conjM) (A : Expr) ← whnfR e | throwError "not conj ap
  let M := conjM.appArg!
  let (_, pfA) ← bestResult A
  let pfNonneg ← try mkAppM ``le_of_lt #[pfA] catch _ => pure pfA
  pure (.nonnegative (← mkAppM ``HermitianMat.conj_nonneg #[M, pfNonneg]))

open MatrixOrder in
example {A : Matrix n n ℂ} (hA : A.PosSemidef) : 0 ≤ A := by
  positivity

open MatrixOrder in
example {A : Matrix n n ℂ} [Nonempty n] (hA : A.PosDef) : 0 < A := by

```

```

positivity

example (hA : A.mat.PosSemidef) :  $0 \leq A$  := by
  positivity

example [Nonempty n] (hA : A.mat.PosDef) :  $0 < A$  := by
  positivity

example [DecidableEq n] [DecidableEq m] [Nonempty n] [Nonempty m]
  (A B : HermitianMat n  $\mathbb{C}$ ) (hA :  $0 \leq A$ ) (hB :  $0 \leq B$ ) (M : Matrix m n  $\mathbb{C}$ ) :
   $0 < (2 : HermitianMat (n \times m) \mathbb{C}) + (3 \cdot A) \otimes_k (\text{Real.pi} \cdot B).conj M$  := by
  positivity

example (A B : HermitianMat n  $\mathbb{C}$ ) (hA :  $0 < A$ ) (hB :  $0 < B$ ) :
   $0 < ((37 \cdot A) \otimes_k ((38 : \mathbb{R}) \cdot B)).trace$  := by
  positivity

omit [Fintype n] in
theorem convex_cone (hA :  $0 \leq A$ ) (hB :  $0 \leq B$ ) {c1 c2 :  $\mathbb{R}$ } (hc1 :  $0 \leq c_1$ ) (hc2 :  $0 \leq$ 
   $0 \leq (c_1 \cdot A + c_2 \cdot B)$  := by
  rw [zero_le_iff] at hA hB ⊢
  exact (hA.smul hc1).add (hB.smul hc2)

theorem sq_nonneg [DecidableEq n] :  $0 \leq A^2$  := by
  simp [zero_le_iff, pow_two]
  nth_rewrite 1 [←Matrix.IsHermitian.eq A.H]
  exact Matrix.posSemidef_conjTranspose_mul_self A.mat

theorem ker_antitone [DecidableEq n] (hA :  $0 \leq A$ ) :  $A \leq B \rightarrow B.ker \leq A.ker$  := by
  intro h x hB
  replace h := (le_iff_mulVec_le.mp h) x
  rw [HermitianMat.mem_ker_iff_mulVec_zero] at hB ⊢
  rw [hB, dotProduct_zero] at h
  rw [zero_le_iff] at hA
  rw [← hA.dotProduct_mulVec_zero_iff]
  rw [Matrix.posSemidef_iff_dotProduct_mulVec] at hA
  exact le_antisymm h (hA.right x)

theorem conj_mono (h :  $A \leq B$ ) :  $A.conj M \leq B.conj M$  := by
  have h_conj_pos :  $(M * (B - A).mat * M^H).PosSemidef$  :=
    Matrix.PosSemidef.mul_mul_conjTranspose_same h M
  constructor;
  · simp [conj, Matrix.IsHermitian, Matrix.mul_assoc]
  · simp [Matrix.mul_sub, Matrix.sub_mul] using h_conj_pos.2

lemma conj_posDef [DecidableEq n] (hA : A.mat.PosDef) (hN : IsUnit N) :
```

```

    (A.conj N).mat.PosDef := by
  rw [Matrix.posDef_iff_dotProduct_mulVec] at hA ⊢
  use HermitianMat.H _
  intro x hx_ne_zero
  open Matrix in
  have h_pos : 0 < star (NH *v x) ⌊v A *v (NH *v x) := by
    apply hA.2
    intro h
    apply hx_ne_zero
    simp [hN] using Matrix.eq_zero_of_mulVec_eq_zero
    (by simp [Matrix.det_conjTranspose] using hN.map Matrix.detMonoidHom)
  convert h_pos using 1
  simp only [conj_apply_mat, mulVec_mulVec, Matrix.mul_assoc]
  simp [dotProduct_mulVec, mulVec_conjTranspose]

lemma inv_conj [DecidableEq n] {M : Matrix n n []} (hM : IsUnit M) :
  (A.conj M)-1 = A-1.conj (M-1)H := by
  have h_inv : (M-1)H * MH = 1 := by
    simp only [Matrix.isUnit_iff_isUnit_det, isUnit_iff_ne_zero, ne_eq] at
    simp [Matrix.conjTranspose_nonsing_inv, hM]
  ext1
  simp only [conj, AddMonoidHom.coe_mk, ZeroHom.coe_mk, Matrix.conjTranspose]
  simp only [mat_inv, mat_mk]
  rw [Matrix.mul_inv_rev, Matrix.mul_inv_rev, Matrix.inv_eq_left_inv h_inv,

theorem le_iff_mulVec_le_mulVec (A B : HermitianMat n []) :
  A ≤ B ↔ ∀ v : n → [], star v ⌊v A.mat *v v ≤ star v ⌊v B.mat *v v := by
  rw [← sub_nonneg, HermitianMat.zero_le_iff]
  conv_rhs => enter [v]; rw [← sub_nonneg]
  have h := (B - A).H
  simp only [HermitianMat.mat_sub] at h
  simp [Matrix.posSemidef_iff_dotProduct_mulVec, Matrix.sub_mulVec, h]

theorem inner_mulVec_nonneg (hA : 0 ≤ A) (v : n → []) :
  0 ≤ star v ⌊v A.mat *v v := by
  rw [le_iff_mulVec_le_mulVec] at hA
  simp using hA v

theorem mem_ker_of_inner_mulVec_zero [DecidableEq n] (hA : 0 ≤ A) (v : Eucl
  (h : star v ⌊v A.mat *v v = 0) : v ∈ A.ker := by
  have := ((zero_le_iff.mp hA).dotProduct_mulVec_zero_iff v).mp h
  exact congr(WithLp.toLp 2 $this)

theorem ker_add [DecidableEq n] (hA : 0 ≤ A) (hB : 0 ≤ B) :
  (A + B).ker = A.ker ⌊ B.ker := by
  have hA' := zero_le_iff.mp hA

```

```

have hB' := zero_le_iff.mp hB
ext v; simp only [Submodule.mem_inf, mem_ker_iff_mulVec_zero]
constructor
· intro hv
  have h3 : star v ⊆v A.mat *v v + star v ⊆v B.mat *v v = 0 := by
    rw [← dotProduct_add, ← Matrix.add_mulVec, ← mat_add, hv, dotProduct_zero]
    rw [Matrix.posSemidef_iff_dotProduct_mulVec] at hA' hB'
    obtain (hzA, hzB) := (add_eq_zero_iff_of_nonneg (hA'.2 v) (hB'.2 v)).mp h3
    rw [← Matrix.posSemidef_iff_dotProduct_mulVec] at hA' hB'
    exact ((hA'.dotProduct_mulVec_zero_iff v).mp hzA,
            (hB'.dotProduct_mulVec_zero_iff v).mp hzB)
· simp +contextual [Matrix.add_mulVec]

theorem ker_sum [DecidableEq n] (f : ι → HermitianMat n) (hf : ∀ i, 0 ≤ f i) :
  (∑ i, f i).ker = ⊆i (f i).ker := by
  ext v
  simp only [Submodule.mem_iInf, mem_ker_iff_mulVec_zero]
  constructor
  · intro hv i
    have hfi := zero_le_iff.mp (hf i)
    rw [← hfi.dotProduct_mulVec_zero_iff]
    have hge : ∀ j, 0 ≤ star v ⊆v (f j).mat *v v := by
      intro j
      have := zero_le_iff.mp (hf j)
      rw [Matrix.posSemidef_iff_dotProduct_mulVec] at this
      exact this.2 v
    have hsum : ∑ j, star v ⊆v (f j).mat *v v = 0 := by
      rw [← dotProduct_sum, ← Matrix.sum_mulVec, ← mat_finset_sum, hv, dotProduct_zero]
    exact le_antisymm
      (hsum ▸ Finset.single_le_sum (fun j _ => hge j) (Finset.mem_univ i))
      (hge i)
  · intro h
    simp [Matrix.sum_mulVec, h]

theorem ker_conj [DecidableEq n] (hA : 0 ≤ A) (B : Matrix n n) :
  (A.conj B).ker = Submodule.comap (Matrix.toEuclideanLin B.conjTranspose) A.ker :

  ext v; simp [HermitianMat.conj];
  constructor <=> intro h;
  · have := Matrix.PosSemidef.dotProduct_mulVec_zero_iff ( show Matrix.PosSemidef A.
    convert this ( BH.mulVec v ) |>.1 _ using 1;
    · rw [ mem_ker_iff_mulVec_zero ];
      congr! 2;
    · convert congr_arg ( fun x : EuclideanSpace _ => star v.ofLp ⊆v x ) h using 1
      simp [Matrix.mulVec_mulVec, Matrix.dotProduct_mulVec]
      · simp [Matrix.mul_assoc, Matrix.dotProduct_mulVec, Matrix.mulVec_mulVec, Matr

```

```

    · simp [dotProduct]
  · simp only [ker, Matrix.mul_assoc, LinearMap.mem_ker]
    convert congr_arg B.toEuclideanLin h using 1
    · simp [HermitianMat.lin, Matrix.toEuclideanLin]
    · exact Eq.symm (LinearMap.map_zero (Matrix.toEuclideanLin B))

theorem ker_le_of_le_smul {α : ℝ} [DecidableEq n] (hα : α ≠ 0) (hA : 0 ≤ A)
  rw [← ker_pos_smul B hα]
  exact ker_antitone hA hAB

--TODO: Positivity extensions for traceLeft, traceRight, rpow, nat powers,
-- the various `proj` function (in Proj.lean), and the inner product.

/-! ## Positivity extensions connecting HermitianMat and Matrix -
/
section MatrixPositivity
open Lean Meta Mathlib.Meta.Positivity

/-- If a HermitianMat is PSD, then its eigenvalues are nonneg. -
/
theorem eigenvalues_nonneg [DecidableEq n] (hA : 0 ≤ A) (i : n) :
  0 ≤ A.H.eigenvalues i :=
  (zero_le_iff.mp hA).eigenvalues_nonneg i

omit [Fintype n] in
open MatrixOrder in
/-- If a HermitianMat is PSD, its underlying matrix is nonneg in the Loewne
/
theorem mat_nonneg (hA : 0 ≤ A) : 0 ≤ A.mat :=
  Matrix.nonneg_iff_posSemidef.mpr (zero_le_iff.mp hA)

omit [Fintype n] in
open MatrixOrder in
/-- If a HermitianMat is positive, its underlying matrix is positive in the
/
theorem mat_pos (hA : 0 < A) : 0 < A.mat :=
  hA

open MatrixOrder in
/-- `M^H * M` is nonneg in the Loewner order, for any matrix `M`. -
/
theorem _root_.Matrix.nonneg_conjTranspose_mul_self {m : Type*} [Fintype m]
  (M : Matrix m n ℝ) : 0 ≤ M.conjTranspose * M :=
  Matrix.nonneg_iff_posSemidef.mpr (Matrix.posSemidef_conjTranspose_mul_sel

open MatrixOrder in

```



```

/-- `M * M^H` is nonneg in the Loewner order, for any matrix `M`. -
/
theorem _root_.Matrix.nonneg_self_mul_conjTranspose {m : Type*} [Fintype m]
  (M : Matrix n m  $\mathbb{C}$ ) :  $0 \leq M * M.conjTranspose$  :=
  Matrix.nonneg_iff_posSemidef.mpr (Matrix.posSemidef_self_mul_conjTranspose M)

omit [Fintype m] in
open MatrixOrder in
theorem subtype_mk_nonneg {M : Matrix m m  $\mathbb{C}$ } (h :  $0 \leq M$ ) :
   $0 \leq (\langle M, (Matrix.LE.le.posSemidef h).isHermitian \rangle : HermitianMat\ m\ \mathbb{C}) :=$ 
  h

omit [Fintype m] in
open MatrixOrder in
theorem subtype_mk_pos {M : Matrix m m  $\mathbb{C}$ } (h :  $0 < M$ ) :
   $0 < (\langle M, (Matrix.LE.le.posSemidef h.le).isHermitian \rangle : HermitianMat\ m\ \mathbb{C}) :=$ 
  h

open MatrixOrder in
private theorem _root_.Matrix.eigenvalues_nonneg [DecidableEq n] {M : Matrix n n  $\mathbb{C}$ }
   $0 \leq (Matrix.LE.le.posSemidef h).isHermitian.eigenvalues\ i :=$ 
  (Matrix.LE.le.posSemidef h).eigenvalues_nonneg i

/-- Positivity extension for `A.mat` where `A : HermitianMat`: nonneg when ` $0 \leq A$ `. -
/
@[positivity HermitianMat.mat _]
def evalHermitianMatMat : PositivityExt where eval {u  $\alpha$  z  $\alpha$  p  $\alpha$  e} := do
  let .app _matFn (A : Expr)  $\leftarrow$  whnfR e | throwError "not HermitianMat.mat"
  match  $\leftarrow$  bestResult A with
  | (true, pa) =>
    pure (.positive ( $\leftarrow$  mkAppM ``HermitianMat.mat_pos #[pa]))
  | (false, pa) =>
    pure (.nonnegative ( $\leftarrow$  mkAppM ``HermitianMat.mat_nonneg #[pa]))

/-- Positivity extension for `A.mat` where `A : HermitianMat`: nonneg when ` $0 \leq A$ `. -
/
@[positivity Subtype.val (_ : HermitianMat _)]
def evalHermitianMatVal : PositivityExt where eval {u  $\alpha$  z  $\alpha$  p  $\alpha$  e} := do
  /- Note: we must not call `whnf` on `e` because `Subtype.val` is a structure
  projection (reducible), so `whnf` would reduce it and destroy the pattern. -
  /
  let A := e.appArg!
  match  $\leftarrow$  bestResult A with
  | (true, pa) =>
    pure (.positive ( $\leftarrow$  mkAppM ``HermitianMat.mat_pos #[pa]))
  | (false, pa) =>

```

```

    pure (.nonnegative (← mkAppM ``HermitianMat.mat_nonneg #[pa]))

/-- Positivity extension for `M * M^H` as a Matrix: always nonneg. -
/
@[positivity HMul.hMul _ (Matrix.conjTranspose _)]
def evalMatrixSelfMulConjTranspose : PositivityExt where eval {_u _α} _zα _
  let .app (.app _hmul _M) Mstar ← whnfR e | throwError "not HMul applicati
  let .app _conjTranspose M' ← whnfR Mstar | throwError "not M * conjTransp
  pure (.nonnegative (← mkAppM ``Matrix.nonneg_self_mul_conjTranspose #[M']

/-- Positivity extension for `M^H * M` as a Matrix: always nonneg. -
/
@[positivity HMul.hMul (Matrix.conjTranspose _) _]
def evalMatrixConjTransposeMulSelf : PositivityExt where eval {_u _α} _zα _
  let .app (.app _hmul Mstar) _M ← whnfR e | throwError "not HMul applicati
  let .app _conjTranspose M' ← whnfR Mstar | throwError "not conjTranspose
  pure (.nonnegative (← mkAppM ``Matrix.nonneg_conjTranspose_mul_self #[M']

/-- Positivity extension for `(M, (pf : M.IsHermitian))` as a HermitianMat:
equivalent to ` $0 \leq M$ ` in `MatrixOrder`. -/
@[positivity (Subtype.mk _ : HermitianMat _)]
def evalHermitianMatMk : PositivityExt where eval {_u _α} _zα _pα e := do
  let .app (.app _mkFn val) _proof ← whnfR e | throwError "not Subtype.mk"
  match ← bestResult val with
  | (true, pa) =>
    pure (.positive (← mkAppM ``HermitianMat.subtype_mk_pos #[pa]))
  | (false, pa) =>
    pure (.nonnegative (← mkAppM ``HermitianMat.subtype_mk_nonneg #[pa]))

/-- Positivity extension for eigenvalues of a Matrix: ` $0 \leq$  ( $_$  : M.IsHermiti
Will try to prove ` $0 \leq M$ ` in the `MatrixOrder`. If the proof is `A.H`, i.e.
HermitianMat, this will give ` $0 \leq A.mat$ ` which becomes ` $0 \leq A$ ` later. -
/
@[positivity Matrix.IsHermitian.eigenvalues _]
def evalMatrixEigenvalues : PositivityExt where eval {_u _α} _zα _pα e := d
  let .app (.app _eigenvaluesFn hProof) _i ← whnfR e | throwError "not eige
  let pType ← inferType hProof
  if pType.isAppOf ``Matrix.IsHermitian then
    let M ← pure pType.appArg!
    let (_, pa) ← bestResult M
    let pa ← try mkAppM ``le_of_lt #[pa] catch _ => pure pa
    pure (.nonnegative (← mkAppM ``Matrix.eigenvalues_nonneg #[pa, _i]))
  else
    throwError "not a Matrix.IsHermitian"

-- Tests

```

section tests

```
variable [DecidableEq n] [DecidableEq m]
open MatrixOrder
```

```
-- Test: eigenvalues nonneg from PSD HermitianMat
```

```
example (A : HermitianMat n ℂ) (hA : 0 < A) (i : n) : 0 ≤ A.H.eigenvalues i := by
  positivity
```

```
-- Test: A.mat nonneg from A nonneg
```

```
example (A : HermitianMat n ℂ) (hA : 0 ≤ A) : 0 ≤ A.mat := by positivity
```

```
example (A : HermitianMat n ℂ) (hA : 0 < A) : 0 < A.mat := by positivity
```

```
example (A : HermitianMat n ℂ) (hA : 0 ≤ A) : 0 ≤ A.val := by positivity
```

```
example (A : HermitianMat n ℂ) (hA : 0 < A) : 0 < A.val := by positivity
```

```
-- Test:  $M^H * M$  nonneg as Matrix
```

```
example (M : Matrix m n ℂ) : 0 ≤ M.conjTranspose * M := by positivity
```

```
-- Test:  $M * M^H$  nonneg as Matrix
```

```
example (M : Matrix n m ℂ) : 0 ≤ M * M.conjTranspose := by positivity
```

```
-- Test:  $\langle M^H * M, \_ \rangle$  nonneg as HermitianMat
```

```
example (M : Matrix m n ℂ) :
```

```
  (0 : HermitianMat n ℂ) ≤ ⟨M.conjTranspose * M, Matrix.isHermitian_conjTranspose_
    positivity
```

```
-- Test:  $\langle M * M^H, \_ \rangle$  nonneg as HermitianMat
```

```
example (M : Matrix n m ℝ) :
```

```
  (0 : HermitianMat n ℝ) ≤ ⟨M * M.conjTranspose, Matrix.isHermitian_mul_conjTransp
    positivity
```

```
example (M : Matrix n n ℂ) (i : n) (A : HermitianMat n ℂ) (hA : 0 ≤ A) :
```

```
  0 ≤ (A + ⟨_, M.isHermitian_mul_conjTranspose_self⟩ + 0).H.eigenvalues i := by
    positivity
```

end tests

end MatrixPositivity

1.463 QuantumInfo/ForMathlib/HermitianMat/Proj.lean

/-

Copyright (c) 2025 Leonardo A Lessa. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Leonardo A Lessa, Alex Meiburg

```

-/
import QuantumInfo.ForMathlib.HermitianMat.CFC

import Mathlib.Analysis.CStarAlgebra.Classes
import Mathlib.Analysis.SpecialFunctions.ContinuousFunctionalCalculus.PosPa

/-!

# Projectors associated to Hermitian matrices

* `projector`: The `HermitianMat` that projects onto a given submodule
* `supportProj`: The `HermitianMat` that projects onto the range (nonzero
* `kerProj`: The `HermitianMat` that projects onto the kernel
* `projLE`: With notation `{A ≤p B}`, `projLE A B` is the projector onto t
  eigenspace of `B - A`.
* `projLT`: With notation `{A <p B}`, `projLT A B` is the projector onto t
  eigenspace of `B - A`.
* Positive and negative part, written `A+` and `A-`, give the restriction
  onto its positive (resp. negative) eigenvalues; equivalently, it's nonne
  nonpositive) eigenvalues.
-/

noncomputable section
namespace HermitianMat

variable {n : Type*} [Fintype n] [DecidableEq n]
variable {α : Type*} [RCLike α]
variable {ι : Type*} [Fintype ι] (S : Submodule α (EuclideanSpace α n))

variable (A B : HermitianMat n α)

open scoped InnerProductSpace

/--
Given a Submodule (EuclideanSpace ...) to HermitianMat, this gives the proj
i.e. a matrix that squares to itself, preserves vectors in the submodule, a
in the orthogonal complement of that submodule.
-/
noncomputable def projector (S : Submodule α (EuclideanSpace α n)) : Hermit
  let P := S.subtypeL.comp S.orthogonalProjection
  (P.toMatrix (EuclideanSpace.basisFun n α).toBasis (EuclideanSpace.basisFu
    ext i j
    simpα [EuclideanSpace.inner_single_right, EuclideanSpace.inner_single_l
      S.inner_starProjection_left_eq_right (EuclideanSpace.single i 1) (Euc

theorem projector_add_orthogonal : projector S + projector S⊥ = 1 := by

```

```

    unfold projector;
    erw [ Subtype.mk_eq_mk ];
    ext i j; simp [ LinearMap.toMatrix_apply, Matrix.one_apply ] ;

@[simp]
theorem trace_projector : (projector S).trace = (Module.finrank  $\mathbb{R}$  S) := by
  suffices h_trace : ((S.subtype  $\circ$   $\iota$  S.orthogonalProjection).toMatrix (EuclideanSpace  $\mathbb{R}$  S)) = (Module.finrank  $\mathbb{R}$  S) := by
    simp [projector, trace_eq_re_trace, h_trace]
  suffices h_trace : ((S.subtype  $\circ$   $\iota$  S.orthogonalProjection).toMatrix (EuclideanSpace  $\mathbb{R}$  S)) = (Module.finrank  $\mathbb{R}$  S) := by
    simp [h_trace]
  rw [LinearMap.toMatrix_comp _ (Module.finBasis  $\iota$  S), Matrix.trace_mul_comm,  $\leftarrow$  LinearMap.trace_mul_comm]
  congr 2
  ext1
  simp [Submodule.orthogonalProjection_mem_subspace_eq_self]

/--
The `HermitianMat.projector` for the `HermitianMat.support` submodule.
-/
noncomputable def supportProj (A : HermitianMat n  $\mathbb{R}$ ) : HermitianMat n  $\mathbb{R}$  := projector A (HermitianMat.support A)

/--
The `HermitianMat.projector` for the `HermitianMat.ker` submodule.
-/
noncomputable def kerProj (A : HermitianMat n  $\mathbb{R}$ ) : HermitianMat n  $\mathbb{R}$  := projector A (HermitianMat.ker A)

@[simp]
theorem kerProj_add_supportProj : A.kerProj + A.supportProj = 1 := by
  rw [ $\leftarrow$  projector_add_orthogonal A.ker, ker_orthogonal_eq_support, kerProj, supportProj]

@[simp]
theorem kerProj_of_nonSingular [NonSingular A] : A.kerProj = 0 := by
  simp only [kerProj, nonSingular_ker_bot, HermitianMat.ext_iff]
  simp [projector]

@[simp]
theorem supportProj_of_nonSingular [NonSingular A] : A.supportProj = 1 := by
  simpa using A.kerProj_add_supportProj

/--
The projector onto a submodule S is the sum of the outer products of the vectors in S.
-/
theorem projector_eq_sum_rankOne (b : OrthonormalBasis  $\iota$   $\mathbb{R}$  S) :
  (projector S).mat =  $\sum i$ , Matrix.vecMulVec (S.subtype (b i)) (star (S.subtype (b i))) := by
  unfold projector;
  ext i j;
  field_simp;

```

```

simp [Matrix.vecMulVec]
-- By definition of orthogonal projection, we can write the projection of
have h_proj :  $\forall j : n$ ,  $S.orthogonalProjection$  (EuclideanSpace.single j 1)
intro j
have h_proj :  $S.orthogonalProjection$  (EuclideanSpace.single j 1) =  $\sum k$ ,
  convert b.sum_repr (  $S.orthogonalProjection$  ( EuclideanSpace.single j
    constructor <;> intro h <;> simp_all [ Subtype.ext_iff, b.repr_apply_
  convert h_proj using 3
  simp [ inner];
convert congr_arg ( fun x : EuclideanSpace ( _ ) n => x i ) ( h_proj j )
simp [ Matrix.sum_apply, mul_comm ]

/--
The projector onto the support of A is the sum of the projections onto the
zero eigenvalues.
-/
lemma projector_support_eq_sum : A.supportProj.mat =
   $\sum i$ , (if A.H.eigenvalues i = 0 then 0 else 1) •
    Matrix.vecMulVec (A.H.eigenvectorBasis i) (star (A.H.eigenvectorBasis
have h_support : A.support = Submodule.span ([]) (Set.image (fun i => A.H.
  refine' le_antisymm _ _;
  • intro x hx;
    -- By definition of  $A.support$ , we know that  $x$  is in the orthogona
    have h_orthogonal_complement :  $x \in (A.ker : Submodule ([]) (EuclideanS$ 
      convert hx using 1;
      exact ker_orthogonal_eq_support A;
    -- By definition of  $A.ker$ , we know that  $x$  is orthogonal to all ei
    have h_orthogonal_zero_eigenvalues :  $\forall i$ ,  $A.H.eigenvalues i = 0 \rightarrow inn$ 
      intro i hi
      have h_eigenvector_zero : A.mat.mulVec (A.H.eigenvectorBasis i) = 0
        have := A.H.mulVec_eigenvectorBasis i; aesop;
      convert h_orthogonal_complement ( A.H.eigenvectorBasis i ) _ using
      exact (mem_ker_iff_mulVec_zero A ((H A).eigenvectorBasis i)).mpr h_
    -- By definition of  $A.ker$ , we know that  $x$  can be written as a lin
zero eigenvalues.
have h_decomp :  $x = \sum i$ , (inner ([]) (A.H.eigenvectorBasis i) x) • A.H.
  exact Eq.symm (OrthonormalBasis.sum_repr' (H A).eigenvectorBasis x)
rw [ h_decomp ];
exact Submodule.sum_mem _ fun i _ => if hi : A.H.eigenvalues i = 0 th
  • rw [ Submodule.span_le, Set.image_subset_iff ];
  intro i hi;
  simp_all [ HermitianMat.support ];
  use (1 / A.H.eigenvalues i) • A.H.eigenvectorBasis i;
  convert congr_arg ( fun x => ( 1 / A.H.eigenvalues i ) • x ) ( A.H.mu
  simp [hi]
  simp [ funext_iff, Matrix.mulVec, dotProduct ];

```

```

    exact PiLp.ext_iff;
  have h_orthonormal_basis :  $\exists b : \text{OrthonormalBasis } \{i : n \mid A.H.\text{eigenvalues } i \neq 0\}$ 
    refine' ( _, _ );
    refine' OrthonormalBasis.mk _ _;
    use fun i => ( A.H.eigenvectorBasis i, Submodule.subset_span ( Set.mem_image_of_
  all_goals simp [ Orthonormal ];
  · intro i j hij; have := A.H.eigenvectorBasis.orthonormal; simp_all [ orthonorma
    exact fun h => hij <| Subtype.ext h;
  · rw [ Submodule.eq_top_iff' ];
    rintro ( x, hx );
    rw [ Submodule.mem_span ] at hx ⊢;
    intro p hp; specialize hx ( Submodule.map ( Submodule.subtype _ ) p ); simp_a
    exact hx fun i hi => ( _, hp i hi, rfl );
  obtain ( b, hb ) := h_orthonormal_basis
  have h_sum_rankOne : (projector A.support).mat =  $\sum i$ , Matrix.vecMulVec (b i) (star
    convert projector_eq_sum_rankOne _ b using 1
    simp [h_support] at *
  simp_all [ Finset.sum_ite ];
  convert h_sum_rankOne using 1;
  · exact h_support ▸ rfl;
  · refine' Finset.sum_bij ( fun i hi => ( i, by simp_a using hi ) ) _ _ _ _ <=> simp

/-
`HermitianMat.supportProj` as a cfc.
-/
theorem supportProj_eq_cfc : A.supportProj = A.cfc (if · = 0 then 0 else 1) := by
  apply HermitianMat.ext;
  rw [HermitianMat.cfc_toMat_eq_sum_smul_proj];
  convert projector_support_eq_sum A using 1;
  refine' Finset.sum_congr rfl fun i _ => _;
  ext x y
  simp [ Matrix.vecMulVec, Matrix.mul_apply ] ;
  simp [ Matrix.single ];
  simp [ Finset.sum_ite, Finset.filter_eq, Finset.filter_and ];
  rw [ Finset.sum_eq_single i ] <=> aesop

/-- Projector onto the non-negative eigenspace of `B - A`. Accessible by the notation
`{A ≤ₚ B}`, which is scoped to `HermitianMat`. This is the unique maximum operator `
such that `P² = P` and `P * A * P ≤ P * B * P` in the Loewner order. -
/
def projLE (A B : HermitianMat n  $\mathbb{R}$ ) : HermitianMat n  $\mathbb{R}$  :=
  (B - A).cfc (fun x ↦ if 0 ≤ x then 1 else 0)

/-- Projector onto the positive eigenspace of `B - A`. Accessible by the notation
`{A <ₚ B}`, which is scoped to `HermitianMat`. Compare with `proj_le`. -
/

```

```

noncomputable def projLT (A B : HermitianMat n  $\mathbb{R}$ ) : HermitianMat n  $\mathbb{R}$  :=
  (B - A).cfc (fun x  $\mapsto$  if 0 < x then 1 else 0)

-- Note this is in the opposite direction as in the Stein's Lemma paper, wh
-- as the default ordering. We offer the ` $\geq_p$ ` notation which is the same wi
-- flipped, similar to how ` $GT.gt$ ` is defeq to ` $LT.lt$ ` with arguments flipp
-- We put the ` $\geq_p$ ` first, since both can delaborate and we want to show the  $\leq$ 
scoped notation "{ A "  $\geq_p$  " B "}" => projLE B A
scoped notation "{ A "  $\leq_p$  " B "}" => projLE A B

scoped notation "{ A "  $>_p$  " B "}" => projLT B A
scoped notation "{ A "  $<_p$  " B "}" => projLT A B

theorem projLE_def : {A  $\leq_p$  B} = (B - A).cfc (fun x  $\mapsto$  if 0  $\leq$  x then 1 else 0)
  rfl

theorem projLT_def : {A  $<_p$  B} = (B - A).cfc (fun x  $\mapsto$  if 0 < x then 1 else 0)
  rfl

theorem projLE_sq : {A  $\leq_p$  B}2 = {A  $\leq_p$  B} := by
  rw [projLE_def,  $\leftarrow$  cfc_pow,  $\leftarrow$  cfc_comp]
  congr! 2 with x
  simp

theorem projLT_sq : {A  $<_p$  B}2 = {A  $<_p$  B} := by
  rw [projLT_def,  $\leftarrow$  cfc_pow,  $\leftarrow$  cfc_comp]
  congr! 2 with x
  simp

theorem projLE_zero_cfc : {0  $\leq_p$  A} = A.cfc (fun x  $\mapsto$  if 0  $\leq$  x then 1 else 0)
  simp only [projLE_def, sub_zero]

theorem projLT_zero_cfc : {0  $<_p$  A} = A.cfc (fun x  $\mapsto$  if 0 < x then 1 else 0)
  simp only [projLT_def, sub_zero]

theorem projLE_zero_cfc' : {A  $\leq_p$  0} = A.cfc (fun x  $\mapsto$  if x  $\leq$  0 then 1 else 0)
  simp only [projLE_def, zero_sub]
  --TODO: Should do a `HermitianMat.cfc_comp_neg`?
  nth_rw 1 [ $\leftarrow$  cfc_id A]
  rw [ $\leftarrow$  cfc_neg,  $\leftarrow$  cfc_comp]
  congr! 2 with x
  simp

theorem projLT_zero_cfc' : {A  $<_p$  0} = A.cfc (fun x  $\mapsto$  if x < 0 then 1 else 0)
  simp only [projLT_def, zero_sub]
  --TODO: Should do a `HermitianMat.cfc_comp_neg`?

```



```

nth_rw 1 [← cfc_id A]
rw [← cfc_neg, ← cfc_comp]
congr! 2 with x
simp

theorem projLE_nonneg :  $0 \leq \{A \leq_p B\}$  := by
  rw [projLE_def, cfc_nonneg_iff]
  intro i
  apply ite_nonneg <;> norm_num

theorem projLT_nonneg :  $0 \leq \{A <_p B\}$  := by
  rw [projLT_def, cfc_nonneg_iff]
  intro i
  apply ite_nonneg <;> norm_num

theorem projLE_le_one :  $\{A \leq_p B\} \leq 1$  := by
  --The whole `rw` line is a defeq, i.e. `change _root_.cfc _ (B - A).mat ≤ 1` works
  --TODO better API.
  open MatrixOrder in
  rw [← Subtype.coe_le_coe, val_eq_coe, selfAdjoint.val_one]
  apply cfc_le_one (f := fun x ↦ if 0 ≤ x then 1 else 0)
  intros; split <;> norm_num

open MatrixOrder in
theorem projLE_mul_nonneg :  $0 \leq \{A \leq_p B\}.mat * (B - A).mat$  := by
  rw [projLE_def]
  nth_rewrite 2 [← cfc_id (B - A)]
  rw [← mat_cfc_mul]
  apply cfc_nonneg
  aesop

open MatrixOrder in
theorem projLE_mul_le :  $\{A \leq_p B\}.mat * A.mat \leq \{A \leq_p B\}.mat * B.mat$  := by
  rw [← sub_nonneg, ← mul_sub_left_distrib]
  exact projLE_mul_nonneg A B

@[simp]
theorem proj_le_add_lt :  $\{A <_p B\} + \{B \leq_p A\} = 1$  := by
  rw [projLE_def, projLT_def]
  rw [← neg_sub A B]
  nth_rw 1 [← cfc_id (A - B)]
  rw [← cfc_neg, ← cfc_comp, ← cfc_add]
  convert cfc_const (A - B) 1 with x
  · simp; grind
  · simp

```

```

theorem conj_lt_add_conj_le : A.conj {A <p 0} + A.conj {0 ≤p A} = A := by
  rw (occs := [2, 4, 5]) [← cfc_id A]
  rw [projLT_zero_cfc', projLE_zero_cfc, cfc_conj, cfc_conj, ← cfc_add]
  congr; ext
  simp; grind

/-
The projection onto the support can be split into the projection onto positive
and negative eigenvalues.
-/
theorem supportProj_eq_proj_lt_add_proj_lt (A : HermitianMat n []) :
  A.supportProj = {A <p 0} + {0 <p A} := by
  rw [supportProj_eq_cfc, projLT_zero_cfc, projLT_zero_cfc', ← cfc_add A]
  congr 1
  grind only [Pi.add_apply]

/-- The positive part of a Hermitian matrix: the projection onto its positive
part.
/
instance : PosPart (HermitianMat n []) where
  posPart A := A.cfc (fun x ↦ x ⊔ 0)

/-- The negative part of a Hermitian matrix: the projection onto its negative
part.
/
instance : NegPart (HermitianMat n []) where
  negPart A := A.cfc (fun x ↦ -x ⊔ 0)

theorem posPart_eq_cfc_max : A+ = A.cfc (fun x ↦ x ⊔ 0) := by
  rfl

theorem negPart_eq_cfc_min : A- = A.cfc (fun x ↦ -x ⊔ 0) := by
  rfl

theorem posPart_eq_cfc_ite : A+ = A.cfc (fun x ↦ if 0 ≤ x then x else 0) :=
  simp only [← max_def', posPart_eq_cfc_max]

theorem negPart_eq_cfc_ite : A- = A.cfc (fun x ↦ if x ≤ 0 then -
x else 0) := by
  simp only [negPart_eq_cfc_min, max_def]
  congr; ext
  split <;> split <;> grind

/-- There is an existing (very slow) `PosPart` instance on `Matrix n n`,
that this is equal. -/
theorem posPart_eq_posPart_toMat : A+ = A.mat+ := by
  rw [CFC.posPart_def, cfc_n_eq_cfc]
  rfl

```

```

/-- There is an existing (very slow) `PosPart` instance on `Matrix n n  $\mathbb{R}$ `, this shows
that this is equal. -/
theorem negPart_eq_negPart_toMat :  $A^- = A.\text{mat}^-$  := by
  rw [CFC.negPart_def, cfc_n_eq_cfc]
  rfl

/-- The positive part can be equivalently described as the nonnegative part. -
/
theorem posPart_eq_cfc_lt :  $A^+ = A.\text{cfc}$  (fun x  $\mapsto$  if  $0 < x$  then  $x$  else  $0$ ) := by
  rw [posPart_eq_cfc_ite]
  congr with x
  rcases lt_trichotomy x 0 <;> grind

/-- The negative part can be equivalently described as the nonpositive part. -
/
theorem negPart_eq_cfc_lt :  $A^- = A.\text{cfc}$  (fun x  $\mapsto$  if  $x < 0$  then  $-x$  else  $0$ ) := by
  rw [negPart_eq_cfc_ite]
  congr with x
  rcases lt_trichotomy x 0 <;> grind

theorem posPart_add_negPart :  $A^+ - A^- = A$  := by
  rw [posPart_eq_cfc_ite, negPart_eq_cfc_lt,  $\leftarrow$  cfc_sub]
  convert cfc_id A
  ext; dsimp; grind

theorem posPart_eq_self {A : HermitianMat n  $\mathbb{R}$ } (hA :  $0 \leq A$ ) :
   $A^+ = A$  := by
  nth_rw 2 [ $\leftarrow$  cfc_id A]
  apply cfc_congr_of_nonneg hA
  grind [Set.EqOn]

theorem posPart_nonneg :  $0 \leq A^+$  := by
  rw [posPart_eq_cfc_ite, cfc_nonneg_iff]
  intro; split <;> order

theorem negPart_nonneg :  $0 \leq A^-$  := by
  rw [negPart_eq_cfc_ite, cfc_nonneg_iff]
  intro; split <;> grind

theorem posPart_le :  $A \leq A^+$  := by
  nth_rw 1 [ $\leftarrow$  cfc_id A]
  rw [posPart_eq_cfc_ite,  $\leftarrow$  sub_nonneg,  $\leftarrow$  cfc_sub, cfc_nonneg_iff]
  intro; simp; split <;> order

```

```

theorem posPart_mul_negPart : A+.mat * A-.mat = 0 := by
  rw [posPart_eq_cfc_ite, negPart_eq_cfc_ite, ← mat_cfc_mul]
  convert congrArg mat (cfc_const A 0)
  · grind [Pi.mul_apply, mul_eq_zero]
  · simp

```

```

open RealInnerProductSpace

```

```

theorem projLE_inner_nonneg : 0 ≤ [A ≤p B], (B - A)[] :=
  --This inner is equal to `(B - A)+.trace`, could be better way to describe
  inner_mul_nonneg (projLE_mul_nonneg A B)

```

```

theorem projLE_inner_le : [A ≤p B], A[] ≤ [A ≤p B], B[] := by
  rw [← sub_nonneg, ← inner_sub_right]
  exact projLE_inner_nonneg A B

```

```

open RealInnerProductSpace in
theorem inner_projLE_nonneg : 0 ≤ [A ≤p B], (B - A)[] :=
  projLE_inner_nonneg A B

```

```

open RealInnerProductSpace in
theorem inner_projLE_le : [A ≤p B], A[] ≤ [A ≤p B], B[] :=
  projLE_inner_le A B

```

```

--TODO: When we upgrade `cfc_continuous` from  $\mathbb{R}$  to  $\mathbb{C}$ , we upgrade these too.
@[fun_prop]

```

```

theorem posPart_Continuous : Continuous (·+ : HermitianMat n  $\mathbb{C}$  → _) := by
  simp_rw [posPart_eq_cfc_max]
  fun_prop

```

```

@[fun_prop]
theorem negPart_Continuous : Continuous (·- : HermitianMat n  $\mathbb{C}$  → _) := by
  simp_rw [negPart_eq_cfc_min]
  fun_prop

```

```

proof_wanted posPart_le_zero_iff : A+ ≤ 0 ↔ A ≤ 0

```

```

proof_wanted posPart_eq_zero_iff : A+ = 0 ↔ A ≤ 0

```

```

/- := by
  rw [← posPart_le_zero_iff]
  have := zero_le_posPart A
  constructor <=> order
-/

```

```

--Many missing lemmas: see `Mathlib.Algebra.Order.Group.PosPart` for examples
-- (They don't apply here since it's not a Lattice, and there's no well-

```

```

defined `max` in
-- the Loewner order.)
-- PosPart is Monotone (so  $A \leq B$  implies  $A^+ \leq B^+$ ), as is NegPart
-- PosPart and NegPart commute with nonnegative scalar multiplication
--  $A^+ \leq 0 \leftrightarrow A^+ = 0 \leftrightarrow A = 0$ 
--  $0 \leq A \rightarrow A^+ = A$ 
--  $0 < A \rightarrow 0 < A^+$  (this is not the PosDef version, this is  $\leq$  &  $\neq$ )
--  $A.\text{PosDef} \rightarrow A^+.\text{PosDef}$ 
-- versions of those  $^+$  for negPart
-- simp:  $0^+ = 0$ ,  $0^- = 0$ ,  $1^+ = 1$ ,  $1^- = 0$ 
--  $(-A)^+ = A^-$ ,  $(-A)^- = A^+$ 
--  $A^{++} = A^+$ ,  $A^{+-} = 0$ 

-- variable {d : Type*} [Fintype d] [DecidableEq d] (A B : HermitianMat d  $\mathbb{C}$ )

theorem one_sub_projLT :  $1 - \{B \leq_p A\} = \{A <_p B\} :=$  by
  rw [sub_eq_iff_eq_add, proj_le_add_lt]

open MatrixOrder ComplexOrder in
theorem projLT_mul_nonneg :  $0 \leq \{A <_p B\}.\text{mat} * (B - A).\text{mat} :=$  by
  rw [projLT_def]
  nth_rewrite 2 [ $\leftarrow$  cfc_id (B - A)]
  rw [ $\leftarrow$  mat_cfc_mul]
  apply cfc_nonneg
  intros
  simp only [Pi.mul_apply, id_eq, ite_mul, one_mul, zero_mul]
  split <;> order

open MatrixOrder ComplexOrder in
theorem proj_lt_mul_lt :  $\{A <_p B\}.\text{mat} * A.\text{mat} \leq \{A <_p B\}.\text{mat} * B.\text{mat} :=$  by
  rw [ $\leftarrow$  sub_nonneg,  $\leftarrow$  mul_sub_left_distrib]
  exact A.projLT_mul_nonneg B

theorem inner_negPart_nonpos :  $\forall A, A^- \leq 0 :=$  by
  rw [ $\leftarrow$  neg_le_neg_iff, neg_zero,  $\leftarrow$  inner_neg_right]
  apply inner_mul_nonneg
  nth_rw 1 [ $\leftarrow$  A.cfc_id]
  rw [negPart_eq_cfc_ite]
  rw [ $\leftarrow$  cfc_neg]
  rw [ $\leftarrow$  mat_cfc_mul]
  change  $0 \leq A.\text{cfc\_}$ 
  rw [cfc_nonneg_iff]
  intro i
  dsimp
  split_ifs with h
  · rw [neg_neg]

```

```

    exact mul_self_nonneg _
  · simp

@[simp]
theorem posPart_inner_negPart_zero :  $\langle A^+, A^- \rangle = 0$  := by
  have hi := inner_eq_trace_rc A+ A-
  rw [posPart_mul_negPart, Matrix.trace_zero] at hi
  simp only [map_eq_zero] using hi

theorem inner_negPart_zero_iff :  $\langle A, A^- \rangle = 0 \leftrightarrow 0 \leq A$  := by
  constructor
  · intro h
    nth_rw 1 [← posPart_add_negPart A] at h
    rw [inner_sub_left, sub_eq_zero, posPart_inner_negPart_zero, eq_comm, i
    rw [← zero_smul ℝ 1, ← cfc_const A, negPart_eq_cfc_ite] at h --
TODO cfc_zero
    rw [cfc_eq_cfc_iff_eq0n, A.H.spectrum_real_eq_range_eigenvalues, Set.eq
    replace h (i) := congrFun h i
    simp only [Function.comp_apply, ite_eq_right_iff, neg_eq_zero] at h
    rw [zero_le_iff, A.H.posSemidef_iff_eigenvalues_nonneg]
    intro i
    contrapose! h
    use i, h.le, h.ne
  · intro h
    apply le_antisymm
    · exact inner_negPart_nonpos A
    · exact inner_ge_zero h (negPart_nonneg A)

theorem inner_negPart_neg_iff :  $\langle A, A^- \rangle < 0 \leftrightarrow \neg 0 \leq A$  := by
  simp [← inner_negPart_zero_iff, lt_iff_le_and_ne, inner_negPart_nonpos A]

/-- The self-duality of the PSD cone: a matrix is PSD iff its inner product
nonnegative matrices is non-negative. -/
theorem nonneg_iff_inner_nonneg (A : HermitianMat n  $\mathbb{R}$ ) :
   $0 \leq A \leftrightarrow \forall B, 0 \leq B \rightarrow 0 \leq \langle A, B \rangle$  := by
  use fun h _  $\mapsto$  inner_ge_zero h
  intro h
  contrapose! h
  classical
  use A-, negPart_nonneg A
  rwa [inner_negPart_neg_iff]

```

1.464 QuantumInfo/ForMathlib/HermitianMat/Reindex.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Basic
import QuantumInfo.ForMathlib.ContinuousLinearMap
import QuantumInfo.ForMathlib.LinearEquiv

/-!
Much like `Matrix.reindex` and `Matrix.submatrix`, we can reindex a Hermitian matrix
Hermitian matrix; however, this only makes sense when both permutations are the same
`HermitianMat.reindex` only takes one `Equiv` argument (as opposed to `Matrix.reindex`)

This file then gives relevant lemmas for simplifying this.
-/
namespace HermitianMat

variable {d d₂ d₃ d₄ α : Type*} [RCLike α]

variable (A B : HermitianMat d α) (e : d ≈ d₂)

def reindex (e : d ≈ d₂) : HermitianMat d₂ α :=
  (A.mat.reindex e e, A.H.submatrix e.symm)

@[simp]
theorem mat_reindex : (A.reindex e).mat = A.mat.reindex e e := by
  rfl

/-! Our simp-normal form for expressions involving `HermitianMat.reindex` is that we
the reindexing as far out as possible, so that it can be absorbed by `HermitianMat.trans`
cancelled out in a `HermitianMat.inner`. In places where it commutes (like `HermitianMat.reindex`
we push it to the rightside. One downside is that we're not as likely to hit `reindex`
-/

@[simp]
theorem reindex_refl (A : HermitianMat d α) :
  A.reindex (·.refl _) = A := by
  rfl

@[simp]
theorem reindex_reindex (A : HermitianMat d α) (e : d ≈ d₂) (f : d₂ ≈ d₃) :
  (A.reindex e).reindex f = A.reindex (e.trans f) := by
  ext1; simp; rfl

```

```

@[simp]
theorem reindex_zero : (0 : HermitianMat d []).reindex e = 0 := by
  ext1; simp

@[simp]
theorem reindex_one [DecidableEq d] [DecidableEq d₂] :
  (1 : HermitianMat d []).reindex e = 1 := by
  ext1
  simp [reindex]

@[simp]
theorem reindex_add : A.reindex e + B.reindex e = (A + B).reindex e := by
  ext1; simp [Matrix.submatrix_add]

@[simp]
theorem reindex_sub : A.reindex e - B.reindex e = (A - B).reindex e := by
  ext1; simp [Matrix.submatrix_sub]

@[simp]
theorem reindex_neg : (-A).reindex e = -(A.reindex e) := by
  ext1; simp [Matrix.submatrix_neg]

@[simp]
theorem reindex_smul (c : ℝ) : (c • A).reindex e = c • (A.reindex e) := by
  ext1; simp [Matrix.submatrix_smul]

@[simp]
theorem reindex_conj [Fintype d₂] [Fintype d] (B : Matrix d₃ d₂ []) :
  (A.reindex e).conj B = A.conj (B.submatrix id e) := by
  ext1
  simp only [conj_apply, mat_reindex, Matrix.reindex_apply, mat_mk]
  rw [← Matrix.submatrix_id_mul_right, Matrix.mul_assoc]
  rw [← Matrix.submatrix_id_mul_left, ← Matrix.mul_assoc]
  simp

variable [Fintype d]

theorem conj_submatrix (B : Matrix d₂ d₄ []) (e : d₃ ≈ d₂) (f : d → d₄) :
  A.conj (B.submatrix e f) = (A.conj (B.submatrix id f)).reindex e.symm :
  ext1
  simp [conj_apply, ← Matrix.submatrix_mul_equiv (e₂ := .refl d)]

theorem reindex_eq_conj [DecidableEq d] (e : d ≈ d₂) : A.reindex e = A.conj
  ext : 3
  simp [-mat_apply, reindex, conj_apply, Matrix.submatrix,

```



```

    Matrix.mul_apply, Matrix.one_apply]

variable [Fintype d₂] [DecidableEq d] [DecidableEq d₂]

theorem ker_reindex :
  (A.reindex e).ker = A.ker.comap (LinearEquiv.euclidean_of_relabel [] e).toLinearMap
  dsimp only [reindex, ker, lin]
  simp only [mat_mk]
  rw [Matrix.reindex_toEuclideanLin, LinearEquiv.ker_comp, LinearMap.ker_comp]

@[simp]
theorem ker_reindex_le_iff :
  (A.reindex e).ker ≤ (B.reindex e).ker ↔ A.ker ≤ B.ker := by
  rw [ker_reindex, ker_reindex]
  apply Submodule.comap_le_comap_iff_of_surjective
  exact LinearEquiv.surjective (LinearEquiv.euclidean_of_relabel [] e)

end HermitianMat

```

1.465 QuantumInfo/ForMathlib/HermitianMat/Rpow.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.LogExp
import QuantumInfo.ForMathlib.HermitianMat.Sqrt
import QuantumInfo.ForMathlib.HermitianMat.Unitary
import Mathlib.Analysis.SpecialFunctions.Integrability.Basic
import Mathlib.Analysis.SpecialFunctions.ImproperIntegrals
import Mathlib.MeasureTheory.Integral.IntegralEqImproper

variable {d d₂ : Type*} [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂]
variable [RCLike []]
variable {A B : HermitianMat d []} {x q r : ℝ}

/-! # Matrix powers with real exponents

-/

noncomputable section
namespace HermitianMat

```

```
/-- Matrix power of a positive semidefinite matrix, as given by the element
    real power of the diagonal in a diagonalized form.
```

```
    Note that this has the usual `Real.rpow` caveats, such as 0 to the power
    1 giving 0. -/
```

```
def rpow (A : HermitianMat d  $\mathbb{R}$ ) (r :  $\mathbb{R}$ ) : HermitianMat d  $\mathbb{R}$  :=
  A.cfc (Real.rpow  $\cdot$  r)
```

```
instance instRPow : Pow (HermitianMat d  $\mathbb{R}$ )  $\mathbb{R}$  :=
  (rpow)
```

```
theorem rpow_conj_unitary (A : HermitianMat d  $\mathbb{R}$ ) (U : Matrix.unitaryGroup d  $\mathbb{R}$ ) :
  (HermitianMat.conj U.val A) ^ r = HermitianMat.conj U.val (A ^ r) := by
  exact A.cfc_conj_unitary ( $\cdot$  ^ r) U
```

```
theorem pow_eq_rpow : A ^ r = A.rpow r :=
  rfl
```

```
theorem rpow_eq_cfc : A ^ r = A.cfc ( $\cdot$  ^ r) :=
  rfl
```

```
theorem diagonal_pow (f : d  $\rightarrow$   $\mathbb{R}$ ) :
  (diagonal  $\mathbb{R}$  f) ^ r = diagonal  $\mathbb{R}$  (fun i  $\mapsto$  (f i) ^ r) := by
  simp [rpow_eq_cfc]
  rfl
```

```
@[fun_prop]
```

```
theorem rpow_const_continuous {r :  $\mathbb{R}$ } (hr : 0  $\leq$  r) : Continuous (fun A : He
  exact HermitianMat.cfc_continuous (Real.continuous_rpow_const hr)
```

```
@[fun_prop]
```

```
theorem const_rpow_continuous [NonSingular A] : Continuous (fun r :  $\mathbb{R}$   $\mapsto$  A ^
  rw [ $\leftarrow$  continuousOn_univ]
  apply continuousOn_cfc_fun_nonsingular
  simp only [Real.rpow_eq_pow]
  fun_prop (disch := assumption)
```

```
/--
```

```
For a fixed Hermitian matrix A, the function  $x \mapsto A^x$  is continuous for  $x >
  -/$ 
```

```
@[fun_prop]
```

```
theorem continuousOn_rpow_pos (A : HermitianMat d  $\mathbb{C}$ ) : ContinuousOn (fun x
  apply A.continuousOn_cfc_fun (hA := subset_rfl)
  intro i _ x hx
  exact (Real.continuousAt_const_rpow' hx.ne').continuousWithinAt
```

```

/--
For a fixed Hermitian matrix A, the function  $x \mapsto A^x$  is continuous for  $x < 0$ .
-/
@[fun_prop]
theorem continuous0n_rpow_neg (A : HermitianMat d  $\mathbb{C}$ ) : ContinuousOn (fun x :  $\mathbb{R} \mapsto A^x$ 
  apply A.continuous0n_cfc_fun (hA := subset_rfl)
  intro i _ x hx
  exact (Real.continuousAt_const_rpow' hx.ne).continuousWithinAt

@[simp]
theorem rpow_one : A ^ (1 :  $\mathbb{R}$ ) = A := by
  simp [rpow_eq_cfc]

/--
Functional calculus of Real.sqrt is equal to functional calculus of  $x^{(1/2)}$ .
-/
lemma sqrt_eq_cfc_rpow_half (A : HermitianMat d  $\square$ ) :
  A.sqrt = A.cfc (fun x  $\mapsto x^{(1/2 : \mathbb{R})}$ ) := by
  rw [sqrt, cfc_eq_cfc_iff_eq0n]
  intro
  simp [Real.sqrt_eq_rpow]

@[simp]
theorem one_rpow : (1 : HermitianMat d  $\square$ ) ^ r = 1 := by
  rcases isEmpty_or_nonempty d
  · apply Subsingleton.allEq
  · nth_rw 2 [← HermitianMat.cfc_id (1 : HermitianMat d  $\square$ )]
    rw [rpow_eq_cfc]
    gcgr
    simp

@[simp]
lemma rpow_zero (A : HermitianMat d  $\square$ ) : A ^ (0 :  $\mathbb{R}$ ) = 1 := by
  simp [rpow_eq_cfc]

lemma rpow_diagonal (a : d  $\rightarrow \mathbb{R}$ ) (r :  $\mathbb{R}$ ) :
  (diagonal  $\mathbb{C}$  a) ^ r = diagonal  $\mathbb{C}$  (fun i  $\Rightarrow$  a i ^ r) := by
  exact cfc_diagonal _ _

/-- Keeps in line with our simp-normal form for moving reindex outwards. -
/
@[simp]
theorem reindex_rpow (e : d  $\approx$  d2) :
  A.reindex e ^ r = (A ^ r).reindex e := by
  apply A.cfc_reindex

```

```

theorem mat_rpow_add (hA : 0 ≤ A) {p q : ℝ} (hpq : p + q ≠ 0) :
  (A ^ (p + q)).mat = (A ^ p).mat * (A ^ q).mat := by
  simp only [rpow_eq_cfc, ← mat_cfc_mul, ← HermitianMat.ext_iff]
  exact cfc_congr_of_nonneg hA (fun i hi ↦ Real.rpow_add' hi hpq)

theorem rpow_mul (hA : 0 ≤ A) {p q : ℝ} : A ^ (p * q) = (A ^ p) ^ q := by
  simp only [rpow_eq_cfc, ← cfc_comp]
  exact cfc_congr_of_nonneg hA (fun i hi ↦ Real.rpow_mul hi p q)

theorem conj_rpow (hA : 0 ≤ A) (hq : q ≠ 0) (hqr : r + 2 * q ≠ 0) :
  (A ^ r).conj (A ^ q) = A ^ (r + 2 * q) := by
  simp only [rpow_eq_cfc, cfc_conj]
  refine cfc_congr_of_nonneg hA (fun i hi ↦ ?_)
  rw [pow_two, Real.rpow_add' hi hqr, two_mul, Real.rpow_add' hi (by simp)]
  rfl

theorem pow_half_mul (hA : 0 ≤ A) :
  (A ^ (1/2 : ℝ)).mat * (A ^ (1/2 : ℝ)).mat = A := by
  rw [← mat_rpow_add hA]
  · norm_num
  · norm_num

theorem rpow_pos {A : HermitianMat d []} (hA : 0 < A) {p : ℝ} : 0 < A ^ p :=
  convert cfc_pos_of_pos hA _
  · exact fun i hi => Real.rpow_pos_of_pos hi _
  · rcases eq_or_ne p 0 with h | h <=> simp [h]

theorem rpow_nonneg (hA : 0 ≤ A) {p : ℝ} : 0 ≤ A ^ p := by
  apply cfc_nonneg_of_nonneg hA
  exact fun i hi => Real.rpow_nonneg hi p

open ComplexOrder in
theorem inv_eq_rpow_neg_one (hA : A.mat.PosDef) : A-1 = A ^ (-1 : ℝ) := by
  have := nonSingular_of_posDef hA
  rw [← cfc_inv, rpow_eq_cfc]
  simp_rw [Real.rpow_neg_one]

open ComplexOrder in
theorem sandwich_self (hB : B.mat.PosDef) :
  (B.conj (B ^ (-1/2 : ℝ)).mat) = 1 := by
  have hB_inv_sqrt : (B ^ (-1 / 2 : ℝ)).mat * (B ^ (-1 / 2 : ℝ)).mat = (B ^ 1 : ℝ).mat := by
    rw [← mat_rpow_add] <=> norm_num
    rw [zero_le_iff]
    exact hB.posSemidef

```

```

have hB_inv : (B ^ (-1 : ℝ)).mat = B.mat-1 := by
  rw [← inv_eq_rpow_neg_one hB, mat_inv]
  rw [hB_inv] at hB_inv_sqrt;
  ext1
  simp [mul_assoc];
  rw [← Matrix.mul_assoc, mul_eq_one_comm.mp];
  rw [← Matrix.mul_assoc, hB_inv_sqrt, Matrix.nonsing_inv_mul _];
  exact isUnit_iff_ne_zero.mpr hB.det_pos.ne'

open ComplexOrder in
lemma rpow_inv_eq_neg_rpow (hA : A.mat.PosDef) (p : ℝ) : (A ^ p)-1 = A ^ (-p) := by
  --TODO cleanup
  ext i j;
  have h_inv : (A ^ p).mat * (A ^ (-p)).mat = 1 := by
    have h_inv : (A ^ p).mat * (A ^ (-p)).mat = 1 := by
      have h_pow : (A ^ p).mat = A.cfc (fun x => x ^ p) := by
        exact rfl
      have h_pow_neg : (A ^ (-p)).mat = A.cfc (fun x => x ^ (-p)) := by
        exact rfl
      have h_inv : (A ^ p).mat * (A ^ (-p)).mat = A.cfc (fun x => x ^ p * x ^ (-p)) := by
        exact rfl
      have h_inv : (A ^ p).mat * (A ^ (-p)).mat = A.cfc (fun x => 1) := by
        rw [h_inv];
        refine' congr_arg _ (cfc_congr_of_posDef hA _);
        exact fun x hx => by simp [← Real.rpow_add hx];
        rw [h_inv, cfc_const]; norm_num;
        exact h_inv;
      -- By definition of matrix inverse, if  $(A^p) * (A^{-p}) = 1$ , then  $(A^{-p})$  is the inverse of  $(A^p)$ .
      have h_inv_def : (A ^ p).mat-1 = (A ^ (-p)).mat := by
        exact Matrix.inv_eq_right_inv h_inv;
      convert congr_fun (congr_fun h_inv_def i) j using 1

open ComplexOrder in
lemma sandwich_le_one (hB : B.mat.PosDef) (h : A ≤ B) :
  (A.conj (B ^ (-1/2 : ℝ)).mat) ≤ 1 := by
  exact le_trans (conj_mono h) (sandwich_self hB).le

open ComplexOrder in
lemma rpow_neg_mul_rpow_self (hA : A.mat.PosDef) (p : ℝ) :
  (A ^ (-p)).mat * (A ^ p).mat = 1 := by
  have := rpow_inv_eq_neg_rpow hA p;

```

```

rw [← this];
-- Since  $A$  is positive definite,  $A^p$  is also positive definite.
have h_pos_def : (A ^ p).mat.PosDef := by
  have h_pos_def : ∀ p : ℝ, A.mat.PosDef → (A ^ p).mat.PosDef := by
    intro p hA_pos_def
    have h_eigenvalues_pos : ∀ i, 0 < (A.H.eigenvalues i) ^ p := by
      exact fun i => Real.rpow_pos_of_pos (by exact Matrix.PosDef.eigenvalues_pos hA_pos_def)
    have h_eigenvalues_pos : (A ^ p).mat.PosDef ↔ ∀ i, 0 < (A ^ p).H.eigenvalues i := by
      exact Matrix.IsHermitian.posDef_iff_eigenvalues_pos (H (A ^ p));
    have h_eigenvalues_pos : ∃ e : d ≈ d, (A ^ p).H.eigenvalues = fun i =>
      exact Matrix.IsHermitian.cfc_eigenvalues (H A) fun x => x.rpow p;
    aesop;
  exact h_pos_def p hA;
convert Matrix.nonsing_inv_mul _ _;
exact isUnit_iff_ne_zero.mpr h_pos_def.det_pos.ne'

open ComplexOrder in
lemma isUnit_rpow_toMat (hA : A.mat.PosDef) (p : ℝ) : IsUnit (A ^ p).mat :=
  have hA_inv : IsUnit (A ^ (-p)).mat := by
    have hA_inv : (A ^ (-p)).mat * (A ^ p).mat = 1 := by
      exact rpow_neg_mul_rpow_self hA p
    exact IsUnit.of_mul_eq_one _ hA_inv
  -- Since  $(A^{-p})(A^p) = 1$ , we have that  $(A^p)$  is the inverse of  $(A^{-p})$ .
  have hA_inv : (A ^ p).mat = (A ^ (-p)).mat⁻¹ := by
    have hA_inv : (A ^ (-p)).mat * (A ^ p).mat = 1 := by
      exact rpow_neg_mul_rpow_self hA p;
    exact Eq.symm (Matrix.inv_eq_right_inv hA_inv);
  aesop

open ComplexOrder in
lemma sandwich_inv (hB : B.mat.PosDef) :
  (A.conj (B ^ (-1/2 : ℝ)).mat)⁻¹ = A⁻¹.conj (B ^ (1/2 : ℝ)).mat := by
  have h_inv : (B ^ (-1 / 2 : ℝ)).mat⁻¹ = (B ^ (1 / 2 : ℝ)).mat := by
    apply Matrix.inv_eq_right_inv
  rw [← rpow_neg_mul_rpow_self hB (1 / 2), neg_div 2 1]
  simp [inv_conj (isUnit_rpow_toMat hB _), h_inv]

theorem ker_rpow_eq_of_nonneg {A : HermitianMat d ℂ} (hA : 0 ≤ A) (hp : r ≠ 0) :
  (A ^ r).ker = A.ker := by
  apply ker_cfc_eq_ker_nonneg hA
  grind [Real.rpow_eq_zero_iff_of_nonneg, Real.rpow_eq_pow]

theorem ker_rpow_le_of_nonneg {A : HermitianMat d ℂ} (hA : 0 ≤ A) :
  (A ^ r).ker ≤ A.ker := by
  apply ker_cfc_le_ker_nonneg hA

```

```

grind [Real.rpow_eq_zero_iff_of_nonneg, Real.rpow_eq_pow]

private lemma rpow_kron_diagonal
  (a : d → ℝ) (b : d₂ → ℝ) (r : ℝ) (ha : ∀ i, 0 ≤ a i) (hb : ∀ j, 0 ≤ b j) :
  ((diagonal ℂ a) ⊗ₖ (diagonal ℂ b)) ^ r =
  ((diagonal ℂ a) ^ r) ⊗ₖ ((diagonal ℂ b) ^ r) := by
  simp only [kronecker_diagonal, rpow_diagonal]
  congr! 2 with x
  apply Real.mul_rpow (ha x.1) (hb x.2)

open scoped Kronecker in
omit [DecidableEq d] [DecidableEq d₂] in
lemma conj_kron
  (A : Matrix d d ℂ) (B : Matrix d₂ d₂ ℂ) (C : HermitianMat d ℂ) (D : HermitianMat d₂ ℂ) :
  conj (A ⊗ₖ B) (C ⊗ₖ D) = conj A C ⊗ₖ conj B D := by
  ext1
  simp [conj, Matrix.mul_kronecker_mul, Matrix.conjTranspose_kronecker]

lemma rpow_kron
  {A : HermitianMat d ℂ} {B : HermitianMat d₂ ℂ} (r : ℝ) (hA : 0 ≤ A) (hB : 0 ≤ B) :
  (A ⊗ₖ B) ^ r = (A ^ r) ⊗ₖ (B ^ r) := by
  obtain ⟨U, a, ha, hA⟩ : ∃ U : [d], ∃ a : d → ℝ, (∀ i, 0 ≤ a i) ∧ A = conj U.val (
    rw [zero_le_iff] at ha
    exact ⟨_, _, hA.eigenvalues_nonneg, eq_conj_diagonal A⟩
  )
  obtain ⟨V, b, hb, hB⟩ : ∃ V : [d₂], ∃ b : d₂ → ℝ, (∀ j, 0 ≤ b j) ∧ B = conj V.val (
    rw [zero_le_iff] at hb
    exact ⟨_, _, hB.eigenvalues_nonneg, eq_conj_diagonal B⟩
  )
  have h_kron_r_pow : (A ⊗ₖ B) ^ r = conj (Matrix.unitary_kron U V).val ((diagonal ℂ
    subst hB hA
    rw [← rpow_conj_unitary, Matrix.unitary_kron, conj_kron]
  )
  rw [h_kron_r_pow]
  subst A B
  have h_kron_r_pow_diag : (diagonal ℂ a ⊗ₖ diagonal ℂ b) ^ r = ((diagonal ℂ a) ^ r)
    exact rpow_kron_diagonal a b r ha hb
  rw [h_kron_r_pow_diag, Matrix.unitary_kron]
  rw [rpow_conj_unitary, rpow_conj_unitary, ← conj_kron]

attribute [fun_prop] ContinuousAt.rpow ContinuousOn.rpow

lemma continuousOn_rpow_uniform {K : Set ℝ} (hK : IsCompact K) :
  ContinuousOn (fun r : ℝ ↦ UniformOnFun.ofFun {K} (fun t : ℝ ↦ t ^ r)) (Set.Ioi 0)
  refine continuousOn_of_forall_continuousAt fun r hr => ?_
  rw [Set.mem_Ioi] at hr
  apply UniformOnFun.tendsto_iff_tendstoUniformlyOn.mpr
  simp only [Set.mem_singleton_iff, UniformOnFun.toFun_ofFun, Metric.tendstoUniformlyOn,
    Function.comp_apply, forall_eq]

```

```

intro  $\varepsilon$  h $\varepsilon$ pos;
have h_unif_cont : UniformContinuousOn (fun (p :  $\mathbb{R} \times \mathbb{R}$ ) => p.1 ^ p.2) (K)
  apply IsCompact.uniformContinuousOn_of_continuous
  · exact hK.prod CompactIccSpace.isCompact_Icc
  · refine continuousOn_of_forall_continuousAt fun p {hp1, {hp21, hp22}}
    have _ : p.1  $\neq$  0  $\vee$  0 < p.2 := by right; linarith
    fun_prop (disch := assumption)
rw [Metric.uniformContinuousOn_iff] at h_unif_cont
obtain { $\delta$ , h $\delta$ pos, H} := h_unif_cont  $\varepsilon$  h $\varepsilon$ pos
filter_upwards [Ioo_mem_nhds (show r / 2 < r by linarith) (show r < r * 2 by linarith)]
refine H (x, r) {hx, ?_} (x, n) {hx, ?_} ?_
· constructor <=> linarith
· constructor <=> linarith
· have : |r - n| <  $\delta$  := abs_lt.mpr (by linarith, by linarith)
  simp

```

section continuity

```

/-- Joint continuity of matrix rpow for Hermitian matrices with positive exponents -/
@[fun_prop]
theorem continuousOn_rpow_joint_nonneg_pos
  {X : Type*} [TopologicalSpace X]
  {A : X  $\rightarrow$  HermitianMat d  $\mathbb{C}$ } {p : X  $\rightarrow$   $\mathbb{R}$ } {S : Set X}
  (hA : ContinuousOn A S) (hp : ContinuousOn p S)
  (hp_pos :  $\forall x \in S, 0 < p x$ ) :
  ContinuousOn (fun x  $\mapsto$  A x ^ p x) S := by
  have h_cont_f : ContinuousOn (fun q : X  $\times$   $\mathbb{R}$  => q.2 ^ p q.1) (S  $\times^s$  Set.univ) := by
    -- fun_prop (disch := grind [Set.MapsTo]) --After BUMP - fix in discharge
    apply continuousOn_snd.rpow
    · exact hp.comp continuousOn_fst (fun x  $\mapsto$  And.left)
    · grind
  simp_rw [rpow_eq_cfc]
  fun_prop (disch := simp)

```

end continuity

```

/-
For a positive Hermitian matrix A, (A^2)^(p/2) = A^p, expressed using function notation.
TODO: Cleanup, generalize...
-/
theorem cfc_sq_rpow_eq_cfc_rpow
  (A : HermitianMat d  $\mathbb{C}$ ) (hA : 0  $\leq$  A) (p :  $\mathbb{R}$ ) (hp : 0 < p) :
  (A ^ 2).cfc (fun x => x ^ (p/2)) = A.cfc (fun x => x ^ p) := by
  have h_sqrt : (A ^ 2).cfc (fun x => x ^ (p / 2)) = (A.cfc (fun x => x ^ 2) ^ p).cfc
    convert rfl;

```



```

    exact cfc_pow A;
  rw [ h_sqrt ];
  have h_sqrt : ∀ (f g : ℝ → ℝ), Continuous f → Continuous g → ∀ (A : HermitianMat d ℂ)
    exact fun f g a a A => Eq.symm (cfc_comp_apply A f g);
  rw [ h_sqrt ];
  · have h_sqrt : ∀ x : ℝ, 0 ≤ x → (x ^ 2) ^ (p / 2) = x ^ p := by
    intro x hx
    rw [ ← Real.rpow_natCast, ← Real.rpow_mul hx ]
    ring_nf
    exact cfc_congr_of_nonneg hA h_sqrt;
  · continuity;
  · exact continuous_id.rpow_const fun x => 0r.inr <| by positivity

/-- Tr[A^p] = ∑i λi^p for a Hermitian matrix A. -/
lemma trace_rpow_eq_sum (A : HermitianMat d ℂ) (p : ℝ) :
  (A ^ p).trace = ∑ i, (A.H.eigenvalues i) ^ p := by
  exact A.trace_cfc_eq (· ^ p)

/-! ## Loewner-Heinz Theorem
The operator monotonicity of `x ↦ x ^ q` for `0 < q ≤ 1`:
if `A ≤ B` (in the Loewner order), then `A ^ q ≤ B ^ q`.
This is proved using the resolvent integral representation, following the same
approach as `log_mono` in `LogExp.lean`. The key identity is:
`x ^ q = c_q * ∫ t in (0,∞), t ^ (q-1) * x / (x + t) dt`
where `c_q = sin(π q) / π`. Since each integrand `x / (x + t)` is operator
monotone (via `inv_antitone`), the integral is operator monotone.
-/
section LoewnerHeinz

variable {A B : HermitianMat d ℂ} {q : ℝ}

open MeasureTheory ComplexOrder Filter in
/-- Finite integral approximation for the rpow monotonicity proof.
    Same integrand as `logApprox` but with weight `t ^ q`. -
/
noncomputable def rpowApprox (A : HermitianMat d ℂ) (q T : ℝ) : HermitianMat d ℂ :=
  ∫ t in (0)..T, t ^ q • ((1 + t)-1 • (1 : HermitianMat d ℂ) - (A + t • 1)-1)

set_option maxHeartbeats 800000 in
open MeasureTheory ComplexOrder in
theorem rpowApprox_mono {A B : HermitianMat d ℂ} (hA : A.mat.PosDef) (hB : B.mat.PosDef)
  (hAB : A ≤ B) (hq : 0 ≤ q) (T : ℝ) (hT : 0 < T) :
  rpowApprox A q T ≤ rpowApprox B q T := by
  unfold HermitianMat.rpowApprox
  have h_integral_mono : ∀m t ∂Measure.restrict volume (Set.Ioc 0 T), t ^ q • ((1 +
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with t ht

```

```

have h_inv_antitone :  $(B + t \cdot 1)^{-1} \leq (A + t \cdot 1)^{-1}$  := by
  apply inv_antitone
    · exact hA.add_posSemidef ( Matrix.PosSemidef.smul ( Matrix.PosSemidef.one hA ) hB )
    · exact add_le_add_left hAB _
  exact smul_le_smul_of_nonneg_left (sub_le_sub_left h_inv_antitone _) (R
rw [ intervalIntegral.integral_of_le hT.le, intervalIntegral.integral_of_
refine integral_mono_ae ?_ ?_ h_integral_mono
· refine ContinuousOn.integrableOn_Icc ?_ |> fun h => h.mono_set <| Set.I
have h_cont : ContinuousOn (fun t :  $\mathbb{R}$  =>  $t^q$ ) (Set.Icc 0 T) := by
  exact continuousOn_id.rpow_const fun x hx => 0r.inr <| by linarith;
have h_cont_inv : ContinuousOn (fun t :  $\mathbb{R}$  =>  $(A + t \cdot 1 : \text{Matrix } d \ d \ \mathbb{C})$ )
  have h_cont_inv :  $\forall t \in \text{Set.Icc } 0 \ T, (A + t \cdot 1 : \text{Matrix } d \ d \ \mathbb{C}).\text{PosDef}$ 
    intro t ht
    exact hA.add_posSemidef (Matrix.PosSemidef.one.smul ht.1)
  have h_cont_inv : ContinuousOn (fun t :  $\mathbb{R}$  =>  $(A + t \cdot 1 : \text{Matrix } d \ d \ \mathbb{C})$ )
    have h_det_cont : ContinuousOn (fun t :  $\mathbb{R}$  =>  $\text{Matrix.det } (A + t \cdot 1)$ )
      fun_prop (disch := solve_by_elim)
    have h_adj_cont : ContinuousOn (fun t :  $\mathbb{R}$  =>  $\text{Matrix.adjugate } (A + t \cdot 1)$ )
      fun_prop (disch := solve_by_elim)
    simp_all [Matrix.inv_def]
    exact ContinuousOn.smul ( h_det_cont.inv₀ fun t ht => by specialize
  exact h_cont_inv
have h_cont_inv : ContinuousOn (fun t :  $\mathbb{R}$  =>  $(1 + t)^{-1} \cdot (1 : \text{Hermitian } d \ d \ \mathbb{C})$ )
  exact ContinuousOn.sub ( ContinuousOn.smul ( ContinuousOn.inv₀ ( cont
convert h_cont.smul h_cont_inv using 1
ext
exact continuousOn_iff_coe _
· have h_cont_tq : ContinuousOn (fun t :  $\mathbb{R}$  =>  $t^q$ ) (Set.Icc 0 T) := by
  exact continuousOn_id.rpow_const fun x hx => 0r.inr hq
have h_cont_inv : ContinuousOn (fun t :  $\mathbb{R}$  =>  $(B + t \cdot 1)^{-1}$ ) (Set.Icc 0 T)
  have h_cont_inv :  $\forall t \in \text{Set.Icc } 0 \ T, (B + t \cdot 1).\text{mat.PosDef}$  := by
    intro t ht
    exact hB.add_posSemidef (Matrix.PosSemidef.one.rsmul ht.1)
  have h_det_cont : ContinuousOn (fun t :  $\mathbb{R}$  =>  $(B + t \cdot 1).\text{mat.det}$ ) (Se
    fun_prop (disch := solve_by_elim)
  have h_adj_cont : ContinuousOn (fun t :  $\mathbb{R}$  =>  $(B + t \cdot 1).\text{mat.adjugate}$ )
    fun_prop (disch := solve_by_elim)
  have h_inv_cont : ContinuousOn (fun t :  $\mathbb{R}$  =>  $(B + t \cdot 1).\text{mat}^{-1}$ ) (Set.
    have h_inv_cont :  $\forall t \in \text{Set.Icc } 0 \ T, (B + t \cdot 1).\text{mat}^{-1} = (B + t \cdot 1)$ 
      intro t ht
      simp [Matrix.inv_def]
    exact ContinuousOn.congr ( ContinuousOn.smul ( h_det_cont.inv₀ fun
  exact (continuousOn_iff_coe fun t =>  $(B + t \cdot 1)^{-1}$ ).mpr h_inv_cont
exact (h_cont_tq.smul (((continuousOn_const.add continuousOn_id).inv₀
  continuousOn_const).sub h_cont_inv)).integrableOn_Icc.mono_set (Set.I

```

```

open MeasureTheory ComplexOrder in
/-- The scalar function underlying rpowApprox via the CFC. -
/
noncomputable def scalarRpowApprox (q T x : ℝ) : ℝ :=
  ∫ t in (0)..T, t ^ q * (1 / (1 + t) - 1 / (x + t))

open MeasureTheory ComplexOrder in
theorem rpowApprox_eq_cfc_scalar (A : HermitianMat d ℂ) (hA : A.mat.PosDef) (q T : ℝ)
  (hq : 0 ≤ q) (hT : 0 < T) :
  rpowApprox A q T = A.cfc (scalarRpowApprox q T) := by
  have rpowApprox_eq_cfc_scalar : ∀ t ∈ Set.Ioc 0 T, t ^ q * ((1 + t)-1 * (1 : HermitianMat d ℂ) - (A + t • 1)-1) = A.cfc
    intro t ht
  have h_integrand : ((1 + t)-1 * (1 : HermitianMat d ℂ) - (A + t • 1)-1) = A.cfc
    have h_integrand : (A + t • 1)-1 = A.cfc (fun u => (u + t)-1) := by
      have h_inv : (A + t • 1)-1 = A.cfc (fun u => (u + t)-1) := by
        have h_inv_def : (A + t • 1)-1 = (A.cfc (fun u => u + t))-1 := by
          rw [ show ( fun u => u + t ) = ( fun u => u ) + fun u => t from rfl, cfc ]
        have h_inv_comp : (A.cfc (fun u => u + t))-1 = A.cfc (fun u => (u + t)-1)
        have h_inv_smul : ∀ {f : ℝ → ℝ} (hf : ∀ i, f (A.H.eigenvalues i) ≠ 0), (
          exact fun {f} hf => inv_cfc_eq_cfc_inv f hf
        )
        apply h_inv_smul
      intro i
      have h_eigenvalue_pos : 0 < A.H.eigenvalues i := by
        exact Matrix.PosDef.eigenvalues_pos hA i
      exact ne_of_gt (add_pos h_eigenvalue_pos ht.left);
    rw [h_inv_def, h_inv_comp];
  exact h_inv
  rw [ h_integrand, ← cfc_const ];
  rw [ ← cfc_sub ];
  rfl;
  aesop;
-- Apply the fact that the integral of a CFC is the CFC of the integral.
have rpowApprox_integral_eq : ∫ t in (0)..T, A.cfc (fun u => t ^ q * (1 / (1 + t)
  have h_integrable : ∀ u : d, IntervalIntegrable (fun t : ℝ => t ^ q * (1 / (1 + t)
    intro u
  have h_integrable : IntervalIntegrable (fun t : ℝ => t ^ q * (1 / (1 + t) - 1
    have h_pos : 0 < A.H.eigenvalues u := by
      exact Matrix.PosDef.eigenvalues_pos hA u
    exact ContinuousOn.intervalIntegrable ( by exact ContinuousOn.mul ( continuo
  exact h_integrable
  exact integral_cfc_eq_cfc_integral _ _ h_integrable
unfold HermitianMat.rpowApprox scalarRpowApprox; simp_all +singlePass;
rw [ ← rpowApprox_integral_eq, intervalIntegral.integral_of_le hT.le, integral_Ioc
rw [ setIntegral_congr_fun measurableSet_Ioc fun t ht => rpowApprox_eq_cfc_scalar
simp [ ← integral_Ioc_eq_integral_Ioc, intervalIntegral.integral_of_le hT.le ]

```

```

/-- The positive constant arising from the resolvent integral.
    Equal to  $\int u \text{ in } \text{Set.Ioi } 0, u^{(q-1)} / (1+u) = \pi / \sin(\pi q)$ ,
    but we only need its positivity. -/
noncomputable def rpowConst (q : ℝ) : ℝ :=
   $\int u \text{ in } \text{Set.Ioi } (0 : \mathbb{R}), (u^{(q-1)} / (1+u)) : \mathbb{R}$ 

open MeasureTheory in
/- The integrand  $u^{(q-1)} / (1+u)$  is integrable on  $(0, \infty)$  for  $0 < q < 1$ 
/
lemma rpowConst_integrableOn (hq : 0 < q) (hq1 : q < 1) :
  IntegrableOn (fun u : ℝ =>  $u^{(q-1)} / (1+u)$ ) (Set.Ioi 0) := by
  rw [← Set.Ioc_union_Ioi_eq_Ioi_zero_le_one]
  apply IntegrableOn.union
  · have h_integrable_0_1 : IntegrableOn (fun u : ℝ =>  $u^{(q-1)}$ ) (Set.Ioi 0) := by
      exact (intervalIntegral.intervalIntegrable_rpow' (by linarith)).1
      apply h_integrable_0_1.mono'
      · apply Measurable.astronglyMeasurable
        fun_prop
      · filter_upwards [ae_restrict_mem measurableSet_Ioc] with x hx
        rw [Real.norm_of_nonneg (div_nonneg (Real.rpow_nonneg hx.1.le _))
          exact div_le_self (Real.rpow_nonneg hx.1.le _)] (by linarith [hx.1])
    · have h_bound :  $\forall u : \mathbb{R}, 1 \leq u \rightarrow u^{(q-1)} / (1+u) \leq u^{(q-2)}$  :=
        intro u hu
        rw [div_le_iff₀ (by positivity)]
        ring_nf
        apply le_add_of_nonneg_of_le (by positivity)
        rw [← Real.rpow_add_one (by positivity)]
        ring_nf
        rfl
      have h_integrable : IntegrableOn (fun u : ℝ =>  $u^{(q-2)}$ ) (Set.Ioi 1) := by
        rw [integrableOn_Ioi_rpow_iff zero_lt_one]
        linarith
      apply h_integrable.mono'
      · apply Measurable.astronglyMeasurable
        fun_prop
      · filter_upwards [ae_restrict_mem measurableSet_Ioi] with u hu
        have _ := hu.out
        rw [Real.norm_of_nonneg (by positivity)]
        exact h_bound u hu.out.le

open MeasureTheory in
/- The resolvent constant is positive. -/
lemma rpowConst_pos (hq : 0 < q) (hq1 : q < 1) : 0 < rpowConst q := by
  unfold rpowConst;
  have h_nonzero : 0 <  $\int u \text{ in } \text{Set.Ioi } (0 : \mathbb{R}), u^{(q-1)} / (1+u) := by
    have h_integrable : IntegrableOn (fun u : ℝ =>  $u^{(q-1)} / (1+u)$ ) ($ 
```

```

    exact rpowConst_integrableOn hq hq1
  rw [ integral_pos_iff_support_of_nonneg_ae ];
  · simp [Function.support]
    exact lt_of_lt_of_le ( by norm_num ) ( measure_mono <| show Set.Ioi ( 0 : ℝ ) )
  · filter_upwards [ ae_restrict_mem measurableSet_Ioi ] with u hu using div_nonneg
  · exact h_integrable;
  linarith

open MeasureTheory Filter in
/-- The scalar rpow approximation converges pointwise.
  `scalarRpowApprox q T x → rpowConst q * (x^q - 1)` as `T → ∞`. -
/
lemma scalarRpowApprox_tendsto {x : ℝ} (hx : 0 < x) (hq : 0 < q) (hq1 : q < 1) :
  Filter.Tendsto (fun T => scalarRpowApprox q T x) atTop (nhds (rpowConst q * (x ^ q))) :=
  have h_def : ∀ T > 0, scalarRpowApprox q T x = x * (∫ t in (0)..T, t ^ (q - 1) / (1 + t)) / (1 + 0)
  intro T hT
  have : ∀ t ∈ Set.Ioc (0 : ℝ) T, t ^ q * (1 / (1 + t) - 1 / (x + t)) = x * (t ^ (q - 1) / (1 + t) - t ^ (q - 1) / (x + t))
  intro t ht; rw [ Real.rpow_sub_one ht.1.ne' ]
  grind
  rw [ intervalIntegral.integral_of_le hT.le, intervalIntegral.integral_of_le hT.le ]
  rw [ ← integral_const_mul, ← integral_sub ];
  · exact Eq.trans ( intervalIntegral.integral_of_le hT.le ) ( setIntegral_congr_f )
  · apply Integrable.const_mul _ _;
    apply Integrable.mono' (g := fun t => t ^ (q - 1) / x)
  · exact ( intervalIntegral.intervalIntegrable_rpow' ( by linarith ) ).1.div_const
  · apply Measurable.aestronglyMeasurable
    fun_prop
  · filter_upwards [ ae_restrict_mem measurableSet_Ioc ] with t ht using by rw [
  · apply Integrable.mono' (g := fun t => t ^ (q - 1) / (1 + 0))
  · exact ( intervalIntegral.intervalIntegrable_rpow' ( by linarith ) ).1.div_const
  · apply Measurable.aestronglyMeasurable
    fun_prop
  · filter_upwards [ ae_restrict_mem measurableSet_Ioc ] with t ht using by rw [
  have h_int_1 : Filter.Tendsto (fun T => ∫ t in (0)..T, t ^ (q - 1) / (1 + t)) Filt.atTop (nhds 0)
  apply intervalIntegral_tendsto_integral_Ioi
  · exact rpowConst_integrableOn hq hq1
  · exact Filter.tendsto_id
  have h_int_x : Filter.Tendsto (fun T => ∫ t in (0)..T, t ^ (q - 1) / (x + t)) Filt.atTop (nhds 0)
  have h_subst : ∀ T > 0, ∫ t in (0)..T, t ^ (q - 1) / (x + t) = x ^ (q - 1) * ∫ u in (0)..T / x, (x * u) ^ (q - 1)
  intro T hT
  have h_subst : ∫ t in (0)..T, t ^ (q - 1) / (x + t) = ∫ u in (0)..T / x, (x * u) ^ (q - 1)
  simp [ mul_comm x, intervalIntegral.integral_comp_mul_right ( fun u => u ^ (q - 1) ) ]
  rw [ inv_mul_eq_div, div_mul_cancel₀ _ hx.ne' ];
  rw [ h_subst, ← intervalIntegral.integral_const_mul ];
  refine intervalIntegral.integral_congr fun u hu => ?_
  rw [ Real.mul_rpow ( by positivity ) ( by cases Set.mem_uIcc.mp hu <=> nlinarith ) ]

```

```

      field_simp
      rw [ Filter.tendsto_congr' ( by filter_upwards [ Filter.eventually_gt_atTop ]
        simp [ mul_comm ] using h_int_1.comp ( Filter.tendsto_id.atTop_div_congr'
      rw [ Filter.tendsto_congr' ( by filter_upwards [ Filter.eventually_gt_atTop ]
      convert Filter.Tendsto.sub ( h_int_x.const_mul x ) h_int_1 using 2
      ring_nf
      rw [ mul_assoc, ← Real.rpow_one_add' hx.le ]
      · simp
      · linarith

open MeasureTheory ComplexOrder Filter in
/-- The matrix rpow approximation converges: `rpowApprox A q T → rpowConst q A` -/
lemma tendsto_rpowApprox (hA : A.mat.PosDef) (hq : 0 < q) (hq1 : q < 1) :
  Tendsto (rpowApprox A q) atTop (nhds (rpowConst q • (A ^ q - 1))) := by
  have h_target : rpowConst q • (A ^ q - 1) = A.cfc (fun x => rpowConst q *
    have h2 : A.cfc (fun x => rpowConst q * (x ^ q - 1)) = rpowConst q • A.
      HermitianMat.cfc_const_mul A (fun x => x ^ q - 1) (rpowConst q)
    have h3 : A.cfc (fun x => x ^ q - 1) = A.cfc (· ^ q) - 1 := by
      conv_rhs => rw [show (1 : HermitianMat d ℂ) = A.cfc (fun _ => (1 : ℝ))
        exact cfc_sub_apply A (f := (· ^ q)) (g := fun _ => 1)
      rw [h2, h3, rpow_eq_cfc]
    have h_eq : ∀f T in atTop, rpowApprox A q T = A.cfc (scalarRpowApprox q T)
      filter_upwards [Filter.eventually_gt_atTop 0] with T hT
      exact rpowApprox_eq_cfc_scalar A hA q T hq.le hT
    rw [Filter.tendsto_congr' h_eq, h_target]
    have h_expand_src : ∀ T, (A.cfc (scalarRpowApprox q T)).mat = ∑ i, scalar
      fun T => cfc_toMat_eq_sum_smul_proj A (scalarRpowApprox q T)
    have h_expand_tgt : (A.cfc (fun x => rpowConst q * (x ^ q - 1))).mat = ∑
      cfc_toMat_eq_sum_smul_proj A (fun x => rpowConst q * (x ^ q - 1))
    have h_sum : Tendsto (fun T : ℝ => ∑ i, scalarRpowApprox q T (A.H.eigenvalues_pos i))
      refine tendsto_finset_sum _ fun i _ => ?_
      have := scalarRpowApprox_tendsto (hA.eigenvalues_pos i) hq hq1
      exact Filter.Tendsto.smul_const (Complex.continuous_ofReal.continuousAt
    open scoped Matrix.Norms.Frobenius in
    rw [tendsto_iff_norm_sub_tendsto_zero] at *
    convert h_sum using 2 with T
    show ||(A.cfc (scalarRpowApprox q T)).mat - (A.cfc (fun x => rpowConst q *
    rw [h_expand_src, h_expand_tgt]

open MeasureTheory ComplexOrder Filter in
theorem rpow_le_rpow_of_posDef (hA : A.mat.PosDef) (hAB : A ≤ B)
  (hq : 0 < q) (hq1 : q ≤ 1) : A ^ q ≤ B ^ q := by
  by_cases hq_eq_one : q = 1;
  · aesop;
  · have h_rpow : rpowConst q • (A ^ q - 1) ≤ rpowConst q • (B ^ q - 1) :=

```

```

    convert le_of_tendsto_of_tendsto ( tendsto_rpowApprox hA hq ( lt_of_le_of_ne h
    generalize_proofs at *; (
    filter_upwards [ Filter.eventually_gt_atTop 0 ] with T hT using rpowApprox_mon
have h_rpow_pos : 0 < rpowConst q := by
  exact rpowConst_pos hq ( lt_of_le_of_ne hq1 hq_eq_one );
simp_all

open ComplexOrder Filter in
/-- The **Löwner–Heinz theorem**: for matrices A and B, if  $0 \leq A \leq B$  and  $0 < q \leq 1$ 
then  $A^q \leq B^q$ . That is, real roots are operator monotone. -
/
theorem rpow_le_rpow_of_le (hA : 0 ≤ A) (hAB : A ≤ B)
  (hq : 0 < q) (hq1 : q ≤ 1) : A ^ q ≤ B ^ q := by
  -- For  $\varepsilon > 0$ , let  $A_\varepsilon = A + \varepsilon \cdot 1$  and  $B_\varepsilon = B + \varepsilon \cdot 1$ .
  set Aε : ℝ → HermitianMat d ℂ := fun ε => A + ε • 1
  set Bε : ℝ → HermitianMat d ℂ := fun ε => B + ε • 1
  -- For  $\varepsilon > 0$ ,  $A_\varepsilon$  and  $B_\varepsilon$  are positive definite and  $A_\varepsilon \leq B_\varepsilon$ .
  have h_pos_def : ∀ ε > 0, (Aε ε).mat.PosDef ∧ (Bε ε).mat.PosDef ∧ Aε ε ≤ Bε ε := by
    intro ε hε_pos
    have h_pos_def_Aε : (Aε ε).mat.PosDef := by
      rw [Matrix.posDef_iff_dotProduct_mulVec] at ⊢
      constructor <|> norm_num [ hε_pos, hA, hAB ];
      · exact H (Aε ε)
      · intro x hx_nonzero
        have h_inner : star x []_v (Aε ε).mat.mulVec x = star x []_v A.mat.mulVec x + ε
          simp [ Aε, Matrix.add_mulVec ]
          ring_nf
          simp [ Matrix.mulVec, dotProduct, Finset.mul_sum _ _ _, mul_assoc, mul_left_comm, mul_one ]
        have h_inner_nonneg : 0 ≤ star x []_v A.mat.mulVec x := by
          exact inner_mulVec_nonneg hA x
        have h_inner_pos : 0 < star x []_v x := by
          simp_all
          exact h_inner.symm ▸ add_pos_of_nonneg_of_pos h_inner_nonneg ( mul_pos ( mod
    have h_pos_def_Bε : (Bε ε).mat.PosDef := by
      convert posDef_of_posDef_le h_pos_def_Aε _ using 1
      exact add_le_add_left hAB _ |> le_trans ( by simp [ Aε ] ) ;
    have h_le_Aε_Bε : Aε ε ≤ Bε ε := by
      exact add_le_add_left hAB _ |> le_trans <| by simp [ Bε ] ;
    exact (h_pos_def_Aε, h_pos_def_Bε, h_le_Aε_Bε)
  -- By the continuity of the function  $M \mapsto M^q$ , we have  $(A_\varepsilon)^q \rightarrow A^q$  and  $(B_\varepsilon)^q \rightarrow B^q$ .
  have h_cont : Filter.Tendsto (fun ε => (Aε ε) ^ q) (nhdsWithin 0 (Set.Ioi 0)) (nhds A ^ q) := by
    have h_cont : ContinuousOn (fun M : HermitianMat d ℂ => M ^ q) (Set.univ : Set (HermitianMat d ℂ)) := by
      -- Apply the continuity of the function  $M \mapsto M^q$  on the set of all Hermitian matrices.
      apply rpow_const_continuous hq.le |> Continuous.continuousOn
    refine' ( h_cont.continuousAt ( by simp ) |> fun h => h.tendsto.comp ( tendsto_n

```

```

-- By the continuity of the function  $M \mapsto M^q$ , we have  $(A \varepsilon \varepsilon)^q \leq (B \varepsilon \varepsilon)^q$ 
have h_le :  $\forall \varepsilon > 0, (A \varepsilon \varepsilon)^q \leq (B \varepsilon \varepsilon)^q$  := by
  exact fun  $\varepsilon h\varepsilon \Rightarrow$  rpow_le_rpow_of_posDef ( h_pos_def  $\varepsilon h\varepsilon |>.1$  ) ( h_pos_def
  exact le_of_tendsto_of_tendsto h_cont.1 h_cont.2 ( Filter.eventually_of_monotone )
end LoewnerHeinz

section ArakiLiebThirring

variable {A B : HermitianMat d  $\mathbb{C}$ } {q r :  $\mathbb{R}$ }

/-- An inequality of Lieb-Thirring type. For  $0 < r \leq 1$ :
 $\text{Tr}[B^r A^r B^r] \leq \text{Tr}[(B A B)^r]$ .
-/
lemma lieb_thirring_le_one
  {A B : HermitianMat d  $\mathbb{C}$ } (hA :  $0 \leq A$ ) (hB :  $0 \leq B$ )
  {r :  $\mathbb{R}$ } (hq0 :  $0 < r$ ) (hq1 :  $q \leq r$ ) :
   $((A^r).conj (B^r).mat).trace \leq ((A.conj B.mat)^r).trace$  := by
  sorry

end ArakiLiebThirring

/-
Trace subadditivity (Rotfel'd inequality): for PSD A, B and  $0 < p \leq 1$ ,
 $\text{Tr}[(A + B)^p] \leq \text{Tr}[A^p] + \text{Tr}[B^p]$ .

This isn't needed for anything else in the repository atm, but it would
be nice to have as a fact.

A stronger version states it as a majorization theorem. See
e.g. https://www.math.uwaterloo.ca/~hwolkowi/henry/reports/thesis mingmay613
-/
lemma trace_rpow_add_le
  {A B : HermitianMat d  $\mathbb{C}$ } (hA :  $0 \leq A$ ) (hB :  $0 \leq B$ )
  (p :  $\mathbb{R}$ ) (hp :  $0 < p$ ) (hp1 :  $p \leq 1$ ) :
   $((A + B)^p).trace \leq (A^p).trace + (B^p).trace$  := by
  sorry

```

1.466 QuantumInfo/ForMathlib/HermitianMat/Schatten.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg

```



```

(Matrix.eigenvalues_conjTranspose_mul_self_nonneg A _) ) _ ) ( sq_nonneg

lemma SchattenNorm_pow_eq
  (A : HermitianMat d ℂ) (hA : 0 ≤ A) (p k : ℝ) (hp : 0 < p) (hk : 0 < k) :
    SchattenNorm (A ^ k).mat p = (SchattenNorm A.mat (k * p)) ^ k := by
  rw [ SchattenNorm_hermitian_pow, SchattenNorm_hermitian_pow ] <;> try pos
  · rw [ ← Real.rpow_mul ] <;> ring_nf <;> norm_num [ hp.ne', hk.ne' ];
  · rw [ mul_comm, ← HermitianMat.rpow_mul ];
  exact hA;
  · -- Since $A$ is positive, $A^{\{k*p\}}$ is also positive, and the trace o
negative.
    have h_pos : 0 ≤ A ^ (k * p) := by
      exact HermitianMat.rpow_nonneg hA;
    exact HermitianMat.trace_nonneg h_pos;
  · exact HermitianMat.rpow_nonneg hA

lemma trace_eq_SchattenNorm_rpow
  (A : HermitianMat d ℂ) (hA : 0 ≤ A) (r : ℝ) (hr : 0 < r) :
    (A ^ r).trace = (SchattenNorm A.mat r) ^ r := by
  rw [SchattenNorm_hermitian_pow hA hr, ← Real.rpow_mul] <;> norm_num [hr.n
  apply HermitianMat.trace_nonneg
  exact HermitianMat.rpow_nonneg hA

/-! ## Relating SchattenNorm to singular values -/

/- The trace of cfc(A†A, t ↦ t^{p/2}) expressed as a sum of eigenvalues. -
/
lemma SchattenNorm_trace_as_eigenvalue_sum (A : Matrix d d ℂ) (p : ℝ) :
  RCLike.re ((Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose).
    ∑ i : d, ((Matrix.isHermitian_mul_conjTranspose_self A.conjTranspose).e
  rw [ Matrix.IsHermitian.cfc ];
  simp [ Matrix.trace_mul_comm, Matrix.mul_assoc ]

/-
The Schatten p-norm raised to the p-th power equals the sum of singular val
raised to the p-th power: `||A||_p^p = ∑ σ_i(A)^p`.
-/
lemma SchattenNorm_rpow_eq_sum_singularValues (A : Matrix d d ℂ) {p : ℝ} (h
  SchattenNorm A p ^ p = ∑ i : d, singularValues A i ^ p := by
  unfold SchattenNorm;
  rw [ ← Real.rpow_mul ( _ ), one_div_mul_cancel hp.ne', Real.rpow_one ];
  · convert SchattenNorm_trace_as_eigenvalue_sum A p using 1
    refine' Finset.sum_congr rfl fun i _ => _;
  unfold singularValues; rw [ Real.sqrt_eq_rpow, ← Real.rpow_mul ( _ ) ]
  ring_nf
  simp +zetaDelta at *;

```

```

    exact Matrix.eigenvalues_conjTranspose_mul_self_nonneg A i;
    · have h_nonneg :  $\forall i : d, 0 \leq ((\text{Matrix.isHermitian\_mul\_conjTranspose\_self } A.\text{conjT}$ 
      exact fun i => Real.rpow_nonneg ( by have := Matrix.eigenvalues_conjTranspose_
      convert Finset.sum_nonneg fun i _ => h_nonneg i using 1;
      convert schattenNorm_trace_as_eigenvalue_sum A p using 1

/- The Schatten p-norm equals the  $\ell^p$  quasi-norm of the singular values:
`||A||_p = ( $\sum \sigma_i(A)^p$ )1/p. -/
lemma schattenNorm_eq_sum_singularValues_rpow (A : Matrix d d  $\mathbb{C}$ ) {p :  $\mathbb{R}$ } (hp : 0 < p) :
  schattenNorm A p = ( $\sum i : d, \text{singularValues } A i ^ p$ ) ^ (1/p) := by
  rw [←schattenNorm_rpow_eq_sum_singularValues A hp];
  rw [← Real.rpow_mul ( by exact Real.rpow_nonneg ( by
    simp [ Matrix.trace ];
    refine' Finset.sum_nonneg fun i _ => _;
    rw [ Matrix.IsHermitian.cfc ];
    simp [ Matrix.mul_apply, Matrix.diagonal ];
    field_simp;
    exact Finset.sum_nonneg fun _ _ => mul_nonneg ( Real.rpow_nonneg ( by
      exact Matrix.eigenvalues_conjTranspose_mul_self_nonneg A _ ) _ ) ( add_nonneg

/-- `||A||_p^p` equals the same sum over sorted singular values. -
/
lemma schattenNorm_rpow_eq_sum_sorted (A : Matrix d d  $\mathbb{C}$ ) {p :  $\mathbb{R}$ } (hp : 0 < p) :
  schattenNorm A p ^ p =
     $\sum i : \text{Fin } (\text{Fintype.card } d), \text{singularValuesSorted } A i ^ p$  := by
  rw [schattenNorm_rpow_eq_sum_singularValues A hp]
  exact sum_singularValues_rpow_eq_sum_sorted A p

open InnerProductSpace in
/--
Scalar trace Young inequality for PSD matrices:
 $\|A\|_p \|B\|_q \leq \text{Tr}[A^p]/p + \text{Tr}[B^q]/q$  for PSD A, B and conjugate p, q > 1.
-/
lemma HermitianMat.trace_young
  (A B : HermitianMat d  $\mathbb{C}$ ) (hA : 0 ≤ A) (hB : 0 ≤ B)
  (p q :  $\mathbb{R}$ ) (hp : 1 < p) (hpq : 1/p + 1/q = 1) :
   $\|A\|_p \|B\|_q \leq (\text{A}^p).\text{trace} / p + (\text{B}^q).\text{trace} / q$  := by
  --TODO Cleanup
  have h_schatten :  $\forall (i j : d), (\text{A.H.eigenvalues } i) * (\text{B.H.eigenvalues } j) \leq (\text{A.H.ei}$ 
    intro i j
  have h_young :  $\forall (a b : \mathbb{R}), 0 \leq a \rightarrow 0 \leq b \rightarrow (1 < p \rightarrow 1 / p + 1 / q = 1 \rightarrow a * b \leq$ 
    intro a b ha hb hp hpq
  have h_young :  $a * b \leq (a^p) / p + (b^q) / q$  := by
    have h_conj : 1 / p + 1 / q = 1 := hpq
    have h_pos : 0 < p  $\wedge$  0 < q := by
      use zero_lt_one.trans hp

```

```

      refine lt_of_not_ge fun h => ?_
      rw [ div_eq_mul_inv, div_eq_mul_inv ] at h_conj
      nlinarith [inv_nonpos.2 h, inv_mul_cancel₀ (by linarith : p ≠ 0)]
      have := @Real.geom_mean_le_arith_mean
      specialize this { 0, 1 } ( fun i => if i = 0 then p⁻¹ else q⁻¹ ) (
      simp only [ div_eq_inv_mul ] using this h_pos.1.le h_pos.2.le ( Re
    exact h_young
  refine h_young _ _ ?_ ?_ hp hpq
  · exact (zero_le_iff.mp hA).eigenvalues_nonneg _
  · exact (zero_le_iff.mp hB).eigenvalues_nonneg _
convert Finset.sum_le_sum fun i _ => Finset.sum_le_sum fun j _ => mul_le_
convert HermitianMat.inner_eq_doubly_stochastic_sum A B using 1;
simp [ Finset.sum_add_distrib, add_mul, Finset.mul_sum, div_eq_mul_inv, m
simp [ ← Finset.mul_sum, ← Finset.sum_comm, ];
congr! 2;
· refine Finset.sum_congr rfl fun i _ => ?_
  have := Matrix.unitary_row_sum_norm_sq ( A.H.eigenvectorUnitary.val.con
  · rw [ this, mul_one ];
  · simp [ Matrix.mul_assoc ];
    simp [ ← Matrix.mul_assoc, Matrix.IsHermitian.eigenvectorUnitary ];
· refine' Finset.sum_congr rfl fun i _ => _;
  have := Matrix.unitary_col_sum_norm_sq ( A.H.eigenvectorUnitary.val.con
  simp [ ← Matrix.mul_assoc, Matrix.IsHermitian.eigenvectorUnitary ]

/-- For PSD `A` and Hermitian `B`, the product
`C = A^{1/2} * B` satisfies `C^* C = (A.conj B.mat).mat = B * A * B`. -
/
lemma conjTranspose_half_mul_eq_conj
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) :
  ((A ^ (1/2 : ℝ)).mat * B.mat).conjTranspose * ((A ^ (1/2 : ℝ)).mat * B.
  = (A.conj B.mat).mat := by
  have := HermitianMat.pow_half_mul hA; simp_all [ ← mul_assoc ] ;
  simp only [mul_assoc, this]

lemma schattenNorm_half_mul_rpow_eq_trace_conj
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A)
  {α : ℝ} (hα : 0 < α) :
  (schattenNorm ((A ^ (1/2 : ℝ)).mat * B.mat) (2 * α)) ^ (2 * α) =
  ((A.conj B.mat) ^ α).trace := by
  have h_conj : ((A ^ (1 / 2 : ℝ)).mat * B.mat).conjTranspose * ((A ^ (1 /
  exact conjTranspose_half_mul_eq_conj hA;
  unfold schattenNorm;
  rw [ ← Real.rpow_mul ] <=> norm_num [ hα.ne' ];
  · ring_nf; norm_num [ hα.ne' ];
  rw [ ← Matrix.IsHermitian.cfc_eq ];
  rw [ Matrix.conjTranspose_conjTranspose ];

```

```

exact congrArg Complex.re (congrArg Matrix.trace (congrArg (cfc fun x => x ^ α)
· have h_eigenvalues_nonneg : ∀ i, 0 ≤ (Matrix.isHermitian_mul_conjTranspose_self
  intro i; exact (by
    have := Matrix.eigenvalues_conjTranspose_mul_self_nonneg ( ( A ^ ( 1 / 2 : ℝ )
    simp_all [ Matrix.trace, Matrix.IsHermitian.cfc ];
    simp_all [ Matrix.mul_apply, Matrix.diagonal ];
    refine' Finset.sum_nonneg fun i _ => Finset.sum_nonneg fun j _ => _;
    field_simp;
    exact mul_nonneg ( Real.rpow_nonneg ( h_eigenvalues_nonneg j ) _ ) (by positiv

```

```

/-!

```

```

## Schatten–Hölder inequality

```

The *Schatten–Hölder inequality* for matrix products:

For matrices `A`, `B` and exponents `r, p, q > 0` with $\frac{1}{r} = \frac{1}{p} + \frac{1}{q}$, the Schatten r -norm of the product satisfies

$$\|A * B\|_{S^r} \leq \|A\|_{S^p} * \|B\|_{S^q}.$$

This version includes the quasi-norm case ($r, p, q < 1$).

```

### Proof sketch

```

The proof proceeds in three steps:

1. Express Schatten norms in terms of singular values:
 $\|A\|_p = (\sum \sigma_i(A)^p)^{1/p}$.
2. Use the *weak log-majorization* of singular values of products
 (`weakLogMaj_singularValues_mul` + `sum_rpow_le_of_weakLogMaj``) to obtain
 $\sum \sigma_i(AB)^r \leq \sum \sigma_{\downarrow i}(A)^r \cdot \sigma_{\downarrow i}(B)^r$.
3. Apply the *classical Hölder inequality* for finite sums
 (`NNReal.inner_le_Lp_mul_Lq`` from Mathlib, with conjugate exponents
 $\frac{p}{r}$ and $\frac{q}{r}$) to bound
 $\sum \sigma_{\downarrow i}(A)^r \cdot \sigma_{\downarrow i}(B)^r \leq (\sum \sigma_i(A)^p)^{r/p} \cdot (\sum \sigma_i(B)^q)^{r/q}$.
4. Take $\frac{1}{r}$ -th powers and combine.

```

-/

```

```

lemma schattenNorm_mul_le (A B : Matrix d d ℂ) {r p q : ℝ}
  (hr : 0 < r) (hp : 0 < p) (hq : 0 < q) (hpqr : 1 / r = 1 / p + 1 / q) :
  schattenNorm (A * B) r ≤ schattenNorm A p * schattenNorm B q := by
-- It suffices to show the inequality for r-th powers, since  $x \mapsto x^{1/r}$  is monotone
rw [schattenNorm_eq_sum_singularValues_rpow (A * B) hr,
    schattenNorm_eq_sum_singularValues_rpow A hp,
    schattenNorm_eq_sum_singularValues_rpow B hq]
-- Rewrite sums over d to sums over Fin (Fintype.card d) via sorted singular values
rw [sum_singularValues_rpow_eq_sum_sorted (A * B) r,
    sum_singularValues_rpow_eq_sum_sorted A p,
    sum_singularValues_rpow_eq_sum_sorted B q]
-- Now we need:
--  $(\sum \sigma_{\downarrow i}(AB)^r)^{1/r} \leq (\sum \sigma_{\downarrow i}(A)^p)^{1/p} \cdot (\sum \sigma_{\downarrow i}(B)^q)^{1/q}$ 

```

```

-- Step 1: From sum_rpow_singularValues_mul_le, we have
--  $\sum \sigma_i(AB)^r \leq \sum \sigma_i(A)^r \cdot \sigma_i(B)^r$ 
have h_sv_ineq := sum_rpow_singularValues_mul_le A B hr
-- Step 2: From holder_step_for_singularValues, we have
--  $\sum \sigma_i(A)^r \cdot \sigma_i(B)^r \leq (\sum \sigma_i(A)^p)^{r/p} \cdot (\sum \sigma_i(B)^q)^{r/q}$ 
have h_holder := holder_step_for_singularValues A B hr hp hq hpqr
-- Step 3: Combine and take 1/r-th power
-- Need:  $(\sum \sigma_i(AB)^r)^{1/r} \leq ((\sum \sigma_i(A)^p)^{r/p})^{1/r} \cdot ((\sum \sigma_i(B)^q)^{r/q})^{1/r}$ 
--  $= (\sum \sigma_i(A)^p)^{1/p} \cdot (\sum \sigma_i(B)^q)^{1/q}$ 
have h_combined :  $\sum i, \text{singularValuesSorted } (A * B) \ i \ ^r \leq$ 
  ( $\sum i, \text{singularValuesSorted } A \ i \ ^p$ ) ^ (r / p) *
  ( $\sum i, \text{singularValuesSorted } B \ i \ ^q$ ) ^ (r / q) :=
  le_trans h_sv_ineq h_holder
-- Take 1/r-th power of both sides
have h_rpow :  $(\sum i, \text{singularValuesSorted } (A * B) \ i \ ^r)^{1/r} \leq$ 
  ( $(\sum i, \text{singularValuesSorted } A \ i \ ^p)^{r/p}$ ) ^ (1/r) *
  ( $(\sum i, \text{singularValuesSorted } B \ i \ ^q)^{r/q}$ ) ^ (1/r) := by
  apply Real.rpow_le_rpow
  · exact Finset.sum_nonneg fun i _ =>
    Real.rpow_nonneg (singularValuesSorted_nonneg _ _) _
  · exact h_combined
  · positivity
-- Simplify the RHS:  $(X^{r/p} \cdot Y^{r/q})^{1/r} = X^{1/p} \cdot Y^{1/q}$ 
have h_simplify :
  ( $(\sum i, \text{singularValuesSorted } A \ i \ ^p)^{r/p}$ ) ^ (1/r) *
  ( $(\sum i, \text{singularValuesSorted } B \ i \ ^q)^{r/q}$ ) ^ (1/r) =
  ( $\sum i, \text{singularValuesSorted } A \ i \ ^p$ ) ^ (1/p) *
  ( $\sum i, \text{singularValuesSorted } B \ i \ ^q$ ) ^ (1/q) := by
  have hsp :  $0 \leq (\sum i, \text{singularValuesSorted } A \ i \ ^p)^{r/p} :=$ 
    Real.rpow_nonneg (Finset.sum_nonneg fun i _ =>
      Real.rpow_nonneg (singularValuesSorted_nonneg _ _) _ ) _
  have hsq :  $0 \leq (\sum i, \text{singularValuesSorted } B \ i \ ^q)^{r/q} :=$ 
    Real.rpow_nonneg (Finset.sum_nonneg fun i _ =>
      Real.rpow_nonneg (singularValuesSorted_nonneg _ _) _ ) _
  rw [Real.mul_rpow hsp hsq]
  have hsp' :  $(0 : \mathbb{R}) \leq \sum i, \text{singularValuesSorted } A \ i \ ^p :=$ 
    Finset.sum_nonneg fun i _ => Real.rpow_nonneg (singularValuesSorted_n
  have hsq' :  $(0 : \mathbb{R}) \leq \sum i, \text{singularValuesSorted } B \ i \ ^q :=$ 
    Finset.sum_nonneg fun i _ => Real.rpow_nonneg (singularValuesSorted_n
  congr 1 <;> rw [← Real.rpow_mul (by assumption)] <;> congr 1 <;> field_
  linarith

lemma HermitianMat.trace_rpow_conj_le
  {A B : HermitianMat d ℂ} (hA : 0 ≤ A) (hB : 0 ≤ B)
  {α p q : ℝ} (hα : 0 < α) (hp : 0 < p) (hq : 0 < q)
  (hpq : 1 / (2 * α) = 1 / p + 1 / q) :

```

```

    ((A.conj B.mat) ^  $\alpha$ ).trace ≤
    (((A ^ (p / 2)).trace) ^ (1 / p) * ((B ^ q).trace) ^ (1 / q)) ^ (2 *  $\alpha$ ) := by
-- Raise both sides of the inequality to the power of  $2\alpha$ .
have h_exp : ((A.conj B.mat) ^  $\alpha$ ).trace ≤ (schattenNorm (A ^ (1 / 2 :  $\mathbb{R}$ )).mat p *
  have h_exp : (schattenNorm ((A ^ (1 / 2 :  $\mathbb{R}$ )).mat * B.mat) (2 *  $\alpha$ )) ^ (2 *  $\alpha$ ) =
    exact schattenNorm_half_mul_rpow_eq_trace_conj hA h $\alpha$ 
  rw [← h_exp]
  -- Apply the Schatten-Hölder inequality to the matrices  $A^{1/2} * B$ .
  refine Real.rpow_le_rpow ?_ (schattenNorm_mul_le _ _ (by positivity) hp hq hpq)
  exact schattenNorm_nonneg _ _
  rw [schattenNorm_hermitian_pow (rpow_nonneg hA) hp, schattenNorm_hermitian_pow hB]
have h_exp_simp : (A ^ (1 / 2 :  $\mathbb{R}$ )) ^ p = A ^ (p / 2 :  $\mathbb{R}$ ) := by
  rw [← HermitianMat.rpow_mul hA]
  ring_nf
  rw [h_exp_simp] at h_exp
  exact h_exp

```

1.467 QuantumInfo/ForMathlib/HermitianMat/Sqrt.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Proj

variable {d  $\square$  : Type*} [Fintype d] [DecidableEq d] [RCLike  $\square$ ]
variable {A B : HermitianMat d  $\square$ } {f g :  $\mathbb{R} \rightarrow \mathbb{R}$ }

noncomputable section

open scoped MatrixOrder ComplexOrder Matrix Kronecker

namespace HermitianMat

/-- The square root of a Hermitian matrix. Negative eigenvalues are mapped to zero.
/
noncomputable def sqrt (A : HermitianMat d  $\square$ ) : HermitianMat d  $\square$  :=
  A.cfc Real.sqrt

theorem sqrt_sq_eq_proj (A : HermitianMat d  $\square$ ) :
  A.sqrt.mat * A.sqrt.mat = A+ := by
  rw [sqrt, ← mat_cfc_mul, ← HermitianMat.ext_iff, posPart_eq_cfc_ite]
  congr! 2 with x

```

```

    grind [Pi.mul_apply, Real.mul_self_sqrt, Real.sqrt_eq_zero']

theorem sqrt_sq (hA : 0 ≤ A) :
  A.sqrt.mat * A.sqrt.mat = A := by
  rw [sqrt_sq_eq_proj, posPart_eq_self hA]

@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem commute_sqrt_left (hAB : Commute A.mat B.mat) :
  Commute A.sqrt.mat B.mat := by
  rw [sqrt]
  commutes

@[aesop unsafe apply 50% (rule_sets := [Commutes])]
theorem commute_sqrt_right (hAB : Commute A.mat B.mat) :
  Commute A.mat B.sqrt.mat := by
  commutes

/--
For a positive definite matrix A,  $A^{-1/2} * A * A^{-1/2} = I$ .
-/
lemma sqrt_inv_mul_self_mul_sqrt_inv_eq_one {A : HermitianMat d} (hA : A.
  A⁻¹.sqrt.mat * A.mat * A⁻¹.sqrt.mat = 1 := by
  have h_inv_def : A⁻¹.sqrt.mat * A⁻¹.sqrt.mat = A⁻¹ := by
    apply HermitianMat.sqrt_sq
    rw [zero_le_iff]
    exact hA.inv.posSemidef
  have h_inv_comm : Commute A⁻¹.sqrt.mat A.mat := by
    commutes
  rw [h_inv_comm, mul_assoc, h_inv_def]
  apply Matrix.mul_nonsing_inv
  exact isUnit_iff_ne_zero.mpr hA.det_pos.ne'

theorem sqrt_nonneg (A : HermitianMat d) : 0 ≤ A.sqrt := by
  rw [sqrt, cfc_nonneg_iff]
  intro; positivity

theorem sqrt_pos (h : 0 < A) : 0 < A.sqrt := by
  rw [sqrt]
  apply cfc_pos_of_pos h (by intros; positivity) (by simp)

theorem sqrt_posDef {A : HermitianMat d} (hA : A.mat.PosDef) :
  A.sqrt.mat.PosDef := by
  rw [sqrt, cfc_posDef]
  simp [hA.eigenvalues_pos]

open Lean Meta Mathlib.Meta.Positivity in

```



```

/-- Positivity extension for `HermitianMat.sqrt` -/
@[positivity HermitianMat.sqrt _]
def evalHermitianMatSqrt : PositivityExt where eval {_u _α} _zα _pα e := do
  let .app _sqrt (A : Expr) ← whnfR e | throwError "not sqrt application"
  try
    let (isStrictA, pfA) ← bestResult A
    if isStrictA then
      pure (.positive (← mkAppM ``HermitianMat.sqrt_pos #[pfA]))
    else
      throwError "Not strictly positive, falling back to nonnegativity"
  catch _ =>
    pure (.nonnegative (← mkAppM ``HermitianMat.sqrt_nonneg #[A]))

example {A : HermitianMat d ℂ} : 0 ≤ A.sqrt := by
  positivity

example [Nonempty d] {A : HermitianMat d ℂ} : 0 < (1 + A.sqrt).sqrt := by
  positivity

```

1.468 QuantumInfo/ForMathlib/HermitianMat/Trace.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Reindex

```

```

/-! # Trace of Hermitian Matrices

```

While the trace of a Hermitian matrix is, in informal math, typically just "the same a matrix that happens to be Hermitian - it is a real number, not a complex number. 0 it is a self-adjoint element of the base `StarAddMonoid`.

Working directly with `Matrix.trace` then means that there would be constant casts b chasing imaginary parts and inequalities and so on. By defining `HermitianMat.trace` operation, we encapsulate the mess and give a clean interface.

The `IsMaximalSelfAdjoint` class is used so that (for example) for matrices over $\mathbb{Z}$ or `HermitianMat.trace` works as well and is in fact defeq to `Matrix.trace`. For $\mathbb{C}$ or it uses the real part.

```

-/

```

```

namespace HermitianMat

```

```

variable {R n m  $\alpha$  : Type*} [Star R] [TrivialStar R] [Fintype n] [Fintype m]

section star
variable [AddGroup  $\alpha$ ] [StarAddMonoid  $\alpha$ ] [CommSemiring R] [Semiring  $\alpha$ ] [Algebra R  $\alpha$ ]

/-- The trace of the matrix. This requires a ` $\text{IsMaximalSelfAdjoint } R \ \alpha$ ` instance of
`HermitianMat n  $\alpha$ ` to ` $R$ `. This means that the trace of (say) a ` $\text{HermitianMat } n \ \mathbb{C}$ `
but that the trace of a ` $\text{HermitianMat } n \ \mathbb{C}$ ` gives values in  $\mathbb{R}$ . The fact that the
real reduces coercions down the line. -/
def trace (A : HermitianMat n  $\alpha$ ) : R :=
  IsMaximalSelfAdjoint.selfadjMap (A.mat.trace)

/-- ` $\text{HermitianMat.trace}$ ` reduces to ` $\text{Matrix.trace}$ ` in the algebra. -/
/
theorem trace_eq_trace (A : HermitianMat n  $\alpha$ ) : algebraMap R  $\alpha$  A.trace = Matrix.trace A.mat
  rw [trace, Matrix.trace, map_sum, map_sum]
  congr! 1
  exact IsMaximalSelfAdjoint.selfadj_algebra (Matrix.IsHermitian.apply A.H)

variable [StarModule R  $\alpha$ ] in
@[simp]
theorem trace_smul (A : HermitianMat n  $\alpha$ ) (r : R) : (r • A).trace = r * A.trace
  simp [trace, IsMaximalSelfAdjoint.selfadj_smul]

end star
section semiring
variable [CommSemiring R] [Ring  $\alpha$ ] [StarAddMonoid  $\alpha$ ] [Algebra R  $\alpha$ ] [IsMaximalSelfAdjoint R  $\alpha$ ]

@[simp]
theorem trace_zero : (0 : HermitianMat n  $\alpha$ ).trace = 0 := by
  simp [trace]

@[simp]
theorem trace_add (A B : HermitianMat n  $\alpha$ ) : (A + B).trace = A.trace + B.trace
  simp [trace]

end semiring
section ring
variable [CommRing R] [Ring  $\alpha$ ] [StarAddMonoid  $\alpha$ ] [Algebra R  $\alpha$ ] [IsMaximalSelfAdjoint R  $\alpha$ ]

@[simp]
theorem trace_neg (A : HermitianMat n  $\alpha$ ) : (-A).trace = -A.trace := by
  simp [trace]

```

```

@[simp]
theorem trace_sub (A B : HermitianMat n  $\alpha$ ) : (A - B).trace = A.trace - B.trace := by
  simp [trace]

end ring
section starring

variable [CommRing R] [CommRing  $\alpha$ ] [StarRing  $\alpha$ ] [Algebra R  $\alpha$ ] [IsMaximalSelfAdjoint  $\alpha$ ]

--Move somewhere else? Needs to import `IsMaximalSelfAdjoint`, so maybe just here.
theorem _root_.Matrix.IsHermitian.isSelfAdjoint_trace {A : Matrix n n  $\alpha$ } (hA : A.IsHermitian)
  IsSelfAdjoint A.trace := by
  simp [Matrix.trace, IsSelfAdjoint, ← Matrix.star_apply, show star A = A from hA]

variable (A : HermitianMat m  $\alpha$ ) (B : HermitianMat n  $\alpha$ )

@[simp]
theorem trace_kronecker [FaithfulSMul R  $\alpha$ ] : (A  $\otimes_k$  B).trace = A.trace * B.trace := by
  apply FaithfulSMul.algebraMap_injective R  $\alpha$ 
  simp only [trace, kronecker_mat]
  rw [Matrix.trace_kronecker A.mat B.mat]
  simp only [map_mul]
  have hA := A.H.isSelfAdjoint_trace
  have hB := B.H.isSelfAdjoint_trace
  open IsMaximalSelfAdjoint in
  rw [selfadj_algebra hA, selfadj_algebra hB, selfadj_algebra (hA.mul hB)]

end starring

section trivialstar

variable [Star  $\alpha$ ] [TrivialStar  $\alpha$ ] [CommSemiring  $\alpha$ ]

/-- `HermitianMat.trace` reduces to `Matrix.trace` when the elements are a `TrivialStar` /
@[simp]
theorem trace_eq_trace_trivial (A : HermitianMat n  $\mathbb{R}$ ) : A.trace = A.mat.trace := by
  rw [← trace_eq_trace]
  rfl

end trivialstar

section RCLike

variable {n m  $\alpha$  : Type*} [Fintype n] [Fintype m] [RCLike  $\alpha$ ]

```

```

theorem trace_eq_re_trace (A : HermitianMat n  $\mathbb{R}$ ) : A.trace = RCLike.re A.mat
  rfl

@[simp]
theorem trace_one [DecidableEq n] : (1 : HermitianMat n  $\mathbb{R}$ ).trace = Fintype.
  simp [trace_eq_re_trace]

/-- `HermitianMat.trace` reduces to `Matrix.trace` when the elements are  $\mathbb{R}$ 
/
@[simp]
theorem trace_eq_trace_rc (A : HermitianMat n  $\mathbb{R}$ ) : A.trace = A.mat.trace :=
  rw [trace, Matrix.trace, map_sum, RCLike.ofReal_sum]
  congr 1
  exact Matrix.IsHermitian.coe_re_diag A.H

theorem trace_diagonal {T : Type*} [Fintype T] [DecidableEq T] (f : T  $\rightarrow$   $\mathbb{R}$ ) :
  (diagonal  $\square$  f).trace =  $\sum i, f i$  := by
  rw [trace_eq_re_trace]
  simp [HermitianMat.diagonal, Matrix.trace]

theorem sum_eigenvalues_eq_trace [DecidableEq n] (A : HermitianMat n  $\mathbb{R}$ ) :
   $\sum i, A.H.eigenvalues i = A.trace$  := by
  convert congrArg RCLike.re A.H.sum_eigenvalues_eq_trace
  rw [RCLike.ofReal_re]

--Proving that traces are 0 or 1 is common enough that we have a convenient
--statements about HermitianMat traces into Matrix traces.
theorem trace_eq_zero_iff (A : HermitianMat n  $\mathbb{R}$ ) : A.trace = 0  $\leftrightarrow$  A.mat.trace = 0
  rw [← trace_eq_trace_rc]
  exact <mod_cast id, mod_cast id>

theorem trace_eq_one_iff (A : HermitianMat n  $\mathbb{R}$ ) : A.trace = 1  $\leftrightarrow$  A.mat.trace = 1
  rw [← trace_eq_trace_rc]
  exact <mod_cast id, mod_cast id>

@[simp]
theorem trace_reindex (A : HermitianMat n  $\mathbb{C}$ ) (e : n  $\approx$  m) :
  (A.reindex e).trace = A.trace := by
  simp [reindex, trace_eq_re_trace]

end RCLike
section partialTrace
section addCommGroup

variable [AddCommGroup  $\alpha$ ] [StarAddMonoid  $\alpha$ ]
omit [Fintype n]

```

```
def traceLeft (A : HermitianMat (m × n) α) : HermitianMat n α :=
  ⟨A.mat.traceLeft, A.H.traceLeft⟩
```

```
def traceRight (A : HermitianMat (m × n) α) : HermitianMat m α :=
  ⟨A.mat.traceRight, A.H.traceRight⟩
```

```
variable (A B : HermitianMat (m × n) α)
```

```
@[simp]
```

```
theorem traceLeft_mat : A.traceLeft.mat = A.mat.traceLeft := by
  rfl
```

```
@[simp]
```

```
theorem traceLeft_add : (A + B).traceLeft = A.traceLeft + B.traceLeft := by
  ext1; simp
```

```
@[simp]
```

```
theorem traceLeft_neg : (-A).traceLeft = -A.traceLeft := by
  ext1; simp
```

```
@[simp]
```

```
theorem traceLeft_sub : (A - B).traceLeft = A.traceLeft - B.traceLeft := by
  ext1; simp
```

```
variable (A B : HermitianMat (n × m) α)
```

```
@[simp]
```

```
theorem traceRight_mat :
  (traceRight A).mat = A.mat.traceRight := by
  rfl
```

```
@[simp]
```

```
theorem traceRight_add : (A + B).traceRight = A.traceRight + B.traceRight := by
  ext1; simp
```

```
@[simp]
```

```
theorem traceRight_neg : (-A).traceRight = -A.traceRight := by
  ext1; simp
```

```
@[simp]
```

```
theorem traceRight_sub : (A - B).traceRight = A.traceRight - B.traceRight := by
  ext1; simp
```

```
end addCommGroup
```

```
section rcLike
```

```

variable {} [RCLike {}]
variable (A : HermitianMat (m × n) {})

omit [Fintype n] in
@[simp]
theorem traceLeft_smul (r : ℝ) : (r • A).traceLeft = r • A.traceLeft := by
  ext1; simp

omit [Fintype m] in
@[simp]
theorem traceRight_smul (r : ℝ) : (r • A).traceRight = r • A.traceRight :=
  ext1; simp

@[simp]
theorem traceLeft_trace : A.traceLeft.trace = A.trace := by
  simp [trace_eq_re_trace]

@[simp]
theorem traceRight_trace : A.traceRight.trace = A.trace := by
  simp [trace_eq_re_trace]

end rCLike
section kron

variable {m n {} : Type*} [RCLike {}]
variable (A : HermitianMat m {}) (B : HermitianMat n {})

@[simp]
theorem traceLeft_kron [Fintype m] : (A ⊗k B).traceLeft = A.trace • B := by
  ext : 2
  simp only [HermitianMat.traceLeft, Matrix.traceLeft, kronecker_mat, mat_m]
  simp [Matrix.trace, RCLike.real_smul_eq_coe_mul, ← Finset.sum_mul]

@[simp]
theorem traceRight_kron [Fintype n] : (A ⊗k B).traceRight = B.trace • A :=
  ext : 2
  simp only [HermitianMat.traceRight, Matrix.traceRight, kronecker_mat, mat_m]
  simp [Matrix.trace, RCLike.real_smul_eq_coe_mul, ← Finset.mul_sum, mul_co]

end kron
end partialTrace

```

1.469 QuantumInfo/ForMathlib/HermitianMat/Unitary.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat.Inner
import QuantumInfo.ForMathlib.HermitianMat.NonSingular
import QuantumInfo.ForMathlib.Isometry

notation "⌈[" n "]" => Matrix.unitaryGroup n ℂ

namespace Matrix

variable {α : Type*} [NonUnitalNonAssocSemiring α] [StarRing α]

variable {α β : Type*} [DecidableEq α] [Fintype α] [DecidableEq β] [Fintype β]

@[simp]
theorem neg_unitary_val (u : ⌈[α]) : (-u).val = -u := by
  rfl

omit [DecidableEq α] [Fintype α] [DecidableEq β] [Fintype β] in
open Kronecker in
@[simp]
theorem star_kron (a : Matrix α α ℂ) (b : Matrix β β ℂ) : star (a ⊗k b) = (star a) ⊗
  ext _ _
  simp

open Kronecker in
theorem kron_unitary (a : ⌈[α]) (b : ⌈[β]) : a.val ⊗k b.val ∈ ⌈[α × β] := by
  simp [Matrix.mem_unitaryGroup_iff, ← Matrix.mul_kronecker_mul]

open Kronecker in
def unitary_kron (a : ⌈[α]) (b : ⌈[β]) : ⌈[α × β] :=
  ⟨_, kron_unitary a b⟩

scoped infixl:60 " ⊗u " => unitary_kron

@[simp]
theorem unitary_kron_apply (a : ⌈[α]) (b : ⌈[β]) (i1 i2 : α) (j1 j2 : β) :
  (a ⊗u b) (i1, j1) (i2, j2) = (a i1 i2) * (b j1 j2) := by
  rfl

@[simp]

```

```

theorem unitary_kron_one_one : (1 :  $\mathbb{C}[\alpha]$ )  $\otimes_u$  (1 :  $\mathbb{C}[\beta]$ ) = (1 :  $\mathbb{C}[\alpha \times \beta]$ ) :=
  simp [Matrix.unitary_kron]

--TODO: Cleanup? Or at least write the signature better. Definitely belongs
--This is really a statement about isometries, not unitaries, too...
variable {d : Type*} [Fintype d] [DecidableEq d]

/--
For a unitary matrix C, the row sums of  $\|C\ i\ j\|^2$  equal 1.
-/
lemma unitary_row_sum_norm_sq (C : Matrix d d  $\mathbb{C}$ ) (hC : C * C.conjTranspose
   $\sum j, \|C\ i\ j\|^2 = 1$  := by
  replace hC := congr( $\$hC\ i\ i$ )
  simp only [Matrix.mul_apply, Matrix.conjTranspose_apply, RCLike.star_def,
    Complex.normSq_eq_norm_sq, Complex.ofReal_pow, Matrix.one_apply_eq] at
  exact_mod_cast hC

/--
For a unitary matrix C, the column sums of  $\|C\ i\ j\|^2$  equal 1.
-/
lemma unitary_col_sum_norm_sq (C : Matrix d d  $\mathbb{C}$ ) (hC : C.conjTranspose * C
   $\sum i, \|C\ i\ j\|^2 = 1$  := by
  replace hC := congr( $\$hC\ j\ j$ )
  simp_rw [Matrix.mul_apply, mul_comm] at hC
  simp only [Matrix.conjTranspose_apply, RCLike.star_def, Matrix.one_apply,
    Complex.mul_conj, Complex.normSq_eq_norm_sq, Complex.ofReal_pow] at hC
  exact_mod_cast hC

end Matrix

```

1.470 QuantumInfo/ForMathlib/IsMaximalSelfAdjoint.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.Matrix.Normed

/- We want to have `HermitianMat.trace` give 0 when 0 is already a TrivialStar
otherwise -- for instance,  $\mathbb{R}$  when the input field is  $\mathbb{C}$ . This typeclass lets
/
class IsMaximalSelfAdjoint (R : outParam Type*) ( $\alpha$  : Type*) [Star  $\alpha$ ] [Star
  [Semiring  $\alpha$ ] [TrivialStar R] [Algebra R  $\alpha$ ] where

```



```

selfadjMap :  $\alpha \rightarrow + R$ 
selfadj_smul :  $\forall (r : R) (a : \alpha), \text{selfadjMap } (r \cdot a) = r * (\text{selfadjMap } a)$ 
selfadj_algebra :  $\forall \{a : \alpha\}, \text{IsSelfAdjoint } a \rightarrow \text{algebraMap } \_ \_ (\text{selfadjMap } a) = a$ 

/-- Every `TrivialStar` `CommSemiring` is its own maximal self adjoints. -
/
instance instTrivialStar_IsMaximalSelfAdjoint {R} [Star R] [TrivialStar R] [CommSemiring R] : IsMaximalSelfAdjoint R :=
  { selfadjMap := AddMonoidHom.id R
    selfadj_smul _ _ := rfl
    selfadj_algebra { _ } _ := rfl }

/--  $\mathbb{R}$  is the maximal self adjoint elements over RCLike -/
instance instRCLike_IsMaximalSelfAdjoint { $\alpha$ } [RCLike  $\alpha$ ] : IsMaximalSelfAdjoint  $\mathbb{R} \alpha$  :=
  { selfadjMap := RCLike.re
    selfadj_smul := RCLike.smul_re
    selfadj_algebra := RCLike.conj_eq_iff_re.mp }

namespace IsMaximalSelfAdjoint

-- In particular instances we care about, simplify selfadjMap should it appear.
-- It _seems_ like `selfadjMap 1 = 1`, always, but I can't find a proof. But these 1
-- take care of proving that anyway.

@[simp]
theorem trivial_selfadjMap {R} [Star R] [TrivialStar R] [CommSemiring R] : (selfadjMap 1) = 1 := rfl

@[simp]
theorem RCLike_selfadjMap { $\alpha$ } [RCLike  $\alpha$ ] : (selfadjMap :  $\alpha \rightarrow + \mathbb{R}$ ) = RCLike.re := by
  rfl

end IsMaximalSelfAdjoint

```

1.471 QuantumInfo/ForMathlib/Isometry.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.InnerProductSpace.JointEigenspace
import Mathlib.Analysis.SpecialFunctions.Bernstein
import Mathlib.Analysis.SpecialFunctions.Pow.NNReal
import Mathlib.LinearAlgebra.Matrix.Permutation

```

```

import Mathlib.LinearAlgebra.Matrix.IsDiag
import Mathlib.Tactic.NormNum.GCD

import QuantumInfo.ForMathlib.Matrix

open scoped Matrix

variable {d d₂ d₃ R : Type*}
variable [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂] [Fintype d₃]
variable [CommRing R] [StarRing R]

variable {α : Type*} [RCLike α] {A B : Matrix d d α}

/-- An isometry is a matrix `A` such that `AA^H = 1`. Compare with a unitary
requires `AA^H = A^H A = 1`. It is common to claim that, in a finite-
dimensional vector
space, a two-sided isometry (`A.Isometry ∧ A^H.Isometry`) must be square and
this is does not work out so well here, since a `Matrix m n R` can be a two-
sided isometry,
but cannot be a `unitary` since the rows and columns are index by different
/
def Matrix.Isometry (A : Matrix d d₂ R) : Prop :=
  A^H * A = 1

omit [Fintype d₃] [DecidableEq d₂] in
theorem Matrix.submatrix_one_isometry {e : d₂ → d} {f : d₃ → d} (he : e.Bij)
  (submatrix (α := R) 1 e f).Isometry := by
  -- Since $e$ is injective and $f$ is bijective, the submatrix of the iden
  have h_perm : ∀ i j, (Matrix.submatrix (1 : Matrix d d R) e f) i j = if e
  -- By definition of the identity matrix, the entry (i, j) in the submat
  simp [Matrix.submatrix, Matrix.one_apply]
  ext i j
  -- Since $e$ is injective and $f$ is bijective, the product $A * A^H$ will
  change ∑ k, (Matrix.conjTranspose (Matrix.submatrix (1 : Matrix d d R) e
  (Matrix.submatrix (1 : Matrix d d R) e f) k j = if i = j then 1 else 0
  simp_all only [Multiset.bijective_iff_map_univ_eq_univ, submatrix_apply,
  symm; split <=> symm
  next h =>
    subst h
    simp_all only [implies_true, mul_ite, ↑reduceIte, star_one, mul_one, st
    Finset.sum_boole]
    have h_unique : ∀ i, ∃! x, e x = f i := by
      intro i
      obtain ⟨x, hx⟩ : ∃ x, e x = f i := by
        replace he := congr_arg Multiset.toFinset he; rw [Finset.ext_iff] a

```

```

    use x
    simp_all only [true_and]
    intro y a
    have := Fintype.bijective_iff_injective_and_card e
    aesop
    obtain { x, hx } := h_unique i;
    rw [show ( Finset.univ.filter fun y => e y = f i ) = { x } from Finset.eq_single
next h => -- Since  $e$  is injective and  $e\ i \neq e\ j$ , there is no  $x$  such that
    have h_no_x :  $\forall x : d_2, \neg(e\ x = f\ i \wedge e\ x = f\ j)$  := by
      exact fun x hx => h ( hf ( hx.1.symm.trans hx.2 ) );
    exact Finset.sum_eq_zero fun x hx => by specialize h_no_x x; aesop

omit [DecidableEq d2] in
theorem Matrix.submatrix_one_id_left_isometry {e : d2 → d} (he : e.Bijective) :
  (submatrix (1 : Matrix d d R) e id).Isometry :=
  submatrix_one_isometry he Function.injective_id

omit [Fintype d2] in
theorem Matrix.submatrix_one_id_right_isometry {e : d2 → d} (he : e.Injective) :
  (submatrix (1 : Matrix d d R) id e).Isometry :=
  submatrix_one_isometry Function.bijective_id he

theorem Matrix.mem_unitaryGroup_iff_isometry (A : Matrix d d R) :
  A ∈ unitaryGroup d R ↔ A.Isometry ∧ AH.Isometry := by
  rw [Isometry, Isometry, conjTranspose_conjTranspose]
  rfl

theorem Equiv.Perm.permMatrix_mem_unitaryGroup (e : Perm d) :
  e.permMatrix R ∈ Matrix.unitaryGroup d R := by
  -- Since  $e$  is a permutation, its permutation matrix  $P_e$  is orthogonal, meaning
  have h_perm_ortho : (Equiv.Perm.permMatrix R e) * (Equiv.Perm.permMatrix R e)T = 1
  ext i j; rw [Matrix.mul_apply] ; aesop;
  constructor
  · simp_all only [Matrix.transpose_permMatrix]
    -- Since the conjugate transpose of a permutation matrix is the permutation matrix
    have h_conj_transpose : star (Equiv.Perm.permMatrix R e) = (Equiv.Perm.permMatrix R e)
    ext i j; simp [Equiv.Perm.permMatrix] ; aesop;
    simp_all [mul_eq_one_comm]
  · simp_all only [Matrix.transpose_permMatrix]
    convert h_perm_ortho using 2;
    simp [Matrix.star_eq_conjTranspose, Equiv.Perm.permMatrix]

omit [Fintype d3] [DecidableEq d2] in
theorem Matrix.reindex_one_isometry (e : d ≈ d2) (f : d ≈ d3) :
  (reindex (α := R) e f 1).Isometry := by
  -- Since  $e$  and  $f$  are bijections, the reindexing of the identity matrix by  $e$ 

```

```

have h_perm : ∀ (e : d ≈ d₂) (f : d ≈ d₃), (Matrix.reindex e f (1 : Matrix d d R)) = 1 := by
  intro e f
  simp [Matrix.Isometry]
  exact h_perm e f

omit [Fintype d] in
theorem Matrix.reindex_one_mem_unitaryGroup (e : d ≈ d₂) :
  reindex (α := R) e e 1 ∈ unitaryGroup d₂ R := by
  -- The reindex of the identity matrix under an equivalence e is just the
  have h_reindex_id : Matrix.reindex e e (1 : Matrix d d R) = 1 := by
  -- By definition of reindex, the entry at (i, j) in the reindexed matrix is
  ext i j
  simp [Matrix.reindex, Matrix.one_apply]
  simp only [h_reindex_id, one_mem]

omit [Fintype d₂] [DecidableEq d₂] [StarRing R] in
theorem Matrix.reindex_eq_conj (A : Matrix d d R) (e : d ≈ d₂) : reindex e
  (reindex (α := R) e (.refl d) 1) * A * (reindex (α := R) (.refl d) e 1) = A := by
  ext i j
  simp only [Matrix.mul_apply, Matrix.reindex]
  simp [Matrix.one_apply]

theorem Matrix.reindex_eq_conj_unitaryGroup' (A : Matrix d d R) (e : Equiv d d) :
  ((_, e⁻¹.permMatrix_mem_unitaryGroup) : unitaryGroup d R) * A * ((_, e) : unitaryGroup d R) = A := by
  ext i j;
  simp [Matrix.mul_apply]
  rw [Finset.sum_eq_single (e.symm j)] <|> aesop

theorem Matrix.IsHermitian.eigenvalue_ext (hA : A.IsHermitian)
  (h : ∀ (v : d → ℝ) (lam : ℝ), A *ₙ v = lam • v → B *ₙ v = lam • v) :
  A = B := by
  -- Since A is Hermitian, it is diagonalizable, and its eigenvectors form an orthonormal basis
  have h_diag : ∀ v : d → ℝ, (A *ₙ v) = (B *ₙ v) := by
  -- Since A is Hermitian, it is diagonalizable, and its eigenvectors form an orthonormal basis
  have h_diag : ∀ v : d → ℝ, ∃ (c : d → ℝ) (lam : d → ℝ), v = ∑ i, c i • (hA.eigenvectorBasis i)
  intro v
  obtain ⟨c, hc⟩ : ∃ c : d → ℝ, v = ∑ i, c i • (hA.eigenvectorBasis i)
  have h_diag : ∀ v : EuclideanSpace ℝ d, ∃ c : d → ℝ, v = ∑ i, c i • (hA.eigenvectorBasis i)
  intro v
  set c := fun i => innerₛₗ ℝ (hA.eigenvectorBasis i) v
  have hv : v = ∑ i, c i • (hA.eigenvectorBasis i) := by
    exact Eq.symm (OrthonormalBasis.sum_repr' hA.eigenvectorBasis v)
  use c;
  obtain ⟨c, hc⟩ := h_diag (WithLp.toLp 2 v)
  use c
  exact congr(WithLp.ofLp $hc)

```

```

    refine' { c, fun i => ( hA.eigenvalues i ), hc, fun i => _ };
    convert hA.mulVec_eigenvectorBasis i;
    ext
    simp only [smul_eq_mul, Pi.smul_apply]
    symm
    exact RCLike.real_smul_eq_coe_mul (hA.eigenvalues i) _
  -- By linearity of A and B, we can distribute them over the sum.
  intros v
  obtain (c, lam, hv, hlam) := h_diag v
  have hA_v : A *v v = ∑ i, c i • lam i • (hA.eigenvectorBasis i) := by
    -- By linearity of matrix multiplication, we can distribute A over the sum.
    have hA_v : A *v (∑ i, c i • (hA.eigenvectorBasis i)) = ∑ i, c i • A *v (hA.ei
      simp [funext_iff]
      simp [Matrix.mulVec, dotProduct, Finset.mul_sum _ _]
      exact fun _ => Finset.sum_comm.trans ( Finset.sum_congr rfl fun _ _ => Finse
    aesop
  have hB_v : B *v v = ∑ i, c i • lam i • (hA.eigenvectorBasis i) := by
  have hBv : B *v v = ∑ i, c i • (B *v (hA.eigenvectorBasis i)) := by
    -- By linearity of matrix multiplication, we can distribute $B$ over the sum
    have hBv : B *v v = B *v (∑ i, c i • (hA.eigenvectorBasis i)) := by
      rw [hv]
      simp only [WithLp.ofLp_sum, WithLp.ofLp_smul]
      simp [hBv, funext_iff]
      simp [Matrix.mulVec, dotProduct, Finset.mul_sum _ _]
      exact fun _ => Finset.sum_comm.trans ( Finset.sum_congr rfl fun _ _ => Finse
    refine hBv.trans ?_
    conv => enter [1, 2, i]; rw [h _ _ ( hlam i )]
    simp only [WithLp.ofLp_sum, WithLp.ofLp_smul]
  rw [hA_v, hB_v]
  -- By the definition of matrix equality, if $A * v = B * v$ for all $v$, then $A =
  apply Matrix.ext; intro i j; exact (by
    simp using congr_fun ( h_diag ( Pi.single j 1 ) ) i)

/-- Generalizes `Matrix.IsHermitian.cfc.eq_1`, which gives a definition for the matr
`Matrix.IsHermitian.eigenvalues` and `Matrix.IsHermitian.eigenvectorUnitary`, to sho
similarly for _any_ diagonalization by a two-sided isometry.
-/
theorem Matrix.IsHermitian.cfc_eq_any_isometry {n m : Type*} [RCLike []]
  [Fintype n] [DecidableEq n] [Fintype m] [DecidableEq m]
  {A : Matrix n n []} (hA : A.IsHermitian) {U : Matrix n m []}
  (hU1 : U * UH = 1) (hU2 : UH * U = 1) {D : m → ℝ}
  (hUD : A = (U * diagonal (RCLike.ofReal ∘ D) : Matrix _ _ ) * UH) (f : ℝ → ℝ) :
  hA.cfc f = (U * diagonal (RCLike.ofReal ∘ f ∘ D) : Matrix _ _ ) * UH := by
  --Thanks Aristotle
  rw [Matrix.IsHermitian.cfc]
  have hUV := hA.spectral_theorem

```

```

set V := hA.eigenvectorUnitary with hV; clear_value V
set D2 := hA.eigenvalues with hD; clear_value D2
rcases V with ⟨V, hV1, hV2⟩
simp only [Unitary.conjStarAlgAut_apply] at hUV ⊢
clear hV hD
subst A; clear hA
have h_diag_eq : diagonal (RCLike.ofReal ∘ D) * (UH * V) = (UH * V) * dia
  have h_mul : (UH * U * diagonal (RCLike.ofReal ∘ D) * UH : Matrix m n []
    simp only [Matrix.mul_assoc, hUV]
    simp_all [Matrix.mul_assoc];
have h_diag_eq_f : diagonal (RCLike.ofReal ∘ f ∘ D) * (UH * V) = (UH * V)
ext i j
simp_all only [diagonal_mul, Function.comp_apply, mul_diagonal]
replace h_diag_eq := congr_fun (congr_fun h_diag_eq i) j
by_cases hi : D i = D2 j <=> simp_all [mul_comm];
have h_final : U * diagonal (RCLike.ofReal ∘ f ∘ D) * UH * V = V * diagon
  have h_final : U * diagonal (RCLike.ofReal ∘ f ∘ D) * (UH * V) = U * (U
    rw [Matrix.mul_assoc, h_diag_eq_f, Matrix.mul_assoc];
    rw [Matrix.mul_assoc, Matrix.mul_assoc];
    simp_all +decide [← Matrix.mul_assoc];
  rw [← h_final, Matrix.mul_assoc];
  rw [hV2, mul_one ]

/-- Generalizes `Matrix.IsHermitian.cfc.eq_1`, which gives a definition for
`Matrix.IsHermitian.eigenvalues` and `Matrix.IsHermitian.eigenvectorUnitary`
similarly for _any_ diagonalization.
-/
theorem Matrix.IsHermitian.cfc_eq_any_unitary {n [] : Type*} [RCLike []] [Fin
  {A : Matrix n n []} (hA : A.IsHermitian) {U : unitaryGroup n []} {D : n → ℝ
  (hUD : A = U.val * diagonal (RCLike.ofReal ∘ D) * star U.val) (f : ℝ → ℝ)
  hA.cfc f = U.val * diagonal (RCLike.ofReal ∘ f ∘ D) * star U.val :=
  Matrix.IsHermitian.cfc_eq_any_isometry hA U.2.2 U.2.1 hUD f

private theorem Matrix.cfc_conj_isometry' (hA : A.IsHermitian) (f : ℝ → ℝ)
  (hu1 : u.Isometry) (hu2 : uH.Isometry) :
  cfc f (u * A * uH) = u * (cfc f A) * uH := by
  let D := hA.eigenvalues
  let U' := u * hA.eigenvectorUnitary.val
  have := IsHermitian.cfc_eq_any_isometry
    (A := u * A * uH) (D := D) (n := d2) (m := d) (U := U') ?_ ?_ ?_ ?_ f;
  · simp using isHermitian_conjTranspose_mul_mul uH hA
  · dsimp [U']
    rw [conjTranspose_mul, Matrix.mul_assoc]
    nth_rw 2 [← Matrix.mul_assoc]
    rw [show _ * _H = 1 from hA.eigenvectorUnitary.2.2, Matrix.one_mul]
    simp [Isometry] using hu2

```

```

· dsimp [U']
  rw [conjTranspose_mul, Matrix.mul_assoc]
  nth_rw 2 [← Matrix.mul_assoc]
  rw [hu₁, Matrix.one_mul]
  exact hA.eigenvectorUnitary.2.1
· rw [hA.spectral_theorem]
  simp [U', Matrix.mul_assoc]
  rfl
rw [Matrix.IsHermitian.cfc_eq, this]
rw [hA.cfc_eq, Matrix.IsHermitian.cfc.eq_1]
simp only [Unitary.conjStarAlgAut_apply]
simp only [Matrix.mul_assoc, conjTranspose_mul, star_eq_conjTranspose, U', D]
exact isHermitian_mul_mul_conjTranspose _ hA

theorem Matrix.cfc_conj_isometry (f : ℝ → ℝ) {u : Matrix d₂ d []}
  (hu₁ : u.Isometry) (hu₂ : uH.Isometry) :
  cfc f (u * A * uH) = u * (cfc f A) * uH := by
  by_cases hA : A.IsHermitian
  · exact cfc_conj_isometry' hA f hu₁ hu₂
  rw [cfc_apply_of_not_predicate, cfc_apply_of_not_predicate]
  · simp
  · exact hA
  · contrapose! hA
    convert isHermitian_conjTranspose_mul_mul u hA
    have hu₃ : uH * u = 1 := by simpa [Isometry] using hu₁
    simp only [Matrix.mul_assoc, hu₃]
    simp [← Matrix.mul_assoc, hu₃]

theorem Matrix.cfc_conj_unitary (f : ℝ → ℝ) (u : unitaryGroup d []) :
  cfc f (u * A * u-1) = u * (cfc f A) * u-1 := by
  have hu := u.prop
  rw [mem_unitaryGroup_iff_isometry] at hu
  exact Matrix.cfc_conj_isometry f hu.left hu.right

theorem Matrix.cfc_conj_unitary' (f : ℝ → ℝ) (u : unitaryGroup d []) :
  cfc f (uH * A * u.val) = uH * (cfc f A) * u.val := by
  simpa only [inv_inv] using cfc_conj_unitary f u-1

theorem Matrix.cfc_reindex (f : ℝ → ℝ) (e : d ≈ d₂) :
  cfc f (reindex e e A) = reindex e e (cfc f A) := by
  rw [reindex_eq_conj, reindex_eq_conj]
  convert Matrix.cfc_conj_isometry f (u := (Matrix.reindex e (Equiv.refl d) : Matrix
  · simp
  · simp
  · apply reindex_one_isometry
  · rw [conjTranspose_reindex, conjTranspose_one]

```

```

    apply reindex_one_isometry

theorem Matrix.commute_euclideanLin (hAB : Commute A B) :
  Commute A.toEuclideanLin B.toEuclideanLin := by
  rw [commute_iff_eq] at hAB
  ext v i
  convert congr(($hAB).mulVec (WithLp.ofLp v) i) using 0
  simp only [Module.End.mul_apply, ← Matrix.mulVec_mulVec];
  simp only [ofLp_toLpLin, toLin'_apply, mulVec_mulVec]

section commute_module
open Module.End

--TODO: All of these have Pi versions (instead of the "just two" operators
-- see the tail end of `JointEigenspace.lean` to see how it should general
-- also give a Pi version for Matrix. That would be useful for e.g. we hav
-- of projectors that all pairwise commute, and we want to simultaneously
-- of them.

/-- Similar to `LinearMap.IsSymmetric.orthogonalFamily_eigenspace_inf_eigen
is indexed by only the pairs of eigenvalues, as opposed to all pairs of `[]
decomposition. -/
theorem LinearMap.IsSymmetric.orthogonalFamily_eigenspace_inf_eigenspace' {
  [NormedAddCommGroup E] [InnerProductSpace ℝ E] {A B : E →ℝ [ ] E}
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) :
  OrthogonalFamily [ ] (fun (μ12 : Eigenvalues A × Eigenvalues B) ↦
    ↗(eigenspace A μ12.1 [ ] eigenspace B μ12.2)) fun μ12 ↦
    (eigenspace A μ12.1 [ ] eigenspace B μ12.2).subtypeℝ := by
  have h := LinearMap.IsSymmetric.orthogonalFamily_eigenspace_inf_eigenspac
  simp only [OrthogonalFamily, Submodule.coe_subtypeℝ, Submodule.subtype_a
    Subtype.forall, Submodule.mem_inf, mem_genEigenspace_one, and_imp] at h
  intro i j hij a ha hb a' ha' hb'
  contrapose! h
  simp only [Pairwise, ne_eq, Prod.forall, Prod.mk.injEq, not_and, not_fora
  refine (⟦_, _⟧, ⟦_, _⟧, ?_, a, ha, hb, a', ha', hb', h)
  intro h' h''
  exact hij (Prod.ext (Subtype.ext h')) (Subtype.ext h'))

/-- Variant of `iSup_mono` that allows for an easier handling of bottom el
/
theorem iSup_mono_bot {α : Type*} {ι ι' : Sort*} [CompleteLattice α]
  {f : ι → α} {g : ι' → α} (h : ∀ (i : ι), f i = ⊥ ∨ ∃ i', f i ≤ g i') :
  iSup f ≤ iSup g := by
  rcases isEmpty_or_nonempty ι'
  · simp only [IsEmpty.exists_iff, or_false] at h
  simp [h]

```



```

· refine iSup_mono' (fun i ↦ ?_)
  rcases h i with h | h <=> simp [h]

noncomputable def Commute.isSymmetric_directSumDecomposition {E : Type*} [RCLike
  [NormedAddCommGroup E] [InnerProductSpace E] {A B : E →1 E} [FiniteDimensional E]
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) :
  DirectSum.Decomposition fun (μ12 : Eigenvalues A × Eigenvalues B) ↦
    (eigenspace A μ12.1 ⊔ eigenspace B μ12.2) := by
  apply (LinearMap.IsSymmetric.orthogonalFamily_eigenspace_inf_eigenspace' hA hB).de
  have h := LinearMap.IsSymmetric.iSup_iSup_eigenspace_inf_eigenspace_eq_top_of_comm
    hA hB hAB
  rw [iSup_prod'] at h
  apply le_antisymm le_top
  rw [← h, iSup_le_iff]
  rintro (fst, snd)
  by_cases h1 : Module.End.HasEigenvalue A fst
  · by_cases h2 : Module.End.HasEigenvalue B snd
    · exact le_iSup_of_le (⟨fst, h1⟩, ⟨snd, h2⟩) le_refl
    · replace h2 : eigenspace B snd = ⊥ := by simp [Module.End.HasUnifEigenvalue] u
      simp [h2]
  · replace h1 : eigenspace A fst = ⊥ := by simp [Module.End.HasUnifEigenvalue] u
    simp [h1]

/-- Similar to `LinearMap.IsSymmetric.directSum_isInternal_of_commute`, but here the
is indexed by only the pairs of eigenvalues, as opposed to all pairs of `E` values,
decomposition. -/
theorem LinearMap.IsSymmetric.directSum_isInternal_of_commute' {E : Type*} [RCLike
  [NormedAddCommGroup E] [InnerProductSpace E] {A B : E →1 E} [FiniteDimensional E]
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) :
  DirectSum.IsInternal fun (μ12 : Eigenvalues A × Eigenvalues B) ↦
    eigenspace A μ12.1 ⊔ eigenspace B μ12.2 := by
  classical
  have h := LinearMap.IsSymmetric.directSum_isInternal_of_commute hA hB hAB
  constructor
  · intro x y hxy
    -- Since the subspaces are orthogonal, the only way their sum can be zero is if
    rw [← sub_eq_zero]
    suffices h_diff_zero : ∀ (x : DirectSum (Eigenvalues A × Eigenvalues B)) fun μ12
      h_diff_zero (x - y) (by simp [hxy])
    clear x y hxy; intro x hx;
    ext μ12
    simp only [DirectSum.zero_apply, ZeroMemClass.coe_zero]
    rw [← inner_self_eq_zero (· := ·)]
    have h_inner_zero : inner (·) (x μ12 : E) (x.coeAddMonoidHom _) = 0 := by
      simp [hx]
    rw [← h_inner_zero]

```

```

simp only [DirectSum.coeAddMonoidHom_eq_dfinsuppSum, ZeroMemClass.coe_z
  DFinsupp.sum_eq_sum_fintype, DFinsupp.equivFunOnFintype_apply]
-- Since the decomposition is orthogonal, the inner product of  $x$   $\mu_{12}$  wi
rw [inner_sum, Finset.sum_eq_add_sum_diff_singleton (Finset.mem_univ  $\mu_1$ 
rw [Finset.sdiff_singleton_eq_erase, left_eq_add]
apply Finset.sum_eq_zero
intro  $\mu$   $h\mu$ 
exact orthogonalFamily_eigenspace_inf_eigenspace' hA hB (Finset.ne_of_m
· -- Since the decomposition is orthogonal, the direct sum of the interse
have h_sum :  $\square (\mu_{12} : \text{Eigenvalues } A \times \text{Eigenvalues } B), \text{eigenspace } A \mu_{12}.$ 
  rw [eq_top_iff]
  intro x hx
  obtain ⟨y, rfl⟩ := h.2 x
  rw [DirectSum.coeAddMonoidHom_eq_dfinsuppSum]
  refine Submodule.sum_mem _ fun i hi => ?_
  have hyi := Submodule.coe_mem (y i)
  simp only [Submodule.mem_inf, mem_genEigenspace_one] at hyi
  refine Submodule.mem_iSup_of_mem ⟨{i.2, ?_}, {i.1, ?_}⟩ (by simp)
  <=> simp only [HasUnifEigenvalue, ne_eq, Submodule.eq_bot_iff, mem_gen
  <=> refine ⟨y i, by tauto, by simp using hi⟩
intro x
rw [Submodule.eq_top_iff'] at h_sum
specialize h_sum x
rw [Submodule.mem_iSup_iff_exists_finsupp] at h_sum
rcases h_sum with ⟨f, hf1, hf2⟩
exact ⟨ $\sum i \in f.\text{support}, .\text{of } _ i \langle f i, hf_1 i \rangle$ , by simp using hf2⟩

noncomputable def LinearMap.sharedEigenbasis {A B : EuclideanSpace  $\square d \rightarrow_1 \square$ 
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) :
  OrthonormalBasis d  $\square$  (EuclideanSpace  $\square d$ ) :=
  ((hA.directSum_isInternal_of_commute' hB hAB).subordinateOrthonormalBasis
  (hA.orthogonalFamily_eigenspace_inf_eigenspace' hB)).reindex
  (Fintype.equivOfCardEq (by simp))

noncomputable def LinearMap.sharedEigenvaluesA {A B : EuclideanSpace  $\square d \rightarrow_1$ 
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) :  $d \rightarrow \mathbb{R} :$ 
  fun i => RCLike.re (inner  $\square$  (LinearMap.sharedEigenbasis hA hB hAB i) (A (

noncomputable def LinearMap.sharedEigenvaluesB {A B : EuclideanSpace  $\square d \rightarrow_1$ 
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) :  $d \rightarrow \mathbb{R} :$ 
  fun i => RCLike.re (inner  $\square$  (LinearMap.sharedEigenbasis hA hB hAB i) (B (

omit [DecidableEq d] in
theorem LinearMap.mem_eigenspace_inf_of_sharedEigenbasis {A B : EuclideanSp
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) (i : d) :
   $\exists (\mu : \text{Module.End.Eigenvalues } A) (\nu : \text{Module.End.Eigenvalues } B),$ 

```

```

      LinearMap.sharedEigenbasis hA hB hAB i ∈ Module.End.eigenspace A μ ⊆ Module.End
rw [LinearMap.sharedEigenbasis]
rw [OrthonormalBasis.reindex_apply]
let hV := hA.directSum_isInternal_of_commute' hB hAB
let hV' := hA.orthogonalFamily_eigenspace_inf_eigenspace' hB
let hn : Module.finiteRank (EuclideanSpace d) = Module.finiteRank (EuclideanSpace d)
let e := Fintype.equivOfCardEq (show Fintype.card (Fin (Module.finiteRank (EuclideanSpace d))))
let j := e.symm i
let idx := hV.subordinateOrthonormalBasisIndex hn j hV'
exists idx.1, idx.2
exact hV.subordinateOrthonormalBasis_subordinate hn j hV'

omit [DecidableEq d] in
theorem LinearMap.apply_A_sharedEigenbasis {A B : EuclideanSpace d →1 [d] EuclideanSpace d}
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) (i : d) :
  A (sharedEigenbasis hA hB hAB i) = (sharedEigenvaluesA hA hB hAB i : ℝ) • (sharedEigenbasis hA hB hAB i)
obtain ⟨μ, v, h⟩ := mem_eigenspace_inf_of_sharedEigenbasis hA hB hAB i
have h₂ := Module.End.mem_eigenspace_iff.mp h.1
rw [h₂]
congr; symm
simp only [sharedEigenvaluesA, h₂, inner_smul_right, OrthonormalBasis.inner_eq_one, mul_one, ← RCLike.conj_eq_iff_re, ← RCLike.star_def]
have h₃ : (sharedEigenbasis hA hB hAB) i ≠ 0 := by
  have := (sharedEigenbasis hA hB hAB).orthonormal.1 i
  grind [norm_zero, zero_ne_one]
simp [inner_smul_left, inner_smul_right, h₂, h₃] using
  hA ((sharedEigenbasis hA hB hAB) i) ((sharedEigenbasis hA hB hAB) i)

omit [DecidableEq d] in
theorem LinearMap.apply_B_sharedEigenbasis {A B : EuclideanSpace d →1 [d] EuclideanSpace d}
  (hA : A.IsSymmetric) (hB : B.IsSymmetric) (hAB : Commute A B) (i : d) :
  B (sharedEigenbasis hA hB hAB i) = (sharedEigenvaluesB hA hB hAB i : ℝ) • (sharedEigenbasis hA hB hAB i)
obtain ⟨μ, v, h⟩ := mem_eigenspace_inf_of_sharedEigenbasis hA hB hAB i
have h₂ := Module.End.mem_eigenspace_iff.mp h.2
rw [h₂]
congr; symm
simp only [sharedEigenvaluesB, h₂, inner_smul_right, OrthonormalBasis.inner_eq_one, mul_one, ← RCLike.conj_eq_iff_re, ← RCLike.star_def]
have h₃ : (sharedEigenbasis hA hB hAB) i ≠ 0 := by
  have := (sharedEigenbasis hA hB hAB).orthonormal.1 i
  grind [norm_zero, zero_ne_one]
simp [inner_smul_left, inner_smul_right, h₂, h₃] using
  hB ((sharedEigenbasis hA hB hAB) i) ((sharedEigenbasis hA hB hAB) i)

noncomputable def Matrix.sharedEigenbasis
  (hA : A.IsHermitian) (hB : B.IsHermitian) (hAB : Commute A B) :

```

```

    OrthonormalBasis d [] (EuclideanSpace [] d) :=
      LinearMap.sharedEigenbasis (isHermitian_iff_isSymmetric.mp hA)
        (isHermitian_iff_isSymmetric.mp hB) (commute_euclideanLin hAB)

noncomputable def Matrix.sharedEigenvectorUnitary (hA : A.IsHermitian) (hB : B.IsHermitian) (hAB : Commute A B) : Matrix.unitaryGroup d [] :=
  ((EuclideanSpace.basisFun d []).toBasis.toMatrix (sharedEigenbasis hA hB hAB))
    (EuclideanSpace.basisFun d []).toMatrix_orthonormalBasis_mem_unitary (sharedEigenbasis hA hB hAB)

namespace Matrix.SharedEigenbasis

variable (hA : A.IsHermitian) (hB : B.IsHermitian) (hAB : Commute A B)

/-- Analogous to `Matrix.IsHermitian.eigenvectorUnitary_mulVec` for the shared eigenbasis /
theorem sharedEigenvectorUnitary_mulVec (j : d) : (sharedEigenvectorUnitary hA hB hAB j) =
  Pi.single j 1 = WithLp.ofLp (sharedEigenbasis hA hB hAB j) := by
  simp_all only [mulVec_single, MulOpposite.op_one, one_smul]
  rfl

noncomputable def sharedEigenvalueA (j : d) : ℝ :=
  LinearMap.sharedEigenvaluesA
    (isHermitian_iff_isSymmetric.mp hA)
    (isHermitian_iff_isSymmetric.mp hB)
    (commute_euclideanLin hAB) j

noncomputable def sharedEigenvalueB (j : d) : ℝ :=
  LinearMap.sharedEigenvaluesB
    (isHermitian_iff_isSymmetric.mp hA)
    (isHermitian_iff_isSymmetric.mp hB)
    (commute_euclideanLin hAB) j

/-- Analogous to `Matrix.IsHermitian.mulVec_eigenvectorBasis` for the shared eigenbasis /
theorem mulVec_sharedEigenbasisA (j : d) :
  A *v (sharedEigenbasis hA hB hAB j) =
    (sharedEigenvalueA hA hB hAB j) • WithLp.ofLp (sharedEigenbasis hA hB hAB j)
  rw [isHermitian_iff_isSymmetric] at hA hB
  have h := LinearMap.apply_A_sharedEigenbasis hA hB (Matrix.commute_euclideanLin hAB)
  simp only [algebraMap_smul] at h
  have := congr_arg WithLp.ofLp h
  simp [Matrix.toEuclideanLin, Matrix.toLpLin_apply] at this
  exact this

theorem mulVec_sharedEigenbasisB (j : d) :
  B *v (sharedEigenbasis hA hB hAB j) =

```

```

    (sharedEigenvalueB hA hB hAB) j • WithLp.ofLp (sharedEigenbasis hA hB hAB j) :=
  rw [isHermitian_iff_isSymmetric] at hA hB
  have h := LinearMap.apply_B_sharedEigenbasis hA hB (Matrix.commute_euclideanLin hA hB)
  simp only [algebraMap_smul] at h
  have := congr_arg WithLp.ofLp h
  simp [Matrix.toEuclideanLin, Matrix.toLpLin_apply] at this
  exact this

/-
PROVIDED SOLUTION
This is exactly analogous to star_shared_mul_B_mul_IsDiag (which is proved below in
-/
set_option maxHeartbeats 0 in

theorem star_shared_mul_A_mul_IsDiag : IsDiag
  ((star (sharedEigenvectorUnitary hA hB hAB : Matrix d d [])) * A *
   (sharedEigenvectorUnitary hA hB hAB : Matrix d d [])) := by
  intro i j hij;
  have := @mulVec_sharedEigenbasisA d;
  specialize this hA hB hAB j;
  replace this := congr_arg ( fun x => star ( ( sharedEigenbasis hA hB hAB i ).ofLp
  simp only [dotProduct, Pi.star_apply, RCLike.star_def, mulVec, Finset.mul_sum, mul
    mul_assoc, Pi.smul_apply, Algebra.smul_mul_assoc] at this
  simp only [mul_assoc]
  convert this using 1;
  · simp [ Matrix.mul_apply, mul_assoc, mul_comm, Finset.sum_mul ]
    congr! 3;
  · have := ( sharedEigenbasis hA hB hAB ).orthonormal;
    rw [ orthonormal_iff_ite ] at this;
    simp only [inner] at this
    rw [ ← Finset.smul_sum, this i j, if_neg hij, smul_zero ]

/-- Analogous to `Matrix.IsHermitian.star_mul_self_mul_eq_diagonal` for the shared b
/
theorem star_shared_mul_B_mul_IsDiag : IsDiag
  ((star (sharedEigenvectorUnitary hA hB hAB : Matrix d d [])) * B *
   (sharedEigenvectorUnitary hA hB hAB : Matrix d d [])) := by
  rw [Matrix.isDiag_iff_diagonal_diag, eq_comm]
  apply Matrix.toEuclideanLin.injective
  apply (EuclideanSpace.basisFun d []).toBasis.ext
  intro i
  simp only [toLpLin_apply, OrthonormalBasis.coe_toBasis, EuclideanSpace.basisFun_ap
    EuclideanSpace.ofLp_single, ← mulVec_mulVec, sharedEigenvectorUnitary_mulVec, ←
    Matrix.diagonal_mulVec_single, mul_one]
  apply PiLp.ext
  intro j

```

```

have h_simp : (Matrix.sharedEigenvectorUnitary hA hB hAB).val.conjTranspose
  (sharedEigenvalueB hA hB hAB i) • (Matrix.sharedEigenvectorUnitary hA h
    convert congr_arg ( fun x => ( Matrix.sharedEigenvectorUnitary hA hB
      symm
        exact (mulVec_smul ((sharedEigenvectorUnitary hA hB hAB).val)H (share
          (WithLp.ofLp ((sharedEigenbasis hA hB hAB) i)))
have h_simp2 : (Matrix.sharedEigenvectorUnitary hA hB hAB).val.conjTranspose
  rw [ ← sharedEigenvectorUnitary_mulVec hA hB hAB i ];
  simp
  ext j
  have := mul_eq_one_comm.mp ( show ( Matrix.sharedEigenvectorUnitary hA
    · convert congr_fun ( congr_fun this j ) i using 1;
      simp [ Pi.single_apply, Matrix.one_apply ];
      · exact Matrix.mem_unitaryGroup_iff.mp ( Matrix.sharedEigenvectorUnitar
simp_all [ Matrix.mulVec, funext_iff ];
simp_all [ Matrix.mul_apply, dotProduct ];
by_cases hij : i = j
· simp [ hij ];
  simp [ hij, Pi.single_apply, Matrix.mulVec, dotProduct ];
  simp only [Finset.mul_sum, mul_left_comm];
  rw [ Finset.sum_comm ]
  simp [ mul_comm, mul_left_comm, Finset.mul_sum]
  congr! 3;
· simp [ hij ];
  simp_all [ mul_comm, Matrix.mulVec, dotProduct ];
  simp_all [ mul_comm, Finset.mul_sum];
  rw [ Finset.sum_comm ]
  simp_all [ mul_assoc, mul_left_comm] ;

end Matrix.SharedEigenbasis

end commute_module

theorem Commute.exists_unitary (hA : A.IsHermitian) (hB : B.IsHermitian) (h
  ∃ U : Matrix.unitaryGroup d [], (U.val * A * UH).IsDiag ∧ (U.val * B * U
  use (Matrix.sharedEigenvectorUnitary hA hB hAB)-1
  constructor
  · convert Matrix.SharedEigenbasis.star_shared_mul_A_mul_IsDiag hA hB hAB
    simp [Matrix.star_eq_conjTranspose]
  · convert Matrix.SharedEigenbasis.star_shared_mul_B_mul_IsDiag hA hB hAB
    simp [Matrix.star_eq_conjTranspose]

variable (U : Matrix.unitaryGroup d [])

instance instInvertibleUnitaryGroup (U : Matrix.unitaryGroup d []) : Inverti

```

```

invertibleOfGroup U

instance (U : Matrix.unitaryGroup d  $\square$ ) : Invertible U.val :=
  (star U.val, U.2.1, U.2.2)

/-- If a matrix is diagonalized by a unitary matrix, then it can be written as a
CFC of a (particular, canonical) diagonal matrix. -/
theorem Matrix.IsDiag.exists_cfc {U : Matrix.unitaryGroup d  $\square$ } {M : Matrix d d  $\square$ }
  (hU : (U.val * M * UH).IsDiag) (hM : M.IsHermitian) (e : d  $\approx$  Fin (Fintype.card d))
   $\exists$  f :  $\mathbb{R} \rightarrow \mathbb{R}$ ,
    M = cfc f (UH * (Matrix.diagonal fun x =>  $\uparrow\uparrow$ (e x)) * U.val) := by
  use fun x  $\mapsto$  if hn :  $\exists$  n : Fin (Fintype.card d), n = x
    then RCLike.re (Matrix.diag (U.val * M * UH) (e.symm hn.choose)) else 0
  rw [Matrix.cfc_conj_unitary']
  rw [Matrix.isDiag_iff_diagonal_diag] at hU
  rw [← Matrix.mul_inv_eq_iff_eq_mul_of_invertible] at hU
  rw [← Matrix.inv_mul_eq_iff_eq_mul_of_invertible] at hU
  rw [← hU, ← Matrix.mul_assoc]
  congr; rotate_right
  · exact Matrix.inv_eq_right_inv U.2.1
  · exact Matrix.inv_eq_left_inv U.2.1
  conv in Nat.cast (e _) =>
    equals (RCLike.ofReal <| e x) => simp only [map_natCast]
  rw [Matrix.cfc_diagonal]
  congr
  ext i
  simp only [Matrix.diag_apply, Function.comp_apply, Nat.cast_inj, exists_apply_eq_a
     $\downarrow$ reduceDite]
  rw [← (Matrix.isHermitian_mul_mul_conjTranspose U.val hM).coe_re_apply_self i]
  congr!
  · rw [mul_assoc, hU]
  all_goals
    ( rw [e.eq_symm_apply]
      symm; convert Classical.choose_eq _
      exact Fin.val_inj)

--TODO: Make Iff version.
/-- If two Hermitian matrices commute, there exists a common matrix that they are bo
/
theorem Commute.exists_cfc (hA : A.IsHermitian) (hB : B.IsHermitian) (hAB : Commute
   $\exists$  C : Matrix d d  $\square$ , ( $\exists$  f :  $\mathbb{R} \rightarrow \mathbb{R}$ , A = cfc f C)  $\wedge$  ( $\exists$  g :  $\mathbb{R} \rightarrow \mathbb{R}$ , B = cfc g C) := b
  obtain ⟨U, hU1, hU2⟩ := hAB.exists_unitary hA hB
  let D : Matrix d d  $\square$  := Matrix.diagonal (Fintype.equivFin d  $\cdot$ )
  exact ⟨UH * D * U.val, hU1.exists_cfc hA _, hU2.exists_cfc hB _⟩

```

1.472 QuantumInfo/ForMathlib/Lieb.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.HermitianMat

/-! Lieb's Inequality .. todo -/

variable {m n : Type*} [Fintype m] [Fintype n] {q r : ℝ}

noncomputable section
open ComplexOrder
open Classical
open RealInnerProductSpace

theorem LiebConcavity (K : Matrix n m ℂ) (hq : 0 ≤ q) (hr : 0 ≤ r) (hqr : q
  let F : (HermitianMat m ℂ × HermitianMat n ℂ) → ℝ :=
    fun (x,y) ↦ (x ^ q).conj K, y ^ r;
    ConcaveOn ℝ .univ F := by
  sorry

```

1.473 QuantumInfo/ForMathlib/LimSupInf.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Algebra.Order.Ring.Star
import Mathlib.Analysis.Normed.Ring.Lemmas
import Mathlib.Data.Finset.Attr
import Mathlib.Data.Int.Star
import Mathlib.Data.Real.StarOrdered
import Mathlib.Tactic.Bound
import Mathlib.Tactic.Peel
import Mathlib.Tactic.Common
import Mathlib.Tactic.Continuity
import Mathlib.Tactic.Finiteness.Attr
import Mathlib.Tactic.SetLike
import Mathlib.Util.CompileInductive
import Mathlib.Topology.Instances.ENNReal.Lemmas

```



```
import Mathlib.Topology.Instances.Nat
```

```
open scoped NNReal
open scoped ENNReal
open Topology
```

```
/-!
```

```
Several 'bespoke' facts about limsup and liminf on ENNReal / NNReal needed in Steins
-/-
```

```
/-
```

```
There exists a strictly increasing sequence of indices  $n_k$  such that  $f(1/(k+1), n_k) \leq y + 1/(k+1)$ 
-/-
```

```
lemma exists_strictMono_seq_le (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞) (hf : ∀ x > 0, Filter.
  ∃ n : ℕ → ℕ, StrictMono n ∧ ∀ k : ℕ, f ((k : ℝ≥0) + 1)-1 (n k) ≤ (y : ℝ≥0∞) + ((k : ℝ≥0) + 1)-1)
  -- Since the liminf is ≤ y, for any ε > 0 and index n, there frequently exists an
  have h_freq (k n : ℕ) : ∃ m > n, f ((k + 1 : ℝ≥0)-1) m ≤ y + (k + 1 : ℝ≥0)-1 := by
    specialize hf ((k + 1 : ℝ≥0)-1) (by positivity)
    rw [Filter.liminf_eq] at hf
    simp only [Filter.eventually_atTop, ge_iff_le, sSup_le_iff, Set.mem_setOf_eq, for]
    contrapose! hf
    refine ⟨_, n + 1, fun m hm ↦ (hf m hm).le, ENNReal.lt_add_right (by norm_num) (b)⟩
  refine ⟨fun k ↦ k.recOn (Classical.choose (h_freq 0 0))
    (fun i ih ↦ Nat.find (h_freq (i + 1) ih)), ?_, ?_⟩
  · exact strictMono_nat_of_lt_succ fun k ↦ (Nat.find_spec (h_freq (k + 1) _)).1
  · rintro ( _ | k)
    · exact (Classical.choose_spec (h_freq 0 _)).2
    · exact (Nat.find_spec (h_freq (k + 1) _)).2
```

```
/-
```

```
There exists a strictly increasing sequence M such that for all k, and all n ≥ M k,
-/-
```

```
lemma exists_seq_bound (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞) (hf : ∀ x > 0, Filter.atTop.li
  ∃ M : ℕ → ℕ, StrictMono M ∧ ∀ k, ∀ n ≥ M k, f ((k + 1 : ℝ≥0)-1) n ≤ y + (k + 1 : ℝ≥0)-1)
  have h_M (k : ℕ) : ∃ M_k, ∀ n ≥ M_k, f (k + 1)-1 n ≤ y + (k + 1 : ℝ≥0∞)-1 := by
    specialize hf (k + 1)-1 (by positivity)
    contrapose! hf
    refine lt_of_lt_of_le (b := ?_) ?_ (le_csInf ⟨τ, by simp⟩ ?_)
    · exact y + (k + 1 : ℝ≥0∞)-1
    · exact ENNReal.lt_add_right (by norm_num) (by norm_num)
    · intro b hb
      simp only [Filter.eventually_map, Filter.eventually_atTop, Set.mem_setOf_eq] a
      obtain ⟨w, h⟩ := hb
      obtain ⟨_, hw_left, hw_right⟩ := hf w
      grw [hw_right]
      exact h _ hw_left
    choose M hM using h_M
```

```

use Nat.rec (M 0) fun k ih => M (k + 1) [] (ih + 1)
constructor
· apply strictMono_nat_of_lt_succ
  exact fun _ => lt_sup_of_lt_right (lt_add_one _)
· intro k n hn
  apply hM
  grw [hn]
  cases k
  · rfl
  · apply le_max_left

/- (∀ x, x > 0 → liminf (n => f x n) ≤ y) →
   ∃ g : ℕ → ℝ, (∀ x, g x > 0) ∧ (liminf g = 0) ∧ (liminf (n => f (g n) n) ≤
/
lemma exists_liminf_zero_of_forall_liminf_le (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞)
(hf : ∀ x > 0, Filter.atTop.liminf (f x) ≤ y) :
  ∃ g : ℕ → ℝ≥0, (∀ x, g x > 0) ∧ Filter.atTop.Tendsto g (0) ∧
    Filter.atTop.liminf (fun n => f (g n) n) ≤ y := by
conv at hf =>
  enter [x, h]
  exact propext (Filter.liminf_le_iff (by isBoundedDefault) (by isBounded
replace hf x hx z hz := Filter.exists_seq_forall_of_frequently (hf x hx z
choose! c hc hc2 using hf
classical
by_contra h_contra;
-- By definition of negation, if $¬ P$ holds, then $P$ does not hold.
push_neg at h_contra;
-- Apply `exists_strictMono_seq_le` to obtain a strictly increasing sequence
obtain ⟨n, hn_mono, hn_le⟩ : ∃ n : ℕ → ℕ, StrictMono n ∧ ∀ k : ℕ, f ((k :
-- Apply `exists_strictMono_seq_le` to obtain a strictly increasing sequence
apply exists_strictMono_seq_le y f; intro x hx_pos; (
refine' le_of_forall_gt_imp_ge_of_dense fun z hz => _;
refine' csSup_le _ _ <=> norm_num;
· exact { 0, { 0, fun _ => zero_le _ } };
· intro b n hn; specialize hc x hx_pos z hz; have := hc.eventually_gt_a
  simp_all only [gt_iff_lt, Filter.eventually_atTop, ge_iff_le]
  obtain ⟨w, h⟩ := this
  exact le_trans (hn _ (le_of_lt (h _ le_refl))) (le_of_lt (hc2
-- Define $g(m) = 1/(k(m)+1)$ where $k(m)$ is the index such that $n_{k(m)}$
set g : ℕ → ℝ≥0 := fun m => (Nat.findGreatest (fun k => m ≥ n k) m + 1 :
have hg_pos : ∀ m, g m > 0 := by
  exact fun m => by positivity;;
have hg_tendsto_zero : Filter.Tendsto g Filter.atTop (0) := by
  -- Since $n$ is strictly monotone, $Nat.findGreatest (fun k => m ≥ n k)
  have h_find_greatest_inf : Filter.Tendsto (fun m => Nat.findGreatest (f
    refine' Filter.tendsto_atTop_atTop.mpr fun x => _;

```

```

use n x; intro a ha
refine' Nat.le_findGreatest _ ha
simp_all only [gt_iff_lt, ne_eq, add_eq_zero, Nat.cast_eq_zero, one_ne_zero, a
  ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast, ENNReal.coe_one, ge_i
  Nat.cast_pos, Nat.findGreatest_pos, zero_lt_one, or_true, implies_true, g]
exact le_trans ( hn_mono.id_le _ ) ha;
rw [ tendsto_order ] at *
simp_all only [gt_iff_lt, ne_eq, add_eq_zero, Nat.cast_eq_zero, one_ne_zero, and
  ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast, ENNReal.coe_one, ge_iff
  Nat.cast_pos, Nat.findGreatest_pos, zero_lt_one, or_true, implies_true, not_lt
  not_isEmpty_of_nonempty, IsEmpty.forall_iff, IsEmpty.exists_iff, true_and, g]
intro a' a
exact Filter.eventually_atTop.mp ( h_find_greatest_inf.eventually_gt_atTop [ ( a
have hg_le : ∀ k, f (g (n k)) (n k) ≤ (y : ℝ≥0∞) + ((k : ℝ≥0) + 1)-1 := by
intro k
specialize hn_le k
simp_all only [gt_iff_lt, ge_iff_le, inv_pos, add_pos_iff, Nat.cast_pos, Nat.fin
  or_true, implies_true, ne_eq, add_eq_zero, Nat.cast_eq_zero, one_ne_zero, and
  ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast, ENNReal.coe_one, g]
rw [ show Nat.findGreatest ( fun k_1 => n k_1 ≤ n k ) ( n k ) = k from _ ]
simp_all only
rw [ Nat.findGreatest_eq_iff ]
simp_all only [ne_eq, le_refl, implies_true, not_le, true_and]
apply And.intro
· exact hn_mono.id_le _;
· intro n_1 a a_1
  exact hn_mono a;
have hg_liminf : Filter.liminf (fun n => f (g n) n) Filter.atTop ≤ y := by
refine' csSup_le _ _ <;> norm_num;
· exact { 0, { 0, fun _ => zero_le _ } };
· intro b x hx; contrapose! hx
  simp_all only [gt_iff_lt, ne_eq, add_eq_zero, Nat.cast_eq_zero, one_ne_zero, a
    ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast, ENNReal.coe_one, ge_i
    Nat.cast_pos, Nat.findGreatest_pos, zero_lt_one, or_true, implies_true, g]
  -- Choose $k$ such that $y + 1/(k+1) < b$.
  obtain ⟨k, hk⟩ : ∃ k : ℕ, y + ((k : ℝ≥0) + 1)-1 < b := by
    rcases ENNReal.lt_iff_exists_nnreal_btwn.mp hx with ⟨z, hz⟩
    simp_all only [ENNReal.coe_lt_coe, ne_eq, add_eq_zero, Nat.cast_eq_zero, one
      not_false_eq_true, ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast,
    obtain ⟨left, right⟩ := hz
    obtain ⟨k, hk⟩ := exists_nat_one_div_lt ( show 0 < ( z : ℝ ) - y from sub_po
    use k
    norm_num at *;
    refine' lt_of_le_of_lt _ right;
    convert add_le_add_left ( ENNReal.ofReal_le_ofReal hk.le ) ( y : ℝ≥0∞ ) usin
    · norm_num [ Real.toNNReal_inv ];

```

```

      rw [add_comm]
      · rw [ENNReal.ofReal_sub _ (by positivity)]
        simp [tsub_add_cancel_of_le, left.le]
    refine' ( n ( Max.max x k ), _, _ )
    · simp_all only [ne_eq, add_eq_zero, Nat.cast_eq_zero, one_ne_zero, a
      ENNReal.coe_add, ENNReal.coe_natCast, ENNReal.coe_one]
      exact le_trans ( le_max_left _ ) ( hn_mono.id_le _ );
    · simp_all only [ne_eq, add_eq_zero, Nat.cast_eq_zero, one_ne_zero, a
      ENNReal.coe_add, ENNReal.coe_natCast, ENNReal.coe_one]
      refine' lt_of_le_of_lt ( hg_le _ ) _;
      refine' lt_of_le_of_lt _ hk
      gcongr
      exact le_sup_right
    exact h_contra g hg_pos hg_tendsto_zero |> not_lt_of_ge hg_liminf;

/- Version of `exists_liminf_zero_of_forall_liminf_le` that lets you also r
to have an upper bound. -/
lemma exists_liminf_zero_of_forall_liminf_le_with_UB (y : ℝ≥0) (f : ℝ≥0 → ℕ
{z : ℝ≥0} (hz : 0 < z)
(hf : ∀ x, x > 0 → Filter.atTop.liminf (f x) ≤ y) :
  ∃ g : ℕ → ℝ≥0, (∀ x, g x > 0) ∧ (∀ x, g x < z) ∧ (Filter.atTop.Tendsto
    (Filter.atTop.liminf (fun n ↦ f (g n) n) ≤ y) := by
obtain ⟨g, hg₀, hg₁, hg₂⟩ := exists_liminf_zero_of_forall_liminf_le y (fu
refine (fun n => min (g n) (z / 2)), by bound, by bound, ?_, ?_)
· convert hg₁.min tendsto_const_nhds using 2
  simp
· beta_reduce
  rwa [Filter.liminf_congr]
  have h := hg₁.eventually (gt_mem_nhds <| half_pos hz)
  peel h with h
  rw [min_eq_left h.le]

/- (∀ x, x > 0 → liminf (n ↦ f x n) ≤ y) →
  ∃ g : ℕ → ℝ, (∀ x, g x > 0) ∧ (liminf g = 0) ∧ (liminf (n ↦ f (g n) n) ≤
/
lemma exists_limsup_zero_of_forall_limsup_le (y : ℝ≥0) (f : ℝ≥0 → ℕ → ℝ≥0∞)
(hf : ∀ x, x > 0 → Filter.atTop.limsup (f x) ≤ y) :
  ∃ g : ℕ → ℝ≥0, (∀ x, g x > 0) ∧ (Filter.atTop.Tendsto g (□ 0)) ∧
    (Filter.atTop.limsup (fun n ↦ f (g n) n) ≤ y) := by
  -- Let's choose a sequence M such that for all k, and all n ≥ M k, f (1
obtain ⟨M, hM⟩ : ∃ M : ℕ → ℕ, StrictMono M ∧ ∀ k, ∀ n ≥ M k, f ((k + 1 :
  -- Apply the hypothesis `exists_seq_bound` to obtain the sequence $M$.
  apply exists_seq_bound y f hf;
  use fun n => 1 / ( Nat.findGreatest ( fun k => M k ≤ n ) n + 1 );
  refine {?_, ?_, ?_}
· aesop;

```

```

· -- We'll use that Nat.findGreatest (fun k => M k ≤ n) n tends to infinity as n t
have h_find_greatest : Filter.Tendsto (fun n => Nat.findGreatest (fun k => M k ≤
  refine' Filter.tendsto_atTop_atTop.mpr fun k => _;
  use M k; intro a ha; refine' Nat.le_findGreatest _ ha
  simp_all only [gt_iff_lt, ge_iff_le]
  obtain ⟨left, right⟩ := hM
  exact le_trans ( left.id_le _ ) ha;
rw [ tendsto_order ]
simp_all only [gt_iff_lt, ge_iff_le, not_lt_zero', one_div, Filter.eventually_at
  IsEmpty.forall_iff, IsEmpty.exists_iff, implies_true, true_and]
intro a' a
obtain ⟨left, right⟩ := hM
have := h_find_greatest.eventually_gt_atTop [a'⁻¹]
simp_all only [Filter.eventually_atTop, ge_iff_le]
obtain ⟨w, h⟩ := this
exact ⟨ w, fun n hn => inv_lt_of_inv_lt₀ a <| by exact lt_of_lt_of_le ( Nat.lt_o
· -- For any ε > 0, choose K such that 1/(K+1) < ε. For n ≥ M K, we have g n = 1/(
have h_eps : ∀ ε > 0, ∃ N, ∀ n ≥ N, f (1 / (Nat.findGreatest (fun k => M k ≤ n)
  intro ε hε_pos
  obtain ⟨K, hK⟩ : ∃ K : ℕ, (K + 1 : ℝ≥0∞)⁻¹ < ε := by
    rcases ENNReal.exists_inv_nat_lt hε_pos.ne' with ⟨ K, hK ⟩
    use K
    simp_all only [gt_iff_lt, ge_iff_le]
    obtain ⟨left, right⟩ := hM
    exact lt_of_le_of_lt ( by gcongr ; norm_num ) hK;
use M K;
intros n hn
have h_k_ge_K : Nat.findGreatest (fun k => M k ≤ n) n ≥ K := by
  refine' Nat.le_findGreatest _ hn
  simp_all only [gt_iff_lt, ge_iff_le]
  obtain ⟨left, right⟩ := hM
  exact le_trans ( left.id_le _ ) hn;
have := hM.2 ( Nat.findGreatest ( fun k => M k ≤ n ) n ) n ?_
· simp_all only [gt_iff_lt, ge_iff_le, one_div]
  obtain ⟨left, right⟩ := hM
  refine le_trans this ( add_le_add_right ( le_trans ( by gcongr ) hK.le ) _ )
· simp_all only [gt_iff_lt, ge_iff_le]
  obtain ⟨left, right⟩ := hM
  have := Nat.findGreatest_eq_iff.mp ( by aesop : Nat.findGreatest ( fun k =>
    by_cases h : Nat.findGreatest ( fun k => M k ≤ n ) n = 0
    · simp_all
    · simp_all
refine' le_of_forall_pos_le_add fun ε hε => _;
refine' csInf_le _ _
· aesop
· aesop

```

```

/-
If  $x_k$  tends to  $L$  and  $g(n) = x_k$  for  $n$  in  $[T_k, T_{k+1})$  where  $T$  is strictly
-/
lemma tendsto_of_block_sequence { $\alpha$  : Type*} [TopologicalSpace  $\alpha$ ] { $x$  :  $\mathbb{N} \rightarrow \alpha$ }
  (hT : StrictMono T) {L :  $\alpha$ } (hx : Filter.atTop.Tendsto x ( $\square$  L)) (g :  $\mathbb{N} \rightarrow \alpha$ )
  (Filter.atTop.Tendsto g ( $\square$  L)) := by
  rw [Filter.tendsto_atTop] at hx
  intro s hs
  -- Let `a` be the witness from the definition of `Tendsto`.
  rcases hx s hs with (a, ha)
  use T a
  intros n hn
  -- Let `k` be such that  $T k \leq n < T (k + 1)$ .
  obtain (k, hk) :  $\exists k, T k \leq n \wedge n < T (k + 1)$  := by
    -- Since  $T$  is strictly increasing, the set  $\{k \mid T k \leq n\}$  is
    empty.
  have h_finite : Set.Finite {k |  $T k \leq n$ } := by
    rw [Set.finite_iff_bddAbove]
    exact (⟦_, (hT.id_le · |>).trans⟧)
  use h_finite.toFinset.max' (a, h_finite.mem_toFinset.mpr hn)
  constructor
  · exact h_finite.mem_toFinset.mp (Finset.max'_mem _ _)
  · rw [← not_le]
    intro h
    exact not_lt_of_ge (Finset.le_max' _ _ (h_finite.mem_toFinset.mpr h))
  rw [hg k n hk]
  exact ha k (le_of_not_gt fun hk' => by linarith [hT.monotone hk'])

/-
Given a lower bound sequence  $M$  and a property  $P$  that can always be satisfied
-/
lemma exists_increasing_sequence_with_property (M :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (P :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Prop}$ )
   $\exists T : \mathbb{N} \rightarrow \mathbb{N}, \text{StrictMono } T \wedge (\forall k, T k \geq M k) \wedge (\forall k, \exists n \in \text{Set.Ico } (T k) (T (k + 1)), P n (T k))$ 
  -- We construct  $T_k$  by induction.
  have hT_ind :  $\forall k : \mathbb{N}, \exists T : \mathbb{N} \rightarrow \mathbb{N}, \text{StrictMono } T \wedge (\forall k, T k \geq M k) \wedge (\forall n \in \text{Set.Ico } (T k) (T (k + 1)), P n (T k))$ 
  intro k
  induction k
  · choose! n hn using hP;
    use fun k => Nat.recOn k (M 0) fun k ih => Max.max (ih + 1) (Max.find (fun n => P n (T k)) n)
    simp_all only [ge_iff_le, le_sup_iff, add_le_add_iff_right, or_true,
      Nat.rec_zero, zero_add]
    refine (⟦?, ?, ?, ?⟧)
    · exact strictMono_nat_of_lt_succ fun k => by cases max_cases (M (k + 1))
    · intro k
      induction k <=> aesop;

```

```

      · intro k
        exact ( n k ( Nat.rec ( M 0 ) ( fun k ih => Max.max ( M ( k + 1 ) ) ( n k ih )
      · exact Or.inr ( by linarith [ hn 0 ( M 0 ) ] );
    · rename_i ih
      obtain ( T, hT1, hT2, hT3, hT4 ) := ih
      use T
      aesop
    exact Exists.elim ( hT_ind 0 ) fun T hT => ( T, hT.1, hT.2.1, hT.2.2.1 )

/-
If g is a block sequence constructed from x and T, and each block contains a witness
-/
lemma liminf_le_of_block_sequence_witnesses {α : Type*} (y : ℝ≥0) (f : α → ℕ → ℝ≥0∞)
  (T : ℕ → ℕ) (hT : StrictMono T) (x : ℕ → α) (g : ℕ → α)
  (hg : ∀ k, ∀ n ∈ Set.Ico (T k) (T (k + 1)), g n = x k)
  (hwit : ∀ k, ∃ n ∈ Set.Ico (T k) (T (k + 1)), f (x k) n ≤ (y : ℝ≥0∞) + (k + 1 :
  Filter.atTop.liminf (fun n ↦ f (g n) n) ≤ y := by
  rw [ Filter.liminf_eq ];
  simp_all only [Set.mem_Ico, and_imp, ne_eq, add_eq_zero, Nat.cast_eq_zero, one_ne_
  not_false_eq_true, ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast, ENNRea
  Filter.eventually_atTop, ge_iff_le, sSup_le_iff, Set.mem_setOf_eq, forall_exists
  intro b x_1 h
  -- Fix an arbitrary $k \ge x_1$.
  suffices h_suff : ∀ k ≥ x_1, ∃ n ≥ k, f (g n) n ≤ y + 1 / (k + 1) by
  -- By taking the limit as $k$ approaches infinity, we get $b \le y$.
  have h_lim : Filter.Tendsto (fun k : ℕ => y + 1 / (k + 1 : ℝ≥0∞)) Filter.atTop (
  have h_lim : Filter.Tendsto (fun k : ℕ => (k + 1 : ℝ≥0∞)-1) Filter.atTop (□ 0)
  rw [ ENNReal.tendsto_nhds_zero ]
  intro ε a
  simp_all only [ge_iff_le, one_div, gt_iff_lt, Filter.eventually_atTop]
  rcases ENNReal.exists_inv_nat_lt a.ne' with ( N, hN );
  exact ( N, fun n hn => le_trans ( by gcongr ; norm_cast ; linarith ) hN.le )
  simp using tendsto_const_nhds.add h_lim;
  refine' le_of_tendsto_of_tendsto tendsto_const_nhds h_lim _;
  filter_upwards [ Filter.eventually_ge_atTop x_1 ] with k hk using by obtain ( n,
  intro k hk
  obtain (n, hn1, hn2) := hwit k;
  exact ( n, le_trans ( show k ≤ T k from hT.id_le _ ) hn1.1, by rw [ hg k n hn1.1 h

/-
If g is a block sequence constructed from x and T, and f is bounded by y + 1/(k+1) o
-/
lemma limsup_le_of_block_sequence_bound {α : Type*} (y : ℝ≥0) (f : α → ℕ → ℝ≥0∞)
  (T : ℕ → ℕ) (hT : StrictMono T) (x : ℕ → α) (g : ℕ → α)
  (hg : ∀ k, ∀ n ∈ Set.Ico (T k) (T (k + 1)), g n = x k)
  (hbound : ∀ k, ∀ n ∈ Set.Ico (T k) (T (k + 1)), f (x k) n ≤ (y : ℝ≥0∞) + (k + 1 :

```

```

Filter.atTop.limsup (fun n ↦ f (g n) n) ≤ y := by
  refine' le_of_forall_pos_le_add fun ε hε => _;
  refine' csInf_le _ _
  · aesop
  simp_all only [Set.mem_Ico, and_imp, ne_eq, add_eq_zero, Nat.cast_eq_zero,
    not_false_eq_true, ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast,
    Filter.eventually_map, Filter.eventually_atTop, ge_iff_le, Set.mem_se]
  -- Choose $K$ such that for all $k \ge K$, we have $1/(k+1) \le \epsilon$
  obtain (K, hK) : ∃ K : ℕ, ∀ k ≥ K, (k + 1 : ℝ≥0)-1 ≤ ε := by
    rcases ENNReal.lt_iff_exists_nnreal_btwn.mp hε with (δ, hδ, hδε)
    simp_all only [ENNReal.coe_pos, ge_iff_le, ne_eq, add_eq_zero, Nat.cast_eq_zero,
      not_false_eq_true, ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast,
      refine' ( [δ-1]+, fun k hk => le_trans _ hδε.le )
    norm_cast
    rw [ ENNReal.inv_le_iff_le_mul ] <=> norm_cast
    · rw [ ← div_le_iff_0 ] at * <=> norm_cast at *
      simp_all only [Nat.ceil_le, Nat.cast_add, Nat.cast_one, one_div]
      exact le_add_of_le_of_nonneg hk zero_le_one;
    · aesop
    · aesop
  use T K
  intro b a
  simp_all only [ge_iff_le, ne_eq, add_eq_zero, Nat.cast_eq_zero, one_ne_zero,
    ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natCast, ENNReal.coe_on]
  -- Let $k$ be such that $b \in [T_k, T_{k+1})$.
  obtain (k, hk) : ∃ k, T k ≤ b ∧ b < T (k + 1) := by
    -- Since $T$ is strictly increasing and unbounded, the set $\{n \mid T n \le b\}$ is
    empty.
    have h_finite : Set.Finite {n | T n ≤ b} := by
      exact Set.finite_iff_bddAbove.2 ( b, fun n hn => le_trans ( hT.id_1 )
        exact ( Finset.max' ( h_finite.toFinset ) ( K, h_finite.mem_toFinset )
      rw [ hg k b hk.1 hk.2 ];
      exact le_trans ( hbound k b hk.1 hk.2 ) ( add_le_add_right ( hK k ( le_

/- Version of `exists_liminf_zero_of_forall_liminf_le_with_UB` that lets you
two different functions simultaneously, one with liminf and one with limsup
/
lemma exists_liminf_zero_of_forall_liminf_limsup_le_with_UB (y1 y2 : ℝ≥0) (
  {z : ℝ≥0} (hz : 0 < z)
  (hf1 : ∀ x > 0, Filter.atTop.liminf (f1 x) ≤ y1)
  (hf2 : ∀ x > 0, Filter.atTop.limsup (f2 x) ≤ y2) :
  ∃ g : ℕ → ℝ≥0, (∀ x, g x > 0) ∧ (∀ x, g x < z) ∧
    Filter.atTop.Tendsto g (□ 0) ∧
    Filter.atTop.liminf (fun n ↦ f1 (g n) n) ≤ y1 ∧
    Filter.atTop.limsup (fun n ↦ f2 (g n) n) ≤ y2 := by
  -- Fix some sequences of positive real numbers $x_k$ and $N_0(k)$.

```



```

obtain ⟨x, hx⟩ : ∃ x : ℕ → ℝ≥0, (∀ k, 0 < x k) ∧ (∀ k, x k ≤ z / 2) ∧ (∀ k, x k ≤
  use fun k => Min.min ( z / 2 ) ( 1 / ( k + 1 ) )
  simp_all only [gt_iff_lt, one_div, lt_inf_iff, div_pos_iff_of_pos_left, Nat.ofNat
    Nat.cast_pos, zero_lt_one, or_true, and_self, implies_true, inf_le_left, inf_l
  rw [ Filter.tendsto_congr' ];
  any_goals filter_upwards [ Filter.eventually_gt_atTop [ ( z / 2 ) ^-1 ]+ ] with k
  · refine' tendsto_order.2 ( fun x => _, fun x hx => _ )
    · aesop
    · simp_all only [gt_iff_lt, Filter.eventually_atTop, ge_iff_le]
      exact ( [x^-1]+, fun n hn => inv_lt_of_inv_lt₀ hx <| lt_of_le_of_lt ( Nat.le
  · rw [ inv_le_comm₀ ] <=> norm_cast
    · simp_all only [inv_div, Nat.cast_add, Nat.cast_one]
      exact le_trans ( Nat.le_ceil _ ) ( mod_cast by linarith )
    · aesop
    · aesop
obtain ⟨N0, hN0⟩ : ∃ N0 : ℕ → ℕ, ∀ k, ∀ n ≥ N0 k, f₂ (x k) n ≤ y₂ + (k + 1 : ℝ≥0)⁻¹
  have h_limsup : ∀ k, Filter.limsup (f₂ (x k)) Filter.atTop ≤ y₂ := by
    exact fun k => hf₂ _ ( hx.1 k );
  have h_limsup_le : ∀ k, ∀ ε > 0, ∃ N, ∀ n ≥ N, f₂ (x k) n ≤ y₂ + ε := by
    intro k ε hε_pos
    have h_limsup_le : Filter.limsup (f₂ (x k)) Filter.atTop ≤ y₂ := h_limsup k;
    rw [ Filter.limsup_eq ] at h_limsup_le;
    have := exists_lt_of_csInf_lt ( show { a : ℝ≥0∞ | ∀ᶜ n in Filter.atTop, f₂ (x
    simp_all only [gt_iff_lt, one_div, ne_eq, add_eq_zero, Nat.cast_eq_zero, one_n
      not_false_eq_true, NNReal.le_inv_iff_mul_le, implies_true, Filter.eventually
    obtain ⟨left, right⟩ := hx
    obtain ⟨w, h⟩ := this
    obtain ⟨left_1, right⟩ := right
    obtain ⟨left_2, right_1⟩ := h
    obtain ⟨left_3, right⟩ := right
    obtain ⟨w_1, h⟩ := left_2
    exact ( w_1, fun n hn => le_trans ( h n hn ) right_1.le );
    exact ( fun k => Classical.choose ( h_limsup_le k _ <| by positivity ), fun k n
-- Define the sequence $T_k$ such that $T_k \geq N_0(k)$ and each interval $[T_k$,
obtain ⟨T, hT_mono, hT_bound, hT_wit⟩ : ∃ T : ℕ → ℕ, StrictMono T ∧ (∀ k, T k ≥ N0
  have hP : ∀ k L, ∃ n ≥ L, f₁ (x k) n ≤ y₁ + (k + 1 : ℝ≥0)⁻¹ := by
    intro k L; specialize hf₁ ( x k ) ( hx.1 k ) ; rw [ Filter.liminf_eq ] at hf₁;
    contrapose! hf₁;
    exact ( y₁ + ( k + 1 : ℝ≥0∞ ) ^-1, L, fun n hn => le_of_lt ( hf₁ n hn ), ENNReal
    have := exists_increasing_sequence_with_property ( fun k => N0 k ) ( fun k n =>
  refine' ( fun n => x ( Nat.find ( show ∃ k, n < T ( k + 1 ) from _ ) ), _, _, _, _
  exact ( n, hT_mono.id_le _ );
  · exact fun n => hx.1 _;
  · intro n
    specialize hx
    simp_all only [gt_iff_lt, ge_iff_le, ne_eq, add_eq_zero, Nat.cast_eq_zero, one_n

```

```

    not_false_eq_true, ENNReal.coe_inv, ENNReal.coe_add, ENNReal.coe_natC
    NNReal.le_inv_iff_mul_le]
  obtain ⟨left, right⟩ := hx
  obtain ⟨left_1, right⟩ := right
  obtain ⟨left_2, right⟩ := right
  exact lt_of_le_of_lt ( left_1 _ ) ( half_lt_self hz );
· refine' hx.2.2.2.comp _;
  refine' Filter.tendsto_atTop_atTop.mpr _;
  intro b; use T b; intro a ha; contrapose! ha
  simp_all only [gt_iff_lt, one_div, ne_eq, add_eq_zero, Nat.cast_eq_zero,
    not_false_eq_true, NNReal.le_inv_iff_mul_le, ge_iff_le, ENNReal.coe_i
    ENNReal.coe_one, Set.mem_Ico, Nat.find_lt_iff]
  obtain ⟨left, right⟩ := hx
  obtain ⟨w, h⟩ := ha
  obtain ⟨left_1, right⟩ := right
  obtain ⟨left_2, right_1⟩ := h
  obtain ⟨left_3, right⟩ := right
  exact right_1.trans_le ( hT_mono.monotone ( by linarith ) );
· refine liminf_le_of_block_sequence_witnesses y_1 f_1 T hT_mono x _ ?_ hT_
  intro k n hn
  congr
  simp_all only [gt_iff_lt, one_div, ne_eq, add_eq_zero, Nat.cast_eq_zero,
    not_false_eq_true, NNReal.le_inv_iff_mul_le, ge_iff_le, ENNReal.coe_i
    ENNReal.coe_one, Set.mem_Ico]
  obtain ⟨left, right⟩ := hx
  obtain ⟨left_1, right_1⟩ := hn
  obtain ⟨left_2, right⟩ := right
  obtain ⟨left_3, right⟩ := right
  rw [ Nat.find_eq_iff ]
  simp_all only [not_lt, true_and]
  intro n_1 a
  exact le_trans ( hT_mono.monotone ( by linarith ) ) left_1
· -- By definition of $g$, we know that for each $k$, $g(n) = x_k$ for $n
  apply limsup_le_of_block_sequence_bound
· exact hT_mono
· intro k n hn
  congr! 1;
  rw [ Nat.find_eq_iff ]
  simp_all only [gt_iff_lt, one_div, ne_eq, add_eq_zero, Nat.cast_eq_zero,
    not_false_eq_true, NNReal.le_inv_iff_mul_le, ge_iff_le, ENNReal.coe_i
    ENNReal.coe_one, Set.mem_Ico, not_lt, true_and]
  intro n_1 a
  obtain ⟨left, right⟩ := hx
  obtain ⟨left_1, right_1⟩ := hn
  obtain ⟨left_2, right⟩ := right
  obtain ⟨left_3, right⟩ := right

```

```

    exact le_trans ( hT_mono.monotone ( by linarith ) ) left_1
  · grind

--PULLOUT.
--PR? This is "not specific to our repo", but might be a bit too specialized to be i
--Definitely would need to clean up the proof first
theorem extracted_limsup_inequality (z :  $\mathbb{R}_{\geq 0\infty}$ ) (hz :  $z \neq \top$ ) (y x :  $\mathbb{N} \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (h_lem5
  : Filter.atTop.limsup (fun n  $\mapsto$  x n / n)  $\leq$  Filter.atTop.limsup (fun n  $\mapsto$  y n / n))
  --Thanks Aristotle!
simp? [Filter.limsup_eq] says simp only [Filter.limsup_eq, Filter.eventually_atTop
  ge_iff_le, le_sInf_iff, Set.mem_setOf_eq, forall_exists_index]
  -- Taking the limit superior of both sides of the inequality  $x_n / n \leq y_n / n + z$ 
  -- get limsup  $x_n / n \leq \limsup (y_n / n + z / n)$ .
intro b n h_bn
have h_le :  $\forall m \geq n, x\ m / (m : \mathbb{R}_{\geq 0\infty}) \leq b + z / (m : \mathbb{R}_{\geq 0\infty}) :=$  by
  intro m hm
  grw [ $\leftarrow$  h_bn m hm,  $\leftarrow$  ENNReal.add_div, h_lem5 m]
  -- Since z is finite, we have  $\lim z / n = 0$ .
have h_z_div_n_zero : Filter.atTop.Tendsto (fun n :  $\mathbb{N} \mapsto z / (n : \mathbb{R}_{\geq 0\infty})$ ) ( $\square$  0) := b
  rw [ENNReal.tendsto_nhds_zero]
  intro  $\varepsilon$  h $\varepsilon$ 
  rw [gt_iff_lt, ENNReal.lt_iff_exists_real_btwn] at h $\varepsilon$ 
  rcases h $\varepsilon$  with ( $\varepsilon'$ , h $\varepsilon_1$ , h $\varepsilon_2$ )
  rw [ENNReal.ofReal_pos] at h $\varepsilon_2$ 
  -- Since z is finite, we can choose k such that for all  $b \geq k$ ,  $z \leq b * \varepsilon'$ .
obtain (k, hk) :  $\exists k : \mathbb{N}, \forall b \geq k, z \leq b * \text{ENNReal.ofReal } \varepsilon' :=$  by
  rcases ENNReal.lt_iff_exists_real_btwn.mp (show  $z < \top$  by finiteness) with (a,
    use [a /  $\varepsilon'$ ])
  intro n hn
  grw [ha.le,  $\leftarrow$  ENNReal.ofReal_natCast,  $\leftarrow$  ENNReal.ofReal_mul (by positivity)]
  gcongr
  nlinarith [Nat.ceil_le.mp hn, mul_div_cancel₀ a h $\varepsilon_2$ .1.ne']
  -- Since  $z \leq b * \varepsilon'$  for all  $b \geq k$ , dividing both sides by b (which is positive)
  rw [Filter.eventually_atTop]
  use k + 1
  intros b _
  grw [ENNReal.div_le_iff_le_mul (by aesop) (by simp), hk b (by omega), mul_comm,
    refine le_of_forall_pos_le_add fun  $\varepsilon$  h $\varepsilon \mapsto$  ?_
  rcases Filter.eventually_atTop.mp (h_z_div_n_zero.eventually <| gt_mem_nhds h $\varepsilon$ ) wi
  apply sInf_le
  use n + m
  intro n hn
  grw [h_le n (by omega), (hm n (by omega)).le]

--PULLOUT and PR

```

```

open Filter in
/-- Like `Filter.tendsto_add_atTop_iff_nat`, but with nat subtraction. -
/
theorem _root_.Filter.tendsto_sub_atTop_iff_nat {α : Type*} {f : ℕ → α} {l : α}
  Tendsto (fun (n : ℕ) ↦ f (n - k)) atTop l ↔ Tendsto f atTop l :=
  show Tendsto (f ∘ fun n ↦ n - k) atTop l ↔ Tendsto f atTop l by
    rw [← tendsto_map'_iff, map_sub_atTop_eq_nat]

--PULLOUT and PR
open ENNReal Filter in
/-- Sort of dual to `ENNReal.tendsto_const_sub_nhds_zero_iff`. Takes a subspace
we don't actually have equality of the limits, or even the fact that the other
need to use `limsup`. -/
theorem _root_.ENNReal.tendsto_sub_const_nhds_zero_iff {α : Type*} {l : Filter}
  : Tendsto (f · - a) l (⊥ 0) ↔ limsup f l ≤ a := by
  rcases eq_or_ne a 0 with rfl | ha
  · simp [tendsto_const_nhds]
  rw [ENNReal.tendsto_nhds_zero, limsup_le_iff']
  simp only [tsub_le_iff_left]
  constructor
  · intro h y hy
    specialize h (y - a) (tsub_pos_of_lt hy)
    rwa [add_comm, tsub_add_cancel_of_le hy.le] at h
  · intro h ε hε
    exact h (a + ε) (lt_add_right ha hε.ne')

```

1.474 QuantumInfo/ForMathlib/LinearEquiv.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.InnerProductSpace.PiL2

variable {d d₁ d₂ d₃ R : Type*} [RCLike R]

namespace LinearEquiv

variable {R : Type*} [Semiring R]

variable (R) in
@[simps]
def of_relabel (e : d ≈ d₂) : (d₂ → R) ≈₁[R] (d → R) := by

```

```

    refine' { e.symm.piCongrLeft (fun _ ↦ R) with .. }
    <;> (intros; ext; simp [Equiv.piCongrLeft_apply])

variable (e : d ≈ d₂)

variable ([]) in
@[simps!]
def euclidean_of_relabel (e : d ≈ d₂) : EuclideanSpace [] d₂ ≈₁[] EuclideanSpace [] d
  (WithLp.linearEquiv 2 [] _).trans ((of_relabel _ e).trans (WithLp.linearEquiv 2 [] _))

@[simp]
theorem of_relabel_refl : of_relabel R (.refl d) = LinearEquiv.refl R (d → R) := by
  rfl

@[simp]
theorem euclidean_of_relabel_refl : euclidean_of_relabel [] (.refl d) =
  LinearEquiv.refl [] (EuclideanSpace [] d) := by
  rfl

end LinearEquiv

namespace Matrix

variable {R : Type*} [CommSemiring R]
variable [Fintype d] [DecidableEq d]
variable [Fintype d₂] [DecidableEq d₂]

theorem reindex_toLin' (e : d₁ ≈ d₃) (f : d₂ ≈ d) (M : Matrix d₁ d₂ R) :
  (M.reindex e f).toLin' = (LinearEquiv.of_relabel R e.symm) ∘₁
  M.toLin' ∘₁ (LinearEquiv.of_relabel R f) := by
  ext
  simp [mulVec, dotProduct, Equiv.piCongrLeft_apply]

theorem reindex_toEuclideanLin (e : d₁ ≈ d₃) (f : d₂ ≈ d) (M : Matrix d₁ d₂ [] ) :
  (M.reindex e f).toEuclideanLin = (LinearEquiv.euclidean_of_relabel [] e.symm) ∘₁
  M.toEuclideanLin ∘₁ (LinearEquiv.euclidean_of_relabel [] f) := by
  ext
  simp [mulVec, dotProduct, Equiv.piCongrLeft_apply]

theorem reindex_right_toLin' (e : d ≈ d₂) (M : Matrix d₃ d R) :
  (M.reindex (.refl d₃) e).toLin' = M.toLin' ∘₁ (LinearEquiv.of_relabel R e) := by
  rw [reindex_toLin']
  simp

theorem reindex_right_toEuclideanLin (e : d ≈ d₂) (M : Matrix d₃ d [] ) :
  (M.reindex (.refl d₃) e).toEuclideanLin =

```

```

      M.toEuclideanLin ◦1 (LinearEquiv.euclidean_of_relabel [] e) := by
    ext
    simp [mulVec, dotProduct, Equiv.piCongrLeft_apply]

theorem reindex_left_toLin' (e : d1 ≈ d3) (M : Matrix d1 d2 R) :
  (M.reindex e (.refl d2)).toLin' = (LinearEquiv.of_relabel R e.symm) ◦ M
  rw [Matrix.reindex_toLin']
  simp

theorem reindex_left_toEuclideanLin (e : d1 ≈ d3) (M : Matrix d1 d2 [] ) :
  (M.reindex e (.refl d2)).toEuclideanLin =
    (LinearEquiv.euclidean_of_relabel [] e.symm) ◦ M.toEuclideanLin := by
  rw [Matrix.reindex_toEuclideanLin]
  simp

end Matrix

```

1.475 QuantumInfo/ForMathlib/Majorization.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib
import QuantumInfo.ForMathlib.Matrix

/-! # Majorization and weak log-majorization

This file develops the theory of majorization for finite sequences, leading
the key singular value inequality needed for the Schatten–Hölder inequality

## Main results

* `sum_rpow_singularValues_mul_le`: for `r > 0`, the singular values of `A`
  satisfy  $\sum \sigma_i(AB)^r \leq \sum \sigma_i(A)^r \cdot \sigma_i(B)^r$ .
* `holder_step_for_singularValues`: the Hölder step giving
   $\sum \sigma_i(A)^r \cdot \sigma_i(B)^r \leq (\sum \sigma_i(A)^p)^{r/p} \cdot (\sum \sigma_i(B)^q)^{r/q}$ .
-/

open Finset BigOperators Matrix

variable {d : Type*} [Fintype d] [DecidableEq d]

```

noncomputable section

`/-! ## Sorted singular values -/`

`/-- The singular values of a square complex matrix `A`, defined as the square roots of the eigenvalues of `A†A`. These are indexed by `d` without a particular ordering.`

`Note: We use `A.conjTranspose` as the argument to `isHermitian_mul_conjTranspose_self` so that the underlying Hermitian matrix is `A† * A` (matching the convention in `schur`)`

`/`
`def singularValues (A : Matrix d d ℂ) : d → ℝ :=`
 `fun i => Real.sqrt ((isHermitian_mul_conjTranspose_self A.conjTranspose).eigenvalue i)`

`/-- Singular values are nonneg. -/`

`lemma singularValues_nonneg (A : Matrix d d ℂ) (i : d) :`
 `0 ≤ singularValues A i :=`
 `Real.sqrt_nonneg _`

`/-- The sorted singular values of a square complex matrix, in decreasing order, indexed by `Fin (Fintype.card d)`.`

`We define them by sorting the multiset of singular values. -`

`/`

`noncomputable def singularValuesSorted (A : Matrix d d ℂ) :`
 `Fin (Fintype.card d) → ℝ :=`
 `fun i =>`
 `let vals : Multiset ℝ := Finset.univ.val.map (singularValues A)`
 `let sorted := vals.sort (· ≥ ·)`
 `sorted.get {i.val, by rw [Multiset.length_sort]}; show i.val < (Multiset.map (singularValues A) sorted).length`

`/-- Sorted singular values are nonneg. -/`

`lemma singularValuesSorted_nonneg (A : Matrix d d ℂ) (i : Fin (Fintype.card d)) :`
 `0 ≤ singularValuesSorted A i := by`
 `have h_nonneg : ∀ i, 0 ≤ (singularValues A i) := by`
 `exact singularValues_nonneg A`
 `have h_sorted_nonneg : ∀ {l : List ℝ}, (∀ x ∈ l, 0 ≤ x) → ∀ i < l.length, 0 ≤ l[i]`
 `aesop`
 `contrapose! h_sorted_nonneg`
 `use Multiset.sort (Finset.univ.val.map (singularValues A)) (· ≥ ·)`
 `refine' {_, i, _, _}`
 `· aesop`
 `· simp [Finset.card_univ]`
 `· convert h_sorted_nonneg using 1`
 `unfold singularValuesSorted; aesop`

```

/-- The sum  $\sum$  singularValues A i ^ p equals the sum over sorted singular
/
lemma sum_singularValues_rpow_eq_sum_sorted (A : Matrix d d  $\mathbb{C}$ ) (p :  $\mathbb{R}$ ) :
   $\sum i : d, \text{singularValues } A \ i \ ^p =$ 
   $\sum i : \text{Fin } (\text{Fintype.card } d), \text{singularValuesSorted } A \ i \ ^p := \text{by}$ 
  have h_perm : Multiset.map (fun i => singularValues A i) Finset.univ.val
  have h_multiset : Multiset.map (fun i => singularValues A i) Finset.univ.val
  aesop
  convert h_multiset using 1
  refine' congr_arg _ (List.ext_get _ _) <=> simp [singularValuesSorted]
  have h_sum_eq : Multiset.sum (Multiset.map (fun x => x ^ p) (Multiset.map
    rw [h_perm]
  convert h_sum_eq using 1
  · erw [Multiset.map_map, Finset.sum_eq_multiset_sum]
  rfl
  · simp [Finset.sum]
  rfl

/-! ## Weak log-majorization and its consequences -/

/-- Sorted singular values are antitone (decreasing). -/
lemma singularValuesSorted_antitone (A : Matrix d d  $\mathbb{C}$ ) :
  Antitone (singularValuesSorted A) := by
  intro i j hij
  have h_sorted : List.Pairwise ( $\cdot \geq \cdot$ ) (Finset.univ.val.map (singularValue
    exact Multiset.pairwise_sort _ _
  exact h_sorted.rel_get_of_le hij

/-- The product of nonneg antitone sequences is antitone. -
/
lemma antitone_mul_of_antitone_nonneg {n :  $\mathbb{N}$ }
  {f g : Fin n  $\rightarrow \mathbb{R}$ } (hf : Antitone f) (hg : Antitone g)
  (hf_nn :  $\forall i, 0 \leq f \ i$ ) (hg_nn :  $\forall i, 0 \leq g \ i$ ) :
  Antitone (fun i => f i * g i) := by
  exact fun i j hij => mul_le_mul (hf hij) (hg hij) (hg_nn _) (hf_nn _)

/-! ### Compound matrices and auxiliary lemmas for Horn's inequality

The proof of Horn's inequality uses the *compound matrix* (or *k-
th exterior power*
of a matrix). For a  $d \times d$  matrix  $M$  and  $k \leq \text{card } d$ , the  $k$ -
th compound matrix
 $C_k(M)$  is indexed by  $k$ -element subsets of the index type, with entry  $\det M[S, T]$ .

```

The key properties are:

1. **Cauchy–Binet**: $C_k(M \ N) = C_k(M) \cdot C_k(N)$.
2. **Spectral characterisation**: the largest singular value of $C_k(M)$ is $\prod_{i=1}^k \sigma_i(M)$.
3. **Operator-norm submultiplicativity**: $\sigma_1(A \ B) \leq \sigma_1(A) \cdot \sigma_1(B)$.

Combining these gives Horn's inequality:

$$\prod \sigma_i(AB) = \sigma_1(C_k(AB)) = \sigma_1(C_k(A) \ C_k(B)) \leq \sigma_1(C_k(A)) \sigma_1(C_k(B)) \\ = (\prod \sigma_i(A)) (\prod \sigma_i(B)).$$

-/

namespace AllOrdered

/-- A `LinearOrder` on any `Fintype`, obtained classically via well-ordering. -/

scoped instance fintypeLinearOrderClassical (α : Type*) [Fintype α] [DecidableEq α]
 LinearOrder α := by
 classical
 exact linearOrderOfSTO (WellOrderingRel)

end AllOrdered

open scoped AllOrdered

/-- The `k`-th compound (exterior-power) matrix of `M`. -
 /

noncomputable def compoundMatrix (M : Matrix d d ℂ) (k : ℕ) :
 Matrix {S : Finset d // S.card = k} {S : Finset d // S.card = k} ℂ :=
 fun S T =>
 @Matrix.det (Fin k) _ _ ℂ _
 (M.submatrix (fun i => S.1.orderEmbOfFin S.2 i) (fun j => T.1.orderEmbOfFin T.2 j))

set_option maxHeartbeats 400000 in

/-- **Cauchy–Binet formula** for rectangular matrices: if `A` is `m × n` and `B` is `n × m`, then $\det(A * B) = \sum_S \det(A[:,S]) * \det(B[S,:])$ where the sum is over `m`-element subsets `S` of the column/row index. -/

lemma cauchyBinet {m : ℕ} {n : Type*} [Fintype n] [DecidableEq n] [LinearOrder n]
 {R : Type*} [CommRing R]
 (A : Matrix (Fin m) n R) (B : Matrix n (Fin m) R) :
 (A * B).det = $\sum S : \{S : \text{Finset } n // S.\text{card} = m\},$
 (A.submatrix id (S.1.orderEmbOfFin S.2)).det *
 (B.submatrix (S.1.orderEmbOfFin S.2) id).det := by
 have h_cauchy_binet : ∀ (A : Matrix (Fin m) n R) (B : Matrix n (Fin m) R), Matrix.
 simp [Matrix.det_apply']
 simp [Matrix.mul_apply, Finset.mul_sum]
 intro A B; rw [← Finset.sum_comm]; congr; ext x; simp [mul_comm]
 simp only [prod_sum, sum_mul]

```

refine' Finset.sum_bij (fun f _ => fun i => f (x.symm i) (Finset.mem_univ i))
· simp
· simp only [univ_pi_univ, mem_univ, funext_iff, forall_true_left, forall_true_right]
  exact fun a1 a2 h i => by simp using h (x i)
· simp only [mem_univ, univ_pi_univ, exists_const, forall_const]
  exact fun b => ⟨fun i _ => b (x i), by ext i; simp⟩
· simp only [univ_pi_univ, mem_univ, prod_mul_distrib, prod_attach_univ,
  Equiv.symm_apply_apply, forall_const]
  intro a
  rw [← Equiv.prod_comp x.symm]
  ring_nf
  rw [← Equiv.prod_comp x.symm]
  simp only [mul_comm, mul_assoc]
  conv_rhs => rw [← Equiv.prod_comp x]
  simp [Equiv.symm_apply_apply]
-- Split the sum into injective and non-injective functions.
have h_split :  $\sum \sigma : \text{Fin } m \rightarrow n, (\prod i, A\ i\ (\sigma\ i)) * \text{Matrix.det } (\text{Matrix.of } \sigma)$ 
  refine Finset.sum_congr rfl fun  $\sigma$  _ => ?_
  split_ifs with h $\sigma$  <=> simp_all [Function.Injective]
  obtain ⟨i, j, hij, hne⟩ := h $\sigma$ 
  exact mul_eq_zero_of_right _ (Matrix.det_zero_of_row_eq hne (by ext1; simp))
-- Group the sum by the image of the injective functions.
have h_group :  $\sum \sigma : \text{Fin } m \rightarrow n, (\text{if } \text{Function.Injective } \sigma \text{ then } (\prod i, A\ i\ (\sigma\ i)) \text{ else } 0)$ 
  rw [← Finset.sum_comm, Finset.sum_congr rfl]
  intro  $\sigma$  _
  by_cases h $\sigma$  : Function.Injective  $\sigma$ 
  · simp [h $\sigma$ ]
    rw [Finset.sum_eq_single ⟨Finset.image  $\sigma$  Finset.univ, by simp [Finset.mem_image]⟩]
    · simp
    · grind
    · simp
  · simp [h $\sigma$ ]
-- For each subset  $S$  of size  $m$ , the inner sum is equal to the product of the elements of  $S$ .
have h_inner :  $\forall S : \{S : \text{Finset } n // S.\text{card} = m\}, \sum \sigma : \text{Fin } m \rightarrow n, (\text{if } \text{Function.Injective } \sigma \text{ then } (\prod i \in S, A\ i\ (\sigma\ i)) \text{ else } 0)$ 
  intro S
  have h_inner_sum :  $\sum \sigma : \text{Fin } m \rightarrow n, (\text{if } \text{Function.Injective } \sigma \wedge \text{Finset.image } \sigma \subseteq S \text{ then } (\prod i \in S, A\ i\ (\sigma\ i)) \text{ else } 0)$ 
    have h_inner_sum : Finset.filter (fun  $\sigma : \text{Fin } m \rightarrow n \Rightarrow \text{Function.Injective } \sigma \wedge \text{Finset.image } \sigma \subseteq S$ ) Finset.univ
      ext  $\sigma$ 
      simp [Finset.mem_image]
      constructor
      · intro h $\sigma$ 
        obtain ⟨a, ha⟩ :  $\exists a : \text{Fin } m \rightarrow \text{Fin } m, \forall i, \sigma\ i = S.\text{val.orderEmbOf } a\ i$ 
        have h_exists_a :  $\forall i, \exists a : \text{Fin } m, \sigma\ i = S.\text{val.orderEmbOf } a\ i$ 
        intro i
        have h_exists_a :  $\sigma\ i \in S.\text{val} :=$  by
          exact h $\sigma$ .2 ▸ Finset.mem_image_of_mem _ (Finset.mem_univ _)

```

```

      have h_exists_a : Finset.image (fun a : Fin m => S.val.orderEmbOffFin S)
        refine' Finset.eq_of_subset_of_card_le (Finset.image_subset_iff.mpr
          rw [Finset.card_image_of_injective _ fun a b h => by simp [Fin.ext]
        grind
      exact (fun i => Classical.choose (h_exists_a i), fun i => Classical.choose
    have ha_inj : Function.Injective a := by
      exact fun i j hij => h.1 <| by simp [ha, hij]
    exact (Equiv.ofBijective a (ha_inj, Finite.injective_iff_surjective.mpr ha
  · rintro (a, rfl)
    constructor
    · exact fun i j hij => a.injective <| by simp using hij
    · refine Finset.eq_of_subset_of_card_le (Finset.image_subset_iff.mpr fun i
      rw [Finset.card_image_of_injective _ fun i j hij => by simp [Fin.ext]
    rw [← Finset.sum_filter, h_inner_sum, Finset.sum_image]
    exact fun τ _ τ' _ h => Equiv.Perm.ext fun i => by simp using congr_fun h i
  rw [h_inner_sum, Matrix.det_apply', Matrix.det_apply']
  simp [Matrix.det_apply', Finset.sum_mul]
  refine' Finset.sum_bij (fun σ _ => σ⁻¹) _ _ _ _ <=> simp [Equiv.Perm.sign_inv]
  · exact fun b => (b⁻¹, inv_inv b)
  · intro σ
    rw [← Equiv.prod_comp σ⁻¹]
    simp [mul_assoc, mul_left_comm, Finset.mul_sum]
    refine' Finset.sum_bij (fun τ _ => σ * τ) _ _ _ _ <=> simp [Equiv.Perm.sign_mu
    · exact fun b => (σ⁻¹ * b, by simp)
    · cases' Int.units_eq_one_or (Equiv.Perm.sign σ) with h h <=> simp [h]
  rw [h_cauchy_binet, h_split, h_group, Finset.sum_congr rfl fun S hS => h_inner S]

/--
The `compoundMatrix` of a product is the product of the `compoundMatrix`s.
-/
lemma compoundMatrix_mul (M N : Matrix d d ℂ) (k : ℕ) :
  compoundMatrix (M * N) k = compoundMatrix M k * compoundMatrix N k := by
  ext1
  apply cauchyBinet

/-- `compoundMatrix` commutes with `conjTranspose`. -/
lemma compoundMatrix_conjTranspose (M : Matrix d d ℂ) (k : ℕ) :
  compoundMatrix M.conjTranspose k = (compoundMatrix M k).conjTranspose := by
  ext1
  unfold compoundMatrix
  rw [Matrix.conjTranspose_apply, ← Matrix.det_conjTranspose]
  simp

/--
The compound matrix of a diagonal matrix is diagonal, with entries being
products of eigenvalues over k-subsets. -/

```

```

lemma compoundMatrix_diagonal (f : d → ℂ) (k : ℕ) :
  compoundMatrix (Matrix.diagonal f) k =
    Matrix.diagonal (fun S : {S : Finset d // S.card = k} =>
      [] i : Fin k, f (S.1.orderEmbOffFin S.2 i)) := by
    ext S T; by_cases h : S = T <=> simp_all [Matrix.diagonal]
    · refine' Matrix.det_of_upperTriangular _ |> fun h => h.trans _
    · intro i j hij; aesop
    · aesop
    · -- Since $S \neq T$, there exists some $i \in S$ such that $i \notin T$
      obtain (i, hiS, hiT) : ∃ i ∈ S.val, i ∉ T.val := by
        contrapose! h
      exact Subtype.ext (Finset.eq_of_subset_of_card_le h (by simp [S.2, T.2])
        obtain (j, hj) : ∃ j : Fin k, (S.val.orderEmbOffFin S.2) j = i := by
          have h_row_zero : Finset.image (fun j : Fin k => S.val.orderEmbOffFin
            refine' Finset.eq_of_subset_of_card_le (Finset.image_subset_iff.mpr
              simp [Finset.card_image_of_injective, Function.Injective, *])
            exact Exists.elim (Finset.mem_image.mp (h_row_zero.symm ▸ hiS)) fun j
          -- Since the row corresponding to $i$ in the submatrix is all zeros, then
          have h_det_zero : Matrix.det (Matrix.of (fun i j => if (S.val.orderEmbOffFin
            rw [Matrix.det_eq_zero_of_row_eq_zero j]; aesop
          convert h_det_zero using 1

```

/--

The eigenvalues of the compound matrix of a Hermitian matrix are the products of eigenvalues over k -subsets. More precisely, the singular values of $\text{compoundMatrix } M \ k$ are the square roots of products of eigenvalues of $M \dagger M$ over k -subsets.

-/

```

lemma singularValues_compoundMatrix_eq (M : Matrix d d ℂ) (k : ℕ) :
  ∀ (S : {S : Finset d // S.card = k}),
  ∃ (j : {S : Finset d // S.card = k}),
  singularValues (compoundMatrix M k) j =
    [] i : Fin k, singularValues M (S.1.orderEmbOffFin S.2 i) := by
  unfold singularValues
  intro S
  have h_eigenvalues : ∃ σ : {S : Finset d // S.card = k} ≈ {S : Finset d // S.card = k}
    apply IsHermitian.eigenvalues_eq_of_unitary_similarity_diagonal
    rotate_right
  exact compoundMatrix (Matrix.IsHermitian.eigenvectorUnitary (isHermitian M))
  · have h_unitary : ∀ (U : Matrix d d ℂ), U ∈ unitaryGroup d ℂ → compoundMatrix (U † * M * U) k =
    intro U hU
    have h_unitary : (compoundMatrix U k).conjTranspose * compoundMatrix M k =
    have h_unitary : (compoundMatrix U k).conjTranspose * compoundMatrix M k =
    rw [← compoundMatrix_conjTranspose, ← compoundMatrix_mul]
    have h_unitary : UH * U = 1 := by
      exact hU.1.symm ▸ by simp

```

```

-- Since the identity matrix's compound matrix is the identity matrix, we
have h_id : compoundMatrix (1 : Matrix d d ℂ) k = 1 := by
  convert compoundMatrix_diagonal (fun _ => 1) k using 1; aesop
grind
have h_unitary' : compoundMatrix U k * (compoundMatrix U k).conjTranspose =
  rw [← mul_eq_one_comm, h_unitary]
exact (by
  exact h_unitary, by
  exact h_unitary')
exact h_unitary _ (by simp [unitaryGroup])
· have h_compoundMatrix_mul : compoundMatrix (M.conjTranspose * M) k = compoundM
  exact compoundMatrix_mul _ _ _
have h_compoundMatrix_conjTranspose : compoundMatrix M.conjTranspose k = (comp
  exact compoundMatrix_conjTranspose M k
have := Matrix.IsHermitian.spectral_theorem (isHermitian_mul_conjTranspose_sel
convert congr_arg (fun x => compoundMatrix x k) this using 1 <=> simp [h_compo
rw [compoundMatrix_mul, compoundMatrix_mul]
rw [← compoundMatrix_conjTranspose]
rw [compoundMatrix_diagonal]; simp [Matrix.mul_assoc]
congr! 3
obtain (σ, hσ) := h_eigenvalues
use σ S; simp_all [funext_iff]
rw [Real.sqrt_eq_iff_mul_self_eq] <=> norm_num [Finset.prod_nonneg, Real.sqrt_nonn
· rw [← Finset.prod_mul_distrib, Finset.prod_congr rfl fun _ _ => Real.mul_self_sq
  intro i hi; exact (by
    apply Matrix.eigenvalues_conjTranspose_mul_self_nonneg)
· refine' Finset.prod_nonneg fun i _ => _
  exact eigenvalues_conjTranspose_mul_self_nonneg M _

/-- The product of nonneg values over a k-subset is at most the product of the
k largest values. -/
lemma prod_le_prod_sorted {n : ℕ} {f : Fin n → ℝ}
  (hf : Antitone f) (hf_nn : ∀ i, 0 ≤ f i)
  (k : ℕ) (hk : k ≤ n)
  (g : Fin k → Fin n) (hg : Function.Injective g) :
  ∏ i : Fin k, f (g i) ≤ ∏ i : Fin k, f (i.val, by omega) := by
  -- The sorted values at positions g(i) are bounded by the sorted values at positio
  -- because g is injective so g(i) ≥ i (in the sorted sense), and f is antitone.
  -- We use the fact that for injective g : Fin k → Fin n, sorting g gives values ≥
  -- First sort g to get g' with g'(0) < g'(1) < ... < g'(k-
1)
  -- Then g'(i) ≥ i since we're choosing k distinct values from Fin n
  -- So f(g'(i)) ≤ f(i) by antitonicity
  -- And the product is preserved under sorting (it's the same set of values)
have h_exists_sorted : ∃ (g' : Fin k → Fin n),
  Function.Injective g' ∧ StrictMono g' ∧

```

```

    Finset.image g Finset.univ = Finset.image g' Finset.univ := by
    exact (Finset.orderEmbOffFin (Finset.image g Finset.univ) (by simp [Fins
      fun a b h => by simpa using h,
      fun a b h => by simpa using h,
      by ext x; simp)
    obtain ⟨g', hg'_inj, hg'_mono, hg'_eq⟩ := h_exists_sorted
    have h_prod_eq : [] i : Fin k, f (g i) = [] i : Fin k, f (g' i) := by
      rw [← Finset.prod_image (f := f) (fun a _ b _ h => hg (by simpa using h
        ← Finset.prod_image (f := f) (fun a _ b _ h => hg'_inj (by simpa us
      exact Finset.prod_congr hg'_eq (fun _ _ => rfl)
    rw [h_prod_eq]
    apply Finset.prod_le_prod (fun i _ => hf_nn _) (fun i _ => ?_)
    apply hf
    -- Need: i.val ≤ (g' i).val for strictly monotone g'
    -- By induction: g'(0) ≥ 0, and g'(j+1) > g'(j) ≥ j implies g'(j+1) ≥ j+1
    suffices h : ∀ (m : ℕ) (hm : m < k), m ≤ (g' ⟨m, hm⟩).val from h i.val i.
    intro m hm
    induction m with
    | zero => exact Nat.zero_le _
    | succ m ih =>
      have ihm := ih (by omega)
      have := hg'_mono (show (⟨m, by omega⟩ : Fin k) < ⟨m + 1, hm⟩ by simp [F
      omega

/-- The 0th sorted singular value is the maximum of the singular values. -
/
lemma singularValuesSorted_zero_eq_sup {e : Type*} [Fintype e] [DecidableEq
  (A : Matrix e e ℂ) (h : 0 < Fintype.card e) :
  singularValuesSorted A ⟨0, h⟩ = Finset.sup' Finset.univ
    (Finset.univ_nonempty_iff.mpr (Fintype.card_pos_iff.mp h))
    (singularValues A) := by
  refine' le_antisymm _ _
  · -- Since the list is sorted in decreasing order, every element in the l
    have h_le_sup : ∀ x ∈ Multiset.sort (Multiset.map (singularValues A) Fi
      aesop
    exact h_le_sup _ (by simp [singularValuesSorted])
  · have h_max_le_ge : ∀ i, singularValues A i ≤ singularValuesSorted A ⟨0,
    intro i
    have h_max_le_ge : ∀ j, singularValuesSorted A j ≤ singularValuesSort
      exact fun j => singularValuesSorted_antitone A (Nat.zero_le _)
    exact (by
      have h_max_le_ge : ∃ j, singularValues A i = singularValuesSorted A j
      have h_exists_j : singularValues A i ∈ Multiset.sort (Finset.univ.v
      simp [Multiset.mem_sort]
      obtain ⟨j, hj⟩ := List.mem_iff_get.mp h_exists_j
      exact ⟨j, by simpa using j.2, hj.symm⟩

```

```

aesop)
exact Finset.sup'_le _ _ fun i _ => h_max_le_ge i

/-- Each singular value appears in the sorted list. -/
lemma singularValues_mem_sorted {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (i : e) :
  ∃ j : Fin (Fintype.card e), singularValues A i = singularValuesSorted A j := by
  have h_mem : singularValues A i ∈ Multiset.sort (Finset.univ.val.map (singularValues))
  simp [Multiset.mem_sort]
  obtain ⟨j, hj⟩ := List.mem_iff_get.mp h_mem
  exact ⟨j, by simpa using j.2⟩, hj.symm)

/-- Each sorted singular value appears among the original singular values. -
/
lemma singularValuesSorted_mem_values {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (j : Fin (Fintype.card e)) :
  ∃ i : e, singularValuesSorted A j = singularValues A i := by
  have h_mem : singularValuesSorted A j ∈ Multiset.sort (Finset.univ.val.map (singularValues))
  simp [singularValuesSorted]
  rw [Multiset.mem_sort] at h_mem
  simp at h_mem
  obtain ⟨i, hi⟩ := h_mem
  exact ⟨i, hi.symm⟩

/-- Stronger version of `singularValues_compoundMatrix_eq` that exposes the permutation -
/
lemma singularValues_compoundMatrix_perm (M : Matrix d d ℂ) (k : ℕ) :
  ∃ σ : {S : Finset d // S.card = k} ≈ {S : Finset d // S.card = k},
  ∀ S, singularValues (compoundMatrix M k) (σ S) =
  [] i : Fin k, singularValues M (S.1.orderEmb0Fin S.2 i) := by
  unfold singularValues
  have h_eigenvalues : ∃ σ : {S : Finset d // S.card = k} ≈ {S : Finset d // S.card = k},
  apply IsHermitian.eigenvalues_eq_of_unitary_similarity_diagonal
  rotate_right
  exact compoundMatrix (Matrix.IsHermitian.eigenvectorUnitary (isHermitian_mul_conjTranspose M))
  · have h_unitary : ∀ (U : Matrix d d ℂ), U ∈ unitaryGroup d ℂ → compoundMatrix U k = 1
    intro U hU
    have h_unitary : (compoundMatrix U k).conjTranspose * compoundMatrix U k = 1
    have h_unitary : (compoundMatrix U k).conjTranspose * compoundMatrix U k = 1
    rw [← compoundMatrix_conjTranspose, ← compoundMatrix_mul]
    have h_unitary : UH * U = 1 := by
      exact hU.1.symm ▸ by simp
    have h_id : compoundMatrix (1 : Matrix d d ℂ) k = 1 := by
      convert compoundMatrix_diagonal (fun _ => 1) k using 1; aesop
    grind
    have h_unitary' : compoundMatrix U k * (compoundMatrix U k).conjTranspose = 1

```

```

      rw [← mul_eq_one_comm, h_unitary]
      exact (by exact h_unitary, by exact h_unitary')
    exact h_unitary _ (by simp [unitaryGroup])
  · have h_compoundMatrix_mul : compoundMatrix (M.conjTranspose * M) k =
    exact compoundMatrix_mul _ _
    have h_compoundMatrix_conjTranspose : compoundMatrix M.conjTranspose
    exact compoundMatrix_conjTranspose M k
    have := Matrix.IsHermitian.spectral_theorem (isHermitian_mul_conjTran
    convert congr_arg (fun x => compoundMatrix x k) this using 1 <=> simp
    rw [compoundMatrix_mul, compoundMatrix_mul]
    rw [← compoundMatrix_conjTranspose]
    rw [compoundMatrix_diagonal]; simp [Matrix.mul_assoc]
    congr! 3
  obtain (σ, hσ) := h_eigenvalues
  use σ
  intro S
  simp_all [funext_iff]
  rw [Real.sqrt_eq_iff_mul_self_eq] <=> norm_num [Finset.prod_nonneg, Real.
  · rw [← Finset.prod_mul_distrib, Finset.prod_congr rfl fun _ => Real.mu
    intro i hi; exact (by apply Matrix.eigenvalues_conjTranspose_mul_self_n
  · refine Finset.prod_nonneg fun i _ => ?_
    exact eigenvalues_conjTranspose_mul_self_nonneg M _

/-- Converse of `singularValues_compoundMatrix_eq`: every singular value of
    compound matrix is a product of singular values of M over some k-
subset. -/
lemma singularValues_compoundMatrix_rev (M : Matrix d d ℂ) (k : ℕ)
  (j : {S : Finset d // S.card = k}) :
  ∃ (S : {S : Finset d // S.card = k}),
    singularValues (compoundMatrix M k) j =
    [] i : Fin k, singularValues M (S.1.orderEmbOffFin S.2 i) := by
  obtain (σ, hσ) := singularValues_compoundMatrix_perm M k
  exact (σ.symm j, by rw [← hσ]; simp)

/-- There exists a bijection `σ : Fin (card d) ≈ d` such that
    `singularValues M (σ i) = singularValuesSorted M i` for all `i`. -
/
lemma exists_sorting_equiv (M : Matrix d d ℂ) :
  ∃ σ : Fin (Fintype.card d) ≈ d,
  ∀ i, singularValues M (σ i) = singularValuesSorted M i := by
  -- Apply the lemma `exists_subset_prod_eq_sorted_prod` to obtain the bije
  have h_bij : ∃ σ : Fin (Fintype.card d) ≈ d, ∀ i, singularValues M (σ i)
  have h_perm : Multiset.ofList (List.ofFn (singularValuesSorted M)) = Mu
  have h_multiset : Multiset.ofList (List.ofFn (singularValues M ∘ (Fin
  have h_multiset : Finset.univ.val = Multiset.map (fun i => (Fintype
  exact Eq.symm (Multiset.map_univ_val_equiv (Fintype.equivFin d |>

```



```

    rw [h_multiset, Multiset.map_map]
    aesop
    have h_multiset_sorted : Multiset.ofList (List.ofFn (singularValuesSorted M))
    have h_multiset_sorted_eq : List.ofFn (singularValuesSorted M) = Multiset.sorted
      refine' List.ext_get _ _ <;> simp [List.ofFn_eq_map] at *; aesop (simp_con
    exact h_multiset_sorted_eq ▸ by simp
    rw [h_multiset, h_multiset_sorted]
    have h_perm : List.Perm (List.ofFn (singularValuesSorted M)) (List.ofFn (singula
    exact Multiset.coe_eq_coe.mp h_perm
    have h_perm : ∃ σ : Fin (Fintype.card d) ≈ Fin (Fintype.card d), ∀ i, singularVa
    have h_perm : ∀ {l1 l2 : List ℝ}, l1.Perm l2 → ∃ σ : Fin l1.length ≈ Fin l2.le
    intros l1 l2 h_perm; induction' h_perm with l1 l2 h_perm ih <;> simp_all
    · obtain ⟨σ, hσ⟩ := <∃ σ : Fin l2.length ≈ Fin h_perm.length, ∀ i : Fin l2.l
      simp_all [Fin.forall_fin_succ]
      · rcases i with (⟨_ | i, hi⟩ <;> rcases j with (⟨_ | j, hj⟩ <;> simp_all [Fi
        simp [Fin.ext_iff] using σ.injective (Fin.ext hij)
      · refine i.cases ?_ ?_
        · simp [Fin.exists_fin_succ]
        · simp only [Fin.exists_fin_succ, Fin.cases_zero, Fin.cases_succ, Fin.su
          exact fun i => Or.inr {σ.symm i, by simp}
      · refine' (Equiv.swap ⟨0, by simp⟩ ⟨1, by simp⟩, _); simp [Fin.forall_fin_su
    · rename_i h1 h2 h3 h4
      obtain ⟨σ1, hσ1⟩ := h3
      obtain ⟨σ2, hσ2⟩ := h4
      use σ1.trans σ2
      simp [hσ1, hσ2]
    obtain ⟨σ, hσ⟩ := h_perm <_>
    refine (Equiv.ofBijective (fun i => ⟨σ ⟨i, ?_⟩, ?_⟩) ⟨?_, ?_⟩, ?_)
    · simp
    · exact lt_of_lt_of_le (Fin.is_lt _) (by simp)
    · norm_num [Function.Injective, Function.Surjective]
      exact fun i j hij => Fin.ext <| by simp [Fin.ext_iff] using σ.injective <|
    · intro b
      obtain ⟨a, ha⟩ : ∃ a : Fin (List.ofFn (singularValuesSorted M)).length, σ a
        exact σ.surjective _
      use ⟨a, lt_of_lt_of_le a.2 (by simp)⟩
      exact Fin.ext (by simp [ha])
    · intro i
      specialize hσ ⟨i, by simp⟩
      simp_all
    exact (Equiv.trans h_perm.choose (Fintype.equivFin d |> Equiv.symm), fun i => h_
    exact ⟨h_bij.choose, fun i => by simp only [Equiv.symm_apply_eq] using h_bij.choo

```

/-- For any k-subset S of d, the product of singular values over S is ≤ the
product of the top k sorted singular values. -/

lemma prod_singularValues_subset_le_sorted_prod (M : Matrix d d ℂ) (k : ℕ)

```

    (hk : k ≤ Fintype.card d) (S : {S : Finset d // S.card = k}) :
    [] i : Fin k, singularValues M (S.1.orderEmbOffFin S.2 i) ≤
    [] i : Fin k, singularValuesSorted M {i.val, by omega} := by
  obtain {σ, hσ} := exists_sorting_equiv M
  -- Define g : Fin k → Fin (card d) as σ-1 ∘ (S.orderEmbOffFin)
  set g : Fin k → Fin (Fintype.card d) := fun i => σ.symm (S.1.orderEmbOffFin i)
  have hg_eq : ∀ i, singularValues M (S.1.orderEmbOffFin S.2 i) = singularValuesSorted M {i.val, by omega}
  intro i
  simp [g]
  rw [← hσ]
  simp
  simp_rw [hg_eq]
  apply prod_le_prod_sorted (singularValuesSorted_antitone M) (singularValuesSorted_val)
  intro i j hij
  simpa [g] using congr_arg σ hij

set_option maxHeartbeats 800000 in
lemma exists_subset_prod_eq_sorted_prod (M : Matrix d d ℂ) (k : ℕ)
  (hk : k ≤ Fintype.card d) :
  ∃ S : {S : Finset d // S.card = k},
  [] i : Fin k, singularValues M (S.1.orderEmbOffFin S.2 i) =
  [] i : Fin k, singularValuesSorted M {i.val, by omega} := by
  by_contra h_contra
  obtain {σ, hσ} : ∃ σ : Fin (Fintype.card d) ≈ d, ∀ i : Fin (Fintype.card d),
    have h_perm : Multiset.ofList (List.ofFn (singularValuesSorted M)) = Multiset.ofList (List.ofFn (singularValuesSorted M))
    have h_perm : Multiset.ofList (List.ofFn (singularValuesSorted M)) = Multiset.ofList (List.ofFn (singularValuesSorted M))
    have h_sorted : List.ofFn (singularValuesSorted M) = Multiset.sort (List.ofFn (singularValuesSorted M))
    refine' List.ext_get _ _ <=> simp
    exact fun n h1 h2 => rfl
  simp [h_sorted, List.ofFn_eq_map]
  refine' Multiset.eq_of_le_of_card_le _ _ <=> simp [Multiset.le_iff_card_le]
  intro a; rw [Multiset.count_map]; simp [List.count]
  rw [← Multiset.toFinset_card_of_nodup] <=> norm_num [Finset.card_image]
  · rw [List.countP_eq_length_filter]
  · simp [Finset.card]
  · rw [← List.toFinset_card_of_nodup] <=> norm_num [List.nodup_finRange]
  · rw [Finset.card_filter]; simp [eq_comm, Fintype.equivFin]
  · rw [← Multiset.toFinset_card_of_nodup] <=> norm_num [Finset.card_image]
  · refine' le_of_eq _; rw [Finset.card_filter, Finset.card_filter]
    · exact fun x _ y _ h => Fintype.equivFin d |>.injective h
    · exact fun b _ => ((Fintype.equivFin d).symm b, Finset.mem_univ)
    · simp [Fintype.equivFin]
    · exact Multiset.Nodup.filter _ (Finset.nodup _)
    · exact List.Nodup.filter _ (List.nodup_finRange _)
    · exact Multiset.Nodup.filter _ (Finset.nodup _)
  exact h_perm.trans (by simp)

```

```

have h_perm : List.Perm (List.ofFn (singularValuesSorted M)) (List.ofFn (singularValuesSorted M))
  exact Multiset.coe_eq_coe.mp h_perm
have h_perm :  $\exists \sigma : \text{Fin } (\text{Fintype.card } d) \approx \text{Fin } (\text{Fintype.card } d), \text{List.ofFn } (\text{singularValuesSorted } M) \approx \text{List.ofFn } (\text{singularValuesSorted } M)$ 
  have := h_perm
  have h_perm :  $\forall \{l_1 \ l_2 : \text{List } \mathbb{R}\}, \text{List.Perm } l_1 \ l_2 \rightarrow \exists \sigma : \text{Fin } l_1.\text{length} \approx \text{Fin } l_2.\text{length}$ 
    intro l_1 l_2 h_perm
    induction' h_perm with l_1 l_2 h_perm ih
    · aesop
    · rename_i a_ih
      obtain  $\langle \sigma, h\sigma \rangle := a_ih$ 
      use Equiv.ofBijective (fun i => Fin.cases {0, by simp} (fun i => Fin.succ i))
      · simp [List.finRange_succ] at *
        exact hσ
      · rcases i with  $\langle \_ \mid i, hi \rangle <;> \text{rcases } j \text{ with } \langle \_ \mid j, hj \rangle <;> \text{simp [Fin.ext_iff, simp_a [Fin.ext_iff]] using } \sigma.\text{injective } (\text{Fin.ext } hij) \mid > \text{fun } h \Rightarrow \text{by simp_a [Fin.ext_iff]}$ 
      · rcases i with  $\langle \_ \mid i, hi \rangle$ 
        · exact  $\langle \{0\}, \text{by simp} \rangle, \text{rfl}$ 
        · exact  $\langle \text{Fin.succ } (\sigma.\text{symm } i, \text{by simp_a using } hi) \rangle, \text{by simp}$ 
      · use Equiv.swap {0, by simp} {1, by simp}; simp [List.finRange_succ]
        refine' List.ext_get _ _ <;> simp [Function.comp]
        intro n hn hn'; rcases n with  $\langle \_ \mid \_ \mid n \rangle <;> \text{trivial}$ 
    · rename_i h_1 h_2 h_3 h_4
      obtain  $\langle \sigma_1, h\sigma_1 \rangle := h_3$ 
      obtain  $\langle \sigma_2, h\sigma_2 \rangle := h_4$ 
      use  $\sigma_1.\text{trans } \sigma_2$ 
      simp [hσ1, hσ2]
      obtain  $\langle \sigma, h\sigma \rangle := h\_perm \ \langle \_ \rangle$ 
      simp [List.ofFn_eq_map] at hσ ⊢
      generalize_proofs at *; (
        exact  $\langle \text{Equiv.ofBijective } (\text{fun } i \Rightarrow \langle \sigma \ i, \text{by simp} \rangle), \text{by solve\_by\_elim} \rangle \langle \text{fun } i \Rightarrow \langle a, ha \rangle := \sigma.\text{surjective } i, \text{by simp} \rangle$ 
        exact  $\langle \langle a, \text{by simp_a using } a.2 \rangle, \text{Fin.ext } <\mid \text{by simp_a [Fin.ext\_iff]} \text{ using cong} \rangle$ 
      )
      obtain  $\langle \sigma, h\sigma \rangle := h\_perm$ 
      use Equiv.ofBijective (fun i => (Fintype.equivFin d).symm (σ i)) (fun i j hij => exact (σ.symm (Fintype.equivFin d i), by simp));)
      intro i
      replace hσ := congr_arg (fun l => l[i!]) hσ
      simp_all
      refine' h_contra  $\langle \langle \text{Finset.image } \sigma \ (\text{Finset.univ.filter fun } i \Rightarrow i.\text{val} < k), \_ \rangle, \_ \rangle$ 
      rw [Finset.card_image_of_injective _ σ.injective, Finset.card_eq_of_bijective]
      use fun i hi =>  $\langle i, \text{by linarith} \rangle$ 
      all_goals norm_num [Fin.ext_iff] at *
      have h_prod_eq :  $\prod x \in \text{Finset.image } \sigma \ (\text{Finset.univ.filter fun } i : \text{Fin } (\text{Fintype.card } d) \Rightarrow \text{Fin } (\text{Fintype.card } d))$ 
        rw [Finset.prod_image] <;> simp [hσ]
        refine' Finset.prod_bij (fun i hi =>  $\langle i, \text{by linarith [Fin.is\_lt } i, \text{Finset.mem\_image } i \rangle$ 
          exact fun b =>  $\langle \langle b, \text{by linarith [Fin.is\_lt } b \rangle \rangle, \text{by simp, rfl} \rangle$ 

```

```

convert h_prod_eq using 1
rw [← Finset.prod_image]
· congr! 1
  refine' Finset.eq_of_subset_of_card_le (Finset.image_subset_iff.mpr fun
  rw [Finset.card_eq_of_bijective]
  use fun i hi => ⟨i, by linarith⟩
  · exact fun a ha => ⟨a, Finset.mem_filter.mp ha |>.2, rfl⟩
  · grind
  · simp +contextual [Fin.ext_iff]
· exact fun x _ y _ hxy => by simpa [Fin.ext_iff] using hxy

lemma prod_singularValuesSorted_eq_compoundSV (M : Matrix d d ℂ) (k : ℕ)
  (hk : k ≤ Fintype.card d) :
  [] i : Fin k, singularValuesSorted M ⟨i.val, by omega⟩ =
  singularValuesSorted (compoundMatrix M k) ⟨0, by
    have : Fintype.card {S : Finset d // S.card = k} = (Fintype.card d).c
    simp [Fintype.card_subtype]
    rw [this]; exact Nat.choose_pos hk) := by
  set hcard : 0 < Fintype.card {S : Finset d // S.card = k} := by
    simp [Fintype.card_subtype]; exact Nat.choose_pos hk
  apply le_antisymm
  · obtain ⟨S₀, hS₀⟩ := exists_subset_prod_eq_sorted_prod M k hk
    obtain ⟨j₀, hj₀⟩ := singularValues_compoundMatrix_eq M k S₀
    obtain ⟨idx, hidx⟩ := singularValues_mem_sorted (compoundMatrix M k) j₀
    rw [← hS₀, ← hj₀, hidx]
    have : NeZero (Fintype.card {S : Finset d // S.card = k}) := ⟨by omega⟩
    exact singularValuesSorted_antitone (compoundMatrix M k) (Fin.zero_le i
  · obtain ⟨j, hj⟩ := singularValuesSorted_mem_values (compoundMatrix M k)
    obtain ⟨S, hS⟩ := singularValues_compoundMatrix_rev M k j
    rw [hj, hS]
    exact prod_singularValues_subset_le_sorted_prod M k hk S

/--
The **Rayleigh quotient bound**:
For a Hermitian matrix H with eigenvalues  $\lambda$ , we have
 $\forall v \ H \ v \leq (\max \lambda) \cdot \forall v \ v$  for all  $v$ . -/
lemma IsHermitian.inner_le_sup_eigenvalue_mul_inner
  {e : Type*} [Fintype e] [DecidableEq e]
  (H : Matrix e e ℂ) (hH : H.IsHermitian)
  (he : 0 < Fintype.card e)
  (v : e → ℂ) :
  Complex.re (star v []v H.mulVec v) ≤
  Finset.sup' Finset.univ
    (Finset.univ_nonempty_iff.mpr (Fintype.card_pos_iff.mp he))
    hH.eigenvalues * Complex.re (star v []v v) := by
  have := hH.spectral_theorem

```

```

-- Let $w = U^* v$, then $v^* H v = w^* D w$.
set w : e → ℂ := star (hH.eigenvectorUnitary : Matrix e e ℂ) |> Matrix.mulVec <| v
have h_eq : (star v []_v H *_v v).re = (star w []_v (Matrix.diagonal (RCLike.ofReal ∘ hH.eigenvalues)) *_v w).re
  replace this := congr_arg (fun m => star v []_v m *_v v) this; simp_all [Matrix.mulVec, Matrix.star_mulVec]
  simp [Matrix.dotProduct_mulVec, Matrix.star_mulVec]
  congr! 3
  ext i j; simp [Matrix.mul_apply, Matrix.diagonal]
-- Since $D$ is diagonal with eigenvalues $\lambda_i$, we have $w^* D w = \sum_i |w_i|^2 \lambda_i$.
have h_diag : (star w []_v (Matrix.diagonal (RCLike.ofReal ∘ hH.eigenvalues)) *_v w).re = \sum_i |w_i|^2 \lambda_i
  simp [dotProduct, Matrix.mulVec, Finset.mul_sum, mul_assoc, mul_comm,]
  simp [Complex.normSq, Complex.sq_norm, diagonal]
  rw [← Finset.sum_sub_distrib]; refine' Finset.sum_congr rfl fun i hi => _; rw [F
    · exact fun j hj => Or.inl (by rw [if_neg (Ne.symm hj)]; norm_num)
    · exact fun j hj => Or.inl (by rw [if_neg (Ne.symm hj)]; norm_num)
-- Since $U$ is unitary, we have $|w|^2 = |v|^2$.
have h_unitary : \sum_i |w_i|^2 = \sum_i |v_i|^2 := by
  have h_unitary : \forall (U : Matrix e e ℂ), U.conjTranspose * U = 1 → \forall (v : e → ℂ),
    intro U hU v
  have h_unitary : (star (U.mulVec v) []_v U.mulVec v) = (star v []_v v) := by
    have h_unitary : (star (U.mulVec v) []_v U.mulVec v) = (star v []_v (U^H * U).mulVec v)
      simp [Matrix.mulVec, dotProduct]
      simp [Matrix.mul_apply, mul_assoc, mul_comm, mul_left_comm, Finset.mul_sum]
      exact Finset.sum_comm.trans (Finset.sum_congr rfl fun _ => Finset.sum_comm)
    rw [h_unitary, hU, Matrix.one_mulVec]
  convert congr_arg Complex.re h_unitary using 1 <=> simp [sq]; ring_nf!
  · simp [dotProduct, sq]; ring_nf!
  · simp [Complex.normSq, Complex.sq_norm]; ring_nf!
  · simp [dotProduct]
    simp [← sq, Complex.normSq_apply, Complex.sq_norm]
  convert h_unitary (star (hH.eigenvectorUnitary : Matrix e e ℂ)) _ v using 1 <=>
  · norm_num [Complex.normSq, Complex.sq_norm]
  · simp [Matrix.star_eq_conjTranspose]
    simp [Matrix.IsHermitian.eigenvectorUnitary]
  rw [h_eq, h_diag, ← h_unitary, Finset.mul_sum]
  exact Finset.sum_le_sum fun i _ => mul_le_mul_of_nonneg_right (Finset.le_sup' (fun
/- All eigenvalues of `A† A` are bounded by (max singular value)². -
/
lemma eigenvalue_le_singularValuesSorted_sq {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (h : 0 < Fintype.card e) (i : e) :
  (isHermitian_mul_conjTranspose_self A.conjTranspose).eigenvalues i ≤
  (singularValuesSorted A {0, h}) ^ 2 := by
  -- By definition of singular values, we know that $\sigma_i(A)^2 = \lambda_i(A^*A)$
  have h_singular_value_squared : \forall i, (singularValues A i) ^ 2 = (isHermitian_mul_c
    intro i

```

```

    simp [singularValues]
    rw [Real.sq_sqrt (_)]
    convert Matrix.eigenvalues_conjTranspose_mul_self_nonneg A i using 1
    rw [← h_singular_value_squared i]
    gcongr
    · exact Real.sq_sqrt_nonneg _
    · convert singularValuesSorted_zero_eq_sup A h |> fun h => h.ge.trans' _
      exact Finset.le_sup' (fun i => singularValues A i) (Finset.mem_univ i)

/-- The quadratic form of `A† A` is bounded by (max singular value)2 * ||v||2
/
lemma quadratic_form_le_singularValuesSorted_sq {e : Type*} [Fintype e] [DecidableEq e]
  (A : Matrix e e ℂ) (h : 0 < Fintype.card e) (v : e → ℂ) :
  Complex.re (star v []_v (A.conjTranspose * A).mulVec v) ≤
  (singularValuesSorted A {0, h}) ^ 2 * Complex.re (star v []_v v) := by
  apply le_trans (IsHermitian.inner_le_sup_eigenvalue_mul_inner (AH * A) (b
    simp [Matrix.IsHermitian]) h v)
  apply_rules [mul_le_mul_of_nonneg_right, Finset.sup'_le]
  · intro i _
    convert eigenvalue_le_singularValuesSorted_sq A h i using 1
    simp [Matrix.conjTranspose_conjTranspose]
  · simp [dotProduct]
    exact Finset.sum_nonneg fun _ => add_nonneg (mul_self_nonneg _) (mul_

/-- The largest singular value of a matrix product is at most the product of
largest singular values: `σ1(M * N) ≤ σ1(M) * σ1(N)`.
This is operator-norm submultiplicativity. -/
lemma singularValuesSorted_mul_le {e : Type*} [Fintype e] [DecidableEq e]
  (M N : Matrix e e ℂ) (h : 0 < Fintype.card e) :
  singularValuesSorted (M * N) {0, h} ≤
  singularValuesSorted M {0, h} * singularValuesSorted N {0, h} := by
  rw [singularValuesSorted_zero_eq_sup, singularValuesSorted_zero_eq_sup, s
  -- Apply the inequality eigenvalue_le_singularValuesSorted_sq to each eig
  have h_eigenvalue_le : ∀ i, (isHermitian_mul_conjTranspose_self (M * N).c
  intro i
  have h_eigenvalue_le : (isHermitian_mul_conjTranspose_self (M * N).conj
  -- By the properties of singular values and eigenvalues, we know that
  have h_eigenvalue_bound : ∀ (v : e → ℂ), Complex.re (star v []_v ((M *
  intro v
  have h_quadratic_form : Complex.re (star v []_v ((M * N).conjTranspos
  have := quadratic_form_le_singularValuesSorted_sq M h (N.mulVec v)
  convert this using 1 <=> simp [Matrix.mul_assoc, Matrix.dotProduct]
  have h_quadratic_form_N : Complex.re (star v []_v (N.conjTranspose *
  convert quadratic_form_le_singularValuesSorted_sq N h v using 1
  simp only [mul_assoc] using h_quadratic_form.trans (mul_le_mul_of_
  convert h_eigenvalue_bound ((isHermitian_mul_conjTranspose_self (M *

```

```

    · have := (isHermitian_mul_conjTranspose_self (M * N).conjTranspose).eigenvalue
    · simp [dotProduct]
      have := (isHermitian_mul_conjTranspose_self (M * N).conjTranspose).eigenvalue
      rw [orthonormal_iff_ite] at this
      simp_all [Complex.ext_iff, inner]
  linarith
-- Apply the inequality eigenvalue_le_singularValuesSorted_sq to each eigenvalue
have h_sqrt_eigenvalue_le : ∀ i, singularValues (M * N) i ≤ singularValuesSorted M i
intro i
have h_sqrt_eigenvalue_le_i : (isHermitian_mul_conjTranspose_self (M * N).conjTranspose).eigenvalue i ≤
have h_sqrt_eigenvalue_le_i' : Real.sqrt ((isHermitian_mul_conjTranspose_self (M * N).conjTranspose).eigenvalue i) ≤
  exact Real.sqrt_le_iff.mpr ⟨mul_nonneg (singularValuesSorted_nonneg M {0, h})⟩
exact h_sqrt_eigenvalue_le_i' |> le_trans (by
  exact le_rfl)
simp_all [Finset.sup'_le_iff]
convert h_sqrt_eigenvalue_le using 1
rw [singularValuesSorted_zero_eq_sup, singularValuesSorted_zero_eq_sup]

/-- Horn's inequality (weak log-majorization of singular values):
For all `k`,  $\prod_{i \leq k} \sigma_i(AB) \leq \prod_{i \leq k} \sigma_i(A) \cdot \prod_{i \leq k} \sigma_i(B)$ .
This follows from submultiplicativity of the operator norm applied to
exterior powers of the matrices. -/
lemma horn_weak_log_majorization (A B : Matrix d d ℂ) (k : ℕ)
  (hk : k ≤ Fintype.card d) :
  ∏ i : Fin k, singularValuesSorted (A * B) (i.val, by omega) ≤
  ∏ i : Fin k, (singularValuesSorted A (i.val, by omega) *
    singularValuesSorted B (i.val, by omega)) := by
-- Rewrite the RHS as (prod of  $\sigma_i(A)$ ) * (prod of  $\sigma_i(B)$ )
rw [Finset.prod_mul_distrib]
-- Use the compound matrix characterisation and submultiplicativity
have hcard : 0 < Fintype.card {S : Finset d // S.card = k} := by
  have : Fintype.card {S : Finset d // S.card = k} = (Fintype.card d).choose k :=
    simp [Fintype.card_subtype]
  rw [this]; exact Nat.choose_pos hk
calc ∏ i : Fin k, singularValuesSorted (A * B) (i.val, by omega)
  = singularValuesSorted (compoundMatrix (A * B) k) (0, hcard) :=
    prod_singularValuesSorted_eq_compoundSV (A * B) k hk
  = singularValuesSorted (compoundMatrix A k * compoundMatrix B k) (0, hcard) :=
    rw [compoundMatrix_mul]
  ≤ singularValuesSorted (compoundMatrix A k) (0, hcard) *
    singularValuesSorted (compoundMatrix B k) (0, hcard) :=
    singularValuesSorted_mul_le _ _ hcard
  = (∏ i : Fin k, singularValuesSorted A (i.val, by omega)) *
    (∏ i : Fin k, singularValuesSorted B (i.val, by omega)) := by
    rw [← prod_singularValuesSorted_eq_compoundSV A k hk,
      ← prod_singularValuesSorted_eq_compoundSV B k hk]

```

```

/-! ### Weak log-majorization implies sum of powers inequality -
/

/-- Raising nonneg antitone sequences to a positive power preserves antitone
/
lemma rpow_antitone_of_nonneg_antitone {n : ℕ}
  {f : Fin n → ℝ} (hf : Antitone f) (hf_nn : ∀ i, 0 ≤ f i)
  {r : ℝ} (hr : 0 < r) :
  Antitone (fun i => f i ^ r) := by
  exact fun i j hij => Real.rpow_le_rpow (hf_nn _) (hf hij) hr.le

/-- Weak log-majorization is preserved under positive powers. -
/
lemma rpow_preserves_weak_log_maj {n : ℕ}
  {x y : Fin n → ℝ}
  (hx_nn : ∀ i, 0 ≤ x i) (hy_nn : ∀ i, 0 ≤ y i)
  (h_log_maj : ∀ (k : ℕ) (_ : k ≤ n),
    ∏ i : Fin k, x ⟨i.val, by omega⟩ ≤
    ∏ i : Fin k, y ⟨i.val, by omega⟩)
  {r : ℝ} (hr : 0 < r) :
  ∀ (k : ℕ) (_ : k ≤ n),
    ∏ i : Fin k, (fun j => x j ^ r) ⟨i.val, by omega⟩ ≤
    ∏ i : Fin k, (fun j => y j ^ r) ⟨i.val, by omega⟩ := by
  intro k hk
  convert Real.rpow_le_rpow _ (h_log_maj k hk) hr.le using 1 <=>
    norm_num [Real.finset_prod_rpow _ _ fun i _ => hx_nn _,
      Real.finset_prod_rpow _ _ fun i _ => hy_nn _]
  exact Finset.prod_nonneg fun _ _ => hx_nn _

/-
The Abel summation identity (a rewriting of the sum) gives:

$$\sum_{i=0}^{n-1} x_i * d_i = x_{n-1} * D_{n-1} + \sum_{i=0}^{n-2} (x_i - x_{i+1}) * D_i$$

(This is essentially Finset.sum_range_by_parts with f = x and g = d.)
Each term is nonneg because:
-  $x_{n-1} \geq 0$  (positive) and  $D_{n-1} \geq 0$  (see below)
-  $x_i - x_{i+1} \geq 0$  (x is antitone) and  $D_i \geq 0$  (see below)
 $D_k = \sum_{j=0}^k \log(y_j/x_j) = \log(\prod_{j=0}^k y_j/x_j) = \log(\prod y_j / \prod x_j)$ 
because  $\prod y_j \geq \prod x_j$  (weak log-majorization) and both products are positive.
So  $\sum x_i * d_i$  is a sum of nonneg terms, hence  $\geq 0$ .
Use Finset.sum_range_by_parts from Mathlib. The key Mathlib lemma is:

$$\text{Finset.sum\_range\_by\_parts } f \ g \ n = f \ (n-1) * \sum_{i < n} g \ i - \sum_{i < n-1} (f(i+1) - f(i)) * \sum_{j < i+1} g \ j$$

Here  $f \ i = x \ \langle i, \dots \rangle$  (antitone) and  $g \ i = \log(y \ \langle i, \dots \rangle / x \ \langle i, \dots \rangle)$ .
Actually, it may be easier to prove this directly by induction on n, without

```


For the direct induction approach on n :

- $n = 0$: sum is empty, $0 \geq 0$.
 - $n = 1$: $x_0 * \log(y_0/x_0) \geq 0$ since $x_0 > 0$ and $\log(y_0/x_0) \geq 0$ (from $\prod_{i < 1} x_i$).
 - $n+1 \rightarrow n+2$: Split $\sum_{i=0}^{n+1} x_i * \log(y_i/x_i) = \sum_{i=0}^n x_i * \log(y_i/x_i) + x_{n+1} * \log(y_{n+1}/x_{n+1})$.
Now $\sum_{i=0}^n x_i * \log(y_i/x_i) \geq \sum_{i=0}^n x_{n+1} * \log(y_i/x_i)$ (since $x_i \leq x_{n+1}$).
- Hmm, this doesn't work cleanly because $\log(y_i/x_i)$ can be negative for some i .
Better approach: prove it directly using the Abel summation identity and nonnegativity.

```

lemma sum_mul_log_nonneg_of_weak_log_maj {n : ℕ}
  {x y : Fin n → ℝ}
  (hx_pos : ∀ i, 0 < x i) (hy_pos : ∀ i, 0 < y i)
  (hx_anti : Antitone x)
  (h_log_maj : ∀ (k : ℕ) (l : k ≤ n),
    ∏ i : Fin k, x (i.val, by omega) ≤
    ∏ i : Fin k, y (i.val, by omega)) :
  0 ≤ ∑ i, x i * Real.log (y i / x i) := by
  by_contra h_neg
  -- Let $d_i = \log(y_i / x_i)$ and $D_k = \sum_{j=0}^{k-1} d_j$.
  set d : Fin n → ℝ := fun i => Real.log (y i / x i)
  set D : Fin n → ℝ := fun k => ∑ i ∈ Finset.Iic k, d i
  -- By Abel's summation formula, we have $\sum_{i=0}^{n-1} x_i d_i = x_{n-1} D_{n-1} + \sum_{i=0}^{n-2} (x_i - x_{i+1}) D_i$.
  1) x_i d_i = x_{n-1} D_{n-1} + \sum_{i=0}^{n-2} (x_i - x_{i+1}) D_i$.
  have hn : n ≠ 0 := by rintro rfl; simp at h_neg
  have h_abel : ∑ i, x i * d i = x (n - 1, by omega) * D (n - 1, by omega) + ∑ i : Fin (n - 1),
    (x (i.val, by omega) - x (i.val + 1, by omega)) * D (i.val, by omega) := by
    rcases n with {} <| > norm_num at *
    rename_i n
    have h_abel : ∀ m : Fin (n + 1), ∑ i ∈ Finset.Iic m, x i * d i = x m * D m + ∑ i : Fin m,
      (x (i.val, by omega) - x (i.val + 1, by omega)) * D (i.val, by omega) := by
      intro m
      induction' m using Fin.inductionOn with m ih
      · simp +zetaDelta at *
      rw [Finset.sum_eq_single 0, Finset.sum_eq_single 0] <| aesop
      · rw [show (Finset.Iic (Fin.succ m) : Finset (Fin (n + 1))) = Finset.Iic (Fin.succ m),
        show (Finset.Iio (Fin.succ m) : Finset (Fin (n + 1))) = Finset.Iio (Fin.succ m)]
        · rw [ih]
        ring_nf!
        rw [show (Finset.Iic (Fin.succ m) : Finset (Fin (n + 1))) = Finset.Iic (Fin.succ m),
          ext i; simp [Finset.mem_Iic, Finset.mem_insert]]
        exact (fun hi => or_iff_not_imp_left.mpr fun hi' => Nat.le_of_lt_succ <| h)
        · ext i; simp [Fin.lt_def, Fin.le_iff_val_le_val]
        · ext i; simp [Finset.mem_Iic, Finset.mem_insert]
        exact (fun hi => or_iff_not_imp_left.mpr fun hi' => Nat.le_of_lt_succ <| h)
    convert h_abel (n, Nat.lt_succ_self _) using 1
    · rw [show (Iic (n, Nat.lt_succ_self _) : Finset (Fin (n + 1))) = Finset.univ fr
    · refine' congr rfl (Finset.sum_bij (fun i hi => (i, by omega)) _ _ _ _) <| > simp
    · exact fun i j h => Fin.ext h

```

```

      · exact fun i hi => ⟨i, by linarith [Fin.is_lt i, show (i : ℕ) < n]⟩
-- Since  $D_k \geq 0$  for all  $k$ , we have  $x_{n-1} D_{n-1} \geq 0$  and  $(x_i - x_{i+1}) D_i \geq 0$  for all  $i$ .
have h_nonneg : ∀ k : Fin n, 0 ≤ D k := by
  intro k
  have h_prod : ∏ i ∈ Finset.Iic k, y i ≥ ∏ i ∈ Finset.Iic k, x i := by
    specialize h_log_maj (k + 1) (by linarith [Fin.is_lt k])
    rw [show (Finset.Iic k : Finset (Fin n)) = Finset.image (fun i : Fin n => i) (Finset.Iic k)]
    · rwa [Finset.prod_image <| by intros i hi j hj hij; simp [Fin.ext_iff]]
    · intro i j hij
      exact Fin.ext <| by simp using congr_arg Fin.val hij
    · ext ⟨i, hi⟩; simp [Fin.ext_iff, Fin.le_iff_val_le_val]
    exact ⟨fun hi' => ⟨i, by linarith [Fin.is_lt k]⟩, rfl⟩, fun ⟨a, ha⟩ =>
      simp +zetaDelta at *
    rw [← Real.log_prod fun i hi => ne_of_gt (div_pos (hy_pos i) (hx_pos i)) (hx_pos i)]
    exact h_neg <| h_abel.symm ▸ add_nonneg (mul_nonneg (le_of_lt (hx_pos _)) (hy_pos _))

```

/-
 PROBLEM
 For positive reals a, b : $b - a \geq a \cdot \log(b/a)$.
 Equivalently: $t - 1 \geq \log(t)$ for $t = b/a$.
 PROVIDED SOLUTION
 We need $b - a \geq a \cdot \log(b/a)$ for $a, b > 0$. Equivalently, dividing by $a > 0$:
 -/
 lemma sub_ge_mul_log_div {a b : ℝ} (ha : 0 < a) (hb : 0 < b) :
 $b - a \geq a \cdot \text{Real.log } (b / a)$:= by
 nlinarith [Real.log_le_sub_one_of_pos (div_pos hb ha), mul_div_cancel₀ b ha]

/- Weak log-majorization of nonneg antitone sequences implies the sum inequality
 /
 lemma weak_log_maj_sum_le {n : ℕ}
 {x y : Fin n → ℝ}
 (hx_nn : ∀ i, 0 ≤ x i) (hy_nn : ∀ i, 0 ≤ y i)
 (hx_anti : Antitone x) (hy_anti : Antitone y)
 (h_log_maj : ∀ (k : ℕ) (n : ℕ), k ≤ n →
 ∏ i : Fin k, x i.val ≤ ∏ i : Fin k, y i.val),
 (∑ i, x i ≤ ∑ i, y i) := by
 induction' n with n ih
 · norm_num +zetaDelta at *
 · by_cases h_last : x (Fin.last n) = 0
 · simp [Fin.sum_univ_castSucc, h_last]
 refine le_add_of_le_of_nonneg (ih (fun i => hx_nn _)) (fun i => hy_nn _)
 intro k hk
 simp
 exact h_log_maj k (by linarith)

```

· -- Since  $x_{\text{last}} > 0$ , we have  $x_i > 0$  for all  $i$ .
  have hx_pos :  $\forall i, 0 < x\ i :=$  by
    exact fun i => lt_of_lt_of_le (lt_of_le_of_ne (hx_nn _) (Ne.symm h_last)) (h
  have hy_pos :  $\forall i, 0 < y\ i :=$  by
    intro i; specialize h_log_maj (n + 1) le_rfl; contrapose! h_log_maj; simp_al
    exact lt_of_le_of_lt (mul_nonpos_of_nonneg_of_nonpos (Finset.prod_nonneg fun
  have h_sum_mul_log_nonneg :  $0 \leq \sum i, x\ i * \text{Real.log } (y\ i / x\ i) :=$  by
    apply sum_mul_log_nonneg_of_weak_log_maj (fun i => hx_pos i) (fun i => hy_po
  have h_sum_mul_log_nonneg :  $\sum i, (y\ i - x\ i) \geq \sum i, x\ i * \text{Real.log } (y\ i / x\ i)$ 
    exact Finset.sum_le_sum fun i _ => by have := sub_ge_mul_log_div (hx_pos i)
    norm_num at *; linarith

/-- Weak log-majorization of nonneg antitone sequences implies the sum of
powers inequality. -/
lemma weak_log_maj_sum_rpow_le {n :  $\mathbb{N}$ }
  {x y : Fin n  $\rightarrow \mathbb{R}$ }
  (hx_nn :  $\forall i, 0 \leq x\ i$ ) (hy_nn :  $\forall i, 0 \leq y\ i$ )
  (hx_anti : Antitone x) (hy_anti : Antitone y)
  (h_log_maj :  $\forall (k : \mathbb{N}) (\_ : k \leq n),$ 
     $\prod i : \text{Fin } k, x\ \langle i.\text{val}, \text{by omega} \rangle \leq$ 
     $\prod i : \text{Fin } k, y\ \langle i.\text{val}, \text{by omega} \rangle$ )
  {r :  $\mathbb{R}$ } (hr :  $0 < r$ ) :
   $\sum i, x\ i ^ r \leq \sum i, y\ i ^ r :=$  by
  apply weak_log_maj_sum_le
  · exact fun i => Real.rpow_nonneg (hx_nn i) r
  · exact fun i => Real.rpow_nonneg (hy_nn i) r
  · exact rpow_antitone_of_nonneg_antitone hx_anti hx_nn hr
  · exact rpow_antitone_of_nonneg_antitone hy_anti hy_nn hr
  · exact rpow_preserves_weak_log_maj hx_nn hy_nn h_log_maj hr

/-! ## Key singular value inequality for products -/
lemma sum_rpow_singularValues_mul_le (A B : Matrix d d  $\mathbb{C}$ ) {r :  $\mathbb{R}$ } (hr :  $0 < r$ ) :
   $\sum i : \text{Fin } (\text{Fintype.card } d), \text{singularValuesSorted } (A * B)\ i ^ r \leq$ 
   $\sum i : \text{Fin } (\text{Fintype.card } d),$ 
   $(\text{singularValuesSorted } A\ i ^ r * \text{singularValuesSorted } B\ i ^ r) :=$  by
  have h_rw :  $\forall i : \text{Fin } (\text{Fintype.card } d),$ 
     $\text{singularValuesSorted } A\ i ^ r * \text{singularValuesSorted } B\ i ^ r =$ 
     $(\text{singularValuesSorted } A\ i * \text{singularValuesSorted } B\ i) ^ r :=$  by
    intro i
    rw [Real.mul_rpow (singularValuesSorted_nonneg A i) (singularValuesSorted_nonneg
  simp_rw [h_rw]
  apply weak_log_maj_sum_rpow_le
  · exact fun i => singularValuesSorted_nonneg (A * B) i
  · exact fun i => mul_nonneg (singularValuesSorted_nonneg A i) (singularValuesSorte
  · exact singularValuesSorted_antitone (A * B)

```

```

    · exact antitone_mul_of_antitone_nonneg
      (singularValuesSorted_antitone A) (singularValuesSorted_antitone B)
      (singularValuesSorted_nonneg A) (singularValuesSorted_nonneg B)
    · exact horn_weak_log_majorization A B
    · exact hr

/-! ## Hölder inequality for singular values -/

/--
The finite-sum Hölder inequality applied to sequences of r-
th powers of
sorted singular values.

With conjugate exponents `p' = p/r > 1` and `q' = q/r > 1` (which satisfy
`1/p' + 1/q' = 1` when `1/r = 1/p + 1/q`), this gives:
`∑ σi(A)r · σi(B)r ≤ (∑ σi(A)p){r/p} · (∑ σi(B)q){r/q}`

Note: The sums on the RHS don't depend on the ordering, so we can replace
sorted singular values with unsorted ones.
-/
lemma holder_step_for_singularValues (A B : Matrix d d ℂ)
  {r p q : ℝ} (hr : 0 < r) (hp : 0 < p) (hq : 0 < q)
  (hpqr : 1 / r = 1 / p + 1 / q) :
  (∑ i : Fin (Fintype.card d),
    (singularValuesSorted A i ^ r * singularValuesSorted B i ^ r)) ≤
    (∑ i : Fin (Fintype.card d), singularValuesSorted A i ^ p) ^ (r / p) *
    (∑ i : Fin (Fintype.card d), singularValuesSorted B i ^ q) ^ (r / q) :=
  have h_holder : (∑ i : Fin (Fintype.card d), (singularValuesSorted A i ^
    have := @Real.inner_le_Lp_mul_Lq
    convert @this (Fin (Fintype.card d)) Finset.univ (fun i => singularValu
    · simp only [abs_of_nonneg (Real.rpow_nonneg (singularValuesSorted_nonneg
      abs_of_nonneg (Real.rpow_nonneg (singularValuesSorted_nonneg
    · constructor <=> norm_num [hr.ne', hp.ne', hq.ne'] at hpqr ⊢ <=> ring_
    convert h_holder using 3 <=> push_cast [← Real.rpow_mul (singularValuesSo

end

```

1.476 QuantumInfo/ForMathlib/Matrix.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/

```

```

import Mathlib.Algebra.Algebra.Spectrum.Quasispectrum
import Mathlib.Algebra.Order.Group.Pointwise.CompleteLattice
import Mathlib.Analysis.CStarAlgebra.Matrix
import Mathlib.Analysis.Matrix.Order
import Mathlib.Analysis.SpecialFunctions.Bernstein
import Mathlib.Analysis.SpecialFunctions.Pow.NNReal
import Mathlib.Data.Multiset.Functor --Can't believe I'm having to import this
import Mathlib.LinearAlgebra.Matrix.Kronecker
import Mathlib.LinearAlgebra.Matrix.PosDef
import Mathlib.LinearAlgebra.Matrix.IsDiag
import Mathlib.Tactic.Bound
import Mathlib.Tactic.NormNum.GCD

import QuantumInfo.ForMathlib.Misc

noncomputable section

open BigOperators

variable {n : Type*}
variable [RCLike n] [DecidableEq n]

namespace Matrix

theorem zero_rank_eq_zero {A : Matrix n n } [Fintype n] (hA : A.rank = 0) : A = 0 :
  have h :  $\forall v, A.mulVecLin v = 0$  := by
    intro v
    rw [rank, Module.finrank_zero_iff] at hA
    have := hA.elim (A.mulVecLin v, (v, rfl)) (0, (0, by rw [mulVecLin_apply, mulVec
    simp only [Subtype.mk.injEq] using this
    rw [LinearEquiv.map_eq_zero_iff toLin'])
    exact LinearMap.ext h

namespace IsHermitian

variable {A : Matrix n n } {B : Matrix n n }
variable (hA : A.IsHermitian) (hB : B.IsHermitian)

include hA in
omit [DecidableEq n] in
theorem smul_selfAdjoint {c : } (hc : _root_.IsSelfAdjoint c) : (c • A).IsHermitian
  exact IsSelfAdjoint.smul hc hA

include hA in
omit [DecidableEq n] in
theorem smul_im_zero {c : } (h : RCLike.im c = 0) : (c • A).IsHermitian :=

```

```

    hA.smul_selfAdjoint (RCLike.conj_eq_iff_im.mpr h)

include hA in
omit [DecidableEq n] in
theorem smul_real (c : ℝ) : (c • A).IsHermitian := by
  convert hA.smul_im_zero (RCLike.ofReal_im c) using 1
  ext
  simp only [smul_apply, smul_eq_mul, RCLike.real_smul_eq_coe_mul]

def HermitianSubspace (n : Type*) [Fintype n] [RCLike ℝ] : Subspace ℝ (Matrix n n ℝ) :=
  { A : Matrix n n ℝ | A.IsHermitian }
add_mem' _ _ := by simp_all only [Set.mem_setOf_eq, IsHermitian.add]
zero_mem' := by simp only [Set.mem_setOf_eq, isHermitian_zero]
smul_mem' c A := by
  simp only [Set.mem_setOf_eq]
  intro hA
  exact IsHermitian.smul_real hA c

variable [Fintype n]

include hA in
omit [DecidableEq n] in
@[simp]
theorem re_trace_eq_trace : RCLike.re (A.trace) = A.trace := by
  rw [trace, map_sum, RCLike.ofReal_sum, IsHermitian.coe_re_diag hA]

section eigenvalues

/-- The sum of the eigenvalues of a Hermitian matrix is equal to its trace. -/
theorem sum_eigenvalues_eq_trace : ∑ i, hA.eigenvalues i = A.trace := by
  nth_rewrite 2 [hA.spectral_theorem]
  simp only [Unitary.conjStarAlgAut_apply]
  rw [trace_mul_comm, ← mul_assoc]
  simp [trace_diagonal]

/-- If all eigenvalues are equal to zero, then the matrix is zero. -/
theorem eigenvalues_zero_eq_zero (h : ∀ i, hA.eigenvalues i = 0) : A = 0 :=
  suffices A.rank = 0 from zero_rank_eq_zero this
  simp only [hA.rank_eq_card_non_zero_eigs, h, ne_eq, not_true_eq_false, Finset.card_eq_zero]

end eigenvalues

end IsHermitian

```

```
section Kronecker
```

```
open Kronecker
```

```
variable [CommRing R] [StarRing R]
variable (A : Matrix m m R) (B : Matrix n n R)
```

```
omit [DecidableEq n] in
theorem kroneckerMap_conjTranspose :  $(A \otimes_k B)^H = (A^H \otimes_k B^H)$  := by
  ext; simp
```

```
variable {A : Matrix m m R} {B : Matrix n n R}
variable (hA : A.IsHermitian) (hB : B.IsHermitian)
```

```
include hA hB in
omit [DecidableEq n] in
theorem kroneckerMap_IsHermitian :  $(A \otimes_k B).IsHermitian$  := by
  exact (hA ▸ hB ▸ kroneckerMap_conjTranspose A B : _ = _)
```

```
end Kronecker
```

```
namespace PosSemidef
```

```
open Kronecker
open scoped ComplexOrder
```

```
variable {m n  $\alpha$  : Type*}
variable [Fintype m] [Fintype n]
variable [RCLike  $\alpha$ ] [dn : DecidableEq n]
```

```
section
```

```
variable {A : Matrix m m  $\alpha$ } {B : Matrix m m  $\alpha$ }
variable (hA : A.PosSemidef) (hB : B.PosSemidef)
```

```
include hA in
theorem trace_zero :  $A.trace = 0 \rightarrow A = 0$  := by
  open Classical in
  intro h
  rw [← hA.isHermitian.sum_eigenvalues_eq_trace, RCLike.ofReal_eq_zero] at h
  rw [Finset.sum_eq_zero_iff_of_nonneg (fun i _ ↦ hA.eigenvalues_nonneg i)] at h
  simp only [Finset.mem_univ, forall_const] at h
  exact hA.isHermitian.eigenvalues_zero_eq_zero h
```

```
include hA in
@[simp]
theorem trace_zero_iff :  $A.trace = 0 \leftrightarrow A = 0$  :=
```

```

    (trace_zero hA, (by simp [·]))

--belongs somewhere else. compare with `Complex.normSq_eq_conj_mul_self`.
open ComplexConjugate in
theorem _root_.RCLike.normSq_eq_conj_mul_self {z : []} : RCLike.normSq z = c
  rw [RCLike.ext_iff]
  simp [RCLike.normSq]
  ring_nf

--PR
theorem Finsupp.sum_eq_ite
  {α : Type u_1} {M : Type u_8} {N : Type u_10} [Zero M] [AddCommMonoid
    [DecidableEq M] (f : α →0 M) (g : α → M → N) :
  f.sum g = ∑ i, if f i ≠ 0 then g i (f i) else 0 := by
  rw [Finsupp.sum, eq_comm]
  classical convert Finset.sum_ite_mem Finset.univ f.support (fun i ↦ g i (
    · simp
    · simp

omit dn in
open ComplexConjugate in
theorem outer_self_conj (v : n → []) : PosSemidef (vecMulVec v (conj v)) :=
  constructor
  · ext
    simp [vecMulVec_apply, mul_comm]
  · intro x
    rw [Finsupp.sum_fintype _ _ (by simp)]
    conv =>
      enter [2, 2, i]
      rw [Finsupp.sum_fintype _ _ (by simp)]
    simp_rw [RCLike.star_def,
      vecMulVec_apply, mul_assoc, ← Finset.mul_sum, ← mul_assoc, ← Finset.s
    change
      0 ≤ (∑ i : n, conj (x i) * v i) * ∑ i : n, conj (v i) * x i
    have : (∑ i : n, conj (x i) * v i) =
      (∑ i : n, conj (conj (v i) * x i)) := by
        simp only [mul_comm (conj (x _)) (v _), map_mul,
          RingHomCompTriple.comp_apply, RingHom.id_apply]
    rw [this, ← map_sum, ← RCLike.normSq_eq_conj_mul_self, RCLike.ofReal_no
    exact RCLike.normSq_nonneg _

omit [Fintype m] in
include hA hB in
theorem convex_cone {c₁ c₂ : []} (hc₁ : 0 ≤ c₁) (hc₂ : 0 ≤ c₂) : (c₁ • A + c
  (hA.smul hc₁).add (hB.smul hc₂)

```



```

variable [dm : DecidableEq m]

/-- A standard basis matrix (with a positive entry) is positive semidefinite iff the
/
theorem stdBasisMatrix_iff_eq (i j : m) {c : ℝ} (hc : 0 < c) : (single i j c).PosSemDef
  constructor
  · intro {hherm, _}
    rw [IsHermitian, ← ext_iff] at hherm
    replace hherm := hherm i j
    simp only [single, conjTranspose_apply, of_apply, true_and, RCLike.star_def, if_
    apply_fun (starRingEnd ℝ) at hherm
    have hcstar := RCLike.conj_eq_iff_im.mpr (RCLike.pos_iff.mp hc).right
    rw [starRingEnd_self_apply, hcstar, ite_eq_left_iff] at hherm
    contrapose! hherm
    have hcnezero : 0 ≠ c := by
      by_contra hczero
      subst hczero
      exact (lt_self_iff_false 0).mp hc
    exact {fun _ => hherm.symm, hcnezero}
  · intro hij
    subst hij
    constructor
    · ext x y
      simp only [conjTranspose_apply, RCLike.star_def, single, of_apply]
      split_ifs <=> try tauto
      · exact RCLike.conj_eq_iff_im.mpr (RCLike.pos_iff.1 hc).2
      · exact RingHom.map_zero (starRingEnd ℝ)
    · intro x
      rw [Finsupp.sum_fintype _ _ (by simp)]
      conv =>
        enter [2, 2, i]
        rw [Finsupp.sum_fintype _ _ (by simp)]
        simp only [single, of_apply]
        convert_to 0 ≤ (star (x i)) * c * (x i)
      · rw [←Fintype.sum_prod_type']
        have h₀ : ∀ x₁ : m × m, x₁ ≠ ⟨i, i⟩ → star (x x₁.1) * ((if i = x₁.1 ∧ i
          have h₁ : ¬(i = z.1 ∧ i = z.2) := by
            rw [ne_eq, Prod.mk_inj] at hz
            by_contra hz'
            apply hz
            exact {hz'.left.symm, hz'.right.symm}
            rw [ite_cond_eq_false _ _ (eq_false h₁)]
            ring
            rw [Fintype.sum_eq_single ⟨i, i⟩]
            · simp [mul_assoc]

```

```

      · simp [mul_assoc] using h₀
    · rw [mul_comm, ←mul_assoc]
      have hpos : 0 ≤ (x i) * star (x i) := by simp only [RCLike.star_def,
        RCLike.mul_conj, RCLike.ofReal_nonneg, norm_nonneg, pow_nonneg]
      exact (mul_nonneg hpos (le_of_lt hc))

end

variable {A : Matrix m m []} {B : Matrix n n []}
variable (hA : A.PosSemidef) (hB : B.PosSemidef)

include hA hB in
theorem PosSemidef_kronecker : (A ⊗ₖ B).PosSemidef := by
  open Classical in
  rw [hA.left.spectral_theorem, hB.left.spectral_theorem]
  simp only [Unitary.conjStarAlgAut_apply]
  rw [mul_kronecker_mul, mul_kronecker_mul]
  rw [star_eq_conjTranspose, star_eq_conjTranspose]
  rw [←kroneckerMap_conjTranspose]
  rw [diagonal_kronecker_diagonal]
  apply mul_mul_conjTranspose_same
  rw [posSemidef_diagonal_iff]
  rintro ⟨i₁, i₂⟩
  convert mul_nonneg (hA.eigenvalues_nonneg i₁) (hB.eigenvalues_nonneg i₂)
  rw [RCLike.nonneg_iff]
  simp

variable [dm : DecidableEq m]

open MatrixOrder
open ComplexOrder

omit [DecidableEq m]

include hA in
theorem zero_dotProduct_zero_iff : (∀ x : m → [], 0 = star x []ₐ A.mulVec x)
  constructor
  · intro h
    ext i j
    have h₂ := fun x ↦ (PosSemidef.dotProduct_mulVec_zero_iff hA x).mp (h x)
    classical have : DecidableEq m := inferInstance
    convert congrFun (h₂ (Pi.single j 1)) i using 1
    simp
  · rintro rfl
    simp

```

```

omit [Fintype m] in
theorem pos_smul {c : ℝ} (hA : (c • A).PosSemidef) (hc : 0 < c) : A.PosSemidef := by
  have : 0 < 1/c := by
    rw [RCLike.pos_iff] at hc ⊢
    aesop
  convert hA.smul (a := 1/c) this.le
  rw [smul_smul, one_div, inv_mul_cancel, hc.ne', one_smul]

theorem zero_posSemidef_neg_posSemidef_iff : A.PosSemidef ∧ (-
A).PosSemidef ↔ A = 0 := by
  constructor
  · intro {hA, hNegA}
    have h0 : ∀ x : m → ℝ, 0 = star x ⌊ A.mulVec x := fun x ↦ by
      simp only [Matrix.posSemidef_iff_dotProduct_mulVec] at hA hNegA
      have hNegA' := hNegA.right (Finsupp.ofSupportFinite x (Function.support x)).toF
      rw [neg_mulVec, dotProduct_neg, le_neg, neg_zero] at hNegA'
      exact le_antisymm (hA.right x) hNegA'
      exact (zero_dotProduct_zero_iff hA).mp h0
  · rintro rfl
    simp [PosSemidef.zero]

end PosSemidef

namespace PosDef
open scoped ComplexOrder

variable {n m : Type*}
variable [Fintype n] [RCLike ℝ] [DecidableEq n]
variable {A : Matrix n n ℝ}

theorem toLin_ker_eq_bot (hA : A.PosDef) : LinearMap.ker A.toLin' = ⊥ := by
  ext v
  rw [Matrix.posDef_iff_dotProduct_mulVec] at hA
  have := @hA.right v
  grind [mulVec_zero, dotProduct_zero, LinearMap.mem_ker, toLin'_apply, Submodule.me

theorem of_toLin_ker_eq_bot (hA : LinearMap.ker A.toLin' = ⊥) (hA₂ : A.PosSemidef) :
  rwa [hA₂.posDef_iff_isUnit, ← Matrix.isUnit_toLin'_iff, LinearMap.isUnit_iff_ker_e

theorem ker_range_antitone {d : Type*} [Fintype d] [DecidableEq d] {A B : Matrix d d
(hA : A.IsHermitian) (hB : B.IsHermitian) :
  LinearMap.ker A.toEuclideanLin ≤ LinearMap.ker B.toEuclideanLin ↔
  LinearMap.range B.toEuclideanLin ≤ LinearMap.range A.toEuclideanLin
:= by
  rw [Matrix.isHermitian_iff_isSymmetric] at hA hB

```

```

    exact ContinuousLinearMap.ker_le_ker_iff_range_le_range
      (T := Matrix.toEuclideanCLM.toFun B) (U := Matrix.toEuclideanCLM.toFun
end PosDef

namespace PosSemidef
section partialOrder
open scoped ComplexOrder
open scoped MatrixOrder

variable {n m : Type*}
variable [Fintype n] [Fintype m] [RCLike  $\alpha$ ] [DecidableEq m]
variable {A : Matrix n n  $\alpha$ } {B : Matrix n n  $\alpha$ }
variable (hA : A.IsHermitian) (hB : B.IsHermitian)

omit [Fintype n] in
theorem le_of_nonneg_imp {R : Type*} [AddCommGroup R] [PartialOrder R] [IsOrderedSemiring R]
  (f : Matrix n n  $\alpha$   $\rightarrow$  R) (h :  $\forall A, A.PosSemidef \rightarrow 0 \leq f A$ ) :
  (A  $\leq$  B  $\rightarrow$  f A  $\leq$  f B) := by
  intro hAB
  rw [← sub_nonneg, ← map_sub]
  exact h (B - A) <| by rwa [← Matrix.le_iff]

omit [Fintype n] in
theorem le_of_nonneg_imp' {R : Type*} [AddCommGroup R] [PartialOrder R] [IsOrderedSemiring R]
  {x y : R} (f : R  $\rightarrow$  Matrix n n  $\alpha$ ) (h :  $\forall x, 0 \leq x \rightarrow (f x).PosSemidef$ ) :
  (x  $\leq$  y  $\rightarrow$  f x  $\leq$  f y) := by
  intro hxy
  rw [← sub_nonneg, ← map_sub, Matrix.nonneg_iff_posSemidef]
  rw [← sub_nonneg] at hxy
  exact h (y - x) hxy

omit [DecidableEq m] in
theorem mul_mul_conjTranspose_mono (C : Matrix m n  $\alpha$ ) :
  A  $\leq$  B  $\rightarrow$  C * A * C.conjTranspose  $\leq$  C * B * C.conjTranspose := fun hAB  $\mapsto$  by
  rw [Matrix.le_iff] at hAB  $\vdash$ 
  have hDistrib : C * B * CH - C * A * CH = C * (B - A) * CH := by
    ext i j
    simp only [sub_apply, mul_apply, conjTranspose_apply, RCLike.star_def,
      ← Finset.sum_sub_distrib, mul_sub_left_distrib, mul_sub_right_distrib]
  rw [hDistrib]
  exact mul_mul_conjTranspose_same hAB C

omit [DecidableEq m] in
theorem conjTranspose_mul_mul_mono (C : Matrix n m  $\alpha$ ) :
  A  $\leq$  B  $\rightarrow$  C.conjTranspose * A * C  $\leq$  C.conjTranspose * B * C := fun hAB  $\mapsto$  by

```

```

    convert mul_mul_conjTranspose_mono CH hAB
    <;> rw [conjTranspose_conjTranspose]

theorem nonneg_iff_eigenvalue_nonneg [DecidableEq n] : 0 ≤ A ↔ ∀ x, 0 ≤ hA.eigenvalue x
  Iff.trans Matrix.nonneg_iff_posSemidef hA.posSemidef_iff_eigenvalues_nonneg

omit [Fintype n] in
theorem diag_monotone : Monotone (diag : Matrix n n ℝ → (n → ℝ)) := fun _ _ =>
  le_of_nonneg_imp (diagAddMonoidHom n ℝ) (fun _ => diag_nonneg)

omit [Fintype n] in
theorem diag_mono : A ≤ B → ∀ i, A.diag i ≤ B.diag i := diag_monotone.imp

theorem trace_monotone : Monotone (@trace n ℝ _) := fun _ _ =>
  le_of_nonneg_imp (traceAddMonoidHom n ℝ) (fun _ => trace_nonneg)

theorem trace_mono : A ≤ B → A.trace ≤ B.trace := trace_monotone.imp

variable [DecidableEq n]

omit [Fintype n] in
theorem diagonal_monotone : Monotone (diagonal : (n → ℝ) → ℝ) := fun _ _ =>
  le_of_nonneg_imp' (diagonalAddMonoidHom n ℝ) (fun _ => PosSemidef.diagonal)

omit [Fintype n] in
theorem diagonal_mono {d1 d2 : n → ℝ} : d1 ≤ d2 → diagonal d1 ≤ diagonal d2 := diagonal_monotone.imp

omit [Fintype n] in
theorem diagonal_le_iff {d1 d2 : n → ℝ} : d1 ≤ d2 ↔ diagonal d1 ≤ diagonal d2 := ⟨diagonal_monotone,
  intro hd
  rw [Matrix.le_iff, diagonal_sub, posSemidef_diagonal_iff] at hd
  simp only [sub_nonneg] at hd
  exact hd⟩

theorem le_smul_one_of_eigenvalues_iff (hA : A.IsHermitian) (c : ℝ) :
  (∀ i, hA.eigenvalues i ≤ c) ↔ A ≤ c • (1 : Matrix n n ℝ) := by
  let U : Matrix n n ℝ := hA.eigenvectorUnitary
  have hU : U.conjTranspose = star U := by simp only [star]
  have hU' : U * star U = 1 := by
    simp only [SetLike.coe_mem, Unitary.mul_star_self_of_mem, U]
  have hc : c • (1 : Matrix n n ℝ) = U * (c • 1) * U.conjTranspose := by
    simp only [Algebra.mul_smul_comm, mul_one, hU, Algebra.smul_mul_assoc, hU']
  have hc' : c • (1 : Matrix n n ℝ) = diagonal (RCLike.ofReal ∘ fun _ : n => c) := by
    ext i j
    simp only [smul_apply, one_apply, smul_ite, RCLike.real_smul_eq_coe_mul, mul_one,
      diagonal, Function.comp_apply, of_apply]

```

```

have hAST : A = U * diagonal (RCLike.ofReal ◦ hA.eigenvalues) * U.conjTra
  rw [hU]
  exact IsHermitian.spectral_theorem hA
constructor
· intro h
  rw [hc, hc', hAST]
  apply mul_mul_conjTranspose_mono
  apply diagonal_mono
  intro i
  simp only [Function.comp_apply, algebraMap_le_algebraMap, h i]
intro hAc i
replace hAc := conjTranspose_mul_mul_mono U hAc
have hU'CT : star U * U = 1 := by
  simp only [SetLike.coe_mem, Unitary.star_mul_self_of_mem, U]
have hcCT : U.conjTranspose * (c • 1) * U = c • (1 : Matrix n n  $\mathbb{C}$ ) := by
  simp only [Algebra.mul_smul_comm, mul_one, hU, Algebra.smul_mul_assoc,
have hASTCT : U.conjTranspose * A * U = diagonal (RCLike.ofReal ◦ hA.eige
  rw [hU]
  convert IsHermitian.conjStarAlgAut_star_eigenvectorUnitary hA using 1
  simp +zetaDelta
  rw [hcCT, hc', hASTCT, <-diagonal_le_iff] at hAc
specialize hAc i
simp only [Function.comp_apply, algebraMap_le_algebraMap] at hAc
exact hAc

theorem smul_one_le_of_eigenvalues_iff (hA : A.IsHermitian) (c :  $\mathbb{R}$ ) :
  ( $\forall i, c \leq hA.eigenvalues\ i$ )  $\leftrightarrow$   $c \cdot (1 : Matrix\ n\ n\ \mathbb{C}) \leq A$  := by
  -- I did the lazy thing and just copied the previous proof
  let U : Matrix n n  $\mathbb{C}$  :=  $\uparrow$ hA.eigenvectorUnitary
  have hU : U.conjTranspose = star U := by simp only [star]
  have hU' : U * star U = 1 := by
    simp only [SetLike.coe_mem, Unitary.mul_star_self_of_mem, U]
  have hc :  $c \cdot (1 : Matrix\ n\ n\ \mathbb{C}) = U * (c \cdot 1) * U.conjTranspose$  := by
    simp only [Algebra.mul_smul_comm, mul_one, hU, Algebra.smul_mul_assoc,
  have hc' :  $c \cdot (1 : Matrix\ n\ n\ \mathbb{C}) = diagonal\ (RCLike.ofReal \circ fun\ _ : n \mapsto c)$ 
    ext i j
    simp only [smul_apply, one_apply, smul_ite, RCLike.real_smul_eq_coe_mul
      diagonal, Function.comp_apply, of_apply]
  have hAST : A = U * diagonal (RCLike.ofReal ◦ hA.eigenvalues) * U.conjTra
    rw [hU]
    exact IsHermitian.spectral_theorem hA
constructor
· intro h
  rw [hc, hc', hAST]
  apply mul_mul_conjTranspose_mono
  apply diagonal_mono

```

```

    intro i
    simp only [Function.comp_apply, algebraMap_le_algebraMap, h i]
  intro hAc i
  replace hAc := conjTranspose_mul_mul_mono U hAc
  have hU'CT : star U * U = 1 := by
    simp only [SetLike.coe_mem, Unitary.star_mul_self_of_mem, U]
  have hcCT : U.conjTranspose * (c • 1) * U = c • (1 : Matrix n n  $\alpha$ ) := by
    simp only [Algebra.mul_smul_comm, mul_one, hU, Algebra.smul_mul_assoc, hU'CT]
  have hASTCT : U.conjTranspose * A * U = diagonal (RCLike.ofReal • hA.eigenvalues)
  rw [hU]
  convert IsHermitian.conjStarAlgAut_star_eigenvectorUnitary hA using 1
  simp +zetaDelta
  rw [hcCT, hc', hASTCT, ←diagonal_le_iff] at hAc
  specialize hAc i
  simp only [Function.comp_apply, algebraMap_le_algebraMap] at hAc
  exact hAc

end partialOrder

end PosSemidef

-- noncomputable section frobenius_inner_product
-- open scoped ComplexOrder
-- variable {A : Matrix n n  $\alpha$ } {B : Matrix n n  $\alpha$ } {C : Matrix n n  $\alpha$ } [Fintype n]

-- /-- The InnerProductSpace on Matrix n n  $\alpha$  defined by the real part of the
--   Frobenius inner product. -/
-- def InnerProductCore : InnerProductSpace.Core ( $\alpha$  :=  $\mathbb{R}$ ) (F := Matrix n n  $\alpha$ ):=
--   {
--     inner A B := RCLike.re (AH * B).trace
--     conj_inner_symm := fun x y  $\mapsto$  by
--       simp [inner, starRingEnd_apply, ← trace_conjTranspose] using
--       RCLike.conj_re (xH * y).trace
--     re_inner_nonneg := fun x  $\mapsto$ 
--       (RCLike.nonneg_iff.mp x.posSemidef_conjTranspose_mul_self.trace_nonneg).1
--     add_left := by simp [inner, add_mul]
--     smul_left x y r := by
--       simp using RCLike.smul_re _ (xH * y).trace
--     definite x h := by
--       ext i j
--       replace h :  $\sum j, \sum i, (RCLike.re (x i j) ^ 2 + RCLike.im (x i j) ^ 2) = 0$  :
--       simp [trace, mul_apply, ← pow_two] using h
--       rw [Fintype.sum_eq_zero_iff_of_nonneg (fun i  $\mapsto$  by positivity)] at h
--       replace h := congrFun h j
--       rw [Pi.zero_apply, Fintype.sum_eq_zero_iff_of_nonneg (fun i  $\mapsto$  by positivity)]
--       replace h := congrFun h i

```

```

--      dsimp at h
--      rw [add_eq_zero_iff_of_nonneg (sq_nonneg _) (sq_nonneg _), sq_eq_z
--      apply RCLike.ext (h.left.trans RCLike.zero_re.symm) (h.right.trans
--    }

-- def instNormed : NormedAddCommGroup (Matrix n n  $\mathbb{C}$ ) :=
--   InnerProductCore.toNormedAddCommGroup

-- scoped[Frobenius] attribute [instance] Matrix.instNormed

-- open scoped Frobenius in
-- def instInnerProductSpace : InnerProductSpace  $\mathbb{R}$  (Matrix n n  $\mathbb{C}$ ) :=
--   InnerProductSpace.ofCore InnerProductCore

-- scoped[Frobenius] attribute [instance] Matrix.instInnerProductSpace

-- instance : Inner  $\mathbb{R}$  (Matrix n n  $\mathbb{C}$ ) :=
--   instInnerProductSpace.toInner

-- /-- The InnerProductSpace on Matrix n n  $\mathbb{C}$  defined by the Frobenius inner
-- /
-- def CInnerProductCore : InnerProductSpace.Core ( $\mathbb{C}$  :=  $\mathbb{C}$ ) (F := Matrix n n  $\mathbb{C}$ )
-- {
--   inner A B := (AH * B).trace
--   conj_inner_symm := fun x y  $\mapsto$  by
--     simp [inner, starRingEnd_apply,  $\leftarrow$  Matrix.trace_conjTranspose]
--   re_inner_nonneg := fun x  $\mapsto$ 
--     (RCLike.nonneg_iff.mp x.posSemidef_conjTranspose_mul_self.trace_no
--   add_left := by simp [inner, add_mul]
--   smul_left x y r := by simp
--   definite x h := by
--     ext i j
--     replace h :  $\sum j, \sum i, ((x \ i \ j).re ^ 2 + (x \ i \ j).im ^ 2) = (0 : \mathbb{C})$ 
--     convert h
--     simp only [Complex.ofReal_sum, Complex.ofReal_add, Complex.ofReal
--       mul_apply, conjTranspose_apply, RCLike.star_def]
--     congr! 2
--     norm_cast
--     rw [Complex.conj_mul',  $\leftarrow$  Complex.sq_norm_sub_sq_re]
--     norm_cast
--     abel
--   rw [Complex.ofReal_eq_zero,
--     Fintype.sum_eq_zero_iff_of_nonneg (fun i  $\mapsto$  by positivity)] at h
--   replace h := congrFun h j
--   rw [Pi.zero_apply, Fintype.sum_eq_zero_iff_of_nonneg (fun i  $\mapsto$  by p
--   replace h := congrFun h i

```



```

--      dsimp at h
--      rw [add_eq_zero_iff_of_nonneg (sq_nonneg _) (sq_nonneg _), sq_eq_zero_iff,
--      apply RCLike.ext (h.left.trans RCLike.zero_re.symm) (h.right.trans (map_zero
--    }

-- open scoped Frobenius in
-- def instCInnerProductSpace : InnerProductSpace ℂ (Matrix n n ℂ) :=
--   InnerProductSpace.ofCore CInnerProductCore

-- scoped[Frobenius] attribute [instance] Matrix.instCInnerProductSpace

-- instance : Inner ℂ (Matrix n n ℂ) :=
--   instCInnerProductSpace.toInner

-- Makes the `Inner ℝ` instance is globally accessible, but the norm instances
-- require `open scoped Frobenius`. e.g.

-- open scoped Frobenius in
-- #synth InnerProductSpace ℝ (Matrix (Fin 5) (Fin 5) ℝ)

-- (no `open` needed):
-- #synth Inner ℝ (Matrix (Fin 5) (Fin 5) ℝ)

-- end frobenius_inner_product

section partial_trace

variable [AddCommMonoid R] [Fintype d]

def traceLeft (m : Matrix (d × d₁) (d × d₂) R) : Matrix d₁ d₂ R :=
  Matrix.of fun i₁ j₁ ↦ ∑ i₂, m (i₂, i₁) (i₂, j₁)

def traceRight (m : Matrix (d₁ × d) (d₂ × d) R) : Matrix d₁ d₂ R :=
  Matrix.of fun i₂ j₂ ↦ ∑ i₁, m (i₂, i₁) (j₂, i₁)

variable [Fintype d₁] [Fintype d₂] in
@[simp]
theorem traceLeft_trace (A : Matrix (d₁ × d₂) (d₁ × d₂) R) : A.traceLeft.trace = A.trace
  convert (Fintype.sum_prod_type_right _).symm
  rfl

variable [Fintype d₁] [Fintype d₂] in
@[simp]
theorem traceRight_trace (A : Matrix (d₁ × d₂) (d₁ × d₂) R) : A.traceRight.trace = A.trace
  convert (Fintype.sum_prod_type _).symm
  rfl

```

```

variable [StarAddMonoid R] in
theorem IsHermitian.traceLeft {A : Matrix (d × d₁) (d × d₁) R} (hA : A.IsHe
  ext
  simp only [Matrix.traceLeft, conjTranspose_apply, of_apply, star_sum]
  congr!
  exact congrFun₂ hA _ _

variable [StarAddMonoid R] in
theorem IsHermitian.traceRight {A : Matrix (d₁ × d) (d₁ × d) R} (hA : A.IsH
  ext
  simp only [Matrix.traceRight, conjTranspose_apply, of_apply, star_sum]
  congr!
  exact congrFun₂ hA _ _

variable [DecidableEq dB] [Fintype dA] [Fintype dB] in
open scoped Kronecker in
/--
`Tr(M (A ⊗ I)) = Tr(Tr_B(M) A)`
-/
theorem trace_mul_kron_one_right {R : Type*} [Ring R]
  (M : Matrix (dA × dB) (dA × dB) R) (A : Matrix dA dA R) :
  (M * (A ⊗ₖ (1 : Matrix dB dB R))).trace = (M.traceRight * A).trace := b
  simp [trace, mul_apply, kroneckerMap_apply, traceRight, one_apply,
    Fintype.sum_prod_type, Finset.sum_mul]
  exact Finset.sum_congr rfl fun _ _ => Finset.sum_comm

variable [DecidableEq dA] [Fintype dA] [Fintype dB] in
open scoped Kronecker in
/--
`Tr(M (I ⊗ B)) = Tr(Tr_A(M) B)`
-/
theorem trace_mul_one_kron_right {R : Type*} [Ring R]
  (M : Matrix (dA × dB) (dA × dB) R) (B : Matrix dB dB R) :
  (M * ((1 : Matrix dA dA R) ⊗ₖ B)).trace = (M.traceLeft * B).trace := by
  simp [trace, mul_apply, kroneckerMap_apply, traceLeft, one_apply,
    Fintype.sum_prod_type, Finset.sum_mul]
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun _ _ => Finset.sum_comm

open ComplexOrder

variable {d₁ d₂ : Type*} {A : Matrix (d₁ × d₂) (d₁ × d₂) []}
variable [Fintype d₂] [Fintype d₁]

theorem PosSemidef.traceLeft [DecidableEq d₁] (hA : A.PosSemidef) : A.trace

```

```

rw [Matrix.posSemidef_iff_dotProduct_mulVec] at hA ⊢
constructor
· exact hA.1.traceLeft
· intro x
  convert Finset.sum_nonneg' (s := .univ) (fun (i : d₁) ↦ hA.2 (fun (j,k) ↦ if i =
  simp_rw [Matrix.traceLeft, dotProduct_mulVec]
  simp [dotProduct, vecMul_eq_sum, ite_apply, Fintype.sum_prod_type, Finset.mul_s
  apply_ite] using Finset.sum_comm_cycle

theorem PosSemidef.traceRight [DecidableEq d₂] (hA : A.PosSemidef) : A.traceRight!Po
rw [Matrix.posSemidef_iff_dotProduct_mulVec] at hA ⊢
constructor
· exact hA.1.traceRight
· intro x
  convert Finset.sum_nonneg' (s := .univ) (fun (i : d₂) ↦ hA.2 (fun (j,k) ↦ if i =
  simp_rw [Matrix.traceRight, dotProduct_mulVec]
  simp [dotProduct, vecMul_eq_sum, ite_apply, Fintype.sum_prod_type, Finset.mul_s
  apply_ite] using Finset.sum_comm_cycle

end partial_trace

section posdef

open ComplexOrder
open Kronecker

theorem PosDef.kron {d₁ d₂ [] : Type*} [Fintype d₁] [DecidableEq d₁] [Fintype d₂] [De
  {A : Matrix d₁ d₁ []} {B : Matrix d₂ d₂ []} (hA : A.PosDef) (hB : B.PosDef) : (A ⊗
  rw [hA.left.spectral_theorem, hB.left.spectral_theorem]
  simp only [Unitary.conjStarAlgAut_apply]
  rw [mul_kronecker_mul, mul_kronecker_mul]
  rw [star_eq_conjTranspose, star_eq_conjTranspose]
  rw [← kroneckerMap_conjTranspose]
  rw [diagonal_kronecker_diagonal]
  apply mul_mul_conjTranspose_same
  · rw [posDef_diagonal_iff]
    rintro ⟨i₁, i₂⟩
    convert mul_pos (hA.eigenvalues_pos i₁) (hB.eigenvalues_pos i₂)
    rw [RCLike.pos_iff]
    simp
  · apply Matrix.vecMul_injective_of_isUnit
    rw [isUnit_iff_exists]
    use (star hA.left.eigenvectorUnitary.val) ⊗ₖ (star hB.left.eigenvectorUnitary.val)
    simp [← Matrix.mul_kronecker_mul]

theorem PosDef.submatrix {d₁ d₂ [] : Type*} [Fintype d₁] [DecidableEq d₁] [Fintype d₂]

```

```

    {M : Matrix d1 d1 []} (hM : M.PosDef) {e : d2 → d1} (he : Function.Injective)
    rw [Matrix.posDef_iff_dotProduct_mulVec] at hM ⊢
    use hM.left.submatrix e
    intro x hx
    let y : d1 → [] := fun i ↦ ∑ j ∈ { j | e j = i }, x j
    have hy : y ≠ 0 := by
      contrapose! hx
      simp only [funext_iff] at hx ⊢
      intro i
      simp [y, he.eq_iff, Finset.sum_eq_single_of_mem] using hx (e i)
    convert @hM.right y hy
    dsimp [Matrix.mulVec, dotProduct, y]
    simp only [map_sum]
    simp only [Finset.sum_mul, Finset.sum_filter, Finset.mul_sum]
    simp [← Finset.mul_sum, Finset.sum_comm]

theorem PosDef.reindex {d1 d2 [] : Type*} [Fintype d1] [DecidableEq d1] [Fintype d2]
  {M : Matrix d1 d1 []} (hM : M.PosDef) (e : d1 ≈ d2) : (M.reindex e e).PosDef
  hM.submatrix e.symm.injective

@[simp]
theorem PosDef.reindex_iff {d1 d2 [] : Type*} [Fintype d1] [DecidableEq d1]
  {M : Matrix d1 d1 []} (e : d1 ≈ d2) : (M.reindex e e).PosDef ↔ M.PosDef
  refine {fun h ↦ ?_, fun h ↦ h.reindex e}
  convert h.reindex e.symm
  simp

theorem PosSemidef.rsmul {n : Type*} [Fintype n] {M : Matrix n n ℝ} (hM : M.PosSemidef)
  (c : ℝ) : (c • M).PosSemidef := by
  rw [Matrix.posSemidef_iff_dotProduct_mulVec] at hM ⊢
  constructor
  · exact hM.1.smul_real c
  · peel hM.2
    rw [smul_mulVec, dotProduct_smul]
    positivity

theorem PosDef.Convex {n [] : Type*} [Fintype n] [RCLike []] : Convex ℝ (Matrix n n ℝ)
  intro A hA B hB a b ha hb hab
  rcases ha.lt_or_eq with ha | rfl
  · apply (hA.smul ha).add_posSemidef
    exact hB.posSemidef.smul hb
  · apply Matrix.PosDef.posSemidef_add
    · simp [Matrix.PosSemidef.zero]
    · exact hB.smul (by linarith)

end posdef

```

section eigenvalues

open ComplexOrder

variable {d $\square$: Type*} [Fintype d] [DecidableEq d] [RCLike $\square$]

theorem PosDef_iff_eigenvalues' (M : Matrix d d $\square$) :
 M.PosDef $\leftrightarrow \exists$ (h : M.IsHermitian), $\forall i, 0 < h.eigenvalues\ i :=$
 (fun h $\mapsto$ (h.left, h.left.posDef_iff_eigenvalues_pos.mp h),
 fun (w, h) $\mapsto$ w.posDef_iff_eigenvalues_pos.mpr h)

--These is disgusting atm. There's cleaner versions of them headed to Mathlib. See #ups

theorem IsHermitian.cfc_eigenvalues {M : Matrix d d $\square$ } (hM : M.IsHermitian) (f : $\mathbb{R} \rightarrow$
 $\exists$ (e : d $\approx$ d), Matrix.IsHermitian.eigenvalues (cfc_predicate f M) = f $\circ$ hM.eigenvalues
 have h_eigenvalues : Multiset.map hM.eigenvalues Finset.univ.val = Multiset.map (f
 rfl
 generalize_proofs at *;
 have h_eigenvalues_cfc : (IsHermitian.cfc hM f).charpoly.roots = Multiset.map (fun
 rw [Matrix.IsHermitian.cfc, Matrix.charpoly];
 -- Since \$U\$ is unitary, we have \$U^* U = I\$, and thus the characteristic polyno
 have h_charpoly : Matrix.det ((hM.eigenvectorUnitary : Matrix d d $\square$) * Matrix.di
 -- Since \$U\$ is unitary, we have \$U^* U = I\$, and thus the characteristic poly
 have h_char_poly : $\forall$ (t : $\square$), Matrix.det (t • 1 - (hM.eigenvectorUnitary : Mat
 intro t;
 -- Since \$U\$ is unitary, we have \$U^* U = I\$, and thus the determinant of \$t
 have h_det : Matrix.det (t • 1 - (hM.eigenvectorUnitary : Matrix d d $\square$) * Ma
 simp [mul_sub, sub_mul, mul_assoc];
 rw [h_det, Matrix.det_mul, Matrix.det_mul];
 rw [mul_right_comm, $\leftarrow$ Matrix.det_mul, mul_comm];
 norm_num +zetaDelta at *;
 refine' Polynomial.funext fun t $\Rightarrow$ _;
 convert h_char_poly t using 1;
 · simp [Matrix.det_apply', Polynomial.eval_finset_sum];
 simp [Matrix.one_apply, Polynomial.eval_prod];
 congr! 3;
 aesop;
 · simp [Matrix.det_apply', Polynomial.eval_finset_sum];
 simp [Matrix.one_apply, Polynomial.eval_prod];
 exact Finset.sum_congr rfl fun _ $\Rightarrow$ by congr; ext; aesop;
 simp_all [Matrix.charmatrix, Matrix.det_diagonal];
 rw [Polynomial.roots_prod];
 · bound;
 · exact Finset.prod_ne_zero_iff.mpr fun i _ $\Rightarrow$ Polynomial.X_sub_C_ne_zero _;
 have := Matrix.IsHermitian.roots_charpoly_eq_eigenvalues (cfc_predicate f M);

```

rw [← Matrix.IsHermitian.cfc_eq] at h_eigenvalues_cfc
rw [ h_eigenvalues_cfc ] at this;
simp [ Function.comp ] at this;
rw [ Multiset.map_univ_eq_iff ] at this;
obtain ⟨ e, he ⟩ := this;
use e.symm
ext x
have := congr_fun he ( e.symm x );
simp_all only [Function.comp_apply, Equiv.apply_symm_apply, algebraMap.co

set_option maxHeartbeats 0 in
--Should be combined the above...? TODO Cleanup
/--
If a Hermitian matrix A is unitarily similar to a diagonal matrix with real
-/
lemma IsHermitian.eigenvalues_eq_of_unitary_similarity_diagonal {d [] : Type
  [Fintype d] [DecidableEq d] [RCLike []]
  {A : Matrix d d []} (hA : A.IsHermitian)
  {U : Matrix d d []} (hU : U ∈ Matrix.unitaryGroup d [])
  {f : d → ℝ}
  (h : A = U * Matrix.diagonal (fun i => (RCLike.ofReal (f i) : [])) * Mat
  ∃ σ : d ≈ d, hA.eigenvalues ∘ σ = f := by
-- Since A is unitarily similar to D, they have the same characteristic p
have h_char_poly : Matrix.charpoly A = Matrix.charpoly (Matrix.diagonal f
have h_char_poly : Matrix.charpoly (U * Matrix.diagonal (fun i => (f i
have h_det : ∀ (t : []), Matrix.det (t • 1 - U * Matrix.diagonal (fun
intro t
have h_det : Matrix.det (t • 1 - U * Matrix.diagonal (fun i => (f i
simp [ mul_sub, sub_mul, Matrix.mul_assoc ];
rw [ show U * UH = 1 from by simp [mul_eq_one_comm] using hU.2 ]
rw [h_det, Matrix.det_mul_comm, ← mul_assoc]
rw [← star_eq_conjTranspose, Matrix.UnitaryGroup.star_mul_self ⟨U,
simp
refine' Polynomial.funext fun t => _;
convert h_det t using 1 <=> simp [ Matrix.charpoly, Matrix.det_apply'
· simp [ Polynomial.eval_finset_sum, Polynomial.eval_mul, Polynomial.
exact Finset.sum_congr rfl fun _ => by congr; ext; aesop;
· simp [ Polynomial.eval_finset_sum, Polynomial.eval_mul, Polynomial.
exact Finset.sum_congr rfl fun _ => by congr; ext; aesop;
rw [ h, h_char_poly ];
-- The roots of the characteristic polynomial of A are its eigenvalues (b
have h_eigenvalues : (Matrix.charpoly A).roots = Multiset.map (RCLike.ofR
exact Matrix.IsHermitian.roots_charpoly_eq_eigenvalues hA;
-- The roots of the characteristic polynomial of D are the diagonal entri
have h_diag_roots : (Matrix.charpoly (Matrix.diagonal fun i => (f i : []))

```

```

    simp [ Matrix.charpoly, Matrix.det_diagonal ];
    rw [ Polynomial.roots_prod ];
    · aesop;
    · exact Finset.prod_ne_zero_iff.mpr fun i _ => Polynomial.X_sub_C_ne_zero _;
  have := Multiset.map_univ_eq_iff ( RCLike.ofReal ∘ hA.eigenvalues ) f
  subst h
  simp_all only [Function.comp_apply, RCLike.ofReal_real_eq_id, id_eq, CompTriple.co
  refine' this.mp _ |> fun ( e, he ) => ( e.symm, _ )
  · simpa [ Function.comp ] using congr_arg ( Multiset.map ( RCLike.re :  $\square \rightarrow \mathbb{R}$  ) ) h
  · exact funext fun x => by simpa using congr_fun he ( e.symm x );

end eigenvalues

section

variable { $\alpha$  n : Type*} [RCLike  $\alpha$ ] [Fintype n] [DecidableEq n]

@[simp]
theorem toEuclideanLin_one : Matrix.toEuclideanLin (1 : Matrix n n  $\alpha$ ) = .id := by
  ext1 x
  simp [Matrix.toEuclideanLin]

end

section more_cfc

open ComplexOrder

variable {d  $\square$  : Type*} [Fintype d] [DecidableEq d] [RCLike  $\square$ ]

@[simp]
theorem cfc_diagonal (g : d  $\rightarrow \mathbb{R}$ ) (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :
  cfc f (Matrix.diagonal (fun x  $\mapsto$  (g x :  $\square$ ))) = diagonal (RCLike.ofReal ∘ f ∘ g) :
  --Thanks Aristotle
  have h_self_adjoint : _root_.IsSelfAdjoint (diagonal (fun x  $\mapsto$  (g x :  $\square$ ))) := by
    change Matrix.conjTranspose _ = _
    simp [Matrix.conjTranspose]
  --TODO cfc_cont_tac
  rw [cfc, dif_pos (h_self_adjoint, continuousOn_iff_continuous_restrict.mpr <| by f
  rw [cfcHom_eq_of_continuous_of_map_id]
  rotate_left
  · refine' { .. }
    use fun f  $\mapsto$  Matrix.diagonal fun x  $\mapsto$  f (g x, (by
      simpa [algebraMap_eq_diagonal, diagonal_apply] using
      congr_arg (· x x) ·.exists_left_inv.choose_spec
    ))

```

```

      · simp
      · simp [diagonal, ← Matrix.ext_iff, mul_apply]
        grind
      · simp
      · simp [diagonal, funext_iff]
        grind [add_zero]
      · simp [← ext_iff, diagonal]
        exact fun r i j ↦ rfl
      · simp [← ext_iff, diagonal]
        grind [RCLike.conj_ofReal]
    · dsimp [diagonal]
      continuity
    · simp [diagonal]
    · simp [diagonal]

theorem PosSemidef.pos_of_mem_spectrum {A : Matrix d d []} (hA : A.PosSemidef)
  r ∈ spectrum ℝ A → 0 ≤ r := by
  intro hr
  rw [hA.left.spectrum_real_eq_range_eigenvalues] at hr
  rcases hr with ⟨i, rfl⟩
  exact hA.eigenvalues_nonneg i

theorem PosSemidef.pow_add {A : Matrix d d []} (hA : A.PosSemidef) {x y : ℝ}
  cfc (· ^ (x + y) : ℝ → ℝ) A = cfc (fun r ↦ r ^ x * r ^ y : ℝ → ℝ) A :=
  refine cfc_congr fun r hr ↦ ?_
  exact Real.rpow_add' (hA.pos_of_mem_spectrum r hr) hxy

theorem PosSemidef.pow_mul {A : Matrix d d []} {x y : ℝ} (hA : A.PosSemidef)
  cfc (· ^ (x * y) : ℝ → ℝ) A = cfc (fun r ↦ (r ^ x) ^ y : ℝ → ℝ) A := by
  refine cfc_congr fun r hr ↦ ?_
  exact Real.rpow_mul (hA.pos_of_mem_spectrum r hr) x y

end more_cfc

section subm

variable {α : Type*} [AddCommMonoid α]
variable {d₁ d₂ : Type*} [Fintype d₁] [Fintype d₂]

@[simp]
theorem trace_submatrix
  (A : Matrix d₁ d₁ α) (e : d₂ ≈ d₁) :
  (A.submatrix e e).trace = A.trace := by
  simpa [Matrix.trace] using e.sum_comp (fun x ↦ A x x)

end subm

```


section spectrum_kron

--This is really really ugly, and already *after* trying to clean it up a bit.
set_option maxHeartbeats 7200000

open Kronecker
open scoped Pointwise

```
private lemma spectrum_prod_complex {d d₂ : Type*}
  [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂]
  {A : Matrix d d ℂ} {B : Matrix d₂ d₂ ℂ}
  (hA : A.IsHermitian) (hB : B.IsHermitian) :
  ∀ x : ℂ, x ∈ spectrum ℂ (A ⊗ₖ B) → ∃ a ∈ spectrum ℂ A, ∃ b ∈ spectrum ℂ B, x = a
intro x hx
have h_det : Matrix.det (A ⊗ₖ B - x • 1) = 0 := by
  rw [ spectrum.mem_iff, Matrix.isUnit_iff_isUnit_det ] at hx;
  rw [ ← neg_sub, Matrix.det_neg ]
  simp_all only [isUnit_iff_ne_zero, ne_eq, Decidable.not_not, Fintype.card_prod,
    mul_eq_zero, pow_eq_zero_iff', neg_eq_zero, one_ne_zero, not_or, false_and, fa
  convert hx using 1;
  congr! 1;
  ext ⟨ i, j ⟩ ⟨ i', j' ⟩;
  simp [ Algebra.smul_def ]
-- Since $A$ and $B$ are Hermitian, they are diagonalizable. Let $P$ and $Q$ be un
obtain ⟨P, hP₁, ⟨D, hD⟩⟩ : ∃ P : Matrix d d ℂ, P.det ≠ 0 ∧ ∃ D : Matrix d d ℂ, D.I
refine' ⟨ hA.eigenvectorUnitary, _, Matrix.diagonal ( RCLike.ofReal ∘ hA.eigenva
· intro h_det_zero;
  exact absurd h_det_zero <| isUnit_iff_ne_zero.mp <| UnitaryGroup.det_isUnit hA
· exact isDiag_diagonal (RCLike.ofReal ∘ hA.eigenvalues);
· -- Since $U$ is unitary, $U^{-1} = U^*$, and thus $U^{-1} * U = I$.
  have h_unitary : (hA.eigenvectorUnitary : Matrix d d ℂ)⁻¹ = star (hA.eigenvect
    rw [ Matrix.inv_eq_left_inv ];
    simp
  -- Substitute h_unitary into the equation.
  rw [h_unitary];
  convert Matrix.IsHermitian.conjStarAlgAut_star_eigenvectorUnitary hA using 1
  simp
obtain ⟨Q, hQ₁, ⟨E, hE⟩⟩ : ∃ Q : Matrix d₂ d₂ ℂ, Q.det ≠ 0 ∧ ∃ E : Matrix d₂ d₂ ℂ,
  have := Matrix.IsHermitian.spectral_theorem hB;
  -- By the spectral theorem, since B is Hermitian, there exists a unitary matrix
obtain ⟨Q, hQ_unitary, D, hD_diag, hQ⟩ : ∃ Q : Matrix d₂ d₂ ℂ, Q.det ≠ 0 ∧ ∃ D :
  refine' ⟨ hB.eigenvectorUnitary, _, Matrix.diagonal ( RCLike.ofReal ∘ hB.eigen
· intro h_det_zero;
  -- Since the eigenvector unitary matrix is unitary, its determinant is non-
zero.
```

```

have h_unitary_det : ∀ (U : Matrix d₂ d₂ ℝ), U * star U = 1 → U.det
  fun U hU => Matrix.det_ne_zero_of_right_inverse hU;
exact h_unitary_det _ ( by simp ) h_det_zero;
· exact isDiag_diagonal (RCLike.ofReal ∘ hB.eigenvalues);
· convert this using 1;
rw [ Matrix.inv_eq_left_inv ];
· simp
  rfl
  · simp only [SetLike.coe_mem, Unitary.star_mul_self_of_mem]
refine ( Q, hQ_unitary, D, hD_diag, ?_ )
simp [ hQ, mul_assoc, hQ_unitary, isUnit_iff_ne_zero ];
-- Then  $(P \otimes Q)^{-1}(A \otimes B)(P \otimes Q) = D \otimes E$ , where
have h_diag : (P.kronecker Q)-1 * (A ⊗k B) * (P.kronecker Q) = D ⊗k E :=
-- Using the properties of the Kronecker product and the fact that  $P$ 
have h_kronecker : (P.kronecker Q)-1 * (A.kronecker B) * (P.kronecker Q)
have h_kronecker : ∀ (X Y : Matrix d d ℝ) (Z W : Matrix d₂ d₂ ℝ), (X.
  intro X Y Z W; ext i j; simp [ Matrix.mul_apply ] ;
  simp only [mul_left_comm, mul_comm, Finset.mul_sum _ _ _];
  exact Fintype.sum_prod_type_right _
  rw [Matrix.inv_eq_right_inv, h_kronecker];
  convert h_kronecker P P-1 Q Q-1 using 1;
  simp [ hP₁, hQ₁, isUnit_iff_ne_zero ];
aesop;
-- Since  $D$  and  $E$  are diagonal matrices, the determinant of  $(D \otimes E)$ 
have h_det_diag : Matrix.det (D ⊗k E - x • 1) = 0 := by
  have h_det_diag : Matrix.det ((P.kronecker Q)-1 * (A ⊗k B - x • 1) * (P
    simp [ ← h_diag, mul_sub, sub_mul ];
    simp [ Matrix.det_kronecker, hP₁, hQ₁ ];
  simp_all [ Matrix.det_mul ];
-- Since  $D$  and  $E$  are diagonal matrices, the determinant of  $(D \otimes E)$ 
obtain (i, j, hij) : ∃ i : d, ∃ j : d₂, D i i * E j j = x := by
  contrapose! h_det_diag;
have h_det_diag : Matrix.det (D ⊗k E - x • 1) = ∏ i : d, ∏ j : d₂, (D i
  have h_det_diag : Matrix.det (D ⊗k E - x • 1) = Matrix.det (Matrix.di
    congr with p q
    simp_all only [ne_eq, kronecker, sub_apply,
      kroneckerMap_apply, smul_apply, smul_eq_mul]
    obtain (fst, snd) := p
    obtain (fst_1, snd_1) := q
    obtain (left, rfl) := hD
    obtain (left_1, rfl) := hE
    simp_all only
    by_cases h : fst = fst_1 <=> by_cases h' : snd = snd_1 <=> simp [ h
    · exact Or.inr ( left_1 ( by aesop ) );
    · exact Or.inl ( left h );
    · exact Or.inl ( left h );

```

```

    simp_all [ Matrix.det_diagonal ];
    exact Fintype.prod_prod_type fun (x_2 : d × d₂) => D x_2.1 x_2.1 * E x_2.2 x_2.2
    exact h_det_diag.symm ▸ Finset.prod_ne_zero_iff.mpr fun i _ => Finset.prod_ne_zero
  refine' ( D i i, _, E j j, _, _ )
  · simp_all [ spectrum.mem_iff ];
    simp_all [ Matrix.isUnit_iff_isUnit_det ];
    have h_det_diag : Matrix.det (P⁻¹ * (D i i • 1 - A) * P) = 0 := by
      simp_all [ mul_sub, sub_mul, mul_assoc ];
      rw [ Matrix.det_eq_zero_of_row_eq_zero i ]
      intro j_1
      subst hij
      simp_all only [map_mul, sub_apply, smul_apply, smul_eq_mul]
      obtain ⟨left, rfl⟩ := hD
      obtain ⟨left_1, rfl⟩ := hE
      by_cases hij : i = j_1 <|> simp_all [ Matrix.one_apply ];
      exact left hij;
    simp_all [ Matrix.det_mul ];
    convert h_det_diag using 1;
    exact congr_arg Matrix.det ( by ext i j; by_cases hi : i = j <|> simp [ hi, Algebra
  · simp_all [ spectrum.mem_iff ];
    -- Since $E$ is diagonal, $E j j - B$ is singular, hence not invertible.
    have h_singular : Matrix.det (E j j • 1 - B) = 0 := by
      have h_singular : Matrix.det (Q⁻¹ * (E j j • 1 - B) * Q) = 0 := by
        simp [ mul_sub, sub_mul, hE.2 ];
        rw [ Matrix.det_eq_zero_of_row_eq_zero j ]
        intro j_1
        subst hij
        simp_all only [map_mul, isUnit_iff_ne_zero, ne_eq, not_false_eq_true, nonsingular
          smul_apply, smul_eq_mul]
        obtain ⟨left, rfl⟩ := hD
        obtain ⟨left_1, rfl⟩ := hE
        by_cases h : j = j_1 <|> aesop;
        simp_all [ Matrix.det_mul ];
      simp_all [ Matrix.isUnit_iff_isUnit_det ];
      convert h_singular using 1;
      simp [ Algebra.smul_def ];
    · simp_all [ spectrum.mem_iff ];

private lemma spectrum_prod_le {d d₂ : Type*}
  [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂]
  {A : Matrix d d ℝ} {B : Matrix d₂ d₂ ℝ}
  (hA : A.IsHermitian) (hB : B.IsHermitian) :
    spectrum ℝ (A ⊗ₖ B) ⊆ spectrum ℝ A * spectrum ℝ B := by
  intro x hx
  suffices h : ∃ a ∈ spectrum ℝ A, ∃ b ∈ spectrum ℝ B, x = a * b by
    rcases h with ⟨a, ha, b, hb, rfl⟩

```

```

    exact ⟨a, ha, b, hb, rfl⟩
  obtain ⟨_, ha, _, hb, h₁, ⟨a', rfl⟩, ⟨b', rfl⟩⟩ : ∃ a ∈ spectrum ℝ A, ∃ b
    x = a * b ∧
    a ∈ Set.range (algebraMap ℝ ℝ) ∧ b ∈ Set.range (algebraMap ℝ ℝ) := by
  obtain ⟨a, ha, b, hb, hx_eq⟩ := spectrum_prod_complex hA hB x (by convey)
  have ha' := ha
  have hb' := hb
  rw [hA.spectrum_eq_image_range] at ha
  rw [hB.spectrum_eq_image_range] at hb
  grind
  use a', ha, b', hb
  simp only [RCLike.algebraMap_eq_ofReal] at h₁
  exact_mod_cast h₁

set_option maxHeartbeats 800000
open Kronecker in
open scoped Pointwise in
theorem spectrum_prod {d d₂ : Type*}
  [Fintype d] [DecidableEq d] [Fintype d₂] [DecidableEq d₂]
  {A : Matrix d d ℝ} {B : Matrix d₂ d₂ ℝ}
  (hA : A.IsHermitian) (hB : B.IsHermitian) :
  spectrum ℝ (A ⊗ₖ B) = spectrum ℝ A * spectrum ℝ B := by
  apply subset_antisymm
  · exact spectrum_prod_le hA hB
  · rintro x ⟨ y, hy, z, hz, rfl ⟩;
    -- Since $y$ is an eigenvalue of $A$ and $z$ is an eigenvalue of $B$, t
    obtain ⟨v, hv⟩ : ∃ v : d → ℝ, v ≠ 0 ∧ A.mulVec v = y • v := by
      rw [spectrum.mem_iff] at hy;
      simp_all [Matrix.isUnit_iff_isUnit_det];
      have := Matrix.exists_mulVec_eq_zero_iff.mpr hy;
      simp_all [funext_iff, Matrix.mulVec, dotProduct];
      simp_all [sub_mul, Matrix.one_apply, Algebra.algebraMap_eq_smul_one]
      exact ⟨ this.choose, this.choose_spec.1, fun x => by linear_combinati
this.choose_spec.2 x ⟩
    obtain ⟨w, hw⟩ : ∃ w : d₂ → ℝ, w ≠ 0 ∧ B.mulVec w = z • w := by
      rw [spectrum.mem_iff] at hz;
      simp_all [Matrix.isUnit_iff_isUnit_det];
      have := Matrix.exists_mulVec_eq_zero_iff.mpr hz;
      simp_all [Matrix.sub_mulVec];
      obtain ⟨ w, hw, hw' ⟩ := this; use w; simp_all [sub_eq_zero, Algebra
      simp_all [funext_iff, Matrix.mulVec, dotProduct];
      simp_all [Matrix.one_apply];
      refine' spectrum.mem_iff.mpr _;
      -- Consider the vector $v \otimes w$.
      set v_tensor_w : (d × d₂) → ℝ := fun p => v p.1 * w p.2;
      -- We need to show that $v \otimes w$ is an eigenvector of $A \otimes B$

```

```

have h_eigenvector : (Matrix.kroneckerMap (· * ·) A B).mulVec v_tensor_w = (y *
  ext { i, j } ;
  simp [ Matrix.mulVec, dotProduct ] at *
  simp [ funext_iff, Matrix.mulVec, dotProduct ] at hv hw ⊢
  erw [ Finset.sum_product ]
  simp_all only [v_tensor_w]
  obtain ⟨left, right⟩ := hv
  obtain ⟨left_1, right_1⟩ := hw
  -- By separating the sums, we can apply the given equalities.
  have h_separate :  $\sum x, \sum x_1, A \ i \ x * B \ j \ x_1 * (v \ x * w \ x_1) = (\sum x : d, A \ i$ 
    simp only [mul_left_comm, mul_comm, Finset.mul_sum _ _ _];
    exact Finset.sum_comm.trans ( Finset.sum_congr rfl fun _ _ => Finset.sum_con
  rw [ h_separate, right, right_1 ]
  simp [RCLike.real_smul_eq_coe_mul]
  ring_nf
  -- Since  $v \otimes w$  is an eigenvector of  $A \otimes B$  with eigenvalue  $yz$ ,
  have h_eigenvector_zero :  $((A \otimes_k B) - (y * z) \cdot 1) * v\_tensor\_w = 0$  := by
    simp [ h_eigenvector, Matrix.sub_mulVec ];
    simp [ Matrix.mulVec, funext_iff ];
    simp [ Matrix.one_apply, dotProduct ];
  -- Since  $v \otimes w$  is non-zero, we have  $(A \otimes B - yzI)(v \otimes w) =$ 
  have h_not_invertible :  $\neg \text{IsUnit } (A \otimes_k B - (y * z) \cdot 1)$  := by
    simp only [ne_eq, isUnit_iff_isUnit_det, isUnit_iff_ne_zero, Decidable.not_not
    rw [ ← Matrix.exists_mulVec_eq_zero_iff ]
    refine' { v_tensor_w, _, h_eigenvector_zero };
    simp [ funext_iff ] at hv hw ⊢
    obtain ⟨left, right⟩ := hv
    obtain ⟨left_1, right_1⟩ := hw
    exact ⟨left.choose, left_1.choose, mul_ne_zero left.choose_spec left_1.choose_
  rw [ ← IsUnit.neg_iff, neg_sub ]
  convert h_not_invertible using 4
  simp [ Algebra.smul_def ]

end spectrum_kron

open ComplexOrder in
open MatrixOrder in
theorem PosDef.zero_lt {n : Type*} [Nonempty n] [Fintype n] {A : Matrix n n ℂ} (hA :
  apply lt_of_le_of_ne
  · replace hA := hA.posSemidef
    rwa [Matrix.nonneg_iff_posSemidef]
  · rintro rfl
    --wtf do better. TODO
    have :  $\neg(0 < 0)$  := by trivial
    classical rw [ ← Matrix.posDef_natCast_iff (n := n) (R := ℂ) ] at this
    revert hA

```

```

    convert this
    ext; simp
    trans ((0 : ℕ) : ℂ)
    · simp
    classical
    change _ = ite _ _ _
    simp

lemma IsHermitian.spectrum_eq_image_eigenvalues [Fintype n] {A : Matrix n n ℂ} (hA : IsHermitian A) :
  spectrum ℝ A = Finset.univ.image hA.eigenvalues := by
  simp using hA.spectrum_real_eq_range_eigenvalues

/- This lemma looks "wrong" in the sense that it's specifically about `Fintype`
why not just use the underlying fact `foo = ↑bar`? It turns out this actually works
with dependent rewrites, given the necessary `Fintype` instance. Using the
trying `rw [hA.spectrum_eq_image_eigenvalues]` fails because of dependent types.
/
lemma IsHermitian.card_spectrum_eq_image [Fintype n] {A : Matrix n n ℂ} (hA : IsHermitian A) :
  [Fintype (spectrum ℝ A)] :
  Fintype.card (spectrum ℝ A) = (Finset.univ.image hA.eigenvalues).card := by
  trans (Set.univ.image hA.eigenvalues).toFinset.card
  · symm
  · convert Set.toFinset_card _
  · rw [Set.image_univ]
  · exact Matrix.IsHermitian.spectrum_real_eq_range_eigenvalues hA
  · simp

section iInf_iSup
namespace IsHermitian

variable {d : Type*} [Fintype d] [DecidableEq d] {A B : Matrix d d ℂ}

open ComplexOrder

lemma sub_iInf_eigenvalues (hA : A.IsHermitian) :
  (A - iInf hA.eigenvalues • 1).PosSemidef := by
  rw [Matrix.posSemidef_iff_dotProduct_mulVec]
  constructor
  · simp [Matrix.IsHermitian, sub_eq_add_neg] using hA
  · intro x
    have h_eigenvalue : ∀ i, hA.eigenvalues i ≥ iInf hA.eigenvalues := by
      -- By definition of infimum, for any eigenvalue $i$, we have $hA.eigenvalues i ≥ iInf hA.eigenvalues$
      intros i
      apply le_of_forall_le
      intro j a

```

```

    exact le_trans a (ciInf_le ( Finite.bddBelow_range hA.eigenvalues ) i );
-- Since  $A$  is Hermitian, we can diagonalize it as  $A = Q \Lambda Q^*$ , where  $Q$  is unitary and  $\Lambda$  is diagonal.
obtain (Q,  $\Lambda$ , hQ, h $\Lambda$ ) :  $\exists Q : \text{Matrix } d \rightarrow d, \exists \Lambda : d \rightarrow d, Q.\text{conjTranspose} * Q = 1$ 
have := hA.spectral_theorem;
refine' ( _, _, _, this, _ );
· simp [  $\leftarrow$  Matrix.ext_iff ];
  intro i j; erw [ Matrix.mul_apply ]; simp [ Matrix.one_apply ];
  have := hA.eigenvectorBasis.orthonormal;
  rw [ orthonormal_iff_ite ] at this;
  rw [  $\leftarrow$  this i j ];
  simp [ PiLp.inner_apply, mul_comm ]
· simp
-- Since  $Q$  is unitary, we have  $Q^* Q = I$ , and thus  $Q^* (A - \lambda_{\min}) Q = \Lambda - \lambda_{\min} I$ .
have h_diag :  $Q.\text{conjTranspose} * (A - (\text{ciInf } (\text{Matrix.IsHermitian.eigenvalues } hA))) = \Lambda - \lambda_{\min} I$ 
  simp [ h $\Lambda$ , mul_sub, sub_mul, mul_assoc, hQ ];
  simp [  $\leftarrow$  mul_assoc, hQ ];
  ext i j; by_cases hij :  $i = j$  <|> aesop;
-- Since  $Q$  is unitary, we have  $Q^* (A - \lambda_{\min}) Q = \Lambda - \lambda_{\min} I$ .
have h_quad_form :  $\text{Star.star } x \cdot (\Lambda - (\text{ciInf } (\text{Matrix.IsHermitian.eigenvalues } hA))) = \Lambda - \lambda_{\min} I$ 
  rw [  $\leftarrow$  h_diag ];
  simp [ Matrix.mul_assoc, Matrix.dotProduct_mulVec, mul_eq_one_comm.mp hQ ];
  simp only [ mulVec_conjTranspose, star_star, vecMul_vecMul ];
  rw [  $\leftarrow$  Matrix.mul_assoc, mul_eq_one_comm.mp hQ, one_mul ];
simp_all only [ ge_iff_le, dotProduct, Pi.star_apply, RCLike.star_def, mulVec, smul_apply, Complex.real_smul, conjTranspose_apply, star_sum, star_mul', RingHomCompTriple.comp_apply, RingHom.id_apply ];
simp_all only [ implies_true, and_self, diagonal_apply, ite_mul, zero_mul, Finset.prod_distrib ];
-- Since the eigenvalues are real and the sums involving  $Q$  and  $x$  are complex, the sum is non-negative.
have h_nonneg :  $\forall i, 0 \leq (\sum x_2, Q x_2 i * \text{star } (x x_2)) * (\sum x_2, \text{star } (Q x_2 i * \text{star } (x x_2)))$ 
  intro i
  have h_nonneg :  $0 \leq (\sum x_2, Q x_2 i * \text{star } (x x_2)) * \text{star } (\sum x_2, Q x_2 i * \text{star } (x x_2))$ 
    exact mul_star_self_nonneg ( $\sum x_2, Q x_2 i * \text{star } (x x_2)$ )
  convert h_nonneg using 1;
  simp [ mul_comm, Finset.mul_sum _ _ ];
-- Since each term in the sum is a product of a non-negative number and a non-negative eigenvalue difference, the entire sum is non-negative.
have h_sum_nonneg :  $\forall i, 0 \leq (\sum x_2, Q x_2 i * \text{star } (x x_2)) * (((\uparrow (hA.eigenvalue i) - \lambda_{\min})))$ 
  intro i
  specialize h_nonneg i
  simp_all only [ mul_assoc, mul_comm, mul_left_comm, RCLike.star_def ];
  rw [  $\leftarrow$  mul_assoc ];
  exact mul_nonneg h_nonneg ( sub_nonneg_of_le <| mod_cast h_eigenvalue i );
convert Finset.sum_nonneg fun i _ => h_sum_nonneg i;
rw [ h $\Lambda$ .1 ]

```

```

lemma iInf_eigenvalues_le_dotProduct_mulVec (hA : A.IsHermitian) (v : d → ℂ) :
  iInf hA.eigenvalues * (star v ⌊v v) ≤ star v ⌊v A *v v := by
  conv_lhs =>
    equals (star v ⌊v (iInf hA.eigenvalues • 1) *v v) =>
      simp only [dotProduct, Pi.star_apply, RCLike.star_def, mul_comm, mulV
        simp [Matrix.one_apply, mul_assoc, mul_left_comm, Finset.mul_sum]
  rw [← sub_nonneg, ← dotProduct_sub, ← Matrix.sub_mulVec]
  replace hA := sub_iInf_eigenvalues hA
  rw [Matrix.posSemidef_iff_dotProduct_mulVec] at hA
  exact hA.right v

lemma iInf_eigenvalues_le_of_posSemidef
  (hAB : (B - A).PosSemidef) (hA : A.IsHermitian) (hB : B.IsHermitian) :
  iInf hA.eigenvalues ≤ iInf hB.eigenvalues := by
  rcases isEmpty_or_nonempty d
  · simp
  contrapose! hAB
  rw [posSemidef_iff_dotProduct_mulVec]
  push_neg
  intro _
  apply exists_lt_of_ciInf_lt at hAB
  rcases hAB with ⟨i, hi⟩
  use WithLp.ofLp (hB.eigenvectorBasis i)
  simp only [sub_mulVec, dotProduct_sub, sub_nonneg]
  rw [hB.mulVec_eigenvectorBasis i]
  simp only [dotProduct_smul, Complex.real_smul]
  nth_rw 2 [dotProduct_comm]
  rw [← EuclideanSpace.inner_eq_star_dotProduct]
  intro h
  replace h := (iInf_eigenvalues_le_dotProduct_mulVec hA _).trans h
  rw [dotProduct_comm, ← EuclideanSpace.inner_eq_star_dotProduct] at h
  simp only [OrthonormalBasis.inner_eq_one, mul_one, Complex.real_le_real]
  order

open MatrixOrder in
lemma iInf_eigenvalues_le (hAB : A ≤ B) (hA : A.IsHermitian) (hB : B.IsHermitian) :
  iInf hA.eigenvalues ≤ iInf hB.eigenvalues :=
  iInf_eigenvalues_le_of_posSemidef hAB hA hB

open MatrixOrder in
lemma iInf_eigenvalues_smul_one_le (hA : A.IsHermitian) : iInf hA.eigenvalues ≤
  (PosSemidef.smul_one_le_of_eigenvalues_iff hA (iInf hA.eigenvalues)).mp
  (ciInf_le (Finite.bddBelow_range _))

end IsHermitian
end iInf_iSup

```



```
section matrix_order
```

```
--Shortcut instances. Having these around speeds things out considerably, in some cases
open MatrixOrder
```

```
variable {d : Type*} [Fintype d]
```

```
lemma _shortcut_posSMulMono : PosSMulMono ℝ (Matrix d d ℂ) :=
  inferInstance
```

```
lemma _shortcut_posSmulReflectLE : PosSMulReflectLE ℝ (Matrix d d ℂ) :=
  inferInstance
```

```
scoped[MatrixOrder] attribute [instance] Matrix._shortcut_posSMulMono
scoped[MatrixOrder] attribute [instance] Matrix._shortcut_posSmulReflectLE
```

```
end matrix_order
```

```
open ComplexOrder in
```

```
theorem IsHermitian.spectrum_subset_Ici_of_sub {d : Type*} [Fintype d] [DecidableEq d]
  {A x : Matrix d d ℂ} (hA : A.IsHermitian) (hl : (x - A).PosSemidef) :
  spectrum ℝ x ⊆ Set.Ici (0, hA.eigenvalues) := by
  --Thanks Aristotle
  intro μ hμ
  obtain ⟨v, hv₁, hv₂⟩ : ∃ v : d → ℂ, v ≠ 0 ∧ x.mulVec v = μ • v := by
    have h_singular : ∃ v : d → ℂ, v ≠ 0 ∧ (μ • 1 - x).mulVec v = 0 := by
      simp only [spectrum.mem_iff, Matrix.isUnit_iff_isUnit_det, isUnit_iff_ne_zero,
        convert Matrix.exists_mulVec_eq_zero_iff.mpr hμ;
        simp [Algebra.smul_def]
      refine h_singular.imp fun v h => ⟨h.left, ?_⟩
      simp_all [Matrix.sub_mulVec, sub_eq_iff_eq_add, funext_iff, Matrix.mulVec, dotPr
    -- Since $x - A$ is positive semidefinite, for any eigenvalue $\lambda$ of $x$, we
    have h_lower_bound : ∀ (v : d → ℂ), v ≠ 0 → (star v ⊙ (x.mulVec v)) ≥ (0, (hA.eigenvalues))
    intro v hv_nonzero
    have h_eigenvalue : (star v ⊙ (A.mulVec v)) ≥ (0, (hA.eigenvalues)) * (star v ⊙ v)
    have h_expand : (star v ⊙ (A.mulVec v)) = ∑ i, (hA.eigenvalues i) * (star (hA.eigenvectorUnitary i) ⊙ v)
    change (star v ⊙ (A.mulVec v)) = ∑ i, (hA.eigenvalues i) * (star (hA.eigenvectorUnitary i) ⊙ v)
    have h_decomp : A = ∑ i, (hA.eigenvalues i) • (Matrix.of (fun j k => (hA.eigenvectorUnitary i j * hA.eigenvectorUnitary i k)))
    convert Matrix.IsHermitian.spectral_theorem hA using 1;
    ext i j
    simp only [RCLike.star_def, Matrix.smul_of, Matrix.sum_apply, Matrix.of_apply,
      Pi.smul_apply, Matrix.diagonal, Function.comp_apply, Matrix.mul_apply,
      Matrix.IsHermitian.eigenvectorUnitary_apply, mul_ite, mul_zero,
      Finset.sum_ite_eq', Finset.mem_univ, ↓reduceIte, Matrix.star_apply,
      Unitary.conjStarAlgAut_apply]
```

```

simp [ mul_comm, mul_left_comm, Algebra.smul_def ]
congr! 1
simp [Algebra.algebraMap_eq_smul_one, mul_assoc]
-- Substitute the decomposition of  $A$  into the expression  $(\text{star } v \sqcap_v (A.\text{mulVec } v)) = \sum i, (hA.\text{eigenvalues } i) * (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i))$ 
have h_subst : (star v  $\sqcap_v$  (A.mulVec v)) =  $\sum i, (hA.\text{eigenvalues } i) * (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i))$ 
-- Substitute the decomposition of  $A$  into the expression  $(\text{star } v \sqcap_v (A.\text{mulVec } v)) = (\text{star } v \sqcap_v ((\sum i, (hA.\text{eigenvalues } i) * (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i))))$ 
have h_subst : (star v  $\sqcap_v$  (A.mulVec v)) = (star v  $\sqcap_v$  (( $\sum i, (hA.\text{eigenvalues } i) * (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i))$ )))
rw [  $\leftarrow$  h_decomp ];
-- By the linearity of matrix multiplication and the dot product,
have h_distribute : (star v  $\sqcap_v$  ( $\sum i, (hA.\text{eigenvalues } i) * (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i))$ )) =  $\sum i, (hA.\text{eigenvalues } i) * (\text{star } v \sqcap_v (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i)))$ 
-- By the linearity of matrix multiplication and the dot product,
have h_distribute :  $\forall (M N : \text{Matrix } d \ d \ \square) (v : d \rightarrow \square), \text{Star.star\_mul\_distrib} (M N) v = \text{Star.star\_mul\_distrib} M (N v)$ 
simp [ Matrix.add_mulVec, dotProduct_add ];
-- By induction on the number of terms in the sum, we can apply
have h_induction :  $\forall (n : \mathbb{N}) (M : \text{Fin } n \rightarrow \text{Matrix } d \ d \ \square) (v : d \rightarrow \square), \text{Star.star\_mul\_distrib} M v = \text{Star.star\_mul\_distrib} M (\sum i, M i v)$ 
intro n M v
induction n
· simp [*]
· simp [Fin.sum_univ_succ, *]
convert h_induction ( Fintype.card d ) ( fun i => Matrix.of ( hA.eigenvalues i * (star v  $\sqcap_v$  (hA.eigenvectorBasis i)) ) )
· rw [  $\leftarrow$  Equiv.sum_comp ( Fintype.equivFin d ) ];
simp [ Fintype.equivFin ];
· rw [  $\leftarrow$  Equiv.sum_comp ( Fintype.equivFin d ) ];
simp [ Fintype.equivFin ];
convert h_distribute using 1;
simp only [dotProduct, Pi.star_apply, RCLike.star_def, Matrix.mul_apply, Finset.mul_sum _ _ _, Matrix.smul_apply, Algebra.smul_mul_assoc, Algebra.mul_smul_comm];
simp [ Algebra.smul_def ];
convert h_subst using 2;
simp only [dotProduct, Pi.star_apply, RCLike.star_def, mul_comm, mul_left_comm, Matrix.of_apply, mul_eq_mul_left_iff, map_eq_zero];
simp [ mul_comm, mul_left_comm, Finset.mul_sum _ _ _ ];
-- Since  $\lambda_i \geq \inf(\text{eigenvalues of } A)$  for all  $i$ ,
have h_bound :  $\forall i, (hA.\text{eigenvalues } i) * (\text{star } (hA.\text{eigenvectorBasis } i) \sqcap_v v) \geq 0$ 
intro i
have h_eigenvalue_bound :  $(hA.\text{eigenvalues } i) \geq (\inf i, (hA.\text{eigenvalues } i))$ 
ciInf_le (Set.finite_range _).bddBelow _
-- Since the product of the inner products is real and non-negative, multiplying both sides of the inequality by this product preserves
have h_nonneg :  $0 \leq (\text{star } (hA.\text{eigenvectorBasis } i) \sqcap_v v) * (\text{star } v \sqcap_v (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i)))$ 
-- Since the inner product is conjugate symmetric, we have  $\text{star } v \sqcap_v (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i)) = \text{star } (\text{star } v \sqcap_v (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i)))$ 
have h_conj_symm :  $\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i) = \text{star } (\text{star } v \sqcap_v (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i)))$ 
simp [ dotProduct, mul_comm ];
rw [ h_conj_symm ];
exact mul_star_self_nonneg (star (hA.eigenvectorBasis i)  $\sqcap_v$  v);

```

```

norm_num [ mul_assoc ];
exact mul_le_mul_of_nonneg_right ( mod_cast h_eigenvalue_bound ) h_nonneg;
-- Since  $\sum_i (\text{star } (hA.\text{eigenvectorBasis } i) \sqcap_v v) * (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i))$ 
have h_sum :  $\sum i, (\text{star } (hA.\text{eigenvectorBasis } i) \sqcap_v v) * (\text{star } v \sqcap_v (hA.\text{eigenvectorBasis } i))$ 
have h_sum :  $\sum i, (\text{star } (hA.\text{eigenvectorBasis } i) \sqcap_v v) \bullet (hA.\text{eigenvectorBasis } i)$ 
have := hA.eigenvectorBasis.sum_repr (WithLp.toLp 2 v);
convert this using 1;
simp only [dotProduct, Pi.star_apply, RCLike.star_def, mul_comm,
  hA.eigenvectorBasis.repr_apply_apply, PiLp.inner_apply, RCLike.inner_apply]
simp only [WithLp.ofLp_sum, WithLp.ofLp_smul]
have key :  $\forall (c : d \rightarrow \mathbb{R}) (f : d \rightarrow \text{EuclideanSpace } \mathbb{R} \ d) (w : d \rightarrow \mathbb{R}),$ 
   $(\sum x, c \ x \bullet (f \ x).ofLp = w) \Leftrightarrow (\sum x, c \ x \bullet f \ x = \text{WithLp.toLp } 2 \ w) :=$  by
  intro c f w
  conv_lhs => rw [show  $\sum x, c \ x \bullet (f \ x).ofLp = (\sum x, c \ x \bullet f \ x).ofLp$  from
    rw [WithLp.ofLp_sum]; simp [WithLp.ofLp_smul]]
  constructor
  · intro h; apply_fun WithLp.toLp 2 at h; simp using h
  · intro h; apply_fun WithLp.ofLp at h; simp using h
exact key _ _ _
-- Taking the inner product of both sides of h_sum with star v, we get the d
have h_inner :  $\text{star } v \sqcap_v (\sum i, (\text{star } (hA.\text{eigenvectorBasis } i) \sqcap_v v) \bullet (hA.\text{eigenvectorBasis } i))$ 
congr 1
simp_rw [← WithLp.ofLp_smul, ← WithLp.ofLp_sum, h_sum]
convert h_inner using 1;
simp [ dotProduct, Finset.mul_sum _ _ _ ];
exact Finset.sum_comm.trans ( Finset.sum_congr rfl fun _ _ => Finset.sum_congr rfl )
rw [ h_expand ];
refine' le_trans _ ( Finset.sum_le_sum fun i _ => h_bound i );
simp only [ mul_assoc ];
rw [ ← Finset.mul_sum _ _ _ , h_sum ];
rw [Matrix.posSemidef_iff_dotProduct_mulVec] at h1
have := h1.2 v
simp [Matrix.sub_mulVec] at this
exact le_trans h_eigenvalue this;
change ( $\sum i, hA.\text{eigenvalues } i) \leq \mu$ 
have := h_lower_bound v hv1
simp_all only [ne_eq, star, RCLike.star_def, Matrix.dotProduct_mulVec, ge_iff_le,
  dotProduct_smul];
simp_all only [dotProduct, mul_comm, RCLike.mul_conj];
rw [ Algebra.smul_def ] at this;
-- Since the sum of the squares of the norms of v is positive, we can divide both
have h_sum_pos :  $0 < \sum x : d, (\|v \ x\| : \mathbb{R}) ^ 2 :=$  by
  contrapose! hv1;
  simp_all only [funext_iff, Pi.zero_apply, not_forall, forall_exists_index, Matrix.mul_smul]
  intro i
  rw [← norm_eq_zero]

```

```

    simp [ sq_nonneg ] using le_antisymm ( le_trans ( Finset.single_le_sum
norm_cast at this;
nlinarith

open ComplexOrder in
theorem IsHermitian.spectrum_subset_Iic_of_sub {d : Type*} [Fintype d] [D
{A x : Matrix d d } (hA : A.IsHermitian) (hl : (A - x).PosSemidef) :
    spectrum  $\mathbb{R}$  x  $\subseteq$  Set.Iic ( i, hA.eigenvalues i) := by
have h := spectrum_subset_Ici_of_sub hA.neg (x := -x) ?_
· rcases isEmpty_or_nonempty d
· simp
  rw [← spectrum.neg_eq] at h
  intro  $\mu$  h $\mu$ 
  specialize h (Set.neg_mem_neg.mpr h $\mu$ )
  rw [← Set.mem_neg, Set.neg_Ici] at h
  convert h
  rw [iInf, iSup, ← spectrum_real_eq_range_eigenvalues, ← spectrum_real_e
  rw [← spectrum.neg_eq, csInf_neg ?_ (A.finite_real_spectrum.bddAbove),
  exact IsSelfAdjoint.spectrum_nonempty hA
· convert hl using 1
  abel

open ComplexOrder in
theorem IsHermitian.spectrum_subset_of_mem_Icc {d : Type*} [Fintype d] [D
{A B x : Matrix d d } (hA : A.IsHermitian) (hB : B.IsHermitian)
(hl : (x - A).PosSemidef) (hr : (B - x).PosSemidef) :
    spectrum  $\mathbb{R}$  x  $\subseteq$  Set.Icc ( i, hA.eigenvalues i) ( i, hB.eigenvalues i)
  rw [← Set.Ici_inter_Iic]
  exact Set.subset_inter (hA.spectrum_subset_Ici_of_sub hl) (hB.spectrum_su

/--
The right partial trace of a matrix is equal to the left partial trace of t
-/
theorem traceRight_eq_traceLeft_reindex {n m R : Type*} [Fintype m] [AddComm
(M : Matrix (n  $\times$  m) (n  $\times$  m) R) :
    M.traceRight = (M.reindex (.prodComm ..) (.prodComm ..)).traceLeft := b
  rfl

open ComplexOrder in
theorem PosSemidef.trace_pos {n : Type*} [Fintype n] [RCLike ]
{A : Matrix n n } (hA : A.PosSemidef) (h : A  $\neq$  0) : 0 < A.trace := by
  apply hA.trace_nonneg.lt_of_ne'
  classical
  rw [hA.left.trace_eq_sum_eigenvalues]
  suffices  $\sum$  i, hA.left.eigenvalues i  $\neq$  0 from mod_cast this
  rwa [ne_eq, Fintype.sum_eq_zero_iff_of_nonneg hA.eigenvalues_nonneg,
```

```

    hA.left.eigenvalues_eq_zero_iff]

section traceLeftRight

variable {m α : Type*} [AddCommGroup α] [Fintype m]
omit [DecidableEq n]

variable {A B : Matrix (m × n) (m × n) α}
@[simp]
theorem traceLeft_add : (A + B).traceLeft = A.traceLeft + B.traceLeft := by
  ext : 2
  simp [Matrix.traceLeft, Finset.sum_add_distrib]

@[simp]
theorem traceLeft_neg : (-A).traceLeft = -A.traceLeft := by
  ext : 2; simp [Matrix.traceLeft]

@[simp]
theorem traceLeft_sub : (A - B).traceLeft = A.traceLeft - B.traceLeft := by
  simp [sub_eq_add_neg]

variable {A B : Matrix (n × m) (n × m) α}

@[simp]
theorem traceRight_add : (A + B).traceRight = A.traceRight + B.traceRight := by
  ext : 2
  simp [Matrix.traceRight, Finset.sum_add_distrib]

@[simp]
theorem traceRight_neg : (-A).traceRight = -A.traceRight := by
  ext : 2; simp [Matrix.traceRight]

@[simp]
theorem traceRight_sub : (A - B).traceRight = A.traceRight - B.traceRight := by
  simp [sub_eq_add_neg]

variable {R : Type*} [DistribSMul R α]
@[simp]
theorem traceLeft_smul {A : Matrix (m × n) (m × n) α} (r : R) :
  (r • A).traceLeft = r • A.traceLeft := by
  ext : 2; simp [Matrix.traceLeft, ← Finset.smul_sum]

@[simp]
theorem traceRight_smul {A : Matrix (n × m) (n × m) α} (r : R) :
  (r • A).traceRight = r • A.traceRight := by
  ext : 2; simp [Matrix.traceRight, ← Finset.smul_sum]

```

```
end traceLeftRight
```

```
theorem unitaryGroup_row_norm [Fintype n] (U : Matrix.unitaryGroup n ℂ) (i : ℕ) :
  ∑ j, ‖U j i‖^2 = 1 := by
  suffices ∑ j, ‖U j i‖^2 = (1 : ℂ) by exact_mod_cast this
  simp [Matrix.mul_apply, Complex.sq_norm, Complex.normSq_eq_conj_mul_self]
  using congr $(U.prop.left) i i
```

```
section finprod
```

```
variable {ι : Type*} {d : ι → Type*} [fι : Fintype ι]
variable {R : Type*}
```

```
def piProd [CommMonoid R] (A : ∀ i, Matrix (d i) (d i) R) : Matrix (∀ i, d i) (d i) R :=
  Matrix.of (fun j k : (∀ i, d i) ↦ ∏ i, A i (j i) (k i))
```

```
variable {A : ∀ i, Matrix (d i) (d i) R}
```

```
theorem IsHermitian.piProd [CommSemiring R] [StarRing R] (hA : ∀ i, (A i).IsHermitian) :
  (piProd A).IsHermitian := by
  ext j k
  simp [Matrix.piProd]
  exact Finset.prod_congr rfl fun i _ => by simp using congr_fun (congr_fun hA i)
```

```
variable [DecidableEq ι] [∀ i, Fintype (d i)] --[∀ i, DecidableEq (d i)]
```

```
theorem trace_piProd [CommSemiring R] :
  (piProd A).trace = ∏ i, (A i).trace := by
  symm
  simp [trace, piProd, Fintype.prod_sum]
```

```
open ComplexOrder in
```

```
set_option maxHeartbeats 400000 in
```

```
theorem PosSemidef.piProd [RCLike R] (hA : ∀ i, (A i).PosSemidef) :
  (piProd A).PosSemidef := by
```

```
-- Let B i be the square root of A i. Let BigB be the pi-product of B i. Show that BigB.conjTranspose * BigB equals the pi-product of A i using Fintype.prod_sum. Then use Matrix.PosSemidef.conjTranspose
```

```
obtain ⟨B, hB⟩ : ∃ B : ∀ i, Matrix (d i) (d i) R, ∀ i, (A i) = B i * star B i
```

```
-- By definition of positive semi-definite matrices, each $A_i$ can be
have h_decomp : ∀ i, ∃ B : Matrix (d i) (d i) R, A i = B * star B := by
  intro i
```

```
obtain ⟨B, hB⟩ : ∃ B : Matrix (d i) (d i) R, A i = B.conjTranspose * B
  exact Matrix.posSemidef_iff_eq_conjTranspose_mul_self.mp (hA i)
  use B.conjTranspose;
```

```

    convert hB using 1;
    simp [ Matrix.star_eq_conjTranspose ];
    exact ( fun i => Classical.choose ( h_decomp i ), fun i => Classical.choose_spec
have hBigB_conjTranspose_mul_BigB : Matrix.of (fun j k : (∀ i, d i) => [] i, (B i *
    ext j k; simp [ Matrix.mul_apply]
    simp only [Finset.prod_sum, ← Finset.prod_mul_distrib];
    refine' Finset.sum_bij ( fun p hp => fun i => p i ( Finset.mem_univ i ) ) _ _ _
    · simp [ funext_iff ];
    · exact fun b => ( fun i _ => b i, rfl );
simp only [Matrix.posSemidef_iff_dotProduct_mulVec] at hA ⊢
simp_all [Matrix.piProd]
constructor
· ext1
  simp [Matrix.mul_apply, mul_comm]
· intro x
  set y := star (Matrix.of (fun j k : (∀ i, d i) => [] i, B i (j i) (k i))) *v x
  convert dotProduct_star_self_nonneg y using 1
  simp [ Matrix.dotProduct_mulVec]
  simp [ Matrix.dotProduct_mulVec, y ];
  simp [ Matrix.vecMul, dotProduct, mul_comm ];
  simp [ Matrix.mul_apply, Matrix.mulVec, dotProduct, Finset.mul_sum _ _ _ , mul_as
  exact Finset.sum_congr rfl fun _ _ => Finset.sum_comm.trans ( Finset.sum_congr r
end finprod

--TODO: Can this be used for `Matrix.reindex_eq_conj` cleanup?
theorem submatrix_eq_mul_mul {d d₂ d₃ R : Type*} [DecidableEq d] [Fintype d] [Semiring R]
  (A : Matrix d d R) (e : d₂ → d) (f : d₃ → d) :
  A.submatrix e f = (submatrix (α := R) 1 e id : Matrix d₂ d R) * A * (submatrix (
  rw [show id = Equiv.refl d by rfl, Matrix.mul_submatrix_one, Matrix.one_submatrix
  simp

open scoped Matrix Kronecker in
/--
The conjugate of a Kronecker product by a Kronecker product is the Kronecker product
-/
lemma kronecker_conj_eq {m n p q α : Type*} [CommSemiring α] [StarRing α] [Fintype m]
  (A : Matrix m m α) (B : Matrix n n α) (C : Matrix p m α) (D : Matrix q n α) :
  (C ⊗k D) * (A ⊗k B) * (C ⊗k D)H = (C * A * CH) ⊗k (D * B * DH) := by
  rw [← Matrix.mul_kronecker_mul]
  ext1
  simp only [Matrix.mul_apply, Matrix.kroneckerMap_apply, Matrix.conjTranspose_apply]
  simp only [← starRingEnd_apply, mul_comm, Finset.mul_sum, mul_left_comm]
  simp only [Finset.sum_mul, mul_assoc, Finset.mul_sum, mul_left_comm]
  rw [Fintype.sum_prod_type_right]

```

end Matrix

1.477 QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.Matrix
import QuantumInfo.ForMathlib.Isometry

open BigOperators
open Classical

namespace Matrix
noncomputable section traceNorm

open scoped ComplexOrder

variable {m n R : Type*}
variable [Fintype m] [Fintype n]
variable [RCLike R]

/-- The trace norm of a matrix:  $\text{Tr}[\sqrt{(A^\dagger A)}]$ . -/
def traceNorm (A : Matrix m n R) : ℝ :=
  open MatrixOrder in
  RCLike.re (CFC.sqrt (AH * A)).trace

@[simp]
theorem traceNorm_zero : traceNorm (0 : Matrix m n R) = 0 := by
  simp [traceNorm]

/-- The trace norm of the negative is equal to the trace norm. -
/
@[simp]
theorem traceNorm_eq_neg_self (A : Matrix m n R) : traceNorm (-A) = traceNorm A := by
  unfold traceNorm
  congr! 3
  rw [Matrix.conjTranspose_neg, Matrix.neg_mul, Matrix.mul_neg]
  exact neg_neg _

open scoped MatrixOrder Isometry

```



```

lemma cfc_sqrt_isometry_conj {A : Matrix n n R} (hA : 0 ≤ A)
  {u : Matrix m n R} (hu₁ : u.Isometry) :
  CFC.sqrt (u * A * uH) = u * CFC.sqrt A * uH := by
  have h_conj (B : Matrix n n R) (hB : 0 ≤ B) : 0 ≤ u * B * uH := by
    rw [Matrix.nonneg_iff_posSemidef] at hB ⊢
    exact hB.mul_mul_conjTranspose_same u
  have hu := h_conj A hA
  have hu' := h_conj (CFC.sqrt A) (CFC.sqrt_nonneg A)
  apply (CFC.sqrt_eq_iff _ _ hu hu').mpr
  . rw [Matrix.mul_assoc, ← Matrix.mul_assoc uH, ← Matrix.mul_assoc uH]
    simp [show uH * u = 1 from hu₁]
  . rw [← Matrix.mul_assoc, Matrix.mul_assoc u, CFC.sqrt_mul_sqrt_self A hA]

theorem traceNorm_isometry_left [Fintype k] {A : Matrix n m R} {u : Matrix k n R}
  (hu₁ : u.Isometry) : traceNorm (u * A) = traceNorm A := by
  unfold traceNorm
  congr 1
  simp [Matrix.mul_assoc]
  nth_rw 2 [← Matrix.mul_assoc]
  simp [show uH * u = 1 from hu₁]

theorem traceNorm_isometry_right [Fintype k] {A : Matrix n m R} {u : Matrix k m R}
  (hu₁ : u.Isometry) : traceNorm (A * uH) = traceNorm A := by
  unfold traceNorm
  congr 1
  simp [← Matrix.mul_assoc]
  nth_rw 2 [Matrix.mul_assoc]
  have hA := (Matrix.posSemidef_conjTranspose_mul_self A).nonneg
  rw [cfc_sqrt_isometry_conj hA hu₁, Matrix.trace_mul_comm, ← Matrix.mul_assoc]
  simp [show uH * u = 1 by exact hu₁]

/-- The trace norm is invariant under isometries u and v, Property 9.1.4 in Wilde -
/
theorem traceNorm_isometry_conj {A : Matrix n n R} {u : Matrix m n R}
  (hu : u.Isometry) {v : Matrix m n R} (hv : v.Isometry) :
  traceNorm (u * A * vH) = traceNorm A := by
  rw [traceNorm_isometry_right hv, traceNorm_isometry_left hu]

@[simp]
theorem traceNorm_unitary_conj {A : Matrix n n R} {U : Matrix.unitaryGroup n R} :
  traceNorm (U.val * A * U.valH) = traceNorm A := by
  have hu := (Matrix.mem_unitaryGroup_iff_isometry U.val).mp U.2
  exact traceNorm_isometry_conj hu.1 hu.1

--More generally sum of abs of singular values.
--Proposition 9.1.1 in Wilde

```

```

theorem traceNorm_Hermitian_eq_sum_abs_eigenvalues {A : Matrix n n R} (hA :
  A.traceNorm =  $\sum i, \text{abs } (hA.\text{eigenvalues } i)$  := by
  obtain ⟨U, D, hD, hA_eq, h_eig⟩ :  $\exists U : \text{Matrix.unitaryGroup } n \text{ } R, \exists D : \text{Matrix.diagonal } n \text{ } R,$ 
    refine' ⟨hA.eigenvectorUnitary, _, isDiag_diagonal _, hA.spectral_theorem⟩
    simp [diagonal]
  nth_rw 1 [hA_eq, traceNorm_unitary_conj]
  unfold traceNorm
  rw [← Matrix.IsDiag.diagonal_diag hD]
  simp [Matrix.diagonal_mul_diagonal, h_eig]
  simp_rw [← sq, ← Real.sqrt_sq_eq_abs, ← Matrix.trace_diagonal]
  set B := ((diagonal fun i => (hA.eigenvalues i : R) ^ 2)) with bD
  rw [CFC.sqrt_eq_real_sqrt B _, bD]
  . rw [cfc_n_eq_cfc]
  . rw [mod_cast [cfc_diagonal (g := fun i => (hA.eigenvalues i) ^ 2)]]
  . simp
  . apply Matrix.PosSemidef.nonneg
  . rw [Matrix.posSemidef_diagonal_iff]
  . exact_mod_cast fun i => sq_nonneg (hA.eigenvalues i)

/-- The trace norm is nonnegative. Property 9.1.1 in Wilde -
/
theorem traceNorm_nonneg (A : Matrix m n R) : 0 ≤ A.traceNorm :=
  open MatrixOrder in
  And.left $ RCLike.nonneg_iff.1
  (Matrix.nonneg_iff_posSemidef.mp (CFC.sqrt_nonneg (AH * A))).trace_nonneg

/-- The trace norm is zero iff. the matrix is zero. -/
theorem traceNorm_zero_iff (A : Matrix m n R) : A.traceNorm = 0 ↔ A = 0 :=
  open MatrixOrder in
  set B := CFC.sqrt (AH * A) with hB_de
  have hB_posSemidef := Matrix.nonneg_iff_posSemidef.mp (CFC.sqrt_nonneg (AH * A))
  have hB_hermitian : B.IsHermitian := hB_posSemidef.1
  have hB_pos : B.PosSemidef := ⟨hB_hermitian, hB_posSemidef.2⟩
  constructor
  · intro h
    have h2 :  $\forall i, hB\_hermitian.\text{eigenvalues } i = 0$  := by
      have h_sum :  $(\uparrow(\sum j, hB\_hermitian.\text{eigenvalues } j) : R) = 0$  := by
        rw [hB_hermitian.sum_eigenvalues_eq_trace, ← hB_hermitian.re_trace]
        unfold traceNorm at h
        norm_cast
      have :  $\sum j, hB\_hermitian.\text{eigenvalues } j = 0$  := by exact_mod_cast h_sum
      intro i
      exact Finset.sum_eq_zero_iff_of_nonneg (λ j _ => hB_pos.eigenvalues j)
        |>.mp this i (Finset.mem_univ i)
    have h3 : CFC.sqrt (AH * A) = 0 := hB_hermitian.eigenvalues_zero_eq_zero
    have h4 : AH * A = 0 := by

```

```

      simp [h₃] using (
        CFC.nnrpow_sqrt_two (AH * A)
        (Matrix.nonneg_iff_posSemidef.mpr A.posSemidef_conjTranspose_mul_self)
      ).symm
    rw [Matrix.conjTranspose_mul_self_eq_zero] at h₄
    exact h₄
  · rintro rfl
    simp

/-- Trace norm is linear under scalar multiplication. Property 9.1.2 in Wilde -
/
theorem traceNorm_smul (A : Matrix m n R) (c : R) : (c • A).traceNorm = ||c|| * A.traceNorm := by
  have h : (c • A)H * (c • A) = (||c||2:R) • (AH * A) := by
    rw [conjTranspose_smul, RCLike.star_def, mul_smul, smul_mul, smul_smul]
    rw [RCLike.mul_conj c]
  rw [traceNorm, h]
  open MatrixOrder in
  have : RCLike.re (trace (||c|| • CFC.sqrt (AH * A))) = ||c|| * traceNorm A := by
    simp [RCLike.smul_re]
    apply Or.inl
    rfl
  convert this using 3
  rw [RCLike.real_smul_eq_coe_smul (K := R) ||c||]
  by_cases h : c = 0
  · subst c
    simp
  · have hM_pd : (AH * A).PosSemidef := by apply posSemidef_conjTranspose_mul_self
    set M := (AH * A)
    rw [sq]
    simp [SemigroupAction.mul_smul]
    apply CFC.sqrt_unique;
    · simp; rw [CFC.sqrt_mul_sqrt_self M hM_pd.nonneg]
    · exact le_trans (by norm_num) (
      smul_le_smul_of_nonneg_left (show 0 ≤ CFC.sqrt M from by exact (CFC.sqrt_nonneg M)))
  /

/-- For square matrices, the trace norm is max Tr[U * A] over unitaries U.-
/
theorem traceNorm_eq_max_tr_U (A : Matrix n n R) : IsGreatest {x | ∃ (U : unitaryGroup n R), x = traceNorm (U * A)} A := by
  sorry

/-- the trace norm satisfies the triangle inequality (for square matrices). TODO: Prove this.
/
theorem traceNorm_triangleIneq (A B : Matrix n n R) : (A + B).traceNorm ≤ A.traceNorm + B.traceNorm := by
  obtain ⟨Uab, h₁⟩ := (traceNorm_eq_max_tr_U (A + B)).left
  rw [Matrix.mul_add, Matrix.trace_add] at h₁
  obtain h₂ := (traceNorm_eq_max_tr_U A).right

```

```

obtain h3 := (traceNorm_eq_max_tr_U B).right
simp only [upperBounds, Subtype.exists, exists_prop, Set.mem_setOf_eq, fo
  and_imp, forall_apply_eq_imp_iff2] at h2 h3
replace h2 := h2 Uab.1 Uab.2
replace h3 := h3 Uab.1 Uab.2
rw [← RCLike.ofReal_le_ofReal (K := R)]
simp only [RCLike.ofReal_add]
calc _
  _ = _ + _ := h1.symm
  _ ≤ ↑(traceNorm A) + trace (↑Uab * B) := by simp only [add_le_add_iff_r
  _ ≤ _ := by simp only [add_le_add_iff_left]; exact h3

theorem traceNorm_triangleIneq' (A B : Matrix n n R) : (A - B).traceNorm ≤
  rw [sub_eq_add_neg A B, ←traceNorm_eq_neg_self B]
  exact traceNorm_triangleIneq A (-B)

theorem PosSemidef.traceNorm_PSD_eq_trace {A : Matrix m m R} (hA : A.PosSem
  have : AH * A = A2 := by rw [hA.1, pow_two]
  open MatrixOrder in
  rw [traceNorm, this, CFC.sqrt_sq A, hA.1.re_trace_eq_trace]

/-- The trace norm is convex. Property 9.1.5 in Wilde -/
theorem traceNorm_convex (M N : Matrix n n R) (l : ℝ) (hl : 0 ≤ l ∧ l ≤ 1)
  ((l : R) • M + ((1 - l) : R) • N).traceNorm ≤ l * M.traceNorm + (1 -
l) * N.traceNorm := by
  refine (traceNorm_triangleIneq _ _).trans ?_
  simp_rw [traceNorm_smul]
  nth_rw 1 [← RCLike.ofReal_one]
  simp_rw [← RCLike.ofReal_sub, RCLike.norm_ofReal]
  rw [abs_of_nonneg (hl.1), abs_of_nonneg (sub_nonneg.mpr (hl.2))]

end traceNorm

end Matrix

```

1.478 QuantumInfo/ForMathlib/Minimax.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.SionMinimax

```

```

--minimize these imports?
import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.GroupTheory.MonoidLocalization.Basic
import Mathlib.LinearAlgebra.FiniteDimensional.Defs
import Mathlib.Topology.Algebra.Module.StrongTopology
import Mathlib.Topology.Algebra.Module.FiniteDimension

--TODO go elsewhere
attribute [fun_prop] LowerSemicontinuous --UpperSemicontinuous
attribute [fun_prop] LowerSemicontinuousOn --UpperSemicontinuousOn
attribute [fun_prop] LowerSemicontinuous.lowerSemicontinuousOn
attribute [fun_prop] UpperSemicontinuous.upperSemicontinuousOn
attribute [fun_prop] Continuous.lowerSemicontinuous Continuous.upperSemicontinuous

attribute [fun_prop] QuasilinearOn QuasiconvexOn QuasiconcaveOn

attribute [fun_prop] QuasiconvexOn.sup
attribute [fun_prop] QuasiconcaveOn.inf
attribute [fun_prop] QuasiconcaveOn.inf

attribute [fun_prop] ConvexOn ConcaveOn

attribute [fun_prop] ConvexOn.quasiconvexOn ConcaveOn.quasiconcaveOn
attribute [fun_prop] LinearMap.convexOn LinearMap.concaveOn

theorem _root_.IsCompact.exists_isMinOn_lowerSemicontinuousOn {α β : Type*}
  [LinearOrder α] [TopologicalSpace α] [TopologicalSpace β] [ClosedIicTopology α]
  {s : Set β} (hs : IsCompact s) (ne_s : s.Nonempty) {f : β → α} (hf : LowerSemicont
    ∃ x ∈ s, IsMinOn f s x := by
  --Thanks Aristotle
  -- By the Extreme Value Theorem for lower semicontinuous functions on compact sets
  have h_extreme : ∃ x ∈ s, ∀ y ∈ s, f x ≤ f y := by
    by_contra! h;
    choose! g hg using h;
    -- For each  $x \in s$ , since  $f$  is lower semicontinuous at  $x$ , there exists a
    have h_neighborhood : ∀ x ∈ s, ∃ U : Set β, IsOpen U ∧ x ∈ U ∧ ∀ y ∈ U ∩ s, f y
      intro x hx;
      have := hf x hx;
      rcases mem_nhdsWithin_iff_exists_mem_nhds_inter.mp ( this ( f ( g x ) ) ( hg x
        exact ( interior U, isOpen_interior, mem_interior_iff_mem_nhds.mpr hU, fun y h
      choose! U hU using h_neighborhood;
    -- Since  $s$  is compact, the open cover  $\{U_x \cap s \mid x \in s\}$  has a fini
    obtain ⟨t, ht⟩ : ∃ t : Finset β, (∀ x ∈ t, x ∈ s) ∧ s ⊆ ⋃ x ∈ t, U x ∩ s := by
    -- Since  $s$  is compact, the open cover  $\{U_x \mid x \in s\}$  has a finite su
    obtain ⟨t, ht⟩ : ∃ t : Finset β, (∀ x ∈ t, x ∈ s) ∧ s ⊆ ⋃ x ∈ t, U x := by
      exact hs.elim_nhds_subcover U fun x hx => IsOpen.mem_nhds ( hU x hx |>.1 ) (

```

```

    exact ( t, ht.1, fun x hx => by rcases Set.mem_iUnion₂.1 ( ht.2 hx )
-- Since $t$ is finite, there exists $x \in t$ such that $f(g(x))$ is m
obtain (x, hx) :  $\exists x \in t, \forall y \in t, f(g\ x) \leq f(g\ y)$  := by
  apply_rules [ Finset.exists_min_image ];
-- Since $$s$ is nonempty, there exists some $y \in s$.
obtain (y, hy) :  $\exists y, y \in s$  := ne_s;
  exact Exists.elim ( Set.mem_iUnion₂.1 ( ht.2 hy ) ) fun x hx => ( x,
obtain ( y, hy ) := ht.2 ( hg x ( ht.1 x hx.1 ) |>.1 );
simp_all only [Set.mem_inter_iff, and_self, and_true, gt_iff_lt, and_im
obtain (left, right) := ht
obtain (left_1, right_1) := hx
obtain ((w, rfl), right_2) := hy
simp_all only [Set.mem_iUnion, Set.mem_inter_iff, and_true, exists_prop
obtain (left_2, right_2) := right_2
  exact lt_irrefl _ ( lt_of_le_of_lt ( right_1 _ left_2 ) ( hU _ ( left _
-- By definition of IsMinOn, we need to show that for all y in s, f(x) ≤
obtain (x, hx_s, hx_min) := h_extreme;
use x, hx_s;
exact hx_min

@[fun_prop]
theorem LinearMap.quasilinearOn {E β α : Type*} [Semiring α] [PartialOrder
  [AddCommMonoid E] [Module α E]
  [PartialOrder β] [AddCommMonoid β] [IsOrderedAddMonoid β] [Module α β] [P
  (f : E →1 [α] β) {s : Set E} (hs : Convex α s) :
    QuasilinearOn α s f := by
  constructor
  · apply ConvexOn.quasiconvexOn
    apply LinearMap.convexOn
    assumption --why doesn't fun_prop find this assumption? TODO learn
  · apply ConcaveOn.quasiconcaveOn
    apply LinearMap.concaveOn
    assumption --why doesn't fun_prop find this assumption? TODO

@[fun_prop]
theorem LinearMap.quasiconvexOn {E β α : Type*} [Semiring α] [PartialOrder
  [AddCommMonoid E] [Module α E]
  [PartialOrder β] [AddCommMonoid β] [IsOrderedAddMonoid β] [Module α β] [P
  (f : E →1 [α] β) {s : Set E} (hs : Convex α s) :
    QuasiconvexOn α s f :=
  (f.quasilinearOn hs).left

@[fun_prop]
theorem LinearMap.quasiconcaveOn {E β α : Type*} [Semiring α] [PartialOrder
  [AddCommMonoid E] [Module α E]
  [PartialOrder β] [AddCommMonoid β] [IsOrderedAddMonoid β] [Module α β] [P

```

```

(f : E →1[[]] β) {s : Set E} (hs : Convex [] s) :
  QuasiconcaveOn [] s f :=
  (f.quasilinearOn hs).right

--??
theorem continuous_stupid.{u_2, u_1} {M : Type u_1} [inst : NormedAddCommGroup M] [i
[inst_3 : ContinuousSMul ℝ M] {N : Type u_2} [inst_4 : NormedAddCommGroup N]
[inst_5 : Module ℝ N]
[FiniteDimensional ℝ M]
(f : N →L[ℝ] M →L[ℝ] ℝ) :
  Continuous fun (x : N × M) ↦ (f x.1) x.2 :=
have h_eval : Continuous (fun p : (M →L[ℝ] ℝ) × M ↦ p.1 p.2) := by
  have h_sum : Continuous (fun p : (M →L[ℝ] ℝ) × M ↦
    ∑ i, p.1 (Module.finBasis ℝ M i) * ((Module.finBasis ℝ M).repr p.2) i) := by
    refine continuous_finset_sum _ fun i _ ↦ .mul (by fun_prop) ?_
    · exact ((Module.finBasis ℝ M).coord i).continuous_of_finiteDimensional.comp c
  convert h_sum with x
  rw [← (Module.finBasis ℝ M).sum_repr x.2, map_sum]
  simp [mul_comm]
h_eval.comp <| .prodMk (f.continuous.comp continuous_fst) continuous_snd

/-- The minimax theorem, at the level of generality we need. For convex, compact, no
and `T` in a real topological vector space `M`, and a bilinear function `f` on M, we
the order of minimizing and maximizing. -/
theorem minimax
  {M : Type*} [NormedAddCommGroup M] [Module ℝ M] [ContinuousAdd M] [ContinuousSMul
  {N : Type*} [NormedAddCommGroup N] [Module ℝ N] [ContinuousAdd N] [ContinuousSMul
  (f : N →L[ℝ] M →L[ℝ] ℝ)
  (S : Set M) (T : Set N) (hS1 : IsCompact S) (hT1 : IsCompact T)
  (hS2 : Convex ℝ S) (hT2 : Convex ℝ T) (hS3 : S.Nonempty) (hT3 : T.Nonempty)
  : [] x : T, [] y : S, f x y = [] y : S, [] x : T, f x y := by
have := sion_minimax (f := (f · ·)) (S := T) (T := S) ?_ hT1 hT3 hS3 ?_ ?_ ?_ hS2
· exact this
· intro y t
  --Why doesn't fun_prop find this, even though this theorem is marked fun_prop? T
  apply LowerSemicontinuous.lowerSemicontinuousOn
  apply Continuous.lowerSemicontinuous
  fun_prop
· intro y t
  --Why doesn't fun_prop find this, even though this theorem is marked fun_prop? T
  apply UpperSemicontinuous.upperSemicontinuousOn
  apply Continuous.upperSemicontinuous
  fun_prop
· intro y hy
  exact LinearMap.quasiconvexOn (f := LinearMap.flip {
    toFun := fun x ↦ (f x).toLinearMap, map_add' := by simp, map_smul' := by sim

```

```

· intro x hx
  apply LinearMap.quasiconcaveOn _ hS₂
· rw [← Set.image_prod]
  apply (hT₁.prod hS₁).bddAbove_image
  apply Continuous.continuousOn
  apply continuous_stupid
· rw [← Set.image_prod]
  apply (hT₁.prod hS₁).bddBelow_image
  apply Continuous.continuousOn
  apply continuous_stupid

/-- **Von-Neumann's Minimax Theorem**, specialized to bilinear forms. -
/
theorem LinearMap.BilinForm.minimax
  {M : Type*} [NormedAddCommGroup M] [Module ℝ M] [ContinuousAdd M] [Contin
  (f : LinearMap.BilinForm ℝ M)
  (S : Set M) (T : Set M) (hS₁ : IsCompact S) (hT₁ : IsCompact T)
  (hS₂ : Convex ℝ S) (hT₂ : Convex ℝ T) (hS₃ : S.Nonempty) (hT₃ : T.Nonempt
    : ∅ x : T, ∅ y : S, f x y = ∅ y : S, ∅ x : T, f x y :=
  _root_.minimax (LinearMap.toContinuousLinearMap {
    toFun := fun x ↦ (f x).toContinuousLinearMap, map_add' := by simp, map_
    S T hS₁ hT₁ hS₂ hT₂ hS₃ hT₃

/-- Convenience form of `LinearMap.BilinForm.minimax` with the order inf/su
/
theorem LinearMap.BilinForm.minimax'
  {M : Type*} [NormedAddCommGroup M] [Module ℝ M] [ContinuousAdd M] [Contin
  (f : LinearMap.BilinForm ℝ M)
  (S : Set M) (T : Set M) (hS₁ : IsCompact S) (hT₁ : IsCompact T)
  (hS₂ : Convex ℝ S) (hT₂ : Convex ℝ T) (hS₃ : S.Nonempty) (hT₃ : T.Nonempt
    : ∅ x : S, ∅ y : T, f x y = ∅ y : T, ∅ x : S, f x y := by
  symm; exact minimax f.flip S T hS₁ hT₁ hS₂ hT₂ hS₃ hT₃

```

1.479 QuantumInfo/ForMathlib/Misc.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Order.CompletePartialOrder

--Can this be rewritten more generally? For `finiteness` to work, I don't k

```



```

--PR'ed in #33105
@[aesop (rule_sets := [finiteness]) unsafe apply]
theorem ite_eq_top {α : Type*} [Top α] (h : Prop) [Decidable h] {x y : α} (hx : x ≠
  (if h then x else y) ≠ ⊤ := by
  split <;> assumption

section subtype_val_iSup

/-
When
https://leanprover.zulipchat.com/#narrow/channel/287929-mathlib4/topic/diamond.20in.
is fixed, the declarations below should be changed to
```
theorem subtype_val_iSup {ι α : Type*} [ConditionallyCompleteLattice α] {s : Set α}
 [Inhabited ιs] [s.OrdConnected] (h : ∀ i, f i ∈ s) :
 (⊓ i, ((f i, h i) : ιs)).val = ⊓ i, f i := by
 sorry

theorem subtype_val_iSup' {ι α : Type*} [ConditionallyCompleteLattice α] {s : Set α}
 [Inhabited ιs] [s.OrdConnected] (h : ∀ i, f i ∈ s) :
 ⊓ i, ((f i, h i) : ιs) = (⊓ i, f i, by sorry) := by
 rw [Subtype.eq_iff, subtype_val_iSup]
```

Sadly, though, there's a "diamond" and we need it with the other data (the one we sp
below).
-/
variable {ι α : Type*} [ι : Nonempty ι] [ConditionallyCompleteLattice α]
  {f : ι → α} {a b : α} [Fact (a ≤ b)]

/- This isn't marked as `simp` because rewriting from a sup over a `CompleteLattice`
`ConditionallyCompleteLattice` would, pretty often, be undesirable. -
/
theorem subtype_val_iSup (h : ∀ i, f i ∈ Set.Icc a b) :
  (⊓ i, ((f i, h i) : ↑(Set.Icc a b))).val = ⊓ i, f i := by
  simp only [iSup, sSup, Set.range_eq_empty_iff, not_isEmpty_of_nonempty, reduceDite
  congr 1; ext1
  simp

theorem subtype_val_iSup' (h : ∀ i, f i ∈ Set.Icc a b) :
  ⊓ i, ((f i, h i) : ↑(Set.Icc a b)) =
  (⊓ i, f i, ((h i.some).1.trans (le_ciSup ⟨b, by intro; grind⟩ _), ciSup_le (h
  rw [Subtype.ext_iff, subtype_val_iSup]

/- This isn't marked as `simp` because rewriting from a sup over a `CompleteLattice`
`ConditionallyCompleteLattice` would, pretty often, be undesirable. -
/

```

```

theorem subtype_val_iInf (h :  $\forall i, f i \in \text{Set.Icc } a \ b$ ) :
  ( $\prod i, (\langle f i, h i \rangle : \uparrow(\text{Set.Icc } a \ b))$ ).val =  $\prod i, f i$  := by
  simp only [iInf, sInf, Set.range_eq_empty_iff, not_isEmpty_of_nonempty, r
  congr 1; ext1
  simp

theorem subtype_val_iInf' (h :  $\forall i, f i \in \text{Set.Icc } a \ b$ ) :
   $\prod i, (\langle f i, h i \rangle : \uparrow(\text{Set.Icc } a \ b)) =$ 
    ( $\prod i, f i, \langle \text{le\_ciInf } (h \cdot |>.1), (\text{ciInf\_le } \langle a, \text{by intro; grind} \rangle \_) \rangle$ ).tra
  rw [Subtype.ext_iff, subtype_val_iInf]

end subtype_val_iSup

open scoped ENNReal Topology in
/-- Analogous to `bdd_le_mul_tendsto_zero`, for `ENNReal` (which otherwise
multiplication function). The product of a sequence that tends to zero with
also tends to zero. -/
protected lemma ENNReal.bdd_le_mul_tendsto_zero
  { $\alpha$  : Type*} {l : Filter  $\alpha$ } {f g :  $\alpha \rightarrow \mathbb{R}_{\geq 0\infty}$ } {b :  $\mathbb{R}_{\geq 0\infty}$ } (hb : b  $\neq \top$ )
  (hf : l.Tendsto f ( $\prod 0$ )) (hg :  $\forall^f (x : \alpha) \text{ in } l, g x \leq b$ ) :
    l.Tendsto (fun x  $\mapsto f x * g x$ ) ( $\prod 0$ ) := by
  rw [ENNReal.tendsto_nhds_zero] at hf  $\vdash$ 
  intro  $\varepsilon$  h $\varepsilon$ pos
  by_cases hb_pos : 0 < b
  · filter_upwards [hf ( $\varepsilon / b$ ) (by simp [hb, h $\varepsilon$ pos.ne']), hg] with x hx1 hx2
    grw [hx1, hx2, ENNReal.div_mul_cancel hb_pos.ne' hb]
  · filter_upwards [hg] with x hx
    grind [not_lt, nonpos_iff_eq_zero, mul_zero, zero_le]

--PULLOUT: Belongs in Mathlib/Algebra/Order/Group/Pointwise/CompleteLattice
-- (after appropriately generalizing to MulPosMono)
open scoped Pointwise in
theorem csInf_mul_nonneg {s t : Set  $\mathbb{R}$ }
  (hs0 : s.Nonempty) (hs1 :  $\forall x \in s, 0 \leq x$ ) (ht0 : t.Nonempty) (ht1 :  $\forall x \in t, 0 \leq x$ ) :
    sInf (s * t) = sInf s * sInf t := by
  apply le_antisymm
  · set a := sInf s
    set b := sInf t
    have h $\varepsilon$ eps :  $\forall \varepsilon > 0, \exists x \in s, x < a + \varepsilon \wedge \exists y \in t, y < b + \varepsilon$  := by
      intro  $\varepsilon$   $\varepsilon$ _pos
      obtain ⟨x, hx1, hx2⟩ := exists_lt_of_csInf_lt hs0 (lt_add_of_pos_right  $\varepsilon$  0)
      obtain ⟨y, hy1, hy2⟩ := exists_lt_of_csInf_lt ht0 (lt_add_of_pos_right  $\varepsilon$  0)
      exact ⟨x, hx1, hx2, y, hy1, hy2⟩
    have h_prod $\varepsilon$ eps :  $\forall \varepsilon > 0, \exists x \in s, \exists y \in t, x * y < (a + \varepsilon) * (b + \varepsilon)$  := by
      intro  $\varepsilon$  h $\varepsilon$ 
      obtain ⟨x, hx1, hx2, y, hy1, hy2⟩ := h $\varepsilon$ eps  $\varepsilon$  h $\varepsilon$ 

```

```

    exact ⟨x, hx₁, y, hy₁, by nlinarith [hs₁ x hx₁, ht₁ y hy₁]⟩
  have h_lim : Filter.Tendsto (fun ε => (a + ε) * (b + ε)) (nhdsWithin 0 (Set.Ioi 0))
    exact tendsto_nhdsWithin_of_tendsto_nhds (Continuous.tendsto' (by continuity))
  apply le_of_tendsto_of_tendsto tendsto_const_nhds h_lim
  filter_upwards [self_mem_nhdsWithin] with ε hε
  specialize h_prod_eps ε hε
  choose x hx y hy using h_prod_eps
  refine le_trans ?_ hy.right.le
  refine csInf_le ⟨0, ?_⟩ ?_
  · rintro x ⟨u, hu, v, hv, rfl⟩
    exact mul_nonneg (hs₁ u hu) (ht₁ v hv)
  · exact ⟨_, hx, _, hy.left, rfl⟩
  · apply le_csInf (hs₀.mul ht₀)
    rintro _ ⟨x, hx, y, hy, rfl⟩
    apply mul_le_mul
    · exact csInf_le ⟨0, hs₁⟩ hx
    · exact csInf_le ⟨0, ht₁⟩ hy
    · exact Real.sInf_nonneg ht₁
    · exact hs₁ x hx

/--
If two functions from finite types have the same multiset of values, there exists a permutation
-/
lemma Multiset.map_univ_eq_iff {α β : Type*} [Fintype α] (f g : α → β) :
  Multiset.map f Finset.univ.val = Multiset.map g Finset.univ.val ↔ ∃ (e : α ≈ α),
  apply Iff.intro
  · intro a
    classical
    -- Since these two multisets are equal, their elements must be equal up to permutation
    have h_perm : ∃ e : α ≈ α, ∀ x, f x = g (e x) := by
      have h_count_eq : ∀ y : β, Finset.card (Finset.filter (fun x => f x = y) Finset.univ) =
        Finset.card (Finset.filter (fun x => g x = y) Finset.univ)
      intro y
      replace a := congr_arg (fun m => m.count y) a;
      simp_all (config := { decide := Bool.true }) [Multiset.count_map];
      simpa [eq_comm, Finset.filter_congr] using a;
      have h_perm : ∀ y : β, ∃ e : { x : α // f x = y } ≈ { x : α // g x = y }, True
      intro y
      simp_all only [exists_const_iff, and_true]
      exact ⟨ Fintype.equivOfCardEq <| by simpa [ Fintype.card_subtype ] using h_count_eq >|,
        choose e he using h_perm;
      refine' ⟨ _, _ ⟩;
      exact (Equiv.sigmaFiberEquiv f).symm.trans (Equiv.sigmaCongrRight e) |> Equiv.trans
    intro x
    specialize e (f x)
    rename_i e_1
    simp_all only [implies_true, Equiv.trans_apply, Equiv.sigmaCongrRight_apply,

```

```

      Equiv.sigmaFiberEquiv_symm_apply_fst, Equiv.sigmaFiberEquiv_apply]
    exact Eq.symm ( e_1 ( f x ) ( x, rfl ) |>.2 );
  exact ( h_perm.choose, funext h_perm.choose_spec );
· intro a
  obtain ⟨w, h⟩ := a
  subst h
  simp_all only [Function.comp_apply, Finset.univ]
  -- Since $w$ is a bijection, the multiset of $w(x)$ for $x$ in the orig
  have h_perm : Multiset.map (fun x => w x) (Finset.val Fintype.elems) =
    exact Multiset.map_univ_val_equiv w;
  conv_rhs => rw [ ← h_perm ];
  simp +zetaDelta at *

/--
If two functions from finite types have the same multiset of values, there
-/
lemma exists_equiv_of_multiset_map_eq {α β γ : Type*} [Fintype α] [Fintype
  (f : α → γ) (g : β → γ) (h : Multiset.map f Finset.univ.val = Multiset.
  ∃ e : α ≈ β, f = g ∘ e := by
  -- Since the multisets of values are equal, the cardinalities of the doma
  obtain ⟨σ, hσ⟩ : ∃ σ : α ≈ β, Multiset.map f Finset.univ.val = Multiset.m
  have h_card : Fintype.card α = Fintype.card β := by
    simp using congr_arg Multiset.card h;
  obtain σ := Fintype.equivOfCardEq h_card
  use σ
  have h_multiset_eq : Multiset.map g Finset.univ.val = Multiset.map (g ∘
    rw [ ← Multiset.map_univ_val_equiv σ ] ;
    rw [ Multiset.map_map ]
  exact h.trans h_multiset_eq;
  -- By `Multiset.map_univ_eq_iff`, there exists `e' : α ≈ α` such that `f
  obtain ⟨e', he'⟩ : ∃ e' : α ≈ α, f = (g ∘ σ) ∘ e' := by
    exact (Multiset.map_univ_eq_iff f (g ∘ σ)).mp hσ;
  exact ( e'.trans σ, by simp [ Function.comp ] using he' )

```

1.480 QuantumInfo/ForMathlib/SionMinimax.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Algebra.Order.Module.Field
import Mathlib.Analysis.Convex.PathConnected
import Mathlib.Analysis.Convex.Quasiconvex

```

```

import Mathlib.Analysis.Convex.Topology
import Mathlib.Analysis.Normed.Order.Lattice
import Mathlib.Analysis.SpecificLimits.Basic
import Mathlib.Data.EReal.Operations
import Mathlib.Data.Fintype.Order
import Mathlib.Topology.Algebra.InfiniteSum.Order
import Mathlib.Topology.MetricSpace.Bounded

@[simp]
theorem Set.image2_flip {α β γ : Type*} {f : α → β → γ} (s : Set α) (t : Set β) :
  image2 (flip f) t s = image2 f s t := by
  grind [image2, flip]

section ciSup

variable {ι α : Type*} [ConditionallyCompleteLattice α] {f g : ι → α} {a : α}

/-- The max-min theorem. A version of `iSup_iInf_le_iInf_iSup` for conditionally
/
theorem ciSup_ciInf_le_ciInf_ciSup {ι' : Type*} [Nonempty ι]
  (f : ι → ι' → α) (Ha : ∀ j, BddAbove (Set.range (f · j))) (Hb : ∀ i, BddBelow (Set
    ⋂ i, ⋂ j, f i j ≤ ⋂ j, ⋂ i, f i j :=
  ciSup_le fun i ↦ ciInf_mono (Hb i) fun j ↦ le_ciSup (Ha j) i

theorem BddAbove.range_max (hf : BddAbove (Set.range f)) (hg : BddAbove (Set.range g)
  BddAbove (Set.range (max f g)) := by
  rcases hf with ⟨a, ha⟩
  rcases hg with ⟨b, hb⟩
  use a ⋂ b
  simp only [mem_upperBounds, Set.mem_range, forall_exists_index, forall_apply_eq_im
  intro i
  specialize ha i
  specialize hb i
  order

theorem BddBelow.range_min (hf : BddBelow (Set.range f)) (hg : BddBelow (Set.range g)
  BddBelow (Set.range (min f g)) :=
  BddAbove.range_max (α := αo d) hf hg

theorem ciInf_eq_min_ciInf_inter_diff (S T : Set ι)
  [Nonempty (S ∩ T : Set ι)] [Nonempty (S \ T : Set ι)] (hf : BddBelow (f '' S)) :
  ⋂ i : S, f i = (⋂ i : (S ∩ T : Set ι), f i) ⋂ ⋂ i : (S \ T : Set ι), f i := by
  apply le_antisymm
  · rw [le_inf_iff]
    constructor
    <;> (

```

```

    apply le_ciInf
    simp only [Subtype.forall, Set.mem_diff, Set.mem_inter_iff, and_imp]
    refine fun a ha hb ↦ ciInf_le ?_ ((a, ha) : S)
    simp [Set.range] using hf;
  )
  · have _ : Nonempty S := .map (fun (x : (S n T : Set _)) ↦ (x, x.2.1)) <_
    apply le_csInf (Set.range_nonempty _)
    rintro i ((b, hb), rfl)
    by_cases hiT : b ∈ T
    · exact inf_le_left.trans (csInf_le (hf.mono (by grind))) (by aesop))
    · exact inf_le_right.trans (csInf_le (hf.mono (by grind))) (by aesop))

variable [Nonempty ι]

theorem lt_ciInf_iff (hf : BddBelow (Set.range f)) :
  a < iInf f ↔ ∃ b, a < b ∧ ∀ (i : ι), b ≤ f i :=
  (((iInf f, ·, (ciInf_le hf ·))), fun (_, hb₁, hb₂) ↦ lt_of_lt_of_le hb₁ (

theorem ciSup_sup_eq (hf : BddAbove (Set.range f)) (hg : BddAbove (Set.rang
  le_antisymm (ciSup_le fun _ => sup_le_sup (le_ciSup hf _) <| le_ciSup hg
    (sup_le (ciSup_mono (hf.range_max hg) fun _ => le_sup_left) <| ciSup_mo

theorem ciInf_inf_eq (hf : BddBelow (Set.range f)) (hg : BddBelow (Set.rang
  ciSup_sup_eq (α := αo d) hf hg

theorem sup_ciSup (hf : BddAbove (Set.range f)) : a ⊔ ⊔ x, f x = ⊔ x, a ⊔ f
  rw [ciSup_sup_eq (by simp) hf, ciSup_const]

theorem inf_ciInf (hf : BddBelow (Set.range f)) : a ⊓ ⊓ x, f x = ⊓ x, a ⊓ f
  sup_ciSup (α := αo d) hf

theorem ciInf_sup_ciInf_le (hf : BddBelow (Set.range f)) (hg : BddBelow (Se
  (⊓ i, f i) ⊓ ⊓ i, g i ≤ ⊓ i, f i ⊓ g i :=
  le_ciInf (fun i ↦ sup_le_sup (ciInf_le hf i) (ciInf_le hg i))

theorem le_ciSup_inf_ciSup (hf : BddAbove (Set.range f)) (hg : BddAbove (Se
  ⊓ (i : ι), f i ⊓ g i ≤ (⊓ (i : ι), f i) ⊓ ⊓ (i : ι), g i :=
  ciInf_sup_ciInf_le (α := αo d) hf hg

end ciSup

theorem Quasiconvex0n.subset
  {α E β : Type*} [Semiring α] [PartialOrder β] [AddCommMonoid E] [LE β] [S
  {s : Set E} {f : E → β} (h : Quasiconvex0n α s f) {t : Set E} (hts : t ⊆
    Quasiconvex0n α t f := by
  intro b

```

```

convert ht.inter (h b) using 1
simp +contextual [Set.ext_iff, @hts _]

theorem Quasiconvex0n.mem_segment_le_max
  {α E β : Type*} [Semiring α] [PartialOrder β] [AddCommMonoid E] [SemilatticeSup β]
  {s : Set E} {f : E → β} (h : Quasiconvex0n α s f)
  {x y z : E} (hx : x ∈ s) (hy : y ∈ s) (hz : z ∈ segment α x y):
  f z ≤ f x ⊔ f y :=
  ((h (f x ⊔ f y)).segment_subset (by simpα) (by simpα) hz).right

theorem Quasiconcave0n.min_le_mem_segment
  {α E β : Type*} [Semiring α] [PartialOrder β] [AddCommMonoid E] [SemilatticeInf β]
  {s : Set E} {f : E → β} (h : Quasiconcave0n α s f)
  {x y z : E} (hx : x ∈ s) (hy : y ∈ s) (hz : z ∈ segment α x y):
  f x ⊔ f y ≤ f z :=
  ((h (f x ⊔ f y)).segment_subset (by simpα) (by simpα) hz).right

theorem LowerSemicontinuous0n.bddBelow {α : Type*} [TopologicalSpace α] {S : Set α}
  (hg : LowerSemicontinuous0n g S) (hS : IsCompact S) : BddBelow (g '' S) := by
  rcases S.eq_empty_or_nonempty with rfl | hS₂
  · simp
  have h_neighborhood : ∀ x ∈ S, ∃ U ∈ nhds x, ∀ y ∈ U ∩ S, g y > g x - 1 := by
    -- By definition of lower semicontinuity, for each x ∈ S, there exists a neighborhood
    intros x hx
    specialize hg x hx (g x - 1) (sub_one_lt (g x))
    rw [eventually_nhdsWithin_iff] at hg
    simp only [Set.mem_inter_iff, gt_iff_lt, and_imp]
    exact ⟨_, hg, fun y hy hyS ↦ hy hyS⟩
  choose! U hU using h_neighborhood
  -- By the finite subcover property, there exists a finite subset t ⊆ S$ such that
  obtain ⟨t, ht⟩ : ∃ t, (∀ x ∈ t, x ∈ S) ∧ S ⊆ ⋃ x ∈ t, U x :=
    hS.elim_nhds_subcover U fun x hx ↦ hU x hx |>.1;
  -- Let $m$ be the minimum value of $g$ on the finite set $t$.
  obtain ⟨m, hm⟩ : ∃ m, ∀ x ∈ t, g x ≥ m := by
    use Finset.min' (t.image g) ?_
    · exact fun x hx ↦ Finset.min'_le _ _ (t.mem_image_of_mem g hx)
    refine ⟨_, Finset.mem_image_of_mem g <| Classical.choose_spec <| Finset.nonempty
    rintro rfl
    simpα using ht.2 hS₂.some_mem
  use m - 1
  refine Set.forall_mem_image.mpr fun x hx ↦ ?_
  rcases Set.mem_iUnion₂.mp (ht.2 hx) with ⟨y, hy, hy'⟩
  linarith [hm y hy, hU y (ht.1 y hy) |>.2 x ⟨hy', hx⟩]

theorem LowerSemicontinuous0n.max {α : Type*} [TopologicalSpace α] {S : Set α} {f g
  (hf : LowerSemicontinuous0n f S) (hg : LowerSemicontinuous0n g S) :

```

```

LowerSemicontinuousOn (fun x ↦ max (f x) (g x)) S := by
  convert lowerSemicontinuousOn_ciSup (s := S) (f := fun (i : Bool) x' ↦ if
    · rw [ciSup_eq_of_forall_le_of_forall_lt_exists_gt] <=> aesop
    · simp
    · simp [hf, hg]

variable {α : Type*} [TopologicalSpace α] {β : Type*} [Preorder β] {f g : α → β}
  {s t : Set α} {y z : β} {γ : Type*} [LinearOrder γ]

theorem lowerSemicontinuousOn_iff_isClosed_preimage {f : α → γ} [IsClosed s]
  LowerSemicontinuousOn f s ↔ ∀ y, IsClosed (s ∩ f-1 Set.Iic y) := by
  constructor
  · intro a y
    refine' isClosed_of_closure_subset fun x hx => _;
    rw [mem_closure_iff] at hx;
    contrapose! hx;
    rcases em (x ∈ s) with hx' | hx'
    · simp only [Set.mem_inter_iff, hx', Set.mem_preimage, Set.mem_Iic, true_and]
      have := a x hx';
      rcases mem_nhdsWithin.1 (this y hx) with (o, ho, h);
      exact (o, ho, h.1, Set.eq_empty_iff_forall_notMem.2 fun z hz => h.2)
    · exact (sc, IsClosed.isOpen_compl, hx', by aesop);
  · intro a x hx y hy
    have hx_not_in : x ∉ s ∩ f-1 Set.Iic y := by
      simp [hy]
    rw [eventually_nhdsWithin_iff]
    filter_upwards [IsOpen.mem_nhds (isOpen_compl_iff.2 (a y)) hx_not_in]
    exact lt_of_not_ge fun h => hz ⟨hzs, h⟩

theorem segment.isConnected {E : Type u_1} [AddCommGroup E] [Module ℝ E] [TopologicalSpace E]
  IsConnected (segment ℝ a b) := by
  rw [← Path.range_segment a b]
  exact isConnected_range (Path.segment a b).continuous

theorem BddAbove.range_inf_of_image2 {M N α : Type*} {f : M → N → α} [ConditionallyCompleteLattice α]
  {S : Set M} {T : Set N} (h_bddA : BddAbove (Set.image2 f S T)) (h_bddB : BddAbove (Set.range fun y : T ↦ ⨅ x : S, f x y)) := by
  rcases isEmpty_or_nonempty T with hT | hT
  · aesop
  rcases S.eq_empty_or_nonempty with rfl | hS
  · simp [Set.range, iInf]
  rw [← Set.nonempty_coe_sort, nonempty_subtype] at hS
  rcases hS with ⟨x, hx⟩
  choose z hz using h_bddA
  choose w hw using h_bddB
  have h_inf_le_M : ∀ y ∈ T, ⨅ x : S, f x y ≤ z := by

```



```

    intro y hy
    apply (ciInf_le (_, Set.forall_mem_range.2 (hw <| Set.mem_image2_of_mem .2 hy))
    exact hz (Set.mem_image2_of_mem hx hy)
    exact (_, Set.forall_mem_range.2 (h_inf_le_M _ .2))

theorem BddBelow.range_sup_of_image2 {M N  $\alpha$  : Type*} {f : M  $\rightarrow$  N  $\rightarrow$   $\alpha$ } [ConditionallyCompleteLattice  $\alpha$ ]
{S : Set M} {T : Set N} (h_bddA : BddAbove (Set.image2 f S T)) (h_bddB : BddBelow
  BddBelow (Set.range fun y : T  $\mapsto$   $\bigsqcup$  x : S, f x y) :=
  BddAbove.range_inf_of_image2 ( $\alpha$  :=  $\alpha^{\text{ord}}$ ) h_bddB h_bddA

theorem ciInf_le_ciInf_of_subset { $\alpha$   $\beta$  : Type*} [ConditionallyCompleteLattice  $\alpha$ ]
{f :  $\beta$   $\rightarrow$   $\alpha$ } {s t : Set  $\beta$ } (hs : s.Nonempty) (hf : BddBelow (f '' t)) (hst : s  $\subseteq$  t)
   $\bigsqcup$  x : t, f x  $\leq$   $\bigsqcup$  x : s, f x := by
  have h_lower_bound :  $\forall$  y  $\in$  s,  $\bigsqcup$  x : t, f x  $\leq$  f y := by
    intro y hy
    obtain <w_1, <left_1, right_1>> : f y  $\in$  f '' t := by
      use y, hst hy
    exact le_trans (ciInf_le (by simp [Set.range] using hf) <w_1, left_1>) (
    apply le_csInf
    · exact (<_, <(<_, hs.choose_spec>), rfl));
    · aesop

theorem LowerSemicontinuousOn.dite_top { $\alpha$   $\beta$  : Type*} [TopologicalSpace  $\alpha$ ] [Preorder
  {s : Set  $\alpha$ } (p :  $\alpha$   $\rightarrow$  Prop) [DecidablePred p] {f : ( $a$  :  $\alpha$ )  $\rightarrow$  p  $a$   $\rightarrow$   $\beta$ }
  (hf : LowerSemicontinuousOn (fun x : Subtype p  $\mapsto$  f x.val x.prop) {x | x.val  $\in$  s})
  (h_relatively_closed :  $\exists$  U : Set  $\alpha$ , IsClosed U  $\wedge$  s  $\cap$  U = s  $\cap$  setOf p) :
  LowerSemicontinuousOn (fun x  $\mapsto$  dite (p x) (f x) (fun _  $\mapsto$   $\top$ )) s := by
  rcases h_relatively_closed with <u, <hu, hsu>>
  simp only [Set.ext_iff, Set.mem_inter_iff, Set.mem_setOf_eq, and_congr_right_iff]
  intro x hx y hy
  dsimp at hy
  split_ifs at hy with h
  · specialize hf <x, h> hx y hy
    rw [eventually_nhdsWithin_iff] at hf  $\vdash$ 
    rw [nhds_subtype_eq_comap, Filter.eventually_comap] at hf;
    filter_upwards [hf]
    simp only [Subtype.forall]
    grind [lt_top_of_lt]
  · filter_upwards [self_mem_nhdsWithin, mem_nhdsWithin_of_mem_nhds (hu.isOpen_compl
    intros
    simp_all only [Set.mem_compl_iff,  $\downarrow$ reduceDite]

theorem LowerSemicontinuousOn.comp_continuousOn { $\alpha$   $\beta$   $\gamma$  : Type*}
  [TopologicalSpace  $\alpha$ ] [TopologicalSpace  $\beta$ ] [Preorder  $\gamma$ ] {f :  $\alpha$   $\rightarrow$   $\beta$ } {s : Set  $\alpha$ } {g
  (hg : LowerSemicontinuousOn g t) (hf : ContinuousOn f s) (h : Set.MapsTo f s t) :
  LowerSemicontinuousOn (g  $\circ$  f) s := by

```

```

intro x hx y hy
have h_neighborhood :  $\exists U \in \text{nhds } (f \ x), \forall z \in U \cap t, y < g \ z :=$  by
  have := hg ( f x ) ( h hx ) y hy
  simp_all only [Function.comp_apply, Set.mem_inter_iff, and_imp]
  rw [eventually_nhdsWithin_iff] at this
  aesop
obtain ⟨U, hU_nhds, hU⟩ :  $\exists U \in \text{nhds } (f \ x), \forall z \in U \cap t, y < g \ z :=$  h_nei
have h_final :  $\forall^f x' \text{ in nhdsWithin } x \ s, f \ x' \in U \wedge f \ x' \in t :=$  by
  filter_upwards [ hf x hx hU_nhds, self_mem_nhdsWithin ] with x' hx' hx'
exact h_final.mono fun x' hx' => hU _ hx'

theorem UpperSemicontinuousOn.comp_continuousOn {α β γ : Type*}
  [TopologicalSpace α] [TopologicalSpace β] [Preorder γ] {f : α → β} {s : Set α}
  (hg : UpperSemicontinuousOn g t) (hf : ContinuousOn f s) (h : Set.Mapsto
    UpperSemicontinuousOn (g ∘ f) s :=
  LowerSemicontinuousOn.comp_continuousOn (γ := γo d) hg hf h

theorem LowerSemicontinuousOn.ite_top {α β : Type*} [TopologicalSpace α] [Preorder β]
  {s : Set α} (p : α → Prop) [DecidablePred p] {f : (a : α) → β} (hf : LowerSemicontinuousOn f s)
  (h_relatively_closed :  $\exists U : \text{Set } \alpha, \text{IsClosed } U \wedge s \cap U = s \cap \text{setOf } p$ ) :
  LowerSemicontinuousOn (fun x ↦ ite (p x) (f x) ⊤) s :=
  dite_top p (hf.comp_continuousOn (by fun_prop) (by intro; simp)) h_relatively_closed

theorem LeftOrdContinuous.comp_lowerSemicontinuousOn_strong_assumptions {α β γ δ : Type*}
  [TopologicalSpace α] [LinearOrder γ] [LinearOrder δ] [TopologicalSpace β] [TopologicalSpace δ]
  [TopologicalSpace γ] [OrderTopology γ] [DenselyOrdered γ] [DenselyOrdered δ]
  {s : Set α} {g : γ → δ} {f : α → γ} (hg : LeftOrdContinuous g) (hf : LowerSemicontinuousOn f s) :
  LowerSemicontinuousOn (g ∘ f) s := by
  intros x hx y hy
  have hU :  $\exists U \in \text{nhds } x, \forall z \in U \cap s, g \ (f \ z) > y :=$  by
    simp_all only [Set.mem_inter_iff, gt_iff_lt, and_imp]
  obtain ⟨z, hz1, hz2⟩ :  $\exists z, y < g \ z \wedge z < f \ x :=$  by
    obtain ⟨z, hz⟩ :  $\exists z, z < f \ x \wedge y < g \ z :=$  by
      by_contra h_contra
      simp only [not_exists, not_and, not_lt] at h_contra
      have h_exists_z :  $\exists z, z < f \ x \wedge g \ z > y :=$  by
        have h_lub : IsLUB (Set.Iio (f x)) (f x) := by
          exact isLUB_Iio
        have := hg h_lub;
        have := this.exists_between hy
        simp_all only [Set.mem_image, Set.mem_Iio, exists_exists_and_eq_ae]
        grind
      exact h_exists_z.choose_spec.2.not_ge ( h_contra _ h_exists_z.choose_spec.1 )
  aesop
specialize hf x hx z hz2
rw [eventually_nhdsWithin_iff] at hf

```

```

    exact (_, hf, fun _ hx hx' ↦ hz1.trans_le (hg2 (hx hx')).le))
  simp only [Set.mem_inter_iff, and_imp] at hU
  obtain ⟨w, ⟨left, right⟩⟩ := hU
  exact Filter.eventually_inf_principal.2 (Filter.mem_of_superset left right)

theorem UpperSemicontinuousOn.frequently_lt_of_tendsto {α β γ : Type*} [TopologicalS
  {f : β → γ} {T : Set β} (hf : UpperSemicontinuousOn f T) {c : γ} {zs : α → β} {z :
  {l : Filter α} [l.NeBot] (hzs : l.Tendsto zs (nhds z)) (hx₂ : f z < c) (hzI : ∀ a,
    ∀f a in l, f (zs a) < c := by
  have h : ∀f n in l, zs n ∈ {y ∈ T | f y < c} := by
    filter_upwards [hzs.eventually (eventually_nhdsWithin_iff.mp ((hf z hzT) c hx₂))]
      using ⟨hzI n, hn (hzI n)⟩
  simp_all

theorem Finset.ciInf_insert {α β : Type*} [DecidableEq α] [ConditionallyCompleteLatt
  (t : Finset α) (ht : t.Nonempty) (x : α) (f : α → β) :
  □ (a : (insert x t : _)), f a = f x □ □ (a : t), f a := by
  apply le_antisymm
  · apply le_inf
    · apply csInf_le <=> aesop
    apply le_csInf
    · exact (_, ⟨(ht.choose, ht.choose_spec), rfl⟩)
    rintro _ ⟨(a, ha), rfl⟩
    exact csInf_le (Finite.bddBelow_range _) ⟨(a, Finset.mem_insert_of_mem ha), rfl⟩
  · apply le_csInf
    · exact (_, ⟨(x, Finset.mem_insert_self _ _), rfl⟩)
    simp only [Set.mem_range, Subtype.exists, mem_insert, exists_prop, exists_eq_or_
    rintro b (rfl | (_, (_, rfl)))
    · exact inf_le_left
    apply inf_le_right.trans
    apply csInf_le (Set.finite_range _).bddBelow
    aesop

theorem Finset.ciSup_insert {α β : Type*} [DecidableEq α] [ConditionallyCompleteLatt
  (t : Finset α) (ht : t.Nonempty) (x : α) (f : α → β) :
  □ (a : (insert x t : _)), f a = f x □ □ (a : t), f a :=
  t.ciInf_insert (β := βo d) ht x f

section sion_minimax
/-!
Following https://projecteuclid.org/journals/kodai-mathematical-journal/volume-11/issue-1/Elementary-proof-for-Sions-minimax-theorem/10.2996/kmj/1138038812.full, with some corrections. There are two errors in
` (∀ x, a < f x) → (a < □ x, f x) `. This is repaired by taking an extra `exists_between
` (∀ x, b < f x) → (b ≤ □ x, f x) ` and so ` (a < □ x, f x) `.
-/

```

```

variable {M : Type*} [NormedAddCommGroup M]
{N : Type*}
{f : M → N → ℝ} {S : Set M} {T : Set N}
(hfc₂ : ∀ y, y ∈ T → LowerSemicontinuousOn (f · y) S)
(hS₁ : IsCompact S) (hS₃ : S.Nonempty) (hT₃ : T.Nonempty)

include hfc₂ hS₁ hS₃ in
private theorem sion_exists_min_lowerSemi (a : ℝ) (hc : ∀ y₀ : T, ∃ (x : S)
  ∃ x ∈ S, f x z ≤ a := by
  let _ := hS₃.to_subtype
  contrapose! hc
  use {z, hzT}
  have h_lower_bound (x : S) : a < f x z := hc x x.2
  apply lt_of_le_of_ne
  · apply le_csInf (Set.range_nonempty _)
    rintro _ {x, rfl}
    exact (h_lower_bound x).le
  intro h
  obtain {x, hxz} : ∃ x : S, f x z ≤ a := by
    obtain {xn, hxn} : ∃ xn : ℕ → S, Filter.atTop.Tendsto (fun n => f (xn n) z)
    have h_eps : ∀ ε > 0, ∃ x : S, f x z < a + ε := by
      intro ε εpos
      have hx : ∃ x : S, f x z < a + ε := by linarith
      simpa using exists_lt_of_csInf_lt (Set.range_nonempty _) hx
    choose! xn hxn using h_eps
    use fun n ↦ xn (1 / (n + 1))
    rw [tendsto_iff_dist_tendsto_zero]
    refine squeeze_zero (fun _ ↦ abs_nonneg _) (fun n ↦ ?_) tendsto_one_d
    refine abs_le.mpr {?, ?} <=> linarith [h_lower_bound (xn (1 / (n + 1)))]
  obtain {x, subseq, hsubseq₁, hsubseq₂} : ∃ x : S, ∃ subseq : ℕ → ℕ,
    StrictMono subseq ∧ Filter.Tendsto (fun n => xn (subseq n)) Filter.atTop
    simpa using (isCompact_iff_isCompact_univ.mp hS₁).isSeqCompact (x := x)
  use x
  refine le_of_forall_pos_le_add fun ε εpos ↦ ?_
  rw [← tsub_le_iff_right]
  apply le_of_tendsto_of_tendsto tendsto_const_nhds (hxn.comp hsubseq₁.te)
  apply Filter.eventually_atTop.mpr
  rw [Metric.tendsto_atTop] at hsubseq₂
  have {δ, δpos, H} : ∃ δ > 0, ∀ y ∈ S, dist y x < δ → f y z > f x z - ε
    obtain {δ, hδ} := Metric.mem_nhdsWithin_iff.mp ((hfc₂ z hzT) x x.2)
    exact {δ, hδ.1, fun y hy hyx => hδ.2 {hyx, hy}}
  rcases hsubseq₂ δ δpos with {N, hN}
  exact {N, fun n hn ↦ (H _ (xn (subseq n)).coe_prop (hN _ hn)).le}
specialize h_lower_bound x
order

```

```

variable [Module  $\mathbb{R}$  M] [ContinuousSMul  $\mathbb{R}$  M]
variable [AddCommGroup N] [TopologicalSpace N] [SequentialSpace N] [T2Space N] [Cont
variable
  (hfc1 :  $\forall x, x \in S \rightarrow \text{UpperSemicontinuousOn } (f \ x) \ T$ )
  (hfc2 :  $\forall y, y \in T \rightarrow \text{QuasiconvexOn } \mathbb{R} \ S \ (f \cdot y)$ )
  (hfc1 :  $\forall x, x \in S \rightarrow \text{QuasiconcaveOn } \mathbb{R} \ T \ (f \ x)$ )
  (hT2 : Convex  $\mathbb{R} \ T$ ) (hS2 : Convex  $\mathbb{R} \ S$ )

include hfc1 hfc2 hS1 hT2 hS3 in
private lemma sion_exists_min_2 (y1 y2 : N) (hy1 : y1 ∈ T) (hy2 : y2 ∈ T)
  (a :  $\mathbb{R}$ ) (ha : a <  $\inf x : S, (\max (f \ x \ y_1) (f \ x \ y_2))$ )
  :  $\exists y_0 : T, a < \inf x : S, f \ x \ y_0 :=$  by
  by_contra! hc
  have _ := isCompact_iff_compactSpace.mp hS1
  have _ := hS1.isClosed
  obtain ⟨β, hβ1, hβ2⟩ := exists_between ha
  let C : N → Set M := fun z ↦ { x ∈ S | f x z ≤ a }
  let C' : N → Set M := fun z ↦ { x ∈ S | f x z ≤ β }
  let A := C' y1
  let B := C' y2
  have hC_subset_C' (z) : C z ⊆ C' z :=
    fun x hx ↦ ⟨hx.1, hx.2.trans hβ1.le⟩
  have hC_nonempty (z) (hz : z ∈ segment  $\mathbb{R} \ y_1 \ y_2$ ) : (C z).Nonempty := by
    simp only [Set.Nonempty, Set.mem_setOf_eq, C]
    exact sion_exists_min_lowerSemi hfc2 hS1 hS3 a hc z (hT2.segment_subset hy1 hy2)
  have hC_closed (z) (hz : z ∈ segment  $\mathbb{R} \ y_1 \ y_2$ ) : IsClosed (C z) := by
    specialize hfc2 z (hT2.segment_subset hy1 hy2 hz)
    rw [lowerSemicontinuousOn_iff_isClosed_preimage] at hfc2
    exact hfc2 a
  have hC'_nonempty (z) (hz : z ∈ segment  $\mathbb{R} \ y_1 \ y_2$ ) : (C' z).Nonempty :=
    (hC_nonempty z hz).mono (hC_subset_C' z)
  have hC'_closed (z) (hz : z ∈ segment  $\mathbb{R} \ y_1 \ y_2$ ) : IsClosed (C' z) := by
    specialize hfc2 z (hT2.segment_subset hy1 hy2 hz)
    rw [lowerSemicontinuousOn_iff_isClosed_preimage] at hfc2
    exact hfc2 β
  have hA_nonempty : A.Nonempty := hC'_nonempty y1 (left_mem_segment  $\mathbb{R} \ y_1 \ y_2$ )
  have hA_closed : IsClosed A := hC'_closed y1 (left_mem_segment  $\mathbb{R} \ y_1 \ y_2$ )
  have hB_nonempty : B.Nonempty := hC'_nonempty y2 (right_mem_segment  $\mathbb{R} \ y_1 \ y_2$ )
  have hB_closed : IsClosed B := hC'_closed y2 (right_mem_segment  $\mathbb{R} \ y_1 \ y_2$ )
  have hAB : A ∩ B = ∅ := by
    simp [A, B, C', Set.ext_iff]
    intro x hx hβy1 _
    by_contra! hβy2
    have hβ := max_le hβy1 hβy2
    revert hβ

```

```

rw [imp_false, not_le]
refine lt_of_lt_of_le hβ₂ ?_
have h := ((hfc₂ y₁ hy₁).max (hfc₂ y₂ hy₂)).bddBelow hS₁
rw [Set.image_eq_range] at h
exact ciInf_le h ⟨x, hx⟩
have hfxz (x) (hx : x ∈ S) (z) (hz : z ∈ segment ℝ y₁ y₂) : min (f x y₁)
(hfq₁ x hx).min_le_mem_segment hy₁ hy₂ hz
have hC'zAB (z) (hz : z ∈ segment ℝ y₁ y₂) : C' z ⊆ A ∪ B := by
--TODO: On newer Mathlib this is just `grind [inf_le_iff, le_trans]`
intro; simp [C', A, B]; grind [inf_le_iff, le_trans]
have hC'z (z) (hz : z ∈ segment ℝ y₁ y₂) : Convex ℝ (C' z) :=
hfq₂ z (hT₂.segment_subset hy₁ hy₂ hz) β
have hC'zAB (z) (hz : z ∈ segment ℝ y₁ y₂) : C' z ⊆ A ∨ C' z ⊆ B := by
specialize hC_subset_C' z
specialize hC'z z hz
have hC' : IsPreconnected (C' z) :=
((hC'z).isConnected ((hC_nonempty z hz).mono hC_subset_C')).isPreconn
rw [isPreconnected_iff_subset_of_disjoint_closed] at hC'
exact hC' A B hA_closed hB_closed (hC'zAB z hz) (by simp [hAB])
have hCzAB (z) (hz : z ∈ segment ℝ y₁ y₂) : C z ⊆ A ∨ C z ⊆ B := by
specialize hC_subset_C' z
rcases hC'zAB z hz with h | h
· exact .inl (hC_subset_C'.trans h)
· exact .inr (hC_subset_C'.trans h)
have h_not_CzAB (z) (hz : z ∈ segment ℝ y₁ y₂) : ¬(C z ⊆ A ∧ C z ⊆ B) :=
rintro ⟨h₁, h₂⟩
specialize hC_nonempty z hz
replace h₁ := Set.mem_of_mem_of_subset hC_nonempty.some_mem h₁
replace h₂ := Set.mem_of_mem_of_subset hC_nonempty.some_mem h₂
simpa [hAB] using Set.mem_inter h₁ h₂
let I : Set N := { z | z ∈ segment ℝ y₁ y₂ ∧ C z ⊆ A }
let J : Set N := { z | z ∈ segment ℝ y₁ y₂ ∧ C z ⊆ B }
have hI₁ : I.Nonempty := by
use y₁
simp [I, A, hC_subset_C', left_mem_segment]
have hJ₁ : J.Nonempty := by
use y₂
simp [J, B, hC_subset_C', right_mem_segment]
rw [Set.nonempty_iff_ne_empty] at hI₁ hJ₁
have hIJ : I ∩ J = ∅ := by
simp [I, J, Set.ext_iff]
intro z hz hCzA _ hCzB
specialize hCzA (hC_nonempty z hz).some_mem
specialize hCzB (hC_nonempty z hz).some_mem
rw [← Set.disjoint_iff_inter_eq_empty, Set.disjoint_left] at hAB
tauto

```

```

have hIJ₂ : I ∪ J = segment ℝ y₁ y₂ := by
  simp [I, J, Set.ext_iff]
  grind
have hI : IsClosed I := by
  apply IsSeqClosed.isClosed
  intro zs z hzI hzs
  simp only [Set.mem_setOf_eq, I, C] at hzI
  replace {hzI, hzI₂} := And.intro (hzI · |>.left) (hzI · |>.right)
have hz_mem : z ∈ segment ℝ y₁ y₂ :=
  have cloL : IsClosed (segment ℝ y₁ y₂) := by
    rw [← closure_openSegment]
    exact isClosed_closure
  cloL.isSeqClosed hzI hzs
let x := (hC_nonempty z hz_mem).some
have hx : x ∈ C z := (hC_nonempty z hz_mem).some_mem
suffices hn : ∃ n, x ∈ C' (zs n) by
  use hz_mem
  rcases hn with ⟨n, hn⟩
  have hCzn : C (zs n) ⊆ A := hzI₂ n
  replace hCzn : C' (zs n) ⊆ A := by
    refine (hC'zAB (zs n) (hzI n)).resolve_right ?_
    intro hC'B
    exact (h_not_CzAB (zs n) (hzI n)) {hCzn, (hC_subset_C' _).trans hC'B}
  have hxA : x ∈ A := Set.mem_of_mem_of_subset hn hCzn
  refine (hCzAB z hz_mem).resolve_right ?_
  intro hCB
  have hAC : A ∩ C z ⊆ A ∩ B := by apply Set.inter_subset_inter_right _ hCB
  rw [hAB, Set.subset_empty_iff, Set.ext_iff] at hAC
  revert hAC
  simp only [Set.mem_inter_iff, Set.mem_empty_iff_false, iff_false, not_and, imp]
  exact ⟨x, hxA, hx⟩
suffices hn : ∃ n, f x (zs n) < β by
  refine hn.imp fun n ↦ ?_
  simp +contextual [C', le_of_lt, hx.left]
simp only [Set.mem_setOf_eq, C] at hx
rcases hx with ⟨hx₁, hx₂⟩
specialize hfc₁ x hx₁
replace hx₂ := hx₂.trans_lt hβ₁
replace hzI (n) := hT₂.segment_subset hy₁ hy₂ (hzI n)
replace hz_mem := hT₂.segment_subset hy₁ hy₂ hz_mem
exact (hfc₁.frequently_lt_of_tendsto hzs hx₂ hzI hz_mem).exists
have hJ : IsClosed J := by
  apply IsSeqClosed.isClosed
  intro zs z hzI hzs
  simp only [Set.mem_setOf_eq, J, C] at hzI
  replace {hzI, hzI₂} := And.intro (hzI · |>.left) (hzI · |>.right)

```

```

have hz_mem : z ∈ segment ℝ y1 y2 :=
  have cloL : IsClosed (segment ℝ y1 y2) := by
    rw [← closure_openSegment]
    exact isClosed_closure
  cloL.isSeqClosed hzI hzs
let x := (hC_nonempty z hz_mem).some
have hx : x ∈ C z := (hC_nonempty z hz_mem).some_mem
suffices hn : ∃ n, x ∈ C' (zs n) by
  use hz_mem
  rcases hn with ⟨n, hn⟩
  have hCzn : C (zs n) ⊆ B := hzI2 n
  replace hCzn : C' (zs n) ⊆ B := by
    refine (hC'zAB (zs n) (hzI n)).resolve_left ?_
    intro hC'B
    exact (h_not_CzAB (zs n) (hzI n)) ((hC_subset_C' _).trans hC'B, hCz)
  have hxA : x ∈ B := Set.mem_of_mem_of_subset hn hCzn
  refine (hCzAB z hz_mem).resolve_left ?_
  intro hCB
  have hAC : C z n B ⊆ A n B := by apply Set.inter_subset_inter_left _
  rw [hAB, Set.subset_empty_iff, Set.ext_iff] at hAC
  revert hAC
  simp only [Set.mem_inter_iff, Set.mem_empty_iff_false, iff_false, not]
  exact ⟨x, hx, hxA⟩
suffices hn : ∃ n, f x (zs n) < β by
  refine hn.imp fun n ↦ ?_
  simp +contextual [C', le_of_lt, hx.left]
simp only [Set.mem_setOf_eq, C] at hx
rcases hx with ⟨hx1, hx2⟩
specialize hfc1 x hx1
replace hx2 := hx2.trans_lt hβ1
replace hzI (n) := hT2.segment_subset hy1 hy2 (hzI n)
replace hz_mem := hT2.segment_subset hy1 hy2 hz_mem
exact (hfc1.frequently_lt_of_tendsto hzs hx2 hzI hz_mem).exists
have hConnected := segment.isConnected y1 y2
rw [IsConnected, isPreconnected_iff_subset_of_fully_disjoint_closed ?_] a
· rw [← closure_openSegment]
  exact isClosed_closure
replace hConnected := hConnected.right I J
simp [hIJ, ← hIJ2, Set.disjoint_iff_inter_eq_empty] at hConnected
obtain hL | hR := hConnected hI hJ
· rw [Set.inter_eq_self_of_subset_right hL] at hIJ
  exact hJ1 hIJ
· rw [Set.inter_eq_self_of_subset_left hR] at hIJ
  exact hI1 hIJ

include hfc1 hfq1 hfc2 hfq2 hS1 hS2 hT2 hS3 in

```



```

private lemma sion_exists_min_fin
  (h_bddA : BddAbove (Set.image2 f S T)) (h_bddB : BddBelow (Set.image2 f S T))
  (ys : Finset N) (hys_n : ys.Nonempty) (hys : (ys : Set N)  $\subseteq$  T)
  (a :  $\mathbb{R}$ ) (ha : a <  $\inf_{x : S, \exists y_i : ys, f x y_i}$ )
  :  $\exists y_0 : T, a < \inf_{x : S, f x y_0} :=$  by
  induction hys_n using Finset.Nonempty.cons_induction generalizing S
  case singleton x =>
    simp at ha hys
    use (x, hys)
  case cons y_n t hxt htn ih =>
    classical rw [Finset.cons_eq_insert] at hys ha
    simp [Set.insert_subset_iff] at hys
    rcases hys with (hy_n, ht)
    let _ := hS3.to_subtype
    obtain (b, hab, hb) := exists_between ha
    let S' := {z : M | z  $\in$  S  $\wedge$  f z y_n  $\leq$  b}
    have hS'_sub : S'  $\subseteq$  S := Set.sep_subset ..
    rcases S'.eq_empty_or_nonempty with hS'_e | hS'_n
    · simp [S'] at hS'_e
      use (y_n, hy_n)
      rw [lt_ciInf_iff]; swap
      · --BddBelow (Set.range fun x : S => f (↑x) y_n)
        convert (hfc2 y_n hy_n).bddBelow hS1
        ext; simp
      use b, hab
      intro i
      exact (hS'_e i i.2).le
    let _ := hS'_n.to_subtype
    have hS'_1 : IsCompact S' := by
      apply hS1.of_isClosed_subset ?_ hS'_sub
      specialize hfc2 y_n
      rw [lowerSemicontinuousOn_iff_isClosed_epigraph hS1.isClosed] at hfc2
      have ha_closed : IsClosed {p : M  $\times$   $\mathbb{R}$  | p.2 = b} := by
        convert isClosed_univ.prod (isClosed_singleton (x := b))
        simp [Set.ext_iff]
      suffices Continuous (fun (z : M)  $\mapsto$  (z, b) : M  $\rightarrow$  M  $\times$   $\mathbb{R}$ ) by
        convert ((hfc2 hy_n).inter ha_closed).preimage this
        simp [S']
      fun_prop
    have hS'_2 : Convex  $\mathbb{R}$  S' := hfq2 _ hy_n _
    have ha' : a <  $\inf_{x : S', \exists y_i : t, f x y_i} :=$  by
      apply lt_of_lt_of_le hab (hb.le.trans ?_)
      classical
      trans ( $\inf_{x : S', \exists y_i : \{x // x \in (\text{Insert.insert } y_n t : \text{Finset } N)\}, f \uparrow x \uparrow y_i}$ )
      · apply ciInf_le_ciInf_of_subset (f := fun x  $\mapsto$   $\inf_{y_i : \{x // x \in (\text{Insert.insert } y_n t : \text{Finset } N)\}, f \uparrow x \uparrow y_i}$ )
        · exact hS'_n

```

```

· --BddBelow ((fun x => [] yi, f x ↑yi) '' S)
  convert BddBelow.range_sup_of_image2 (T := S) (S := { x | x ∈ ins
    (by apply h_bddA.mono; simp [flip]; grind) (by apply h_bddB.mon
    simp [Set.ext_iff, flip]
· exact hS'_sub
· gcongr with x
· --BddBelow (Set.range fun x : S' => [] yi, f ↑x ↑yi)
  exact BddBelow.range_sup_of_image2 (T := S') (S := { x | x ∈ ins
    (by apply h_bddA.mono; simp [flip]; grind) (by apply h_bddB.mon
rw [t.ciSup_insert htn]
simp only [sup_le_iff, le_refl, and_true]
conv at hb => enter [2, 1, x]; rw [t.ciSup_insert htn]
contrapose! hb
refine ciInf_le_of_le ?_ (x.val, hS'_sub x.2) ?_
· --BddBelow (Set.range fun x => max (f (↑x) yn) ([] a, f ↑x ↑a))
  apply BddBelow.range_mono (h := fun x ↦ le_max_left _ _)
  convert (hfc2 yn hyn).bddBelow hS1
  ext; simp
· have := x.2.2
  simp [this, hb.le.trans this]
specialize @ih S'
(hfc2 · · |>.mono hS'_sub) hS'1 hS'_n
(hfc1 · <| hS'_sub ·) (hfq2 · · |>.subset hS'_sub hS'2)
(hfq1 · <| hS'_sub ·) hS'2
(h_bddA.mono <| Set.image2_subset_right hS'_sub)
(h_bddB.mono <| Set.image2_subset_right hS'_sub) ht ha'
clear ha'
obtain ⟨y0', hy0'⟩ := ih
refine (sion_exists_min_2 hfc2 hS1 hS3 hfc1 hfq2 hfq1
  hT2 y0' yn y0'.2 hyn a ?_)
by_cases hS'eq : S' = S
· rw [hS'eq] at hy0'
  apply hy0'.trans_le
  gcongr
  · --BddBelow (Set.range fun x : S => f (↑x) y0')
    convert (hfc2 y0' y0'.2).bddBelow hS1
    ext; simp
  exact le_sup_left
have hS_diff_ne : (S \ S').Nonempty :=
  Set.nonempty_of_ssubset (hS'_sub.ssubset_of_ne hS'eq)
apply Set.Nonempty.to_subtype at hS_diff_ne

have h_non_inter : Nonempty ↑(S ∩ S') := by
  rwa [Set.inter_eq_self_of_subset_right hS'_sub]
rw [ciInf_eq_min_cInf_inter_diff (f := fun x ↦ max (f x y0') (f x yn))]
· --BddAbove ((fun x => max (f x y0') (f x yn)) '' S)

```

```

    apply h_bddB.mono
    rintro _ (x, hx, rfl)
    use x, hx
    -- TODO: On a newer mathlib this line is just `grind`
    rcases max_cases (f x ↑y₀') (f x yₙ) <=> grind
  clear h_non_inter
  rw [lt_inf_iff]
  constructor
  · rw [Set.inter_eq_self_of_subset_right hS'_sub]
    apply hy₀'.trans_le
    gcongr
    · --BddBelow (Set.range fun x : S' => f (↑x) y₀')
      apply h_bddB.mono
      rintro _ (x, rfl)
      use x, hS'_sub x.2, y₀', y₀'.2
    exact le_sup_left
  · have hS'_compl : S \ S' = { z | z ∈ S ∧ b < f z yₙ } := by
      ext
      simp [S']
    rw [hS'_compl] at hS_diff_ne
    rw [Set.coe_setOf] at hS_diff_ne
    refine hab.trans_le ?_
    simp
    apply le_ciInf
    intro x
    have := x.2.2.le
    exact le_sup_of_le_right this

include hfc₁ hfq₁ hfc₂ hfq₂ hS₁ hS₂ hT₂ hS₃ hT₃ in
/-- **Sion's Minimax theorem**. Because of `ciSup` and `ciInf` junk values when f is
bounded, we need to assume that it's bounded above and below. -
/
theorem sion_minimax
  (h_bddA : BddAbove (Set.image2 f S T))
  (h_bddB : BddBelow (Set.image2 f S T))
  : [] x : S, [] y : T, f x y = [] y : T, [] x : S, f x y := by
  have _ := hS₁.isClosed
  have _ := hS₃.to_subtype
  have _ := hT₃.to_subtype
  have h_bdd_0 (i : T) : BddBelow (Set.range fun j : S => f j i) := by
    --This one actually doesn't require h_bddB, we can just prove it from compactnes
    convert (hfc₂ i i.2).bddBelow hS₁
  ext; simp
  have h_bdd_1 (j : S) : BddAbove (Set.range fun (x : T) => f j x) :=
    h_bddA.mono (T.range_restrict (f j) ▶ Set.image_subset_image2_right j.coe_prop)
  have h_bdd_2 : BddAbove (Set.range fun y : T => [] x : S, f x y) :=

```

```

    h_bddA.range_inf_of_image2 h_bddB
have h_bdd_3 : BddBelow (Set.range fun x : S => ∅ y : T, f x y) :=
  BddBelow.range_sup_of_image2 (f := flip f) (by simpa) (by simpa)
apply le_antisymm; swap
· exact ciSup_ciInf_le_ciInf_ciSup _ h_bdd_1 h_bdd_0
by_contra! h
obtain ⟨a, ha1, ha2⟩ := exists_between h; clear h
obtain ⟨b, hb1, hb2⟩ := exists_between ha2; clear ha2
revert ha1
rw [imp_false, not_lt]
have := hS1.elim_finite_subfamily_closed (fun (y : T) => { x | x ∈ S ∧ f x y })
· rcases this with ⟨u, hu⟩
  have hu' : u.Nonempty := by
    grind [Finset.not_nonempty_iff_eq_empty, Set.iInter_univ,
      Set.inter_univ, Set.not_nonempty_empty]
  have hau : a < ∅ x : S, ∅ yi : u.map ⟨_, Subtype.val_injective⟩, f ↑x ↑yi
    simp +contextual only [Set.iInter_coe_set, Set.ext_iff, Set.mem_inter,
      Set.mem_setOf_eq, Set.mem_empty_iff_false, iff_false, not_and, true
        not_le] at hu
  rw [lt_ciInf_iff]; swap
  · --BddBelow (Set.range fun x => ∅ yi : Finset.map ..., f ↑x ↑yi)
    exact BddBelow.range_sup_of_image2 (T := S) (S := u.map ⟨_, Subtype
      (f := flip f) (by apply h_bddA.mono; simp [flip]; grind) (by appl
        use b, hb1
    intro i
    specialize hu i i.2
    rcases hu with ⟨c, hc1, hc2, hc3⟩
    refine le_ciSup_of_le ?_ ⟨c, by simpa using ⟨hc1, hc2⟩⟩ hc3.le
    --BddAbove (Set.range fun yi : Finset.map ... => f ↑i ↑yi)
    apply h_bddA.mono
    simp [Set.range, Set.image2]; grind
  obtain ⟨y0, hy0⟩ := sion_exists_min_fin hfc2 hS1 hS3 hfc1 hfq2 hfq1 hT2
    h_bddA h_bddB (u.map ⟨_, Subtype.val_injective⟩) (by simpa) (by simp)
  exact hy0.le.trans (le_ciSup h_bdd_2 y0)
· intro i
  specialize hfc2 i i.2
  rw [lowerSemicontinuousOn_iff_isClosed_preimage] at hfc2
  exact hfc2 b
· convert Set.inter_empty _
  by_contra hu
  simp only [Set.iInter_coe_set, Set.iInter_eq_empty_iff, Set.mem_iInter,
    Classical.not_imp, not_and, not_le, not_forall, not_exists, not_lt] a
  obtain ⟨x, hx⟩ := hu
  apply hb2.not_ge
  apply (ciInf_le h_bdd_3 ⟨x, hx _ hT3.some_mem |>.1⟩).trans _
  apply ciSup_le (hx _ .2 |>.2)

```

1.481 QuantumInfo/ForMathlib/Superadditive.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Analysis.Subadditive

def Superadditive (u :  $\mathbb{N} \rightarrow \mathbb{R}$ ) : Prop :=
   $\forall m n, u (m + n) \geq u m + u n$ 

namespace Superadditive

variable {u :  $\mathbb{N} \rightarrow \mathbb{R}$ } (h : Superadditive u)

include h in
theorem to_Subadditive : Subadditive (-u ·) :=
  (by dsimp; linarith [h · ·])

noncomputable def lim (_h : Superadditive u) :=
  sSup ((fun n :  $\mathbb{N} \Rightarrow u n / n$ ) '' .Ici 1)

/-- Fekete's lemma for superadditive sequences: a superadditive sequence which is
    bounded above converges. -/
theorem tendsto_lim (hbdd : BddAbove (Set.range fun n => u n / n)) :
  Filter.Tendsto (fun n => u n / n) .atTop (nhds (h.lim)) := by
  have := h.to_Subadditive.tendsto_lim (hbdd.recOn
    ((·, fun _ hx => neg_le.mp <| ·
      (hx.recOn ((·, by rw [← ·, neg_div', neg_neg])))))
  )
  convert this.neg using 1
  · ext; rw [neg_div', neg_neg]
  · simp only [lim, Subadditive.lim, Real.sInf_def, neg_neg, nhds_eq_nhds_iff,
    ← Set.image_neg_eq_neg, Set.image_image, neg_div', neg_neg]

end Superadditive

namespace Subadditive

variable {u :  $\mathbb{N} \rightarrow \mathbb{R}$ } (h : Subadditive u)

```

```

include h in
theorem to_Superadditive : Superadditive (-u ·) :=
  (by dsimp; linarith [h · ·])

end Subadditive

```

1.482 QuantumInfo/ForMathlib/Tactic/Commutes.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/

import QuantumInfo.ForMathlib.Tactic.Commutes.Attribute
import Mathlib.Algebra.BigOperators.Group.List.Basic
import Mathlib.Algebra.BigOperators.Ring.Finset
import Mathlib.Algebra.BigOperators.Ring.List
import Mathlib.Algebra.Group.Action.Defs
import Mathlib.Algebra.Group.Center
import Mathlib.Algebra.Group.Commute.Basic
import Mathlib.Algebra.Group.Commute.Hom
import Mathlib.Algebra.Group.Commute.Units
import Mathlib.Algebra.Group.Invertible.Basic
import Mathlib.Algebra.Group.Opposite
import Mathlib.Algebra.Group.Pi.Lemmas
import Mathlib.Algebra.Group.Prod
import Mathlib.Algebra.GroupWithZero.Commute
import Mathlib.Algebra.GroupWithZero.Semiconj
import Mathlib.Algebra.Polynomial.Basic
import Mathlib.Algebra.Ring.Commute
import Mathlib.Algebra.Ring.Nat
import Mathlib.Algebra.Star.SelfAdjoint
import Mathlib.Data.Int.Cast.Lemmas
import Mathlib.Data.Matrix.Basic
import Mathlib.Data.Nat.Cast.Commute
import Mathlib.Data.Rat.Cast.Defs
import Mathlib.GroupTheory.GroupAction.Ring
import Mathlib.LinearAlgebra.Matrix.ZPow

/-!

# Tactic for `Commute`

```

This tactic uses `aesop` to discharge goals relating to the `Commute` relation. It tries to do straightforward recursion on expressions, along with some basic normalization of ring operations.

-/

/-- A tactic for proving goals of the `Commute` relation.

You can use `commutes?` to get a corresponding proof script.

-/

syntax (name := commutesTac) "commutes" : tactic

macro_rules

| `(tactic| commutes) => `(tactic|

(aesop

(config := { introsTransparency? := some .reducible, terminal := true,
useSimpAll := false, useDefaultSimpSet := false })

(rule_sets := [\$(Lean.mkIdent `Commutes):ident, -default]))) --

include `builtin`

/-- A variant of the `commutes` tactic for proving `Commute` goals that produces a proof script. -/

syntax (name := commutesTac?) "commutes?" : tactic

macro_rules

| `(tactic| commutes?) => `(tactic|

(aesop?

(config := { introsTransparency? := some .reducible, terminal := true,
useSimpAll := false, useDefaultSimpSet := false })

(rule_sets := [\$(Lean.mkIdent `Commutes):ident, -default])))

attribute [aesop safe apply (rule_sets := [Commutes])]

Commute.all Commute.refl

Commute.one_left Commute.one_right

Commute.pow_self Commute.self_pow Commute.pow_pow_self

Units.commute_coe_inv Units.commute_inv_coe

Commute.self_zpow Commute.zpow_self Commute.zpow_zpow_self

Commute.self_zpow₀ Commute.zpow_self₀ Commute.zpow_zpow_self₀

Commute.zero_left Commute.zero_right

Commute.neg_one_left Commute.neg_one_right

Nat.cast_commute Nat.commute_cast

Commute.intCast_left Commute.intCast_right

-- Commute.natCast_mul_self Commute.self_natCast_mul --

redundant

-- Commute.intCast_mul_self Commute.intCast_mul_self_right --

redundant

commute_invOf

NNRat.cast_commute NNRat.commute_cast

Rat.cast_commute Rat.commute_cast

```

Polynomial.commute_X Polynomial.commute_X_pow
Matrix.commute_diagonal Matrix.scalar_commute
IsStarNormal.star_comm_self
Matrix.Commute.self_zpow Matrix.Commute.zpow_self Matrix.Commute.zpow_zpow

--It's extremely rare that we need to use Commute.symm in the middle of a p
--quite common that we specifically need it at the end (or equivalently, at
--which is why we give a safe + fast tactic to apply. This often cuts heart
--factor.
attribute [aesop unsafe apply 5% (rule_sets := [Commutes])] Commute.symm

--This essentially does `symm + assumption`. Note that `assumption` is alre
add_aesop_rules safe tactic (rule_sets := [Commutes]) (by exact Commute.symm

--Due to indexing OfNat issues, these don't work as `apply` rules. We add t
add_aesop_rules safe tactic (rule_sets := [Commutes]) (by apply Commute.ofN
add_aesop_rules safe tactic (rule_sets := [Commutes]) (by apply Commute.ofN

--Try to normalize ring operations
add_aesop_rules safe tactic (rule_sets := [Commutes]) (by apply Commute.ofN

attribute [aesop unsafe apply 50% (rule_sets := [Commutes])]
  Commute.mul_left Commute.mul_right
  Commute.pow_left Commute.pow_right --Commute.pow_pow (redundant)
  Commute.smul_left Commute.smul_right
  Commute.op Commute.unop
  Commute.prod
  Commute.zpow_left Commute.zpow_right --Commute.zpow_zpow (redundant)
  Commute.zpow_left₀ Commute.zpow_right₀ --Commute.zpow_zpow_self₀ (redunda
  Commute.units_of_val Commute.units_val
  Commute.units_inv_left Commute.units_inv_right
  Commute.units_zpow_left Commute.units_zpow_right
  Commute.ringInverse_ringInverse
  Commute.inv_left₀ Commute.inv_right₀
  Commute.div_left Commute.div_right
  Commute.neg_left Commute.neg_right
  Commute.add_left Commute.add_right
  Commute.sub_left Commute.sub_right
  -- Commute.natCast_mul_left Commute.natCast_mul_right --
redundant
  -- Commute.intCast_mul_left Commute.intCast_mul_right --
redundant
  Commute.invOf_left Commute.invOf_right
  Commute.inv_left Commute.inv_right --Commute.inv_inv (redundant)
  Commute.list_prod_left Commute.list_prod_right
  Commute.list_sum_left Commute.list_sum_right

```



```

Commute.sum_left Commute.sum_right
Commute.conj
Commute.pi
Commute.map
Commute.star_star --Commute.star_left Commute.star_right
Matrix.Commute.zpow_left Matrix.Commute.zpow_right --Matrix.Commute.zpow_zpow (red

--TODO: In `Mathlib.Analysis.Normed.Algebra.Exponential`, tag
--   `Commute.exp_left`, `Commute.exp_right`, `Commute.exp`

--TODO: In cfc-related files, tag `Commute.cfc`, `IsSelfAdjoint.commute_cfc`,
-- `Commute.cfc_real`, `cfc_commute_cfc`, `cfc_n_commute_cfc_n`, `Commute.expUnitary`

attribute [aesop simp (rule_sets := [Commutes])]
  List.mem_map --Needed for `Commute.list_prod_left` to be interesting
  --Do some weak normalization of ring operations. This is similar to the list
  -- of lemmas used in `noncomm_ring`.
  mul_one one_mul mul_zero zero_mul add_zero zero_add pow_one pow_zero
  one_pow zero_pow

--Chekcing that `commutes` can now prove several other lemmas, which were not part
-- of the set of rules given above.
example : type_of% @Commute.natCast_mul_self := by
  commutes

example : type_of% @Commute.natCast_mul_left := by
  commutes

example : type_of% @Commute.zpow_zpow_self₀ := by
  commutes

example : type_of% @Commute.natCast_mul_natCast_mul := by
  commutes

example : type_of% @Commute.intCast_mul_self := by
  commutes

example : type_of% @Commute.intCast_mul_left := by
  commutes

example : type_of% @Matrix.Commute.zpow_zpow := by
  commutes

--Very basic example: all natural numbers commute.
example (A B : Nat) : Commute A B := by
  commutes

```

```

--Check that we can also recognize assumptions stated in equivalent forms
example (R : Type*) [NonUnitalNonAssocSemiring R] (x y : R) (h : SemiconjBy
  Commute (x + y) x := by
  commutes

example (R : Type*) [Semiring R] (x y : R) (h : x * y * 1 = 0 + y * x) :
  Commute (y ^ 2) (x + 1) := by
  commutes

--Example of a more complex goal that it can prove.
example (l : List ℕ) (R : Type*) [Ring R] (x y : R) (h : Commute y x) :
  Commute (l.map (fun n => n • x + y ^ n - 37)).prod x := by
  commutes --Runs in 5676 heartbeats

--Produced proof script from `commutes?` for the above goal. Takes 850 heartbeats
example (l : List ℕ) (R : Type*) [Ring R] (x y : R) (h : Commute y x) :
  Commute (l.map (fun n => n • x + y ^ n - 37)).prod x := by
  apply Commute.list_prod_left
  intro x_1 a
  simp only [List.mem_map] at *
  obtain ⟨w, h_1⟩ := a
  obtain ⟨left, right⟩ := h_1
  subst right
  apply Commute.sub_left
  · apply Commute.add_left
    · apply Commute.smul_left
      rfl
    · apply Commute.pow_left
      exact h
  · apply Commute.ofNat_left

```

1.483 QuantumInfo/ForMathlib/Tactic/Commutes/Attribute.lean

```

/-
Copyright (c) 2026 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/

import Mathlib.Init
import Aesop.Frontend.Command

/-!

```

Commutes Rule Set

This module defines the `Commutes` Aesop rule set which is used by the `commutes` tactic. Aesop rule sets only become visible once the file in which they're declared is imported, so we must put this declaration into its own file.
-/

declare_aesop_rule_sets [Commutes] (default := false)

1.484 QuantumInfo/ForMathlib/ULift.lean

/-

Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg

-/

import Mathlib

open ComplexOrder

universe u v

instance AddZero.ulift {α : Type u} [AddZero α] : AddZero (ULift.{v, u} α) where

instance ULift.instStar {α : Type u} [Star α] : Star (ULift.{v, u} α) where
star x := .up (star x.down)

@[simp]

theorem ULift.star_eq {α : Type u} [Star α] (x : ULift.{v, u} α) : star x = .up (star x.down) := rfl

instance ULift.instInvolutiveStar {α : Type u} [InvolutiveStar α] : InvolutiveStar (ULift.{v, u} α) where
star_involutive x := by simp

instance ULift.instStarMul {α : Type u} [Mul α] [StarMul α] : StarMul (ULift.{v, u} α) where
star_mul x y := by simp; rfl

instance {α : Type u} [NonUnitalNonAssocSemiring α] [StarRing α] : StarRing (ULift.{v, u} α) where
star_add x y := by simp; rfl

@[simp]

theorem ULift.starRingEnd_down {α : Type u} (x : ULift.{v, u} α) [CommSemiring α] [StarRing α] :
((starRingEnd (ULift.{v, u} α)) x).down = star x.down := by
rfl

```

instance {α : Type u} [NormedField α] : NormedField (ULift.{v,u} α) where
  dist_eq x y := NormedField.dist_eq x.down y.down
  norm_mul a b := NormedField.norm_mul a.down b.down

instance {α : Type u} [DenselyNormedField α] : DenselyNormedField (ULift.{v,u} α) where
  lt_norm_lt x y := by simpa using DenselyNormedField.lt_norm_lt x y

@[simp]
theorem AddEquiv.ulift_apply {α : Type u} [Add α] (x : ULift.{v, u} α) :
  AddEquiv.ulift.{u, v} x = x.down := by
  rfl

noncomputable instance {α : Type u} [RCLike α] : RCLike (ULift.{v,u} α) where
  re := RCLike.re.comp AddEquiv.ulift.toAddMonoidHom (N := α)
  im := RCLike.im.comp AddEquiv.ulift.toAddMonoidHom (N := α)
  I := .up RCLike.I
  I_re_ax := by simp
  I_mul_I_ax := by
    rcases RCLike.I_mul_I_ax (K := α) with h | h
    · left
      ext1
      exact h
    · right
      ext1
      exact h
  re_add_im_ax z := by exact congrArg ULift.up (RCLike.re_add_im_ax z.down)
  ofReal_re_ax r := by simp
  ofReal_im_ax := by simp
  mul_re_ax z w := by simp
  mul_im_ax := by simp
  conj_re_ax x := by simp
  conj_im_ax := by simp
  conj_I_ax := by ext1; simp
  norm_sq_eq_def_ax x := by simpa using RCLike.norm_sq_eq_def_ax x.down
  mul_im_I_ax := by simp
  le_iff_re_im {z w} := by simpa using RCLike.le_iff_re_im (z := z.down) (w := w.down)

```

1.485 QuantumInfo/ForMathlib/Unitary.lean

/-

Copyright (c) 2025 Alex Meiburg. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Alex Meiburg

```

-/
import Mathlib.LinearAlgebra.Matrix.Kronecker
import Mathlib.LinearAlgebra.Matrix.PosDef
import QuantumInfo.ForMathlib.HermitianMat.Unitary

open BigOperators
open Classical

namespace LinearMap
section unitary

variable {α : Type*} [RCLike α]
variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace α E]
variable [FiniteDimensional α E]

open Module.End

@[simp]
theorem unitary_star_apply_eq (U : unitary (E →1 [α] E)) (v : E) :
  (star U.val) (U.val v) = v := by
  rw [← mul_apply, (Unitary.mem_iff.mp U.prop).left, one_apply]

@[simp]
theorem unitary_apply_star_eq (U : unitary (E →1 [α] E)) (v : E) :
  U.val ((star U.val) v) = v := by
  rw [← mul_apply, (Unitary.mem_iff.mp U.prop).right, one_apply]

/-- Conjugating a linear map by a unitary operator gives a map whose  $\mu$ -
eigenspace is
isomorphic (same dimension) as those of the original linear map. -
/
noncomputable def conj_unitary_eigenspace_equiv (T : E →1 [α] E) (U : unitary (E →1 [α] E)) :
  eigenspace T  $\mu$   $\approx$ 1 [α] eigenspace (U.val * T * star (U.val))  $\mu$  where
  toFun v := ⟨U.val v.val, by
    have hv := v.2
    rw [mem_eigenspace_iff] at hv ⊢
    simp [hv]⟩
  invFun v := ⟨(star U.val) v, by
    have hv := v.2
    rw [mem_eigenspace_iff] at hv ⊢
    simp using congrArg ((star U.val) ·) hv⟩
  map_add' := by simp
  map_smul' := by simp
  left_inv _ := by simp
  right_inv _ := by simp

```

```

end unitary
namespace IsSymmetric

open Module.End

variable {α : Type*} [RCLike α]
variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace α E]
variable [FiniteDimensional α E]
variable {T : E →ℓ [α] E}

/-- A symmetric operator conjugated by a unitary is symmetric. -
/
theorem conj_unitary_IsSymmetric (U : unitary (E →ℓ [α] E)) (hT : T.IsSymmetric) :
  (U.val * T * star U.val).IsSymmetric := by
  intro i j
  rw [mul_assoc, mul_apply, ← LinearMap.adjoint_inner_right]
  rw [mul_apply, mul_apply, mul_apply, ← LinearMap.adjoint_inner_left U.val]
  exact hT (star U.val <| i) (star U.val j)

variable {n : ℕ} (hn : Module.finiteRank α E = n)

/-- There is an equivalence between the eigenvalues of a finite dimensional
and the eigenvalues of that operator conjugated by a unitary. -
/
def conj_unitary_eigenvalue_equiv (U : unitary (E →ℓ [α] E)) (hT : T.IsSymmetric) :
  { σ : Equiv.Perm (Fin n) // (hT.conj_unitary_IsSymmetric U).eigenvalues σ }
  sorry --use conj_unitary_eigenspace_equiv

end IsSymmetric
end LinearMap

```

1.486 QuantumInfo/InfiniteDim/QState.lean

```

--Copyright 2024: https://github.com/dupuisf/LFTCM2024TraceClass
-- import Mathlib
-- -- import QuantumInfo.InfiniteDim.TraceClass

-- variable (H : Type*)
-- variable [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H]

-- namespace Quantum

-- local notation "α" x "β", "γ" y "δ" => @inner ℂ _ _ x y

```

```

-- open InnerProduct

-- /-- Ket in a Hilbert space H, defined as a vector of unit norm. -
/
-- structure Ket :=
--   vec : H
--   normalized : ||vec||=1

-- /-- An operator on a Hilbert space H is a mixed state iff it's a PSD operator with
/
-- structure IsMixedState (op : H → L[ℂ] H) : Prop :=
--   /-- The operator of a mixed state is Hermitian -/
--   herm : IsSelfAdjoint op
--   /-- The operator of a mixed state is Positive Semidefinite -
/
--   posSemiDef : ∀ (x : H), 0 ≤ ⟨x, op x⟩.re
--   /-- The operator is traceclass. -/
--   traceClass : IsTraceClass op
--   /-- The trace of a mixed state is 1. -/
--   traceOne : TraceClass.trace op = 1

-- /-- Mixed state in a Hilbert space H, defined as a PSD operator with trace 1. -
/
-- structure MState :=
--   /-- The underlying operator of a state -/
--   op : H → L[ℂ] H
--   /-- The operator is a valid mixed state. -/
--   prop : IsMixedState H op

-- /-- The stateSpace of a Hilbert space H is the set of all mixed states in H. -
/
-- def stateSpace : Set (H → L[ℂ] H) := { p | IsMixedState H p }

-- set_option pp.analyze true

-- theorem stateSpace_convex : Convex ℝ (stateSpace H) := by
--   rw [convex_iff_pointwise_add_subset]
--   intros x y xpos ypos xyl z hz
--   rw [stateSpace] at hz ⊢
--   rw [Set.mem_setOf_eq]
--   obtain ⟨x₁', hx₁', y₁', hy₁', hp₁p₂⟩ := Set.mem_add.1 hz
--   obtain ⟨p₁, hp₁, dx₁'⟩ := Set.mem_smul_set.1 hx₁'
--   obtain ⟨p₂, hp₂, dy₁'⟩ := Set.mem_smul_set.1 hy₁'
--   clear hz hx₁' hy₁'
--   subst dx₁'
--   subst dy₁'

```

```

--   subst z
--   rw [Set.mem_setOf_eq] at hp1 hp2
--   constructor
--   · have h1 := hp1.1
--     have h2 := hp2.1
--     apply IsSelfAdjoint.add
--     <=> apply IsSelfAdjoint.smul (by rfl) <_>
--   · intro v
--     simp only [ContinuousLinearMap.add_apply, ContinuousLinearMap.coe_smul,
--       Pi.smul_apply, inner_add_right, Complex.add_re]
--     change 0 ≤ (∑v, (x:ℂ) • ρ1 v ∑_ℂ).re + (∑v, (y:ℂ) • ρ2 v ∑_ℂ).re
--     simp only [inner_smul_right, Complex.mul_re, Complex.ofReal_re, Complex.zero_mul,
--       sub_zero, ge_iff_le]
--     have h1 := hp1.2 v
--     have h2 := hp2.2 v
--     positivity
--   ·
--   -- · obtain ⟨ι, _⟩ := hp1.3
--   --   apply TraceClass.mk_of_exists
--   --   sorry
--   sorry
--   · sorry

```

1.487 QuantumInfo/QI_DOC.md

QuantumInfo Documentation

This document provides a comprehensive overview of mathematical and physical QuantumInfo part of PhysLib, explaining the exact conventions adopted, their meaning and highlighting key theorems.

Table of Contents

****Part I: Mathematical Preliminaries****

1. [Basic Types and Structures](#basic-types-and-structures)
2. [Hermitian Matrices](#hermitian-matrices)
3. [The Loewner Order](#the-loewner-order)
4. [Continuous Functional Calculus](#continuous-functional-calculus)
5. [Projectors and the Order](#projectors-and-the-order)
6. [Logarithm and Special Functions](#logarithm-and-special-functions)

7. [Other Mathematical Results](#other-mathematical-results)

****Part II: Quantum Information Theory****

8. [Pure States: Bra and Ket](#pure-states-bra-and-ket)

9. [Mixed States: MState](#mixed-states-mstate)

10. [Matrix Maps and Quantum Channels](#matrix-maps-and-quantum-channels)

11. [Entropy and Information Theory](#entropy-and-information-theory)

12. [Named Subsystems and Permutations](#named-subsystems-and-permutations)

13. [Distances and Fidelity](#distances-and-fidelity)

14. [Kronecker Products](#kronecker-products)

****Part III: Reference****

15. [Notations and Scopes](#notations-and-scopes)

16. [Major Theorems](#major-theorems)

Part I: Mathematical Preliminaries

Basic Types and Structures

Prob

****`Prob`**** is a real number between 0 and 1, representing a probability:

```lean

```
def Prob := { p : ℝ // 0 ≤ p ∧ p ≤ 1 }
```

```

****Key operations:****

- ****`toReal`** (p : Prob) : ℝ : Extract the underlying real number
- Arithmetic operations when they preserve the bounds
- ****`Mixable`**** instance: Probabilities form a convex set

Distribution

****`Distribution α`**** on a finite type `α` is a probability distribution:

```lean

```
def Distribution (α : Type u) [Fintype α] : Type u :=
 { f : α → Prob // Finset.sum Finset.univ (fun i => (f i).toReal) = 1 }
```

```

A function from α to Prob whose values sum to 1.

```

**Key operations:**
- **`uniform` : Distribution  $\alpha$ ** : The uniform distribution
- **`expect_val`** : Expected value of a function under the distribution
- **`Mixable`** instance: Distributions are convex

---

## Hermitian Matrices

### Definition

**`HermitianMat n  $\alpha$ `** is the type of Hermitian  $n \times n$  matrices over a field.

```lean
def HermitianMat (n : Type*) (α : Type*) [AddGroup α] [StarAddMonoid α] :=
 { selfAdjoint (Matrix n n α) : Type (max u_1 u_2) }
```

```

It's defined as the subtype of self-adjoint elements in $\text{Matrix } n \ n \ \alpha$, which

```

```lean
{ M : Matrix n n α // M.IsHermitian }
```

```

where M.IsHermitian means $M^H = M$ (conjugate transpose equals itself).

```

**Key fields and functions:**
- **`mat` ( $A$  : HermitianMat n  $\alpha$ ) : Matrix n n  $\alpha$ ** : The underlying matrix
- **`H` ( $A$  : HermitianMat n  $\alpha$ )** : Proof that  $A.\text{mat}.\text{IsHermitian}$ 
- **Construction** : Use  $\langle M, h \rangle$  where  $h : M.\text{IsHermitian}$ 

```

Basic Operations

```

**Algebraic structure:**
- `HermitianMat n  $\alpha$ ` is an `AddGroup` (inherits from matrices)
- Has `0`, `1` when appropriate
- Addition and subtraction defined componentwise
- Scalar multiplication by real numbers (see below)

**Key operations:**
- **`conj` ( $B$  : Matrix m n  $\alpha$ ) : HermitianMat n  $\alpha$   $\rightarrow$  HermitianMat m  $\alpha$ ** :
  Conjugation by a (possibly rectangular) matrix:  $A \mapsto B * A * B^H$ 
  - Returns a `HermitianMat m  $\alpha$ ` (different size!)
  - This is an `AddMonoidHom`

```

- `**`conjLinear` (B : Matrix m n  $\alpha$ ) : HermitianMat n  $\alpha$   $\rightarrow$   $\iota$ [R] HermitianMat m  $\alpha$  `**:`
Same as ``conj`` but as a linear map (when we have scalar multiplication by ``R``)
- `**`diagonal` (f : n  $\rightarrow$   $\mathbb{R}$ ) : HermitianMat n  $\square$  `**:`
Diagonal matrix with real entries on the diagonal
- `**`kronecker` (A : HermitianMat m  $\alpha$ ) (B : HermitianMat n  $\alpha$ ) : HermitianMat (m  $\times$  n)`
Kronecker product. Notation: ``A  $\otimes_k$  B`` (scoped to ``HermitianMat``)

The Trace and IsMaximalSelfAdjoint

The trace of a Hermitian matrix has a subtlety related to scalar multiplication.

****The Problem:****

Hermitian matrices over `` $\mathbb{C}$ `` can be viewed as matrices over a ****real**** vector space (behaved with respect to scalar multiplication).

****The ``IsMaximalSelfAdjoint`` Typeclass:****

This typeclass handles the relationship between the scalar field for matrices (`` $\alpha$ `` =

```lean`

```
class IsMaximalSelfAdjoint (R A : Type*) [CommSemiring R] [AddCommGroup A] [Module R A]
 [Star A] [StarAddMonoid A] where
 selfadjMap : selfAdjoint A $\rightarrow$ ι [R] R
````
```

For ``HermitianMat n  $\mathbb{C}$ ``, this provides a way to take real-valued functions (like trace) and make them respect the real scalar structure.

****The Trace Definition:****

```lean`

```
def trace (A : HermitianMat n α) : R :=
 IsMaximalSelfAdjoint.selfadjMap (A.mat.trace)
````
```

This wraps the standard matrix trace to ensure it respects the scalar structure prop

****Why this matters:****

- The trace of a Hermitian matrix is always ****real****
- We want ``trace (r  $\cdot$  A) = r  $\cdot$  trace A`` for ****real**** ``r``
- But ``A.mat.trace`` is technically in `` $\mathbb{C}$ ``
- ``IsMaximalSelfAdjoint`` provides the canonical map from the self-adjoint elements to the real scalars

****Key properties:****

```
- **`trace_eq_re_trace`**: The HermitianMat trace equals the real part of t
- **`trace_add`**: `(A + B).trace = A.trace + B.trace`
- **`trace_smul`**: `(r • A).trace = r • A.trace` for real `r`
```

Partial Traces

For bipartite systems:

```
- **`traceLeft (A : HermitianMat (m × n) α) : HermitianMat n α`**:
  Trace out the first subsystem
- **`traceRight (A : HermitianMat (m × n) α) : HermitianMat m α`**:
  Trace out the second subsystem
```

These operations are crucial for dealing with composite quantum systems.

Inner Product

Hermitian matrices have a real inner product (the Hilbert-Schmidt inner product):

```
```lean
[A, B] := (A.mat * B.mat).trace.re
```
```

This is written with double angle brackets and is always **real-valued**.

Properties:

```
- Bilinearity (over real scalars)
- Symmetry:  $[A, B] = [B, A]$ 
- Positive definiteness:  $[A, A] \geq 0$  with equality iff  $A = 0$ 
- Cauchy-Schwarz:  $|[A, B]| \leq [A, A]^{(1/2)} * [B, B]^{(1/2)}$ 
```

The Loewner Order

Definition

The **Loewner order** is a partial order on Hermitian matrices:

```
```lean
A ≤ B ↔ (B - A).mat.PosSemidef
```
```

****In words:**** $A \leq B$ if and only if $B - A$ is positive semidefinite (all eigenvalues $\langle x, (B - A)x \rangle \geq 0$ for all vectors x).

Implementation

****For HermitianMat:****

- Implemented via `PartialOrder (HermitianMat n [])` instance
- Always in scope (no need to open a namespace)
- Comes with `IsOrderedAddMonoid` structure

****For Matrix:****

- In the `MatrixOrder` namespace
- Must open with `open MatrixOrder` to use
- Same definition: $A \leq B \leftrightarrow (B - A).PosSemidef$

Characterizations

Multiple equivalent ways to express the order:

1. ****`le\_iff`**:** $A \leq B \leftrightarrow (B - A).mat.PosSemidef$

2. ****`zero\_le\_iff`**:** $0 \leq A \leftrightarrow A.mat.PosSemidef$

3. ****`le\_iff\_mulVec\_le`**:**

$$A \leq B \leftrightarrow \forall x, \langle x, A \cdot x \rangle \leq \langle x, B \cdot x \rangle$$

(All quadratic forms are ordered)

4. ****Via inner product**:** $A \leq B \leftrightarrow \forall C \geq 0, \text{tr}(C \cdot A) \leq \text{tr}(C \cdot B)$

Order Properties

****Compatibility with operations:****

- ****Addition**:** $A \leq B \rightarrow A + C \leq B + C$
- ****Scalar multiplication**** (nonnegative): $0 \leq r, A \leq B \rightarrow r \cdot A \leq r \cdot B$
- ****Conjugation**** (`conj_mono`): $A \leq B \rightarrow M \cdot A \cdot M^H \leq M \cdot B \cdot M^H$
- ****Trace**** (`trace_mono`): $A \leq B \rightarrow A.trace \leq B.trace$
- ****Diagonal**** (`diag_mono`): $A \leq B \rightarrow \forall i, A.diag\ i \leq B.diag\ i$
- ****Inner product**** (`inner_mono`): If $0 \leq A$ and $B \leq C$, then $\langle A, B \rangle \leq \langle A, C \rangle$

****Kernels:****

- ****`ker\_antitone`**:** If $0 \leq A \leq B$, then $\ker(B) \subseteq \ker(A)$
- Larger matrices in Loewner order have smaller kernels

****Convexity:****

- **``convex_cone``**: If $\mathbf{0} \leq A, B$ and $\mathbf{0} \leq c_1, c_2$, then $\mathbf{0} \leq c_1 \bullet A + c_2 \bullet B$
- The set of positive semidefinite matrices is a convex cone

Topological Properties

- **``OrderClosedTopology``**: The order relation is closed
- If $A_n \leq B_n$ and $A_n \rightarrow A$, $B_n \rightarrow B$, then $A \leq B$
- **``CompactIccSpace``**: Order intervals are compact
- The set $\{X \mid A \leq X \leq B\}$ is compact for any $A \leq B$

Special Elements

- **``ZeroLEOneClass``**: $\mathbf{0} \leq \mathbf{1}$ in the Loewner order
- The identity matrix is positive semidefinite

Continuous Functional Calculus

The CFC Framework

The **continuous functional calculus (CFC)** allows us to apply continuous

`lean`

```
def cfc (A : HermitianMat d) (f : ℝ → ℝ) : HermitianMat d
```

Conceptually: If $A = \sum_i \lambda_i |e_i\rangle\langle e_i|$ (spectral decomposition), then:

$$A.\text{cfc } f = \sum_i f(\lambda_i) |e_i\rangle\langle e_i|$$

Mathematical basis: Uses Mathlib's CFC for C*-algebras, specialized to

Basic CFC Operations

Identity and constants:

- **``cfc_id``**: $A.\text{cfc id} = A$
- **``cfc_const``**: $A.\text{cfc } (\text{fun } _ \Rightarrow c) = c \bullet \mathbf{1}$

Arithmetic:

- **``cfc_add``**: $A.\text{cfc } (f + g) = A.\text{cfc } f + A.\text{cfc } g$
- **``cfc_mul``**: $A.\text{cfc } (f * g) = A.\text{cfc } f * A.\text{cfc } g$ (when commutative)
- **``cfc_smul``**: $A.\text{cfc } (c \bullet f) = c \bullet A.\text{cfc } f$
- **``cfc_neg``**: $A.\text{cfc } (-f) = -(A.\text{cfc } f)$

- **``cfc_sub``**: $A.cfc (f - g) = A.cfc f - A.cfc g$

`**Composition:**`

- **``cfc_comp``**: $A.cfc (f \circ g) = (A.cfc g).cfc f$ (under appropriate conditions)

`**Conjugation:**`

- **``cfc_conj_unitary``**: $(A.conj U).cfc f = (A.cfc f).conj U$ for unitary U

CFC and the Order

`**Monotonicity:**`

- **``cfc_le_cfc_of_commute_mono0n``**: If f is monotone on $(0, \infty)$, A and B co

This is crucial for proving operator monotonicity and concavity results.

Relationship to Diagonal Matrices

For diagonal matrices, the CFC is particularly simple:

``cfc_diagonal``:

$(\text{diagonal } d).cfc f = \text{diagonal } (f \circ d)$

This just applies f componentwise to the diagonal entries.

Projectors and the Order

Definitions

``proj_le (A B : HermitianMat n) : HermitianMat n`` (notation $\{A \leq_p B\}$):

`lean`

`def proj_le (A B : HermitianMat n) : HermitianMat n :=
 (B - A).cfc (fun x ↦ if 0 ≤ x then 1 else 0)`

Projects onto the eigenspace where $B - A$ has non-negative eigenvalues.

``proj_lt (A B : HermitianMat n) : HermitianMat n`` (notation $\{A <_p B\}$):

`lean`

`def proj_lt (A B : HermitianMat n) : HermitianMat n :=
 (B - A).cfc (fun x ↦ if 0 < x then 1 else 0)`

```

Projects onto the eigenspace where  $B - A$  has strictly positive eigenvalue

**\*\*Flipped notations:\*\***

- $\{A \geq_p B\} = \{B \leq_p A\}$
- $\{A >_p B\} = \{B <_p A\}$

**\*\*Note:\*\*** The default ordering here uses  $\leq_p$ , opposite to some papers that

**### Properties**

**\*\*Idempotency:\*\***

- **\*\*`proj\_le\_sq`\*\*:**  $\{A \leq_p B\}^2 = \{A \leq_p B\}$
- **\*\*`proj\_lt\_sq`\*\*:**  $\{A <_p B\}^2 = \{A <_p B\}$

These are genuine projectors (satisfy  $P^2 = P$ ).

**\*\*Bounds:\*\***

- **\*\*`proj\_le\_nonneg`\*\*:**  $0 \leq \{A \leq_p B\}$
- **\*\*`proj\_lt\_nonneg`\*\*:**  $0 \leq \{A <_p B\}$
- **\*\*`proj\_le\_le\_one`\*\*:**  $\{A \leq_p B\} \leq 1$
- **\*\*`proj\_lt\_le\_one`\*\*:**  $\{A <_p B\} \leq 1$

**\*\*Relationship:\*\***

- **\*\*`proj\_le\_add\_lt`\*\*:**  $\{A \leq_p B\} + \{A <_p B\} \leq 1$  (typically equality when
- **\*\*`one\_sub\_proj\_le`\*\*:**  $1 - \{B \leq_p A\} = \{A <_p B\}$

**\*\*Special cases:\*\***

- **\*\*`proj\_zero\_le\_cfc`\*\*:**  $\{0 \leq_p A\} = A.cfc$  (fun x => if 0 ≤ x then 1 else
- This is the projector onto the non-negative eigenspace of  $A$

**\*\*Inner products:\*\***

- **\*\*`proj\_le\_inner\_nonneg`\*\*:**  $0 \leq \langle \{A \leq_p B\}, B - A \rangle$
- **\*\*`proj\_le\_inner\_le`\*\*:**  $\langle \{A \leq_p B\}, A \rangle \leq \langle \{A \leq_p B\}, B \rangle$

**\*\*Interpretation:\*\*** The projector  $\{A \leq_p B\}$  is the unique maximum projector

**### Positive and Negative Parts**

Related to projectors are the positive and negative parts:

**\*\*`posPart`** (A : HermitianMat d [] ) : HermitianMat d []\*\* (notation  $A^+$ ):  
````lean`

`def posPart (A : HermitianMat d []) : HermitianMat d [] :=
 A.cfc (fun x => max x 0)`


```

```

**`negPart` (A : HermitianMat d $\mathbb{R}$) : HermitianMat d $\mathbb{R}$ (notation `A-):
```lean
def negPart (A : HermitianMat d  $\mathbb{R}$ ) : HermitianMat d  $\mathbb{R}$  :=
  A.cfc (fun x => max (-x) 0)
```

```

**Properties:**

- **posPart\_add\_negPart**:  $A^+ - A^- = A$
- **zero\_le\_posPart**:  $0 \leq A^+$
- **posPart\_continuous**: The map  $A \mapsto A^+$  is continuous
- **negPart\_continuous**: The map  $A \mapsto A^-$  is continuous

---

## ## Logarithm and Special Functions

### ### Matrix Logarithm

**Definition:**

```

```lean
def log (A : HermitianMat d  $\mathbb{R}$ ) : HermitianMat d  $\mathbb{R}$  :=
  A.cfc Real.log
```

```

**Crucial convention:** Uses Mathlib's `Real.log`, which satisfies:

- $\text{Real.log } 0 = 0$
- $\text{Real.log } (-x) = \text{Real.log } x$  for  $x > 0$

### ### Consequences of the Convention

1. **Works on singular matrices:**
  - If  $A$  has zero eigenvalues,  $A.\text{log}$  is still defined
  - Zero eigenvalues map to zero
2. **Kernel preservation:**
  - $\ker(A) = \ker(A.\text{log})$
  - Vectors in the kernel of  $A$  remain in the kernel of  $A.\text{log}$
3. **Negation property:**
  - **log\_smul\_of\_pos**: For PSD  $A$  and  $x \neq 0$ ,  $(x \cdot A).\text{log} = \text{Real.log } x \cdot \{0\}$
  - In particular, for negative  $x$ , the sign is absorbed by  $\text{Real.log}$

### ### Basic Properties

**\*\*Special values:\*\***

- **\*\*`log\_zero`\*\*:**  $(0 : \text{HermitianMat } n \ \square).log = 0$
- **\*\*`log\_one`\*\*:**  $(1 : \text{HermitianMat } n \ \square).log = 0$

**\*\*Scaling:\*\***

- **\*\*`log\_smul`\*\*:** For nonsingular  $A$  and  $x \neq 0$ :  

$$(x \cdot A).log = \text{Real.log } x \cdot 1 + A.log$$

**\*\*Conjugation:\*\***

- **\*\*`log\_conj\_unitary`\*\*:**  $(A.conj \ U).log = A.log.conj \ U$  for unitary  $U$   
 - The logarithm commutes with unitary conjugation

**### Monotonicity**

**\*\*`log\_mono`\*\* (Operator Monotonicity):**

If  $A, B$  are positive definite and  $A \leq B$ , then  $A.log \leq B.log$ .

**\*\*Proof method:\*\*** Uses an integral approximation:

**``lean**

**def** logApprox (x : HermitianMat n  $\square$ ) (T :  $\mathbb{R}$ ) : HermitianMat n  $\square$  :=  
 $\int t \text{ in } (0)..T, ((1 + t)^{-1} \cdot 1 - (x + t \cdot 1)^{-1})$   
**``**

**\*\*Key steps:\*\***

1. **\*\*`logApprox\_mono`\*\*:** The approximation is monotone for each  $T$
2. **\*\*`tendsto\_logApprox`\*\*:** As  $T \rightarrow \infty$ ,  $\text{logApprox } x \ T \rightarrow x.log$
3. Monotonicity is preserved in the limit

**### Concavity**

**\*\*`log\_concave`\*\* (Operator Concavity):**

For positive definite  $A, B$  and  $a, b \geq 0$  with  $a + b = 1$ :

$$a \cdot A.log + b \cdot B.log \leq (a \cdot A + b \cdot B).log$$

**\*\*Proof method:\*\*** Similar integral approximation:

1. **\*\*`logApprox\_concave`\*\*:** The approximation is concave
2. Take the limit as  $T \rightarrow \infty$

**\*\*Related result\*\*** – **\*\*`inv\_antitone`\*\*:**

For positive definite  $A \leq B$ , we have  $B^{-1} \leq A^{-1}$ .  
 The matrix inverse is **\*\*operator antitone\*\***.

**\*\*And\*\*** — **\*\*`inv\_convex`\*\***:  
 For positive definite `A, B` and convex weights:  
 ```

$$(a \cdot A + b \cdot B)^{-1} \leq a \cdot A^{-1} + b \cdot B^{-1}$$

```

The matrix inverse is **\*\*operator convex\*\***.

### ### Kronecker Products

**\*\*`log\_kron`\*\*** (Positive Definite Case):  
 For positive definite `A : HermitianMat m []` and `B : HermitianMat n []`:  
 ```

$$(A \otimes_k B).log = A.log \otimes_k 1 + 1 \otimes_k B.log$$

```

**\*\*Interpretation:\*\*** The logarithm distributes over tensor products (with identity pa

**\*\*`log\_kron\_with\_proj`\*\*** (General Case):  
 For any Hermitian `A, B` (possibly singular):  
 ```

$$(A \otimes_k B).log = (A.log \otimes_k B.cfc \chi_{\neq 0}) + (A.cfc \chi_{\neq 0} \otimes_k B.log)$$

```

where ` $\chi_{\neq 0}$ ` is the indicator function of non-zero eigenvalues: `fun x => if x = 0

**\*\*Why the projectors?\*\*** They account for the kernel, ensuring the formula works even

### ### Matrix Exponential

**\*\*Definition:\*\***  
 ```lean  
 def exp (A : HermitianMat d []) : HermitianMat d [] :=
 A.cfc Real.exp
 ```

**\*\*Properties:\*\***  
 - **\*\*Positivity\*\***: ` $0 \leq A.exp$ ` always (exponential of a Hermitian matrix is PSD)  
 - **\*\*Exponential of zero\*\***: ` $(0 : HermitianMat n []).exp = 1$ `  
 - **\*\*Exponential is always nonsingular\*\***: ` $A.exp$ ` is always invertible  
 - **\*\*Inverse\*\***: ` $(A.exp)^{-1} = (-A).exp$ `

---

### ## Other Mathematical Results

#### ### Sion's Minimax Theorem

The library includes a formalization of **Sion's minimax theorem**, a generalization of the following:

**Context:** In classical minimax theory, we want to exchange the order of

max\_x min\_y

$$\max_x \min_y f(x, y) \leq \min_y \max_x f(x, y)$$

The question is: when does equality hold?

**Von Neumann's theorem:** If  $X$  and  $Y$  are compact convex subsets of Euclidean space, then

**Sion's theorem:** Generalizes to locally convex topological vector spaces

- Continuous
- Quasi-convex in  $x$  for each fixed  $y$
- Quasi-concave in  $y$  for each fixed  $x$

Then the minimax equality holds.

**Formalization:** Located in `QuantumInfo/ForMathlib/SionMinimax.lean`

**Key definitions:**

- `QuasiconvexOn`: Generalizes convexity
- `QuasiconcaveOn`: Generalizes concavity

**The theorem:** States that under appropriate compactness and continuity assumptions, the minimax equality holds.

**Why it matters:**

- Used in quantum resource theory for optimization problems
- Essential for capacity formulas
- Provides a general framework for saddle point problems

### Regularization and Limits

The library also formalizes notions of regularized quantities:

**SupRegularized** ( $fn : \mathbb{N} \rightarrow \mathbb{R}$ ) :  $\mathbb{R}$ : The limit of  $fn\ n / n$  as  $n \rightarrow \infty$  when it exists, using supremum-based regularization.

The limit of  $fn\ n / n$  as  $n \rightarrow \infty$  when it exists, using supremum-based regularization.

**InfRegularized** ( $fn : \mathbb{N} \rightarrow \mathbb{R}$ ) :  $\mathbb{R}$ : The limit of  $fn\ n / n$  as  $n \rightarrow \infty$  when it exists, using infimum-based regularization.

The limit of  $fn\ n / n$  as  $n \rightarrow \infty$  when it exists, using infimum-based regularization.

**Key theorems:**

- `mono_sup`: If  $fn$  is monotone, the regularization equals the supremum
- `InfRegularized.of_Subadditive`: For subadditive sequences, the infimum

**\*\*Application:\*\*** These are used for regularized entropies and channel capacities, wh

### ### Superadditivity and Subadditivity

**\*\*Definitions:\*\***

```lean

def Superadditive (f : $\mathbb{N} \rightarrow \mathbb{R}$) : Prop := $\forall m n, f\ m + f\ n \leq f\ (m + n)$

def Subadditive (f : $\mathbb{N} \rightarrow \mathbb{R}$) : Prop := $\forall m n, f\ (m + n) \leq f\ m + f\ n$

```

**\*\*Key theorems:\*\***

- **\*\*`Subadditive.to\_Superadditive`\*\*:** If ``f`` is subadditive, then ``-f`` is superadditive

- **\*\*`Superadditive.to\_Subadditive`\*\*:** If ``f`` is superadditive, then ``-f`` is subadditive

**\*\*Application:\*\*** Quantum channel capacities often exhibit superadditivity or subaddi

---

## # Part II: Quantum Information Theory

### ## Pure States: Bra and Ket

#### ### Ket Vectors

**\*\*`Ket d`\*\*** represents a normalized quantum state as a vector:

```lean

structure Ket (d : Type*) [Fintype d] where

vec : d → $\mathbb{C}$

normalized' : $\sum x, \|\text{vec } x\|^2 = 1$

```

**\*\*Interpretation:\*\*** A ``Ket d`` is a unit vector in the Hilbert space  $\mathbb{C}^d$ . The ``norma`

**\*\*Notation:\*\*** ``|ψ⟩`` (scoped to ``Braket``)

**\*\*Function-like behavior:\*\***

- Has ``FunLike`` instance: can apply to indices

- ``ψ i`` gives the ``i``-th component

**\*\*Key operations:\*\***

- **\*\*`Ket.prod`** (``ψ₁ ⊗ ψ₂``): Tensor product giving ``Ket (d₁ × d₂)``

- Defined as:  $(\psi_1 \otimes \psi_2)(i, j) = \psi_1\ i * \psi_2\ j$

- **`uniform\_superposition`** : Ket d`\*\* : The equal superposition  $|+\rangle = (1/\sqrt{d}) \sum x$

### ### Bra Vectors

**`Bra d`** is defined identically to **`Ket d`**:

```

`lean
structure Bra (d : Type*) [Fintype d] where
 vec : d → ℂ
 'normalized' : ∑ x, ||vec x|| ^ 2 = 1
`

```

**Why separate from Ket?** To avoid confusion with complex conjugation. In

**Notation:**  $\langle \psi |$  (scoped to **`Braket`**)

**Conversion:**

- **`Ket.to\_bra`** ( $\psi$  : Ket d) : Bra d`\*\* : Componentwise conjugation  
 - **`Bra.to\_ket`** ( $\psi$  : Bra d) : Ket d`\*\* : Componentwise conjugation

### ### Inner Product

**`dot`** ( $\xi$  : Bra d) ( $\psi$  : Ket d) : ℂ`\*\*:

```

`lean
def dot (ξ : Bra d) (ψ : Ket d) : ℂ := ∑ x, (ξ x) * (ψ x)
`

```

**Notation:**  $\langle \xi | \psi \rangle$  (scoped to **`Braket`**)

**Note:** This is the "mixed" form where we don't automatically conjugate.

**Properties:**

- **Linearity** (in the second argument)  
 - **`dot\_self\_nonneg`**:  $\langle \psi | \psi \rangle$  is real and non-negative  
 - **Normalization**:  $\langle \psi | \psi \rangle = 1$  for any ket  $\psi$

### ### Design Philosophy

**Why not use Mathlib's Hilbert spaces directly?**

The library uses basis-dependent vectors ( $d \rightarrow \mathbb{C}$ ) rather than abstract Hil

1. **Computational basis is canonical** in quantum information theory
2. **Classical limit** is obvious: diagonal elements correspond to classical
3. **Explicit finite dimension** makes many proofs simpler
4. **Connection to matrices** is straightforward

The trade-off is that we don't get basis-independent formulations, but this is usual dimensional quantum information theory.

---

## Mixed States: MState

### Definition

**MState**  $d$  is a mixed quantum state (density matrix):

```

`lean
structure MState (d : Type*) [Fintype d] [DecidableEq d] where
 M : HermitianMat d ℂ
 zero_le : 0 ≤ M
 tr : M.trace = 1
`

```

**Interpretation:** A positive semidefinite Hermitian matrix with trace 1. Represent

**Key fields:**

- **M**: The underlying `HermitianMat d ℂ`
- **m**: The underlying `Matrix d d ℂ` (lowercase)
- **zero\_le**: Proof that `M` is PSD (equivalently, all eigenvalues  $\geq 0$ )
- **tr**: Proof that `trace(M) = 1`

### Constructing MStates

**From pure states:**

```

`lean
def pure (ψ : Ket d) : MState d
`

```

Creates the rank-1 projector  $|\psi\rangle\langle\psi|$ .

**Properties:**

- `pure ψ` is PSD with a single non-zero eigenvalue (equal to 1)
- `(pure ψ).m = vecMulVec ψ (ψ : Bra d)`
- `(pure ψ).purity = 1`

**From classical distributions:**

```

`lean
def ofClassical (dist : Distribution d) : MState d
`

```

Embeds a probability distribution as a diagonal density matrix.

```

Properties:
- `(ofClassical dist).m = diagonal (fun i => dist i)`
- Represents classical mixture with no quantum coherence
- Commutes with all matrices in the computational basis

Uniform state:
```lean
def uniform : MState d := ofClassical Distribution.uniform
```

The maximally mixed state `I/d`.

Properties:
- `uniform.m = (1/d) • 1`
- Maximum entropy: `Sv n uniform = log d`
- Minimum purity: `purity uniform = 1/d`

Basic Properties

Spectrum:
```lean
def spectrum (ρ : MState d) : Distribution d
```

The eigenvalues of `ρ`, viewed as a probability distribution.

Key facts:
- Eigenvalues are non-negative (from `zero_le`)
- Eigenvalues sum to 1 (from `tr`)
- Entropy can be computed from spectrum: `Sv n ρ = Hs (ρ.spectrum)`

Purity:
```lean
def purity (ρ : MState d) : Prob := [ρ, ρ]
```

Measures how "pure" a state is:
- `purity ρ = 1` iff `ρ` is a pure state
- `purity ρ = 1/d` for the uniform state
- Equivalently: `Tr[ρ²]`

Eigenvalue bounds:
- **`eigenvalue_nonneg`**: All eigenvalues are non-negative
- **`eigenvalue_le_one`**: All eigenvalues are at most 1
- **`le_one`**: `ρ.M ≤ 1` in the Loewner order

Tensor Products

`prod` (`ρ1 ⊗M ρ2`):

```



For  $\rho_1 : \text{MState } d_1$  and  $\rho_2 : \text{MState } d_2$ , gives  $\text{MState } (d_1 \times d_2)$ .

```

'''lean
def prod (ρ₁ : MState d₁) (ρ₂ : MState d₂) : MState (d₁ × d₂)
'''

Properties:
- $(\rho_1 \otimes^M \rho_2).m = \rho_1.m \otimes_k \rho_2.m$ (Kronecker product of matrices)
- Product of pure states: $\text{pure } \psi_1 \otimes^M \text{pure } \psi_2 = \text{pure } (\psi_1 \otimes \psi_2)$
- $\text{prod_inner_prod} : \langle \rho_1 \otimes^M \rho_2, \sigma_1 \otimes^M \sigma_2 \rangle = \langle \rho_1, \sigma_1 \rangle * \langle \rho_2, \sigma_2 \rangle$

```

```

n-fold products:
'''lean
def npow (ρ : MState d) (n : ℕ) : MState (Fin n → d)
'''

```

Notation:  $\rho \otimes^M n$

Creates  $n$  identical copies on the product space.

### Partial Traces

For bipartite states  $\rho : \text{MState } (d_1 \times d_2)$ :

```

`traceLeft ρ : MState d₂`: Reduces to second subsystem
'''lean
def traceLeft (ρ : MState (d₁ × d₂)) : MState d₂
'''

```

Mathematically:  $\rho_B = \text{Tr}_A[\rho_{AB}] = \sum_i \langle i | \rho | i \rangle_A$  where the trace is over the first subsystem

```

`traceRight ρ : MState d₁`: Reduces to first subsystem
'''lean
def traceRight (ρ : MState (d₁ × d₂)) : MState d₁
'''

```

Mathematically:  $\rho_A = \text{Tr}_B[\rho_{AB}]$

```

Properties:
- **Linearity**: Both are linear maps
- **Composition**: $\rho.\text{traceLeft}.\text{traceLeft}$ makes sense for tripartite systems
- **With SWAP**: $\rho.\text{SWAP}.\text{traceLeft} = \rho.\text{traceRight}$

```

### Subsystem Manipulation

```

SWAP:
'''lean
def SWAP (ρ : MState (d₁ × d₂)) : MState (d₂ × d₁) :=
 ρ.relabel (Equiv.prodComm d₁ d₂).symm

```

```
```
```

Exchanges the two subsystems.

```
**Properties:**
- **`SWAP_SWAP`**:  $\rho.\text{SWAP}.\text{SWAP} = \rho$ 
- **`traceLeft_SWAP`**:  $\rho.\text{SWAP}.\text{traceLeft} = \rho.\text{traceRight}$ 
- **`spectrum_SWAP`**: Eigenvalues preserved (up to relabeling)
```

```
**Associators:**
```lean
def assoc (ρ : MState ((d1 × d2) × d3)) : MState (d1 × d2 × d3)
def assoc' (ρ : MState (d1 × d2 × d3)) : MState ((d1 × d2) × d3)
```
```

```
**Properties:**
- **`assoc_assoc`**:  $\rho.\text{assoc}.\text{assoc} = \rho$ 
- **`assoc'_assoc`**:  $\rho.\text{assoc}.\text{assoc}' = \rho$ 
- **`traceRight_assoc`**:  $\rho.\text{assoc}.\text{traceRight} = \rho.\text{traceRight}.\text{traceRight}$ 
```

Purification

```
**`purify (ρ : MState d) : Ket (d × d)`**:
```

Produces a pure state $|\Psi\rangle$ on an enlarged space such that tracing out half

```
```lean
def purify (ρ : MState d) : Ket (d × d)
```
```

```
**Key theorem:**
- **`traceRight_of_purify`**:  $(\text{pure } \rho.\text{purify}).\text{traceRight} = \rho$ 
```

This is the **purification theorem**: every mixed state has a purification.

Unitary Conjugation

```
**`U_conj (ρ : MState d) (U : Matrix.unitaryGroup d ℂ) : MState d`**:
```

Applies unitary transformation: $\rho \mapsto U \rho U^\dagger$.

```
**Notation:**  $U \triangleleft \rho$  (scoped to MState`)
```

```
**Properties:**
- Preserves all eigenvalues
- Preserves entropy:  $S_{v_n}(U \triangleleft \rho) = S_{v_n} \rho$ 
```

- Preserves purity: $\text{purity}(U \triangleleft \rho) = \text{purity } \rho$

Relabeling

`relabel` ($\rho : \text{MState } d_1$) ($e : d_2 \approx d_1$) : $\text{MState } d_2$

Changes the index type via an equivalence. The underlying matrix gets permuted.

Properties:

- **`spectrum_relabel_eq`**: Multiset of eigenvalues preserved
- **`relabel_kron`**: Commutes with tensor products (appropriately)

Topology and Convexity

Topological structure:

- **`CompactSpace`** ($\text{MState } d$): The space of states is compact
- **`MetricSpace`** ($\text{MState } d$): Inherits metric from Hermitian matrices
- **`BoundedSpace`** ($\text{MState } d$): All states are within bounded distance

Convexity:

- **`instMixable`**: MStates have a `Mixable` instance
- **`convex`**: The set of MStates is convex in the ambient space

Interpretation: Quantum states form a compact convex set (the **Bloch ball** in

Matrix Maps and Quantum Channels

MatrixMap: The Base Type

`lean`

`abbrev MatrixMap (A B R : Type*) [Semiring R] := Matrix A A R →l [R] Matrix B B R`

A `MatrixMap A B R` is a linear map from `A×A` matrices to `B×B` matrices over `R`.

Philosophy: We view quantum channels as linear maps on the space of matrices, not

Choi Matrix Representation

Every matrix map has two representations:

1. **As a linear map** (the definition)
2. **Via the Choi matrix**

```

**`choi_matrix (M : MatrixMap A B R) : Matrix (B × A) (B × A) R`**:
```lean
def choi_matrix (M : MatrixMap A B R) : Matrix (B × A) (B × A) R :=
 fun (j₁,i₁) (j₂,i₂) ↦ M (Matrix.single i₁ i₂ 1) j₁ j₂
```

```

Interpretation: Apply the map to all elementary matrices and collect results.

```

**`of_choi_matrix (M : Matrix (B × A) (B × A) R) : MatrixMap A B R`**:

```

Inverse operation: given a matrix, construct the corresponding map.

Key theorems:

- **`choi\_map\_inv`**: $\text{of_choi_matrix}(\text{choi_matrix } M) = M$
- **`map\_choi\_inv`**: $\text{choi_matrix}(\text{of_choi_matrix } M) = M$
- **`choi\_equiv`**: These form a linear equivalence

Transfer matrix: There's also a ``toMatrix`` function giving the "transfer matrix".

Kraus Representation

```

**`of_kraus (M N : κ → Matrix B A R)`**:

```

Constructs the map $X \mapsto \sum_k M_k * X * N_k^H$.

Key fact: Every CPTP map has a Kraus representation.

The Hierarchy: Unbundled Properties

These are **propositions** about matrix maps:

1. Trace Preserving:

```

```lean
def IsTracePreserving (M : MatrixMap A B R) : Prop :=
 ∀ (x : Matrix A A R), (M x).trace = x.trace
```

```

Equivalent characterization:

- **`IsTracePreserving\_iff\_trace\_choi`**: $M.\text{IsTracePreserving} \leftrightarrow M.\text{choi_matrix}.\text{IsTracePreserving}$

Key theorems:

- **`IsTracePreserving.comp`**: Composition preserves TP
- **`IsTracePreserving.kron`**: Tensor products preserve TP
- **`IsTracePreserving.of\_kraus\_isTracePreserving`**: Kraus form is TP iff

2. Unital:

```

```lean
def Unital (M : MatrixMap A B R) : Prop := M 1 = 1
```

```

Maps identity to identity.

```

**3. Hermitian Preserving:**
```lean
def IsHermitianPreserving (M : MatrixMap A B $\square$) : Prop :=
 $\forall \{x\}, x.IsHermitian \rightarrow (M x).IsHermitian$
```

```

```

**4. Positive:**
```lean
def IsPositive (M : MatrixMap A B $\square$) : Prop :=
 $\forall \{x\}, x.PosSemidef \rightarrow (M x).PosSemidef$
```

```

Maps PSD matrices to PSD matrices.

```

**5. Completely Positive:**
```lean
def IsCompletelyPositive (M : MatrixMap A B $\square$) : Prop :=
 $\forall (n : \mathbb{N}), (M \otimes_{k m} MatrixMap.id (Fin n) \square).IsPositive$
```

```

Positive even when tensored with identity maps.

```

**Key theorem:**
- **`choi_PSD_iff_CP_map`** (Choi's Theorem): `M.IsCompletelyPositive  $\leftrightarrow$  M.choi_matrix.IsPositive`

```

The Hierarchy: Bundled Structures

These **bundle** a map with proofs of properties:

```

**1. HPMMap (Hermitian Preserving):**
```lean
structure HPMMap extends MatrixMap dIn dOut $\square$ where
 HP : MatrixMap.IsHermitianPreserving toLinearMap
```

```

```

- Has `FunLike` instance: can apply to `HermitianMat`
- **`funext_mstate`**: Two HP maps are equal if they agree on all `MState`s

```

```

**2. TPMMap (Trace Preserving):**
```lean

```

```

structure TMap extends MatrixMap dIn dOut R where
 TP : MatrixMap.IsTracePreserving toLinearMap
```

```

```

**3. UnitalMap:**
```lean

```

```

structure UnitalMap extends MatrixMap dIn dOut R where
 unital : MatrixMap.Unital toLinearMap
```

```

```

**4. PMap (Positive):**
```lean

```

```

structure PMap extends HMap dIn dOut $\mathbb{R}$ where
 pos : MatrixMap.IsPositive toLinearMap
```

```

Note: Extends `HMap` (positive maps are automatically Hermitian preserving)

```

**5. CMap (Completely Positive):**
```lean

```

```

structure CMap extends PMap dIn dOut $\mathbb{R}$ where
 cp : MatrixMap.IsCompletelyPositive toLinearMap
```

```

```

**6. PTPMap (Positive Trace Preserving):**
```lean

```

```

structure PTPMap extends PMap dIn dOut $\mathbb{R}$, TMap dIn dOut $\mathbb{R}$
```

```

Combines positivity and trace preservation.

```

**7. PUMap (Positive Unital):**
```lean

```

```

structure PUMap extends PMap dIn dOut $\mathbb{R}$, UnitalMap dIn dOut $\mathbb{R}$
```

```

Dual notion to PTP.

```

**8. CPTPMap (Completely Positive Trace Preserving):**
```lean

```

```

structure CPTPMap extends PTPMap dIn dOut, CMap dIn dOut where
```

```

The main object: Quantum channels. Over $\mathbb{C}$ by default.

```

**9. CPUMap (Completely Positive Unital):**

```

```

```lean
structure CUPMap extends CMap dIn dOut [], PUMap dIn dOut []
```

```

Dual to CPTP: maps observables (Hermitian matrices) in the opposite direction.

CPTPMap: Quantum Channels

CPTPMap $dIn\ dOut$ represents a quantum channel.

Key operations:

- **id** : CPTPMap $dIn\ dIn$: Identity channel
- **compose** ($M_2 \circ_m M_1$): Sequential composition
- **prod** ($M_1 \otimes_k M_2$): Parallel composition (tensor product)
- **CPTP_of_choi_PSD_Tr**: Construct from Choi matrix

Important examples:

- **of_unitary** (U : Matrix.unitaryGroup $d\ \mathbb{C}$): The unitary channel $\rho \mapsto U \rho U^\dagger$
- **replacement** (σ : MState $dOut$): Constant channel outputting σ
- **SWAP** : CPTPMap $(d_1 \times d_2)\ (d_2 \times d_1)$: Swaps subsystems
- **assoc** : CPTPMap $((d_1 \times d_2) \times d_3)\ (d_1 \times d_2 \times d_3)$: Reassociates
- **traceLeft** : CPTPMap $(d_1 \times d_2)\ d_2$: Partial trace as a channel
- **traceRight** : CPTPMap $(d_1 \times d_2)\ d_1$: Partial trace as a channel

Function-like behavior:

- Has `FunLike` instance when `DecidableEq dOut`
- Can apply to states: $\forall \rho : \text{MState } dOut$ for $\rho : \text{MState } dIn$

Extensionality:

- **funext**: Two channels are equal if they agree on all states

State-Channel Correspondence:

- **choi_MState_iff_CPTP**: Bijection between `CPTPMap  $dIn\ dOut$` and `MState ( $dIn\ dOut$ )`
- This is the **Choi-Jamiołkowski isomorphism**

Mixability:

- **instMixable**: Channels are convex (via Choi matrices)
- Can take convex combinations of channels

Duality

dual (M : MatrixMap $A\ B$ []) : MatrixMap $B\ A$ []:

The adjoint map with respect to the Hilbert-Schmidt inner product.

Defining property: $\text{Tr}[M(X) * Y] = \text{Tr}[X * M.\text{dual}(Y)]$

****Key theorems:****

- ****`IsCompletelyPositive.dual`****: CP maps have CP duals
- ****`dual\_kron`****: $(M \otimes_{k,m} N).dual = M.dual \otimes_{k,m} N.dual$
- ****`dual\_id`****: $(MatrixMap.id A \square).dual = MatrixMap.id A \square$

****Interpretation:**** The dual map acts on observables in the "backwards" direction

Entropy and Information Theory

Von Neumann Entropy

```lean

def S<sub>vn</sub> (ρ : MState d) : ℝ := H<sub>s</sub> (ρ.spectrum)

```

The von Neumann entropy, defined via the Shannon entropy of the eigenvalue

```

$$S(\rho) = -\sum_i \lambda_i \log \lambda_i$$

```

****Properties:****

- ****`S<sub>vn</sub>\_nonneg`****: $0 \leq S_{vn} \rho$
- ****`S<sub>vn</sub>\_le\_log\_d`****: $S_{vn} \rho \leq \log(d)$ with equality for uniform state
- ****Concavity:**** Von Neumann entropy is concave
- ****Unitary invariance:**** $S_{vn}(U \triangleleft \rho) = S_{vn} \rho$
- ****Subadditivity:**** $S_{vn} \rho \leq S_{vn} \rho.traceLeft + S_{vn} \rho.traceRight$

Conditional Entropy

```lean

def qConditionalEnt (ρ : MState (dA × dB)) : ℝ :=  
  S<sub>vn</sub> ρ - S<sub>vn</sub> ρ.traceLeft

```

Quantum conditional entropy: $S(A|B) = S(\rho_{AB}) - S(\rho_B)$.

****Key difference from classical:**** Can be ****negative**** (indicates entanglement)

****Interpretation:****

- Negative conditional entropy means the joint system has less uncertainty
- This is impossible classically but common for entangled quantum states

Mutual Information


```

```lean
def qMutualInfo (ρ : MState (dA × dB)) : ℝ :=
 Svn ρ.traceLeft + Svn ρ.traceRight - Svn ρ
```

```

Quantum mutual information: $I(A:B) = S(\rho_A) + S(\rho_B) - S(\rho_{AB})$.

Properties:

- **Non-negativity:** $0 \leq qMutualInfo \rho$
- **Symmetry:** $qMutualInfo \rho = qMutualInfo \rho.SWAP$
- Measures total correlations (classical + quantum)

Relationship to relative entropy:

- **$qMutualInfo_as_qRelativeEnt$:** $I(A:B) = D(\rho_{AB} \parallel \rho_A \otimes \rho_B)$

Conditional Mutual Information

```

```lean
def qcmi (ρ : MState (dA × dB × dC)) : ℝ :=
 qConditionalEnt ρ.assoc'.traceRight - qConditionalEnt ρ
```

```

Quantum conditional mutual information: $I(A;C|B) = S(A|B) - S(A|BC)$.

Physical meaning: Information about A contained in C , given knowledge of B .

Key theorem:

- **$S_{vn_weak_monotonicity}$:** Related to strong subadditivity

Coherent Information

```

```lean
def coherentInfo (ρ : MState d1) (Λ : CPTPMap d1 d2) : ℝ :=
 -qConditionalEnt (Λ.prod CPTPMap.id (pure ρ.purify))
```

```

The coherent information of ρ through channel Λ .

Physical meaning: Maximum rate at which quantum information can be transmitted through the channel.

Connection to capacity: Related to the quantum capacity of the channel.

Relative Entropy

```

```lean

```

```
def qRelativeEnt (ρ σ : MState d) : EReal := D̃_1(ρ||σ)
\`\`
```

**\*\*Notation:\*\***  $\mathbb{D}(\rho||\sigma)$

The quantum relative entropy (also called quantum Kullback-Leibler divergence):

$$\mathbb{D}(\rho||\sigma) = \text{Tr}[\rho (\log \rho - \log \sigma)]$$

When supports are compatible, otherwise  $+\infty$ .

**\*\*Properties:\*\***

- **\*\*Non-negativity:\*\***  $\mathbb{D}(\rho||\sigma) \geq 0$  with equality iff  $\rho = \sigma$  (quantum Pinsker's inequality)
- **\*\*`qRelativeEnt\_additive`\*\*:**  $\mathbb{D}(\rho_1 \otimes \rho_2 || \sigma_1 \otimes \sigma_2) = \mathbb{D}(\rho_1 || \sigma_1) + \mathbb{D}(\rho_2 || \sigma_2)$
- **\*\*`qRelativeEnt\_joint\_convexity`\*\*:** Jointly convex in  $(\rho, \sigma)$
- **\*\*Data processing inequality:\*\***  $\mathbb{D}(\Lambda \rho || \Lambda \sigma) \leq \mathbb{D}(\rho || \sigma)$  for any CPTP  $\Lambda$

**\*\*Relationship to mutual information:\*\***

Mutual information is relative entropy to the product state:

$$I(A:B) = \mathbb{D}(\rho_{AB} || \rho_A \otimes \rho_B)$$

### ### Sandwiched Rényi Relative Entropy

```
\`\`lean
```

```
def SandwichedRelRentropy (α : ℝ) (ρ σ : MState d) : EReal
\`\`
```

**\*\*Notation:\*\***  $\tilde{D}_\alpha(\rho||\sigma)$

A family of relative entropies parameterized by  $\alpha \in [0, \infty]$ :

$$\tilde{D}_\alpha(\rho||\sigma) = (1/(\alpha-1)) \log \text{Tr}[(\sigma^{((1-\alpha)/(2\alpha))} \rho \sigma^{((1-\alpha)/(2\alpha))})^\alpha]$$

**\*\*Special cases:\*\***

- $\alpha = 1$ : Quantum relative entropy  $\mathbb{D}(\rho||\sigma)$  (defined as limit)
- $\alpha = 1/2$ : Related to fidelity
- $\alpha \rightarrow \infty$ : Min-relative entropy

**\*\*Properties:\*\***

- **\*\*`sandwichedRelRentropy\_additive`\*\*:**  $\tilde{D}_\alpha(\rho_1 \otimes \rho_2 || \sigma_1 \otimes \sigma_2) = \tilde{D}_\alpha(\rho_1 || \sigma_1) + \tilde{D}_\alpha(\rho_2 || \sigma_2)$
- Monotone in  $\alpha$

- Data processing inequality

---

## ## Named Subsystems and Permutations

### ### The Subsystem Problem

**\*\*The issue:\*\*** In standard quantum information notation, we write:

- $\rho_{\{ABC\}}$  for a tripartite state
- $\rho_{\{AC\}}$  after tracing out  $B$
- $I(A:C|B)$  for conditional mutual information

This treats subsystems by **\*\*name\*\***.

**\*\*In Lean:\*\*** A type like  $\text{MState } (dA \times dB \times dC)$  is actually  $\text{MState } (dA \times (dB \times dC))$  associativity of  $\times$ .

**\*\*Consequence:\*\*** Partial traces act on the outermost split. We can directly talk about

- The partition  $dA$  vs  $(dB \times dC)$
- But **\*\*not\*\*** directly about  $dA \times dC$  vs  $dB$

### ### Solution: Explicit Permutations

To access different bipartitions, we use explicit permutation functions.

**\*\*SWAP:\*\***

```lean

```
def SWAP (p : MState (d1 × d2)) : MState (d2 × d1) :=
  p.relabel (Equiv.prodComm d1 d2).symm
```

Exchanges two subsystems.

****Properties:****

- ****`SWAP\_SWAP`****: $\rho.\text{SWAP}.\text{SWAP} = \rho$ (involutive)
- ****`traceLeft\_SWAP`****: $\rho.\text{SWAP}.\text{traceLeft} = \rho.\text{traceRight}$
- ****`traceRight\_SWAP`****: $\rho.\text{SWAP}.\text{traceRight} = \rho.\text{traceLeft}$
- ****`spectrum\_SWAP`****: Eigenvalues preserved up to relabeling

****Associators:****

```lean

```
def assoc (p : MState ((d1 × d2) × d3)) : MState (d1 × d2 × d3) :=
 p.relabel (Equiv.prodAssoc d1 d2 d3).symm
```

```
def assoc' (p : MState (d1 × d2 × d3)) : MState ((d1 × d2) × d3) :=
```

```

 ρ.SWAP.assoc.SWAP.assoc.SWAP
 \,\,

```

Change the grouping of a tripartite system.

**\*\*Properties:\*\***

```

- **`assoc_assoc`**: `ρ.assoc'.assoc = ρ`
- **`assoc'_assoc`**: `ρ.assoc.assoc' = ρ`
- **`traceRight_assoc`**: `ρ.assoc.traceRight = ρ.traceRight.traceRight`
- **`traceLeft_assoc`**: `ρ.assoc'.traceLeft = ρ.traceLeft.traceLeft`
- **`traceLeft_right_assoc`**: `ρ.assoc.traceLeft.traceRight = ρ.traceRight`

```

**### Example: Computing  $S(A|C)$**

**\*\*Setup:\*\*** We have  $\rho : \text{MState } (d_A \times d_B \times d_C)$  which is really  $d_A \times (d_B \times d_C)$

**\*\*Goal:\*\*** Compute  $S(A|C)$  = conditional entropy of  $A$  given  $C$ , ignoring  $B$

**\*\*Challenge:\*\*** We need access to the bipartition  $(d_A \times d_C)$  vs  $d_B$ , but only  $d_A \times d_B \times d_C$  is available

**\*\*Method:\*\***

1. Use `SWAP` and `assoc` to rearrange: move subsystems so  $A$  and  $C$  are adjacent
2. Apply partial traces to get  $\rho_{AC}$
3. Compute conditional entropy on the result

**\*\*Current state:\*\*** Entropy inequalities explicitly use these permutations  $\rho_{AC}$

**\*\*Future direction:\*\*** A more flexible "named subsystems" API is desired but not yet implemented

**### Channel Versions**

All permutations exist as CPTPMap channels:

```

- **`CPTPMap.SWAP` : CPTPMap $(d_1 \times d_2) (d_2 \times d_1)$
- **`CPTPMap.assoc` : CPTPMap $((d_1 \times d_2) \times d_3) (d_1 \times d_2 \times d_3)$
- **`CPTPMap.assoc'` : CPTPMap $(d_1 \times d_2 \times d_3) ((d_1 \times d_2) \times d_3)$
- **`CPTPMap.traceLeft` : CPTPMap $(d_1 \times d_2) d_2$
- **`CPTPMap.traceRight` : CPTPMap $(d_1 \times d_2) d_1$

```

**\*\*Consistency:\*\*** These satisfy  $\text{CPTPMap.SWAP } \rho = \rho.\text{SWAP}$ , etc.

---

**## Distances and Fidelity**

**### Trace Distance**

```

```lean
def TrDistance (ρ σ : MState d) : ℝ :=
  (1/2) * (ρ.m - σ.m).traceNorm
```

```

The **trace distance** between states:  $D(\rho, \sigma) = (1/2) \|\rho - \sigma\|_1$ .

**Interpretation:** The trace norm  $\|A\|_1 = \text{Tr}[\sqrt{A^\dagger A}]$  is the sum of singular values

**Properties:**

- **Bounds:**  $0 \leq \text{TrDistance } \rho \sigma \leq 1$
- **TrDistance.prob:** Can be bundled as a `Prob`
- **Metric:** Forms a metric on `MState d`
- **Contractivity:**  $\text{TrDistance } (\Lambda \rho) (\Lambda \sigma) \leq \text{TrDistance } \rho \sigma$  for CPTP  $\Lambda$ 
  - Channels cannot increase distinguishability

**Physical meaning:** Optimal success probability in distinguishing  $\rho$  from  $\sigma$  using

### ### Fidelity

```

```lean
def Fidelity (ρ σ : MState d) : ℝ
```

```

The **fidelity** between states:  $F(\rho, \sigma) = \text{Tr}[\sqrt{\sqrt{\rho} \sigma \sqrt{\rho}}]$ .

**Alternative definition (via purifications):** Maximum overlap between purifications

**Properties:**

- **Bounds:**  $0 \leq \text{Fidelity } \rho \sigma \leq 1$
- **Fidelity.prob:** Can be bundled as a `Prob`
- **Symmetry:**  $\text{Fidelity } \rho \sigma = \text{Fidelity } \sigma \rho$
- **Monotonicity:**  $\text{Fidelity } (\Lambda \rho) (\Lambda \sigma) \geq \text{Fidelity } \rho \sigma$  for CPTP  $\Lambda$ 
  - Opposite direction from trace distance!

**Physical meaning:** Measures "closeness" of quantum states, with different operati

**Uhlmann's theorem:** The fidelity is achieved by optimal choice of purifications.

### ### Relationships

The trace distance and fidelity are related by various inequalities:

**Fuchs-van de Graaf inequalities:**

```

```

```

$$1 - F(\rho, \sigma) \leq D(\rho, \sigma) \leq \sqrt{1 - F(\rho, \sigma)^2}$$

These relate the two fundamental distance measures in quantum information t

Kronecker Products

Overview

Kronecker products (tensor products) are fundamental in quantum information

****Key convention:**** The product of types ``d1`` and ``d2`` has type ``d1 × d2`` (

Product Notations by Type

Type	Notation	Input Types	Output Type	Scope
Matrix	<code>`⊗<sub>k</sub>`</code>	<code>`Matrix A B R`</code> , <code>`Matrix C D R`</code>	<code>`Matrix (A×B) (C×D)`</code>	<code>a</code>
Ket	<code>`⊗`</code>	<code>`Ket d1`</code> , <code>`Ket d2`</code>	<code>`Ket (d1 × d2)`</code>	default
MState	<code>`⊗<sup>M</sup>`</code>	<code>`MState d1`</code> , <code>`MState d2`</code>	<code>`MState (d1 × d2)`</code>	default
HermitianMat	<code>`⊗<sub>k</sub>`</code>	<code>`HermitianMat m`</code> , <code>`HermitianMat n`</code>	<code>`HermitianMat (m × n)`</code>	
CPTPMap	<code>`⊗<sup>C P</sup>`</code>	<code>`CPTPMap dIn1 dOut1`</code> , <code>`CPTPMap dIn2 dOut2`</code>	<code>`CPTPMap (dIn1 × dIn2) (dOut1 × dOut2)`</code>	
MatrixMap	<code>`⊗<sub>k m</sub>`</code>	<code>`MatrixMap A B R`</code> , <code>`MatrixMap C D R`</code>	<code>`MatrixMap (A × C) (B × D)`</code>	
Unitary	<code>`⊗<sub>u</sub>`</code>	Unitary, Unitary	Unitary on product	<code>`Matrix`</code>
Categorical	<code>`⊗<sub>r</sub>`</code>	<code>`MState (H i)`</code> , <code>`MState (H j)`</code>	<code>`MState (H (i × j))`</code>	

The first of these, for bare matrices, comes from Mathlib. The last of these

****Why different notations?****

- Avoids ambiguity when multiple products are in scope
- Makes code more readable
- Each notation is tailored to its domain

Properties

****Linearity:**** All products are (bi)linear.

****Associativity:**** Products are associative up to canonical isomorphism:

$$(\rho_1 \otimes^M \rho_2) \otimes^M \rho_3 \cong \rho_1 \otimes^M (\rho_2 \otimes^M \rho_3)$$

But they live in different types: ``((d1 × d2) × d3)`` vs ``(d1 × (d2 × d3))``.

****For pure states:****

```

\ \ \
pure  $\psi_1 \otimes^M \text{pure } \psi_2 = \text{pure } (\psi_1 \otimes \psi_2)$ 
\ \ \

**For channels:**
\ \ \
 $(\Lambda_1 \otimes_k \Lambda_2) (\rho_1 \otimes^M \rho_2) = (\Lambda_1 \rho_1) \otimes^M (\Lambda_2 \rho_2)$ 
\ \ \

### n-fold Products

**For MState:**
\ \ \lean
def npow ( $\rho : \text{MState } d$ ) ( $n : \mathbb{N}$ ) :  $\text{MState } (\text{Fin } n \rightarrow d)$ 
\ \ \

**Notation:**  $\rho \otimes^M n$ 

Creates  $n$  identical copies on the type  $\text{Fin } n \rightarrow d$ .

**Example:**  $\rho \otimes^M 3$  represents three copies of  $\rho$ .

### Key Theorems Involving Products

**Logarithm:**
-  $\text{log\_kron}$ :  $(A \otimes_k B).log = A.log \otimes_k 1 + 1 \otimes_k B.log$ 

**Trace preservation:**
-  $\text{IsTracePreserving.kron}$ : Tensor products of TP maps are TP

**Relative entropy:**
-  $\text{qRelativeEnt\_additive}$ : Relative entropy is additive on product states

**Duality:**
-  $\text{dual\_kron}$ :  $(M \otimes_{k\ m} N).dual = M.dual \otimes_{k\ m} N.dual$ 

---

# Part III: Reference

## Notations and Scopes

### Quantum States



| Notation       | Meaning    | Type            | Scope  |
|----------------|------------|-----------------|--------|
| $ \psi\rangle$ | Bra vector | $\text{Bra } d$ | Braket |


```

$ \psi\rangle$	Ket vector	<code>`Ket d`</code>	<code>`Braket`</code>
$\langle\psi \psi\rangle$	Inner product (bra-ket)	<code>`C`</code>	<code>`Braket`</code>
$\langle A, B \rangle$	Hilbert-Schmidt inner product	<code>`R`</code>	default
$ \psi_1\rangle \otimes \psi_2\rangle$	Ket tensor product	<code>`Ket (d1 x d2)`</code>	default
$\rho_1 \otimes^M \rho_2$	State tensor product	<code>`MState (d1 x d2)`</code>	default
$\rho \otimes^{M^n}$	n-fold state power	<code>`MState (Fin n -> d)`</code>	default

Hermitian Matrices

Notation	Meaning	Type	Scope
$A \otimes_k B$	Kronecker product	<code>`HermitianMat (m x n) C`</code>	<code>`HermitianMat`</code>
$\{A \leq_p B\}$	Projection onto $B-A \geq 0$ eigenspace	<code>`HermitianMat n C`</code>	
$\{A <_p B\}$	Projection onto $B-A > 0$ eigenspace	<code>`HermitianMat n C`</code>	
$\{A \geq_p B\}$	Same as $\{B \leq_p A\}$	<code>`HermitianMat n C`</code>	<code>`HermitianMat`</code>
$\{A >_p B\}$	Same as $\{B <_p A\}$	<code>`HermitianMat n C`</code>	<code>`HermitianMat`</code>
A^+	Positive part $\max(A, 0)$	<code>`HermitianMat d C`</code>	<code>`HermitianMat`</code>
A^-	Negative part $\max(-A, 0)$	<code>`HermitianMat d C`</code>	<code>`HermitianMat`</code>

Quantum Channels

Notation	Meaning	Type	Scope
$M_1 \otimes_k M_2$	Channel tensor product	<code>`CPTPMap ...`</code>	default
$M_1 \circ_m M_2$	Channel composition (M_1 after M_2)	<code>`CPTPMap dIn dOut`</code>	default
$U \triangleleft \rho$	Unitary conjugation $U \rho U^\dagger$	<code>`MState d`</code>	<code>`MState`</code>
$M \otimes_{k,m} N$	MatrixMap tensor product	<code>`MatrixMap ...`</code>	<code>`MatrixMap`</code>
$U[n]$	Unitary group $U(n)$	<code>`Matrix.unitaryGroup n C`</code>	default
$C[g]$	Controlled unitary gate	Qubit unitary	qubit-specific

Information Theory

Notation	Meaning	Value Type	Scope
$D(\rho \sigma)$	Quantum relative entropy	<code>`EReal`</code>	default
$\tilde{D}_\alpha(\rho \sigma)$	Sandwiched Rényi entropy (α -parametrized)	<code>`EReal`</code>	default
$\beta_\epsilon(\rho S)$	Optimal hypothesis test error rate	<code>`Prob`</code>	<code>`OptimalHypothesis`</code>

Resource Theory

Notation	Meaning	Type	Scope
$\rho_1 \otimes_r \rho_2$	Resource-theoretic product	<code>`MState (H (i * j))`</code>	resource
$M_1 \otimes^{C P_r} M_2$	RT channel product	<code>`CPTPMap ...`</code>	resource theory
$i \otimes^H[n]$	n-fold RT space power	Space type	resource theory

$\rho^{\otimes n}$	n-fold RT state power	$\text{MState}(H(i^n))$	resource theory
$\square_r$	Relative entropy resource measure	$\mathbb{R}$	resource theory
$\square_{r\infty}$	Regularized RE resource	$\mathbb{R}$	resource theory

Scope Usage

****To use scoped notations:****

```lean

open scoped HermitianMat -- enables  $\otimes_k$  for HermitianMat

open scoped MatrixMap -- enables  $\otimes_{k,m}$

open scoped Braket -- enables  $\square\psi\square$ ,  $\square\psi\square$ , etc.

```

****Default scope:**** Notations like $\otimes^M$, $\circ_m$ are always available.

Some Theorems

Operator Theory

1. ****`log\_mono`**** (Logarithm Monotonicity):
 For positive definite $A \leq B$, we have $A.\log \leq B.\log$.
 - The matrix logarithm is ****operator monotone****
2. ****`log\_concave`**** (Logarithm Concavity):
 For positive definite A, B and convex combination weights $a, b \geq 0$ with $a + b = 1$,

$$a \cdot A.\log + b \cdot B.\log \leq (a \cdot A + b \cdot B).\log$$

 - The matrix logarithm is ****operator concave****
3. ****`inv\_antitone`**** (Inverse Antitonicity):
 For positive definite $A \leq B$, we have $B^{-1} \leq A^{-1}$.
 - The matrix inverse is ****operator antitone****
4. ****`inv\_convex`**** (Inverse Convexity):
 For positive definite A, B and convex weights:

$$(a \cdot A + b \cdot B)^{-1} \leq a \cdot A^{-1} + b \cdot B^{-1}$$

 - The matrix inverse is ****operator convex****
5. ****`log\_kron`**** (Logarithm of Kronecker Product):
 For positive definite A, B :

$$(A \otimes_k B).log = A.log \otimes_k 1 + 1 \otimes_k B.log$$

Quantum Channels

7. **choi_PSD_iff_CP_map** (Choi's Theorem):
A map is completely positive iff its Choi matrix is positive semidefinite.
- This is the fundamental **Choi-Jamiołkowski** correspondence
8. **choi_MState_iff_CPTP** (State-Channel Correspondence):
CPTP maps from $\mathcal{H}_{In}$ to $\mathcal{H}_{Out}$ correspond bijectively to states on $\mathcal{H}_{In}$
9. **IsTracePreserving.of_kraus_isTracePreserving** (Kraus TP Criterion):
Kraus operators $\{M_k, N_k\}$ define a TP map iff $\sum_k N_k^\dagger M_k = I$.
10. **IsCompletelyPositive.dual** (Duality of CP Maps):
The dual of a completely positive map is completely positive.
11. **dual_kron** (Duality Preserves Kronecker Products):
 $(M \otimes_{k,m} N).dual = M.dual \otimes_{k,m} N.dual$

Information Theory

12. **qRelativeEnt_additive** (Additivity of Relative Entropy):
$$\mathbb{D}(\rho_1 \otimes^M \rho_2 \| \sigma_1 \otimes^M \sigma_2) = \mathbb{D}(\rho_1 \| \sigma_1) + \mathbb{D}(\rho_2 \| \sigma_2)$$
13. **qRelativeEnt_joint_convexity** (Joint Convexity):
Quantum relative entropy is jointly convex in both arguments.
14. **sandwichedRelRentropy_additive** (Rényi Additivity):
Sandwiched Rényi entropy is additive on product states for all α .
15. **S_vn_weak_monotonicity** (Weak Monotonicity):
Related to strong subadditivity of von Neumann entropy.
16. **qMutualInfo_as_qRelativeEnt** (Mutual Information as Relative Entropy):
$$I(A:B) = \mathbb{D}(\rho_{\{AB\}} \| \rho_A \otimes \rho_B)$$

Convexity and Order

17. **MState.convex** (States Form Convex Set):
The set of quantum states is convex in the ambient Hermitian matrix space.

18. `**`HermitianMat.convex_cone`**` (PSD Cone is Convex):
Non-negative combinations of PSD Hermitian matrices are PSD.

19. `**`Matrix.PosSemidef.convex_cone`**` (Matrix PSD Cone):
Same for plain matrices.

Monotonicity

20. `**`trace_mono`**` (Trace Monotonicity):
 $A \leq B \rightarrow A.\text{trace} \leq B.\text{trace}$ in Loewner order.

21. `**`conj_mono`**` (Conjugation Preserves Order):
 $A \leq B \rightarrow M * A * M^H \leq M * B * M^H$

22. `**`inner_mono`**` (Inner Product Monotonicity):
If $0 \leq A$ and $B \leq C$, then $\langle A, B \rangle \leq \langle A, C \rangle$.

23. `**`diagonal_mono`**` (Diagonal Monotonicity):
Componentwise order on diagonal entries corresponds to Loewner order.

24. `**`optimalHypothesisRate_antitone`**` (Data Processing for Hypothesis Testing):
Quantum channels cannot improve hypothesis testing performance.

Topology and Analysis

25. `**`CompactSpace (MState d)`**` (States are Compact):
The space of quantum states on a finite-dimensional system is compact.

26. `**`OrderClosedTopology (HermitianMat d)`**` (Order is Closed):
The Loewner order on Hermitian matrices is topologically closed.

27. `**`CompactIccSpace (HermitianMat d)`**` (Order Intervals Compact):
Order intervals $[A, B] = \{X \mid A \leq X \leq B\}$ are compact.

28. `**`tendsto_logApprox`**` (Logarithm Approximation Convergence):
For positive definite x , $\logApprox x T \rightarrow x.\log$ as $T \rightarrow \infty$.

Extensionality

29. `**`HPMap.funext_mstate`**` (Extensionality for HP Maps):
Two Hermitian-preserving maps are equal if they agree on all `MState`s`.

30. `**`CPTPMap.funext`**` (Extensionality for Channels):
Two CPTP maps are equal if they agree on all input states.

Resource Theory

31. **RelativeEntResource.Subadditive** (Relative Entropy Resource):
The relative entropy of resource is subadditive under tensor products.

Minimax Theory

32. **Sion's Minimax Theorem**:
Under appropriate compactness and quasi-convexity conditions, $\sup_x \inf_y$ in

1.488 QuantumInfo/Regularized.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.ForMathlib.Superadditive
import Mathlib.Order.LiminfLimsup
import Mathlib.Topology.Order.MonotoneConvergence

/-! Definition of "Regularized quantities" as are common in information theory
from one-shot versions, and good properties coming from Fekete's lemma.
-/

variable {T : Type*} [ConditionallyCompleteLattice T]

/-- An `InfRegularized` value is the  $\limsup$  of value at each natural number
a proof of lower- and upper-bounds to be defined. -/
noncomputable def InfRegularized (fn :  $\mathbb{N} \rightarrow T$ ) {lb ub : T}
  (_ :  $\forall n, lb \leq fn\ n$ ) (_ :  $\forall n, fn\ n \leq ub$ ) : T :=
  Filter.atTop.liminf fn

/-- A `SupRegularized` value is the  $\limsup$  of value at each natural number
a proof of lower- and upper-bounds to be defined. -/
noncomputable def SupRegularized (fn :  $\mathbb{N} \rightarrow T$ ) {lb ub : T}
  (_ :  $\forall n, lb \leq fn\ n$ ) (_ :  $\forall n, fn\ n \leq ub$ ) : T :=
  Filter.atTop.limsup fn

namespace InfRegularized

variable {fn :  $\mathbb{N} \rightarrow T$ } {_lb _ub : T} {hl :  $\forall n, _lb \leq fn\ n$ } {hu :  $\forall n, fn\ n \leq _ub$ }

/-- The `InfRegularized` value is also lower bounded. -/

```

```

theorem lb : _lb ≤ InfRegularized fn hl hu := by
  convert le_csSup ?_ ?_;
  · exact ( _ub, fun a ha => by rcases Filter.eventually_atTop.mp ha with ( n, hn ) )
  · aesop

/-- The `InfRegularized` value is also upper bounded. -/
theorem ub : InfRegularized fn hl hu ≤ _ub := by
  unfold InfRegularized;
  simp +decide [Filter.liminf_eq];
  refine' csSup_le _;
  · exact ( _, ( 0, fun n _ => hl n ) );
  · exact fun b hb => by rcases hb with ( n, hn ) ; exact le_trans ( hn n le_rfl ) (

/-- For `Antitone` functions, the `InfRegularized` is the supremum of values. -
/
theorem anti_inf (h : Antitone fn) :
  InfRegularized fn hl hu = sInf (Set.range fn) := by
  unfold InfRegularized; simp +decide [ Filter.liminf_eq ] ;
  rw [ @csSup_eq_of_forall_le_of_forall_lt_exists_gt ];
  · exact ( _, ( 0, fun n _ => hl n ) );
  · rintro a ( n, hn );
    exact le_csInf ( _, Set.mem_range_self n ) fun x hx => by rcases hx with ( m, rf
  · exact fun w hw => ( _, ( 0, fun n _ => csInf_le ( _lb, Set.forall_mem_range.2 hl

/-- For `Antitone` functions, the `InfRegularized` is lower bounded by
any particular value. -/
theorem anti_ub (h : Antitone fn) : ∀ n, InfRegularized fn hl hu ≤ fn n := by
  intro n
  have h_inf_le : InfRegularized fn hl hu ≤ fn n := by
    convert csSup_le _;
    · exact ( _lb, Filter.eventually_atTop.2 ( 0, fun n hn => hl n ) );
    · simp +zetaDelta at *;
      exact fun b x hx => le_trans ( hx ( Max.max x n ) ( le_max_left _ _ ) ) ( h (
    exact h_inf_le

end InfRegularized

namespace SupRegularized

variable {fn : ℕ → T} { _lb _ub : T} {hl : ∀ n, _lb ≤ fn n} {hu : ∀ n, fn n ≤ _ub}

/-- The `SupRegularized` value is also lower bounded. -/
theorem lb : _lb ≤ SupRegularized fn hl hu := by
  -- Suppose, for contradiction, that  $\mathrm{InfRegularized} \setminus; fn < \mathrm{SupRegularized} fn$ 
  by_contra h_contra;
  -- By definition of `SupRegularized`, we have that  $\mathrm{SupRegularized} \setminus; fn$ 

```

```

have h_sup_ge_inf : SupRegularized fn hl hu ≥ InfRegularized fn hl hu :=
  apply_rules [ Filter.liminf_le_limsup ];
  · exact ( _ub, Filter.eventually_atTop.2 ( 0, fun n hn => hu n ) );
  · exact ( _, Filter.eventually_atTop.2 ( 0, fun n hn => hl n ) );
exact h_contra ( le_trans ( by exact InfRegularized.lb ) h_sup_ge_inf )

/-- The `SupRegularized` value is also upper bounded. -/
theorem ub : SupRegularized fn hl hu ≤ _ub := by
  -- By definition of `limsup`, we know that for any `ε > 0`, there exists
  apply csInf_le;
  · exact ( _, fun x hx => hx.exists.choose_spec.trans' ( hl _ ) );
  · aesop

/-- For `Monotone` functions, the `SupRegularized` is the supremum of value
/
theorem mono_sup (h : Monotone fn) :
  SupRegularized fn hl hu = sSup { fn n | n : ℕ } := by
  unfold SupRegularized;
  simp +decide [ Filter.limsup_eq, Filter.eventually_atTop ];
  refine' le_antisymm _ _;
  · refine' csInf_le _ _;
    · exact ( _lb, by rintro x ( n, hn ) ; exact le_trans ( hl n ) ( hn n l
    · exact ( 0, fun n _ => le_csSup ( _ub, by rintro x ( m, rfl ) ; exact
  · refine' csSup_le _ _;
    · exact ( _, ( 0, rfl ) );
    · norm_num +zetaDelta at *;
      exact fun n => le_csInf ( _ub, ( n, fun m hm => hu m ) ) fun x hx =>

/-- For `Monotone` functions, the `SupRegularized` is lower bounded by
any particular value. -/
theorem mono_lb (h : Monotone fn) : ∀ n, fn n ≤ SupRegularized fn hl hu :=
  intro n
  unfold SupRegularized;
  refine' le_csInf _ _;
  · exact ( _ub, Filter.eventually_atTop.2 ( 0, fun n hn => hu n ) );
  · simp +zetaDelta at *;
    exact fun b m hm => le_trans ( h ( Nat.le_max_left _ _ ) ) ( hm _ ( Nat

end SupRegularized

section real

variable {fn : ℕ → ℝ} { _lb _ub : ℝ } {hl : ∀ n, _lb ≤ fn n} {hu : ∀ n, fn n

theorem InfRegularized.to_SupRegularized : InfRegularized fn hl hu = -
SupRegularized (-fn ·)

```

```

    (lb := -_ub) (ub := -_lb) (neg_le_neg_iff.mpr <| hu ·) (neg_le_neg_iff.mpr <| hl
sorry

theorem SupRegularized.to_InfRegularized : SupRegularized fn hl hu = -
InfRegularized (-fn ·)
  (lb := -_ub) (ub := -_lb) (neg_le_neg_iff.mpr <| hu ·) (neg_le_neg_iff.mpr <| hl
  -- By definition of `InfRegularized` and `SupRegularized`, we can rewrite the goal
  have h_limsup_limsup : Filter.limsup fn Filter.atTop = -
Filter.liminf (fun n => -fn n) Filter.atTop := by
  -- By definition of liminf and limsup, we know that for any sequence of real num
  have h_limsup_neg : ∀ (s : ℕ → ℝ), Filter.liminf (fun n => -
s n) Filter.atTop = -Filter.limsup s Filter.atTop := by
  -- Apply the definition of liminf and limsup.
  intros s
  rw [Filter.liminf_eq, Filter.limsup_eq];
  -- By definition of supremum and infimum, we know that the supremum of a set i
  have h_sup_neg_inf : ∀ (S : Set ℝ), sSup S = -sInf (-
S) := by
    intro S; rw [Real.sInf_def] ; aesop;
    convert h_sup_neg_inf _ using 3 ; aesop;
    rw [h_limsup_neg, neg_neg];
    exact h_limsup_limsup

/-- For `Antitone` functions, the value `Filter.Tendsto` the `InfRegularized` value.
/
theorem InfRegularized.anti_tendsto (h : Antitone fn) :
  Filter.Tendsto fn .atTop (nhds (InfRegularized fn hl hu)) := by
  convert tendsto_atTop_ciInf h { _lb, fun _ {a,b} => b > hl a }
  rw [InfRegularized.anti_inf h, iInf.eq_1]

variable {f₁ : ℕ → ℝ} { _lb _ub : ℝ} {hl : ∀ n, _lb ≤ fn n} {hu : ∀ n, fn n ≤ _ub}

theorem InfRegularized.of_Subadditive (hf : Subadditive (fun n => fn n * n))
:
  hf.lim = InfRegularized fn hl hu := by
  have h₁ := hf.tendsto_lim (by
    use min 0 _lb
    rw [mem_lowerBounds]
    rintro x ⟨y, (rfl : _ / _ = _)⟩
    rcases y with (_, n)
    · simp
    · rw [inf_le_iff]
      convert 0r.inr (hl (n+1))
      field_simp
  )
  apply tendsto_nhds_unique h₁

```

```
have := InfRegularized.anti_tendsto (fn := fn) (hl := hl) (hu := hu) (sorry)
sorry
```

1.489 QuantumInfo/StatMech/Hamiltonian.lean

```
/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import Mathlib.Data.Fintype.Defs
import Mathlib.Data.Real.Basic

section ThermodynamicEnsemble

/-- The Hamiltonian for a microcanonical ensemble, parameterized by any ext
Since it's an arbitrary type, 'T' would be nice, but we use 'D' for data
We define Hamiltonians as an energy function on a real manifold, whose di
Energy is a WithTop  $\mathbb{R}$ ,  $\tau$  is used to mean "excluded as a possibility in ph
This formalization does exclude the possibility of discrete degrees of fr
A better formalization could have an arbitrary measurable space.
-/
structure MicroHamiltonian (D : Type) where
  --For extrinsic parameters D (e.g. the number of particles of different c
  --how many continuous degrees of freedom are there?
  dim : D → Type
  --We require that the number of degrees of freedom is finite.
  [dim_fin : ∀ d, Fintype (dim d)]
  --Given the configuration, what is its energy?
  H : {d : D} → (dim d →  $\mathbb{R}$ ) → WithTop  $\mathbb{R}$ 

/-- We add the dim_fin to the instance cache so that things like the measur
/
instance microHamiltonianFintype (H : MicroHamiltonian D) (d : D) : Fintype
  H.dim_fin d

/-- The standard microcanonical ensemble Hamiltonian, where the data is the
/
abbrev NVEHamiltonian := MicroHamiltonian ( $\mathbb{N} \times \mathbb{R}$ )

/-- Helper to get the number in an N-V Hamiltonian -/
def NVEHamiltonian.N : ( $\mathbb{N} \times \mathbb{R}$ ) →  $\mathbb{N}$  := Prod.fst

/-- Helper to get the volume in an N-V Hamiltonian -/
```



```
def NVEHamiltonian.V : ( $\mathbb{N} \times \mathbb{R}$ )  $\rightarrow \mathbb{R}$  := Prod.snd
```

1.490 QuantumInfo/StatMech/IdealGas.lean

```
/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.StatMech.ThermoQuantities
import Mathlib.Analysis.SpecialFunctions.Gaussian.FourierTransform

noncomputable section

--! Specializing to an ideal gas of distinguishable particles.

/-- The Hamiltonian for an ideal gas: particles live in a cube of volume  $V^{(1/3)}$ , and
The per-particle mass is normalized to 1. -/
def IdealGas : NVEHamiltonian where
  --The dimension of the manifold is 6 times the number of particles: three for posi
  dim := fun (n,_)  $\mapsto$  Fin n  $\times$  (Fin 3  $\oplus$  Fin 3)
  --The energy is  $\infty$  if any positions are outside the cube, otherwise it's the sum of
  H := fun {d} config  $\mapsto$ 
    let (n,V) := d
    let R :=  $V^{(1/3:\mathbb{R})} / 2$  --half-sidelength of a cubical box
    if  $\forall$  (i : Fin n) (ax : Fin 3), |config (i,.inl ax)| <= R then
       $\sum$  (i : Fin n) (ax : Fin 3), config (i,.inr ax)^2 / (2 :  $\mathbb{R}$ )
    else
      T

namespace IdealGas
open MicroHamiltonian
open NVEHamiltonian

variable (n :  $\mathbb{N}$ ) {V  $\beta$  T :  $\mathbb{R}$ }

open MeasureTheory in
/-- The partition function Z for an ideal gas. -/
theorem PartitionZ_eq (hV : 0 < V) (h $\beta$  : 0 <  $\beta$ ) :
  IdealGas.PartitionZ (n,V)  $\beta$  =  $V^n * (2 * \text{Real.pi} / \beta)^{(3 * n / 2 : \mathbb{R})}$  := by
  rw [PartitionZ, IdealGas]
  simp only [Finset.univ_product_univ, one_div, ite_eq_right_iff, WithTop.sum_eq_top
    Finset.mem_univ, WithTop.coe_ne_top, and_false, exists_false, imp_false, not_for
    neg_mul]
```

```

have h₀ : ∀ (config : Fin n × (Fin 3 ⊕ Fin 3) → ℝ) proof,
  ((if ∀ (i : Fin n) (ax : Fin 3), |config (i, Sum.inl ax)| ≤ V ^ (3 :
    ∑ x : Fin n × Fin 3, config (x.1, Sum.inr x.2) ^ 2 / (2 :
    else 1) : WithTop ℝ).untop proof = ∑ x : Fin n × Fin 3, con
intro config proof
rw [WithTop.untop_eq_iff]
split_ifs with h
· simp
· simp [h] at proof
simp only [h₀, dite_eq_ite]; clear h₀

let eq_pm : MeasurableEquiv ((Fin n × Fin 3 → ℝ) × (Fin n × Fin 3 → ℝ)) (
  let e1 := (MeasurableEquiv.sumPiEquivProdPi (α := fun _ : (Fin n × Fin
  let e2 := (MeasurableEquiv.piCongrLeft _ (MeasurableEquiv.prodSumDistri
e1.symm.trans e2

have h_preserve : MeasurePreserving eq_pm := by
  unfold eq_pm
  -- fun_prop --this *should* be a fun_prop!
  rw [MeasurableEquiv.coe_trans]
  apply MeasureTheory.MeasurePreserving.comp (μb := by volume_tac)
  · apply MeasurePreserving.symm
    apply MeasureTheory.volume_measurePreserving_piCongrLeft
  · apply MeasurePreserving.symm
    apply measurePreserving_sumPiEquivProdPi
rw [← MeasurePreserving.integral_comp h_preserve eq_pm.measurableEmbeddin

rw [show volume = Measure.prod volume volume from rfl]

have h_eval_eq_pm : ∀ (x y i p_i), eq_pm (x, y) (i, Sum.inl p_i) = x (i,
  intros; rfl
have h_eval_eq_pm' : ∀ (x y i m_i), eq_pm (x, y) (i, Sum.inr m_i) = y (i,
  intros; rfl
simp_rw [h_eval_eq_pm, h_eval_eq_pm']
clear h_eval_eq_pm h_eval_eq_pm'

have h_measurable_box : Measurable fun (a : (Fin n × Fin 3 → ℝ))
  => ∃ x_1 x_2, V ^ (3⁻¹:ℝ) / 2 < |a (x_1, x_2)| := by
  simp_rw [← Classical.not_forall_not, not_not, not_lt, abs_le]
  apply Measurable.not
  apply Measurable.forall
  intro i
  apply Measurable.forall
  intro j
  refine Measurable.comp (measurableSet_setOf.mp measurableSet_Icc) (meas

```

```

have h_measurability : Measurable fun x : (Fin n × Fin 3 → ℝ) × (Fin n × Fin 3 → ℝ)
  if ∃ x_1 x_2, V ^ (3⁻¹:ℝ) / 2 < |x.1 (x_1, x_2)| then 0
  else Real.exp (-(β * ∑ x_1 : Fin n × Fin 3, x.2 (x_1.1, x_1.2) ^ 2 / 2)) := by
  apply Measurable.ite
  · simp_rw [measurableSet_setOf]
    convert Measurable.comp h_measurable_box measurable_fst
  · fun_prop
  · fun_prop

rw [MeasureTheory.integral_eq_lintegral_of_nonneg_ae]
rotate_left
· apply Filter.Eventually.of_forall
  intros
  positivity
· apply Measurable.aestronglyMeasurable
  fun_prop
rw [MeasureTheory.lintegral_prod]; swap
· exact (Measurable.comp (g := ENNReal.ofReal)
  ENNReal.measurable_ofReal h_measurability).ameasurable

conv =>
  enter [1, 1, 2, x, 2, y]
  rw [← ite_not _ _ (0:ℝ), ← boole_mul _ (Real.exp _)]
  rw [ENNReal.ofReal_mul (by split_ifs <=> positivity)]
dsimp
conv =>
  enter [1, 1, 2, x]
  rw [MeasureTheory.lintegral_const_mul' _ _ (ENNReal.ofReal_ne_top)]

rw [MeasureTheory.lintegral_mul_const, ENNReal.toReal_mul]
rw [← MeasureTheory.integral_eq_lintegral_of_nonneg_ae]
rw [← MeasureTheory.integral_eq_lintegral_of_nonneg_ae]
rotate_left
· apply Filter.Eventually.of_forall
  intros
  positivity
· apply Measurable.aestronglyMeasurable
  fun_prop
· apply Filter.Eventually.of_forall
  intros
  positivity
· apply Measurable.aestronglyMeasurable
  apply Measurable.ite
  · rw [measurableSet_setOf]
    apply Measurable.not

```

```

    exact h_measurable_box
  · fun_prop
  · fun_prop
  · apply Measurable.comp ENNReal.measurable_ofReal
    apply Measurable.ite
  · rw [measurableSet_setOf]
    apply Measurable.not
    exact h_measurable_box
  · fun_prop
  · fun_prop

congr 1
· --Volume of the box
  have h_integrand_prod : ∀ (a : Fin n × Fin 3 → ℝ),
    (if ¬∃ x y, V ^ (3⁻¹ : ℝ) / 2 < |a (x, y)| then 1 else 0) =
    (∏ xy, if |a xy| ≤ V ^ (3⁻¹ : ℝ) / 2 then 1 else 0 : ℝ) := by
    intro a
    push_neg
    simp_rw [← Prod.forall (p := fun xy ↦ |a xy| ≤ V ^ (3⁻¹ : ℝ) / 2)]
    exact Fintype.prod_boole.symm
  simp_rw [h_integrand_prod]; clear h_integrand_prod
  convert ← MeasureTheory.integral_fintype_prod_eq_prod (ι := Fin n × Fin 3)
    (f := fun _ r ↦ if |r| ≤ V ^ (3⁻¹ : ℝ) / 2 then 1 else 0); swap
  · infer_instance
  rw [Finset.prod_const]
  rw [Finset.card_univ, Fintype.card_prod, Fintype.card_fin, Fintype.card]
  have h_integral_ld : (∫ (x : ℝ), if |x| ≤ V ^ (3⁻¹ : ℝ) / 2 then 1 else 0) =
    have h_indicator := integral_indicator (f := fun _ ↦ (1:ℝ)) (μ := by
      (measurableSet_Icc (a := -(V ^ (3⁻¹ : ℝ) / 2)) (b := (V ^ (3⁻¹ : ℝ) / 2))
    simp_rw [Set.indicator] at h_indicator
    simp_rw [abs_le, ← Set.mem_Icc, h_indicator]
    simp
    positivity
  rw [h_integral_ld]; clear h_integral_ld
  rw [← Real.rpow_mul_natCast hV.le]
  field_simp
  simp
· --Gaussian integral
  have h_gaussian :=
    GaussianFourier.integral_rexp_neg_mul_sq_norm (V := PiLp 2 (fun _ : ℝ ↦ 1))
  apply (Eq.trans ?_ h_gaussian).trans ?_
  · have := EuclideanSpace.volume_preserving_symm_measurableEquiv_toLp (F := ℝ)
    rw [← this.integral_comp (MeasurableEquiv.measurableEmbedding _)]
    congr! 3 with x
    simp_rw [div_eq_inv_mul, ← Finset.mul_sum, ← mul_assoc, neg_mul, mul_assoc]
    congr! 3

```

```

    simp only [Prod.mk.eta, Real.norm_eq_abs, sq_abs]
    congr
  · field_simp
    congr
    simp
    ring_nf

/-- The Helmholtz Free Energy A for an ideal gas. -/
theorem HelmholtzA_eq (hV : 0 < V) (hT : 0 < T) : IdealGas.HelmholtzA (n,V) T =
  -n * T * (Real.log V + (3/2) * Real.log (2 * Real.pi * T)) := by
  rw [HelmholtzA, PartitionZT, PartitionZ_eq n hV (one_div_pos.mpr hT), Real.log_mul
    Real.log_pow, Real.log_rpow, one_div, div_inv_eq_mul]
  ring_nf
  all_goals positivity

theorem ZIntegrable (hV : 0 < V) (hβ : 0 < β) : IdealGas.ZIntegrable (n,V) β := by
  have hZpos : 0 < PartitionZ IdealGas (n, V) β := by
    rw [PartitionZ_eq n hV hβ]
    positivity
  constructor
  · apply MeasureTheory.Integrable.of_integral_ne_zero
    rw [← PartitionZ]
    exact hZpos.ne'
  · exact hZpos.ne'

/-- The ideal gas law:  $PV = nRT$ . In our unitsless system,  $R = 1$ . -/
theorem IdealGasLaw (hV : 0 < V) (hT : 0 < T) :
  let P := IdealGas.Pressure (n,V) T;
  let R := 1;
  P * V = n * R * T := by
  dsimp [Pressure]
  rw [← derivWithin_of_isOpen (s := Set.Ioi 0) isOpen_Ioi hV]
  rw [derivWithin_congr (f := fun V' ↦ -n * T * (Real.log V' + (3/2) * Real.log (2 *
  rw [derivWithin_of_isOpen (s := Set.Ioi 0) isOpen_Ioi hV]
  erw [deriv_mul (by fun_prop) (by fun_prop (disch := exact hV.ne')))]
  simp [field]
  ring_nf
  · intro _ hV'
    dsimp
    rw [HelmholtzA_eq n hV' hT]
    ring_nf
  · exact HelmholtzA_eq n hV hT

-- Now proving e.g. Boyle's Law ("for an ideal gas with a fixed particle number, P a
-- is a trivial consequence of the ideal gas law.

```

end IdealGas

1.491 QuantumInfo/StatMech/ThermoQuantities.lean

```

/-
Copyright (c) 2025 Alex Meiburg. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Alex Meiburg
-/
import QuantumInfo.StatMech.Hamiltonian
import Mathlib.Analysis.SpecialFunctions.Log.Deriv
import Mathlib.Data.Real.StarOrdered
import Mathlib.MeasureTheory.Constructions.Pi
import Mathlib.MeasureTheory.Integral.Bochner.Basic
import Mathlib.MeasureTheory.Integral.Bochner.L1
import Mathlib.MeasureTheory.Integral.Bochner.VitaliCaratheodory
import Mathlib.MeasureTheory.Measure.Haar.OfBasis
import Mathlib.Order.CompletePartialOrder

noncomputable section
namespace MicroHamiltonian

variable {D : Type} (H : MicroHamiltonian D) (d : D)

/-- The partition function corresponding to a given MicroHamiltonian. This
It also depends on the data D defining the system extrinsincs.

* Ideally this would be an NNReal, but  $\int$  (NNReal) doesn't work right now,
-/
def PartitionZ ( $\beta$  :  $\mathbb{R}$ ) :  $\mathbb{R}$  :=
   $\int$  (config : H.dim d  $\rightarrow$   $\mathbb{R}$ ),
    let E := H.H config
    if h : E =  $\tau$  then 0 else Real.exp (- $\beta$  * (E.untop h))

/-- The partition function as a function of temperature T instead of  $\beta$ . -
/
def PartitionZT (T :  $\mathbb{R}$ ) :  $\mathbb{R}$  :=
  PartitionZ H d (1/T)

/-- The Internal Energy, U or E, defined as  $-\partial(\ln Z)/\partial\beta$ . Parameterized here
/
def InternalU ( $\beta$  :  $\mathbb{R}$ ) :  $\mathbb{R}$  :=
  -deriv (fun  $\beta'$   $\mapsto$  (PartitionZ H d  $\beta'$ ).log)  $\beta$ 

```

```

/-- The Helmholtz Free Energy,  $-T * \ln Z$ . Also denoted  $F$ . Parameterized here with  $\tau$ 
/
def HelmholtzA (T : ℝ) : ℝ :=
  -T * (PartitionZT H d T).log

/-- The entropy, defined as the  $-\partial A/\partial T$ . Function of  $T$ . -/
def EntropyS (T : ℝ) : ℝ :=
  -deriv (HelmholtzA H d) T

/-- The entropy, defined as  $\ln Z + \beta * U$ . Function of  $\beta$ . -/
def EntropySβ (β : ℝ) : ℝ :=
  (PartitionZ H d β).log + β * InternalU H d β

/-- To be able to compute or define anything from a Hamiltonian, we need its partition
a computable integral. A Hamiltonian is ZIntegrable at  $\beta$  if PartitionZ is Lebesgue integrable
-/
def ZIntegrable (β : ℝ) : Prop :=
  MeasureTheory.Integrable (fun (config : H.dim d → ℝ) ↦
    let E := H.H config;
    if h : E = τ then 0 else Real.exp (-β * (E.untop h))
  ) ∧ (H.PartitionZ d β ≠ 0)

/--
This Prop defines the most common case of ZIntegrable, that it is integrable at all
(aka all positive β).
-/
def PositiveβIntegrable : Prop :=
  ∀ β > 0, H.ZIntegrable d β

variable {H d}

/-
Need the fact that the partition function  $Z$  is differentiable. Assume it's integrable.
Letting  $\mu^-(H,E)$  be the measure of  $\{x \mid H(x) \leq E\}$ , then for nonzero  $\beta$ ,
 $\int_{-\infty}^{\infty} \exp(-\beta E) (d\mu^-/dE) dE =$ 
 $\int \exp(-\beta H) d\mu =$ 
 $\int (1/\beta * \int_{-\infty}^{\infty} \exp(-\beta E) dE) d\mu =$ 
 $\int (1/\beta * \int_{-\infty}^{\infty} \exp(-\beta E) \chi(E \leq H) dE) d\mu =$ 
 $1/\beta * \int (\int \exp(-\beta E) \chi(E \leq H) d\mu) dE =$ 
 $1/\beta * \int \exp(-\beta E) * \mu^-(H,E) dE$ 

so this will be differentiable if
 $\int \exp(-\beta E) * \mu^-(H,E) dE$ 
is, aka if the Laplace transform is differentiable.
See e.g. https://math.stackexchange.com/q/84382/127777

```

For this we really want the fact that the Laplace transform is analytic where it is (as Wikipedia informs) an easy consequence of Fubini's theorem + M. theorem isn't in mathlib yet. So this is a sorry for now

```

-/
open scoped ContDiff in
theorem DifferentiableAt_Z_if_ZIntegrable {β : ℝ} (h : H.ZIntegrable d β) :
  sorry

/-- The two definitions of entropy, in terms of T or β, are equivalent. -
/
theorem entropy_A_eq_entropy_Z (T β : ℝ) (hβT : T * β = 1) (hi : H.ZIntegrable d β) :
  EntropyS H d T = EntropySβ H d β := by
  have hTnz : T ≠ 0 := left_ne_zero_of_mul_eq_one hβT
  have hβnz : β ≠ 0 := right_ne_zero_of_mul_eq_one hβT
  have hβT' := eq_one_div_of_mul_eq_one_right hβT
  dsimp [EntropyS, EntropySβ, InternalU, PartitionZT]
  unfold HelmholtzA
  erw [deriv_mul]
  rw [deriv_neg'', neg_mul, one_mul, neg_add_rev, neg_neg, mul_neg, add_comm]
  congr 1
  · rw [PartitionZT, hβT']
  simp_rw [PartitionZT]
  have hdc := deriv_comp (h := fun T ↦ T⁻¹) (h₂ := fun β => Real.log (H.PartitionZT β))
  unfold Function.comp at hdc
  simp only [hdc, one_div, deriv_inv', mul_neg, neg_inj, hβT']
  field_simp
  ring_nf
  --Show the differentiability side-goals
  · rw [← one_div, ← hβT']
    have h₁ := hi.2
    have := (DifferentiableAt_Z_if_ZIntegrable hi).differentiableAt WithTop
    fun_prop (disch := assumption)
  · fun_prop (disch := assumption)
  · fun_prop
  · simp_rw [PartitionZT]
    rw [hβT'] at hi
    have := hi.2
    have := (DifferentiableAt_Z_if_ZIntegrable hi).differentiableAt WithTop
    fun_prop (disch := assumption)

/--
The "definition of temperature from entropy":
1/T = (∂S/∂U), when the derivative is at constant extrinsic d (typically N/
Here we use β instead of 1/T on the left, and express the right actually as
as all our things are ultimately parameterized by β.
-/

```



```

theorem  $\beta$ _eq_deriv_S_U { $\beta$  :  $\mathbb{R}$ } (hi : H.ZIntegrable d  $\beta$ ) :  $\beta$  = (deriv (H.EntropyS $\beta$  d))
  unfold EntropyS $\beta$ 
  unfold InternalU

  --Show the differentiability side-goals
  have : DifferentiableAt  $\mathbb{R}$  (fun  $\beta$  => Real.log (H.PartitionZ d  $\beta$ ))  $\beta$  := by
    have := hi.2
    have := (DifferentiableAt_Z_if_ZIntegrable hi).differentiableAt WithTop.top_ne_z
    fun_prop (disch := assumption)
  have : DifferentiableAt  $\mathbb{R}$  (deriv fun  $\beta$  => Real.log (H.PartitionZ d  $\beta$ ))  $\beta$  := by
    have this := (DifferentiableAt_Z_if_ZIntegrable hi).log hi.2
    replace this :=
      (this.fderiv_right (m :=  $\tau$ ) (OrderTop.le_top _)).differentiableAt WithTop.top_
    unfold deriv
    fun_prop

  --Main goal
  simp only [mul_neg]
  erw [deriv.neg', deriv_add, deriv.neg']
  dsimp
  erw [deriv_mul]
  simp only [deriv_id'', one_mul, neg_add_rev, add_neg_cancel_comm_assoc, neg_div_ne]
  have : deriv (deriv fun  $\beta$  => Real.log (H.PartitionZ d  $\beta$ ))  $\beta \neq 0$  := ?_
  exact (mul_div_cancel_right0  $\beta$  this).symm
  --Discharge those side-goals
  · sorry
  · fun_prop (disch := assumption)
  · fun_prop (disch := assumption)
  · fun_prop (disch := assumption)
  · fun_prop (disch := assumption)

open scoped ContDiff in
example (x :  $\mathbb{R}$ ) (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hf : ContDiffAt  $\mathbb{R}$   $\omega$  f x) : DifferentiableAt  $\mathbb{R}$  (deriv f)
  have := (hf.fderiv_right (m :=  $\tau$ ) (OrderTop.le_top _)).differentiableAt WithTop.to
  unfold deriv
  fun_prop

end MicroHamiltonian

--! Specializing to a system of particles in space

namespace NVEHamiltonian
open MicroHamiltonian

variable (H : NVEHamiltonian) (d :  $\mathbb{N} \times \mathbb{R}$ )

```

```

/-- Pressure, as a function of T. Defined as the conjugate variable to volume
/
def Pressure (T : ℝ) : ℝ :=
  let (n, V) := d;
  -deriv (fun V' ↦ HelmholtzA H (n, V') T) V

end NVEHamiltonian

```

1.492 QuantumInfo/WildeTODO.md

Matching QuantumInfo to _Quantum Information Theory_

Mark Wilde's book "Quantum Information Theory" provides a solid guide to QIT. In the development of `QuantumInfo`, it was conceived that we could try to match that book where reasonably possible as a way to ensure good coverage. This gives that correspondence, although it may be rather out of date (probably early 2025). Of course there's also a lot of content in `QuantumInfo` that is not in that book, too.

Each of the items below is an item in Mark Wilde's book, "Quantum Information Theory".

- * ☐ - Completed, and proved with no `sorry`s.
- * ☐ - All required definitions are present and the question is stated / the answer is given.
- * ☐ - This has not yet been stated or fully defined.
- * ☐ - This is unlikely / undesirable to be formalized.
- * ☐ - This didn't need to be proved, because it was just the definition (or a simple fact).
- * ☐ - This is not a fact in the book but something related that would be nice to have.

There's very little from Chapters 2 or 3 to include right now. Chapter 2 is mostly covered.

Chapter 2 - Classical Shannon Theory

Exercise 2.1.1: The entropy of a uniform variable is $\log |X|$, where $|X|$ is the number of elements in X .
☐ `Hs\_uniform` in `ClassicalInfo.Entropy`.

Property 2.1.1, 2.1.2, 2.1.3: Basic properties of typical sets.

☐ Typical sets aren't defined until Chapter 14, we will prove things then.

Exercise 2.2.1: If the average (across codewords) probability of a codeword is p , then the probability of a codeword being in a set of size k is k^p .
☐ `ENNReal.finset\_card\_const\_le\_le\_of\_tsum\_le` states the interesting bit (and the proof).

Chapter 3 - The Noiseless Quantum Theory

Bras, Kets

☐ `Bra` and `Ket` in `QuantumInfo.Finite.Braket`.

Exercise 3.2.1: Determine the Bloch sphere angles for the states $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.

$\square$ and $\square + \square$.

$\square$ Eventually we'll define qubits, basic states like these, and the Bloch sphere, and

Exercise 3.3.1: A linear operator unitary iff it is norm-preserving.

$\square$ This is stated for linear operators as `unitary.linearIsometryEquiv`, and the "only

_There's a whole bunch about Pauli matrices. We haven't specialized to qubits yet, s

Exercise 3.3.2: Find a matrix representation of $[X, Z]$ in the basis $\square 0 \square, \square 1 \square$.

Exercise 3.3.3: Show that the Pauli matrices are all Hermitian, unitary, and square

Exercise 3.3.4: Represent the eigenstates of the Y operator in the standard computat

Exercise 3.3.5: Show that the Paulis either commute or anticommute.

Exercise 3.3.6: With $\sigma_i, i=0..3$, show that $\text{Tr}\{\sigma_i \sigma_j\} = \delta_{ij}$.

Exercise 3.3.7: Matrix representation of Hadamard gate.

Exercise 3.3.8: Hadamard is its own inverse.

Exercise 3.3.9: A particular exercise in changing bases; this is $\square$.

Exercise 3.3.10: $HXH = Z$ and $HZH = X$. _Sure, if we make simp lemmas for Clifford produ

Definition 3.3.1: Function of a Hermitian operator (by a function on its spectrum).

$\square$ This we definitely would like, currently they're all ad-hoc.

Exercise 3.3.11: Explicit forms for rotation operators.

$\square$ Requires Bloch sphere and rotation gates. Will probably be the definitions of them

Postulate: Born rule.

$\square$ We have `MState.measure` which has this baked in. But we could also have it for br

Definition: von Neumann measurement. A POVM that is not just projective, but given b

$\square$ Could be a nice Prop on POVMs.

Exercise 3.4.1: POVM associated to the X operator measurement.

$\square$ Definitely want this.

Exercise 3.4.2, 3.4.3: Asking (something like) $\langle \psi | H | \psi \rangle = \text{Ex}[H.\text{measure}(\text{pure } \psi)]$.

$\square$ Definitely want to prove. Doesn't need qubits, either.

Uncertainty principle

$\square$ Could do the state-dependent version given in the book. The averaged-case expectation value one comes as a result. But this would require defining + work

Exercise 3.4.4, 3.4.5: Longish story about uncertainty princple.

$\square$ Won't prove the thing asked, it's a very specific thing.

Exercise 3.5.1: This is askings for products of operators on qubits as tensor produc

□ This is precisely how we define it to start with.

Exercise 3.5.2: Asking that $\langle \xi_1 \otimes \xi_2 | \psi_1 \otimes \psi_2 \rangle = \langle \xi_1 | \psi_1 \rangle \langle \xi_2 | \psi_2 \rangle$.

□ Basic fact about bracket products types.

Exercise 3.5.3: Matrix representation of CNOT gate as $[1, 0, 0, 0; 0, 1, \dots]$ etc.

□ In `Qubits.Basic`, defined using `Qubit.controllize` and proven equivalent.

Exercise 3.5.4: Prove $(H \otimes H)(\text{CNOT})(H \otimes H) = |+\rangle\langle+| \otimes I + |-\rangle\langle-| \otimes Z$.

□ Very specific equality.

Exercise 3.5.5: `CNOT(1→2)` commutes with `CNOT(1→3)`.

Exercise 3.5.6: `CNOT(2→1)` commutes with `CNOT(3→1)`.

□ Depending on how we define the qubits to a gate, might be fine (in which

No-Cloning theorem (for unitaries): There is no unitary that clones perfectly.

□ Should be easy to state and prove.

□ No-Cloning theorem (for channels): There is no channel that clones perfectly.

□ Proving the bounds on the optimal universal quantum clone (5/6 fidelity etc.)

Exercise 3.5.7: Orthogonal states can be perfectly cloned.

□ Stated for two orthogonal qubit states, we'll prove for orthogonal sets of

Exercise 3.5.8: No-Deletion theorem. No unitary from $|\psi\rangle\langle\psi|A$ to $|\psi\rangle\langle\psi|B$ for fixed

□

Definition 3.6.1: Entangled pure states.

□ `Ket.IsEntangled` in `QuantumInfo.Finite.Braket`

Exercise 3.6.1, 3.6.2: The Bell state $= (|++\rangle + |--\rangle)/\sqrt{2}$. This means sharing

□ Too specific. _Could_ say something like "the Bell state is identical in

Exercise 3.6.3: Cloning implies signalling.

□ If we can prove that cloning doesn't exist, this gives an immediate contradiction.

CHSH Game

□ Defining it, giving optimal classical and quantum strategies.

Exercise 3.6.5, 3.6.6, 3.6.7: Numerical exercises with Bell states.

□ Maybe just defining Bell states as a basis for 2-qubit states.

Exercise 3.7.1 - 3.7.11: Lemmas about qudit X and Z operators.

□ Sure, if we define them.

Exercise 3.7.12: "Transpose trick" for Bell state $|\Phi\rangle$, that $(U \otimes I)|\Phi\rangle = (I \otimes U^T)|\Phi\rangle$

□ ``transposeTrick`` in ``Braket.lean``.

Theorem 3.8.1, Exercise 3.8.1: Schmidt Decomposition.

□

Exercise 3.8.2: Schmidt decomposition determines if a state is entangled or product.

□

Chapter 4 - The Noisy Quantum Theory

Definition 4.1.1: Matrix trace.

□ ``Matrix.trace`` in `Mathlib`.

Exercise 4.1.1: The trace is cyclic.

□ ``Matrix.trace_mul_cycle`` in `Mathlib`.

Definition 4.1.2: Defining the density operator of an ensemble.

□ We define density operators, but could write an "of_ensemble" function.

Exercise 4.1.2: The density matrix of ``pure (indicator 0)``.

□ Feels specific and not like a useful lemma.

Exercise 4.1.3: Matrix trace as the expectation value of maximally entangled state.

□

Exercise 4.1.4: Trace of operator functions, ``Tr[f(GG*)] = Tr[f(G*G)]``.

□ Requires Definition 3.3.1 first.

Exercise 4.1.5: Computing density operators of some particular ensembles.

□

Exercise 4.1.6: The spectrum of a density operator forms a distribution.

□ ``MState.spectrum`` is a ``Distribution``, in ``QuantumInfo.Finite.MState``.

Definition 4.1.3: The density operator is a PSD operator with trace 1.

□ ``MState`` in ``QuantumInfo.Finite.MState``.

Definition 4.1.4: The maximally mixed state.

□ ``MState.uniform`` in ``QuantumInfo.Finite.MState``.

Exercise 4.1.7: A uniform ensemble of $\{|0\rangle, |1\rangle, |+\rangle, |-\rangle\}$ is the maximally mixed state.

□

Exercise 4.1.8: The set of density operators is a convex set.

□ ``MState.instMixable`` in ``QuantumInfo.Finite.MState``.

Definition 4.1.5: Purity of a mixed state.

□ `MState.purity`` in `QuantumInfo.Finite.MState``.

Exercise 4.1.9: Purity is equal to 1 iff the state is pure.

□ `MState.pure_iff_purity_one`` in `QuantumInfo.Finite.MState``.

Exercise 4.1.10-4.1.13: Specific to qubits and the Bloch sphere.

□

Unitary Evolution of a Mixed State

□ `MState.U_conj`` in `QuantumInfo.Finite.Unitary``

Evolution of an ensemble of mixed states

□ Could define ensemble as distribution over a finite set of MStates.

Exercise 4.1.14, 4.1.15: Some facts about embedding classical probabilities

□ Probably not useful for anything else.

Definition 4.2.1: POVMs as PSD matrices that sum to the identity.

□ `POVM`` in `QuantumInfo.Finite.POVM``.

Exercise 4.2.1: The five "Chrysler" states on a qubit, scaled down by (2/5)

□ Very specific.

Exercise 4.2.2(a): Suppose an ensemble ρ_X has a bound τ , so that $\forall(x \in X)$,

Exercise 4.2.2(b): In a d dimensional ensemble, no more than d possible sym

□ Seems unlikely to be used later but fine to state and prove.

Definition 4.3.1: Product State (density operator).

□ `MState.prod`` in `QuantumInfo.Finite.MState`` constructs product states. N

Exercise 4.3.1(a): `ρ.purity = (U_swap * ρ ⊗ ρ).trace``.

Exercise 4.3.1(b): For an operator function `f``, `Tr[f(ρ)] = ( f(ρ) ⊗ I(d)`

□ Should be an easy index chase.

Exercise 4.3.2: Just a step of a scenario, not really a theorem.

□

Exercise 4.3.3: Every separable state $\sum x, p_x \otimes o_x`$  is also of the form  $\sum`$

Definition 4.3.2: Separable state.

□ `MState.IsSeparable`` in `QuantumInfo.Finite.MState``.

Definition 4.3.3: Entangled (mixed) state.

□ `MState.IsEntangled`` in `QuantumInfo.Finite.MState``.

Exercise 4.3.4: Convexity of separable states.

□ Will have to write this as ``Mixable { p : MState (d1 × d2) // p.IsSeparable }``.

1.493 README.md

```
> [!NOTE]
> Things look different? We've recently undergone a name change and move from /lean-phys-community/PhysLean
> to /leanprover-community/physlib. Same repo, different location and name. This should be the average user, but please be patient as we update things.
```

```

□ The project shall contain results (definitions, theorems, lemmas and calculations) including quantum information, formalized (or digitalized) into the interactive
□ The project shall not be tied to physics axiomizations (e.g. axiomatic QFT), but
□ The project shall be for mainstream physics only.
```

Contributing to PhysLib

PhysLib is open-source and community run, and we welcome contributions from anyone. All you need to do is open a pull-request with your changes and our team of maintainers will review it and iterate with you on feedback until it can be merged.

If you unsure where you would like to contribute, you may find ideas on:

- our [open issues](https://github.com/leanprover-community/physlib/issues).
- our [todo list](https://physlean.com/TODOList)
- our [Get Involved page](https://physlean.com/GetInvolved.html)
- the [quantumInfo todo page](./QuantumInfo/WildeTODO.md)

> If stuck at any point there are lots of people happy to help on the [PhysLib zulip channel] (PhysLib)

Installing PhysLib

At the moment PhysLib is divided into two essentially disjoint halves, ``PhysLean`` and ``QuantumInfo``. These were two repositories that merged in an effort to create a more cohesive ecosystem in Lean. There is ongoing effort to integrate them more deeply and share code, but at the moment they offer two separate `_build` targets: ``PhysLean`` and ``QuantumInfo``, as specified in the `lakefile.toml`. They are both default targets, so ``lake build`` will build both.

If you only want to build one, ``lake build PhysLean`` or ``lake build QuantumInfo`` will build only one or the other. This could be useful if you're working on one part or the other and want to ensure all your changes worked, or if you're only interested in having one or the other as part of your project.

Making a pull-request

There are lots of guides on how to make a pull-request on GitHub. The first thing you need to do is fork the repository. Once you've made your pull request we will review it.

- Guide to [PhysLib reviews](https://github.com/leanprover-community/physlib/blob/master/docs/ReviewGuidelines.md).

It will also undergo a number of automated checks called linters. Sometimes to run locally:

- Guide to [linters and running them locally](https://github.com/leanprover-community/physlib/blob/master/scripts/README.md).

Most importantly:

> [!NOTE]

> When making contributing to PhysLib it is much better to do it with small

Maintainers

Below are the maintainers of the project, however the best way to reach them on the [Lean Zulip](https://leanprover.zulipchat.com/#narrow/channel/479953-PhysLib)

- Léo Lessa (@Megaleo)
- Alex Meiburg (@Timeroot)
- Daniel Morrison (@morrison-daniel)
- Zhi-Kai Pong (@zhikaip)
- Rodolfo Soldati (@rodolfo-s)
- Joseph Tooby-Smith (@jstoobysmith)
- Winston Yin (@winstonysin)

Citing the project

If you want to cite the project as a whole please cite:

...

```
@misc{physlib,
  author = {The PhysLib community},
  title = {PhysLib: The Lean Physics Library},
  year = {2024},
  publisher = {GitHub},
  journal = {GitHub repository},
  howpublished = {\url{https://github.com/leanprover-community/physlib}},
}
```

...

PhysLib was formed by merging the general physics Lean library PhysLean (for) with the quantum-information library Lean-QuantumInfo. Where appropriate please citing the papers associated with the origin of these projects. For the for

```
@article{Tooby-Smith:2024vqu,
  author = "Tooby-Smith, Joseph",
```



```

    title = "{HepLean: Digitalising high energy physics}",
    eprint = "2405.08863",
    archivePrefix = "arXiv",
    primaryClass = "hep-ph",
    doi = "10.1016/j.cpc.2024.109457",
    journal = "Comput. Phys. Commun.",
    volume = "308",
    pages = "109457",
    year = "2025"
}

```

and for the latter please use:

```

...
@article{Meiburg:2025mwn,
  author = "Meiburg, Alex and Lessa, Leonardo A. and Soldati, Rodolfo R.",
  title = "{A Formalization of the Generalized Quantum Stein's Lemma in Lean}",
  eprint = "2510.08672",
  archivePrefix = "arXiv",
  primaryClass = "quant-ph",
  month = "10",
  year = "2025"
}

```

1.494 docs/ReviewGuidelines.md

Review guidelines

Below we give guidelines to what reviews of pull-requests (PRs) to PhysLib are looking for. This list is not exhaustive and reviews are within the judgement of the reviewer.

A good place to start for both authors and reviewers is:

- [What to consider when reviewing](<https://leanprover-community.github.io/contributing/review.html#what-to-consider-when-reviewing>)

Note that your PR does not need to be perfect once you make it, we will work with you to improve its quality, if you are also willing to work with us. Just do your best. You should expect that about 50% of the work happens after you make your PR and it should not discourage you.

Code quality

- Correct abstraction of lemmas and definitions.
- Correct use of type theory in definitions.
- Correct use of lemmas from Mathlib, e.g. not reproving things which are already proven.
- Concise proofs where possible.

In another programming language these points are analogous to not just making a function that does the right thing, but making sure that that function is as efficient as possible, uses the best algorithm available etc.

Organization

- Lemmas and definitions appear in the correct place
- Modules are easy to read and have a well-defined scope
- Any new correct files are suitably named.
- Any new correct files are suitably located.
- Modules have sufficient documentation to understand their flow.

These points are about navigability of the project as a whole and how easy

Style conventions

In addition to those here:

<https://leanprover-community.github.io/contribute/style.html>

- Use of `lemma` instead of `theorem` except for the most important results

These points are related to the readability of the project and results there

PR and authorship

- The author of the PR understands the material in the PR.
- The PR is concise and adds a single new concept (which may be multiple lemmas)

These points are not to the code-base itself but the history of the project. It is easy for someone to find information on a given area from the git history.

PR lengths

As a rule of thumb:

- < 50 lines = easy to check PR
- < 100 lines = average sized PR
- 100-200 lines = large PR, okay but try to break up if possible
- 200+ lines = very large PR, should be broken up into smaller pieces

There is no strict rule, as some very large PRs can be very simple e.g. documentation or file reorganization and some small PRs can be very complicated e.g. adding definitions.

Remember, the reviewer needs to understand your PR and the more there is for them to understand, the harder it is.

Small PRs are generally good because:

- Small changes can be merged faster and not get blocked by unrelated work
- Reduces the chance and impact of merge conflicts
- Easier for reviewers to understand

Tag system

We operate a tag system with our PRs. This makes it easier for reviewers and authors to understand where in the process PRs are.

- Please tag your PR with the appropriate ``t-subject`` tag.
- PRs marked as ``draft`` will not be reviewed.
- PRs marked with a ``awaiting-author`` tag will also not be reviewed until the author has addressed any reviewer comments and has removed this tag.
- If your PR is not getting reviewed and you would like to draw attention to it, please post [here](https://leanprover.zulipchat.com/#narrow/channel/479953-PhysLib/topic/PR.20reviews/with/577663418).
- Once a PR is marked with a ``ready-to-merge`` the author does not need to do anything; the maintainers will make sure it gets merged into the project.

1.495 lake-manifest.json

```
"name": "PhysLib",
```

1.496 lakefile.lean

```
import Lake
open System Lake DSL
```

```
package «PhysLib»
```

```
require "mathlib" from git "https://github.com/leanprover-
community/mathlib4.git" @ "v4.28.0"
```

```
require «doc-gen4» from git "https://github.com/leanprover/doc-
gen4" @ "v4.28.0"
```

```

@[default_target]
lean_lib PhysLean

@[default_target]
lean_lib QuantumInfo

-- These were their own lean_lib in Lean-QuantumInfo, we should move them t
-- lean_lib ClassicalInfo
-- lean_lib StatMech

lean_exe check_file_imports where
  srcDir := "scripts"

lean_exe style_lint where
  srcDir := "scripts/MetaPrograms"

lean_exe type_former_lint where
  srcDir := "scripts"

lean_exe lint_all where
  supportInterpreter := true
  srcDir := "scripts"

lean_exe check_dup_tags where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

lean_exe sorry_lint where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

lean_exe runPhysLeanLinters where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

-- Other linting scripts

lean_exe free_simps where
  srcDir := "scripts/MetaPrograms"

lean_exe spelling where
  srcDir := "scripts/MetaPrograms"

lean_exe check_rfl where
  srcDir := "scripts/MetaPrograms"

```

```
lean_exe redundant_imports where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

lean_exe module_doc_lint where
  srcDir := "scripts/MetaPrograms"

-- Stats

lean_exe stats where
  supportInterpreter := true
  srcDir := "scripts"

lean_exe local_stats where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

-- Scripts for the website

lean_exe make_tag where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

lean_exe TODO_to_yaml where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

lean_exe informal where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

-- Old

lean_exe find_TODOs where
  srcDir := "scripts"

lean_exe notes where
  supportInterpreter := true
  srcDir := "scripts/MetaPrograms"

lean_exe mathlib_textLint_on_hepLean where
  srcDir := "scripts"

lean_exe no_docs where
  srcDir := "scripts/MetaPrograms"
```

```
-- Optional scripts

-- Optional inclusion of llm. Needed for `openAI_doc_check`
-- require llm from git "https://github.com/leanprover-community/llm" @ "ma

-- Optional inclusion of tryAtEachStep
-- require tryAtEachStep from git "https://github.com/dwrensha/tryAtEachSte

-- Optional inclusion of LeanCopilot
-- require LeanCopilot from git "https://github.com/lean-
dojo/LeanCopilot.git" @ "v1.4.1"

-- lean_exe commands defined specifically for PhysLean.

-- Optional inclusion of openAI_doc_check. Needs `llm` above.
-- lean_exe openAI_doc_check where
--   srcDir := "scripts"
```

1.497 scripts/lint-style.py

```
    if words[0] != "module" and words[0] != "public" and words[0] != "m
" and words[0] != "/-!" and words[0] != "#align_import":
```